

Mathematics, Computer Science & AI Compendium

Henry Ndubuaku et al.

- [1 Vector Spaces](#)
 - [1.1 Coding Tasks \(use CoLab or notebook\)](#)
 - [1.2 Side Quest \(Some mathematical symbols to know\)](#)
- [2 Vector Properties](#)
 - [2.1 Coding Tasks \(use CoLab or notebook\)](#)
- [3 Metrics and Norms](#)
 - [3.1 Coding Tasks \(use CoLab or notebook\)](#)
- [4 Vector Products](#)
 - [4.1 Coding Tasks \(use CoLab or notebook\)](#)
- [5 Basis and Duality](#)
 - [5.1 Coding Tasks \(use CoLab or notebook\)](#)
- [6 Matrix Properties](#)
 - [6.1 Coding Tasks \(use CoLab or notebook\)](#)
- [7 Matrix Types](#)
 - [7.1 Coding Tasks \(use CoLab or notebook\)](#)
- [8 Matrix Operations](#)
 - [8.1 Coding Tasks \(use CoLab or notebook\)](#)
- [9 Linear Transformations](#)
 - [9.1 Coding Tasks \(use CoLab or notebook\)](#)
- [10 Matrix Decompositions](#)
 - [10.1 Coding Tasks \(use CoLab or notebook\)](#)
- [11 Differential Calculus](#)
 - [11.1 Coding Tasks \(use CoLab or notebook\)](#)
- [12 Integral Calculus](#)
 - [12.1 Coding Tasks \(use CoLab or notebook\)](#)
- [13 Multivariate Calculus](#)
 - [13.1 Coding Tasks \(use CoLab or notebook\)](#)
- [14 \$f\(x, y\) = xxy + x\$ at \$\(3, 2\)\$](#)
- [15 Optimisation](#)
 - [15.1 Coding Tasks \(use CoLab or notebook\)](#)

- [16 Fundamentals of Statistics](#)
 - [16.1 Coding Tasks \(use CoLab or notebook\)](#)
- [17 Statistical Measures](#)
 - [17.1 Coding Tasks \(use CoLab or notebook\)](#)
- [18 Sampling](#)
 - [18.1 Coding Tasks \(use CoLab or notebook\)](#)
- [19 Hypothesis Testing](#)
 - [19.1 Coding Tasks \(use CoLab or notebook\)](#)
- [20 Statistical Inference](#)
 - [20.1 Coding Tasks \(use CoLab or notebook\)](#)
- [21 Counting](#)
 - [21.1 Coding Tasks \(use CoLab or notebook\)](#)
- [22 Probability Concepts](#)
 - [22.1 Coding Tasks \(use CoLab or notebook\)](#)
- [23 Probability Distributions](#)
 - [23.1 Coding Tasks \(use CoLab or notebook\)](#)
- [24 Bayesian Methods and Sequential Models](#)
 - [24.1 Coding Tasks \(use CoLab or notebook\)](#)
- [25 Information Theory](#)
 - [25.1 Coding Tasks \(use CoLab or notebook\)](#)
- [26 Classical Machine Learning](#)
 - [26.1 Coding Tasks \(use CoLab or notebook\)](#)
- [27 Gradient Machine Learning](#)
 - [27.1 Coding Tasks \(use CoLab or notebook\)](#)
- [28 Deep Learning](#)
 - [28.1 Coding Tasks \(use CoLab or notebook\)](#)
- [29 Reinforcement Learning](#)
 - [29.1 Coding Tasks \(use CoLab or notebook\)](#)
- [30 Distributed Deep Learning](#)
 - [30.1 Coding Tasks \(use CoLab or notebook\)](#)
- [31 Linguistic Foundations](#)
 - [31.1 Coding Tasks \(use CoLab or notebook\)](#)
- [32 Text Processing and Classic NLP](#)
 - [32.1 Coding Tasks \(use CoLab or notebook\)](#)
- [33 Embeddings and Sequence Models](#)
 - [33.1 Coding Tasks \(use CoLab or notebook\)](#)
- [34 Transformers and Language Models](#)
 - [34.1 Coding Tasks \(use CoLab or notebook\)](#)

- [35 Advanced Text Generation](#)
 - [35.1 Coding Tasks \(use CoLab or notebook\)](#)
- [36 Image Fundamentals](#)
 - [36.1 Coding Tasks \(use CoLab or notebook\)](#)
- [37 Convolutional Networks](#)
 - [37.1 Coding Tasks \(use CoLab or notebook\)](#)
- [38 Object Detection and Segmentation](#)
 - [38.1 Coding Tasks \(use CoLab or notebook\)](#)
- [39 Vision Transformers and Generation](#)
 - [39.1 Coding Tasks \(use CoLab or notebook\)](#)
- [40 Video and 3D Vision](#)
 - [40.1 Coding Tasks \(use CoLab or notebook\)](#)
- [41 Digital Signal Processing](#)
 - [41.1 Coding Tasks \(use CoLab or notebook\)](#)
- [42 Automatic Speech Recognition](#)
 - [42.1 Coding Tasks \(use CoLab or notebook\)](#)
- [43 Text to Speech and Voice](#)
 - [43.1 Coding Tasks \(use CoLab or notebook\)](#)
- [44 Speaker and Audio Analysis](#)
 - [44.1 Coding Tasks \(use CoLab or notebook\)](#)
- [45 Source Separation and Noise Cancellation](#)
 - [45.1 Coding Tasks \(use CoLab or notebook\)](#)
- [46 Multimodal Representations](#)
 - [46.1 Fusion Strategies](#)
 - [46.2 Joint Embedding Spaces](#)
 - [46.3 Contrastive Learning for Multimodal Alignment](#)
 - [46.4 Contrastive Loss Functions in Detail](#)
 - [46.5 Image-Text Retrieval and Zero-Shot Classification](#)
 - [46.6 Audio-Visual Correspondence](#)
 - [46.7 Evaluation](#)
 - [46.8 Putting It All Together](#)
 - [46.9 Coding Tasks \(use CoLab or notebook\)](#)
- [47 Vision Language Models](#)
 - [47.1 Visual Question Answering](#)
 - [47.2 Image Captioning](#)
 - [47.3 Architecture Patterns](#)
 - [47.4 Flamingo: Few-Shot Multimodal Learning](#)
 - [47.5 LLaVA and Visual Instruction Tuning](#)

- [47.6 Scaling Vision-Language Models](#)
- [47.7 Grounding and Referring](#)
- [47.8 OCR-Free Document Understanding](#)
- [47.9 The Visual Token Pipeline](#)
- [47.10 Training Objectives](#)
- [47.11 Coding Tasks \(use CoLab or notebook\)](#)
- [48 Image and Video Tokenisation](#)
 - [48.1 Why Tokenise Images](#)
 - [48.2 VQ-VAE: Vector Quantisation](#)
 - [48.3 VQ-GAN: Adversarial Training for Higher Fidelity](#)
 - [48.4 Residual Quantisation and Multi-Scale Codebooks](#)
 - [48.5 Image Tokenisers in Practice](#)
 - [48.6 Video Tokenisation](#)
 - [48.7 Continuous vs Discrete Tokens](#)
 - [48.8 Applications](#)
 - [48.9 Coding Tasks \(use CoLab or notebook\)](#)
- [49 Cross-Modal Generation](#)
 - [49.1 Text-to-Image Generation](#)
 - [49.2 Text-to-Video Generation](#)
 - [49.3 Text-to-Audio Generation](#)
 - [49.4 Image-to-Text Generation](#)
 - [49.5 Video-Audio Co-Generation](#)
 - [49.6 Instruction-Following Generation](#)
 - [49.7 Consistency and Alignment Metrics](#)
 - [49.8 Ethical Considerations](#)
 - [49.9 Coding Tasks \(use CoLab or notebook\)](#)
- [50 Unified Multimodal Architectures](#)
 - [50.1 The Case for Unification](#)
 - [50.2 Any-to-Any Models](#)
 - [50.3 Modality-Specific Encoders and Decoders with a Shared Backbone](#)
 - [50.4 Multimodal Tokenisation](#)
 - [50.5 Training Recipes: Staged Pretraining and Joint Fine-Tuning](#)
 - [50.6 Multimodal Chain-of-Thought Reasoning](#)
 - [50.7 Multimodal Agents](#)
 - [50.8 Benchmarks and Evaluation](#)
 - [50.9 World Models](#)
 - [50.10 Coding Tasks \(use CoLab or notebook\)](#)

- [51 Perception](#)
 - [51.1 Sensor Modalities](#)
 - [51.2 Sensor Calibration](#)
 - [51.3 Sensor Fusion](#)
 - [51.4 3D Object Detection](#)
 - [51.5 Depth Estimation](#)
 - [51.6 Occupancy Networks](#)
 - [51.7 Lane Detection and Road Topology](#)
 - [51.8 Semantic Mapping](#)
 - [51.9 Coding Tasks \(use CoLab or notebook\)](#)
- [52 Robot Learning](#)
 - [52.1 Robot Kinematics](#)
 - [52.2 Dynamics and Control](#)
 - [52.3 Imitation Learning](#)
 - [52.4 Sim-to-Real Transfer](#)
 - [52.5 World Models for Robotics](#)
 - [52.6 Manipulation](#)
 - [52.7 Locomotion](#)
 - [52.8 Safety in Robot Learning](#)
 - [52.9 Coding Tasks \(use CoLab or notebook\)](#)
- [53 Vision-Language-Action Models](#)
 - [53.1 From Vision-Language to Action](#)
 - [53.2 VLA Architecture](#)
 - [53.3 Action Tokenisation](#)
 - [53.4 Key VLA Models](#)
 - [53.5 Pretraining Recipes](#)
 - [53.6 Generalisation](#)
 - [53.7 Embodiment-Agnostic Models](#)
 - [53.8 Benchmarks and Evaluation](#)
 - [53.9 Coding Tasks \(use CoLab or notebook\)](#)
- [54 Self-Driving Cars](#)
 - [54.1 The Autonomous Driving Stack](#)
 - [54.2 HD Maps](#)
 - [54.3 Motion Prediction](#)
 - [54.4 Planning](#)
 - [54.5 End-to-End Driving](#)
 - [54.6 World Models for Driving](#)
 - [54.7 Simulation](#)

- [54.8 Safety](#)
- [54.9 Levels of Autonomy](#)
- [54.10 Coding Tasks \(use CoLab or notebook\)](#)
- [55 Space and Extreme Robotics](#)
 - [55.1 Space Robotics](#)
 - [55.2 Communication Constraints](#)
 - [55.3 Radiation-Hardened Computing](#)
 - [55.4 Autonomous Navigation in Unstructured Terrain](#)
 - [55.5 Underwater Robotics](#)
 - [55.6 Search and Rescue Robotics](#)
 - [55.7 Swarm Robotics](#)
 - [55.8 Human-Robot Interaction](#)
 - [55.9 Coding Tasks \(use CoLab or notebook\)](#)
- [56 Geometric Deep Learning](#)
 - [56.1 Symmetry and Groups](#)
 - [56.2 Invariance and Equivariance](#)
 - [56.3 The Five Geometric Domains](#)
 - [56.4 Scale Separation and Coarsening](#)
 - [56.5 Coding Tasks \(use CoLab or notebook\)](#)
- [57 Graph Theory](#)
 - [57.1 Nodes, Edges, and Adjacency](#)
 - [57.2 Graph Types](#)
 - [57.3 Degree, Paths, and Connectivity](#)
 - [57.4 The Graph Laplacian](#)
 - [57.5 Spectral Graph Theory](#)
 - [57.6 Community Detection](#)
 - [57.7 Real-World Graphs](#)
 - [57.8 Coding Tasks \(use CoLab or notebook\)](#)
- [58 Graph Neural Networks](#)
 - [58.1 The Message-Passing Framework](#)
 - [58.2 Graph Convolutional Network \(GCN\)](#)
 - [58.3 GraphSAGE](#)
 - [58.4 Graph Isomorphism Network \(GIN\)](#)
 - [58.5 Over-Smoothing](#)
 - [58.6 Graph Pooling](#)
 - [58.7 Heterogeneous Graphs](#)
 - [58.8 Link Prediction and Knowledge Graph Completion](#)
 - [58.9 Task Types](#)

- [58.10 Coding Tasks \(use CoLab or notebook\)](#)
- [59 Graph Attention Networks](#)
 - [59.1 GAT: Graph Attention Network](#)
 - [59.2 Multi-Head Graph Attention](#)
 - [59.3 GATv2: Fixing Static Attention](#)
 - [59.4 Graph Transformers](#)
 - [59.5 Positional and Structural Encodings](#)
 - [59.6 Scalability](#)
 - [59.7 Temporal and Dynamic Graphs](#)
 - [59.8 Coding Tasks \(use CoLab or notebook\)](#)
- [60 3D Graph Networks](#)
 - [60.1 Geometric Graphs](#)
 - [60.2 SE\(3\) and E\(n\) Equivariance](#)
 - [60.3 SchNet: Distance-Based Message Passing](#)
 - [60.4 DimeNet and SphereNet: Angles and Dihedrals](#)
 - [60.5 E\(n\) Equivariant GNNs \(EGNN\)](#)
 - [60.6 Tensor Field Networks and Higher-Order Representations](#)
 - [60.7 Applications](#)
 - [60.8 Graph Generation](#)
 - [60.9 Coding Tasks \(use CoLab or notebook\)](#)
- [61 Discrete Maths](#)
 - [61.1 Propositional Logic](#)
 - [61.2 Predicate Logic and Quantifiers](#)
 - [61.3 Proof Techniques](#)
 - [61.4 Sets](#)
 - [61.5 Relations](#)
 - [61.6 Functions](#)
 - [61.7 Graph Theory Fundamentals](#)
 - [61.8 Recurrence Relations](#)
 - [61.9 Computability](#)
 - [61.10 Complexity Theory](#)
 - [61.11 Coding Tasks \(use CoLab or notebook\)](#)
- [62 Computer Architecture](#)
 - [62.1 Number Systems](#)
 - [62.2 Logic Gates](#)
 - [62.3 CPU Architecture](#)
 - [62.4 Instruction Set Architectures](#)
 - [62.5 Pipelining](#)

- [62.6 Memory Hierarchy](#)
- [62.7 Virtual Memory](#)
- [62.8 I/O, Interrupts, and DMA](#)
- [62.9 Coding Tasks \(use CoLab or notebook\)](#)
- [63 Operating Systems](#)
 - [63.1 What an Operating System Does](#)
 - [63.2 Processes](#)
 - [63.3 Threads](#)
 - [63.4 CPU Scheduling](#)
 - [63.5 Memory Management](#)
 - [63.6 File Systems](#)
 - [63.7 System Calls and Kernel Mode](#)
 - [63.8 Networking Fundamentals](#)
 - [63.9 Virtualisation and Containers](#)
 - [63.10 Security Basics](#)
 - [63.11 Coding Tasks \(use CoLab or notebook\)](#)
- [64 Concurrency and Parallelism](#)
 - [64.1 Concurrency vs Parallelism](#)
 - [64.2 Synchronisation Primitives](#)
 - [64.3 Classic Concurrency Problems](#)
 - [64.4 Deadlock](#)
 - [64.5 Lock-Free and Wait-Free Data Structures](#)
 - [64.6 Parallel Programming Models](#)
 - [64.7 Async and Event-Driven Programming](#)
 - [64.8 Scaling Laws](#)
 - [64.9 Coding Tasks \(use CoLab or notebook\)](#)
- [65 Programming Languages](#)
 - [65.1 Language Paradigms](#)
 - [65.2 Type Systems](#)
 - [65.3 Memory Management](#)
 - [65.4 Compilation Pipeline](#)
 - [65.5 Interpretation and JIT Compilation](#)
 - [65.6 Key Language Features](#)
 - [65.7 Domain-Specific Languages](#)
 - [65.8 Language Design Tradeoffs](#)
 - [65.9 Coding Tasks \(use CoLab or notebook\)](#)
- [66 Foundations: Big O, Recursion, Backtracking, and Dynamic Programming](#)
 - [66.1 Why Patterns, Not Memorisation](#)

- [66.2 Big O Notation](#)
- [66.3 Recursion](#)
- [66.4 Backtracking](#)
- [66.5 Dynamic Programming](#)
- [66.6 Putting It All Together](#)
- [67 Arrays and Hashing](#)
 - [67.1 Arrays](#)
 - [67.2 Strings](#)
 - [67.3 Hash Tables](#)
 - [67.4 Pattern: Hash Map Lookups](#)
 - [67.5 Pattern: Two Pointers](#)
 - [67.6 Pattern: Sliding Window](#)
 - [67.7 Pattern: Prefix Sums](#)
 - [67.8 Common Pitfalls Summary](#)
 - [67.9 Take-Home Problems \(NeetCode\)](#)
- [68 Linked Lists, Stacks, and Queues](#)
 - [68.1 Linked Lists](#)
 - [68.2 Pattern: Fast and Slow Pointers \(Floyd's\)](#)
 - [68.3 Stacks](#)
 - [68.4 Pattern: Monotonic Stack](#)
 - [68.5 Queues](#)
 - [68.6 Priority Queues and Heaps](#)
 - [68.7 Common Pitfalls Summary](#)
 - [68.8 Take-Home Problems \(NeetCode\)](#)
- [69 Trees](#)
 - [69.1 Binary Tree Traversals](#)
 - [69.2 Binary Search Trees \(BSTs\)](#)
 - [69.3 Tries \(Prefix Trees\)](#)
 - [69.4 Union-Find \(Disjoint Set Union\)](#)
 - [69.5 Segment Trees and Fenwick Trees](#)
 - [69.6 Common Pitfalls Summary](#)
 - [69.7 Take-Home Problems \(NeetCode\)](#)
- [70 Graphs](#)
 - [70.1 Graph Representations](#)
 - [70.2 Pattern: BFS \(Breadth-First Search\)](#)
 - [70.3 Pattern: DFS \(Depth-First Search\)](#)
 - [70.4 Shortest Paths](#)
 - [70.5 Strongly Connected Components](#)

- [70.6 Common Pitfalls Summary](#)
- [70.7 Take-Home Problems \(NeetCode\)](#)
- [71 Sorting, Search, and Algorithm Design](#)
 - [71.1 Sorting Algorithms](#)
 - [71.2 Pattern: Binary Search](#)
 - [71.3 Pattern: Greedy Algorithms](#)
 - [71.4 Pattern: Dynamic Programming](#)
 - [71.5 Pattern: Backtracking](#)
 - [71.6 Common Pitfalls Summary](#)
 - [71.7 Take-Home Problems \(NeetCode\)](#)
- [72 Linux and the Command Line](#)
 - [72.1 The Shell](#)
 - [72.2 File System Hierarchy](#)
 - [72.3 File Permissions](#)
 - [72.4 Process Management](#)
 - [72.5 Package Managers](#)
 - [72.6 Environment Variables](#)
 - [72.7 SSH \(Secure Shell\)](#)
 - [72.8 Essential ML Commands Cheat Sheet](#)
- [73 Git and Version Control](#)
 - [73.1 The Mental Model](#)
 - [73.2 Branching](#)
 - [73.3 Merging and Rebasing](#)
 - [73.4 Resolving Conflicts](#)
 - [73.5 Writing Good Commit Messages](#)
 - [73.6 Pull Requests and Code Review](#)
 - [73.7 .gitignore](#)
 - [73.8 Git for ML](#)
- [74 Codebase Design and Patterns](#)
 - [74.1 Project Structure](#)
 - [74.2 Clean Code Principles](#)
 - [74.3 Design Patterns for ML](#)
 - [74.4 Configuration Management](#)
 - [74.5 Logging and Observability](#)
 - [74.6 API Design](#)
 - [74.7 Packaging and Distribution](#)
 - [74.8 Working with AI Coding Agents](#)

- [75 Testing and Quality Assurance](#)
 - [75.1 The Test Pyramid](#)
 - [75.2 Unit Tests with pytest](#)
 - [75.3 Mocking and Patching](#)
 - [75.4 Testing ML Code](#)
 - [75.5 CI/CD Pipelines](#)
 - [75.6 Linting and Formatting](#)
 - [75.7 Code Review Best Practices](#)
- [76 Deployment and DevOps](#)
 - [76.1 Docker for ML](#)
 - [76.2 Model Serving](#)
 - [76.3 Experiment Tracking](#)
 - [76.4 Reproducibility](#)
 - [76.5 Monitoring in Production](#)
 - [76.6 Feature Stores](#)
 - [76.7 Pipeline Orchestration](#)
- [77 Why C++ and How ML Frameworks Work](#)
 - [77.1 Why Python Frontend, C++ Backend](#)
 - [77.2 How ML Frameworks Are Structured](#)
 - [77.3 Quick C++ Fundamentals for Python Engineers](#)
 - [77.4 When to Write Custom C++ Kernels](#)
 - [77.5 How to Bind C++ to Python](#)
 - [77.6 The Big Picture](#)
 - [77.7 Coding Tasks \(compile with g++ or clang++\)](#)
- [78 Hardware Fundamentals](#)
 - [78.1 The End of Free Performance](#)
 - [78.2 How Modern CPUs Execute Instructions](#)
 - [78.3 SIMD: Single Instruction, Multiple Data](#)
 - [78.4 The Roofline Model](#)
 - [78.5 Latency vs Throughput](#)
 - [78.6 Chip Architecture Landscape](#)
 - [78.7 Thermal and Power Constraints](#)
 - [78.8 Practical: Measuring Performance in C++](#)
 - [78.9 Coding Tasks \(use CoLab or notebook\)](#)
- [79 ARM and NEON](#)
 - [79.1 ARM Architecture Basics](#)
 - [79.2 NEON: 128-bit SIMD](#)
 - [79.3 I8MM: Integer Matrix Multiply](#)

- [79.4 SME and SME2: Scalable Matrix Extension](#)
- [79.5 SVE and SVE2: Scalable Vector Extensions](#)
- [79.6 Apple Silicon Specifics](#)
- [79.7 Auto-Vectorisation](#)
- [79.8 Coding Tasks \(compile with g++ or clang++ on ARM — Mac M-series or Linux aarch64\)](#)
- [80 x86 and AVX](#)
 - [80.1 The x86 SIMD Evolution](#)
 - [80.2 AVX2 Programming](#)
 - [80.3 AVX-512](#)
 - [80.4 Intel AMX: Matrix Multiply Hardware](#)
 - [80.5 Memory Alignment](#)
 - [80.6 Performance Pitfalls](#)
 - [80.7 Profiling](#)
 - [80.8 Coding Tasks \(compile with g++ or clang++ on x86 — Intel/AMD\)](#)
- [81 GPU Architecture and CUDA](#)
 - [81.1 GPU vs CPU: Fundamentally Different Designs](#)
 - [81.2 GPU Memory Hierarchy](#)
 - [81.3 CUDA Programming Model](#)
 - [81.4 Streams and Concurrency](#)
 - [81.5 Profiling CUDA Code](#)
 - [81.6 Advanced Optimisation Techniques](#)
 - [81.7 NVIDIA GPU Generations](#)
 - [81.8 Coding Tasks \(compile with nvcc\)](#)
- [82 Triton and TPUs](#)
 - [82.1 Triton: GPU Kernels in Python](#)
 - [82.2 Case Study: Flash Attention](#)
 - [82.3 TPU Architecture](#)
 - [82.4 Choosing the Right Tool](#)
 - [82.5 Coding Tasks \(use CoLab with GPU runtime\)](#)
- [83 RISC-V and Embedded Systems](#)
 - [83.1 The RISC-V Philosophy](#)
 - [83.2 RISC-V Base Architecture](#)
 - [83.3 The V Extension: RISC-V Vector Processing](#)
 - [83.4 Embedded ML: TinyML](#)
 - [83.5 RISC-V in AI Accelerators](#)
 - [83.6 Edge Deployment Constraints](#)
 - [83.7 Coding Tasks \(compile with g++ or riscv64-gcc cross-compiler\)](#)

- [84 Vulkan Compute and Cross-Platform GPU](#)
 - [84.1 Vulkan Architecture Overview](#)
 - [84.2 Compute Shaders in GLSL](#)
 - [84.3 The C++ Vulkan Setup](#)
 - [84.4 Kompute: Simplified Vulkan for ML](#)
 - [84.5 WebGPU: GPU Compute in the Browser](#)
 - [84.6 When to Use Vulkan](#)
 - [84.7 Coding Tasks \(compile with g++ -lvulkan, requires Vulkan SDK\)](#)
- [85 Quantisation](#)
 - [85.1 Why Quantise](#)
 - [85.2 Number Formats](#)
 - [85.3 The Quantisation Equation](#)
 - [85.4 Post-Training Quantisation \(PTQ\)](#)
 - [85.5 Quantisation-Aware Training \(QAT\)](#)
 - [85.6 Weight-Only Quantisation](#)
 - [85.7 Activation Quantisation](#)
 - [85.8 Mixed-Precision Quantisation](#)
 - [85.9 KV-Cache Quantisation](#)
 - [85.10 Coding Tasks \(use CoLab or notebook\)](#)
- [86 Efficient Architectures](#)
 - [86.1 StreamingLLM: Infinite-Length Generation](#)
 - [86.2 Sparse Attention](#)
 - [86.3 Linear Attention and State-Space Models](#)
 - [86.4 Multi-Query and Grouped-Query Attention](#)
 - [86.5 Mixture of Experts at Inference](#)
 - [86.6 Knowledge Distillation](#)
 - [86.7 Pruning](#)
 - [86.8 Neural Architecture Search \(NAS\)](#)
 - [86.9 Coding Tasks \(use CoLab or notebook\)](#)
- [87 Serving and Batching](#)
 - [87.1 Prefill vs Decode: Two Very Different Phases](#)
 - [87.2 Static Batching \(Naive\)](#)
 - [87.3 Continuous Batching](#)
 - [87.4 PagedAttention and vLLM](#)
 - [87.5 Scheduling Strategies](#)
 - [87.6 Disaggregated Serving](#)
 - [87.7 Multi-Model and LoRA Serving](#)
 - [87.8 Constrained and Guided Generation](#)

- [87.9 Request Routing](#)
- [87.10 Inference Metrics](#)
- [87.11 Coding Tasks \(use CoLab or notebook\)](#)
- [88 Edge Inference](#)
 - [88.1 Edge Constraints](#)
 - [88.2 The Model Compression Pipeline](#)
 - [88.3 On-Device Runtimes](#)
 - [88.4 The Compiler Stack](#)
 - [88.5 Hardware Targets](#)
 - [88.6 On-Device LLMs](#)
 - [88.7 Federated Learning](#)
 - [88.8 Latency Optimisation](#)
 - [88.9 Coding Tasks \(use CoLab or notebook\)](#)
- [89 Scaling and Deployment](#)
 - [89.1 Model Parallelism at Inference](#)
 - [89.2 Speculative Decoding](#)
 - [89.3 Prefix Caching](#)
 - [89.4 KV-Cache Eviction](#)
 - [89.5 Inference Frameworks](#)
 - [89.6 Cost Optimisation](#)
 - [89.7 Monitoring](#)
 - [89.8 Coding Tasks \(use CoLab or notebook\)](#)
- [90 Systems Design Fundamentals](#)
 - [90.1 Client-Server Architecture](#)
 - [90.2 Networking Protocols](#)
 - [90.3 DNS](#)
 - [90.4 Proxies](#)
 - [90.5 Load Balancing](#)
 - [90.6 Caching](#)
 - [90.7 Databases](#)
 - [90.8 Message Queues](#)
 - [90.9 Consistency Models](#)
 - [90.10 Resilience Patterns](#)
- [91 Cloud Computing](#)
 - [91.1 Cloud Service Models](#)
 - [91.2 Major Providers](#)
 - [91.3 Containers and Kubernetes](#)
 - [91.4 Storage](#)

- [91.5 Cloud Networking](#)
- [91.6 Serverless](#)
- [91.7 Cost Management](#)
- [91.8 Multi-Region Deployment](#)
- [91.9 Infrastructure as Code](#)
- [92 Large Scale Infrastructure](#)
 - [92.1 Scalability](#)
 - [92.2 Distributed Systems](#)
 - [92.3 Microservices](#)
 - [92.4 Data Pipelines](#)
 - [92.5 ML Training Infrastructure](#)
 - [92.6 Database Scaling](#)
 - [92.7 Search and Vector Systems](#)
 - [92.8 Observability](#)
 - [92.9 Reliability](#)
 - [92.10 CI/CD](#)
- [93 ML Systems Design](#)
 - [93.1 The ML System Lifecycle](#)
 - [93.2 Data Management](#)
 - [93.3 Training Infrastructure](#)
 - [93.4 Model Evaluation](#)
 - [93.5 Model Serving](#)
 - [93.6 Feature Engineering](#)
 - [93.7 ML Pipelines](#)
 - [93.8 Monitoring](#)
- [94 ML Design Examples](#)
 - [94.1 1. Recommendation System \(e.g., YouTube, Netflix, Spotify\)](#)
 - [94.2 2. Search Ranking \(e.g., Google, Bing\)](#)
 - [94.3 3. Ads Click Prediction](#)
 - [94.4 4. Fraud Detection](#)
 - [94.5 5. Content Moderation](#)
 - [94.6 6. Conversational AI \(RAG-Based Chatbot\)](#)
 - [94.7 7. Large-Scale Image Search](#)
 - [94.8 The Interview Framework](#)
- [95 AI for Finance](#)
- [96 AI for Biology](#)
- [97 Quantum Machine Learning](#)
- [98 Neuromorphic Computing](#)

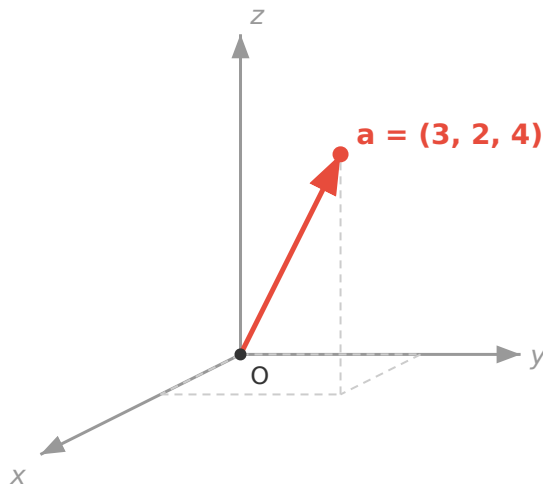
- [99 Maths, CS & AI Compendium](#)
 - [99.1 Overview](#)
 - [99.2 Background](#)
 - [99.3 MCP Server](#)
 - [99.4 Outline](#)
 - [99.5 Foreword](#)
 - [99.6 How To Study Better](#)
 - [99.7 Who is Henry Ndubuaku?](#)
 - [99.8 Citation](#)

1 Vector Spaces

Vector spaces form the mathematical playground where ML lives. This file covers vector addition, scalar multiplication, closure axioms, subspaces, and why nearly everything in AI is represented as vectors.

- Think of a Vector Space as a specific kind of playground where mathematical objects live, and each object is called a **vector**.
- Vector space is formally defined as an collection of vectors that can be added and scaled together without leaving the space.
- A useful non-example: the integers $\mathbb{Z}$ are **not** a vector space over the real numbers, since scaling 3 by 0.5 gives 1.5, which falls outside the set. The “without leaving the space” part of the definition is doing real work.
- For geometric intuition in machine learning (ML), we will always think of vectors as a point in Euclidean space, represented by it’ s coordinates.
- The vector **a** (denoted mathematically as lowercase letters in bold) has n coordinates, each representing a position along an axis.

$$\mathbf{a} = [a_1, a_2, a_3]$$

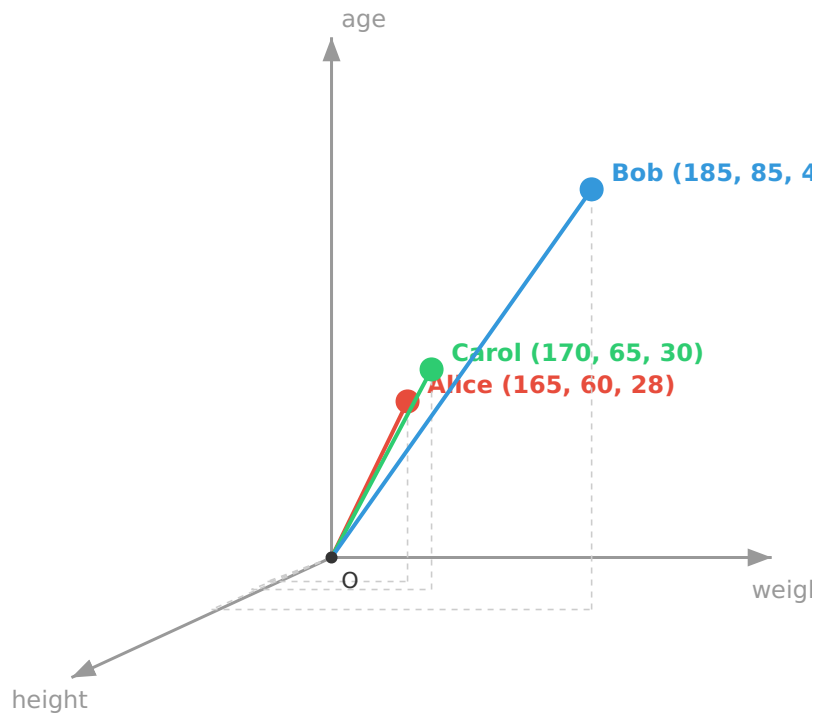


A vector $\mathbf{a} = (3, 2, 4)$ plotted in 3D space with x, y, z axes

- The vectors in the vector space live under a very specific, unbreakable set of rules:
 - **Vector Addition (Combining)**: You can take any two vectors and combine them to create a new one. Think of vectors as instructions for movement. If vector A means “walk 3 steps forward” and vector B means “walk 2 steps right,” adding them ($A + B$) creates a new, single instruction: “walk 3 steps forward and 2 steps right.”
 - **Scalar Multiplication (Scaling)**: You can take any vector and scale it using a regular number (a “scalar”). You can stretch it, shrink it, or reverse it. If vector A is “walk 3 steps forward,” scaling it by 2 makes it “walk 6 steps forward.” Scaling it by -1 flips it entirely to “walk 3 steps backward.”
- The **dimension** of a vector space is the number of independent directions it contains. $\mathbb{R}^2$ is 2-dimensional (needs 2 coordinates), while $\mathbf{a}$ above lives in $\mathbb{R}^3$.
- We can for instance represent any object, say, a human, as a vector, where h_1 = height in cm, h_2 = weight in kg, h_3 = age.

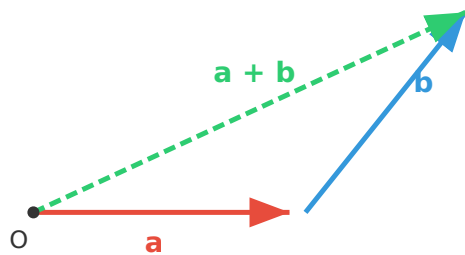
$$\mathbf{h} = [185, 75, 30]$$

- We have now created a vector space with a vector representing a human.
- We can represent multiple humans, and see how close or apart they are!



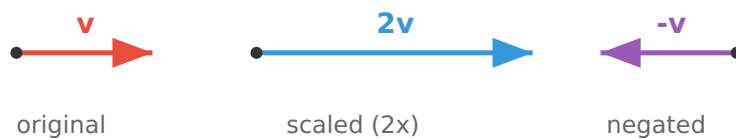
Three humans as vectors: Alice and Carol are close, Alice and Bob are far apart

- We can add more features, creating a rich representation of a human, often called feature vectors in ML.
- The more unique and meaningful features you have, the more descriptive the feature vector is, an important factor to remember.
- Beyond 3 dimensions, vectors become very difficult to visually inspect, inspiring a field of mathematics called **Linear Algebra**.
- Now, **Linear algebra** is the study of vectors, vector spaces and mappings between vectors.
- We represent almost every thing in AI/ML as vectors, making linear algebra the bedrock of the field.
- Vector addition can be performed by placing one vector on the tail of the other visually, and drawing from the origin to the endpoint.



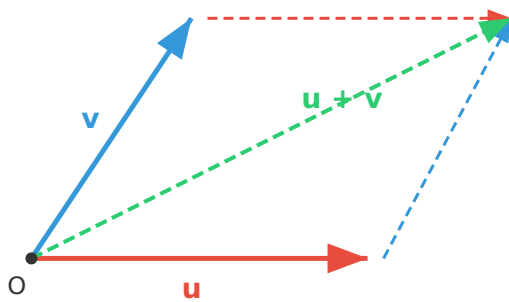
Vector addition: $\mathbf{a}$ (red) plus $\mathbf{b}$ (blue) gives resultant $\mathbf{a} + \mathbf{b}$ (green dashed)

- For two vectors $\mathbf{a} = (a_1, a_2)$ and $\mathbf{b} = (b_1, b_2)$: $\mathbf{a} + \mathbf{b} = (a_1 + b_1, a_2 + b_2)$
- Vectors can also be subtracted, with all addition rules applying too.
- Multiplying a vector by a scalar scales the vector by that factor in the same direction.



Scalar multiplication: $\mathbf{v}$ (red), $2\mathbf{v}$ (blue, doubled), $-\mathbf{v}$ (purple, reversed)

- For a scalar c and vector $\mathbf{v} = (v_1, v_2)$: $c\mathbf{v} = (cv_1, cv_2)$
- **Closure under Addition:** If you add any two vectors from the vector space, the result is also a vector within the same space: If $\mathbf{u} \in V$ and $\mathbf{v} \in V$, then $\mathbf{u} + \mathbf{v} \in V$
- **Closure under Scalar Multiplication:** If you multiply any vector from the vector space by a scalar, the result is a vector within the same space: If $\mathbf{v} \in V$ and $c \in F$, then $c\mathbf{v} \in V$
- **Commutativity of Addition:** For any two vectors $\mathbf{u}$ and $\mathbf{v}$: $\mathbf{u} + \mathbf{v} = \mathbf{v} + \mathbf{u}$



Parallelogram rule: both paths (u then v, or v then u) reach the same point

- Both paths through the parallelogram arrive at the same point.
- **(Zero Vector):** There exists a vector **0** such that for any vector **v**: $\mathbf{v} + \mathbf{0} = \mathbf{v}$



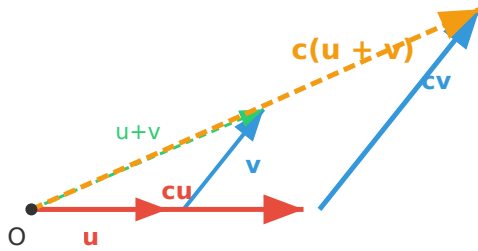
Zero vector: $\mathbf{v} + \mathbf{0} = \mathbf{v}$

- **Additive Inverse:** For every vector **v**, there exists a vector $-\mathbf{v}$ such that: $\mathbf{v} + (-\mathbf{v}) = \mathbf{0}$



Additive inverse: v (red) and -v (blue) cancel to zero

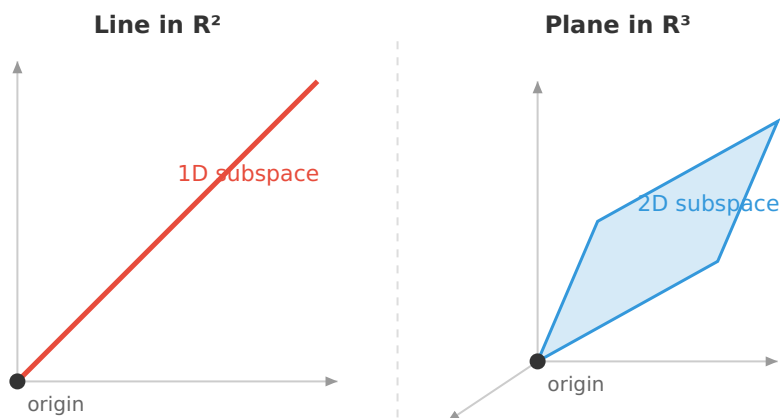
- **Distributivity 1:** For any scalar c and vectors **u**, **v**: $c(\mathbf{u} + \mathbf{v}) = c\mathbf{u} + c\mathbf{v}$



Distributivity: scaling the sum (gold) equals summing the scaled vectors

- Scaling the sum (gold) gives the same result as summing the scaled vectors.
- **Distributivity 2:** For any scalars c, d and vector $\mathbf{v}$: $(c + d)\mathbf{v} = c\mathbf{v} + d\mathbf{v}$
- **Associativity:** For any scalars c, d and vector $\mathbf{v}$: $(cd)\mathbf{v} = c(d\mathbf{v})$
- **Identity Element:** For any vector $\mathbf{v}$: $1\mathbf{v} = \mathbf{v}$, where 1 is the multiplicative identity in the field of scalars.
- Some examples of vector spaces:
 - **$\mathbb{R}^n$ (n-dimensional space):** all real numbers in an n-dimensional space, example vector $[1, 4, 3000, \dots, \text{nth item}]$, add two points or scale one, and you still land somewhere on the plane.
 - **Grayscale images:** a 28×28 image is just 784 pixel intensities, i.e. a vector in $\mathbb{R}^{784}$. Adding two images (blending) or scaling one (brightening) gives another image of the same size.
 - **Audio signals:** a 1-second clip sampled at 44.1kHz is a vector with 44,100 entries. Mixing two clips together is just vector addition.
 - **Polynomials:** adding two polynomials or scaling one by a number gives another polynomial, so they form a vector space too. Vectors don't have to look like arrows!
- A **subspace** is just a smaller playground inside the bigger one. Imagine 3D space as a room. A flat sheet of paper passing through the centre of the room is a subspace, and so is a single straight wire through the centre.

- The key requirement is that the subspace must pass through the origin. If you shift that sheet of paper off-centre, it stops being a subspace because the zero vector is no longer on it.



Subspaces: a line and a plane through the origin inside 3D space

- All the same rules from the vector space (addition, scaling, closure) still work inside a subspace. You can add or scale vectors within it and never “fall off” into the larger space.
- A line through the origin is a 1-dimensional subspace, a plane through the origin is a 2-dimensional subspace, and the full space is a subspace of itself.
- In ML, subspaces appear naturally. High-dimensional data often has structure that lives on a lower-dimensional subspace. Techniques like PCA find that subspace so we can work with the data more efficiently.

1.1 Coding Tasks (use CoLab or notebook)

1. Run code to verify the distributivity property, then Modify and play around to test other rules!

```
import jax.numpy as jnp

u = jnp.array([1, 2])
v = jnp.array([3, 0])
c = 2

lhs = c * (u + v)
rhs = c*u + c*v
```

```
print(f"LHS: {lhs}")
print(f"RHS: {rhs}")
```

1. Run code to visualise different vectors, then modify values for different coordinates to understand how each axis affects position.

```
import jax.numpy as jnp
import matplotlib.pyplot as plt

# Try changing these vectors!
a = jnp.array([3, 2, 4])
b = jnp.array([1, 4, 2])
c = jnp.array([4, 1, 3])

fig = plt.figure()
ax = fig.add_subplot(111, projection="3d")

for vec, name, color in [(a, "a", "red"), (b, "b", "blue"), (c, "c", "green")]:
    ax.quiver(0, 0, 0, *vec, color=color, arrow_length_ratio=0.1, linewidth=2,
label=name)

lim = int(jnp.abs(jnp.stack([a, b, c])).max()) + 1
ax.set_xlim([0, lim]); ax.set_ylim([0, lim]); ax.set_zlim([0, lim])
ax.set_xlabel("X"); ax.set_ylabel("Y"); ax.set_zlabel("Z")
ax.legend()
plt.show()
```

1.2 Side Quest (Some mathematical symbols to know)

Symbol	Meaning	Example
$\in$	is an element of	$x \in A$: x is in set A
$\notin$	is not an element of	$x \notin A$
$\subset$	is a proper subset of	$A \subset B$
$\subseteq$	is a subset of or equal to	$A \subseteq B$
$\cup$	union	$A \cup B$: things in A or B
$\cap$	intersection	$A \cap B$: things in both A and B
$\emptyset$	empty set	$A = \emptyset$

Symbol	Meaning	Example
$\mathbb{R}$	real numbers	$x \in \mathbb{R}$
$\mathbb{Z}$	integers	-2, -1, 0, 1, 2
$\mathbb{N}$	natural numbers	1, 2, 3, ...
$\mathbb{Q}$	rational numbers	fractions like 1/2
$\Rightarrow$	implies	$x > 2 \Rightarrow x > 1$
$\Leftrightarrow$	if and only if	$x = 2 \Leftrightarrow x^2 = 4$, with extra conditions
$\forall$	for all	$\forall x \in \mathbb{R}$
$\exists$	there exists	$\exists x$ such that $x^2 = 4$
$\neg$	not	$\neg P$: not P
$\wedge$	and	$P \wedge Q$
$\vee$	or	$P \vee Q$
$\therefore$	therefore	$x = 2, \therefore x^2 = 4$

2 Vector Properties

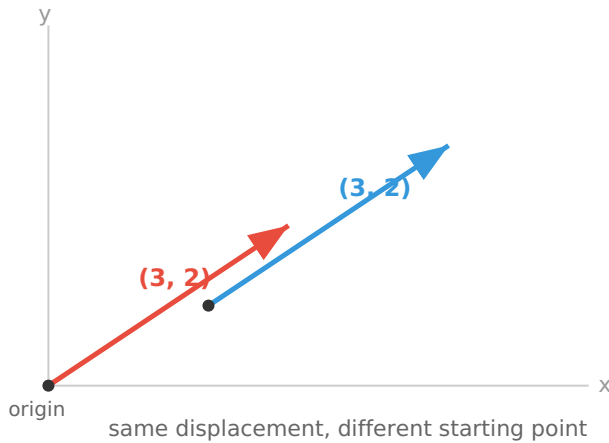
Vector properties describe the geometric and algebraic characteristics that define how vectors behave. This file covers magnitude, direction, unit vectors, equality, parallelism, orthogonality, and linear independence, the building blocks of every ML feature space.

- The **magnitude** (or length) of a vector tells you how far it reaches. Think of it as the length of the arrow. For a vector $\mathbf{a} = (a_1, a_2, a_3)$, its magnitude is:

$$\|\mathbf{a}\| = \sqrt{a_1^2 + a_2^2 + a_3^2}$$

- This is just the Pythagorean theorem extended to higher dimensions and measuring the straight-line distance from the origin to the point.
- The **direction** of a vector tells you where it points; simply visualise a straight line from the origin to the coordinate's point.

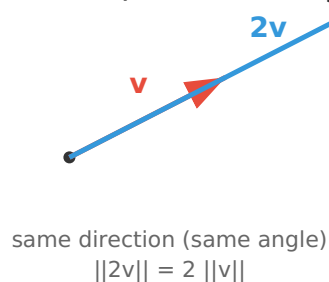
- When origin is not explicitly specified, we often imply $(0,0,\dots,0)$, the centerpoint, at least for visualisation purposes.
- Position doesn't matter, it's always about displacement: a vector $(3, 2)$ drawn from the origin and the same $(3, 2)$ drawn from another point are still equal.



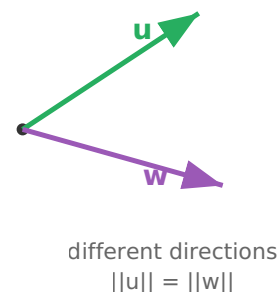
Vector equality: same $(3,2)$ vector drawn from two different starting points

- Two vectors can have the same length but point in completely different directions, or point the same way but differ in length.

Same direction, different magnitude



Same magnitude, different direction



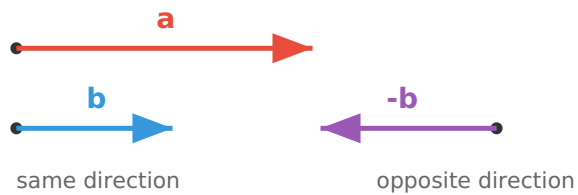
Same direction, different magnitudes (v and $2v$) vs same magnitude, different directions

- Two vectors are **equal** if and only if all their corresponding components match; same length, same direction, the exact same arrow.

$$\mathbf{a} = \mathbf{b} \Leftrightarrow a_i = b_i \text{ for all } i$$

- Two vectors are **parallel** if one is a scalar multiple of the other. They point along the same line, either in the same direction or exactly opposite.

$$\mathbf{a} \parallel \mathbf{b} \Leftrightarrow \mathbf{a} = k\mathbf{b} \text{ for some scalar } k \neq 0$$



Parallel vectors: a and b point the same way, a and $-b$ point opposite

- If $k > 0$, they point the same way. If $k < 0$, they point in opposite directions. Either way, they lie on the same line through the origin.
- Intuitively, parallel vectors carry no “new” directional information. One is just a stretched or flipped version of the other.
- Two vectors are **orthogonal** (perpendicular) if they point in completely independent directions. Moving along one gives you zero progress along the other.



90° , completely independent

Orthogonal vectors: u and v meet at a right angle

- Think of walking north and then walking east, these are orthogonal directions, no amount of walking north will ever move you east. We will encounter orthogonality very often.
- Orthogonality is central to ML: features that are orthogonal carry completely independent information, which is ideal for representation.
- More generally, any two vectors have an **angle** θ between them, ranging from 0° to 180° .
- This angle captures the entire relationship between two directions: 0° means parallel (same direction), 180° means parallel (opposite direction), and 90° means orthogonal. Everything in between is a blend.
- Most vector relationships in ML live somewhere in this spectrum. Later, we will see exact tools (dot product, cosine similarity) to compute this angle.

- A set of vectors is **linearly dependent** if at least one of them can be built from the others by scaling and adding. It brings no new information to the set.
- For example, if $\mathbf{c} = 2\mathbf{a} + 3\mathbf{b}$, then $\mathbf{c}$ is redundant, you already have everything $\mathbf{c}$ offers through $\mathbf{a}$ and $\mathbf{b}$.
- Parallel vectors are always linearly dependent, since one is just a scaled copy of the other. Any set containing the zero vector is also linearly dependent.
- Vectors are **linearly independent** if none of them can be built from the others. Each one contributes a genuinely new direction. Orthogonal vectors are always linearly independent.
- Some intuition: if you want to study different humans and represent them as a vector, linearly dependent vectors (humans) will skew the observation in favour of the oversampled data point, an important factor in designing dataset for training AI.
- In 2D, two linearly independent vectors can reach any point in the plane. In 3D, you need three. This idea of “how many independent vectors you need” connects directly to dimension.
- A vector is **sparse** when most of its components are zero. The opposite, most components being nonzero, is called **dense**.

$$\mathbf{s} = [0, 0, 3, 0, 0, 0, 1, 0, 0, 0]$$

- Sparsity matters because it affects both storage and computation. Sparse vectors can be stored and processed much more efficiently by only tracking the nonzero entries.
- A **unit vector** is a vector with magnitude exactly 1. It purely represents a direction with no length information. You can turn any vector into a unit vector by dividing by its magnitude:



a

$$= \frac{\mathbf{a}}{\|\mathbf{a}\|}$$

- This process is called **normalisation**. It strips away “how far” and keeps only “which way”, this is an important factor in machine learning.

- The standard unit vectors point along each axis:



i

$$= (1, 0, 0),$$



j

$$= (0, 1, 0),$$



k

$= (0, 0, 1)$. Any vector can be written as a combination of these, e.g. $(3, 2, 4) = 3$



i

$$+ 2$$



j

$$+ 4$$



k

2.1 Coding Tasks (use CoLab or notebook)

1. Compute the magnitude of a vector and verify it matches the Pythagorean theorem, then modify to compute the unit vector.

```
import jax.numpy as jnp

a = jnp.array([3.0, 4.0])

magnitude = jnp.sqrt(jnp.sum(a ** 2))
print(f"Magnitude of a: {magnitude}")
```

1. Check whether two vectors are parallel by testing if one is a scalar multiple of the other.

```
import jax.numpy as jnp

a = jnp.array([2, 4, 6])
b = jnp.array([1, 2, 3])

ratios = a / b
```

```
print(f"Ratios: {ratios}")
print(f"Parallel: {jnp.allclose(ratios, ratios[0])}")
```

3 Metrics and Norms

Norms measure the size of a vector; metrics measure the distance between two vectors. This file covers L1, L2, and L-infinity norms, Euclidean and cosine distance, and why choosing the right distance function is critical for kNN, clustering, and retrieval in ML.

- We know vectors have magnitude and direction. But how do we actually measure “how big” a single vector is, or “how far apart” two vectors are? This is where **norms** and **metrics** come in.
- In scalars, we know that $10 > 5$, because their values quantify them, but how can we quantify a vector? It’s **norm**, it measures the size of a single vector.
- The most familiar norm is the **Euclidean norm** (L2), which is just the magnitude formula we already know:

$$\|\mathbf{v}\|_2 = \sqrt{v_1^2 + v_2^2 + \dots + v_n^2}$$

- But there are other ways to measure size. Imagine you are in a city with a grid of streets. You cannot walk diagonally through buildings, so the “length” of your journey is the total blocks walked along each street. This is the **Manhattan norm** (L1):

$$\|\mathbf{v}\|_1 = |v_1| + |v_2| + \dots + |v_n|$$

- Or you might only care about the single largest component, ignoring the rest. This is the **Max norm** (L-infinity):

$$\|\mathbf{v}\|_\infty = \max(|v_1|, |v_2|, \dots, |v_n|)$$

- All three are special cases of the **general Lp norm**:

$$\|\mathbf{v}\|_p = (|v_1|^p + |v_2|^p + \dots + |v_n|^p)^{1/p}$$

- Setting $p = 2$ gives Euclidean, $p = 1$ gives Manhattan, and as $p \rightarrow \infty$ you get the Max norm. As p grows, the largest component contributes more and more, until eventually only it matters.
- Every norm must obey three rules:
 - **Non-negativity:** $\|\mathbf{v}\| \geq 0$, and $\|\mathbf{v}\| = 0$ only if $\mathbf{v} = \mathbf{0}$. Size is never negative, and only the zero vector has zero size.
 - **Scaling:** $\|c\mathbf{v}\| = |c| \cdot \|\mathbf{v}\|$. Doubling a vector doubles its size.
 - **Triangle inequality:** $\|\mathbf{u} + \mathbf{v}\| \leq \|\mathbf{u}\| + \|\mathbf{v}\|$. The shortcut is never longer than going the long way round.
- Now, a **metric** measures the distance between two vectors. Think of it as asking: “how far apart are these two points?”
- The simplest way to get a metric is to use a norm on the difference:
 $d(\mathbf{u}, \mathbf{v}) = \|\mathbf{u} - \mathbf{v}\|$. Subtract the two vectors, then measure the size of what remains.
- Using the Euclidean norm this gives us the familiar **Euclidean distance**:

$$d(\mathbf{u}, \mathbf{v}) = \sqrt{(u_1 - v_1)^2 + (u_2 - v_2)^2 + \dots + (u_n - v_n)^2}$$

- Using the Manhattan norm gives **Manhattan distance**, the total difference along each axis, like counting city blocks between two locations.
- Every metric must obey four rules:
 - **Non-negativity:** $d(\mathbf{u}, \mathbf{v}) \geq 0$. Distance is never negative.
 - **Identity:** $d(\mathbf{u}, \mathbf{v}) = 0$ if and only if $\mathbf{u} = \mathbf{v}$. Zero distance means the same point.
 - **Symmetry:** $d(\mathbf{u}, \mathbf{v}) = d(\mathbf{v}, \mathbf{u})$. The distance from A to B is the same as from B to A.
 - **Triangle inequality:** $d(\mathbf{u}, \mathbf{w}) \leq d(\mathbf{u}, \mathbf{v}) + d(\mathbf{v}, \mathbf{w})$. Going directly is never longer than taking a detour.

- So what is the relationship between the two? A norm measures one vector, a metric measures the gap between two. Every norm naturally creates a metric (by measuring the difference), but not every metric comes from a norm.
- For example, **Hamming distance** counts the number of positions where two vectors differ. It is a valid metric, but it does not come from any norm.
- In ML, choosing the right norm or metric matters.
- L2 distance squares each difference before summing, so a single large difference dominates the result.
- L1 distance sums the absolute differences, treating each one equally. A single large difference has less influence compared to L2.

3.1 Coding Tasks (use CoLab or notebook)

1. Compute L1, and L2 norms of the same vector. Try changing the values and notice which norm is most sensitive to large components vs many small ones. Then try computing the L_p norm for increasing values of p (e.g. 1, 2, 5, 10, 50, 100) and watch it converge towards the L-infinity value.

```
import jax.numpy as jnp

v = jnp.array([3.0, -4.0, 1.0])

l1 = jnp.sum(jnp.abs(v))
l2 = jnp.sqrt(jnp.sum(v ** 2))

print(f"L1: {l1}, L2: {l2:.2f}")
```

1. Compute the Euclidean and Manhattan distance between two vectors. Try moving the vectors closer or further apart and observe how each distance responds differently.

```
import jax.numpy as jnp

u = jnp.array([1.0, 2.0, 3.0])
v = jnp.array([4.0, 0.0, 1.0])

euclidean = jnp.sqrt(jnp.sum((u - v) ** 2))
manhattan = jnp.sum(jnp.abs(u - v))
```

```
print(f"Euclidean: {euclidean:.2f}, Manhattan: {manhattan}")
```

4 Vector Products

Vector products are the fundamental operations for measuring similarity and computing projections. This file covers inner products, the dot product, cosine similarity, the cross product, and outer products, operations that power attention mechanisms, embeddings, and geometric reasoning in AI.

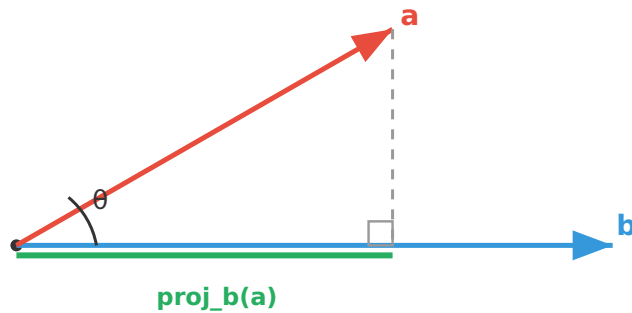
- We have seen how to add and scale vectors. But can we multiply two vectors together? It turns out there is more than one way to do it, and each answers a different question.
- An **inner product** is the general idea: a function that takes two vectors and produces a single number (a scalar). It is the abstract blueprint for “multiplying” vectors.
- Any inner product must satisfy three rules:
 - **Positive definiteness:** $\langle \mathbf{v}, \mathbf{v} \rangle \geq 0$, and equals zero only for the zero vector. Multiplying a vector with itself always gives a non-negative result.
 - **Symmetry:** $\langle \mathbf{u}, \mathbf{v} \rangle = \langle \mathbf{v}, \mathbf{u} \rangle$. The order does not matter.
 - **Linearity:** $\langle a\mathbf{u} + b\mathbf{v}, \mathbf{w} \rangle = a \langle \mathbf{u}, \mathbf{w} \rangle + b \langle \mathbf{v}, \mathbf{w} \rangle$. It distributes over addition and scaling.
- The **dot product** is the most common inner product. It is the concrete version you will use almost everywhere. For two vectors $\mathbf{a} = (a_1, a_2, \dots, a_n)$ and $\mathbf{b} = (b_1, b_2, \dots, b_n)$:

$$\mathbf{a} \cdot \mathbf{b} = a_1 b_1 + a_2 b_2 + \dots + a_n b_n$$

- Multiply matching components, then add everything up. That is all it is.
- But what does this number mean? The dot product has a beautiful geometric interpretation:

$$\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \|\mathbf{b}\| \cos(\theta)$$

$$\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \|\mathbf{b}\| \cos(\theta)$$



Dot product: vector $\mathbf{a}$ projected onto $\mathbf{b}$, with angle θ and projection shown

- This connects the dot product directly to the angle θ between the two vectors. The result tells you how much the two vectors “agree” in direction.
- If they point the same way ($\theta = 0^\circ$), $\cos(\theta) = 1$ and the dot product is maximised.
- If they are orthogonal ($\theta = 90^\circ$), $\cos(\theta) = 0$ and the dot product is exactly zero. This gives us a precise test for orthogonality.
- If they point in opposite directions ($\theta = 180^\circ$), $\cos(\theta) = -1$ and the dot product is negative.
- A vector dotted with itself gives its magnitude squared: $\mathbf{a} \cdot \mathbf{a} = \|\mathbf{a}\|^2$.
- The dot product also gives us **projection**, the shadow one vector casts onto another. The projection of $\mathbf{a}$ onto $\mathbf{b}$ is:

$$\text{proj}_{\mathbf{b}}(\mathbf{a}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{b}\|^2} \mathbf{b}$$

- Think of shining a light straight down onto $\mathbf{b}$. The shadow of $\mathbf{a}$ on that line is the projection. It tells you how much of $\mathbf{a}$ lies in the direction of $\mathbf{b}$.
- **Cosine similarity** normalises the dot product by dividing out both magnitudes:

$$\cos(\theta) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}$$

- This gives a value between -1 and 1 that measures direction alignment, ignoring how long the vectors are. It is widely used in ML to compare things like documents, embeddings, and user preferences.

- Now, the dot product takes two vectors and returns a scalar. The **cross product** does the opposite, it takes two vectors and returns a new vector.
- The cross product $\mathbf{a} \times \mathbf{b}$ produces a vector that is perpendicular to both $\mathbf{a}$ and $\mathbf{b}$:

$$\mathbf{a} \times \mathbf{b} = (a_2b_3 - a_3b_2, a_3b_1 - a_1b_3, a_1b_2 - a_2b_1)$$

- The cross product only works in 3D. While the dot product works in any number of dimensions, the cross product is specific to three-dimensional space.
- Its magnitude equals the area of the parallelogram formed by the two vectors:

$$\|\mathbf{a} \times \mathbf{b}\| = \|\mathbf{a}\| \|\mathbf{b}\| \sin(\theta)$$

- Notice the pattern: the dot product uses $\cos(\theta)$ and the cross product uses $\sin(\theta)$. The dot product measures how much two vectors align, the cross product measures how much they differ in direction.
- The direction of the result follows the **right-hand rule**: curl the fingers of your right hand from $\mathbf{a}$ towards $\mathbf{b}$, and your thumb points in the direction of $\mathbf{a} \times \mathbf{b}$.
- Unlike the dot product, the cross product is **not commutative**:
 $\mathbf{a} \times \mathbf{b} = -(\mathbf{b} \times \mathbf{a})$. Swapping the order flips the direction.
- If two vectors are parallel, their cross product is the zero vector (since $\sin(0^\circ) = 0$). No area, no perpendicular direction.
- What happens when you combine three vectors using both products? This gives us **triple products**.
- The **scalar triple product** $\mathbf{a} \cdot (\mathbf{b} \times \mathbf{c})$ first takes the cross product of two vectors, then dots the result with the third. The output is a single number that equals the volume of the parallelepiped (a slanted 3D box) formed by the three vectors.
- If the scalar triple product is zero, the three vectors are **coplanar**, they all lie in the same flat plane and form no volume.
- The order can be cycled without changing the result:
 $\mathbf{a} \cdot (\mathbf{b} \times \mathbf{c}) = \mathbf{b} \cdot (\mathbf{c} \times \mathbf{a}) = \mathbf{c} \cdot (\mathbf{a} \times \mathbf{b})$.
- The **vector triple product** $\mathbf{a} \times (\mathbf{b} \times \mathbf{c})$ applies the cross product twice and returns a vector. It expands neatly using the identity:

$$\mathbf{a} \times (\mathbf{b} \times \mathbf{c}) = (\mathbf{a} \cdot \mathbf{c})\mathbf{b} - (\mathbf{a} \cdot \mathbf{b})\mathbf{c}$$

- The result always lies in the plane spanned by $\mathbf{b}$ and $\mathbf{c}$. Note that the cross product is **not associative**: $\mathbf{a} \times (\mathbf{b} \times \mathbf{c}) \neq (\mathbf{a} \times \mathbf{b}) \times \mathbf{c}$.

4.1 Coding Tasks (use CoLab or notebook)

1. Compute the dot product of two vectors and use it to find the angle between them. Try making them orthogonal, parallel, or opposite and see how the angle changes.

```
import jax.numpy as jnp

a = jnp.array([1.0, 2.0, 3.0])
b = jnp.array([4.0, -1.0, 2.0])

dot = jnp.dot(a, b)
angle = jnp.arccos(dot / (jnp.linalg.norm(a) * jnp.linalg.norm(b)))

print(f"Dot product: {dot}")
print(f"Angle: {jnp.degrees(angle):.1f}°")
```

1. Compute the cross product of two 3D vectors and verify the result is perpendicular to both by checking that its dot product with each original vector is zero.

```
import jax.numpy as jnp

a = jnp.array([1.0, 0.0, 0.0])
b = jnp.array([0.0, 1.0, 0.0])

cross = jnp.cross(a, b)

print(f"a x b = {cross}")
print(f"Perpendicular to a: {jnp.dot(cross, a) == 0}")
print(f"Perpendicular to b: {jnp.dot(cross, b) == 0}")
```

5 Basis and Duality

Bases define the coordinate systems of vector spaces, and duality reveals how linear functions act on vectors. This file covers linear independence, spanning sets, change

of basis, dual spaces, and covectors, concepts behind PCA, feature transforms, and attention queries in ML.

- We have seen that vectors live in spaces with a certain number of dimensions. But what defines those dimensions? This is where **basis vectors** come in.
- A **basis** is a set of vectors that can build every other vector in the space through scaling and adding (linear combination), with no redundancy. They are the building blocks of the space.
- A basis must satisfy two conditions:
 - **Linearly independent**: No basis vector can be built from the others. Each one contributes a genuinely new direction.
 - **Spanning**: Every vector in the space can be expressed as a combination of the basis vectors. Nothing is left out.
- The number of vectors in a basis equals the **dimension** of the space. In $\mathbb{R}^2$ you need 2, in $\mathbb{R}^3$ you need 3, and so on.
- The most natural basis is the **standard basis**, the unit vectors along each axis:
 - In $\mathbb{R}^2$:
 - ⌚
i
= (1, 0) and
 - ⌚
j
= (0, 1)
 - In $\mathbb{R}^3$:
 - ⌚
i
= (1, 0, 0),
 - ⌚
j
= (0, 1, 0),
 - ⌚
k
= (0, 0, 1)

- Any vector is just a weighted sum of these basis vectors. The vector $(3, 2)$ is really 3

⊙

i

+ 2

⊙

j

The weights (3 and 2) are the **coordinates** of the vector in that basis.

- But the standard basis is not the only valid basis. In $\mathbb{R}^2$, the vectors $(1, 1)$ and $(-1, 1)$ also form a basis. They are linearly independent and can reach any point in the plane. The same vector will just have different coordinates in this new basis.
- A **change of basis** re-expresses the same vector using different basis. The vector has not moved, we are just describing it from a different perspective.
- This is done by multiplying by a **change of basis matrix** P , whose columns are the new basis vectors written in the old coordinates. To go back, multiply by P^{-1} .
- Think of it as two friends giving directions to the same café. You navigate by streets: $(3, 2)$ means “3 blocks east, 2 blocks north”. Your friend navigates by diagonals: their directions are $\mathbf{b}_1 = (1, 1)$ (north-east) and $\mathbf{b}_2 = (-1, 1)$ (north-west), the new basis from earlier. Same café, two languages for describing how to get there.
- Your friend says the café is at $(2.5, -0.5)$: “walk 2.5 steps along my north-east diagonal, then half a step backwards along my north-west diagonal”. To find out what that means in your street language, you simply follow their recipe:

$$2.5 \begin{bmatrix} 1 \\ 1 \end{bmatrix} - 0.5 \begin{bmatrix} -1 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 2 \end{bmatrix}$$

- It is the same café you call $(3, 2)$!
- That “follow the recipe” step is exactly what multiplying by P does. Stack the friend’s basis vectors as columns, and multiplying by a vector of their coordinates takes 2.5 of the first column and -0.5 of the second, the recipe again, just written compactly:

$$P = \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}, \quad P \begin{bmatrix} 2.5 \\ -0.5 \end{bmatrix} = \begin{bmatrix} 3 \\ 2 \end{bmatrix}$$

- Translating the other way, from your (3, 2) into the friend's language, means undoing the recipe, and undoing P is precisely what P^{-1} does:

$$P^{-1} \begin{bmatrix} 3 \\ 2 \end{bmatrix} = \begin{bmatrix} 2.5 \\ -0.5 \end{bmatrix}$$

- Nothing moved. (3, 2) and (2.5, -0.5) are two descriptions of one point, and P is the dictionary for translating between them.
- In ML, change of basis appears frequently. PCA, for example, finds a new basis (the principal components) where the data is easier to understand, the axes align with the directions of greatest variation.
- The next set of concepts would be abstract and quite challenging to grasp for now, until when we apply the concepts in later chapters, so brace yourself for impact, sorry.

- Now, there is a deeper idea hiding here. When we write $\mathbf{v} = (3, 2)$, the coordinates 3 and 2 are really the result of “measuring” $\mathbf{v}$ along each basis direction. The first coordinate asks “how much of

⊙

i

is in $\mathbf{v}$?” , the second asks “how much of

⊙

j

?”

- In the café story, these measurements were the friend's questions: “how far along my north-east diagonal did you walk?” and “how far along my north-west one?” . Every basis comes with its own set of questions, one per direction.
- Each question is a **linear functional**: a function that takes in a vector and returns a single number, the reading.
- The word linear means the functional respects the two vector space operations. Measure a sum of vectors and you get the sum of the readings; double a vector and the reading doubles:

$$f(\mathbf{u} + \mathbf{v}) = f(\mathbf{u}) + f(\mathbf{v}), \quad f(c\mathbf{v}) = c f(\mathbf{v})$$

- In other words, linear functionals are honest rulers. Vectors are the objects, linear functionals are the rulers that measure them, and the collection of all possible rulers forms the **dual space** V^* .
- For every basis $\{\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_n\}$ there is a matched set of rulers, the **dual basis** $\{\mathbf{e}_1^*, \mathbf{e}_2^*, \dots, \mathbf{e}_n^*\}$, where $\mathbf{e}_i^*$ answers exactly one question: “how many steps of $\mathbf{e}_i$?” . A well-calibrated ruler reads 1 on its own basis vector and completely ignores all the others, which is written compactly using the **Kronecker delta** δ_{ij} :

$$\mathbf{e}_i^*(\mathbf{e}_j) = \delta_{ij} = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{if } i \neq j \end{cases}$$

- That tiny calibration rule is all a ruler needs, because linearity does the rest. Watch $\mathbf{e}_1^*$ read the first coordinate of $\mathbf{v} = 3\mathbf{e}_1 + 2\mathbf{e}_2$:

$$\mathbf{e}_1^*(3\mathbf{e}_1 + 2\mathbf{e}_2) = 3 \mathbf{e}_1^*(\mathbf{e}_1) + 2 \mathbf{e}_1^*(\mathbf{e}_2) = 3 \cdot 1 + 2 \cdot 0 = 3$$

- The ruler ignores every direction except its own and reports the coordinate. Coordinates are dual basis readings.
- So what does a ruler look like concretely? In $\mathbb{R}^n$, every linear functional is just a row of numbers applied with a dot product. And for the friend’ s basis we have already built the rulers without noticing: translating $(3, 2)$ into the friend’ s language meant multiplying by P^{-1} , and each row of P^{-1} produced one coordinate. **The rows of P^{-1} are the dual basis:**

$$P^{-1} = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 2 & -1 \end{bmatrix} \Rightarrow \mathbf{b}_1^* = (0.5, 0.5), \quad \mathbf{b}_2^* = (-0.5, 0.5)$$

- Check it against the café trip: $\mathbf{b}_1^* \cdot (3, 2) = 0.5 \cdot 3 + 0.5 \cdot 2 = 2.5$, exactly the friend’ s first coordinate. The two halves of this file are one idea: a change of basis swaps the building blocks, and the dual basis is the matched set of rulers that reads off the new coordinates.
- The calibration rule was hiding in plain sight too: $P^{-1}P = I$ says precisely that ruler i reads 1 on basis vector i and 0 on the others. The identity matrix is the Kronecker delta written out as a table.

- One warning. It is tempting to guess that the ruler for $\mathbf{b}_1$ is $\mathbf{b}_1$ itself, i.e. just dot with $(1, 1)$. But $(1, 1) \cdot (3, 2) = 5$, double the true coordinate of 2.5. Dotting with a basis vector reads off its own coordinate only when the basis is **orthonormal** (mutually perpendicular, each of length 1), like the standard basis. That is the only reason $\mathbf{e}_1^*$ happens to look identical to $\mathbf{e}_1$. For every other basis, the rulers live in the rows of P^{-1} .
- This also explains the dot product's double life. Every vector $\mathbf{u}$ secretly defines a ruler (measure $\mathbf{v}$ by computing $\mathbf{u} \cdot \mathbf{v}$), and every ruler is a dot product with some vector. In finite dimensions, the dual space is essentially a mirror image of the original space.
- Duality may seem abstract now, but it underlies many practical ideas: coordinates are dual basis evaluations, the dot product is a duality pairing, and transformations like attention in neural networks operate by having one set of vectors “query” another, which is duality in action.

5.1 Coding Tasks (use CoLab or notebook)

1. Express a vector in two different bases and verify they represent the same point. Try creating your own basis and see what coordinates the vector gets.

```
import jax.numpy as jnp

v = jnp.array([3.0, 2.0])

# Standard basis: coordinates are just the components
print(f"Standard basis coords: {v}")

# New basis: (1,1) and (-1,1)
P = jnp.array([[1.0, -1.0],
               [1.0,  1.0]])
new_coords = jnp.linalg.solve(P, v)
print(f"New basis coords: {new_coords}")

# Verify: reconstruct from new coords
```



```
reconstructed = new_coords[0] * P[:, 0] + new_coords[1] * P[:, 1]
print(f"Reconstructed: {reconstructed}")
```

1. Build the dual basis for the friend's basis from task 1 (the rows of P^{-1}) and verify each ruler reads off exactly one coordinate. Then see why dotting with a basis vector itself only works for orthonormal bases.

```
import jax.numpy as jnp

v = jnp.array([3.0, 2.0])

# The friend's basis (1,1) and (-1,1), stacked as columns
P = jnp.array([[1.0, -1.0],
               [1.0, 1.0]])

# The dual basis: the rows of  $P^{-1}$ 
P_inv = jnp.linalg.inv(P)
b1_star, b2_star = P_inv[0], P_inv[1]
print(f"b1* = {b1_star}, b2* = {b2_star}")

# Each ruler reads one coordinate in the friend's basis (matches task 1!)
print(f"b1*(v) = {jnp.dot(b1_star, v)}" # 2.5
print(f"b2*(v) = {jnp.dot(b2_star, v)}" # -0.5

# Calibration: 1 on its own basis vector, 0 on the other
print(f"b1*(b1) = {jnp.dot(b1_star, P[:, 0])}, b1*(b2) = {jnp.dot(b1_star, P[:,
1])}")

# The trap: dotting with b1 itself does NOT give the coordinate
print(f"b1 . v = {jnp.dot(P[:, 0], v)} (5.0, not 2.5 -- b1 is not orthonormal)")

# The standard basis IS orthonormal, so there the shortcut works
e1 = jnp.array([1.0, 0.0])
print(f"e1 . v = {jnp.dot(e1, v)} (3.0, the first standard coordinate)")
```

6 Matrix Properties

Matrices are the data structures that store datasets, encode transformations, and define every neural network layer. This file covers matrix dimensions, elements,

transpose, trace, determinant, inverse, rank, and null space, the foundational properties used throughout linear algebra and ML.

- At its core, a **matrix** is a rectangular grid of numbers arranged in rows and columns. If a vector is a single list of numbers, a matrix as a stack of vectors.

```
A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}
```

- If a single person is described by the vector [age, height, weight], then three people form a matrix where each row is one person:

```
\begin{bmatrix} 25 & 170 & 65 \\ 30 & 180 & 80 \\ 22 & 160 & 55 \end{bmatrix}
```

- This matrix has 3 rows and 3 columns, so we call it a 3×3 matrix.
- Each number in the grid is called an **element** or **entry**, identified by its row and column: A_{ij} is the element in row i , column j .
- The **transpose** of a matrix flips it along its diagonal, turning rows into columns and columns into rows. If A is $m \times n$, then A^T is $n \times m$.

```
A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \quad \rightarrow \quad A^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}
```

- Multiplying a matrix by its transpose always gives a square matrix: AA^T is $m \times m$ and A^TA is $n \times n$.
- The **trace** of a square matrix is the sum of its diagonal elements:
 $\text{tr}(A) = A_{11} + A_{22} + \dots + A_{nn}$. The trace equals the sum of the eigenvalues (which we will see later).

$$\begin{bmatrix} \mathbf{1} & 5 & 3 \\ 2 & \mathbf{4} & 7 \\ 6 & 8 & \mathbf{9} \end{bmatrix}$$

Trace: the sum of diagonal entries

- For the matrix above, $\text{tr}(A) = 1 + 4 + 9 = 14$. Only the highlighted diagonal matters.

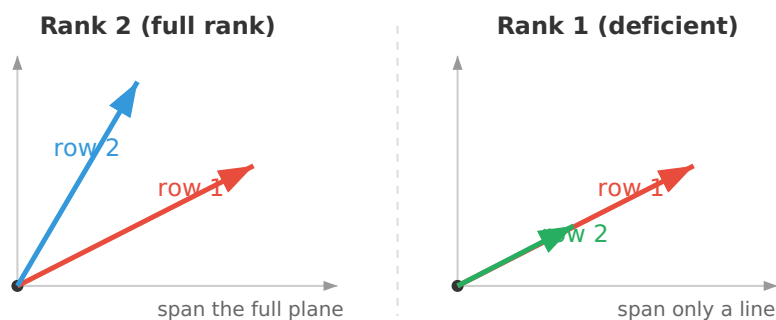
- If two matrices represent the same linear transformation under different bases, their traces will be the same. The trace is “basis-independent.”
- The **rank** of a matrix is the number of linearly independent rows (or equivalently, columns). It tells you how much “useful information” the matrix carries.
- For example, the following matrix has rank 2 because neither row is a multiple of the other:

```
\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}
```

But this matrix has rank 1 because the second row is just twice the first, so it adds no new information:

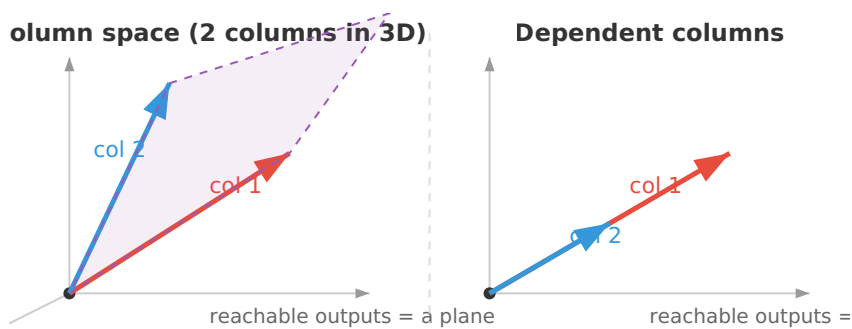
```
\begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}
```

- A 5×3 matrix can have rank at most 3. If some rows are just scaled or combined versions of others, the rank drops. A matrix with maximum possible rank is called **full rank**.



Rank: independent rows span the full space vs. dependent rows span only a subspace

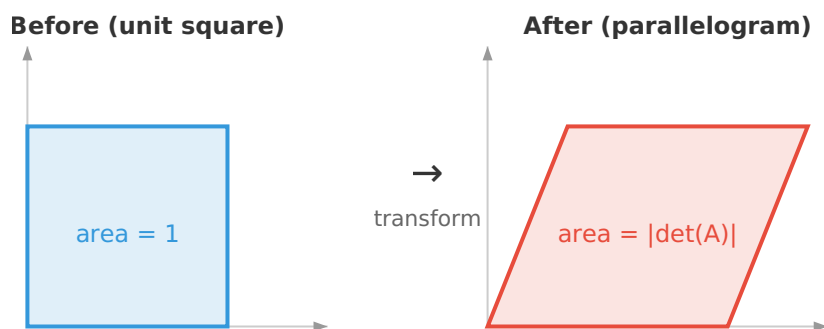
- A square matrix is invertible (has an inverse) if and only if it is full rank.
- The rank is connected to the **null space** (the set of vectors that the matrix maps to zero) through the **rank-nullity theorem**: $\text{rank}(A) + \text{nullity}(A) = \text{number of columns of } A$. What the matrix keeps (rank) plus what it destroys (nullity) equals the total dimension.
- The **column space** of a matrix is the set of all possible outputs when you multiply the matrix by any vector. It is spanned by the columns of the matrix. If a matrix has 3 columns but only 2 are independent, the column space is a 2D plane, not all of 3D space.



Column space: independent columns span a plane, dependent columns span only a line

- The **row space** is the same idea but from the perspective of rows. The rank equals the dimension of both the column space and the row space, so they always agree.
- Together, the column space tells you “what outputs can this matrix produce?” and the null space tells you “what inputs get mapped to zero?” These two spaces completely describe what the matrix does.
- The **determinant** of a square matrix is a single number that captures how the matrix scales space. Think of a 2×2 matrix as transforming a unit square into a parallelogram. The determinant is the area of that parallelogram (with a sign).

$$\det \begin{bmatrix} a & b \\ c & d \end{bmatrix} = ad - bc$$



Determinant: the area scaling factor of a linear transformation

- For example:

$$\det \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix} = 2 \cdot 3 - 1 \cdot 0 = 6$$

The transformation stretches the unit square into a parallelogram with area 6.

- If the determinant is positive, the transformation preserves orientation (things don't get “flipped”). If negative, it flips orientation (like a mirror reflection).

If zero, the matrix squashes space into a lower dimension, collapsing the parallelogram to a line or point.

- A matrix with determinant zero is called **singular**. It has no inverse and has lost information permanently.
- For matrices larger than 2×2 , the determinant is computed using **minors** and **cofactors**. The **minor** M_{ij} is the determinant of the smaller matrix you get by deleting row i and column j .

$$\begin{bmatrix} 1 & 5 & 3 \\ 2 & 7 & 4 \\ 6 & 8 & 9 \end{bmatrix} = \begin{bmatrix} 2 & 4 \\ 6 & 9 \end{bmatrix}$$

Minor M_{12} : delete row 1 and column 2

Minor: delete a row and column to get a smaller matrix

- The **cofactor** $C_{ij} = (-1)^{i+j} M_{ij}$ attaches a sign to each minor (alternating like a checkerboard: $+, -, +, \dots$). The determinant of the full matrix is then the sum along any row or column: $\det(A) = \sum_j A_{1j} \cdot C_{1j}$. This is called **cofactor expansion**.
- The **inverse** of a square matrix A , written A^{-1} , is the matrix that undoes what A does: $AA^{-1} = A^{-1}A = I$ (the identity matrix). Only non-singular matrices have inverses.
- For a 2×2 matrix, the inverse has a direct formula:

$$\begin{bmatrix} a & b \\ d & -b \end{bmatrix}^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

Notice the determinant in the denominator, which is why singular matrices (determinant zero) have no inverse.

- The **condition number** measures how sensitive a matrix is to small changes in its input. It is defined as $\kappa(A) = \|A\| \cdot \|A^{-1}\|$.
- A condition number close to 1 means the matrix is **well-conditioned**: small input changes produce small output changes. A large condition number means it is **ill-conditioned**: tiny errors get amplified enormously. Orthogonal and

identity matrices have condition number 1, while singular matrices have infinite condition number.

- For example, the following matrix has condition number 10^8 . One direction is scaled normally while the other is nearly squashed to zero, so small perturbations along that direction get wildly distorted:

```
\begin{bmatrix} 1 & 0 \\ 0 & 10^{-8} \end{bmatrix}
```

- Just as vectors have norms (length), matrices have **norms** that measure their “size.” The most common is the **Frobenius norm**, which treats the matrix as a long vector and computes its length:

```
\|A\|_F = \sqrt{\sum_i \sum_j A_{ij}^2}
```

- For example:

```
\left\| \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \right\|_F = \sqrt{1 + 4 + 9 + 16} = \sqrt{30} \approx 5.48
```

- The **spectral norm** $\|A\|_2$ is the largest singular value of A. It measures the maximum amount the matrix can stretch any unit vector. In ML, matrix norms are used for weight regularisation (penalising large weights) and monitoring training stability.
- A symmetric matrix A is **positive definite** if for every non-zero vector $\mathbf{x}$: $\mathbf{x}^T \mathbf{A} \mathbf{x} > 0$. This quadratic form always produces a positive number.
- For example, the following matrix is positive definite:

```
A = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}
```

Pick any vector, say $\mathbf{x} = [1, -1]^T$: $\mathbf{x}^T \mathbf{A} \mathbf{x} = 2 - 1 - 1 + 3 = 3 > 0$. No matter which non-zero $\mathbf{x}$ you try, you always get a positive result.

- Positive definite matrices are important because they guarantee that optimisation problems have a unique minimum.
- If the condition is relaxed to $\mathbf{x}^T \mathbf{A} \mathbf{x} \geq 0$ (allowing zero), the matrix is **positive semi-definite** (PSD). PSD matrices come up constantly: covariance matrices, kernel matrices in SVMs, and Hessians at local minima are all PSD. The

difference is that PSD allows some directions to be “flat” (zero curvature) rather than strictly curving upward.

6.1 Coding Tasks (use CoLab or notebook)

1. Compute the trace, rank, and determinant of a matrix. Try making one row a multiple of another and see how rank and determinant change.

```
import jax.numpy as jnp

A = jnp.array([[1.0, 2.0],
               [3.0, 4.0]])

print(f"Trace: {jnp.trace(A)}")
print(f"Rank: {jnp.linalg.matrix_rank(A)}")
print(f"Determinant: {jnp.linalg.det(A):.2f}")
```

1. Compute the inverse of a matrix, multiply it by the original, and verify you get the identity. Then try a singular matrix and observe what happens.

```
import jax.numpy as jnp

A = jnp.array([[1.0, 2.0],
               [3.0, 4.0]])

A_inv = jnp.linalg.inv(A)
print(f"A * A_inv:\n{A @ A_inv}")
```

7 Matrix Types

Special matrix structures unlock computational shortcuts and mathematical guarantees. This file covers identity, diagonal, symmetric, triangular, orthogonal, positive definite, sparse, and stochastic matrices, types that appear in covariance estimation, graph algorithms, regularisation, and Markov chains.

- Not all matrices are the same. Different structures give matrices special properties that make them faster to compute with, easier to reason about, or both. Here are the types you will encounter most.

- A **square matrix** has the same number of rows and columns ($n \times n$). Most of the interesting properties (determinant, eigenvalues, inverse) only apply to square matrices.
- The **identity matrix** I is a square matrix with 1s on the diagonal and 0s everywhere else. It is the “do nothing” transformation: $AI = IA = A$ for any compatible matrix A .

$$I = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- The **zero matrix** O has all elements equal to zero. It maps every vector to the zero vector, destroying all information.
- A **diagonal matrix** is all zeros except on the main diagonal. Multiplying a vector by a diagonal matrix simply scales each component independently, making it very efficient.

$$D = \begin{bmatrix} 3 & 0 \\ 0 & 7 \end{bmatrix}$$

- A **symmetric matrix** equals its own transpose: $A = A^T$, meaning $A_{ij} = A_{ji}$. Symmetric matrices have the special property that their eigenvectors are always perpendicular to each other. Covariance matrices are always symmetric.

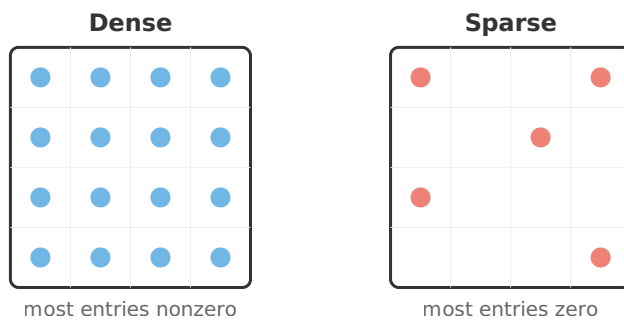
$$S = \begin{bmatrix} 3 & -1 \\ -1 & 6 \end{bmatrix}$$

- A **triangular matrix** has all zeros on one side of the diagonal. **Lower triangular** has zeros above, **upper triangular** has zeros below. They are essential for solving systems of equations efficiently through forward or back substitution.

$$L = \begin{bmatrix} 2 & 0 & 0 \\ 1 & 3 & 0 \\ -1 & 2 & 4 \end{bmatrix} \quad U = \begin{bmatrix} 5 & -1 & 2 \\ 0 & 1 & 3 \\ 0 & 0 & -2 \end{bmatrix}$$

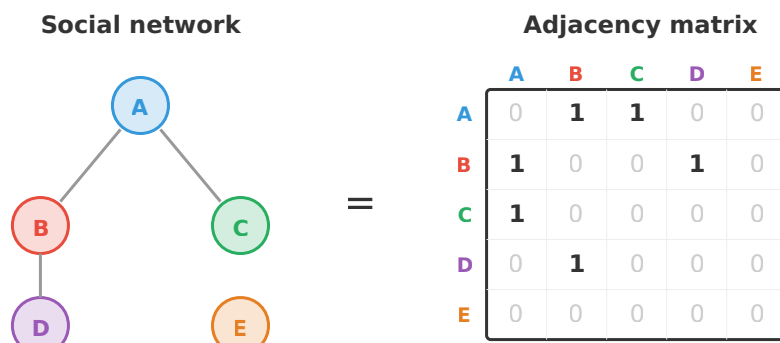
- The determinant of a triangular matrix is simply the product of its diagonal elements.
- An **orthogonal matrix** has the property that its transpose equals its inverse: $Q^T Q = Q Q^T = I$.
- This means you can “undo” the transformation just by transposing, which is computationally cheap. Its columns are orthonormal (unit length and mutually perpendicular).

- A **sparse matrix** has most of its elements equal to zero, while a **dense matrix** has most elements nonzero.



Sparse vs dense: dots represent nonzero entries

- In practice, many real-world matrices are extremely sparse.
- A social network with a million users could be represented as a $10^6 \times 10^6$ matrix, but each person only connects to a handful of others, so nearly all entries are zero.



A small social network and its adjacency matrix: most entries are zero

- A **permutation matrix** is obtained by rearranging the rows of an identity matrix. Multiplying by it shuffles the elements of a vector. Every row and every column has exactly one 1 and the rest are 0s.
- For example, the matrix below moves element 3 to position 1, element 1 to position 2, and element 2 to position 3:

```
P = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}
```

- A **Toeplitz matrix** has the same value along every diagonal (upper-left to lower-right). Notice how each diagonal is constant:

```
T = \begin{bmatrix} a & b & c \\ d & a & b \\ e & d & a \end{bmatrix}
```

- This structure appears in signal processing and convolution, because sliding a fixed filter across a signal is equivalent to multiplying by a Toeplitz matrix.
- A **circulant matrix** is a special Toeplitz matrix where each row is a cyclic shift of the one above. When a row reaches the end, it wraps around:

```
C = \begin{bmatrix} 1 & 3 & 2 \\ 2 & 1 & 3 \\ 3 & 2 & 1 \end{bmatrix}
```

- Circulant matrices are closely connected to the discrete Fourier transform (DFT) and are central to how circular convolution works.
- A **Hermitian matrix** is the complex equivalent of a symmetric matrix: $A = A^*$ (where A^* is the conjugate transpose).
- For real-valued matrices, Hermitian and symmetric are the same thing. You will encounter these in quantum computing and signal processing.
- A **unitary matrix** is the complex equivalent of an orthogonal matrix: $U^*U = UU^* = I$. Just as orthogonal matrices preserve lengths in real spaces, unitary matrices preserve lengths in complex spaces.
- An **idempotent matrix** satisfies $A^2 = A$. Applying the transformation twice is the same as applying it once, which makes it a **projection**. Once you have projected, projecting again changes nothing.
- A **nilpotent matrix** satisfies $A^k = O$ (the zero matrix) for some power k . Apply the transformation enough times and everything collapses to zero. For example:

```
\begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}^2 = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}
```

- A **Boolean matrix** (or binary matrix) contains only 0s and 1s. It represents yes/no relationships. For example, in a graph with 3 nodes, the **adjacency matrix** records which nodes are connected:

```
B = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix}
```

- Here, node 1 connects to nodes 2 and 3, but nodes 2 and 3 are not connected to each other.

- A **Vandermonde matrix** is built from consecutive powers of a set of values. Given values x_1, x_2, x_3 :

```
V = \begin{bmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ 1 & x_3 & x_3^2 \end{bmatrix}
```

- This structure appears in polynomial interpolation: finding the unique polynomial that passes through a given set of points.
- A **Hessenberg matrix** is “almost” triangular, with zeros below the first subdiagonal:

```
H = \begin{bmatrix} 4 & 2 & 1 \\ 3 & 5 & -1 \\ 0 & 1 & 6 \end{bmatrix}
```

- It is a useful intermediate form for computing eigenvalues efficiently. Reducing a matrix to Hessenberg form first makes iterative algorithms converge faster.

7.1 Coding Tasks (use CoLab or notebook)

1. Create an orthogonal matrix (rotation matrix), multiply it by its transpose, and verify you get the identity. Try different angles.

```
import jax.numpy as jnp

theta = jnp.pi / 4
Q = jnp.array([[jnp.cos(theta), -jnp.sin(theta)],
               [jnp.sin(theta),  jnp.cos(theta)]])

print(f"Q @ Q.T:\n{Q @ Q.T}")
print(f"Determinant: {jnp.linalg.det(Q):.2f}")
```

1. Create a symmetric matrix and verify that it equals its transpose. Then compute its eigenvalues and check that the eigenvectors are perpendicular.

```
import jax.numpy as jnp

S = jnp.array([[4.0, 2.0],
               [2.0, 3.0]])

print(f"Symmetric: {jnp.allclose(S, S.T)}")

eigenvalues, eigenvectors = jnp.linalg.eigh(S)
```

```
print(f"Eigenvalues: {eigenvalues}")
print(f"Dot product of eigenvectors: {jnp.dot(eigenvectors[:, 0],
eigenvectors[:, 1]):.6f}")
```

8 Matrix Operations

Matrix operations are the computational engine of deep learning. This file covers matrix addition, scalar multiplication, matrix-vector products, matrix multiplication, element-wise operations, Kronecker products, and broadcasting, the operations behind every forward pass and gradient update.

- Matrices can be added and scaled just like vectors.
- For addition, both matrices must have the same dimensions, and you add element by element:

```
\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} + \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix}
```

- For scalar multiplication, you multiply every element by the scalar:

```
3 \times \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} = \begin{bmatrix} 3 & 6 \\ 9 & 12 \end{bmatrix}
```

- The simplest thing you can do with a matrix is multiply it by a vector. **Matrix-vector multiplication** $\mathbf{Ax}$ combines the columns of $\mathbf{A}$ using the entries of $\mathbf{x}$ as weights:

```
\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 5 \\ 6 \end{bmatrix} = 5 \begin{bmatrix} 1 \\ 3 \end{bmatrix} + 6 \begin{bmatrix} 2 \\ 4 \end{bmatrix} = \begin{bmatrix} 17 \\ 39 \end{bmatrix}
```

- This is the core operation in ML. Every neural network layer computes $\mathbf{Ax} + \mathbf{b}$: a matrix times an input vector, plus a bias.
- The general case is **matrix multiplication**. Given $\mathbf{A}$ ($m \times n$) and $\mathbf{B}$ ($n \times p$), the product $\mathbf{C} = \mathbf{AB}$ is an $m \times p$ matrix where each element is a dot product:

$$C_{ij} = \sum_{k=1}^n A_{ik} B_{kj}$$

- Each entry in the result is the dot product of a row from A with a column from B. The inner dimensions must match (n), and the result takes the outer dimensions ($m \times p$).
- Another way to see it: each column of the result is a **weighted sum** of the columns of A, where the weights come from the corresponding column of B.
- If B has column $[2, 3]^T$, the result column is $2 \times (\text{column 1 of } A) + 3 \times (\text{column 2 of } A)$.
- A useful special case: multiplying a matrix by its transpose always gives a square matrix. AA^T is $m \times m$ and $A^T A$ is $n \times n$:

```
\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix} = \begin{bmatrix} 14 & 32 \\ 32 & 77 \end{bmatrix}
```

- Matrix multiplication has important rules:
 - **Not commutative:** $AB \neq BA$ in general. The order matters.
 - **Associative:** $(AB)C = A(BC)$. You can group multiplications however you like.
 - **Distributive:** $A(B + C) = AB + AC$.
 - **Identity:** $AI = IA = A$.
- The **Hadamard product** (element-wise product) multiplies two matrices of the same size entry by entry, written $A \odot B$:

```
\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \odot \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} 5 & 12 \\ 21 & 32 \end{bmatrix}
```

- Unlike standard matrix multiplication, the Hadamard product is commutative ($A \odot B = B \odot A$) and requires both matrices to have the same dimensions. It is used heavily in ML for gating: multiplying element-wise by a mask of values between 0 and 1 controls how much of each entry “passes through.”

- The **outer product** of two vectors $\mathbf{u}$ and $\mathbf{v}$ produces a matrix: $\mathbf{uv}^T$. Each entry is the product of one element from $\mathbf{u}$ and one from $\mathbf{v}$:

```
\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \begin{bmatrix} 4 & 5 \end{bmatrix} =
\begin{bmatrix} 4 & 5 \\ 8 & 10 \\ 12 & 15 \end{bmatrix}
```

- The result always has rank 1, because every row is a scaled version of $\mathbf{v}^T$. Any matrix can be written as a sum of rank-1 outer products, which is exactly what SVD does (covered in decompositions).
- Matrix multiplication is computationally expensive. Multiplying two $n \times n$ matrices takes $O(n^3)$ operations. For a 1000×1000 matrix, that is a billion multiplications.
- When matrices are **sparse** (mostly zeros), naive multiplication wastes time multiplying by zero. The **Compressed Sparse Row (CSR)** format stores only the nonzero elements along with their positions:
 - **Values**: the nonzero entries in row order
 - **Column indices**: which column each value belongs to
 - **Row offsets**: where each row starts in the values list
- For example, the matrix:

```
A = \begin{bmatrix} 5 & 0 & 0 & 2 \\ 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix}
```

- Is stored as: values = [5, 2, 3, -1], columns = [0, 3, 2, 3], row offsets = [0, 2, 3, 4]. This skips all the zeros and makes sparse operations much faster.
- A core use of matrices is solving **systems of linear equations**. The system $A\mathbf{x} = \mathbf{b}$ asks: “what vector $\mathbf{x}$, when transformed by A, produces $\mathbf{b}$?”
- For example, say you are buying fruit. Apples cost x_1 dollars each and bananas cost x_2 dollars each. You know that 2 apples and 1 banana cost \$5, and 1 apple and 3 bananas cost \$10. In matrix form:

```
\begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \\ \end{bmatrix} = \begin{bmatrix} 5 \\ 10 \end{bmatrix}
```

- Multiplying the matrix by the vector row by row (each row dotted with $[x_1, x_2]^T$) gives two equations:

$$2x_1 + 1x_2 = 5 \text{ (row 1)} \quad x_1 + 3x_2 = 10 \text{ (row 2)}$$

- From row 1, $x_2 = 5 - 2x_1$. Substituting into row 2: $x_1 + 3(5 - 2x_1) = 10$, which gives $x_1 = 1$, then $x_2 = 3$. Apples cost \$1 and bananas cost \$3.
- Verify — it checks out:

```
\begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} 1 \\ 3 \end{bmatrix} \\ = \begin{bmatrix} 2 + 3 \\ 1 + 9 \end{bmatrix} = \begin{bmatrix} 5 \\ 10 \end{bmatrix}
```

- If A has an inverse, the solution is simply $\mathbf{x} = A^{-1}\mathbf{b}$. But computing the inverse directly is expensive and numerically unstable. In practice, we use decompositions instead.
- Not every matrix is square, and not every square matrix is invertible. The **pseudo-inverse** A^+ generalises the inverse to any matrix. It always exists and provides the “best possible” inverse:

$$A^+ = (A^T A)^{-1} A^T$$

- When A is lower triangular, solving $L\mathbf{x} = \mathbf{b}$ is easy by **forward substitution**: solve for x_1 first, then use it to find x_2 , and so on down.
- When A is upper triangular, solving $U\mathbf{x} = \mathbf{b}$ works by **back substitution**: solve for the last variable first, then work upward.
- This is why decomposing a matrix into triangular factors (as we will see in decompositions) is so useful. It turns a hard problem into two easy ones.

8.1 Coding Tasks (use CoLab or notebook)

1. Multiply two matrices and verify the dimensions. Then swap the order and observe that the result changes (or that it fails if dimensions don't match).

```
import jax.numpy as jnp

A = jnp.array([[1.0, 2.0],
               [3.0, 4.0]])
B = jnp.array([[5.0, 6.0],
               [7.0, 8.0]])

print(f"A @ B:\n{A @ B}")
print(f"B @ A:\n{B @ A}")
print(f"Equal: {jnp.allclose(A @ B, B @ A)}")
```

1. Solve a system of linear equations $A\mathbf{x}=\mathbf{b}$ and verify the solution by multiplying back. Try changing $\mathbf{b}$ to see how the solution shifts.

```
import jax.numpy as jnp

A = jnp.array([[2.0, 1.0],
               [5.0, 3.0]])
b = jnp.array([4.0, 7.0])

x = jnp.linalg.solve(A, b)
print(f"Solution x: {x}")
print(f"A @ x: {A @ x}")
```

9 Linear Transformations

Every matrix multiplication is a linear transformation, a function that reshapes, rotates, or projects vectors while preserving linearity. This file covers rotation, reflection, scaling, shearing, projection, the kernel and image of a map, and how neural network layers chain these transformations.

- A **linear transformation** (or linear map) is a function that takes a vector and produces another vector, while preserving addition and scaling. If T is linear, then:
 - $T(\mathbf{u} + \mathbf{v}) = T(\mathbf{u}) + T(\mathbf{v})$
 - $T(c\mathbf{u}) = cT(\mathbf{u})$
- Every linear transformation can be represented as multiplication by a matrix. The matrix is the transformation. When you multiply a vector by a matrix, you are applying a linear transformation to it.

- Think of a 2×2 matrix as a machine that takes in 2D vectors and outputs new 2D vectors. The columns of the matrix tell you where the standard basis vectors

⌚

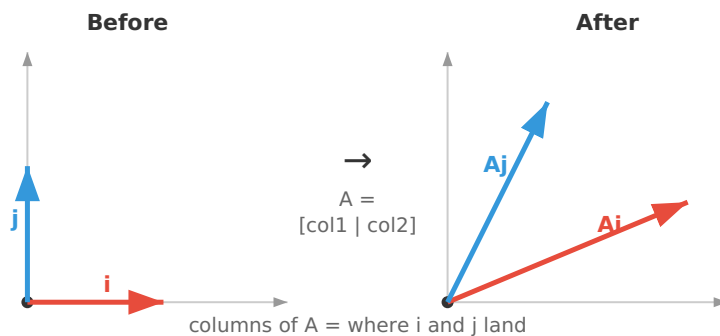
i

and

⌚

j

end up after the transformation. Everything else follows from linearity.



The columns of a matrix show where the basis vectors land

- For example, if

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

then

⌚

i

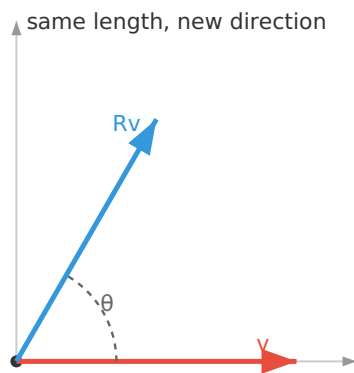
$= [1, 0]^T$ lands at $[2, 1]^T$ (column 1) and

⌚

j

$= [0, 1]^T$ lands at $[1, 2]^T$ (column 2). Every other vector is a combination of these two, so its output follows automatically.

- Multiplying two matrices can be thought of as applying one transformation after another. If B transforms vectors from one space and A transforms the result, then AB does both in sequence. In a game engine, rotating a character and then moving them forward is a different result from moving them first and then rotating, which is why matrix multiplication is not commutative.
- **Rotation** turns vectors by an angle θ without changing their length. The vector stays the same size, it just points in a new direction.



Rotation preserves length but changes direction

- In 2D, the rotation matrix is:

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

- For $\theta = 90^\circ$:

$$R = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$$

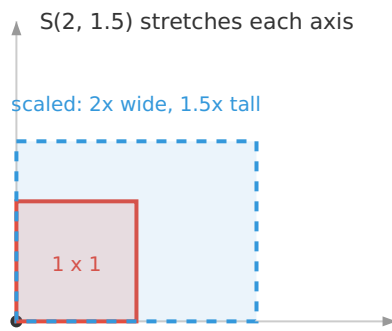
so $[1, 0]^T$ becomes $[0, 1]^T$. The vector pointing right now points up. Rotation matrices are orthogonal and always have determinant 1. When you rotate a photo on your phone, this is the exact matrix being applied to every pixel coordinate.

- In 3D, there are separate rotation matrices for each axis. A robotic arm rotates each joint around a specific axis, and each joint is one rotation matrix. Rotation around the z-axis looks like the 2D case embedded in 3D:

$$R_z(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- **Scaling** stretches or shrinks vectors along each axis independently:

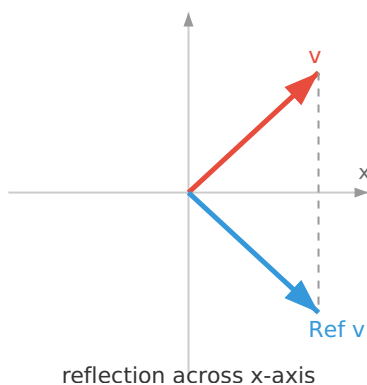
$$S(s_x, s_y) = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix}$$



Scaling stretches each axis by a different factor

- $S(2, 1.5)$ doubles the x-component and multiplies the y-component by 1.5. Scaling by -1 along an axis flips that component. A diagonal matrix is always a scaling transformation. When you resize an image to 50%, you are applying $S(0.5, 0.5)$ to every pixel coordinate.
- **Reflection** flips vectors across an axis or line, like a mirror. Reflecting across the x-axis keeps the x-component and negates the y-component:

```
\text{Ref}_x = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}
```



Reflection across the x-axis flips the y-component

- For example, $[3, 2]^T$ becomes $[3, -2]^T$. When your phone flips a selfie horizontally so text reads correctly, it is applying a reflection matrix. Reflecting across the line $y = x$ swaps the two components:

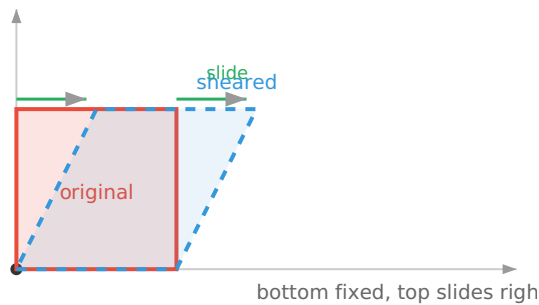
```
\text{Ref}_{y=x} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}
```

- Reflection matrices have determinant -1 , confirming they flip orientation.
- Rotations and reflections are both **rigid transformations**: they preserve distances and angles. The matrices that represent them are orthogonal

matrices, which is why orthogonal matrices always have determinant +1 (rotation) or -1 (reflection).

- **Shearing** skews vectors along one axis proportionally to the other. A horizontal shear by factor k :

```
\text{Sh}_x(k) = \begin{bmatrix} 1 & k \\ 0 & 1 \end{bmatrix}
```



Shearing slides the top sideways while the bottom stays fixed

- Each point slides horizontally by k times its height. With $k = 0.5$, a point at height 2 shifts right by 1. The bottom row stays put, the top row slides. This is how italic text works: upright letters are sheared so they slant to the right.
- All of the above (rotation, scaling, reflection, shearing) are **linear** transformations. They keep the origin fixed and preserve straight lines. But what about **translation** (shifting everything by a fixed amount)?
- Translation is not a linear transformation because it moves the origin. If you shift every point right by 3, the zero vector moves to $[3, 0]^T$, breaking linearity. To handle it, we use an **affine transformation**, which combines a linear transformation with a translation:

$$\mathbf{y} = \mathbf{A}\mathbf{x} + \mathbf{t}$$

- To represent this as a single matrix multiplication, we use **homogeneous coordinates**: add an extra 1 to every vector and use an $(n + 1) \times (n + 1)$ matrix:

```
\begin{bmatrix} A & \mathbf{t} \\ \mathbf{0}^T & 1 \end{bmatrix} \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix} = \begin{bmatrix} A\mathbf{x} + \mathbf{t} \\ 1 \end{bmatrix}
```

- Affine transformations preserve straight lines and parallelism, but not necessarily angles or lengths. Every object in a video game is positioned using

affine transformations: rotate it, scale it, then place it at the right location, all encoded in a single matrix.

- A **degenerate transformation** (singular matrix) collapses space into a lower dimension.
- For example, the matrix

```
\begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}
```

maps every 2D vector onto a single line, because both columns point in the same direction. The determinant is zero, information is lost, and the transformation cannot be undone.

- Converting a colour image (3 values per pixel: red, green, blue) to grayscale (1 value per pixel) is a degenerate transformation: the colour information is permanently gone.
- In ML, linear transformations are the core of neural networks, data is represented as a matrix (a stack of vectors representing features of an object like humans, planes, text, image...anything!)
- Each layer applies a matrix multiplication (linear transformation), details are provided in other chapters, we need to explain how to structure these data and motivate neural networks properly.
- However, the most used techniques today often almost exclusively pass the data through a bunch of linear transformations, we call these **Transformers**.
- Gemini, ChatGPT, Claude, Qwen, DeepSeek and the best performing AI in the world today, are transformers!

9.1 Coding Tasks (use CoLab or notebook)

1. Apply a rotation matrix to a vector and plot both the original and rotated vector. Try different angles.

```
import jax.numpy as jnp
import matplotlib.pyplot as plt

theta = jnp.pi / 3
R = jnp.array([[jnp.cos(theta), -jnp.sin(theta)],
               [jnp.sin(theta),  jnp.cos(theta)]])
```

```

v = jnp.array([1.0, 0.0])
v_rot = R @ v

plt.figure(figsize=(5, 5))
plt.quiver(0, 0, v[0], v[1], angles='xy', scale_units='xy', scale=1,
color='red', label='original')
plt.quiver(0, 0, v_rot[0], v_rot[1], angles='xy', scale_units='xy', scale=1,
color='blue', label='rotated')
plt.xlim(-1.5, 1.5); plt.ylim(-1.5, 1.5)
plt.grid(True); plt.legend(); plt.gca().set_aspect('equal')
plt.show()

```

1. Apply a shearing transformation to a set of points forming a square and visualise the deformed shape.

```

import jax.numpy as jnp
import matplotlib.pyplot as plt

square = jnp.array([[0,0],[1,0],[1,1],[0,1],[0,0]]).T

k = 0.5
shear = jnp.array([[1, k],
                   [0, 1]])
sheared = shear @ square

plt.figure(figsize=(6, 4))
plt.plot(square[0], square[1], 'r-o', label='original')
plt.plot(sheared[0], sheared[1], 'b-o', label='sheared')
plt.grid(True); plt.legend(); plt.gca().set_aspect('equal')
plt.show()

```

10 Matrix Decompositions

Matrix decompositions break complex matrices into simpler factors for solving systems, computing inverses, and compressing data. This file covers Gaussian elimination, LU, QR, Cholesky, eigendecomposition, and SVD, the algorithms behind PCA, recommender systems, and numerical stability in ML.

- A matrix decomposition (or factorisation) breaks a matrix into simpler pieces that are easier to work with. Think of it like factoring a number: $12 = 3 \times 4$ is easier to reason about than 12 alone.

- We decompose matrices to solve systems of equations faster, compute inverses stably, find eigenvalues, compress data, and understand the geometry of transformations.
- The most fundamental technique is **Gaussian elimination** (row reduction). The idea is simple: given a system $\mathbf{Ax} = \mathbf{b}$, use three allowed operations to simplify A until the answer is obvious.
- The operations are: swap two rows, multiply a row by a nonzero scalar, or add a multiple of one row to another.
- For example, to eliminate the first column below the pivot, subtract multiples of row 1 from the rows below:

```
\begin{bmatrix} 2 & 1 & 5 \\ 4 & 3 & 7 \\ 6 & 5 & 9 \end{bmatrix}
\xrightarrow{R_2 - 2R_1} \begin{bmatrix} 2 & 1 & 5 \\ 0 & 1 & -3 \\ 6 & 5 & 9 \end{bmatrix}
\xrightarrow{R_3 - 3R_1} \begin{bmatrix} 2 & 1 & 5 \\ 0 & 1 & -3 \\ 0 & 2 & -6 \end{bmatrix}
```

- The goal is **row echelon form (REF)**: zeros below each pivot (the first nonzero entry in each row), with each pivot to the right of the one above it. The matrix becomes a staircase shape.

$$\begin{bmatrix} 2 & 1 & 5 \\ 4 & 3 & 7 \\ 6 & 5 & 9 \end{bmatrix} \xrightarrow{\text{row ops}} \begin{bmatrix} \mathbf{2} & 1 & 5 \\ 0 & \mathbf{1} & -3 \\ 0 & 0 & \mathbf{-2} \end{bmatrix} \xrightarrow{\text{solve 1}} \begin{bmatrix} x_1 = ? \\ x_2 = ? \\ \mathbf{x_3 = 1} \end{bmatrix}$$

back substitution: solve for

Gaussian elimination: row operations produce a triangular form, then solve from bottom up

- Going further to **reduced row echelon form (RREF)** makes every pivot equal to 1 and the only nonzero entry in its column. Every matrix has a unique RREF.
- Once in triangular form, we solve by **back substitution**: the bottom row gives the last variable directly, then work upward.
- This is the foundation that all other decompositions build upon, the goal of decompositions is to reduce a matrix to a triangular form, so we can back substitute and solve for the variables.

- **LU decomposition** formalises Gaussian elimination by factoring a square matrix into $A = LU$ (or $A = PLU$ with row swaps), where L is lower triangular and U is upper triangular.

$$\begin{bmatrix} A \end{bmatrix} = \begin{bmatrix} L \end{bmatrix} \times \begin{bmatrix} U \end{bmatrix}$$

two easy triangular solves

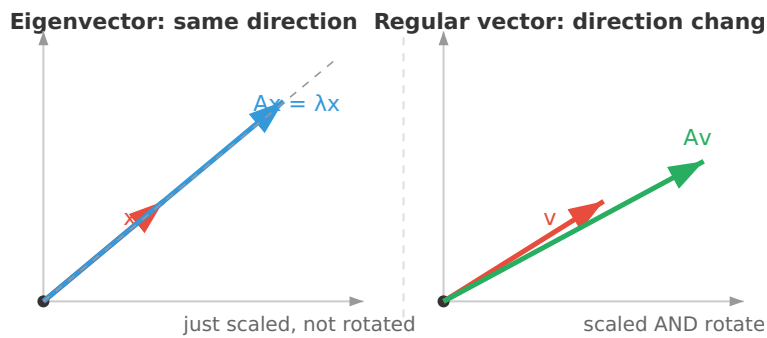
LU decomposition: split one hard matrix into two easy triangular ones

- To solve $A\mathbf{x} = \mathbf{b}$: first solve $L\mathbf{y} = \mathbf{b}$ by forward substitution (top to bottom), then solve $U\mathbf{x} = \mathbf{y}$ by back substitution (bottom to top). Two easy triangular solves instead of one hard general solve.
- The advantage over raw Gaussian elimination is reuse. Once you have L and U , you can solve for many different $\mathbf{b}$ vectors without redoing the factorisation.
- If you need to solve the same system with 1000 different right-hand sides (common in simulations), you factorise once and reuse.
- When a matrix is symmetric and positive definite (like a covariance matrix), we can do even better.
- **Cholesky decomposition** factors it as $A = LL^T$, where L is lower triangular. For example:

$$\begin{bmatrix} 4 & 2 \\ 2 & 5 \end{bmatrix} = \begin{bmatrix} 2 & 0 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} 2 & 1 \\ 0 & 2 \end{bmatrix}$$

- This is roughly twice as fast as LU and is guaranteed to be numerically stable. Think of it as a “square root” of the matrix.
- If the decomposition fails (a negative value under a square root), the matrix is not positive definite. Cholesky thus doubles as a test for positive definiteness.
- The **eigenvectors** of a square matrix A are the special directions that the transformation only stretches or shrinks, without rotating. The **eigenvalue** is the scaling factor:

$$A\mathbf{x} = \lambda\mathbf{x}$$



Eigenvector stays on the same line (just scaled), regular vector gets rotated

- Most vectors change direction when multiplied by a matrix. But eigenvectors are special: the output points in the same direction as the input, just scaled by λ . If $\lambda = 2$, the eigenvector doubles in length. If $\lambda = -1$, it flips direction. If $\lambda = 0$, it gets squashed to zero.
- For example, with:

$$A = \begin{bmatrix} 3 & 1 \\ 0 & 2 \end{bmatrix}$$

the vector $[1, 0]^T$ is an eigenvector with $\lambda = 3$ because $A[1, 0]^T = [3, 0]^T = 3[1, 0]^T$.

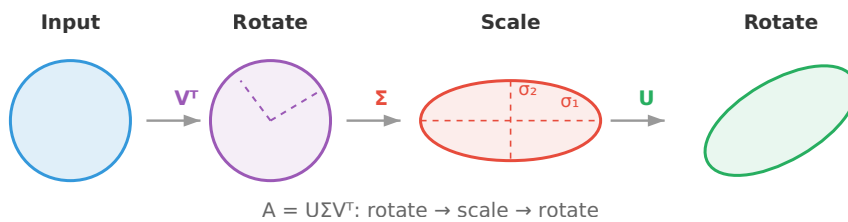
- To find eigenvalues, solve the **characteristic polynomial** $\det(A - \lambda I) = 0$. The roots are the eigenvalues. Then substitute each λ back into $(A - \lambda I)\mathbf{x} = \mathbf{0}$ to find the corresponding eigenvectors.
- Key properties:
 - The trace of A equals the sum of its eigenvalues.
 - The determinant of A equals the product of its eigenvalues.
 - Symmetric matrices have perpendicular eigenvectors and real eigenvalues.
 - Positive definite matrices have all positive eigenvalues.
 - Covariance matrices (which we will encounter in statistics) are always positive semi-definite.
- Computing eigenvalues via the characteristic polynomial is impractical for large matrices. Instead, iterative methods are used:
 - **Power iteration:** repeatedly multiply by A and normalise. Converges to the dominant eigenvector (largest eigenvalue). Simple but only finds one eigenpair.

- **QR algorithm:** the workhorse method. Repeatedly decompose and recombine using QR factorisation until the matrix converges to triangular form, revealing all eigenvalues on the diagonal.
 - **Inverse iteration:** finds the eigenvector closest to a given target value. Useful when you know roughly which eigenvalue you want.
 - For large sparse matrices, **Arnoldi** and **Lanczos** iterations exploit the sparsity for efficiency.
- If a square matrix has a full set of linearly independent eigenvectors, it can be **diagonalised**: $A = PDP^{-1}$, where D is a diagonal matrix of eigenvalues and the columns of P are the eigenvectors.
 - Why is this useful? Diagonal matrices are trivial to work with. Need A^{100} ? Instead of multiplying A by itself 100 times, compute $PD^{100}P^{-1}$, and raising a diagonal matrix to a power just raises each entry independently. This turns an expensive operation into a cheap one.
 - An **eigenbasis** is a basis made entirely of eigenvectors. In this basis, the matrix becomes diagonal and the transformation is just independent scaling along each eigenvector direction. This is like finding the natural coordinate system for the transformation.
 - **QR decomposition** factors any matrix A into $A = QR$, where Q is orthogonal (its columns are orthonormal) and R is upper triangular. Think of it as separating the “direction” information (Q) from the “scaling and mixing” information (R).
 - The **Gram-Schmidt process** builds Q column by column. Take the first column of A and normalise it. Take the second column, subtract its projection onto the first (to make it perpendicular), and normalise. Repeat for each column. The result is an orthonormal set of vectors.
 - QR decomposition is the engine behind the QR algorithm for eigenvalues. It is also used directly for solving least-squares problems: when $A\mathbf{x} = \mathbf{b}$ has no exact solution (more equations than unknowns), QR finds the best approximate answer.

- **SVD** (Singular Value Decomposition) is the most general and arguably the most important decomposition. Every matrix (any shape, any rank) has an SVD:

$$A = U\Sigma V^T$$

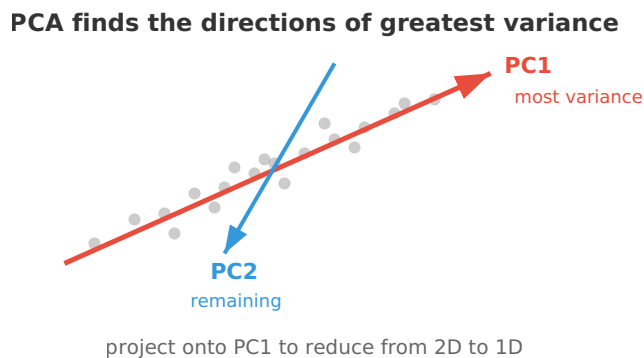
- V^T ($n \times n$, orthogonal): rotates the input
- Σ ($m \times n$, diagonal): scales along orthogonal axes (the singular values, non-negative, in descending order)
- U ($m \times m$, orthogonal): rotates the output



SVD: any transformation = rotate, then scale, then rotate again

- Geometrically, SVD says that every linear transformation, no matter how complicated, is just a rotation, followed by a stretch along the axes, followed by another rotation. A circle becomes an ellipse.
- The singular values ($\sigma_1 \geq \sigma_2 \geq \dots$) reveal the “importance” of each direction. Large singular values correspond to directions that matter most. The rank of A equals the number of nonzero singular values.
- **Low-rank approximation:** by keeping only the k largest singular values and zeroing the rest, you get the best possible rank- k approximation of A . This is how image compression works: a 1000×1000 image might need only $k = 50$ singular values to look nearly identical, compressing it by 20x.
- SVD also provides the pseudo-inverse: $A^+ = V\Sigma^+U^T$, where Σ^+ inverts the nonzero singular values.
- While eigendecomposition only works for square matrices, SVD works for any matrix. This is its key advantage.
- **PCA** (Principal Component Analysis) uses eigendecomposition (or SVD) for dimensionality reduction.
- Imagine a dataset with 100 features per sample (vector of dim 100 stacked into a matrix). Many of those features are correlated and redundant.

- PCA finds the directions along which the data actually varies, letting you keep only what matters.



PCA finds the directions of greatest variance in the data

- The first principal component (PC1) is the direction of greatest variance.
- The second (PC2) captures the most variance of what remains, and is perpendicular to the first.
- If most of the variance lives along just a few directions, you can project the data down to those dimensions and discard the rest with minimal loss.
- The steps:
 - Standardise the data (subtract mean, divide by standard deviation) so all features contribute equally
 - Compute the covariance matrix
 - Find its eigenvalues and eigenvectors
 - Select the k eigenvectors with the largest eigenvalues (these are the principal components)
 - Project the data onto these components
- Standardisation is critical: without it, a feature measured in kilometres would dominate one measured in centimetres, regardless of actual importance.
- In practice, PCA is used for visualisation (projecting high-dimensional data to 2D or 3D), noise reduction (discarding low-variance directions that are mostly noise), and speeding up ML models by reducing the number of input features.
- **Kernel PCA** extends PCA to nonlinear relationships. It maps the data through a kernel function into a higher-dimensional space where the structure becomes linear, then applies standard PCA and projects back.

- **Schur decomposition** factors a square matrix as $A = QTQ^*$, where Q is unitary and T is upper triangular. Every square matrix has a Schur decomposition, even if it cannot be diagonalised.
- **Non-negative Matrix Factorisation (NMF)** decomposes a matrix into two non-negative matrices: $A \approx WH$, where all entries in W and H are ≥ 0 . Unlike SVD, which can produce negative entries, NMF only adds, never subtracts. This makes the parts interpretable: in topic modelling, W gives the topic weights per document and H gives the word weights per topic, all non-negative, matching how we think about “how much of each topic” a document contains.
- The **spectral theorem** states that symmetric (or Hermitian) matrices can always be diagonalised with an orthogonal (or unitary) matrix. Their eigenvalues are always real and their eigenvectors always orthogonal. This is the theoretical foundation behind PCA.

10.1 Coding Tasks (use CoLab or notebook)

1. Compute the eigenvalues and eigenvectors of a symmetric matrix. Verify that the eigenvectors are perpendicular and reconstruct the matrix from its eigendecomposition.

```
import jax.numpy as jnp

A = jnp.array([[4.0, 2.0],
               [2.0, 3.0]])

eigenvalues, eigenvectors = jnp.linalg.eigh(A)
print(f"Eigenvalues: {eigenvalues}")
print(f"Eigenvectors orthogonal: {jnp.dot(eigenvectors[:,0], eigenvectors[:,1]):.6f}")

# Reconstruct: A = P D P^T
D = jnp.diag(eigenvalues)
A_reconstructed = eigenvectors @ D @ eigenvectors.T
print(f"Reconstruction matches: {jnp.allclose(A, A_reconstructed)}")
```

1. Implement power iteration to find the largest eigenvalue, and inverse iteration to find the smallest. Compare with `jnp.linalg.eigh`. Then try implementing the QR algorithm yourself.

```

import jax.numpy as jnp

A = jnp.array([[4.0, 2.0],
               [2.0, 3.0]])

# Power iteration: finds the LARGEST eigenvalue
v = jnp.array([1.0, 0.0])
for _ in range(20):
    v = A @ v
    v = v / jnp.linalg.norm(v)
print(f"Largest eigenvalue: {v @ A @ v:.4f}")

# Inverse iteration: multiply by A^{-1} instead of A, finds the SMALLEST
eigenvalue
v = jnp.array([1.0, 0.0])
for _ in range(20):
    v = jnp.linalg.solve(A, v)
    v = v / jnp.linalg.norm(v)
print(f"Smallest eigenvalue: {1.0 / (v @ jnp.linalg.solve(A, v)):.4f}")

print(f"jnp.linalg.eigh: {jnp.linalg.eigh(A)[0]}")

```

1. Compute the SVD of a matrix, then reconstruct it using only the top-k singular values and observe how the approximation quality changes with k.

```

import jax.numpy as jnp

A = jnp.array([[1.0, 2.0, 3.0],
               [4.0, 5.0, 6.0],
               [7.0, 8.0, 9.0]])

U, S, Vt = jnp.linalg.svd(A)

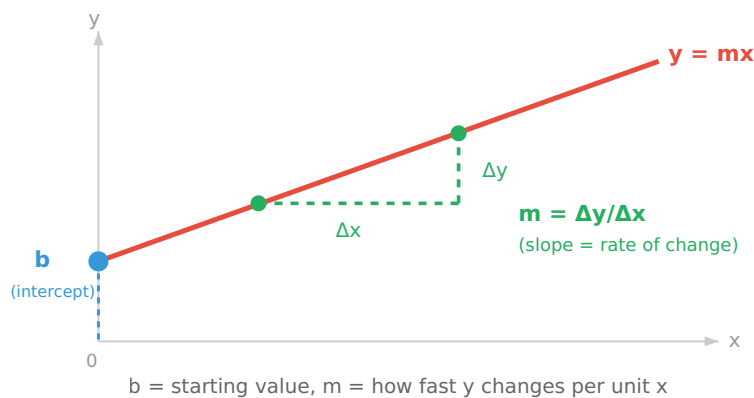
for k in [1, 2, 3]:
    approx = U[:, :k] @ jnp.diag(S[:k]) @ Vt[:k, :]
    error = jnp.linalg.norm(A - approx)
    print(f"k={k}, reconstruction error: {error:.4f}")

```

11 Differential Calculus

Differential calculus captures instantaneous rates of change. This file covers limits, derivatives, differentiation rules, the chain rule (the foundation of backpropagation), and common derivatives used throughout ML.

- In the previous chapters, we learned how to represent data as vectors and transform it with matrices. But many real-world phenomena are not static. A car accelerates, a stock price fluctuates, a neural network's loss changes as weights update. **Calculus** is the mathematics of change.
- Calculus asks two questions: how fast is something changing right now? (differential calculus) and how much has it accumulated over time? (integral calculus). This section tackles the “how fast” question.
- Imagine you are driving and glance at your speedometer. It reads 60 km/h. That number is not the average speed of your entire trip; it is your speed at this exact instant. Differential calculus gives us the tools to compute such instantaneous rates of change.
- But first, let us revisit the equation of a straight line: $y = mx + b$.
- This is the simplest relationship between two quantities.
 - b is the **y-intercept**, where the line crosses the y-axis (the starting value when $x = 0$).
 - m is the **slope**, the rate of change: for every 1 unit increase in x , y changes by m .
- If $m = 3$, the line rises steeply; if $m = 0$, the line is flat; if $m = -2$, the line falls.
- The slope is computed as $m = \frac{\Delta y}{\Delta x} = \frac{y_2 - y_1}{x_2 - x_1}$, the ratio of “how much did y change” to “how much did x change.”



The equation of a line: b is the y -intercept, m is the slope (rise over run)

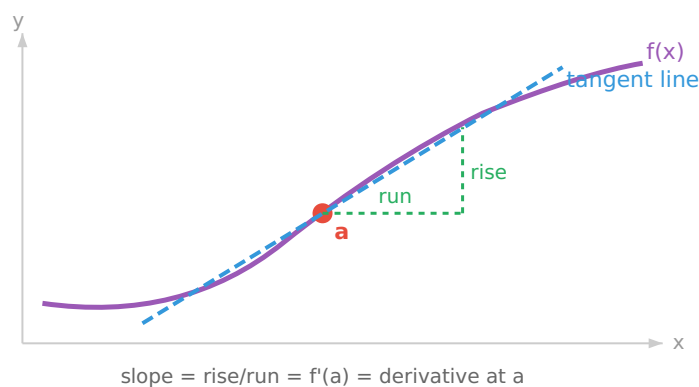
- Once you know m and b , you can compute y for any x .
- For example, if $m = 2$ and $b = 3$, then at $x = 5$: $y = 2(5) + 3 = 13$.
- The two parameters fully determine the line, and predicting any output is just plugging in.
- For a straight line, the slope is the same everywhere.
- This idea generalises beyond lines. Any function is a rule that maps inputs to outputs, and once you know its formula (its parameters and shape), you can compute the output for any input and plot the result.
- $y = x^2$ gives a parabola, $y = \sin(x)$ gives a wave, $y = e^x$ gives exponential growth. Each formula defines a specific curve, and being comfortable reading a function as a shape is essential for everything that follows.
- For a straight line, the slope is the same everywhere. But most interesting functions are curved, so the slope varies from point to point. Calculus gives us a way to find the slope at any single point on a curve.
- We also need the concept of a **limit**. A limit describes what value a function approaches as its input gets closer and closer to some target, without necessarily reaching it.

$$\lim_{x \rightarrow a}$$

$$f(x) = L$$

- This reads: “as x approaches a , $f(x)$ approaches L .” The function does not need to actually equal L at $x = a$. It just needs to get arbitrarily close.

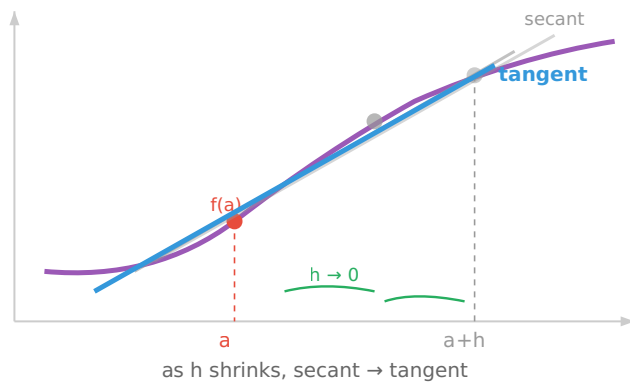
- For example, take $f(x) = \frac{x^2 - 1}{x - 1}$. If you plug in $x = 1$ directly, you get $\frac{0}{0}$, which is undefined.
- But try values close to 1: $f(0.9) = 1.9$, $f(0.99) = 1.99$, $f(1.01) = 2.01$. The outputs are clearly heading towards 2.
- Algebraically, we can see why: factor the numerator as $(x - 1)(x + 1)$, cancel the $(x - 1)$ terms, and we get $f(x) = x + 1$ for all $x \neq 1$. So as $x \rightarrow 1$, $f(x) \rightarrow 2$.
- The function has a hole at $x = 1$, but the limit still exists.
- Limits are the foundation that everything else in calculus rests on.
- The **derivative** of a function $f(x)$ at a point $x = a$ measures the instantaneous rate of change. Geometrically, it is the slope of the tangent line to the curve at that point.



The derivative is the slope of the tangent line at a point on the curve

- To compute this slope, we start with two points on the curve and compute the slope of the line through them (a **secant line**). Then we slide the second point closer and closer to the first, and see what slope the secant line approaches. This is the **difference quotient**:

$$f'(a) = \lim_{h \rightarrow 0} \frac{f(a + h) - f(a)}{h}$$



As h shrinks, the secant line approaches the tangent line

- The numerator $f(a+h) - f(a)$ is the change in output. The denominator h is the change in input. Their ratio is the average rate of change over a tiny interval. As $h \rightarrow 0$, this average becomes the instantaneous rate.
- For example, let $f(x) = x^2$. At $x = 3$:

$$f'(3) =$$

$$\lim_{h \rightarrow 0}$$

$$\frac{(3+h)^2 - 9}{h} =$$

$$\lim_{h \rightarrow 0}$$

$$\frac{9 + 6h + h^2 - 9}{h} =$$

$$\lim_{h \rightarrow 0}$$

$$(6 + h) = 6$$

- So at $x = 3$, the function x^2 is increasing at a rate of 6 units of output per unit of input.
- A function is **differentiable** at a point if this limit exists. For that to happen, the function must be continuous (no jumps), smooth (no sharp corners), and defined in a neighbourhood around the point.
- If you can draw the curve without lifting your pen and without any kinks, it is probably differentiable there.
- Computing derivatives from the limit definition every time would be tedious. Fortunately, a handful of rules let us differentiate almost any function quickly.

- **Constant rule:** the derivative of a constant is zero. If $f(x) = 5$, then $f'(x) = 0$. A flat line has zero slope.
- **Power rule:** the workhorse of differentiation. Bring the exponent down and reduce it by one:

$$\frac{d}{dx}x^n = nx^{n-1}$$

- For example: $\frac{d}{dx}x^3 = 3x^2$. The cubic becomes a quadratic. This works for any real exponent, including negatives and fractions: $\frac{d}{dx}x^{-1} = -x^{-2}$ and $\frac{d}{dx}$

$\sqrt{x}$

$$= \frac{d}{dx}x^{1/2} = \frac{1}{2}x^{-1/2}.$$

- **Sum/Difference rule:** differentiate term by term.

$$\frac{d}{dx}[f(x) \pm g(x)] = f'(x) \pm g'(x)$$

- **Product rule:** when two functions are multiplied, the derivative is not simply the product of the derivatives. Instead:

$$\frac{d}{dx}[f(x) \cdot g(x)] = f'(x)g(x) + f(x)g'(x)$$

- Think of it as: “the rate of change of the first times the second, plus the first times the rate of change of the second.” For example, $\frac{d}{dx}$
 $[x^2 \sin x] = 2x \sin x + x^2 \cos x$.

- **Quotient rule:** for a ratio of functions:

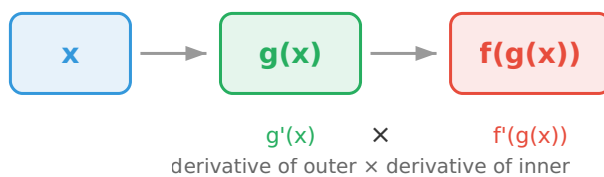
$$\frac{d}{dx}\left[\frac{f(x)}{g(x)}\right] = \frac{f'(x)g(x) - f(x)g'(x)}{[g(x)]^2}$$

- A useful mnemonic: “low d-high minus high d-low, over the square of what’s below.”

- **Chain rule:** the most important rule for ML. When functions are composed (one inside another), the derivative is the product of the derivatives along the chain:

$$\frac{d}{dx}f(g(x)) = f'(g(x)) \cdot g'(x)$$

- Think of it as peeling an onion. Differentiate the outer function (keeping the inner function untouched), then multiply by the derivative of the inner function.



Chain rule: differentiate the outer, multiply by the derivative of the inner

- For example, $\frac{d}{dx}(3x+1)^5 = 5(3x+1)^4 \cdot 3 = 15(3x+1)^4$. The outer function is $(\cdot)^5$ and the inner is $3x+1$.
- The chain rule is the mathematical foundation of **backpropagation** in neural networks. A deep network is a long chain of composed functions. To compute how the loss changes with respect to each weight, we apply the chain rule repeatedly from the output layer back to the input, multiplying local derivatives at each step.
- Here are the most common derivatives you will encounter. Each one can be derived from the limit definition, but knowing them by heart saves time:

Function	Derivative	Notes
e^x	e^x	The only function that is its own derivative
a^x	$a^x \ln a$	Generalises the exponential
$\ln x$	$\frac{1}{x}$	The natural logarithm
$\log_a x$	$\frac{1}{x \ln a}$	General logarithm

Function	Derivative	Notes
$\sin x$	$\cos x$	
$\cos x$	$-\sin x$	Note the negative sign
$\tan x$	$\sec^2 x$	

- The exponential function e^x is remarkable: it is the only function that equals its own derivative. This is why e appears everywhere in ML, from softmax activations to probability distributions.

- **L' Hopital' s Rule** handles limits that produce indeterminate forms like $\frac{0}{0}$ or

$$\frac{\infty}{\infty}.$$

When direct substitution gives one of these forms, you can take the

derivative of the numerator and denominator separately and try the limit again:

$$\lim_{x \rightarrow a}$$

$$\frac{f(x)}{g(x)} =$$

$$\lim_{x \rightarrow a}$$

$$\frac{f'(x)}{g'(x)}$$

- Conditions: both f and g must be differentiable near a , and $g'(x) \neq 0$ near a (except possibly at a itself). The original limit must give an indeterminate form.

- For example: $\lim_{x \rightarrow 0} \frac{\sin x}{x}$. Direct substitution gives $\frac{0}{0}$. Applying L' Hopital' s

Rule: $\lim_{x \rightarrow 0} \frac{\cos x}{1} = 1$. This limit is fundamental, it appears in signal processing and Fourier analysis.

- You can apply the rule repeatedly if the result is still indeterminate. For

instance, $\lim_{x \rightarrow 0} \frac{1 - \cos x}{x^2}$ gives $\frac{0}{0}$. First application: $\lim_{x \rightarrow 0} \frac{\sin x}{2x}$, still $\frac{0}{0}$. Second

application: $\lim_{x \rightarrow 0} \frac{\cos x}{2} = \frac{1}{2}$.

- If two functions are differentiable, their sum, difference, product, composition, and quotient (where the denominator is non-zero) are also differentiable. This is why we can confidently differentiate complex expressions built from simple pieces.

11.1 Coding Tasks (use CoLab or notebook)

1. Visualise common functions. Plot x^2 , $\sin(x)$, and e^x side by side to build intuition for how different formulas produce different shapes. Try changing parameters (e.g. $2x^2$, $\sin(2x)$) and observe how the curves change.

```
import jax.numpy as jnp
import matplotlib.pyplot as plt

x = jnp.linspace(-3, 3, 300)

fig, axes = plt.subplots(1, 3, figsize=(12, 3))
axes[0].plot(x, x**2, color="#e74c3c")
axes[0].set_title("x² (parabola)")
axes[1].plot(x, jnp.sin(x), color="#3498db")
axes[1].set_title("sin(x) (wave)")
axes[2].plot(x, jnp.exp(x), color="#27ae60")
axes[2].set_title("eˣ (exponential)")
for ax in axes:
    ax.axhline(0, color="gray", linewidth=0.5)
    ax.axvline(0, color="gray", linewidth=0.5)
plt.tight_layout()
plt.show()
```

1. Use JAX' s automatic differentiation to compute the derivative of $f(x) = x^3 - 2x + 1$ at several points. Compare with the analytical derivative $f'(x) = 3x^2 - 2$.

```
import jax
import jax.numpy as jnp

f = lambda x: x**3 - 2*x + 1
df = jax.grad(f)
```

```
for x in [0.0, 1.0, 2.0, -1.0]:
    print(f"x={x:5.1f}  autodiff: {df(x):.4f}  analytical: {3*x**2 - 2:.4f}")
```

1. Verify the chain rule numerically. Define $f(x) = \sin(x^2)$, compute its derivative via `jax.grad`, and compare with the analytical result $2x \cos(x^2)$.

```
import jax
import jax.numpy as jnp

f = lambda x: jnp.sin(x**2)
df = jax.grad(f)

for x in [0.5, 1.0, 2.0]:
    auto = df(x)
    analytical = 2*x * jnp.cos(x**2)
    print(f"x={x:.1f}  autodiff: {auto:.6f}  analytical: {analytical:.6f}")
```

1. Visualise the derivative. Plot $f(x) = x^3 - 3x$ and its derivative $f'(x) = 3x^2 - 3$ on the same graph. Notice where $f'(x) = 0$ corresponds to the peaks and valleys of f .

```
import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

f = lambda x: x**3 - 3*x
# jax.grad works on scalars; jax.vmap vectorises it to operate on an array of
inputs at once
df = jax.vmap(jax.grad(f))

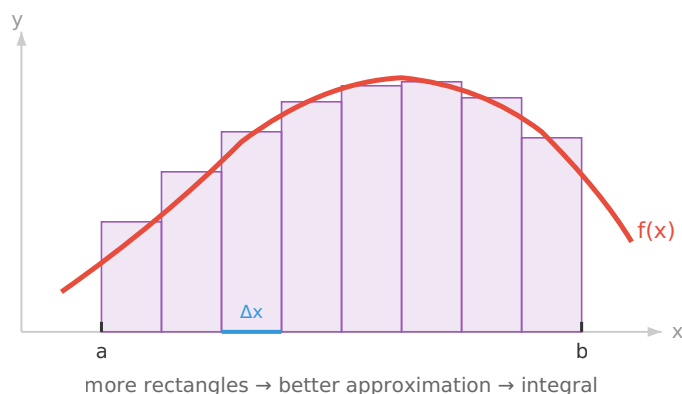
x = jnp.linspace(-2.5, 2.5, 200)
plt.plot(x, jax.vmap(f)(x), label="f(x)")
plt.plot(x, df(x), label="f'(x)", linestyle="--")
plt.axhline(0, color="gray", linewidth=0.5)
plt.legend()
plt.title("A function and its derivative")
plt.show()
```

12 Integral Calculus

Integral calculus accumulates quantities over intervals, turning local rates back into totals. This file covers definite and indefinite integrals, the Fundamental Theorem of

Calculus, integration techniques, and applications to probability densities and expected values in ML.

- Differentiation tells us the rate of change at a single point. Integration goes the other way: it accumulates many tiny pieces to compute a total.
- If the derivative answers “how fast?” , the integral answers “how much?”
- The simplest way to think about integration is as the **area under a curve**. If you plot a function $f(x)$ and shade the region between the curve and the x-axis from $x = a$ to $x = b$, the integral gives the signed area of that region.



Integration computes the area under a curve by summing thin rectangles

- Why “signed” ? Regions above the x-axis contribute positive area, regions below contribute negative area. This makes physical sense: if $f(x)$ represents velocity, the integral gives net displacement (forward minus backward), not total distance.
- To compute this area, imagine slicing the region into n thin vertical rectangles, each of width Δx . The height of each rectangle is the function value at some point in that slice. Sum them up:

$$\text{Area} \approx \sum_{i=1}^n f(x_i^*) \Delta x$$

- As we make the rectangles thinner and thinner ($n \rightarrow \infty$, $\Delta x \rightarrow 0$), the sum becomes exact. This limiting process defines the **definite integral**:

$$\int_a^b f(x) \, dx =$$

$$\lim_{n \rightarrow \infty} \sum_{i=1}^n$$

$$f(x_i^*) \Delta x$$

- The $\int$ symbol is an elongated “S” for “sum.” The dx reminds us that we are summing infinitesimally thin slices along the x -axis.
- An **indefinite integral** (or **antiderivative**) is a function $F(x)$ whose derivative is $f(x)$. We write:

$$\int f(x) \, dx = F(x) + C$$

- The $+C$ is the **constant of integration**. Since the derivative of any constant is zero, there are infinitely many antiderivatives that differ only by a constant. For example, $\int 2x \, dx = x^2 + C$, because the derivative of $x^2 + 7$ or $x^2 - 3$ is still $2x$.
- The **Fundamental Theorem of Calculus** is the bridge that connects differentiation and integration. It has two parts:
- **Part 1:** If $F(x)$ is an antiderivative of $f(x)$, then the definite integral equals the difference of F at the endpoints:

$$\int_a^b f(x) \, dx = F(b) - F(a)$$

- This is remarkably practical. Instead of computing a limit of sums (which is hard), we find an antiderivative and evaluate it at two points (which is usually easy).
- **Part 2:** If we define $F(x) = \int_a^x f(t) \, dt$, then $F'(x) = f(x)$. Differentiation and integration are inverse operations, they undo each other.

· For example, to compute $\int_1^3 x^2 \, dx$: the antiderivative of x^2 is $\frac{x^3}{3}$. So $\int_1^3 x^2 \, dx =$

$$\frac{27}{3} - \frac{1}{3} = \frac{26}{3} \approx 8.67.$$

- Just as differentiation has rules, integration has corresponding rules that reverse them:

Function	Integral	Condition
x^n	$\frac{x^{n+1}}{n+1} + C$	$n \neq -1$
$\frac{1}{x}$	$\ln x + C$	
e^x	$e^x + C$	
a^x	$\frac{a^x}{\ln a} + C$	
$\sin x$	$-\cos x + C$	
$\cos x$	$\sin x + C$	
k (constant)	$kx + C$	

- The **sum/difference rule** carries over:
 $\int [f(x) \pm g(x)] dx = \int f(x) dx \pm \int g(x) dx$. Constants can be pulled out:
 $\int k f(x) dx = k \int f(x) dx$.
- When a function is too complex to integrate directly, we have techniques to simplify it.
- **u-substitution** is the reverse of the chain rule. If you spot a composite function $f(g(x))$ multiplied by $g'(x)$, substitute $u = g(x)$ so that $du = g'(x) dx$, and the integral simplifies.
- For example: $\int 2x \cos(x^2) dx$. Let $u = x^2$, so $du = 2x dx$. The integral becomes
 $\int \cos(u) du = \sin(u) + C = \sin(x^2) + C$.
- **Integration by parts** is the reverse of the product rule. If the integrand is a product of two functions:

$$\int u dv = uv - \int v du$$

- Choose u and dv strategically so that the remaining integral $\int v du$ is simpler than the original. A common mnemonic for choosing u is **LIATE**: Logarithmic, Inverse trig, Algebraic, Trigonometric, Exponential (pick u from the earlier category).

- For example: $\int x e^x dx$. Let $u = x$ (algebraic) and $dv = e^x dx$. Then $du = dx$ and $v = e^x$. So: $\int x e^x dx = x e^x - \int e^x dx = x e^x - e^x + C = e^x(x - 1) + C$.
- In ML, integration appears in probability theory (computing probabilities by integrating density functions), in expected values (weighted averages over continuous distributions), and in computing the area under ROC curves. While we rarely integrate by hand in practice, understanding what integration means helps interpret these quantities.

12.1 Coding Tasks (use CoLab or notebook)

1. Numerically approximate $\int_0^1 x^2 dx$ using a Riemann sum with increasing numbers of rectangles. Compare with the exact answer $\frac{1}{3}$.

```
import jax.numpy as jnp

for n in [10, 100, 1000, 10000]:
    x = jnp.linspace(0, 1, n, endpoint=False)
    dx = 1.0 / n
    area = jnp.sum(x**2 * dx)
    print(f"n={n:5d} approx: {area:.6f} exact: {1/3:.6f}")
```

1. Verify the Fundamental Theorem of Calculus numerically. Define

$F(x) = \int_0^x t^2 dt = \frac{x^3}{3}$ and check that its derivative (computed via `jax.grad`) equals x^2 .

```
import jax
import jax.numpy as jnp

F = lambda x: x**3 / 3
dF = jax.grad(F)

for x in [0.5, 1.0, 2.0, 3.0]:
    print(f"x={x:.1f} F'(x)={dF(x):.4f} x^2={x**2:.4f}")
```

1. Visualise the area under $f(x) = \sin(x)$ from 0 to π . Use `plt.fill_between` to shade the area and compute it numerically with a Riemann sum.

```
import jax.numpy as jnp
import matplotlib.pyplot as plt

x = jnp.linspace(0, jnp.pi, 500)
y = jnp.sin(x)

plt.plot(x, y, color="purple", linewidth=2)
plt.fill_between(x, y, alpha=0.2, color="purple")
plt.title(f"Area = {jnp.sum(jnp.sin(x) * (jnp.pi / 500)):.4f} (exact: 2.0)")
plt.show()
```

13 Multivariate Calculus

Multivariate calculus extends derivatives and integrals to functions of many variables, which is essential since ML models have millions of parameters. This file covers partial derivatives, gradients, the Jacobian, the Hessian, and the multivariable chain rule that makes backpropagation possible.

- So far, our functions have taken a single input x and produced a single output $f(x)$. But in ML, we almost never work with just one variable.
- Consider a function of two variables, like $f(x, y) = x^2 + y^2$. This defines a surface in 3D space, a bowl shape. We want to know: if we nudge x a little while keeping y fixed, how does f change? That is a **partial derivative**.

- The **partial derivative** of f with respect to x , written $\frac{\partial f}{\partial x}$, treats every other variable as a constant and differentiates normally with respect to x .

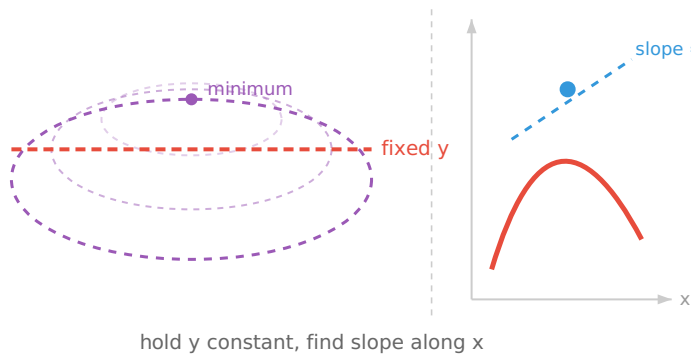
- For $f(x, y) = x^2y + 3x - 2y$:

$$\frac{\partial f}{\partial x} = 2xy + 3 \quad \frac{\partial f}{\partial y} = x^2 - 2$$

- To compute $\frac{\partial f}{\partial x}$, we treated y as a constant, so x^2y differentiated to $2xy$, $3x$ to 3 , and $-2y$ to 0 .
- To compute $\frac{\partial f}{\partial y}$, we treated x as a constant, so x^2y differentiated to x^2 , $3x$ to 0 , and $-2y$ to -2 .

- Geometrically, taking a partial derivative with respect to x is like slicing the 3D surface with a plane parallel to the xz -plane (at a fixed y value) and finding the slope of the resulting curve.

Partial derivative: slice the surface at fixed y



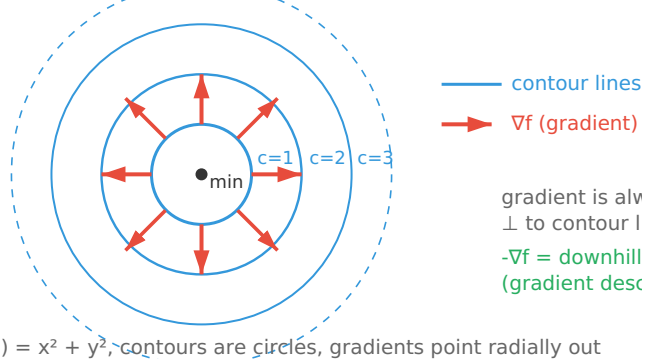
Partial derivative: slice the surface by holding one variable constant

- The **gradient** collects all the partial derivatives into a single vector:

$$\nabla f = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right)$$

- For $f(x, y) = x^2 + y^2$: $\nabla f(x, y) = (2x, 2y)$. At the point $(1, 2)$: $\nabla f(1, 2) = (2, 4)$.
- The gradient has two key properties:
 - Direction:** it points in the direction of steepest increase. Imagine a hiker on a mountain. The gradient at their position points straight uphill, along the steepest path.
 - Magnitude:** $\|\nabla f\|$ gives the rate of increase in that steepest direction. A large gradient means the terrain is steep; a small gradient means it is nearly flat.

Gradient vectors on contour lines



Gradient vectors point uphill, perpendicular to contour lines

- Since the gradient points uphill, moving in the opposite direction ($-\nabla f$) goes downhill, towards lower values. This simple idea is the basis of **gradient descent**, an optimisation technique we will explore in detail in later chapters. For now, the key takeaway is that the gradient tells you which way is “up” and how steep the climb is.
- The **directional derivative** generalises partial derivatives. Instead of asking “how does f change along the x -axis?” , it asks “how does f change along any direction $\mathbf{u}$?” It is computed as the dot product of the gradient with a unit vector:

$$D_{\mathbf{u}}f = \nabla f \cdot \mathbf{u}$$

- For $f(x, y) = x^2 + y^2$ at $(1, 2)$ in the direction of $\mathbf{v} = (3, 4)$: first normalise to get $\mathbf{u} = (3/5, 4/5)$, then $D_{\mathbf{u}}f = (2, 4) \cdot (3/5, 4/5) = 6/5 + 16/5 = 22/5$.
- Partial derivatives are special cases of directional derivatives where the direction is along a coordinate axis. If the directional derivative is zero in some direction, the function is flat in that direction at that point.
- **Contour lines** (or level curves) connect points where a function has the same value. For $f(x, y) = x^2 + y^2$, the contour lines are circles centred at the origin: $x^2 + y^2 = c$ for different values of c .
- Contour lines never cross each other (a point cannot have two different function values).
- The gradient is always perpendicular to the contour lines, pointing from lower to higher values.
- Closely spaced contour lines indicate steep terrain; widely spaced lines indicate gentle slopes.
- So far, our functions produced a single output. But many functions produce multiple outputs. A function $\mathbf{F} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ takes n inputs and produces m outputs. The **Jacobian matrix** organises all the partial derivatives of such a vector-valued function:

```
J = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}
```

- Each row of the Jacobian is the gradient of one output component. For a function with 3 inputs and 2 outputs, the Jacobian is a 2×3 matrix.
- The Jacobian generalises the derivative to vector-valued functions.
- Just as the derivative of a scalar function tells you how much the output changes per unit input change, the Jacobian tells you how each output changes with respect to each input.
- The **determinant of the Jacobian** measures how much a transformation locally stretches or compresses space.
- If the determinant is 2, small regions double in area. If it is 0, the transformation squashes space to a lower dimension (recall from our chapter on matrices that a zero determinant means a singular, non-invertible transformation).
- When several transformations are composed (one feeding into the next), the Jacobian of the overall mapping is the product of the individual Jacobians. We will see this idea become central in later chapters.
- Where the gradient captures first-order information (slopes), the **Hessian matrix** captures second-order information (curvature).
- For a scalar function $f(x_1, \dots, x_n)$, the Hessian is the $n \times n$ matrix of all second partial derivatives:

```
H = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix}
```

- For $f(x, y) = x^3 + 2xy^2 - y^3$, the gradient is $(3x^2 + 2y^2, 4xy - 3y^2)$, and the Hessian is:

$$H = \begin{bmatrix} 6x & 4y \\ 4y & 4x - 6y \end{bmatrix}$$

- The diagonal entries ($6x$ and $4x - 6y$) tell you how the slope in the x -direction changes as you move in x , and similarly for y .
- The off-diagonal entries ($4y$) tell you how the slope in one direction changes as you move in the other direction.
- **Clairaut's theorem** guarantees that for functions with continuous second derivatives, the mixed partial derivatives are equal: $\frac{\partial^2 f}{\partial x \partial y} = \frac{\partial^2 f}{\partial y \partial x}$.
- This means the Hessian is symmetric, which (as we saw in the matrices chapter) guarantees real eigenvalues and orthogonal eigenvectors.
- The Hessian tells us about the shape of the function near a critical point (where the gradient is zero):
 - If H is positive definite (all eigenvalues positive), the point is a **local minimum**, the surface curves upward in every direction like a bowl.
 - If H is negative definite (all eigenvalues negative), the point is a **local maximum**, the surface curves downward like an inverted bowl.
 - If H has both positive and negative eigenvalues, the point is a **saddle point**, the surface curves up in some directions and down in others, like a mountain pass.
- The **multivariate chain rule** extends the chain rule to functions of several variables. If $z = f(x, y)$ where $x = g(t)$ and $y = h(t)$, then:

$$\frac{dz}{dt} = \frac{\partial f}{\partial x} \frac{dx}{dt} + \frac{\partial f}{\partial y} \frac{dy}{dt}$$

- Each path from t to z contributes a term: the partial derivative along that path times the derivative of the intermediate variable with respect to t .
- For example, if $z = x^2y + 3x - y^2$, $x = \cos(t)$, $y = \sin(t)$:

$$\frac{dz}{dt} = (2xy + 3)(-\sin t) + (x^2 - 2y)(\cos t)$$

- Beyond computing derivatives by hand, there are three approaches:
 - **Numerical differentiation:** approximate $f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$ for small h . Simple but noisy and inaccurate.
 - **Symbolic differentiation:** apply differentiation rules algebraically to produce an exact formula. Can produce expressions that grow exponentially large.
 - **Automatic differentiation (autodiff):** tracks the chain of operations and computes exact derivatives efficiently. This is what JAX, PyTorch, and TensorFlow use. It gives exact numerical values (not approximate) without producing bloated symbolic expressions.

13.1 Coding Tasks (use CoLab or notebook)

1. Compute the gradient of $f(x, y) = x^2y + 3x - 2y$ at the point $(1, 2)$ using `jax.grad`. Since f takes a vector input, use `jax.grad` with `argnums`.

```
import jax
import jax.numpy as jnp

def f(x, y):
    return x**2 * y + 3*x - 2*y

df_dx = jax.grad(f, argnums=0)
df_dy = jax.grad(f, argnums=1)

x, y = 1.0, 2.0
print(f"∂f/∂x = {df_dx(x, y):.4f} (expected: {2*x*y + 3:.4f})")
print(f"∂f/∂y = {df_dy(x, y):.4f} (expected: {x**2 - 2:.4f})")
```

1. Compute the Jacobian of a vector-valued function using `jax.jacobian`. Compare with manual calculation.

```
import jax
import jax.numpy as jnp

def F(x):
    return jnp.array([x[0]**2 + x[1], x[0] * x[1]**2])
```

```
J = jax.jacobian(F)
x = jnp.array([1.0, 2.0])
print(f"Jacobian at (1,2):\n{J(x)}")
# Expected: [[2*x[0], 1], [x[1]**2, 2*x[0]*x[1]]] = [[2, 1], [4, 4]]
```

1. Compute the Hessian of $f(x, y) = x^3 + 2xy^2 - y^3$ using `jax.hessian` and verify it is symmetric.

```
import jax
import jax.numpy as jnp

def f(xy):
    x, y = xy[0], xy[1]
    return x**3 + 2*x*y**2 - y**3

H = jax.hessian(f)
point = jnp.array([1.0, 2.0])
hess = H(point)
print(f"Hessian:\n{hess}")
print(f"Symmetric: {jnp.allclose(hess, hess.T)}")
# Expected: [[6x, 4y], [4y, 4x-6y]] = [[6, 8], [8, -8]]
```

1. Build a minimal autodiff engine from scratch.
 - Each `Var` tracks its value and how to propagate gradients backward through the chain rule.
 - Try extending it with more operations (division, power, etc.).
 - This is the foundations of how JAX, PyTorch and Numpy were designed.

```
python class Var: def init(self, val, children=(), backward_fn=None):
    self.val = val self.grad = 0.0 self.children = children self.backward_fn =
    backward_fn
```

```
def add(self, other): out = Var(self.val + other.val, children=(self, other)) def
    _backward(): self.grad += out.grad # d(a+b)/da = 1 other.grad += out.grad #
    d(a+b)/db = 1 out.backward_fn = _backward return out
```

```
def mul(self, other): out = Var(self.val * other.val, children=(self, other)) def
    _backward(): self.grad += other.val * out.grad # d(ab)/da = b other.grad +=
    self.val out.grad # d(a*b)/db = a out.backward_fn = _backward return out
```

```
def backward(self): # topological sort then propagate gradients # we will go
    through this in data structures and algorithms order, visited = [], set() def
```

```

topo(v): if v not in visited: visited.add(v) for c in v.children: topo(c)
order.append(v) topo(self) self.grad = 1.0 for v in reversed(order): if
v.backward_fn: v.backward_fn()

```

14 $f(x, y) = xxy + x$ at (3, 2)

```
x = Var(3.0) y = Var(2.0) f = x * x * y + x # = 332 + 3 = 21
```

```
f.backward() print(f" f = {f.val} ") # 21.0 print(f" df/dx = {x.grad} ") # 2xy + 1 = 13.0
print(f" df/dy = {y.grad} ") # x*x = 9.0
```

Function Approximation

Function approximation replaces complex functions with simpler ones that are close enough to be useful. This file covers linearisation, Taylor series, polynomial approximation, Fourier series, and the universal approximation theorem, the theoretical backbone of why neural networks can learn arbitrary mappings.

- Many functions we encounter are too complex to work with directly. Computing $e^{0.1}$ on paper, predicting the trajectory of a satellite, etc. all involve functions that do not have simple closed-form answers.

- **Function approximation** replaces a complicated function with a simpler one that is "close enough" over the region we care about.

- The most natural approximation is a polynomial. Polynomials are just sums of powers of x with coefficients, and they are easy to evaluate, differentiate, and integrate.

- But why do polynomials work so well as approximators? Consider what each power of x contributes.

- The constant term a_0 sets the baseline value.
- The $a_1 x$ term adds a slope.
- The $a_2 x^2$ term adds curvature.
- Each higher power captures finer detail about the function's shape.

![Each polynomial term adds another layer of detail to the approximation](../images/polynomial_buildup.svg)

- By choosing the right coefficients, we can match a function's value, slope, curvature, and higher-order behaviour at a point, one piece at a time.

- With enough terms, the polynomial can mimic almost any smooth function.
- The question becomes: how do we find the right coefficients?
- **Linearisation** is the simplest approximation. Near a point $x = a$, we replace the function with its tangent line:

$$L(x) = f(a) + f'(a)(x - a)$$

- This is the first-order **Taylor approximation**. It says: start at the known value $f(a)$, then adjust by the slope times the distance from a .
- For example, linearise $\sin(x)$ at $x = 0$: $f(0) = 0$, $f'(0) = \cos(0) = 1$, so $L(x) = x$. Near zero, $\sin(x) \approx x$. Try it: $\sin(0.1) = 0.0998 \dots \approx 0.1$.
- But linearisation is only good very close to a . Move further away and the approximation falls apart. To do better, we include higher-order terms.
- The **Taylor series** represents a function as an infinite sum of polynomial terms, each capturing finer detail about the function's behaviour near a point a :

$$f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!} (x - a)^n = f(a) + f'(a)(x-a) + \frac{f''(a)}{2!}(x-a)^2 + \frac{f'''(a)}{3!}(x-a)^3 + \dots$$

![[Taylor series: adding more terms gives a better approximation](../images/taylor_approximation.svg)]

- Each successive term adds a correction. The first term matches the value, the second matches the slope, the third matches the curvature, and so on. The more terms we include, the larger the region where the approximation is accurate.
- The $n!$ in the denominator is not arbitrary. When you differentiate $(x - a)^n$ exactly n times, you get $n!$. The factorial cancels this out, ensuring that the n -th derivative of the Taylor polynomial equals the n -th derivative of the original function at $x = a$.
- A **Maclaurin series** is simply a Taylor series centred at $a = 0$:

$$f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(0)}{n!} x^n$$

- Some famous Maclaurin series:

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots$$

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$$

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \dots$$

- Notice that $\sin x$ has only odd powers (it is an odd function) and $\cos x$ has only even powers (it is an even function). The alternating signs cause the approximation to oscillate around the true value, converging from both sides.

- Let us approximate $e^{0.5}$ using four terms: $1 + 0.5 + \frac{0.25}{2} + \frac{0.125}{6} = 1 + 0.5 + 0.125 + 0.02083 \approx 1.6458$. The true value is $1.6487\dots$, so four terms already give us three correct decimal places.

- Not every Taylor series converges everywhere. The **radius of convergence** tells us how far from the centre a the series gives valid results. Within that radius, the polynomial approximation can be made as accurate as we want by adding more terms. Outside it, the series diverges.

- A **power series** is the general form: $\sum_{n=0}^{\infty} a_n (x - c)^n$. Taylor series are power series where the coefficients are determined by derivatives. Other power series might be defined by some other rule. The **ratio test** determines convergence: compute $\lim_{n \rightarrow \infty} \left| \frac{a_{n+1}}{a_n} \right|$. If this limit is L , the radius of convergence is $R = 1/L$.

- When we truncate a Taylor series after n terms, we incur an error. The **Lagrange remainder** bounds this error:

$$R_n(x) = \frac{f^{(n+1)}(c)}{(n+1)!} (x-a)^{n+1}$$

- Here c is some unknown point between a and x . We do not know c exactly, but we can often bound $|f^{(n+1)}(c)|$ to get a worst-case error estimate. The $(n+1)!$ in the denominator grows extremely fast, so the error shrinks rapidly as we add more terms (for functions within the radius of convergence).

- For a function of multiple variables, the Taylor expansion includes mixed partial derivatives. The second-order approximation of $f(\mathbf{x})$ around a point $\mathbf{a}$ is:

$$f(\mathbf{x}) \approx f(\mathbf{a}) + \nabla f(\mathbf{a})^T (\mathbf{x} - \mathbf{a}) + \frac{1}{2} (\mathbf{x} - \mathbf{a})^T H(\mathbf{a}) (\mathbf{x} - \mathbf{a})$$

- The first term is the value, the second uses the gradient (a vector, as we saw in multivariate calculus), and the third uses the Hessian matrix (which captures curvature). This connects our matrices chapter directly to calculus: the Hessian is a matrix of second derivatives that describes the shape of the function's surface.

- This multivariate second-order approximation is the foundation of Newton's method and other second-order optimisation techniques, which we will see in the next file.

- Beyond polynomials, there are other approximation methods worth knowing about:

- **Spline interpolation**: instead of one high-degree polynomial, use many low-degree polynomials stitched together smoothly. This avoids the wild oscillations that high-degree polynomials can produce.

- **Fourier series**: approximate periodic functions as sums of sines and cosines. Essential in signal processing and audio.

- **Neural networks**: universal function approximators. With enough neurons, they can approximate any continuous function to arbitrary accuracy. This is the theoretical justification for deep learning.

- A function is called "well-behaved" if it has properties that make approximation reliable: continuity (no jumps), differentiability (no sharp corners), smoothness (derivatives of all orders exist), and boundedness (outputs stay finite).

- Polynomials, exponentials, and trigonometric functions are all well-behaved. The better-behaved a function is, the fewer Taylor terms you need for a good approximation.

Coding Tasks (use Colab or notebook)

1. Approximate e^x using increasing numbers of Taylor terms and visualise how the approximation improves.

```
```python
```

```
import jax.numpy as jnp
```

```
import matplotlib.pyplot as plt
```

```
x = jnp.linspace(-2, 3, 300)
```

```
plt.plot(x, jnp.exp(x), "k-", linewidth=2, label="ex (exact)")
```

```
colors = ["#e74c3c", "#3498db", "#27ae60", "#9b59b6"]
```

```
for n, color in zip([1, 2, 4, 8], colors):
```

```
 approx = sum(x**k / jnp.array(float(jnp.prod(jnp.arange(1, k+1)) if k > 0
else 1))
```

```
 for k in range(n+1))
```

```
 plt.plot(x, approx, color=color, linestyle="--", label=f"{n} terms")
```

```
plt.ylim(-2, 15)
```

```
plt.legend()
```

```
plt.title("Taylor approximation of ex")
plt.show()
```

1. Compute the Lagrange remainder to bound the error of approximating  $\sin(1)$  with different numbers of Taylor terms.

```
import jax.numpy as jnp

x = 1.0
exact = jnp.sin(x)

taylor = 0.0
for n in range(8):
 sign = (-1)**n
 factorial = float(jnp.prod(jnp.arange(1, 2*n+2)))
 taylor += sign * x**(2*n+1) / factorial
 error = abs(exact - taylor)
 bound = x**(2*n+3) / float(jnp.prod(jnp.arange(1, 2*n+4)))
 print(f"terms={n+1} approx={taylor:.10f} error={error:.2e} bound={bound:.2e}")
```

1. Compare linearisation vs quadratic Taylor approximation of  $\cos(x)$  near  $x = 0$ . Plot both approximations alongside the true function and observe the range where each is accurate.

```
import jax.numpy as jnp
import matplotlib.pyplot as plt

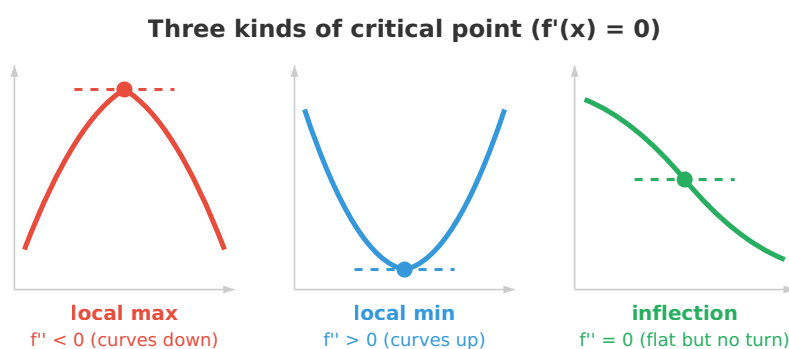
x = jnp.linspace(-3, 3, 300)
plt.plot(x, jnp.cos(x), "k-", linewidth=2, label="cos(x)")
plt.plot(x, jnp.ones_like(x), "--", color="#e74c3c", label="linear: 1")
plt.plot(x, 1 - x**2/2, "--", color="#3498db", label="quadratic: 1 - x2/2")
plt.plot(x, 1 - x**2/2 + x**4/24, "--", color="#27ae60", label="4th order")
plt.ylim(-2, 2)
plt.legend()
plt.title("Taylor approximations of cos(x)")
plt.show()
```

## 15 Optimisation

Optimisation is the mathematical core of model training, finding the parameters that minimise a loss function. This file covers critical points, convexity, gradient descent,

Newton's method, constrained optimisation with Lagrange multipliers, and the optimisers (SGD, Adam) that power modern deep learning.

- Training a neural network, fitting a regression line, tuning hyperparameters: at the core of almost every ML algorithm is an **optimisation** problem.
- We have some function (a loss, a cost, an objective) and we want to find the inputs that make it as small (or large) as possible.
- Before optimising, we need to understand **zeros** (or roots) of functions. A zero of  $f(x)$  is a value  $x$  where  $f(x) = 0$ . Graphically, these are the x-intercepts.
- For example,  $f(x) = x^2 - 3x + 2 = (x - 1)(x - 2)$  has zeros at  $x = 1$  and  $x = 2$ . Between the zeros, the function is negative ( $f(1.5) = -0.25$ ); outside the zeros, it is positive. The zeros divide the number line into regions where the function has constant sign.
- The **multiplicity** of a zero is how many times the corresponding factor appears.
- At a simple zero (multiplicity 1), the graph crosses the x-axis. At a double zero (multiplicity 2), the graph touches the x-axis but bounces back without crossing, appearing “flat” at that point.
- Finding zeros matters because the zeros of the derivative  $f'(x)$  are the **critical points** of  $f(x)$ , the candidates for maxima and minima.
- At a maximum or minimum, the tangent line is flat (slope = 0), so  $f'(x) = 0$ .

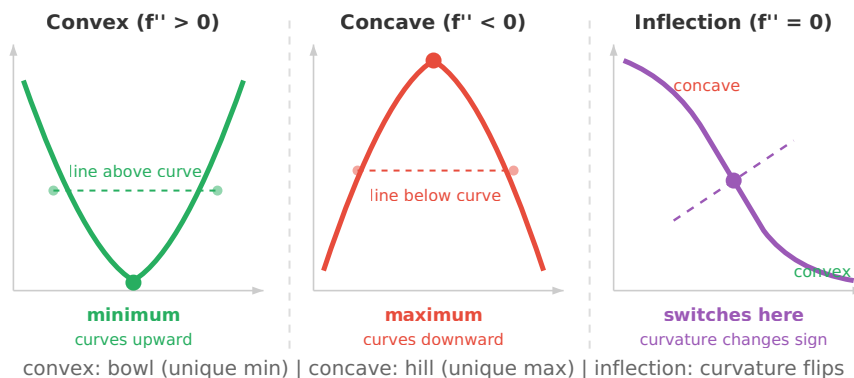


Critical points: where the derivative equals zero, the function has a peak, valley, or saddle

- But not every critical point is a maximum or minimum. A point where  $f'(x) = 0$  could also be an **inflection point** (like  $x = 0$  for  $f(x) = x^3$ ), where the function flattens momentarily but does not change direction.



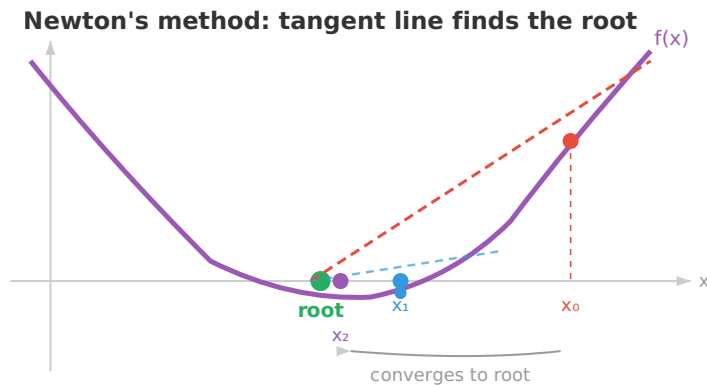
- The **second derivative test** resolves this. At a critical point  $x = c$  where  $f'(c) = 0$ :
  - If  $f''(c) > 0$ : the curve is concave up (like a bowl), so  $c$  is a **local minimum**.
  - If  $f''(c) < 0$ : the curve is concave down (like a hill), so  $c$  is a **local maximum**.
  - If  $f''(c) = 0$ : the test is inconclusive; higher derivatives or other methods are needed.
- For example,  $f(x) = x^3 - 3x$ . The derivative is  $f'(x) = 3x^2 - 3 = 3(x-1)(x+1)$ , so critical points are at  $x = -1$  and  $x = 1$ . The second derivative is  $f''(x) = 6x$ . At  $x = -1$ :  $f''(-1) = -6 < 0$  (local max). At  $x = 1$ :  $f''(1) = 6 > 0$  (local min).
- A function is **convex** if the line segment between any two points on its graph lies above (or on) the graph. Think of it as a bowl shape, curving upward everywhere. Mathematically,  $f$  is convex if  $f''(x) \geq 0$  for all  $x$ .



Convex functions have a unique global minimum; non-convex functions can have many local minima

- Convexity is powerful because convex functions have a remarkable property: every local minimum is also the **global minimum**. There are no deceptive local valleys to get trapped in. If you roll a ball into a convex bowl, it will always reach the bottom.
- A function is **concave** (curving downward) if  $-f$  is convex. Points where the function transitions between concave and convex are **inflection points**, occurring where  $f''(x) = 0$ .
- **Newton's method** finds zeros of functions (and by extension, critical points of their derivatives) using tangent lines. Starting from an initial guess  $x_0$ , it iteratively refines:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$



Newton's method: follow the tangent line to find a better approximation of the root

- The idea: at  $x_n$ , draw the tangent line and find where it crosses the x-axis. That crossing point becomes  $x_{n+1}$ . For well-behaved functions with a good starting point, Newton's method converges very quickly (quadratically, meaning the number of correct digits roughly doubles each step).

- For example, to find

---

$\sqrt{5}$

(a zero of  $f(x) = x^2 - 5$ ):  $f'(x) = 2x$ , so  $x_{n+1} = x_n - \frac{x_n^2 - 5}{2x_n}$ . Starting at  $x_0 = 2$ :  $x_1 = 2.25$ ,

$x_2 = 2.2361\dots$ , which is already accurate to four decimal places.

- Newton's method can fail if the initial guess is far from the root, if  $f'(x) = 0$  near the root, or if the function has inflection points nearby. It also requires computing the derivative, which may be expensive.
- For optimisation (finding minima instead of zeros), we apply Newton's method to  $f'(x) = 0$ , which gives the update:

$$x_{n+1} = x_n - \frac{f'(x_n)}{f''(x_n)}$$

- In multiple dimensions, this becomes  $\mathbf{x}_{n+1} = \mathbf{x}_n - H^{-1} \nabla f(\mathbf{x}_n)$ , where  $H$  is the Hessian matrix. This is the second-order Taylor approximation from the previous file in action: approximate the function as a quadratic, jump to the minimum of that quadratic, repeat.

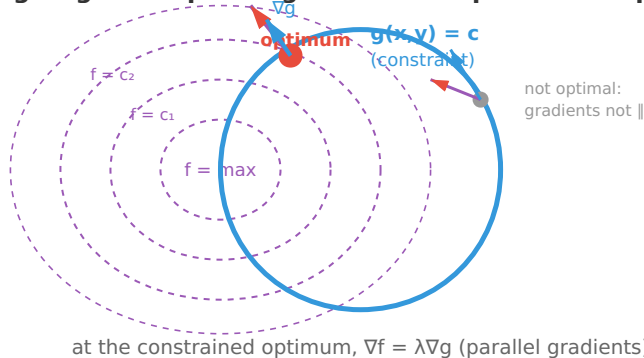
- **Lagrange multipliers solve constrained optimisation:** find the optimum of  $f(x, y)$  subject to a constraint  $g(x, y) = c$ . Instead of searching all of  $\mathbb{R}^n$ , we are restricted to the set where the constraint holds (a curve or surface).
- The key insight is geometric: at the constrained optimum, the gradient of  $f$  must be parallel to the gradient of  $g$ . If they were not parallel, we could move along the constraint in a direction that still improves  $f$ , so we would not be at the optimum yet.
- We introduce a new variable  $\lambda$  (the Lagrange multiplier) and define the **Lagrangian**:

$$\mathcal{L}(x, y, \lambda) = f(x, y) - \lambda(g(x, y) - c)$$

- Setting all partial derivatives to zero gives a system of equations whose solutions are the constrained optima:

$$\frac{\partial \mathcal{L}}{\partial x} = 0, \quad \frac{\partial \mathcal{L}}{\partial y} = 0, \quad \frac{\partial \mathcal{L}}{\partial \lambda} = 0$$

**Lagrange multipliers: gradients are parallel at optimum**



Lagrange multipliers: at the optimum, gradients of  $f$  and  $g$  are parallel

- For example, maximise  $f(x, y) = x^2y$  subject to  $x^2 + y^2 = 1$ . The Lagrangian is  $\mathcal{L} = x^2y - \lambda(x^2 + y^2 - 1)$ . Taking partials:

$$2xy - 2\lambda x = 0, \quad x^2 - 2\lambda y = 0, \quad x^2 + y^2 = 1$$

- From the first equation (assuming  $x \neq 0$ ):  $\lambda = y$ . Substituting into the second:  $x^2 = 2y^2$ . Combined with the constraint:  $2y^2 + y^2 = 1$ , so  $y = \frac{1}{\sqrt{3}}$ . The maximum

value is  $f = \frac{2}{3\sqrt{3}}$ .

- For inequality constraints ( $g(x, y) \leq c$  instead of  $=c$ ), the **Karush-Kuhn-Tucker (KKT) conditions** generalise Lagrange multipliers. The constraint is either active (binding, treated as equality) or inactive (the solution lies in the interior and the constraint is irrelevant).
- In practice, we rarely optimise by hand. Here are the main algorithmic families:
  - **First-order methods** (use only gradient): gradient descent, stochastic gradient descent (SGD), Adam. These are cheap per step but can converge slowly, especially on ill-conditioned problems.
  - **Second-order methods** (use gradient and Hessian): Newton's method converges fast but computing and inverting the Hessian is expensive ( $O(n^3)$  for  $n$  parameters). **Quasi-Newton methods** (like BFGS and L-BFGS) approximate the Hessian using only gradient information, achieving faster convergence than first-order methods without the full cost of second-order methods.
  - **Conjugate gradient**: efficient for large sparse systems, using only matrix-vector products instead of storing the full Hessian.
  - **Gauss-Newton** and **Levenberg-Marquardt**: specialised for least-squares problems (common in regression), approximating the Hessian via the Jacobian.
  - **Natural gradient descent**: accounts for the geometry of the parameter space using the Fisher information matrix, which can be more effective for probabilistic models.
- The choice of optimiser depends on the problem. For deep learning, first-order methods (especially Adam) dominate because the number of parameters is enormous (millions to billions), making Hessian computation impractical. For smaller problems with smooth objectives, second-order methods can be dramatically faster.

## 15.1 Coding Tasks (use CoLab or notebook)

---

1. Implement Newton's method to find

$$\sqrt{7}$$

(a zero of  $f(x) = x^2 - 7$ ). Observe the rapid convergence.

```

import jax.numpy as jnp

f = lambda x: x**2 - 7
df = lambda x: 2*x

x = 3.0 # initial guess
for i in range(6):
 x = x - f(x) / df(x)
 print(f"step {i+1}: x = {x:.10f} (error: {abs(x - jnp.sqrt(7.0)):.2e})")

```

1. Use gradient descent to minimise  $f(x, y) = (x - 3)^2 + (y + 1)^2$ . The minimum is at  $(3, -1)$ . Experiment with different learning rates.

```

import jax
import jax.numpy as jnp

def f(params):
 x, y = params
 return (x - 3)**2 + (y + 1)**2

grad_f = jax.grad(f)
params = jnp.array([0.0, 0.0])
lr = 0.1

for i in range(20):
 g = grad_f(params)
 params = params - lr * g
 if i % 5 == 0 or i == 19:
 print(f"step {i:2d}: ({params[0]:.4f}, {params[1]:.4f})")
loss={f(params):.6f}")

```

1. Solve a constrained optimisation problem numerically. Maximise  $f(x, y) = xy$  subject to  $x + y = 10$  by parameterising  $y = 10 - x$  and finding the optimum of the single-variable function.

```

import jax
import jax.numpy as jnp

Substitute constraint: y = 10 - x, so f = x(10 - x) = 10x - x^2
f = lambda x: x * (10 - x)
df = jax.grad(f)

Gradient ascent (we want maximum, so add gradient)
x = 1.0
lr = 0.1

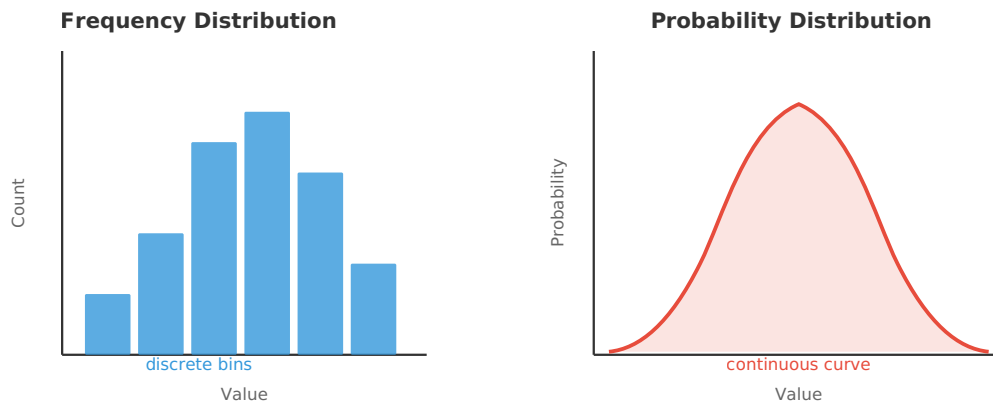
```

```
for i in range(20):
 x = x + lr * df(x)
print(f"x={x:.4f}, y={10-x:.4f}, f={f(x):.4f}") # should be x=5, y=5, f=25
```

## 16 Fundamentals of Statistics

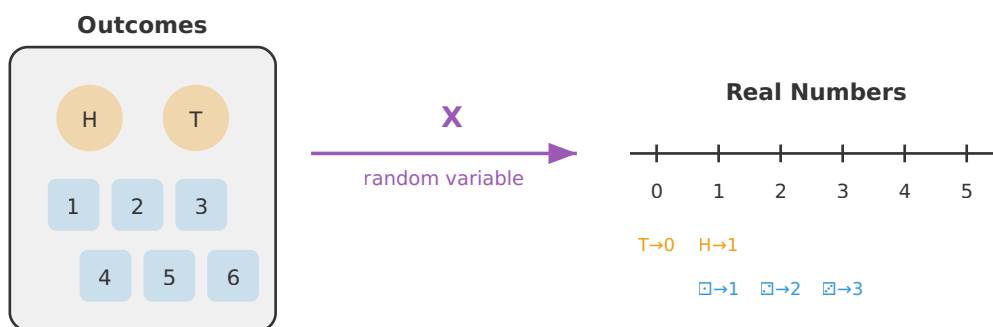
Statistics provides the language for describing data and quantifying uncertainty. This file covers distributions, random variables, PMFs, PDFs, CDFs, expectation, variance, moments, and the central limit theorem, the concepts that underpin every ML evaluation metric and loss function.

- Statistics is the science of learning from data. You collect observations, summarise them, and draw conclusions, often about things you cannot measure directly.
- Imagine you want to know the average height of every adult in a country. You cannot measure everyone, so you measure a sample and use statistics to make an informed guess about the whole population.
- There are two main branches:
  - **Descriptive statistics:** summarising data you already have (averages, charts, tables)
  - **Inferential statistics:** using a sample to make claims about a larger group
- The building block of statistics is the **distribution**, a description of how values are spread out. Everything else, averages, tests, predictions, flows from understanding distributions.
- A **frequency distribution** counts how often each value (or range of values) appears in your data. Think of sorting exam scores into bins and counting how many students fall in each bin. The result is a histogram.
- A **probability distribution** replaces raw counts with probabilities. Instead of “12 students scored between 70 and 80,” it says “there is a 0.24 probability of scoring between 70 and 80.” The histogram bars become a smooth curve when the data is continuous.



Frequency distribution as a histogram vs probability distribution as a smooth curve

- The histogram on the left is built from actual data you collected. The smooth curve on the right is a mathematical model that describes the pattern behind the data. One is empirical, the other is theoretical.
- To work with distributions mathematically, we need a way to assign numbers to outcomes. That is exactly what a **random variable** does.
- A random variable is a function that maps each outcome of an experiment to a real number. Flip a coin: the outcome is “heads” or “tails,” but a random variable  $X$  converts this to  $X(\text{heads}) = 1$  and  $X(\text{tails}) = 0$ . Now we can do arithmetic.



Random variable mapping outcomes (coin, die) to a number line

- A **discrete** random variable takes on a countable set of values: the number of heads in 10 flips, the roll of a die, the number of emails you receive in an hour.
- A **continuous** random variable can take any value in an interval: your exact height, the time until the next bus arrives, the temperature at noon.

- The distinction matters because it changes how we compute probabilities. For discrete variables, we sum. For continuous variables, we integrate (recall integrals from Chapter 3).
- For a discrete random variable, the **probability mass function (PMF)** gives the probability of each specific value:

$$P(X = x) = p(x), \text{ where } \sum_x p(x) = 1$$

- For a continuous random variable, the **probability density function (PDF)** gives the probability of falling within a range. The probability of any single exact value is zero; only intervals have positive probability:

$$P(a \leq X \leq b) = \int_a^b f(x) dx, \text{ where } \int_{-\infty}^{\infty} f(x) dx = 1$$

- Now that we can assign numbers to outcomes, the most natural question is: what value do we expect on average?
- **Expectation** (or expected value) is the weighted average of all possible values, where the weights are the probabilities. Think of it as the “centre of gravity” of the distribution.
- If you roll a fair die many times, your average roll converges to 3.5. That is the expected value, even though you can never actually roll a 3.5.
- For a discrete random variable:

$$E[X] = \sum_x x \cdot p(x)$$

- For a continuous random variable (using the integral from Chapter 3):

$$E[X] = \int_{-\infty}^{\infty} x \cdot f(x) dx$$

- Example: a fair six-sided die has  $p(x) = 1/6$  for  $x = 1, 2, 3, 4, 5, 6$ .



$$E[X] = 1 \cdot \frac{1}{6} + 2 \cdot \frac{1}{6} + 3 \cdot \frac{1}{6} + 4 \cdot \frac{1}{6} + 5 \cdot \frac{1}{6} + 6 \cdot \frac{1}{6} = \frac{21}{6} = 3.5$$

- Expectation is linear, meaning  $E[aX + b] = aE[X] + b$ . This property is extremely useful and shows up constantly in ML loss functions.
- Expectation tells us the centre, but it says nothing about how spread out the values are. To describe the full shape of a distribution, we need **moments**.
- A moment is an expectation of a power of X. The k-th **raw moment** is:

$$\mu_k' = E[X^k]$$

- The first raw moment ( $k=1$ ) is just the mean:  $\mu_1' = E[X] = \mu$ .
- Raw moments are measured from zero. Often we care about deviation from the mean instead. The k-th **central moment** centres the measurement:

$$\mu_k = E[(X - \mu)^k]$$

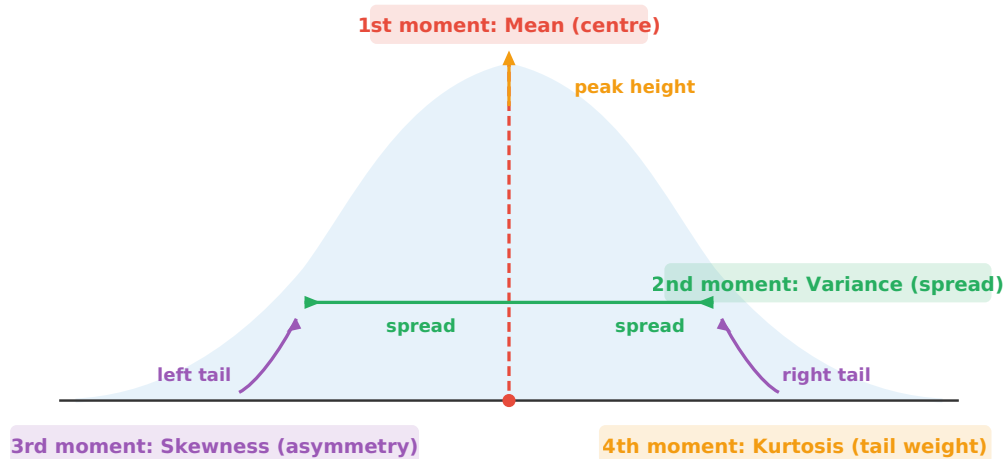
- The first central moment is always zero (deviations above and below the mean cancel). The second central moment is the **variance**.
- To compare distributions on different scales, we **standardise** by dividing by the appropriate power of the standard deviation  $\sigma$ :

◉

$\mu$

$$z_k = \frac{\mu_k}{\sigma^k}$$

- Each moment captures a different aspect of the distribution's shape:



Bell curve annotated with what each moment captures: mean (centre), variance (spread), skewness (asymmetry), kurtosis (tail weight)

- **1st moment (Mean):** Where the distribution is centred. The balance point.
- **2nd moment (Variance):** How spread out values are around the mean. Higher variance means wider.
- **3rd moment (Skewness):** Whether the distribution leans left or right. Zero skewness means symmetric.
- **4th moment (Kurtosis):** How heavy the tails are. Higher kurtosis means more extreme outliers.
- Let us work through all four moments for a concrete dataset:  $X = \{2, 4, 4, 4, 5, 5, 7, 9\}$ .
- **Step 1: Mean** (1st raw moment)

$$\mu = \frac{2 + 4 + 4 + 4 + 5 + 5 + 7 + 9}{8} = \frac{40}{8} = 5$$

- **Step 2: Variance** (2nd central moment). Subtract the mean from each value, square, then average:

$$\sigma^2 = \frac{(2-5)^2 + (4-5)^2 + (4-5)^2 + (4-5)^2 + (5-5)^2 + (5-5)^2 + (7-5)^2 + (9-5)^2}{8}$$

$$= \frac{9 + 1 + 1 + 1 + 0 + 0 + 4 + 16}{8} = \frac{32}{8} = 4$$

- The **standard deviation** is  $\sigma =$

$$\sqrt{4}$$

$$= 2.$$

- **Step 3: Skewness** (standardised 3rd central moment). Cube the deviations, average, divide by  $\sigma^3$ :

↺

$\mu$

$$^3 = \frac{1}{8} \cdot \frac{(-3)^3 + (-1)^3 + (-1)^3 + (-1)^3 + 0^3 + 0^3 + 2^3 + 4^3}{2^3}$$

$$= \frac{1}{8} \cdot \frac{-27 - 1 - 1 - 1 + 0 + 0 + 8 + 64}{8} = \frac{42}{64} = 0.656$$

- Positive skewness means the right tail is longer, which makes sense since 9 is far above the mean.
- **Step 4: Kurtosis** (standardised 4th central moment). Raise deviations to the 4th power:

↺

$\mu$

$$^4 = \frac{1}{8} \cdot \frac{(-3)^4 + (-1)^4 + (-1)^4 + (-1)^4 + 0^4 + 0^4 + 2^4 + 4^4}{2^4}$$

$$= \frac{1}{8} \cdot \frac{81 + 1 + 1 + 1 + 0 + 0 + 16 + 256}{16} = \frac{356}{128} = 2.781$$

- A normal distribution has kurtosis of 3 (called “mesokurtic” ). Our value of 2.781 is close, suggesting the tails are roughly normal. Values above 3 ( “leptokurtic” ) signal heavier tails; below 3 ( “platykurtic” ) signal lighter tails. Some formulas report **excess kurtosis** by subtracting 3, so our excess kurtosis would be -0.219.

## 16.1 Coding Tasks (use CoLab or notebook)

1. Compute the expected value of a loaded die where face 6 has probability 0.3 and all other faces share the remaining probability equally. Verify by simulating 100,000 rolls.

```
import jax
import jax.numpy as jnp

Loaded die: face 6 has p=0.3, others share 0.7 equally
probs = jnp.array([0.14, 0.14, 0.14, 0.14, 0.14, 0.30])
faces = jnp.array([1, 2, 3, 4, 5, 6])

Analytical expected value
ev = jnp.sum(faces * probs)
print(f"Expected value (formula): {ev:.4f}")

Simulation
key = jax.random.PRNGKey(42)
rolls = jax.random.choice(key, faces, shape=(100_000,), p=probs)
print(f"Expected value (simulation): {rolls.mean():.4f}")
```

1. Compute all four moments (mean, variance, skewness, kurtosis) for the dataset from the worked example, then modify the data and observe how each moment changes.

```
import jax.numpy as jnp

x = jnp.array([2, 4, 4, 4, 5, 5, 7, 9], dtype=jnp.float32)

mean = jnp.mean(x)
variance = jnp.mean((x - mean) ** 2)
std = jnp.sqrt(variance)
skewness = jnp.mean(((x - mean) / std) ** 3)
kurtosis = jnp.mean(((x - mean) / std) ** 4)

print(f"Mean: {mean:.3f}")
print(f"Variance: {variance:.3f}")
print(f"Std Dev: {std:.3f}")
print(f"Skewness: {skewness:.3f}")
```

```
print(f"Kurtosis: {kurtosis:.3f}")
print(f"Excess K: {kurtosis - 3:.3f}")
```

1. Visualise a PMF and CDF side by side for a fair die roll. Try changing the probabilities to see how the shapes shift.

```
import jax.numpy as jnp
import matplotlib.pyplot as plt

faces = jnp.array([1, 2, 3, 4, 5, 6])
pmf = jnp.ones(6) / 6 # fair die; try changing these!
cdf = jnp.cumsum(pmf)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 4))

ax1.bar(faces, pmf, color="#3498db", alpha=0.8)
ax1.set_title("PMF")
ax1.set_xlabel("Face")
ax1.set_ylabel("P(X = x)")
ax1.set_ylim(0, 0.5)

ax2.step(faces, cdf, where="mid", color="#e74c3c", linewidth=2)
ax2.set_title("CDF")
ax2.set_xlabel("Face")
ax2.set_ylabel("P(X ≤ x)")
ax2.set_ylim(0, 1.1)

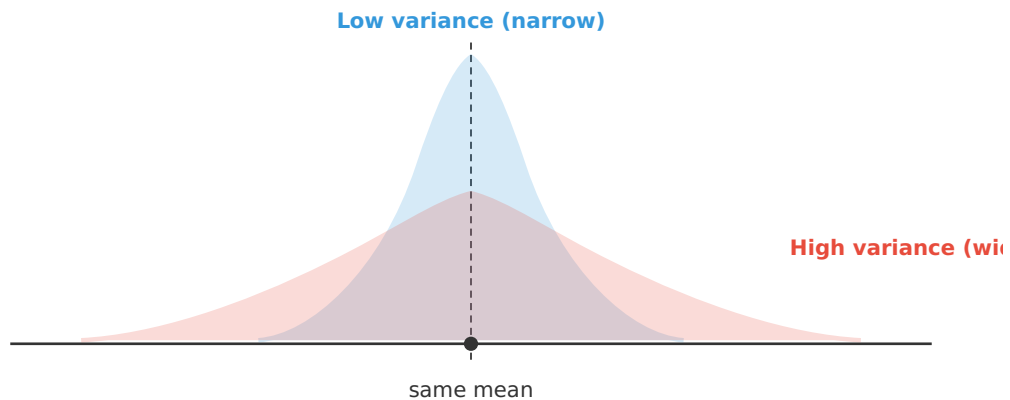
plt.tight_layout()
plt.show()
```

## 17 Statistical Measures

Statistical measures summarise data with single numbers that capture spread, position, shape, and association. This file covers variance, standard deviation, quartiles, skewness, kurtosis, covariance, correlation, and z-scores, the toolkit for exploratory data analysis and feature engineering in ML.

- In the previous file we introduced moments as a family of summary statistics. Here we unpack the practical tools that flow from them: measures of dispersion, position, shape, and association.

- **Dispersion** answers the question: how spread out is the data? Two classrooms can have the same average test score, but very different spreads.



Two distributions with the same mean but different spreads

- The narrow (blue) distribution has low variance: most values cluster tightly around the mean. The wide (red) distribution has high variance: values are scattered further out.
- **Variance** is the average squared distance from the mean. We square to avoid positive and negative deviations cancelling each other out.

$$\sigma^2 = \frac{1}{N}$$

$\sum_{i=1}^N$

$$(x_i - \mu)^2$$

- When working with a sample (not the full population), we divide by  $N - 1$  instead of  $N$ . This correction (called Bessel's correction) accounts for the fact that a sample tends to underestimate the true variability:

$$s^2 = \frac{1}{N - 1}$$

$\sum_{i=1}^N$

$$(x_i - \bar{x})^2$$

$$)^2$$

- **Standard deviation** is the square root of variance:  $\sigma =$

$$\sqrt{\sigma^2}$$

It brings the measure back to the original units. If your data is in centimetres, variance is in  $\text{cm}^2$ , but standard deviation is back in cm.

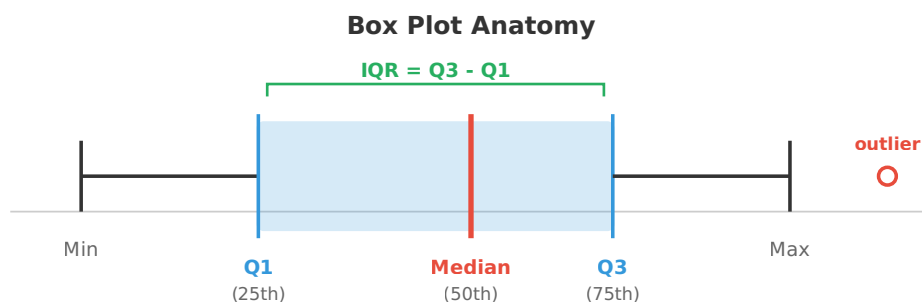
- **Mean Absolute Deviation (MAD)** is a simpler alternative. Instead of squaring, take the absolute value of each deviation:

$$\text{MAD} = \frac{1}{N} \sum_{i=1}^N |x_i - \mu|$$

N  
Σ  
i = 1

$$|x_i - \mu|$$

- MAD is more robust to outliers than variance because it does not amplify large deviations by squaring them. However, variance is more mathematically convenient (it decomposes nicely in proofs and ML optimisation).
- **Position** answers a different question: where does a specific value sit relative to the rest of the data?
- **Quartiles** split sorted data into four equal parts. Q1 (25th percentile) is the value below which 25% of data falls. Q2 is the median (50th percentile). Q3 is the 75th percentile.
- The **Interquartile Range (IQR)** is  $Q3 - Q1$ . It captures the spread of the middle 50% of data, ignoring extremes.



Box plot showing Q1, median, Q3, IQR, whiskers, and an outlier

- The box plot is one of the most useful visualisations in statistics. The box spans Q1 to Q3, the line inside is the median, whiskers extend to the most extreme non-outlier values, and dots beyond the whiskers are outliers.

- **Percentiles** generalise quartiles. The p-th percentile is the value below which p% of observations fall. Q1 is the 25th percentile, the median is the 50th, and Q3 is the 75th.
- The **z-score** tells you how many standard deviations a value is from the mean:

$$z = \frac{x - \mu}{\sigma}$$

- A z-score of 2 means the value is 2 standard deviations above the mean. A z-score of -1.5 means it is 1.5 standard deviations below. This is also called **standardisation** and is used heavily in ML for feature scaling, as it transforms any distribution to have mean 0 and standard deviation 1.
- **Shape** describes the geometry of a distribution beyond its centre and spread.
- **Skewness** (the standardised 3rd moment from the previous file) measures asymmetry. A perfectly symmetric distribution like the normal curve has skewness of zero. Positive skewness means a longer right tail (e.g. income distributions). Negative skewness means a longer left tail (e.g. age at retirement).

$$\text{Skewness} = \frac{1}{N}$$

$$\sum_{i=1}^N$$

$$\left(\frac{x_i - \mu}{\sigma}\right)^3$$

- **Kurtosis** (the standardised 4th moment) measures tail heaviness. The normal distribution has kurtosis of 3. Distributions with heavier tails (more prone to outliers) have kurtosis greater than 3.

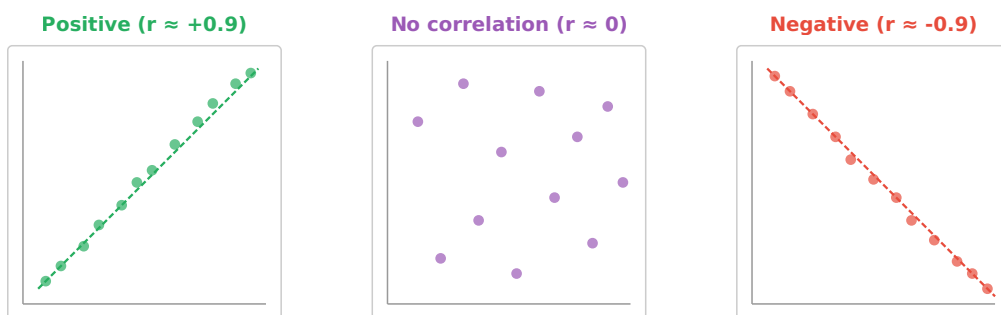


$$\text{Kurtosis} = \frac{1}{N}$$

$$\sum_{i=1}^N$$

$$\left( \frac{x_i - \mu}{\sigma} \right)^4$$

- **Correlation** measures the strength and direction of a relationship between two variables. It answers: when one variable goes up, does the other tend to go up, go down, or do nothing?



Three scatter plots showing positive, no, and negative correlation

- **Pearson correlation** ( $r$ ) measures linear association. It ranges from  $-1$  (perfect negative) through  $0$  (none) to  $+1$  (perfect positive).

$$r = \frac{\sum_{i=1}^N (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^N (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^N (y_i - \bar{y})^2}}$$

- If you recall dot products from Chapter 1, Pearson correlation is essentially the cosine similarity between the mean-centred versions of  $\mathbf{x}$  and  $\mathbf{y}$ .

- **Spearman correlation** ( $\rho$ ) measures monotonic association. Instead of using raw values, it ranks them first and then computes Pearson correlation on the ranks. This makes it robust to outliers and works even when the relationship is nonlinear, as long as it is consistently increasing or decreasing.
- **Geometric mean** is the appropriate average when values multiply together, like growth rates. If your investment grows by 10%, then 20%, then 30%, the average growth factor is not the arithmetic mean of those rates. Instead:

–

x

$$x_{\text{geo}} = \left( \prod_{i=1}^N x_i \right)^{1/N}$$

- For growth rates specifically, convert percentages to factors first (1.10, 1.20, 1.30), compute the geometric mean, then subtract 1.
- **Exponential Moving Average (EMA)** gives more weight to recent observations. Unlike a simple moving average where all points in the window are equally weighted, EMA decays exponentially:

$$\text{EMA}_t = \alpha \cdot x_t + (1 - \alpha) \cdot \text{EMA}_{t-1}$$

- The smoothing factor  $\alpha$  (between 0 and 1) controls how quickly old observations lose influence. Higher  $\alpha$  means more responsive to recent changes, lower  $\alpha$  means smoother. In ML, EMA is used in optimisers like Adam and in batch normalisation's running statistics.
- **Outlier detection** identifies data points that are unusually far from the rest. Two common methods:
  - **IQR method:** a point is an outlier if it falls below  $Q1 - 1.5 \times \text{IQR}$  or above  $Q3 + 1.5 \times \text{IQR}$
  - **Z-score method:** a point is an outlier if  $|z| > 3$  (more than 3 standard deviations from the mean)
- The IQR method is more robust because it does not assume a normal distribution. The z-score method works well when data is approximately normal but can fail when the distribution is heavily skewed.

## 17.1 Coding Tasks (use CoLab or notebook)

1. Compute variance, standard deviation, and MAD for a dataset and compare them. Observe what happens when you add an extreme outlier.

```
import jax.numpy as jnp

data = jnp.array([4, 8, 6, 5, 3, 7, 9, 5, 6, 7], dtype=jnp.float32)

mean = jnp.mean(data)
variance = jnp.var(data)
std = jnp.std(data)
mad = jnp.mean(jnp.abs(data - mean))

print("Original data:")
print(f" Variance: {variance:.3f}, Std: {std:.3f}, MAD: {mad:.3f}")

Add an outlier and recompute
data_outlier = jnp.append(data, 100.0)
mean2 = jnp.mean(data_outlier)
print(f"\nWith outlier (100):")
print(f" Variance: {jnp.var(data_outlier):.3f}, Std: {jnp.std(data_outlier):.3f}, MAD: {jnp.mean(jnp.abs(data_outlier - mean2)):.3f}")
```

1. Compute Pearson and Spearman correlation between two variables. Experiment with different relationships.

```
import jax
import jax.numpy as jnp

Perfect linear relationship
x = jnp.array([1, 2, 3, 4, 5, 6, 7, 8], dtype=jnp.float32)
y = 2 * x + 1 # try changing this!

def pearson(a, b):
 a_c = a - jnp.mean(a)
 b_c = b - jnp.mean(b)
 return jnp.sum(a_c * b_c) / (jnp.sqrt(jnp.sum(a_c**2)) *
jnp.sqrt(jnp.sum(b_c**2)))

def spearman(a, b):
 rank_a = jnp.argsort(jnp.argsort(a)).astype(jnp.float32)
 rank_b = jnp.argsort(jnp.argsort(b)).astype(jnp.float32)
 return pearson(rank_a, rank_b)
```

```
print(f"Pearson r: {pearson(x, y):.4f}")
print(f"Spearman p: {spearman(x, y):.4f}")
```

1. Implement outlier detection using both the IQR and z-score methods, then compare their results on skewed data.

```
import jax.numpy as jnp

data = jnp.array([2, 3, 3, 4, 5, 5, 5, 6, 6, 7, 50], dtype=jnp.float32)

IQR method
q1, q3 = jnp.percentile(data, 25), jnp.percentile(data, 75)
iqr = q3 - q1
lower, upper = q1 - 1.5 * iqr, q3 + 1.5 * iqr
iqr_outliers = data[(data < lower) | (data > upper)]
print(f"IQR bounds: [{lower:.1f}, {upper:.1f}]")
print(f"IQR outliers: {iqr_outliers}")

Z-score method
z_scores = (data - jnp.mean(data)) / jnp.std(data)
z_outliers = data[jnp.abs(z_scores) > 3]
print(f"\nZ-scores: {z_scores}")
print(f"Z-score outliers (|z| > 3): {z_outliers}")
```

1. Compute and plot an Exponential Moving Average with different smoothing factors on noisy data.

```
import jax.numpy as jnp
import matplotlib.pyplot as plt

Generate noisy data
key = __import__("jax").random.PRNGKey(0)
noise = __import__("jax").random.normal(key, shape=(50,))
signal = jnp.linspace(0, 5, 50) + noise

def ema(data, alpha):
 result = jnp.zeros_like(data)
 result = result.at[0].set(data[0])
 for t in range(1, len(data)):
 result = result.at[t].set(alpha * data[t] + (1 - alpha) * result[t - 1])
 return result

plt.figure(figsize=(10, 4))
plt.plot(signal, "o", alpha=0.3, label="raw data", color="#999")
for alpha, color in [(0.1, "#e74c3c"), (0.3, "#3498db"), (0.7, "#27ae60")]:
```

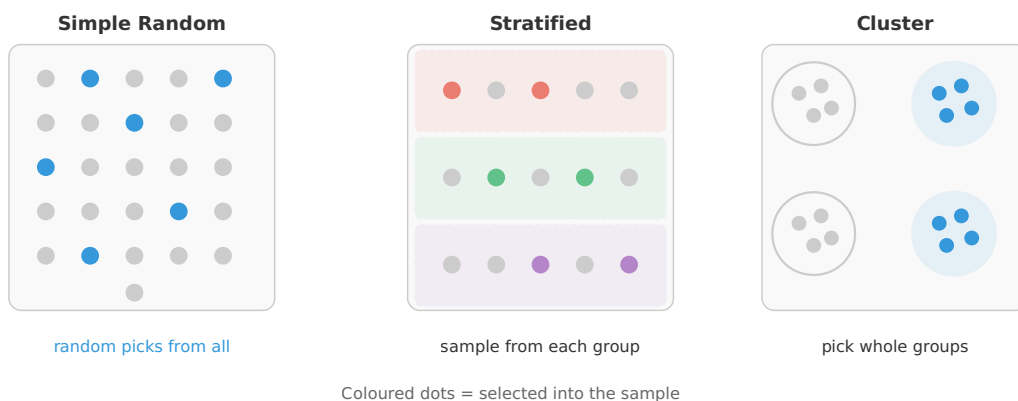
```
plt.plot(ema(signal, alpha), label=f" α ={alpha}", color=color, linewidth=2)
plt.legend()
plt.title("EMA with different smoothing factors")
plt.show()
```

## 18 Sampling

Sampling determines how we collect data and directly controls the quality of every conclusion we draw. This file covers random, stratified, cluster, and systematic sampling, sampling distributions, the law of large numbers, and bootstrapping, methods essential for training/test splits and dataset curation in ML.

- In an ideal world, you would measure every single member of the group you care about. In practice, that is almost never possible. You cannot survey every voter, test every light bulb, or scan every patient. So you take a **sample** and use it to learn about the whole.
- The **population** is the complete set of individuals or items you want to study. The **sample** is the subset you actually observe.
- A **parameter** is a number that describes the population (e.g. the true average height of all adults in a country).
- A **statistic** is a number computed from your sample (e.g. the average height of the 500 people you measured). Statistics are used to estimate parameters.
- The quality of your conclusions depends entirely on how you select your sample. A biased sample leads to biased conclusions, no matter how sophisticated your analysis.
- The **sampling frame** is the list of all individuals from which you actually draw your sample. Ideally this matches the population perfectly, but in practice there are gaps.
- For instance, if you survey people by phone, you miss everyone without a phone. The difference between the frame and the population is called **coverage error**.
- **Sampling error** is the natural discrepancy between a sample statistic and the population parameter.

- Even a perfectly random sample will not match the population exactly. Larger samples reduce sampling error.
- There are two broad families of sampling: probability and non-probability.
- **Probability sampling** means every member of the population has a known, nonzero chance of being selected. This lets you quantify uncertainty and generalise results.
- **Simple random sampling**: every individual has an equal chance of being selected, and every possible sample of size  $n$  is equally likely. Think of putting every name in a hat and drawing blindly.
- **Stratified sampling**: divide the population into non-overlapping groups (strata) based on a shared characteristic (e.g. age group, region), then randomly sample from each stratum. This guarantees representation from every group and reduces variance when strata differ from each other.
- **Cluster sampling**: divide the population into groups (clusters), randomly select some clusters, then include everyone in the chosen clusters. This is practical when the population is spread out geographically, like sampling entire schools rather than individual students across a district.
- **Systematic sampling**: pick a random starting point, then select every  $k$ -th individual from the list. For example, start at person 7 and then take every 10th person (7, 17, 27, ...). Simple to implement but can introduce bias if the list has a hidden pattern.

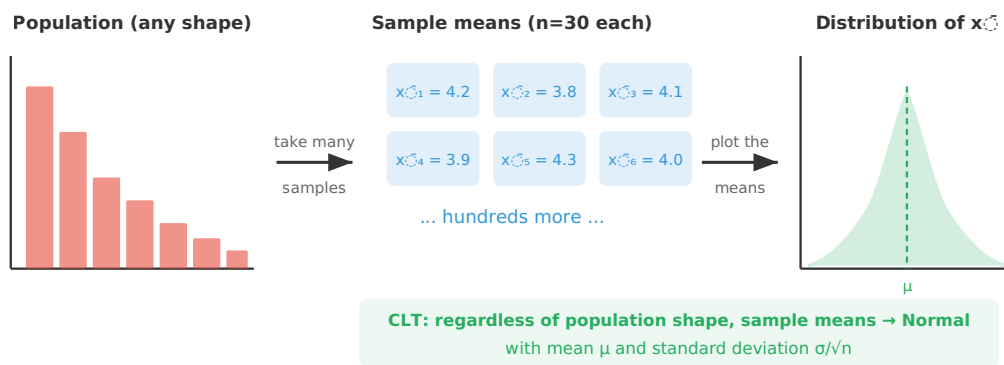


Three probability sampling methods side by side: simple random, stratified, and cluster

- **Non-probability sampling** does not give every member a known chance of selection. Results cannot be rigorously generalised, but these methods are often faster and cheaper.
- **Convenience sampling**: select whoever is easiest to reach. Surveying people at a shopping mall is convenient but misses those who do not shop there.
- **Quota sampling**: like stratified sampling, but without randomness. The researcher fills quotas (e.g. 50 men and 50 women) by picking accessible individuals from each group.
- **Snowball sampling**: start with a few participants and ask them to recruit others. Useful for hard-to-reach populations (e.g. studying rare diseases), but heavily biased toward connected individuals.
- Once you have a sampling method, a natural question arises: if I took a different sample, would I get a different statistic? Almost certainly yes. The **sampling distribution** is the distribution of a statistic (like the sample mean) across all possible samples of the same size.
- Imagine drawing 1,000 different samples of 30 people and computing the mean height of each. Those 1,000 means form a distribution. Some will be a bit above the true population mean, some a bit below, and most will cluster around the true value.
- The standard deviation of this sampling distribution is called the **standard error**:

$$SE = \frac{\sigma}{\sqrt{n}}$$

- Notice that the standard error shrinks as  $n$  grows. Larger samples give more precise estimates. Quadrupling the sample size halves the standard error.
- The most important result in statistics is the **Central Limit Theorem (CLT)**. It says: no matter what the shape of the original population, the distribution of sample means approaches a normal distribution as the sample size increases.



CLT: a skewed population produces normally distributed sample means

- More precisely, if  $X_1, X_2, \dots, X_n$  are independent observations from any distribution with mean  $\mu$  and finite variance  $\sigma^2$ , then as  $n$  grows:

—

X

$$\approx \text{Normal} \left( \mu, \frac{\sigma^2}{n} \right)$$

- The CLT is what makes most of inferential statistics work. It lets us use the normal distribution as an approximation even when the underlying data is not normal, as long as the sample is large enough.
- How large is “large enough”? A common rule of thumb is  $n \geq 30$ , but this depends on how non-normal the population is. For highly skewed distributions, you may need more. For roughly symmetric populations, even  $n = 10$  can be sufficient.
- The CLT has three key conditions:
  - **Independence:** each observation must not influence the others
  - **Finite variance:** the population variance must exist (rules out some exotic distributions)
  - **Identical distribution:** all observations come from the same distribution

## 18.1 Coding Tasks (use CoLab or notebook)

1. Demonstrate the CLT visually: draw samples from a highly skewed distribution, compute sample means, and watch the histogram of means become bell-shaped.



```

import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

key = jax.random.PRNGKey(0)

Exponential distribution (very skewed)
population = jax.random.exponential(key, shape=(100_000,))

fig, axes = plt.subplots(1, 4, figsize=(14, 3))
sample_sizes = [1, 5, 30, 100]

for ax, n in zip(axes, sample_sizes):
 keys = jax.random.split(key, 2000)
 means = jnp.array([jax.random.choice(k, population, shape=(n,)).mean() for k
in keys])
 ax.hist(means, bins=40, color="#3498db", alpha=0.7, density=True)
 ax.set_title(f"n = {n}")
 ax.set_xlim(0, 4)

fig.suptitle("CLT: sample means become normal as n increases", fontsize=13)
plt.tight_layout()
plt.show()

```

1. Compare simple random sampling with stratified sampling. Create a population with distinct groups and show that stratified sampling gives lower variance in estimates.

```

import jax
import jax.numpy as jnp

key = jax.random.PRNGKey(42)

Population: two distinct groups
group_a = jax.random.normal(key, shape=(500,)) + 10 # mean ~10
key, subkey = jax.random.split(key)
group_b = jax.random.normal(subkey, shape=(500,)) + 20 # mean ~20
population = jnp.concatenate([group_a, group_b])

Simple random sampling: 1000 trials, sample size 20
srs_means = []
for i in range(1000):
 key, subkey = jax.random.split(key)
 sample = jax.random.choice(subkey, population, shape=(20,), replace=False)
 srs_means.append(sample.mean())
srs_means = jnp.array(srs_means)

```

```

Stratified sampling: 10 from each group
strat_means = []
for i in range(1000):
 key, k1, k2 = jax.random.split(key, 3)
 s_a = jax.random.choice(k1, group_a, shape=(10,), replace=False)
 s_b = jax.random.choice(k2, group_b, shape=(10,), replace=False)
 strat_means.append(jnp.concatenate([s_a, s_b]).mean())
strat_means = jnp.array(strat_means)

print(f"Simple Random - Mean: {srs_means.mean():.3f}, Std: {srs_means.std():.3f}")
print(f"Stratified - Mean: {strat_means.mean():.3f}, Std: {strat_means.std():.3f}")
print(f"Stratified sampling reduced variance by {(1 - strat_means.var()/srs_means.var())*100:.1f}%")

```

1. Explore how sample size affects standard error. Plot the standard error against sample size and confirm the  $1/\sqrt{n}$

relationship.

```

import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

key = jax.random.PRNGKey(7)
population = jax.random.normal(key, shape=(50_000,)) * 10 + 50

sample_sizes = [5, 10, 20, 50, 100, 200, 500, 1000]
std_errors = []

for n in sample_sizes:
 means = []
 for _ in range(500):
 key, subkey = jax.random.split(key)
 sample = jax.random.choice(subkey, population, shape=(n,))
 means.append(sample.mean())
 std_errors.append(jnp.array(means).std())

plt.figure(figsize=(8, 4))
plt.plot(sample_sizes, std_errors, "o-", color="#e74c3c", label="Observed SE")
theoretical = population.std() / jnp.sqrt(jnp.array(sample_sizes, dtype=jnp.float32))
plt.plot(sample_sizes, theoretical, "--", color="#3498db", label="σ/√n (theoretical)")

```

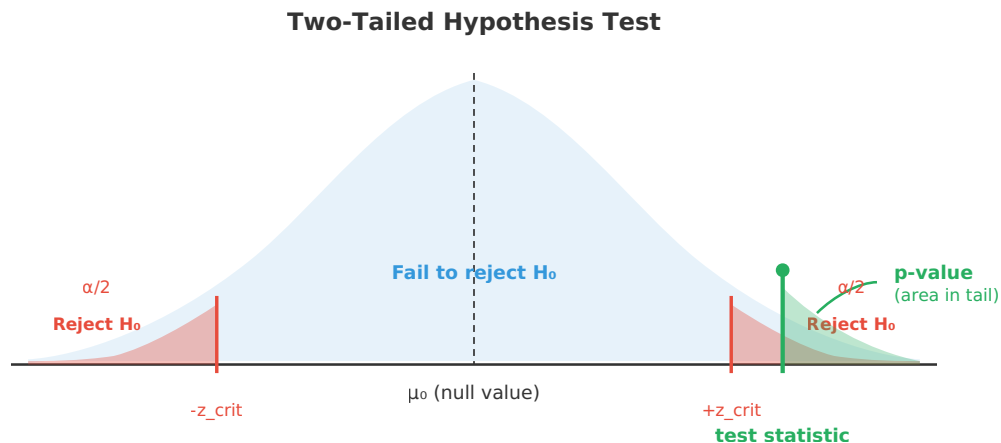
```
plt.xlabel("Sample size (n)")
plt.ylabel("Standard error")
plt.legend()
plt.title("Standard error shrinks with larger samples")
plt.show()
```

## 19 Hypothesis Testing

Hypothesis testing provides a rigorous framework for deciding whether observed effects are real or due to chance. This file covers null and alternative hypotheses, p-values, significance levels, t-tests, chi-squared tests, ANOVA, and Type I/II errors, the same logic used in A/B testing, model comparison and research.

- Statistics is not just about describing data. Often you need to make a decision: does a new drug work? Is one algorithm faster than another? Has the average changed? Hypothesis testing gives you a structured framework for answering these questions using data.
- The idea is simple: assume nothing has changed (the “null hypothesis” ), then check whether the data is so extreme that this assumption becomes hard to believe.
- The **null hypothesis** ( $H_0$ ) is the default claim, usually a statement of “no effect” or “no difference.” For example: “the average delivery time is still 30 minutes” or “the new model is no better than the old one.”
- The **alternative hypothesis** ( $H_1$  or  $H_a$ ) is what you suspect might be true instead: “the average delivery time has changed” or “the new model is better.”
- You never prove  $H_1$  directly. Instead, you ask: if  $H_0$  were true, how likely is it that I would see data this extreme? If it is very unlikely, you reject  $H_0$  in favour of  $H_1$ .
- The **test statistic** is a single number that summarises how far your sample result is from what  $H_0$  predicts. Different tests use different formulas, but the logic is always the same: measure the distance between observed and expected.

- The **p-value** is the probability of observing a test statistic at least as extreme as yours, assuming  $H_0$  is true. A small p-value means the data is surprising under  $H_0$ .
- The **significance level** ( $\alpha$ ) is the threshold you set before looking at the data. If  $p \leq \alpha$ , you reject  $H_0$ . Common choices are  $\alpha = 0.05$  (5%) and  $\alpha = 0.01$  (1%).



Normal curve with rejection regions shaded, test statistic marked, and p-value area highlighted

- The shaded tails are the rejection regions. If your test statistic lands there, the data is surprising enough under  $H_0$  that you reject it. The green area shows the p-value for a particular test statistic.
- Here is the step-by-step procedure:
  - **Step 1:** State  $H_0$  and  $H_1$
  - **Step 2:** Choose a significance level  $\alpha$
  - **Step 3:** Collect data and compute the test statistic
  - **Step 4:** Find the p-value (or compare the test statistic to a critical value)
  - **Step 5:** If  $p \leq \alpha$ , reject  $H_0$ . Otherwise, fail to reject  $H_0$
- **Worked example:** A factory claims their bolts have a mean length of 10 cm. You measure 36 bolts and find a sample mean of 10.3 cm. The known population standard deviation is 0.9 cm. Is there evidence that the mean has changed?
- $H_0: \mu = 10$ ,  $H_1: \mu \neq 10$ ,  $\alpha = 0.05$
- Test statistic (z-test, since  $\sigma$  is known and  $n$  is large):

$$z = \frac{\bar{x} - \mu_0}{\frac{\sigma}{\sqrt{n}}} = \frac{10.3 - 10}{\frac{0.9}{\sqrt{36}}} = \frac{0.3}{0.15} = 2.0$$

- For a two-tailed test at  $\alpha = 0.05$ , the critical values are  $\pm 1.96$ . Our  $z = 2.0 > 1.96$ , so we reject  $H_0$ . The p-value is approximately 0.046, which is less than 0.05.
- Conclusion: there is statistically significant evidence that the mean bolt length differs from 10 cm.
- A **one-tailed test** checks for an effect in one specific direction ( $H_1: \mu > 10$  or  $\mu < 10$ ). The entire  $\alpha$  goes into one tail, making it easier to reject  $H_0$  in that direction but impossible to detect an effect in the opposite direction.
- A **two-tailed test** checks for any difference ( $H_1: \mu \neq 10$ ). The  $\alpha$  is split between both tails ( $\alpha/2$  each). This is more conservative but catches effects in either direction.
- Even with a good procedure, mistakes happen. There are exactly two types of errors:

### Decision vs Reality

	$H_0$ is actually TRUE	$H_0$ is actually FALSE
Reject $H_0$	<b>Type I Error</b> (false positive) prob = $\alpha$	<b>Correct!</b> (true positive) prob = $1 - \beta$
Fail to reject $H_0$	<b>Correct!</b> (true negative) prob = $1 - \alpha$	<b>Type II Error</b> (false negative) prob = $\beta$

2x2 grid showing Type I and Type II errors: reality vs decision

- **Type I Error** (false positive): you reject  $H_0$  when it is actually true. The probability of this is  $\alpha$ , which you control by choosing your significance level. Like a fire alarm going off when there is no fire.

- **Type II Error** (false negative): you fail to reject  $H_0$  when it is actually false. The probability of this is  $\beta$ . Like a fire alarm staying silent during a real fire.
- **Power** is  $1 - \beta$ , the probability of correctly rejecting a false  $H_0$ . Higher power means you are better at detecting real effects. Power increases when:
  - The true effect size is larger (bigger differences are easier to detect)
  - The sample size is larger (more data = more precision)
  - The significance level  $\alpha$  is larger (but this raises Type I error risk)
  - The variability is lower (less noise)
- There is a tension between Type I and Type II errors. Lowering  $\alpha$  (being more cautious about false positives) increases  $\beta$  (more false negatives). You cannot minimise both simultaneously with a fixed sample size.
- **Parametric tests** assume the data follows a specific distribution (usually normal). They are more powerful when the assumptions hold.
- **Z-test**: compares a sample mean to a known value when  $\sigma$  is known and  $n$  is large ( $n \geq 30$ ). Test statistic:

$$z = \frac{\bar{x} - \mu_0}{\frac{\sigma}{\sqrt{n}}}$$

- **T-test**: like the z-test, but for when  $\sigma$  is unknown (estimated from the sample) or  $n$  is small. Uses the t-distribution, which has heavier tails than the normal. The heavier tails account for the extra uncertainty from estimating  $\sigma$ .

$$t = \frac{\bar{x} - \mu_0}{\frac{s}{\sqrt{n}}}$$

- The t-distribution has a parameter called **degrees of freedom** ( $df = n - 1$ ). As  $df$  increases, the t-distribution approaches the normal distribution.

- There are several flavours of t-test:
  - **One-sample t-test:** is the sample mean different from a specific value?
  - **Independent two-sample t-test:** are the means of two separate groups different?
  - **Paired t-test:** are the means of two related measurements different (e.g. before and after treatment on the same subjects)?
- **ANOVA (Analysis of Variance):** tests whether three or more group means are equal. Instead of running multiple t-tests (which inflates the Type I error rate), ANOVA does a single test by comparing the variance between groups to the variance within groups.

$$F = \frac{\text{variance between groups}}{\text{variance within groups}}$$

- A large F ratio means the groups differ more than you would expect from random variation alone.
- **Non-parametric tests** make fewer assumptions about the data distribution. They work on ranks rather than raw values, making them robust to outliers and non-normality.
- **Chi-square test ( $\chi^2$ ):** tests whether observed frequencies match expected frequencies. Used for categorical data. For example: do the proportions of red, blue, and green cars match the manufacturer's claimed proportions?

$$\chi^2 = \sum \frac{(O_i - E_i)^2}{E_i}$$

- **Mann-Whitney U test:** the non-parametric alternative to the independent two-sample t-test. It tests whether one group tends to have larger values than the other by comparing ranks.
- **Wilcoxon signed-rank test:** the non-parametric alternative to the paired t-test. Compares paired observations by looking at the magnitude and direction of differences.
- **Kruskal-Wallis test:** the non-parametric alternative to one-way ANOVA. Tests whether multiple groups come from the same distribution by comparing ranks across all groups.

- **Goodness-of-fit tests** check whether your data follows a specific theoretical distribution. The chi-square goodness-of-fit test compares observed bin counts to expected counts under the hypothesised distribution.
- **Normality tests** specifically check whether data is normally distributed. Common ones include the Shapiro-Wilk test (powerful for small samples) and the Kolmogorov-Smirnov test (compares the sample CDF to the theoretical CDF).
- In ML, hypothesis testing appears when you compare model performance. If model A achieves 92% accuracy and model B achieves 91%, is the difference real or just noise? A paired t-test on cross-validation scores can answer this.

## 19.1 Coding Tasks (use CoLab or notebook)

1. Perform a z-test for the bolt factory example from the text. Compute the test statistic, p-value, and make a decision.

```
import jax.numpy as jnp

x_bar = 10.3 # sample mean
mu_0 = 10.0 # null hypothesis value
sigma = 0.9 # known population std
n = 36 # sample size
alpha = 0.05

Test statistic
z = (x_bar - mu_0) / (sigma / jnp.sqrt(n))
print(f"z = {z:.4f}")

p-value (two-tailed) using the normal CDF approximation
For |z| = 2.0, p ≈ 0.0456
from jax.scipy.stats import norm
p_value = 2 * (1 - norm.cdf(jnp.abs(z)))
print(f"p-value = {p_value:.4f}")
print(f"Reject H₀? {p_value <= alpha}")
```

1. Simulate Type I error: when  $H_0$  is true, how often do we mistakenly reject it?  
Run 10,000 experiments and check that the rejection rate matches  $\alpha$ .

```
import jax
import jax.numpy as jnp
```



```

key = jax.random.PRNGKey(0)
mu_0 = 50.0
sigma = 10.0
n = 30
alpha = 0.05
n_experiments = 10_000

rejections = 0
for i in range(n_experiments):
 key, subkey = jax.random.split(key)
 sample = mu_0 + sigma * jax.random.normal(subkey, shape=(n,))
 z = (sample.mean() - mu_0) / (sigma / jnp.sqrt(n))
 p_value = 2 * (1 - __import__("jax").scipy.stats.norm.cdf(jnp.abs(z)))
 if p_value <= alpha:
 rejections += 1

print(f"Rejection rate: {rejections/n_experiments:.4f}")
print(f"Expected (α): {alpha}")

```

1. Compare a t-test and a Mann-Whitney U test on two groups. Generate data where one group has a slightly higher mean and see which test detects the difference.

```

import jax
import jax.numpy as jnp

key = jax.random.PRNGKey(99)
k1, k2 = jax.random.split(key)

group_a = jax.random.normal(k1, shape=(25,)) * 5 + 100
group_b = jax.random.normal(k2, shape=(25,)) * 5 + 103 # slightly higher mean

Two-sample t-test (equal variance assumed)
n_a, n_b = len(group_a), len(group_b)
mean_a, mean_b = group_a.mean(), group_b.mean()
pooled_var = ((n_a - 1) * group_a.var() + (n_b - 1) * group_b.var()) / (n_a +
n_b - 2)
se = jnp.sqrt(pooled_var * (1/n_a + 1/n_b))
t_stat = (mean_a - mean_b) / se
print(f"T-test statistic: {t_stat:.4f}")

Mann-Whitney: count how often group_a values beat group_b values
u_stat = jnp.sum(group_a[:, None] < group_b[None, :])
print(f"Mann-Whitney U: {u_stat}")
print(f"\nGroup A mean: {mean_a:.2f}, Group B mean: {mean_b:.2f}")

```

## 20 Statistical Inference

Statistical inference goes beyond yes/no decisions to estimate population parameters with quantified uncertainty. This file covers confidence intervals, point and interval estimation, maximum likelihood estimation, the method of moments, and regression analysis, the bridge between raw data and predictive models in ML.

- Hypothesis testing gives you a yes/no decision: reject or fail to reject. But often you want something more informative, a range of plausible values for the parameter you are estimating. That is what **confidence intervals** provide.
- A **point estimate** is a single number computed from your sample, like the sample mean

$\bar{x}$

It is your best guess for the population parameter, but on its own it gives no sense of how precise the estimate is.

- A **confidence interval** wraps that point estimate with a range that reflects uncertainty. It takes the form:

CI =

$\bar{x}$

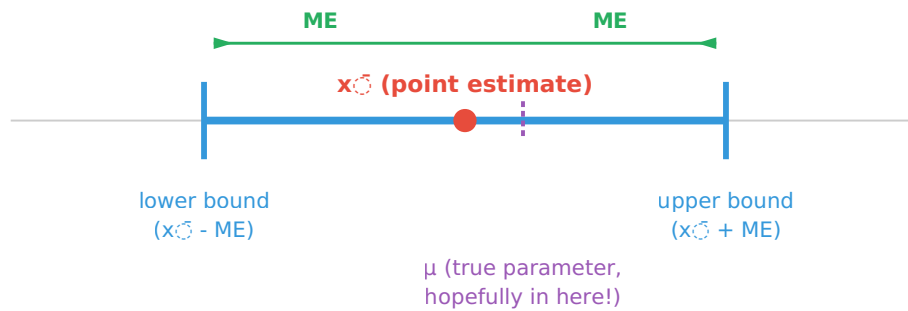
$\pm$  ME

- The **margin of error (ME)** depends on three things: how confident you want to be, how much variability is in the data, and how large your sample is:

$$ME = z^* \cdot \frac{\sigma}{\sqrt{n}}$$

- Here  $z^*$  is the critical value from the normal distribution that matches your desired confidence level. For 95% confidence,  $z^* = 1.96$ . For 99% confidence,  $z^* = 2.576$ .

## Confidence Interval = Point Estimate $\pm$ Margin of Error



Confidence interval: point estimate with margin of error on either side

- A **95% confidence interval** means: if you repeated the experiment many times and built an interval each time, about 95% of those intervals would contain the true population parameter. It does not mean there is a 95% probability the parameter is in this specific interval. The parameter is fixed; the intervals are what vary.

- **Worked example:** You measure the heights of 50 people and find

–

$\bar{x}$

= 170 cm with  $\sigma = 8$  cm. Construct a 95% confidence interval.

$$ME = 1.96 \cdot \frac{8}{\sqrt{50}} = 1.96 \cdot 1.131 = 2.22 \text{ cm}$$

$$CI = [170 - 2.22, 170 + 2.22] = [167.78, 172.22]$$

- You can say with 95% confidence that the true mean height lies between 167.78 and 172.22 cm.
- When  $\sigma$  is unknown (the usual case), use the sample standard deviation  $s$  and the t-distribution instead:

$$CI =$$

–

$\bar{x}$

$$\pm t_{n-1}^* \cdot \frac{s}{\sqrt{n}}$$

- Wider intervals are more confident but less precise. Narrower intervals are more precise but less confident. You can narrow an interval without losing confidence by increasing the sample size.

- **Power analysis** helps you plan an experiment before you run it. The question is: how large a sample do I need to detect an effect of a given size with a specified power?
- Recall from the previous file that power =  $1 - \beta$ , the probability of correctly rejecting a false  $H_0$ . A common target is 80% power.
- The required sample size for a z-test detecting a difference  $\delta$  with significance  $\alpha$  and power  $1 - \beta$  is:

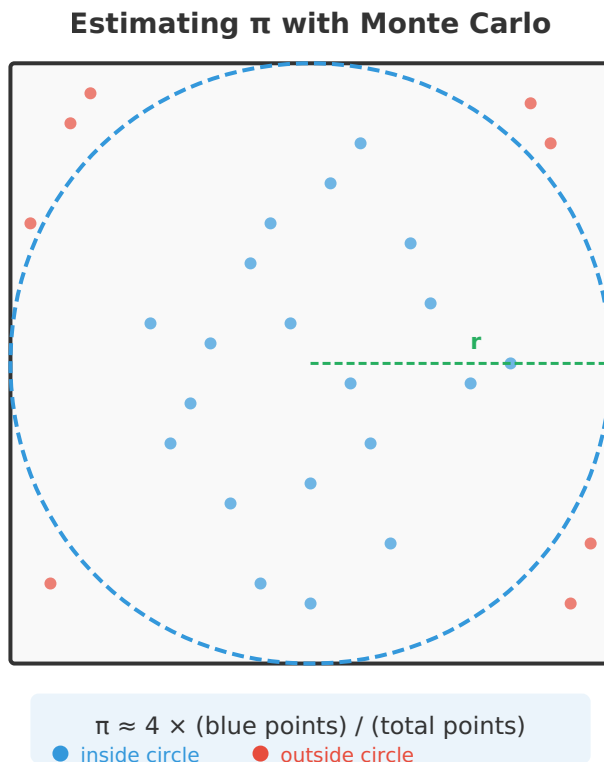
$$n = \left( \frac{(z_{\alpha/2} + z_{\beta}) \cdot \sigma}{\delta} \right)^2$$

- For example, to detect a 2 cm difference in mean height ( $\sigma = 8$ ) with  $\alpha = 0.05$  and 80% power ( $z_{0.025} = 1.96$ ,  $z_{0.20} = 0.84$ ):

$$n = \left( \frac{(1.96 + 0.84) \cdot 8}{2} \right)^2 = \left( \frac{22.4}{2} \right)^2 = 11.2^2 \approx 126$$

- You would need about 126 people per group.
- Power analysis prevents two common mistakes: running an experiment too small to detect a real effect (underpowered), or wasting resources on an experiment far larger than necessary (overpowered).
- **Monte Carlo methods** use random sampling to solve problems that are difficult or impossible to solve analytically. The core idea: if you cannot compute something exactly, simulate it many times and use the results as an approximation.
- The name comes from the Monte Carlo casino, a nod to the role of randomness. These methods are workhorses in ML for tasks like estimating integrals, evaluating model uncertainty, and approximating complex distributions.
- The general Monte Carlo recipe:
  - Define a domain of possible inputs
  - Generate random inputs from that domain
  - Evaluate a function on each input
  - Aggregate the results (average, count, etc.)

- A classic example is estimating  $\pi$ . Imagine a square with side length 2, centred at the origin, with a circle of radius 1 inscribed inside it. The area of the square is 4, and the area of the circle is  $\pi$ .



Square with inscribed circle, random points coloured by inside/outside

- Drop random points uniformly in the square. The fraction that land inside the circle approximates  $\pi/4$ :

$$\pi \approx 4 \times \frac{\text{points inside circle}}{\text{total points}}$$

- A point  $(x, y)$  is inside the circle if  $x^2 + y^2 \leq 1$ . The more points you throw, the closer your estimate gets to the true value of  $\pi$ .
- In ML, Monte Carlo methods appear in:
  - **Monte Carlo dropout:** run inference multiple times with dropout enabled to estimate prediction uncertainty
  - **MCMC (Markov Chain Monte Carlo):** sample from complex posterior distributions in Bayesian models
  - **Policy gradient methods:** estimate gradients in reinforcement learning by sampling trajectories

- **Factor analysis** is a technique for discovering hidden (latent) variables that explain the correlations among observed variables. If 10 personality survey questions can be explained by 3 underlying traits (extraversion, agreeableness, conscientiousness), factor analysis finds those traits.
- The model assumes each observed variable  $x_i$  is a linear combination of a few latent factors  $f_j$  plus noise:

$$x_i = \lambda_{i1}f_1 + \lambda_{i2}f_2 + \dots + \lambda_{ik}f_k + \epsilon_i$$

- The  $\lambda$  values are called **factor loadings** and tell you how strongly each observed variable relates to each factor. This connects directly to the matrix decompositions from Chapter 2; factor analysis is closely related to eigenvalue decomposition and SVD.
- **Experimental design** is the art of structuring an experiment so that you can draw valid conclusions. Poor design can make even a large dataset useless.
- Key components of a well-designed experiment:
  - **Independent variable (IV)**: what you manipulate (e.g. drug dose, model architecture)
  - **Dependent variable (DV)**: what you measure (e.g. recovery time, accuracy)
  - **Control group**: receives no treatment (or a placebo), providing a baseline for comparison
  - **Random assignment**: participants are assigned to groups randomly, which balances out confounding variables you did not measure
- **Common experimental designs**:
  - **Completely randomised design**: subjects are randomly assigned to treatment groups. Simple and effective when groups are comparable.
  - **Randomised block design**: subjects are first grouped into blocks (e.g. by age), then randomly assigned to treatments within each block. This reduces variability from the blocking factor, similar in spirit to stratified sampling.
  - **Factorial design**: tests multiple IVs simultaneously. A  $2 \times 3$  factorial design has 2 levels of one variable and 3 of another, giving 6 treatment combinations. This lets you detect **interactions**, where the effect of one variable depends on the level of another.

- **Crossover design:** each subject receives all treatments in sequence (with washout periods in between). Every subject serves as their own control, reducing the effect of individual differences.
- In ML experiments, these principles are critical. When comparing models, you should control for random seed, dataset split, and hardware. Cross-validation is a form of crossover design. Ablation studies, where you remove one component at a time, follow the logic of factorial designs.

## 20.1 Coding Tasks (use CoLab or notebook)

1. Construct a 95% confidence interval for the height example, then experiment with different confidence levels and sample sizes.

```
import jax.numpy as jnp

x_bar = 170.0 # sample mean
sigma = 8.0 # population std (known)
n = 50 # sample size

Critical values for common confidence levels
z_stars = {0.90: 1.645, 0.95: 1.960, 0.99: 2.576}

for conf, z_star in z_stars.items():
 me = z_star * (sigma / jnp.sqrt(n))
 lower, upper = x_bar - me, x_bar + me
 print(f"{conf*100:.0f}% CI: [{lower:.2f}, {upper:.2f}] (ME = {me:.2f})")
```

1. Estimate  $\pi$  using Monte Carlo simulation. Plot how the estimate converges as you increase the number of points.

```
import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

key = jax.random.PRNGKey(42)

Generate random points in [-1, 1] x [-1, 1]
n_points = 100_000
k1, k2 = jax.random.split(key)
x = jax.random.uniform(k1, shape=(n_points,), minval=-1, maxval=1)
y = jax.random.uniform(k2, shape=(n_points,), minval=-1, maxval=1)
```

```

Check which points are inside the unit circle
inside = (x**2 + y**2) <= 1.0
cumulative_inside = jnp.cumsum(inside)
counts = jnp.arange(1, n_points + 1)
pi_estimates = 4.0 * cumulative_inside / counts

plt.figure(figsize=(10, 4))
plt.plot(pi_estimates, color="#3498db", alpha=0.7, linewidth=0.5)
plt.axhline(y=jnp.pi, color="#e74c3c", linestyle="--", label=f" $\pi = \{{jnp.pi:.6f}\}$ ")
plt.xlabel("Number of points")
plt.ylabel("Estimate of π ")
plt.title("Monte Carlo estimation of π ")
plt.legend()
plt.ylim(2.8, 3.5)
plt.show()

print(f"Final estimate: {pi_estimates[-1]:.6f}")
print(f"True value: {jnp.pi:.6f}")
print(f"Error: {abs(pi_estimates[-1] - jnp.pi):.6f}")

```

1. Perform a simple power analysis: for a given effect size and standard deviation, compute the required sample size and verify it by simulation.

```

import jax
import jax.numpy as jnp

Parameters
delta = 2.0 # effect size (difference in means)
sigma = 8.0 # population std
alpha = 0.05
power_target = 0.80

Analytical sample size
z_alpha = 1.96 # two-tailed, alpha=0.05
z_beta = 0.84 # power=0.80
n_required = ((z_alpha + z_beta) * sigma / delta) ** 2
print(f"Required n per group: {n_required:.0f}")

Verify by simulation
key = jax.random.PRNGKey(7)
n = int(jnp.ceil(n_required))
n_sims = 5000
rejections = 0

for _ in range(n_sims):
 key, k1, k2 = jax.random.split(key, 3)

```



```

group_a = jax.random.normal(k1, shape=(n,)) * sigma + 50
group_b = jax.random.normal(k2, shape=(n,)) * sigma + 50 + delta
pooled_se = jnp.sqrt(2 * sigma**2 / n)
z = (group_b.mean() - group_a.mean()) / pooled_se
p = 2 * (1 - __import__("jax").scipy.stats.norm.cdf(jnp.abs(z)))
if p <= alpha:
 rejections += 1

print(f"Simulated power: {rejections/n_sims:.3f}")
print(f"Target power: {power_target:.3f}")

```

1. Visualise how confidence interval width changes with sample size. This shows why collecting more data gives more precise estimates.

```

import jax.numpy as jnp
import matplotlib.pyplot as plt

sigma = 8.0
z_star = 1.96 # 95% confidence

sample_sizes = jnp.array([10, 20, 30, 50, 100, 200, 500, 1000],
dtype=jnp.float32)
margins = z_star * sigma / jnp.sqrt(sample_sizes)

plt.figure(figsize=(8, 4))
plt.bar([str(int(n)) for n in sample_sizes], margins, color="#3498db", alpha=0.7)
plt.xlabel("Sample size")
plt.ylabel("Margin of error (cm)")
plt.title("95% CI margin of error shrinks with larger samples")
plt.show()

```

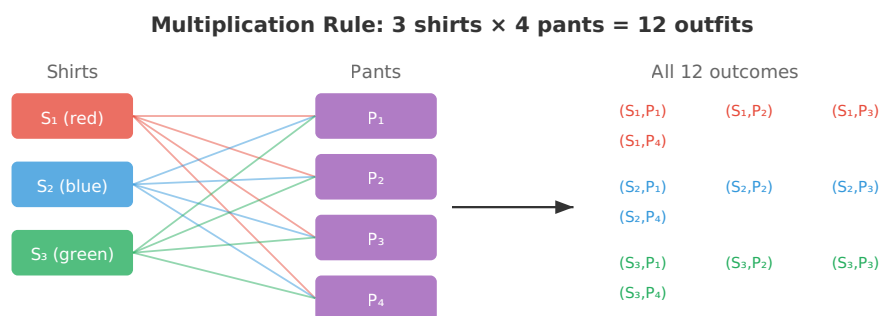
## 21 Counting

Counting is the prerequisite for computing probabilities, you must know how many outcomes exist before you can assign likelihoods. This file covers the multiplication and addition rules, factorials, permutations, combinations, and the binomial coefficient, combinatorial tools that underpin sampling, hashing, and probabilistic analysis in ML.

- Before we can compute probabilities, we need to count outcomes. If you want to know the chance of drawing a winning hand in poker, you first need to know

how many possible hands exist and how many of those are winners. Counting is the machinery that makes probability precise.

- The simplest counting principle is the **multiplication rule**. If one decision has  $m$  options and a second independent decision has  $n$  options, the total number of combined outcomes is  $m \times n$ .
- Picture getting dressed in the morning. You have 3 shirts and 4 pants. Each shirt can pair with every pant, giving you  $3 \times 4 = 12$  possible outfits.



Each shirt pairs with every pant:  $3 \times 4 = 12$  total combinations  
 This is the multiplication rule: if choice A has  $m$  options and choice B has  $n$  options,  
 the total number of combined outcomes is  $m \times n$ .

Tree diagram showing 3 shirts times 4 pants equals 12 outfits

- The multiplication rule extends to any number of choices. If you also have 2 pairs of shoes, the total outfits become  $3 \times 4 \times 2 = 24$ . Each new independent choice multiplies the count.
- The **addition rule** handles “or” scenarios. If event A can happen in  $m$  ways and event B can happen in  $n$  ways, and they cannot both happen at the same time (mutually exclusive), the total number of ways is  $m + n$ .
- Suppose you can travel from city X to city Y by car (3 routes) or by train (2 routes). You cannot take both simultaneously, so the total options are  $3 + 2 = 5$ .
- When events overlap, you need to subtract the double-counted outcomes. If A and B can co-occur, the count is  $|A \cup B| = |A| + |B| - |A \cap B|$ . This is the inclusion-exclusion principle, and it will reappear when we discuss probability addition rules.
- The **factorial** of a non-negative integer  $n$  is the product of all positive integers up to  $n$ :

$$n! = n \times (n-1) \times (n-2) \times \cdots \times 2 \times 1$$

- Think of the factorial as answering: in how many ways can you arrange  $n$  distinct objects in a line? Three books on a shelf can be arranged in  $3! = 3 \times 2 \times 1 = 6$  ways. By convention,  $0! = 1$ .
- Factorials grow extremely fast.  $10! = 3,628,800$  and  $20!$  is already over  $2.4 \times 10^{18}$ . This explosive growth is why brute-force search becomes impractical in combinatorial problems.

- A **permutation** is an ordered arrangement of objects. When you pick  $r$  items from  $n$  distinct objects and the order matters, the number of permutations is:

$$P(n, r) = \frac{n!}{(n-r)!}$$

- Imagine picking a president, vice president, and treasurer from a club of 10 people. The first role has 10 candidates, the second has 9 remaining, the third has 8. That gives  $P(10, 3) = 10 \times 9 \times 8 = 720$ . The formula confirms this:  $\frac{10!}{7!} = 720$ .

- A **combination** is an unordered selection. When you pick  $r$  items from  $n$  and the order does not matter, we divide out the redundant orderings:

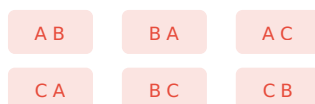
$$C(n, r) = \binom{n}{r} = \frac{n!}{r!(n-r)!}$$

- The notation  $\binom{n}{r}$  is read “ $n$  choose  $r$ .” The key insight is that every combination corresponds to  $r!$  permutations (the  $r$  chosen items can be rearranged in  $r!$  ways), so we divide the permutation count by  $r!$ .

### Permutations vs Combinations

#### Permutations (order matters)

Choosing 2 from {A, B, C}



6 arrangements

$$P(3,2) = 3!/(3-2)! = 6$$

Who is 1st vs 2nd matters  
(president vs VP)

#### Combinations (order ignored)

Choosing 2 from {A, B, C}



3 groups

$$C(3,2) = 3!/(2! \cdot 1!) = 3$$

A B and B A are the same group {A,B}  
so we divide by  $2! = 2$  orderings

Only membership matters  
(picking a committee)

Side-by-side comparison: permutations count all orderings, combinations collapse identical groups

- Example: from a group of 10 people, how many ways can you form a committee of 3? Order does not matter (there is no president or vice president, just members), so we use combinations:

$$\binom{10}{3} = \frac{10!}{3! \cdot 7!} = \frac{10 \times 9 \times 8}{3 \times 2 \times 1} = 120$$

- The same 10 people produce 720 permutations but only 120 combinations, because each group of 3 has  $3! = 6$  internal orderings.
- Combinations are central to probability. The binomial coefficient  $\binom{n}{r}$  counts the number of ways to get exactly  $r$  successes in  $n$  trials, which is the heart of the binomial distribution (covered in file 03).
- Let us work through a classic committee problem that combines multiple counting ideas.
- **Problem:** A club has 8 men and 6 women. How many ways can you form a committee of 5 that includes exactly 3 men and 2 women?
- **Step 1:** Choose 3 men from 8.

$$\binom{8}{3} = \frac{8!}{3! \cdot 5!} = \frac{8 \times 7 \times 6}{3 \times 2 \times 1} = 56$$

- **Step 2:** Choose 2 women from 6.

$$\binom{6}{2} = \frac{6!}{2! \cdot 4!} = \frac{6 \times 5}{2 \times 1} = 15$$

- **Step 3:** Apply the multiplication rule. Each selection of men can pair with each selection of women:

$$56 \times 15 = 840 \text{ committees}$$

- This pattern, breaking a complex counting problem into independent sub-choices and multiplying, is the standard approach in combinatorics.
- There are also **permutations with repetition**. When items can repeat, choosing  $r$  items from  $n$  types gives  $n^r$  outcomes. A 4-digit PIN using digits 0-9 has

$10^4 = 10,000$  possibilities. Each position has 10 options, and the multiplication rule handles the rest.

- **Combinations with repetition** (also called “stars and bars”) count how many ways to choose  $r$  items from  $n$  types when repeats are allowed and order does not matter:

$$\binom{n+r-1}{r} = \frac{(n+r-1)!}{r!(n-1)!}$$

- Example: choosing 3 scoops from 4 ice cream flavours (repeats allowed) gives

$$\binom{4+3-1}{3} = \binom{6}{3} = 20 \text{ options.}$$

- To summarise the counting toolkit:

Scenario	Formula
Ordered, no repetition (permutation)	$P(n, r) = \frac{n!}{(n-r)!}$
Unordered, no repetition (combination)	$\binom{n}{r} = \frac{n!}{r!(n-r)!}$
Ordered, with repetition	$n^r$
Unordered, with repetition	$\binom{n+r-1}{r}$

- Every probability calculation involving equally likely outcomes uses the formula  $P(\text{event}) = \frac{\text{favourable outcomes}}{\text{total outcomes}}$ . Counting gives us both numbers. With this foundation, we are ready to formalise probability itself in the next file.

## 21.1 Coding Tasks (use CoLab or notebook)

1. Compute  $P(10, 3)$  and  $\binom{10}{3}$  using both the factorial formula and direct computation. Verify that the permutation count is always  $r!$  times the combination count.

```

import jax.numpy as jnp
from math import factorial

n, r = 10, 3

perm = factorial(n) // factorial(n - r)
comb = factorial(n) // (factorial(r) * factorial(n - r))

print(f"P({n},{r}) = {perm}")
print(f"C({n},{r}) = {comb}")
print(f"P / C = {perm // comb} (should equal {r}! = {factorial(r)})")

```

1. Solve the committee problem (3 men from 8, 2 women from 6) programmatically and verify by enumerating all valid committees.

```

from itertools import combinations
from math import factorial

def comb_count(n, r):
 return factorial(n) // (factorial(r) * factorial(n - r))

Formula approach
men_ways = comb_count(8, 3)
women_ways = comb_count(6, 2)
print(f"Formula: {men_ways} × {women_ways} = {men_ways * women_ways}")

Enumeration approach
men = [f"M{i}" for i in range(1, 9)]
women = [f"W{i}" for i in range(1, 7)]
count = sum(1 for _ in combinations(men, 3) for _ in combinations(women, 2))
print(f"Enumeration: {count}")

```

1. Count how many 4-character passwords can be made from 26 lowercase letters (with repetition allowed). Then count how many contain no repeated letters.

```

from math import factorial

n = 26
r = 4

with_rep = n ** r
without_rep = factorial(n) // factorial(n - r)

print(f"With repetition: {with_rep:>10}")

```

```
print(f"Without repetition: {without_rep:>10,}")
print(f"Fraction with repeats: {1 - without_rep/with_rep:.2%}")
```

1. Simulate the birthday problem: in a group of  $k$  people, what is the probability that at least two share a birthday? Plot the probability for  $k = 1$  to 60 and find where it crosses 50%.

```
import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

def birthday_prob_exact(k):
 """Probability of at least one shared birthday in group of k."""
 p_no_match = 1.0
 for i in range(k):
 p_no_match *= (365 - i) / 365
 return 1 - p_no_match

ks = list(range(1, 61))
probs = [birthday_prob_exact(k) for k in ks]

plt.figure(figsize=(8, 4))
plt.plot(ks, probs, color="#3498db", linewidth=2)
plt.axhline(y=0.5, color="#e74c3c", linestyle="--", alpha=0.7, label="50%")
cross = next(k for k, p in zip(ks, probs) if p >= 0.5)
plt.axvline(x=cross, color="#e74c3c", linestyle="--", alpha=0.7)
plt.xlabel("Group size (k)")
plt.ylabel("P(at least one shared birthday)")
plt.title(f"Birthday Problem (crosses 50% at k={cross})")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

## 22 Probability Concepts

Probability theory formalises uncertainty and provides the rules for reasoning under it. This file covers sample spaces, events, axioms of probability, conditional probability, independence, Bayes' theorem, and the frequentist vs. Bayesian

interpretations, the mathematical framework behind every generative and discriminative model in ML.

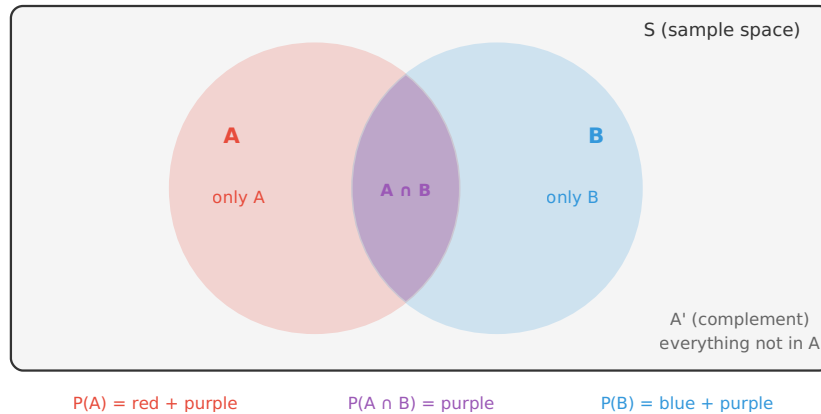
- Probability assigns a number between 0 and 1 to an event, measuring how likely it is to happen.
- A probability of 0 means impossible, 1 means certain, and 0.5 means a coin toss.
- There are two main interpretations. The **frequentist** view says probability is the long-run relative frequency: flip a fair coin 10,000 times and heads will appear roughly 50% of the time.
- The **Bayesian** view says probability is a degree of belief: you might say there is a 70% chance it rains tomorrow, even though tomorrow only happens once.
- Both interpretations use the same mathematical rules. The difference is philosophical, but it matters in ML. Frequentist methods give you point estimates. Bayesian methods give you full distributions over parameters.
- The **sample space**  $S$  is the set of all possible outcomes of an experiment. Flip a coin:  $S = \{H, T\}$ . Roll a die:  $S = \{1, 2, 3, 4, 5, 6\}$ .
- An **event** is any subset of the sample space. “Rolling an even number” is the event  $A = \{2, 4, 6\}$ , which is a subset of  $S$ .
- The probability of an event when all outcomes are equally likely is simply counting (from file 01):

$$P(A) = \frac{|A|}{|S|} = \frac{\text{favourable outcomes}}{\text{total outcomes}}$$

- For the even-number example:  $P(\text{even}) = \frac{3}{6} = 0.5$ .



### Sample Space, Events, and Set Operations



Venn diagram showing events A and B within sample space S, with intersection and complement

- The **complement** of event A, written  $A'$  or  $A^c$ , is everything in S that is not in A. Since every outcome is either in A or not:

$$P(A') = 1 - P(A)$$

- Complements are often the easier route. Instead of counting all the ways to get at least one head in 5 coin flips, count the one way to get no heads and subtract:  
 $P(\text{at least one head}) = 1 - P(\text{all tails}) = 1 - (0.5)^5 = 0.969$ .

- Two events are **mutually exclusive** (disjoint) if they cannot both happen:  
 $A \cap B = \emptyset$ . Rolling a 2 and rolling a 5 on a single die are mutually exclusive.

- The **addition rule for mutually exclusive events** is straightforward:

$$P(A \cup B) = P(A) + P(B) \quad (\text{if } A \cap B = \emptyset)$$

- When events can overlap, you need the **general addition rule** to avoid double-counting the intersection:

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

- This mirrors the inclusion-exclusion principle from counting. The Venn diagram above shows why: the purple region (intersection) gets counted once in  $P(A)$  and again in  $P(B)$ , so we subtract it once.

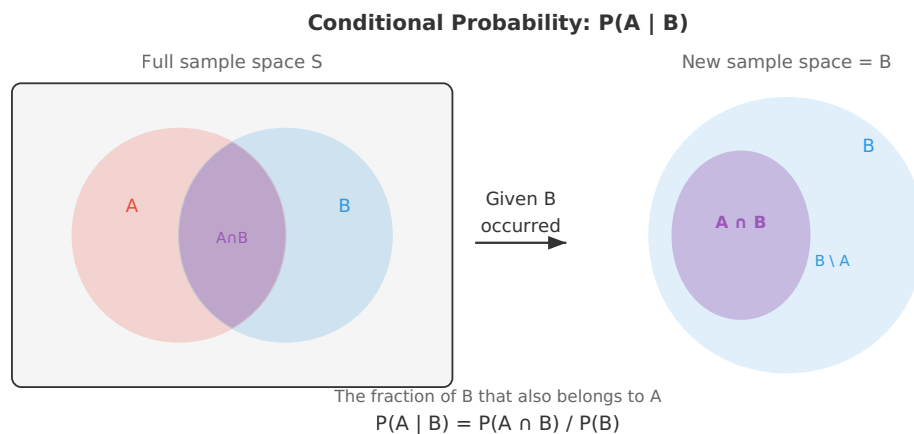
- **Joint probability**  $P(A \cap B)$  is the probability that both A and B occur. In a deck of cards,  $P(\text{red} \cap \text{king}) = \frac{2}{52}$  because there are 2 red kings.

- **Marginal probability** is the probability of a single event regardless of others.

$P(\text{red}) = \frac{26}{52} = 0.5$  is a marginal probability. If you have a joint distribution over two variables, the marginal is obtained by summing (or integrating) over the other variable.

- **Conditional probability** answers: given that B has already happened, what is the probability of A? We shrink the sample space from S down to B, and ask what fraction of B also belongs to A:

$$P(A | B) = \frac{P(A \cap B)}{P(B)}, \quad P(B) > 0$$



Conditional probability as shrinking the sample space from S to B

- Example: you draw a card and someone tells you it is red. What is the probability it is a king? There are 26 red cards and 2 of them are kings, so

$$P(\text{king} | \text{red}) = \frac{2}{26} = \frac{1}{13}. \text{ Using the formula: } P(\text{king} \cap \text{red}) / P(\text{red}) = \frac{2/52}{26/52} = \frac{1}{13}.$$

- Two events are **independent** if knowing one happened tells you nothing about the other. Formally:

$$P(A \cap B) = P(A) \cdot P(B)$$

- Equivalently,  $P(A | B) = P(A)$ . Flipping two separate coins are independent events. Drawing two cards without replacement are not independent (the first draw changes what remains).
- Independence is a massive simplifier. For independent events, joint probabilities factor into products, which makes computation tractable. Many

ML models assume independence between features (e.g. Naive Bayes) precisely because of this simplification.

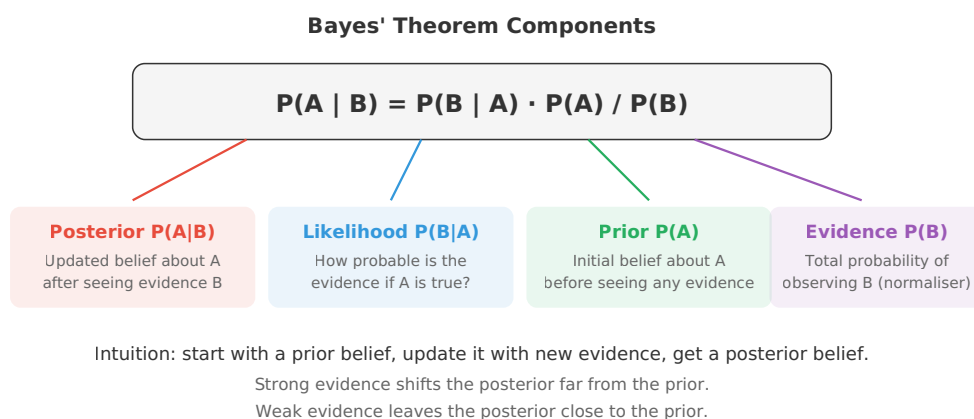
- The **multiplication rule** for any two events rearranges the conditional probability formula:

$$P(A \cap B) = P(A | B) \cdot P(B) = P(B | A) \cdot P(A)$$

- For independent events, this simplifies to  $P(A \cap B) = P(A) \cdot P(B)$  since the conditional equals the marginal.
- **Bayes' theorem** is one of the most important results in probability and the foundation of Bayesian ML. It lets you reverse the direction of a conditional probability:

$$P(A | B) = \frac{P(B | A) \cdot P(A)}{P(B)}$$

- The theorem follows directly from writing  $P(A \cap B)$  two ways:  
 $P(B | A) \cdot P(A) = P(A | B) \cdot P(B)$ , then solving for  $P(A | B)$ .



Bayes' theorem components: posterior, likelihood, prior, and evidence

- Each component has a name:
  - **Prior**  $P(A)$ : your initial belief before seeing evidence
  - **Likelihood**  $P(B | A)$ : how probable the evidence is, assuming A is true
  - **Evidence**  $P(B)$ : the total probability of seeing the evidence, acts as a normaliser
  - **Posterior**  $P(A | B)$ : your updated belief after seeing the evidence
- Let us work through the classic medical diagnosis example. Suppose a disease affects 1% of the population. A test for the disease is 95% accurate: it correctly

identifies 95% of sick people (sensitivity) and correctly identifies 90% of healthy people (specificity).

- You test positive. What is the probability you actually have the disease?
- Let  $D$  = having the disease,  $+$  = testing positive.
  - Prior:  $P(D) = 0.01$
  - Likelihood:  $P(+ | D) = 0.95$
  - False positive rate:  $P(+ | D') = 0.10$
- We need  $P(+)$ . By the law of total probability:

$$\begin{aligned} P(+) &= P(+ | D) \cdot P(D) + P(+ | D') \cdot P(D') \\ &= 0.95 \times 0.01 + 0.10 \times 0.99 = 0.0095 + 0.099 = 0.1085 \end{aligned}$$

- Now apply Bayes' theorem:

$$P(D | +) = \frac{P(+ | D) \cdot P(D)}{P(+)} = \frac{0.95 \times 0.01}{0.1085} \approx 0.088$$

- Despite the test being “95% accurate,” a positive result only gives you about an 8.8% chance of having the disease. The prior matters enormously. Because the disease is rare, most positive results are false positives. This is a crucial insight for any classification problem in ML: when classes are imbalanced, accuracy alone is misleading.
- The **law of total probability** partitions the sample space into mutually exclusive, exhaustive events  $B_1, B_2, \dots, B_n$  and expresses any event  $A$  as:

$$P(A) = \sum_{i=1}^n P(A | B_i) \cdot P(B_i)$$

- This is exactly what we used to compute  $P(+)$  in the medical example: we split the population into “has disease” and “does not have disease.”
- The **chain rule of probability** generalises the multiplication rule to any number of events:

$$P(A_1 \cap A_2 \cap \dots \cap A_n) = P(A_1) \cdot P(A_2 | A_1) \cdot P(A_3 | A_1 \cap A_2) \dots P(A_n | A_1 \cap \dots \cap A_{n-1})$$

- Each factor conditions on everything that came before. This is the backbone of autoregressive language models: the probability of a sentence is the product of each word's probability given all previous words.
- **Conditional independence** means two events are independent given a third. A and B are conditionally independent given C if:

$$P(A \cap B | C) = P(A | C) \cdot P(B | C)$$

- Events can be marginally dependent but conditionally independent, or vice versa. For example, two students' exam scores may be correlated (both depend on the difficulty of the exam), but given the exam difficulty, their scores are independent.
- Conditional independence is the key assumption behind graphical models like Bayesian networks. It lets you factorise complex joint distributions into manageable pieces, making inference computationally feasible.

## 22.1 Coding Tasks (use CoLab or notebook)

---

1. Simulate the medical diagnosis problem. Generate a population of 100,000 people, apply the disease prevalence and test accuracy, and verify that Bayes' theorem gives the correct posterior.

```
import jax
import jax.numpy as jnp

key = jax.random.PRNGKey(42)
n = 100_000

Generate population
k1, k2 = jax.random.split(key)
has_disease = jax.random.bernoulli(k1, p=0.01, shape=(n,))

Generate test results
k3, k4 = jax.random.split(k2)
Sensitivity: P(+|D) = 0.95, Specificity: P(-|D') = 0.90
test_positive = jnp.where(
 has_disease,
 jax.random.bernoulli(k3, p=0.95, shape=(n,)),
 jax.random.bernoulli(k4, p=0.10, shape=(n,))
```

```

)

Among those who tested positive, what fraction actually has the disease?
positives = test_positive.astype(bool)
true_positives = (has_disease & positives).sum()
total_positives = positives.sum()

print(f"Total positive tests: {total_positives}")
print(f"True positives: {true_positives}")
print(f"P(Disease | Positive) = {true_positives / total_positives:.4f}")
print(f"Bayes' formula: {0.95 * 0.01 / 0.1085:.4f}")

```

1. Verify the addition rule by simulation. Generate random events A and B with known probabilities and overlap, then check that  $P(A \cup B) = P(A) + P(B) - P(A \cap B)$ .

```

import jax
import jax.numpy as jnp

key = jax.random.PRNGKey(0)
n = 200_000
k1, k2 = jax.random.split(key)

Events: A = value < 0.4, B = value < 0.6 (overlap at < 0.4)
vals_a = jax.random.uniform(k1, shape=(n,))
vals_b = jax.random.uniform(k2, shape=(n,))

A = vals_a < 0.4
B = vals_b < 0.6

p_a = A.mean()
p_b = B.mean()
p_a_and_b = (A & B).mean()
p_a_or_b = (A | B).mean()

print(f"P(A) = {p_a:.4f}")
print(f"P(B) = {p_b:.4f}")
print(f"P(A ∩ B) = {p_a_and_b:.4f}")
print(f"P(A ∪ B) simulated = {p_a_or_b:.4f}")
print(f"P(A) + P(B) - P(A∩B) = {p_a + p_b - p_a_and_b:.4f}")

```

1. Demonstrate that conditional probability changes with evidence. Simulate rolling two dice and compute  $P(\text{sum} = 7)$ , then  $P(\text{sum} = 7 \mid \text{first die} = 3)$ .

```

import jax
import jax.numpy as jnp

key = jax.random.PRNGKey(1)
n = 500_000
k1, k2 = jax.random.split(key)

d1 = jax.random.randint(k1, shape=(n,), minval=1, maxval=7)
d2 = jax.random.randint(k2, shape=(n,), minval=1, maxval=7)
total = d1 + d2

Unconditional
p_sum7 = (total == 7).mean()
print(f"P(sum=7) = {p_sum7:.4f} (exact: {6/36:.4f})")

Conditional on first die = 3
mask = d1 == 3
p_sum7_given_d1_3 = (total[mask] == 7).mean()
print(f"P(sum=7 | d1=3) = {p_sum7_given_d1_3:.4f} (exact: {1/6:.4f})")

```

1. Implement Bayes' theorem as a function and use it to update beliefs iteratively. Start with a uniform prior over a coin's bias and update after observing each flip.

```

import jax.numpy as jnp
import matplotlib.pyplot as plt

def bayes_update(prior, likelihood):
 """Multiply prior by likelihood and normalise."""
 posterior = prior * likelihood
 return posterior / posterior.sum()

Discretise possible bias values
theta = jnp.linspace(0, 1, 200)
prior = jnp.ones_like(theta) # uniform prior
prior = prior / prior.sum()

Observed flips: 1=heads, 0=tails
flips = [1, 1, 0, 1, 1, 1, 0, 1, 0, 1]

plt.figure(figsize=(10, 5))
plt.plot(theta, prior, "--", color="#999", label="prior")

for i, flip in enumerate(flips):
 likelihood = theta if flip == 1 else (1 - theta)
 prior = bayes_update(prior, likelihood)

```

```

if i in [0, 2, 4, 9]:
 plt.plot(theta, prior, label=f"after {i+1} flips", linewidth=2)

plt.xlabel("Coin bias θ ")
plt.ylabel("Belief (normalised)")
plt.title("Bayesian updating: belief about coin bias")
plt.legend()
plt.grid(alpha=0.3)
plt.show()

```

## 23 Probability Distributions

Probability distributions describe how random outcomes are spread across possible values. This file catalogues the key discrete and continuous distributions; Bernoulli, binomial, Poisson, Gaussian, exponential, beta, and more, giving the formulas, intuitions, and ML applications (loss functions, priors, noise models) for each.

- In Chapter 4 we introduced random variables, PMFs, PDFs, and CDFs. Here we catalogue the most important probability distributions you will encounter in ML and statistics, giving the intuition, formula, mean, and variance for each.
- Quick recap of the three core functions (see Chapter 4 for full definitions):
  - **PMF**  $P(X = x)$ : gives the probability of each discrete outcome. The bars in a bar chart.
  - **PDF**  $f(x)$ : gives the density at each point for continuous variables. The area under the curve between two points is the probability.
  - **CDF**  $F(x) = P(X \leq x)$ : the cumulative probability up to  $x$ . Always goes from 0 to 1 and never decreases.
- The **support** of a distribution is the set of values where the PMF or PDF is positive. For a die roll, the support is  $\{1, 2, 3, 4, 5, 6\}$ . For the normal distribution, the support is all real numbers  $(-\infty, \infty)$ .
- Distributions divide cleanly into two families: discrete (countable outcomes, use PMFs) and continuous (uncountable outcomes, use PDFs).
- **Bernoulli distribution**: the simplest distribution. A single trial with two outcomes: success (1) with probability  $p$  and failure (0) with probability  $1 - p$ .

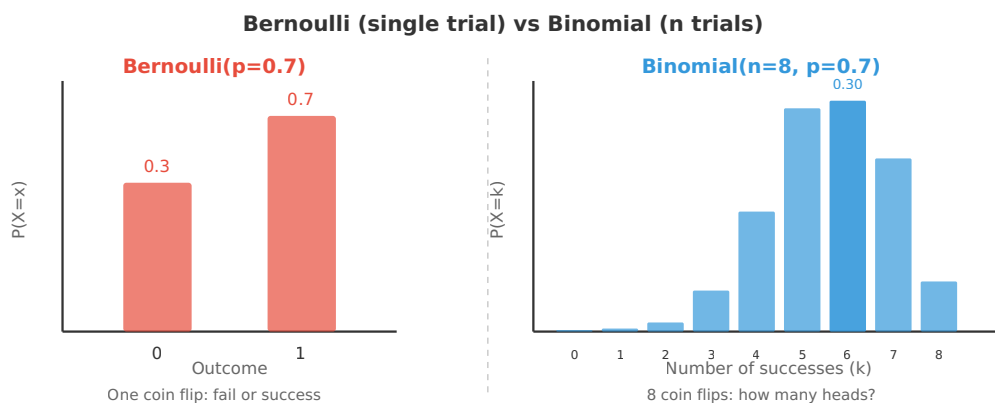


$$P(X=x) = p^x(1-p)^{1-x}, \quad x \in \{0, 1\}$$

- Mean:  $E[X] = p$ . Variance:  $\text{Var}(X) = p(1-p)$ .
- Every coin flip, every yes/no classification, every binary outcome is a Bernoulli trial. In ML, the output of a sigmoid function is exactly the  $p$  parameter of a Bernoulli distribution.
- **Binomial distribution:** count the number of successes in  $n$  independent Bernoulli trials, each with the same probability  $p$ .

$$P(X=k) = \binom{n}{k} p^k (1-p)^{n-k}, \quad k=0, 1, \dots, n$$

- The binomial coefficient  $\binom{n}{k}$  from file 01 counts how many ways to arrange  $k$  successes among  $n$  trials.
- Mean:  $E[X] = np$ . Variance:  $\text{Var}(X) = np(1-p)$ .



Bernoulli as a single bar chart vs Binomial as a distribution over counts

- Example: flip a biased coin ( $p = 0.7$ ) eight times. The probability of getting exactly 6 heads is  $\binom{8}{6} (0.7)^6 (0.3)^2 = 28 \times 0.1176 \times 0.09 \approx 0.296$ .
- **Poisson distribution:** counts the number of events in a fixed interval of time or space, given a known average rate  $\lambda$ . Useful when events are rare and independent.

$$P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}, \quad k = 0, 1, 2, \dots$$

- Mean:  $E[X] = \lambda$ . Variance:  $\text{Var}(X) = \lambda$ . The mean equals the variance, which is a signature property.
- Examples: emails per hour ( $\lambda = 5$ ), typos per page, server requests per second. In ML, Poisson regression models count data where a linear model would predict negative counts.
- As  $n \rightarrow \infty$  and  $p \rightarrow 0$  with  $np = \lambda$  held constant, the  $\text{Binomial}(n, p)$  converges to  $\text{Poisson}(\lambda)$ . This is why the Poisson works well for rare events in large populations.
- **Geometric distribution:** counts the number of trials until the first success.  
“How many coins do I flip before I get my first heads?”

$$P(X = k) = (1 - p)^{k-1} p, \quad k = 1, 2, 3, \dots$$

- Mean:  $E[X] = 1/p$ . Variance:  $\text{Var}(X) = (1 - p)/p^2$ .
- The geometric distribution is **memoryless**: the probability of waiting  $k$  more trials for success does not depend on how many trials you have already waited. This makes it special among discrete distributions.
- **Negative Binomial distribution:** generalises the geometric by counting trials until the  $r$ -th success (geometric is the special case  $r = 1$ ).

$$P(X = k) = \binom{k-1}{r-1} p^r (1-p)^{k-r}, \quad k = r, r+1, r+2, \dots$$

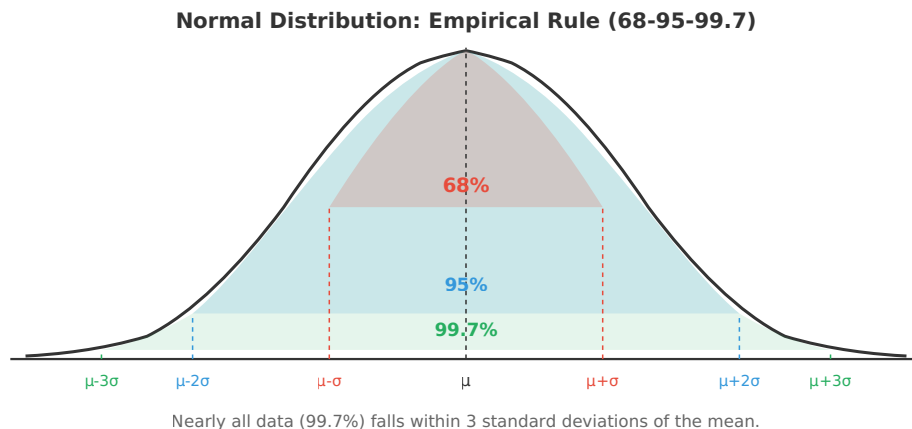
- Mean:  $E[X] = r/p$ . Variance:  $\text{Var}(X) = r(1-p)/p^2$ .
- The Negative Binomial is also used in practice to model overdispersed count data (where the variance exceeds the mean), which the Poisson cannot handle.
- Now we move to continuous distributions.
- **Uniform distribution:** all values in an interval  $[a, b]$  are equally likely. The PDF is a flat rectangle.

$$f(x) = \frac{1}{b-a}, \quad a \leq x \leq b$$

- Mean:  $E[X] = \frac{a+b}{2}$ . Variance:  $\text{Var}(X) = \frac{(b-a)^2}{12}$ .
- Random number generators produce Uniform(0,1) samples as their starting point. Other distributions are generated by transforming these uniform samples.
- **Normal (Gaussian) distribution:** the most important distribution in statistics. It arises naturally from the Central Limit Theorem (see Chapter 4): averages of many independent random variables tend toward a normal distribution regardless of the original distribution.

$$f(x) = \frac{1}{\frac{\sigma}{\sqrt{2\pi}}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

- Mean:  $E[X] = \mu$ . Variance:  $\text{Var}(X) = \sigma^2$ .
- The **standard normal** has  $\mu = 0$  and  $\sigma = 1$ . Any normal variable  $X$  can be standardised to a standard normal  $Z$  using  $Z = (X - \mu)/\sigma$ .



Bell curve with 68-95-99.7 empirical rule regions shaded

- The **empirical rule** (68-95-99.7 rule) says:
  - About 68% of data falls within  $\pm 1\sigma$  of the mean
  - About 95% falls within  $\pm 2\sigma$
  - About 99.7% falls within  $\pm 3\sigma$

- In ML, normal distributions appear everywhere: weight initialisation, noise in data augmentation, the assumption behind MSE loss (which implicitly assumes Gaussian errors), and the reparameterisation trick in variational autoencoders.
- **Exponential distribution:** models the time between events in a Poisson process. If events arrive at rate  $\lambda$ , the waiting time between them follows  $\text{Exponential}(\lambda)$ .

$$f(x) = \lambda e^{-\lambda x}, \quad x \geq 0$$

- Mean:  $E[X] = 1/\lambda$ . Variance:  $\text{Var}(X) = 1/\lambda^2$ .
- Like the geometric distribution for discrete variables, the exponential is **memoryless**:  $P(X > s + t \mid X > s) = P(X > t)$ . The probability of waiting another  $t$  units does not depend on how long you have already waited.
- **Gamma distribution:** generalises the exponential. It models the time until the  $\alpha$ -th event in a Poisson process (exponential is  $\alpha = 1$ ).

$$f(x) = \frac{\beta^\alpha}{\Gamma(\alpha)} x^{\alpha-1} e^{-\beta x}, \quad x > 0$$

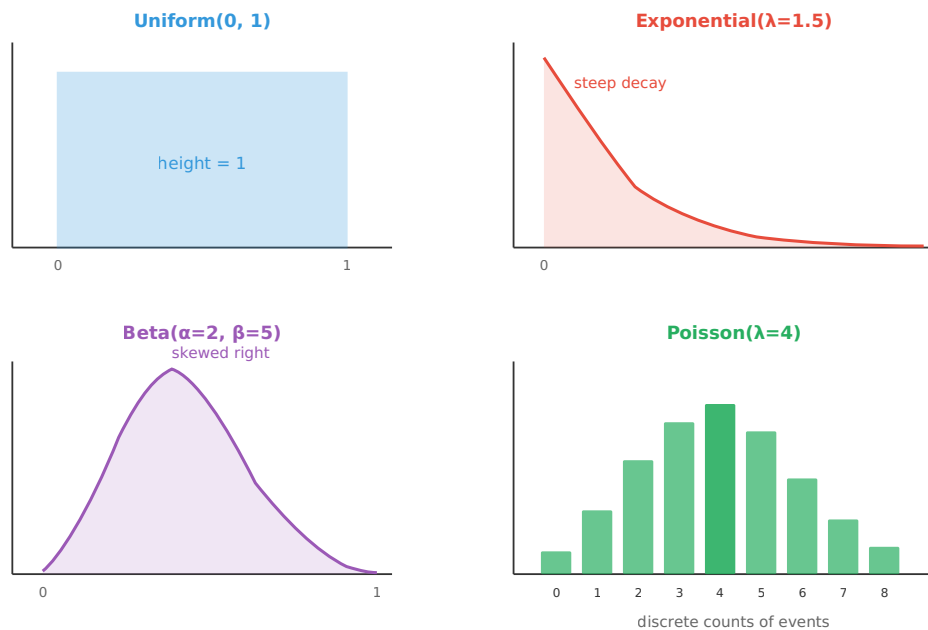
- Here  $\alpha$  (shape) controls the shape and  $\beta$  (rate) controls the scale.  $\Gamma(\alpha)$  is the gamma function, which extends factorials to real numbers:  $\Gamma(n) = (n-1)!$  for positive integers.
- Mean:  $E[X] = \alpha/\beta$ . Variance:  $\text{Var}(X) = \alpha/\beta^2$ .
- **Beta distribution:** defined on the interval  $[0, 1]$ , making it perfect for modelling probabilities, proportions, and rates.

$$f(x) = \frac{x^{\alpha-1}(1-x)^{\beta-1}}{B(\alpha, \beta)}, \quad 0 \leq x \leq 1$$

- The denominator  $B(\alpha, \beta) = \frac{\Gamma(\alpha)\Gamma(\beta)}{\Gamma(\alpha+\beta)}$  is the beta function, a normalising constant.
- Mean:  $E[X] = \frac{\alpha}{\alpha+\beta}$ . Variance:  $\text{Var}(X) = \frac{\alpha\beta}{(\alpha+\beta)^2(\alpha+\beta+1)}$ .
- The Beta distribution is the conjugate prior for the Bernoulli and Binomial likelihoods. This means if your prior is Beta and your data is Bernoulli, the

posterior is also Beta, which makes Bayesian updating analytically tractable. We will use this in file 04.

#### Common Distribution Shapes



Four common distribution shapes: Uniform, Exponential, Beta, Poisson

- **Chi-squared distribution** ( $\chi^2$ ): if you take  $k$  independent standard normal random variables and sum their squares, the result follows a  $\chi^2$  distribution with  $k$  degrees of freedom.

$$f(x) = \frac{1}{2^{k/2}\Gamma(k/2)} x^{k/2-1} e^{-x/2}, \quad x > 0$$

- Mean:  $E[X] = k$ . Variance:  $\text{Var}(X) = 2k$ .
- The  $\chi^2$  distribution is actually a special case of the Gamma distribution with  $\alpha = k/2$  and  $\beta = 1/2$ . It appears in hypothesis testing (the chi-squared test from Chapter 4), goodness-of-fit tests, and in computing confidence intervals for variance.
- **Student's t-distribution**: looks like a normal distribution but with heavier tails. It arises when you estimate the mean of a normally distributed population using a small sample and the population variance is unknown.

$$f(x) = \frac{\Gamma\left(\frac{\nu+1}{2}\right)}{\sqrt{\nu\pi} \Gamma\left(\frac{\nu}{2}\right)} \left(1 + \frac{x^2}{\nu}\right)^{-(\nu+1)/2}$$

- The parameter  $\nu$  (nu) is the degrees of freedom. As  $\nu \rightarrow \infty$ , the t-distribution converges to the standard normal. With small  $\nu$ , the heavier tails give more probability to extreme values, reflecting the extra uncertainty from a small sample.
- Mean:  $E[X] = 0$  (for  $\nu > 1$ ). Variance:  $\text{Var}(X) = \frac{\nu}{\nu-2}$  (for  $\nu > 2$ ).
- The t-distribution is used in t-tests (Chapter 4) and shows up in Bayesian inference as a marginal distribution when integrating out unknown variance.
- To summarise the key distributions:

Distribution	Type	Support	Mean	Variance
Bernoulli( $p$ )	Discrete	$\{0, 1\}$	$p$	$p(1-p)$
Binomial( $n, p$ )	Discrete	$\{0, \dots, n\}$	$np$	$np(1-p)$
Poisson( $\lambda$ )	Discrete	$\{0, 1, 2, \dots\}$	$\lambda$	$\lambda$
Geometric( $p$ )	Discrete	$\{1, 2, 3, \dots\}$	$1/p$	$(1-p)/p^2$
Uniform( $a, b$ )	Continuous	$[a, b]$	$(a+b)/2$	$(b-a)^2/12$
Normal( $\mu, \sigma^2$ )	Continuous	$(-\infty, \infty)$	$\mu$	$\sigma^2$
Exponential( $\lambda$ )	Continuous	$[0, \infty)$	$1/\lambda$	$1/\lambda^2$
Gamma( $\alpha, \beta$ )	Continuous	$(0, \infty)$	$\alpha/\beta$	$\alpha/\beta^2$
Beta( $\alpha, \beta$ )	Continuous	$[0, 1]$	$\alpha/(\alpha+\beta)$	see above
$\chi^2(k)$	Continuous	$(0, \infty)$	$k$	$2k$
Student's t( $\nu$ )	Continuous	$(-\infty, \infty)$	$0$	$\nu/(\nu-2)$

## 23.1 Coding Tasks (use CoLab or notebook)

1. Plot the Binomial PMF for  $n = 20$  with several values of  $p$ . Observe how the shape shifts from left-skewed to symmetric to right-skewed.

```
import jax.numpy as jnp
import matplotlib.pyplot as plt
from math import comb

n = 20
ks = jnp.arange(0, n + 1)

fig, axes = plt.subplots(1, 3, figsize=(12, 4), sharey=True)
for ax, p, color in zip(axes, [0.2, 0.5, 0.8], ["#e74c3c", "#3498db",
"#27ae60"]):
 pmf = jnp.array([comb(n, int(k)) * p**k * (1-p)**(n-k) for k in ks])
 ax.bar(ks, pmf, color=color, alpha=0.7)
 ax.set_title(f"Binomial(n={n}, p={p})")
 ax.set_xlabel("k")
axes[0].set_ylabel("P(X = k)")
plt.tight_layout()
plt.show()
```

1. Verify the Poisson approximation to the Binomial. Set  $n = 1000$ ,  $p = 0.003$ , and compare  $\text{Binomial}(n, p)$  with  $\text{Poisson}(\lambda = np)$ .

```
import jax.numpy as jnp
import matplotlib.pyplot as plt
from math import comb, factorial, exp

n, p = 1000, 0.003
lam = n * p
ks = jnp.arange(0, 15)

binom_pmf = jnp.array([comb(n, int(k)) * p**k * (1-p)**(n-k) for k in ks])
poisson_pmf = jnp.array([lam**k * exp(-lam) / factorial(int(k)) for k in ks])

plt.figure(figsize=(8, 4))
plt.bar(ks - 0.15, binom_pmf, width=0.3, color="#3498db", alpha=0.7,
label=f"Binomial({n},{p})")
plt.bar(ks + 0.15, poisson_pmf, width=0.3, color="#e74c3c", alpha=0.7,
label=f"Poisson({lam})")
plt.xlabel("k")
plt.ylabel("P(X = k)")
plt.title("Poisson approximation to Binomial")
```

```
plt.legend()
plt.show()
```

1. Sample from a Normal distribution and verify the empirical rule. Count what fraction of samples fall within 1, 2, and 3 standard deviations.

```
import jax
import jax.numpy as jnp

key = jax.random.PRNGKey(42)
mu, sigma = 5.0, 2.0
samples = mu + sigma * jax.random.normal(key, shape=(100_000,))

for k in [1, 2, 3]:
 within = jnp.abs(samples - mu) <= k * sigma
 print(f"Within {k}σ: {within.mean():.4f} (expected: {[0.6827, 0.9545, 0.9973][k-1]:.4f})")
```

1. Explore the Beta distribution by varying  $\alpha$  and  $\beta$ . Plot several shapes and see how the distribution changes from uniform to skewed to concentrated.

```
import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

x = jnp.linspace(0.01, 0.99, 200)

def beta_pdf(x, a, b):
 # Unnormalised for shape comparison
 return x**(a-1) * (1-x)**(b-1)

plt.figure(figsize=(10, 5))
params = [(1,1,"Uniform"), (2,5,"Left skew"), (5,2,"Right skew"),
 (5,5,"Symmetric"), (0.5,0.5,"U-shape")]
colors = ["#999", "#e74c3c", "#3498db", "#27ae60", "#9b59b6"]

for (a, b, label), color in zip(params, colors):
 y = beta_pdf(x, a, b)
 y = y / jnp.trapezoid(y, x) # normalise
 plt.plot(x, y, label=f"α={a}, β={b} ({label})", color=color, linewidth=2)

plt.xlabel("x")
plt.ylabel("Density")
plt.title("Beta distribution shapes")
plt.legend()
```



```
plt.grid(alpha=0.3)
plt.show()
```

## 24 Bayesian Methods and Sequential Models

Bayesian methods combine prior beliefs with observed data to produce posterior distributions over model parameters. This file covers maximum likelihood estimation, MAP estimation, conjugate priors, Bayesian inference, hidden Markov models, and the EM algorithm, techniques behind spam filters, language models, and uncertainty-aware ML.

- So far we have described distributions and how to compute probabilities. Now we tackle the question at the heart of ML: given observed data, how do we find the best parameters for our model?
- **Maximum Likelihood Estimation (MLE)** answers this directly. Pick the parameter values that make the observed data most probable.
- Formally, given data  $D = \{x_1, x_2, \dots, x_n\}$  and a model with parameter  $\theta$ , the **likelihood function** is:

$$L(\theta | D) = P(D | \theta) = \prod_{i=1}^n P(x_i | \theta)$$

- The product assumes the data points are independent and identically distributed (i.i.d.). The MLE estimate is:

$$\hat{\theta}_{MLE} = \arg \max_{\theta} L(\theta | D)$$

- In practice we maximise the **log-likelihood** instead, because the log turns products into sums and prevents numerical underflow:

$$\ell(\theta) = \log L(\theta | D) = \sum_{i=1}^n \log P(x_i | \theta)$$

- Since log is monotonically increasing, the  $\theta$  that maximises  $\ell(\theta)$  also maximises  $L(\theta)$ .
- **Coin toss example:** you flip a coin 10 times and get 7 heads. What is the MLE estimate of the coin's bias  $p$  (probability of heads)?
- Each flip is Bernoulli( $p$ ), so the likelihood of 7 heads in 10 flips is:

$$L(p) = \binom{10}{7} p^7 (1-p)^3$$

- Taking the log and differentiating:  $\frac{d\ell}{dp} = \frac{7}{p} - \frac{3}{1-p} = 0$ , which gives

↺

$p$

$$p_{MLE} = 7/10 = 0.7.$$

- MLE is intuitive and simple. If you got 7 heads in 10 flips, the most likely bias is 0.7. But notice the problem: if you got 10 heads in 10 flips, MLE says

↺

$p$

$= 1$ , meaning the coin will always land heads. That seems overconfident given only 10 observations.

- **Maximum A Posteriori (MAP)** estimation fixes this by adding prior beliefs. Instead of maximising just the likelihood, MAP maximises the posterior:

↺

$\theta$

$$p_{MAP} = \arg$$

max

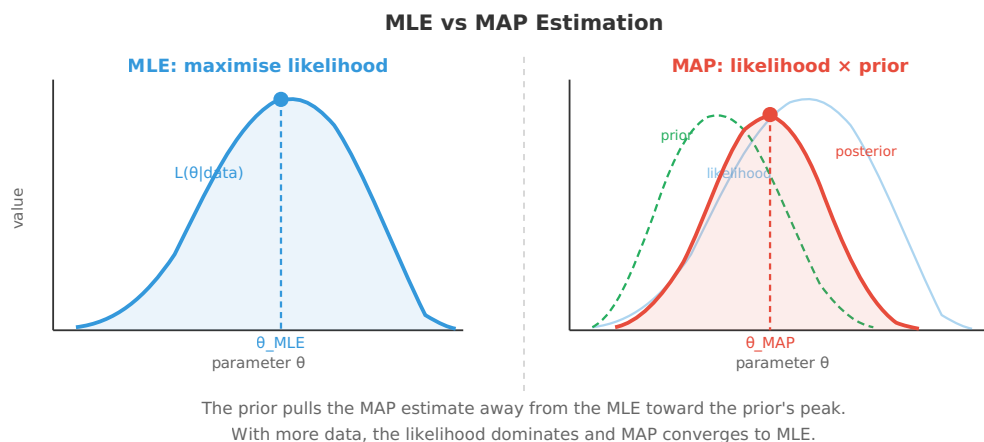
$\theta$

$$P(\theta | D) = \arg$$

max  
 $\theta$

$$P(D \mid \theta) \cdot P(\theta)$$

- We dropped  $P(D)$  from the denominator because it does not depend on  $\theta$  and does not affect the argmax.
- The prior  $P(\theta)$  encodes what we believed about  $\theta$  before seeing data. If we use a Beta(2, 2) prior for our coin bias (expressing a mild belief that the coin is roughly fair), the MAP estimate is no longer simply the proportion of heads. It gets pulled toward 0.5.



MLE finds the peak of the likelihood; MAP finds the peak of likelihood times prior

- With a Beta( $\alpha$ ,  $\beta$ ) prior and observing  $h$  heads and  $t$  tails, the posterior is Beta( $\alpha + h$ ,  $\beta + t$ ), and the MAP estimate is:

⌚  
 p

$$\text{MAP} = \frac{\alpha + h - 1}{\alpha + \beta + h + t - 2}$$

- For our example with Beta(2,2) prior, 7 heads, 3 tails:

⌚  
 p

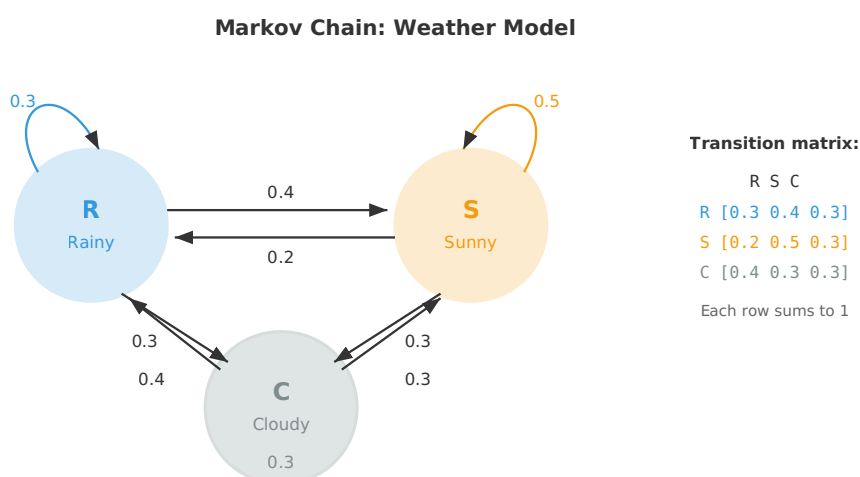
$$\text{MAP} = \frac{2 + 7 - 1}{2 + 2 + 10 - 2} = \frac{8}{12} = 0.667.$$

- Notice how the MAP estimate (0.667) is pulled toward 0.5 compared to the MLE (0.7). The prior acts as regularisation. In ML, L2 regularisation (weight decay) is exactly equivalent to MAP estimation with a Gaussian prior on the weights.

- **Full Bayesian inference** goes further than MAP. Instead of finding a single best  $\theta$ , it maintains the entire posterior distribution  $P(\theta | D)$ . This gives you not just a point estimate but a measure of uncertainty.
- For the biased coin with Beta(2,2) prior and 7 heads, 3 tails, the full posterior is Beta(9, 5). The mean of this distribution is  $9/14 \approx 0.643$ , and its spread tells us how confident we are. With more data, the posterior narrows.
- The three approaches form a spectrum:
  - **MLE**: no prior, just data. Fast, but can overfit with little data.
  - **MAP**: point estimate with prior regularisation. Adds robustness.
  - **Full Bayesian**: entire posterior distribution. Most informative, but often computationally expensive.
- **Markov chains** model sequences where the next state depends only on the current state, not on the history. This “memorylessness” is called the **Markov property**:

$$P(X_{t+1} | X_t, X_{t-1}, \dots, X_1) = P(X_{t+1} | X_t)$$

- Think of weather. Tomorrow’s weather depends on today’s weather, but not on last week’s (a simplification, but surprisingly useful).
- A Markov chain has a finite set of **states** and a **transition matrix**  $T$  where entry  $T_{ij}$  gives the probability of moving from state  $i$  to state  $j$ . Each row sums to 1.

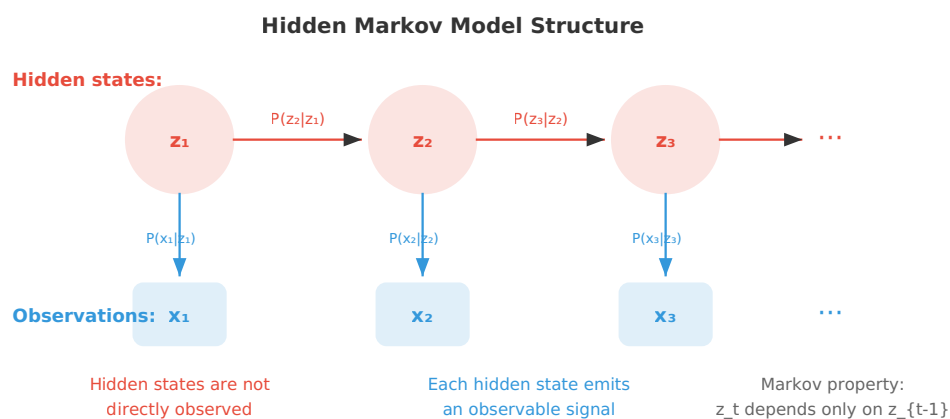


Weather Markov chain with states Rainy, Sunny, Cloudy and transition probabilities

- For the weather example above, the transition matrix is:

```
T = \begin{pmatrix} 0.3 & 0.4 & 0.3 \\ 0.2 & 0.5 & 0.3 \\ 0.4 & 0.3 & 0.3 \end{pmatrix}
```

- If today is rainy (state vector  $\mathbf{s}_0 = [1, 0, 0]$ ), the probability distribution over tomorrow's weather is  $\mathbf{s}_1 = \mathbf{s}_0 T = [0.3, 0.4, 0.3]$ . Two days from now:  $\mathbf{s}_2 = \mathbf{s}_0 T^2$ . This uses matrix multiplication from Chapter 1.
- Many Markov chains converge to a **stationary distribution**  $\pi$  such that  $\pi T = \pi$ . No matter where you start, after enough steps the chain settles into  $\pi$ . This property is the foundation of MCMC (Markov Chain Monte Carlo), a widely used sampling technique in Bayesian ML.
- **Hidden Markov Models (HMMs)** extend Markov chains by adding a layer of indirection. The true states are hidden (unobserved), and at each time step the hidden state emits an observable signal.



HMM structure: hidden states on top connected by transitions, observations below connected by emissions

- An HMM has three components:
  - **Transition probabilities**  $P(z_t | z_{t-1})$ : how hidden states evolve (the Markov chain)
  - **Emission probabilities**  $P(x_t | z_t)$ : what each hidden state produces as observable output
  - **Initial distribution**  $P(z_1)$ : the starting hidden state probabilities
- **Umbrella example**: suppose you cannot see the weather directly but you can observe whether your friend carries an umbrella. The hidden states are {Rainy, Sunny} and the observation is {Umbrella, No umbrella}.

- Transition probabilities:  $P(\text{Rainy} | \text{Rainy}) = 0.7$ ,  $P(\text{Sunny} | \text{Rainy}) = 0.3$ ,  $P(\text{Rainy} | \text{Sunny}) = 0.4$ ,  $P(\text{Sunny} | \text{Sunny}) = 0.6$ .
- Emission probabilities:  $P(\text{Umbrella} | \text{Rainy}) = 0.9$ ,  $P(\text{No umbrella} | \text{Rainy}) = 0.1$ ,  $P(\text{Umbrella} | \text{Sunny}) = 0.2$ ,  $P(\text{No umbrella} | \text{Sunny}) = 0.8$ .
- The key questions for HMMs are:
  - **Decoding**: given observations, what is the most likely sequence of hidden states? Solved by the **Viterbi algorithm**.
  - **Evaluation**: what is the probability of an observation sequence? Solved by the **Forward algorithm**.
  - **Learning**: given observations, what are the best model parameters? Solved by the **Baum-Welch algorithm** (an instance of Expectation-Maximisation).
- **Viterbi walkthrough**: suppose you observe [Umbrella, Umbrella, No umbrella] and want to find the most likely weather sequence.
- Start with initial probabilities. Assume  $P(R) = 0.5$ ,  $P(S) = 0.5$ .
- **Day 1** (observe Umbrella):
  - $V_1(R) = P(R) \cdot P(U | R) = 0.5 \times 0.9 = 0.45$
  - $V_1(S) = P(S) \cdot P(U | S) = 0.5 \times 0.2 = 0.10$
- **Day 2** (observe Umbrella):
  - $V_2(R) = \max(V_1(R) \cdot P(R | R), V_1(S) \cdot P(R | S)) \cdot P(U | R)$
  - $= \max(0.45 \times 0.7, 0.10 \times 0.4) \times 0.9 = \max(0.315, 0.04) \times 0.9 = 0.2835$
  - $V_2(S) = \max(V_1(R) \cdot P(S | R), V_1(S) \cdot P(S | S)) \cdot P(U | S)$
  - $= \max(0.45 \times 0.3, 0.10 \times 0.6) \times 0.2 = \max(0.135, 0.06) \times 0.2 = 0.027$
- **Day 3** (observe No umbrella):
  - $V_3(R) = \max(0.2835 \times 0.7, 0.027 \times 0.4) \times 0.1 = 0.1985 \times 0.1 = 0.01985$
  - $V_3(S) = \max(0.2835 \times 0.3, 0.027 \times 0.6) \times 0.8 = 0.08505 \times 0.8 = 0.06804$
- Day 3's maximum is at Sunny. Backtracking: Day 3 = Sunny (from R), Day 2 = Rainy (from R), Day 1 = Rainy. Most likely sequence: **Rainy, Rainy, Sunny**.
- The **Forward-Backward algorithm** computes the probability of being in each hidden state at each time step, given the entire observation sequence. The

forward pass computes  $P(z_t, x_{1:t})$  and the backward pass computes  $P(x_{t+1:T} | z_t)$ . Multiplying these gives the smoothed state probabilities.

- The **Baum-Welch algorithm** learns HMM parameters from data when the hidden states are unobserved. It is an Expectation-Maximisation (EM) algorithm: the E-step uses forward-backward to estimate which hidden states generated the observations, and the M-step updates the transition and emission probabilities.
- HMMs were historically dominant in speech recognition (hidden phoneme states emit acoustic signals) and bioinformatics (hidden gene states emit DNA base pairs). While deep learning has largely superseded HMMs in these fields, the ideas of hidden states, emissions, and sequential inference remain central to sequence models.
- **Conditional Random Fields (CRFs)** improve on HMMs by removing the independence assumption on emissions. In an HMM, the observation at time  $t$  depends only on the hidden state at time  $t$ . CRFs allow the label at position  $t$  to depend on the entire input sequence.
- A linear-chain CRF models the conditional probability of a label sequence  $\mathbf{y}$  given an input sequence  $\mathbf{x}$ :

$$P(\mathbf{y} | \mathbf{x}) = \frac{1}{Z(\mathbf{x})}$$

$\exp \left( \sum_t \right.$

$\left. \sum_k \right)$

$$\lambda_k f_k(y_t, y_{t-1}, \mathbf{x}, t)]$$

- Here  $f_k$  are feature functions (which can look at any part of the input),  $\lambda_k$  are learned weights, and  $Z(\mathbf{x})$  is a normalising constant.
- CRFs are discriminative models (they model  $P(\mathbf{y} | \mathbf{x})$  directly) while HMMs are generative (they model  $P(\mathbf{x}, \mathbf{y})$ ). This distinction is the same as logistic regression (discriminative) vs Naive Bayes (generative).
- In modern NLP, CRF layers are often added on top of neural networks (BiLSTM-CRF, BERT-CRF) for tasks like named entity recognition and part-of-speech tagging, where capturing label dependencies is important.

## 24.1 Coding Tasks (use CoLab or notebook)

1. Implement MLE and MAP for a coin toss experiment. Observe how the MAP estimate changes with different priors and different amounts of data.

```
import jax.numpy as jnp
import matplotlib.pyplot as plt

Data: observed coin flips
heads, tails = 7, 3

MLE
p_mle = heads / (heads + tails)
print(f"MLE: {p_mle:.4f}")

MAP with Beta prior
for alpha, beta in [(1,1), (2,2), (5,5), (10,10)]:
 p_map = (alpha + heads - 1) / (alpha + beta + heads + tails - 2)
 print(f"MAP (Beta({alpha},{beta})): {p_map:.4f}")

Visualise posterior for Beta(2,2) prior
theta = jnp.linspace(0.01, 0.99, 200)
Posterior is Beta(alpha+heads, beta+tails)
a_post, b_post = 2 + heads, 2 + tails
posterior = theta**((a_post-1)) * (1-theta)**((b_post-1))
posterior = posterior / jnp.trapezoid(posterior, theta)

plt.figure(figsize=(8, 4))
plt.plot(theta, posterior, color="#e74c3c", linewidth=2, label=f"Posterior
Beta({a_post},{b_post})")
plt.axvline(p_mle, color="#3498db", linestyle="--", label=f"MLE = {p_mle:.2f}")
plt.axvline((a_post-1)/(a_post+b_post-2), color="#e74c3c", linestyle="--",
label=f"MAP = {(a_post-1)/(a_post+b_post-2):.3f}")
plt.xlabel("θ (coin bias)")
plt.ylabel("Density")
plt.title("Posterior distribution after 7H, 3T with Beta(2,2) prior")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

1. Build a Markov chain for the weather model and simulate it. Compute the stationary distribution both by simulation and by solving  $\pi T = \pi$ .

```
import jax
import jax.numpy as jnp
```



```

Transition matrix: R, S, C
T = jnp.array([
 [0.3, 0.4, 0.3],
 [0.2, 0.5, 0.3],
 [0.4, 0.3, 0.3]
])
states = ["Rainy", "Sunny", "Cloudy"]

Simulate 100,000 steps
key = jax.random.PRNGKey(42)
n_steps = 100_000
state = 0 # start rainy
counts = jnp.zeros(3)

for i in range(n_steps):
 key, subkey = jax.random.split(key)
 state = jax.random.choice(subkey, 3, p=T[state])
 counts = counts.at[state].add(1)

sim_stationary = counts / n_steps
print("Simulated stationary distribution:")
for s, p in zip(states, sim_stationary):
 print(f" {s}: {p:.4f}")

Analytical: find left eigenvector with eigenvalue 1
eigenvalues, eigenvectors = jnp.linalg.eig(T.T)
idx = jnp.argmin(jnp.abs(eigenvalues - 1.0))
pi = jnp.real(eigenvectors[:, idx])
pi = pi / pi.sum()
print("\nAnalytical stationary distribution:")
for s, p in zip(states, pi):
 print(f" {s}: {p:.4f}")

```

1. Implement the Viterbi algorithm for the umbrella HMM and decode a sequence of observations.

```

import jax.numpy as jnp

HMM parameters
states = ["Rainy", "Sunny"]
obs_names = ["Umbrella", "No umbrella"]

trans = jnp.array([[0.7, 0.3], # R->R, R->S
 [0.4, 0.6]]) # S->R, S->S

```

```

emit = jnp.array([[0.9, 0.1], # R->U, R->noU
 [0.2, 0.8]]) # S->U, S->noU

init = jnp.array([0.5, 0.5])

Observations: U=0, noU=1
observations = [0, 0, 1] # Umbrella, Umbrella, No umbrella

def viterbi(obs, init, trans, emit):
 n_states = len(init)
 T = len(obs)
 V = jnp.zeros((T, n_states))
 path = jnp.zeros((T, n_states), dtype=int)

 # Initialisation
 V = V.at[0].set(init * emit[:, obs[0]])

 # Recursion
 for t in range(1, T):
 for j in range(n_states):
 probs = V[t-1] * trans[:, j]
 V = V.at[t, j].set(jnp.max(probs) * emit[j, obs[t]])
 path = path.at[t, j].set(jnp.argmax(probs))

 # Backtrack
 best = [int(jnp.argmax(V[-1]))]
 for t in range(T-1, 0, -1):
 best.insert(0, int(path[t, best[0]]))
 return best, V

decoded, scores = viterbi(observations, init, trans, emit)
print("Observations:", [obs_names[o] for o in observations])
print("Decoded: ", [states[s] for s in decoded])

```

1. Visualise how the posterior evolves as you observe more coin flips. Start with a Beta(1,1) prior (uniform) and update after each flip.

```

import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

theta = jnp.linspace(0.01, 0.99, 300)
key = jax.random.PRNGKey(7)

True bias = 0.65
flips = jax.random.bernoulli(key, p=0.65, shape=(50,))

```

```

plt.figure(figsize=(10, 5))
a, b = 1, 1 # Beta(1,1) = uniform

for n_obs in [0, 1, 5, 10, 25, 50]:
 h = int(flips[:n_obs].sum())
 t = n_obs - h
 a_post = a + h
 b_post = b + t
 y = theta**(a_post-1) * (1-theta)**(b_post-1)
 y = y / jnp.trapezoid(y, theta)
 plt.plot(theta, y, linewidth=2, label=f"n={n_obs} (h={h})")

plt.axvline(0.65, color="black", linestyle=":", alpha=0.5, label="true p=0.65")
plt.xlabel("θ")
plt.ylabel("Density")
plt.title("Bayesian updating: posterior narrows with more data")
plt.legend()
plt.grid(alpha=0.3)
plt.show()

```

## 25 Information Theory

Information theory quantifies information, surprise, and the difference between probability distributions. This file covers entropy, cross-entropy, KL divergence, mutual information, and surprisal, the concepts behind every classification loss function, VAE objective, and data compression scheme used in ML.

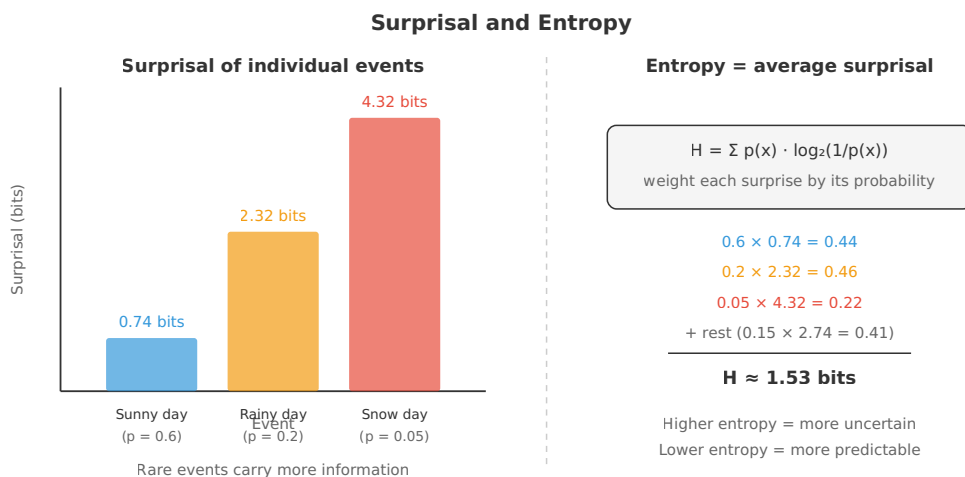
- Information theory, founded by Claude Shannon in 1948, gives us a mathematical framework for quantifying information. It answers questions like: how surprised should you be by an event? How much information does a message carry? How different are two probability distributions?
- These questions sound abstract, but they are the foundation of ML loss functions, data compression, and communication systems. Cross-entropy loss, the most common loss function in classification, comes directly from information theory.
- Start with the simplest question: how much information does a single event carry?

- **Surprisal** (also called self-information) measures how surprising an event is. If something very likely happens, you learn almost nothing. If something rare happens, you learn a lot.
- If you live in a desert and someone tells you it is sunny, that is not very informative. If they tell you it is snowing, that is extremely informative. Surprisal formalises this intuition:

$$I(x) = \log_2 \frac{1}{p(x)} = -\log_2 p(x)$$

- The unit is **bits** when we use  $\log_2$ . A fair coin flip has surprisal  $-\log_2(0.5) = 1$  bit. An event with probability  $1/8$  has surprisal  $\log_2(8) = 3$  bits.
- Why logarithm and not just  $1/p$ ? Three reasons:
  - A certain event ( $p = 1$ ) should give zero information:  $\log(1) = 0$  but  $1/1 = 1$ .
  - Independent events should have additive information:  $\log(1/p_1 p_2) = \log(1/p_1) + \log(1/p_2)$ .
  - We want a smooth, well-behaved function.  $1/p$  explodes;  $\log(1/p)$  grows gently.
- **Entropy** is the expected surprisal, the average amount of information you get per event sampled from a distribution. It measures the uncertainty or “unpredictability” of the distribution:

$$H(X) = E[I(X)] = - \sum_x p(x) \log_2 p(x)$$



Bar chart showing high-probability events have low surprisal and vice versa; entropy is the weighted average

- A fair coin has entropy  $H = -0.5\log_2(0.5) - 0.5\log_2(0.5) = 1$  bit. Maximum uncertainty.
- A biased coin with  $p = 0.9$  has entropy  $H = -0.9\log_2(0.9) - 0.1\log_2(0.1) \approx 0.469$  bits. Less uncertain, so less entropy.
- A deterministic event ( $p = 1$ ) has entropy  $H = 0$ . No uncertainty at all.
- Entropy is maximised when all outcomes are equally likely. For  $n$  equally likely outcomes,  $H = \log_2 n$ . A fair die has entropy  $\log_2 6 \approx 2.585$  bits.
- The practical meaning of entropy is **compression**. Shannon's source coding theorem says you cannot compress data below its entropy rate without losing information. An image where every pixel is equally likely (maximum entropy) cannot be compressed. An image that is mostly white (low entropy) compresses well.
- For a quick sense of scale: a grayscale pixel (256 values) has a maximum entropy of 8 bits. A 1080p grayscale image has at most  $1920 \times 1080 \times 8 \approx 16.6$  million bits. Real images have much lower entropy because neighbouring pixels are correlated, which is why JPEG compression works.

- For continuous random variables, the discrete sum becomes an integral.

**Differential entropy** is:

$$h(X) = - \int_{-\infty}^{\infty} f(x) \log f(x) \, dx$$

- A Gaussian with variance  $\sigma^2$  has differential entropy  $h = \frac{1}{2} \log_2(2\pi e \sigma^2)$ . Among all distributions with the same variance, the Gaussian has the maximum entropy. This is one reason the Gaussian is so common in modelling: it makes the fewest assumptions beyond the specified mean and variance.
- **Mutual information** measures how much knowing one variable tells you about another. It is the reduction in uncertainty about  $X$  when you observe  $Y$ :

$$I(X; Y) = H(X) - H(X | Y) = H(Y) - H(Y | X)$$

- Equivalently:

$$I(X; Y) =$$

$$\sum_{x, y}$$

$$p(x, y) \log_2 \frac{p(x, y)}{p(x)p(y)}$$

- If X and Y are independent,  $p(x, y) = p(x)p(y)$  and mutual information is zero. The more dependent they are, the higher the mutual information.
- In ML, mutual information is used in feature selection (pick features with high MI with the target), in information bottleneck methods, and in evaluating clustering quality.
- **Cross-entropy** measures the average number of bits needed to encode events from distribution p using a code optimised for distribution q:

$$H(p, q) = - \sum_x$$

$$p(x) \log_2 q(x)$$

- If q matches p perfectly, cross-entropy equals entropy:  $H(p, p) = H(p)$ . If q is a bad approximation, cross-entropy is higher. The “extra” bits come from the mismatch.
- This is exactly why cross-entropy is the standard loss function for classification in ML. The true labels define p (a one-hot distribution), and the model’s predicted probabilities define q. Minimising cross-entropy pushes q toward p:

$$\mathcal{L} = - \sum_c$$

$$y_c \log$$

$$\hat{y}_c$$

$$y$$

$$c$$

- For a single sample with true class c, this simplifies to  $\mathcal{L} = - \log$

$$\hat{y}_c$$

$$y$$

$c$ . The loss is the surprisal of the true class under the model’s predictions. If the model assigns high probability to the correct class, the loss is low.

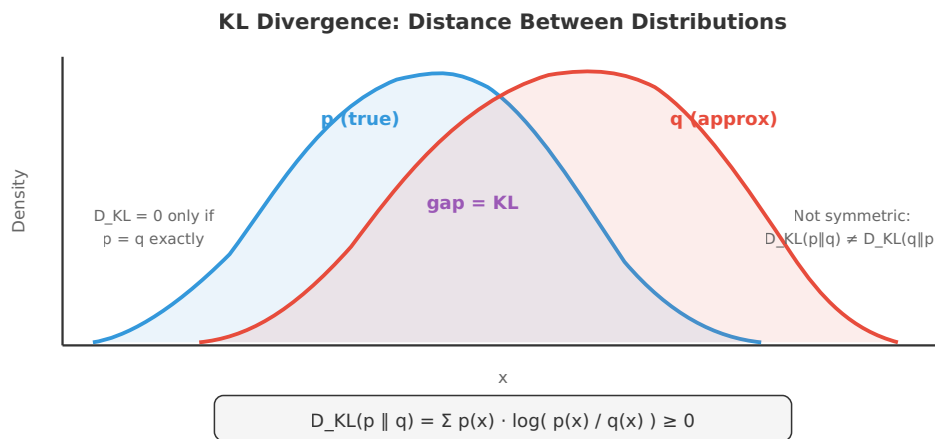
- **KL divergence** (Kullback-Leibler divergence, also called relative entropy) measures how much one distribution differs from another:

$$D_{KL}(p \parallel q) =$$

$\sum_x$

$$p(x) \log \frac{p(x)}{q(x)} = H(p, q) - H(p)$$

- KL divergence is the “extra cost” of using distribution  $q$  instead of the true distribution  $p$ . It is always non-negative ( $D_{KL} \geq 0$ ) and equals zero only when  $p = q$ .



Two distributions  $p$  and  $q$  with the gap between them representing KL divergence

- KL divergence is not symmetric:  $D_{KL}(p \parallel q) \neq D_{KL}(q \parallel p)$ . This asymmetry matters.  $D_{KL}(p \parallel q)$  penalises  $q$  for placing low probability where  $p$  has high probability (because  $\log(p/q)$  blows up).  $D_{KL}(q \parallel p)$  penalises the reverse.
- This asymmetry leads to two styles of approximation:
  - Minimising  $D_{KL}(p \parallel q)$  produces **moment-matching** behaviour:  $q$  covers all modes of  $p$  but may be too spread out.
  - Minimising  $D_{KL}(q \parallel p)$  produces **mode-seeking** behaviour:  $q$  concentrates on one mode of  $p$  but may miss others. This is what variational inference uses.
- Since  $H(p)$  is constant with respect to the model, minimising cross-entropy  $H(p, q)$  is equivalent to minimising  $D_{KL}(p \parallel q)$ . This is why we can use cross-

entropy loss and know that we are also minimising the KL divergence between the true and predicted distributions.

- KL divergence plays a central role in **Bayesian updating**. The posterior  $P(\theta | D)$  is the distribution closest to the prior  $P(\theta)$  (in KL divergence terms) that is consistent with the observed data. Each new observation updates the posterior, reducing uncertainty about  $\theta$ .
- In variational autoencoders (VAEs), the loss function has two terms: a reconstruction loss (cross-entropy) and a KL divergence term that regularises the latent space to stay close to a standard normal distribution.
- To tie everything together: entropy tells you the intrinsic uncertainty in a distribution, cross-entropy tells you how well your model approximates reality, and KL divergence tells you the gap between the two. These three quantities form the backbone of modern ML optimisation.

## 25.1 Coding Tasks (use CoLab or notebook)

---

1. Compute the entropy of various distributions and verify that the uniform distribution has maximum entropy for a given number of outcomes.

```
import jax.numpy as jnp

def entropy(p):
 """Compute entropy in bits. Filter out zero-probability events."""
 p = p[p > 0]
 return -jnp.sum(p * jnp.log2(p))

Fair die
fair = jnp.ones(6) / 6
print(f"Fair die entropy: {entropy(fair):.4f} bits (max = log2(6) = {jnp.log2(6):.4f})")

Loaded die
loaded = jnp.array([0.1, 0.1, 0.1, 0.1, 0.1, 0.5])
print(f"Loaded die entropy: {entropy(loaded):.4f} bits")

Deterministic
det = jnp.array([0.0, 0.0, 0.0, 0.0, 0.0, 1.0])
print(f"Deterministic: {entropy(det):.4f} bits")

Fair coin
```



```
coin = jnp.array([0.5, 0.5])
print(f"Fair coin entropy: {entropy(coin):.4f} bits")
```

1. Compute cross-entropy and KL divergence between a true distribution and several approximations. Verify that  $D_{\text{KL}}(p \parallel q) = H(p, q) - H(p)$ .

```
import jax.numpy as jnp

def cross_entropy(p, q):
 return -jnp.sum(p * jnp.log2(jnp.clip(q, 1e-10, 1.0)))

def kl_divergence(p, q):
 mask = p > 0
 return jnp.sum(jnp.where(mask, p * jnp.log2(p / jnp.clip(q, 1e-10, 1.0)),
0.0))

def entropy(p):
 p = p[p > 0]
 return -jnp.sum(p * jnp.log2(p))

p = jnp.array([0.4, 0.3, 0.2, 0.1]) # true distribution

for name, q in [("perfect match", p),
 ("slight mismatch", jnp.array([0.35, 0.30, 0.25, 0.10])),
 ("big mismatch", jnp.array([0.1, 0.1, 0.1, 0.7]))]:
 h_p = entropy(p)
 h_pq = cross_entropy(p, q)
 kl = kl_divergence(p, q)
 print(f"{name:20s}: H(p)={h_p:.4f}, H(p,q)={h_pq:.4f}, "
 f"KL={kl:.4f}, H(p,q)-H(p)={h_pq-h_p:.4f}")
```

1. Show that KL divergence is not symmetric by computing  $D_{\text{KL}}(p \parallel q)$  and  $D_{\text{KL}}(q \parallel p)$  for two different distributions.

```
import jax.numpy as jnp

def kl_div(p, q):
 mask = p > 0
 return float(jnp.sum(jnp.where(mask, p * jnp.log2(p / jnp.clip(q, 1e-10,
1.0)), 0.0)))

p = jnp.array([0.9, 0.1])
q = jnp.array([0.5, 0.5])

print(f"D_KL(p || q) = {kl_div(p, q):.4f}")
```

```
print(f"D_KL(q || p) = {kl_div(q, p):.4f}")
print(f"Not the same! KL divergence is asymmetric.")
```

1. Simulate cross-entropy loss during training. Create a “true” one-hot label and show how the loss decreases as the model’s predicted probabilities improve.

```
import jax.numpy as jnp
import matplotlib.pyplot as plt

True label: class 2 out of 4
true_label = jnp.array([0, 0, 1, 0])

Simulate improving predictions
steps = []
losses = []
for confidence in jnp.linspace(0.25, 0.99, 50):
 # Model becomes more confident in class 2
 remaining = (1 - confidence) / 3
 pred = jnp.array([remaining, remaining, confidence, remaining])
 loss = -jnp.sum(true_label * jnp.log(jnp.clip(pred, 1e-10, 1.0)))
 steps.append(float(confidence))
 losses.append(float(loss))

plt.figure(figsize=(8, 4))
plt.plot(steps, losses, color="#e74c3c", linewidth=2)
plt.xlabel("Model confidence in true class")
plt.ylabel("Cross-entropy loss")
plt.title("Cross-entropy loss decreases as predictions improve")
plt.grid(alpha=0.3)
plt.show()
```

## 26 Classical Machine Learning

Classical ML algorithms learn patterns from data without explicit programming, using closed-form solutions or heuristic search rather than gradient descent. This file covers Naive Bayes, k-NN, decision trees, random forests, SVMs, k-means clustering, and PCA

- Machine learning is the study of algorithms that improve their performance on some task by learning from data, rather than being explicitly programmed with rules. Instead of writing “if income > 50k and age < 30 then approve loan,”

you hand the algorithm thousands of past loan decisions and let it figure out the pattern.

- There are three broad paradigms. **Supervised learning** uses labelled data, meaning each input comes with a known correct output. The algorithm learns a mapping from inputs to outputs. **Unsupervised learning** works with unlabelled data and tries to discover hidden structure, like clusters or compressed representations. **Reinforcement learning** learns through trial and error, receiving rewards or penalties for actions taken in an environment (covered in file 04).
- Within supervised learning, **classification** predicts discrete categories (spam or not spam, cat or dog) while **regression** predicts continuous values (house price, temperature tomorrow). The boundary is not always sharp: logistic regression is named “regression” but performs classification.
- A key distinction in probabilistic models is **generative vs discriminative**. A generative model learns the joint distribution  $P(x, y)$ , which means it understands how the data itself is generated. It can produce new samples. A discriminative model learns  $P(y | x)$  directly, focusing only on the boundary between classes. Naive Bayes is generative; logistic regression (file 02) is discriminative. Generative models are more flexible but harder to train well; discriminative models often give better classification accuracy when you have enough data.
- **Naive Bayes** is one of the simplest and most effective classifiers. It applies Bayes’ theorem (from chapter 05) directly:

$$P(C_k | x) = \frac{P(x | C_k) P(C_k)}{P(x)}$$

- The “naive” part is a strong independence assumption: it treats every feature as independent given the class. If you are classifying emails as spam, Naive Bayes assumes the presence of the word “free” tells you nothing about the presence of the word “winner,” once you know the email is spam. This is almost never true in reality, but the classifier works surprisingly well anyway.
- Since  $P(x)$  is the same for all classes, classification simplifies to picking the class that maximises the numerator:

$\hat{y}$

$= \arg$

$\max_k \prod_{i=1}^n$

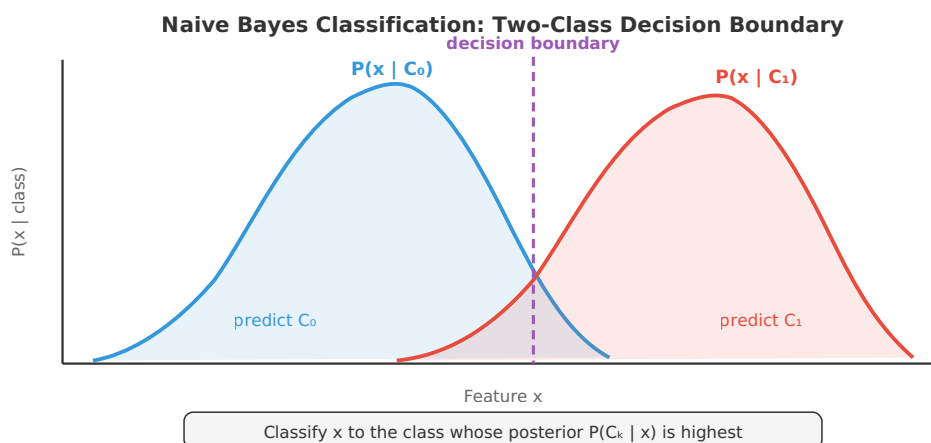
$P(C_k)$

$P(x_i \mid C_k)$

- The prior  $P(C_k)$  is just the fraction of training examples in each class. The likelihoods  $P(x_i \mid C_k)$  depend on what kind of features you have, which gives rise to three common variants.
- **Multinomial Naive Bayes** is designed for count data, like word frequencies in documents. Each feature  $x_i$  represents how many times word  $i$  appears, and the likelihood follows a multinomial distribution. This is the standard choice for text classification, sentiment analysis, and spam filtering.
- **Gaussian Naive Bayes** assumes each feature follows a normal distribution within each class. You estimate the mean  $\mu_{ik}$  and variance  $\sigma_{ik}^2$  of feature  $i$  for class  $k$  from the training data, then compute:

$$P(x_i \mid C_k) = \frac{1}{\sqrt{2\pi\sigma_{ik}^2}} \exp\left(-\frac{(x_i - \mu_{ik})^2}{2\sigma_{ik}^2}\right)$$

- This is the natural choice when your features are continuous measurements, like height, weight, or sensor readings.

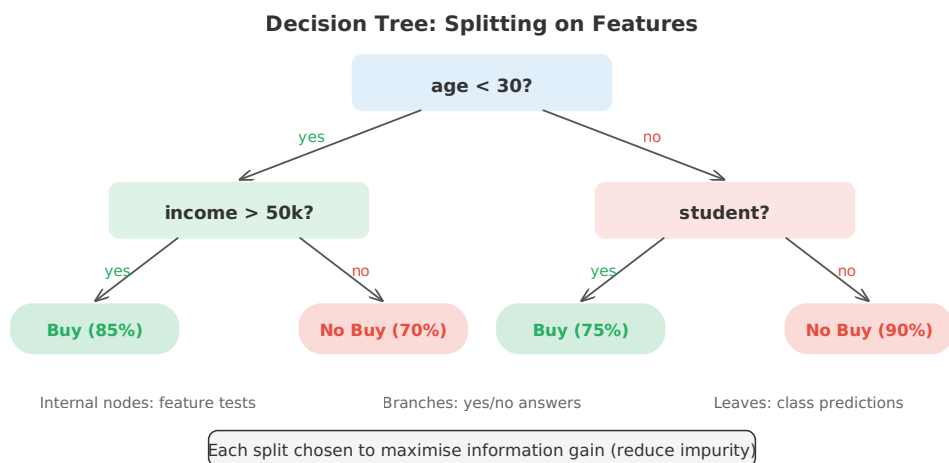


Two overlapping Gaussian class-conditional distributions with a decision boundary where posteriors cross

- **Bernoulli Naive Bayes** models binary features: each feature is either present (1) or absent (0). Instead of counting how many times a word appears, you only track whether it appears at all. This works well for short texts or binary feature vectors.
- A practical problem arises when a feature value never appears with a certain class in training data. The likelihood becomes zero, and because everything is multiplied together, the entire posterior collapses to zero. **Laplace smoothing** fixes this by adding a small count (usually 1) to every feature-class combination:

$$P(x_i | C_k) = \frac{\text{count}(x_i, C_k) + \alpha}{\text{count}(C_k) + \alpha \cdot V}$$

- Here  $\alpha$  is the smoothing parameter (typically 1) and  $V$  is the number of possible values for that feature. This ensures no probability is ever exactly zero.
- **Decision trees** take a completely different approach. Instead of computing probabilities, they partition the feature space through a sequence of yes/no questions. Think of the game Twenty Questions: at each step, you ask the question that narrows down the possibilities the most.
- A tree starts at the root with all training examples. At each internal node, it picks a feature and a threshold to split on (e.g., “is age < 30?”). Examples flow left or right based on the answer. This continues recursively until the leaves, which hold predictions: the majority class for classification, or the mean value for regression.



Decision tree with depth-2 splits on features, yes/no branches, and colored leaf nodes showing class predictions

- The critical question is: which feature should you split on? You want splits that produce the “purest” child nodes, where most examples belong to the same class. Two common measures of impurity are **Gini impurity** and **entropy**.
- **Gini impurity** measures the probability that a randomly chosen sample would be misclassified if labelled according to the distribution in that node:

$$\text{Gini}(S) = 1 - \sum_{k=1}^K p_k^2$$

- If a node is perfectly pure (all one class), Gini is 0. If classes are equally balanced (say 50/50 for two classes), Gini reaches its maximum of 0.5.
- **Entropy** (from chapter 05’ s information theory section) measures the average surprise:

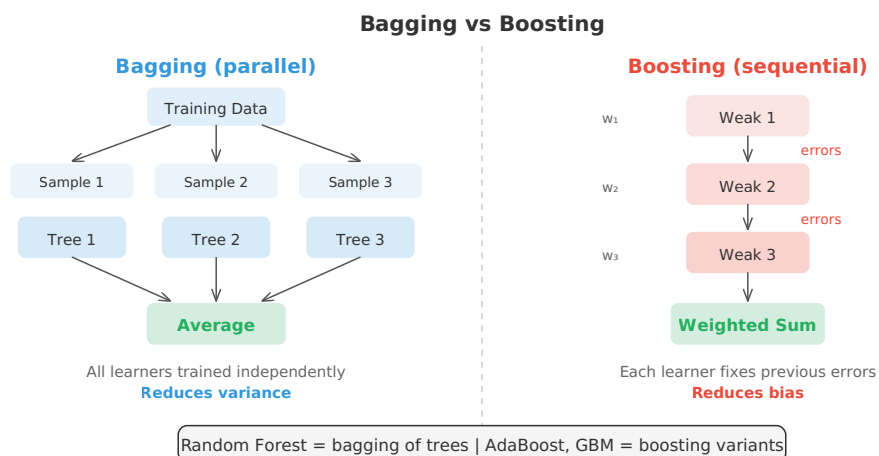
$$H(S) = - \sum_{k=1}^K p_k \log_2 p_k$$

- A pure node has entropy 0. A perfectly balanced binary node has entropy 1 bit. In practice, Gini and entropy give very similar trees; Gini is slightly faster to compute since it avoids the logarithm.
- **Information gain** is the reduction in impurity achieved by a split. For a split that divides set S into subsets  $S_L$  and  $S_R$ :

$$\text{IG}(S, \text{split}) = H(S) - \frac{|S_L|}{|S|} H(S_L) - \frac{|S_R|}{|S|} H(S_R)$$

- The algorithm greedily picks the split with the highest information gain at each node. This is a locally optimal strategy, not globally optimal, but it works well in practice.
- **Regression trees** work the same way, but leaves predict a continuous value (the mean of the examples that reach that leaf) and the splitting criterion uses variance reduction instead of Gini or entropy.

- Left unchecked, a decision tree will keep splitting until every leaf is pure, essentially memorising the training data. This is severe overfitting. **Pruning** combats this. Pre-pruning sets limits before growing the tree: maximum depth, minimum samples per leaf, or minimum information gain to make a split. Post-pruning grows the full tree first, then removes branches that do not improve performance on a validation set.
- A single decision tree is easy to interpret but tends to be unstable: small changes in the data can produce a very different tree. **Ensemble methods** combine many models to get better predictions than any single model could achieve.
- The core idea is the “wisdom of crowds.” If you ask 100 mediocre classifiers and take a majority vote, the ensemble can be excellent, as long as the individual classifiers make somewhat independent errors.
- **Bagging** (bootstrap aggregating) trains multiple models on different random subsets of the data, sampled with replacement (bootstrap samples). Each model sees roughly 63% of the original data. At prediction time, you average the outputs (regression) or take a majority vote (classification). Because each model sees different data, they make different mistakes, and averaging cancels out much of the variance.
- **Random Forests** are bagging applied to decision trees with one extra twist: at each split, the tree only considers a random subset of features (typically  $\sqrt{d}$  features out of  $d$  total). This further decorrelates the trees, making the ensemble even more powerful. Random forests are one of the most reliable off-the-shelf classifiers in all of machine learning.



Side-by-side comparison: bagging trains parallel models and averages,  
boosting trains sequential models that correct previous errors

- **Boosting** takes the opposite philosophy. Instead of training models independently, it trains them sequentially, with each new model focusing on the examples that previous models got wrong.
- **AdaBoost** (Adaptive Boosting) maintains a weight for each training example. Initially all weights are equal. After training a weak learner (often a very shallow decision tree, called a “stump” ), examples that were misclassified get higher weights, so the next learner pays more attention to them. The final prediction is a weighted vote of all learners, where better-performing learners get more say:

$$H(x) = \text{sign} \left( \sum_{t=1}^T \alpha_t h_t(x) \right)$$

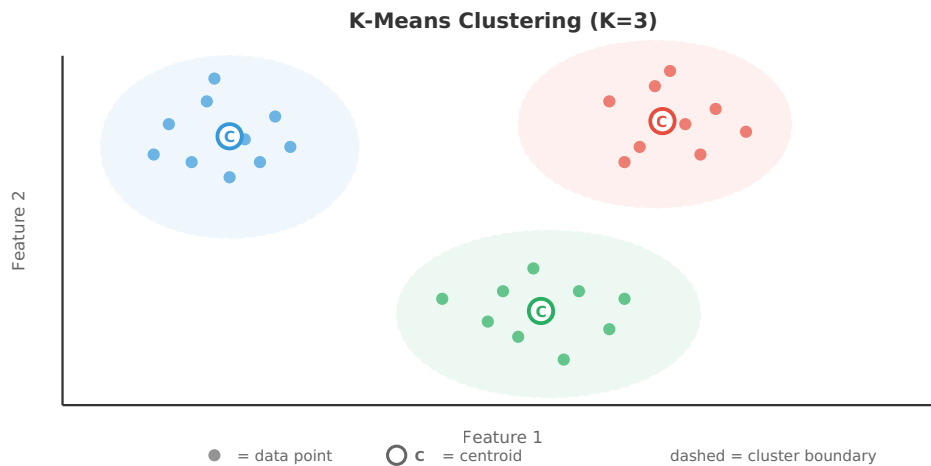
- The weight  $\alpha_t$  for learner  $t$  depends on its error rate  $\epsilon_t$ :

$$\alpha_t = \frac{1}{2} \ln \left( \frac{1 - \epsilon_t}{\epsilon_t} \right)$$

- A learner with low error gets a large positive weight; one performing at chance ( $\epsilon = 0.5$ ) gets zero weight.
- **Gradient Boosting** generalises this idea. Instead of reweighting examples, each new model is trained to predict the residual errors (negative gradient of the loss function) of the combined ensemble so far. For squared error loss, the residuals are literally the differences between predictions and targets. Gradient boosting with decision trees (GBDT) is behind many winning solutions in structured data competitions (XGBoost, LightGBM, CatBoost are popular implementations).
- The key contrast: bagging reduces **variance** (averaging out noise) while boosting reduces **bias** (correcting systematic errors). Bagging works best when individual models overfit; boosting works best when they underfit.
- Shifting to unsupervised learning, **K-Means clustering** is the simplest and most widely used clustering algorithm. Given  $n$  data points and a target number of clusters  $K$ , it assigns each point to one of  $K$  groups by minimising the total distance from each point to its cluster centre.



- The algorithm alternates two steps. First, **assign** each point to the nearest centroid. Second, **update** each centroid to the mean of all points assigned to it. Repeat until assignments stop changing. This is guaranteed to converge because the total within-cluster distance decreases (or stays the same) at every step.



2D scatter plot with three clusters of coloured points, centroid markers, and dashed cluster boundaries

- Formally, K-Means minimises the within-cluster sum of squares, called **inertia**:

$$J = \sum_{k=1}^K \sum_{x \in C_k} \|x - \mu_k\|^2$$

- where  $\mu_k$  is the centroid of cluster  $C_k$ .
- K-Means is sensitive to initialisation. Bad starting centroids can lead to poor local minima. The **K-Means++** initialisation strategy picks the first centroid randomly, then chooses each subsequent centroid with probability proportional to its squared distance from the nearest existing centroid. This spreads out the initial centres and almost always gives better results.
- How do you choose K? Two common tools. The **elbow method** plots inertia vs K and looks for the “elbow” where adding more clusters stops helping much. The **silhouette score** measures how similar a point is to its own cluster compared to the nearest other cluster, ranging from -1 (wrong cluster) to +1

(well clustered). Average silhouette score across all points gives an overall measure of cluster quality.

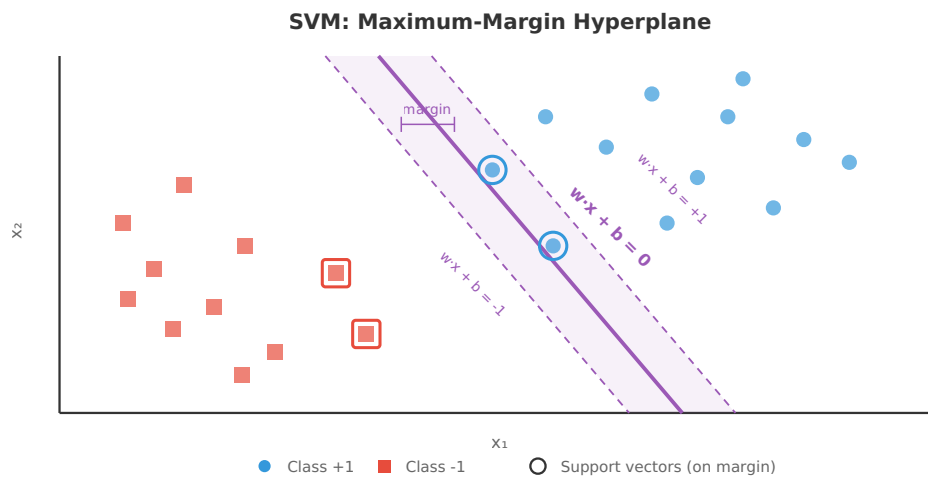
- K-Means has limitations: it assumes spherical clusters of roughly equal size, and it makes “hard” assignments (each point belongs to exactly one cluster).

**Gaussian Mixture Models (GMMs)** relax both restrictions.

- A GMM models the data as a mixture of  $K$  Gaussian distributions, each with its own mean  $\mu_k$ , covariance  $\Sigma_k$ , and mixing weight  $\pi_k$  (where the weights sum to 1):

$$P(x) = \sum_{k=1}^K \pi_k \mathcal{N}(x \mid \mu_k, \Sigma_k)$$

- Instead of hard assignments, each point gets a **soft assignment**: the probability (called the “responsibility”) that it belongs to each cluster. A point near the boundary between two Gaussians might be 60% cluster A and 40% cluster B.
- GMMs are fitted using the **Expectation-Maximisation (EM) algorithm**, which alternates two steps, much like K-Means. The **E-step** computes responsibilities: for each point, what is the probability it came from each Gaussian? The **M-step** updates parameters: given the responsibilities, what are the best means, covariances, and mixing weights? EM is guaranteed to increase the data likelihood at each iteration and converges to a local maximum.
- K-Means is actually a special case of EM with GMMs: it corresponds to spherical Gaussians with equal covariance and hard (0/1) responsibilities.
- **Support Vector Machines (SVMs)** approach classification from a geometric perspective. Given two linearly separable classes, there are infinitely many hyperplanes that separate them. SVM finds the one with the **maximum margin**, the largest possible gap between the hyperplane and the nearest data points from each class.
- The nearest points, the ones sitting right on the edge of the margin, are called **support vectors**. They are the only points that matter for defining the boundary; you could remove all other training points and get the same hyperplane.



Two classes separated by a maximum-margin hyperplane, with margin bands and circled support vectors

- For a linear classifier  $f(x) = w \cdot x + b$ , finding the maximum margin amounts to solving:

$$\min_{w, b} \frac{1}{2} \|w\|^2 \text{ subject to } y_i(w \cdot x_i + b) \geq 1 \text{ for all } i$$

- This is a convex quadratic program, so it has a unique global solution (no local minima to worry about).
- Real data is rarely perfectly separable. **Soft-margin SVM** allows some points to violate the margin by introducing slack variables  $\xi_i \geq 0$ :

$$\min_{w, b, \xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i \text{ subject to } y_i(w \cdot x_i + b) \geq 1 - \xi_i$$

- The hyperparameter  $C$  controls the tradeoff: large  $C$  penalises misclassifications heavily (tighter fit, risk of overfitting), small  $C$  allows more violations (wider margin, more regularised).
- The most powerful feature of SVMs is the **kernel trick**. Many datasets that are not linearly separable in the original feature space become separable when

mapped to a higher-dimensional space. The kernel trick lets you compute dot products in that high-dimensional space without ever explicitly computing the transformation.

- A kernel function  $K(x_i, x_j) = \phi(x_i) \cdot \phi(x_j)$  replaces every dot product in the SVM optimisation. The most popular kernel is the **Radial Basis Function (RBF)** kernel:

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$$

- The RBF kernel implicitly maps data to an infinite-dimensional space. The parameter  $\gamma$  controls how far the influence of a single training point reaches: large  $\gamma$  means each point only influences its immediate neighbourhood (risk of overfitting), small  $\gamma$  gives smoother boundaries.
- Other common kernels include the polynomial kernel  $K(x_i, x_j) = (x_i \cdot x_j + c)^d$  and the linear kernel  $K(x_i, x_j) = x_i \cdot x_j$  (which is just the standard SVM without any transformation).
- In practice, SVMs with RBF kernels were the dominant classifier before deep learning took over. They still work well on small-to-medium datasets, especially when the number of features is large relative to the number of samples.
- The SVM's connection to chapter 02 (matrices) runs deep. The optimisation is typically solved in its dual form, where the solution depends only on dot products between training examples, which is exactly what makes the kernel trick possible. The entire algorithm operates in the language of inner products and linear algebra.
- To summarise the classical ML toolkit:

Algorithm	Type	Key Strength	Key Weakness
Naive Bayes	Supervised (generative)	Fast, works with little data	Independence assumption
Decision Tree	Supervised	Interpretable	Overfits easily
Random Forest	Supervised (ensemble)	Robust, few hyperparameters	Less interpretable
Gradient Boosting			

Algorithm	Type	Key Strength	Key Weakness
	Supervised (ensemble)	State-of-the-art on tabular data	Slower, more tuning
K-Means	Unsupervised (clustering)	Simple, scalable	Assumes spherical clusters
GMM	Unsupervised (clustering)	Soft assignments, flexible shapes	Sensitive to initialisation
SVM	Supervised	Effective in high dimensions	Slow on large datasets

## 26.1 Coding Tasks (use CoLab or notebook)

1. Implement Gaussian Naive Bayes from scratch. Train on synthetic 2D data with two classes and visualise the decision boundary. Compare with scikit-learn's implementation.

```

import jax.numpy as jnp
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification

Generate synthetic data
X, y = make_classification(n_samples=300, n_features=2, n_redundant=0,
 n_informative=2, n_clusters_per_class=1,
 random_state=42)
X, y = jnp.array(X), jnp.array(y)

Fit Gaussian Naive Bayes from scratch
classes = jnp.unique(y)
params = {}
for c in classes:
 c = int(c)
 mask = y == c
 X_c = X[mask]
 params[c] = {
 'mean': jnp.mean(X_c, axis=0),
 'var': jnp.var(X_c, axis=0),
 'prior': jnp.sum(mask) / len(y)
 }

def gaussian_log_likelihood(x, mean, var):

```

```

 return -0.5 * jnp.sum(jnp.log(2 * jnp.pi * var) + (x - mean)**2 / var)

def predict(X):
 preds = []
 for x in X:
 log_posts = []
 for c in [0, 1]:
 log_post = jnp.log(params[c]['prior']) + gaussian_log_likelihood(
 x, params[c]['mean'], params[c]['var'])
 log_posts.append(log_post)
 preds.append(jnp.argmax(jnp.array(log_posts)))
 return jnp.array(preds)

Decision boundary visualisation
xx, yy = jnp.meshgrid(jnp.linspace(X[:,0].min()-1, X[:,0].max()+1, 200),
 jnp.linspace(X[:,1].min()-1, X[:,1].max()+1, 200))
grid = jnp.column_stack([xx.ravel(), yy.ravel()])
zz = predict(grid).reshape(xx.shape)

plt.figure(figsize=(8, 6))
plt.contourf(xx, yy, zz, alpha=0.3, cmap='coolwarm')
plt.scatter(X[y==0, 0], X[y==0, 1], c='#3498db', label='Class 0',
 edgecolors='k', s=20)
plt.scatter(X[y==1, 0], X[y==1, 1], c='#e74c3c', label='Class 1',
 edgecolors='k', s=20)
plt.title("Gaussian Naive Bayes Decision Boundary")
plt.legend()
plt.grid(alpha=0.3)
plt.show()

accuracy = jnp.mean(predict(X) == y)
print(f"Training accuracy: {accuracy:.2%}")

```

1. Build a decision tree that splits using Gini impurity. Implement the splitting logic for a single node and show how information gain selects the best feature and threshold.

```

import jax.numpy as jnp

def gini_impurity(y):
 """Gini impurity of a label array."""
 classes, counts = jnp.unique(y, return_counts=True)
 probs = counts / len(y)
 return 1.0 - jnp.sum(probs ** 2)

def information_gain(y, left_mask):

```

```

 """IG from splitting y into left/right by boolean mask."""
 parent_gini = gini_impurity(y)
 left_y, right_y = y[left_mask], y[~left_mask]
 n = len(y)
 if len(left_y) == 0 or len(right_y) == 0:
 return 0.0
 child_gini = (len(left_y)/n) * gini_impurity(left_y) + \
 (len(right_y)/n) * gini_impurity(right_y)
 return float(parent_gini - child_gini)

def best_split(X, y):
 """Find the feature and threshold that maximise information gain."""
 best_ig, best_feat, best_thresh = -1, None, None
 for feat in range(X.shape[1]):
 thresholds = jnp.unique(X[:, feat])
 for thresh in thresholds:
 mask = X[:, feat] <= float(thresh)
 ig = information_gain(y, mask)
 if ig > best_ig:
 best_ig, best_feat, best_thresh = ig, feat, float(thresh)
 return best_feat, best_thresh, best_ig

Example: synthetic data
from sklearn.datasets import make_classification
X, y = make_classification(n_samples=100, n_features=4, n_redundant=0,
random_state=0)
X, y = jnp.array(X), jnp.array(y)

feat, thresh, ig = best_split(X, y)
print(f"Best split: feature {feat}, threshold {thresh:.3f}, info gain {ig:.4f}")
print(f"Parent Gini: {gini_impurity(y):.4f}")
mask = X[:, feat] <= thresh
print(f"Left Gini: {gini_impurity(y[mask]):.4f} ({int(jnp.sum(mask))}
samples)")
print(f"Right Gini: {gini_impurity(y[~mask]):.4f} ({int(jnp.sum(~mask))}
samples)")

```

1. Implement K-Means from scratch with K-Means++ initialisation. Cluster a synthetic dataset and visualise the clusters at each iteration.

```

import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs

Generate synthetic clusters

```

```

X, y_true = make_blobs(n_samples=300, centers=4, cluster_std=0.8,
random_state=42)
X = jnp.array(X)

def kmeans_plus_plus_init(X, K, key):
 """K-Means++ initialisation."""
 n = X.shape[0]
 idx = jax.random.randint(key, (), 0, n)
 centroids = [X[idx]]
 for _ in range(1, K):
 dists = jnp.min(jnp.stack([jnp.sum((X - c)**2, axis=1) for c in
centroids]), axis=0)
 probs = dists / jnp.sum(dists)
 key, subkey = jax.random.split(key)
 idx = jax.random.choice(subkey, n, p=probs)
 centroids.append(X[idx])
 return jnp.stack(centroids)

def kmeans(X, K, max_iters=20, key=jax.random.PRNGKey(0)):
 centroids = kmeans_plus_plus_init(X, K, key)
 history = [centroids]
 for _ in range(max_iters):
 # Assign step
 dists = jnp.stack([jnp.sum((X - c)**2, axis=1) for c in centroids])
 labels = jnp.argmin(dists, axis=0)
 # Update step
 new_centroids = jnp.stack([
 jnp.mean(X[labels == k], axis=0) for k in range(K)
])
 history.append(new_centroids)
 if jnp.allclose(centroids, new_centroids):
 break
 centroids = new_centroids
 return labels, centroids, history

K = 4
labels, centroids, history = kmeans(X, K)

Plot final result
colors = ['#3498db', '#e74c3c', '#27ae60', '#9b59b6']
plt.figure(figsize=(8, 6))
for k in range(K):
 mask = labels == k
 plt.scatter(X[mask, 0], X[mask, 1], c=colors[k], s=20, alpha=0.6)
 plt.scatter(centroids[k, 0], centroids[k, 1], c=colors[k], marker='X',
 s=200, edgecolors='k', linewidths=1.5)
plt.title(f"K-Means Clustering (K={K}, {len(history)-1} iterations)")

```



```
plt.grid(alpha=0.3)
plt.show()

Compute inertia
inertia = sum(jnp.sum((X[labels == k] - centroids[k])**2) for k in range(K))
print(f"Final inertia: {inertia:.2f}")
```

1. Demonstrate the kernel trick. Show that an RBF kernel computes dot products in a high-dimensional space by comparing the kernel matrix with explicit feature mapping for a polynomial kernel.

```
import jax.numpy as jnp

Simple 2D data
X = jnp.array([[1.0, 2.0], [3.0, 4.0], [5.0, 6.0]])

Polynomial kernel: $K(x,y) = (x \cdot y + 1)^2$
def poly_kernel(X, degree=2, c=1.0):
 return (X @ X.T + c) ** degree

Explicit degree-2 feature map for 2D: $(1, \sqrt{2} \cdot x_1, \sqrt{2} \cdot x_2, x_1^2, x_2^2, \sqrt{2} \cdot x_1 \cdot x_2)$
def poly_features(X):
 x1, x2 = X[:, 0], X[:, 1]
 return jnp.column_stack([
 jnp.ones(len(X)),
 jnp.sqrt(2) * x1,
 jnp.sqrt(2) * x2,
 x1 ** 2,
 x2 ** 2,
 jnp.sqrt(2) * x1 * x2
])

K_trick = poly_kernel(X)
phi = poly_features(X)
K_explicit = phi @ phi.T

print("Kernel trick (polynomial degree 2):")
print(K_trick)
print("\nExplicit feature map dot products:")
print(K_explicit)
print(f"\nMatrices match: {jnp.allclose(K_trick, K_explicit)}")

RBF kernel: no finite explicit map exists
def rbf_kernel(X, gamma=0.5):
 sq_dists = jnp.sum(X**2, axis=1, keepdims=True) + \
```

```

 jnp.sum(X**2, axis=1) - 2 * X @ X.T
 return jnp.exp(-gamma * sq_dists)

K_rbf = rbf_kernel(X)
print("\nRBF kernel matrix:")
print(K_rbf)
print("Diagonal is always 1 (a point is identical to itself)")
print("Off-diagonal entries decay with distance")

```

## 27 Gradient Machine Learning

Gradient-based learning optimises model parameters by iteratively following the slope of a loss surface. This file covers linear regression, logistic regression, softmax classification, gradient descent variants, regularisation (L1/L2), and the bias-variance tradeoff

- The classical methods in file 01 use clever heuristics or closed-form solutions. This file covers algorithms that learn by following gradients, taking small steps downhill on a loss surface until they find good parameters. Gradient-based learning is the engine behind everything from linear regression to the largest neural networks.
- **Linear regression** is the simplest gradient-based model, and it also has a closed-form solution, which makes it a perfect starting point. The model is a line (or hyperplane in higher dimensions):

↺

y

$$= w \cdot x + b =$$

d

$\Sigma$

i=1

$$w_i x_i + b$$

- In matrix notation (from chapter 02), if we stack all training inputs as rows of a matrix X and absorb the bias into w by appending a column of ones, this becomes

↺

y

$$= Xw.$$

- The goal is to minimise the **mean squared error (MSE)**, the average squared difference between predictions and actual values:

$$\mathcal{L}(w) = \frac{1}{n}$$

$$\sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$\frac{1}{n} \|y - Xw\|^2$$

- Why squared error? It has a probabilistic justification: if you assume the targets are generated as  $y = Xw + \epsilon$  where  $\epsilon \sim \mathcal{N}(0, \sigma^2)$ , then maximising the Gaussian likelihood of the data (chapter 05) is equivalent to minimising MSE. Squared error also penalises large mistakes more than small ones, which is often desirable.



Scatter of data points with a best-fit line and dashed vertical residual lines showing the errors

- Because MSE is a quadratic function of  $w$ , it has a unique global minimum that we can find analytically. Taking the derivative, setting it to zero, and solving gives the **normal equation**:

$$w^* = (X^T X)^{-1} X^T y$$

- This directly uses the matrix inverse from chapter 02. The expression  $X^T X$  is a  $d \times d$  matrix (where  $d$  is the number of features), and  $X^T y$  is a  $d$ -dimensional vector. The normal equation gives the exact optimal weights in one shot.

- When does the normal equation fail? When  $X^T X$  is singular (not invertible), which happens if features are linearly dependent or if you have more features than samples ( $d > n$ ). In these cases you need regularisation (covered later) or gradient descent.
- **Logistic regression** adapts the linear model for binary classification. Instead of predicting a continuous value, we want a probability between 0 and 1. The **sigmoid function** squashes any real number into this range:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- The model computes  $z = w \cdot x + b$  (a linear score, just like linear regression) and then passes it through the sigmoid:

⦿

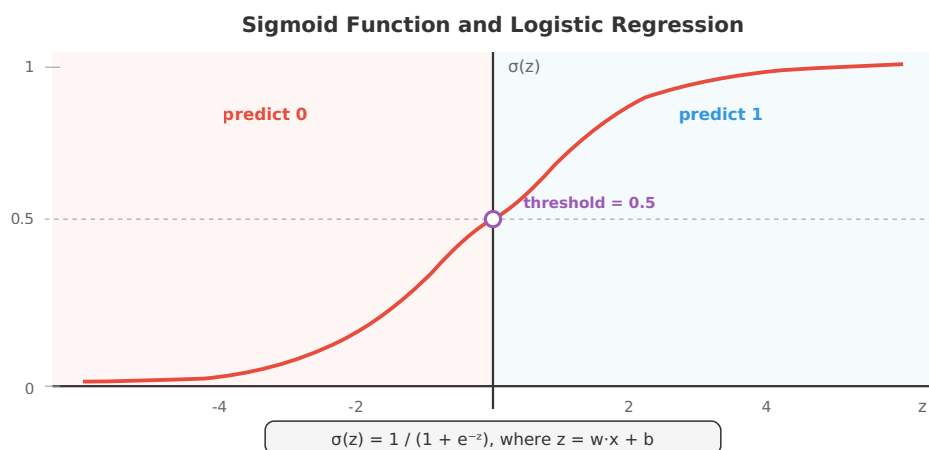
y

=  $\sigma(w \cdot x + b)$ . The output

⦿

y

is interpreted as  $P(y = 1 \mid x)$ .



Sigmoid curve with 0.5 threshold marked, showing classification regions for predict 0 and predict 1

- The sigmoid has nice properties:  $\sigma(0) = 0.5$ ,  $\sigma(z) \rightarrow 1$  as  $z \rightarrow \infty$ ,  $\sigma(z) \rightarrow 0$  as  $z \rightarrow -\infty$ , and its derivative has the elegant form  $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ .
- The loss function for logistic regression is **binary cross-entropy (BCE)**, which comes directly from the Bernoulli likelihood (chapter 05):

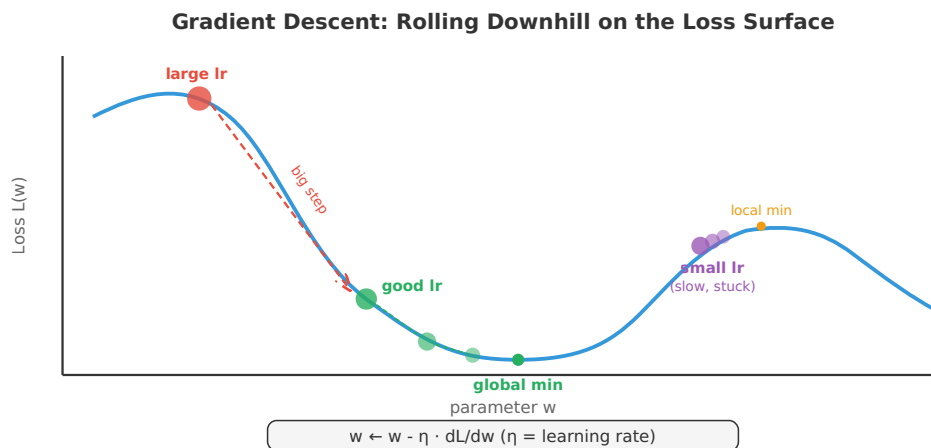
$$\mathcal{L} = -\frac{1}{n}$$

$$\sum_{i=1}^n [y_i \log(y_i) + (1 - y_i) \log(1 - y_i)]$$

- When the true label is 1, only the first term is active and it penalises low predictions. When the true label is 0, only the second term is active and it penalises high predictions. The logarithm makes the penalty extremely steep for confident wrong predictions: predicting 0.01 when the true label is 1 costs much more than predicting 0.4.
- Unlike MSE for linear regression, there is no closed-form solution for the BCE-minimising weights. We need an iterative approach: **gradient descent**.
- The intuition behind gradient descent is simple: imagine you are standing on a hilly landscape (the loss surface) in fog. You cannot see the global minimum, but you can feel the slope under your feet. You take a step downhill, feel the slope again, and repeat. Eventually you reach a valley.

$$w \leftarrow w - \eta \frac{\partial \mathcal{L}}{\partial w}$$

- The learning rate  $\eta$  controls your step size. Too large and you overshoot valleys, bouncing around without converging. Too small and you inch along painfully slowly, possibly getting stuck in a local minimum.



1D loss curve with three balls: large learning rate overshoots, good learning rate converges, small learning rate gets stuck

- The gradient  $\frac{\partial \mathcal{L}}{\partial w}$  is a vector pointing in the direction of steepest ascent. We subtract it because we want to go downhill. This is the chain rule from chapter 03 applied to the loss function.
- **Batch gradient descent** computes the gradient using the entire training set at every step. This gives an exact gradient but is expensive when  $n$  is large.
- **Stochastic gradient descent (SGD)** uses a single random example per step. The gradient is noisy (it estimates the true gradient from one sample) but each step is extremely fast. The noise can actually help escape shallow local minima.
- **Mini-batch gradient descent** splits the difference: use a batch of  $B$  examples (typically 32, 64, or 256) per step. This balances computational efficiency (vectorised operations on the batch) with gradient quality. Almost all deep learning uses mini-batch SGD.
- **Backpropagation** is how we actually compute gradients in models with many parameters, like neural networks. It is the chain rule from chapter 03 applied systematically through a computational graph.
- Any model can be represented as a directed acyclic graph of operations: inputs flow in, get multiplied by weights, added together, passed through nonlinear functions, and eventually produce a loss value. The **forward pass** computes the output (and loss) by flowing data through this graph from input to output.
- The **backward pass** (backpropagation) flows gradients in reverse. Starting from the loss, you compute how the loss changes with respect to each intermediate

value, using the chain rule at every node. If  $L$  depends on  $z$  which depends on  $w$ , then:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial w}$$

- Each node only needs to know its own local derivative and the gradient flowing in from above. This makes backpropagation modular and efficient: the cost is roughly twice the forward pass (one pass forward, one backward).
- Vanilla SGD has a problem: it oscillates in directions with steep curvature while making slow progress in flat directions. **Optimisers** improve on this by adapting the step based on gradient history.
- **SGD with momentum** keeps a running average of past gradients (an exponential moving average, from chapter 04). This smooths out oscillations and accelerates progress along consistent directions:

$$\begin{aligned} v_t &= \beta v_{t-1} + (1 - \beta) \nabla \mathcal{L} \\ w &\leftarrow w - \eta v_t \end{aligned}$$

- Think of a ball rolling downhill: momentum lets it build up speed in a consistent direction and dampens the side-to-side jitter. The typical value is  $\beta = 0.9$ .
- **Nesterov Accelerated Gradient (NAG)** is a small but clever tweak: instead of computing the gradient at the current position, compute it at the “look-ahead” position  $w - \eta \beta v_{t-1}$ . This corrective step reduces overshooting:

$$\begin{aligned} v_t &= \beta v_{t-1} + \nabla \mathcal{L}(w - \eta \beta v_{t-1}) \\ w &\leftarrow w - \eta v_t \end{aligned}$$

- **Adagrad** adapts the learning rate per parameter. Parameters that receive large gradients get smaller learning rates, and vice versa. It accumulates the squared gradients:

$$G_t = G_{t-1} + g_t^2, \quad w \leftarrow w - \frac{\eta}{\sqrt{G_t + \epsilon}} g_t$$

- The problem:  $G_t$  only grows, so the effective learning rate monotonically decreases and eventually becomes too small to learn anything.

- **RMSprop** fixes this by using an exponential moving average of squared gradients instead of a sum, so recent gradients matter more than ancient ones:

$$s_t = \beta s_{t-1} + (1 - \beta)g_t^2, \quad w \leftarrow w - \frac{\eta}{\sqrt{s_t + \epsilon}} g_t$$

- **Adam** (Adaptive Moment Estimation) combines momentum and RMSprop. It maintains both a first-moment estimate (mean of gradients, like momentum) and a second-moment estimate (mean of squared gradients, like RMSprop):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2)g_t^2$$

- Since  $m_t$  and  $v_t$  are initialised at zero, they are biased toward zero in early steps. Bias correction fixes this:

$\hat{m}$

$t =$

$$\frac{m_t}{1 - \beta_1^t}, \quad \hat{v}$$

$$t = \frac{v_t}{1 - \beta_2^t}$$

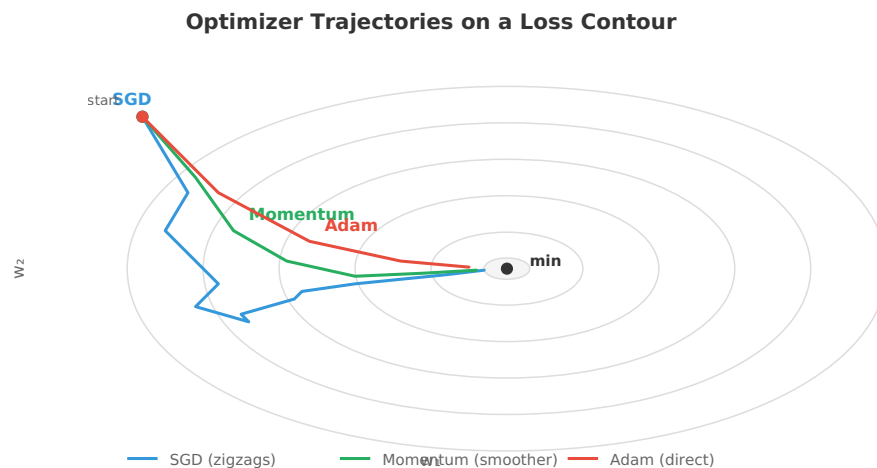
$$w \leftarrow w - \frac{\eta}{\sqrt{\hat{v}}}$$

$$t + \epsilon$$

$\hat{m}$

$t$





2D contour plot showing SGD zigzagging, Momentum following a smoother path, and Adam taking the most direct route to the minimum

- Default hyperparameters ( $\beta_1 = 0.9$ ,  $\beta_2 = 0.999$ ,  $\epsilon = 10^{-8}$ ) work well across a wide range of problems, which is why Adam is the default optimiser in most deep learning work.
- **AdamW** decouples weight decay from the gradient update. Standard L2 regularisation and weight decay are equivalent for SGD but not for Adam. AdamW applies weight decay directly to the parameters rather than adding  $\lambda w$  to the gradient. This gives better generalisation and is now the standard in transformer training:

$$w \leftarrow w - \eta \left( \frac{g_t}{\sqrt{\frac{m_t}{t} + \epsilon}} + \lambda w \right)$$

- **LION** (EvoLved Sign Momentum) is a newer optimiser discovered through program search. It uses only the sign of the momentum update (not the magnitude), which makes each update uniform in scale. LION uses less memory than Adam (no second-moment buffer) and can match or beat Adam on many tasks:

$$w \leftarrow w - \eta \cdot \text{sign}(\beta_1 m_{t-1} + (1 - \beta_1) g_t)$$

$$\mathbf{m}_t = \beta_2 \mathbf{m}_{t-1} + (1 - \beta_2) \mathbf{g}_t$$

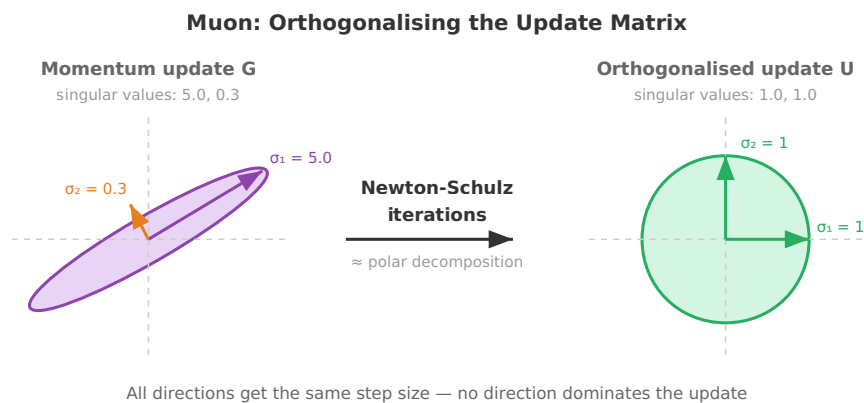
- **Muon** (Momentum + Orthogonalisation) applies Nesterov momentum and then orthogonalises the update matrix using Newton-Schulz iterations, which approximate the polar decomposition. The resulting update direction lies on the Stiefel manifold, every update has roughly equal magnitude across all singular directions, preventing any single direction from dominating. This removes the need for adaptive second-moment estimates (no  $\mathbf{v}_t$  buffer like Adam), reducing memory. Muon has shown strong results on transformer training, often matching AdamW quality at faster convergence, particularly for the attention and MLP weight matrices. Embedding and output layers are typically still handled by AdamW.

$$\mathbf{G}_t = \text{NesterovMomentum}(\nabla \mathcal{L})$$

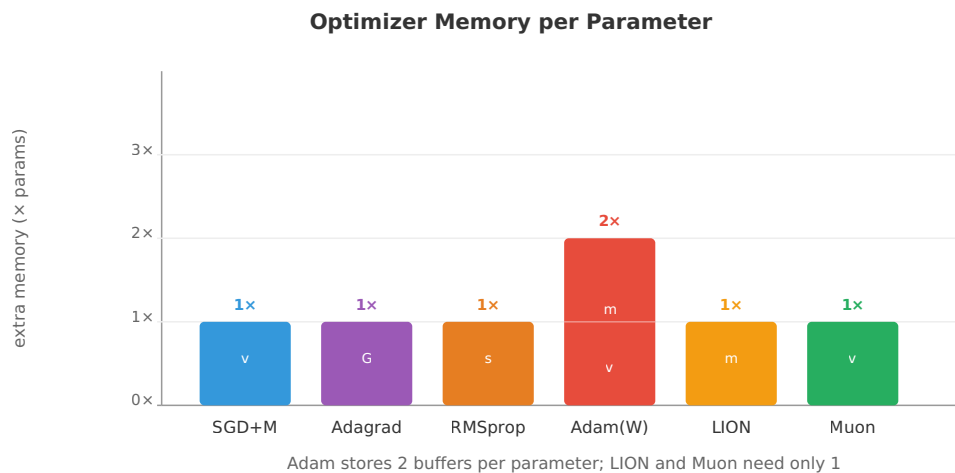
$$\mathbf{U}_t = \text{NewtonSchulz}(\mathbf{G}_t) \approx \mathbf{G}_t (\mathbf{G}_t^T \mathbf{G}_t)^{-1/2}$$

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \mathbf{U}_t$$

- The Newton-Schulz iteration computes the orthogonal factor by repeating  $\mathbf{X}_{k+1} = \frac{1}{2} \mathbf{X}_k (3\mathbf{I} - \mathbf{X}_k^T \mathbf{X}_k)$  for a few steps (typically 5-10). This avoids the cost of a full SVD while giving a good approximation.



Muon orthogonalisation: momentum update has skewed singular values, Newton-Schulz iterations equalise them so all directions update uniformly



Optimizer memory comparison: what each optimizer stores per parameter

- Beyond MSE and BCE, several other **loss functions** are commonly used.

- **Mean Absolute Error (MAE)**, or L1 loss, takes the average of absolute

differences:  $\frac{1}{n} \sum |$

$y_i - y$

$|_i|$ .

It is more robust to outliers than MSE because it does not square large errors.

- **Huber loss** combines the best of both: it behaves like MSE for small errors (smooth, easy to optimise) and like MAE for large errors (robust to outliers). It has a threshold  $\delta$  that controls the transition.

- **Categorical cross-entropy (CCE)** generalises BCE to multiple classes. If

$y$

$y_k$

is the predicted probability for class  $k$  and the true class is  $c$ :

$\mathcal{L} = -\log(y_c)$

- This is just the negative log-probability of the correct class. Minimising cross-entropy is equivalent to maximising the likelihood, which connects back to the information theory in chapter 05: cross-entropy measures how many extra bits you need when using your predicted distribution instead of the true distribution.

- **Hinge loss** is used by SVMs:  $\mathcal{L} = \max(0, 1 - y \cdot f(x))$ . It only penalises predictions that are on the wrong side of the margin or within the margin. Once a point is correctly classified with sufficient confidence, the loss is zero.
- **Regularisation** prevents overfitting by adding a penalty for complex models. The regularised loss is:

$$\mathcal{L}_{\text{reg}} = \mathcal{L}_{\text{data}} + \lambda R(w)$$

- **L2 regularisation** (Ridge, weight decay) penalises the sum of squared weights:  $R(w) = \|w\|^2 = \sum w_i^2$ . It discourages any single weight from becoming too large, effectively shrinking all weights toward zero but rarely making them exactly zero.
- **L1 regularisation** (Lasso) penalises the sum of absolute weights:  $R(w) = \|w\|_1 = \sum |w_i|$ . It encourages sparsity, driving many weights to exactly zero, which performs automatic feature selection.
- **Elastic Net** combines both:  $R(w) = \alpha \|w\|_1 + (1 - \alpha) \|w\|^2$ , blending sparsity and shrinkage.
- There is a beautiful Bayesian interpretation (from chapter 05). L2 regularisation is equivalent to placing a Gaussian prior on the weights and finding the MAP estimate. L1 regularisation corresponds to a Laplace prior. The regularisation strength  $\lambda$  controls how much you trust the prior relative to the data.
- **Evaluation metrics** tell you whether your model is actually working. For regression, MSE and MAE are standard. For classification, things are more nuanced.
- A **confusion matrix** is a table of four counts for binary classification:
  - True Positive (TP): predicted positive, actually positive
  - False Positive (FP): predicted positive, actually negative
  - True Negative (TN): predicted negative, actually negative
  - False Negative (FN): predicted negative, actually positive

· **Accuracy** =  $\frac{TP + TN}{TP + TN + FP + FN}$  can be misleading when classes are imbalanced.

If 99% of emails are not spam, a model that always predicts “not spam” has 99% accuracy but is useless.

- **Precision** =  $\frac{TP}{TP + FP}$  answers: of all predicted positives, how many are actually positive? High precision means few false alarms.
- **Recall** (sensitivity) =  $\frac{TP}{TP + FN}$  answers: of all actual positives, how many did you catch? High recall means few missed cases.
- **F1 score** =  $\frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$  is the harmonic mean of precision and recall, balancing both.
- The **ROC curve** plots the true positive rate (recall) against the false positive rate ( $\frac{FP}{FP + TN}$ ) as you vary the classification threshold from 0 to 1. A perfect classifier hugs the top-left corner. The **AUC** (area under the ROC curve) summarises performance in a single number: 1.0 is perfect, 0.5 is random guessing.
- **Cross-validation** provides a more reliable estimate of generalisation performance. In k-fold cross-validation, you split the data into k folds, train on k - 1 of them, test on the remaining fold, and rotate. The average test performance across all k folds is your estimate. This uses all data for both training and testing (just never at the same time), which is especially valuable when data is scarce.
- The **bias-variance tradeoff** (from chapter 04) is the fundamental tension in ML. A model's expected error decomposes into:

$$\text{Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Noise}$$

- **Bias** is systematic error from wrong assumptions (e.g., fitting a line to curved data). **Variance** is sensitivity to training data fluctuations (e.g., a degree-20 polynomial fitting noise). Simple models have high bias and low variance; complex models have low bias and high variance. The sweet spot minimises total error.
- **Learning rate scheduling** adjusts  $\eta$  during training. Common strategies:
  - Step decay: multiply  $\eta$  by a factor (e.g., 0.1) every N epochs
  - Cosine annealing: smoothly decrease  $\eta$  following a cosine curve from the initial value to near zero

- Warmup: start with a very small  $\eta$  and linearly increase it for the first few thousand steps, then decay. This prevents large initial gradients from destabilising training
- 1cycle: one cosine cycle up then down, which can give faster convergence
- **Hyperparameter tuning** is the process of finding good values for learning rate, batch size, regularisation strength, and other settings that are not learned by gradient descent. Common approaches:
  - Grid search: try every combination on a predefined grid (exhaustive but expensive)
  - Random search: sample combinations randomly, which is often more efficient because not all hyperparameters matter equally
  - Bayesian optimisation: build a model of the objective function and intelligently choose the next hyperparameters to try
  - **ASHA** (Asynchronous Successive Halving Algorithm): runs many trials in parallel with small budgets, then promotes the most promising ones to larger budgets while killing the rest early. It combines the efficiency of early stopping with massive parallelism — instead of running 100 full training runs, start all 100 cheaply, keep the top quarter at each rung, and only a handful run to completion. This is the backbone of modern large-scale tuning frameworks like Ray Tune.
- **Schedule-free learning** eliminates the need for a learning rate schedule altogether. Instead of decaying  $\eta$  on a fixed curve, it maintains two sequences: a slow-moving average of iterates  $z_t$  (which converges to the optimum) and a fast exploratory iterate  $y_t$  (where gradients are evaluated). The final output is the averaged sequence, which provably matches the convergence rate of the best schedule in hindsight. This removes the schedule as a hyperparameter entirely — you only set the base learning rate and the optimizer handles the rest. Schedule-free variants of both SGD and Adam have been shown to match or exceed their tuned-schedule counterparts.

## 27.1 Coding Tasks (use CoLab or notebook)

---

1. Implement linear regression with both the normal equation and gradient descent. Compare the solutions and plot the convergence of the GD loss over iterations.

```

import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

Generate synthetic data: $y = 3x + 2 + \text{noise}$
key = jax.random.PRNGKey(42)
n = 100
X = jax.random.uniform(key, (n, 1), minval=0, maxval=10)
y = 3 * X[:, 0] + 2 + jax.random.normal(key, (n,)) * 1.5

Add bias column
X_b = jnp.column_stack([X, jnp.ones(n)])

Normal equation
w_exact = jnp.linalg.solve(X_b.T @ X_b, X_b.T @ y)
print(f"Normal equation: w={w_exact[0]:.4f}, b={w_exact[1]:.4f}")

Gradient descent
w_gd = jnp.zeros(2)
lr = 0.005
losses = []
for step in range(500):
 pred = X_b @ w_gd
 error = pred - y
 loss = jnp.mean(error ** 2)
 losses.append(float(loss))
 grad = (2 / n) * X_b.T @ error
 w_gd = w_gd - lr * grad

print(f"Gradient descent: w={w_gd[0]:.4f}, b={w_gd[1]:.4f}")

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].scatter(X[:, 0], y, s=15, alpha=0.5, color='#3498db')
axes[0].plot([0, 10], [w_exact[1], w_exact[0]*10 + w_exact[1]], color='#e74c3c',
linewidth=2)
axes[0].set_title("Linear Regression Fit")
axes[0].set_xlabel("x"); axes[0].set_ylabel("y")

axes[1].plot(losses, color='#27ae60', linewidth=1.5)
axes[1].set_title("GD Loss Convergence")
axes[1].set_xlabel("Step"); axes[1].set_ylabel("MSE")
axes[1].set_yscale('log')

```

```
plt.tight_layout()
plt.show()
```

1. Implement logistic regression from scratch with gradient descent. Train on a 2D dataset and visualise the learned decision boundary.

```
import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons

Generate data
X, y = make_moons(n_samples=300, noise=0.2, random_state=42)
X, y = jnp.array(X), jnp.array(y, dtype=jnp.float32)

def sigmoid(z):
 return 1 / (1 + jnp.exp(-z))

Add bias column
X_b = jnp.column_stack([X, jnp.ones(len(X))])
w = jnp.zeros(3)
lr = 0.5
losses = []

for step in range(2000):
 z = X_b @ w
 pred = sigmoid(z)
 # BCE loss
 loss = -jnp.mean(y * jnp.log(pred + 1e-8) + (1 - y) * jnp.log(1 - pred +
1e-8))
 losses.append(float(loss))
 # Gradient
 grad = X_b.T @ (pred - y) / len(y)
 w = w - lr * grad

Decision boundary
xx, yy = jnp.meshgrid(jnp.linspace(-2, 3, 200), jnp.linspace(-1.5, 2, 200))
grid = jnp.column_stack([xx.ravel(), yy.ravel(), jnp.ones(xx.size)])
zz = sigmoid(grid @ w).reshape(xx.shape)

plt.figure(figsize=(8, 6))
plt.contourf(xx, yy, zz, levels=[0, 0.5, 1], alpha=0.3, colors=['#e74c3c',
'#3498db'])
plt.contour(xx, yy, zz, levels=[0.5], colors='#9b59b6', linewidths=2)
plt.scatter(X[y==0, 0], X[y==0, 1], c='#e74c3c', s=15, label='Class 0')
plt.scatter(X[y==1, 0], X[y==1, 1], c='#3498db', s=15, label='Class 1')
```



```
plt.title("Logistic Regression Decision Boundary")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

1. Compare optimiser trajectories on a 2D quadratic surface. Run SGD, SGD+Momentum, and Adam from the same starting point and plot their paths.

```
import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

Elongated quadratic: $L(w_1, w_2) = 0.5*w_1^2 + 10*w_2^2$
def loss_fn(w):
 return 0.5 * w[0]**2 + 10 * w[1]**2

grad_fn = jax.grad(loss_fn)

def run_sgd(w0, lr=0.05, steps=80):
 w = w0.copy()
 path = [w.copy()]
 for _ in range(steps):
 g = grad_fn(w)
 w = w - lr * g
 path.append(w.copy())
 return jnp.stack(path)

def run_momentum(w0, lr=0.05, beta=0.9, steps=80):
 w, v = w0.copy(), jnp.zeros(2)
 path = [w.copy()]
 for _ in range(steps):
 g = grad_fn(w)
 v = beta * v + (1 - beta) * g
 w = w - lr * v
 path.append(w.copy())
 return jnp.stack(path)

def run_adam(w0, lr=0.05, b1=0.9, b2=0.999, eps=1e-8, steps=80):
 w, m, v = w0.copy(), jnp.zeros(2), jnp.zeros(2)
 path = [w.copy()]
 for t in range(1, steps + 1):
 g = grad_fn(w)
 m = b1 * m + (1 - b1) * g
 v = b2 * v + (1 - b2) * g**2
 m_hat = m / (1 - b1**t)
 v_hat = v / (1 - b2**t)
```

```

 w = w - lr * m_hat / (jnp.sqrt(v_hat) + eps)
 path.append(w.copy())
 return jnp.stack(path)

w0 = jnp.array([8.0, 3.0])
sgd_path = run_sgd(w0)
mom_path = run_momentum(w0)
adam_path = run_adam(w0)

Plot
fig, ax = plt.subplots(figsize=(8, 6))
w1 = jnp.linspace(-10, 10, 100)
w2 = jnp.linspace(-4, 4, 100)
W1, W2 = jnp.meshgrid(w1, w2)
L = 0.5 * W1**2 + 10 * W2**2
ax.contour(W1, W2, L, levels=20, cmap='Greys', alpha=0.4)
ax.plot(sgd_path[:,0], sgd_path[:,1], 'o-', color='#3498db', markersize=2,
linewidth=1, label='SGD')
ax.plot(mom_path[:,0], mom_path[:,1], 'o-', color='#27ae60', markersize=2,
linewidth=1, label='Momentum')
ax.plot(adam_path[:,0], adam_path[:,1], 'o-', color='#e74c3c', markersize=2,
linewidth=1, label='Adam')
ax.plot(0, 0, 'k*', markersize=15, label='Minimum')
ax.set_xlabel('w1'); ax.set_ylabel('w2')
ax.set_title("Optimizer Trajectories on Elongated Quadratic")
ax.legend()
plt.grid(alpha=0.3)
plt.show()

```

1. Show the effect of L1 vs L2 regularisation on weight sparsity. Train linear regression with both penalties and compare the resulting weight vectors.

```

import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

Synthetic data: only first 3 of 20 features are relevant
key = jax.random.PRNGKey(0)
n, d = 200, 20
w_true = jnp.zeros(d).at[:3].set(jnp.array([3.0, -2.0, 1.5]))
X = jax.random.normal(key, (n, d))
y = X @ w_true + 0.5 * jax.random.normal(key, (n,))

def train_ridge(X, y, lam=1.0, lr=0.01, steps=2000):
 """L2 regularised linear regression via GD."""
 w = jnp.zeros(X.shape[1])

```

```

 for _ in range(steps):
 pred = X @ w
 grad = (2/len(y)) * X.T @ (pred - y) + 2 * lam * w
 w = w - lr * grad
 return w

def train_lasso(X, y, lam=1.0, lr=0.01, steps=2000):
 """L1 regularised linear regression via proximal GD."""
 w = jnp.zeros(X.shape[1])
 for _ in range(steps):
 pred = X @ w
 grad = (2/len(y)) * X.T @ (pred - y)
 w = w - lr * grad
 # Soft thresholding (proximal operator for L1)
 w = jnp.sign(w) * jnp.maximum(jnp.abs(w) - lr * lam, 0)
 return w

w_l2 = train_lasso(X, y, lam=0.1)
w_l1 = train_lasso(X, y, lam=0.1)

fig, axes = plt.subplots(1, 3, figsize=(14, 4))
axes[0].bar(range(d), w_true, color='#333', alpha=0.7)
axes[0].set_title("True Weights"); axes[0].set_xlabel("Feature")
axes[1].bar(range(d), w_l2, color='#3498db', alpha=0.7)
axes[1].set_title("L2 (Ridge): shrinks all"); axes[1].set_xlabel("Feature")
axes[2].bar(range(d), w_l1, color='#e74c3c', alpha=0.7)
axes[2].set_title("L1 (Lasso): zeros out irrelevant");
axes[2].set_xlabel("Feature")
plt.tight_layout()
plt.show()

print(f"L2 non-zero weights: {int(jnp.sum(jnp.abs(w_l2) > 0.01))}/{d}")
print(f"L1 non-zero weights: {int(jnp.sum(jnp.abs(w_l1) > 0.01))}/{d}")

```

## 28 Deep Learning

Deep learning stacks nonlinear layers to build hierarchical representations that transform raw inputs into useful features automatically. This file covers MLPs, activation functions, backpropagation, CNNs, RNNs, LSTMs, attention, transformers, GANs, VAEs, diffusion models, and normalisation techniques

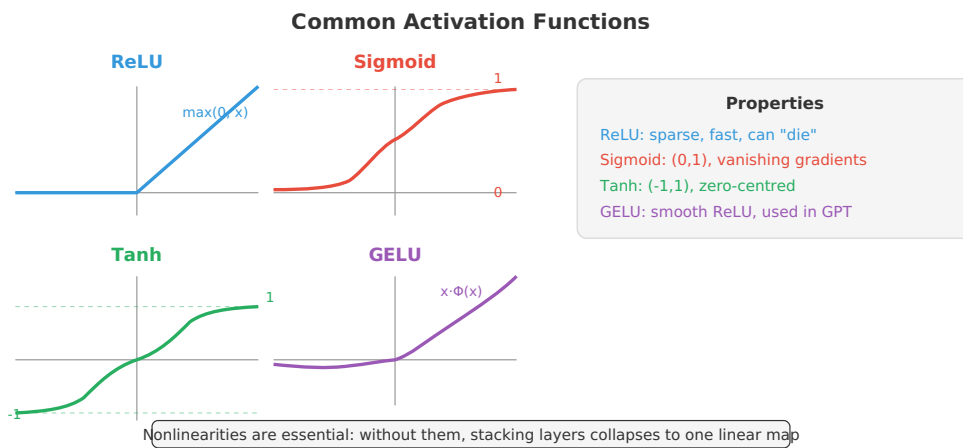
- What makes a network “deep” ? A shallow network has one hidden layer; a deep network has many. Depth lets the network build hierarchical

representations, with early layers learning simple features (edges, tones) and later layers composing them into complex concepts (faces, sentences). This compositionality is what gives deep learning its power.

- The simplest deep network is the **multi-layer perceptron (MLP)**, also called a fully connected or dense network. Each layer computes:

$$h = \sigma(Wx + b)$$

- Here  $W$  is a weight matrix (chapter 02),  $b$  is a bias vector, and  $\sigma$  is a nonlinear activation function. The output of one layer becomes the input to the next. Without the nonlinearity, stacking layers would be pointless:  $W_2(W_1x) = (W_2W_1)x$ , which is just another linear transformation. This is exactly the matrix multiplication collapse from chapter 02.
- **Activation functions** introduce the nonlinearity that makes depth meaningful.
- **ReLU** (Rectified Linear Unit):  $\text{ReLU}(x) = \max(0, x)$ . It is the most widely used activation. It is fast to compute, does not saturate for positive inputs, and produces sparse activations (many neurons output exactly zero). The downside: neurons with negative input always output zero, and if they get stuck there permanently, they “die” and stop learning.
- **Sigmoid**:  $\sigma(x) = \frac{1}{1 + e^{-x}}$ , squashing inputs to  $(0, 1)$ . Useful for output layers in binary classification, but problematic in hidden layers because gradients vanish when the input is far from zero (the curve is nearly flat).
- **Tanh**:  $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$ , squashing to  $(-1, 1)$ . Zero-centred (unlike sigmoid), which helps gradient flow, but still suffers from vanishing gradients at extremes.
- **GELU** (Gaussian Error Linear Unit):  $\text{GELU}(x) = x \cdot \Phi(x)$ , where  $\Phi$  is the standard normal CDF. It is a smooth approximation to ReLU that allows small negative values through. GELU is the default in GPT and BERT.
- **Swish**:  $\text{Swish}(x) = x \cdot \sigma(x)$ , another smooth gate. Similar to GELU in practice.



Side-by-side plots of ReLU, Sigmoid, Tanh, and GELU with their key properties

- A dense layer with  $d_{\text{in}}$  inputs and  $d_{\text{out}}$  outputs has  $d_{\text{in}} \times d_{\text{out}} + d_{\text{out}}$  parameters (weights plus biases). The matrix multiply  $Wx$  is just matrix-vector multiplication from chapter 02. In a batch setting, the input is a matrix  $X$  of shape  $(B, d_{\text{in}})$  and the output is  $XW^T + b$  of shape  $(B, d_{\text{out}})$ .
- The **universal approximation theorem** states that a single hidden layer with enough neurons can approximate any continuous function on a compact domain to arbitrary accuracy. This sounds like depth should not matter, but the catch is “enough neurons.” In practice, deep networks can represent the same functions with exponentially fewer parameters than shallow ones. Depth gives you efficiency, not just expressiveness.
- As networks get deeper, two gradient pathologies emerge. **Vanishing gradients:** when gradients pass through many layers (via the chain rule, chapter 03), they get multiplied by many factors. If these factors are consistently less than 1 (as happens with sigmoid and tanh saturating), the gradient shrinks exponentially toward zero. Early layers barely learn. **Exploding gradients:** if factors are consistently greater than 1, gradients grow exponentially, causing numerical overflow and unstable training.
- Solutions to vanishing/exploding gradients:
  - Use ReLU or GELU activations (gradient is 1 for positive inputs, no saturation)
  - Careful weight initialisation
  - Normalisation layers
  - Residual connections (skip connections)

- Gradient clipping (for exploding gradients): cap the gradient norm at a maximum value
- **Weight initialisation** matters because it determines the scale of activations and gradients at the start of training. If weights are too large, activations explode; too small, they vanish.
- **Xavier (Glorot) initialisation** sets weights from a distribution with variance  $\frac{2}{d_{\text{in}} + d_{\text{out}}}$ . This keeps the variance of activations roughly constant across layers, assuming linear or tanh activations.
- **He (Kaiming) initialisation** uses variance  $\frac{2}{d_{\text{in}}}$ , which is calibrated for ReLU activations (since ReLU zeros out half the activations, you need double the variance to compensate).
- **Normalisation layers** stabilise training by ensuring that the inputs to each layer have consistent statistics (roughly zero mean, unit variance).
- **Batch Normalisation (BatchNorm)** normalises across the batch dimension: for each channel/feature, compute the mean and variance across all samples in the mini-batch, then normalise. It adds learnable scale ( $\gamma$ ) and shift ( $\beta$ ) parameters so the network can undo the normalisation if needed:

⦿

x

$$= \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y = \gamma$$

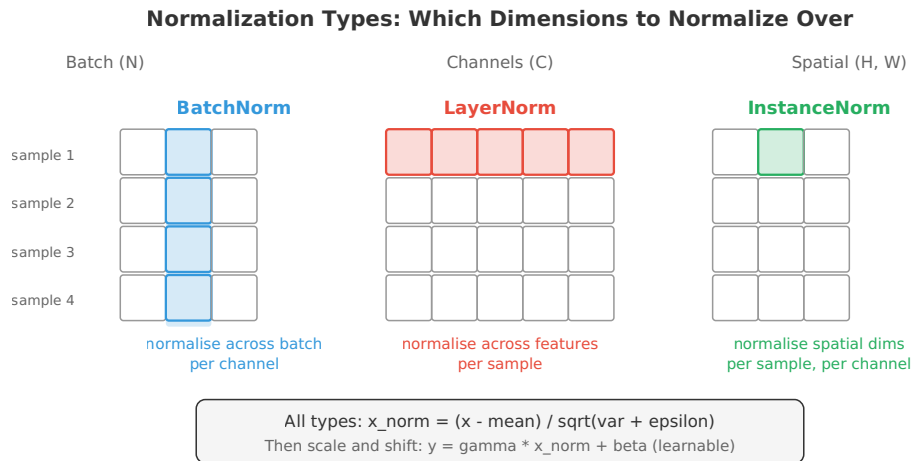
⦿

x

$$+ \beta$$

- BatchNorm has a problem: it depends on the batch size. With very small batches, the statistics are noisy. At inference time, you use running averages instead of batch statistics, which creates a train/test discrepancy.
- **Layer Normalisation (LayerNorm)** normalises across the feature dimension for each individual sample. It does not depend on other samples in the batch, making it the standard choice for transformers and recurrent networks.

- **Instance Normalisation** normalises across spatial dimensions for each sample and each channel independently. It is popular in style transfer.
- **Group Normalisation** splits channels into groups and normalises within each group. It is a compromise between LayerNorm and InstanceNorm.

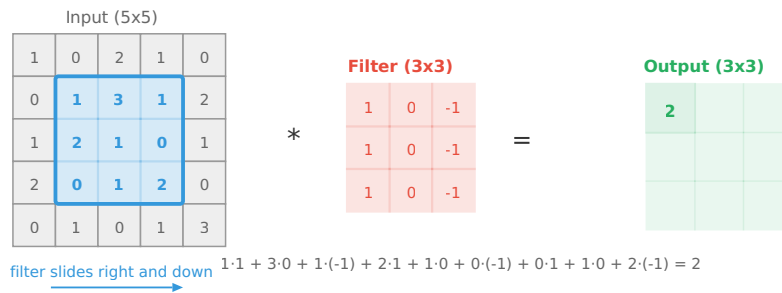


3D tensor with coloured slices showing which dimensions BatchNorm, LayerNorm, and InstanceNorm normalise over

- **Dropout** is a regularisation technique that randomly zeroes out a fraction  $p$  of neurons during training. This forces the network to not rely on any single neuron, encouraging redundant representations. At test time, all neurons are active. **Inverted dropout** scales activations by  $\frac{1}{1-p}$  during training so that no scaling is needed at test time. This is the standard implementation.
- **Convolutional Neural Networks (CNNs)** exploit spatial structure. Instead of connecting every input to every output (as in dense layers), a convolutional layer slides a small filter (kernel) across the input, computing a dot product at each position. The same filter weights are shared across all positions, which drastically reduces parameters and builds in translation invariance.
- The **convolution operation** for a 2D input with filter  $K$  of size  $k \times k$ :

$$\begin{aligned}
 & (\text{input} * K)[i, j] = \\
 & \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} \text{input}[i+m, j+n] \cdot K[m, n]
 \end{aligned}$$

## 2D Convolution: Filter Slides Over Input



Output size = (input - filter + 2·padding) / stride + 1  
Same filter weights applied at every position (weight sharing)

Input grid with a 3x3 filter sliding over it, producing an output feature map with element-wise multiply-and-sum at each position

- The output size depends on three hyperparameters. **Stride** controls how many pixels the filter moves between positions (stride 2 halves the spatial dimensions). **Padding** adds zeros around the input border ( “same” padding preserves spatial size, “valid” padding does not). The output size formula:  $\text{out} = \lfloor (\text{in} - k + 2p) / s \rfloor + 1$ .
- **Pooling** layers downsample feature maps. Max pooling takes the maximum value in each window; average pooling takes the mean. Pooling reduces spatial dimensions while keeping the most important information.
- **Dilated convolutions** insert gaps between filter elements, increasing the receptive field without increasing parameters. A dilation rate of 2 means the 3x3 filter covers a 5x5 area.
- **1x1 convolutions** are convolutions with a 1x1 filter. They do not look at spatial neighbours; instead, they mix information across channels. Think of them as applying a dense layer at every spatial position. They are used to change the number of channels cheaply.
- **Skip connections** (residual connections) let the input bypass one or more layers:  $\text{output} = F(x) + x$ . The layer only needs to learn the residual  $F(x) = \text{output} - x$ , which is easier when the optimal transformation is close to identity. ResNets (Residual Networks) stacked over 100 layers using this trick, solving the degradation problem where deeper networks performed worse than shallower ones.
- CNNs build a **feature hierarchy**. Early layers detect edges and textures. Middle layers combine these into parts (eyes, wheels). Late layers recognise whole



objects. Each layer's receptive field (the region of the input it can "see") grows with depth.

- **Embeddings** map discrete tokens (words, characters, item IDs) to dense vectors. An embedding layer is just a lookup table: a matrix  $E$  of shape (vocabulary size, embedding dimension). Looking up token  $i$  means selecting row  $i$  of  $E$ . This is equivalent to multiplying by a one-hot vector, which is just a special case of matrix-vector multiplication (chapter 02). Embeddings are learned during training, so similar tokens end up with similar vectors.
- **Tokenisation** is the process of converting raw text into a sequence of tokens. Word-level tokenisation splits on spaces but cannot handle unseen words. **Subword tokenisation** (BPE, WordPiece, SentencePiece) breaks text into frequent subword units, balancing vocabulary size and coverage. The word "unhappiness" might become [ "un" , "happiness" ] or [ "un" , "happ" , "iness" ].
- **Recurrent Neural Networks (RNNs)** process sequences one element at a time, maintaining a hidden state that carries information forward:

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$$

- The hidden state  $h_t$  is a compressed summary of everything the network has seen up to time  $t$ . The same weights  $W_h$  and  $W_x$  are shared across all time steps (weight sharing, like CNNs share spatial weights).
- Vanilla RNNs struggle with long sequences because of vanishing gradients: the gradient signal from step  $t$  to step  $t - k$  passes through  $k$  multiplications by  $W_h$ , and it shrinks (or explodes) exponentially.
- **LSTM** (Long Short-Term Memory) solves this by introducing a separate cell state  $c_t$  that flows through time with minimal interference. Three gates control what information enters, leaves, and persists:
- The **forget gate** decides what to erase from the cell state:  $f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$
- The **input gate** decides what new information to write:  $i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$ , with candidate values

⌘

c

$$h_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

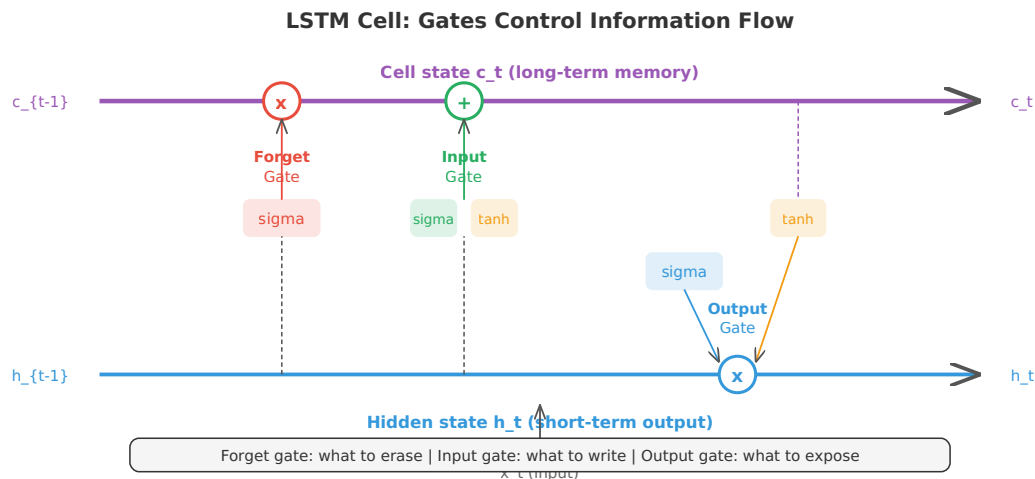
- The cell state updates:  $c_t = f_t \odot c_{t-1} + i_t \odot$

⊙

c

t

- The **output gate** decides what to expose:  $o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$ , and  $h_t = o_t \odot \tanh(c_t)$



LSTM cell showing forget gate, input gate, output gate, cell state highway, and data flow connections

- The cell state acts like a conveyor belt: information can flow unchanged across many time steps (the forget gate stays close to 1), which solves the vanishing gradient problem for long-range dependencies.
- **GRU** (Gated Recurrent Unit) simplifies the LSTM by merging the cell state and hidden state into one, and using two gates instead of three: an update gate (combines forget and input) and a reset gate. GRUs have fewer parameters and often perform comparably to LSTMs.
- The fundamental limitation of RNNs (including LSTMs) is sequential processing: you must process token 1 before token 2 before token 3. This prevents parallelisation and creates an information bottleneck, as all context must squeeze through the fixed-size hidden state.
- **Attention** solves both problems. Instead of compressing the entire input into a fixed vector, attention lets the model look back at all input positions and decide which ones are relevant for the current output.
- The modern formulation uses **queries, keys, and values (Q, K, V)**. Think of it like a library search: you have a query (what you are looking for), keys (labels on

each book), and values (the actual book contents). You compare your query against all keys to figure out which values to retrieve.

- **Scaled dot-product attention:**

$$\text{Attention}(Q, K, V) = \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} \right) V$$

- $QK^T$  computes the similarity between every query and every key. This is a matrix multiply (chapter 02), and the entries are dot products, which measure cosine similarity (chapter 01). Dividing by

---

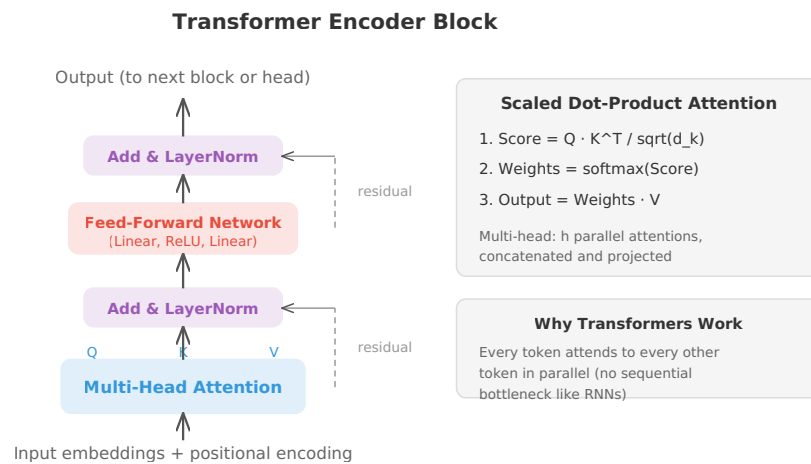
$\sqrt{d_k}$

prevents the dot products from becoming too large (which would make the softmax saturate and produce near-one-hot distributions with vanishing gradients). The softmax converts similarities to a probability distribution. Multiplying by  $V$  produces a weighted combination of values.

- **Multi-head attention** runs  $h$  parallel attention operations, each with different learned projections of  $Q$ ,  $K$ , and  $V$ . This lets the model attend to information from different representation subspaces simultaneously. One head might attend to syntactic relationships while another attends to semantic ones. The outputs are concatenated and projected:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

- The **Transformer** architecture (Vaswani et al., 2017) is built entirely from attention and feed-forward layers, with no recurrence. The encoder block repeats: multi-head self-attention, add and layer-norm, feed-forward network, add and layer-norm. The decoder block adds a masked self-attention (preventing the model from seeing future tokens) and a cross-attention layer that attends to the encoder output.



Transformer encoder block: multi-head attention, add and layernorm, feed-forward network, add and layernorm, with residual connections

- **Positional encoding** is necessary because attention is permutation-equivariant, meaning it treats the input as a set, not a sequence. Without position information, “the cat sat on the mat” and “the mat sat on the cat” would be identical. The original Transformer uses sinusoidal positional encodings:

$$PE_{(\text{pos}, 2i)} = \sin\left(\frac{\text{pos}}{10000^{2i/d}}\right), \quad PE_{(\text{pos}, 2i+1)} = \cos\left(\frac{\text{pos}}{10000^{2i/d}}\right)$$

- Each position gets a unique vector that the model can use to distinguish positions. Modern models often use learned positional embeddings or relative positional encodings (RoPE, ALiBi) instead.
- Transformers process all tokens in parallel (the self-attention matrix  $QK^T$  is computed in one matrix multiply), which makes them much faster to train than RNNs on modern hardware. The tradeoff is that self-attention is  $O(n^2)$  in sequence length (every token attends to every other), while RNNs are  $O(n)$ . This is why long-context models require special attention variants (sparse attention, linear attention, flash attention).
- **Vision Transformers (ViT)** apply the Transformer to images by splitting the image into fixed-size patches (e.g., 16x16), flattening each patch into a vector, and treating the patches as a sequence of tokens. A learnable [CLS] token is prepended, and its final representation is used for classification. Despite having no convolutional inductive biases, ViTs match or surpass CNNs when trained on enough data.
- **MLP-Mixer** is an even simpler architecture that replaces both attention and convolution with MLPs. It alternates between “token-mixing” MLPs (applied

across spatial positions) and “channel-mixing” MLPs (applied across features). It performs competitively, suggesting that the key insight of modern architectures is not attention itself, but rather efficient mixing of information across tokens and features.

- **Autoencoders** learn compressed representations by training a network to reconstruct its own input. The encoder maps the input to a lower-dimensional bottleneck (the latent code), and the decoder maps it back:

$$\begin{aligned}
 & \text{Encoder: } \mathbf{z} = f_{\text{enc}}(\mathbf{x}) \\
 & \text{Decoder: } \hat{\mathbf{x}} = f_{\text{dec}}(\mathbf{z}) \\
 & \text{Loss: } \mathcal{L} = \|\mathbf{x} - \hat{\mathbf{x}}\|^2
 \end{aligned}$$

- The bottleneck forces the network to learn the most important features. Autoencoders are used for dimensionality reduction, denoising (train on noisy input, reconstruct clean output), and anomaly detection (high reconstruction error signals an unusual input).
- **Variational Autoencoders (VAEs)** add a probabilistic twist. Instead of encoding to a single point  $\mathbf{z}$ , the encoder outputs the parameters of a distribution (mean  $\mu$  and variance  $\sigma^2$  of a Gaussian). The latent code is sampled from this distribution:  $\mathbf{z} = \mu + \sigma \odot \epsilon$ , where  $\epsilon \sim \mathcal{N}(0, \mathbf{I})$ . This **reparameterisation trick** makes the sampling differentiable so gradients can flow through.
- The VAE loss has two terms:

$$\begin{aligned}
 \mathcal{L} = & \underbrace{\|\mathbf{x} - \hat{\mathbf{x}}\|^2}_{\text{reconstruction}} \\
 & + \underbrace{D_{\text{KL}}(q(\mathbf{z} | \mathbf{x}) \| p(\mathbf{z}))}_{\text{regularisation}}
 \end{aligned}$$

- The KL divergence term (from chapter 05) pushes the learned posterior  $q(\mathbf{z} | \mathbf{x})$  toward the prior  $p(\mathbf{z}) = \mathcal{N}(0, \mathbf{I})$ , ensuring the latent space is smooth and well-

structured. You can then sample from the prior and decode to generate new data. This is what makes VAEs generative models.

## 28.1 Coding Tasks (use CoLab or notebook)

---

1. Build a simple MLP from scratch in JAX. Train it on a 2D classification problem (e.g., concentric circles) and visualise the decision boundary.

```
import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt
from sklearn.datasets import make_circles

Data
X, y = make_circles(n_samples=500, noise=0.1, factor=0.5, random_state=42)
X, y = jnp.array(X), jnp.array(y, dtype=jnp.float32)

Initialise a 2-layer MLP: 2 -> 16 -> 16 -> 1
def init_params(key):
 k1, k2, k3 = jax.random.split(key, 3)
 return {
 'W1': jax.random.normal(k1, (2, 16)) * 0.5,
 'b1': jnp.zeros(16),
 'W2': jax.random.normal(k2, (16, 16)) * 0.5,
 'b2': jnp.zeros(16),
 'W3': jax.random.normal(k3, (16, 1)) * 0.5,
 'b3': jnp.zeros(1),
 }

def forward(params, x):
 h = jnp.maximum(0, x @ params['W1'] + params['b1']) # ReLU
 h = jnp.maximum(0, h @ params['W2'] + params['b2']) # ReLU
 logit = (h @ params['W3'] + params['b3']).squeeze()
 return jax.nn.sigmoid(logit)

def loss_fn(params, X, y):
 pred = forward(params, X)
 return -jnp.mean(y * jnp.log(pred + 1e-7) + (1 - y) * jnp.log(1 - pred +
1e-7))

grad_fn = jax.jit(jax.grad(loss_fn))
params = init_params(jax.random.PRNGKey(0))
lr = 0.1

for step in range(2000):
```

```

grads = grad_fn(params, X, y)
params = {k: params[k] - lr * grads[k] for k in params}

Plot decision boundary
xx, yy = jnp.meshgrid(jnp.linspace(-2, 2, 200), jnp.linspace(-2, 2, 200))
grid = jnp.column_stack([xx.ravel(), yy.ravel()])
zz = forward(params, grid).reshape(xx.shape)

plt.figure(figsize=(7, 6))
plt.contourf(xx, yy, zz, levels=[0, 0.5, 1], alpha=0.3, colors=['#e74c3c',
'#3498db'])
plt.scatter(X[y==0,0], X[y==0,1], c='#e74c3c', s=10, label='Class 0')
plt.scatter(X[y==1,0], X[y==1,1], c='#3498db', s=10, label='Class 1')
plt.title("MLP Decision Boundary on Concentric Circles")
plt.legend(); plt.grid(alpha=0.3); plt.show()

acc = jnp.mean((forward(params, X) > 0.5) == y)
print(f"Accuracy: {acc:.2%}")

```

1. Implement 1D convolution from scratch. Apply a simple edge-detection filter to a signal and compare with the built-in `jnp.convolve`.

```

import jax.numpy as jnp
import matplotlib.pyplot as plt

def conv1d(signal, kernel):
 """1D convolution (valid mode) from scratch."""
 n, k = len(signal), len(kernel)
 output = jnp.zeros(n - k + 1)
 for i in range(n - k + 1):
 output = output.at[i].set(jnp.sum(signal[i:i+k] * kernel))
 return output

Create a signal with a step function
t = jnp.linspace(0, 4, 200)
signal = jnp.where(t < 1, 0.0, jnp.where(t < 2, 1.0, jnp.where(t < 3, 0.5, 1.5)))

Edge detection kernel
edge_kernel = jnp.array([-1.0, 0.0, 1.0])

Our implementation vs built-in
our_output = conv1d(signal, edge_kernel)
jnp_output = jnp.convolve(signal, edge_kernel, mode='valid')

fig, axes = plt.subplots(3, 1, figsize=(10, 6), sharex=True)
axes[0].plot(t, signal, color='#3498db', linewidth=1.5)

```

```

axes[0].set_title("Original Signal"); axes[0].set_ylabel("Value")

axes[1].plot(t[:len(our_output)], our_output, color='#e74c3c', linewidth=1.5)
axes[1].set_title("After Edge Detection (our conv1d)");
axes[1].set_ylabel("Value")

axes[2].plot(t[:len(jnp_output)], jnp_output, color='#27ae60', linewidth=1.5,
linestyle='--')
axes[2].set_title("After Edge Detection (jnp.convolve)");
axes[2].set_ylabel("Value")
axes[2].set_xlabel("t")

plt.tight_layout(); plt.show()
print(f"Outputs match: {jnp.allclose(our_output, jnp_output)}")

```

1. Implement scaled dot-product attention from scratch. Compute attention weights for a small example and visualise the attention matrix as a heatmap.

```

import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

def scaled_dot_product_attention(Q, K, V):
 """Scaled dot-product attention."""
 d_k = Q.shape[-1]
 scores = Q @ K.T / jnp.sqrt(d_k)
 weights = jax.nn.softmax(scores, axis=-1)
 output = weights @ V
 return output, weights

Example: 4 tokens, embedding dim 8
key = jax.random.PRNGKey(42)
k1, k2, k3 = jax.random.split(key, 3)
seq_len, d_model = 4, 8

Q = jax.random.normal(k1, (seq_len, d_model))
K = jax.random.normal(k2, (seq_len, d_model))
V = jax.random.normal(k3, (seq_len, d_model))

output, weights = scaled_dot_product_attention(Q, K, V)

print(f"Q shape: {Q.shape}")
print(f"Attention weights shape: {weights.shape}")
print(f"Output shape: {output.shape}")
print(f"\nAttention weights (rows sum to 1):")
print(weights)

```



```

print(f"Row sums: {weights.sum(axis=-1)}")

Visualise attention
fig, ax = plt.subplots(figsize=(5, 4))
im = ax.imshow(weights, cmap='Blues', vmin=0, vmax=1)
ax.set_xlabel("Key position"); ax.set_ylabel("Query position")
ax.set_title("Attention Weights")
tokens = ['tok 0', 'tok 1', 'tok 2', 'tok 3']
ax.set_xticks(range(4)); ax.set_xticklabels(tokens)
ax.set_yticks(range(4)); ax.set_yticklabels(tokens)
for i in range(4):
 for j in range(4):
 ax.text(j, i, f"{weights[i,j]:.2f}", ha='center', va='center',
fontSize=10)
plt.colorbar(im); plt.tight_layout(); plt.show()

```

1. Build a simple autoencoder that compresses 2D data through a 1D bottleneck and reconstructs it. Visualise the latent space and reconstructions.

```

import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons

Data
X, _ = make_moons(n_samples=500, noise=0.05, random_state=42)
X = jnp.array(X)

Autoencoder: 2 -> 8 -> 1 -> 8 -> 2
def init_ae(key):
 k1, k2, k3, k4 = jax.random.split(key, 4)
 return {
 'enc_W1': jax.random.normal(k1, (2, 8)) * 0.5, 'enc_b1': jnp.zeros(8),
 'enc_W2': jax.random.normal(k2, (8, 1)) * 0.5, 'enc_b2': jnp.zeros(1),
 'dec_W1': jax.random.normal(k3, (1, 8)) * 0.5, 'dec_b1': jnp.zeros(8),
 'dec_W2': jax.random.normal(k4, (8, 2)) * 0.5, 'dec_b2': jnp.zeros(2),
 }

def encode(p, x):
 h = jnp.tanh(x @ p['enc_W1'] + p['enc_b1'])
 return h @ p['enc_W2'] + p['enc_b2']

def decode(p, z):
 h = jnp.tanh(z @ p['dec_W1'] + p['dec_b1'])
 return h @ p['dec_W2'] + p['dec_b2']

```

```

def ae_loss(p, X):
 z = encode(p, X)
 X_hat = decode(p, z)
 return jnp.mean((X - X_hat) ** 2)

grad_fn = jax.jit(jax.grad(ae_loss))
params = init_ae(jax.random.PRNGKey(0))
lr = 0.01

for step in range(3000):
 grads = grad_fn(params, X)
 params = {k: params[k] - lr * grads[k] for k in params}

z = encode(params, X)
X_hat = decode(params, z)

fig, axes = plt.subplots(1, 2, figsize=(12, 5))
axes[0].scatter(X[:,0], X[:,1], c=z.squeeze(), cmap='viridis', s=10)
axes[0].set_title("Original Data (coloured by latent code)")
axes[1].scatter(X_hat[:,0], X_hat[:,1], c=z.squeeze(), cmap='viridis', s=10)
axes[1].set_title("Reconstruction from 1D bottleneck")
for ax in axes:
 ax.set_aspect('equal'); ax.grid(alpha=0.3)
plt.tight_layout(); plt.show()

print(f"Reconstruction MSE: {ae_loss(params, X):.4f}")

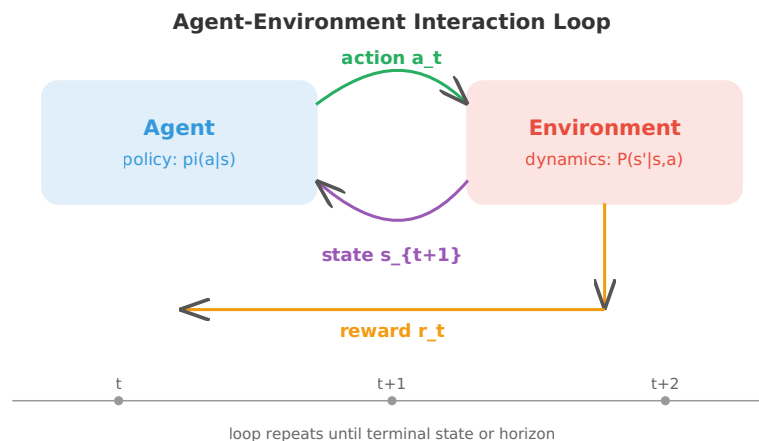
```

## 29 Reinforcement Learning

Reinforcement learning trains agents to make sequential decisions by maximising cumulative reward through trial and error. This file covers MDPs, value functions, Bellman equations, Q-learning, policy gradients, actor-critic methods, PPO, and RLHF, the framework behind game-playing agents and language model alignment.

- Supervised learning needs labelled data. Unsupervised learning finds patterns in unlabelled data. **Reinforcement learning (RL)** is different from both: an agent learns by interacting with an environment, taking actions, and receiving rewards. There are no correct labels; the agent must discover good behaviour through trial and error.

- Think of teaching a dog a new trick. You do not show it a dataset of correct behaviours. Instead, it tries things, you give treats for good actions, and over time it figures out what you want. RL formalises this process.
- The RL setup has five core components. The **agent** is the learner and decision-maker. The **environment** is everything outside the agent that it interacts with. At each time step, the agent observes a **state**  $s_t$ , chooses an **action**  $a_t$ , receives a **reward**  $r_t$ , and transitions to a new state  $s_{t+1}$ . The agent's goal is to maximise the total reward it collects over time.



Agent-environment loop: agent observes state, takes action, receives reward, environment transitions to new state

- A **policy**  $\pi$  is the agent's strategy: a mapping from states to actions. A deterministic policy gives one action per state:  $a = \pi(s)$ . A stochastic policy gives a probability distribution over actions:  $\pi(a \mid s)$ . The goal of RL is to find the optimal policy, the one that maximises expected cumulative reward.
- The mathematical framework for RL is the **Markov Decision Process (MDP)**, defined by a tuple  $(S, A, P, R, \gamma)$ : a set of states  $S$ , a set of actions  $A$ , transition probabilities  $P(s' \mid s, a)$ , a reward function  $R(s, a)$ , and a discount factor  $\gamma$ .
- The **Markov property** (from chapter 05) says the future depends only on the current state, not on the history of how you got there:  

$$P(s_{t+1} \mid s_t, a_t, s_{t-1}, \dots) = P(s_{t+1} \mid s_t, a_t)$$
This means the state contains all the information needed to make a decision.
- The **discount factor**  $\gamma \in [0, 1)$  determines how much the agent cares about future rewards versus immediate ones. The discounted return from time  $t$  is:

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

- With  $\gamma = 0$ , the agent is completely myopic, caring only about the next reward. With  $\gamma$  close to 1, the agent is far-sighted. The discount factor also ensures the sum converges (if rewards are bounded), which is important for mathematical well-definedness.

- **Value functions** estimate how good it is to be in a state (or to take an action in a state). The **state-value function**  $V^\pi(s)$  is the expected return starting from state  $s$  and following policy  $\pi$ :

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s]$$

- The **action-value function**  $Q^\pi(s, a)$  is the expected return starting from state  $s$ , taking action  $a$ , and then following  $\pi$ :

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a]$$

- The relationship:  $V^\pi(s) = \sum_a \pi(a \mid s) Q^\pi(s, a)$ . The state value is the average of action values, weighted by the policy.
- The **Bellman equation** expresses a recursive relationship: the value of a state equals the immediate reward plus the discounted value of the next state. For the state-value function:

$$V^\pi(s) = \sum_a \pi(a \mid s) \sum_{s'} P(s' \mid s, a) [R(s, a) + \gamma V^\pi(s')]$$

- For the optimal value function  $V^*(s)$ , the agent always picks the best action:

$$V^*(s) = \max_a \sum_{s'} P(s' \mid s, a) [R(s, a) + \gamma V^*(s')]$$

- Similarly, the **Bellman optimality equation** for  $Q^*$ :

$$Q^*(s, a) = \sum_{s'} P(s' \mid s, a) [R(s, a) + \gamma \max_{a'} Q^*(s', a')]$$

- Once you have  $Q^*$ , the optimal policy is trivial: always pick the action with the highest Q-value:  $\pi^*(s) = \arg \max_a Q^*(s, a)$ .
- **Dynamic programming** methods solve MDPs when you know the transition probabilities and rewards (the full model). **Policy evaluation** computes  $V^\pi$  for a given policy by iteratively applying the Bellman equation until convergence. **Policy improvement** takes the value function and constructs a better policy by acting greedily:  $\pi'(s) = \arg \max_a \sum_{s'} P(s' \mid s, a) [R(s, a) + \gamma V^\pi(s')]$ .
- **Policy iteration** alternates between evaluation and improvement until the policy stops changing. It is guaranteed to converge to the optimal policy.
- **Value iteration** combines both steps into one: it repeatedly applies the Bellman optimality equation until  $V^*$  converges, then extracts the policy.

$$V(s) \leftarrow \max_a \sum_{s'} P(s' \mid s, a) [R(s, a) + \gamma V(s')]$$

$\Sigma$   
 $s'$

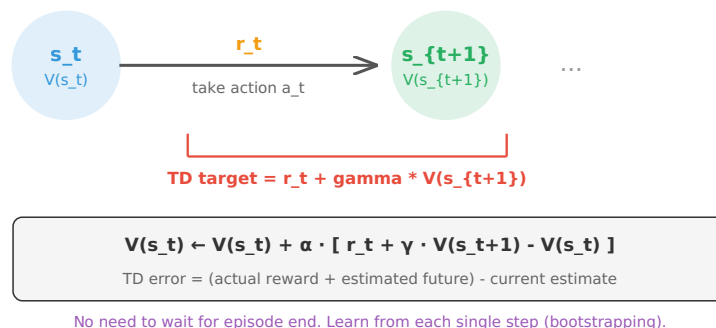
$$P(s' | s, a)[R(s, a) + \gamma V(s')]$$

- Dynamic programming requires knowing  $P(s' | s, a)$ , which is often impractical. In most real problems, the agent does not know the environment's dynamics; it can only interact with it. This is where **model-free** methods come in.
- **Temporal Difference (TD) learning** learns from experience without knowing the model. The key idea is **bootstrapping**: instead of waiting until the end of an episode to compute the actual return  $G_t$ , you estimate it using the current value function:

$$V(s_t) \leftarrow V(s_t) + \alpha [r_t + \gamma V(s_{t+1}) - V(s_t)]$$

- The term in brackets is the **TD error**: the difference between the **TD target** ( $r_t + \gamma V(s_{t+1})$ ) and the current estimate  $V(s_t)$ . If the TD error is positive, the state was better than expected, so we increase its value. If negative, we decrease it.

#### TD Learning: Bootstrapping from the Next State



State transition showing TD target: current value, reward, and bootstrapped next value with the update formula

- TD learning updates after every single step (not after complete episodes), which makes it much more efficient than Monte Carlo methods. It also works in continuing (non-episodic) environments.
- **SARSA** (State-Action-Reward-State-Action) is TD learning applied to Q-values. The agent takes action  $a$  in state  $s$ , observes reward  $r$  and next state  $s'$ , then chooses next action  $a'$  according to its policy:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma Q(s', a') - Q(s, a)]$$

- SARSA is **on-policy**: it updates using the action the agent actually takes, which includes exploration. This makes SARSA more conservative; it learns a policy that accounts for its own exploration noise.
- **Q-learning** is the most famous RL algorithm. It is like SARSA, but instead of using the action the agent actually takes, it uses the best possible action:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma$$

max

$a'$

$$Q(s', a') - Q(s, a)]$$

- Q-learning is **off-policy**: it learns the optimal Q-values regardless of the policy being followed. The agent can explore randomly while still learning the optimal action values. This makes Q-learning more aggressive and often faster to converge, but it can overestimate values.
- **Exploration vs exploitation** is the fundamental dilemma: should the agent exploit what it already knows (choose the action with the highest estimated value) or explore unknown actions (which might turn out to be better)?
- The simplest strategy is **epsilon-greedy**: with probability  $\epsilon$ , take a random action (explore); with probability  $1 - \epsilon$ , take the greedy action (exploit). A common schedule starts with high  $\epsilon$  (lots of exploration) and decays it over time.
- Tabular methods (storing a value for each state-action pair in a table) work for small, discrete state spaces. For large or continuous state spaces, you need function approximation. **Deep Q-Networks (DQN)** use a neural network to approximate  $Q(s, a; \theta)$ , where  $\theta$  are the network weights.
- DQN introduced two critical stabilisation techniques. **Experience replay**: instead of learning from consecutive transitions (which are highly correlated), store transitions in a replay buffer and sample random mini-batches for training. This breaks correlations and reuses data efficiently.
- **Target network**: use a separate, slowly-updated copy of the network to compute TD targets. Without this, the target moves every time you update the network, creating a “chasing your own tail” instability. The target network is updated

periodically (hard update every N steps) or continuously (soft update:  $\theta^- \leftarrow \tau\theta + (1 - \tau)\theta^-$ ).

- The DQN loss is just MSE between predicted Q-values and TD targets:

$$\mathcal{L}(\theta) = \mathbb{E}[(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta))^2]$$

- All the methods so far learn value functions and derive policies from them. **Policy gradient** methods take a different approach: they directly parameterise the policy  $\pi(a \mid s; \theta)$  and optimise it by gradient ascent on expected return.
- The **policy gradient theorem** gives the gradient of expected return with respect to policy parameters:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi}[\nabla_{\theta} \log \pi(a \mid s; \theta) \cdot G_t]$$

- This says: increase the probability of actions that led to high returns, decrease the probability of actions that led to low returns. The log-probability gradient gives the direction to change the policy, and  $G_t$  scales how much to change it.
- **REINFORCE** is the simplest policy gradient algorithm. Run an episode, compute returns  $G_t$  for each step, and update:

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} \log \pi(a_t \mid s_t; \theta) \cdot G_t$$

- REINFORCE has high variance because  $G_t$  is a noisy, single-sample estimate of the expected return. A common fix is to subtract a **baseline** (typically the average return or a learned value function) to reduce variance without introducing bias:

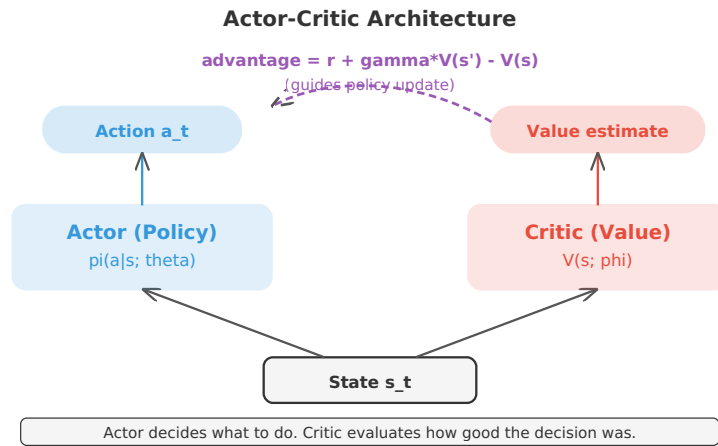
$$\theta \leftarrow \theta + \alpha \nabla_{\theta} \log \pi(a_t \mid s_t; \theta) \cdot (G_t - b)$$

- **Actor-Critic** methods use two networks. The **actor** is the policy  $\pi(a \mid s; \theta)$ . The **critic** is a value function  $V(s; \phi)$  that serves as the baseline. The advantage  $A_t = r_t + \gamma V(s_{t+1}) - V(s_t)$  replaces  $G_t - b$ :



$$\theta \leftarrow \theta + \alpha \nabla_{\theta} \log \pi(a_t | s_t; \theta) \cdot A_t$$

- The critic is updated by minimising TD error, just like value-based methods. The actor is updated using the policy gradient, with the critic's advantage estimate reducing variance. This is the best of both worlds.



Two-headed architecture: actor outputs action probabilities, critic outputs value estimate, advantage signal guides actor updates

- **PPO** (Proximal Policy Optimization) is the most widely used policy gradient algorithm in practice. It addresses a key problem: if a policy update is too large, performance can collapse catastrophically.

- PPO uses a **clipped surrogate objective**. Let  $r_t(\theta) = \frac{\pi(a_t | s_t; \theta)}{\pi(a_t | s_t; \theta_{old})}$  be the probability ratio between new and old policies. The loss is:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}[\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)]$$

- The clipping (typically  $\epsilon = 0.2$ ) prevents the ratio from moving too far from 1, which keeps updates small and stable. If the advantage is positive (action was good), the ratio is capped at  $1 + \epsilon$ . If negative (action was bad), the ratio is capped at  $1 - \epsilon$ . This is simpler and more stable than earlier trust-region methods (TRPO).
- PPO is what was used to train ChatGPT-style models via **RLHF** (Reinforcement Learning from Human Feedback). In RLHF, a reward model is trained on human preference data (which of two outputs do humans prefer?), and then PPO optimises the language model's policy to maximise this learned reward.

- **DPO** (Direct Preference Optimization) simplifies RLHF by eliminating the reward model entirely. Instead of training a reward model and then running RL, DPO derives a closed-form loss that directly optimises the policy from preference data:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}[\log \sigma(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)})]$$

- Here  $y_w$  is the preferred (winning) response and  $y_l$  is the dispreferred (losing) response. DPO increases the relative probability of preferred outputs and is much simpler to implement than PPO-based RLHF.
- Two important distinctions in RL algorithms. **On-policy vs off-policy**: on-policy methods (SARSA, PPO) learn from data generated by the current policy; off-policy methods (Q-learning, DQN) can learn from data generated by any policy. Off-policy methods are more sample-efficient (they reuse old data) but can be less stable.
- **Model-based vs model-free**: model-free methods (everything discussed so far) learn values or policies directly from experience. Model-based methods learn a model of the environment ( $P(s' | s, a)$  and  $R(s, a)$ ) and use it for planning (imagining future trajectories without actually taking actions). Model-based methods are more sample-efficient but add the complexity of learning an accurate model.
- To summarise the RL landscape:

Method	Type	Key Idea	Strength
Value Iteration	DP, model-based	Bellman optimality	Exact solution (small MDPs)
SARSA	TD, on-policy	Learn Q on-policy	Conservative, safe
Q-Learning	TD, off-policy	Learn $Q^*$ , greedy target	Simple, effective
DQN	Deep, off-policy	Neural Q + replay + target net	Scales to high-dim states
REINFORCE	Policy gradient	Gradient of log-prob * return	Simple policy optimisation

Method	Type	Key Idea	Strength
Actor-Critic	PG + value	Actor + critic for low variance	Practical and flexible
PPO	PG, clipped	Trust-region-like stability	Industry standard
DPO	Direct preference	Skip reward model	Simpler RLHF

## 29.1 Coding Tasks (use CoLab or notebook)

1. Implement value iteration for a simple gridworld. Compute the optimal value function and extract the optimal policy. Visualise both as a heatmap and arrow plot.

```

import jax.numpy as jnp
import matplotlib.pyplot as plt

4x4 gridworld: goal at (3,3), reward -1 per step, 0 at goal
grid_size = 4
gamma = 0.99
goal = (3, 3)

Actions: up, down, left, right
actions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
action_names = ['up', 'down', 'left', 'right']
action_arrows = ['\u2191', '\u2193', '\u2190', '\u2192']

def step(s, a):
 """Deterministic transition."""
 ns = (max(0, min(grid_size-1, s[0]+a[0])),
 max(0, min(grid_size-1, s[1]+a[1])))
 return ns

Value iteration
V = jnp.zeros((grid_size, grid_size))
for iteration in range(100):
 V_new = jnp.array(V)
 for i in range(grid_size):
 for j in range(grid_size):
 if (i, j) == goal:
 continue

```

```

 values = []
 for a in actions:
 ns = step((i, j), a)
 values.append(-1 + gamma * float(V[ns[0], ns[1]]))
 V_new = V_new.at[i, j].set(max(values))
 if jnp.max(jnp.abs(V_new - V)) < 1e-6:
 print(f"Converged in {iteration+1} iterations")
 break
 V = V_new

Extract policy
policy = [['' for _ in range(grid_size)] for _ in range(grid_size)]
for i in range(grid_size):
 for j in range(grid_size):
 if (i, j) == goal:
 policy[i][j] = 'G'
 continue
 best_a = max(range(4), key=lambda a: -1 + gamma * float(V[step((i,j),
actions[a])[0], step((i,j), actions[a])[1]]))
 policy[i][j] = action_arrows[best_a]

fig, axes = plt.subplots(1, 2, figsize=(10, 4))
im = axes[0].imshow(V, cmap='YlOrRd_r')
axes[0].set_title("Optimal Value Function")
for i in range(grid_size):
 for j in range(grid_size):
 axes[0].text(j, i, f"{V[i,j]:.1f}", ha='center', va='center',
fontsize=10)
plt.colorbar(im, ax=axes[0])

axes[1].imshow(jnp.ones((grid_size, grid_size)), cmap='Greys', vmin=0, vmax=2)
axes[1].set_title("Optimal Policy")
for i in range(grid_size):
 for j in range(grid_size):
 axes[1].text(j, i, policy[i][j], ha='center', va='center', fontsize=18)
plt.tight_layout(); plt.show()

```

1. Implement tabular Q-learning on a simple gridworld. Train the agent, plot the learning curve, and show the learned Q-values.

```

import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

grid_size = 5
goal = (4, 4)

```

```

actions = [(-1,0), (1,0), (0,-1), (0,1)]

Q-table
Q = {}
for i in range(grid_size):
 for j in range(grid_size):
 Q[(i,j)] = [0.0] * 4

alpha = 0.1
gamma = 0.95
epsilon = 1.0
epsilon_decay = 0.995
min_epsilon = 0.01

def step(s, a_idx):
 a = actions[a_idx]
 ns = (max(0, min(grid_size-1, s[0]+a[0])),
 max(0, min(grid_size-1, s[1]+a[1])))
 r = 0.0 if ns == goal else -1.0
 done = ns == goal
 return ns, r, done

key = jax.random.PRNGKey(42)
rewards_per_episode = []

for ep in range(500):
 s = (0, 0)
 total_reward = 0
 for _ in range(100):
 key, subkey = jax.random.split(key)
 if float(jax.random.uniform(subkey)) < epsilon:
 key, subkey = jax.random.split(key)
 a = int(jax.random.randint(subkey, (), 0, 4))
 else:
 a = max(range(4), key=lambda i: Q[s][i])

 ns, r, done = step(s, a)
 total_reward += r
 # Q-learning update
 Q[s][a] += alpha * (r + gamma * max(Q[ns]) - Q[s][a])
 s = ns
 if done:
 break
 rewards_per_episode.append(total_reward)
 epsilon = max(min_epsilon, epsilon * epsilon_decay)

plt.figure(figsize=(8, 4))

```

```

Smooth the curve
window = 20
smoothed = [sum(rewards_per_episode[max(0,i-window):i+1])/min(i+1, window)
 for i in range(len(rewards_per_episode))]
plt.plot(smoothed, color='#3498db', linewidth=1.5)
plt.xlabel("Episode"); plt.ylabel("Total Reward (smoothed)")
plt.title("Q-Learning on Gridworld")
plt.grid(alpha=0.3); plt.show()

Show learned policy
arrow = ['\u2191', '\u2193', '\u2190', '\u2192']
print("Learned policy:")
for i in range(grid_size):
 row = ""
 for j in range(grid_size):
 if (i,j) == goal:
 row += " G "
 else:
 row += f" {arrow[max(range(4), key=lambda a: Q[(i,j)][a])]} "
 print(row)

```

1. Implement REINFORCE on a multi-armed bandit problem. Show how the policy evolves over training to favour the best arm.

```

import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

5-armed bandit with different expected rewards
true_rewards = jnp.array([0.2, 0.5, 0.8, 0.3, 0.1])
n_arms = len(true_rewards)

Policy: softmax over logits
logits = jnp.zeros(n_arms)
lr = 0.1
key = jax.random.PRNGKey(42)

policy_history = []
reward_history = []

for step in range(2000):
 probs = jax.nn.softmax(logits)
 policy_history.append(probs)

 # Sample action
 key, subkey = jax.random.split(key)

```

```

 action = jax.random.choice(subkey, n_arms, p=probs)

 # Get reward (Bernoulli)
 key, subkey = jax.random.split(key)
 reward = float(jax.random.uniform(subkey) < true_rewards[action])
 reward_history.append(reward)

 # REINFORCE update
 # grad log pi(a) = e_a - probs (for softmax parameterisation)
 grad_log_pi = -probs.at[action].add(1.0) # one-hot(a) - probs
 logits = logits + lr * reward * grad_log_pi

 policy_history = jnp.stack(policy_history)

 fig, axes = plt.subplots(1, 2, figsize=(12, 4))
 colors = ['#3498db', '#e74c3c', '#27ae60', '#9b59b6', '#f39c12']
 for i in range(n_arms):
 axes[0].plot(policy_history[:, i], color=colors[i],
 label=f'Arm {i} (true={true_rewards[i]:.1f})', linewidth=1.5)
 axes[0].set_xlabel("Step"); axes[0].set_ylabel("P(arm)")
 axes[0].set_title("Policy Evolution (REINFORCE)")
 axes[0].legend(fontsize=8); axes[0].grid(alpha=0.3)

 # Smoothed reward
 window = 50
 smoothed = [sum(reward_history[max(0, i-window):i+1])/min(i+1, window)
 for i in range(len(reward_history))]
 axes[1].plot(smoothed, color='#27ae60', linewidth=1.5)
 axes[1].axhline(y=0.8, color='#e74c3c', linestyle='--', alpha=0.5, label='Best
arm')
 axes[1].set_xlabel("Step"); axes[1].set_ylabel("Avg Reward")
 axes[1].set_title("Reward Over Time"); axes[1].legend()
 axes[1].grid(alpha=0.3)
 plt.tight_layout(); plt.show()

```

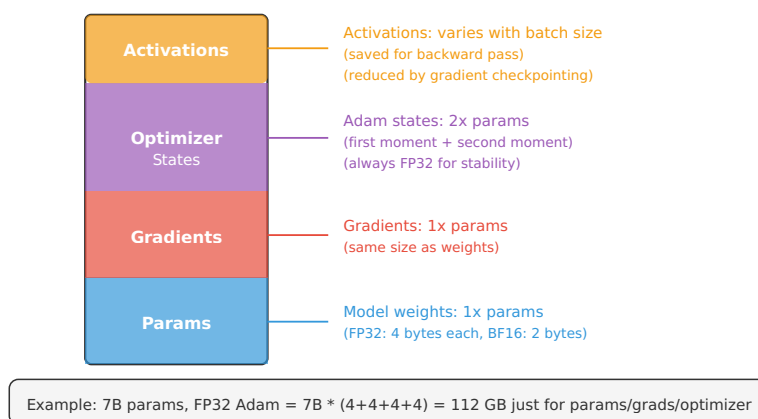
## 30 Distributed Deep Learning

Distributed training spreads computation across multiple GPUs and machines to train models that are too large or too slow for a single device. This file covers mixed precision, data parallelism, model parallelism, pipeline parallelism, ZeRO, FSDP,

tensor parallelism, and communication primitives like all-reduce, essential for training LLMs at scale.

- Training a large neural network on a single GPU eventually hits a wall. The model might not fit in memory, or training might take months. Distributed training spreads the work across multiple devices (GPUs, TPUs, or entire machines) to train faster and train bigger models. This file covers the techniques that make that possible.
- To understand why distribution matters, start with the **computational cost** of training. A single forward pass through a dense layer with  $d_{in}$  inputs and  $d_{out}$  outputs on a batch of  $B$  examples requires roughly  $2 \cdot B \cdot d_{in} \cdot d_{out}$  FLOPs (floating-point operations): one multiply and one add for each element of the output matrix. The backward pass costs roughly twice the forward pass (computing gradients with respect to both the inputs and the weights), so one training step on a dense layer is about  $6 \cdot B \cdot d_{in} \cdot d_{out}$  FLOPs.
- For a transformer layer with hidden dimension  $d$ , the self-attention block involves four projections (Q, K, V, and output) each costing  $O(B \cdot n \cdot d^2)$  FLOPs (where  $n$  is the sequence length), plus the attention matrix computation at  $O(B \cdot n^2 \cdot d)$ . The feed-forward block has two dense layers, typically expanding to  $4d$  and back:  $O(B \cdot n \cdot 8d^2)$ . Total per layer: roughly  $O(B \cdot n \cdot 12d^2 + B \cdot n^2 \cdot d)$ . Multiply by the number of layers and you see why training GPT-scale models requires thousands of GPU-hours.
- The **memory wall** is often the tighter constraint. During training, GPU memory must hold four things simultaneously:

**Training Memory Breakdown (per GPU)**



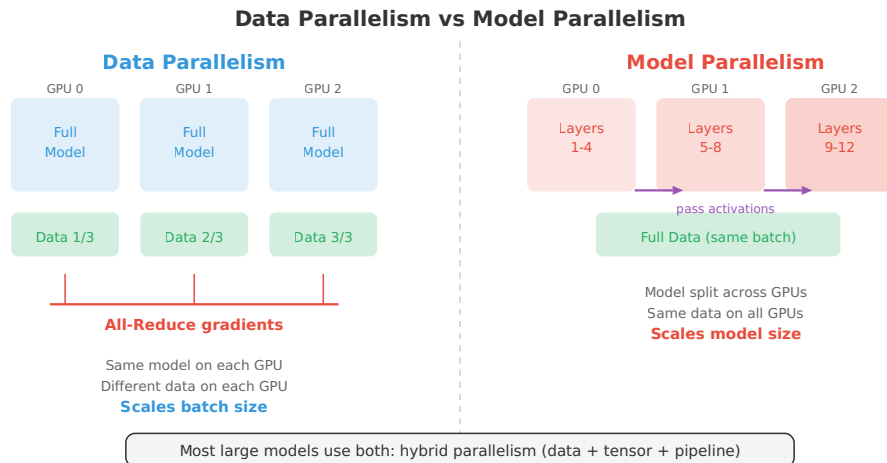


Stacked bar showing training memory breakdown: parameters, gradients, optimizer states, activations

- **Parameters:** the model weights. A 7-billion parameter model in FP32 (4 bytes per parameter) needs 28 GB just for weights.
- **Gradients:** same size as the parameters. Another 28 GB.
- **Optimizer states:** Adam maintains two additional buffers (first and second moment estimates), each the size of the parameters. These are kept in FP32 for numerical stability, even when the model uses lower precision. For our 7B model, that is  $2 \times 28 = 56$  GB.
- **Activations:** intermediate values saved during the forward pass for use in the backward pass. The size depends on batch size, sequence length, and model width. This is often the largest component and grows linearly with batch size.
- For our 7B model with FP32 Adam: 28 (params) + 28 (grads) + 56 (optimizer) = 112 GB, before we even count activations. A single 80 GB A100 GPU cannot hold this. This is why distributed strategies are essential.
- **Mixed precision training** is the first line of defence. Instead of storing everything in FP32 (32-bit floating point), you train using FP16 or BF16 (16-bit) for the forward and backward passes, while keeping a master copy of weights in FP32 for the optimizer update.
- **FP16** has high precision (10-bit mantissa) but a limited range, which can cause overflow/underflow. Loss scaling (multiplying the loss by a large factor before the backward pass, then dividing gradients by the same factor) mitigates this.
- **BF16** (brain float) has the same exponent range as FP32 (8-bit exponent) but less precision (7-bit mantissa). It almost never overflows and rarely needs loss scaling, making it simpler to use. BF16 is the default for modern transformer training.
- Mixed precision roughly halves the memory for activations and gradients (the dominant costs during forward/backward passes), while keeping optimizer states in FP32 for numerical stability.
- **Data parallelism** is the simplest distributed strategy. You replicate the entire model on N GPUs, split each mini-batch into N equal chunks, and send one chunk to each GPU. Each GPU runs the forward and backward pass on its chunk

independently. Then the gradients are averaged across all GPUs (using an all-reduce operation), and each GPU updates its local copy of the model.

- From the model's perspective, this is equivalent to training with a mini-batch that is  $N$  times larger. If each GPU processes a batch of size  $B$ , the effective batch size is  $N \cdot B$ .

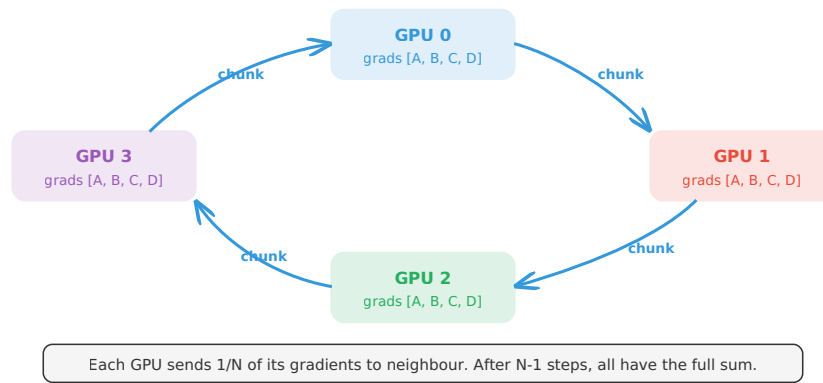


Side-by-side comparison: data parallelism replicates model and splits data, model parallelism splits model and shares data

- The gradient averaging can be done synchronously or asynchronously. **Synchronous SGD** waits for all GPUs to finish before averaging, ensuring mathematical equivalence to single-GPU training with a larger batch. The downside is that the slowest GPU (the “straggler”) holds everyone up.
- **Asynchronous SGD** lets each GPU update a shared parameter server independently, without waiting. This eliminates the straggler problem but introduces “stale gradients”: a GPU might compute gradients based on slightly outdated parameters. Stale gradients add noise and can slow convergence. In practice, synchronous SGD with efficient communication is preferred.
- **Gradient accumulation** is a software trick for simulating larger batch sizes on limited hardware. Instead of doing one update per mini-batch, you run several forward/backward passes and accumulate the gradients, then do one update. This gives the same result as a larger batch without needing more GPU memory for activations (only one mini-batch of activations is in memory at a time).
- When the model itself is too large to fit on a single GPU, you need **model parallelism**. There are two main flavours.

- **Tensor parallelism** splits individual layers across GPUs. A large matrix multiply  $Y = XW$  can be split column-wise: partition  $W$  into  $[W_1, W_2]$  across two GPUs, compute  $Y_1 = XW_1$  and  $Y_2 = XW_2$  in parallel, then concatenate. This works for attention projections and feed-forward layers. It requires fast communication between GPUs (typically NVLink within a node) because partial results must be combined at every layer.
- **Pipeline parallelism** assigns different layers to different GPUs. GPU 0 runs layers 1-4, GPU 1 runs layers 5-8, and so on. Data flows through the pipeline like an assembly line. The naive approach has a “pipeline bubble” : while GPU 0 processes the forward pass for micro-batch 1, GPUs 1-3 sit idle. **Micro-batching** mitigates this by splitting the mini-batch into smaller micro-batches that flow through the pipeline in sequence, keeping all GPUs busy most of the time.
- **Hybrid parallelism** combines data, tensor, and pipeline parallelism. A typical large-model setup might use tensor parallelism within a node (8 GPUs connected by fast NVLink), pipeline parallelism across nodes, and data parallelism across groups of nodes. This is how models like GPT-4 and Llama are trained.
- The efficiency of distributed training depends heavily on **communication**. The key operation is **all-reduce**: given a value on each of  $N$  GPUs, compute the sum (or average) and distribute the result to all GPUs.
- A naive all-reduce sends all data to one GPU, sums it, and broadcasts back. This is  $O(N)$  in communication and creates a bottleneck at the root.
- **Ring all-reduce** is much more efficient. Arrange the  $N$  GPUs in a ring. Each GPU splits its data into  $N$  chunks. In  $N - 1$  steps, each GPU sends one chunk to its neighbour and receives a chunk from its other neighbour, accumulating partial sums. After another  $N - 1$  steps, the full sum is propagated to all GPUs. Total data transferred per GPU:  $2(N - 1)/N$  times the data size, which approaches  $2 \times$  as  $N$  grows. Crucially, this does not increase with  $N$ , making it bandwidth-optimal.

### Ring All-Reduce: Efficient Gradient Synchronisation



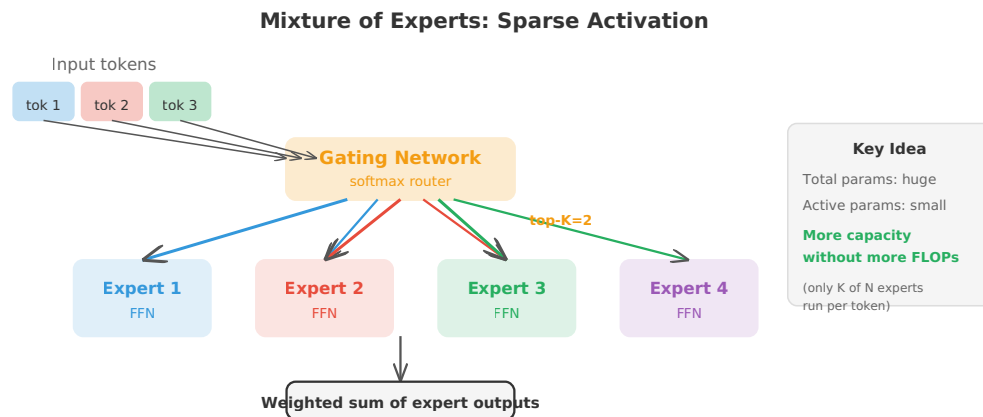
Four GPUs arranged in a ring, each passing gradient chunks to its neighbour until all have the complete sum

- **Parameter servers** are an alternative architecture where dedicated server nodes hold the model parameters. Workers compute gradients and send them to the server, which updates parameters and sends them back. This is simpler but can create communication bottlenecks at the server.
- **NCCL** (NVIDIA Collective Communications Library) is the standard library for GPU-to-GPU communication. It provides optimised implementations of all-reduce, all-gather, broadcast, and other collective operations, automatically choosing the best algorithm for the network topology.
- **Scaling laws** describe how model performance improves with compute, data, and model size. The original Kaplan et al. (2020) scaling laws found that loss decreases as a power law with each:

$$L(N) \propto N^{-\alpha_N}, \quad L(D) \propto D^{-\alpha_D}, \quad L(C) \propto C^{-\alpha_C}$$

- where  $N$  is the number of parameters,  $D$  is the dataset size, and  $C$  is the compute budget.
- The **Chinchilla scaling laws** (Hoffmann et al., 2022) showed that most models were undertrained: for a given compute budget, you should train a smaller model on more data than previously thought. The optimal ratio is roughly 20 tokens per parameter. A 7B model should see about 140B tokens, not the 300B tokens that Llama 1 used with a 65B model. This finding shifted the field toward “compute-optimal” training.
- **Mixture of Experts (MoE)** is an architecture that scales model capacity without proportionally scaling compute. Instead of one feed-forward network per transformer layer, you have  $N$  “expert” networks (each a standard FFN). A

**gating network** (router) examines each token and sends it to the top-K experts (typically  $K = 1$  or  $K = 2$ ).



Tokens routed through a gating network to selected experts, with top-K sparse routing and weighted combination of outputs

- The total parameter count is much larger (because you have  $N$  experts), but the FLOPs per token stay roughly constant (because only  $K$  experts activate per token). For example, Mixtral 8x7B has 47B total parameters but only uses about 13B per forward pass, giving the performance of a much larger model at the cost of a smaller one.
- MoE introduces challenges. **Load balancing**: if the router sends most tokens to the same expert, the others are wasted. An auxiliary loss encourages uniform routing. **Communication**: different experts may live on different GPUs, so routing tokens requires all-to-all communication, which is expensive.
- **Fault tolerance** is critical when training runs last weeks or months on thousands of GPUs. If a single GPU fails, you do not want to lose all progress. **Checkpointing** periodically saves model weights, optimizer states, and the training state (learning rate, step count, data position) to disk. If a failure occurs, you restart from the last checkpoint.
- **Gradient checkpointing** (also called activation recomputation) is a memory optimisation, not a fault-tolerance mechanism. During the forward pass, instead of saving all activations for the backward pass, you only save activations at certain checkpoints. During the backward pass, you recompute the missing activations from the checkpoints. This trades compute for memory: it increases the forward-pass cost by roughly 33% but can reduce activation memory by a factor of

---

$\sqrt{L}$

(where  $L$  is the number of layers).

- Putting it all together, training a frontier model combines all of these techniques: BF16 mixed precision, data parallelism across thousands of GPUs with ring all-reduce, tensor parallelism within nodes, pipeline parallelism across nodes, gradient checkpointing to reduce memory, MoE for parameter efficiency, and regular checkpointing for fault tolerance. The systems engineering is as challenging as the algorithm design.
- To summarise the distributed training toolkit:

Technique	What It Does	Tradeoff
Mixed precision (BF16)	Halves memory for activations/grads	Slight numerical differences
Data parallelism	Scales batch size across GPUs	Communication overhead for gradient sync
Tensor parallelism	Splits layers across GPUs	Requires fast interconnect
Pipeline parallelism	Splits model stages across GPUs	Pipeline bubble (wasted compute)
Gradient accumulation	Simulates large batches	Slower (multiple forward/backward passes)
Gradient checkpointing	Reduces activation memory	~33% more compute
Ring all-reduce	Efficient gradient averaging	Bandwidth-limited for large models
MoE	More capacity, same FLOPs	Load balancing, routing complexity
Scaling laws	Guides compute allocation	Empirical, may not hold at all scales

## 30.1 Coding Tasks (use CoLab or notebook)

1. Compute the FLOPs and memory requirements for a transformer layer. Given hidden dimension  $d$ , sequence length  $n$ , batch size  $B$ , and number of layers, estimate the total training cost.

```
import jax.numpy as jnp

def transformer_layer_flops(d, n, B):
 """Approximate FLOPs for one transformer layer forward pass."""
 # QKV projections: $3 * (B * n * d * d) * 2$ (multiply-add)
 qkv_flops = 3 * 2 * B * n * d * d
 # Attention: $(B * n * n * d) * 2$ for QK^T , $(B * n * n * d) * 2$ for $attn * V$
 attn_flops = 2 * 2 * B * n * n * d
 # Output projection: $(B * n * d * d) * 2$
 out_flops = 2 * B * n * d * d
 # FFN: two layers, $d \rightarrow 4d$ and $4d \rightarrow d$: $2 * (B * n * d * 4d) * 2$
 ffn_flops = 2 * 2 * B * n * d * 4 * d
 return qkv_flops + attn_flops + out_flops + ffn_flops

def transformer_layer_memory(d, n, B, dtype_bytes=2):
 """Approximate activation memory (bytes) for one layer."""
 # QKV: $3 * B * n * d$
 qkv_mem = 3 * B * n * d * dtype_bytes
 # Attention weights: $B * heads * n * n$ (approx $B * n * n * sizeof$)
 attn_mem = B * n * n * dtype_bytes
 # FFN intermediate: $B * n * 4d$
 ffn_mem = B * n * 4 * d * dtype_bytes
 return qkv_mem + attn_mem + ffn_mem

Example: GPT-2 scale
d, n, B, L = 1024, 1024, 8, 24
fwd_flops = transformer_layer_flops(d, n, B)
total_flops = 3 * L * fwd_flops # 3x for forward + backward
act_mem = L * transformer_layer_memory(d, n, B)
param_count = L * (12 * d * d + 13 * d) # approximate

print(f"Model: d={d}, n={n}, B={B}, L={L}")
print(f"Parameters: {param_count / 1e6:.0f}M")
print(f"FLOPs per step: {total_flops / 1e12:.2f} TFLOPs")
print(f"Activation memory: {act_mem / 1e9:.2f} GB (BF16)")
print(f"Parameter memory (FP32): {param_count * 4 / 1e9:.2f} GB")
```

```
print(f"Adam optimizer memory: {param_count * 8 / 1e9:.2f} GB")
print(f"Total training memory: {(param_count * 16 + act_mem) / 1e9:.2f} GB")
```

1. Simulate data-parallel training. Split a dataset across multiple “virtual GPUs,” compute gradients independently, average them, and verify the result matches single-GPU training.

```
import jax
import jax.numpy as jnp

Simple linear model: $y = wx + b$
key = jax.random.PRNGKey(0)
X = jax.random.normal(key, (64, 4))
w_true = jnp.array([1.0, -2.0, 3.0, 0.5])
y = X @ w_true + 0.1 * jax.random.normal(key, (64,))

def loss_fn(w, X, y):
 return jnp.mean((X @ w - y) ** 2)

grad_fn = jax.grad(loss_fn)

Single GPU: full batch gradient
w = jnp.zeros(4)
grad_single = grad_fn(w, X, y)

Data parallel: split across 4 "GPUs"
n_gpus = 4
chunk_size = len(X) // n_gpus
grads = []
for i in range(n_gpus):
 X_chunk = X[i*chunk_size:(i+1)*chunk_size]
 y_chunk = y[i*chunk_size:(i+1)*chunk_size]
 grads.append(grad_fn(w, X_chunk, y_chunk))

All-reduce: average gradients
grad_parallel = jnp.mean(jnp.stack(grads), axis=0)

print("Single-GPU gradient:", grad_single)
print("Data-parallel gradient (avg):", grad_parallel)
print(f"Match: {jnp.allclose(grad_single, grad_parallel, atol=1e-5)}")

Train both and compare
w_single, w_parallel = jnp.zeros(4), jnp.zeros(4)
lr = 0.1
for step in range(100):
 w_single = w_single - lr * grad_fn(w_single, X, y)
```



```

grads = [grad_fn(w_parallel, X[i*chunk_size:(i+1)*chunk_size],
 y[i*chunk_size:(i+1)*chunk_size]) for i in range(n_gpus)]
avg_grad = jnp.mean(jnp.stack(grads), axis=0)
w_parallel = w_parallel - lr * avg_grad

print(f"\nAfter 100 steps:")
print(f"Single-GPU weights: {w_single}")
print(f>Data-parallel weights: {w_parallel}")
print(f"Max difference: {jnp.max(jnp.abs(w_single - w_parallel)).2e}")

```

1. Implement a simple Mixture of Experts layer. Create a gating network that routes tokens to top-K experts and combine their outputs.

```

import jax
import jax.numpy as jnp

def expert_fn(x, W1, b1, W2, b2):
 """Simple 2-layer FFN expert."""
 h = jnp.maximum(0, x @ W1 + b1) # ReLU
 return h @ W2 + b2

def moe_layer(x, gate_W, experts_params, top_k=2):
 """
 MoE forward pass.
 x: (batch, d_model)
 gate_W: (d_model, n_experts)
 experts_params: list of (W1, b1, W2, b2) per expert
 """
 n_experts = len(experts_params)

 # Gating: compute routing scores
 gate_logits = x @ gate_W # (batch, n_experts)
 gate_probs = jax.nn.softmax(gate_logits, axis=-1)

 # Top-K selection
 top_k_indices = jnp.argsort(-gate_probs, axis=-1)[: , :top_k]
 top_k_probs = jnp.take_along_axis(gate_probs, top_k_indices, axis=-1)
 # Renormalise
 top_k_probs = top_k_probs / jnp.sum(top_k_probs, axis=-1, keepdims=True)

 # Compute expert outputs (simplified: run all experts, mask later)
 expert_outputs = jnp.stack([
 expert_fn(x, *experts_params[i]) for i in range(n_experts)
], axis=1) # (batch, n_experts, d_model)

```

```

 # Gather top-K expert outputs and weight them
 batch_idx = jnp.arange(x.shape[0])[:, None]
 selected_outputs = expert_outputs[batch_idx, top_k_indices]
 # (batch, top_k, d_model)
 output = jnp.sum(selected_outputs * top_k_probs[:, :, None], axis=1)

 return output, gate_probs

Setup
key = jax.random.PRNGKey(42)
batch, d_model, d_ff, n_experts = 8, 16, 32, 4

Initialise experts
experts_params = []
for i in range(n_experts):
 k1, k2, key = jax.random.split(key, 3)[0], jax.random.split(key, 3)[1],
 jax.random.split(key, 3)[2]
 experts_params.append((
 jax.random.normal(k1, (d_model, d_ff)) * 0.1,
 jnp.zeros(d_ff),
 jax.random.normal(k2, (d_ff, d_model)) * 0.1,
 jnp.zeros(d_model),
))

key, subkey = jax.random.split(key)
gate_W = jax.random.normal(subkey, (d_model, n_experts)) * 0.1
x = jax.random.normal(key, (batch, d_model))

output, gate_probs = moe_layer(x, gate_W, experts_params, top_k=2)

print(f"Input shape: {x.shape}")
print(f"Output shape: {output.shape}")
print(f"Gate probabilities (first sample): {gate_probs[0]}")
print(f"Expert usage (avg across batch):")
for i in range(n_experts):
 usage = jnp.mean(gate_probs[:, i])
 print(f" Expert {i}: {usage:.3f}")

```

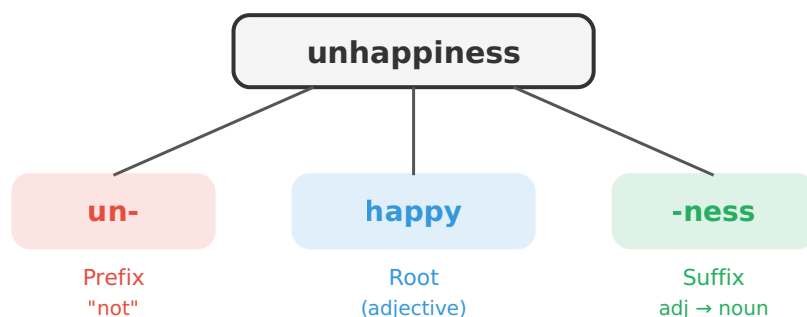
## 31 Linguistic Foundations

Linguistics provides the structural vocabulary that NLP systems implicitly learn and exploit. This file covers morphology, syntax, semantics, pragmatics, phonology,

constituency and dependency parsing, and the distributional hypothesis, the human-language science that grounds tokenisation, grammar, and meaning in AI.

- Before we can build systems that understand or generate language, we need to understand how language itself works.
- Linguistics is the scientific study of language, and it provides the conceptual vocabulary that NLP borrows from constantly.
- Even modern neural models, which learn language from raw data, implicitly rediscover many of the structures that linguists have catalogued for decades.
- Language has structure at every level: the sounds that make up words, the parts that make up words, the rules that combine words into sentences, the meaning those sentences carry, and the way context shapes interpretation. We will work through each level from the bottom up.
- **Morphology** is the study of the internal structure of words. Words are not atomic; they are built from smaller meaningful units called **morphemes**.
- The word “unhappiness” contains three morphemes: “un-” (a prefix meaning “not” ), “happy” (the root), and “-ness” (a suffix that turns an adjective into a noun). Each morpheme contributes to the meaning.
- A **root** (or stem) is the core morpheme that carries the primary meaning. “Happy,” “run,” “compute” are roots.
- An **affix** is a morpheme that attaches to a root to modify it.
- English has **prefixes** (before the root: un-, re-, pre-) and **suffixes** (after: -ing, -ed, -tion). Some languages also have infixes (inserted inside the root) and circumfixes (wrapping around).

### Morpheme Analysis: "unhappiness"



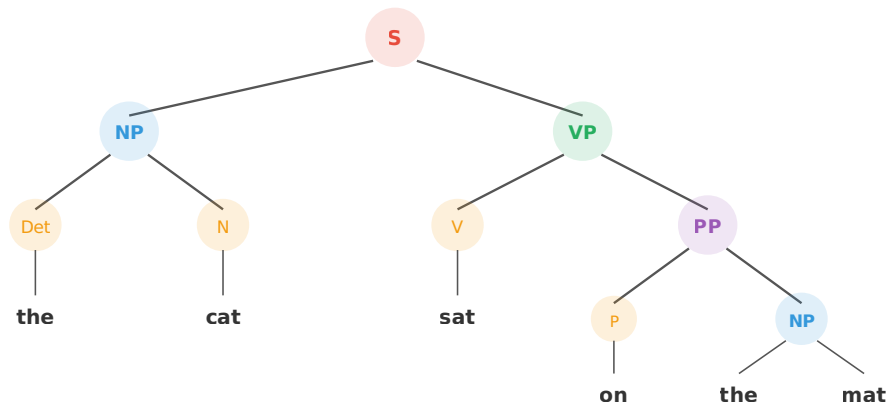
3 morphemes: prefix + root + suffix → derived noun meaning "state of not being happy"

Morpheme tree: “unhappiness” split into prefix “un” , root “happy” , suffix “ness”

- There are two kinds of morphological processes. **Inflection** changes the grammatical properties of a word without changing its core meaning or part of speech: “run” becomes “runs” (third person), “running” (progressive), “ran” (past tense). The word is still a verb meaning the same thing.
- **Derivation** creates a new word, often changing the part of speech: “happy” (adjective) becomes “happiness” (noun), “compute” (verb) becomes “computation” (noun) becomes “computational” (adjective). Each derivation shifts meaning and grammatical category.
- Languages vary enormously in morphological complexity. English is relatively **analytic** (few morphemes per word, relying on word order).
- Turkish and Finnish are **agglutinative** (words can contain many morphemes strung together). Arabic and Hebrew use **templatic** morphology (roots are consonant skeletons like k-t-b for “write,” and vowel patterns are inserted to create different words: kitab “book,” kataba “he wrote,” maktub “written” ).
- Morphology matters for NLP because it affects tokenisation. A word-level tokeniser treats “run,” “runs,” “running,” and “ran” as four unrelated symbols.
- A morphologically-aware system recognises they share a root. Subword tokenisation (BPE, WordPiece), which we will cover in file 02, is a statistical approximation to morphological analysis.
- **Syntax** is the study of how words combine into phrases and sentences. Every language has rules governing word order and structure; violating them produces gibberish.
- “The cat sat on the mat” is grammatical English; “Mat the on sat cat the” is not.
- There are two main frameworks for describing syntactic structure.
- **Phrase structure grammar** (also called constituency grammar) says sentences are built by nesting phrases inside phrases. A sentence (S) consists of a noun phrase (NP) and a verb phrase (VP).

- A noun phrase might be a determiner (Det) followed by a noun (N). A verb phrase might be a verb (V) followed by a noun phrase. These rules build a tree:

**Constituency (Parse) Tree: "the cat sat on the mat"**



Each internal node is a phrase type; each leaf is a word. Phrases nest inside phrases.

Constituency tree for “the cat sat on the mat” : S branches into NP and VP, NP into Det “the” and N “cat” , VP into V “sat” and PP, PP into P “on” and NP

- This tree is called a **constituency tree** (or parse tree). Each internal node is a phrase type, each leaf is a word. The tree captures the hierarchical grouping: “on the mat” is a unit (prepositional phrase), “sat on the mat” is a unit (verb phrase), and the whole thing is a sentence.
- A **context-free grammar (CFG)** formalises these rules. It consists of a set of production rules, each of the form  $A \rightarrow \alpha$ , where A is a non-terminal symbol (a phrase type like NP or VP) and  $\alpha$  is a sequence of terminals (words) and non-terminals. For example:

```

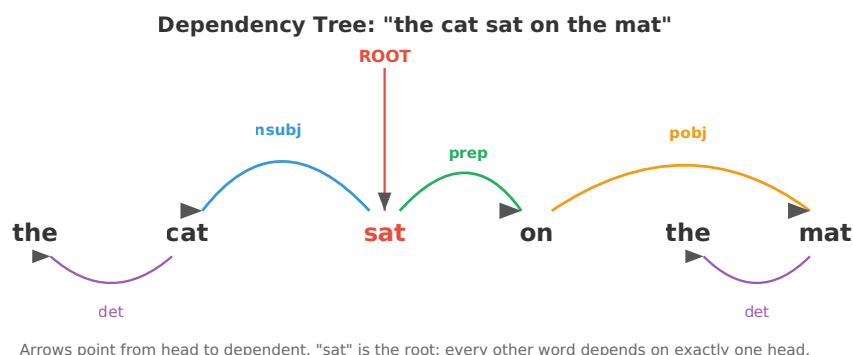
S → NP VP
NP → Det N
NP → Det N PP
VP → V NP
VP → V PP
PP → P NP
Det → "the" | "a"
N → "cat" | "mat" | "dog"
V → "sat" | "chased"
P → "on" | "under"

```

- Starting from S and repeatedly applying rules, you can generate all sentences the grammar allows. Parsing is the reverse: given a sentence, find the tree (or trees) that produced it. A sentence with multiple valid parse trees is

**syntactically ambiguous.** “I saw the man with the telescope” has two parses: I used a telescope to see the man, or I saw a man who had a telescope.

- **Dependency grammar** takes a different perspective. Instead of phrase nesting, it describes direct relationships between words. Each word in a sentence depends on exactly one other word (its **head**), except the root of the sentence. The result is a **dependency tree** where edges are labelled with grammatical relations (subject, object, modifier, etc.).



Dependency tree for “the cat sat on the mat” : arrows from “sat” to “cat” (nsubj) and to “on” (prep), from “on” to “mat” (pobj), from “cat” to “the” (det), from “mat” to “the” (det)

- In the dependency view, “sat” is the root. “Cat” depends on “sat” as its subject (nsubj). “On” depends on “sat” as a prepositional modifier. “Mat” depends on “on” as the object of the preposition. Every word hangs off exactly one head, creating a tree.
- Dependency grammar has become the dominant framework in modern NLP because dependency trees are easier to produce with statistical parsers and the relations map more directly to semantic roles (who did what to whom).
- **Valency** describes how many arguments a verb requires. “Sleep” is **intransitive** (one argument: the sleeper). “Eat” is **transitive** (two: the eater and the eaten). “Give” is **ditransitive** (three: the giver, the thing given, and the receiver). Knowing a verb’s valency constrains which parse trees are valid.
- **Semantics** is the study of meaning. Syntax tells you how a sentence is structured; semantics tells you what it means.

- **Lexical semantics** concerns the meaning of individual words. Words are related to each other in systematic ways:
  - **Synonymy**: words with (nearly) the same meaning. “Big” and “large” are synonyms. True perfect synonymy is rare; there are almost always subtle differences in connotation or usage.
  - **Antonymy**: words with opposite meanings. “Hot” and “cold,” “buy” and “sell.”
  - **Hypernymy/hyponymy**: “is-a” relationships. “Dog” is a hyponym of “animal” (a dog is a kind of animal). “Animal” is a hypernym of “dog.” These form taxonomic hierarchies.
  - **Meronymy**: “part-of” relationships. “Wheel” is a meronym of “car.”
  - **Polysemy**: a single word with multiple related meanings. “Bank” means a financial institution or a river bank. Context disambiguates.
  
- **Word sense disambiguation (WSD)** is the task of determining which sense of a polysemous word is intended in a given context. In “I deposited money at the bank,” the financial sense is correct. In “We sat by the river bank,” the geographical sense is. WSD was a central problem in early NLP; modern contextual embeddings (ELMo, BERT) largely solve it by producing different vector representations for different uses of the same word.
  
- **Compositional semantics** asks how the meanings of individual words combine to form the meaning of a phrase or sentence. The principle of **compositionality** (attributed to Frege) states that the meaning of a complex expression is determined by the meanings of its parts and the rules used to combine them. “The cat chased the dog” means something different from “the dog chased the cat” because the syntactic structure (who is the subject vs object) interacts with the word meanings.
  
- Not all meaning is compositional. **Idioms** like “kick the bucket” (meaning “to die” ) have meanings that cannot be derived from their parts. These are a challenge for any compositional approach.
  
- **Distributional semantics** is the computational approach to meaning that underpins modern NLP. The **distributional hypothesis** (Firth, 1957) states: “You shall know a word by the company it keeps.” Words that appear in similar contexts tend to have similar meanings. This is the theoretical foundation for word embeddings (Word2Vec, GloVe), which we will explore in file 03.

- **Pragmatics** studies how context affects meaning. The same sentence can mean different things depending on who says it, when, where, and why.
- “Can you pass the salt?” is syntactically a yes/no question about ability. Pragmatically, it is a request. You would not answer “Yes, I can” and then sit still. Understanding this requires knowledge beyond the literal words, specifically, the conventions of **speech acts**.
- **Speech act theory** (Austin, Searle) distinguishes between:
  - **Locutionary act**: the literal content ( “Can you pass the salt?” )
  - **Illocutionary act**: the intended function (a request)
  - **Perlocutionary act**: the effect on the listener (they pass the salt)
- **Implicature** (Grice) is meaning that is implied but not explicitly stated. If someone asks “Is John a good cook?” and you reply “He’ s British,” you have not answered the question literally, but the listener can infer (through cultural stereotypes, fairly or not) that you mean “no.” Grice’ s **cooperative principle** says speakers generally try to be informative, truthful, relevant, and clear, and listeners interpret utterances assuming these maxims hold.
- **Coreference** is a pragmatic phenomenon where different expressions refer to the same entity. In “Alice went to the store. She bought milk,” “she” refers to Alice. Resolving coreference is essential for understanding multi-sentence text and is a key NLP task.
- **Discourse structure** describes how sentences connect to form coherent text. A narrative has a beginning, middle, and end. An argument has claims and evidence. **Rhetorical Structure Theory (RST)** analyses text as a tree of discourse relations (elaboration, contrast, cause, etc.) between segments.
- Pragmatics is where NLP gets hardest. Modern language models handle much of syntax and semantics implicitly through training data, but pragmatic reasoning, understanding sarcasm, implicature, and context-dependent meaning, remains a frontier challenge.
- **Phonology** studies the sound systems of languages. While this chapter focuses on text, a brief overview bridges to the audio and speech chapter (Chapter 09).
- A **phoneme** is the smallest unit of sound that distinguishes meaning. English has about 44 phonemes. The words “bat” and “pat” differ by one phoneme (/b/ vs /p/), which changes the meaning entirely. This is called a **minimal pair**.



- **Allophones** are different physical realisations of the same phoneme that do not change meaning. The “p” in “pin” (aspirated, with a puff of air) and the “p” in “spin” (unaspirated) are allophones of /p/ in English; a native speaker treats them as the same sound.
- The **International Phonetic Alphabet (IPA)** provides a standardised notation for phonemes across all languages. The word “cat” is transcribed as /kæt/. IPA is the bridge between written text and speech systems.
- **Prosody** covers the rhythm, stress, and intonation of speech. “I didn’ t say he stole the money” has seven different meanings depending on which word is stressed. Prosody carries information that text alone loses, which is why text-to-speech systems must model it carefully.
- In NLP, phonological knowledge appears in text-to-speech (grapheme-to-phoneme conversion), speech recognition (mapping acoustic signals to phonemes), and even in spelling correction and transliteration.

## 31.1 Coding Tasks (use CoLab or notebook)

---

1. Build a simple morphological analyser that splits English words into likely morphemes using a list of common prefixes and suffixes.

```

prefixes = ['un', 're', 'pre', 'dis', 'mis', 'over', 'under', 'out', 'non']
suffixes = ['ing', 'ed', 'ly', 'ness', 'ment', 'tion', 'able', 'ible', 'er',
'est', 'ful', 'less', 'ous']

def analyse_morphemes(word):
 """Simple morpheme analysis using known affixes."""
 parts = []
 remaining = word.lower()

 # Check prefixes
 for p in sorted(prefixes, key=len, reverse=True):
 if remaining.startswith(p) and len(remaining) > len(p) + 2:
 parts.append(f"prefix: {p}")
 remaining = remaining[len(p):]
 break

 # Check suffixes
 for s in sorted(suffixes, key=len, reverse=True):
 if remaining.endswith(s) and len(remaining) > len(s) + 2:
 root = remaining[:-len(s)]

```

```

 parts.append(f"[root: {root}]")
 parts.append(f"[suffix: {s}]")
 remaining = None
 break

 if remaining is not None:
 parts.append(f"[root: {remaining}]")

 return parts

for word in ['unhappiness', 'reusable', 'disconnected', 'overreacting',
'kindness']:
 print(f"{word:20s} → {' + '.join(analyse_morphemes(word))}")

```

1. Implement a simple context-free grammar parser using recursive descent.  
Define a small grammar and parse a sentence into a constituency tree.

```

class CFGParser:
 """Recursive descent parser for a tiny English grammar."""
 def __init__(self, tokens):
 self.tokens = tokens
 self.pos = 0

 def peek(self):
 return self.tokens[self.pos] if self.pos < len(self.tokens) else None

 def consume(self, expected=None):
 tok = self.peek()
 if expected and tok != expected:
 return None
 self.pos += 1
 return tok

 def parse_det(self):
 if self.peek() in ('the', 'a'):
 return ('Det', self.consume())
 return None

 def parse_noun(self):
 if self.peek() in ('cat', 'dog', 'mat', 'man'):
 return ('N', self.consume())
 return None

 def parse_verb(self):
 if self.peek() in ('sat', 'chased', 'saw'):
 return ('V', self.consume())

```

```

 return None

 def parse_prep(self):
 if self.peek() in ('on', 'under', 'with'):
 return ('P', self.consume())
 return None

 def parse_np(self):
 save = self.pos
 det = self.parse_det()
 noun = self.parse_noun()
 if det and noun:
 # Check for optional PP
 pp = self.parse_pp()
 if pp:
 return ('NP', det, noun, pp)
 return ('NP', det, noun)
 self.pos = save
 return None

 def parse_pp(self):
 save = self.pos
 prep = self.parse_prep()
 np = self.parse_np()
 if prep and np:
 return ('PP', prep, np)
 self.pos = save
 return None

 def parse_vp(self):
 save = self.pos
 verb = self.parse_verb()
 if verb:
 np = self.parse_np()
 if np:
 return ('VP', verb, np)
 pp = self.parse_pp()
 if pp:
 return ('VP', verb, pp)
 self.pos = save
 return None

 def parse_sentence(self):
 np = self.parse_np()
 vp = self.parse_vp()
 if np and vp and self.pos == len(self.tokens):
 return ('S', np, vp)

```

```

 return None

def print_tree(tree, indent=0):
 if isinstance(tree, str):
 print(' ' * indent + tree)
 elif isinstance(tree, tuple):
 print(' ' * indent + tree[0])
 for child in tree[1:]:
 print_tree(child, indent + 2)

sentences = [
 "the cat sat on the mat",
 "a dog chased the cat",
]

for sent in sentences:
 tokens = sent.split()
 parser = CFGParser(tokens)
 tree = parser.parse_sentence()
 print(f"\n'{sent}':")
 if tree:
 print_tree(tree)
 else:
 print(" (no parse found)")

```

1. Explore lexical relations by building a simple word graph. Given a small vocabulary with synonym, antonym, and hypernym relations, find paths between words.

```

relations = {
 ('big', 'large'): 'synonym',
 ('big', 'small'): 'antonym',
 ('small', 'tiny'): 'synonym',
 ('dog', 'animal'): 'hypernym',
 ('cat', 'animal'): 'hypernym',
 ('puppy', 'dog'): 'hypernym',
 ('happy', 'glad'): 'synonym',
 ('happy', 'sad'): 'antonym',
 ('hot', 'cold'): 'antonym',
 ('hot', 'warm'): 'synonym',
}

Build adjacency list
from collections import defaultdict, deque

graph = defaultdict(list)

```

```

for (w1, w2), rel in relations.items():
 graph[w1].append((w2, rel))
 graph[w2].append((w1, rel))

def find_path(start, end):
 """BFS to find a path between two words through the relation graph."""
 queue = deque([(start, [(start, None)])])
 visited = {start}
 while queue:
 node, path = queue.popleft()
 if node == end:
 return path
 for neighbor, rel in graph[node]:
 if neighbor not in visited:
 visited.add(neighbor)
 queue.append((neighbor, path + [(neighbor, rel)]))
 return None

pairs = [('big', 'tiny'), ('puppy', 'cat'), ('happy', 'sad')]
for w1, w2 in pairs:
 path = find_path(w1, w2)
 if path:
 steps = " → ".join(f"{w}({r})" if r else w for w, r in path)
 print(f"{w1} → {w2}: {steps}")
 else:
 print(f"{w1} → {w2}: no path found")

```

## 32 Text Processing and Classic NLP

Text processing transforms raw characters into structured representations that models can consume. This file covers tokenisation (word, subword, BPE, WordPiece), text normalisation, edit distance, TF-IDF, n-gram language models, POS tagging, NER, and sentiment analysis, the classical NLP pipeline that still underpins modern systems.

- Raw text is messy. Before any NLP model can work with language, the text must be cleaned, normalised, and converted into a structured representation. This file covers the pipeline from raw characters to features that models can consume, along with the classical NLP algorithms that dominated before deep learning.

- **Text normalisation** transforms raw text into a canonical form. The goal is to reduce irrelevant variation so that “Hello” , “hello” , “HELLO” and “héllo” are treated appropriately.
- **Case folding** converts text to lowercase. This collapses “The” and “the” into one token. It helps for most tasks, but destroys useful information in some cases: “US” (the country) vs “us” (the pronoun), or “Apple” (the company) vs “apple” (the fruit).
- **Unicode normalisation** handles the fact that the same character can be encoded multiple ways. The character “é” can be a single code point (U+00E9) or a base “e” plus a combining accent (U+0065 + U+0301). NFC normalisation composes them into one code point; NFD decomposes them. Without normalisation, two identical-looking strings may not match.
- **Edit distance** measures how different two strings are. The **Levenshtein distance** counts the minimum number of single-character insertions, deletions, and substitutions needed to transform one string into another. “kitten” → “sitting” has edit distance 3 (k→s, e→i, insert g).
- Edit distance is computed using dynamic programming (we review in algorithms chapter). Define  $D[i][j]$  as the distance between the first  $i$  characters of string  $s$  and the first  $j$  characters of string  $t$ :

```

D[i][j] = \begin{cases} j & \text{if } i = 0 \\ i & \text{if } j = 0 \\ D[i-1][j-1] & \text{if } s[i] = t[j] \\ 1 + \min(D[i-1][j], D[i][j-1], D[i-1][j-1]) & \text{otherwise} \end{cases}

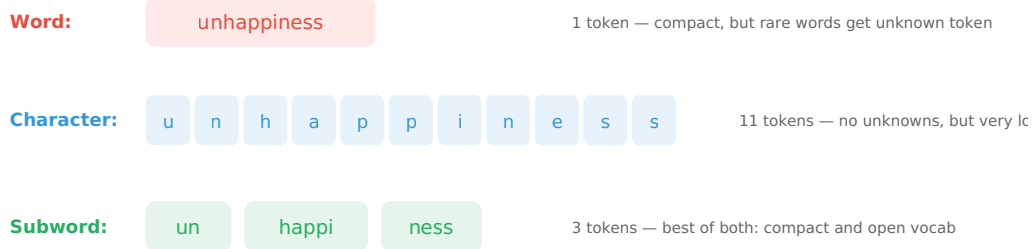
```

- Edit distance powers spelling correction, fuzzy matching, and DNA sequence alignment. In NLP, it is used to handle typos and find similar words.
- **Tokenisation** splits text into discrete units (tokens) that a model can process. This is the first and arguably most important preprocessing step. The choice of tokenisation strategy profoundly affects model behaviour.
- **Whitespace tokenisation** splits on spaces. Simple but naive: “New York” becomes two tokens, “don’ t” is one token (or split into “don” and “ t” depending on the splitter), and languages like Chinese and Japanese have no spaces between words at all.
- **Rule-based tokenisation** uses handcrafted patterns (regular expressions) to handle contractions, punctuation, and special cases. “I’ m” → “I” +

“ m” , “U.S.A.” stays as one token. Every language needs its own rules, which is labour-intensive.

- **Subword tokenisation** is the modern solution. Instead of splitting at word boundaries, it learns a vocabulary of frequent subword units from data. This elegantly handles unknown words: if “unhappiness” is not in the vocabulary, it might be split into “un” + “happi” + “ness” , preserving morphological structure.

#### Tokenisation Strategies for "unhappiness"



Subword tokenisation (BPE) approximates morphological analysis from data

Comparison of word-level, character-level, and subword tokenisation of “unhappiness” and “transformers”

- **Byte-Pair Encoding (BPE)** starts with individual characters as the vocabulary. It repeatedly finds the most frequent adjacent pair and merges them into a new token. After enough merges, common words are single tokens and rare words are split into frequent subword pieces.
- The BPE algorithm:
  1. Initialise the vocabulary with all individual characters in the training corpus
  2. Count the frequency of every adjacent token pair
  3. Merge the most frequent pair into a new token
  4. Repeat steps 2-3 for a desired number of merges (vocabulary size)
- For example, starting with “l o w” (5 times), “l o w e r” (2 times), “n e w e s t” (6 times): the most frequent pair might be “e s” → merge into “es” . Then “es t” → “est” . Then “n e w” → “new” . The final vocabulary contains both full words and subword pieces.
- **WordPiece** (used by BERT) is similar to BPE but selects merges based on likelihood rather than frequency. It merges the pair that maximises the

language model likelihood of the training data. Subword tokens that are not word-initial are prefixed with “##” (e.g., “playing” → “play” + “##ing” ).

- **Unigram** (used by SentencePiece) takes the opposite approach: start with a large vocabulary and iteratively remove tokens whose removal least hurts the training data likelihood. The final vocabulary is the set of subword units that best explain the corpus.
- **SentencePiece** is a language-agnostic tokenisation library that treats the input as a raw byte stream (no pre-tokenisation on spaces). This makes it work for any language, including those without spaces. It implements both BPE and Unigram algorithms.
- The vocabulary size is a key hyperparameter. Typical choices range from 30,000 to 100,000 tokens. Larger vocabularies mean fewer tokens per sequence (more efficient) but a larger embedding table. Smaller vocabularies mean more subword splits and longer sequences.
- Both techniques reduce words to a base form, but they differ in approach.
- **Stemming** chops off suffixes using crude rules. The Porter stemmer reduces “running” to “run” , “happiness” to “happi” , and “studies” to “studi” . It is fast but imprecise: “university” and “universe” both stem to “univers” despite being unrelated.
- **Lemmatisation** uses vocabulary and morphological analysis to find the true dictionary form (lemma). “Running” → “run” , “better” → “good” , “mice” → “mouse” . It requires knowing the part of speech: “saw” as a verb lemmatises to “see” , but as a noun it stays “saw” .
- Modern subword tokenisation has largely replaced stemming and lemmatisation in neural NLP, but they remain useful in information retrieval and when working with smaller models or limited data.
- **Part-of-speech (POS) tagging** assigns a grammatical category to each word: noun, verb, adjective, determiner, etc. This is one of the oldest NLP tasks and is fundamental to syntactic analysis.
- The Penn Treebank tagset is the most common for English, with 36 tags (NN for singular noun, NNS for plural noun, VB for base verb, VBD for past tense, JJ for adjective, etc.).



- POS tagging is tricky because many words are ambiguous. “Book” can be a noun ( “the book” ) or a verb ( “book a flight” ). “Run” has dozens of senses across parts of speech. Context is essential.
- Early taggers used **Hidden Markov Models (HMMs)** from chapter 05. The hidden states are POS tags, the observations are words. The transition probabilities capture tag sequences (a determiner is likely followed by a noun or adjective), and the emission probabilities capture which words appear with which tags. The Viterbi algorithm finds the most likely tag sequence.
- The HMM model for POS tagging:

⊙

t

$$t_{1:n} = \arg$$

max

$t_{1:n}$

n

$\prod$

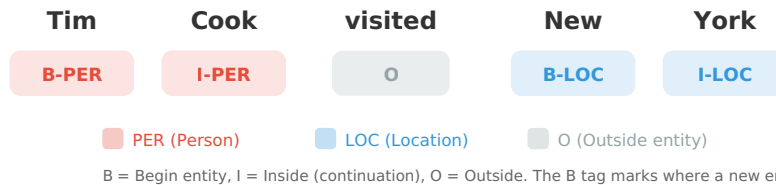
i=1

$$P(w_i | t_i) \cdot P(t_i | t_{i-1})$$

- Modern POS taggers use neural networks (bidirectional LSTMs or transformers) and achieve over 97% accuracy on English, approaching human performance.
- **Named Entity Recognition (NER)** identifies and classifies proper names and other specific entities in text: persons, organisations, locations, dates, monetary amounts, etc.
- In “Apple CEO Tim Cook announced the event in Cupertino on Monday,” a NER system should identify: Apple (ORG), Tim Cook (PER), Cupertino (LOC), Monday (DATE).
- NER is typically framed as **sequence labelling** using **BIO tagging** (also called IOB tagging). Each token gets a tag:
  - **B-TYPE**: beginning of an entity of type TYPE
  - **I-TYPE**: inside (continuation of) an entity of type TYPE
  - **O**: outside any entity

- “Tim Cook visited New York” becomes: Tim/B-PER Cook/I-PER visited/O New/B-LOC York/I-LOC. The B tag marks where a new entity starts, which is important when two entities of the same type are adjacent.

#### BIO Tagging for Named Entity Recognition



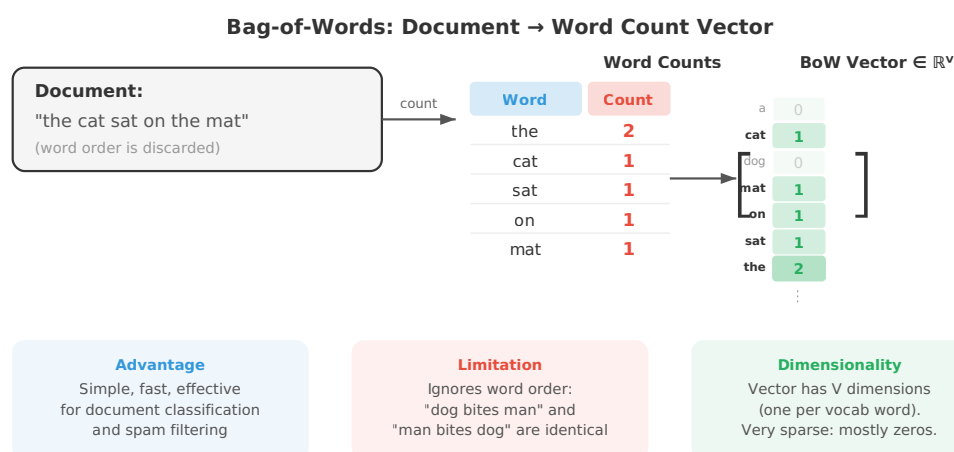
Sentence with BIO tags colour-coded: B-PER (red), I-PER (red), O (grey), B-LOC (blue), I-LOC (blue)

- Classical NER used **Conditional Random Fields (CRFs)** from chapter 05, which model the conditional probability of the entire tag sequence given the input. Unlike HMMs, which are generative ( $P(x, y)$ ), CRFs are discriminative and model  $P(y \mid x)$  directly. A linear-chain CRF defines:

$$P(y_{1:n} \mid x_{1:n}) = \frac{1}{Z(x)} \exp \left( \sum_{i=1}^n \left[ \sum_k \lambda_k f_k(y_i, x, i) + \sum_j \mu_j g_j(y_i, y_{i-1}, x, i) \right] \right)$$

- Here  $f_k$  are **emission features** (how likely tag  $y_i$  is given the input at position  $i$ ) and  $g_j$  are **transition features** (how likely tag  $y_i$  is given the previous tag  $y_{i-1}$ ).
- The partition function  $Z(x) = \sum_y \exp(\dots)$  sums over all possible tag sequences to normalise the distribution. Training maximises the conditional log-likelihood, which requires computing  $Z(x)$  efficiently using the forward algorithm (chapter 05).
- The key advantage over independently classifying each token: the CRF's transition features enforce structural constraints (e.g., I-PER should only follow B-PER or I-PER, never appear after O).

- Modern NER stacks a CRF on top of a neural encoder (BiLSTM-CRF or BERT-CRF), where the neural network produces the emission scores and the CRF layer learns transition structure.
- **Syntactic parsing** converts a sentence into its syntactic structure, either a constituency tree or a dependency tree (both from file 01).
- The **CYK algorithm** (Cocke-Younger-Kasami) parses sentences with context-free grammars using dynamic programming.
- It requires the grammar to be in **Chomsky Normal Form** (every rule has either two non-terminals or one terminal on the right side). It fills a triangular table bottom-up: cells represent spans of the sentence, and each cell stores the non-terminals that can generate that span.
- CYK runs in  $O(n^3 \cdot |G|)$  time, where  $n$  is the sentence length and  $|G|$  is the grammar size. This is exact but slow for large grammars.
- **Shift-reduce parsing** processes the sentence left to right, maintaining a stack. At each step, it either **shifts** (pushes the next word onto the stack) or **reduces** (pops elements from the stack and replaces them with a phrase). A trained classifier decides the action at each step. This runs in  $O(n)$  time, making it much faster than CYK.
- **Dependency parsing** is now more common than constituency parsing in practice. Transition-based dependency parsers (like shift-reduce) and graph-based parsers (which score all possible edges and find the maximum spanning tree) are the two main approaches. Neural dependency parsers using BiLSTMs or transformers achieve state-of-the-art results.
- Before embeddings, NLP represented documents as vectors using simple counting methods.
- The **bag-of-words (BoW)** model represents a document as a vector of word counts, ignoring word order entirely. If the vocabulary has  $V$  words, each document is a vector in  $\mathbb{R}^V$  (connecting back to vector spaces from chapter 01). The entry for word  $w$  is the number of times  $w$  appears in the document.



Bag-of-words: a document is converted to a word count table, then to a sparse vector in  $\mathbb{R}^V$  with one entry per vocabulary word

- BoW is simple but surprisingly effective for tasks like document classification and spam filtering. Its main weakness is that it treats every word as equally important: “the” and “revolutionary” get equal weight.
- TF-IDF** (Term Frequency-Inverse Document Frequency) fixes this by weighting words based on how informative they are. Words that appear frequently in one document but rarely across the corpus are likely important for that document.

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

- Term frequency**  $\text{TF}(t, d)$  is often the raw count of term  $t$  in document  $d$  (or its log:  $1 + \log(\text{count})$ ).
- Inverse document frequency**  $\text{IDF}(t) = \log \frac{N}{|\{d : t \in d\}|}$ , where  $N$  is the total number of documents. Words appearing in every document (like “the”) get IDF close to 0. Rare words get high IDF.
- TF-IDF vectors can be compared using cosine similarity (from chapter 01) to measure document similarity. This is the foundation of classical information retrieval and search engines.
- A **language model** assigns a probability to a sequence of words. It answers: how likely is this sentence? Language models are central to machine translation, speech recognition, spelling correction, and text generation.
- The probability of a sentence  $w_1, w_2, \dots, w_n$  is, by the chain rule of probability (chapter 05):

$$P(w_1, w_2, \dots, w_n) = \prod_{i=1}^n$$

$$P(w_i \mid w_1, \dots, w_{i-1})$$

- This is exact but impractical: you would need to store probabilities for every possible history. The **Markov assumption** (chapter 05) truncates the history to the last  $k - 1$  words, giving an **n-gram model** (where  $n = k$ ).

- A **bigram model** ( $n = 2$ ) conditions only on the previous word:

$$P(w_i \mid w_1, \dots, w_{i-1}) \approx P(w_i \mid w_{i-1})$$

- A **trigram model** ( $n = 3$ ) conditions on the previous two words. N-gram probabilities are estimated by counting in a corpus:

$$P(w_i \mid w_{i-1}) = \frac{\text{count}(w_{i-1}, w_i)}{\text{count}(w_{i-1})}$$

- **Perplexity** measures how well a language model predicts a test set. It is the inverse probability of the test set, normalised by the number of words:

$$\text{PPL} = P(w_1, \dots, w_N)^{-1/N} = \exp \left( -\frac{1}{N} \right)$$

$$\sum_{i=1}^N$$

$$\log P(w_i \mid w_{<i})$$

- Lower perplexity means the model is less “surprised” by the test data and therefore better. A model that assigns uniform probability over a 10,000-word vocabulary has perplexity 10,000. A good bigram model might achieve perplexity around 200. Modern neural language models achieve perplexity below 20.
- Notice that perplexity is the exponentiated cross-entropy (from chapter 05’s information theory). Minimising cross-entropy loss during training directly minimises perplexity.

- **Smoothing** handles the zero-probability problem: if an n-gram never appeared in training, the model assigns it probability 0, which makes the entire sentence probability 0. **Laplace smoothing** (add-1) adds a small count to every n-gram:

$$P_{\text{Laplace}}(w_i \mid w_{i-1}) = \frac{\text{count}(w_{i-1}, w_i) + 1}{\text{count}(w_{i-1}) + V}$$

- This is too aggressive for large vocabularies (it steals too much probability from observed n-grams). **Kneser-Ney smoothing** is the gold standard for n-gram models. It combines two ideas: absolute discounting and a continuation probability for backoff.
- First, **absolute discounting** subtracts a fixed discount  $d$  (typically  $d \approx 0.75$ ) from each observed count, rather than adding pseudocounts. The freed probability mass is redistributed to unseen n-grams. The interpolated form is:

$$P_{\text{KN}}(w_i \mid w_{i-1}) = \frac{\max(\text{count}(w_{i-1}, w_i) - d, 0)}{\text{count}(w_{i-1})} + \lambda(w_{i-1}) \cdot P_{\text{cont}}(w_i)$$

- where  $\lambda(w_{i-1})$  is a normalising constant that distributes the discounted mass. The key innovation is the **continuation probability**  $P_{\text{cont}}(w_i)$ , which measures how many different contexts  $w_i$  appears in, rather than how often it appears overall:

$$P_{\text{cont}}(w_i) = \frac{|\{w' : \text{count}(w', w_i) > 0\}|}{|\{(w', w'') : \text{count}(w', w'') > 0\}|}$$

- The numerator counts how many distinct words precede  $w_i$  in the corpus. A word like “Francisco” appears in few contexts (almost always after “San” ), so even if “San Francisco” is very frequent, “Francisco” gets a low continuation probability and will not be predicted spuriously in other contexts.
- Conversely, common words like “the” appear after many different words and get high continuation probability. This captures the intuition that a word’s versatility matters more than its raw frequency for backoff estimation.
- N-gram models were the state of the art for decades. They are fast, interpretable, and require no training (just counting). But they struggle with long-range dependencies ( “The keys that I left on the table **are** missing” requires knowing the subject “keys” is plural, which is far from the verb).

Neural language models, starting with RNNs and culminating in transformers, address this limitation.

## 32.1 Coding Tasks (use CoLab or notebook)

---

1. Implement the Levenshtein edit distance using dynamic programming. Test it on word pairs and use it for simple spelling correction.

```
import jax.numpy as jnp

def edit_distance(s, t):
 """Compute Levenshtein edit distance using DP."""
 m, n = len(s), len(t)
 D = [[0] * (n + 1) for _ in range(m + 1)]

 for i in range(m + 1):
 D[i][0] = i
 for j in range(n + 1):
 D[0][j] = j

 for i in range(1, m + 1):
 for j in range(1, n + 1):
 if s[i-1] == t[j-1]:
 D[i][j] = D[i-1][j-1]
 else:
 D[i][j] = 1 + min(D[i-1][j], D[i][j-1], D[i-1][j-1])

 return D[m][n]

Test
pairs = [("kitten", "sitting"), ("sunday", "saturday"), ("hello", "hallo")]
for s, t in pairs:
 print(f"d('{s}', '{t}') = {edit_distance(s, t)}")

Simple spelling correction
dictionary = ["the", "their", "there", "then", "than", "this", "that", "these",
"those"]
misspelled = "thier"
corrections = sorted(dictionary, key=lambda w: edit_distance(misspelled, w))
print(f"\nClosest to '{misspelled}': {corrections[:3]}")
```

1. Implement BPE tokenisation from scratch. Start with character-level tokens and iteratively merge the most frequent pairs.

```

from collections import Counter

def get_pairs(corpus):
 """Count adjacent token pairs across all words."""
 pairs = Counter()
 for word, freq in corpus.items():
 symbols = word.split()
 for i in range(len(symbols) - 1):
 pairs[(symbols[i], symbols[i+1])] += freq
 return pairs

def merge_pair(pair, corpus):
 """Merge all occurrences of a pair in the corpus."""
 new_corpus = {}
 bigram = ' '.join(pair)
 replacement = ''.join(pair)
 for word, freq in corpus.items():
 new_word = word.replace(bigram, replacement)
 new_corpus[new_word] = freq
 return new_corpus

Training corpus with word frequencies
text = "low low low low low lower lower newest newest newest newest newest\nnewest"

word_freqs = Counter(text.split())
Initialise: split each word into characters with end-of-word marker
corpus = {' '.join(word) + ' _': freq for word, freq in word_freqs.items()}

print("Initial corpus:")
for word, freq in corpus.items():
 print(f" {word}: {freq}")

Run BPE for 10 merges
for i in range(10):
 pairs = get_pairs(corpus)
 if not pairs:
 break
 best_pair = max(pairs, key=pairs.get)
 corpus = merge_pair(best_pair, corpus)
 print(f"\nMerge {i+1}: {best_pair} (freq={pairs[best_pair]})")
 for word, freq in corpus.items():
 print(f" {word}: {freq}")

```

1. Build a bigram language model and compute perplexity on a test sentence. Experiment with Laplace smoothing.



```

from collections import Counter, defaultdict
import math

Training corpus
train = """the cat sat on the mat . the dog chased the cat .
the cat ran from the dog . a dog sat on a mat .""".split()

Count bigrams and unigrams
bigrams = Counter(zip(train[:-1], train[1:]))
unigrams = Counter(train)
vocab_size = len(set(train))

def bigram_prob(w2, w1, alpha=0):
 """P(w2 | w1) with optional Laplace smoothing."""
 return (bigrams[(w1, w2)] + alpha) / (unigrams[w1] + alpha * vocab_size)

Compute perplexity
test = "the cat sat on a mat .".split()

for alpha in [0, 1, 0.1]:
 log_prob = 0
 for w1, w2 in zip(test[:-1], test[1:]):
 p = bigram_prob(w2, w1, alpha=alpha)
 if p > 0:
 log_prob += math.log(p)
 else:
 log_prob += float('-inf')

 ppl = math.exp(-log_prob / (len(test) - 1)) if log_prob > float('-inf') else float('inf')
 print(f"Smoothing α={alpha}: perplexity = {ppl:.2f}")

```

1. Implement TF-IDF from scratch and use cosine similarity to find the most similar document to a query.

```

import jax.numpy as jnp
import math
from collections import Counter

documents = [
 "the cat sat on the mat",
 "the dog chased the cat around the park",
 "a mat was placed on the floor by the door",
 "the quick brown fox jumped over the lazy dog",
]

```

```

Build vocabulary
vocab = sorted(set(word for doc in documents for word in doc.split()))
word_to_idx = {w: i for i, w in enumerate(vocab)}
V = len(vocab)
N = len(documents)

Compute TF-IDF matrix
doc_freq = Counter()
for doc in documents:
 for word in set(doc.split()):
 doc_freq[word] += 1

tfidf_matrix = jnp.zeros((N, V))
for i, doc in enumerate(documents):
 word_counts = Counter(doc.split())
 for word, count in word_counts.items():
 tf = 1 + math.log(count)
 idf = math.log(N / doc_freq[word])
 j = word_to_idx[word]
 tfidf_matrix = tfidf_matrix.at[i, j].set(tf * idf)

Query
query = "cat on the mat"
query_vec = jnp.zeros(V)
query_counts = Counter(query.split())
for word, count in query_counts.items():
 if word in word_to_idx:
 tf = 1 + math.log(count)
 idf = math.log(N / doc_freq.get(word, 1))
 query_vec = query_vec.at[word_to_idx[word]].set(tf * idf)

Cosine similarity (from chapter 01)
def cosine_sim(a, b):
 return jnp.dot(a, b) / (jnp.linalg.norm(a) * jnp.linalg.norm(b) + 1e-8)

print(f"Query: '{query}'\n")
for i, doc in enumerate(documents):
 sim = cosine_sim(query_vec, tfidf_matrix[i])
 print(f" Doc {i} (sim={sim:.3f}): '{doc}'")

```

## 33 Embeddings and Sequence Models

Word embeddings compress sparse, symbolic text into dense vector spaces where semantic similarity becomes geometric proximity. This file covers Word2Vec (CBOW,

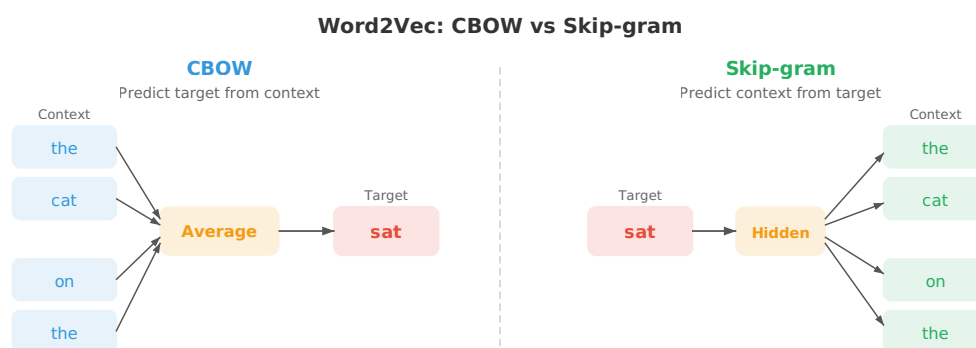
Skip-gram), GloVe, FastText, RNNs, LSTMs, GRUs, seq2seq with attention, and the encoder-decoder paradigm, the progression from bag-of-words to contextual representations.

- In file 01, we introduced the distributional hypothesis: words that appear in similar contexts tend to have similar meanings. In file 02, we represented text using sparse, hand-crafted features like TF-IDF vectors. These vectors live in very high-dimensional spaces (one dimension per vocabulary word) and are mostly zeros. **Word embeddings** compress this information into dense, low-dimensional vectors that capture semantic relationships, and they are learned directly from data.
- **Word2Vec** (Mikolov et al., 2013) learns word embeddings by training a shallow neural network on a simple prediction task. There are two architectures.
- The **Continuous Bag of Words (CBOW)** model predicts a target word from its surrounding context words. Given a window of context words (e.g., “the cat \_\_\_ on the” ), the model averages their embedding vectors and passes the result through a linear layer to predict the missing word ( “sat” ). The training objective maximises:

$$P(w_t \mid w_{t-k}, \dots, w_{t-1}, w_{t+1}, \dots, w_{t+k})$$

- The **Skip-gram** model does the reverse: given a target word, predict the surrounding context words. For the target word “sat” , the model tries to predict “the” , “cat” , “on” , “the” in separate predictions. The objective maximises:

$$P(w_{t+j} \mid w_t) \text{ for each } j \in [-k, k], j \neq 0$$



Both architectures learn dense word embeddings as a byproduct of prediction.  
Skip-gram works better for rare words; CBOW is faster for frequent words.

Skip-gram and CBOW architectures side by side: CBOW averages context embeddings to predict the centre word, skip-gram uses the centre word embedding to predict each context word

- Skip-gram tends to work better for rare words because each word generates multiple training examples (one per context position). CBOW is faster and slightly better for frequent words because it averages over multiple context signals.
- Training on the full vocabulary is expensive because the softmax denominator sums over all  $V$  words. **Negative sampling** approximates this by turning the problem into binary classification: distinguish the true context word (positive sample) from randomly sampled noise words (negative samples). Instead of computing the full softmax, the model only updates embeddings for the target, the true context word, and a handful of negatives:

$$\mathcal{L} = \log \sigma(\mathbf{v}_{w_o}^T \mathbf{v}_{w_i}) + \sum_{i=1}^k \mathbb{E}_{w_i \sim P_n} [\log \sigma(-\mathbf{v}_{w_i}^T \mathbf{v}_{w_i})]$$

- Here  $\mathbf{v}_{w_i}$  is the input word embedding,  $\mathbf{v}_{w_o}$  is the output (context) word embedding, and  $P_n$  is the noise distribution, typically the unigram frequency raised to the 3/4 power (which downweights very frequent words like “the” ).
- Why does this simple objective produce meaningful embeddings? Levy and Goldberg (2014) showed that skip-gram with negative sampling is implicitly factorising a **shifted pointwise mutual information (PMI)** matrix. At convergence, the dot product of two word vectors approximates:

$$\mathbf{v}_w^T \mathbf{v}_c \approx \text{PMI}(w, c) - \log k$$

- where  $\text{PMI}(w, c) = \log \frac{P(w, c)}{P(w)P(c)}$  measures how much more likely words  $w$  and  $c$  co-occur than expected by chance (chapter 05 information theory), and  $k$  is the number of negative samples. Words that co-occur much more than chance have high PMI and therefore high dot product (similar embeddings). Words that co-occur less than expected have negative PMI and dissimilar embeddings. This reveals that Word2Vec is doing the same thing as classical distributional

semantics methods like latent semantic analysis (SVD on co-occurrence matrices), but in a more scalable, online fashion.

- The most surprising property of Word2Vec embeddings is that they capture **analogies** through vector arithmetic. The vector  $v_{\text{king}} - v_{\text{man}} + v_{\text{woman}}$  is closest to  $v_{\text{queen}}$ . This works because the embedding space encodes semantic relationships as approximately linear directions: the “royalty” direction is roughly  $v_{\text{king}} - v_{\text{man}}$ , and adding it to  $v_{\text{woman}}$  lands near  $v_{\text{queen}}$ . This connects to the linear algebra of chapter 01: semantic relationships are vector translations.
- **GloVe** (Global Vectors for Word Representation, Pennington et al., 2014) takes a different approach. Instead of learning from local context windows one at a time, it builds a global word co-occurrence matrix  $X$  where  $X_{ij}$  counts how often word  $j$  appears in the context of word  $i$  across the entire corpus. The model then learns embeddings whose dot product approximates the log co-occurrence:

$$w_i^T$$

⊗

$w$

⊗

$$j + b_i + b$$

$$j = \log X_{ij}$$

- The loss function weights each pair by a capping function  $f(X_{ij})$  that prevents very frequent co-occurrences from dominating:

$$\mathcal{L} =$$

$v$

$\sum$

$i, j = 1$

$$f(X_{ij})(w_i^T$$

⊗

$w$

$$j + b_i + b$$

$$j - \log X_{ij})^2$$

- GloVe combines the benefits of global matrix factorisation (like latent semantic analysis) with the local context learning of Word2Vec. In practice, GloVe and Word2Vec produce embeddings of similar quality.
- **FastText** (Bojanowski et al., 2017) extends skip-gram by representing each word as a bag of character n-grams. The word “where” with  $n = 3$  becomes: “<wh” , “whe” , “her” , “ere” , “re>” , plus the whole-word token “” . The word’s embedding is the sum of all its n-gram embeddings.
- This has a crucial advantage: FastText can produce embeddings for words it has never seen during training. The word “whereabouts” shares n-grams with “where” , so its embedding will be reasonable even if “whereabouts” never appeared in the training data. This is especially useful for morphologically rich languages (file 01) where words have many inflected forms.
- **Embedding evaluation** typically uses two types of benchmarks. **Analogy tasks** test whether  $v_a - v_b + v_c \approx v_d$  (e.g., “Paris” - “France” + “Italy”  $\approx$  “Rome” ). **Similarity benchmarks** compare the cosine similarity (chapter 01) between word pairs to human judgements. Common datasets include WordSim-353, SimLex-999, and the Google analogy test set. A practical caveat: embeddings that excel at analogies may not be best for downstream tasks like sentiment classification. The best evaluation is often the task itself.
- In chapter 06, we introduced RNNs, LSTMs, and GRUs as architectures for sequential data. Here we focus on how they are applied to language tasks specifically.
- A **language model RNN** reads tokens one at a time and predicts the next token at each step. The hidden state  $h_t$  compresses the entire history  $w_1, \dots, w_t$  into a fixed-size vector, and a linear layer plus softmax maps  $h_t$  to a distribution over the vocabulary. Training uses cross-entropy loss against the true next token, which is identical to minimising perplexity (file 02). The key limitation: the fixed-size hidden state must encode everything about the history, and information from early tokens gets progressively overwritten.

- **Bidirectional RNNs** process the sequence in both directions: one RNN reads left-to-right, another reads right-to-left. At each position  $t$ , the forward hidden state

$\rightarrow$

$h_{\rightarrow t}$

and backward hidden state

$\leftarrow$

$h_{\leftarrow t}$

are concatenated to form a context-aware representation

$\rightarrow$

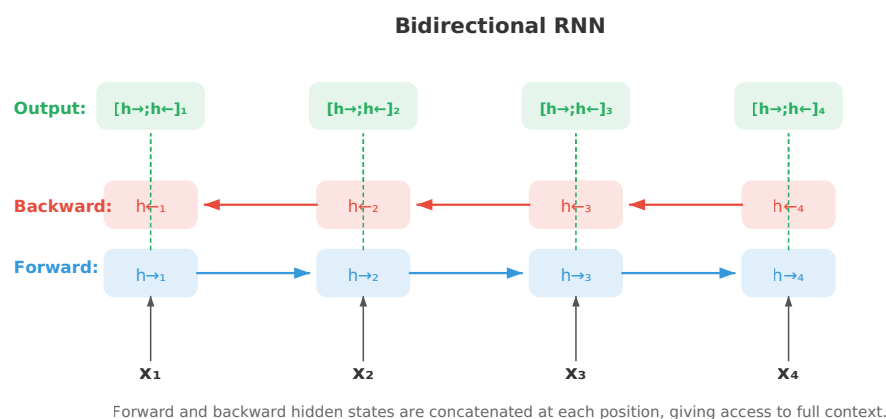
$h_t = [h_{\rightarrow t}; h_{\leftarrow t}]$

;

$\leftarrow$

$h_{\leftarrow t}$

]. This gives the model access to both past and future context, which is powerful for tasks like POS tagging and NER (file 02) where a word's label depends on words both before and after it. Bidirectional RNNs cannot be used for language modelling because you cannot peek at future tokens when predicting them.

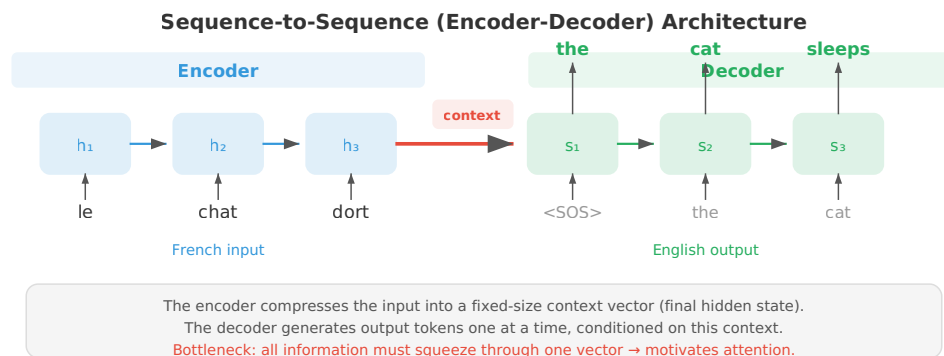


Bidirectional RNN: forward RNN reads left-to-right producing hidden states, backward RNN reads right-to-left, outputs are concatenated at each position

- **Deep stacked RNNs** place multiple RNN layers on top of each other. The hidden states of layer  $l$  at all time steps become the input sequence for layer  $l + 1$ . Stacking 2-4 layers typically improves performance by building hierarchical representations, similar to how deeper CNNs build feature hierarchies (chapter

06). Beyond 4 layers, vanishing gradients and overfitting become problems unless residual connections are added between layers.

- The **sequence-to-sequence (seq2seq)** architecture (Sutskever et al., 2014) maps a variable-length input sequence to a variable-length output sequence. It consists of an **encoder** RNN that reads the input and compresses it into a context vector (the final hidden state), and a **decoder** RNN that generates the output one token at a time, conditioned on this context vector.



Seq2seq encoder-decoder: encoder RNN reads input tokens left-to-right, final hidden state passed as initial state to decoder RNN, which generates output tokens autoregressively

- Seq2seq was the breakthrough architecture for machine translation. The encoder reads a French sentence, the decoder produces the English translation. The decoder starts with a special start-of-sequence token and generates tokens autoregressively until it produces an end-of-sequence token. A practical trick: reversing the input sequence (feeding “chat le” instead of “le chat” ) improved results because it placed the first input word closer to the first output word in the computation graph, shortening the gradient path.
- The bottleneck problem: the entire input must be compressed into a single fixed-size vector. For long sentences, this vector cannot capture all the information, and performance degrades. This motivated **attention mechanisms**.
- Chapter 06 introduced the modern Q, K, V formulation of attention. The original attention mechanisms for NLP were formulated differently, as alignment models between encoder and decoder states.
- **Bahdanau attention** (additive attention, Bahdanau et al., 2015) computes an alignment score between the decoder hidden state  $s_t$  and each encoder hidden state  $h_i$  using a learned feed-forward network:



$$e_{ti} = v^T \tanh(W_s s_{t-1} + W_h h_i)$$

- The scores are normalised to attention weights via softmax, and the context vector is a weighted sum of encoder states:

$$\alpha_{ti} = \frac{\exp(e_{ti})}{\sum_j \exp(e_{tj})}, \quad c_t = \sum_i \alpha_{ti} h_i$$

$\sum_i$

$$\alpha_{ti} h_i$$

- The decoder then uses both  $s_{t-1}$  and  $c_t$  to produce the next output. The key insight: instead of one fixed context vector for the entire sentence, each decoder step gets a different weighted combination of encoder states, allowing the model to “look back” at the relevant parts of the input.
- **Luong attention** (multiplicative attention, Luong et al., 2015) simplifies the score computation. The **dot** variant uses  $e_{ti} = s_t^T h_i$ . The **general** variant uses  $e_{ti} = s_t^T W h_i$ . These are faster than Bahdanau’s additive score because they use matrix multiplication instead of a feed-forward network. Luong attention also computes the context vector from the current decoder state  $s_t$  (rather than  $s_{t-1}$ ), which gives it access to more information but makes the computation slightly different.

### Attention Alignment: French → English Translation

		Source (French)				
		le	chat	noir	dort	bien
Target (English)	the	.82	.05	.03	.06	.04
	black	.03	.06	.85	.03	.03
	cat	.04	.80	.07	.05	.04
	sleeps	.02	.05	.04	.83	.06
	well	.03	.02	.03	.06	.86

Attention alignment heatmap between a source sentence and its translation, showing which source words each target word attends to, with brighter cells indicating higher attention weights

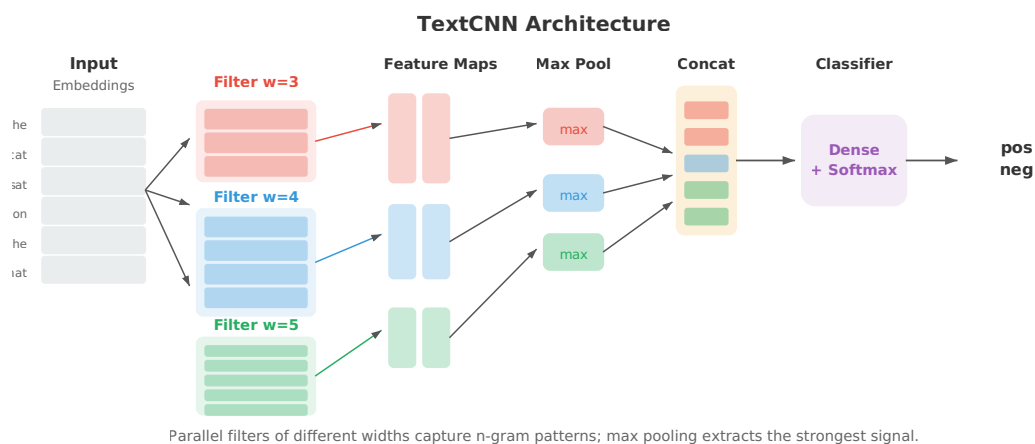
- Attention weights are often visualised as heatmaps showing which input tokens the decoder focuses on when producing each output token. In translation, these heatmaps roughly trace the word alignment between source and target languages, with the diagonal pattern broken by reordering (e.g., adjective-noun order differs between French and English).
- At inference time, the decoder must choose a token at each step. **Greedy decoding** picks the highest-probability token at each position, but this can lead to suboptimal sequences: a locally good choice may force the model into a globally bad sentence. **Beam search** maintains the top k (the beam width) partial sequences at each step, expanding each by all possible next tokens and keeping the best k overall.
- With beam width  $k = 1$ , beam search reduces to greedy decoding. Typical values are  $k = 4$  to  $k = 10$ . Larger beams find better sequences but are proportionally slower. Beam search also needs **length normalisation** to avoid favouring shorter sequences, which naturally have higher total probability because they multiply fewer terms. The normalised score is:

$|y|$   
 $\sum$   
 $t=1$

$$\text{score}(y) = \frac{1}{|y|^\alpha}$$

$$\log P(y_t \mid y_{<t})$$

- where  $|y|$  is the sequence length and  $\alpha$  (typically 0.6-0.7) controls the strength of the length penalty. With  $\alpha = 0$ , there is no length normalisation. With  $\alpha = 1$ , the score is the per-token log-probability (geometric mean). The intermediate value balances between favouring concise outputs and not truncating too early.
- While RNNs process text sequentially, **1D CNNs** process it in parallel by sliding filters across the token sequence. Each filter detects a local pattern (an n-gram feature).
- **TextCNN** (Kim, 2014) applies multiple 1D convolutional filters of different widths (e.g., 3, 4, 5 tokens) to the input embedding matrix. Each filter produces a feature map, and **max-over-time pooling** takes the single maximum value from each feature map, capturing whether the pattern was detected anywhere in the text regardless of position. The pooled features from all filters are concatenated and passed to a classifier.



TextCNN architecture: input embeddings pass through parallel convolutional filters of widths 3, 4, and 5, each followed by max-over-time pooling, then concatenated and fed to a fully connected classifier

- TextCNN is fast and surprisingly effective for text classification tasks like sentiment analysis. It captures local n-gram patterns but cannot model long-range dependencies: a filter of width 5 only sees 5 consecutive tokens. **Dilated causal convolutions** address this by inserting gaps (dilations) between filter

elements. Stacking layers with exponentially increasing dilation rates (1, 2, 4, 8, ...) grows the receptive field exponentially without increasing parameters, allowing the model to capture dependencies across hundreds of tokens.

- All the embeddings discussed so far (Word2Vec, GloVe, FastText) produce a single vector per word type regardless of context. “Bank” gets the same embedding whether it means a financial institution or a river bank. This is a fundamental limitation that **contextual embeddings** address.
- **ELMo** (Embeddings from Language Models, Peters et al., 2018) produces contextual word representations by running a deep bidirectional LSTM language model on the input text. The forward LSTM predicts the next word at each position; a separate backward LSTM predicts the previous word. Both are trained as language models on large corpora.
- At each position  $k$ , ELMo combines the hidden states from all  $L$  layers using task-specific learned weights:

$$\text{ELMo}_k = \gamma \sum_{j=0}^L s_j h_{k,j}$$

- Here  $h_{k,j}$  is the hidden state at position  $k$  and layer  $j$  (layer 0 is the raw token embedding),  $s_j$  are softmax-normalised scalar weights, and  $\gamma$  is a task-specific scaling factor. Different layers capture different information: lower layers capture syntax (POS tags, word morphology), upper layers capture semantics (word sense, semantic role). By mixing all layers with learned weights, ELMo embeddings adapt to diverse downstream tasks.
- ELMo marked the beginning of the **pre-train then fine-tune** paradigm: train a large language model on massive unlabelled text, then use its representations for downstream tasks. ELMo specifically uses the pre-trained representations as fixed or lightly tuned features that are concatenated with task-specific inputs. BERT and GPT (file 04) push this further by fine-tuning the entire model end-to-end, which proves dramatically more effective.
- The progression from Word2Vec to ELMo illustrates a recurring theme in NLP: moving from static to dynamic representations, from local to global context, and from shallow to deep models. Each step trades computational cost for

richer representations. Transformers (file 04) complete this progression by replacing recurrence entirely with attention, enabling both deep contextualisation and parallel computation.

## 33.1 Coding Tasks (use CoLab or notebook)

---

1. Implement Word2Vec skip-gram with negative sampling from scratch. Train on a small corpus and visualise the learned embeddings using PCA.

```
import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

Small corpus
corpus = """the king ruled the kingdom . the queen ruled the kingdom .
the prince is the son of the king . the princess is the daughter of the queen .
a man worked in the castle . a woman worked in the castle .
the king and queen lived in the castle . the prince and princess played
outside """

vocab = sorted(set(corpus))
word2idx = {w: i for i, w in enumerate(vocab)}
idx2word = {i: w for w, i in word2idx.items()}
V = len(vocab)

Generate skip-gram pairs with window size 2
window = 2
pairs = []
for i, word in enumerate(corpus):
 for j in range(max(0, i - window), min(len(corpus), i + window + 1)):
 if i != j:
 pairs.append((word2idx[word], word2idx[corpus[j]]))

pairs = jnp.array(pairs)
print(f"Vocabulary: {V} words, Training pairs: {len(pairs)}")

Model parameters
embed_dim = 16
key = jax.random.PRNGKey(42)
k1, k2 = jax.random.split(key)
W_in = jax.random.normal(k1, (V, embed_dim)) * 0.1 # input embeddings
W_out = jax.random.normal(k2, (V, embed_dim)) * 0.1 # output embeddings

Negative sampling loss for one pair
def neg_sampling_loss(W_in, W_out, target, context, neg_ids):
```

```

v_in = W_in[target] # (embed_dim,)
v_out = W_out[context] # (embed_dim,)
v_neg = W_out[neg_ids] # (k, embed_dim)

pos_loss = -jax.nn.log_sigmoid(jnp.dot(v_in, v_out))
neg_loss = -jnp.sum(jax.nn.log_sigmoid(-v_neg @ v_in))
return pos_loss + neg_loss

Training loop
num_neg = 5
lr = 0.05

@jax.jit
def train_step(W_in, W_out, target, context, neg_ids):
 loss, (g_in, g_out) = jax.value_and_grad(neg_sampling_loss, argnums=(0, 1))(
 W_in, W_out, target, context, neg_ids)
 return loss, W_in - lr * g_in, W_out - lr * g_out

key = jax.random.PRNGKey(0)
for epoch in range(50):
 total_loss = 0.0
 for i in range(len(pairs)):
 key, subkey = jax.random.split(key)
 neg_ids = jax.random.randint(subkey, (num_neg,), 0, V)
 loss, W_in, W_out = train_step(W_in, W_out, pairs[i, 0], pairs[i, 1],
neg_ids)

 total_loss += loss
 if (epoch + 1) % 10 == 0:
 print(f"Epoch {epoch+1}: avg loss = {total_loss / len(pairs):.4f}")

Visualise with PCA (chapter 01)
embeddings = W_in
mean = embeddings.mean(axis=0)
centered = embeddings - mean
U, S, Vt = jnp.linalg.svd(centered, full_matrices=False)
coords = centered @ Vt[:2].T # project onto top 2 PCs

plt.figure(figsize=(10, 8))
for i, word in idx2word.items():
 plt.scatter(coords[i, 0], coords[i, 1], c='#3498db', s=40)
 plt.annotate(word, (coords[i, 0] + 0.02, coords[i, 1] + 0.02), fontsize=9)
plt.title("Word2Vec Skip-gram Embeddings (PCA projection)")
plt.grid(alpha=0.3); plt.show()

```

1. Build a character-level RNN language model that learns to generate text from a small training string.

```

import jax
import jax.numpy as jnp

Tiny training text
text = "to be or not to be that is the question "
chars = sorted(set(text))
char2idx = {c: i for i, c in enumerate(chars)}
idx2char = {i: c for c, i in char2idx.items()}
V = len(chars)
data = jnp.array([char2idx[c] for c in text])

RNN parameters
hidden_dim = 64
key = jax.random.PRNGKey(0)
k1, k2, k3, k4, k5 = jax.random.split(key, 5)

params = {
 'Wx': jax.random.normal(k1, (V, hidden_dim)) * 0.1,
 'Wh': jax.random.normal(k2, (hidden_dim, hidden_dim)) * 0.05,
 'bh': jnp.zeros(hidden_dim),
 'Wy': jax.random.normal(k3, (hidden_dim, V)) * 0.1,
 'by': jnp.zeros(V),
}

def rnn_step(params, h, x_idx):
 x = jnp.eye(V)[x_idx] # one-hot
 h = jnp.tanh(x @ params['Wx'] + h @ params['Wh'] + params['bh'])
 logits = h @ params['Wy'] + params['by']
 return h, logits

def loss_fn(params, inputs, targets):
 h = jnp.zeros(hidden_dim)
 total_loss = 0.0
 for t in range(len(inputs)):
 h, logits = rnn_step(params, h, inputs[t])
 log_probs = jax.nn.log_softmax(logits)
 total_loss -= log_probs[targets[t]]
 return total_loss / len(inputs)

grad_fn = jax.jit(jax.grad(loss_fn))

Training
inputs = data[:-1]
targets = data[1:]
lr = 0.01

for step in range(500):

```

```

 grads = grad_fn(params, inputs, targets)
 params = {k: params[k] - lr * grads[k] for k in params}
 if (step + 1) % 100 == 0:
 l = loss_fn(params, inputs, targets)
 print(f"Step {step+1}: loss = {l:.4f}")

Generate text
def generate(params, seed_char, length=60):
 h = jnp.zeros(hidden_dim)
 idx = char2idx[seed_char]
 result = [seed_char]
 key = jax.random.PRNGKey(42)
 for _ in range(length):
 h, logits = rnn_step(params, h, idx)
 key, subkey = jax.random.split(key)
 idx = jax.random.categorical(subkey, logits)
 result.append(idx2char[int(idx)])
 return ''.join(result)

print(f"\nGenerated: {generate(params, 't')}")

```

1. Implement a toy seq2seq model with Bahdanau attention for sequence reversal. Visualise the attention alignment matrix.

```

import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

Task: reverse a sequence of digits (e.g., [3, 1, 4] -> [4, 1, 3])
vocab_size = 10 # digits 0-9
SOS, EOS = 10, 11 # special tokens
total_vocab = 12
embed_dim, hidden_dim = 16, 32
max_len = 5

key = jax.random.PRNGKey(42)
keys = jax.random.split(key, 8)

params = {
 'embed': jax.random.normal(keys[0], (total_vocab, embed_dim)) * 0.1,
 'enc_Wx': jax.random.normal(keys[1], (embed_dim, hidden_dim)) * 0.1,
 'enc_Wh': jax.random.normal(keys[2], (hidden_dim, hidden_dim)) * 0.05,
 'dec_Wx': jax.random.normal(keys[3], (embed_dim, hidden_dim)) * 0.1,
 'dec_Wh': jax.random.normal(keys[4], (hidden_dim, hidden_dim)) * 0.05,
 # Bahdanau attention
 'Ws': jax.random.normal(keys[5], (hidden_dim, hidden_dim)) * 0.1,

```



```

 'Wh_att': jax.random.normal(keys[6], (hidden_dim, hidden_dim)) * 0.1,
 'v_att': jax.random.normal(keys[7], (hidden_dim,)) * 0.1,
 # Output projection (from hidden + context to vocab)
 'Wo': jax.random.normal(keys[0], (hidden_dim * 2, total_vocab)) * 0.1,
}

def encode(params, seq):
 """Encode input sequence, return all hidden states."""
 h = jnp.zeros(hidden_dim)
 states = []
 for t in range(len(seq)):
 x = params['embed'][seq[t]]
 h = jnp.tanh(x @ params['enc_Wx'] + h @ params['enc_Wh'])
 states.append(h)
 return jnp.stack(states), h

def bahdanau_attention(params, dec_state, enc_states):
 """Compute Bahdanau attention weights and context vector."""
 scores = jnp.tanh(enc_states @ params['Wh_att'] + dec_state @ params['Ws'])
 e = scores @ params['v_att'] # (src_len,)
 alpha = jax.nn.softmax(e)
 context = alpha @ enc_states
 return context, alpha

def decode_step(params, dec_h, prev_token, enc_states):
 x = params['embed'][prev_token]
 dec_h = jnp.tanh(x @ params['dec_Wx'] + dec_h @ params['dec_Wh'])
 context, alpha = bahdanau_attention(params, dec_h, enc_states)
 combined = jnp.concatenate([dec_h, context])
 logits = combined @ params['Wo']
 return dec_h, logits, alpha

def seq2seq_loss(params, src, tgt):
 enc_states, enc_final = encode(params, src)
 dec_h = enc_final
 loss = 0.0
 prev_token = SOS
 for t in range(len(tgt)):
 dec_h, logits, _ = decode_step(params, dec_h, prev_token, enc_states)
 log_probs = jax.nn.log_softmax(logits)
 loss -= log_probs[tgt[t]]
 prev_token = tgt[t]
 return loss / len(tgt)

Generate training data: reverse sequences
key = jax.random.PRNGKey(0)
train_srcs, train_tgts = [], []

```

```

for _ in range(200):
 key, subkey = jax.random.split(key)
 length = jax.random.randint(subkey, (), 3, max_len + 1)
 key, subkey = jax.random.split(key)
 seq = jax.random.randint(subkey, (int(length),), 0, vocab_size)
 train_srcs.append(seq)
 train_tgts.append(seq[::-1]) # reverse

Training
grad_fn = jax.grad(seq2seq_loss)
lr = 0.01

for epoch in range(100):
 total_loss = 0.0
 for src, tgt in zip(train_srcs, train_tgts):
 grads = grad_fn(params, src, tgt)
 params = {k: params[k] - lr * grads[k] for k in params}
 total_loss += seq2seq_loss(params, src, tgt)
 if (epoch + 1) % 20 == 0:
 print(f"Epoch {epoch+1}: avg loss = {total_loss / len(train_srcs):.4f}")

Visualise attention for one example
test_src = jnp.array([3, 1, 4, 1, 5])
test_tgt = test_src[::-1]

enc_states, enc_final = encode(params, test_src)
dec_h = enc_final
attentions = []
prev_token = SOS
for t in range(len(test_tgt)):
 dec_h, logits, alpha = decode_step(params, dec_h, prev_token, enc_states)
 attentions.append(alpha)
 prev_token = test_tgt[t]

att_matrix = jnp.stack(attentions)
fig, ax = plt.subplots(figsize=(6, 5))
im = ax.imshow(att_matrix, cmap='Blues')
ax.set_xlabel("Source position"); ax.set_ylabel("Target position")
src_labels = [str(int(x)) for x in test_src]
tgt_labels = [str(int(x)) for x in test_tgt]
ax.set_xticks(range(len(src_labels))); ax.set_xticklabels(src_labels)
ax.set_yticks(range(len(tgt_labels))); ax.set_yticklabels(tgt_labels)
for i in range(len(tgt_labels)):
 for j in range(len(src_labels)):
 ax.text(j, i, f"{att_matrix[i,j]:.2f}", ha='center', va='center',
fontSize=9)

```

```
ax.set_title("Bahdanau Attention Alignment (sequence reversal)")
plt.colorbar(im); plt.tight_layout(); plt.show()
```

## 34 Transformers and Language Models

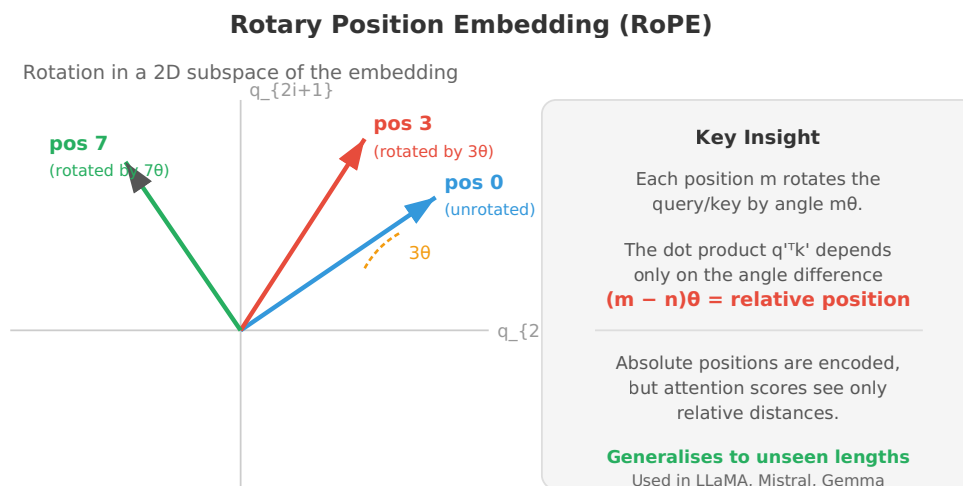
Transformers replaced recurrence with self-attention and became the dominant architecture for language understanding and generation. This file covers BERT, GPT, T5, positional encodings (sinusoidal, RoPE), pre-training objectives (MLM, CLM), fine-tuning, prompt engineering, and scaling laws, the blueprint behind modern LLMs.

- In chapter 06, we introduced the Transformer architecture: self-attention, multi-head attention, positional encoding, and the encoder-decoder structure. Here we focus on how transformers are adapted for specific NLP paradigms, the models that defined modern NLP (BERT, GPT, T5), and the techniques that make them practical at scale.
- Recall the core operation: **scaled dot-product attention** computes 
$$\frac{\text{softmax}(QK^T)}{\sqrt{d_k}}V,$$
 where queries, keys, and values are linear projections of the input. **Multi-head attention** runs  $h$  parallel attention heads, each with different learned projections, and concatenates the results. The Transformer block wraps this with residual connections, layer normalisation, and a position-wise feed-forward network (chapter 06).
- A subtle but important architectural choice is the placement of **layer normalisation**. The original Transformer uses **post-norm**: the residual and normalisation come after the sublayer, as  $\text{LayerNorm}(x + \text{Sublayer}(x))$ .
- Most modern models use **pre-norm**: normalise before the sublayer, as  $x + \text{Sublayer}(\text{LayerNorm}(x))$ . Pre-norm is more stable during training because the residual connection passes gradients directly through the identity path without them being affected by the normalisation. This makes it easier to train very deep models without careful learning rate warmup.
- The **feed-forward sublayer** in each Transformer block is a two-layer MLP applied independently to each token position:

$$\text{FFN}(x) = W_2 \cdot \text{GELU}(W_1 x + b_1) + b_2$$

- The inner dimension is typically 4 times the model dimension (e.g.,  $d_{\text{model}} = 768$ ,  $d_{\text{ff}} = 3072$ ). This FFN accounts for about two-thirds of the parameters in each block and is thought to function as a key-value memory that stores factual knowledge learned during training.
- **Positional encoding** gives the model information about token order, since attention itself is permutation-equivariant. The original **sinusoidal encoding** (chapter 06) uses fixed sine and cosine functions at different frequencies. **Learned positional embeddings** simply add a trainable vector for each position (used in BERT and GPT-2). Both are absolute encodings: position 5 gets the same vector regardless of context.
- **Rotary Position Embedding (RoPE)** encodes position by rotating the query and key vectors in 2D subspaces. For a pair of dimensions ( $q_{2i}, q_{2i+1}$ ), the rotation by angle  $m\theta_i$  (where  $m$  is the position and  $\theta_i = 10000^{-2i/d}$ ) applies:

$$\begin{bmatrix} q'_{2i} \\ q'_{2i+1} \end{bmatrix} = \begin{bmatrix} \cos m\theta_i & -\sin m\theta_i \\ \sin m\theta_i & \cos m\theta_i \end{bmatrix} \begin{bmatrix} q_{2i} \\ q_{2i+1} \end{bmatrix}$$



RoPE: each position rotates query and key vectors by a different angle in 2D subspaces, so attention scores depend only on relative position

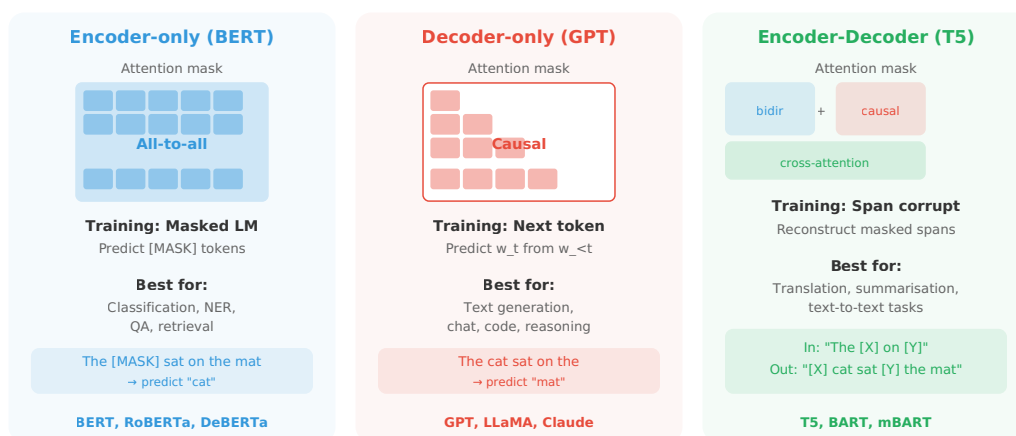
- The beauty of RoPE is that the dot product  $q'^T k'$  between rotated queries and keys depends only on the relative position  $m - n$ , not the absolute positions.

- To see why, write the rotation as  $q' = R_m q$  and  $k' = R_n k$ , where  $R_m$  is a block-diagonal rotation matrix. The attention score becomes:

$$q'^T k' = (R_m q)^T (R_n k) = q^T R_m^T R_n k = q^T R_{n-m} k$$

- The last step follows from the rotation group property:  $R_m^T R_n = R_{n-m}$  (rotating back by  $m$  then forward by  $n$  equals rotating by  $n - m$ ).
- This means the attention score depends only on the relative distance  $n - m$ , not the absolute positions  $m$  and  $n$  individually.
- The model gains a natural notion of distance without any learned position parameters and can generalise to sequence lengths not seen during training.
- **ALiBi** (Attention with Linear Biases) takes an even simpler approach: it adds a fixed linear penalty to the attention scores based on distance, as  $\text{score}_{ij} = q_i^T k_j - m \cdot |i - j|$ , where  $m$  is a head-specific slope. Different heads use different slopes, allowing some heads to focus locally and others globally. ALiBi requires no learned parameters for position and generalises well to sequences longer than those seen during training.
- The three dominant paradigms for Transformer-based language models are **encoder-only**, **decoder-only**, and **encoder-decoder**. They differ in what the model can see (the attention mask) and how they are trained.

### Three Transformer Paradigms

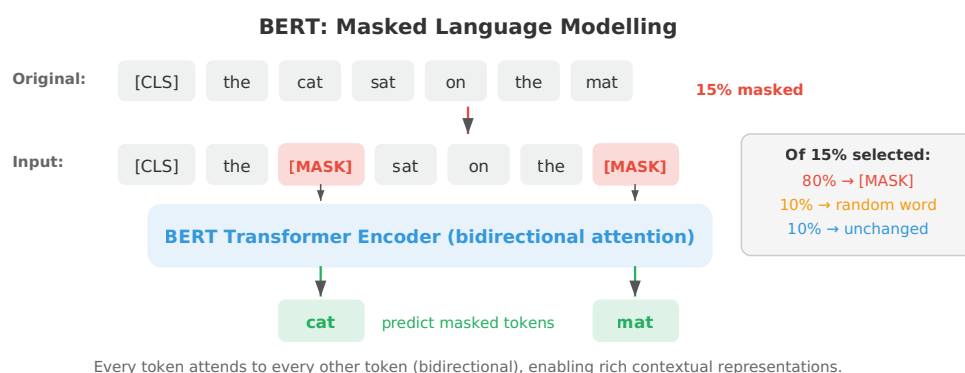


Three Transformer paradigms: encoder-only (BERT) with bidirectional attention for classification, decoder-only (GPT) with causal attention for generation, encoder-decoder (T5) combining both for seq2seq tasks

- **BERT** (Bidirectional Encoder Representations from Transformers, Devlin et al., 2019) is the canonical encoder-only model. It processes text with full bidirectional attention: every token can attend to every other token, both left and right. This gives BERT rich contextual representations but means it cannot generate text autoregressively.
- BERT is pre-trained with two objectives. **Masked language modelling (MLM)** randomly masks 15% of input tokens and trains the model to predict them. Of the selected tokens, 80% are replaced with a [MASK] token, 10% with a random word, and 10% are left unchanged (to prevent the model from learning to only predict when it sees [MASK]). The training objective is:

$$\mathcal{L}_{\text{MLM}} = - \sum_{i \in \mathcal{M}} \log P(w_i \mid w_{\setminus \mathcal{M}})$$

- where  $\mathcal{M}$  is the set of masked positions and  $w_{\setminus \mathcal{M}}$  is the sentence with those positions masked. This is a **denoising** objective: the model learns to reconstruct corrupted input.



BERT masked language modelling: 15% of tokens are masked in the input, the bidirectional Transformer predicts the original tokens at masked positions

- **Next Sentence Prediction (NSP)** trains BERT to predict whether two sentences are consecutive in the original text. A special [CLS] token at the start of the input is used for this binary classification. NSP was included to help with tasks like question answering that require understanding sentence relationships, though later work (RoBERTa) showed it contributes little and can be dropped.

- BERT' s pre-trained representations are adapted to downstream tasks by adding a task-specific head (a simple linear layer) on top and fine-tuning the entire model. For classification tasks, the [CLS] token representation is used. For token-level tasks (NER, POS tagging), each token' s representation is used. This **fine-tuning** approach transfers the linguistic knowledge learned during pre-training to new tasks with relatively little labelled data.
- **GPT** (Generative Pre-trained Transformer, Radford et al., 2018) is the canonical decoder-only model. It uses **causal (autoregressive) attention**: each token can only attend to tokens at earlier positions (and itself). This is enforced by masking future positions in the attention matrix (setting their scores to  $-\infty$  before the softmax). The training objective is simple **causal language modelling**: predict the next token given all previous tokens.

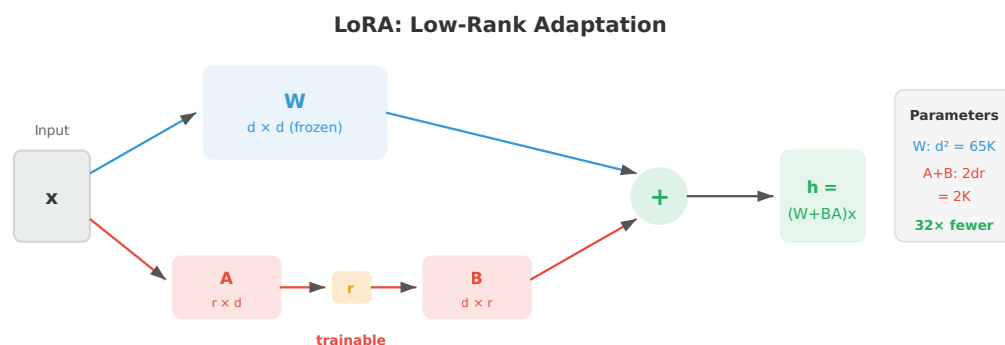
$$\mathcal{L}_{\text{CLM}} = - \sum_{i=1}^n \log P(w_i \mid w_1, \dots, w_{i-1})$$

- This is the same n-gram language model objective from file 02, but with a Transformer parameterisation that can condition on the entire preceding context rather than just the last  $k - 1$  tokens.
- **GPT-2** scaled this up to 1.5 billion parameters and demonstrated strong zero-shot performance: without any fine-tuning, it could perform tasks by conditioning on a natural language prompt ( “Translate English to French: ...” ).
- **GPT-3** (175 billion parameters) showed that scale alone could enable **in-context learning**: by providing a few input-output examples in the prompt, the model could perform new tasks without any gradient updates.
- **Encoder-decoder models** like **T5** (Text-to-Text Transfer Transformer, Raffel et al., 2020) frame every NLP task as text-to-text: the input is a text string (possibly with a task prefix like “translate English to German:” ) and the output is a text string. The encoder processes the input with bidirectional attention, and the decoder generates the output autoregressively with cross-attention to the encoder.
- T5 is pre-trained with **span corruption**: random contiguous spans of tokens are replaced with sentinel tokens, and the model must generate the original tokens. For example, “The cat sat on the mat” might become “The [X] on [Y]” as

input, and the target is “[X] cat sat [Y] the mat” . This is a generalisation of BERT’ s MLM to spans rather than individual tokens.

- **BART** (Lewis et al., 2020) is another encoder-decoder model pre-trained with a denoising objective, but it applies a broader set of corruption strategies: token masking, token deletion, span masking, sentence permutation, and document rotation. The diversity of corruption forces the model to learn more robust representations.
- As language models grow larger, **full fine-tuning** (updating all parameters) becomes impractical: a 175B parameter model requires hundreds of gigabytes just to store the optimizer states. **Parameter-efficient fine-tuning (PEFT)** methods adapt only a small fraction of parameters.
- **Adapters** insert small bottleneck layers (typically two linear layers with a nonlinearity: down-project to a small dimension, then up-project back) between the existing Transformer layers. Only the adapter weights are trained; the original model weights are frozen. This adds less than 5% new parameters while matching full fine-tuning performance on most tasks.
- **LoRA** (Low-Rank Adaptation) modifies the weight matrices themselves without adding new layers. Instead of updating the full weight matrix  $W$ , LoRA learns a low-rank decomposition of the update:  $W' = W + BA$ , where  $B$  is  $d \times r$  and  $A$  is  $r \times d$  with  $r \ll d$  (typically  $r = 4$  to  $r = 64$ ). The original  $W$  is frozen; only  $A$  and  $B$  are trained. At inference time, the update can be merged into the original weights with no additional latency:

$$W' = W + BA$$



$W$  is frozen (no gradients). Only the small matrices  $A$  and  $B$  are trained. At inference, merge:  $W' = W + BA$ .



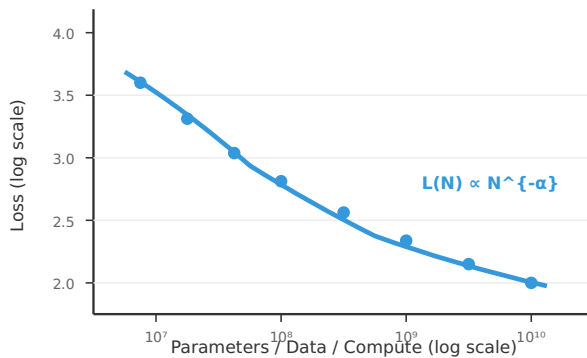
LoRA: the frozen weight matrix  $W$  is bypassed by a low-rank path through small matrices  $A$  and  $B$ , reducing trainable parameters by 32x while matching full fine-tuning

- **Prefix tuning** prepends a sequence of learnable “virtual tokens” to the key and value matrices of each attention layer. The model attends to these prefix vectors as if they were real tokens, and only the prefix parameters are trained. This is similar to prompt tuning but operates in the activation space rather than the embedding space.
- **Prompt engineering** is the art of designing input text that elicits the desired behaviour from a pre-trained model without any parameter updates.
  - **Zero-shot prompting** describes the task in natural language ( “Classify the sentiment of the following review:” ).
  - **Few-shot prompting** provides input-output examples before the actual query.
  - **Chain-of-thought (CoT)** prompting adds “Let’ s think step by step” or includes reasoning traces in the examples, which dramatically improves performance on arithmetic and logical reasoning tasks by guiding the model to decompose problems.
- **In-context learning (ICL)** is the phenomenon where large language models can learn to perform tasks from examples provided in the prompt, without any gradient updates. The model’ s weights do not change; it uses the examples as a kind of implicit specification.
- How ICL works mechanically remains an active research question; one hypothesis is that the attention layers implement a form of gradient descent in their forward pass, effectively “training” on the in-context examples.
- **Scaling laws** describe predictable relationships between model size, data size, compute budget, and performance (measured by loss). Kaplan et al. (2020) found that loss follows a power law in each variable:

$$L(N) \propto N^{-\alpha_N}, \quad L(D) \propto D^{-\alpha_D}, \quad L(C) \propto C^{-\alpha_C}$$

- where  $N$  is the number of parameters,  $D$  is the dataset size, and  $C$  is the compute budget. These power laws hold over many orders of magnitude and suggest that simply scaling up yields predictable improvements.

### Scaling Laws: Loss Decreases as a Power Law



#### Key Findings

##### Kaplan et al. (2020)

Loss follows power laws in N, D, C.  
Bigger models are more sample-efficient.

##### Chinchilla (Hoffmann, 2022)

Scale params and data equally.  
Rule: ~20 tokens per parameter.

##### Implication

Performance is predictable.  
Invest more compute → lower loss.

Scaling laws: loss decreases as a power law on log-log axes, with Kaplan and Chinchilla findings showing predictable improvement with scale

- The **Chinchilla scaling laws** (Hoffmann et al., 2022) revised this by showing that most large models are undertrained. For a fixed compute budget C, the optimal allocation scales model size and training data equally:

$$N_{\text{opt}} \propto C^{0.5}, \quad D_{\text{opt}} \propto C^{0.5}$$

- This means that if you double your compute budget, you should increase both model size and dataset size by a factor of

---

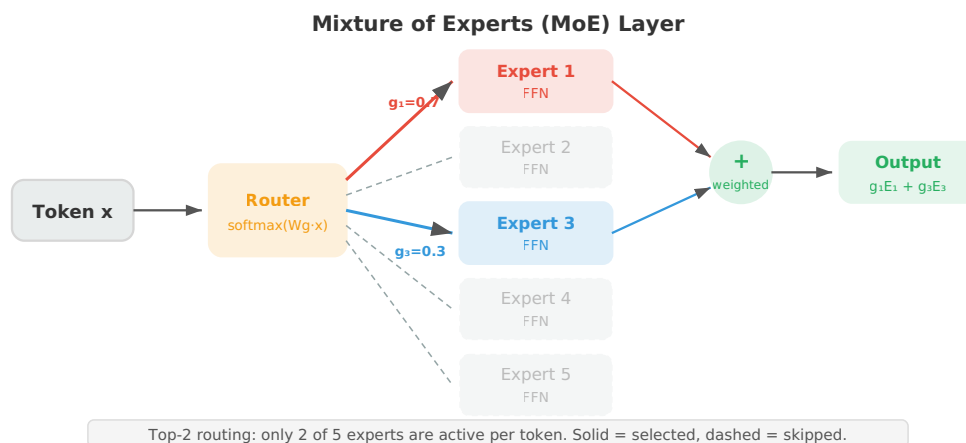
$\sqrt{2}$

not just make the model bigger.

- Kaplan et al. had recommended scaling N faster than D, which led to very large but undertrained models. Chinchilla (70B parameters, 1.4T tokens) matched the performance of Gopher (280B parameters, 300B tokens) with the same compute budget, demonstrating that the earlier models were severely data-starved.
- The practical rule of thumb: train on roughly 20 tokens per parameter.
- **Mixture of Experts (MoE)** is an architecture that scales model capacity without proportionally scaling computation. Instead of one large feed-forward layer, MoE uses multiple **expert** FFN layers and a **gating network** (router) that selects which experts to activate for each token.
- The gating function computes a routing score for each expert and selects the top-k (typically k = 1 or k = 2):

$$G(x) = \text{TopK}(\text{softmax}(W_g x))$$

- Only the selected experts process the token, so the computational cost scales with  $k$  (the number of active experts) rather than the total number of experts  $E$ . A model with 8 experts and top-2 routing has 4x the parameters of a dense model but only 2x the computation.



MoE layer: input token passes through a router that computes scores for each expert, top-2 experts are selected, their outputs are weighted by the gating scores and summed

- A critical challenge in MoE is **load balancing**: if the router sends most tokens to a few popular experts, the others are wasted. Training adds an auxiliary **load balancing loss** that encourages uniform expert utilisation:

$$\mathcal{L}_{\text{balance}} = E \cdot$$

$$\sum_{i=1}^E$$

$$f_i \cdot p_i$$

- where  $f_i$  is the fraction of tokens assigned to expert  $i$  and  $p_i$  is the average routing probability for expert  $i$ . This product is minimised when both the token fractions and probabilities are uniform (each equal to  $1/E$ ).
- Expert parallelism** distributes different experts across different accelerators. During the forward pass, an all-to-all communication step routes tokens to the device hosting their assigned expert, then routes the results back. This communication cost is the main engineering challenge of MoE at scale. Models like Switch Transformer, Mixtral, and GShard use MoE to achieve strong performance with practical inference costs.

- Building models is half the job; measuring whether they work is the other half. NLP evaluation is uniquely difficult because language is ambiguous, subjective, and open-ended.
- A translation can be correct in many different ways. A summary can be good even if it shares no exact words with a reference.
- A chatbot response can be helpful, harmless, and honest, yet reasonable humans will disagree.
- **Exact match (EM)** is the simplest metric: does the model’s output exactly match the gold answer? It is used for tasks with short, unambiguous answers like extractive question answering (SQuAD) or closed-form maths.
- EM is harsh; “New York City” and “new york city” fail to match unless normalisation is applied — but its simplicity makes it unambiguous.
- **Token-level metrics** treat NLP as a classification problem at the token level, using precision, recall, and F1 from chapter 06.
- **Precision** measures what fraction of the model’s predicted tokens are correct:  $P = TP / (TP + FP)$ . A model that predicts very few entities but gets them all right has high precision.
- **Recall** measures what fraction of the gold tokens the model found:  $R = TP / (TP + FN)$ . A model that predicts every token as an entity has perfect recall but terrible precision.
- **F1** is the harmonic mean of precision and recall:

$$F_1 = \frac{2PR}{P + R}$$

- The harmonic mean (rather than arithmetic) penalises imbalance: if either P or R is low,  $F_1$  is low. For NER (file 02), F1 is computed per entity type and then macro-averaged across types. For POS tagging, token-level accuracy is more common because every token gets a tag.
- **Span-level F1** (used in SQuAD) compares the set of tokens in the predicted span to the set in the gold span. This is more forgiving than exact match: if the gold answer is “the Eiffel Tower” and the model predicts “Eiffel Tower”, the span F1 is high (4 overlapping tokens out of 5) even though EM is zero.

- **BLEU** (Bilingual Evaluation Understudy, Papineni et al., 2002) is the classic metric for machine translation. It measures n-gram overlap between the candidate translation and one or more reference translations. The score combines precision at multiple n-gram levels (unigram through 4-gram) with a brevity penalty:

$$\text{BLEU} = \text{BP} \cdot \exp \left( \sum_{n=1}^N w_n \log p_n \right)$$

- where  $p_n$  is the **modified n-gram precision**: the count of each n-gram in the candidate is clipped to its maximum count in any reference, preventing a degenerate candidate like “the the the the” from scoring high. The weights  $w_n$  are typically uniform ( $w_n = 1/N$ , with  $N = 4$ ).
- The **brevity penalty**  $\text{BP} = \min(1, \exp(1 - r/c))$  penalises candidates shorter than the reference ( $c$  is candidate length,  $r$  is reference length). Without this, a model could achieve high precision by outputting very few, very safe words.
- BLEU correlates reasonably with human judgement at the corpus level (averaged over many sentences) but poorly at the sentence level.
- It rewards exact n-gram matches and misses valid paraphrases: “the cat is on the mat” and “a feline sits atop the rug” have zero bigram overlap despite meaning the same thing.
- BLEU also ignores recall entirely — a candidate that produces only the most common words scores well on precision.
- **ROUGE** (Recall-Oriented Understudy for Gisting Evaluation, Lin, 2004) is the standard metric for summarisation. Unlike BLEU, which emphasises precision, ROUGE emphasises recall: what fraction of the reference n-grams appear in the candidate?
- **ROUGE-N** computes recall of n-grams:  $\text{ROUGE-N} = \frac{|\text{n-grams}_{\text{ref}} \cap \text{n-grams}_{\text{cand}}|}{|\text{n-grams}_{\text{ref}}|}$ .  
ROUGE-1 (unigram) and ROUGE-2 (bigram) are most common.

- ROUGE-L uses the **longest common subsequence (LCS)** between candidate and reference, which captures sentence-level word ordering without requiring consecutive matches.
- The LCS length normalised by reference length gives recall, normalised by candidate length gives precision, and the F-measure combines them.
- LCS is computed via dynamic programming in  $O(mn)$  time (similar to edit distance from file 02):

$$R_{LCS} = \frac{LCS(X, Y)}{m}, \quad P_{LCS} = \frac{LCS(X, Y)}{n}, \quad F_{LCS} = \frac{(1 + \beta^2)R_{LCS}P_{LCS}}{R_{LCS} + \beta^2P_{LCS}}$$

- where  $m$  and  $n$  are the lengths of reference and candidate, and  $\beta$  is typically set to favour recall ( $\beta \rightarrow \infty$  gives pure recall).
- **METEOR** (Metric for Evaluation of Translation with Explicit ORdering, Banerjee and Lavie, 2005) addresses BLEU's weaknesses by incorporating synonyms, stemming, and word order.
- It first aligns words between candidate and reference using exact matches, stem matches (via Porter stemming from file 02), and synonym matches (via WordNet from file 01).
- Then it computes a harmonic mean of unigram precision and recall weighted toward recall, and applies a fragmentation penalty that penalises candidates where matched words appear in a different order than the reference.
- **ChrF** (Character n-gram F-score) computes F-score over character n-grams rather than word n-grams. This makes it robust to morphological variation (critical for agglutinative languages from file 01) and partially handles tokenisation differences. ChrF++ adds word bigrams to the character n-grams.
- It has become a recommended metric for machine translation alongside BLEU, especially for morphologically rich languages.
- **Perplexity** (file 02) measures how well a language model predicts a held-out test set. It is the standard intrinsic metric for language models:  $PPL = \exp\left(-\frac{1}{N} \sum_i \log P(w_i \mid w_{<i})\right)$ . Lower is better.

- Perplexity is comparable only between models using the same tokenisation, since different tokenisers produce different sequence lengths  $N$  for the same text.
- A model with a larger vocabulary tends to have lower perplexity per token but processes fewer tokens per sentence.
- **Bits-per-byte** (BPB) normalises by the number of UTF-8 bytes in the text rather than the number of tokens, making it tokenisation-independent:

$$\text{BPB} = \frac{-\sum_i \log_2 P(w_i | w_{<i})}{\text{number of UTF-8 bytes}}$$

- **BERTScore** (Zhang et al., 2020) moves beyond surface-level n-gram matching by computing similarity in embedding space. Each token in the candidate is matched to its most similar token in the reference using cosine similarity of contextual embeddings (typically from a pre-trained BERT model). The scores are aggregated into precision, recall, and F1:

$$R_{\text{BERT}} = \frac{1}{|r|} \sum_{r_i \in r} \max_{c_j \in c} \cos(r_i, c_j), \quad P_{\text{BERT}} = \frac{1}{|c|} \sum_{c_j \in c} \max_{r_i \in r} \cos(c_j, r_i)$$

- where  $r_i$  and  $c_j$  are contextual embeddings of reference and candidate tokens. This captures semantic similarity that n-gram metrics miss: “automobile” and “car” score highly because their BERT embeddings are similar, even though they share no characters.
- **BLEURT** (Sellam et al., 2020) takes this further by fine-tuning a BERT model directly on human quality judgements. Given a reference and candidate pair, it

outputs a scalar quality score. BLEURT is trained on synthetic data (random perturbations of reference translations rated by metrics like BLEU and METEOR) and then fine-tuned on human ratings. It correlates better with human judgement than any surface-level metric.

- **COMET** (Crosslingual Optimized Metric for Evaluation of Translation, Rei et al., 2020) is a learned metric for machine translation that conditions on the source sentence, reference, and candidate — not just reference and candidate. It uses a multilingual encoder (XLM-R) to embed all three and predicts a quality score. By seeing the source, COMET can detect meaning errors that reference-only metrics miss (e.g., a fluent but factually wrong translation).
- **LLM-as-judge** is the modern approach to evaluation at scale. Instead of computing metrics against references, a powerful language model (GPT-4, Claude) is prompted to evaluate the quality of model outputs. The judge receives the input, the model's response, and optionally a reference answer, and produces a rating (e.g., 1-5) or a pairwise preference (response A is better than response B).
- **Pairwise comparison** (used in Chatbot Arena) is the most reliable LLM-as-judge format. The judge sees two responses and picks the better one, rather than assigning absolute scores. This avoids calibration issues (different judges may have different baselines for “3 out of 5”). Results are aggregated into **Elo ratings** (from chess), where each model starts with a base rating and gains or loses points based on wins and losses against other models. The expected win probability of model A against model B is:

$$P(A > B) = \frac{1}{1 + 10^{(R_B - R_A)/400}}$$

- where  $R_A, R_B$  are the Elo ratings. After each comparison, ratings are updated:  $R'_A = R_A + K(S - P(A > B))$ , where  $S \in \{0, 1\}$  is the actual outcome and  $K$  controls the update magnitude. Models that consistently beat strong opponents rise quickly; models that lose to weak opponents fall.
- **Position bias** is a known issue with LLM judges: they tend to prefer the response presented first (or in some models, the response presented second). **Swapping** (evaluating each pair twice with responses in both orders) and averaging the results mitigates this.



- **Verbosity bias** is another: judges tend to prefer longer, more detailed responses even when a concise answer is better.
- **Self-consistency** checks whether the judge gives the same rating across multiple evaluations of the same input. High variance indicates the evaluation signal is noisy.
- **Inter-annotator agreement** (Cohen's kappa or Krippendorff's alpha) measures whether multiple judges agree, providing an upper bound on evaluation reliability.
- **Contamination** is a critical concern: if the evaluation data appeared in the model's training set, benchmark scores are inflated and meaningless.
- This is especially problematic for LLMs trained on web-scraped data, where popular benchmarks are likely present. Mitigation strategies include: using held-out test sets that are not publicly released, creating dynamic benchmarks that regenerate questions periodically, **canary strings** (unique identifiers embedded in benchmark data to detect leakage), and comparing performance on contaminated vs clean subsets.
- **Standard NLU benchmarks** evaluate language understanding across diverse tasks.
- **GLUE** (General Language Understanding Evaluation) and **SuperGLUE** are multi-task benchmarks covering sentiment (SST-2), textual similarity (STS-B), natural language inference (MNLI, RTE), coreference (WSC), and question answering (BoolQ).
- Models are evaluated on each task separately and scored by an aggregate metric. GLUE is now considered saturated (models exceed human performance on most tasks); SuperGLUE remains more challenging.
- **MMLU** (Massive Multitask Language Understanding) evaluates knowledge and reasoning across 57 academic subjects (mathematics, history, law, medicine, computer science, etc.) using multiple-choice questions.
- It tests whether a model has absorbed broad knowledge during pre-training. Scores are reported per subject and as a macro-average.
- **MMLU-Pro** adds harder, multi-step reasoning questions with 10 answer choices instead of 4.

- **HellaSwag** tests commonsense reasoning by asking the model to choose the most plausible continuation of a scenario. The wrong answers are generated adversarially (using models) to be superficially plausible but semantically wrong.
- **WinoGrande** tests commonsense coreference resolution with minimal pairs that differ by one word.
- **ARC** (AI2 Reasoning Challenge) uses grade-school science questions in easy and challenge sets, testing factual and reasoning ability.
- **Reasoning and maths benchmarks** evaluate the problem-solving capabilities that separate strong LLMs from weak ones.
- **GSM8K** (Grade School Math 8K) contains 8,500 elementary maths word problems requiring multi-step arithmetic reasoning. It is the standard benchmark for basic mathematical reasoning and for evaluating chain-of-thought prompting (file 04).
- **MATH** is a harder dataset of competition-level maths problems across algebra, number theory, geometry, counting, and probability. Problems require multi-step symbolic reasoning, and MATH-500 is a commonly reported 500-problem subset.
- **AIME** (American Invitational Mathematics Examination) problems are competition-level: solving them correctly requires deep mathematical reasoning over many steps. DeepSeek-R1 scores 79.8% on AIME 2024, demonstrating that RL-trained reasoning models (file 05) can approach strong human competitors.
- **HumanEval** and **MBPP** (Mostly Basic Programming Problems) evaluate code generation by checking whether the model's code passes unit tests. HumanEval contains 164 Python problems with function signatures and docstrings; the model must generate the function body.
- The metric is **pass@k**: the probability that at least one of k generated solutions passes all tests. For a single sample:

$$\text{pass@k} = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}$$

- where  $n$  is the total number of generated samples and  $c$  is the number that pass. This formula corrects for the bias in simply taking the best of  $k$  samples.
- **SWE-bench** goes further, evaluating whether models can resolve real GitHub issues by modifying existing codebases — a much harder test of practical software engineering ability.
- **GPQA** (Graduate-Level Google-Proof QA) contains expert-level questions in biology, physics, and chemistry that are difficult even for domain experts. It tests whether models have genuine understanding rather than pattern matching. The “Diamond” subset is the hardest.
- **Safety and alignment benchmarks** evaluate whether models are helpful, harmless, and honest.
- **TruthfulQA** tests whether models reproduce common misconceptions. Questions are designed so that the most common internet answers are wrong (e.g., “What happens if you swallow gum?” , the common myth is that it stays for 7 years, but the truthful answer is that it passes through normally). Models that have memorised popular but incorrect claims score poorly.
- **BBQ** (Bias Benchmark for QA) tests for social biases across categories like age, gender, race, and religion. Questions are structured so that a biased model would systematically choose stereotypical answers. **Toxigen** evaluates the model’s tendency to generate toxic content about specific demographic groups.
- **MT-Bench** evaluates multi-turn conversation ability using 80 carefully designed questions across writing, roleplay, reasoning, maths, coding, extraction, STEM, and humanities. An LLM judge (GPT-4) scores responses on a 1-10 scale. The multi-turn format tests whether models can follow up, maintain context, and handle clarification requests.
- **Chatbot Arena** (LMSYS) uses real users to conduct blind pairwise comparisons between anonymous models. Users submit prompts and vote for the better response without knowing which model produced it. The resulting Elo leaderboard is considered the most ecologically valid evaluation of general-

purpose LLM quality because it reflects real user preferences on diverse, uncurated prompts.

- **AlpacaEval** automates pairwise evaluation by comparing model outputs against a reference model (GPT-4) on a fixed set of instructions. A judge model determines the win rate.
- **AlpacaEval 2.0** uses length-controlled win rates to correct for verbosity bias.
- **Task-specific evaluation** requires tailored metrics for specialised domains.
- **Word Error Rate (WER)** for speech recognition:  $WER = (S + D + I)/N$ , where S, D, I are substitution, deletion, and insertion errors and N is the number of reference words. This is the edit distance (file 02) normalised by reference length, applied at the word level.
- **Slot F1** for task-oriented dialogue systems measures whether the model correctly extracts structured information from user utterances (e.g., extracting “destination: Paris” and “date: tomorrow” from “Book me a flight to Paris tomorrow” ).
- **Citation accuracy** for RAG systems (file 05) checks whether the model’s generated citations actually support the claims made. A claim is verified against the retrieved passage, and the metric counts the fraction of claims that are fully, partially, or not supported.
- **Evaluation pitfalls** are common and can invalidate entire benchmark comparisons.
- **Teaching to the test:** optimising for benchmark performance rather than genuine capability. A model fine-tuned on MMLU-style multiple choice will score well on MMLU but may fail on the same questions posed in open-ended format.
- **Metric gaming:** models can be optimised to produce outputs that score well on automatic metrics (high BLEU, low perplexity) without being genuinely good. The BLEU-optimal translation is often a safe, generic paraphrase rather than a natural, fluent one.
- **Benchmark saturation:** when models approach or exceed human performance on a benchmark, the benchmark stops being informative. GLUE, SQuAD 1.1, and several others are now saturated.

- The field continuously creates harder benchmarks, but the cycle of creation, saturation, and replacement makes longitudinal comparison difficult.
- **Human evaluation** remains the gold standard but is expensive, slow, and hard to reproduce. Different annotator pools (crowdworkers vs domain experts, different cultures, different languages) produce different judgements. Reporting inter-annotator agreement and annotator demographics is essential for reproducibility.

## 34.1 Coding Tasks (use CoLab or notebook)

1. Implement a full Transformer encoder block from scratch (multi-head attention, feed-forward, residual connections, layer norm). Apply it to a simple sequence classification task.

```
import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

def layer_norm(x, gamma, beta, eps=1e-5):
 mean = x.mean(axis=-1, keepdims=True)
 var = x.var(axis=-1, keepdims=True)
 return gamma * (x - mean) / jnp.sqrt(var + eps) + beta

def multi_head_attention(Q, K, V, W_q, W_k, W_v, W_o, n_heads):
 B, T, D = Q.shape
 head_dim = D // n_heads

 q = Q @ W_q # (B, T, D)
 k = K @ W_k
 v = V @ W_v

 # Reshape to (B, n_heads, T, head_dim)
 q = q.reshape(B, T, n_heads, head_dim).transpose(0, 2, 1, 3)
 k = k.reshape(B, T, n_heads, head_dim).transpose(0, 2, 1, 3)
 v = v.reshape(B, T, n_heads, head_dim).transpose(0, 2, 1, 3)

 scores = q @ k.transpose(0, 1, 3, 2) / jnp.sqrt(head_dim)
 weights = jax.nn.softmax(scores, axis=-1)
 out = (weights @ v).transpose(0, 2, 1, 3).reshape(B, T, D)
 return out @ W_o, weights

def transformer_block(x, params):
 # Pre-norm multi-head self-attention
```

```

 normed = layer_norm(x, params['ln1_g'], params['ln1_b'])
 attn_out, weights = multi_head_attention(
 normed, normed, normed,
 params['W_q'], params['W_k'], params['W_v'], params['W_o'],
 n_heads=4
)
 x = x + attn_out

 # Pre-norm feed-forward
 normed = layer_norm(x, params['ln2_g'], params['ln2_b'])
 ff = jax.nn.gelu(normed @ params['W1'] + params['b1'])
 ff = ff @ params['W2'] + params['b2']
 x = x + ff
 return x, weights

Initialise parameters
d_model, d_ff, n_heads = 32, 128, 4
key = jax.random.PRNGKey(42)
keys = jax.random.split(key, 10)

params = {
 'W_q': jax.random.normal(keys[0], (d_model, d_model)) * 0.05,
 'W_k': jax.random.normal(keys[1], (d_model, d_model)) * 0.05,
 'W_v': jax.random.normal(keys[2], (d_model, d_model)) * 0.05,
 'W_o': jax.random.normal(keys[3], (d_model, d_model)) * 0.05,
 'ln1_g': jnp.ones(d_model), 'ln1_b': jnp.zeros(d_model),
 'ln2_g': jnp.ones(d_model), 'ln2_b': jnp.zeros(d_model),
 'W1': jax.random.normal(keys[4], (d_model, d_ff)) * 0.05,
 'b1': jnp.zeros(d_ff),
 'W2': jax.random.normal(keys[5], (d_ff, d_model)) * 0.05,
 'b2': jnp.zeros(d_model),
}

Test with random input
x = jax.random.normal(keys[6], (2, 8, d_model)) # batch=2, seq_len=8
out, attn_weights = transformer_block(x, params)
print(f"Input shape: {x.shape}")
print(f"Output shape: {out.shape}")
print(f"Attention weights shape: {attn_weights.shape}") # (B, n_heads, T, T)

Visualise attention patterns for each head
fig, axes = plt.subplots(1, 4, figsize=(16, 3.5))
for h in range(4):
 im = axes[h].imshow(attn_weights[0, h], cmap='Blues', vmin=0)
 axes[h].set_title(f"Head {h}")
 axes[h].set_xlabel("Key pos"); axes[h].set_ylabel("Query pos")

```

```
plt.suptitle("Multi-Head Attention Patterns")
plt.tight_layout(); plt.show()
```

1. Implement causal (autoregressive) attention masking and compare it with bidirectional attention. Show how the mask prevents information from flowing from future to past tokens.

```
import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

def attention(Q, K, V, mask=None):
 d_k = Q.shape[-1]
 scores = Q @ K.T / jnp.sqrt(d_k)
 if mask is not None:
 scores = jnp.where(mask, scores, -1e9)
 weights = jax.nn.softmax(scores, axis=-1)
 return weights @ V, weights

seq_len, d_model = 6, 8
key = jax.random.PRNGKey(0)
k1, k2, k3 = jax.random.split(key, 3)
Q = jax.random.normal(k1, (seq_len, d_model))
K = jax.random.normal(k2, (seq_len, d_model))
V = jax.random.normal(k3, (seq_len, d_model))

Bidirectional (encoder-style): all positions visible
bidir_mask = jnp.ones((seq_len, seq_len), dtype=bool)
bidir_out, bidir_weights = attention(Q, K, V, bidir_mask)

Causal (decoder-style): only past and current positions visible
causal_mask = jnp.tril(jnp.ones((seq_len, seq_len), dtype=bool))
causal_out, causal_weights = attention(Q, K, V, causal_mask)

fig, axes = plt.subplots(1, 3, figsize=(14, 4))
tokens = [f"t{i}" for i in range(seq_len)]

axes[0].imshow(bidir_weights, cmap='Blues', vmin=0, vmax=0.5)
axes[0].set_title("Bidirectional Attention\n(BERT-style)")
axes[0].set_xticks(range(seq_len)); axes[0].set_xticklabels(tokens)
axes[0].set_yticks(range(seq_len)); axes[0].set_yticklabels(tokens)

axes[1].imshow(causal_mask.astype(float), cmap='Greys', vmin=0, vmax=1)
axes[1].set_title("Causal Mask\n(1 = allowed, 0 = blocked)")
axes[1].set_xticks(range(seq_len)); axes[1].set_xticklabels(tokens)
axes[1].set_yticks(range(seq_len)); axes[1].set_yticklabels(tokens)
```

```

axes[2].imshow(causal_weights, cmap='Blues', vmin=0, vmax=0.5)
axes[2].set_title("Causal Attention\n(GPT-style)")
axes[2].set_xticks(range(seq_len)); axes[2].set_xticklabels(tokens)
axes[2].set_yticks(range(seq_len)); axes[2].set_yticklabels(tokens)

for ax in axes:
 ax.set_xlabel("Key"); ax.set_ylabel("Query")
plt.tight_layout(); plt.show()

Verify: in causal attention, output at position i depends only on positions <=
i

print("Causal attention weight at position 2 (should only attend to 0, 1, 2):")
print(f" Weights: {causal_weights[2]}")
print(f" Sum of future weights (should be ~0): {causal_weights[2, 3:].sum():.6f}")

```

1. Implement LoRA (Low-Rank Adaptation) and show how it modifies a weight matrix with far fewer trainable parameters than full fine-tuning.

```

import jax
import jax.numpy as jnp

d_model = 256
rank = 4 # LoRA rank (much smaller than d_model)

key = jax.random.PRNGKey(42)
k1, k2, k3 = jax.random.split(key, 3)

Original frozen weight matrix
W_frozen = jax.random.normal(k1, (d_model, d_model)) * 0.02

LoRA matrices (only these are trainable)
B = jnp.zeros((d_model, rank)) # initialised to zero
A = jax.random.normal(k2, (rank, d_model)) * 0.01 # random init

Forward pass: W_effective = W_frozen + B @ A
x = jax.random.normal(k3, (8, d_model))

Without LoRA
y_original = x @ W_frozen.T

With LoRA
W_effective = W_frozen + B @ A
y_lora = x @ W_effective.T

```



```

Parameter counts
full_params = d_model * d_model
lora_params = d_model * rank + rank * d_model # B + A

print(f"Model dimension: {d_model}")
print(f"LoRA rank: {rank}")
print(f"Full fine-tuning parameters: {full_params:,}")
print(f"LoRA parameters: {lora_params:,}")
print(f"Parameter reduction: {full_params / lora_params:.1f}x")
print(f"\nSince B is initialised to zeros, initial LoRA output matches
original:")
print(f" Max difference: {jnp.abs(y_original - y_lora).max():.2e}")

Simulate training: only update A and B
def lora_forward(A, B, W_frozen, x):
 return x @ (W_frozen + B @ A).T

def dummy_loss(A, B, W_frozen, x, target):
 pred = lora_forward(A, B, W_frozen, x)
 return jnp.mean((pred - target) ** 2)

Target: some transformation of x
target = x @ jax.random.normal(jax.random.PRNGKey(99), (d_model, d_model)).T *
0.02

grad_fn = jax.jit(jax.grad(dummy_loss, argnums=(0, 1)))
lr = 0.01

for step in range(200):
 gA, gB = grad_fn(A, B, W_frozen, x, target)
 A = A - lr * gA
 B = B - lr * gB

loss_before = dummy_loss(jnp.zeros_like(A), jnp.zeros_like(B), W_frozen, x,
target)
loss_after = dummy_loss(A, B, W_frozen, x, target)
print(f"\nLoss before LoRA: {loss_before:.6f}")
print(f"Loss after LoRA: {loss_after:.6f}")
print(f"Effective weight change rank: {jnp.linalg.matrix_rank(B @ A)}")

```

## 35 Advanced Text Generation

Advanced text generation goes beyond vanilla autoregressive decoding to improve quality, controllability, and speed. This file covers text diffusion models (D3PM,

MDLM), OCR, RLHF and DPO for alignment, long-context methods (RoPE scaling, ring attention), retrieval-augmented generation, and speculative decoding for faster inference.

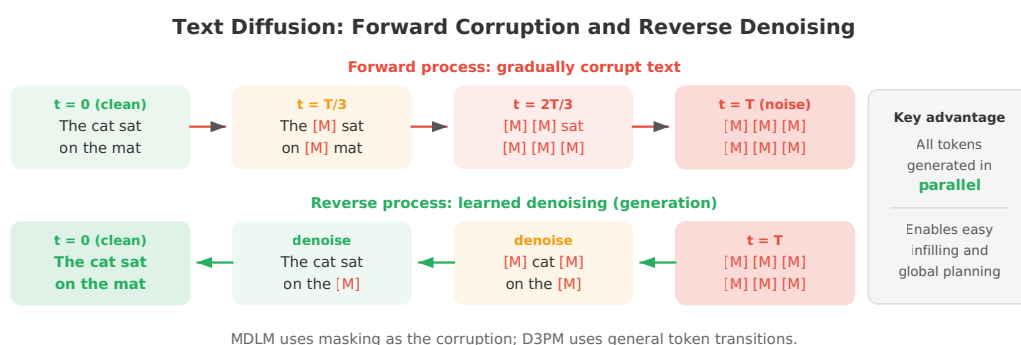
- Standard autoregressive generation (file 04) produces text one token at a time, left to right. This is simple and effective, but it is inherently sequential, allows no global planning, and gives limited control over the output. This file covers methods that go beyond vanilla autoregressive decoding: diffusion models for text, optical character recognition, controllable generation through human feedback, handling long contexts, retrieval-augmented generation, and speculative decoding for faster inference.
- **Text diffusion models** apply the diffusion framework (introduced for images in chapter 08) to discrete text. The core challenge is that text is discrete: you cannot add continuous Gaussian noise to tokens the way you add noise to pixels. Several approaches address this.
- **D3PM** (Discrete Denoising Diffusion Probabilistic Models, Austin et al., 2021) defines a forward corruption process directly over discrete tokens using transition matrices. At each forward step, a token has some probability of being replaced by another token (uniform noise), masked (absorbing state), or staying the same. The reverse process learns to denoise, predicting the clean token from the corrupted one. The transition matrix  $Q_t$  at step  $t$  controls corruption:

$$q(x_t \mid x_{t-1}) = \text{Cat}(x_t; x_{t-1} Q_t)$$

- where  $\text{Cat}$  denotes a categorical distribution and  $x$  is a one-hot vector. The multi-step forward process  $q(x_t \mid x_0)$  has a closed form:  $q(x_t \mid x_0) = \text{Cat}(x_t; x_0 Q_t)$  where  $Q_t = Q_1 Q_2 \cdots Q_t$  is the product of all transition matrices up to step  $t$ . Training minimises a variational lower bound (ELBO) that decomposes across timesteps, similar to the continuous case (chapter 08):

$$\mathcal{L}_{\text{D3PM}} = D_{\text{KL}}(q(x_T \mid x_0) \parallel p(x_T)) + \sum_{t=2}^T D_{\text{KL}}(q(x_{t-1} \mid x_t, x_0) \parallel p_{\theta}(x_{t-1} \mid x_t)) - \log p_{\theta}(x_0 \mid x_1)$$

- The first term ensures the fully corrupted distribution matches the prior (uniform or all-mask). The sum of KL terms trains the model to reverse each corruption step: the true reverse posterior  $q(x_{t-1} \mid x_t, x_0)$  can be computed in closed form using Bayes' rule and the known transition matrices, and the model  $p_{\theta}(x_{t-1} \mid x_t)$  is trained to match it.
- Since both distributions are categorical, the KL divergence is a simple sum over vocabulary entries. The final term measures reconstruction quality from the least corrupted state.
- **MDLM** (Masked Diffusion Language Models, Sahoo et al., 2024) simplifies D3PM by using masking as the only corruption operation: the forward process gradually replaces tokens with a [MASK] token, and the reverse process predicts the original tokens. This connects text diffusion to masked language modelling (BERT, file 04), with the diffusion timestep controlling what fraction of tokens are masked. At  $t=0$  the text is fully clean; at  $t=T$  it is fully masked.
- **Continuous text diffusion** sidesteps the discrete problem by working in the continuous embedding space. Tokens are first mapped to their embedding vectors (chapter 06), noise is added in this continuous space, and a denoising model (typically a Transformer) learns to reverse the process. At generation time, the model produces continuous vectors that are mapped back to discrete tokens by finding the nearest embedding. The challenge is that small errors in continuous space can map to completely wrong tokens, so careful rounding and clamping are needed.



## Text Diffusion Process

- The appeal of text diffusion is that it generates all tokens simultaneously through iterative refinement, rather than left-to-right. This allows global coherence and easy infilling (generating missing text in the middle of a passage), but current text diffusion models still lag behind autoregressive models in generation quality for long-form text.
- **Text OCR** (Optical Character Recognition) is the task of extracting text from images. While not traditionally grouped with language generation, modern OCR systems are deeply integrated with NLP and increasingly use language model components.
- **Scene text detection** locates text regions in natural images (street signs, product labels, licence plates). This is challenging because text in the wild appears at arbitrary angles, scales, fonts, and against cluttered backgrounds. Detection methods typically use CNN or Transformer backbones to produce bounding boxes or segmentation masks around text regions.
- **CRNN** (Convolutional Recurrent Neural Network, Shi et al., 2017) is a classic text recognition architecture. A CNN extracts visual features from the text image, the feature map is sliced into a sequence of columns (one per horizontal position), and a bidirectional LSTM reads this sequence to model context. The output is decoded using **CTC** (Connectionist Temporal Classification), which handles the alignment between input columns and output characters without requiring explicit segmentation.
- The fundamental problem CTC solves: the model produces  $T$  output distributions (one per input column), but the target text has  $L \leq T$  characters.
- We do not know which columns correspond to which characters. CTC introduces a **blank token**  $\epsilon$  and defines a many-to-one mapping  $\mathcal{B}$  that collapses repeated characters and removes blanks:  $\mathcal{B}(\text{"HH-ee-ll-ll-oo"}) = \text{"Hello"}$  (where “-” is blank).
- The probability of the target sequence  $y$  is the sum over all input alignments that collapse to  $y$ :

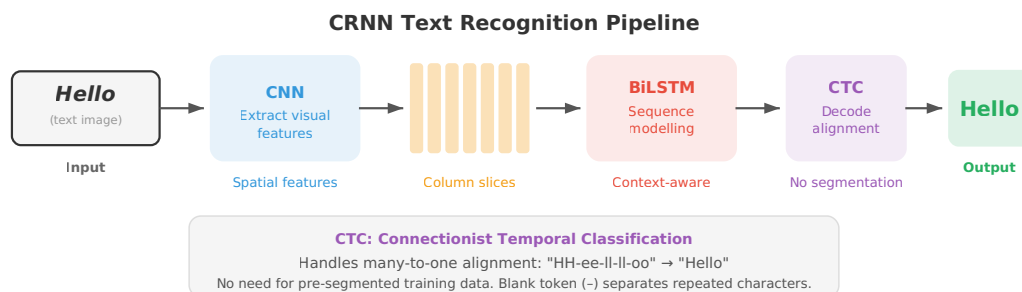
$$P(y \mid x) =$$

$$\sum_{\pi \in \mathcal{B}^{-1}(y)}$$

T  
 $\prod$   
 $t=1$

$$P(\pi_t \mid \mathbf{x})$$

- where  $\pi$  is an alignment path of length  $T$  (one label per column, including blanks). Naively summing over all paths is exponential, but the **forward algorithm** (chapter 05 HMMs) computes this sum efficiently in  $O(T \cdot L)$  time using dynamic programming.
- The blank token is essential: without it, repeated characters like “ll” in “Hello” would be indistinguishable from a single “l”. Training maximises  $\log P(y \mid x)$ , and at inference time, the best path is found by beam search or greedy decoding over the CTC output.
- **Document OCR** processes structured documents (invoices, forms, scientific papers) and must understand layout in addition to recognising characters. Modern systems like LayoutLM combine text recognition with spatial position features: each token gets both its text embedding and a positional embedding encoding its  $(x, y)$  coordinates on the page. This allows the model to understand that a number appearing below “Total:” is the total amount.



## CRNN OCR Pipeline

- **Vision-language OCR** models like TrOCR treat text recognition as image-to-text generation: a Vision Transformer encoder processes the image, and a language model decoder generates the text character by character. This leverages the power of pre-trained vision and language models and handles diverse scripts, fonts, and layouts without handcrafted feature engineering.
- **Controllable generation** is the challenge of steering a language model to produce outputs with desired properties: a particular style, topic, sentiment, safety level, or factual accuracy. The model should follow instructions while remaining fluent and coherent.

- **Classifier-free guidance (CFG)** for text adapts a technique from image generation. During training, the conditioning signal (e.g., a prompt) is randomly dropped some fraction of the time, training both a conditional and unconditional model in one. At inference, the output logits are interpolated:

$$\text{logits}_{\text{guided}} = (1 + w) \cdot \text{logits}_{\text{conditional}} - w \cdot \text{logits}_{\text{unconditional}}$$

- where  $w > 0$  amplifies the influence of the condition. Higher  $w$  makes the output more strongly follow the prompt but reduces diversity.
- **RLHF** (Reinforcement Learning from Human Feedback, Ouyang et al., 2022) is the dominant method for aligning language models with human preferences. The process has three stages:

- First, **supervised fine-tuning (SFT)**: fine-tune the base language model on a dataset of high-quality human-written responses to prompts.
- Second, **reward model training**: collect human comparisons (given prompt  $x$  and two responses  $y_1, y_2$ , which is better?) and train a reward model  $r_\phi(x, y)$  to predict human preferences. The reward model is trained with a pairwise ranking loss:

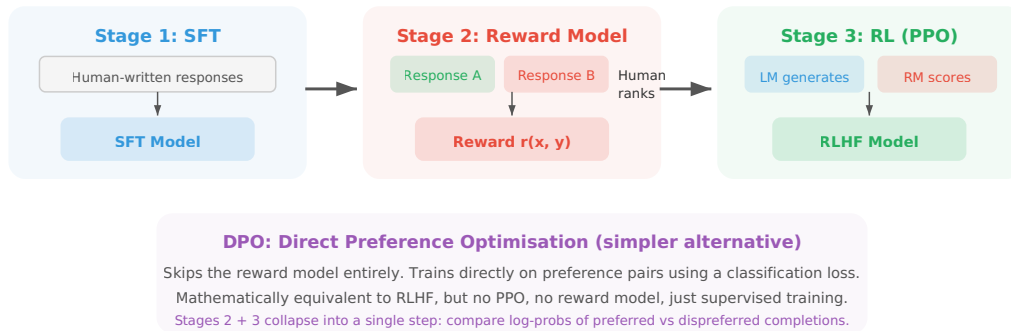
$$\mathcal{L}_{\text{RM}} = -\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))$$

- where  $y_w$  is the preferred response and  $y_l$  is the dispreferred one.
- Third, **RL fine-tuning**: optimise the language model to maximise the reward while staying close to the SFT model (to prevent mode collapse). This uses PPO (Proximal Policy Optimisation, from chapter 06) with a KL penalty:

$$\mathcal{L}_{\text{RL}} = -\mathbb{E}[r_\phi(x, y) - \beta D_{\text{KL}}(\pi_\theta // \pi_{\text{SFT}})]$$

- The KL term prevents the model from drifting too far from the base model and exploiting quirks of the reward model ( “reward hacking” ).

## RLHF: Reinforcement Learning from Human Feedback



### RLHF Pipeline

- **DPO** (Direct Preference Optimisation, Rafailov et al., 2023) simplifies RLHF by eliminating the reward model entirely. The key mathematical insight is that the KL-constrained RL objective above has a closed-form optimal policy:

$$\pi^*(y \mid x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y \mid x) \exp \left( \frac{r(x, y)}{\beta} \right)$$

- where  $Z(x)$  is a normalising partition function. Rearranging this for the reward gives  $r(x, y) = \beta \log \frac{\pi^*(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta \log Z(x)$ . Substituting this implicit reward into the Bradley-Terry preference model  $P(y_w > y_l) = \sigma(r(x, y_w) - r(x, y_l))$  causes the intractable  $Z(x)$  terms to cancel, yielding the DPO loss directly:

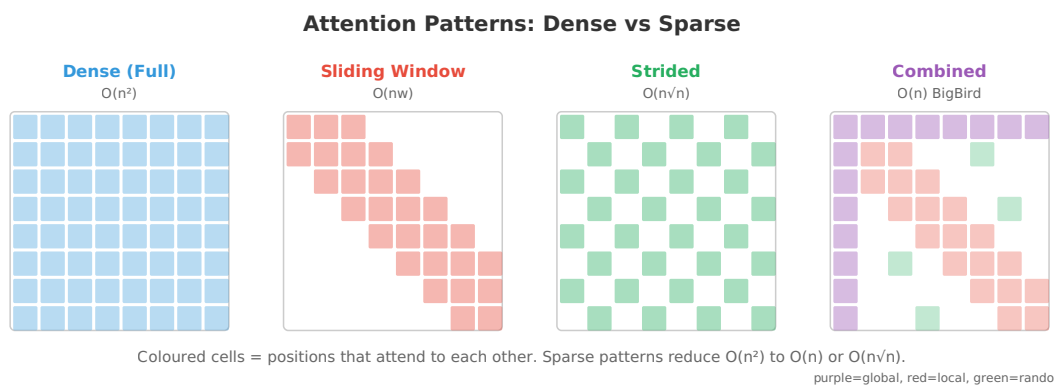
$$\mathcal{L}_{\text{DPO}} = - \log \sigma \left( \beta \log \frac{\pi_{\theta}(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_{\theta}(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} \right)$$

- This is mathematically equivalent to RLHF but collapses the reward model and RL training into a single supervised step.
- The expression inside the sigmoid can be read as: “increase the relative probability of the preferred response and decrease the relative probability of the dispreferred response, measured against the reference model.”
- The  $\beta$  parameter controls how much the policy can deviate from the reference. In practice, DPO is simpler to implement (just compute log-probabilities under the current and reference models for both completions) and avoids the instabilities of PPO training.
- **Constitutional AI** (Bai et al., 2022) automates parts of the alignment process. Instead of collecting human comparisons, it uses the language model itself to

critique and revise its own outputs according to a set of principles (the “constitution” ), such as “choose the response that is less harmful.” The AI-generated comparisons are then used for preference training (RLAIF: RL from AI Feedback).

- **Long-context methods** address the  $O(n^2)$  memory and compute cost of standard self-attention, which limits sequence length. As  $n$  grows into the tens or hundreds of thousands of tokens, standard attention becomes infeasible.
- **Sparse attention** replaces the dense  $n \times n$  attention matrix with a sparse pattern where each token attends to only a subset of other tokens. Common patterns include **local attention** (each token attends to a fixed-size window of neighbours), **strided attention** (attend to every  $k$ -th token), and **random attention** (attend to a random subset). Combinations of these patterns (used in BigBird, Longformer) achieve  $O(n)$  or  $O(n\sqrt{n})$

$\sqrt{n}$  complexity while maintaining the ability to capture both local and global dependencies.



### Sparse Attention Patterns

- **Sliding window attention** restricts each token to attend only to the previous  $w$  tokens (its local window). This is  $O(nw)$  rather than  $O(n^2)$ , but long-range information must propagate through overlapping windows across layers. With  $L$  layers and window size  $w$ , the effective receptive field is  $L \times w$  tokens.
- **Ring attention** distributes long sequences across multiple devices by arranging them in a ring topology. Each device holds a chunk of the sequence and computes attention for its chunk while simultaneously sending key-value blocks to the next device in the ring. This overlaps computation with communication and allows sequences of arbitrary length limited only by the total memory across all devices, not the memory of any single one.



- **Memory-augmented models** extend context by equipping the Transformer with an external memory bank. At each layer, the model can read from and write to this memory using attention. Memorizing Transformers cache key-value pairs from previous chunks and attend to them in subsequent chunks, effectively extending context beyond the training window. The retrieval is approximate (using k-nearest neighbours over cached keys) to keep it efficient.
- The methods above are **architectural** solutions to long context. Equally important is how models are **trained** to use long contexts effectively.
- **Progressive context extension** is the standard approach. Training on very long sequences from the start is prohibitively expensive ( $O(n^2)$  attention cost), so models are pre-trained at a short context length (typically 4K–8K tokens) and then **continued pre-training** extends to the target length in stages.
- Llama 3.1 extends from 8K to 128K over 800B tokens with gradually increasing sequence length. DeepSeek-V3 trains at 4K, then extends to 32K, then 128K.
- Each stage uses a modest number of tokens (relative to the full pre-training budget) because the model only needs to learn how to use longer positions, not relearn language itself.
- The position encoding must be adjusted during extension. **RoPE interpolation** scales down the position indices so that the model sees the same rotation angles it was trained on, just spread over a longer sequence. If the model was trained at length  $L$  and you want to extend to  $L' = 4L$ , you divide all position indices by 4.
- This means the model never sees a rotation angle it has not encountered, but the effective resolution between adjacent positions drops.
- **RoPE extrapolation** keeps the original position indices unchanged and simply applies RoPE to positions beyond  $L$ , relying on the model generalising to unseen angles.
- Interpolation is much more stable; extrapolation degrades rapidly without base frequency adjustment (ABF).
- **YaRN** (Yet another RoPE extension) improves on naive interpolation by recognising that not all RoPE dimensions should be treated equally.

- High-frequency dimensions (small  $i$  in  $\theta_i = \theta_{\text{base}}^{-2i/d}$ ) rotate many times within the training length and can extrapolate well.
- Low-frequency dimensions (large  $i$ ) rotate slowly and are more sensitive to length extension.
- YaRN interpolates only the low-frequency dimensions, extrapolates the high-frequency ones, and applies a temperature scaling  $t$  to the attention logits to compensate for the distributional shift:

$$\text{score}'_{ij} = \frac{q_i^T k_j}{\frac{t}{\sqrt{d_k}}}$$

- where  $t > 1$  flattens the attention distribution, preventing the model from attending too sharply to nearby tokens when position signals are compressed.
- **Long-context data curation** is a critical and often underestimated challenge. Most pre-training corpora consist of short documents (news articles, web pages, social media posts).
- Long-context training requires a data mix that actually exercises the full context window: books, code repositories, long-form scientific articles, multi-turn conversation logs, and concatenated thematically related documents.
- If the model is only trained on short documents padded or packed to fill the context window, it learns to ignore distant tokens because they are never relevant.
- **Sequence packing** is a training efficiency technique: multiple documents are concatenated into a single training sequence to avoid padding waste, with attention masks preventing cross-document attention.
- For long-context training, the packing strategy matters: packing many unrelated short documents teaches the model that distant tokens are noise, while packing fewer, genuinely long documents teaches it to use the full context.
- A known failure mode is the “**lost in the middle**” phenomenon (Liu et al., 2023): language models tend to use information at the beginning and end of the context window effectively but struggle with information placed in the middle.

- This resembles the serial position effect in human memory (primacy and recency).
- It arises partly from training data distributions (important information is often at the start or end of documents) and partly from attention patterns that concentrate on nearby and initial tokens.
- Long-context training with diverse placement of key information mitigates but does not fully solve this.
- **Needle-in-a-haystack** evaluation tests whether a model can retrieve a specific fact ( “the needle” ) placed at various positions within a long distractor context ( “the haystack” ).
- A model with genuine long-context ability should achieve near-perfect retrieval regardless of where the needle is placed.
- This test reveals the lost-in-the-middle effect clearly and is used to benchmark context extension methods.
- **Long-context fine-tuning** after pre-training uses targeted SFT data: long multi-turn dialogues, document QA with evidence scattered across thousands of tokens, long-form summarisation, and repository-level code understanding.
- Qwen3 uses **Dual Chunk Attention (DCA)** during this stage, which processes long sequences as pairs of chunks where intra-chunk attention is full and inter-chunk attention is efficient, achieving 4x the effective sequence capacity during fine-tuning.
- **State Space Models (SSMs)** offer a fundamentally different approach to long-sequence modelling. Rather than modifying attention, they replace it entirely with a linear dynamical system inspired by continuous-time control theory.
- An SSM maps an input sequence  $u(t)$  to an output  $y(t)$  through a latent state  $x(t) \in \mathbb{R}^N$  governed by:

$$x'(t) = Ax(t) + Bu(t) , \quad y(t) = Cx(t) + Du(t)$$

- where  $A \in \mathbb{R}^{N \times N}$  is the state transition matrix,  $B \in \mathbb{R}^{N \times 1}$  is the input projection,  $C \in \mathbb{R}^{1 \times N}$  is the output projection, and  $D$  is a skip connection.
- To apply this to discrete sequences (tokens), the continuous system is **discretised** using a step size  $\Delta$ . The zero-order hold discretisation gives:

–

A

–

$$= \exp(\Delta A) \cdot B$$

$$= (\Delta A)^{-1} (\exp(\Delta A) - I) \cdot \Delta B$$

- The discrete recurrence then becomes  $x_k =$

–

A

–

$$x_{k-1} + B$$

$u_k, y_k = Cx_k + Du_k$ , which looks like an RNN: process one token at a time with a hidden state.

- Unlike RNNs, this recurrence can also be unrolled as a **global convolution**: because the system is linear, the output is  $y =$

–

K

$* u$  where the kernel

–

K

$$= (C$$

–

B

, C

–

A

–

B

, C

–

A

2

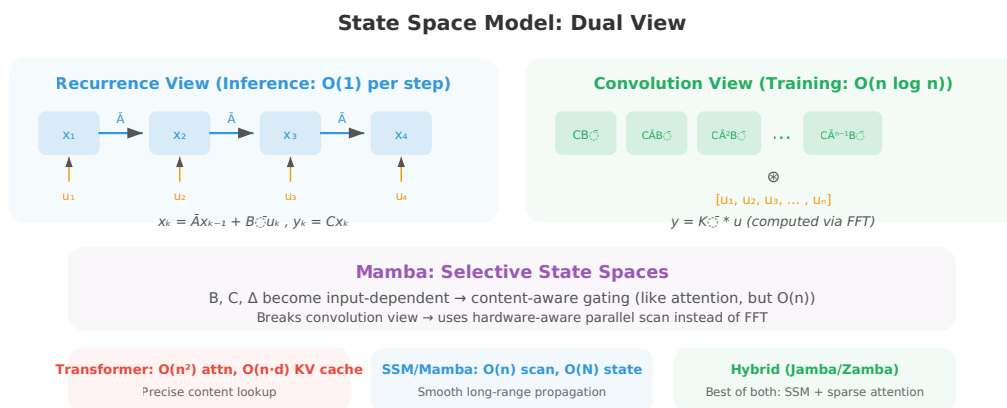
–

B

, ...)

depends only on the fixed parameters.

- This **dual view** — recurrence for efficient autoregressive inference ( $O(1)$  per step) and convolution for efficient parallel training ( $O(n \log n)$  via FFT) — is the central insight of SSMs.



SSM Dual View: recurrence for inference, convolution for training, and Mamba' s selective extension

- **S4** (Structured State Spaces for Sequence Modeling, Gu et al., 2022) made SSMs practical by solving the key numerical challenge: the state matrix  $A$  must capture long-range dependencies, but naively parameterising it leads to vanishing or exploding dynamics (the same problem as vanilla RNNs).
- S4 initialises  $A$  using the **HiPPO** (High-order Polynomial Projection Operators) matrix, which is derived from the theory of optimal polynomial approximation of continuous signals. The HiPPO matrix has a specific structure that provably enables the state to maintain a compressed representation of the entire input history with graceful decay:

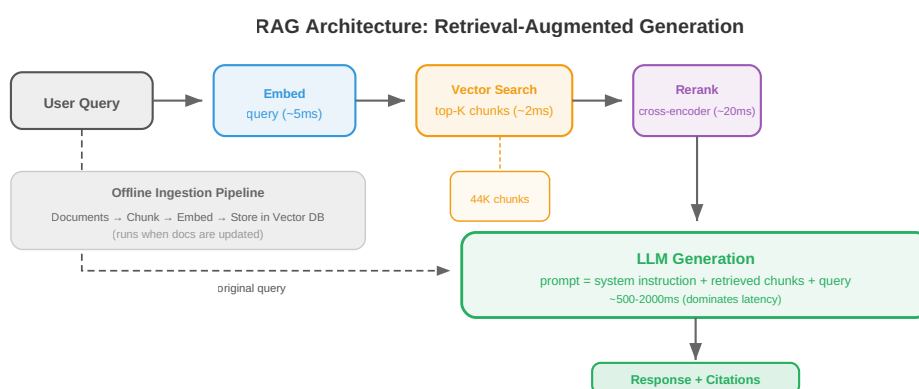
$$A_{nk} = -\begin{cases} (2n+1)^{1/2}(2k+1)^{1/2} & \text{if } n > k \\ 0 & \text{if } n = k \\ (2n+1)^{1/2}(2k+1)^{1/2} & \text{if } n < k \end{cases}$$

- This lower-triangular structure ensures that the state acts as an online approximation of the input signal using Legendre polynomials. Computing  $A^k$  for long kernels is expensive, so S4 uses the fact that the HiPPO matrix can be decomposed as a sum of low-rank and diagonal terms, enabling  $O(n \log n)$  kernel computation.
- **Mamba** (Gu and Dao, 2023) introduces the critical innovation of **selective state spaces**: making the SSM parameters input-dependent. In S4, the matrices  $A, B, C$ , and the step size  $\Delta$  are fixed — the same dynamics apply to every token regardless of content. Mamba makes  $B, C$ , and  $\Delta$  functions of the input:

$$B_k = \text{Linear}(u_k), C_k = \text{Linear}(u_k), \Delta_k = \text{softplus}(\text{Linear}(u_k))$$

- This selectivity allows the model to decide, at each position, what information to store in the state and what to ignore — analogous to how attention selects relevant tokens, but without the quadratic cost. The step size  $\Delta_k$  controls the “gate” : a large  $\Delta$  causes the state to integrate the current input strongly (the continuous dynamics advance a large step, effectively resetting the state), while a small  $\Delta$  preserves the existing state and ignores the current input.
- The trade-off is that input-dependent parameters break the convolution view (the kernel is no longer fixed), so Mamba cannot use FFT-based training. Instead, it uses a **hardware-aware parallel scan** algorithm that exploits the associativity of the recurrence: the state update  $(x_k, u_k) \mapsto x_{k+1}$  can be expressed as a sequence of associative operations and parallelised using a prefix sum (scan), analogous to parallel prefix addition in hardware design. This runs in  $O(n)$  time with  $O(\log n)$  depth on a GPU, nearly matching the efficiency of convolution.
- Mamba achieves inference that is truly  $O(1)$  per token (just update the fixed-size state, no KV cache that grows with context), making it fundamentally more memory-efficient than Transformers at long sequence lengths. The state size  $N$  (typically 16) is much smaller than a Transformer’s KV cache, which stores  $O(n \cdot d)$  values. In practice, Mamba matches or exceeds Transformer quality at the same parameter count on language modelling benchmarks, with significantly faster inference on long sequences.
- **Hybrid architectures** combine SSM layers with attention layers, using SSMs for the majority of layers (efficient long-range propagation) and sprinkling in a few attention layers (precise content-based retrieval). Models like Jamba and Zamba interleave Mamba and Transformer blocks, achieving better quality than pure SSMs while maintaining much of the inference efficiency advantage. This suggests that attention and SSMs capture complementary capabilities: SSMs excel at smooth, long-range state propagation while attention excels at precise, content-dependent lookups.
- **Retrieval-Augmented Generation (RAG)** addresses the knowledge limitations of language models by giving them access to an external knowledge base at inference time. Instead of relying solely on knowledge encoded in model parameters during training, RAG retrieves relevant documents and conditions generation on them.

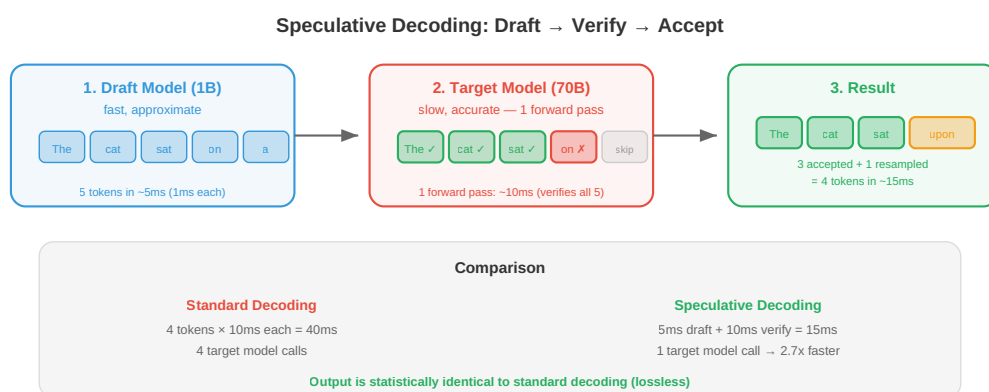
- The classic **retriever-reader architecture** has two components. The **retriever** takes a query and fetches the top-k most relevant passages from a corpus. The **reader** (a language model) generates the answer conditioned on both the query and the retrieved passages. The retriever can use sparse methods (BM25, which extends TF-IDF from file 02) or dense methods.
- **Dense passage retrieval (DPR)** uses a dual-encoder architecture: one encoder maps questions to vectors, another maps passages to vectors. Both are typically BERT-based. At indexing time, all passages are encoded and stored. At query time, the question is encoded and the nearest passages are found using approximate nearest neighbour search (such as FAISS). The similarity metric is the dot product between question and passage vectors.
- **Chunking strategies** affect retrieval quality significantly. Documents must be split into passages small enough for the retriever to handle, but large enough to contain complete ideas. Fixed-size chunking (e.g., 256 tokens with 50-token overlap) is simple but may split sentences awkwardly. Semantic chunking splits at paragraph or section boundaries. Hierarchical chunking creates a tree of summaries at different granularities.



## RAG Architecture

- RAG provides several advantages: the knowledge base can be updated without retraining the model, the model can cite sources, and hallucination is reduced because the model can ground its answers in retrieved text. The main challenges are retrieval quality (if the wrong passages are retrieved, the model may produce wrong answers confidently) and latency (retrieval adds a step to inference).
- **Speculative decoding** accelerates autoregressive generation by using a small, fast **draft model** to propose multiple tokens in parallel, which are then verified by the large **target model** in a single forward pass.

- The algorithm works as follows: the draft model generates  $k$  candidate tokens autoregressively (this is fast because the draft model is small).
- The target model then scores all  $k$  tokens simultaneously in a single forward pass (this is efficient because the work is batched).
- For each candidate token  $t$  sampled from the draft distribution  $p_d(t)$ , it is accepted with probability  $\min(1, p_{\text{target}}(t)/p_d(t))$ . If rejected, a corrected token is resampled from the **adjusted distribution**  $p_{\text{adj}}(t) = \max(0, p_{\text{target}}(t) - p_d(t))$ , normalised.
- This acceptance-rejection scheme guarantees that the output distribution is identical to the target model alone.
- To see why, consider the effective probability of emitting token  $t$ . It can be accepted directly (probability  $p_d(t) \cdot \min(1, p_{\text{target}}(t)/p_d(t))$ ) or produced through resampling.
- For tokens where  $p_{\text{target}}(t) \leq p_d(t)$ , the direct acceptance contributes  $p_{\text{target}}(t)$ . For tokens where  $p_{\text{target}}(t) > p_d(t)$ , direct acceptance contributes  $p_d(t)$  and resampling contributes the remainder  $p_{\text{target}}(t) - p_d(t)$  (after accounting for the rejection probability).
- In both cases, the total probability of emitting  $t$  equals  $p_{\text{target}}(t)$ . The draft model affects only speed, not quality.



## Speculative Decoding

- The speedup depends on the acceptance rate: if the draft model is well-aligned with the target model, most tokens are accepted and the wall-clock time is



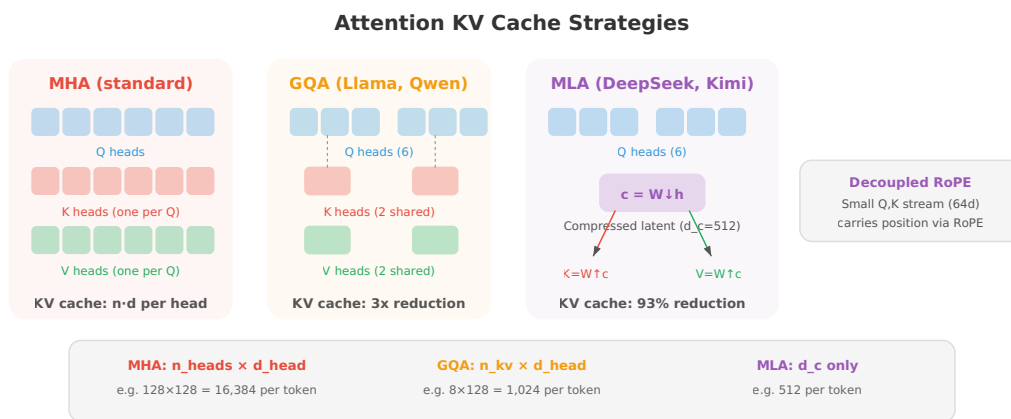
roughly that of the draft model. Typical speedups are 2-3x with no quality degradation.

- **Medusa** (Cai et al., 2024) takes a different approach: instead of a separate draft model, it adds multiple lightweight prediction heads to the target model itself. Each head predicts a different future token position simultaneously ( $k = 1, 2, 3, \dots$  steps ahead). At each step, Medusa proposes several candidate continuations using a tree structure, and a single forward pass through the target model's attention layers verifies which candidates are consistent. This avoids the need for a separate draft model entirely.
- **Parallel generation** methods more broadly aim to break the sequential bottleneck of autoregressive decoding. Jacobi decoding initialises all positions with guesses and iteratively refines them in parallel until convergence, treating generation as a fixed-point iteration. Non-autoregressive models (NAT) generate all tokens simultaneously in a single forward pass but typically suffer quality degradation and require techniques like iterative refinement, CTC loss, or knowledge distillation from autoregressive teachers to close the gap.
- The techniques above — alignment, long context, retrieval, efficient decoding, state space models — come together in modern production LLMs.
- The remainder of this file surveys the architectural innovations in frontier models, showing how theoretical ideas from files 01–04 and the methods above are combined in practice.
- **Grouped Query Attention (GQA)** is the most widely adopted attention efficiency technique. Standard multi-head attention (MHA) maintains separate key and value projections per head, requiring  $n_{\text{heads}} \times d_{\text{head}}$  values cached per token. GQA groups multiple query heads to share a single key-value head.
- With 64 query heads and 8 KV heads (a common configuration in Llama 3, Qwen, Gemma), each KV head is shared by 8 query heads, reducing the KV cache by 8x compared to MHA.
- The output quality is nearly identical to MHA because the queries can still attend to different patterns, they just share the same key-value subspace. Multi-query attention (MQA) is the extreme case with a single KV head for all queries, but GQA provides a better quality-efficiency trade-off.
- **Multi-head Latent Attention (MLA)**, introduced in DeepSeek-V2, achieves even more aggressive KV cache compression. Instead of caching the full key-value

projections (even with GQA), MLA down-projects the hidden state into a low-rank **latent vector**  $c_t \in \mathbb{R}^{d_c}$  with  $d_c \ll n_{\text{heads}} \times d_{\text{head}}$ :

$$c_t = W_{\text{down}} h_t$$

- Only this compressed vector is cached. At attention time, the full key and value representations are reconstructed via up-projection:  $k_t = W_{\text{up}}^K c_t$ ,  $v_t = W_{\text{up}}^V c_t$ . In DeepSeek-V3 (671B total parameters, 37B active), the compression dimension is  $d_c = 512$  versus  $128 \times 128 = 16,384$  for full MHA, a 93% reduction in KV cache.
- A subtlety: standard RoPE is position-dependent and incompatible with the shared compression, so MLA uses **decoupled RoPE**: a small separate stream of the query and key (64 dimensions per head) carries position information via RoPE, while the bulk of the representation flows through the compressed latent path.



### Attention KV Cache Strategies: MHA, GQA, and MLA compared

- Position encoding at scale** has diverged significantly from the original sinusoidal scheme. All frontier models use **RoPE** (file 04), but with key modifications for long context. The base frequency  $\theta_{\text{base}}$  in the original RoPE formula  $\theta_i = \theta_{\text{base}}^{-2i/d}$  is typically 10,000, which limits extrapolation beyond the training length.
- Adjusted Base Frequency (ABF)** simply increases  $\theta_{\text{base}}$  to 500,000 (Llama 3) or 1,000,000 (Qwen3, Gemma 3), stretching the rotation periods so the model encounters fewer full rotations during training and can extrapolate further.
- YaRN** (Yet another RoPE extensionN) applies frequency-dependent interpolation: low-frequency dimensions are interpolated (scaled down), high-frequency dimensions are extrapolated, and a temperature factor adjusts the attention

distribution. DeepSeek-V3, Qwen, and Kimi K2 all use YaRN-based extension to reach 128K context from models pre-trained at 4K–8K.

- **iRoPE** (interleaved RoPE), introduced in Llama 4, takes a more radical approach: every 4th attention layer uses **no positional encoding at all** (NoPE), while the other layers use standard RoPE with chunked attention.
- The NoPE layers can attend to all positions without any positional bias, while the RoPE layers provide local ordering. Combined with temperature scaling at inference, this enables Llama 4 Scout's 10M-token context window — orders of magnitude beyond any pure RoPE approach.
- **Mixture of Experts at scale** has become the dominant architecture for frontier models (file 04 introduced MoE fundamentals). The key design choices are the number of experts, routing sparsity, and load balancing.
- **Routing sparsity** varies significantly: DeepSeek-V3 uses 256 experts with top-8 routing (32x sparsity), Qwen3 uses 128 experts with top-8 (16x sparsity), Mixtral uses 8 experts with top-2 (4x sparsity), and Llama 4 Maverick uses 128 experts with top-1 plus a shared expert (128x sparsity).
- Higher sparsity means more total parameters for the same active compute, but requires more careful load balancing and communication infrastructure.
- **Auxiliary-loss-free load balancing** (DeepSeek-V3) replaces the traditional load balancing loss (file 04) which was found to degrade model quality. Instead, each expert maintains a dynamic bias term adjusted per training step: overloaded experts have their bias decreased (receiving fewer tokens), underloaded experts have their bias increased. This achieves balanced routing without any auxiliary loss polluting the main training signal.
- **Shared experts** appear in most MoE designs: one or more expert FFNs that process every token regardless of routing. These handle common patterns that all tokens need (basic syntax, function words), freeing the routed experts to specialise. Llama 4 uses 1 shared expert plus 1 routed expert per token (very sparse); DeepSeek-V3 uses 1 shared plus 8 routed.
- **Alternating dense and MoE layers** provide another design axis. Gemma 2 and 3 alternate local/global attention layers (5:1 ratio in Gemma 3, where local layers use a 1,024-token sliding window and only global layers cache the full 128K context).

- Llama 4 Maverick interleaves dense FFN layers with MoE layers. Kimi K2 uses hybrid-sparsity layers (one dense layer interspersed among expert layers). This heterogeneous design allows different layers to serve different functions.
- **Multi-token prediction (MTP)**, used in DeepSeek-V3, trains the model to predict not just the next token but also the token after that. At each position, a secondary prediction module (sharing the main model's embeddings) predicts one additional future token. The MTP loss is weighted at 0.1–0.3 relative to the main next-token loss. Beyond improving representation quality during training, the MTP heads can serve as draft heads for speculative decoding at inference time, providing a free speedup.
- **Knowledge distillation** is a training strategy where a large “teacher” model's outputs guide the training of a smaller “student” model. Gemma 2 and 3 use distillation extensively: the smaller models (2B, 4B) are trained on 50x the compute-optimal amount of data with the teacher's probability distributions as soft targets. This is why Gemma 3-4B matches Gemma 2-27B in quality.
- The distillation loss replaces or supplements the standard cross-entropy: the student minimises the KL divergence between its output distribution and the teacher's:

$$\mathcal{L}_{\text{distill}} = D_{\text{KL}}(p_{\text{teacher}}(\cdot \mid x) \parallel p_{\text{student}}(\cdot \mid x))$$

- DeepSeek-R1 distilled its 671B reasoning model into dense models as small as 1.5B using 800K curated chain-of-thought samples, producing small models with disproportionately strong reasoning.
- **Reasoning via reinforcement learning** represents the most significant recent advance in LLM capabilities. DeepSeek-R1 demonstrated that pure reinforcement learning on a base model (without supervised fine-tuning) can elicit chain-of-thought reasoning, self-verification, and error correction, behaviours that emerge spontaneously when the model is rewarded for correct final answers.
- DeepSeek-R1 uses **GRPO** (Group Relative Policy Optimisation), which eliminates the value network required by PPO. For each prompt, GRPO samples a group of G outputs, computes their rewards, and normalises advantages within the group:

$$A_i = \frac{r_i - \text{mean}(r_1, \dots, r_G)}{\text{std}(r_1, \dots, r_G)}$$

- The policy gradient then uses these group-relative advantages with a clipped objective (similar to PPO's clipping).
- Eliminating the critic network halves the memory and compute requirements of RL training, making it practical to train 671B-parameter models with RL.
- A critical design choice: DeepSeek-R1 uses **rule-based rewards** (checking mathematical answers against ground truth, running code test cases) rather than neural reward models, because neural reward models were found to be susceptible to reward hacking at this scale.
- **Qwen3's hybrid thinking mode** integrates reasoning (with <think> tags for step-by-step chain-of-thought) and fast direct response into a single model, allowing users to control a "thinking budget" that trades latency for reasoning depth.
- This is achieved by training on both thinking and non-thinking data, not through separate model checkpoints.
- **Training stabilisation at scale** requires new techniques beyond standard practices. **Logit soft-capping** (Gemma 2) passes attention scores through  $s \cdot \tanh(\text{logits}/s)$  with a soft cap  $s$  (typically 30–50) to prevent unbounded growth.
- **QK-Norm** (Qwen3) applies RMSNorm to query and key vectors before computing attention scores, replacing the need for QKV bias. **QK-Clip** (Kimi K2's MuonClip optimiser) monitors the maximum attention logit during training and rescales query-key weight matrices when they exceed a threshold, enabling stable pre-training of 1T-parameter models with zero instability events.
- **FP8 mixed-precision training** (DeepSeek-V3) uses 8-bit floating point for the compute-intensive matrix multiplications in the forward and backward passes while keeping master weights in higher precision.
- This roughly doubles throughput compared to BF16/FP16 training with negligible quality loss. DeepSeek-V3 trained its 671B-parameter model for only 2.8M H800 GPU-hours — a fraction of comparable models — largely due to this and other engineering optimisations.

## 35.1 Coding Tasks (use CoLab or notebook)

---

1. Implement a simple retrieval-augmented generation pipeline from scratch. Index a set of documents using TF-IDF (file 02), retrieve the most relevant passage for a query, and prepend it to a prompt.

```
import jax.numpy as jnp
import math
from collections import Counter

Knowledge base: a set of short passages
knowledge_base = [
 "The Eiffel Tower is a wrought-iron lattice tower in Paris, France. It was constructed from 1887 to 1889 as the centerpiece of the 1889 World's Fair.",
 "The Great Wall of China is a series of fortifications built along the northern borders of China. Construction began in the 7th century BC.",
 "Photosynthesis is the process by which plants convert sunlight, water, and carbon dioxide into glucose and oxygen using chlorophyll.",
 "The theory of general relativity, published by Albert Einstein in 1915, describes gravity as the curvature of spacetime caused by mass and energy.",
 "Python is a high-level programming language known for its simple syntax and readability. It was created by Guido van Rossum and released in 1991.",
 "The mitochondria are organelles found in eukaryotic cells. They generate most of the cell's supply of ATP, used as a source of chemical energy.",
]

Build TF-IDF index (reusing concepts from file 02)
def tokenise(text):
 return text.lower().split()

vocab = sorted(set(w for doc in knowledge_base for w in tokenise(doc)))
word2idx = {w: i for i, w in enumerate(vocab)}
V = len(vocab)
N = len(knowledge_base)

Document frequencies
doc_freq = Counter()
for doc in knowledge_base:
 for w in set(tokenise(doc)):
 doc_freq[w] += 1

def tfidf_vector(text):
 words = tokenise(text)
 counts = Counter(words)
 vec = jnp.zeros(V)
 for w, c in counts.items():
```

```

 if w in word2idx:
 tf = 1 + math.log(c)
 idf = math.log(N / (doc_freq.get(w, 0) + 1))
 vec = vec.at[word2idx[w]].set(tf * idf)
 return vec

Index all documents
doc_vectors = jnp.stack([tfidf_vector(doc) for doc in knowledge_base])

def cosine_sim(a, b):
 return jnp.dot(a, b) / (jnp.linalg.norm(a) * jnp.linalg.norm(b) + 1e-8)

def retrieve(query, top_k=2):
 """Retrieve top-k most relevant passages for a query."""
 q_vec = tfidf_vector(query)
 sims = jnp.array([cosine_sim(q_vec, doc_vectors[i]) for i in range(N)])
 top_indices = jnp.argsort(-sims)[:top_k]
 return [(int(i), float(sims[i]), knowledge_base[int(i)]) for i in
top_indices]

Test retrieval
queries = [
 "Who built the Eiffel Tower?",
 "How do plants make food?",
 "What did Einstein discover?",
]

for query in queries:
 results = retrieve(query, top_k=1)
 print(f"\nQuery: '{query}'")
 for idx, sim, passage in results:
 print(f" Retrieved (sim={sim:.3f}): '{passage[:80]}...'")

RAG-style prompt construction
context = results[0][2]
rag_prompt = f"Context: {context}\n\nQuestion: {query}\nAnswer:"
print(f" RAG prompt:\n {rag_prompt[:120]}...")

```

1. Implement speculative decoding with a toy draft and target model. Show that the accepted output matches the target model's distribution.

```

import jax
import jax.numpy as jnp

Simulate a draft model (fast, less accurate) and target model (slow, accurate)
vocab_size = 8

```

```

seq_len = 5

key = jax.random.PRNGKey(42)

Target model: returns logits given a sequence
def target_model(seq, key):
 """Simulated target model: produces token logits (expensive)."""
 # In practice this would be a large Transformer forward pass
 k1, k2 = jax.random.split(key)
 logits = jax.random.normal(k1, (len(seq), vocab_size)) * 2
 # Make it somewhat predictable: bias toward token (seq[-1] + 1) % vocab_size
 for i in range(len(seq)):
 logits = logits.at[i, (seq[i] + 1) % vocab_size].add(3.0)
 return logits

def draft_model(seq, key):
 """Simulated draft model: similar but noisier (cheap)."""
 k1, k2 = jax.random.split(key)
 logits = jax.random.normal(k1, (len(seq), vocab_size))
 for i in range(len(seq)):
 logits = logits.at[i, (seq[i] + 1) % vocab_size].add(2.0)
 return logits

def sample_token(logits, key):
 return jax.random.categorical(key, logits)

def speculative_decode(prefix, draft_steps=3, key=jax.random.PRNGKey(0)):
 """Speculative decoding: draft proposes, target verifies."""
 seq = list(prefix)
 total_accepted = 0
 total_proposed = 0

 for _ in range(4): # generate 4 rounds
 key, *subkeys = jax.random.split(key, draft_steps + 3)

 # Draft model proposes draft_steps tokens
 draft_tokens = []
 draft_probs = []
 draft_seq = list(seq)
 for i in range(draft_steps):
 d_logits = draft_model(jnp.array(draft_seq), subkeys[i])
 d_probs = jax.nn.softmax(d_logits[-1])
 tok = sample_token(d_logits[-1], subkeys[i])
 draft_tokens.append(int(tok))
 draft_probs.append(d_probs)
 draft_seq.append(int(tok))

```



```

Target model scores all draft tokens in one pass
target_logits = target_model(jnp.array(draft_seq), subkeys[draft_steps])
target_start = len(seq) - 1 # position of last prefix token

Accept/reject each draft token
accepted = 0
for i in range(draft_steps):
 t_probs = jax.nn.softmax(target_logits[target_start + i])
 d_prob = draft_probs[i][draft_tokens[i]]
 t_prob = t_probs[draft_tokens[i]]

 # Accept with probability min(1, target_prob / draft_prob)
 accept_prob = jnp.minimum(1.0, t_prob / (d_prob + 1e-10))
 key, accept_key = jax.random.split(key)
 if jax.random.uniform(accept_key) < accept_prob:
 seq.append(draft_tokens[i])
 accepted += 1
 else:
 # Reject: sample from adjusted distribution
 key, resample_key = jax.random.split(key)
 adjusted = jnp.maximum(0, t_probs - draft_probs[i])
 adjusted = adjusted / (adjusted.sum() + 1e-10)
 new_tok = jax.random.categorical(resample_key, jnp.log(adjusted
+ 1e-10))

 seq.append(int(new_tok))
 break

 total_accepted += accepted
 total_proposed += draft_steps

return seq, total_accepted, total_proposed

Run speculative decoding
prefix = [0, 1]
result_seq, accepted, proposed = speculative_decode(prefix)
acceptance_rate = accepted / proposed if proposed > 0 else 0

print(f"Prefix: {prefix}")
print(f"Generated sequence: {result_seq}")
print(f"Draft proposals: {proposed}")
print(f"Accepted: {accepted}")
print(f"Acceptance rate: {acceptance_rate:.1%}")
print(f"Speedup potential: {(accepted + proposed) / proposed:.2f}x")

```

1. Build a simple DPO training loop. Given pairs of preferred and dispreferred completions, update a small model using the DPO loss.

```

import jax
import jax.numpy as jnp

Tiny language model: linear projection from one-hot to logits
vocab_size = 10
seq_len = 4

key = jax.random.PRNGKey(42)
k1, k2 = jax.random.split(key)

Current policy parameters (trainable)
theta = jax.random.normal(k1, (vocab_size, vocab_size)) * 0.1
Reference policy parameters (frozen copy of initial theta)
theta_ref = theta.copy()

def log_prob_sequence(params, sequence):
 """Compute log P(sequence) under a simple autoregressive model."""
 total = 0.0
 for t in range(1, len(sequence)):
 # Simple: logits at position t depend on token at t-1
 logits = params[sequence[t-1]]
 log_probs = jax.nn.log_softmax(logits)
 total += log_probs[sequence[t]]
 return total

def dpo_loss(theta, theta_ref, preferred, dispreferred, beta=0.1):
 """Direct Preference Optimisation loss for one pair."""
 log_pi_w = log_prob_sequence(theta, preferred)
 log_pi_l = log_prob_sequence(theta, dispreferred)
 log_ref_w = log_prob_sequence(theta_ref, preferred)
 log_ref_l = log_prob_sequence(theta_ref, dispreferred)

 # DPO objective
 return -jax.nn.log_sigmoid(
 beta * ((log_pi_w - log_ref_w) - (log_pi_l - log_ref_l))
)

Preference dataset: (prompt_prefix, preferred_completion,
dispreferred_completion)
preferences = [
 (jnp.array([1, 3, 5, 7]), jnp.array([1, 3, 5, 2])), # prefer 7 over 2 at end
 (jnp.array([0, 2, 4, 6]), jnp.array([0, 2, 4, 9])), # prefer 6 over 9
 (jnp.array([3, 3, 3, 3]), jnp.array([3, 3, 3, 0])), # prefer repeating over
0
 (jnp.array([5, 6, 7, 8]), jnp.array([5, 6, 7, 1])), # prefer 8 over 1
]

```

```

grad_fn = jax.jit(jax.grad(dpo_loss))
lr = 0.05

print("Training DPO...")
for epoch in range(100):
 total_loss = 0.0
 for preferred, dispreferred in preferences:
 loss = dpo_loss(theta, theta_ref, preferred, dispreferred)
 grads = grad_fn(theta, theta_ref, preferred, dispreferred)
 theta = theta - lr * grads
 total_loss += loss
 if (epoch + 1) % 20 == 0:
 avg_loss = total_loss / len(preferences)
 print(f" Epoch {epoch+1}: avg DPO loss = {avg_loss:.4f}")

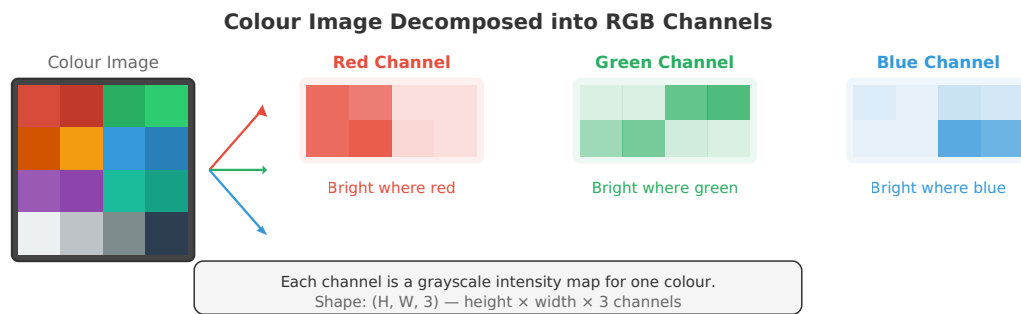
Check: the model should now prefer the preferred completions
print("\nPreference check after DPO training:")
for preferred, dispreferred in preferences:
 lp_w = log_prob_sequence(theta, preferred)
 lp_l = log_prob_sequence(theta, dispreferred)
 print(f" Preferred {list(preferred.astype(int))}: logP={lp_w:.3f} "
 f"Dispreferred {list(dispreferred.astype(int))}: logP={lp_l:.3f} "
 f"{'correct' if lp_w > lp_l else 'WRONG'}")

```

## 36 Image Fundamentals

Image fundamentals explain how digital images are represented, formed, and pre-processed before any model sees them. This file covers pixels, colour spaces (RGB, HSV, YCbCr, LAB), the pinhole camera model, convolution, edge detection (Sobel, Canny), histograms, and feature descriptors (SIFT, ORB), the low-level vision toolkit.

- A **digital image** is a 2D grid of numbers. Each cell in the grid is a **pixel** (picture element), and its value represents intensity or colour. A grayscale image is a single 2D matrix where each pixel holds a brightness value, typically from 0 (black) to 255 (white) for 8-bit images.
- A colour image extends this to three channels. In the **RGB** colour space, each pixel stores three values: red, green, and blue intensity.
- The coloured image is a 3D tensor (matrix) of shape (height, width, 3). Mixing these three channels at different intensities produces the full spectrum of visible colour.



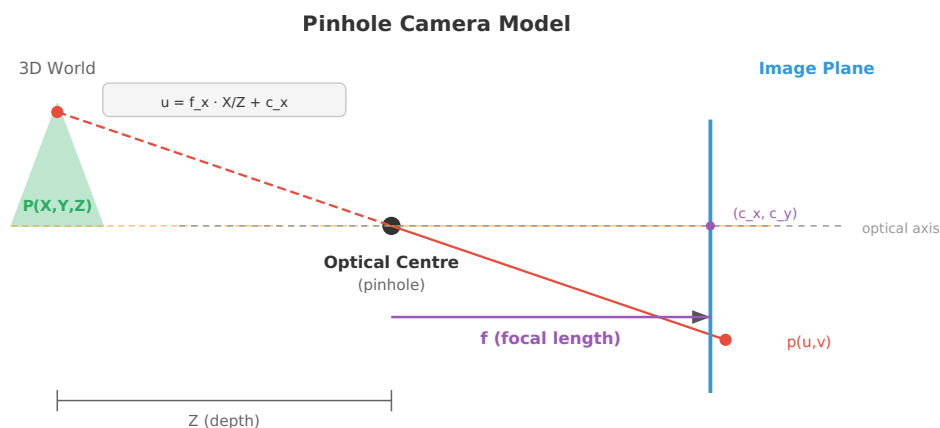
Colour image decomposed into its red, green, and blue channels, each shown as a grayscale intensity map

- **Bit depth** determines how many distinct intensity levels each channel can represent.
- An 8-bit image has  $2^8 = 256$  levels per channel, giving  $256^3 \approx 16.7$  million possible colours. A 16-bit image has 65,536 levels per channel, used in medical imaging and HDR photography where fine intensity distinctions matter.
- RGB is convenient for displays, but other colour spaces are better suited for different tasks.
- **HSV** (Hue, Saturation, Value) separates colour information from brightness. Hue is the pure colour (0-360 degrees around a colour wheel), saturation is how vivid the colour is (0 = grey, 1 = pure colour), and value is brightness. HSV is useful for colour-based segmentation because you can threshold on hue alone, regardless of lighting conditions. Detecting “red objects” is much easier in HSV than in RGB.
- **YCbCr** separates luminance (Y, perceived brightness) from chrominance (Cb, Cr, colour difference signals). This is the colour space used in JPEG compression and video codecs. Human vision is more sensitive to brightness than colour, so chrominance can be stored at lower resolution (chroma subsampling) with little perceptual loss.
- **LAB** (CIELAB) is designed so that numerical distance between two colours corresponds to perceptual difference. Equal steps in LAB space look like equal steps to a human observer. The L channel is lightness, A goes from green to red, and B goes from blue to yellow. LAB is used when you need perceptually uniform colour comparisons.
- **Image formation** describes how a 3D scene becomes a 2D image. The simplest model is the **pinhole camera**: light from the scene passes through a tiny hole

and projects onto a sensor plane behind it. A point  $(X, Y, Z)$  in world coordinates projects to pixel coordinates  $(u, v)$ :

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \frac{1}{Z} \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}$$

- The  $3 \times 3$  matrix is the **intrinsic matrix**  $K$ . It encodes the camera's internal properties: focal lengths  $f_x, f_y$  (how strongly the lens converges light) and the principal point  $(c_x, c_y)$  (where the optical axis meets the sensor, usually near the image centre). These are fixed for a given camera and lens combination.



Pinhole camera model: 3D point projects through the optical centre onto the image plane, with focal length and principal point labelled

- The **extrinsic parameters** describe where the camera is in the world: a rotation matrix  $R$  ( $3 \times 3$ , from chapter 02) and a translation vector  $t$  ( $3 \times 1$ ). Together, they transform world coordinates to camera coordinates. The full projection is:

$$\mathbf{p} = K[R \mid t]\mathbf{P}$$

- where  $\mathbf{P} = [X, Y, Z, 1]^T$  is the 3D point in homogeneous coordinates and  $\mathbf{p} = [u, v, 1]^T$  is the projected pixel. The  $[R \mid t]$  matrix is  $3 \times 4$ , stacking the rotation and translation side by side. This is all linear algebra from chapter 02.
- Real lenses introduce **distortion**.
  - **Radial distortion** bends straight lines into curves (barrel distortion makes the image bulge outward; pincushion distortion squeezes it inward).
  - Tangential distortion** arises when the lens is not perfectly parallel to the sensor.

- Camera calibration estimates both intrinsic parameters and distortion coefficients from images of a known pattern (like a checkerboard), then corrects (undistorts) images.
- **Spatial filtering** is the foundation of classical image processing. A **filter** (or kernel) is a small matrix (typically 3x3 or 5x5) that slides over the image. At each position, the filter values are multiplied element-wise with the overlapping image patch and summed to produce one output pixel. This is a **2D convolution**, the same operation that powers CNNs (file 02), but here the filter weights are hand-designed rather than learned.

$$\begin{aligned}
 & (\text{image} * K)[i, j] = \\
 & \sum_m \sum_n \\
 & \quad \text{image}[i + m, j + n] \cdot K[m, n]
 \end{aligned}$$

- This is a 2D extension of the 1D convolution from chapter 06. The filter determines what the operation detects: different filters detect different features.
- **Blurring** smooths an image by averaging neighbouring pixels. A **box filter** gives equal weight to all neighbours.
- A **Gaussian filter** weights neighbours by a 2D Gaussian (chapter 05), giving more weight to nearby pixels and less to distant ones. Gaussian blur is the most common smoothing operation and is parametrised by  $\sigma$ : larger  $\sigma$  means more smoothing.
- **Median filtering** replaces each pixel with the median of its neighbourhood instead of a weighted average. It is particularly effective at removing salt-and-pepper noise (random black and white pixels) while preserving edges, because the median is robust to outliers (as discussed in chapter 04).
- **Edge detection** identifies boundaries where pixel intensity changes sharply. Edges carry most of the structural information in an image; you can recognise objects from their edges alone.
- The **Sobel operator** uses two 3x3 filters to estimate the gradient in the horizontal and vertical directions:

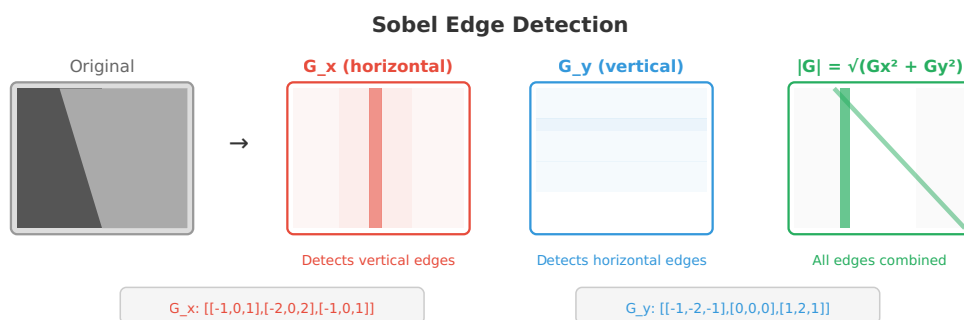
```
G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad \text{quad} \\ G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}
```

- Convoluting the image with  $G_x$  gives the horizontal gradient (strong response at vertical edges), and  $G_y$  gives the vertical gradient (strong response at horizontal edges).

- The gradient magnitude

$$\sqrt{G_x^2 + G_y^2}$$

and direction  $\arctan(G_y/G_x)$  together describe the edge strength and orientation at each pixel. This is the image-domain analogue of the gradient from chapter 03.



Original image, Sobel horizontal gradient, Sobel vertical gradient, and combined edge magnitude

- The **Canny edge detector** is the gold standard for edge detection. It applies four steps:
  1. Smooth the image with a Gaussian filter to reduce noise
  2. Compute gradient magnitude and direction (using Sobel)
  3. **Non-maximum suppression**: thin edges by keeping only pixels that are local maxima along the gradient direction
  4. **Hysteresis thresholding**: use two thresholds (high and low). Pixels above the high threshold are definite edges. Pixels between the thresholds are edges only if connected to a definite edge. Pixels below the low threshold are discarded.
- The two thresholds in Canny make it more robust than a single threshold: strong edges are always kept, and weak edges are kept only if they are part of a continuous edge structure.

- **Frequency domain** analysis reveals patterns that are hard to see in the spatial domain. The **2D Fourier transform** (extending the 1D version from chapter 03) decomposes an image into a sum of 2D sinusoidal patterns at different frequencies and orientations:

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) \cdot e^{-j2\pi(ux/M + vy/N)}$$

- Low frequencies correspond to smooth, slowly-varying regions (the sky, a wall). High frequencies correspond to sharp transitions (edges, textures, noise). The **magnitude spectrum** shows how much energy exists at each frequency, and the **phase spectrum** encodes the spatial arrangement.
- **Low-pass filtering** removes high frequencies, which smooths the image (equivalent to Gaussian blur in the spatial domain). **High-pass filtering** removes low frequencies, emphasising edges and fine detail. **Band-pass filtering** keeps only a range of frequencies, useful for texture analysis.
- In practice, filtering in the frequency domain can be faster than spatial convolution for large filters, because convolution in the spatial domain is equivalent to element-wise multiplication in the frequency domain (the **convolution theorem**). This connects directly to the Fourier transform properties from chapter 03.
- **Histograms** summarise the distribution of pixel intensities. A histogram counts how many pixels have each intensity value (0-255 for 8-bit images). It is the same frequency distribution from chapter 04 applied to pixel values.



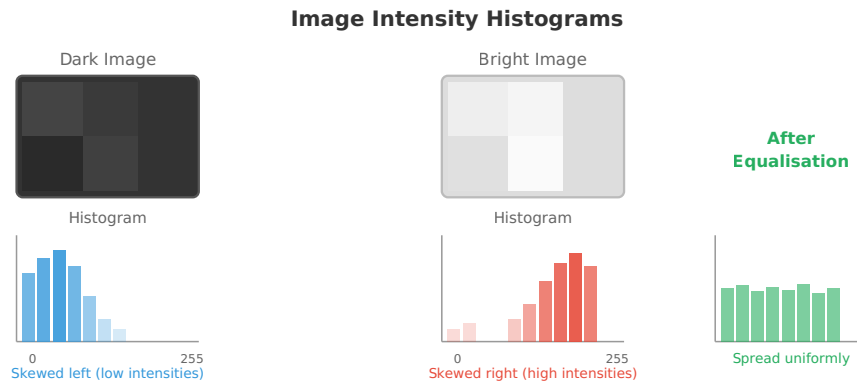


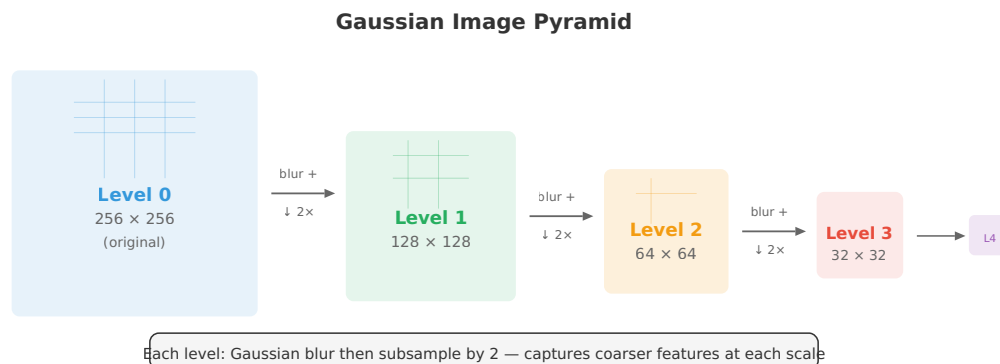
Image with its intensity histogram: dark image has histogram skewed left, bright image has histogram skewed right

- A dark image has its histogram concentrated on the left (low values). A bright image has it concentrated on the right. A low-contrast image has a narrow histogram. A high-contrast image has a wide, spread-out histogram.
- **Histogram equalisation** stretches the histogram to span the full intensity range, improving contrast. The idea is to find a mapping that makes the cumulative distribution function (CDF) of pixel intensities approximately linear. This is a direct application of the CDF concept from chapter 04.
- **Otsu's method** automatically finds the best threshold to separate an image into foreground and background. It tries every possible threshold and picks the one that minimises the within-class variance (or equivalently, maximises the between-class variance). This is the same variance concept from chapter 04, applied to pixel intensity populations.
- **Feature extraction** identifies distinctive points or regions in an image that can be used for matching, recognition, and 3D reconstruction. Good features should be repeatable (found again in a different view), distinctive (distinguishable from other features), and efficient to compute.
- **Corner detection** finds points where the image intensity changes significantly in multiple directions. A smooth region has little change in any direction. An edge has change in one direction. A corner has change in at least two directions, making it locally unique and therefore a reliable landmark.
- The **Harris corner detector** analyses the **structure tensor** (also called the second-moment matrix) at each pixel:

$$M = \sum_{(x,y) \in W} w(x,y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

- where  $I_x$  and  $I_y$  are the image gradients (computed with Sobel),  $W$  is a local window, and  $w$  is a Gaussian weighting function. The eigenvalues of  $M$  (from chapter 02) tell you the type of feature:
  - Both eigenvalues small: flat region (no feature)
  - One large, one small: edge
  - Both large: corner
- Instead of computing eigenvalues explicitly, Harris uses a corner response function:  $R = \det(M) - k \cdot (\text{trace}(M))^2$ , where  $\det(M) = \lambda_1 \lambda_2$  and  $\text{trace}(M) = \lambda_1 + \lambda_2$  (both from chapter 02). Large positive  $R$  indicates a corner. The constant  $k$  is typically 0.04-0.06.
- The **Shi-Tomasi** detector simplifies this to  $R = \min(\lambda_1, \lambda_2)$ , directly checking that the smaller eigenvalue is large enough. This is slightly more stable in practice.
- **Blob detection** finds regions that differ from their surroundings. Unlike corners (which are point features), blobs have a characteristic size.
- **SIFT** (Scale-Invariant Feature Transform, Lowe, 2004) detects blobs at multiple scales and constructs a descriptor that is invariant to rotation, scale, and partially invariant to illumination changes. It works by:
  1. Building a **scale space** (see below) using Gaussian blur at increasing  $\sigma$
  2. Finding extrema in the Difference of Gaussians (DoG) across scales
  3. Refining keypoint locations and removing low-contrast points and edge responses
  4. Assigning a dominant orientation based on local gradient directions
  5. Building a 128-dimensional descriptor from gradient histograms in a 16x16 patch around the keypoint
- **SURF** (Speeded-Up Robust Features) approximates SIFT using box filters and integral images for faster computation. **ORB** (Oriented FAST and Rotated BRIEF) is a fast, open-source alternative that combines the FAST corner detector with the BRIEF binary descriptor, adding rotation invariance.

- **HOG** (Histogram of Oriented Gradients) descriptors divide the image into small cells, compute a histogram of gradient directions within each cell, and normalise across blocks of cells. HOG captures the distribution of edge orientations, which is highly informative for object shape. Before deep learning, HOG + SVM (chapter 06) was the dominant approach for pedestrian detection and object recognition.
- **Image pyramids** represent an image at multiple resolutions.
  - A **Gaussian pyramid** is built by repeatedly blurring and downsampling (halving the resolution). Each level is a coarser version of the original.
  - A **Laplacian pyramid** stores the difference between consecutive Gaussian levels, capturing the detail lost at each downsampling step. The Laplacian pyramid is invertible: you can reconstruct the original image from it.



Gaussian pyramid: original image at full resolution, then progressively smaller versions at half resolution each level

- **Scale space** formalises the idea that objects exist at different scales. A tree is a large blob; a leaf on that tree is a small blob. To detect both, you need to search across scales. The scale space of an image is the family of images produced by convolving with Gaussians at increasing  $\sigma$ :

$$L(x, y, \sigma) = G(x, y, \sigma) * I(x, y)$$

- where  $G$  is a 2D Gaussian with standard deviation  $\sigma$ . Features that persist across multiple scales are more likely to be meaningful structures rather than noise. Scale space is the theoretical foundation of SIFT and of the multi-scale processing used throughout modern computer vision, including the feature pyramid networks in object detection (file 03).

## 36.1 Coding Tasks (use CoLab or notebook)

1. Load an image, convert it to different colour spaces (RGB, HSV, LAB), and visualise the individual channels. Observe how colour information is distributed differently across spaces.

```
import jax.numpy as jnp
import matplotlib.pyplot as plt
from PIL import Image
import numpy as np

Create a synthetic test image with distinct colours
H, W = 128, 256
img = np.zeros((H, W, 3), dtype=np.uint8)
img[:, :64] = [255, 50, 50] # red
img[:, 64:128] = [50, 255, 50] # green
img[:, 128:192] = [50, 50, 255] # blue
img[:, 192:] = [255, 255, 50] # yellow

Add a brightness gradient
for y in range(H):
 scale = 0.3 + 0.7 * y / H
 img[y] = (img[y] * scale).astype(np.uint8)

img_jnp = jnp.array(img, dtype=jnp.float32) / 255.0

Manual RGB to HSV conversion
def rgb_to_hsv(rgb):
 r, g, b = rgb[..., 0], rgb[..., 1], rgb[..., 2]
 maxc = jnp.max(rgb, axis=-1)
 minc = jnp.min(rgb, axis=-1)
 diff = maxc - minc + 1e-7

 # Hue
 h = jnp.where(maxc == minc, 0.0,
 jnp.where(maxc == r, 60 * ((g - b) / diff % 6),
 jnp.where(maxc == g, 60 * ((b - r) / diff + 2),
 60 * ((r - g) / diff + 4))))
 s = jnp.where(maxc < 1e-7, 0.0, diff / maxc)
 v = maxc
 return jnp.stack([h / 360, s, v], axis=-1)

hsv = rgb_to_hsv(img_jnp)

fig, axes = plt.subplots(2, 3, figsize=(14, 8))
for i, (ch, name) in enumerate(zip([img_jnp[...,0], img_jnp[...,1], img_jnp[...,
```

```

2]],

 ['Red', 'Green', 'Blue'])):
 axes[0, i].imshow(ch, cmap='gray', vmin=0, vmax=1)
 axes[0, i].set_title(f'RGB: {name}'); axes[0, i].axis('off')

 for i, (ch, name) in enumerate(zip([hsv[...,0], hsv[...,1], hsv[...,2]],
 ['Hue', 'Saturation', 'Value'])):
 axes[1, i].imshow(ch, cmap='gray', vmin=0, vmax=1)
 axes[1, i].set_title(f'HSV: {name}'); axes[1, i].axis('off')

plt.suptitle('RGB vs HSV Channels')
plt.tight_layout(); plt.show()

```

1. Implement Sobel edge detection and Gaussian blur from scratch using 2D convolution. Apply them to an image and compare the results.

```

import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

def conv2d(image, kernel):
 """2D convolution (valid mode) from scratch."""
 H, W = image.shape
 kH, kW = kernel.shape
 out_h, out_w = H - kH + 1, W - kW + 1
 output = jnp.zeros((out_h, out_w))
 for i in range(out_h):
 for j in range(out_w):
 patch = image[i:i+kH, j:j+kW]
 output = output.at[i, j].set(jnp.sum(patch * kernel))
 return output

Create a test image: white rectangle on dark background
img = jnp.zeros((64, 64))
img = img.at[15:50, 20:45].set(1.0)
Add some noise
key = jax.random.PRNGKey(42)
img = img + jax.random.normal(key, img.shape) * 0.05

Sobel filters
sobel_x = jnp.array([[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]], dtype=jnp.float32)
sobel_y = jnp.array([[-1, -2, -1], [0, 0, 0], [1, 2, 1]], dtype=jnp.float32)

Gaussian blur kernel (5x5, sigma=1)
ax = jnp.arange(-2, 3, dtype=jnp.float32)
xx, yy = jnp.meshgrid(ax, ax)

```

```

gaussian = jnp.exp(-(xx**2 + yy**2) / (2 * 1.0**2))
gaussian = gaussian / gaussian.sum()

Apply filters
gx = conv2d(img, sobel_x)
gy = conv2d(img, sobel_y)
edges = jnp.sqrt(gx**2 + gy**2)
blurred = conv2d(img, gaussian)

fig, axes = plt.subplots(1, 4, figsize=(16, 4))
for ax, data, title in zip(axes,
 [img, edges, blurred, gx],
 ['Original', 'Edge Magnitude', 'Gaussian Blur', 'Horizontal Gradient']):
 ax.imshow(data, cmap='gray')
 ax.set_title(title); ax.axis('off')
plt.tight_layout(); plt.show()

```

1. Implement histogram equalisation from scratch and apply it to a low-contrast grayscale image. Compare histograms before and after.

```

import jax.numpy as jnp
import matplotlib.pyplot as plt

Create a low-contrast image (values clustered in a narrow range)
key = __import__('jax').random.PRNGKey(42)
img = __import__('jax').random.uniform(key, (128, 128)) * 0.3 + 0.3 # values in
[0.3, 0.6]

def histogram_equalise(img, n_bins=256):
 """Histogram equalisation for a grayscale image."""
 # Quantise to bins
 bins = jnp.linspace(0, 1, n_bins + 1)
 hist = jnp.histogram(img, bins=bins)[0]

 # Compute CDF
 cdf = jnp.cumsum(hist)
 cdf_normalised = (cdf - cdf.min()) / (cdf.max() - cdf.min())

 # Map each pixel through the CDF
 indices = jnp.clip((img * n_bins).astype(jnp.int32), 0, n_bins - 1)
 equalised = cdf_normalised[indices]
 return equalised

eq_img = histogram_equalise(img)

fig, axes = plt.subplots(2, 2, figsize=(12, 10))

```

```

axes[0, 0].imshow(img, cmap='gray', vmin=0, vmax=1)
axes[0, 0].set_title('Original (Low Contrast)'); axes[0, 0].axis('off')
axes[0, 1].imshow(eq_img, cmap='gray', vmin=0, vmax=1)
axes[0, 1].set_title('After Histogram Equalisation'); axes[0, 1].axis('off')

axes[1, 0].hist(img.ravel(), bins=64, color='#3498db', alpha=0.8)
axes[1, 0].set_title('Histogram Before'); axes[1, 0].set_xlim(0, 1)
axes[1, 1].hist(eq_img.ravel(), bins=64, color='#e74c3c', alpha=0.8)
axes[1, 1].set_title('Histogram After'); axes[1, 1].set_xlim(0, 1)

plt.tight_layout(); plt.show()

```

1. Implement the Harris corner detector from scratch. Detect corners in a simple image and visualise them.

```

import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

def harris_corners(img, k=0.05, threshold=0.01):
 """Harris corner detection from scratch."""
 # Compute gradients with Sobel
 sobel_x = jnp.array([[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]], dtype=jnp.float32)
 sobel_y = jnp.array([[-1, -2, -1], [0, 0, 0], [1, 2, 1]], dtype=jnp.float32)

 # Pad image for valid convolution to preserve size
 img_pad = jnp.pad(img, 1, mode='edge')
 H, W = img.shape

 Ix = jnp.zeros_like(img)
 Iy = jnp.zeros_like(img)
 for i in range(H):
 for j in range(W):
 patch = img_pad[i:i+3, j:j+3]
 Ix = Ix.at[i, j].set(jnp.sum(patch * sobel_x))
 Iy = Iy.at[i, j].set(jnp.sum(patch * sobel_y))

 # Structure tensor components
 Ixx = Ix * Ix
 Iyy = Iy * Iy
 Ixy = Ix * Iy

 # Gaussian smoothing of structure tensor (approximate with window sum)
 w = 3 # window half-size
 R = jnp.zeros_like(img)
 pad_xx = jnp.pad(Ixx, w, mode='constant')

```

```

pad_yy = jnp.pad(Iyy, w, mode='constant')
pad_xy = jnp.pad(Ixy, w, mode='constant')

for i in range(H):
 for j in range(W):
 sxx = jnp.sum(pad_xx[i:i+2*w+1, j:j+2*w+1])
 syy = jnp.sum(pad_yy[i:i+2*w+1, j:j+2*w+1])
 sxy = jnp.sum(pad_xy[i:i+2*w+1, j:j+2*w+1])
 det = sxx * syy - sxy * sxy
 trace = sxx + syy
 R = R.at[i, j].set(det - k * trace * trace)

Threshold
corners = R > threshold * R.max()
return R, corners

Test image: checkerboard pattern (lots of corners)
block = 16
n = 4
checker = jnp.zeros((block * n, block * n))
for i in range(n):
 for j in range(n):
 if (i + j) % 2 == 0:
 checker = checker.at[i*block:(i+1)*block, j*block:
(j+1)*block].set(1.0)

R, corners = harris_corners(checker)
cy, cx = jnp.where(corners)

fig, axes = plt.subplots(1, 3, figsize=(14, 4))
axes[0].imshow(checker, cmap='gray')
axes[0].set_title('Checkerboard'); axes[0].axis('off')
axes[1].imshow(R, cmap='hot')
axes[1].set_title('Harris Response'); axes[1].axis('off')
axes[2].imshow(checker, cmap='gray')
axes[2].scatter(cx, cy, c='#e74c3c', s=15, marker='x')
axes[2].set_title(f'Detected Corners ({len(cx)})'); axes[2].axis('off')
plt.tight_layout(); plt.show()

```

## 37 Convolutional Networks

Convolutional neural networks learn spatial feature hierarchies directly from pixel data, replacing hand-designed filters with gradient-optimised ones. This file covers convolution mechanics, pooling, stride, dilation, receptive fields, and landmark



architectures (LeNet, AlexNet, VGG, ResNet, Inception, EfficientNet) that defined image classification.

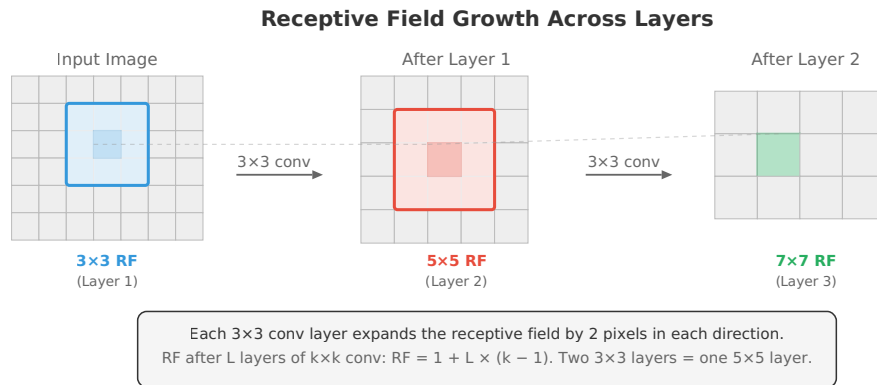
- In file 01, we hand-designed filters for edge detection, blurring, and corner detection. The natural question is: can we learn the optimal filters from data? That is exactly what convolutional neural networks (CNNs) do.
- Instead of choosing filter weights by hand, CNNs learn them via gradient descent (chapter 06), discovering features that are directly useful for the task at hand.
- In chapter 06, we introduced the convolution operation, CNN basics, and the idea of filter learning. Here we go deeper into the architectural innovations that made CNNs the dominant paradigm in computer vision for over a decade.
- Recall the core **convolution operation**: a filter  $K$  of size  $k \times k$  slides over the input feature map, computing a dot product at each position (chapter 06). The output size is controlled by three hyperparameters:
  - **Stride**: how many pixels the filter moves between positions. Stride 1 means the filter shifts one pixel at a time. Stride 2 means it shifts two pixels, halving the spatial dimensions. Strided convolution is an alternative to pooling for downsampling.
  - **Padding**: adding zeros around the input border. “Same” padding ( $p = \lceil k/2 \rceil$ ) preserves spatial dimensions. “Valid” padding ( $p = 0$ ) reduces them.
  - **Dilation**: inserting gaps between filter elements. A  $3 \times 3$  filter with dilation 2 covers a  $5 \times 5$  receptive field using only 9 parameters. Dilated convolutions expand the receptive field without increasing computation.

- The output spatial size after convolution:

$$\text{out} = \lfloor \frac{\text{in} - k + 2p}{s} \rfloor + 1$$

- where  $\text{in}$  is the input size,  $k$  is the kernel size,  $p$  is padding, and  $s$  is stride. This formula applies independently to height and width.
- The **receptive field** of a neuron is the region of the original input that can influence its value.
  - Early layers have small receptive fields (they see local patterns like edges).

- Deeper layers have larger receptive fields (they see larger structures like object parts).
- The receptive field grows with each layer: roughly by  $k - 1$  pixels per convolutional layer (more with stride or dilation).



Receptive field growing across layers: layer 1 neurons see a 3x3 patch, layer 2 neurons see a 5x5 patch, layer 3 neurons see a 7x7 patch of the original input

- **Pooling** layers reduce spatial dimensions while retaining the most important information.
  - **Max pooling** takes the maximum value in each window, preserving the strongest activation (the most prominent feature).
  - **Average pooling** takes the mean, smoothing the feature map. A 2x2 pool with stride 2 halves both spatial dimensions.
- **Global Average Pooling (GAP)** averages the entire spatial extent of each channel into a single number, producing a vector of length equal to the number of channels. GAP replaces the fully connected layers at the end of many modern architectures, drastically reducing parameter count and acting as a structural regulariser.
- **Batch Normalisation (BatchNorm)** normalises activations within each mini-batch to have zero mean and unit variance, then applies a learnable scale and shift (chapter 06). In CNNs, BatchNorm is applied per-channel: statistics are computed across the batch and spatial dimensions for each channel independently. It stabilises training, allows higher learning rates, and acts as a mild regulariser.
- **Dropout** (chapter 06) randomly zeroes neurons during training.

- In CNNs, **spatial dropout** (Dropout2D) drops entire feature map channels rather than individual pixels, which is more effective because neighbouring pixels in a feature map are highly correlated.
- **Data augmentation** artificially expands the training set by applying random transformations to each image during training: horizontal flips, random crops, rotations, colour jitter (adjusting brightness, contrast, saturation, hue), and cutout (masking random rectangular patches). The network sees each image in many different forms, forcing it to learn transformation-invariant features rather than memorising specific pixel patterns.
- Advanced augmentation strategies include **Mixup** (blending two images and their labels):
 

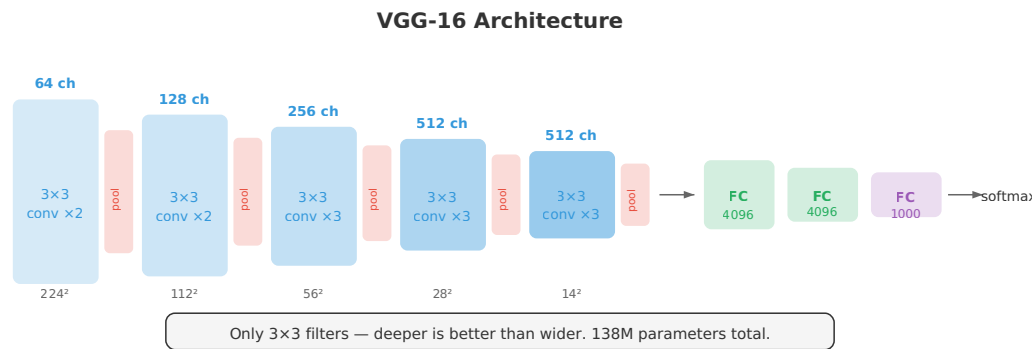


$$z = \lambda x_i + (1 - \lambda) x_j,$$


$$y = \lambda y_i + (1 - \lambda) y_j,$$

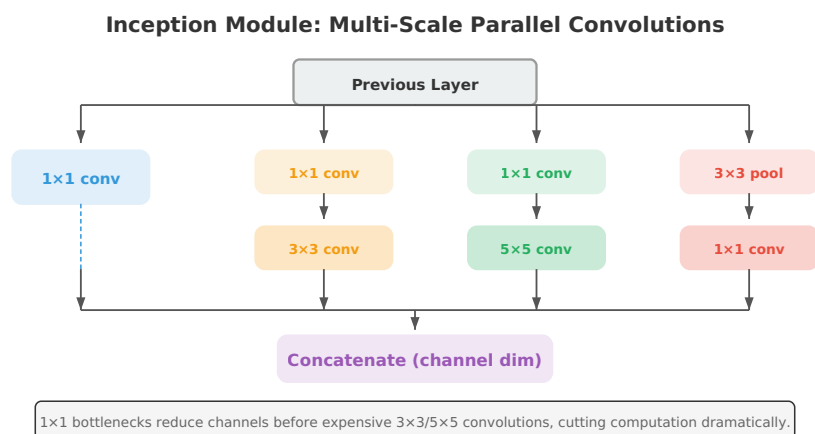
**CutMix** (pasting a rectangular patch from one image onto another and mixing labels proportionally to area), and **RandAugment** (randomly sampling a sequence of augmentations from a fixed set with a single strength parameter).
- The history of CNN architectures is a story of progressively deeper, more efficient designs, each solving a problem that limited its predecessor.
- **LeNet-5** (LeCun et al., 1998) was the original CNN, designed for handwritten digit recognition. Two convolutional layers followed by three fully connected layers, with average pooling and tanh activations. It proved that learned filters outperform hand-designed features, but it was tiny by modern standards (60K parameters).
- **AlexNet** (Krizhevsky et al., 2012) won the ImageNet competition by a massive margin, igniting the deep learning revolution. Key innovations: ReLU activation (instead of tanh, which suffers from vanishing gradients), dropout for regularisation, data augmentation, and training on GPUs. Five convolutional layers, three fully connected layers, 60 million parameters.
- **VGG** (Simonyan and Zisserman, 2014) showed that using only 3x3 filters stacked deeply works better than larger filters. Two stacked 3x3 filters have the same

receptive field as one 5x5 filter but fewer parameters ( $2 \times 3^2 = 18$  vs  $5^2 = 25$ ) and an extra nonlinearity. VGG-16 (16 layers) and VGG-19 (19 layers) are still widely used as feature extractors. The architecture is remarkably simple: convolution blocks with increasing channels (64, 128, 256, 512), each followed by max pooling.



VGG architecture: stacked  $3 \times 3$  conv blocks with increasing channel depth (64→128→256→512), max pooling between blocks, ending with fully connected layers

- **GoogLeNet/Inception** (Szegedy et al., 2014) introduced the **Inception module**: instead of choosing a single filter size, use  $1 \times 1$ ,  $3 \times 3$ , and  $5 \times 5$  convolutions in parallel, concatenate their outputs, and let the network decide which scale is most useful.  $1 \times 1$  convolutions are used as bottlenecks before the larger filters to reduce computation. GoogLeNet achieved better accuracy than VGG with 12x fewer parameters (6.8M vs 138M).



Inception module: four parallel branches ( $1 \times 1$ ,  $3 \times 3$ ,  $5 \times 5$ , and pooling) with  $1 \times 1$  bottlenecks, concatenated along the channel dimension

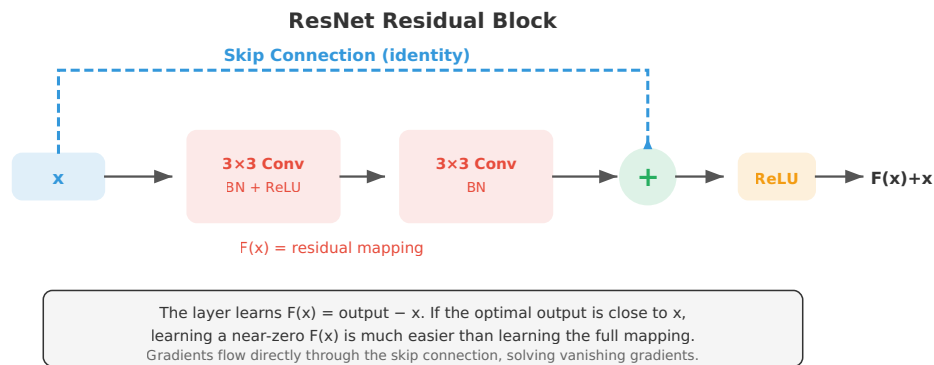
- The Inception module captures features at multiple scales simultaneously. A  $1 \times 1$  filter captures point-wise patterns, a  $3 \times 3$  captures local texture, and a  $5 \times 5$

captures larger structures. The concatenation combines all perspectives into a rich representation.

- **ResNet** (He et al., 2016) solved the **degradation problem**: deeper networks performed worse than shallower ones, not because of overfitting, but because they were harder to optimise. The solution is the **skip connection** (residual connection):

$$\text{output} = F(x) + x$$

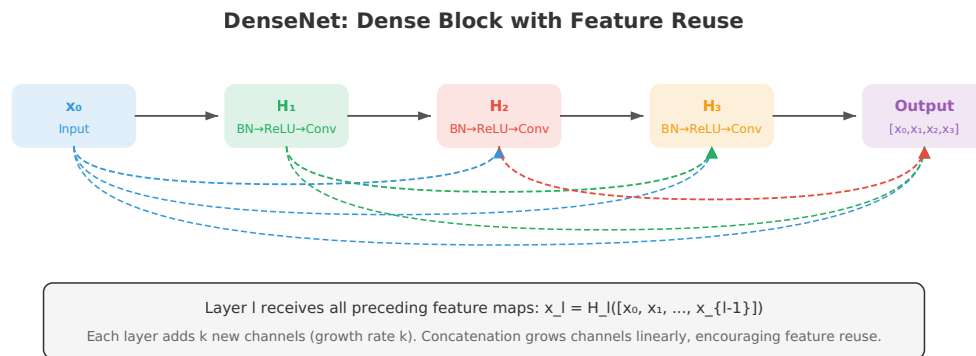
- The layer learns the residual  $F(x) = \text{output} - x$ . If the optimal transformation is close to identity (which is common in deep networks), learning a near-zero residual is much easier than learning the full mapping. Skip connections also provide a direct gradient highway, reducing vanishing gradients. ResNet trained networks with 152 layers, far deeper than anything before.



ResNet block: input  $x$  passes through two conv layers to produce  $F(x)$ , then the skip connection adds  $x$  back, giving output  $F(x) + x$

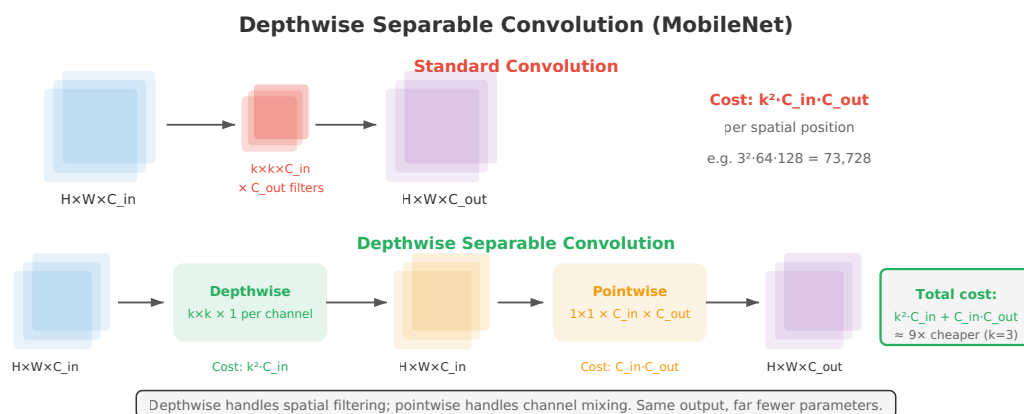
- When the input and output dimensions differ (due to stride or channel change), a **projection shortcut** applies a  $1 \times 1$  convolution to  $x$  to match dimensions:  
 $\text{output} = F(x) + W_s x$ .
- The **bottleneck block** (used in ResNet-50 and deeper) uses three convolutions:  $1 \times 1$  to reduce channels,  $3 \times 3$  for spatial processing, and  $1 \times 1$  to expand channels back. This is cheaper than two  $3 \times 3$  convolutions and allows much deeper networks.
- **DenseNet** (Huang et al., 2017) takes the skip connection idea further: every layer is connected to every subsequent layer within a dense block. Layer  $l$  receives the feature maps from all preceding layers as input:  
 $x_l = H_l([x_0, x_1, \dots, x_{l-1}])$ , where  $[\cdot]$  denotes concatenation along the channel

dimension. This encourages feature reuse, strengthens gradient flow, and reduces the total number of parameters.



DenseNet dense block: every layer receives feature maps from all preceding layers via concatenation, creating dense connectivity for maximum feature reuse

- **Efficient architectures** target deployment on mobile devices and edge hardware, where compute, memory, and energy are constrained.
- **MobileNet** (Howard et al., 2017) replaces standard convolutions with **depthwise separable convolutions**, which factorise the operation into two steps:
  1. **Depthwise convolution**: apply a single  $k \times k$  filter per input channel (no cross-channel interaction)
  2. **Pointwise convolution**: apply  $1 \times 1$  convolutions to combine information across channels
- A standard  $k \times k$  convolution with  $C_{in}$  input channels and  $C_{out}$  output channels costs  $k^2 \cdot C_{in} \cdot C_{out}$  multiplications per spatial position. Depthwise separable convolution costs  $k^2 \cdot C_{in} + C_{in} \cdot C_{out}$ , a reduction of roughly  $k^2$  times. For a  $3 \times 3$  filter, this is approximately 9x cheaper.

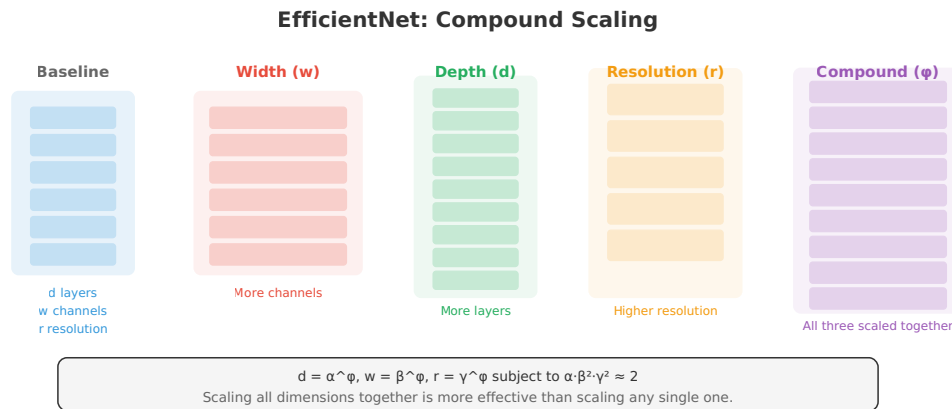


Depthwise separable convolution: depthwise step applies one  $k \times k$  filter per channel, then pointwise  $1 \times 1$  convolutions mix channels — same output shape,  $\sim 9 \times$  fewer operations

- **MobileNet-V2** introduced the **inverted residual block**: expand channels with a  $1 \times 1$  convolution, apply depthwise convolution in the expanded space, then project back down with a  $1 \times 1$  convolution. The skip connection is placed on the narrow (bottleneck) layers, inverting the ResNet pattern. The expansion ratio is typically 6.
- **EfficientNet** (Tan and Le, 2019) introduced **compound scaling**: instead of scaling only depth, only width, or only resolution independently, scale all three dimensions together using a fixed ratio. Given a scaling coefficient  $\phi$ :

$$\text{depth} : d = \alpha^\phi, \text{ width} : w = \beta^\phi, \text{ resolution} : r = \gamma^\phi$$

- subject to  $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$  (so that total computation roughly doubles per unit increase in  $\phi$ ). A grid search finds  $\alpha = 1.2$ ,  $\beta = 1.1$ ,  $\gamma = 1.15$  as the baseline ratios. EfficientNet-B0 through B7 scale up progressively, achieving state-of-the-art accuracy with far fewer parameters and FLOPs than previous models.



EfficientNet compound scaling: scaling width, depth, or resolution alone vs scaling all three together with a single coefficient  $\phi$

- **ShuffleNet** reduces the cost of  $1 \times 1$  convolutions (which dominate in MobileNet-style architectures) by using **group convolutions** followed by a **channel shuffle**. Group convolutions split channels into groups and convolve within each group independently, but this prevents cross-group information flow. The shuffle operation rearranges channels between groups, restoring the information mixing at negligible cost.

- **Transfer learning** is the practice of taking a model trained on one task and adapting it to a different task. In computer vision, this almost always means starting from a model pre-trained on ImageNet (1.4 million images, 1,000 classes) and adapting to a domain-specific dataset (medical images, satellite images, manufacturing defects).
- **Feature extraction:** freeze all convolutional layers, remove the final classification head, and train only a new head on top. The frozen layers act as a generic feature extractor. This works well when the target domain is similar to ImageNet and the target dataset is small.
- **Fine-tuning:** unfreeze some or all convolutional layers and train with a small learning rate. The pre-trained weights serve as a starting point rather than fixed features. Fine-tuning typically starts by unfreezing only the later layers (which capture high-level, task-specific features) and optionally unfreezing earlier layers as well.
- Transfer learning works because the early layers of a CNN learn universal features (edges, textures, colours) that are useful across tasks, while later layers learn task-specific features. A network trained to classify animals still has useful edge detectors for classifying buildings.
- **Visualising CNNs** reveals what the network has learned and helps debug unexpected behaviour.
- **Activation maps** (feature maps) show the output of each filter for a given input image. Early layer activations look like edge maps; deeper layers produce increasingly abstract, spatially coarse activations.
- **Grad-CAM** (Gradient-weighted Class Activation Mapping, Selvaraju et al., 2017) highlights the regions of the input image that were most important for the model's prediction. It works by:
  1. Computing the gradient of the target class score with respect to the feature maps of the last convolutional layer (using the chain rule from chapter 03)
  2. Global average pooling these gradients to get per-channel importance weights
  3. Computing a weighted combination of the feature maps and applying ReLU

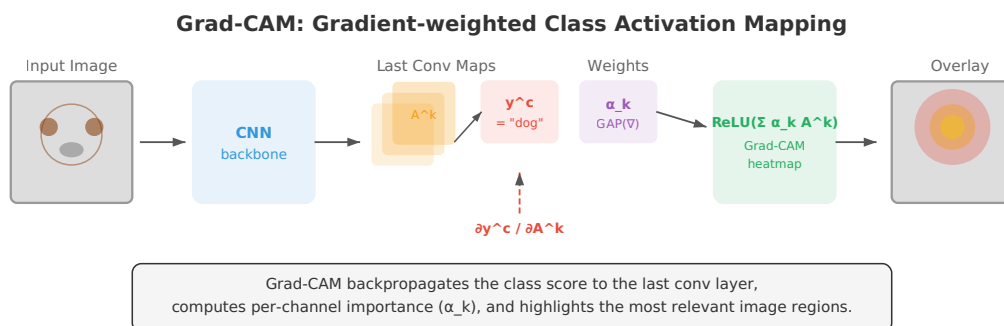


$$L_{\text{Grad-CAM}} = \text{ReLU} \left( \sum_k \alpha_k A^k \right), \quad \alpha_k = \frac{1}{Z}$$

$\sum_i \sum_j$

$$\frac{\partial y^c}{\partial A_{ij}^k}$$

- where  $A^k$  is the  $k$ -th feature map,  $\alpha_k$  is the importance weight for channel  $k$ , and  $y^c$  is the score for class  $c$ . The result is a coarse heatmap showing which regions drove the classification. ReLU is applied because we are interested in features that have a positive influence on the class.



Grad-CAM: input image of a dog, feature maps from the last conv layer, gradient-weighted combination, and the resulting heatmap overlaid on the original image highlighting the dog's face

- **Feature inversion** reconstructs an input image from its feature representation by optimising a random image to match the target features (using gradient descent on the pixel values). This reveals what information the network retains at each layer. Early layers reconstruct near-perfect images; deeper layers produce recognisable but distorted images, showing that fine spatial detail is lost while semantic content is preserved.
- **Deep Dream** and **neural style transfer** are creative applications of feature visualisation. Deep Dream maximises the activation of neurons at a chosen layer to produce surreal, pattern-amplified images. Neural style transfer optimises a target image to match the content features (from a deep layer) of

one image and the style features (Gram matrix of filter activations, which captures texture statistics) of another.

## 37.1 Coding Tasks (use CoLab or notebook)

---

1. Implement a simple CNN from scratch in JAX with two convolutional layers, max pooling, and a classification head. Train it on a synthetic 2D pattern classification task.

```
import jax
import jax.numpy as jnp
import jax.lax as lax
import matplotlib.pyplot as plt

def conv2d(x, kernel, stride=1):
 """Simple 2D convolution for single input, single filter."""
 return lax.conv(x[None, None], kernel[None, None], (stride, stride), 'SAME')

[0, 0]

def max_pool(x, size=2):
 """2x2 max pooling."""
 H, W = x.shape
 x = x[:H//size*size, :W//size*size]
 return x.reshape(H//size, size, W//size, size).max(axis=(1, 3))

def init_cnn(key):
 k1, k2, k3 = jax.random.split(key, 3)
 return {
 'conv1': jax.random.normal(k1, (5, 5)) * 0.3,
 'conv2': jax.random.normal(k2, (3, 3)) * 0.3,
 'fc_w': jax.random.normal(k3, (64, 1)) * 0.1,
 'fc_b': jnp.zeros(1),
 }

def forward_cnn(params, img):
 # Conv1 -> ReLU -> Pool
 h = jnp.maximum(0, conv2d(img, params['conv1']))
 h = max_pool(h)
 # Conv2 -> ReLU -> Pool
 h = jnp.maximum(0, conv2d(h, params['conv2']))
 h = max_pool(h)
 # Flatten and classify
 flat = h.ravel()
 # Pad or truncate to fixed size
 flat = jnp.pad(flat, (0, max(0, 64 - len(flat))))[:64]
```

```

logit = (flat @ params['fc_w'] + params['fc_b']).squeeze()
return jax.nn.sigmoid(logit)

Generate synthetic data: class 0 = low-freq pattern, class 1 = high-freq
def make_data(key, n=200):
 images, labels = [], []
 for i in range(n):
 k1, key = jax.random.split(key)
 x, y = jnp.meshgrid(jnp.linspace(0, 4*jnp.pi, 32), jnp.linspace(0,
4*jnp.pi, 32))
 if i < n // 2:
 img = jnp.sin(x) + jax.random.normal(k1, (32, 32)) * 0.1
 labels.append(0)
 else:
 img = jnp.sin(4 * x) * jnp.sin(4 * y) + jax.random.normal(k1, (32,
32)) * 0.1
 labels.append(1)
 images.append(img)
 return images, jnp.array(labels, dtype=jnp.float32)

key = jax.random.PRNGKey(42)
images, labels = make_data(key)
params = init_cnn(jax.random.PRNGKey(0))

def loss_fn(params, img, label):
 pred = forward_cnn(params, img)
 return -(label * jnp.log(pred + 1e-7) + (1 - label) * jnp.log(1 - pred +
1e-7))

grad_fn = jax.grad(loss_fn)
lr = 0.01

for epoch in range(5):
 total_loss = 0.0
 for img, label in zip(images, labels):
 grads = grad_fn(params, img, label)
 params = {k: params[k] - lr * grads[k] for k in params}
 total_loss += loss_fn(params, img, label)
 print(f"Epoch {epoch}: loss = {total_loss / len(images):.4f}")

Test accuracy
preds = jnp.array([forward_cnn(params, img) > 0.5 for img in images])

```

```
acc = jnp.mean(preds == labels)
print(f"Accuracy: {acc:.2%}")
```

1. Visualise how different filter sizes affect the receptive field. Show that two stacked 3x3 filters cover the same receptive field as one 5x5 filter but with fewer parameters.

```
import jax.numpy as jnp
import matplotlib.pyplot as plt

def compute_receptive_field(layers):
 """Compute receptive field size from a list of (kernel_size, stride)
 tuples."""
 rf = 1 # start with 1 pixel
 stride_product = 1
 for k, s in layers:
 rf += (k - 1) * stride_product
 stride_product *= s
 return rf

Compare architectures
configs = {
 'Single 5x5': [(5, 1)],
 'Two 3x3': [(3, 1), (3, 1)],
 'Three 3x3': [(3, 1), (3, 1), (3, 1)],
 'Single 7x7': [(7, 1)],
 '3x3 stride 2 + 3x3': [(3, 2), (3, 1)],
}

print(f"{'Config':<25} {'RF':>4} {'Params (per channel)':>20}")
print('-' * 55)
for name, layers in configs.items():
 rf = compute_receptive_field(layers)
 # Parameters: sum of k^2 for each layer (per input-output channel pair)
 params = sum(k * k for k, s in layers)
 print(f"{'name':<25} {'rf':>4} {'params':>20}")

Visualise receptive fields
fig, axes = plt.subplots(1, 3, figsize=(14, 4))
for ax, (name, rf_size) in zip(axes, [('5x5 filter', 5), ('Two 3x3 filters', 5),
('Three 3x3 filters', 7)]):
 grid = jnp.zeros((9, 9))
 c = 4 # centre
 half = rf_size // 2
 grid = grid.at[c-half:c+half+1, c-half:c+half+1].set(1.0)
 ax.imshow(grid, cmap='Blues', vmin=0, vmax=1)
```

```

ax.set_title(f'{name}\nRF = {rf_size}x{rf_size}')
ax.set_xticks(range(9)); ax.set_yticks(range(9))
ax.grid(True, alpha=0.3)
plt.suptitle('Receptive Field Comparison')
plt.tight_layout(); plt.show()

```

1. Implement Grad-CAM from scratch. Given a pre-built simple CNN, compute the gradient-weighted activation map for a specific class and visualise it as a heatmap.

```

import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

def simple_cnn(params, img):
 """Simple CNN that returns both the prediction and last conv activations."""
 # Conv layer (our "last conv layer" for Grad-CAM)
 H, W = img.shape
 k = params['conv'].shape[0]
 pad = k // 2
 img_pad = jnp.pad(img, pad, mode='edge')
 activation_map = jnp.zeros((H, W))
 for i in range(H):
 for j in range(W):
 activation_map = activation_map.at[i, j].set(
 jnp.sum(img_pad[i:i+k, j:j+k] * params['conv'])
)
 activation_map = jnp.maximum(0, activation_map) # ReLU

 # Global average pool -> dense -> output
 pooled = activation_map.mean()
 logit = pooled * params['w'] + params['b']
 return jax.nn.sigmoid(logit), activation_map

Create test image: bright region on the left (class indicator)
img = jnp.zeros((32, 32))
img = img.at[8:24, 4:16].set(1.0)
img = img.at[5:10, 20:28].set(0.3)

key = jax.random.PRNGKey(42)
params = {
 'conv': jax.random.normal(key, (5, 5)) * 0.3,
 'w': jnp.array(2.0),
 'b': jnp.array(-0.5),
}

```

```

Compute Grad-CAM
def class_score(params, img):
 pred, _ = simple_cnn(params, img)
 return pred

Get activation map and gradients
pred, act_map = simple_cnn(params, img)
grad_fn = jax.grad(lambda img: simple_cnn(params, img)[0])
img_grad = grad_fn(img)

Weight = global average of gradients (simplified 1-channel Grad-CAM)
alpha = img_grad.mean()
grad_cam = jnp.maximum(0, alpha * act_map) # ReLU
grad_cam = (grad_cam - grad_cam.min()) / (grad_cam.max() - grad_cam.min() + 1e-8)

fig, axes = plt.subplots(1, 3, figsize=(14, 4))
axes[0].imshow(img, cmap='gray'); axes[0].set_title('Input Image');
axes[0].axis('off')
axes[1].imshow(act_map, cmap='viridis'); axes[1].set_title('Activation Map');
axes[1].axis('off')
axes[2].imshow(img, cmap='gray', alpha=0.6)
axes[2].imshow(grad_cam, cmap='jet', alpha=0.4)
axes[2].set_title(f'Grad-CAM (pred={pred:.2f})'); axes[2].axis('off')
plt.tight_layout(); plt.show()

```

1. Compare depthwise separable convolution with standard convolution. Count the parameters and FLOPs for both and show they produce similar outputs with far less computation.

```

import jax
import jax.numpy as jnp

def standard_conv(x, kernel):
 """Standard convolution: (H, W, C_in) * (k, k, C_in, C_out) -> (H, W, C_out)."""
 H, W, C_in = x.shape
 k, _, _, C_out = kernel.shape
 pad = k // 2
 x_pad = jnp.pad(x, ((pad, pad), (pad, pad), (0, 0)), mode='constant')
 out = jnp.zeros((H, W, C_out))
 for i in range(H):
 for j in range(W):
 patch = x_pad[i:i+k, j:j+k, :] # (k, k, C_in)
 for c in range(C_out):
 out = out.at[i, j, c].set(jnp.sum(patch * kernel[:, :, :, c]))
 return out

```

```

def depthwise_separable_conv(x, dw_kernel, pw_kernel):
 """Depthwise separable: depthwise (k,k,C_in) then pointwise (C_in, C_out)."""
 H, W, C_in = x.shape
 k = dw_kernel.shape[0]
 pad = k // 2
 x_pad = jnp.pad(x, ((pad, pad), (pad, pad), (0, 0)), mode='constant')

 # Depthwise: one filter per channel
 dw_out = jnp.zeros((H, W, C_in))
 for i in range(H):
 for j in range(W):
 for c in range(C_in):
 patch = x_pad[i:i+k, j:j+k, c]
 dw_out = dw_out.at[i, j, c].set(jnp.sum(patch * dw_kernel[:, :,
c]))

 # Pointwise: 1x1 conv across channels
 out = dw_out @ pw_kernel
 return out

Setup
H, W, C_in, C_out, k = 8, 8, 16, 32, 3
key = jax.random.PRNGKey(42)
k1, k2, k3, k4 = jax.random.split(key, 4)

x = jax.random.normal(k1, (H, W, C_in))
std_kernel = jax.random.normal(k2, (k, k, C_in, C_out)) * 0.1
dw_kernel = jax.random.normal(k3, (k, k, C_in)) * 0.1
pw_kernel = jax.random.normal(k4, (C_in, C_out)) * 0.1

Compare
std_params = k * k * C_in * C_out
dw_params = k * k * C_in + C_in * C_out

std_flops = H * W * k * k * C_in * C_out
dw_flops = H * W * (k * k * C_in + C_in * C_out)

print(f"Standard conv: {std_params:>8,} params, {std_flops:>10,}
FLOPs")
print(f"Depthwise separable conv: {dw_params:>8,} params, {dw_flops:>10,}
FLOPs")
print(f"Parameter reduction: {std_params / dw_params:.1f}x")
print(f"FLOP reduction: {std_flops / dw_flops:.1f}x")

std_out = standard_conv(x, std_kernel)
ds_out = depthwise_separable_conv(x, dw_kernel, pw_kernel)

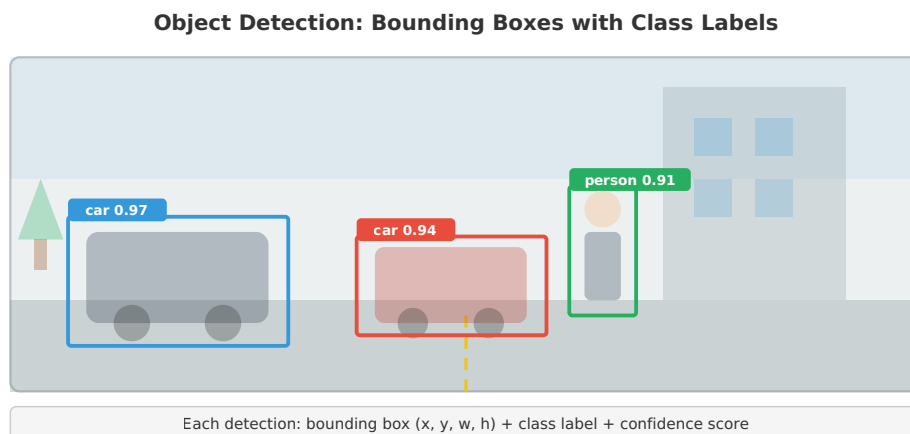
```

```
print(f"\nStandard output shape: {std_out.shape}")
print(f"Depthwise sep output shape: {ds_out.shape}")
```

## 38 Object Detection and Segmentation

Object detection localises and classifies every object in an image; segmentation assigns a label to every pixel. This file covers IoU, mAP, anchor boxes, R-CNN family, YOLO, SSD, Feature Pyramid Networks, semantic/instance/panoptic segmentation (U-Net, Mask R-CNN, SAM), and the metrics that benchmark them.

- Image classification (file 02) answers “what is in this image?” Object detection asks a harder question: “what objects are in this image, and where are they?”
- Segmentation goes further still: “which pixels belong to which object or category?” These tasks form a hierarchy of increasingly precise spatial understanding.
- An **object detection** model outputs a set of **bounding boxes**, each defined by four coordinates (top-left corner x, y, width, height) and a class label with a confidence score. A single image may contain zero, one, or hundreds of objects from multiple classes.



Input image with multiple objects, each enclosed by a coloured bounding box with a class label and confidence score

- **Intersection over Union (IoU)** measures how well a predicted bounding box matches the ground truth. It is the area of overlap divided by the area of union:



$$\text{IoU} = \frac{\text{Area of Intersection}}{\text{Area of Union}}$$

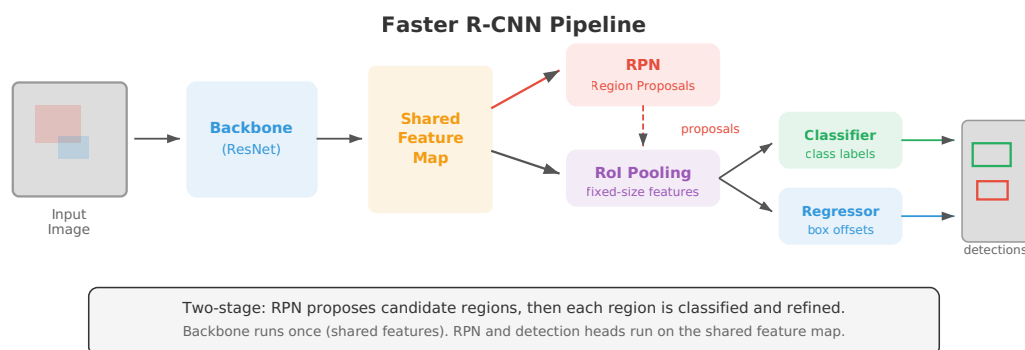
- An IoU of 1 means perfect overlap; an IoU of 0 means no overlap at all. The standard threshold for a “correct” detection is  $\text{IoU} \geq 0.5$ , though stricter thresholds (0.75, 0.9) are also used.
- A detection is a **true positive (TP)** if its IoU with a ground truth box exceeds the threshold and the class is correct.
- A **false positive (FP)** is a predicted box that does not match any ground truth.
- A **false negative (FN)** is a ground truth object that no prediction matched. These are the same precision/recall concepts from chapter 06.
- **Average Precision (AP)** summarises detection quality for one class. For each class, rank all detections by confidence score, compute precision and recall at each rank, and calculate the area under the precision-recall curve:

$$\text{AP} = \int_0^1 p(r) \, dr$$

- In practice, the curve is interpolated: at each recall level, precision is set to the maximum precision at any recall  $\geq r$ . This smooths the curve and makes it monotonically decreasing.
- **Mean Average Precision (mAP)** averages AP across all classes. “mAP@0.5” uses IoU threshold 0.5. “mAP@[.5:.95]” (the COCO standard) averages mAP over ten IoU thresholds from 0.5 to 0.95 in steps of 0.05, rewarding both detection and precise localisation.
- **Non-Maximum Suppression (NMS)** removes duplicate detections. When a model predicts multiple overlapping boxes for the same object, NMS keeps the highest-confidence box and removes all others that overlap with it above an IoU threshold. This is applied per class after the model produces its raw predictions.
- **Two-stage detectors** first propose candidate regions, then classify and refine each proposal.
- **R-CNN** (Girshick et al., 2014) was the first successful deep learning detector. It uses selective search (a classical algorithm) to propose ~2,000 candidate regions, warps each region to a fixed size, runs each through a CNN

independently, and classifies with an SVM (chapter 06). R-CNN was accurate but extremely slow: it ran the CNN 2,000 times per image.

- **Fast R-CNN** (Girshick, 2015) solved the redundancy by running the CNN once on the entire image to produce a shared feature map, then extracting features for each proposal from that shared map using **RoI pooling** (Region of Interest pooling).
- RoI pooling takes a variable-sized region of the feature map and produces a fixed-size output by dividing the region into a grid and max-pooling within each cell. This is much faster because the expensive CNN computation happens only once.
- **Faster R-CNN** (Ren et al., 2015) eliminated the external region proposal algorithm by introducing the **Region Proposal Network (RPN)**, a small CNN that runs on top of the shared feature map and predicts proposals directly. The RPN slides a small window over the feature map and, at each position, predicts  $k$  proposals (one for each **anchor box**).



Faster R-CNN pipeline: input image → backbone CNN → shared feature map → RPN generates proposals → RoI pooling → classification and box regression heads

- **Anchor boxes** are predefined bounding boxes at each spatial position of the feature map, covering different scales and aspect ratios (e.g., three scales × three ratios = 9 anchors per position). The RPN predicts two things for each anchor: an objectness score (object vs background) and coordinate offsets that refine the anchor into a tighter proposal. This parametrisation makes the regression problem easier: instead of predicting absolute coordinates, the network predicts small adjustments to a reasonable starting box.
- The anchor offsets are parametrised as:

$$t_x = \frac{x - x_a}{w_a}, \quad t_y = \frac{y - y_a}{h_a}, \quad t_w = \log \frac{w}{w_a}, \quad t_h = \log \frac{h}{h_a}$$

- where  $(x, y, w, h)$  are the predicted box centre and size, and  $(x_a, y_a, w_a, h_a)$  are the anchor. The log transform for width and height ensures the predicted box is always positive and makes the regression scale-invariant.
- Faster R-CNN trains with a multi-task loss: classification loss (cross-entropy from chapter 05) for the class label, plus a **smooth L1 loss** for box regression. Smooth L1 is less sensitive to outliers than L2:

```
\text{smooth}_{L1}(x) = \begin{cases} 0.5x^2 & \text{if } |x| < 1 \\ |x| - 0.5 & \text{otherwise} \end{cases}
```

- **Feature Pyramid Networks (FPN)** (Lin et al., 2017) address the multi-scale problem by building a top-down pathway with lateral connections that merges high-level semantics with low-level spatial detail. The backbone produces feature maps at multiple scales (each pooling layer halves the resolution). FPN adds a top-down path where each level receives upsampled features from the level above and merges them with the corresponding bottom-up level via lateral 1x1 convolutions. The result is a pyramid of feature maps, each with both strong semantics and good spatial resolution.
- Small objects are detected from the higher-resolution levels of the pyramid; large objects from the lower-resolution levels. FPN is now a standard component in most modern detection architectures.
- **One-stage detectors** skip the proposal step entirely, predicting class labels and bounding boxes in a single pass. This is faster but was historically less accurate than two-stage detectors, until focal loss closed the gap.
- **YOLO** (You Only Look Once, Redmon et al., 2016) divides the image into an  $S \times S$  grid. Each grid cell predicts B bounding boxes and C class probabilities. If the centre of an object falls in a grid cell, that cell is responsible for detecting it. YOLO is extremely fast because the entire detection is a single forward pass with no proposal stage.
- **YOLOv2** added anchor boxes, batch normalisation, and multi-scale training. **YOLOv3** used a Feature Pyramid Network and predicted at three scales. **YOLOv4-v8** continued improving with better backbones, path aggregation

networks, and mosaic data augmentation (stitching four images together during training to increase context diversity).

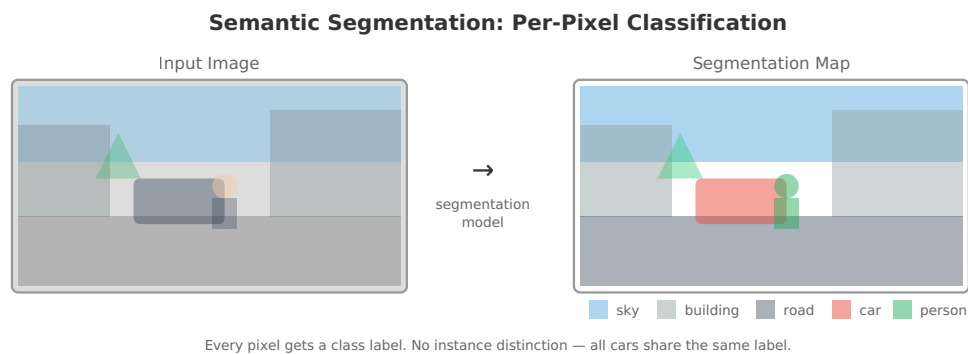
- **SSD** (Single Shot MultiBox Detector, Liu et al., 2016) predicts at multiple feature map scales within the backbone, using anchor boxes at each scale. Early (high-resolution) feature maps detect small objects; later (low-resolution) maps detect large objects. SSD is faster than Faster R-CNN with competitive accuracy.
- **RetinaNet** (Lin et al., 2017) identified the core problem with one-stage detectors: class imbalance. The vast majority of anchor boxes correspond to background, which generates easy negatives that dominate the loss and overwhelm the gradients from the rare positive examples.
- **Focal loss** solves this by down-weighting easy examples:

$$FL(p_t) = -\alpha_t(1 - p_t)^\gamma \log(p_t)$$

- where  $p_t$  is the predicted probability for the correct class. When the model is confident and correct ( $p_t$  is high),  $(1 - p_t)^\gamma$  is small, reducing the loss contribution from easy negatives. The hyperparameter  $\gamma$  (typically 2) controls the strength of the down-weighting. With  $\gamma = 0$ , focal loss reduces to standard cross-entropy. With focal loss, RetinaNet achieved accuracy comparable to two-stage detectors at one-stage speed.
- **Anchor-free detection** eliminates anchor boxes entirely, reducing hyperparameter tuning and simplifying the pipeline.
- **FCOS** (Fully Convolutional One-Stage, Tian et al., 2019) predicts, at every spatial position of the feature map, the distances from that position to the four sides of the nearest bounding box (left, top, right, bottom) plus a class label. A **centerness** score down-weights predictions far from the object centre, improving quality. FCOS uses FPN to handle multiple scales.
- **CenterNet** (Zhou et al., 2019) detects objects as points: it predicts a heatmap where peaks correspond to object centres, then regresses the width and height at each peak. Detection becomes keypoint estimation. This is elegant and anchor-free, but requires careful heatmap post-processing.
- **CornerNet** detects objects as pairs of corners (top-left and bottom-right). It predicts two heatmaps (one for each corner type) and uses an **associative**

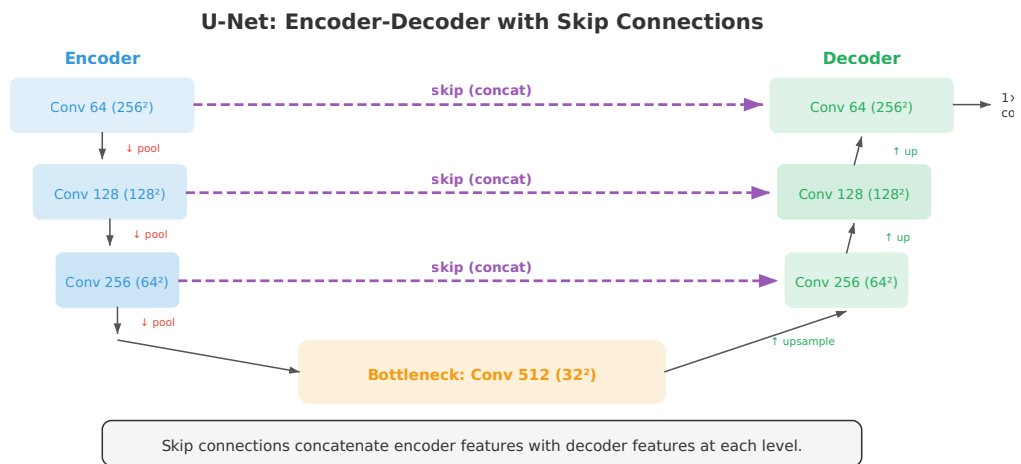
**embedding** to match corresponding corners into bounding boxes. This avoids the need for anchors and handles objects of arbitrary shape.

- **Semantic segmentation** assigns a class label to every pixel in the image. Unlike detection (which outputs boxes), segmentation produces a dense pixel-level map. A street scene might label every pixel as road, sidewalk, car, pedestrian, building, sky, etc.



Semantic segmentation: input street scene and its pixel-level label map where each colour represents a class

- **Fully Convolutional Networks (FCN)** (Long et al., 2015) adapted classification CNNs for segmentation by replacing fully connected layers with convolutional layers, allowing the network to output a spatial map rather than a single class. Upsampling (via transposed convolutions or bilinear interpolation) restores the output to the input resolution. Skip connections from earlier layers add back spatial detail lost during downsampling.
- **Transposed convolution** (sometimes called “deconvolution”) is the upsampling counterpart of convolution. Where strided convolution reduces spatial dimensions, transposed convolution increases them. It inserts zeros between input elements and then applies a standard convolution, effectively learning how to upsample.
- **U-Net** (Ronneberger et al., 2015) introduced a symmetric encoder-decoder architecture with skip connections at every level. The encoder (contracting path) reduces spatial resolution while increasing channels, exactly like a classification CNN. The decoder (expanding path) upsamples back to full resolution. Skip connections concatenate encoder feature maps with decoder feature maps at each level, providing fine spatial detail to the decoder. This combination of high-level semantics and low-level detail produces sharp, accurate segmentation boundaries.



U-Net architecture: encoder path on the left with downsampling, decoder path on the right with upsampling, and skip connections bridging corresponding levels

- U-Net was originally designed for biomedical image segmentation (where training data is scarce) and its architecture has become the foundation for many subsequent models, including the U-Net in latent diffusion models (file 04).
- **DeepLab** (Chen et al., 2014-2018) introduced two key innovations for segmentation:
  - **Atrous (dilated) convolution**: standard convolution with gaps inserted between filter elements, controlled by a dilation rate  $r$ . A  $3 \times 3$  filter with dilation  $r$  has a receptive field of  $(2r + 1) \times (2r + 1)$  while using only 9 parameters. This captures context at multiple scales without downsampling, preserving spatial resolution.
  - **Atrous Spatial Pyramid Pooling (ASPP)**: applies multiple atrous convolutions with different dilation rates in parallel (e.g., rates 1, 6, 12, 18), concatenates the results, and fuses with a  $1 \times 1$  convolution. ASPP captures context at multiple scales simultaneously, similar in spirit to the Inception module (file 02) but using dilation instead of different kernel sizes.
- DeepLab also used a **Conditional Random Field (CRF)** (chapter 05) as a post-processing step to refine segmentation boundaries by encouraging spatially nearby pixels with similar colours to share the same label.

- **Instance segmentation** combines detection and segmentation: it identifies each individual object instance and produces a pixel-level mask for each. Two cars in a scene get two separate masks, not just “car” for both.
- **Mask R-CNN** (He et al., 2017) extends Faster R-CNN by adding a small segmentation head that predicts a binary mask for each detected object. The architecture is Faster R-CNN + a mask branch: the mask branch takes the RoI-pooled features and outputs a  $m \times m$  binary mask per class. It uses **RoIAlign** instead of RoI pooling: bilinear interpolation at precisely sampled points rather than quantised grid cells, which avoids the spatial misalignment that quantisation causes. This small change significantly improves mask quality.
- Mask R-CNN is trained with a multi-task loss: classification loss + box regression loss + mask loss (per-pixel binary cross-entropy). The mask branch predicts a mask for every class independently; only the mask corresponding to the predicted class is used, which decouples mask prediction from classification and improves both.
- **Panoptic segmentation** unifies semantic and instance segmentation into a single task. Every pixel gets both a class label (semantic) and an instance ID (instance, for “thing” classes like cars and people). “Stuff” classes (sky, road, grass) get only semantic labels because they are amorphous regions without countable instances.
- The panoptic quality (PQ) metric evaluates this by decomposing into a segmentation quality (average IoU of matched segments) and a recognition quality (F1 score of matched segments):

$$PQ =$$

$$\frac{\sum_{(p, g) \in TP} \text{IoU}(p, g)}{|TP|}$$

~  
SQ

×

$$\frac{|TP|}{|TP| + \frac{1}{2}|FP| + \frac{1}{2}|FN|}$$

~  
RQ

- **Real-time segmentation** is critical for applications like autonomous driving and augmented reality, where latency budgets are tight (often under 30 milliseconds per frame).
- **BiSeNet** (Bilateral Segmentation Network, Yu et al., 2018) uses two parallel paths: a **spatial path** with wide, shallow layers that preserves spatial detail, and a **context path** with deep, narrow layers that captures semantics. The outputs are fused, giving both speed and accuracy.
- **DDRNet** (Deep Dual-Resolution Network, Hong et al., 2021) maintains two branches at different resolutions throughout the network, with repeated information exchange between them. The high-resolution branch preserves spatial detail while the low-resolution branch captures global context. Multiple bilateral fusion modules merge information in both directions.
- The general trend in real-time segmentation is to avoid the heavy encoder-decoder pattern and instead maintain sufficient spatial resolution throughout the network, trading some accuracy for dramatically lower latency.

## 38.1 Coding Tasks (use CoLab or notebook)

---

1. Implement IoU computation and Non-Maximum Suppression from scratch. Apply NMS to a set of overlapping bounding boxes and visualise the result.

```
import jax.numpy as jnp
import matplotlib.pyplot as plt
import matplotlib.patches as patches

def compute_iou(box1, box2):
 """Compute IoU between two boxes [x1, y1, x2, y2]."""
 x1 = jnp.maximum(box1[0], box2[0])
 y1 = jnp.maximum(box1[1], box2[1])
 x2 = jnp.minimum(box1[2], box2[2])
 y2 = jnp.minimum(box1[3], box2[3])
```



```

intersection = jnp.maximum(0, x2 - x1) * jnp.maximum(0, y2 - y1)
area1 = (box1[2] - box1[0]) * (box1[3] - box1[1])
area2 = (box2[2] - box2[0]) * (box2[3] - box2[1])
union = area1 + area2 - intersection

return intersection / (union + 1e-6)

def nms(boxes, scores, iou_threshold=0.5):
 """Non-Maximum Suppression."""
 order = jnp.argsort(-scores) # sort by descending confidence
 keep = []

 remaining = list(range(len(scores)))
 order_list = order.tolist()

 while order_list:
 idx = order_list[0]
 keep.append(idx)
 order_list = order_list[1:]

 new_order = []
 for j in order_list:
 iou = compute_iou(boxes[idx], boxes[j])
 if iou < iou_threshold:
 new_order.append(j)
 order_list = new_order

 return keep

Example: overlapping detections of the same object
boxes = jnp.array([
 [50, 60, 150, 160], # high confidence
 [55, 65, 155, 165], # overlapping duplicate
 [52, 58, 148, 158], # overlapping duplicate
 [200, 100, 300, 200], # different object
 [205, 105, 305, 205], # overlapping duplicate
])
scores = jnp.array([0.95, 0.80, 0.70, 0.90, 0.60])

keep = nms(boxes, scores, iou_threshold=0.5)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
colors = ['#3498db', '#e74c3c', '#27ae60', '#9b59b6', '#f39c12']

for ax, title, indices in zip(axes, ['Before NMS', 'After NMS'],
 [range(len(boxes)), keep]):
 ax.set_xlim(0, 400); ax.set_ylim(0, 300)

```

```

ax.set_aspect('equal'); ax.invert_yaxis()
ax.set_title(title)
for i in indices:
 b = boxes[i]
 rect = patches.Rectangle((b[0], b[1]), b[2]-b[0], b[3]-b[1],
 linewidth=2, edgecolor=colors[i],
 facecolor='none')
 ax.add_patch(rect)
 ax.text(b[0], b[1]-5, f'{scores[i]:.2f}', color=colors[i], fontsize=10)

plt.tight_layout(); plt.show()
print(f"Kept {len(keep)} of {len(boxes)} boxes after NMS")

```

1. Implement a simplified Region Proposal Network (RPN). Given a feature map, generate anchor boxes at multiple scales and aspect ratios, and predict objectness scores and box offsets.

```

import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt
import matplotlib.patches as patches

def generate_anchors(feature_h, feature_w, stride, scales, ratios):
 """Generate anchor boxes for each position on the feature map."""
 anchors = []
 for y in range(feature_h):
 for x in range(feature_w):
 cx = (x + 0.5) * stride
 cy = (y + 0.5) * stride
 for s in scales:
 for r in ratios:
 w = s * jnp.sqrt(r)
 h = s / jnp.sqrt(r)
 anchors.append([cx - w/2, cy - h/2, cx + w/2, cy + h/2])
 return jnp.array(anchors)

def rpn_forward(feature_map, params):
 """Simplified RPN: predicts objectness and box offsets per anchor."""
 H, W, C = feature_map.shape
 n_anchors = params['cls_w'].shape[1]

 # Slide a 1x1 conv over the feature map (simplified)
 cls_scores = feature_map.reshape(-1, C) @ params['cls_w'] # (H*W, n_anchors)
 box_offsets = feature_map.reshape(-1, C) @ params['reg_w'] # (H*W,
n_anchors*4)

```

```

 cls_scores = jax.nn.sigmoid(cls_scores)
 return cls_scores.ravel(), box_offsets.reshape(-1, 4)

Setup
feature_h, feature_w, channels = 4, 4, 16
stride = 16 # each feature map cell covers 16x16 pixels
scales = [32, 64, 128]
ratios = [0.5, 1.0, 2.0]
n_anchors_per_pos = len(scales) * len(ratios)

key = jax.random.PRNGKey(42)
k1, k2, k3 = jax.random.split(key, 3)

feature_map = jax.random.normal(k1, (feature_h, feature_w, channels))
params = {
 'cls_w': jax.random.normal(k2, (channels, n_anchors_per_pos)) * 0.01,
 'reg_w': jax.random.normal(k3, (channels, n_anchors_per_pos * 4)) * 0.01,
}

anchors = generate_anchors(feature_h, feature_w, stride, scales, ratios)
scores, offsets = rpn_forward(feature_map, params)

print(f"Feature map: {feature_h}x{feature_w}, stride={stride}")
print(f"Anchors per position: {n_anchors_per_pos}")
print(f"Total anchors: {len(anchors)}")
print(f"Objectness scores shape: {scores.shape}")
print(f"Box offsets shape: {offsets.shape}")

Visualise anchors for one position
fig, ax = plt.subplots(figsize=(6, 6))
img_size = feature_h * stride
ax.set_xlim(0, img_size); ax.set_ylim(0, img_size)
ax.invert_yaxis(); ax.set_aspect('equal')

pos_idx = feature_h // 2 * feature_w + feature_w // 2 # centre position
colors = ['#3498db', '#e74c3c', '#27ae60']
for i, s in enumerate(scales):
 for j, r in enumerate(ratios):
 idx = pos_idx * n_anchors_per_pos + i * len(ratios) + j
 a = anchors[idx]
 rect = patches.Rectangle((a[0], a[1]), a[2]-a[0], a[3]-a[1],
 linewidth=1.5, edgecolor=colors[i],
 facecolor='none', linestyle=['--', '-', ':'])

 ax.add_patch(rect)

ax.scatter([img_size/2], [img_size/2], c='red', s=50, zorder=5)

```

```

ax.set_title(f'Anchors at centre position\n3 scales × 3 ratios =
{n_anchors_per_pos}')
ax.grid(True, alpha=0.3)
plt.tight_layout(); plt.show()

```

1. Implement a simplified U-Net encoder-decoder with skip connections for 1D segmentation (binary labelling of a 1D signal).

```

import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

def conv1d_same(x, kernel):
 """1D convolution with same padding."""
 k = len(kernel)
 pad = k // 2
 x_pad = jnp.pad(x, pad, mode='edge')
 n = len(x)
 out = jnp.zeros(n)
 for i in range(n):
 out = out.at[i].set(jnp.sum(x_pad[i:i+k] * kernel))
 return out

def downsample(x):
 return x[::2]

def upsample(x, target_len):
 return jnp.interp(jnp.linspace(0, 1, target_len), jnp.linspace(0, 1,
len(x)), x)

def unet_1d(x, params):
 """Simplified 1D U-Net with 2 encoder/decoder levels."""
 # Encoder
 e1 = jnp.maximum(0, conv1d_same(x, params['enc1']))
 e1_down = downsample(e1)

 e2 = jnp.maximum(0, conv1d_same(e1_down, params['enc2']))
 e2_down = downsample(e2)

 # Bottleneck
 bottleneck = jnp.maximum(0, conv1d_same(e2_down, params['bottleneck']))

 # Decoder with skip connections
 d2_up = upsample(bottleneck, len(e2))
 d2 = jnp.maximum(0, conv1d_same(d2_up + e2, params['dec2'])) # skip
connection

```

```

d1_up = upsample(d2, len(e1))
d1 = conv1d_same(d1_up + e1, params['dec1']) # skip connection

return jax.nn.sigmoid(d1)

Create signal with labelled regions
n = 128
t = jnp.linspace(0, 4 * jnp.pi, n)
signal = jnp.sin(t) + 0.5 * jnp.sin(3 * t)
labels = (signal > 0.5).astype(jnp.float32) # binary segmentation target

key = jax.random.PRNGKey(42)
keys = jax.random.split(key, 5)
params = {
 'enc1': jax.random.normal(keys[0], (5,)) * 0.3,
 'enc2': jax.random.normal(keys[1], (5,)) * 0.3,
 'bottleneck': jax.random.normal(keys[2], (3,)) * 0.3,
 'dec2': jax.random.normal(keys[3], (5,)) * 0.3,
 'dec1': jax.random.normal(keys[4], (5,)) * 0.3,
}

def loss_fn(params, signal, labels):
 pred = unet_1d(signal, params)
 return -jnp.mean(labels * jnp.log(pred + 1e-7) + (1 - labels) * jnp.log(1 -
pred + 1e-7))

grad_fn = jax.jit(jax.grad(loss_fn))
lr = 0.05

for step in range(500):
 grads = grad_fn(params, signal, labels)
 params = {k: params[k] - lr * grads[k] for k in params}

pred = unet_1d(signal, params)

fig, axes = plt.subplots(3, 1, figsize=(12, 7), sharex=True)
axes[0].plot(t, signal, color='#3498db', linewidth=1.5)
axes[0].set_title('Input Signal'); axes[0].set_ylabel('Value')

axes[1].fill_between(t, 0, labels, alpha=0.3, color='#27ae60')
axes[1].set_title('Ground Truth Labels'); axes[1].set_ylabel('Label')

axes[2].plot(t, pred, color='#e74c3c', linewidth=1.5)
axes[2].fill_between(t, 0, (pred > 0.5).astype(float), alpha=0.2,
color='#e74c3c')
axes[2].set_title('U-Net Prediction'); axes[2].set_ylabel('Probability')

```

```

axes[2].set_xlabel('t')

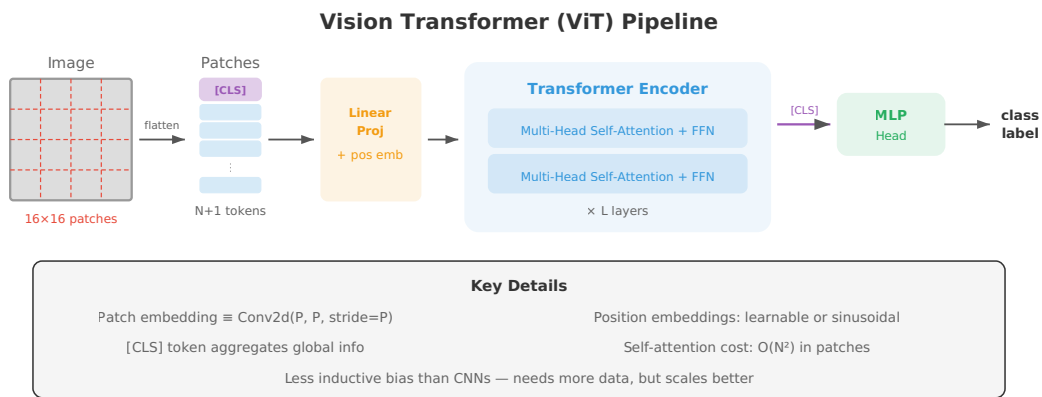
plt.tight_layout(); plt.show()
print(f"Final loss: {loss_fn(params, signal, labels):.4f}")
print(f"Pixel accuracy: {jnp.mean((pred > 0.5) == labels):.2%}")

```

## 39 Vision Transformers and Generation

Vision transformers apply self-attention to image patches, challenging CNN dominance with data-driven spatial learning. This file covers ViT, DeiT, Swin Transformer, image generation with GANs (StyleGAN), VAEs, and diffusion models (DDPM, Stable Diffusion), plus super-resolution and neural style transfer.

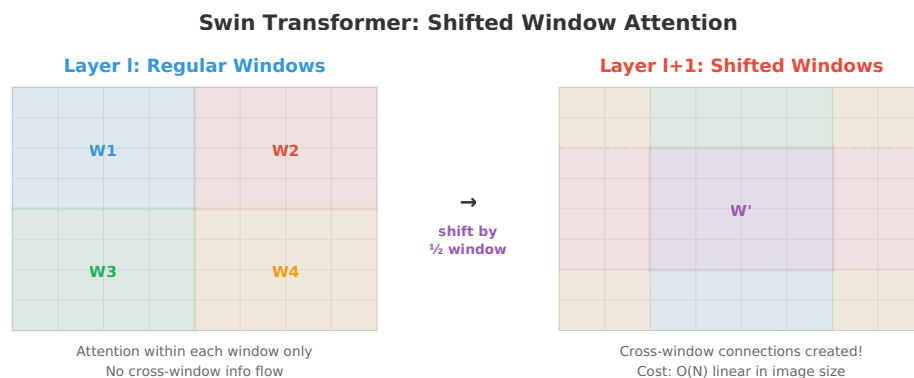
- CNNs (file 02) build in strong spatial inductive biases: local connectivity, weight sharing, and translation equivariance. Vision Transformers (ViTs) ask a provocative question: what if we drop these biases entirely and let the model learn spatial structure from data, using only the attention mechanism from chapter 06?
- The **Vision Transformer (ViT)** (Dosovitskiy et al., 2021) applies the standard Transformer encoder directly to images. The key idea is to treat an image as a sequence of patches, just as NLP treats text as a sequence of tokens.
- The process works as follows:
  1. Split the image (height  $H$ , width  $W$ , channels  $C$ ) into a grid of non-overlapping patches of size  $P \times P$ . This produces  $N = HW/P^2$  patches.
  2. Flatten each patch into a vector of length  $P^2 \cdot C$  and project it to the model dimension  $D$  via a learned linear embedding (a single matrix multiply, chapter 02).
  3. Prepend a learnable **[CLS] token** embedding (analogous to BERT's [CLS], chapter 07). This token attends to all patches and its final representation is used for classification.
  4. Add **position embeddings** (one learnable vector per position) to provide spatial information, since attention is permutation-equivariant.
  5. Pass the sequence of  $(N + 1)$  token embeddings through a standard Transformer encoder (multi-head self-attention + FFN, chapter 06).
  6. The [CLS] token's final representation is passed through a classification head (a small MLP).



ViT pipeline: image split into  $16 \times 16$  patches, each flattened and linearly projected, [CLS] token prepended, position embeddings added, then processed by Transformer encoder blocks

- The **patch embedding** is equivalent to a convolution with kernel size  $P$  and stride  $P$  (non-overlapping). ViT literally converts the 2D image into a 1D sequence, then processes it with the same architecture used for language.
- ViT has less inductive bias than CNNs: it does not enforce local connectivity or translation equivariance. This means it needs more training data to learn spatial structure from scratch. On small datasets, CNNs outperform ViT. But when trained on very large datasets (JFT-300M, 300 million images), ViT matches or exceeds the best CNNs, suggesting that the inductive biases of CNNs are helpful for data efficiency but not necessary for ultimate performance.
- ViT self-attention is  $O(N^2)$  in the number of patches. For a  $224 \times 224$  image with  $16 \times 16$  patches,  $N = 196$ , which is manageable. But for higher-resolution images or smaller patches, the quadratic cost becomes prohibitive.
- **DeiT** (Data-efficient Image Transformer, Touvron et al., 2021) showed that ViT can be trained effectively on ImageNet alone (without the massive JFT dataset) using strong data augmentation, regularisation (stochastic depth, label smoothing, dropout), and **knowledge distillation**: a pre-trained CNN teacher provides soft labels that the ViT student learns to match. DeiT added a **distillation token** alongside the [CLS] token, trained to predict the teacher's output.
- **Swin Transformer** (Liu et al., 2021) addresses ViT's two main limitations: its quadratic cost with image size and its lack of hierarchical feature maps (which detection and segmentation need).

- Swin introduces **shifted windows**: instead of global self-attention over all patches, attention is computed within local windows (e.g., 7x7 patches). This makes the cost linear in image size:  $O(N)$  rather than  $O(N^2)$ . But local windows alone would prevent information flow between regions.
- **Window shifting** solves this: in alternating layers, the window partition is shifted by half the window size. This creates cross-window connections, allowing information to flow between all parts of the image across layers without the cost of global attention.



Swin Transformer: layer  $l$  computes attention within regular windows, layer  $l+1$  shifts the window partition by half, creating cross-window connections

- Swin also builds a **hierarchical representation** by merging patches across stages. After each stage, neighbouring  $2 \times 2$  patches are concatenated and projected to double the channel dimension and halve the spatial resolution. This produces multi-scale feature maps analogous to those in CNNs and FPN (file 03), making Swin directly compatible with detection heads like Faster R-CNN and segmentation heads like U-Net.
- **PVT** (Pyramid Vision Transformer) takes a similar hierarchical approach with spatial-reduction attention: at each stage, the keys and values are spatially downsampled before computing attention, reducing the quadratic cost while maintaining the global receptive field.
- **Self-supervised visual learning** trains representations from unlabelled images. Labels are expensive to collect, but images are abundant. The goal is to learn features that transfer well to downstream tasks without any human annotation.
- **Contrastive learning** trains the model to recognise that two augmented views of the same image (a “positive pair”) should have similar representations,



while views of different images ( “negative pairs” ) should have dissimilar representations.

- **SimCLR** (Chen et al., 2020) creates two augmented views of each image in a batch, encodes both with a shared backbone + projection head, and applies the **NT-Xent loss** (normalised temperature-scaled cross-entropy):

$$\ell_{i,j} = - \log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k \neq i} \exp(\text{sim}(z_i, z_k)/\tau)}$$

- where  $\text{sim}$  is cosine similarity (chapter 01) and  $\tau$  is a temperature parameter. The numerator pushes positive pairs together; the denominator pushes negative pairs apart. SimCLR requires large batch sizes (4,096+) to provide enough negatives.
- **MoCo** (Momentum Contrast, He et al., 2020) solves the large-batch requirement by maintaining a **momentum-updated queue** of negative embeddings. The query encoder is updated by gradient descent; the key encoder is updated as an exponential moving average (EMA, chapter 04) of the query encoder:  $\theta_k \leftarrow m\theta_k + (1 - m)\theta_q$ , with  $m = 0.999$ . The queue stores recent key embeddings, providing a large and consistent set of negatives without needing huge batches.
- **BYOL** (Bootstrap Your Own Latent, Grill et al., 2020) eliminates negative pairs entirely. It uses two networks: an “online” network and a “target” network (EMA of the online). The online network predicts the target network’s representation of a different augmented view. Without negatives, BYOL avoids the collapse problem (where the model outputs the same vector for everything) through the asymmetry of the predictor head and the EMA target.
- **DINO** (Self-Distillation with No Labels, Caron et al., 2021) applies self-distillation to ViT. A student network predicts the output of a teacher network (EMA of the student) across different augmented views. The teacher uses larger crops; the student uses smaller crops. DINO produces features that contain explicit information about the scene layout: the self-attention maps of DINO-trained ViTs naturally segment objects without any segmentation supervision.
- **Masked image modelling** is the visual analogue of BERT’s masked language modelling (chapter 07). A large fraction of input patches is masked, and the model learns to reconstruct them.

- **MAE** (Masked Autoencoders, He et al., 2022) masks 75% of patches and trains a ViT encoder-decoder to reconstruct the missing pixel values. Only the unmasked patches are processed by the encoder (saving 4x computation during pre-training), and the lightweight decoder reconstructs the full image from the encoded visible patches plus learnable mask tokens.
- **BEiT** (BERT Pre-training of Image Transformers, Bao et al., 2022) masks patches and predicts discrete visual tokens (obtained from a pre-trained dVAE tokeniser) rather than raw pixels. This parallels BERT's prediction of discrete word tokens and avoids the low-level detail of pixel reconstruction.
- **Image generation** aims to produce new, realistic images that do not exist in the training set. The core challenge is modelling the high-dimensional probability distribution of natural images.
- **Generative Adversarial Networks (GANs)** (Goodfellow et al., 2014) use two competing networks: a **generator** G that creates fake images from random noise, and a **discriminator** D that tries to distinguish real images from fakes. They are trained adversarially: G tries to fool D, and D tries to catch G.

min  
G  
max  
D

$$+ \mathbb{E}_{z \sim p(z)} [\log (1 - D(G(z)))]$$

$$\mathbb{E}_{x \sim p_{\text{data}}} [\log$$

- The generator takes a random latent vector  $z$  (sampled from a simple distribution like a Gaussian) and maps it through a series of transposed convolutions to produce an image. The discriminator is a standard CNN classifier. At equilibrium, G produces images indistinguishable from real data, and D outputs 0.5 for all inputs.
- **Mode collapse** is the main failure mode of GANs: the generator learns to produce only a few types of images that fool the discriminator, ignoring the diversity of the training data. The generator finds a small set of “safe” outputs rather than covering the full distribution.
- Training tricks that stabilise GANs include: spectral normalisation (constraining the Lipschitz constant of the discriminator), progressive growing (training at low resolution first, then gradually increasing), feature matching (matching the statistics of intermediate discriminator features rather than the

final output), and using Wasserstein distance instead of the original JS divergence objective.

- **StyleGAN** (Karras et al., 2019) is the most influential GAN architecture for high-quality image synthesis. Its key innovation is the **style-based generator**: instead of feeding the latent vector  $z$  directly into the generator, it is first mapped through a **mapping network** (an 8-layer MLP) to produce a style vector  $w$ . This style vector is injected into each layer of the generator via **adaptive instance normalisation (AdaIN)**, which modulates the feature map statistics:

$$\text{AdaIN}(x, y) = y_s \cdot \frac{x - \mu(x)}{\sigma(x)} + y_b$$

- where  $y_s$  and  $y_b$  are the scale and bias derived from  $w$ . Different layers control different aspects: early layers control coarse features (pose, face shape), middle layers control medium features (hair style, eyes), and late layers control fine details (freckles, hair texture). StyleGAN can generate photorealistic faces at 1024x1024 resolution.
- **Variational Autoencoders (VAEs)** (chapter 06) provide an alternative generative approach. Unlike GANs, VAEs have a principled probabilistic framework with a clear training objective (ELBO). They tend to produce blurrier images than GANs but offer a smoother, more structured latent space. VAEs are the encoder-decoder pair used in latent diffusion models for compressing images to and from latent space.
- **Diffusion models** have become the dominant paradigm for image generation, surpassing GANs in both quality and diversity. The idea is conceptually simple: gradually add noise to data until it becomes pure Gaussian noise (the **forward process**), then learn to reverse this process step by step (the **reverse process**).
- The **forward process** adds Gaussian noise over  $T$  timesteps:

$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I)$$

- where  $\beta_t$  is a noise schedule that increases over time. After enough steps,  $x_T$  is approximately pure Gaussian noise regardless of the original image  $x_0$ . Using the reparametrisation trick (chapter 06) and setting  $\alpha_t = 1 - \beta_t$ ,

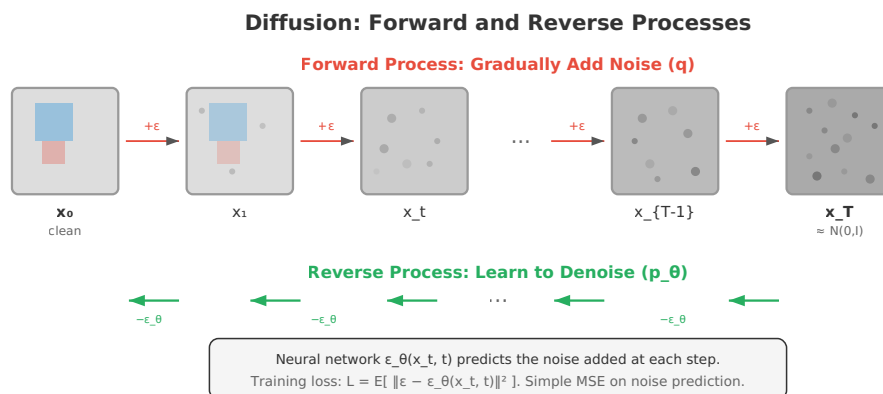
$\alpha$

$x_t = \prod_{s=1}^t \alpha_s$ , we can sample  $x_t$  directly from  $x_0$ :

$$x_t = \sqrt{\alpha_t} x_0 + \sqrt{1 - \alpha_t} \epsilon$$

- The **reverse process** learns to denoise: starting from pure noise  $x_T$ , the model predicts the noise  $\epsilon$  added at each step and subtracts it to recover  $x_{t-1}$ . This is parametrised by a neural network  $\epsilon_\theta$  (typically a U-Net, from file 03), trained with a simple MSE loss:

$$\mathcal{L} = \mathbb{E}_{t, x_0, \epsilon} [\| \epsilon - \epsilon_\theta(x_t, t) \|^2]$$



Diffusion forward and reverse processes: clean image gradually corrupted by noise over  $T$  steps (forward), neural network learns to reverse each step (reverse), starting from pure noise to generate a clean image

- **DDPM** (Denoising Diffusion Probabilistic Models, Ho et al., 2020) established this framework. Sampling requires iterating through all  $T$  steps (typically 1,000), which is slow. **DDIM** (Denoising Diffusion Implicit Models, Song et al., 2021) reformulates the sampling process as a deterministic mapping, allowing large step skips (e.g., 50 steps instead of 1,000) with minimal quality loss.
- **Score-based models** (Song and Ermon, 2019) provide an alternative perspective. Instead of predicting the noise  $\epsilon$ , the model estimates the **score function**  $\nabla_{x_t} \log p(x_t)$ , the gradient of the log-probability with respect to the noisy image. This gradient points toward higher-probability (cleaner) regions of

the data distribution. Sampling follows this gradient using Langevin dynamics. Score-based models and DDPM were unified in the framework of **stochastic differential equations (SDEs)**: the forward process is an SDE that adds noise, and the reverse process is the time-reversed SDE.

- **Classifier-free guidance** (Ho and Salimans, 2022) controls the tradeoff between sample quality and diversity. The model is trained both conditionally (with a text prompt or class label) and unconditionally (with the condition dropped randomly). At sampling time, the prediction is a weighted combination:

⊙

$\epsilon$

$$= \epsilon_{\theta}(x_t, \emptyset) + s \cdot (\epsilon_{\theta}(x_t, c) - \epsilon_{\theta}(x_t, \emptyset))$$

- where  $c$  is the condition,  $\emptyset$  is the null condition, and  $s > 1$  is the guidance scale. Higher  $s$  produces images that match the condition more strongly but with less diversity.  $s = 1$  gives the unguided model;  $s = 7.5$  is a common default.
- **Latent diffusion** (Rombach et al., 2022; Stable Diffusion) moves the diffusion process from pixel space to a learned latent space. A pre-trained VAE encoder compresses images to a lower-dimensional latent representation (typically 4x or 8x spatial downsampling), diffusion operates in this compressed space, and the VAE decoder reconstructs pixels from the denoised latent. This is dramatically more efficient: diffusing a 512x512 image in pixel space means processing a  $512 \times 512 \times 3$  tensor, but in latent space only a  $64 \times 64 \times 4$  tensor.
- The denoising U-Net in latent diffusion receives the noisy latent, the timestep (encoded as a sinusoidal embedding, analogous to positional encoding in transformers), and a conditioning signal (text embedding from a frozen CLIP or T5 text encoder). The text condition enters via cross-attention layers within the U-Net: the text embeddings serve as keys and values, and the image features serve as queries. This lets the model attend to relevant parts of the text prompt at each spatial location.
- **Flow matching** is an emerging alternative to diffusion that learns a direct transport path between noise and data, rather than the iterative denoising of DDPM.
- A **continuous normalising flow (CNF)** defines a time-dependent velocity field  $v_{\theta}(x, t)$  that pushes samples from a simple distribution  $p_0$  (noise) to the data

distribution  $p_1$  along smooth trajectories. The transformation follows an ordinary differential equation (ODE):

$$\frac{dx}{dt} = v_{\theta}(x, t), \quad t \in [0, 1]$$

- Starting from  $x_0 \sim \mathcal{N}(0, I)$ , integrating the ODE forward to  $t = 1$  produces a sample from the data distribution. The velocity field is parametrised by a neural network and trained to match a target conditional flow.
- **Optimal transport** (OT) flow matching (Lipman et al., 2023) uses straight-line paths between noise and data as the target flow: the conditional path from noise sample  $x_0$  to data sample  $x_1$  is simply  $x_t = (1 - t)x_0 + tx_1$ , and the target velocity is  $v = x_1 - x_0$ . The training loss becomes:

$$\mathcal{L} = \mathbb{E}_{t, x_0, x_1} [\| v_{\theta}(x_t, t) - (x_1 - x_0) \|^2]$$

- **Rectified flows** (Liu et al., 2022) iteratively straighten the learned flow paths. After an initial training pass, the model is used to generate (noise, data) pairs by simulating the ODE. These pairs, which are more closely aligned than random pairings, are used to retrain the model. Repeating this process produces increasingly straight paths, which can be traversed in fewer ODE steps (even a single step), enabling extremely fast generation.
- Flow matching has several advantages over diffusion: the training objective is simpler (direct velocity regression, no noise schedule), the sampling ODE is smoother (requiring fewer integration steps), and the connection to optimal transport provides theoretical grounding. Stable Diffusion 3 and Flux use flow matching instead of traditional DDPM.

## 39.1 Coding Tasks (use CoLab or notebook)

---

1. Implement the ViT patch embedding from scratch. Split an image into patches, flatten them, project to the model dimension, add position embeddings, and prepend a [CLS] token.

```
import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt
```

```

def create_patch_embedding(image, patch_size, d_model, params):
 """Convert an image into a sequence of patch embeddings."""
 H, W, C = image.shape
 n_patches_h = H // patch_size
 n_patches_w = W // patch_size
 n_patches = n_patches_h * n_patches_w

 # Extract patches
 patches = []
 for i in range(n_patches_h):
 for j in range(n_patches_w):
 patch = image[i*patch_size:(i+1)*patch_size,
 j*patch_size:(j+1)*patch_size, :]
 patches.append(patch.ravel())
 patches = jnp.stack(patches) # (N, P*P*C)

 # Linear projection to d_model
 embeddings = patches @ params['proj_w'] + params['proj_b'] # (N, d_model)

 # Prepend CLS token
 cls_token = params['cls_token'] # (1, d_model)
 embeddings = jnp.concatenate([cls_token, embeddings], axis=0) # (N+1,
d_model)

 # Add position embeddings
 embeddings = embeddings + params['pos_embed'] # (N+1, d_model)

 return embeddings, patches

Setup
H, W, C = 32, 32, 3
patch_size = 8
d_model = 64
n_patches = (H // patch_size) * (W // patch_size) # 16

key = jax.random.PRNGKey(42)
keys = jax.random.split(key, 5)

Create a synthetic image with distinct quadrants
image = jnp.zeros((H, W, C))
image = image.at[:16, :16, 0].set(1.0) # red top-left
image = image.at[:16, 16:, 1].set(1.0) # green top-right
image = image.at[16:, :16, 2].set(1.0) # blue bottom-left
image = image.at[16:, 16:, :2].set(1.0) # yellow bottom-right

params = {
 'proj_w': jax.random.normal(keys[0], (patch_size**2 * C, d_model)) * 0.02,

```

```

 'proj_b': jnp.zeros(d_model),
 'cls_token': jax.random.normal(keys[1], (1, d_model)) * 0.02,
 'pos_embed': jax.random.normal(keys[2], (n_patches + 1, d_model)) * 0.02,
 }

 embeddings, patches = create_patch_embedding(image, patch_size, d_model, params)

 print(f"Image shape: {image.shape}")
 print(f"Patch size: {patch_size}x{patch_size}")
 print(f"Number of patches: {n_patches}")
 print(f"Patch vector length: {patch_size**2 * C}")
 print(f"Embedding shape: {embeddings.shape} (CLS + {n_patches} patches)")

 # Visualise patches
 fig, axes = plt.subplots(2, 5, figsize=(14, 6))
 axes[0, 0].imshow(image); axes[0, 0].set_title('Full Image'); axes[0,
0].axis('off')
 for idx in range(min(9, n_patches)):
 ax = axes[(idx+1) // 5, (idx+1) % 5]
 patch_img = patches[idx].reshape(patch_size, patch_size, C)
 ax.imshow(patch_img); ax.set_title(f'Patch {idx}'); ax.axis('off')
 plt.suptitle('ViT Patch Decomposition')
 plt.tight_layout(); plt.show()

```

1. Implement a simple GAN training loop. Train a generator and discriminator on 2D data and visualise the generated distribution converging to the real distribution.

```

import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

def generator(z, params):
 h = jnp.tanh(z @ params['g_w1'] + params['g_b1'])
 h = jnp.tanh(h @ params['g_w2'] + params['g_b2'])
 return h @ params['g_w3'] + params['g_b3']

def discriminator(x, params):
 h = jax.nn.leaky_relu(x @ params['d_w1'] + params['d_b1'], 0.2)
 h = jax.nn.leaky_relu(h @ params['d_w2'] + params['d_b2'], 0.2)
 return jax.nn.sigmoid(h @ params['d_w3'] + params['d_b3'])

def init_params(key):
 keys = jax.random.split(key, 6)
 z_dim, h_dim, data_dim = 2, 32, 2
 scale = 0.1

```



```

 return {
 'g_w1': jax.random.normal(keys[0], (z_dim, h_dim)) * scale,
 'g_b1': jnp.zeros(h_dim),
 'g_w2': jax.random.normal(keys[1], (h_dim, h_dim)) * scale,
 'g_b2': jnp.zeros(h_dim),
 'g_w3': jax.random.normal(keys[2], (h_dim, data_dim)) * scale,
 'g_b3': jnp.zeros(data_dim),
 'd_w1': jax.random.normal(keys[3], (data_dim, h_dim)) * scale,
 'd_b1': jnp.zeros(h_dim),
 'd_w2': jax.random.normal(keys[4], (h_dim, h_dim)) * scale,
 'd_b2': jnp.zeros(h_dim),
 'd_w3': jax.random.normal(keys[5], (h_dim, 1)) * scale,
 'd_b3': jnp.zeros(1),
 }

def d_loss(params, real_data, fake_data):
 real_score = discriminator(real_data, params)
 fake_score = discriminator(fake_data, params)
 return -jnp.mean(jnp.log(real_score + 1e-7) + jnp.log(1 - fake_score + 1e-7))

def g_loss(params, fake_data):
 fake_score = discriminator(fake_data, params)
 return -jnp.mean(jnp.log(fake_score + 1e-7))

Real data: ring distribution
key = jax.random.PRNGKey(42)
theta = jax.random.uniform(key, (512,)) * 2 * jnp.pi
real_data = jnp.stack([jnp.cos(theta), jnp.sin(theta)], axis=1)
real_data = real_data + jax.random.normal(key, real_data.shape) * 0.05

params = init_params(jax.random.PRNGKey(0))
d_grad = jax.grad(d_loss)
g_grad = jax.grad(g_loss)
lr = 0.001

snapshots = []
for step in range(3000):
 key, k1 = jax.random.split(key)
 z = jax.random.normal(k1, (512, 2))
 fake_data = generator(z, params)

 # Update discriminator
 grads = d_grad(params, real_data, fake_data)
 for k in ['d_w1', 'd_b1', 'd_w2', 'd_b2', 'd_w3', 'd_b3']:
 params[k] = params[k] - lr * grads[k]

 # Update generator

```

```

fake_data = generator(z, params)
grads = g_grad(params, fake_data)
for k in ['g_w1', 'g_b1', 'g_w2', 'g_b2', 'g_w3', 'g_b3']:
 params[k] = params[k] - lr * grads[k]

if step in [0, 500, 1500, 2999]:
 snapshots.append((step, fake_data.copy()))

fig, axes = plt.subplots(1, 4, figsize=(16, 4))
for ax, (step, fake) in zip(axes, snapshots):
 ax.scatter(real_data[:, 0], real_data[:, 1], s=5, alpha=0.3, c='#3498db',
label='Real')
 ax.scatter(fake[:, 0], fake[:, 1], s=5, alpha=0.3, c='#e74c3c',
label='Generated')
 ax.set_title(f'Step {step}'); ax.set_xlim(-2, 2); ax.set_ylim(-2, 2)
 ax.set_aspect('equal'); ax.legend(markerscale=3)
plt.suptitle('GAN Training: Generator Learns the Ring Distribution')
plt.tight_layout(); plt.show()

```

1. Implement the diffusion forward process: add noise to an image at increasing timesteps and visualise the progressive corruption. Then implement a single denoising step.

```

import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

def noise_schedule(T, beta_start=0.0001, beta_end=0.02):
 """Linear noise schedule."""
 betas = jnp.linspace(beta_start, beta_end, T)
 alphas = 1.0 - betas
 alpha_bars = jnp.cumprod(alphas)
 return betas, alphas, alpha_bars

def forward_diffusion(x0, t, alpha_bars, key):
 """Add noise to x0 at timestep t."""
 alpha_bar_t = alpha_bars[t]
 noise = jax.random.normal(key, x0.shape)
 xt = jnp.sqrt(alpha_bar_t) * x0 + jnp.sqrt(1 - alpha_bar_t) * noise
 return xt, noise

Create a simple 2D "image" (checkerboard)
img = jnp.zeros((32, 32))
for i in range(4):
 for j in range(4):
 if (i + j) % 2 == 0:

```

```

img = img.at[i*8:(i+1)*8, j*8:(j+1)*8].set(1.0)

T = 1000
betas, alphas, alpha_bars = noise_schedule(T)

Visualise forward process
timesteps = [0, 50, 200, 500, 999]
key = jax.random.PRNGKey(42)

fig, axes = plt.subplots(1, len(timesteps), figsize=(16, 3.5))
for ax, t in zip(axes, timesteps):
 key, subkey = jax.random.split(key)
 xt, noise = forward_diffusion(img, t, alpha_bars, subkey)
 ax.imshow(xt, cmap='gray', vmin=-2, vmax=2)
 ax.set_title(f't={t}\n $\alpha_{\{t\}}$ ={alpha_bars[t]:.3f}')
 ax.axis('off')
plt.suptitle('Diffusion Forward Process: Progressive Noise Addition')
plt.tight_layout(); plt.show()

Simple denoising: train a tiny network to predict noise at t=200
t_denoise = 200
key, k1 = jax.random.split(key)
xt, true_noise = forward_diffusion(img, t_denoise, alpha_bars, k1)

Tiny "denoiser": just learn a constant noise estimate (for illustration)
noise_estimate = jnp.zeros_like(img)
lr = 0.01
for step in range(100):
 residual = noise_estimate - true_noise
 noise_estimate = noise_estimate - lr * residual

Reverse one step
alpha_bar_t = alpha_bars[t_denoise]
x_denoised = (xt - jnp.sqrt(1 - alpha_bar_t) * noise_estimate) /
jnp.sqrt(alpha_bar_t)

fig, axes = plt.subplots(1, 3, figsize=(12, 4))
axes[0].imshow(img, cmap='gray'); axes[0].set_title('Original x_0 ');
axes[0].axis('off')
axes[1].imshow(xt, cmap='gray', vmin=-2, vmax=2)
axes[1].set_title(f'Noisy $x_{\{200\}}$ '); axes[1].axis('off')
axes[2].imshow(x_denoised, cmap='gray')
axes[2].set_title('Denoised (one step)'); axes[2].axis('off')
plt.tight_layout(); plt.show()

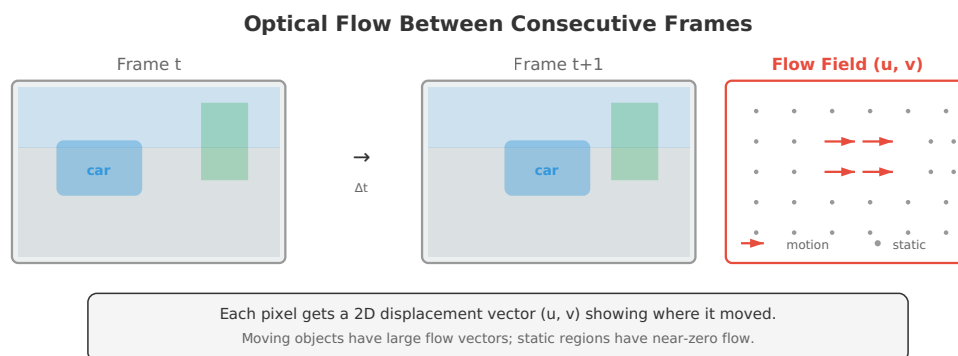
mse = jnp.mean((x_denoised - img)**2)
print(f"Denoising MSE: {mse:.4f}")

```

# 40 Video and 3D Vision

Video and 3D vision extend image understanding into the temporal and spatial domains. This file covers optical flow, video classification (3D CNNs, TimeSformer), object tracking (SORT, DeepSORT), action recognition, depth estimation (monocular and stereo), point clouds, NeRFs, and 3D Gaussian splatting.

- Files 01-04 treated images as isolated snapshots. But the visual world is continuous: objects move, scenes change, and depth exists. This file extends computer vision into the temporal domain (video) and the spatial domain (3D), covering how models understand motion, track objects, estimate depth, and reconstruct scenes.
- A **video** is a sequence of images (frames) captured over time. At 30 frames per second, a 10-second clip contains 300 frames. The key challenge is modelling the **temporal dimension**: how do objects move, how do scenes evolve, and how can we relate information across frames?
- Optical flow** estimates the apparent motion of pixels between two consecutive frames. For each pixel in frame  $t$ , optical flow produces a 2D displacement vector  $(u, v)$  pointing to where that pixel moved in frame  $t + 1$ . The result is a dense motion field the same size as the image.



Two consecutive video frames and the optical flow field between them, visualised as coloured arrows showing pixel motion direction and magnitude

- Optical flow is computed under the **brightness constancy assumption**: a pixel's intensity does not change as it moves. If a pixel at position  $(x, y)$  in frame  $t$  has intensity  $I(x, y, t)$  and moves by  $(u, v)$  in a small time interval  $\delta t$ :

$$I(x + u\delta t, y + v\delta t, t + \delta t) = I(x, y, t)$$

- Taking a first-order Taylor expansion (chapter 03) and dividing by  $\delta t$ :

$$I_x u + I_y v + I_t = 0$$

- where  $I_x, I_y$  are the spatial gradients (Sobel, file 01) and  $I_t$  is the temporal gradient (difference between consecutive frames). This is the **optical flow constraint equation**. One equation, two unknowns ( $u, v$ ): we need an additional constraint.
- **Lucas-Kanade** assumes the flow is constant within a small window (e.g., 5x5 pixels). This gives an overdetermined system (25 equations, 2 unknowns) solved by least squares (the normal equation from chapter 06):

$$\begin{bmatrix} u & v \end{bmatrix} = \begin{bmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{bmatrix}^{-1} \begin{bmatrix} -\sum I_x I_t \\ -\sum I_y I_t \end{bmatrix}$$

- The 2x2 matrix is the structure tensor from file 01 (the same matrix used in Harris corner detection). Lucas-Kanade works well for small motions but fails when objects move more than a few pixels between frames.
- **Farneback's method** fits a polynomial expansion to the neighbourhood of each pixel and estimates the displacement field that best explains the change between frames. It produces dense flow (a vector for every pixel) and handles larger motions than Lucas-Kanade.
- Modern **deep learning optical flow** methods (FlowNet, RAFT) learn to predict optical flow end-to-end from pairs of frames. **RAFT** (Recurrent All-Pairs Field Transforms, Teed and Deng, 2020) computes a 4D correlation volume between all pairs of pixels in the two frames and iteratively refines the flow estimate using a GRU-based update operator. RAFT achieves state-of-the-art accuracy and has become the standard flow backbone.
- **Two-stream networks** (Simonyan and Zisserman, 2014) were an early approach to video understanding. One stream processes a single RGB frame (appearance), the other processes a stack of optical flow frames (motion). The two streams are fused at the end (by averaging or concatenation). This architecture explicitly separates “what things look like” from “how they move.”

- **3D Convolutional Networks** extend 2D convolutions to the temporal dimension. A 3D convolution applies a filter of size  $k \times k \times k_t$  that spans both spatial and temporal dimensions, directly learning spatiotemporal features.
- **C3D** (Tran et al., 2015) stacked 3D convolutions with  $3 \times 3 \times 3$  filters, showing that temporal convolutions can learn motion features without explicit optical flow. The cost is high: 3D convolutions have  $k_t$  times more parameters and computation than their 2D counterparts.
- **I3D** (Inflated 3D, Carreira and Zisserman, 2017) took a more practical approach: start with a pre-trained 2D CNN (like Inception or ResNet) and “inflate” all 2D filters to 3D by repeating the weights along the temporal dimension and dividing by  $k_t$ . This transfers ImageNet pre-training to video while adding temporal modelling. A 2D  $k \times k$  filter becomes a  $k \times k \times k_t$  filter initialised as  $W_{3D}[:, :, j] = W_{2D}/k_t$  for all temporal positions  $j$ .
- **SlowFast Networks** (Feichtenhofer et al., 2019) use two parallel pathways operating at different temporal resolutions:
  - The **Slow pathway** processes frames at a low frame rate (e.g., every 16th frame) with high spatial resolution and many channels, capturing fine spatial detail.
  - The **Fast pathway** processes frames at a high frame rate (every 2nd frame) with reduced spatial resolution and fewer channels (typically 1/8 of the Slow pathway), capturing rapid temporal changes.
  - Lateral connections fuse information from Fast to Slow via strided convolutions.
- The insight is that spatial and temporal information have different bandwidth requirements: object appearance changes slowly, but motion can be rapid. SlowFast matches this asymmetry by design.
- **TimeSformer** (Bertasius et al., 2021) applies the Vision Transformer to video. It decomposes full spatiotemporal attention (which would be prohibitively expensive:  $O((T \times N)^2)$  for  $T$  frames and  $N$  patches per frame) into **divided attention**: each block alternates between temporal attention (each patch attends across time at the same spatial position) and spatial attention (each patch attends across space within the same frame). This reduces the cost from  $O(T^2N^2)$  to  $O(T^2 + N^2)$ .

- **VideoMAE** (Tong et al., 2022) extends the masked autoencoder idea (file 04) to video. An extremely high masking ratio (90-95%) is used because video has high temporal redundancy: neighbouring frames look nearly identical, so masking most patches still leaves enough information for reconstruction. VideoMAE pre-trains a ViT backbone on unlabelled video and transfers to downstream tasks.
- **Action recognition** classifies a video clip into one of many action categories (e.g., “running,” “cooking,” “playing guitar” ). It is the video analogue of image classification. Standard benchmarks include Kinetics-400 (400 action classes, ~300K clips), Something-Something (174 fine-grained actions requiring temporal reasoning), and ActivityNet (200 classes with long, untrimmed videos).
- **Temporal action detection** goes beyond classification: given a long untrimmed video, find the start time, end time, and class of each action. This is the temporal analogue of object detection. Methods like ActionFormer use a Transformer to process temporal features and predict action boundaries.
- **Video object tracking** follows a specific object across frames after it is identified in the first frame.
- **SORT** (Simple Online and Realtime Tracking, Bewley et al., 2016) combines a detection model (which detects objects in each frame independently) with the **Kalman filter** for motion prediction and the **Hungarian algorithm** for assignment.
- The **Kalman filter** maintains a state estimate (position, velocity, size) for each tracked object and predicts where it will be in the next frame using a linear motion model. When a new detection arrives, the Kalman filter updates its estimate by combining the prediction with the observation, weighted by their respective uncertainties. This is Bayesian updating (chapter 05) applied to tracking.
- The **Hungarian algorithm** solves the bilinear assignment problem: given M tracked objects and N new detections, find the optimal one-to-one matching that minimises total cost (using IoU distance from file 03). Unmatched detections start new tracks; unmatched tracks are terminated after a grace period.
- **DeepSORT** extends SORT by adding a **deep appearance feature**: each detected object is passed through a small CNN that produces an appearance embedding (a descriptor vector). The matching cost combines IoU distance with cosine

distance (chapter 01) in embedding space. This handles occlusion and re-identification: even if an object disappears behind another for several frames, its appearance embedding allows re-matching when it reappears.

- **ByteTrack** (Zhang et al., 2022) improves tracking by using every detection, including low-confidence ones. Most trackers discard detections below a confidence threshold. ByteTrack first matches high-confidence detections to existing tracks, then matches the remaining low-confidence detections to unmatched tracks. This recovers objects that are temporarily occluded or blurry (and thus have low detection confidence).
- **3D vision** recovers the third spatial dimension that is lost in the 2D image projection (file 01).
- **Depth estimation** predicts the distance from the camera to each point in the scene.
- **Stereo depth** uses two cameras separated by a known baseline  $b$ . The same point appears at different horizontal positions in the left and right images (this offset is called **disparity**  $d$ ). Depth is inversely proportional to disparity:

$$Z = \frac{f \cdot b}{d}$$

- where  $f$  is the focal length and  $b$  is the baseline distance. Computing disparity requires finding corresponding points between the two images (stereo matching), which is a 1D search along the horizontal scan line (because the cameras are horizontally aligned, a point at the same height in 3D projects to the same row in both images).
- **Monocular depth estimation** predicts depth from a single image, which is fundamentally ill-posed (infinitely many 3D scenes can produce the same 2D image). Yet humans do it effortlessly using cues like relative size, texture gradients, occlusion, and atmospheric haze. Deep networks learn these cues from training data.
- Models like **MiDaS** and **Depth Anything** predict relative depth maps (ranking which objects are closer) from single images. They are trained on diverse datasets with a scale-invariant loss and produce remarkably accurate results despite the theoretical ambiguity.



- **Point clouds** are sets of 3D points  $(x, y, z)$ , optionally with colour or other attributes, captured by LiDAR sensors or stereo reconstruction. Unlike images, point clouds are unordered and irregularly spaced.
- **PointNet** (Qi et al., 2017) processes point clouds directly by applying shared MLPs to each point independently, then aggregating with max pooling (which is permutation-invariant, solving the ordering problem). **PointNet++** adds hierarchical grouping to capture local structure at multiple scales.
- **Neural Radiance Fields (NeRFs)** (Mildenhall et al., 2020) represent a 3D scene as a continuous function that maps a 3D position  $(x, y, z)$  and viewing direction  $(\theta, \phi)$  to a colour  $(r, g, b)$  and density  $\sigma$ . This function is parameterised by an MLP:

$$F_{\theta} : (x, y, z, \theta, \phi) \rightarrow (r, g, b, \sigma)$$

- To render a pixel, a ray is cast from the camera through that pixel into the scene. Points are sampled along the ray, and the MLP predicts colour and density at each point. The pixel colour is computed by **volume rendering**: integrating colour weighted by density along the ray:

$$C(\mathbf{r}) = \int_{t_n}^t T(t) \cdot \sigma(\mathbf{r}(t)) \cdot \mathbf{c}(\mathbf{r}(t), \mathbf{d}) dt$$

- where  $T(t) = \exp(-\int_{t_n}^t \sigma(\mathbf{r}(s)) ds)$  is the accumulated transmittance (how much light has been absorbed so far). In practice, this integral is approximated by sampling  $N$  points along the ray and summing:

↺

C

=

N

Σ

i=1

$$T_i \cdot (1 - \exp(-\sigma_i \delta_i)) \cdot c_i$$

- NeRF is trained by minimising the MSE between rendered pixels and ground truth pixels from a set of posed photographs. After training, the NeRF can render photorealistic novel views from any camera position. The limitation is speed: rendering requires evaluating the MLP millions of times (one per sample point per pixel), making real-time rendering difficult.

- **3D Gaussian Splatting** (Kerbl et al., 2023) addresses NeRF's speed limitation by representing the scene as a collection of 3D Gaussian primitives rather than a continuous volumetric function. Each Gaussian has a 3D position (mean), a 3D covariance matrix (controlling shape and orientation), opacity, and colour (represented as spherical harmonics for view-dependent effects).
- Rendering projects each 3D Gaussian onto the image plane (producing a 2D Gaussian "splat"), sorts by depth, and composites front-to-back using alpha blending. This is a rasterisation process that runs on the GPU in real time (100+ FPS), orders of magnitude faster than NeRF's ray marching. Gaussian splatting matches or exceeds NeRF quality while enabling real-time rendering.
- **SLAM** (Simultaneous Localisation and Mapping) is the problem of building a map of an unknown environment while simultaneously tracking the camera's position within it. It is fundamental to robotics, autonomous driving, and AR.
- **Visual odometry** estimates camera motion from frame to frame by tracking features across images. Feature points (SIFT, ORB from file 01) are matched between consecutive frames, and the camera's rotation and translation are estimated from the correspondences using the **essential matrix** (which encodes the geometric relationship between two views, derived from the intrinsic and extrinsic parameters of file 01).
- **Feature-based SLAM** extends visual odometry by maintaining a persistent map. **ORB-SLAM** (Mur-Artal et al., 2015) is the most widely used feature-based SLAM system. It has three parallel threads:
  1. **Tracking:** match ORB features in each new frame to the map, estimate camera pose using PnP (Perspective-n-Point) and RANSAC
  2. **Local mapping:** triangulate new map points from matched features, optimise their positions using bundle adjustment (minimising reprojection error across all views that see each point)
  3. **Loop closure:** detect when the camera revisits a previously mapped area (using bag-of-visual-words), then correct the accumulated drift by globally optimising the map
- **LiDAR SLAM** uses 3D point clouds from LiDAR sensors instead of (or in addition to) camera images. LiDAR provides direct depth measurements, making the geometry estimation more robust but at higher hardware cost. Methods like LOAM (LiDAR Odometry and Mapping) register point clouds between consecutive scans using iterative closest point (ICP) alignment.

- **Visual-Inertial SLAM** fuses camera data with measurements from an IMU (accelerometer + gyroscope). The IMU provides high-frequency rotation and acceleration estimates that bridge the gaps between camera frames and handle fast motion or temporary visual feature loss.
- **VR/AR** applications are among the most demanding consumers of computer vision.
- **Pose estimation** determines the position and orientation of the human body (or face, or hands) from images. **Body pose** is typically represented as a set of 2D or 3D keypoint locations (joints: shoulders, elbows, wrists, hips, knees, ankles). Models like **OpenPose** and **MediaPipe** predict these keypoints using heatmap regression: for each joint, the model outputs a heatmap where the peak indicates the joint's location.
- **Top-down** methods first detect people with a bounding box detector (file 03), then estimate the pose within each box. **Bottom-up** methods detect all keypoints in the image first, then group them into individuals using part affinity fields (vector fields that encode the association between connected joints).
- **Scene reconstruction** builds a 3D model of the environment from sensor data. In AR, this enables placing virtual objects on real surfaces, occluding virtual objects behind real ones, and casting virtual shadows. Real-time scene reconstruction methods (like depth sensor-based systems in ARKit and ARCore) build a sparse mesh of the environment that updates as the user moves.
- **Real-time rendering** constraints in VR are extreme: both eyes need separate renders at 90+ FPS (to avoid motion sickness), with latency under 20 milliseconds from head movement to display update. Techniques like **foveated rendering** (rendering at high resolution only where the user is looking, using eye tracking) and **reprojection** (warping the previous frame based on new head pose to fill the gap while the next frame renders) are essential for meeting these constraints.
- The convergence of real-time neural rendering (3D Gaussian splatting), robust tracking (visual-inertial SLAM), and efficient pose estimation is making photorealistic, interactive AR/VR experiences increasingly feasible.

## 40.1 Coding Tasks (use CoLab or notebook)

1. Implement the Lucas-Kanade optical flow algorithm from scratch. Compute flow between two synthetic frames where a square moves to the right.

```
import jax.numpy as jnp
import matplotlib.pyplot as plt

def lucas_kanade(frame1, frame2, window_size=5):
 """Lucas-Kanade optical flow."""
 # Compute gradients
 Ix = jnp.zeros_like(frame1)
 Iy = jnp.zeros_like(frame1)
 It = frame2 - frame1

 # Sobel-like gradients
 Ix = Ix.at[1:-1, :].set((frame1[2:, :] - frame1[:-2, :]) / 2)
 Iy = Iy.at[:, 1:-1].set((frame1[:, 2:] - frame1[:, :-2]) / 2)

 H, W = frame1.shape
 half_w = window_size // 2
 u = jnp.zeros_like(frame1)
 v = jnp.zeros_like(frame1)

 for i in range(half_w, H - half_w):
 for j in range(half_w, W - half_w):
 Ix_win = Ix[i-half_w:i+half_w+1, j-half_w:j+half_w+1].ravel()
 Iy_win = Iy[i-half_w:i+half_w+1, j-half_w:j+half_w+1].ravel()
 It_win = It[i-half_w:i+half_w+1, j-half_w:j+half_w+1].ravel()

 A = jnp.stack([Ix_win, Iy_win], axis=1)
 ATA = A.T @ A
 ATb = -A.T @ It_win

 # Check if the system is well-conditioned
 det = ATA[0,0] * ATA[1,1] - ATA[0,1] * ATA[1,0]
 if jnp.abs(det) > 1e-6:
 flow = jnp.linalg.solve(ATA, ATb)
 u = u.at[i, j].set(flow[0])
 v = v.at[i, j].set(flow[1])

 return u, v

Create two frames: a white square that moves right
frame1 = jnp.zeros((64, 64))
frame1 = frame1.at[20:40, 15:35].set(1.0)
```

```

frame2 = jnp.zeros((64, 64))
frame2 = frame2.at[20:40, 20:40].set(1.0) # shifted 5 pixels right

u, v = lucas_kanade(frame1, frame2, window_size=7)

Visualise
fig, axes = plt.subplots(1, 3, figsize=(14, 4))
axes[0].imshow(frame1, cmap='gray'); axes[0].set_title('Frame 1');
axes[0].axis('off')
axes[1].imshow(frame2, cmap='gray'); axes[1].set_title('Frame 2');
axes[1].axis('off')

Quiver plot of flow (subsample for clarity)
step = 4
Y, X = jnp.mgrid[0:64:step, 0:64:step]
axes[2].imshow(frame1, cmap='gray', alpha=0.5)
axes[2].quiver(X, Y, u[::step, ::step], v[::step, ::step],
 color='#e74c3c', scale=50, width=0.005)
axes[2].set_title('Optical Flow'); axes[2].axis('off')

plt.tight_layout(); plt.show()

Check average flow in the moving region
region_u = u[20:40, 15:35]
print(f"Average horizontal flow in object region: {region_u[region_u !=
0].mean():.2f} pixels")

```

1. Implement a simple Kalman filter for 2D object tracking. Simulate a noisy trajectory and show how the Kalman filter smooths the estimates.

```

import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

def kalman_predict(x, P, F, Q):
 """Kalman filter prediction step."""
 x_pred = F @ x
 P_pred = F @ P @ F.T + Q
 return x_pred, P_pred

def kalman_update(x_pred, P_pred, z, H, R):
 """Kalman filter update step."""
 y = z - H @ x_pred # innovation
 S = H @ P_pred @ H.T + R # innovation covariance
 K = P_pred @ H.T @ jnp.linalg.inv(S) # Kalman gain

```

```

 x_updated = x_pred + K @ y
 P_updated = (jnp.eye(len(x_pred)) - K @ H) @ P_pred
 return x_updated, P_updated

State: [x, y, vx, vy]
dt = 1.0
F = jnp.array([[1, 0, dt, 0], # state transition
 [0, 1, 0, dt],
 [0, 0, 1, 0],
 [0, 0, 0, 1]])
H = jnp.array([[1, 0, 0, 0], # observation: we measure x, y
 [0, 1, 0, 0]])
Q = jnp.eye(4) * 0.01 # process noise
R = jnp.eye(2) * 4.0 # measurement noise (noisy detector)

Simulate ground truth: circular motion
n_steps = 50
t = jnp.linspace(0, 2 * jnp.pi, n_steps)
true_x = 10 * jnp.cos(t) + 20
true_y = 10 * jnp.sin(t) + 20

Noisy observations
key = jax.random.PRNGKey(42)
noise = jax.random.normal(key, (n_steps, 2)) * 2.0
obs_x = true_x + noise[:, 0]
obs_y = true_y + noise[:, 1]

Run Kalman filter
x = jnp.array([obs_x[0], obs_y[0], 0.0, 0.0]) # initial state
P = jnp.eye(4) * 10.0 # initial uncertainty

kalman_x, kalman_y = [], []
for i in range(n_steps):
 x, P = kalman_predict(x, P, F, Q)
 z = jnp.array([obs_x[i], obs_y[i]])
 x, P = kalman_update(x, P, z, H, R)
 kalman_x.append(x[0])
 kalman_y.append(x[1])

kalman_x = jnp.array(kalman_x)
kalman_y = jnp.array(kalman_y)

Visualise
plt.figure(figsize=(8, 8))
plt.plot(true_x, true_y, 'k-', linewidth=2, label='Ground Truth')
plt.scatter(obs_x, obs_y, c='#e74c3c', s=20, alpha=0.5, label='Noisy
Observations')

```

```

plt.plot(kalman_x, kalman_y, '#3498db', linewidth=2, label='Kalman Filter')
plt.legend(); plt.grid(alpha=0.3)
plt.title('Kalman Filter Tracking')
plt.xlabel('x'); plt.ylabel('y')
plt.axis('equal'); plt.show()

obs_error = jnp.mean(jnp.sqrt((obs_x - true_x)**2 + (obs_y - true_y)**2))
kalman_error = jnp.mean(jnp.sqrt((kalman_x - true_x)**2 + (kalman_y -
true_y)**2))
print(f"Observation RMSE: {obs_error:.2f}")
print(f"Kalman filter RMSE: {kalman_error:.2f}")
print(f"Error reduction: {(1 - kalman_error/obs_error) * 100:.1f}%")

```

1. Implement a simplified NeRF-style volume rendering pipeline. Cast rays through a simple 3D scene (spheres of known colour and density) and render an image by integrating along each ray.

```

import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

def render_ray(origin, direction, spheres, n_samples=64, t_near=1.0, t_far=6.0):
 """Volume render a single ray through a scene of spheres."""
 t_vals = jnp.linspace(t_near, t_far, n_samples)
 deltas = jnp.concatenate([jnp.diff(t_vals), jnp.array([1e-3])])

 colour = jnp.zeros(3)
 transmittance = 1.0

 for i in range(n_samples):
 point = origin + t_vals[i] * direction

 # Compute density and colour at this point
 density = 0.0
 point_colour = jnp.zeros(3)

 for center, radius, col, sigma in spheres:
 dist = jnp.linalg.norm(point - center)
 # Soft sphere: density falls off with distance from surface
 d = jnp.exp(-jnp.maximum(0, dist - radius) * sigma) * sigma
 density += d
 point_colour += d * jnp.array(col)

 # Normalise colour by total density
 point_colour = jnp.where(density > 1e-6, point_colour / density,
point_colour)

```

```

 # Volume rendering equation
 alpha = 1.0 - jnp.exp(-density * deltas[i])
 colour += transmittance * alpha * point_colour
 transmittance *= (1.0 - alpha)

 return colour

Scene: three coloured spheres
spheres = [
 (jnp.array([0.0, 0.0, 4.0]), 0.8, [1.0, 0.2, 0.2], 5.0), # red
 (jnp.array([1.5, 0.5, 5.0]), 0.6, [0.2, 1.0, 0.2], 5.0), # green
 (jnp.array([-1.0, -0.5, 3.5]), 0.5, [0.2, 0.2, 1.0], 5.0), # blue
]

Camera setup
img_h, img_w = 64, 64
focal = 60.0
origin = jnp.array([0.0, 0.0, 0.0])

image = jnp.zeros((img_h, img_w, 3))
for i in range(img_h):
 for j in range(img_w):
 # Compute ray direction
 px = (j - img_w / 2) / focal
 py = -(i - img_h / 2) / focal
 direction = jnp.array([px, py, 1.0])
 direction = direction / jnp.linalg.norm(direction)

 colour = render_ray(origin, direction, spheres)
 image = image.at[i, j].set(jnp.clip(colour, 0, 1))

plt.figure(figsize=(6, 6))
plt.imshow(image)
plt.title('NeRF-style Volume Rendering\n(3 spheres)')
plt.axis('off')
plt.tight_layout(); plt.show()
print(f"Image shape: {image.shape}")
print(f"Rendered {img_h * img_w} rays with 64 samples each")

```

## 41 Digital Signal Processing

Digital signal processing converts raw audio waveforms into structured representations that ML models can learn from. This file covers sound physics,



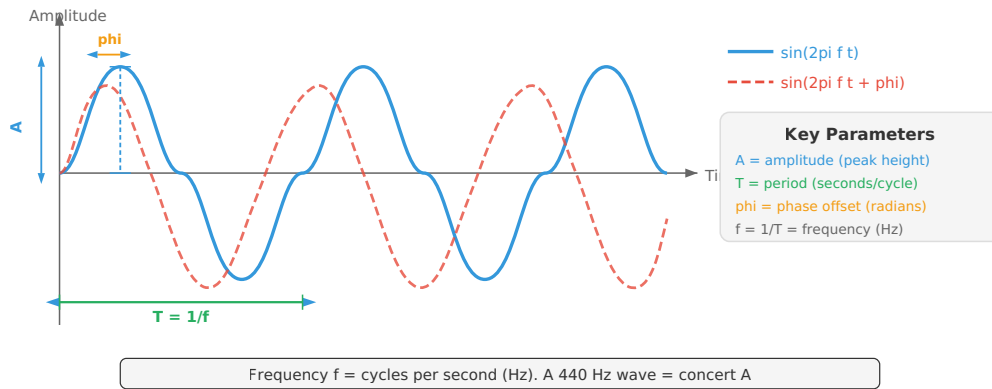
sampling and quantisation, the Fourier transform (DFT, FFT), spectrograms, mel filterbanks, MFCCs, and windowing, the feature extraction pipeline for all speech and audio AI.

- **Sound** is a pressure wave that propagates through a medium (air, water, solids). A vibrating object (vocal cord, guitar string, speaker cone) pushes and pulls on air molecules, creating alternating regions of high pressure (compression) and low pressure (rarefaction).
- These pressure variations travel outward at roughly 343 m/s in air and reach your ear, where they vibrate the eardrum and are transduced into neural signals.
- Think of sound like dropping a stone into a still pond: the stone is the vibrating source, the ripples are the pressure wave, and a cork bobbing on the surface is the microphone or eardrum responding to the wave's arrival.
- The height of the cork's bobbing is the **amplitude**, how often it bobs per second is the **frequency**, and whether it starts at the top or bottom of its bob when the wave arrives is the **phase**.
- A **waveform** is a plot of pressure (or voltage, after a microphone converts sound to an electrical signal) against time. The simplest waveform is a **pure tone**, a single sinusoid:

$$x(t) = A \sin (2\pi ft + \phi)$$

- where:
  - A is the amplitude (peak deviation from zero, determining loudness),
  - f is the frequency in Hz (cycles per second, determining pitch),
  - $\phi$  is the phase in radians (time offset of the wave).
- The **period** is  $T = 1/f$ , the duration of one complete cycle.

### Audio Waveform: Anatomy of a Sine Wave



A sine wave annotated with amplitude, period, frequency, and phase shift

- **Amplitude** determines the perceived loudness. Doubling the amplitude quadruples the power (since power is proportional to amplitude squared).
- Human hearing spans an enormous range of amplitudes, so we use a logarithmic scale: the **decibel** (dB). The sound pressure level is:

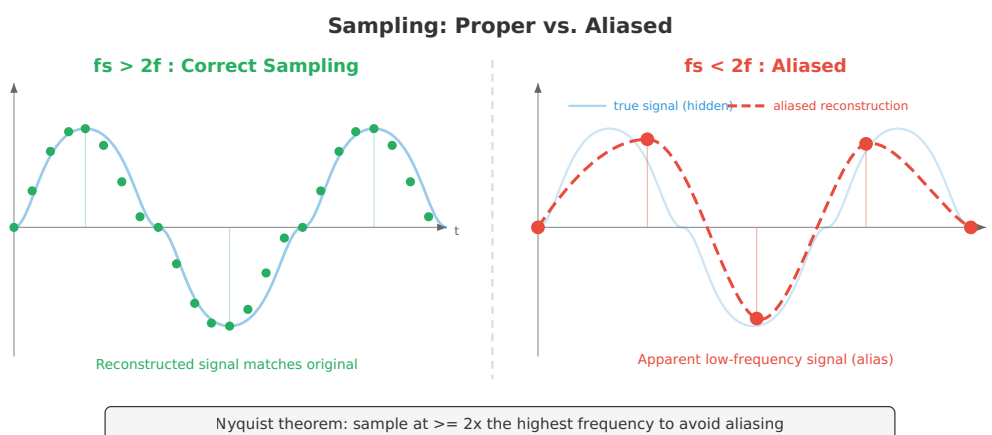
$$L = 20 \log_{10} \left( \frac{A}{A_{\text{ref}}} \right) \text{ dB}$$

- where  $A_{\text{ref}}$  is a reference amplitude (typically the threshold of hearing,  $20 \mu\text{Pa}$ ).  
A whisper is about 30 dB, normal conversation 60 dB, a rock concert 110 dB. Each 6 dB increase roughly doubles the amplitude; each 10 dB increase roughly doubles perceived loudness. The logarithm here is the same function from chapter 03.
- **Frequency** determines pitch. Low frequencies (20-250 Hz) sound bass; high frequencies (2000-20000 Hz) sound treble. Human hearing spans roughly 20 Hz to 20 kHz. Concert A is 440 Hz. Doubling the frequency raises the pitch by one **octave**.
- Most natural sounds are not pure tones but complex mixtures of many frequencies, which is why a piano and a violin playing the same note sound different: they share the same **fundamental frequency** but differ in their **harmonics** (integer multiples of the fundamental) and their relative amplitudes (the **timbre**).
- **Phase** determines where in its cycle the wave starts. Two waves with identical amplitude and frequency but different phases can interfere constructively (phases aligned, amplitudes add) or destructively (phases opposite, amplitudes cancel).

- Phase is critical in stereo audio and beamforming but is largely discarded in many speech processing pipelines because human pitch and timbre perception is mostly phase-invariant.
- Real-world audio signals are **continuous** functions of time, but computers work with discrete numbers. **Sampling** converts a continuous signal into a discrete sequence by measuring the signal's value at regular intervals.
- The **sample rate**  $f_s$  is how many measurements per second. CD audio uses  $f_s = 44,100$  Hz; telephony uses 8000 Hz; modern speech models typically use 16000 Hz.
- The **Nyquist-Shannon sampling theorem** states that a continuous signal can be perfectly reconstructed from its samples if and only if the sample rate is at least twice the highest frequency present in the signal:

$$f_s \geq 2f_{\max}$$

- The frequency  $f_s/2$  is called the **Nyquist frequency**. If the signal contains frequencies above the Nyquist frequency, those frequencies fold back into the valid range and appear as false lower-frequency components. This phenomenon is called **aliasing**. Aliasing is irreversible: once it occurs, the original signal cannot be recovered from the samples.
- The everyday analogy for aliasing is the wagon-wheel effect in films: a wheel spinning at just above the frame rate appears to spin slowly backward because the camera undersamples the rotation. In audio, a 15 kHz tone sampled at 16 kHz ( $f_{\text{Nyquist}} = 8$  kHz) aliases down to  $16 - 15 = 1$  kHz, a completely different pitch.



Proper sampling of a signal versus aliased sampling where the sample rate is too low

- To prevent aliasing, an **anti-aliasing filter** (a low-pass filter) removes all frequencies above  $f_s/2$  before sampling. This is applied by the analog-to-digital converter (ADC) hardware before the signal is digitised.
- **Quantisation** maps each continuous-valued sample to the nearest value in a finite set of levels. An n-bit quantiser has  $2^n$  levels. CD audio uses 16-bit quantisation ( $2^{16} = 65,536$  levels); telephony often uses 8-bit with  $\mu$ -law or A-law **companding** (a nonlinear mapping that devotes more levels to small amplitudes, matching human perception). Quantisation introduces **quantisation noise**, a form of rounding error with variance  $\Delta^2/12$  where  $\Delta$  is the step size between levels.
- **Time-domain analysis** extracts features directly from the waveform without transforming it to another domain. These features are simple, fast to compute, and capture basic signal properties.
- **Energy** of a frame of N samples measures the overall loudness:

$$E = \sum_{n=0}^{N-1} x[n]^2$$

- Speech segments have high energy; silence has low energy. Energy is the squared  $\ell_2$  norm from chapter 01 applied to a signal vector.
- **Zero-crossing rate** (ZCR) counts how often the signal changes sign in a frame:

$$ZCR = \frac{1}{2(N-1)} \sum_{n=1}^{N-1} |\text{sign}(x[n]) - \text{sign}(x[n-1])|$$

- High ZCR indicates high-frequency content or noise; low ZCR indicates low-frequency or voiced speech (where the vocal cords vibrate periodically). ZCR is

a crude frequency estimator: a pure tone at  $f$  Hz crosses zero  $2f$  times per second.

- **Autocorrelation** measures how similar a signal is to a delayed copy of itself:

$$R[k] = \sum_{n=0}^{N-1-k} x[n] \cdot x[n+k]$$

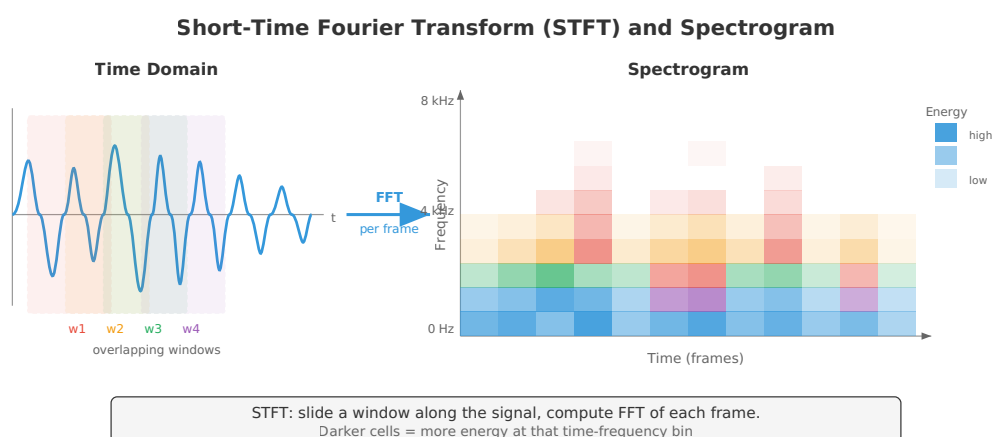
- At lag  $k=0$ , autocorrelation equals the energy. For periodic signals, the autocorrelation has peaks at lags equal to the period and its multiples. This is the standard technique for **pitch detection**: find the first significant peak in  $R[k]$  after  $k=0$ , and the pitch is  $f_s/k_{\text{peak}}$ . Autocorrelation is related to the dot product from chapter 01:  $R[k]$  is the dot product of the signal with its  $k$ -shifted version.
- **Frequency-domain analysis** reveals the spectral content of a signal, information invisible in the waveform. The key tool is the **Discrete Fourier Transform** (DFT), which decomposes a signal of  $N$  samples into  $N$  complex-valued frequency components:

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-j2\pi kn/N}, \quad k = 0, 1, \dots, N-1$$

- Each  $X[k]$  is a complex number whose magnitude  $|X[k]|$  gives the amplitude of the frequency component at  $f_k = k \cdot f_s/N$  Hz, and whose phase  $\angle X[k]$  gives the phase offset. The DFT is a change of basis from the time-domain basis (unit impulses) to the frequency-domain basis (complex exponentials), which is a direct application of the basis concept from chapter 02. The DFT can be written as a matrix multiplication  $\mathbf{X} = \mathbf{W}\mathbf{x}$  where  $\mathbf{W}$  is the  $N \times N$  DFT matrix with entries  $W_{kn} = e^{-j2\pi kn/N}$ .
- The **Fast Fourier Transform** (FFT) is an algorithm that computes the DFT in  $O(N \log N)$  operations instead of the naive  $O(N^2)$ , by recursively splitting the problem into even-indexed and odd-indexed subproblems (the Cooley-Tukey

algorithm). This speedup is what makes real-time spectral analysis practical. FFT is one of the most important algorithms in all of computing.

- The **power spectrum**  $|X[k]|^2$  shows the distribution of energy across frequencies. The **magnitude spectrum**  $|X[k]|$  shows amplitude. Plotting these reveals which frequencies dominate the signal: a vowel has strong harmonics at integer multiples of the fundamental frequency; a fricative (like “s”) has broad high-frequency energy.
- A **spectrogram** is a visual representation of how the frequency content of a signal changes over time. It is computed by chopping the signal into short overlapping frames, computing the FFT of each frame, and stacking the resulting magnitude spectra side by side. The horizontal axis is time, the vertical axis is frequency, and the colour (or brightness) at each point represents the magnitude. The spectrogram is the single most important visualisation in audio processing.



Spectrogram showing time on the horizontal axis, frequency on the vertical axis, and intensity as colour

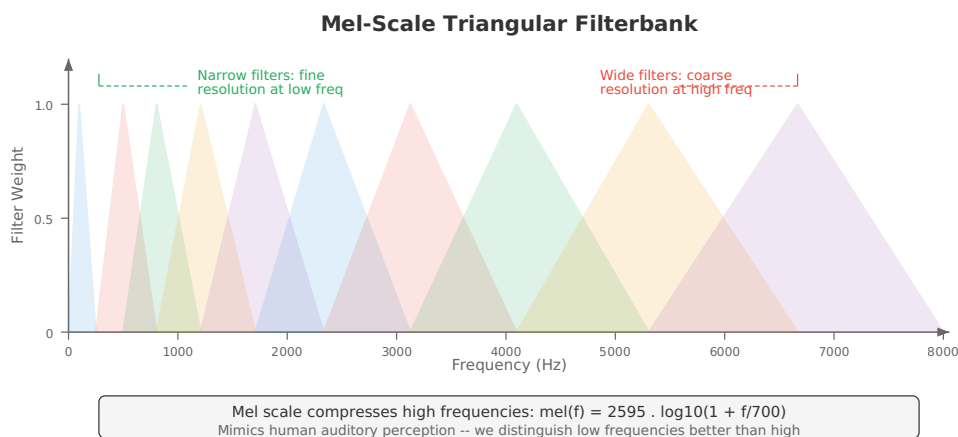
- The **mel scale** is a perceptual frequency scale that reflects how humans perceive pitch. Humans perceive equal ratios of frequency as equal intervals of pitch (just as we perceive equal ratios of intensity as equal intervals of loudness). Below about 1000 Hz, the mel scale is approximately linear; above 1000 Hz, it becomes approximately logarithmic:

$$m = 2595 \log_{10} \left( 1 + \frac{f}{700} \right)$$

- The inverse is  $f = 700(10^{m/2595} - 1)$ . The mel scale is the reason that musical semitones are equally spaced on a log-frequency axis: A4 (440 Hz) to A5 (880 Hz)

and A5 to A6 (1760 Hz) both sound like “one octave up” even though the Hz gaps are 440 and 880 respectively.

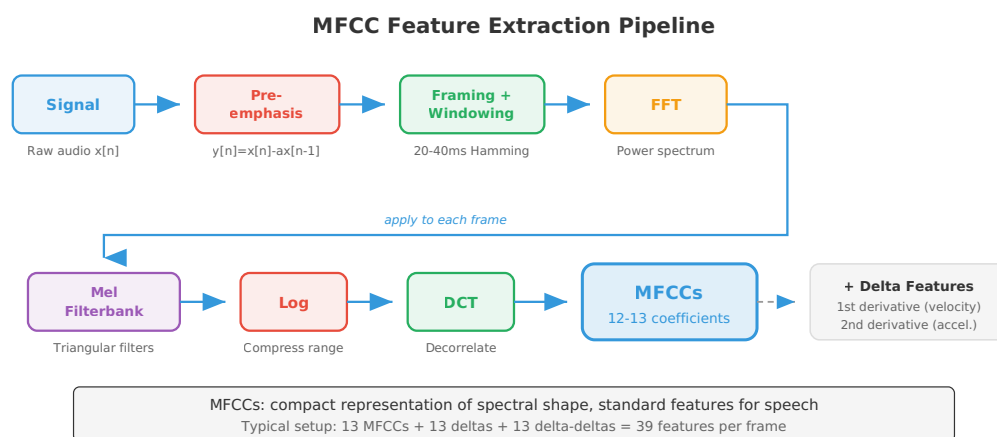
- A **mel filterbank** is a set of triangular bandpass filters spaced uniformly on the mel scale. Each filter covers a frequency band and sums the spectral energy within that band, producing a single number. Typical speech systems use 40-80 mel filters. Low-frequency filters are narrow (high frequency resolution where we are perceptually sensitive) and high-frequency filters are wide (low resolution where we are less sensitive). This mimics the frequency resolution of the human cochlea.



Triangular mel-scale filterbank overlaid on a linear frequency axis, showing narrow filters at low frequencies and wide filters at high frequencies

- **Mel-Frequency Cepstral Coefficients** (MFCCs) are the classic feature representation for speech and audio. They compress the mel spectrum into a small number of decorrelated coefficients that capture the shape of the spectral envelope (which encodes vocal tract configuration and therefore phonetic identity) while discarding fine spectral detail (which encodes pitch and phase).
- The MFCC pipeline:
  1. **Pre-emphasis**: apply a first-order high-pass filter  $y[n] = x[n] - \alpha x[n - 1]$  (typically  $\alpha = 0.97$ ) to boost high frequencies that are attenuated by the vocal tract.
  2. **Framing**: chop the signal into overlapping frames (typically 25 ms long, with 10 ms hop).
  3. **Windowing**: multiply each frame by a window function (Hamming) to reduce spectral leakage (see below).
  4. **FFT**: compute the power spectrum of each windowed frame.

5. **Mel filterbank**: apply the triangular mel filterbank to the power spectrum, producing mel-band energies.
6. **Log**: take the logarithm of the mel-band energies. The log compresses the dynamic range and converts multiplication (of spectral components) to addition, matching human loudness perception.
7. **DCT**: apply the Discrete Cosine Transform to the log-mel energies. The DCT decorrelates the mel bands (since adjacent bands are highly correlated) and compacts the energy into the first few coefficients. Keep the first 13 coefficients (MFCC-0 through MFCC-12).



MFCC pipeline from raw audio through windowed frames, FFT, mel filterbank, log compression, and DCT to final MFCC features

- The DCT in step 7 is essentially a “Fourier transform of the spectrum” (hence the name **cepstrum** = anagram of spectrum). Low-order cepstral coefficients capture the broad spectral shape (vocal tract resonances, called **formants**), while high-order coefficients capture fine spectral detail (pitch harmonics). By keeping only the first 13, we retain formant information and discard pitch detail.
- **Delta** and **delta-delta** MFCCs (first and second time derivatives of the MFCCs, computed by finite differences across adjacent frames) capture the dynamics of the spectral shape, adding temporal context. A full MFCC feature vector is often 39-dimensional: 13 static + 13 delta + 13 delta-delta.
- Modern neural network models (chapter 06) have largely replaced MFCCs with learned features: log-mel spectrograms (the output of step 6, skipping the DCT) are the standard input for deep learning ASR and audio classification. The model learns its own decorrelation. Still, MFCCs remain important for low-resource settings, classical ML pipelines, and understanding the signal processing foundations.



- **Windowing** is the process of multiplying a signal frame by a smooth window function before computing the FFT. Without windowing, the FFT assumes the frame repeats infinitely; the abrupt start and end of the frame create artificial discontinuities that spread energy across all frequencies, an artifact called **spectral leakage**.
- **Rectangular window**  $w[n] = 1$  for all  $n$ : no tapering, maximum leakage, but widest main lobe (best frequency resolution for a given frame length). Rarely used in practice.
- **Hamming window**:  $w[n] = 0.54 - 0.46 \cos(2\pi n/(N-1))$ . Tapers to near zero at the edges, greatly reducing leakage. The standard choice for speech processing.
- **Hann window** (also called Hanning):  $w[n] = 0.5 - 0.5 \cos(2\pi n/(N-1))$ . Tapers to exactly zero at the edges. Very similar to Hamming but with slightly better sidelobe suppression.
- **Blackman window**:  $w[n] = 0.42 - 0.5 \cos(2\pi n/(N-1)) + 0.08 \cos(4\pi n/(N-1))$ . Even better sidelobe suppression but wider main lobe (worse frequency resolution). Used when sidelobe artifacts are particularly problematic.
- There is a fundamental tradeoff: windows with less leakage have wider main lobes, meaning they cannot resolve two closely spaced frequencies. This is the **spectral resolution vs. leakage tradeoff**, a consequence of the uncertainty principle from chapter 03.
- **Overlap-add** (OLA) is a technique for reconstructing a signal from windowed, processed frames. Frames overlap (typically 50-75%), and after processing, the windowed outputs are summed. If the window and overlap are chosen correctly (e.g., Hann window with 50% overlap), the overlapping windows sum to a constant, giving perfect reconstruction. This is essential for any frame-based modification of audio (noise reduction, pitch shifting, time stretching).
- The **Short-Time Fourier Transform** (STFT) is the formal framework underlying spectrograms. It applies the DFT to each windowed frame of the signal:

$$\text{STFT}\{x[n]\}(m, k) = \sum_{n=0}^{N-1} x[n + mH] \cdot w[n] \cdot e^{-j 2\pi k n / N}$$

- where  $m$  is the frame index,  $H$  is the hop size (number of samples between consecutive frames),  $w[n]$  is the window function, and  $N$  is the FFT size. The

output is a 2D complex-valued matrix: the **time-frequency representation** of the signal.

- The STFT embodies a fundamental **time-frequency tradeoff**:
  - Long frames (large N): high frequency resolution (can distinguish closely-spaced frequencies) but poor time resolution (cannot pinpoint when a frequency changes).
  - Short frames (small N): high time resolution but poor frequency resolution.
  - The product of time resolution and frequency resolution is bounded below:  $\Delta t \cdot \Delta f \geq \frac{1}{4\pi}$ . This is the **Gabor limit**, the signal processing analogue of the Heisenberg uncertainty principle from physics.
- Typical speech STFT parameters: 25 ms frame length (N = 400 at 16 kHz), 10 ms hop (H = 160), Hamming window, 512-point FFT (zero-padded from 400 for efficiency and smoother spectral interpolation).
- **Filtering** modifies the frequency content of a signal by amplifying some frequencies and attenuating others. A **filter** is a system that takes an input signal and produces an output signal. Filters are characterised by their **frequency response**  $H(f)$ , which describes the gain and phase shift applied to each frequency.
- **Low-pass filter**: passes frequencies below a cutoff frequency  $f_c$  and attenuates frequencies above it. Removes high-frequency noise and detail. The anti-aliasing filter before sampling is a low-pass filter.
- **High-pass filter**: passes frequencies above  $f_c$  and attenuates below. Removes low-frequency rumble and DC offset. The pre-emphasis filter in MFCC extraction ( $y[n] = x[n] - 0.97x[n-1]$ ) is a simple high-pass filter.
- **Band-pass filter**: passes frequencies within a range  $[f_1, f_2]$  and attenuates outside. Each triangle in the mel filterbank is a band-pass filter.
- **Band-stop (notch) filter**: attenuates a specific narrow frequency range. Used to remove specific interference (e.g., 50/60 Hz power line hum).
- A **Finite Impulse Response** (FIR) filter computes each output sample as a weighted sum of the current and past input samples:

$$y[n] = \sum_{k=0}^M b_k \cdot x[n-k]$$

- The weights  $b_k$  are the **filter coefficients** (also called **taps**). The filter has order  $M$ . FIR filters are always stable (the output never diverges) and can be designed to have perfectly linear phase (all frequencies are delayed by the same amount, preserving waveform shape). Their downside is that achieving a sharp cutoff requires many taps (high  $M$ ), increasing computation. The output is a convolution of the input with the coefficient vector, exactly the 1D convolution operation from chapter 06.
- An **Infinite Impulse Response** (IIR) filter uses feedback: the output depends on both past inputs and past outputs:

$$y[n] = \sum_{k=0}^M b_k \cdot x[n-k] - \sum_{k=1}^L a_k \cdot y[n-k]$$

- The feedback terms  $a_k$  create a recursive structure whose impulse response is theoretically infinite in duration. IIR filters achieve sharp cutoffs with far fewer coefficients than FIR filters, but they can be unstable (output grows without bound if the poles of the transfer function lie outside the unit circle, a concept from the z-transform). They also have nonlinear phase, which can distort the waveform shape. Classical filter designs (Butterworth, Chebyshev, elliptic) are IIR.
- The **transfer function** of a discrete-time filter is obtained via the z-transform:

$$H(z) = \frac{\sum_{k=0}^M b_k z^{-k}}{1 + \sum_{k=1}^L a_k z^{-k}}$$

- The roots of the numerator are called **zeros** and the roots of the denominator are called **poles**. The pole-zero plot fully characterises the filter's behaviour. Poles near the unit circle amplify nearby frequencies; zeros near the unit circle

attenuate them. FIR filters have only zeros (the denominator is 1). This connects to the eigenvalue and root-finding concepts from chapter 02 and chapter 03.

- **Convolution theorem:** convolution in the time domain equals element-wise multiplication in the frequency domain. This means filtering can be performed either by directly convolving the signal with the filter impulse response, or by multiplying their Fourier transforms and inverse-transforming the result. For long filters, the frequency-domain approach (using FFT) is faster:  $O(N \log N)$  versus  $O(NM)$ .
- The **inverse STFT** (iSTFT) reconstructs a time-domain signal from its STFT representation. This is essential for any system that modifies audio in the frequency domain (noise reduction, source separation, voice conversion). The reconstruction uses overlap-add:

$$x[n] = \frac{\sum_m w[n - mH] \cdot \text{IDFT}\{X(m, k)\}[n - mH]}{\sum_m w[n - mH]^2}$$

- The denominator normalises for the window overlap, ensuring perfect reconstruction when the synthesis window matches the analysis window and the overlap is sufficient.
- **Summary of the DSP chain for speech:** raw audio is sampled at 16 kHz, pre-emphasised, chopped into 25 ms Hamming-windowed frames with 10 ms hop, each frame is FFT-transformed, passed through a mel filterbank, log-compressed, and either kept as log-mel features (for neural network models) or DCT-transformed to produce MFCCs (for classical models). This entire chain converts a 1D time-domain signal into a 2D time-frequency representation suitable for downstream machine learning, which will be the subject of file 02.

## 41.1 Coding Tasks (use CoLab or notebook)

1. Generate a sine wave, sample it at different rates, and demonstrate aliasing. Plot the continuous signal, properly-sampled version, and under-sampled (aliased) version.

```
import jax.numpy as jnp
import matplotlib.pyplot as plt

Parameters
f_signal = 5.0 # 5 Hz signal
```

```

duration = 1.0 # 1 second

"Continuous" signal (very high sample rate)
t_cont = jnp.linspace(0, duration, 10000)
x_cont = jnp.sin(2 * jnp.pi * f_signal * t_cont)

Properly sampled (fs = 50 Hz, well above Nyquist = 10 Hz)
fs_good = 50
t_good = jnp.arange(0, duration, 1.0 / fs_good)
x_good = jnp.sin(2 * jnp.pi * f_signal * t_good)

Under-sampled (fs = 7 Hz, below Nyquist = 10 Hz) -> aliasing
fs_bad = 7
t_bad = jnp.arange(0, duration, 1.0 / fs_bad)
x_bad = jnp.sin(2 * jnp.pi * f_signal * t_bad)

The aliased frequency: |f_signal - fs_bad| = |5 - 7| = 2 Hz
f_alias = abs(f_signal - fs_bad)
x_alias_cont = jnp.sin(2 * jnp.pi * f_alias * t_cont)

fig, axes = plt.subplots(3, 1, figsize=(12, 9))

Plot 1: original signal
axes[0].plot(t_cont, x_cont, color='#3498db', linewidth=1.5, label=f'Original {f_signal} Hz')
axes[0].set_title(f'Original {f_signal} Hz Signal')
axes[0].set_xlabel('Time (s)'); axes[0].set_ylabel('Amplitude')
axes[0].legend(); axes[0].grid(True, alpha=0.3)

Plot 2: proper sampling
axes[1].plot(t_cont, x_cont, color='#3498db', linewidth=1, alpha=0.4,
label='Original')
axes[1].stem(t_good, x_good, linefmt='#27ae60', markerfmt='o', basefmt='k-',
label=f'Sampled at {fs_good} Hz (above Nyquist)')
axes[1].set_title(f'Proper Sampling: fs = {fs_good} Hz > 2 x {f_signal} Hz')
axes[1].set_xlabel('Time (s)'); axes[1].set_ylabel('Amplitude')
axes[1].legend(); axes[1].grid(True, alpha=0.3)

Plot 3: aliased sampling
axes[2].plot(t_cont, x_cont, color='#3498db', linewidth=1, alpha=0.4,
label='Original')
axes[2].stem(t_bad, x_bad, linefmt='#e74c3c', markerfmt='o', basefmt='k-',
label=f'Sampled at {fs_bad} Hz (below Nyquist)')
axes[2].plot(t_cont, x_alias_cont, color='#f39c12', linewidth=1.5,
linestyle='--',
label=f'Aliased signal appears as {f_alias} Hz')
axes[2].set_title(f'Aliased Sampling: fs = {fs_bad} Hz < 2 x {f_signal} Hz')

```

```

axes[2].set_xlabel('Time (s)'); axes[2].set_ylabel('Amplitude')
axes[2].legend(); axes[2].grid(True, alpha=0.3)

plt.tight_layout(); plt.show()

```

1. Compute and visualise the FFT of a signal composed of multiple sinusoids. Show the magnitude spectrum and identify the constituent frequencies.

```

import jax.numpy as jnp
import matplotlib.pyplot as plt

Create a composite signal: 220 Hz + 440 Hz + 880 Hz (A3 + A4 + A5)
fs = 8000 # 8 kHz sample rate
duration = 0.1 # 100 ms
t = jnp.arange(0, duration, 1.0 / fs)
n_samples = len(t)

Three frequency components with different amplitudes
x = 1.0 * jnp.sin(2 * jnp.pi * 220 * t) + \
 0.6 * jnp.sin(2 * jnp.pi * 440 * t) + \
 0.3 * jnp.sin(2 * jnp.pi * 880 * t)

Compute FFT
X = jnp.fft.fft(x)
freqs = jnp.fft.fftfreq(n_samples, d=1.0 / fs)
magnitude = jnp.abs(X) / n_samples # normalise

Only plot positive frequencies
pos_mask = freqs >= 0
freqs_pos = freqs[pos_mask]
mag_pos = magnitude[pos_mask] * 2 # double to account for negative freq energy

fig, axes = plt.subplots(2, 1, figsize=(12, 7))

Time domain
axes[0].plot(t * 1000, x, color='#3498db', linewidth=1)
axes[0].set_title('Composite Signal: 220 Hz + 440 Hz + 880 Hz')
axes[0].set_xlabel('Time (ms)'); axes[0].set_ylabel('Amplitude')
axes[0].grid(True, alpha=0.3)

Frequency domain
axes[1].plot(freqs_pos, mag_pos, color='#e74c3c', linewidth=1.5)
axes[1].set_title('Magnitude Spectrum (FFT)')
axes[1].set_xlabel('Frequency (Hz)'); axes[1].set_ylabel('Magnitude')
axes[1].set_xlim(0, 1500)

Annotate peaks

```

```

for f_peak, amp in [(220, 1.0), (440, 0.6), (880, 0.3)]:
 axes[1].annotate(f'{f_peak} Hz', xy=(f_peak, amp), fontsize=10,
 ha='center', va='bottom', color='#9b59b6',
 arrowprops=dict(arrowstyle='->', color='#9b59b6'))
axes[1].grid(True, alpha=0.3)

plt.tight_layout(); plt.show()

```

1. Build the full MFCC pipeline from scratch in JAX: pre-emphasis, framing, windowing, FFT, mel filterbank, log, DCT. Visualise the mel filterbank and the resulting MFCCs as a heatmap.

```

import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

--- Generate a synthetic speech-like signal ---
key = jax.random.PRNGKey(42)
fs = 16000
duration = 1.0
t = jnp.arange(0, duration, 1.0 / fs)

Simulate voiced speech: fundamental + harmonics with amplitude decay
f0 = 150.0 # fundamental frequency
x = sum(jnp.sin(2 * jnp.pi * f0 * k * t) / k for k in range(1, 8))
Add some noise
x = x + 0.1 * jax.random.normal(key, t.shape)
x = x / jnp.max(jnp.abs(x)) # normalise

--- Step 1: Pre-emphasis ---
alpha = 0.97
x_pre = jnp.concatenate([x[:1], x[1:] - alpha * x[:-1]])

--- Step 2: Framing ---
frame_len = int(0.025 * fs) # 25 ms = 400 samples
hop_len = int(0.010 * fs) # 10 ms = 160 samples
n_frames = (len(x_pre) - frame_len) // hop_len + 1
frames = jnp.stack([x_pre[i * hop_len : i * hop_len + frame_len]
 for i in range(n_frames)])

--- Step 3: Hamming window ---
hamming = 0.54 - 0.46 * jnp.cos(2 * jnp.pi * jnp.arange(frame_len) / (frame_len
- 1))
windowed = frames * hamming

--- Step 4: FFT ---

```

```

n_fft = 512
spectra = jnp.fft.rfft(windowed, n=n_fft)
power_spectra = jnp.abs(spectra) ** 2 / n_fft

--- Step 5: Mel filterbank ---
n_mels = 40
f_min, f_max = 0.0, fs / 2.0

def hz_to_mel(f):
 return 2595 * jnp.log10(1 + f / 700)

def mel_to_hz(m):
 return 700 * (10 ** (m / 2595) - 1)

mel_min = hz_to_mel(f_min)
mel_max = hz_to_mel(f_max)
mel_points = jnp.linspace(mel_min, mel_max, n_mels + 2)
hz_points = mel_to_hz(mel_points)

freq_bins = jnp.floor((n_fft + 1) * hz_points / fs).astype(jnp.int32)
n_freqs = n_fft // 2 + 1
filterbank = jnp.zeros((n_mels, n_freqs))

for m in range(n_mels):
 f_left = freq_bins[m]
 f_center = freq_bins[m + 1]
 f_right = freq_bins[m + 2]
 # Rising slope
 for k in range(int(f_left), int(f_center)):
 if f_center != f_left:
 filterbank = filterbank.at[m, k].set((k - f_left) / (f_center -
f_left))
 # Falling slope
 for k in range(int(f_center), int(f_right)):
 if f_right != f_center:
 filterbank = filterbank.at[m, k].set((f_right - k) / (f_right -
f_center))

Apply filterbank
mel_spectra = jnp.dot(power_spectra, filterbank.T)

--- Step 6: Log ---
log_mel = jnp.log(mel_spectra + 1e-10)

--- Step 7: DCT (type-II) ---
n_mfcc = 13
n_mel_channels = log_mel.shape[1]

```



```

dct_matrix = jnp.zeros((n_mfcc, n_mel_channels))
for i in range(n_mfcc):
 for j in range(n_mel_channels):
 dct_matrix = dct_matrix.at[i, j].set(
 jnp.cos(jnp.pi * i * (j + 0.5) / n_mel_channels)
)
mfccs = jnp.dot(log_mel, dct_matrix.T)

--- Visualisation ---
fig, axes = plt.subplots(3, 1, figsize=(14, 11))

Mel filterbank
freq_axis = jnp.linspace(0, fs / 2, n_freqs)
for m in range(n_mels):
 color = '#3498db' if m % 2 == 0 else '#e74c3c'
 axes[0].plot(freq_axis, filterbank[m], color=color, alpha=0.6, linewidth=0.8)
axes[0].set_title(f'Mel Filterbank ({n_mels} filters)')
axes[0].set_xlabel('Frequency (Hz)'); axes[0].set_ylabel('Weight')
axes[0].grid(True, alpha=0.3)

Log-mel spectrogram
im1 = axes[1].imshow(log_mel.T, aspect='auto', origin='lower',
 extent=[0, duration, 0, n_mels], cmap='viridis')
axes[1].set_title('Log-Mel Spectrogram')
axes[1].set_xlabel('Time (s)'); axes[1].set_ylabel('Mel Band')
plt.colorbar(im1, ax=axes[1], label='Log Energy')

MFCCs
im2 = axes[2].imshow(mfccs.T, aspect='auto', origin='lower',
 extent=[0, duration, 0, n_mfcc], cmap='coolwarm')
axes[2].set_title(f'MFCCs (first {n_mfcc} coefficients)')
axes[2].set_xlabel('Time (s)'); axes[2].set_ylabel('MFCC Index')
plt.colorbar(im2, ax=axes[2], label='Coefficient Value')

plt.tight_layout(); plt.show()

```

1. Implement FIR low-pass and high-pass filters and visualise their effect on a signal containing both low and high frequency components. Show both time-domain and frequency-domain views.

```

import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

Create a signal with low (100 Hz) and high (2000 Hz) components
fs = 8000

```

```

duration = 0.05 # 50 ms for clear visualisation
t = jnp.arange(0, duration, 1.0 / fs)

x_low = jnp.sin(2 * jnp.pi * 100 * t)
x_high = 0.5 * jnp.sin(2 * jnp.pi * 2000 * t)
x = x_low + x_high

Design a simple FIR low-pass filter using windowed sinc
def fir_lowpass(cutoff_hz, fs, n_taps=51):
 """Design FIR low-pass filter using windowed sinc method."""
 fc = cutoff_hz / fs # normalised cutoff
 n = jnp.arange(n_taps)
 mid = (n_taps - 1) / 2.0
 # Sinc function (ideal low-pass impulse response)
 h = jnp.where(n == mid, 2 * fc,
 jnp.sin(2 * jnp.pi * fc * (n - mid)) / (jnp.pi * (n - mid)))
 # Apply Hamming window
 window = 0.54 - 0.46 * jnp.cos(2 * jnp.pi * n / (n_taps - 1))
 h = h * window
 h = h / jnp.sum(h) # normalise to unity gain at DC
 return h

def apply_filter(x, h):
 """Apply FIR filter via convolution."""
 return jnp.convolve(x, h, mode='same')

Low-pass filter at 500 Hz (passes 100 Hz, blocks 2000 Hz)
h_lp = fir_lowpass(500, fs, n_taps=51)
x_lp = apply_filter(x, h_lp)

High-pass = delta - low-pass (spectral inversion)
delta = jnp.zeros(51)
delta = delta.at[25].set(1.0)
h_hp = delta - h_lp
x_hp = apply_filter(x, h_hp)

Compute spectra for all signals
def compute_spectrum(signal, fs):
 X = jnp.fft.rfft(signal)
 freqs = jnp.fft.rfftfreq(len(signal), d=1.0 / fs)
 mag = jnp.abs(X) / len(signal) * 2
 return freqs, mag

fig, axes = plt.subplots(3, 2, figsize=(14, 10))

Time domain plots
for i, (sig, title, color) in enumerate([

```

```

(x, 'Original (100 Hz + 2000 Hz)', '#3498db'),
(x_lp, 'Low-pass filtered (< 500 Hz)', '#27ae60'),
(x_hp, 'High-pass filtered (> 500 Hz)', '#e74c3c')
]):
 axes[i, 0].plot(t * 1000, sig[:len(t)], color=color, linewidth=1)
 axes[i, 0].set_title(f'Time Domain: {title}')
 axes[i, 0].set_xlabel('Time (ms)'); axes[i, 0].set_ylabel('Amplitude')
 axes[i, 0].grid(True, alpha=0.3)

Frequency domain plots
for i, (sig, title, color) in enumerate([
 (x, 'Original', '#3498db'),
 (x_lp, 'Low-pass', '#27ae60'),
 (x_hp, 'High-pass', '#e74c3c')
]):
 freqs, mag = compute_spectrum(sig, fs)
 axes[i, 1].plot(freqs, mag, color=color, linewidth=1.5)
 axes[i, 1].set_title(f'Spectrum: {title}')
 axes[i, 1].set_xlabel('Frequency (Hz)'); axes[i, 1].set_ylabel('Magnitude')
 axes[i, 1].set_xlim(0, 3000)
 axes[i, 1].axvline(x=500, color='#f39c12', linestyle='--', alpha=0.7,
 label='Cutoff (500 Hz)')
 axes[i, 1].legend(); axes[i, 1].grid(True, alpha=0.3)

plt.tight_layout(); plt.show()

```

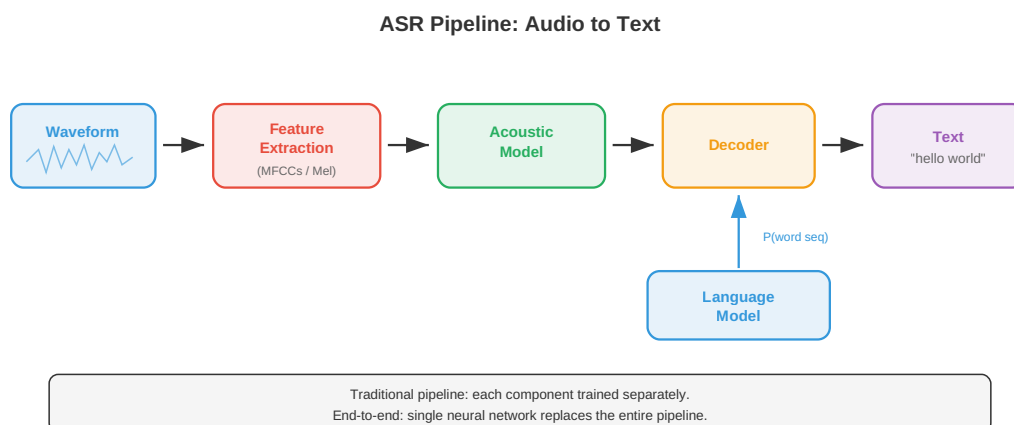
## 42 Automatic Speech Recognition

Automatic speech recognition converts spoken audio into written text, bridging the gap between human speech and machine-readable language. This file covers GMM-HMMs, CTC loss, the RNN-Transducer, attention-based encoder-decoder models (LAS), Whisper, and end-to-end ASR, from classical pipelines to modern neural architectures.

- **Automatic speech recognition** (ASR) is the task of converting spoken audio into written text. It is one of the oldest problems in AI (first systems in the 1950s recognised single digits) and one of the most commercially deployed (voice assistants, transcription services, subtitling).
- The difficulty comes from the enormous variability of speech: different speakers, accents, speaking rates, background noise, microphone

characteristics, and the fundamental ambiguity of mapping a continuous acoustic signal to discrete words.

- Think of ASR like a courtroom stenographer. The stenographer hears a continuous stream of sound, mentally segments it into words, resolves ambiguities using context ( “they’ re” vs “their” vs “there” ), and types the result. An ASR system does the same thing, but in stages that can be made explicit and optimised independently or jointly.
- The **classical ASR pipeline** processes audio in a chain of distinct stages: raw audio is transformed into features (MFCCs or log-mel spectrograms, from file 01), an **acoustic model** scores how well each feature frame matches each phonetic unit, a **pronunciation model** (lexicon) maps phonetic units to words, a **language model** scores how likely word sequences are, and a **decoder** searches for the word sequence that maximises the combined score. Each component is trained and tuned separately.



ASR pipeline from raw audio through feature extraction, acoustic model, decoder, and language model to output text

- **Phonemes** are the smallest units of sound that distinguish words in a language. English has roughly 39-44 phonemes (the exact number depends on the dialect and the phoneme inventory used). For example, “bat” and “pat” differ in one phoneme (/b/ vs /p/). Most ASR systems model **context-dependent phonemes** called **triphones**: a phoneme defined by its left and right neighbours (e.g., “a” in the context of “b\_t” is a different unit from “a” in “c\_t” ), because the acoustic realisation of a phoneme is heavily influenced by its neighbours (this is called **coarticulation**).
- The number of possible triphones is enormous (40 phonemes cubed = 64,000), so **decision tree clustering** groups acoustically similar triphones into **senones**

(typically 2000-10,000 classes). Each senone gets its own acoustic model. This clustering is a form of the decision tree algorithms from chapter 06.

- **GMM-HMM** (Gaussian Mixture Model - Hidden Markov Model) was the dominant acoustic modelling approach from the 1980s to the early 2010s. The HMM (from chapter 05) models the temporal structure of speech: each phoneme is a left-to-right HMM with 3-5 states, where each state represents a sub-phonetic segment (onset, middle, offset). The state-to-state transitions model duration implicitly.
- At each HMM state, the emission probability (how likely a particular feature vector is given that state) is modelled by a **Gaussian mixture model** (GMM): a weighted sum of multivariate Gaussian distributions (from chapter 05):

$$p(\mathbf{x} \mid s) = \sum_{m=1}^M w_m \prod_{n=1}^N \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_m, \boldsymbol{\Sigma}_m)$$

- where  $\mathbf{x}$  is the feature vector (e.g., 39-dimensional MFCCs),  $s$  is the HMM state,  $M$  is the number of mixture components (typically 8-64),  $w_m$  are mixture weights, and  $\boldsymbol{\mu}_m, \boldsymbol{\Sigma}_m$  are the mean and covariance of each Gaussian component. The covariance matrices are usually diagonal for computational efficiency (assuming feature dimensions are independent, which is approximately true for MFCCs due to the DCT decorrelation).
- Training uses the **Baum-Welch algorithm** (a special case of EM, from chapter 05) to iteratively estimate the GMM parameters and HMM transition probabilities from transcribed speech data. Decoding (finding the most likely state sequence) uses the **Viterbi algorithm** (dynamic programming, from chapter 05):

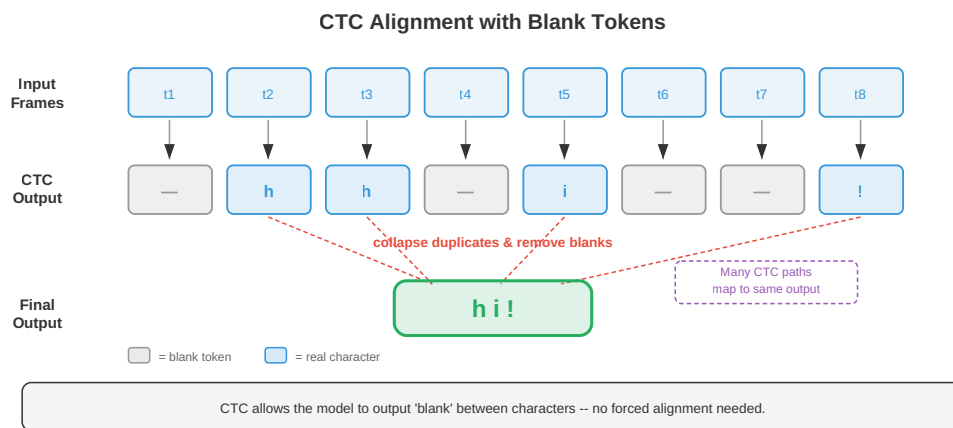
$$\delta_t(j) = \max_i \left[ \delta_{t-1}(i) \cdot a_{ij} \right] \cdot b_j(\mathbf{x}_t)$$

- where  $\delta_t(j)$  is the probability of the best path ending in state  $j$  at time  $t$ ,  $a_{ij}$  is the transition probability from state  $i$  to state  $j$ , and  $b_j(\mathbf{x}_t)$  is the emission probability of feature  $\mathbf{x}_t$  in state  $j$ .
- **DNN-HMM** (Hinton et al., 2012) replaced the GMM emission model with a deep neural network (DNN, from chapter 06) that predicts senone posterior probabilities  $p(s \mid \mathbf{x})$  from a window of feature frames. The HMM still handles

temporal structure and sequencing, but the neural network provides far more discriminative emission scores. This hybrid approach reduced word error rates by 20-30% relative to GMMs and was the dominant paradigm from 2012-2016.

- **WFST decoding** (Weighted Finite-State Transducer) is the standard decoding framework for traditional ASR. Each component (HMM topology  $H$ , context-dependency  $C$ , lexicon  $L$ , grammar/language model  $G$ ) is represented as a weighted finite-state transducer, and they are composed into a single search graph  $H \circ C \circ L \circ G$ . The Viterbi search then finds the lowest-cost path through this composed graph. WFSTs allow modular combination of knowledge sources and efficient dynamic programming search. The mathematical framework is from finite automata theory (related to the state machines in chapter 05).
- **End-to-end ASR** eliminates the separate components (pronunciation model, phoneme inventory, WFST decoder) and trains a single neural network that maps directly from audio features to characters or word pieces. The key challenge is the **alignment problem**: the input (hundreds of feature frames per second) and output (a few characters per second) have very different lengths, and the alignment between them is unknown during training.
- **Connectionist Temporal Classification** (CTC) (Graves et al., 2006) solves the alignment problem by introducing a special **blank** token and allowing the network to output any sequence of characters and blanks, as long as collapsing consecutive duplicates and removing blanks yields the correct transcript. For example, the transcript “cat” could be produced by the output sequence “-cc-aa-t-” (where “-” is blank).
- Formally, CTC defines a many-to-one mapping  $\mathcal{B}$  from the set of all length- $T$  output sequences (over the alphabet plus blank) to label sequences. The probability of a label sequence  $\mathbf{y}$  is the sum over all alignments that collapse to it:

$$P(\mathbf{y} | \mathbf{x}) = \sum_{\mathbf{\pi} \in \mathcal{B}^{-1}(\mathbf{y})} \prod_{t=1}^T p(\pi_t | \mathbf{x})$$



CTC alignment showing many possible paths through blank and character tokens that all collapse to the same output text

- Computing this sum naively would require enumerating exponentially many alignments, but the **CTC forward-backward algorithm** computes it efficiently in  $O(T \cdot |y|)$  using dynamic programming, analogous to the HMM forward-backward algorithm from chapter 05.
- CTC makes a **conditional independence assumption**: the output at each time step is independent of all other outputs given the input. This means CTC cannot model output dependencies (e.g., it cannot learn that “q” is almost always followed by “u” ). An external language model must be used to handle such dependencies.
- **CTC decoding** options:
  - **Greedy decoding**: take the most probable token at each time step, then collapse. Fast but suboptimal.
  - **Beam search**: maintain the top-k partial hypotheses at each step, merging hypotheses that collapse to the same prefix. Can incorporate a language model score.
  - **Prefix beam search**: a modified beam search that correctly handles the CTC blank merging, ensuring hypotheses are compared after collapsing.
- **RNN-Transducer (RNN-T)** (Graves, 2012) extends CTC by adding an explicit **prediction network** (a language model-like RNN) that conditions each output on the previous outputs, removing the conditional independence assumption. RNN-T has three components:
  - **Encoder**: processes the audio features to produce hidden representations  $\mathbf{h}_t^{\text{enc}}$  (typically a stack of LSTMs or Conformer layers).

- **Prediction network:** an autoregressive RNN that produces hidden representations  $\mathbf{h}_u^{\text{pred}}$  from the previously emitted labels.
- **Joint network:** combines the encoder and prediction network outputs at each (time, label) position and produces a distribution over the next token (including blank):

$$p(y | t, u) = \text{softmax}(W \cdot \tanh(W_{\text{enc}} \mathbf{h}_t^{\text{enc}} + W_{\text{pred}} \mathbf{h}_u^{\text{pred}} + b))$$

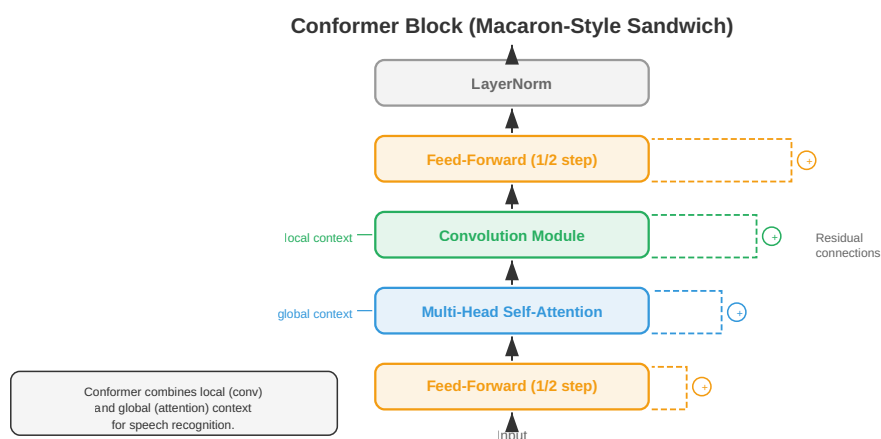
- RNN-T can emit zero or more labels per time step (by emitting non-blank tokens before advancing to the next time step, or emitting blank to advance without output). Training uses a forward-backward algorithm over the 2D (time, label) lattice, with complexity  $O(T \cdot U)$  where  $U$  is the output length. RNN-T is the dominant architecture for on-device streaming ASR (used in Google's Pixel phones and similar products) because it naturally supports streaming: the encoder processes audio left-to-right and the prediction network generates output incrementally.
- **Listen, Attend and Spell (LAS)** (Chan et al., 2016) is an attention-based encoder-decoder model (the sequence-to-sequence architecture from chapter 06). It has three components:
  - **Listener** (encoder): a pyramidal bidirectional LSTM that processes the full input sequence and downsamples by a factor of 8 (by concatenating pairs of consecutive hidden states at each layer), producing a shorter sequence of encoder hidden states.
  - **Attention:** at each decoder step, computes attention weights over all encoder states to form a context vector (the same attention mechanism from chapter 07).
  - **Speller** (decoder): an autoregressive LSTM that generates the output transcript one character at a time, conditioned on the context vector and previously generated characters.
- LAS achieves strong results but requires the full utterance to be available before decoding (because the attention attends to all encoder states), making it unsuitable for streaming applications. It also struggles with very long utterances because attention over long sequences becomes diffuse.



- The **Conformer** (Gulati et al., 2020) combines the local pattern-capturing ability of convolutions with the global dependency modelling of self-attention. Each Conformer block has four modules in a sandwich structure:

1. **Feed-forward module** (half-step): a feed-forward network with residual connection, using half the residual weight.
2. **Multi-head self-attention module**: standard transformer self-attention (from chapter 07) with relative positional encoding.
3. **Convolution module**: a pointwise convolution, a gated linear unit (GLU), a 1D depthwise convolution, batch normalisation, a Swish activation, and another pointwise convolution. The depthwise convolution captures local context (like an n-gram over the feature sequence).
4. **Feed-forward module** (half-step): identical to module 1.

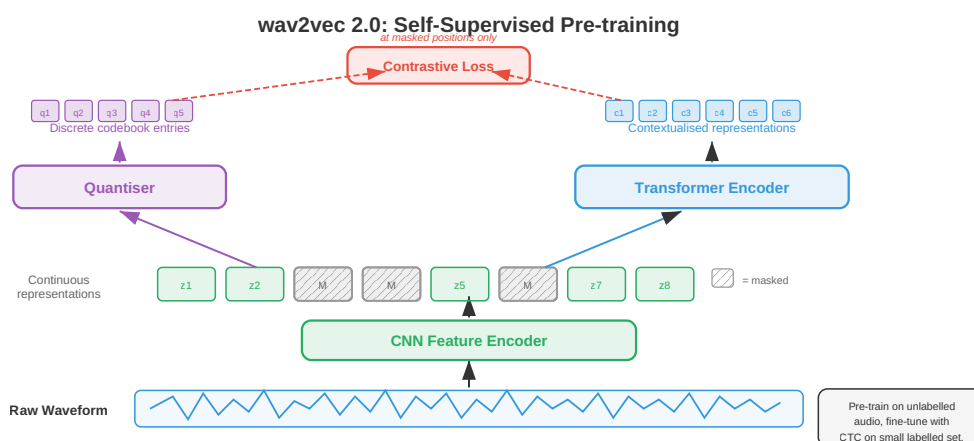
- The output is:  $\mathbf{y} = \text{LayerNorm}(\mathbf{x} + \frac{1}{2}\text{FFN}_1 + \text{MHSA} + \text{Conv} + \frac{1}{2}\text{FFN}_2)$ . The macaron-like structure (FFN-Attention-Conv-FFN) with half-step residuals was found empirically to outperform other orderings. Conformers have become the default encoder for both CTC and RNN-T systems, outperforming pure transformer and pure LSTM encoders.



Conformer block showing the sandwich structure of feed-forward, self-attention, convolution, and feed-forward modules

- **Whisper** (Radford et al., 2023) is OpenAI' s large-scale attention-based ASR model. It uses a standard encoder-decoder transformer architecture (from chapter 07) trained on 680,000 hours of weakly supervised data scraped from the internet (audio paired with approximate transcripts). Key design choices:
  - Input: 80-channel log-mel spectrogram (from file 01) with 25 ms windows and 10 ms hops, normalised to zero mean and unit variance.

- Encoder: standard transformer encoder with sinusoidal positional embeddings and pre-activation layer normalisation.
  - Decoder: transformer decoder that autoregressively generates tokens using a byte-level BPE tokenizer (from chapter 07).
  - Multitask: a single model handles transcription, translation, language identification, and timestamp prediction, conditioned on special task tokens in the decoder prompt.
  - The scale of the training data (rather than architectural novelty) is the primary driver of Whisper' s strong generalisation across domains, accents, and languages.
- **wav2vec 2.0** (Baevski et al., 2020) is a **self-supervised** pre-training framework for speech representations. The core idea is to learn speech representations from large amounts of unlabelled audio, then fine-tune with a small amount of labelled data. This follows the same self-supervised paradigm as BERT (from chapter 07) but adapted for continuous audio signals.
  - The wav2vec 2.0 architecture has three parts:
    - **Feature encoder**: a multi-layer 1D CNN that processes raw waveform samples and produces latent representations  $\mathbf{z}_t$  at a 20 ms frame rate (one vector every 320 samples at 16 kHz).
    - **Quantisation module**: discretises the latent representations into a finite codebook using **product quantisation** (dividing the vector into groups and quantising each group independently, choosing from G codebooks of V entries each). This produces targets  $\mathbf{q}_t$  for the contrastive learning objective.
    - **Context network**: a transformer encoder that takes the (partially masked) latent representations and produces contextualised representations  $\mathbf{c}_t$ .



wav2vec 2.0 architecture showing the CNN feature encoder, masking, transformer context network, and contrastive learning with quantised targets

- During pre-training, random spans of latent representations are **masked** (replaced with a learned mask embedding), and the model must identify the true quantised representation of the masked position from a set of distractors (negatives sampled from other positions in the same utterance). The contrastive loss is:

$$\mathcal{L} = -\log \frac{\exp(\text{sim}(\mathbf{c}_t, \mathbf{q}_t)/\kappa)}{\sum_{\mathbf{q} \in Q_t} \exp(\text{sim}(\mathbf{c}_t, \mathbf{q})/\kappa)}$$

- where sim is cosine similarity,  $\kappa$  is a temperature parameter, and  $Q_t$  includes the true quantised target plus distractors. An additional **diversity loss** encourages equal use of all codebook entries. This loss is essentially the InfoNCE contrastive loss, the same family of contrastive objectives used in visual self-supervised learning.
- After pre-training, a linear projection and CTC head are added on top, and the model is fine-tuned on labelled data. wav2vec 2.0 achieved near state-of-the-art results with only 10 minutes of labelled data (using 53,000 hours of unlabelled audio for pre-training), demonstrating the power of self-supervised learning for low-resource speech recognition.
- **HuBERT** (Hsu et al., 2021) is another self-supervised approach that replaces the contrastive objective with a **masked prediction** objective (predicting discrete cluster assignments of the masked frames). The targets are produced by an offline clustering step (k-means on MFCCs in the first iteration, then k-means on HuBERT features in subsequent iterations). HuBERT simplifies the training pipeline compared to wav2vec 2.0 (no quantisation module or contrastive sampling needed) and achieves comparable or better results.

- **Fast Conformer** (Rekesh et al., 2023, NVIDIA NeMo) replaces the quadratic self-attention in the standard Conformer with a **down-sampled attention** mechanism: the input sequence is compressed (typically  $8\times$  via strided convolution) before computing attention, then expanded back. This reduces the attention cost from  $O(T^2)$  to  $O(T^2/64)$  while retaining global context, enabling training on very long utterances (up to several minutes) without memory issues. Fast Conformer is the default encoder in NVIDIA's NeMo toolkit and forms the backbone of their production-grade models.
- **Parakeet** (NVIDIA, 2024) is a family of high-accuracy English ASR models built on the Fast Conformer encoder with CTC and RNN-T decoders, trained on 64,000 hours of English speech. Parakeet models (0.6B and 1.1B parameters) achieved the lowest word error rates on standard benchmarks at the time of release, surpassing Whisper large-v3 on most English test sets. The key ingredients are the efficient Fast Conformer architecture, aggressive data augmentation (SpecAugment, speed perturbation, noise mixing), and large-scale supervised training data — demonstrating that careful engineering of known components can still push the state of the art.
- **Canary** (NVIDIA, 2024) extends the NeMo framework to multilingual and multitask ASR. It uses a Fast Conformer encoder with an attention-based decoder (rather than CTC or RNN-T) and handles transcription plus translation across multiple languages in a single model (similar to Whisper's multitask design but with the more efficient Fast Conformer backbone). Canary models support English, German, Spanish, and French with competitive accuracy.
- **Moonshine** (Useful Sensors, 2024) is a family of ASR models specifically optimised for **on-device and edge deployment**. The encoder uses a hybrid architecture that replaces the initial transformer/conformer layers with a small CNN followed by a few transformer layers, dramatically reducing the model size (the base model is under 30M parameters). Moonshine targets real-time streaming on CPUs and low-power devices where Whisper would be too large and slow, trading some accuracy for  $5\text{-}10\times$  lower latency and memory footprint.
- **Distil-Whisper** (Gandhi et al., 2023) applies **knowledge distillation** (chapter 06) to compress Whisper into a smaller, faster model. The student model uses only 2 decoder layers (compared to Whisper's 32) while keeping the full encoder, and is trained to match Whisper's output distributions. Distil-Whisper

achieves within 1% WER of the teacher while being  $6\times$  faster, making it practical for real-time applications where the full Whisper model is too slow.

- **Universal Speech Model (USM)** (Zhang et al., 2023, Google) scales self-supervised pre-training to 12 million hours of unlabelled audio across 300+ languages, followed by supervised fine-tuning. USM demonstrates that the wav2vec 2.0 / self-supervised paradigm scales to truly massive data regimes, achieving strong performance on low-resource languages with very limited labelled data.
- **Massively Multilingual Speech (MMS)** (Pratap et al., 2023, Meta) extends wav2vec 2.0 pre-training to over 1,100 languages using religious recordings and other sources of multilingual audio. MMS covers far more languages than any previous ASR system, enabling speech recognition for many under-resourced languages for the first time.
- The landscape of modern ASR is converging on a few dominant patterns: (1) Conformer-family encoders with CTC or RNN-T for streaming, (2) encoder-decoder transformers for offline/multitask, (3) self-supervised pre-training for low-resource settings, and (4) scale — more data and larger models consistently improve accuracy. The choice between these depends on the deployment constraints: latency budget, compute available, number of languages, and whether the application is streaming or batch.
- **Language model integration** improves ASR by incorporating linguistic knowledge beyond what the acoustic model captures. The basic idea is to combine the acoustic model score  $p(\mathbf{x} | \mathbf{y})$  (how well the audio matches the transcript) with a language model score  $p(\mathbf{y})$  (how likely the transcript is as a sentence) during decoding.
- **Shallow fusion** combines the scores at beam search time:

⊙

$\mathbf{y}$

$= \arg$

max  
y

$$[\log p_{\text{AM}}(\mathbf{y} | \mathbf{x}) + \lambda \log p_{\text{LM}}(\mathbf{y})]$$

- where  $\lambda$  is a tunable weight and  $p_{\text{LM}}$  is an external language model (typically an n-gram or neural LM from chapter 07). This is simple and effective but requires the LM to operate on the same token vocabulary as the ASR model.
- **Deep fusion** (Gulcehre et al., 2015) integrates the language model inside the decoder network: the LM hidden state is concatenated with the decoder hidden state and passed through a gating mechanism before the output projection. The entire system (including the pre-trained LM) is fine-tuned jointly. This allows deeper integration but is more complex to train.
- **Cold fusion** (Sriram et al., 2018) is similar to deep fusion but trains the ASR decoder from scratch with the language model integrated, rather than fine-tuning a pre-trained decoder. This forces the acoustic model to learn complementary information rather than duplicating what the LM already knows.
- **Rescoring** (N-best rescoring) is a two-pass approach: first generate N candidate transcripts using beam search, then re-rank them using a more powerful language model (e.g., a large transformer LM). This is simple to implement and allows using very large LMs that would be too slow for first-pass decoding.
- **Internal language model estimation** (ILME) addresses a subtle problem: end-to-end models implicitly learn an internal LM from the training transcripts, which can conflict with the external LM during shallow fusion (essentially double-counting the linguistic prior). ILME estimates the internal LM and subtracts its score during fusion:

⌚

y

= arg

max  
y

$$[\log p_{\text{E2E}}(\mathbf{y} | \mathbf{x}) - \beta \log p_{\text{ILM}}(\mathbf{y}) + \lambda \log p_{\text{LM}}(\mathbf{y})]$$

- **Streaming vs. offline ASR** is a fundamental architectural choice. Offline (or batch) ASR processes the entire utterance before producing any output.

Streaming ASR produces output incrementally as audio arrives, with bounded latency.

- Streaming is essential for real-time applications: live captioning, voice assistants (the user expects a response before they finish speaking), telephone call transcription. The challenge is that some future context is helpful for recognition (knowing that the next word is “York” disambiguates “New” ), but streaming systems cannot wait for arbitrarily long future context.
- **Unidirectional encoders** (left-to-right LSTMs, causal convolutions, causal transformers) naturally support streaming because each output depends only on past and present input. Bidirectional encoders (which look at future context) do not support streaming directly.
- **Chunked attention** (also called blockwise or segmental attention) divides the input into fixed-length chunks and applies self-attention only within each chunk (and optionally to a few preceding chunks). This limits the latency to the chunk size plus processing time, while still allowing some local bidirectional context within each chunk. The tradeoff is that accuracy degrades as the chunk size decreases.
- **Lookahead** allows a streaming encoder to peek at a small number of future frames (e.g., 300-900 ms) before producing output for the current frame. This is implemented by adding a small right-context to the unidirectional computation. The lookahead window adds latency but significantly improves accuracy.
- **Latency** in streaming ASR has several components:
  - **Algorithmic latency**: the delay from when audio arrives to when the model can process it (determined by chunk size, lookahead, and feature extraction).
  - **Computational latency**: the time to run the model’s forward pass.
  - **Endpointer latency**: the delay in detecting that the user has finished speaking.
  - **First-token latency**: how quickly the first word appears. **Finalization latency**: how quickly the final output is confirmed (streaming systems often produce provisional output that gets corrected as more audio arrives).
- **Evaluation metrics** for ASR:



- **Word Error Rate** (WER) is the primary metric. It is computed by aligning the hypothesis (system output) to the reference (ground truth transcript) using edit distance (minimum number of substitutions, insertions, and deletions to transform one into the other), then:

$$\text{WER} = \frac{S + D + I}{N}$$

- where S is substitutions, D is deletions, I is insertions, and N is the total number of words in the reference. WER can exceed 100% if there are many insertions. A WER of 5% is considered roughly human-level for clean read speech; conversational or noisy speech is much harder (10-20%+).
- **Character Error Rate** (CER) is the same formula applied at the character level rather than the word level. CER is more informative for languages without clear word boundaries (Chinese, Japanese) and for evaluating how close near-misses are ( “cat” vs “bat” is 100% WER but 33% CER).
- **Word Information Lost** (WIL) and **Word Information Preserved** (WIP) are information-theoretic alternatives that account for the correlation between reference and hypothesis more precisely than WER, but they are less commonly reported.
- **Real-Time Factor** (RTF) measures computational efficiency: the ratio of processing time to audio duration.  $\text{RTF} < 1$  means the system runs faster than real time;  $\text{RTF} > 1$  means it cannot keep up with live audio. Streaming systems must maintain  $\text{RTF} < 1$ .
- **Data augmentation** is critical for robust ASR. Common techniques:
  - **Speed perturbation**: resampling audio at 0.9x and 1.1x speed (changing pitch and duration).
  - **SpecAugment** (Park et al., 2019): masking random frequency bands and time steps in the spectrogram. This is the audio analogue of dropout and is one of the most effective regularisation techniques for ASR. It requires no additional data.
  - **Noise augmentation**: mixing clean speech with recorded noise at various signal-to-noise ratios.
  - **Room impulse response simulation**: convolving clean speech with simulated room acoustics to simulate reverberant environments.



- **Tokenisation** for ASR determines the output vocabulary of the model. Options include:
  - **Characters**: simple, small vocabulary (~30 for English), but long output sequences and no implicit language modelling.
  - **Word pieces / BPE** (from chapter 07): subword units that balance vocabulary size and sequence length. The standard for modern systems (Whisper uses byte-level BPE with ~50,000 tokens).
  - **Words**: large vocabulary (50,000+), short output sequences, but cannot handle out-of-vocabulary words.
  - **Phonemes**: linguistically motivated, compact, but requires a pronunciation lexicon.
- The evolution of ASR can be summarised as a progression from heavily engineered modular systems (GMM-HMM + WFST decoding, 1990s-2010s) to hybrid systems (DNN-HMM, 2012-2016) to end-to-end systems that absorb more and more of the pipeline into a single neural network (CTC, RNN-T, LAS, 2016-2020) to large-scale pre-trained models that leverage vast amounts of unlabelled or weakly labelled data (wav2vec 2.0, Whisper, 2020-present). Each transition simplified the engineering while improving accuracy, following the broader trend in machine learning toward learning representations from data rather than hand-designing them (the same story told in chapter 06 for image features replaced by CNNs, and in chapter 07 for NLP features replaced by transformers).

## 42.1 Coding Tasks (use CoLab or notebook)

---

1. Implement CTC loss from scratch in JAX. Create a toy example with a short sequence of logits and a target label, compute the CTC forward algorithm to get the total probability, and compute the negative log-likelihood loss.

```
import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

def ctc_forward(log_probs, targets):
 """
 CTC forward algorithm (log-domain for numerical stability).
 log_probs: (T, V) log probabilities over vocabulary (index 0 = blank)
 targets: (U,) target label indices (no blanks)
 Returns: log probability of the target sequence under CTC.
```

```

"""
T, V = log_probs.shape
U = len(targets)

Build the extended label sequence with blanks: [blank, y1, blank, y2, ...,
yU, blank]
S = 2 * U + 1
labels = jnp.zeros(S, dtype=jnp.int32) # all blanks
for i in range(U):
 labels = labels.at[2 * i + 1].set(targets[i])

Initialise alpha (log domain)
NEG_INF = -1e30
alpha = jnp.full((T, S), NEG_INF)
alpha = alpha.at[0, 0].set(log_probs[0, labels[0]]) # start with blank
alpha = alpha.at[0, 1].set(log_probs[0, labels[1]]) # or first label

Fill forward
for t in range(1, T):
 for s in range(S):
 # Same state
 a = alpha[t - 1, s]
 # From previous state
 if s > 0:
 a = jnp.logaddexp(a, alpha[t - 1, s - 1])
 # Skip blank (if current and two-back labels are different)
 if s > 1 and labels[s] != 0 and labels[s] != labels[s - 2]:
 a = jnp.logaddexp(a, alpha[t - 1, s - 2])
 alpha = alpha.at[t, s].set(a + log_probs[t, labels[s]])

Total log probability: sum of last two states at final time step
log_prob = jnp.logaddexp(alpha[T - 1, S - 1], alpha[T - 1, S - 2])
return log_prob, alpha

--- Toy example ---
T = 12 # input length (time steps)
V = 5 # vocab size (0=blank, 1='c', 2='a', 3='t', 4='x')
targets = jnp.array([1, 2, 3]) # "c", "a", "t"

Create random logits and convert to log-probabilities
key = jax.random.PRNGKey(42)
logits = jax.random.normal(key, (T, V))
log_probs = jax.nn.log_softmax(logits, axis=-1)

log_prob, alpha = ctc_forward(log_probs, targets)
ctc_loss = -log_prob

```

```

print(f"Target sequence: {targets.tolist()} ('c', 'a', 't')")
print(f"Input length T={T}, Vocab size V={V}")
print(f"CTC log-probability: {log_prob:.4f}")
print(f"CTC loss (neg log-prob): {ctc_loss:.4f}")

Visualise the forward variable (alpha) lattice
fig, ax = plt.subplots(figsize=(12, 5))
Convert from log to linear for visualisation
alpha_linear = jnp.exp(alpha - jnp.max(alpha)) # normalise for visibility
im = ax.imshow(alpha_linear.T, aspect='auto', origin='lower', cmap='viridis')
ax.set_xlabel('Time step (t)')
ax.set_ylabel('Extended label index (s)')

label_names = ['_', 'c', '_', 'a', '_', 't', '_'] # _ = blank
ax.set_yticks(range(len(label_names)))
ax.set_yticklabels(label_names)
ax.set_title(f'CTC Forward Variable (alpha lattice) | Loss = {ctc_loss:.2f}')
plt.colorbar(im, ax=ax, label='Normalised probability')
plt.tight_layout(); plt.show()

```

1. Build a simple encoder-decoder attention-based ASR model (a minimal LAS-like architecture) in JAX. Use a 1D convolution encoder and a single-layer decoder with dot-product attention. Run it on synthetic data and visualise the attention weights.

```

import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

--- Minimal attention-based encoder-decoder for ASR ---

def init_params(key, input_dim, hidden_dim, vocab_size):
 """Initialise parameters for a tiny LAS-like model."""
 keys = jax.random.split(key, 8)
 scale = 0.1
 params = {
 # Encoder: simple linear projection (simulating conv output)
 'enc_w': jax.random.normal(keys[0], (input_dim, hidden_dim)) * scale,
 'enc_b': jnp.zeros(hidden_dim),
 # Attention: query, key, value projections
 'attn_q': jax.random.normal(keys[1], (hidden_dim, hidden_dim)) * scale,
 'attn_k': jax.random.normal(keys[2], (hidden_dim, hidden_dim)) * scale,
 'attn_v': jax.random.normal(keys[3], (hidden_dim, hidden_dim)) * scale,
 # Decoder RNN (simple Elman RNN for illustration)
 'dec_wh': jax.random.normal(keys[4], (hidden_dim, hidden_dim)) * scale,
 'dec_wx': jax.random.normal(keys[5], (vocab_size, hidden_dim)) * scale,
 }

```

```

 'dec_wc': jax.random.normal(keys[6], (hidden_dim, hidden_dim)) * scale,
 'dec_b': jnp.zeros(hidden_dim),
 # Output projection
 'out_w': jax.random.normal(keys[7], (hidden_dim, vocab_size)) * scale,
 'out_b': jnp.zeros(vocab_size),
 }
 return params

def encode(params, x):
 """Encoder: linear projection (placeholder for conv/LSTM stack)."""
 return jnp.tanh(x @ params['enc_w'] + params['enc_b'])

def attend(params, query, enc_out):
 """Dot-product attention over encoder outputs."""
 q = query @ params['attn_q'] # (hidden,)
 k = enc_out @ params['attn_k'] # (T_enc, hidden)
 v = enc_out @ params['attn_v'] # (T_enc, hidden)
 d_k = q.shape[-1]
 scores = (k @ q) / jnp.sqrt(d_k) # (T_enc,)
 weights = jax.nn.softmax(scores) # (T_enc,)
 context = weights @ v # (hidden,)
 return context, weights

def decode_step(params, h_prev, y_prev_onehot, enc_out):
 """Single decoder step: RNN + attention."""
 # Embed previous token
 y_emb = y_prev_onehot @ params['dec_wx'] # (hidden,)
 # Attend to encoder
 context, attn_w = attend(params, h_prev, enc_out)
 # RNN update
 h = jnp.tanh(h_prev @ params['dec_wh'] + y_emb + context @ params['dec_wc']
 + params['dec_b'])
 # Output logits
 logits = h @ params['out_w'] + params['out_b']
 return h, logits, attn_w

--- Setup ---
key = jax.random.PRNGKey(0)
input_dim = 40 # e.g., 40 mel bands
hidden_dim = 64
vocab_size = 10 # small vocab for demo
T_enc = 30 # encoder time steps
T_dec = 8 # decoder steps

params = init_params(key, input_dim, hidden_dim, vocab_size)

Synthetic input: random mel-like features

```

```

key, subkey = jax.random.split(key)
x = jax.random.normal(subkey, (T_enc, input_dim))

Encode
enc_out = encode(params, x)

Decode (teacher forcing with random targets)
key, subkey = jax.random.split(key)
targets = jax.random.randint(subkey, (T_dec,), 0, vocab_size)

h = jnp.zeros(hidden_dim)
all_logits = []
all_attn = []

for t in range(T_dec):
 y_prev = jax.nn.one_hot(targets[t] if t > 0 else 0, vocab_size)
 h, logits, attn_w = decode_step(params, h, y_prev, enc_out)
 all_logits.append(logits)
 all_attn.append(attn_w)

all_attn = jnp.stack(all_attn) # (T_dec, T_enc)
all_logits = jnp.stack(all_logits) # (T_dec, vocab_size)

--- Visualise attention weights ---
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

im = axes[0].imshow(all_attn, aspect='auto', cmap='Blues', origin='lower')
axes[0].set_xlabel('Encoder time step')
axes[0].set_ylabel('Decoder step')
axes[0].set_title('Attention Weights (decoder -> encoder)')
plt.colorbar(im, ax=axes[0])

Show predicted token distribution for each decoder step
im2 = axes[1].imshow(jax.nn.softmax(all_logits, axis=-1), aspect='auto',
 cmap='Oranges', origin='lower')
axes[1].set_xlabel('Vocabulary index')
axes[1].set_ylabel('Decoder step')
axes[1].set_title('Output Token Probabilities')
plt.colorbar(im2, ax=axes[1])

plt.suptitle('Minimal Attention-based ASR Model (untrained)')
plt.tight_layout(); plt.show()

```

1. Compute Word Error Rate (WER) from scratch using dynamic programming (edit distance), and evaluate multiple hypotheses against a reference. Visualise the edit distance matrix.

```

import jax.numpy as jnp
import matplotlib.pyplot as plt
import numpy as np

def compute_wer(reference, hypothesis):
 """
 Compute WER using dynamic programming (Levenshtein distance on words).
 Returns WER, number of substitutions, deletions, insertions, and the DP
 matrix.
 """
 ref_words = reference.split()
 hyp_words = hypothesis.split()
 N = len(ref_words)
 M = len(hyp_words)

 # DP matrix: d[i][j] = edit distance between ref[:i] and hyp[:j]
 d = np.zeros((N + 1, M + 1), dtype=np.int32)
 # Backtrack matrix to count S, D, I
 ops = np.zeros((N + 1, M + 1, 3), dtype=np.int32) # [sub, del, ins]

 for i in range(N + 1):
 d[i][0] = i # all deletions
 for j in range(M + 1):
 d[0][j] = j # all insertions

 for i in range(1, N + 1):
 for j in range(1, M + 1):
 if ref_words[i - 1] == hyp_words[j - 1]:
 sub_cost = d[i - 1][j - 1] # match, no edit
 else:
 sub_cost = d[i - 1][j - 1] + 1 # substitution
 del_cost = d[i - 1][j] + 1 # deletion
 ins_cost = d[i][j - 1] + 1 # insertion

 d[i][j] = min(sub_cost, del_cost, ins_cost)

 # Backtrack to count operations
 i, j = N, M
 S, D, I = 0, 0, 0
 while i > 0 or j > 0:
 if i > 0 and j > 0 and d[i][j] == d[i-1][j-1] and ref_words[i-1] ==
hyp_words[j-1]:
 i -= 1; j -= 1 # correct
 elif i > 0 and j > 0 and d[i][j] == d[i-1][j-1] + 1:
 S += 1; i -= 1; j -= 1 # substitution
 elif i > 0 and d[i][j] == d[i-1][j] + 1:
 D += 1; i -= 1 # deletion

```

```

 elif j > 0 and d[i][j] == d[i][j-1] + 1:
 I += 1; j -= 1 # insertion
 else:
 break

wer = (S + D + I) / N if N > 0 else 0.0
return wer, S, D, I, d

--- Test cases ---
reference = "the cat sat on the mat"
hypotheses = [
 "the cat sat on the mat", # perfect
 "the cat sit on the mat", # 1 substitution
 "the cat on the mat", # 1 deletion
 "the big cat sat on the mat", # 1 insertion
 "a dog sat in a rug", # multiple errors
]

print(f"Reference: '{reference}'\n")
print(f"{'Hypothesis':<40s} {'WER':>6s} {'S':>3s} {'D':>3s} {'I':>3s}")
print("-" * 60)
results = []
for hyp in hypotheses:
 wer, S, D, I, dp = compute_wer(reference, hyp)
 results.append((hyp, wer, S, D, I, dp))
 print(f"{'hyp':<40s} {'wer':>6.1%} {'S':>3d} {'D':>3d} {'I':>3d}")

Visualise the DP matrix for the worst case
worst = results[-1]
hyp_words = worst[0].split()
ref_words = reference.split()
dp_matrix = worst[5]

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

DP matrix
im = axes[0].imshow(dp_matrix, cmap='YlOrRd', origin='upper')
axes[0].set_xticks(range(len(hyp_words) + 1))
axes[0].set_xticklabels([''] + hyp_words, rotation=45, ha='right', fontsize=9)
axes[0].set_yticks(range(len(ref_words) + 1))
axes[0].set_yticklabels([''] + ref_words, fontsize=9)
axes[0].set_xlabel('Hypothesis words')
axes[0].set_ylabel('Reference words')
axes[0].set_title(f'Edit Distance Matrix\nWER = {worst[1]:.1%}')
for i in range(dp_matrix.shape[0]):
 for j in range(dp_matrix.shape[1]):
 axes[0].text(j, i, str(dp_matrix[i, j]), ha='center', va='center',

```

```

fontsize=8)
 plt.colorbar(im, ax=axes[0])

 # WER comparison bar chart
 names = [f'Hyp {i+1}' for i in range(len(results))]
 wers = [r[1] * 100 for r in results]
 colors = ['#27ae60' if w == 0 else '#f39c12' if w < 30 else '#e74c3c' for w in
wers]

 axes[1].barh(names, wers, color=colors)
 axes[1].set_xlabel('WER (%)')
 axes[1].set_title('Word Error Rate Comparison')
 for i, (w, r) in enumerate(zip(wers, results)):
 axes[1].text(w + 1, i, f'{w:.0f}% (S={r[2]}, D={r[3]}, I={r[4]})',
 va='center', fontsize=9)
 axes[1].set_xlim(0, max(wers) * 1.4)

 plt.tight_layout(); plt.show()

```

1. Implement SpecAugment (frequency masking and time masking) on a log-mel spectrogram and visualise the original vs. augmented versions. Generate the spectrogram from a synthetic signal.

```

import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

--- Generate synthetic log-mel spectrogram ---
key = jax.random.PRNGKey(42)
fs = 16000
duration = 2.0
t = jnp.arange(0, duration, 1.0 / fs)

Simulate speech: chirp signal with harmonics
f0 = 120.0
x = sum(jnp.sin(2 * jnp.pi * f0 * k * t * (1 + 0.1 * t)) / k for k in range(1,
10))

key, subkey = jax.random.split(key)
x = x + 0.05 * jax.random.normal(subkey, t.shape)

Compute log-mel spectrogram (simplified)
frame_len = 400 # 25 ms
hop_len = 160 # 10 ms
n_fft = 512
n_mels = 80

n_frames = (len(x) - frame_len) // hop_len + 1

```



```

 hamming = 0.54 - 0.46 * jnp.cos(2 * jnp.pi * jnp.arange(frame_len) / (frame_len
- 1))

 frames = jnp.stack([x[i * hop_len : i * hop_len + frame_len] for i in
range(n_frames)])
 windowed = frames * hamming
 spectra = jnp.abs(jnp.fft.rfft(windowed, n=n_fft)) ** 2

Simple mel filterbank
 def hz_to_mel(f): return 2595 * jnp.log10(1 + f / 700)
 def mel_to_hz(m): return 700 * (10 ** (m / 2595) - 1)

 mel_points = jnp.linspace(hz_to_mel(0), hz_to_mel(fs / 2), n_mels + 2)
 hz_pts = mel_to_hz(mel_points)
 bins = jnp.floor((n_fft + 1) * hz_pts / fs).astype(jnp.int32)

 n_freqs = n_fft // 2 + 1
 fb = jnp.zeros((n_mels, n_freqs))
 for m in range(n_mels):
 lo, mid, hi = int(bins[m]), int(bins[m+1]), int(bins[m+2])
 for k in range(lo, mid):
 if mid != lo:
 fb = fb.at[m, k].set((k - lo) / (mid - lo))
 for k in range(mid, hi):
 if hi != mid:
 fb = fb.at[m, k].set((hi - k) / (hi - mid))

 log_mel = jnp.log(spectra @ fb.T + 1e-10)

--- SpecAugment ---
 def spec_augment(spec, key, n_freq_masks=2, freq_mask_width=15,
 n_time_masks=2, time_mask_width=25):
 """Apply SpecAugment: frequency and time masking."""
 augmented = spec.copy()
 T, F = spec.shape

 # Frequency masking
 for _ in range(n_freq_masks):
 key, k1, k2 = jax.random.split(key, 3)
 f_width = jax.random.randint(k1, (), 1, freq_mask_width + 1)
 f_start = jax.random.randint(k2, (), 0, max(1, F - freq_mask_width))
 mask = (jnp.arange(F) >= f_start) & (jnp.arange(F) < f_start + f_width)
 augmented = jnp.where(mask[None, :], 0.0, augmented)

 # Time masking
 for _ in range(n_time_masks):
 key, k1, k2 = jax.random.split(key, 3)

```

```

 t_width = jax.random.randint(k1, (), 1, time_mask_width + 1)
 t_start = jax.random.randint(k2, (), 0, max(1, T - time_mask_width))
 mask = (jnp.arange(T) >= t_start) & (jnp.arange(T) < t_start + t_width)
 augmented = jnp.where(mask[:, None], 0.0, augmented)

 return augmented

key, subkey = jax.random.split(key)
log_mel_aug = spec_augment(log_mel, subkey)

--- Visualise ---
fig, axes = plt.subplots(2, 1, figsize=(14, 8))

im0 = axes[0].imshow(log_mel.T, aspect='auto', origin='lower', cmap='inferno',
 extent=[0, duration, 0, n_mels])
axes[0].set_title('Original Log-Mel Spectrogram')
axes[0].set_xlabel('Time (s)'); axes[0].set_ylabel('Mel Band')
plt.colorbar(im0, ax=axes[0], label='Log Energy')

im1 = axes[1].imshow(log_mel_aug.T, aspect='auto', origin='lower',
 cmap='inferno',
 extent=[0, duration, 0, n_mels])
axes[1].set_title('After SpecAugment (frequency + time masking)')
axes[1].set_xlabel('Time (s)'); axes[1].set_ylabel('Mel Band')
plt.colorbar(im1, ax=axes[1], label='Log Energy')

plt.tight_layout(); plt.show()

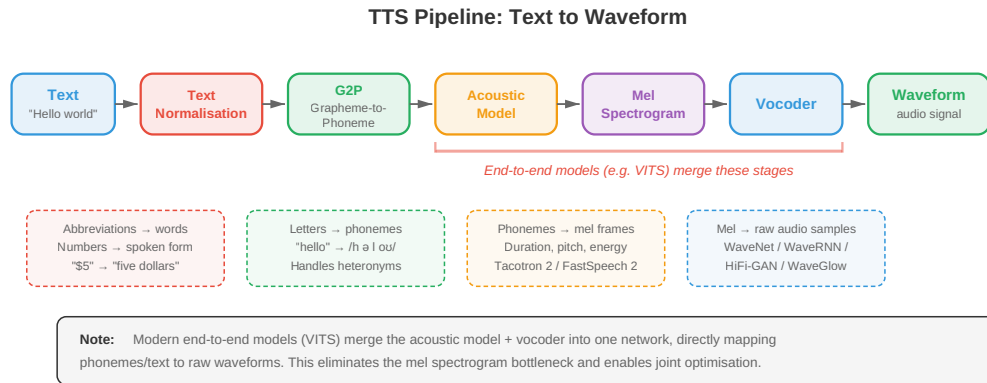
```

## 43 Text to Speech and Voice

Text-to-speech synthesis reverses the ASR pipeline, generating natural-sounding audio from written text. This file covers the TTS pipeline (text normalisation, G2P, acoustic models, vocoders), Tacotron, WaveNet, HiFi-GAN, voice cloning, voice conversion, and voice activity detection (VAD).

- In file 01, we built the signal-processing toolkit: waveforms, spectrograms, mel filterbanks, and MFCCs. In file 02, we turned speech into text. Now we reverse the arrow: given text, synthesise natural-sounding speech. This is **text-to-speech (TTS)**, a problem that also opens the door to voice conversion, voice cloning, and voice activity detection.
- Think of TTS like a stage performance. The script is the text input. A director (the acoustic model) decides how each line should sound, its pitch, timing,

emphasis. The orchestra (the vocoder) then performs the score, producing the actual sound waves the audience hears. Modern neural TTS replaces the stiff, robotic delivery of rule-based systems with performances that rival human speakers.



TTS pipeline: text is normalised, converted to phonemes, processed by an acoustic model to produce a mel spectrogram, then passed through a vocoder to generate the final waveform

- **Text-to-speech pipeline** the standard TTS pipeline has four stages: (1) text normalisation, (2) phoneme conversion, (3) acoustic model, and (4) vocoder. Some modern systems collapse stages 3 and 4 into a single end-to-end model, but the conceptual decomposition remains useful.
- **Text normalisation** converts raw text into a pronounceable form. Abbreviations expand ( “Dr.” to “Doctor” ), numbers become words ( “1984” to “nineteen eighty-four” ), currency symbols are verbalised ( “\$5” to “five dollars” ), and URLs or special characters are handled. This stage is often rule-based with language-specific grammars, though neural normalisation models exist. Errors here propagate to every downstream stage: if “St.” is read as “saint” instead of “street” , the entire utterance is wrong.
- **Grapheme-to-phoneme (G2P) conversion** maps normalised text to a phoneme sequence. English is notoriously irregular ( “though” , “through” , “tough” all use “ough” differently), so dictionary lookup (the CMU Pronouncing Dictionary) handles common words while a neural sequence-to-sequence model (chapter 06’ s encoder-decoder or chapter 07’ s transformer) handles out-of-vocabulary words. Languages with shallow orthographies (Spanish, Finnish) need simpler G2P. The output is typically an IPA (International Phonetic Alphabet) sequence or an equivalent internal phoneme set.

- **Acoustic models** consume the phoneme sequence and produce an intermediate acoustic representation, almost always a **mel spectrogram** (file 01). The mel spectrogram captures the spectral envelope at each time frame, which encodes the perceptually relevant information a vocoder needs to reconstruct the waveform. The acoustic model must decide timing (how long each phoneme lasts), pitch (fundamental frequency  $F_0$ ), and energy (loudness).
- **Vocoders** take the mel spectrogram and produce the raw audio waveform. This is an ill-posed inversion problem: many waveforms can produce the same spectrogram because phase information was discarded. Classical vocoders (Griffin-Lim, WORLD) use iterative or signal-model approaches, but neural vocoders now dominate in quality.
- **Vocoders: WaveNet** (van den Oord et al., 2016) was the first neural vocoder to produce speech nearly indistinguishable from human recordings. It models the waveform autoregressively, predicting each sample  $x_t$  conditioned on all previous samples:

$$P(x) = \prod_{t=1}^T P(x_t \mid x_1, \dots, x_{t-1}, c)$$

- where  $c$  is the conditioning signal (mel spectrogram). Each sample is 16-bit, so a naive softmax over 65536 values is impractical. WaveNet uses **mu-law companding** to reduce to 256 quantisation levels, or later variants use a mixture of logistics distribution.
- WaveNet's core building block is the **dilated causal convolution**. Causal means filter weights only look at past samples (no future leakage). Dilated means the filter skips samples with exponentially increasing gaps: dilation factors  $1, 2, 4, 8, \dots, 512$ . This gives an exponentially large receptive field while keeping the parameter count linear.
- The gated activation for each layer is:

$$z = \tanh(W_f * x) \odot \sigma(W_g * x)$$

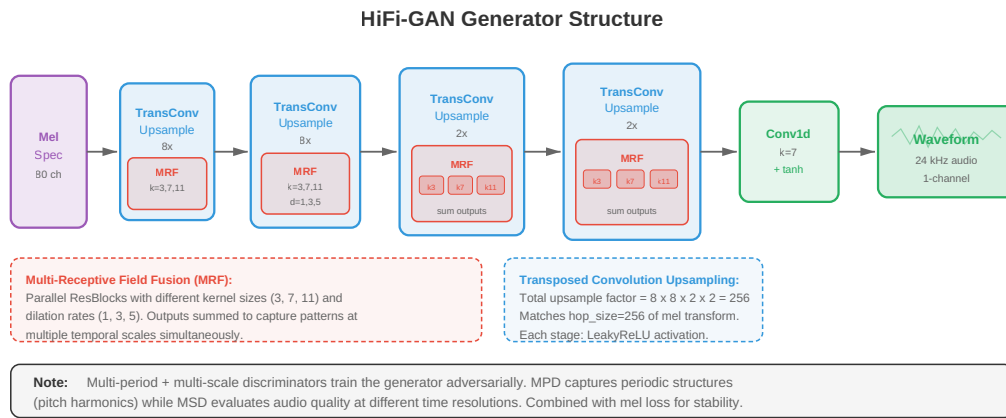
- where  $W_f$  and  $W_g$  are filter and gate convolution weights,  $*$  denotes dilated causal convolution, and  $\odot$  is element-wise multiplication. This gating

mechanism (from chapter 06's LSTMs) allows the network to control information flow.

- WaveNet produces exceptional quality but is painfully slow at inference: generating one second of 24 kHz audio requires 24000 sequential forward passes. This motivated all subsequent vocoder research.
- **WaveRNN** (Kalchbrenner et al., 2018) replaces WaveNet's deep convolutional stack with a single-layer recurrent network. It splits each 16-bit sample into coarse (upper 8 bits) and fine (lower 8 bits) components, predicting each with a GRU (chapter 06). This dual softmax approach reduces computation significantly while maintaining high quality. WaveRNN is fast enough for real-time on mobile CPUs with careful kernel optimisation.
- **WaveGlow** (Prenger et al., 2019) is a **flow-based** vocoder that avoids autoregressive generation entirely. It uses a sequence of invertible transformations (affine coupling layers, chapter 06's normalising flows) to map a simple Gaussian distribution to the waveform distribution. Training maximises the exact log-likelihood using the change-of-variables formula:

$$\log P(x) = \log P(z) + \sum_i \log \left| \det \frac{\partial f_i}{\partial f_{i-1}} \right|$$

- where  $z = f(x)$  is the latent variable obtained by passing  $x$  through the flow. At inference, a sample  $z \sim \mathcal{N}(0, I)$  is drawn and pushed through the inverted flow in a single parallel pass. WaveGlow trades model size (large networks for the coupling layers) for generation speed.
- **HiFi-GAN** (Kong et al., 2020) uses a **generative adversarial network** to synthesise waveforms from mel spectrograms. The generator upsamples the mel spectrogram through a series of transposed convolutions, each followed by a **multi-receptive field fusion (MRF)** module. The MRF module applies multiple residual blocks with different kernel sizes and dilation rates in parallel, then sums their outputs. This allows the generator to capture patterns at multiple time scales simultaneously.



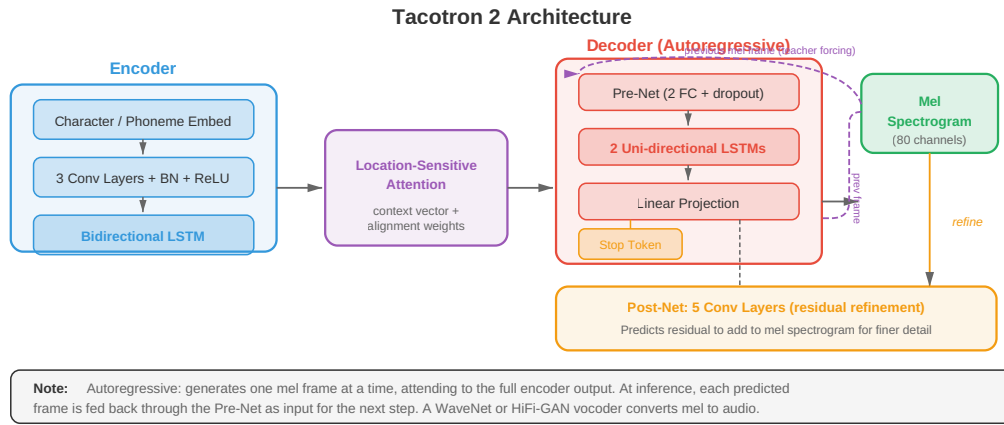
HiFi-GAN generator architecture: mel spectrogram input passes through transposed convolution upsampling layers, each followed by multi-receptive field fusion blocks that combine parallel residual stacks with different dilation patterns

- HiFi-GAN uses two discriminator types. The **multi-period discriminator (MPD)** reshapes the 1D waveform into 2D by folding it at different periods (2, 3, 5, 7, 11), then applies 2D convolutions. This captures periodic structures at different fundamental frequencies. The **multi-scale discriminator (MSD)** operates on the raw waveform, 2x downsampled, and 4x downsampled versions, capturing patterns at different temporal resolutions.
- The training objective combines adversarial loss, **mel spectrogram reconstruction loss** (L1 distance between the mel spectrogram of synthesised and ground truth audio), and **feature matching loss** (L1 distance between intermediate discriminator features):

$$\mathcal{L}_G = \mathcal{L}_{\text{adv}}(G) + \lambda_{\text{mel}} \mathcal{L}_{\text{mel}}(G) + \lambda_{\text{fm}} \mathcal{L}_{\text{fm}}(G)$$

- HiFi-GAN achieves synthesis quality comparable to WaveNet while being over 1000x faster, enabling real-time generation on a single GPU.
- Neural source-filter (NSF) models** combine traditional signal processing with neural networks. In the classical source-filter model, voiced speech is produced by a source excitation (periodic pulse train at the fundamental frequency  $F_0$ ) passed through a vocal tract filter (the spectral envelope). NSF models replace the handcrafted filter with a neural network while keeping the explicit source signal. The input  $F_0$  contour provides fine pitch control that purely data-driven vocoders sometimes struggle with.

- **Acoustic models: Tacotron** (Wang et al., 2017) was the first end-to-end neural TTS system that directly converted character sequences to mel spectrograms. It uses an encoder-decoder architecture with attention (chapter 07). The encoder processes the character/phoneme sequence with a convolution bank, highway network, and bidirectional GRU. The decoder is an autoregressive GRU that predicts mel frames one at a time, using the previous frame and the attention context as input.
- **Tacotron 2** (Shen et al., 2018) refines the architecture significantly. The encoder is a 3-layer 1D convolution stack followed by a bidirectional LSTM (chapter 06). The decoder is a 2-layer LSTM with **location-sensitive attention**, which conditions the attention mechanism not only on the encoder outputs and decoder state but also on the cumulative attention weights from previous steps. This prevents the common failure mode of attention skipping or repeating words.



Tacotron 2 architecture: character/phoneme encoder with convolution layers and BiLSTM, location-sensitive attention aligning to mel spectrogram frames, autoregressive decoder with stop token prediction

- The location-sensitive attention energy for encoder position  $j$  at decoder step  $i$  is:

$$e_{i,j} = w^T \tanh(W_s s_{i-1} + W_h h_j + W_f f_{i,j} + b)$$

- where  $s_{i-1}$  is the previous decoder state,  $h_j$  is the encoder output at position  $j$ , and  $f_{i,j}$  is the location feature obtained by convolving the cumulative attention weights  $\sum_{k < i} \alpha_{k,j}$  with a 1D convolution filter. The attention weights are  $\alpha_{i,j} = \text{softmax}(e_{i,j})$ .

- Tacotron 2' s decoder also predicts a **stop token** probability at each step, indicating when the mel spectrogram is complete. The output mel spectrogram is then passed to a vocoder (originally WaveNet, later replaced by HiFi-GAN or similar).
- The autoregressive nature of Tacotron 2 means synthesis speed is limited by the number of mel frames. For a typical 80-frame-per-second mel spectrogram, a 5-second utterance requires 400 sequential decoder steps.
- **FastSpeech** (Ren et al., 2019) solves the speed problem with a **non-autoregressive** acoustic model. Instead of generating mel frames sequentially, FastSpeech generates all frames in parallel. The key challenge is determining how many mel frames each phoneme should produce, which FastSpeech handles with a **duration predictor**.
- The duration predictor is a small convolutional network that predicts the integer duration (number of mel frames) for each phoneme. During training, ground-truth durations are extracted from a pre-trained autoregressive teacher model (Tacotron 2) using its attention alignments. During inference, the predicted durations are used to expand the phoneme-level hidden sequence to the frame level using a **length regulator** that simply repeats each phoneme' s hidden representation for the predicted number of frames.
- **FastSpeech 2** (Ren et al., 2021) improves on FastSpeech by removing the teacher-student distillation. It extracts ground-truth durations directly using forced alignment (from file 02' s acoustic model frameworks) and adds explicit **variance adaptors** for pitch ( $F_0$ ) and energy in addition to duration. Each adaptor is a small convolutional predictor whose output conditions the decoder:

```
\begin{aligned}
\hat{d}_i &= \text{DurationPredictor}(h_i) \\\
\hat{p}_i &= \text{PitchPredictor}(h_i) \\\
\hat{e}_i &= \text{EnergyPredictor}(h_i)
\end{aligned}
```

- where  $h_i$  is the encoder hidden state for phoneme  $i$ . At training time, ground-truth values are used; at inference, the predicted values give explicit control over prosody. This controllability is a major advantage of FastSpeech 2: adjusting pitch, speed, or energy is as simple as scaling the predictor outputs.



- FastSpeech 2 is typically 10-20x faster than Tacotron 2 at inference and avoids common autoregressive failure modes like word skipping, repetition, and attention collapse.
- **VITS** (Kim et al., 2021) is an **end-to-end** TTS model that directly generates waveforms from text, eliminating the separate vocoder stage. VITS combines a conditional variational autoencoder (chapter 06) with normalising flows and adversarial training. The posterior encoder maps ground-truth mel spectrograms to a latent space, the prior encoder maps phonemes (through a transformer-based text encoder and duration predictor) to the same latent space, and the decoder (HiFi-GAN-based) generates waveforms from latent samples.
- The training objective for VITS combines:
  - **Reconstruction loss:** the VAE forces the latent distribution to encode acoustic information
  - **KL divergence:** aligns the text-conditioned prior with the audio-conditioned posterior
  - **Adversarial loss:** discriminators ensure waveform quality
  - **Duration loss:** trains the stochastic duration predictor
- VITS produces higher quality than two-stage systems (FastSpeech 2 + HiFi-GAN) because the acoustic model and vocoder are jointly optimised, avoiding the mismatch between predicted and ground-truth mel spectrograms that degrades two-stage systems.
- **VALL-E** (Wang et al., 2023) radically reframes TTS as a **language modelling problem** over discrete audio tokens. It uses a neural audio codec (EnCodec) to represent speech as a sequence of discrete codes from multiple codebook levels. Given a text prompt and a 3-second enrollment utterance (also encoded as discrete tokens), VALL-E uses a transformer language model to predict the audio tokens autoregressively.
- VALL-E uses two models: an **autoregressive (AR) model** that generates the first codebook level token-by-token, and a **non-autoregressive (NAR) model** that predicts the remaining codebook levels in parallel, conditioned on the first level and each other. This codec language model approach enables remarkable zero-shot voice cloning: a 3-second sample is enough to reproduce a speaker's voice, timbre, and even emotional tone.

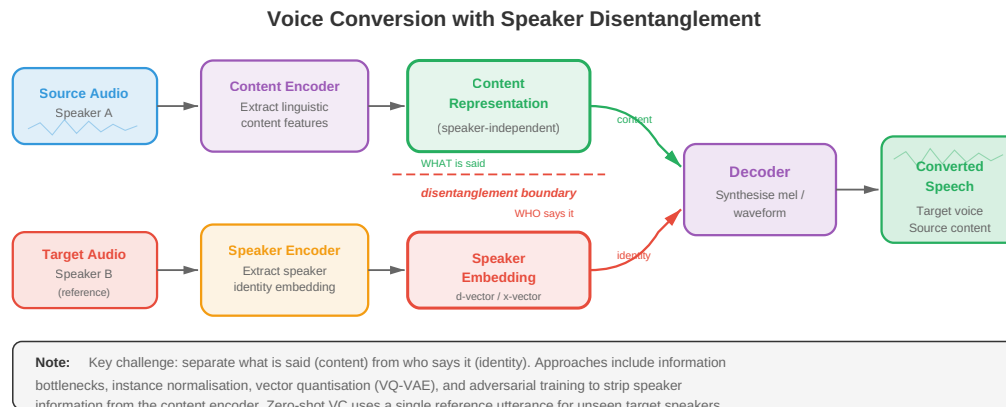
- **StyleTTS** (Li et al., 2022) and **StyleTTS 2** disentangle speech into content and style components. A style encoder extracts a style vector from reference audio, capturing speaker identity, prosody, and recording conditions. During inference, style can be sampled from a learned prior distribution or transferred from a reference utterance. StyleTTS 2 uses diffusion models (chapter 08) for the style prior, generating diverse and natural prosody.
- **Kokoro** (2024) is a lightweight, high-quality open-source TTS model notable for its small size (~82M parameters) and impressive naturalness. It uses a StyleTTS 2-inspired architecture with a diffusion-based style prior and a fine-tuned ISTFTNet vocoder that directly predicts STFT coefficients (from file 01) rather than raw waveform samples. Despite being a fraction of the size of models like VALL-E, Kokoro achieves near-human naturalness for English, Japanese, French, Korean, and Chinese, demonstrating that carefully curated training data and efficient architecture design can compete with brute-force scale. Kokoro's small footprint makes it practical for local and edge deployment.
- **Orpheus** (Canopy Labs, 2025) is a family of open-source TTS models (1B and 3B parameters) built on the **codec language model** paradigm pioneered by VALL-E. Orpheus takes the idea further with an LLM backbone (fine-tuned Llama 3) that generates SNAC audio codec tokens directly. Its standout feature is human-like emotional expressiveness: it handles laughter, sighs, hesitations, and affective prosody with remarkable naturalness. Orpheus can be prompted with tags like [laugh] or [sigh] in the input text, giving fine-grained control over paralinguistic expression.
- **Dia** (Nari Labs, 2025) is an open-source dialogue TTS model that generates realistic multi-speaker conversations from a single text transcript. Built on a 1.6B-parameter encoder-decoder transformer, Dia handles turn-taking, speaker-specific voices, and non-verbal cues (laughter, pauses) within a conversation. It also supports voice cloning from a short audio prompt, enabling zero-shot speaker generation in dialogue context.
- **Sesame CSM** (Conversational Speech Model, 2025) focuses on natural multi-turn conversational speech. Rather than optimising for reading-style TTS, Sesame models the dynamics of real conversation: backchannels ( “uh huh” ), interruptions, rhythm changes between speakers, and emotional responsiveness. The model uses a transformer backbone conditioned on conversational context (both text and audio history), producing speech that adapts its style to the flow of the dialogue.

- **Fish Speech** (Fish Audio, 2024) is an open-source TTS system that uses a dual autoregressive architecture: a large language model generates semantic tokens from text, and a smaller model converts these to VQGAN acoustic tokens, which are decoded into waveforms by a vocoder. Fish Speech supports zero-shot voice cloning from a 10-15 second reference and achieves low latency suitable for real-time applications. Its modular design allows swapping components (e.g., different vocoders) independently.
- **ChatTTS** (2024) is an open-source conversational TTS model designed for dialogue applications like chatbots and virtual assistants. It generates natural, conversational-sounding speech with fine-grained control over prosodic features (laughter, pauses, filler words) using special tokens embedded in the text input. ChatTTS supports mixed Chinese-English synthesis and multi-speaker generation.
- **Bark** (Suno, 2023) is a transformer-based open-source model that generates speech, music, and sound effects from text prompts. It uses a three-stage pipeline of transformer models (text → semantic tokens → coarse acoustic tokens → fine acoustic tokens) and supports voice cloning, multilingual synthesis, and non-speech audio like music and ambient sounds. Bark's generality comes at the cost of controllability — it is less precise than dedicated TTS systems but more flexible.
- **Parler-TTS** (Hugging Face, 2024) takes a **natural language description** approach to voice control: instead of requiring a reference audio clip for style, the user provides a text description like “a female speaker with a warm, expressive voice in a quiet room.” Parler-TTS is trained on annotated speech data where each utterance is paired with a natural language description of the speaking style, enabling intuitive control without any reference audio.
- **Neuphonic** is an API-based TTS platform optimised for ultra-low-latency speech synthesis, targeting real-time voice agents and conversational AI applications. It achieves time-to-first-audio under 100 ms through a streaming architecture that begins generating audio before the full input text is available. Neuphonic focuses on the deployment and latency optimisation layer rather than novel model architecture, providing production-grade infrastructure around modern neural TTS.
- **KittenTTS** is a compact, fast TTS model designed for efficiency and low-resource deployment. It prioritises minimal latency and small model size for

edge and embedded applications, trading some naturalness for real-time performance on CPUs and mobile devices.

- The modern TTS landscape is bifurcating into two paradigms: (1) **codec language models** (VALL-E, Orpheus, Fish Speech) that treat speech generation as next-token prediction over discrete audio codes, leveraging the scaling laws of LLMs; and (2) **flow/diffusion-based models** (VITS, StyleTTS 2, Kokoro) that generate continuous mel spectrograms or waveforms through iterative refinement. Codec LMs excel at zero-shot cloning and expressiveness; flow/diffusion models tend to be smaller and faster. Both are rapidly converging toward human-level naturalness.
- **Prosody modelling** controls the “music” of speech: pitch, duration, energy, rhythm, and intonation. Without good prosody, synthesised speech sounds flat and robotic even if individual phonemes are clear. Think of prosody as the difference between a monotone GPS voice and an expressive audiobook narrator.
- **Pitch** (fundamental frequency  $F_0$ ) is the perceived highness or lowness of speech. It rises at the end of questions, falls at the end of statements, and varies continuously during emotional speech.  $F_0$  is extracted from audio using algorithms like CREPE (a neural pitch tracker) or YIN (autocorrelation-based, from file 01). In TTS, pitch is either predicted by the acoustic model (FastSpeech 2’ s pitch predictor) or implicitly learned (Tacotron 2).
- **Duration** determines the speaking rate and rhythm. Stressed syllables are longer, function words are shortened, and pauses mark phrase boundaries. Duration modelling is explicit in non-autoregressive models (FastSpeech) and implicit in autoregressive models (Tacotron’ s attention alignment determines duration).
- **Energy** (loudness) carries emphasis. “I didn’ t say HE stole it” vs “I didn’ t say he STOLE it” have different meanings conveyed entirely through energy patterns.
- **Style embeddings** capture higher-level prosodic patterns. The **Global Style Token (GST)** framework (Wang et al., 2018) learns a bank of style tokens (soft attention over a learned set of embeddings) that capture speaking styles like “excited” , “sad” , or “whispering” . The style embedding is extracted from a reference utterance and added to the encoder output, allowing style transfer at inference.

- **Voice conversion (VC)** changes the speaker identity of an utterance while preserving the linguistic content. Imagine recording yourself and having the output sound like a specific target speaker. VC requires disentangling speaker identity from content.



Voice conversion pipeline: source speech is decomposed into content representation and speaker embedding, the target speaker embedding replaces the source, and the decoder reconstructs speech in the target voice

- **Speaker embeddings** (detailed further in file 04) encode speaker identity as a fixed-dimensional vector. These can come from a pre-trained speaker verification model (x-vectors, ECAPA-TDNN). In VC, the source speech is encoded into a content representation that is speaker-independent, then decoded with the target speaker embedding.
- **Disentangled representations** separate speech into independent factors: content (phonemes), speaker identity, pitch, and rhythm. Approaches include:
  - **Information bottleneck:** compress the content representation so tightly that speaker information is lost (AutoVC)
  - **Adversarial training:** train a speaker classifier on the content representation and use gradient reversal to remove speaker information
  - **Vector quantisation:** VQ-VAE forces the content through a discrete bottleneck, which naturally strips speaker identity (since codebook entries represent phonetic categories, not speaker traits)
- **Voice cloning** synthesises speech in a target speaker's voice. **Multi-speaker TTS** trains on data from many speakers, conditioning the model on a speaker embedding. At inference, a new speaker's embedding is extracted from enrollment audio and used to condition generation.

- **Few-shot voice cloning** adapts to a new speaker using a small amount of data (a few minutes). The speaker encoder extracts an embedding from the enrollment audio, and the TTS model generates speech conditioned on this embedding. This is the approach used in SV2TTS (Jia et al., 2018): a separately trained speaker encoder, a Tacotron 2 synthesiser conditioned on the speaker embedding, and a WaveRNN vocoder.
- **Zero-shot voice cloning** requires no adaptation at all: a single short utterance (3-30 seconds) is enough. VALL-E achieves this by treating the enrollment audio as a prompt for the language model. The model learns to continue generating in the same voice because it was trained on large-scale multi-speaker data where voice consistency within an utterance is the statistical norm.
- **Voice activity detection (VAD)** answers a simple binary question at each time frame: is someone speaking or not? Despite its simplicity, VAD is a critical preprocessing step for ASR (file 02), speaker diarisation (file 04), and noise reduction (file 05). A good VAD reduces computation by skipping silence and improves accuracy by preventing noise from being processed as speech.
- Classical VAD uses energy thresholding (speech is louder than silence), zero-crossing rate (speech has characteristic crossing patterns), and spectral features. These fail in noisy environments where the signal-to-noise ratio is low.
- **Neural VAD** models treat the problem as frame-level binary classification. A small RNN or CNN takes acoustic features (log mel energies from file 01) and predicts speech/non-speech probabilities.
- **WebRTC VAD** (Google) is a classic lightweight VAD using a GMM-based classifier on simple spectral features. It operates at four aggressiveness levels (0-3) and is extremely fast, but struggles with music, non-speech vocalisations, and low-SNR environments. It remains widely used as a baseline due to its zero-dependency simplicity.
- **Silero VAD** (Silero Team, 2021) is the de facto standard neural VAD for production use. Its architecture is a small stack of depthwise separable 1D convolutions (chapter 08's MobileNet idea applied to audio) followed by a single LSTM layer for temporal context, with a final linear head producing a



speech probability per frame. The entire model is under 2MB (~1M parameters) and processes audio in 30-100 ms chunks.

- **Input:** raw 16 kHz audio (no manual feature extraction — the convolutional front-end learns its own features from the waveform directly).
  - **Windowed stateful inference:** the LSTM hidden state carries over between chunks, so the model handles streaming audio without reprocessing the full history. Each call processes a 30, 60, or 100 ms chunk and returns a speech probability in [0, 1].
  - **Adaptive thresholding:** rather than a single fixed threshold, Silero VAD uses separate start and end thresholds with a minimum speech/silence duration, preventing rapid toggling on noisy boundaries. A speech segment must exceed the start threshold for a minimum duration before being confirmed, and silence must persist below the end threshold before the segment is closed.
  - **Performance:** Silero VAD runs at 1-2% real-time factor on CPU (processing 1 second of audio takes ~10-20 ms), making it suitable for edge devices, mobile phones, and real-time pipelines. It significantly outperforms WebRTC VAD on noisy and music-heavy audio while remaining small enough for on-device deployment.
  - Silero VAD is commonly used as the front-end for Whisper (file 02) to segment long audio into utterance-level chunks before transcription, and for speaker diarisation pipelines (file 04) to identify speech regions before extracting speaker embeddings.
- **Acoustic activity detection (AAD)** generalises VAD to detect any acoustic activity, not just speech. This is useful in smart home devices, security systems, and wildlife monitoring. AAD models detect events like glass breaking, dogs barking, or alarms, often using the audio classification frameworks described in file 04.
  - **Evaluation metrics for TTS** measure both objective quality and subjective naturalness:
    - **Mean Opinion Score (MOS):** human listeners rate naturalness on a 1-5 scale. The gold standard, but expensive and slow.
    - **Mel cepstral distortion (MCD):** measures the distance between synthesised and reference mel cepstra. Lower is better, but does not always correlate with perception.

- **PESQ / POLQA**: standardised perceptual evaluation metrics originally designed for telephony.
- **Speaker similarity**: cosine similarity between speaker embeddings of synthesised and reference audio (relevant for voice cloning).
- **Intelligibility**: measured by feeding synthesised audio through an ASR system (file 02) and computing WER.

## 43.1 Coding Tasks (use CoLab or notebook)

- **Task 1: Griffin-Lim vocoder from mel spectrogram.** Implement the Griffin-Lim iterative phase reconstruction algorithm to convert a mel spectrogram back to a waveform. This demonstrates the vocoder problem and why neural vocoders are needed.

```
import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

Generate a synthetic waveform (sum of harmonics simulating a vowel)
sr = 16000
duration = 1.0
t = jnp.linspace(0, duration, int(sr * duration))
f0 = 220.0 # fundamental frequency
waveform = (
 0.6 * jnp.sin(2 * jnp.pi * f0 * t) +
 0.3 * jnp.sin(2 * jnp.pi * 2 * f0 * t) +
 0.1 * jnp.sin(2 * jnp.pi * 3 * f0 * t)
)

Compute STFT
n_fft = 1024
hop_length = 256
window = jnp.hanning(n_fft)

def stft(signal, n_fft, hop_length, window):
 """Compute Short-Time Fourier Transform."""
 n_frames = 1 + (len(signal) - n_fft) // hop_length
 frames = jnp.stack([
 signal[i * hop_length : i * hop_length + n_fft] * window
 for i in range(n_frames)
])
 return jnp.fft.rfft(frames, n=n_fft)

def istft(stft_matrix, hop_length, window, length):
```



```

"""Compute inverse STFT with overlap-add."""
n_fft = (stft_matrix.shape[1] - 1) * 2
n_frames = stft_matrix.shape[0]
frames = jnp.fft.irfft(stft_matrix, n=n_fft)
frames = frames * window[None, :]
output = jnp.zeros(length)
for i in range(n_frames):
 start = i * hop_length
 end = start + n_fft
 if end <= length:
 output = output.at[start:end].add(frames[i])
return output

Forward STFT
S = stft(waveform, n_fft, hop_length, window)
magnitude = jnp.abs(S)

Mel filterbank
n_mels = 80
mel_low = 0.0
mel_high = 2595 * jnp.log10(1 + (sr / 2) / 700)
mel_points = jnp.linspace(mel_low, mel_high, n_mels + 2)
hz_points = 700 * (10 ** (mel_points / 2595) - 1)
freq_bins = jnp.floor((n_fft + 1) * hz_points / sr).astype(int)

mel_filterbank = jnp.zeros((n_mels, n_fft // 2 + 1))
for m in range(n_mels):
 f_left = freq_bins[m]
 f_center = freq_bins[m + 1]
 f_right = freq_bins[m + 2]
 for k in range(f_left, f_center):
 mel_filterbank = mel_filterbank.at[m, k].set(
 (k - f_left) / max(f_center - f_left, 1)
)
 for k in range(f_center, f_right):
 mel_filterbank = mel_filterbank.at[m, k].set(
 (f_right - k) / max(f_right - f_center, 1)
)

To mel and back (pseudo-inverse)
mel_spec = magnitude @ mel_filterbank.T
magnitude_reconstructed = mel_spec @ jnp.linalg.pinv(mel_filterbank.T)
magnitude_reconstructed = jnp.maximum(magnitude_reconstructed, 1e-7)

Griffin-Lim algorithm
def griffin_lim(magnitude, n_iter, hop_length, window, signal_length):
 """Iterative phase reconstruction."""

```

```

n_fft = (magnitude.shape[1] - 1) * 2
key = jax.random.PRNGKey(42)
phase = jax.random.uniform(key, magnitude.shape, minval=-jnp.pi,
maxval=jnp.pi)

 for _ in range(n_iter):
 complex_spec = magnitude * jnp.exp(1j * phase)
 signal = istft(complex_spec, hop_length, window, signal_length)
 reanalysis = stft(signal, n_fft, hop_length, window)
 phase = jnp.angle(reanalysis)

 complex_spec = magnitude * jnp.exp(1j * phase)
 return istft(complex_spec, hop_length, window, signal_length)

reconstructed = griffin_lim(magnitude_reconstructed, n_iter=60,
hop_length=hop_length,
 window=window, signal_length=len(waveform))

Plot comparison
fig, axes = plt.subplots(3, 1, figsize=(12, 8))

axes[0].plot(t[:1000], waveform[:1000], color='#3498db', linewidth=0.8)
axes[0].set_title('Original Waveform')
axes[0].set_ylabel('Amplitude')

axes[1].imshow(jnp.log1p(mel_spec.T), aspect='auto', origin='lower',
cmap='magma')
axes[1].set_title('Mel Spectrogram (intermediate representation)')
axes[1].set_ylabel('Mel bin')

axes[2].plot(t[:1000], reconstructed[:1000], color='#e74c3c', linewidth=0.8)
axes[2].set_title('Griffin-Lim Reconstructed Waveform (60 iterations)')
axes[2].set_xlabel('Time (s)')
axes[2].set_ylabel('Amplitude')

plt.tight_layout()
plt.show()

Measure reconstruction error
mse = jnp.mean((waveform[:len(reconstructed)] - reconstructed[:len(waveform)]))

```

**\*\* 2)**

```
print(f"MSE between original and reconstructed: {mse:.6f}")
print("Note: phase information loss through mel inversion causes artifacts.")
```

- **Task 2: Duration predictor (FastSpeech-style).** Train a small convolutional duration predictor that maps phoneme embeddings to durations. This is the core component enabling non-autoregressive TTS.

```
import jax
import jax.numpy as jnp
import jax.random as jr
import matplotlib.pyplot as plt

Simulate phoneme sequences with ground-truth durations
In real TTS, durations come from forced alignment or teacher attention
def generate_synthetic_data(key, n_samples=200, max_phonemes=30, embed_dim=64):
 """Generate synthetic phoneme embeddings and durations."""
 keys = jr.split(key, 4)
 lengths = jr.randint(keys[0], (n_samples,), 5, max_phonemes)

 all_embeddings = []
 all_durations = []
 all_masks = []

 for i in range(n_samples):
 L = int(lengths[i])
 emb = jr.normal(keys[1], (max_phonemes, embed_dim))
 # Durations: vowels (even indices) are longer, consonants shorter
 base_dur = jnp.where(jnp.arange(max_phonemes) % 2 == 0, 8.0, 4.0)
 noise = jr.normal(jr.fold_in(keys[2], i), (max_phonemes,)) * 1.5
 dur = jnp.clip(base_dur + noise, 1.0, 20.0).astype(jnp.float32)
 mask = (jnp.arange(max_phonemes) < L).astype(jnp.float32)

 all_embeddings.append(emb)
 all_durations.append(dur * mask)
 all_masks.append(mask)

 return (jnp.stack(all_embeddings), jnp.stack(all_durations),
 jnp.stack(all_masks))

key = jr.PRNGKey(42)
embeddings, durations, masks = generate_synthetic_data(key)

Duration predictor: 2-layer 1D convolution + linear projection
def init_duration_predictor(key, embed_dim=64, hidden_dim=128, kernel_size=3):
 """Initialise duration predictor weights."""
 keys = jr.split(key, 4)
```

```

scale1 = jnp.sqrt(2.0 / (embed_dim * kernel_size))
scale2 = jnp.sqrt(2.0 / (hidden_dim * kernel_size))
params = {
 'conv1_w': jr.normal(keys[0], (kernel_size, embed_dim, hidden_dim)) *
scale1,
 'conv1_b': jnp.zeros(hidden_dim),
 'conv2_w': jr.normal(keys[1], (kernel_size, hidden_dim, hidden_dim)) *
scale2,
 'conv2_b': jnp.zeros(hidden_dim),
 'linear_w': jr.normal(keys[2], (hidden_dim, 1)) * jnp.sqrt(2.0 /
hidden_dim),
 'linear_b': jnp.zeros(1),
}
return params

def duration_predictor(params, x):
 """Predict log-durations from phoneme embeddings. x: (batch, seq, embed)."""
 # Conv layer 1 with ReLU
 h = jax.lax.conv_general_dilated(
 x.transpose(0, 2, 1), # (batch, embed, seq)
 params['conv1_w'].transpose(2, 1, 0), # (out, in, kernel)
 window_strides=(1,), padding='SAME'
).transpose(0, 2, 1) + params['conv1_b'] # back to (batch, seq, hidden)
 h = jax.nn.relu(h)

 # Conv layer 2 with ReLU
 h = jax.lax.conv_general_dilated(
 h.transpose(0, 2, 1),
 params['conv2_w'].transpose(2, 1, 0),
 window_strides=(1,), padding='SAME'
).transpose(0, 2, 1) + params['conv2_b']
 h = jax.nn.relu(h)

 # Linear projection to scalar
 log_dur = (h @ params['linear_w'] + params['linear_b']).squeeze(-1)
 return log_dur

Loss: MSE on log-durations (standard in FastSpeech)
def loss_fn(params, embeddings, durations, masks):
 log_dur_pred = duration_predictor(params, embeddings)
 log_dur_true = jnp.log(jnp.clip(durations, 1.0, None))
 sq_err = (log_dur_pred - log_dur_true) ** 2 * masks
 return jnp.sum(sq_err) / jnp.sum(masks)

grad_fn = jax.jit(jax.value_and_grad(loss_fn))

Training loop

```

```

params = init_duration_predictor(jr.PRNGKey(0))
lr = 1e-3
losses = []

for epoch in range(300):
 loss_val, grads = grad_fn(params, embeddings, durations, masks)
 params = jax.tree.map(lambda p, g: p - lr * g, params, grads)
 losses.append(float(loss_val))

Evaluate on a sample
log_dur_pred = duration_predictor(params, embeddings[:1])
dur_pred = jnp.exp(log_dur_pred[0])
dur_true = durations[0]
mask = masks[0]
valid_len = int(jnp.sum(mask))

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].plot(losses, color='#3498db', linewidth=1.5)
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('MSE Loss (log-duration)')
axes[0].set_title('Duration Predictor Training')
axes[0].set_yscale('log')

x_pos = jnp.arange(valid_len)
width = 0.35
axes[1].bar(x_pos - width/2, dur_true[:valid_len], width, color='#27ae60',
 label='Ground truth', alpha=0.8)
axes[1].bar(x_pos + width/2, dur_pred[:valid_len], width, color='#e74c3c',
 label='Predicted', alpha=0.8)
axes[1].set_xlabel('Phoneme index')
axes[1].set_ylabel('Duration (frames)')
axes[1].set_title('Duration Prediction vs Ground Truth')
axes[1].legend()

plt.tight_layout()
plt.show()

```

· **Task 3: Simple neural vocoder with upsampling convolutions.** Build a minimal HiFi-GAN-style generator that upsamples a mel spectrogram to a waveform using transposed convolutions and residual blocks.

```

import jax
import jax.numpy as jnp
import jax.random as jr
import matplotlib.pyplot as plt

```

```

def init_residual_block(key, channels, kernel_size, dilation):
 """Initialise a dilated residual convolution block."""
 k1, k2 = jr.split(key)
 scale = jnp.sqrt(2.0 / (channels * kernel_size))
 return {
 'conv1_w': jr.normal(k1, (kernel_size, channels, channels)) * scale,
 'conv1_b': jnp.zeros(channels),
 'conv2_w': jr.normal(k2, (kernel_size, channels, channels)) * scale,
 'conv2_b': jnp.zeros(channels),
 'dilation': dilation
 }

def residual_block(params, x):
 """x: (batch, time, channels). Dilated conv residual block with LeakyReLU."""
 h = jax.nn.leaky_relu(x, negative_slope=0.1)
 # Simplified: use standard conv (dilation handled conceptually)
 h = jax.lax.conv_general_dilated(
 h.transpose(0, 2, 1),
 params['conv1_w'].transpose(2, 1, 0),
 window_strides=(1,),
 padding='SAME',
 rhs_dilation=(params['dilation'],)
).transpose(0, 2, 1) + params['conv1_b']
 h = jax.nn.leaky_relu(h, negative_slope=0.1)
 h = jax.lax.conv_general_dilated(
 h.transpose(0, 2, 1),
 params['conv2_w'].transpose(2, 1, 0),
 window_strides=(1,),
 padding='SAME'
).transpose(0, 2, 1) + params['conv2_b']
 return x + h

def init_generator(key, n_mels=80, upsample_rates=(8, 8, 4),
 channels=128):
 """Initialise a minimal HiFi-GAN-style generator."""
 keys = jr.split(key, 10)
 params = {}

 # Input projection: mel bins -> channels
 params['input_w'] = jr.normal(keys[0], (7, n_mels, channels)) * 0.02
 params['input_b'] = jnp.zeros(channels)

 # Upsample blocks (transposed convolutions)
 in_ch = channels
 for i, rate in enumerate(upsample_rates):
 k_size = rate * 2

```

scale

```
scale = jnp.sqrt(2.0 / (in_ch * k_size))
out_ch = in_ch // 2
params[f'up{i}_w'] = jr.normal(keys[i+1], (k_size, in_ch, out_ch)) *

params[f'up{i}_b'] = jnp.zeros(out_ch)
Residual blocks at each scale
params[f'res{i}_0'] = init_residual_block(jr.fold_in(keys[i+4], 0),
 out_ch, 3, 1)
params[f'res{i}_1'] = init_residual_block(jr.fold_in(keys[i+4], 1),
 out_ch, 3, 3)

in_ch = out_ch

Output projection to mono waveform
params['output_w'] = jr.normal(keys[8], (7, in_ch, 1)) * 0.02
params['output_b'] = jnp.zeros(1)
params['upsample_rates'] = upsample_rates

return params

def generator_forward(params, mel):
 """mel: (batch, time, n_mels) -> waveform: (batch, time * prod(rates), 1)."""
 # Input projection
 h = jax.lax.conv_general_dilated(
 mel.transpose(0, 2, 1),
 params['input_w'].transpose(2, 1, 0),
 window_strides=(1,), padding='SAME'
).transpose(0, 2, 1) + params['input_b']

 for i, rate in enumerate(params['upsample_rates']):
 h = jax.nn.leaky_relu(h, negative_slope=0.1)
 # Upsample via transposed convolution
 k_size = rate * 2
 h = jax.lax.conv_transpose(
 h.transpose(0, 2, 1),
 params[f'up{i}_w'].transpose(2, 1, 0),
 strides=(rate,),
 padding='SAME'
).transpose(0, 2, 1) + params[f'up{i}_b']
 # Residual blocks
 h = residual_block(params[f'res{i}_0'], h)
 h = residual_block(params[f'res{i}_1'], h)

 h = jax.nn.leaky_relu(h, negative_slope=0.1)
 out = jax.lax.conv_general_dilated(
 h.transpose(0, 2, 1),
 params['output_w'].transpose(2, 1, 0),
 window_strides=(1,), padding='SAME'
```

```

).transpose(0, 2, 1) + params['output_b']

return jnp.tanh(out)

Create a synthetic mel spectrogram (simulating a vowel)
n_mels = 80
n_frames = 50
mel = jnp.zeros((1, n_frames, n_mels))
Add energy in low-frequency mel bins (simulating formants)
mel = mel.at[:, :, 5:15].set(1.0)
mel = mel.at[:, :, 20:25].set(0.6)

Initialise and run generator
key = jr.PRNGKey(42)
params = init_generator(key, n_mels=n_mels, upsample_rates=(8, 8, 4),
 channels=128)
waveform = generator_forward(params, mel)

print(f"Input mel shape: {mel.shape}")
print(f"Output waveform shape: {waveform.shape}")
print(f"Upsample factor: {8 * 8 * 4} = {8*8*4}x")

fig, axes = plt.subplots(2, 1, figsize=(12, 6))

axes[0].imshow(mel[0].T, aspect='auto', origin='lower', cmap='magma')
axes[0].set_title('Input Mel Spectrogram')
axes[0].set_ylabel('Mel bin')
axes[0].set_xlabel('Frame')

waveform_np = waveform[0, :, 0]
axes[1].plot(waveform_np[:2000], color='#9b59b6', linewidth=0.5)
axes[1].set_title('Generator Output Waveform (untrained - random noise)')
axes[1].set_ylabel('Amplitude')
axes[1].set_xlabel('Sample')

plt.tight_layout()
plt.show()
print("Note: The output is noise because the generator is untrained.")
print("In practice, adversarial + mel loss training shapes this into speech.")

```

· **Task 4: Voice activity detection with a simple RNN.** Train a small GRU-based VAD model on synthetic audio features to classify frames as speech or silence.

```

import jax
import jax.numpy as jnp
import jax.random as jr

```



```

import matplotlib.pyplot as plt

Generate synthetic log-mel energy features with speech/silence labels
def generate_vad_data(key, n_sequences=100, n_frames=200, n_features=40):
 """Simulate log-mel features: speech regions are higher energy with
 structure."""
 keys = jr.split(key, 5)
 all_features = []
 all_labels = []

 for i in range(n_sequences):
 k = jr.fold_in(keys[0], i)
 k1, k2, k3 = jr.split(k, 3)

 # Random speech/silence pattern
 label = jnp.zeros(n_frames)
 n_segments = jr.randint(k1, (), 2, 6)
 for seg in range(int(n_segments)):
 start = jr.randint(jr.fold_in(k2, seg), (), 0, n_frames - 20)
 length = jr.randint(jr.fold_in(k3, seg), (), 10, 50)
 end = jnp.minimum(start + length, n_frames)
 label = label.at[int(start):int(end)].set(1.0)

 # Features: speech frames have higher energy + spectral structure
 noise = jr.normal(jr.fold_in(keys[1], i), (n_frames, n_features)) * 0.3
 speech_pattern = jnp.outer(label, jnp.exp(-jnp.arange(n_features) /
15.0))

 features = speech_pattern * 2.0 + noise + 0.1

 all_features.append(features)
 all_labels.append(label)

 return jnp.stack(all_features), jnp.stack(all_labels)

key = jr.PRNGKey(123)
features, labels = generate_vad_data(key)
train_features, train_labels = features[:80], labels[:80]
test_features, test_labels = features[80:], labels[80:]

Simple GRU-based VAD model
def init_vad_model(key, input_dim=40, hidden_dim=64):
 keys = jr.split(key, 6)
 scale_ih = jnp.sqrt(2.0 / input_dim)
 scale_hh = jnp.sqrt(2.0 / hidden_dim)
 return {
 'W_z': jr.normal(keys[0], (input_dim, hidden_dim)) * scale_ih,
 'U_z': jr.normal(keys[1], (hidden_dim, hidden_dim)) * scale_hh,

```

```

 'b_z': jnp.zeros(hidden_dim),
 'W_r': jr.normal(keys[2], (input_dim, hidden_dim)) * scale_ih,
 'U_r': jr.normal(keys[3], (hidden_dim, hidden_dim)) * scale_hh,
 'b_r': jnp.zeros(hidden_dim),
 'W_h': jr.normal(keys[4], (input_dim, hidden_dim)) * scale_ih,
 'U_h': jr.normal(keys[5], (hidden_dim, hidden_dim)) * scale_hh,
 'b_h': jnp.zeros(hidden_dim),
 'W_out': jr.normal(jr.fold_in(keys[0], 99), (hidden_dim, 1)) * 0.1,
 'b_out': jnp.zeros(1),
 }

def gru_step(params, h, x):
 """Single GRU step."""
 z = jax.nn.sigmoid(x @ params['W_z'] + h @ params['U_z'] + params['b_z'])
 r = jax.nn.sigmoid(x @ params['W_r'] + h @ params['U_r'] + params['b_r'])
 h_tilde = jnp.tanh(x @ params['W_h'] + (r * h) @ params['U_h'] +
params['b_h'])
 h_new = (1 - z) * h + z * h_tilde
 return h_new

def vad_forward(params, x):
 """x: (batch, time, features) -> logits: (batch, time)."""
 batch_size, n_frames, _ = x.shape
 hidden_dim = params['W_z'].shape[1]
 h = jnp.zeros((batch_size, hidden_dim))

 outputs = []
 for t in range(n_frames):
 h = gru_step(params, h, x[:, t, :])
 logit = (h @ params['W_out'] + params['b_out']).squeeze(-1)
 outputs.append(logit)

 return jnp.stack(outputs, axis=1)

def bce_loss(params, features, labels):
 """Binary cross-entropy loss for VAD."""
 logits = vad_forward(params, features)
 probs = jax.nn.sigmoid(logits)
 probs = jnp.clip(probs, 1e-7, 1 - 1e-7)
 loss = -(labels * jnp.log(probs) + (1 - labels) * jnp.log(1 - probs))
 return jnp.mean(loss)

grad_fn = jax.jit(jax.value_and_grad(bce_loss))

Training
params = init_vad_model(jr.PRNGKey(0))
lr = 5e-3

```

```

losses = []

for epoch in range(200):
 loss_val, grads = grad_fn(params, train_features, train_labels)
 params = jax.tree.map(lambda p, g: p - lr * g, params, grads)
 losses.append(float(loss_val))
 if epoch % 50 == 0:
 print(f"Epoch {epoch}: loss = {loss_val:.4f}")

Evaluate on test set
test_logits = vad_forward(params, test_features)
test_preds = (jax.nn.sigmoid(test_logits) > 0.5).astype(jnp.float32)
accuracy = jnp.mean(test_preds == test_labels)
print(f"\nTest accuracy: {accuracy:.4f}")

Visualise a test example
idx = 0
fig, axes = plt.subplots(3, 1, figsize=(14, 7))

axes[0].imshow(test_features[idx].T, aspect='auto', origin='lower', cmap='magma')
axes[0].set_title('Log-Mel Energy Features')
axes[0].set_ylabel('Mel bin')

axes[1].fill_between(range(200), test_labels[idx], alpha=0.4, color='#27ae60',
 label='Ground truth')
axes[1].plot(jax.nn.sigmoid(test_logits[idx]), color='#e74c3c',
 linewidth=1.5, label='Predicted probability')
axes[1].axhline(0.5, color='gray', linestyle='--', linewidth=0.8)
axes[1].set_ylabel('Speech probability')
axes[1].legend()
axes[1].set_title('VAD Predictions')

axes[2].fill_between(range(200), test_labels[idx], alpha=0.4, color='#27ae60',
 label='Ground truth')
axes[2].fill_between(range(200), test_preds[idx], alpha=0.4, color='#f39c12',
 label='Predicted (threshold=0.5)')
axes[2].set_ylabel('Speech / Silence')
axes[2].set_xlabel('Frame')
axes[2].legend()
axes[2].set_title('VAD Binary Decision')

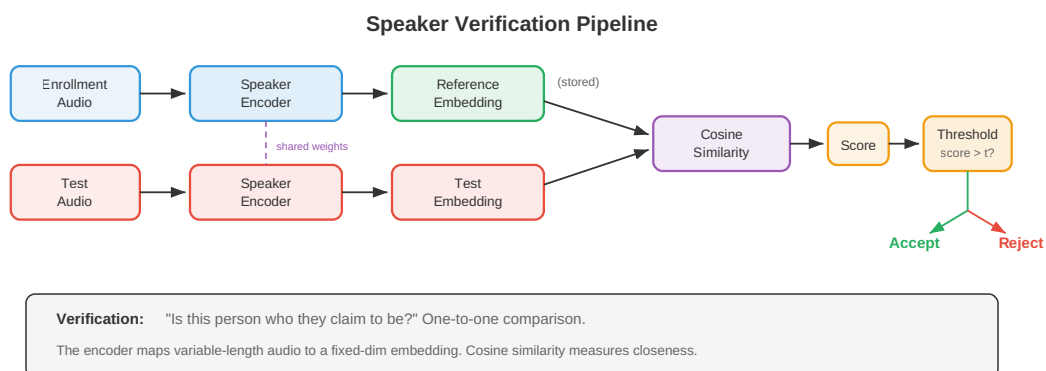
plt.tight_layout()
plt.show()

```

## 44 Speaker and Audio Analysis

Speaker and audio analysis identifies who is speaking, when they speak, and what non-speech sounds are present. This file covers speaker verification and identification, i-vectors, d-vectors, x-vectors, speaker diarisation, audio event classification, music information retrieval, and emotion recognition from speech.

- In file 01 we built the signal-processing foundations: spectrograms, MFCCs, and mel filterbanks. In file 02 we recognised what was said. Now we ask who said it, when they said it, and what else is happening in the audio. Speaker recognition, diarisation, audio classification, and music analysis all share a common thread: learning compact embeddings that capture the right invariances for the task at hand, echoing the embedding ideas from chapter 06.
- Think of identifying a speaker like recognising a friend’s voice on the phone. You do not need to understand the words; something about the timbre, pacing, and vocal quality is unique to that person. Speaker recognition systems learn to extract exactly this “voiceprint” from raw audio, ignoring what is said and focusing on how it is said.
- **Speaker recognition** is the umbrella term for two related tasks:
  - **Speaker verification** (SV): given a claimed identity and an audio clip, determine whether the speaker is who they claim to be. This is a binary decision (accept or reject) and is the technology behind voice-based authentication ( “Hey Siri, is this my voice?” ).
  - **Speaker identification** (SI): given an audio clip and a gallery of known speakers, determine which speaker produced the clip. This is a multi-class classification problem.



Speaker verification: enrollment audio is embedded, test audio is embedded, cosine similarity is computed between embeddings, and a threshold determines accept or reject

- Both tasks share the same underlying representation: a fixed-dimensional **speaker embedding** that captures the speaker's identity regardless of what they say. The difference is only in the decision stage: verification compares two embeddings, identification finds the nearest embedding among candidates.
- **Cosine similarity** is the standard metric for comparing speaker embeddings. Given enrollment embedding  $e$  and test embedding  $t$ :

$$s = \frac{e \cdot t}{\|e\| \|t\|}$$

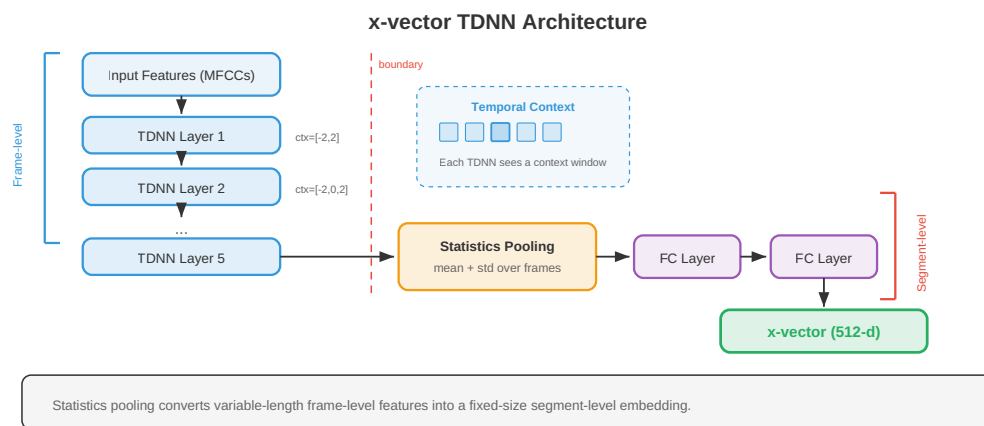
- A threshold  $\theta$  determines the accept/reject decision: if  $s > \theta$ , accept. The threshold trades off between the **false acceptance rate (FAR)** and **false rejection rate (FRR)**. The **equal error rate (EER)**, where  $FAR = FRR$ , is the standard evaluation metric. Lower EER means better performance. State-of-the-art systems achieve EER below 1% on standard benchmarks (VoxCeleb).
- **i-vectors** (Dehak et al., 2010) were the dominant speaker embedding before deep learning. The idea comes from factor analysis (chapter 02's matrix factorisation and chapter 04's dimensionality reduction). A **universal background model (UBM)**, a large GMM trained on diverse speakers, defines a supervector space. Each utterance's GMM supervector is projected into a low-dimensional **total variability space**:

$$M = m + Tw$$

- where  $M$  is the utterance's GMM supervector,  $m$  is the UBM mean supervector,  $T$  is the total variability matrix (learned from data), and  $w$  is the i-vector, a low-dimensional (typically 400-600) representation capturing both speaker and channel variability.
- To remove channel variability from i-vectors, **Probabilistic Linear Discriminant Analysis (PLDA)** models the i-vector as a sum of speaker-specific and channel-specific latent variables. PLDA provides a principled log-likelihood ratio score for verification:

$$\text{score}(w_1, w_2) = \log \frac{P(w_1, w_2 \mid \text{same speaker})}{P(w_1 \mid \text{speaker}_1) P(w_2 \mid \text{speaker}_2)}$$

- **d-vectors** (Variani et al., 2014) were the first neural speaker embeddings. A DNN trained for speaker classification on frame-level features extracts a fixed-dimensional representation by averaging the last hidden layer activations over all frames in an utterance. Simple but effective, d-vectors demonstrated that neural networks could learn speaker-discriminative features without the complex statistical machinery of i-vectors.
- **x-vectors** (Snyder et al., 2018) significantly advanced neural speaker embeddings using a **Time Delay Neural Network (TDNN)** architecture. TDNNs are 1D convolutions with specific context windows at each layer, related to the dilated convolutions from file 03' s WaveNet but applied to frame-level features rather than raw waveform samples.



x-vector architecture: TDNN layers process frame-level features with increasing context, statistics pooling aggregates across time, and fully connected layers produce the speaker embedding

- The x-vector architecture has three stages:
  - **Frame-level layers:** a stack of TDNN layers processes MFCCs (from file 01) with progressively wider temporal context. Each layer sees a fixed context window (e.g.,  $\{t-2, t-1, t, t+1, t+2\}$  for the first layer, wider for subsequent layers).
  - **Statistics pooling:** after the frame-level layers, the mean and standard deviation of the frame-level outputs are computed over the entire utterance, producing a fixed-dimensional vector regardless of utterance length:

```

\begin{aligned}
\mu &= \frac{1}{T} \sum_{t=1}^T h_t \\
\sigma &= \sqrt{\frac{1}{T} \sum_{t=1}^T (h_t - \mu)^2}
\end{aligned}

```

- where  $h_t$  is the frame-level output at time  $t$ . The concatenation  $[\mu; \sigma]$  is the pooled representation.
  - **Segment-level layers:** fully connected layers process the pooled representation. The output of the first segment-level layer (before the softmax) is the x-vector embedding.
- x-vectors are trained with a standard cross-entropy loss over speaker identities. Despite being trained for classification, the learned intermediate representation (the x-vector) generalises well to unseen speakers because the network learns to extract speaker-discriminative features rather than memorising specific speakers.
- **ECAPA-TDNN** (Desplanques et al., 2020) is the current state-of-the-art TDNN-based architecture for speaker recognition. It introduces three improvements over x-vectors:
  - **Squeeze-Excitation (SE) blocks:** channel attention (from chapter 08's SENet) that re-weights feature channels based on global context, allowing the model to emphasise speaker-relevant channels.
  - **Res2Net-style multi-scale features:** within each TDNN block, the channels are split into groups that are processed hierarchically, creating features at multiple temporal resolutions (analogous to chapter 08's multi-scale feature extraction).
  - **Attentive statistics pooling:** instead of equal-weight averaging, an attention mechanism weights each frame's contribution to the pooled statistics. Frames with more speaker-discriminative content (e.g., vowels, which carry more speaker information) receive higher attention weights:

$$\alpha_t = \frac{\exp(v^T f(h_t))}{\sum_{\tau} \exp(v^T f(h_{\tau}))}$$

- where  $f$  is a small neural network and  $v$  is a learned attention vector. The attended mean and standard deviation become

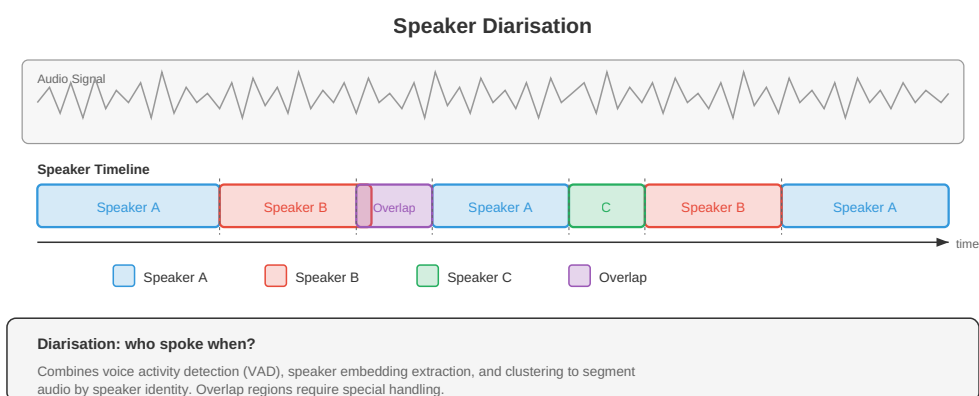
$$\begin{aligned} \mu &= \sum_t \alpha_t h_t \text{ and} \\ \sigma &= \sqrt{\sum_t \alpha_t (h_t - \mu)^2} \end{aligned}$$

- ECAPA-TDNN is typically trained with **AAM-Softmax** (Additive Angular Margin Softmax), which adds an angular margin penalty to the classification loss, pushing embeddings of the same speaker closer together and different speakers further apart on the hypersphere:

$$L = - \log \frac{e^{s \cos(\theta_{y_i} + m)}}{\sum_{j \neq y_i} e^{s \cos \theta_j}}$$

- where  $\theta_{y_i}$  is the angle between the embedding and the weight vector of the true class,  $m$  is the margin (typically 0.2), and  $s$  is a scaling factor (typically 30). This loss comes from face recognition (chapter 08's ArcFace) and is highly effective for speaker verification.
- **Speaker diarisation** answers “who spoke when” in a multi-speaker recording. Think of it as colouring a timeline: each colour represents a different speaker, and the system must determine when each speaker is active, including overlapping speech.





Speaker diarisation: an audio timeline is segmented and labelled with speaker identities, showing turn-taking and overlap regions

- **Clustering-based diarisation** is the traditional pipeline approach:
  - **Segmentation**: divide the audio into short segments (typically 1-2 seconds) using a sliding window or speaker change detection.
  - **Embedding extraction**: extract a speaker embedding (x-vector, ECAPA-TDNN) for each segment.
  - **Clustering**: group segments by speaker. **Agglomerative Hierarchical Clustering (AHC)** is standard: start with each segment as its own cluster, then iteratively merge the two most similar clusters until a stopping criterion is met (based on a distance threshold or a target number of speakers).
  - **Re-segmentation**: refine boundaries using a Viterbi-based re-alignment.
- The number of speakers is typically unknown a priori, which makes this problem harder than standard clustering. Spectral clustering with an eigenvalue-based threshold for determining  $k$  is another common approach.
- **End-to-end neural diarisation (EEND)** (Fujita et al., 2019) frames diarisation as a multi-label classification problem. A neural network (typically a self-attention-based model, chapter 07's transformer) takes the entire recording as input and outputs a binary activity label for each speaker at each frame. This directly handles overlapping speech, which is a major weakness of clustering-based methods.
- The EEND output for  $S$  speakers at frame  $t$  is:

$$y_{t,s} = \sigma(f_s(h_t))$$

- where  $h_t$  is the transformer output at frame  $t$  and  $f_s$  is a linear projection for speaker  $s$ . The training loss is binary cross-entropy summed over speakers and frames. A key challenge is that the number of speakers must be fixed or handled with a variable-output architecture (EEND-EDA uses an encoder-decoder with attractors).
- **Permutation invariant training (PIT)** for diarisation handles the label ambiguity problem: since speakers have no inherent ordering, the loss is computed for all possible speaker-to-output assignments and the minimum is taken (this is the same PIT used in source separation, covered in file 05).
- **Audio classification** assigns a label to an entire audio clip. Unlike ASR (file 02), which transcribes speech, audio classification covers a broader range: environmental sounds (siren, rain, dog bark), music genres (rock, jazz, classical), and general audio events.
- The standard approach follows the image classification paradigm from chapter 08: represent the audio as a spectrogram (a 2D time-frequency image), then apply a CNN or transformer classifier. This spectral-image approach leverages decades of progress in computer vision.
- **Environmental sound classification (ESC)** uses datasets like ESC-50 (50 classes, 2000 clips) and UrbanSound8K. Typical architectures are CNNs (chapter 06) applied to log-mel spectrograms. Data augmentation is crucial: time stretching, pitch shifting, adding background noise, and **SpecAugment** (file 02' s masking approach applied to spectrograms) all improve generalisation.
- **Audio event detection** (Sound Event Detection, SED) is the temporal analogue of classification: not just what events are present, but when they start and end. **AudioSet** (Gemmeke et al., 2017) is the large-scale benchmark with 527 event classes and over 2 million 10-second clips from YouTube, each weakly labelled (clip-level labels, not frame-level).
- **Weakly-supervised SED** must learn frame-level predictions from clip-level labels. The standard approach uses a CNN that produces frame-level class probabilities, then aggregates them to clip-level predictions via attention pooling:

⌚

Y

$$c = \sigma \left( \sum_t \right)$$

$$\alpha_{t,c} \cdot f_{t,c}$$

- where  $f_{t,c}$  is the frame-level logit for class  $c$  at time  $t$ , and  $\alpha_{t,c}$  is an attention weight. The clip-level prediction

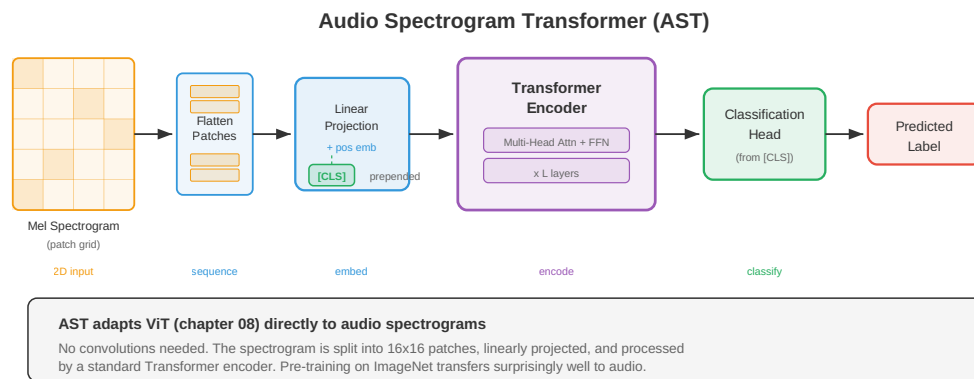
⌚

Y

$c$  is trained against the clip-level label.

- **Acoustic scene classification (ASC)** categorises the overall environment: “airport” , “park” , “metro station” , “office” . This is a holistic task: the model must capture the general acoustic texture rather than specific events. The DCASE challenge series benchmarks ASC annually, with winning systems typically using ensembles of CNNs on multi-resolution spectrograms.
- **Audio embeddings** are general-purpose representations learned from large-scale audio data, analogous to word embeddings (chapter 07) or image features (chapter 08) that transfer to downstream tasks.
- **VGGish** (Hershey et al., 2017) adapts the VGG image classification network (chapter 08) to audio. It processes 0.96-second log-mel spectrogram patches through a VGG-like CNN pre-trained on AudioSet, producing a 128-dimensional embedding per patch. VGGish embeddings serve as general-purpose audio features for downstream tasks, similar to how ImageNet-pretrained CNNs provide visual features.
- **PANNs** (Pre-trained Audio Neural Networks, Kong et al., 2020) are a family of CNN architectures (CNN6, CNN10, CNN14) trained on the full AudioSet for audio tagging. CNN14, the most widely used, is a 14-layer CNN with  $3 \times 3$  convolutions applied to log-mel spectrograms. PANNs produce 2048-dimensional embeddings that achieve state-of-the-art transfer learning on diverse audio tasks.
- **Audio Spectrogram Transformer (AST)** (Gong et al., 2021) applies the Vision Transformer (ViT, chapter 08) architecture directly to audio spectrograms. The spectrogram is split into  $16 \times 16$  patches (just like ViT splits images), each patch is linearly projected to a token embedding, positional embeddings are added,

and a standard transformer encoder (chapter 07) processes the sequence. A [CLS] token's output is used for classification.



Audio Spectrogram Transformer: the mel spectrogram is divided into patches, each patch is flattened and linearly projected into a token, positional embeddings are added, and a transformer encoder produces the classification output via a CLS token

- AST benefits from **ImageNet pre-training**: since spectrograms are 2D images, AST initialises from a ViT pre-trained on ImageNet images, then fine-tunes on audio. This cross-modal transfer is surprisingly effective because both domains share low-level features (edges, textures) and the positional embeddings can be interpolated to handle different spectrogram sizes.
- **HTS-AT** (Chen et al., 2022) improves on AST with a hierarchical Swin Transformer architecture (chapter 08's shifted-window attention), reducing computational cost while improving performance through multi-scale feature extraction.
- **BEATs** (Chen et al., 2023) uses an audio-specific pre-training strategy: iterative masked prediction with a discrete tokeniser (similar to wav2vec 2.0's approach from file 02 but applied to general audio). The tokeniser is progressively refined, creating increasingly semantically meaningful discrete audio tokens.
- **Speaker diarisation with embeddings** combines speaker embeddings with temporal modelling. Modern systems like Pyannote.audio use a three-stage pipeline: (1) a neural segmentation model that detects speaker turns and overlapping speech, (2) an embedding extraction stage (ECAPA-TDNN) applied to each detected segment, and (3) clustering to assign speaker identities across the recording.

- **Music information retrieval (MIR)** applies audio analysis to music. The spectral representations from file 01 are particularly useful here because music has rich harmonic structure.
- **Beat tracking** detects the rhythmic pulse of music. The standard approach computes an **onset strength envelope** from the spectrogram (detecting energy increases that signal note onsets), then finds the tempo using autocorrelation or a tempogram, and finally tracks individual beat positions using dynamic programming to find the sequence of beat times that best matches the onset envelope while maintaining a consistent tempo.
- **Chord recognition** identifies the harmonic content over time. The input is typically a **chromagram** (also called a pitch class profile): a 12-dimensional representation that folds all octaves together, showing the energy in each of the 12 pitch classes (C, C#, D, ..., B). A CNN or RNN (chapter 06) classifies each time frame into one of the standard chord labels (C major, A minor, G7, etc.).
- The chromagram is computed from the STFT (file 01) by mapping each frequency bin to its pitch class:

$$\text{chroma}(p) = \sum_{k: \text{pitch}(k) \bmod 12 = p} |X(k)|^2$$

- where  $p \in \{0, 1, \dots, 11\}$  is the pitch class and  $\text{pitch}(k)$  maps frequency bin  $k$  to its MIDI note number.
- **Source separation basics** (detailed further in file 05) separate a music recording into individual instruments (vocals, drums, bass, other). This is central to MIR applications like remixing, karaoke, and music transcription. Models like Demucs (file 05) achieve remarkably good separation quality on the standard MUSDB18 benchmark.
- **Music tagging** assigns labels to songs (genre, mood, instruments, era). It is essentially audio classification applied to music, using the same CNN-on-spectrogram approach. The Million Song Dataset and MagnaTagATune are standard benchmarks.
- **Audio fingerprinting** identifies a specific recording from a short excerpt, even with noise, reverberation, or compression artifacts. The classic system is Shazam, which hashes constellation points (prominent peaks in the

spectrogram). Neural approaches learn robust embeddings that are invariant to acoustic degradation while remaining discriminative between different recordings, echoing the invariant feature learning from chapter 06 and chapter 08.

## 44.1 Coding Tasks (use CoLab or notebook)

---

- **Task 1: Speaker embedding extraction with statistics pooling.** Build a simple x-vector-style model that processes frame-level features through TDNN layers and statistics pooling to produce speaker embeddings.

```
import jax
import jax.numpy as jnp
import jax.random as jr
import matplotlib.pyplot as plt

Simulate frame-level MFCC features for multiple speakers
def generate_speaker_data(key, n_speakers=5, utterances_per_speaker=20,
 n_frames=100, n_features=40):
 """Generate synthetic speaker data with speaker-dependent patterns."""
 keys = jr.split(key, 3)
 all_features = []
 all_labels = []

 # Each speaker has a characteristic spectral pattern
 speaker_patterns = jr.normal(keys[0], (n_speakers, n_features)) * 0.5

 for spk in range(n_speakers):
 for utt in range(utterances_per_speaker):
 k = jr.fold_in(keys[1], spk * utterances_per_speaker + utt)
 noise = jr.normal(k, (n_frames, n_features)) * 0.3
 features = speaker_patterns[spk][None, :] + noise
 all_features.append(features)
 all_labels.append(spk)

 perm = jr.permutation(keys[2], len(all_features))
 features = jnp.stack(all_features)[perm]
 labels = jnp.array(all_labels)[perm]
 return features, labels

key = jr.PRNGKey(42)
features, labels = generate_speaker_data(key)
n_speakers = 5
n_features = 40
```

```

x-vector-style model
def init_xvector(key, n_features=40, hidden=128, embed_dim=64, n_speakers=5):
 keys = jr.split(key, 8)
 params = {
 # TDNN layer 1: context [-2, 2]
 'tdnn1_w': jr.normal(keys[0], (5, n_features, hidden)) * jnp.sqrt(2.0 /
(5 * n_features)),
 'tdnn1_b': jnp.zeros(hidden),
 # TDNN layer 2: context [-2, 2]
 'tdnn2_w': jr.normal(keys[1], (5, hidden, hidden)) * jnp.sqrt(2.0 / (5 *
hidden)),
 'tdnn2_b': jnp.zeros(hidden),
 # TDNN layer 3: context [-3, 3]
 'tdnn3_w': jr.normal(keys[2], (7, hidden, hidden)) * jnp.sqrt(2.0 / (7 *
hidden)),
 'tdnn3_b': jnp.zeros(hidden),
 # Segment-level layers (after pooling: 2*hidden -> embed_dim)
 'seg1_w': jr.normal(keys[3], (2 * hidden, embed_dim)) * jnp.sqrt(2.0 /
(2 * hidden)),
 'seg1_b': jnp.zeros(embed_dim),
 # Classification head
 'cls_w': jr.normal(keys[4], (embed_dim, n_speakers)) * jnp.sqrt(2.0 /
embed_dim),
 'cls_b': jnp.zeros(n_speakers),
 }
 return params

def xvector_forward(params, x, return_embedding=False):
 """x: (batch, frames, features) -> logits or embeddings."""
 # TDNN layers (1D convolutions)
 h = jax.lax.conv_general_dilated(
 x.transpose(0, 2, 1), params['tdnn1_w'].transpose(2, 1, 0),
 window_strides=(1,), padding='SAME'
).transpose(0, 2, 1) + params['tdnn1_b']
 h = jax.nn.relu(h)

 h = jax.lax.conv_general_dilated(
 h.transpose(0, 2, 1), params['tdnn2_w'].transpose(2, 1, 0),
 window_strides=(1,), padding='SAME'
).transpose(0, 2, 1) + params['tdnn2_b']
 h = jax.nn.relu(h)

 h = jax.lax.conv_general_dilated(
 h.transpose(0, 2, 1), params['tdnn3_w'].transpose(2, 1, 0),
 window_strides=(1,), padding='SAME'
).transpose(0, 2, 1) + params['tdnn3_b']

```

```

h = jax.nn.relu(h)

Statistics pooling: mean and std over time
mu = jnp.mean(h, axis=1)
sigma = jnp.std(h, axis=1)
pooled = jnp.concatenate([mu, sigma], axis=-1)

Segment-level layer -> embedding
embedding = jax.nn.relu(pooled @ params['seg1_w'] + params['seg1_b'])

if return_embedding:
 return embedding

Classification
logits = embedding @ params['cls_w'] + params['cls_b']
return logits

def cross_entropy_loss(params, features, labels):
 logits = xvector_forward(params, features)
 one_hot = jax.nn.one_hot(labels, n_speakers)
 log_probs = jax.nn.log_softmax(logits)
 return -jnp.mean(jnp.sum(one_hot * log_probs, axis=-1))

grad_fn = jax.jit(jax.value_and_grad(cross_entropy_loss))

Train
params = init_xvector(jr.PRNGKey(0))
lr = 1e-3
losses = []

for epoch in range(300):
 loss_val, grads = grad_fn(params, features, labels)
 params = jax.tree.map(lambda p, g: p - lr * g, params, grads)
 losses.append(float(loss_val))

Extract embeddings and visualise with t-SNE-style 2D projection (using PCA)
embeddings = xvector_forward(params, features, return_embedding=True)

Simple PCA to 2D
emb_centered = embeddings - jnp.mean(embeddings, axis=0)
_, _, Vt = jnp.linalg.svd(emb_centered, full_matrices=False)
proj_2d = emb_centered @ Vt[:2].T

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].plot(losses, color='#3498db', linewidth=1.5)
axes[0].set_xlabel('Epoch')

```



```

axes[0].set_ylabel('Cross-Entropy Loss')
axes[0].set_title('Speaker Classification Training')
axes[0].set_yscale('log')

colors = ['#3498db', '#e74c3c', '#27ae60', '#f39c12', '#9b59b6']
for spk in range(n_speakers):
 mask = labels == spk
 axes[1].scatter(proj_2d[mask, 0], proj_2d[mask, 1], c=colors[spk],
 label=f'Speaker {spk}', alpha=0.7, s=30)
axes[1].set_xlabel('PC 1')
axes[1].set_ylabel('PC 2')
axes[1].set_title('Speaker Embeddings (PCA projection)')
axes[1].legend()

plt.tight_layout()
plt.show()

Verification demo: cosine similarity
emb_norm = embeddings / jnp.linalg.norm(embeddings, axis=-1, keepdims=True)
sim_matrix = emb_norm @ emb_norm.T
print(f"Embedding shape: {embeddings.shape}")
print(f"Avg same-speaker similarity: {jnp.mean(sim_matrix[labels[:, None] ==
labels[None, :]]):.4f}")
print(f"Avg diff-speaker similarity: {jnp.mean(sim_matrix[labels[:, None] !=
labels[None, :]]):.4f}")

```

- **Task 2: Speaker verification with cosine similarity scoring.** Given pre-computed speaker embeddings, implement a verification system that computes EER (Equal Error Rate) and plots the DET curve.

```

import jax
import jax.numpy as jnp
import jax.random as jr
import matplotlib.pyplot as plt

def generate_verification_pairs(key, n_speakers=20, dim=64, n_pairs=2000):
 """Generate speaker embeddings and verification trial pairs."""
 keys = jr.split(key, 5)

 # Speaker centroids with some variance
 centroids = jr.normal(keys[0], (n_speakers, dim))
 centroids = centroids / jnp.linalg.norm(centroids, axis=-1, keepdims=True)

 # Generate enrollment and test embeddings with intra-speaker variance
 enroll_embs = []
 test_embs = []

```

```

trial_labels = [] # 1 = same speaker (target), 0 = different (impostor)

for i in range(n_pairs):
 k1, k2, k3 = jr.split(jr.fold_in(keys[1], i), 3)
 is_target = jr.bernoulli(k1).astype(int)

 spk1 = jr.randint(k2, (), 0, n_speakers)
 emb1 = centroids[spk1] + jr.normal(jr.fold_in(k3, 0), (dim,)) * 0.15

 if is_target:
 spk2 = spk1
 else:
 spk2 = (spk1 + jr.randint(jr.fold_in(k3, 1), (), 1, n_speakers)) %
n_speakers

 emb2 = centroids[spk2] + jr.normal(jr.fold_in(k3, 2), (dim,)) * 0.15

 enroll_embs.append(emb1)
 test_embs.append(emb2)
 trial_labels.append(int(is_target))

return (jnp.stack(enroll_embs), jnp.stack(test_embs),
 jnp.array(trial_labels))

key = jr.PRNGKey(42)
enroll, test, labels = generate_verification_pairs(key)

Compute cosine similarity scores
enroll_norm = enroll / jnp.linalg.norm(enroll, axis=-1, keepdims=True)
test_norm = test / jnp.linalg.norm(test, axis=-1, keepdims=True)
scores = jnp.sum(enroll_norm * test_norm, axis=-1)

Compute FAR and FRR at various thresholds
thresholds = jnp.linspace(-1.0, 1.0, 500)

target_scores = scores[labels == 1]
impostor_scores = scores[labels == 0]

fars = []
frrs = []
for thresh in thresholds:
 far = jnp.mean(impostor_scores >= thresh) # false accepts
 frr = jnp.mean(target_scores < thresh) # false rejects
 fars.append(float(far))
 frrs.append(float(frr))

fars = jnp.array(fars)

```

```

frrs = jnp.array(frrs)

Find EER: where FAR ≈ FRR
eer_idx = jnp.argmin(jnp.abs(fars - frrs))
eer = float((fars[eer_idx] + frrs[eer_idx]) / 2)
eer_threshold = float(thresholds[eer_idx])

print(f"Equal Error Rate (EER): {eer:.4f} ({eer*100:.2f}%)")
print(f"EER threshold: {eer_threshold:.4f}")

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

Score distributions
bins = jnp.linspace(-0.5, 1.0, 60)
axes[0].hist(target_scores, bins=bins, alpha=0.6, color='#27ae60',
 label='Target (same speaker)', density=True)
axes[0].hist(impostor_scores, bins=bins, alpha=0.6, color='#e74c3c',
 label='Impostor (different speaker)', density=True)
axes[0].axvline(eer_threshold, color='#f39c12', linestyle='--', linewidth=2,
 label=f'EER threshold = {eer_threshold:.3f}')
axes[0].set_xlabel('Cosine Similarity Score')
axes[0].set_ylabel('Density')
axes[0].set_title('Score Distributions')
axes[0].legend()

FAR vs FRR
axes[1].plot(thresholds, fars, color='#e74c3c', linewidth=2, label='FAR')
axes[1].plot(thresholds, frrs, color='#3498db', linewidth=2, label='FRR')
axes[1].axvline(eer_threshold, color='#f39c12', linestyle='--', linewidth=1.5)
axes[1].scatter([eer_threshold], [eer], color='#f39c12', s=100, zorder=5,
 label=f'EER = {eer:.4f}')
axes[1].set_xlabel('Threshold')
axes[1].set_ylabel('Error Rate')
axes[1].set_title('FAR and FRR vs Threshold')
axes[1].legend()

DET curve (FAR vs FRR)
axes[2].plot(fars, frrs, color='#9b59b6', linewidth=2)
axes[2].plot([0, 1], [0, 1], 'k--', alpha=0.3)
axes[2].scatter([eer], [eer], color='#f39c12', s=100, zorder=5,
 label=f'EER = {eer:.4f}')
axes[2].set_xlabel('False Acceptance Rate')
axes[2].set_ylabel('False Rejection Rate')
axes[2].set_title('DET Curve')
axes[2].set_xlim([0, 0.5])
axes[2].set_ylim([0, 0.5])
axes[2].legend()

```

```

axes[2].set_aspect('equal')

plt.tight_layout()
plt.show()

```

· **Task 3: Audio spectrogram patch embedding (AST-style).** Implement the patch extraction and embedding layer of the Audio Spectrogram Transformer, visualising how a spectrogram is tokenised.

```

import jax
import jax.numpy as jnp
import jax.random as jr
import matplotlib.pyplot as plt

Generate a synthetic spectrogram (harmonic structure + noise)
def generate_spectrogram(key, n_time=128, n_freq=128):
 """Create a synthetic spectrogram with harmonic patterns."""
 k1, k2 = jr.split(key)
 spec = jr.normal(k1, (n_time, n_freq)) * 0.1

 # Add harmonic bands (simulating speech formants)
 for f0 in [15, 30, 45, 70]:
 width = 3
 envelope = jnp.exp(-0.5 * ((jnp.arange(n_freq) - f0) / width) ** 2)
 time_mod = 0.5 + 0.5 * jnp.sin(2 * jnp.pi * jnp.arange(n_time) / 40)
 spec += jnp.outer(time_mod, envelope)

 return jnp.clip(spec, 0, None)

key = jr.PRNGKey(42)
spectrogram = generate_spectrogram(key)
n_time, n_freq = spectrogram.shape

Patch extraction parameters
patch_h = 16 # time
patch_w = 16 # frequency
stride_h = 16
stride_w = 16
embed_dim = 192 # ViT-Small dimension

n_patches_h = n_time // stride_h
n_patches_w = n_freq // stride_w
n_patches = n_patches_h * n_patches_w

print(f"Spectrogram: {n_time} x {n_freq}")
print(f"Patch size: {patch_h} x {patch_w}")

```

```

print(f"Number of patches: {n_patches_h} x {n_patches_w} = {n_patches}")

Extract patches
def extract_patches(spec, patch_h, patch_w, stride_h, stride_w):
 """Extract non-overlapping patches from spectrogram."""
 patches = []
 positions = []
 for i in range(0, spec.shape[0] - patch_h + 1, stride_h):
 for j in range(0, spec.shape[1] - patch_w + 1, stride_w):
 patch = spec[i:i+patch_h, j:j+patch_w]
 patches.append(patch.flatten())
 positions.append((i, j))
 return jnp.stack(patches), positions

patches, positions = extract_patches(spectrogram, patch_h, patch_w, stride_h,
stride_w)
print(f"Patches shape: {patches.shape}") # (n_patches, patch_h * patch_w)

Linear projection (patch embedding)
patch_dim = patch_h * patch_w
k1, k2 = jr.split(jr.PRNGKey(0))
W_embed = jr.normal(k1, (patch_dim, embed_dim)) * jnp.sqrt(2.0 / patch_dim)
b_embed = jnp.zeros(embed_dim)

Learnable positional embeddings
pos_embed = jr.normal(k2, (n_patches + 1, embed_dim)) * 0.02 # +1 for CLS

CLS token
cls_token = jnp.zeros((1, embed_dim))

Forward pass
patch_tokens = patches @ W_embed + b_embed # (n_patches, embed_dim)
tokens = jnp.concatenate([cls_token, patch_tokens], axis=0) # (n_patches+1,
embed_dim)
tokens = tokens + pos_embed # Add positional embeddings

print(f"Token sequence shape: {tokens.shape}")
print(f"Each token has dimension: {embed_dim}")

Visualisation
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

Original spectrogram with patch grid
axes[0, 0].imshow(spectrogram.T, aspect='auto', origin='lower', cmap='magma')
for i in range(0, n_time + 1, stride_h):
 axes[0, 0].axvline(i - 0.5, color='white', linewidth=0.5, alpha=0.5)
for j in range(0, n_freq + 1, stride_w):

```

```

 axes[0, 0].axhline(j - 0.5, color='white', linewidth=0.5, alpha=0.5)
 axes[0, 0].set_title(f'Spectrogram with {patch_h}x{patch_w} Patch Grid')
 axes[0, 0].set_xlabel('Time frame')
 axes[0, 0].set_ylabel('Frequency bin')

 # Individual patches visualised
 n_show = min(16, n_patches)
 patch_grid = patches[:n_show].reshape(n_show, patch_h, patch_w)
 combined = jnp.concatenate([patch_grid[i] for i in range(min(8, n_show))],
axis=1)

 axes[0, 1].imshow(combined.T, aspect='auto', origin='lower', cmap='magma')
 axes[0, 1].set_title(f'First {min(8, n_show)} Patches (concatenated)')
 axes[0, 1].set_xlabel('Patch index (horizontal)')
 axes[0, 1].set_ylabel('Frequency within patch')

 # Token embeddings similarity matrix
 token_norms = tokens / jnp.linalg.norm(tokens, axis=-1, keepdims=True)
 sim = token_norms @ token_norms.T
 im = axes[1, 0].imshow(sim, cmap='RdBu_r', vmin=-1, vmax=1)
 axes[1, 0].set_title('Token Similarity Matrix (cosine)')
 axes[1, 0].set_xlabel('Token index')
 axes[1, 0].set_ylabel('Token index')
 plt.colorbar(im, ax=axes[1, 0], fraction=0.046)

 # Positional embedding similarity
 pos_norms = pos_embed / jnp.linalg.norm(pos_embed, axis=-1, keepdims=True)
 pos_sim = pos_norms @ pos_norms.T
 im2 = axes[1, 1].imshow(pos_sim, cmap='RdBu_r', vmin=-1, vmax=1)
 axes[1, 1].set_title('Positional Embedding Similarity')
 axes[1, 1].set_xlabel('Position index')
 axes[1, 1].set_ylabel('Position index')
 plt.colorbar(im2, ax=axes[1, 1], fraction=0.046)

 plt.tight_layout()
 plt.show()

```

- **Task 4: Simple chromagram computation for chord analysis.** Compute and visualise a chromagram from a synthetic harmonic signal, demonstrating the pitch class folding used in music information retrieval.

```

import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

Generate a synthetic musical signal: C major chord -> G major chord
sr = 16000

```

```

duration = 2.0
t = jnp.linspace(0, duration, int(sr * duration))

C major (C4=261.6, E4=329.6, G4=392.0) for first half
G major (G3=196.0, B3=246.9, D4=293.7) for second half
half = len(t) // 2

c_major = (0.5 * jnp.sin(2 * jnp.pi * 261.63 * t[:half]) +
 0.4 * jnp.sin(2 * jnp.pi * 329.63 * t[:half]) +
 0.3 * jnp.sin(2 * jnp.pi * 392.00 * t[:half]))

g_major = (0.5 * jnp.sin(2 * jnp.pi * 196.00 * t[:half]) +
 0.4 * jnp.sin(2 * jnp.pi * 246.94 * t[:half]) +
 0.3 * jnp.sin(2 * jnp.pi * 293.66 * t[:half]))

signal = jnp.concatenate([c_major, g_major])

Compute STFT
n_fft = 4096 # high resolution for pitch accuracy
hop_length = 512
window = jnp.hanning(n_fft)

def stft(signal, n_fft, hop_length, window):
 n_frames = 1 + (len(signal) - n_fft) // hop_length
 frames = jnp.stack([
 signal[i * hop_length : i * hop_length + n_fft] * window
 for i in range(n_frames)
])
 return jnp.fft.rfft(frames, n=n_fft)

S = stft(signal, n_fft, hop_length, window)
power_spec = jnp.abs(S) ** 2
freqs = jnp.fft.rfftfreq(n_fft, 1.0 / sr)

Compute chromagram by mapping frequency bins to pitch classes
MIDI note number from frequency: 69 + 12 * log2(f / 440)
note_names = ['C', 'C#', 'D', 'D#', 'E', 'F', 'F#', 'G', 'G#', 'A', 'A#', 'B']

def freq_to_chroma(freq):
 """Map frequency to pitch class (0-11). Returns -1 for freq <= 0."""
 midi = 69 + 12 * jnp.log2(jnp.clip(freq, 1e-10, None) / 440.0)
 return jnp.round(midi).astype(int) % 12

Build chromagram: sum power spectrum energy for each pitch class
chromagram = jnp.zeros((power_spec.shape[0], 12))
valid_freqs = freqs[1:] # skip DC
valid_power = power_spec[:, 1:]

```

```

for p in range(12):
 # Find frequency bins belonging to this pitch class
 chroma_bins = freq_to_chroma(valid_freqs)
 mask = (chroma_bins == p).astype(jnp.float32)
 chromagram = chromagram.at[:, p].set(
 jnp.sum(valid_power * mask[None, :], axis=1)
)

Normalise each frame
chromagram = chromagram / (jnp.max(chromagram, axis=1, keepdims=True) + 1e-8)

Visualisation
fig, axes = plt.subplots(3, 1, figsize=(14, 10))

Waveform
axes[0].plot(t[:3000], signal[:3000], color='#3498db', linewidth=0.5,
 label='C major')
axes[0].plot(t[half:half+3000], signal[half:half+3000], color='#e74c3c',
 linewidth=0.5, label='G major')
axes[0].set_title('Waveform: C major → G major')
axes[0].set_ylabel('Amplitude')
axes[0].set_xlabel('Time (s)')
axes[0].legend()

Spectrogram (log scale)
time_axis = jnp.arange(power_spec.shape[0]) * hop_length / sr
axes[1].imshow(jnp.log1p(power_spec[:, :500]).T, aspect='auto', origin='lower',
 cmap='magma', extent=[0, time_axis[-1], 0, freqs[500]])
axes[1].set_title('Power Spectrogram')
axes[1].set_ylabel('Frequency (Hz)')
axes[1].set_xlabel('Time (s)')

Chromagram
im = axes[2].imshow(chromagram.T, aspect='auto', origin='lower', cmap='YlOrRd',
 extent=[0, time_axis[-1], -0.5, 11.5])
axes[2].set_yticks(range(12))
axes[2].set_yticklabels(note_names)
axes[2].set_title('Chromagram (pitch class energy over time)')
axes[2].set_ylabel('Pitch class')
axes[2].set_xlabel('Time (s)')
plt.colorbar(im, ax=axes[2], fraction=0.046, label='Normalised energy')

Mark expected active pitch classes
mid_frame = chromagram.shape[0] // 2
print(f"C major region - expected: C, E, G")
print(f"Chroma values: {dict(zip(note_names, [f'{v:.2f}' for v in

```



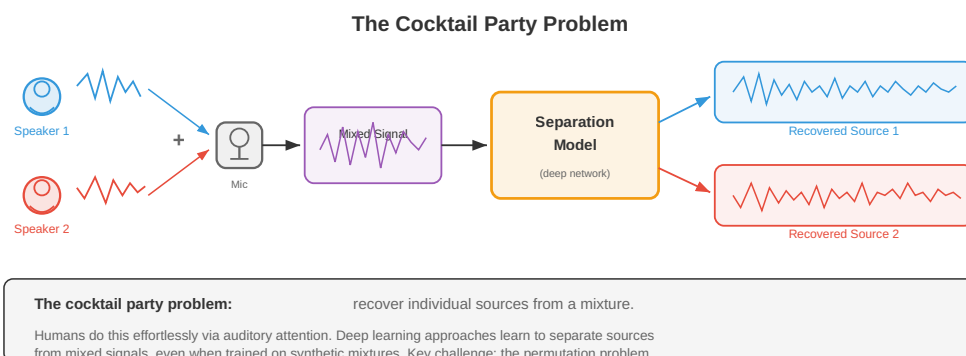
```
chromagram[mid_frame//2]]))}")
 print(f"G major region - expected: G, B, D")
 print(f"Chroma values: {dict(zip(note_names, [f'{v:.2f}' for v in
chromagram[mid_frame + mid_frame//2]]))}")

plt.tight_layout()
plt.show()
```

## 45 Source Separation and Noise Cancellation

Source separation and noise cancellation recover individual signals from mixed audio; the computational cocktail party problem. This file covers ICA, NMF, time-frequency masking, beamforming, deep learning separation networks (Conv-TasNet, SepFormer), speech enhancement, and adaptive noise cancellation.

- Imagine standing at a crowded cocktail party. Dozens of people are talking simultaneously, music is playing, glasses are clinking, yet you can focus on one conversation and follow it clearly. This remarkable ability, the **cocktail party problem** (Cherry, 1953), is something the human auditory system solves effortlessly but machines find extraordinarily difficult. This file covers the algorithms that attempt it: separating mixed audio sources, cancelling unwanted noise, and enhancing speech in adverse conditions.
- The signal-processing foundations from file 01 (STFT, spectrograms, filterbanks) underpin every method here. The matrix decomposition techniques from chapter 02 (NMF, ICA, SVD) provide the classical toolkit. The deep learning architectures from chapter 06 (CNNs, RNNs, attention) and the probability theory from chapter 04/05 inform the modern approaches.



The cocktail party problem: multiple speakers and sound sources are mixed at microphone arrays, and the separation system must recover individual source signals from the mixture

- **Problem formulation:** a mixture signal  $x(t)$  is observed at one or more microphones. The mixture is a sum (in the simplest case) of  $C$  source signals:

$$x(t) = \sum_{c=1}^C s_c(t) + n(t)$$

- where  $s_c(t)$  is the  $c$ -th source signal and  $n(t)$  is background noise. The goal is to recover the individual  $s_c(t)$  from  $x(t)$ . In the single-microphone case this is severely underdetermined: one equation,  $C$  unknowns. Additional assumptions (statistical independence, spectral structure, learned priors) are needed to make the problem tractable.
- In the frequency domain (via STFT from file 01), the mixture becomes:

$$X(t, f) = \sum_{c=1}^C S_c(t, f) + N(t, f)$$

- Many separation methods work in the time-frequency domain by estimating a **mask**  $M_c(t, f) \in [0, 1]$  for each source, then recovering the source as

$$\hat{s}_c(t, f) = M_c(t, f) \cdot X(t, f)$$

The **ideal binary mask (IBM)** sets  $M_c(t, f) = 1$  if source  $c$  dominates that time-frequency bin and 0 otherwise. The **ideal ratio mask (IRM)** is a soft version:

$$\text{IRM}_c(t, f) = \frac{|S_c(t, f)|^2}{\sum_{j=1}^c |S_j(t, f)|^2}$$

- **Independent Component Analysis (ICA)** is the classical approach when the number of microphones equals or exceeds the number of sources. ICA (chapter 02) finds a linear unmixing matrix  $W$  such that

↺

$\hat{s}$

$= Wx$ , where the recovered sources

↺

$s$

are maximally statistically independent. The key assumption is that source signals are non-Gaussian and independent, which is typically valid for speech and music.

- For the multi-microphone instantaneous mixing model  $x = As$  (where  $A$  is the mixing matrix), ICA recovers  $W \approx A^{-1}$  by maximising the non-Gaussianity of the outputs (FastICA uses negentropy) or by minimising mutual information. ICA works well in controlled settings but fails when mixing involves convolution (room reverberation), when sources outnumber microphones, or when the independence assumption is violated.
- **Non-negative Matrix Factorisation (NMF)** decomposes the magnitude spectrogram  $V \in \mathbb{R}_+^{F \times T}$  into a product of two non-negative matrices (chapter 02):

$$V \approx WH$$

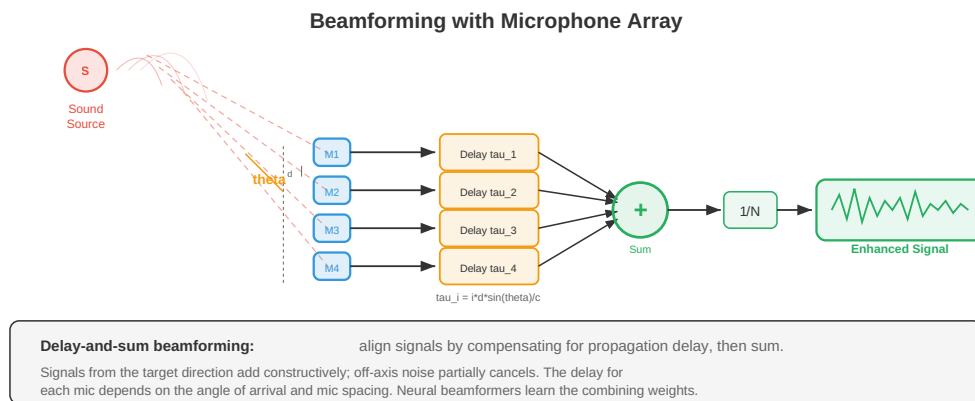
- where  $W \in \mathbb{R}_+^{F \times K}$  is a dictionary of  $K$  spectral basis vectors and  $H \in \mathbb{R}_+^{K \times T}$  contains the activation coefficients over time. The non-negativity constraint is physically motivated: magnitudes are non-negative, and sounds combine additively.
- For source separation, NMF learns separate dictionaries for each source:  $W_{\text{speech}}$  captures the spectral patterns of speech (formant structures), while  $W_{\text{noise}}$  captures noise patterns. The mixture is decomposed as

$V \approx W_{\text{speech}} H_{\text{speech}} + W_{\text{noise}} H_{\text{noise}}$ , and each source is recovered by masking.

NMF is minimised using multiplicative updates with either the Frobenius norm or KL divergence as the cost function:

```
\begin{aligned}
\text{Frobenius:} \quad D_F(V \parallel WH) &= \|V - WH\|_F^2 \\
\text{KL:} \quad D_{\text{KL}}(V \parallel WH) &= \sum_{f,t} \left[V_{ft} \log \frac{V_{ft}}{(WH)_{ft}} - V_{ft} + (WH)_{ft} \right]
\end{aligned}
```

- **Beamforming** exploits spatial information from microphone arrays. When a source signal arrives at different microphones with different delays (due to the spatial arrangement), these delays can be used to enhance the signal from one direction while suppressing others.



Beamforming: a microphone array receives signals from different directions with different time delays, and the beamformer combines them to enhance the desired direction while suppressing others

- **Delay-and-sum beamforming** is the simplest approach. If the desired source is at angle  $\theta$  relative to the array, the time delay at microphone  $m$  is  $\tau_m(\theta) = d_m \sin \theta / c$ , where  $d_m$  is the microphone position and  $c$  is the speed of sound. The beamformer output aligns and sums the microphone signals:

$$y(t) = \frac{1}{M}$$

M  
 $\sum$   
 $m = 1$

$$x_m(t - \tau_m(\theta))$$

- Signals from the target direction add coherently, while signals from other directions add incoherently, providing spatial filtering. The array geometry determines the spatial resolution: larger arrays give narrower beams.
- **Minimum Variance Distortionless Response (MVDR)** beamforming optimises the weights to minimise total output power while passing the target direction without distortion:

```
\begin{aligned}
\min_{\mathbf{w}} \quad & \mathbf{w}^H \Phi_{nn} \mathbf{w} \\
\text{subject to} \quad & \mathbf{w}^H \mathbf{d}(\theta) = 1
\end{aligned}
```

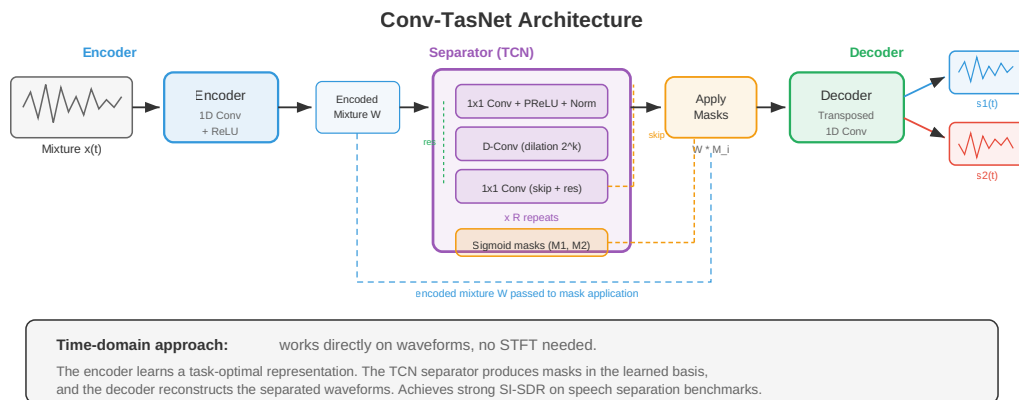
- where  $\Phi_{nn}$  is the noise spatial covariance matrix and  $\mathbf{d}(\theta)$  is the steering vector for direction  $\theta$ . The closed-form solution is:

$$\mathbf{w}_{\text{MVDR}} = \frac{\Phi_{nn}^{-1} \mathbf{d}(\theta)}{\mathbf{d}(\theta)^H \Phi_{nn}^{-1} \mathbf{d}(\theta)}$$

- MVDR adapts to the noise environment by using the estimated noise covariance, providing better interference rejection than delay-and-sum. It is widely used in hearing aids, smart speakers, and teleconferencing systems.
- **Deep learning for source separation** has dramatically improved performance, especially in the single-microphone case where classical methods struggle. The general paradigm is: encode the mixture, estimate masks or source representations with a neural network, and decode to recover individual sources.
- **Deep clustering** (Hershey et al., 2016) embeds each time-frequency bin into a high-dimensional space where bins belonging to the same source are close together and bins from different sources are far apart. A bidirectional LSTM (chapter 06) maps each T-F bin  $(t, f)$  to an embedding  $v_{t,f} \in \mathbb{R}^D$ . The training objective is:

$$\mathcal{L} = \|VV^T - YY^T\|_F^2$$

- where  $V$  is the matrix of embeddings and  $Y$  is the one-hot matrix of source assignments. The product  $VV^T$  is an affinity matrix (how similar two bins' embeddings are), and  $YY^T$  is the ideal affinity (1 if same source, 0 otherwise). At inference, K-means clustering on the embeddings produces binary masks.
- **Conv-TasNet** (Luo and Mesgarani, 2019) operates entirely in the time domain, bypassing the STFT. It has three components:



Conv-TasNet architecture: an encoder converts the mixture waveform to a latent representation, a temporal convolutional network separator estimates source masks, and a decoder reconstructs individual source waveforms

- **Encoder:** a 1D convolution maps short segments of the mixture waveform to a latent representation. For a mixture  $x \in \mathbb{R}^T$ , the encoder output is  $w = \text{ReLU}(U * x) \in \mathbb{R}^N \times L$ , where  $U$  is a learnable basis (analogous to the STFT basis but learned from data),  $N$  is the number of basis functions, and  $L$  is the number of segments. The encoder kernel size and stride (typically 2ms and 1ms) determine the temporal resolution.
- **Separator:** a **Temporal Convolutional Network (TCN)** processes the encoded mixture and outputs  $C$  masks. The TCN stacks dilated 1D depthwise separable convolutions (from chapter 08's efficient convolutions) in blocks with exponentially increasing dilation factors  $1, 2, 4, \dots, 2^{B-1}$ , repeated  $R$  times. This gives a very large receptive field while keeping computation efficient.
- **Decoder:** a transposed 1D convolution (with a learned basis  $V$ ) converts each masked representation back to the time domain:



S

$$c = V^T(M_c \odot w).$$

- Conv-TasNet significantly outperforms spectrogram-based methods because the learned encoder-decoder basis can capture information (particularly phase) that the STFT magnitude discards.
- **Dual-Path RNN (DPRNN)** (Luo et al., 2020) addresses the long sequence modelling problem in separation. Rather than processing the entire encoded sequence with a single RNN or TCN, DPRNN splits the sequence into overlapping chunks and applies RNNs along two paths: an **intra-chunk** path (modelling local patterns within each chunk) and an **inter-chunk** path (modelling global patterns across chunks). This reduces the RNN sequence length from  $L$  to

---


$$\sqrt{L}$$

in each dimension:

```
\begin{aligned}
\text{Intra-chunk:} \quad & h_{k,n}^{\text{intra}} = \text{BiLSTM}_{\text{intra}}(z_{k,n}) \\
\text{Inter-chunk:} \quad & h_{k,n}^{\text{inter}} = \text{BiLSTM}_{\text{inter}}(h_{k,n}^{\text{intra}})
\end{aligned}
```

- where  $k$  indexes the chunk and  $n$  indexes the position within the chunk. The intra-chunk LSTM processes across  $n$  for a fixed  $k$ ; the inter-chunk LSTM processes across  $k$  for a fixed  $n$ .
- **SepFormer** (Subakan et al., 2021) replaces the RNNs in the dual-path framework with transformers (chapter 07). The intra-chunk transformer captures local dependencies with self-attention, and the inter-chunk transformer captures global dependencies. Multi-head attention's ability to model long-range dependencies without the vanishing gradient problem (chapter 06) makes SepFormer particularly effective for long recordings. SepFormer achieves state-of-the-art results on the WSJ0-2mix benchmark.
- **Permutation Invariant Training (PIT)** solves a fundamental problem in supervised source separation: the label assignment ambiguity. If the network has two outputs (for two speakers), which output should correspond to which

speaker? There is no natural ordering. PIT computes the loss for all possible assignments and takes the minimum:

$$\mathcal{L}_{\text{PIT}} = \min_{\pi \in \mathcal{P}} \sum_{c=1}^C \ell(s_{\pi(c)}, s_c)$$

- where  $\mathcal{P}$  is the set of all permutations of  $\{1, \dots, C\}$  and  $\ell$  is the per-source loss (typically scale-invariant signal-to-distortion ratio, SI-SDR). For  $C = 2$  sources there are only 2 permutations; for  $C = 3$  there are 6. This is computed efficiently using the Hungarian algorithm for larger  $C$ .
- **Scale-Invariant Signal-to-Distortion Ratio (SI-SDR)** is the standard evaluation metric for source separation:

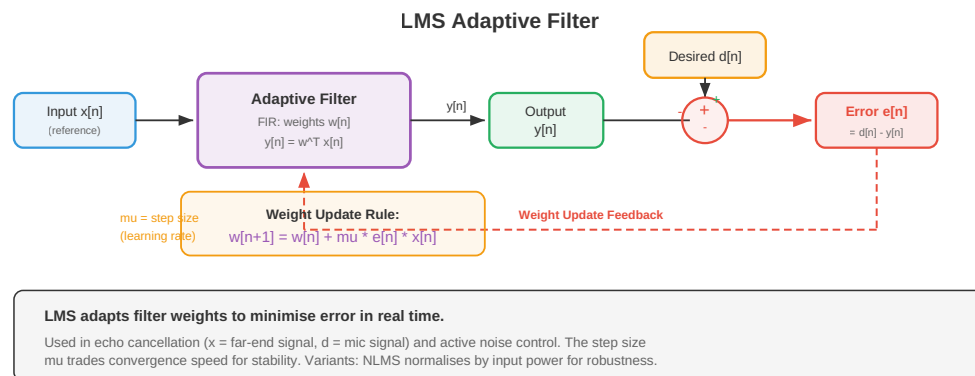
```
\begin{aligned}
s_{\text{target}} &= \frac{\langle \hat{s}, s \rangle}{\|s\|^2} s \\
e_{\text{noise}} &= \hat{s} - s_{\text{target}} \\
\text{SI-SDR} &= 10 \log_{10} \frac{\|s_{\text{target}}\|^2}{\|e_{\text{noise}}\|^2}
\end{aligned}
```

- where  $\hat{s}$  is the estimated source and  $s$  is the ground truth. SI-SDR is invariant to the overall scale of the estimate, which is desirable because absolute volume is less important than the quality of the separation. Higher SI-SDR (in dB) is better. State-of-the-art systems achieve around 20-22 dB SI-SDR improvement on WSJ0-2mix.
- **Music source separation** separates a music recording into stems: vocals, drums, bass, and other instruments. This enables applications like karaoke (remove vocals), remixing (adjust instrument levels), and transcription (analyse one instrument at a time).



- **Open-Unmix** (Stoter et al., 2019) is a reference baseline that uses a 3-layer bidirectional LSTM to predict a soft mask for each source in the magnitude STFT domain. It processes each source independently with a dedicated model. Simple but effective, Open-Unmix established reproducible benchmarks on MUSDB18.
- **Demucs** (Defossez et al., 2019; updated as Hybrid Demucs, 2021) uses a U-Net architecture (chapter 08) that operates directly on the waveform. The encoder compresses the mixture through strided convolutions, the decoder expands it back through transposed convolutions with skip connections, and each source gets its own decoder head. **Hybrid Demucs** combines time-domain and frequency-domain processing: the encoder has parallel time-domain and STFT branches whose features are fused before the decoder. This captures both fine temporal details and spectral structure.
- Demucs achieves state-of-the-art separation quality on MUSDB18, with particularly strong vocal separation. Its U-Net architecture is reminiscent of the image segmentation architectures from chapter 08, treating the separation problem as a form of “audio segmentation” .
- **Active noise cancellation (ANC)** reduces unwanted sound by generating an anti-noise signal that destructively interferes with the noise. Think of noise-cancelling headphones: a microphone picks up ambient noise, the ANC system generates an inverted version, and the combined signal (noise + anti-noise) ideally cancels to silence.
- The physics is simple: if the noise is  $n(t)$ , generating  $-n(t)$  at the same point in space produces silence:  $n(t) + (-n(t)) = 0$ . The challenge is that the anti-noise must be precisely aligned in time, amplitude, and phase. Even small errors produce residual noise or artifacts.
- **Feedforward ANC** uses a reference microphone that picks up the noise before it reaches the listener. The system has time to process the noise and generate the anti-noise. The reference signal passes through an adaptive filter whose output is subtracted from the noise at the error microphone (near the listener). This works well for predictable, broadband noise (engine hum, fan noise).
- **Feedback ANC** uses only an error microphone at the listener’s ear. The system estimates the noise from the residual signal (what the listener actually hears) and adjusts the anti-noise. Feedback ANC is simpler (no reference microphone needed) but has limited bandwidth and can become unstable.

- **Adaptive filtering** is the mathematical engine behind ANC. The filter coefficients must continuously adapt to the changing noise environment. The most common algorithm is the **Least Mean Squares (LMS)** filter.



LMS adaptive filter: the reference signal passes through an FIR filter, the output is subtracted from the desired signal to produce an error, and the error feeds back to update the filter coefficients

- **LMS algorithm:** an FIR filter with coefficients  $\mathbf{w} = [w_0, w_1, \dots, w_{L-1}]^T$  processes the reference signal  $\mathbf{x}(n) = [x(n), x(n-1), \dots, x(n-L+1)]^T$ . The output is  $y(n) = \mathbf{w}^T \mathbf{x}(n)$ , the error is  $e(n) = d(n) - y(n)$  (where  $d(n)$  is the desired/primary signal), and the weight update is:

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \mu e(n) \mathbf{x}(n)$$

- where  $\mu$  is the step size (learning rate). This is a stochastic gradient descent step on the mean squared error  $E[e^2(n)]$ , using the instantaneous gradient estimate  $-2e(n)\mathbf{x}(n)$  instead of the true gradient (chapter 03' s gradient descent and chapter 06' s SGD).
- The step size  $\mu$  controls the trade-off between convergence speed and steady-state error. Too large and the filter oscillates or diverges; too small and adaptation is sluggish. The stability condition is  $0 < \mu < 2/(\lambda_{\max})$ , where  $\lambda_{\max}$  is the largest eigenvalue of the input autocorrelation matrix  $R = E[\mathbf{x}\mathbf{x}^T]$ .
- **Normalised LMS (NLMS)** normalises the step size by the input power, making convergence independent of the signal level:

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \frac{\mu}{\|\mathbf{x}(n)\|^2 + \epsilon} e(n) \mathbf{x}(n)$$

- where  $\epsilon$  is a small regularisation constant to prevent division by zero. NLMS converges more reliably than LMS because the effective step size adapts to the input power.
- **Recursive Least Squares (RLS)** is a faster-converging alternative that minimises the weighted least squares cost  $\sum_{k=1}^n \lambda^{n-k} e^2(k)$ , where  $\lambda \in (0, 1]$  is a forgetting factor. RLS maintains an estimate of the inverse autocorrelation matrix and updates it recursively, achieving optimal convergence at the cost of  $O(L^2)$  computation per sample (versus  $O(L)$  for LMS).
- **Noise reduction and speech enhancement** aim to improve speech quality and intelligibility in noisy recordings. Unlike source separation (which separates distinct sources), speech enhancement specifically targets the speech-plus-noise case, recovering clean speech from a noisy observation.
- **Spectral subtraction** is the simplest approach. During noise-only frames (detected by VAD from file 03), estimate the noise spectrum |

⊗

N

$|f|^2$ . Then subtract it from each frame:

|

⊗

S

$$|f|^2 = \max(|X(f)|^2 - \alpha|$$

⊗

N

$$|f|^2, \beta|X(f)|^2)$$

- where  $\alpha$  is an over-subtraction factor (typically 1-4, aggressive subtraction removes more noise but introduces more artifacts) and  $\beta$  is a spectral floor that prevents negative values and reduces “musical noise” artifacts (isolated tonal remnants that sound like random musical notes).
- **Wiener filtering** provides the minimum mean squared error estimate of the clean speech spectrum:

$$G(t, f) = \frac{|S(t, f)|^2}{|S(t, f)|^2 + |N(t, f)|^2} \cdot X(t, f) = G(t, f) \cdot X(t, f)$$

- The Wiener gain  $G(t, f) = \text{SNR}(t, f) / (1 + \text{SNR}(t, f))$  ranges from 0 (pure noise) to 1 (pure speech), acting as a soft mask. The challenge is estimating the speech and noise power spectra. The **a priori SNR**  $\xi(t, f) = |S(t, f)|^2 / |N(t, f)|^2$  is estimated using the “decision-directed” approach: a smoothed combination of the current frame’s estimate and the previous frame’s Wiener-filtered output.
- **Neural speech enhancement** uses deep learning to estimate either a mask (like the Wiener gain) or the clean spectrogram directly. Architectures range from simple feedforward networks to U-Nets (chapter 08), CRNs (Convolutional Recurrent Networks), and transformers.
- **DCCRN** (Deep Complex Convolutional Recurrent Network) operates on the complex STFT (both magnitude and phase), using complex-valued convolutions that naturally handle the real and imaginary parts. This avoids the phase estimation problem that plagues magnitude-only approaches.
- **FullSubNet** uses a dual-path architecture with a full-band model (capturing global spectral patterns) and a sub-band model (capturing local harmonic details). The full-band model processes the entire spectrum, while the sub-band model processes narrow frequency bands centred on each frequency bin. Their outputs are combined for the final mask estimate.
- **DNS (Deep Noise Suppression) Challenge** by Microsoft benchmarks speech enhancement systems annually. Winners typically use large-scale training with diverse noise types, data augmentation (adding noise at various SNRs, reverberation, codec artifacts), and real-time-capable architectures.
- **Echo cancellation** removes acoustic echo in two-way communication. When you are on a phone call, the far-end speaker’s voice plays through your loudspeaker, bounces around the room, and is picked up by your microphone, creating an echo that the far-end speaker hears. **Acoustic Echo Cancellation (AEC)** models the acoustic path from loudspeaker to microphone and subtracts the predicted echo.
- The acoustic path is modelled as an adaptive FIR filter (using LMS or NLMS) with the far-end signal as input. The filter models the room impulse response,

which includes direct path, early reflections, and late reverberation. Room impulse responses can be hundreds of milliseconds long, requiring filters with thousands of taps.

- **Double-talk detection** is critical for AEC: when both the near-end and far-end speakers talk simultaneously, the adaptive filter must freeze (stop updating) to prevent it from cancelling the near-end speaker's voice. Double-talk detectors compare the energy of the error signal with the far-end signal energy; a sudden increase in error energy that is not explained by the far-end signal suggests near-end speech.
- The **normalised cross-correlation** between the far-end signal  $x(n)$  and the microphone signal  $d(n)$  provides a double-talk indicator:

$$\xi(n) = \frac{\sum_{k=0}^{L-1} x(n-k)d(n-k)}{\sqrt{\sum_{k=0}^{L-1} x^2(n-k) \sum_{k=0}^{L-1} d^2(n-k)}}$$

- During single-talk (far-end only),  $\xi$  is high because  $d$  is mostly echo of  $x$ . During double-talk,  $\xi$  drops because the near-end speech is uncorrelated with  $x$ .
- Modern AEC systems combine adaptive filtering with neural networks: the adaptive filter provides an initial echo estimate, and a neural network (similar to the speech enhancement models above) cleans up residual echo and handles non-linearities (loudspeaker distortion) that linear filters cannot capture.
- **Evaluation metrics for separation and enhancement:**
  - **SI-SDR** (defined above): standard for source separation.
  - **SDR** (Signal-to-Distortion Ratio): from BSS Eval, measures overall separation quality including artifacts and interference.
  - **PESQ** (Perceptual Evaluation of Speech Quality): ITU standard that predicts subjective quality scores. Range: -0.5 to 4.5.

- **STOI** (Short-Time Objective Intelligibility): predicts speech intelligibility. Range: 0 to 1.
- **DNSMOS**: Microsoft's deep noise suppression MOS predictor, a neural network trained to predict human MOS scores without requiring clean reference audio.

## 45.1 Coding Tasks (use CoLab or notebook)

- **Task 1: Independent Component Analysis for source separation.** Implement FastICA to separate two mixed audio sources, demonstrating the classical cocktail party solution for the determined case (equal sources and microphones).

```
import jax
import jax.numpy as jnp
import jax.random as jr
import matplotlib.pyplot as plt

Generate two source signals
sr = 8000
duration = 1.0
t = jnp.linspace(0, duration, int(sr * duration))

Source 1: sinusoidal (like a tone)
s1 = jnp.sin(2 * jnp.pi * 440 * t) + 0.3 * jnp.sin(2 * jnp.pi * 880 * t)

Source 2: sawtooth-like (rich harmonics)
s2 = 2 * (t * 200 % 1) - 1 # sawtooth at 200 Hz

Normalise sources
s1 = s1 / jnp.max(jnp.abs(s1))
s2 = s2 / jnp.max(jnp.abs(s2))
sources = jnp.stack([s1, s2]) # (2, T)

Mixing matrix (unknown to the algorithm)
A = jnp.array([[0.8, 0.4],
 [0.3, 0.9]])
mixtures = A @ sources # (2, T)

FastICA implementation
def whiten(X):
 """Centre and whiten the data."""
 X_centered = X - jnp.mean(X, axis=1, keepdims=True)
 cov = (X_centered @ X_centered.T) / X_centered.shape[1]
```

```

eigvals, eigvecs = jnp.linalg.eigh(cov)
D_inv_sqrt = jnp.diag(1.0 / jnp.sqrt(eigvals + 1e-8))
whitening = D_inv_sqrt @ eigvecs.T
return whitening @ X_centered, whitening

def fastica(X, n_components=2, max_iter=200, tol=1e-6):
 """FastICA using tanh non-linearity (approximation to negentropy)."""
 X_white, whitening = whiten(X)
 n, T = X_white.shape

 key = jr.PRNGKey(42)
 W = jr.normal(key, (n_components, n))
 # Orthogonalise W
 U, _, Vt = jnp.linalg.svd(W, full_matrices=False)
 W = U @ Vt

 for iteration in range(max_iter):
 W_old = W.copy()

 # For each component
 for i in range(n_components):
 w = W[i]
 # $w^T X_{white}$: (T,)
 wx = w @ X_white # (T,)

 # $g(u) = \tanh(u)$, $g'(u) = 1 - \tanh^2(u)$
 g_wx = jnp.tanh(wx)
 g_prime_wx = 1 - g_wx ** 2

 # Newton update: $w_{new} = E[X * g(w^T X)] - E[g'(w^T X)] * w$
 w_new = jnp.mean(X_white * g_wx[None, :], axis=1) - \
 jnp.mean(g_prime_wx) * w

 # Decorrelate from previous components (deflation)
 for j in range(i):
 w_new = w_new - jnp.dot(w_new, W[j]) * W[j]

 w_new = w_new / jnp.linalg.norm(w_new)
 W = W.at[i].set(w_new)

 # Check convergence
 convergence = jnp.min(jnp.abs(jnp.diag(W @ W_old.T)))
 if convergence > 1 - tol:
 print(f"FastICA converged in {iteration + 1} iterations")
 break

 # Unmixing matrix

```

```

 unmixing = W @ whitening
 recovered = unmixing @ X
 return recovered, unmixing

recovered, W_unmix = fastica(mixtures)

Fix sign ambiguity (ICA can flip signs)
for i in range(2):
 if jnp.corrcoef(recovered[i], sources[i])[0, 1] < -0.5:
 recovered = recovered.at[i].set(-recovered[i])

If sources are swapped, fix permutation
corr_00 = jnp.abs(jnp.corrcoef(recovered[0], sources[0])[0, 1])
corr_01 = jnp.abs(jnp.corrcoef(recovered[0], sources[1])[0, 1])
if corr_01 > corr_00:
 recovered = recovered[::-1]

Normalise for display
recovered = recovered / jnp.max(jnp.abs(recovered), axis=1, keepdims=True)

fig, axes = plt.subplots(3, 2, figsize=(14, 9))

axes[0, 0].plot(t[:1000], s1[:1000], color='#3498db', linewidth=0.8)
axes[0, 0].set_title('Source 1 (Original)')
axes[0, 0].set_ylabel('Amplitude')

axes[0, 1].plot(t[:1000], s2[:1000], color='#e74c3c', linewidth=0.8)
axes[0, 1].set_title('Source 2 (Original)')

axes[1, 0].plot(t[:1000], mixtures[0, :1000], color='#9b59b6', linewidth=0.8)
axes[1, 0].set_title('Mixture 1 (Microphone 1)')
axes[1, 0].set_ylabel('Amplitude')

axes[1, 1].plot(t[:1000], mixtures[1, :1000], color='#9b59b6', linewidth=0.8)
axes[1, 1].set_title('Mixture 2 (Microphone 2)')

axes[2, 0].plot(t[:1000], recovered[0, :1000], color='#27ae60', linewidth=0.8)
axes[2, 0].set_title('Recovered Source 1 (FastICA)')
axes[2, 0].set_ylabel('Amplitude')
axes[2, 0].set_xlabel('Time (s)')

axes[2, 1].plot(t[:1000], recovered[1, :1000], color='#f39c12', linewidth=0.8)
axes[2, 1].set_title('Recovered Source 2 (FastICA)')
axes[2, 1].set_xlabel('Time (s)')

plt.tight_layout()
plt.show()

```



```

Report correlation with originals
for i in range(2):
 corr = jnp.corrcoef(recovered[i], sources[i])[0, 1]
 print(f"Source {i+1} recovery correlation: {corr:.4f}")

```

- **Task 2: NMF-based source separation on spectrograms.** Use non-negative matrix factorisation (chapter 02) to separate a spectrogram into two components, demonstrating how NMF learns spectral dictionaries for each source.

```

import jax
import jax.numpy as jnp
import jax.random as jr
import matplotlib.pyplot as plt

Generate two signals with distinct spectral characteristics
sr = 8000
duration = 1.0
t = jnp.linspace(0, duration, int(sr * duration))

Source 1: low-frequency harmonic (simulating bass)
src1 = (jnp.sin(2 * jnp.pi * 100 * t) +
 0.5 * jnp.sin(2 * jnp.pi * 200 * t) +
 0.3 * jnp.sin(2 * jnp.pi * 300 * t))

Source 2: high-frequency harmonic (simulating a flute)
src2 = (jnp.sin(2 * jnp.pi * 800 * t) +
 0.4 * jnp.sin(2 * jnp.pi * 1600 * t))

Time-varying amplitudes (sources active at different times)
env1 = jnp.where(t < 0.5, 1.0, 0.3)
env2 = jnp.where(t > 0.3, 1.0, 0.2)
src1 = src1 * env1
src2 = src2 * env2

mixture = src1 + src2

Compute magnitude spectrogram (STFT)
n_fft = 512
hop = 128
window = jnp.hanning(n_fft)

def compute_stft(signal, n_fft, hop, window):
 n_frames = 1 + (len(signal) - n_fft) // hop
 frames = jnp.stack([

```

```

 signal[i * hop : i * hop + n_fft] * window
 for i in range(n_frames)
])
 return jnp.fft.rfft(frames, n=n_fft)

S_mix = compute_stft(mixture, n_fft, hop, window)
V = jnp.abs(S_mix).T # (F, T) - frequency x time
phase = jnp.angle(S_mix).T

F, T = V.shape
print(f"Spectrogram shape: {F} freq bins x {T} time frames")

NMF: $V \approx WH$ using multiplicative update rules
def nmf(V, K, n_iter=200, key=jr.PRNGKey(0)):
 """Non-negative Matrix Factorisation with Frobenius norm."""
 k1, k2 = jr.split(key)
 W = jnp.abs(jr.normal(k1, (F, K))) * 0.1 + 0.01 # (F, K)
 H = jnp.abs(jr.normal(k2, (K, T))) * 0.1 + 0.01 # (K, T)

 costs = []
 for i in range(n_iter):
 # Multiplicative update for H
 WtV = W.T @ V
 WtWH = W.T @ W @ H + 1e-8
 H = H * (WtV / WtWH)

 # Multiplicative update for W
 VHt = V @ H.T
 WHHt = W @ H @ H.T + 1e-8
 W = W * (VHt / WHHt)

 cost = jnp.sum((V - W @ H) ** 2)
 costs.append(float(cost))

 return W, H, costs

Run NMF with K=2 components
K = 2
W, H, costs = nmf(V, K, n_iter=300)

Reconstruct each source using soft masks
V_hat = W @ H
mask1 = (W[:, 0:1] @ H[0:1, :]) / (V_hat + 1e-8)
mask2 = (W[:, 1:2] @ H[1:2, :]) / (V_hat + 1e-8)

V_src1 = mask1 * V
V_src2 = mask2 * V

```

```

Visualisation
fig, axes = plt.subplots(3, 2, figsize=(14, 10))

Mixture spectrogram
axes[0, 0].imshow(jnp.log1p(V), aspect='auto', origin='lower', cmap='magma')
axes[0, 0].set_title('Mixture Spectrogram $|X|$ ')
axes[0, 0].set_ylabel('Frequency bin')

NMF convergence
axes[0, 1].plot(costs, color='#3498db', linewidth=1.5)
axes[0, 1].set_title('NMF Convergence')
axes[0, 1].set_xlabel('Iteration')
axes[0, 1].set_ylabel('Frobenius cost')
axes[0, 1].set_yscale('log')

Spectral basis vectors W
freq_hz = jnp.arange(F) * sr / n_fft
axes[1, 0].plot(freq_hz, W[:, 0], color='#27ae60', linewidth=1.5,
 label='Basis 1 (low freq)')
axes[1, 0].plot(freq_hz, W[:, 1], color='#e74c3c', linewidth=1.5,
 label='Basis 2 (high freq)')
axes[1, 0].set_title('Learned Spectral Bases W')
axes[1, 0].set_xlabel('Frequency (Hz)')
axes[1, 0].set_ylabel('Magnitude')
axes[1, 0].legend()

Temporal activations H
time_s = jnp.arange(T) * hop / sr
axes[1, 1].plot(time_s, H[0], color='#27ae60', linewidth=1.5,
 label='Activation 1')
axes[1, 1].plot(time_s, H[1], color='#e74c3c', linewidth=1.5,
 label='Activation 2')
axes[1, 1].set_title('Temporal Activations H')
axes[1, 1].set_xlabel('Time (s)')
axes[1, 1].set_ylabel('Activation')
axes[1, 1].legend()

Separated spectrograms
axes[2, 0].imshow(jnp.log1p(V_src1), aspect='auto', origin='lower', cmap='magma')
axes[2, 0].set_title('Separated Source 1 (low-frequency)')
axes[2, 0].set_ylabel('Frequency bin')
axes[2, 0].set_xlabel('Time frame')

axes[2, 1].imshow(jnp.log1p(V_src2), aspect='auto', origin='lower', cmap='magma')
axes[2, 1].set_title('Separated Source 2 (high-frequency)')
axes[2, 1].set_xlabel('Time frame')

```

```
plt.tight_layout()
plt.show()

print(f"Reconstruction error: {jnp.sum((V - W @ H)**2):.2f}")
print(f"NMF learns spectral bases that capture each source's frequency profile.")
```

- **Task 3: LMS adaptive filter for noise cancellation.** Implement the LMS and NLMS algorithms for echo/noise cancellation, showing convergence behaviour and the effect of step size.

```
import jax
import jax.numpy as jnp
import jax.random as jr
import matplotlib.pyplot as plt

Simulate an echo cancellation scenario
Far-end signal -> room impulse response -> echo at microphone
Near-end speech is the desired signal we want to preserve

sr = 8000
duration = 2.0
n_samples = int(sr * duration)
key = jr.PRNGKey(42)
keys = jr.split(key, 5)

Far-end signal (reference): random speech-like signal
far_end = jr.normal(keys[0], (n_samples,)) * 0.5

Room impulse response (unknown to the algorithm)
rir_length = 64
rir = jnp.zeros(rir_length)
rir = rir.at[0].set(0.8) # direct path
rir = rir.at[5].set(0.3) # early reflection
rir = rir.at[12].set(-0.2) # reflection
rir = rir.at[25].set(0.1) # late reflection
rir = rir.at[40].set(-0.05)

Echo: convolution of far-end with RIR
echo = jnp.convolve(far_end, rir)[:n_samples]

Near-end speech (active in a portion of the signal)
near_end = jnp.zeros(n_samples)
start, end = n_samples // 3, 2 * n_samples // 3
near_speech = 0.3 * jnp.sin(
 2 * jnp.pi * 300 * jnp.linspace(0, (end - start) / sr, end - start)
```

```

)
near_end = near_end.at[start:end].set(near_speech)

Microphone signal: echo + near-end + noise
noise = jr.normal(keys[1], (n_samples,)) * 0.01
mic_signal = echo + near_end + noise

LMS adaptive filter
def lms_filter(reference, desired, filter_length, mu):
 """Standard LMS adaptive filter."""
 n = len(reference)
 w = jnp.zeros(filter_length)
 output = jnp.zeros(n)
 error = jnp.zeros(n)
 w_history = []

 for i in range(filter_length, n):
 x = reference[i:i-filter_length:-1] # reversed segment
 if len(x) < filter_length:
 x = jnp.pad(x, (0, filter_length - len(x)))
 x = reference[max(0, i-filter_length+1):i+1][::-1]

 y = jnp.dot(w, x)
 e = desired[i] - y
 w = w + mu * e * x

 output = output.at[i].set(y)
 error = error.at[i].set(e)

 if i % 500 == 0:
 w_history.append(w.copy())

 return output, error, w_history

NLMS adaptive filter
def nlms_filter(reference, desired, filter_length, mu, eps=1e-6):
 """Normalised LMS adaptive filter."""
 n = len(reference)
 w = jnp.zeros(filter_length)
 output = jnp.zeros(n)
 error = jnp.zeros(n)

 for i in range(filter_length, n):
 x = reference[max(0, i-filter_length+1):i+1][::-1]

 y = jnp.dot(w, x)
 e = desired[i] - y

```

```

 norm_factor = jnp.dot(x, x) + eps
 w = w + (mu / norm_factor) * e * x

 output = output.at[i].set(y)
 error = error.at[i].set(e)

 return output, error

Run LMS with different step sizes
filter_len = 64
mu_values = [0.001, 0.01, 0.05]
colors_mu = ['#3498db', '#e74c3c', '#27ae60']

fig, axes = plt.subplots(2, 2, figsize=(14, 10))

Original signals
t = jnp.arange(n_samples) / sr
axes[0, 0].plot(t, mic_signal, color='#9b59b6', linewidth=0.5, alpha=0.7,
 label='Mic (echo + near-end)')
axes[0, 0].plot(t, echo, color='#e74c3c', linewidth=0.5, alpha=0.7,
 label='Echo (to cancel)')
axes[0, 0].plot(t, near_end, color='#27ae60', linewidth=0.8,
 label='Near-end speech (to preserve)')
axes[0, 0].set_title('Signal Components')
axes[0, 0].set_xlabel('Time (s)')
axes[0, 0].set_ylabel('Amplitude')
axes[0, 0].legend(fontsize=8)

LMS convergence for different step sizes
for mu, color in zip(mu_values, colors_mu):
 _, err, _ = lms_filter(far_end, mic_signal, filter_len, mu)
 # Smoothed squared error
 sq_err = err ** 2
 window_size = 200
 smoothed = jnp.convolve(sq_err, jnp.ones(window_size)/window_size,
 mode='valid')
 axes[0, 1].plot(smoothed, color=color, linewidth=1.2,
 label=f'mu={mu}')

axes[0, 1].set_title('LMS Convergence (smoothed MSE)')
axes[0, 1].set_xlabel('Sample')
axes[0, 1].set_ylabel('Squared Error')
axes[0, 1].set_yscale('log')
axes[0, 1].legend()

Best LMS result
_, err_lms, w_hist = lms_filter(far_end, mic_signal, filter_len, 0.01)

```

```

axes[1, 0].plot(t, mic_signal, color='#9b59b6', linewidth=0.5, alpha=0.4,
 label='Before cancellation')
axes[1, 0].plot(t, err_lms, color='#3498db', linewidth=0.5, alpha=0.8,
 label='After LMS cancellation')
axes[1, 0].plot(t, near_end, color='#27ae60', linewidth=0.8, alpha=0.5,
 label='True near-end')
axes[1, 0].set_title('LMS Echo Cancellation Result (mu=0.01)')
axes[1, 0].set_xlabel('Time (s)')
axes[1, 0].set_ylabel('Amplitude')
axes[1, 0].legend(fontsize=8)

NLMS result
_, err_nlms = nlms_filter(far_end, mic_signal, filter_len, 0.5)
axes[1, 1].plot(t, mic_signal, color='#9b59b6', linewidth=0.5, alpha=0.4,
 label='Before cancellation')
axes[1, 1].plot(t, err_nlms, color='#f39c12', linewidth=0.5, alpha=0.8,
 label='After NLMS cancellation')
axes[1, 1].plot(t, near_end, color='#27ae60', linewidth=0.8, alpha=0.5,
 label='True near-end')
axes[1, 1].set_title('NLMS Echo Cancellation Result (mu=0.5)')
axes[1, 1].set_xlabel('Time (s)')
axes[1, 1].set_ylabel('Amplitude')
axes[1, 1].legend(fontsize=8)

plt.tight_layout()
plt.show()

Measure echo reduction
echo_power = jnp.mean(echo ** 2)
lms_residual = jnp.mean(err_lms[n_samples//2:] ** 2) # after convergence
nlms_residual = jnp.mean(err_nlms[n_samples//2:] ** 2)
print(f"Echo power: {10*jnp.log10(echo_power):.1f} dB")
print(f"LMS residual: {10*jnp.log10(lms_residual):.1f} dB "
 f"(ERLE: {10*jnp.log10(echo_power/lms_residual):.1f} dB)")
print(f"NLMS residual: {10*jnp.log10(nlms_residual):.1f} dB "
 f"(ERLE: {10*jnp.log10(echo_power/nlms_residual):.1f} dB)")

```

- **Task 4: Time-frequency masking for speech enhancement.** Implement a simple spectral masking approach (ideal ratio mask) and compare it with spectral subtraction, visualising the separation quality on a synthetic noisy speech signal.

```

import jax
import jax.numpy as jnp
import jax.random as jr
import matplotlib.pyplot as plt

```

```

Create synthetic "speech" and "noise" signals
sr = 8000
duration = 2.0
t = jnp.linspace(0, duration, int(sr * duration))

Speech: harmonic series with time-varying amplitude (simulating speech)
speech = jnp.zeros_like(t)
for f0 in [150, 300, 450, 600, 900]:
 amp_env = 0.5 + 0.5 * jnp.sin(2 * jnp.pi * 2.0 * t) # 2 Hz modulation
 speech = speech + (0.5 / (f0/150)) * amp_env * jnp.sin(2 * jnp.pi * f0 * t)
speech = speech / jnp.max(jnp.abs(speech))

Noise: band-limited noise
key = jr.PRNGKey(42)
noise_raw = jr.normal(key, t.shape) * 0.4

Mix at a given SNR
snr_db = 5.0
speech_power = jnp.mean(speech ** 2)
noise_power = jnp.mean(noise_raw ** 2)
noise_scale = jnp.sqrt(speech_power / (noise_power * 10 ** (snr_db / 10)))
noise = noise_raw * noise_scale
mixture = speech + noise

STFT
n_fft = 512
hop = 128
window = jnp.hanning(n_fft)

def stft(signal, n_fft, hop, window):
 n_frames = 1 + (len(signal) - n_fft) // hop
 frames = jnp.stack([
 signal[i * hop : i * hop + n_fft] * window
 for i in range(n_frames)
])
 return jnp.fft.rfft(frames, n=n_fft)

def istft(S, hop, window, length):
 n_fft = (S.shape[1] - 1) * 2
 n_frames = S.shape[0]
 frames = jnp.fft.irfft(S, n=n_fft) * window[None, :]
 output = jnp.zeros(length)
 window_sum = jnp.zeros(length)
 for i in range(n_frames):
 start = i * hop
 end = start + n_fft

```



[illegible]

```

si_sdr_mix = si_sdr(mixture, speech)
si_sdr_irm_val = si_sdr(enhanced_irm, speech)
si_sdr_sub_val = si_sdr(enhanced_sub, speech)
si_sdr_wiener_val = si_sdr(enhanced_wiener, speech)

Visualisation
fig, axes = plt.subplots(3, 2, figsize=(14, 12))

Spectrograms
axes[0, 0].imshow(jnp.log1p(mag_speech.T), aspect='auto', origin='lower',
 cmap='magma')
axes[0, 0].set_title('Clean Speech Spectrogram')
axes[0, 0].set_ylabel('Frequency bin')

axes[0, 1].imshow(jnp.log1p(mag_mix.T), aspect='auto', origin='lower',
 cmap='magma')
axes[0, 1].set_title(f'Noisy Mixture ({snr_db:.0f} dB SNR)')

Masks
axes[1, 0].imshow(irm.T, aspect='auto', origin='lower', cmap='RdYlGn')
axes[1, 0].set_title('Ideal Ratio Mask (Oracle)')
axes[1, 0].set_ylabel('Frequency bin')

axes[1, 1].imshow(wiener_gain.T, aspect='auto', origin='lower', cmap='RdYlGn',
 vmin=0, vmax=1)
axes[1, 1].set_title('Estimated Wiener Gain')

Enhanced waveforms comparison
n_show = 3000
axes[2, 0].plot(t[:n_show], speech[:n_show], color='#27ae60', linewidth=0.8,
 alpha=0.5, label='Clean')
axes[2, 0].plot(t[:n_show], mixture[:n_show], color='#e74c3c', linewidth=0.5,
 alpha=0.4, label='Noisy')
axes[2, 0].plot(t[:n_show], enhanced_irm[:n_show], color='#3498db',
 linewidth=0.8, label='IRM enhanced')
axes[2, 0].set_title('Waveform Comparison (IRM)')
axes[2, 0].set_xlabel('Time (s)')
axes[2, 0].set_ylabel('Amplitude')
axes[2, 0].legend(fontsize=8)

SI-SDR bar chart
methods = ['Mixture', 'Spectral\nSubtraction', 'Wiener\nFilter', 'Ideal
Ratio\nMask']
sdr_values = [float(si_sdr_mix), float(si_sdr_sub_val),
 float(si_sdr_wiener_val), float(si_sdr_irm_val)]
bar_colors = ['#e74c3c', '#f39c12', '#9b59b6', '#27ae60']
bars = axes[2, 1].bar(methods, sdr_values, color=bar_colors, alpha=0.8)

```

```

axes[2, 1].set_ylabel('SI-SDR (dB)')
axes[2, 1].set_title('Enhancement Quality Comparison')
for bar, val in zip(bars, sdr_values):
 axes[2, 1].text(bar.get_x() + bar.get_width()/2., bar.get_height() + 0.3,
 f'{val:.1f}', ha='center', fontsize=10)
axes[2, 1].axhline(0, color='gray', linestyle='--', linewidth=0.8)

plt.tight_layout()
plt.show()

print(f"SI-SDR (noisy mixture): {si_sdr_mix:.2f} dB")
print(f"SI-SDR (spectral subtraction): {si_sdr_sub_val:.2f} dB")
print(f"SI-SDR (Wiener filter): {si_sdr_wiener_val:.2f} dB")
print(f"SI-SDR (ideal ratio mask): {si_sdr_irm_val:.2f} dB (oracle upper
bound)")

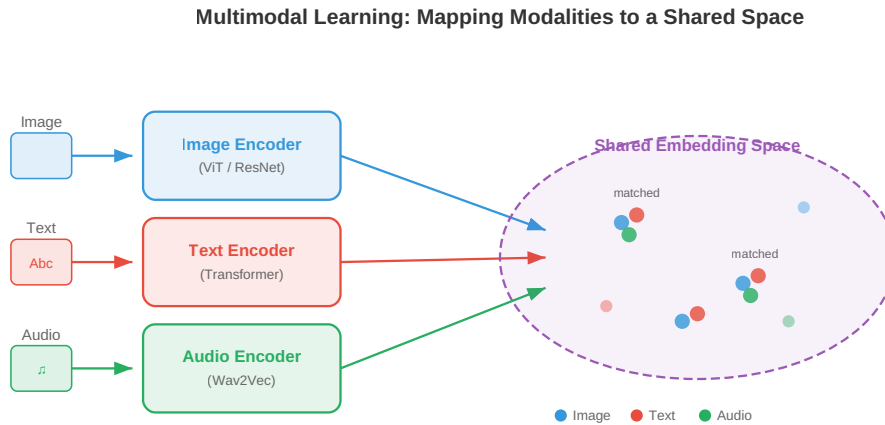
```

## 46 Multimodal Representations

Multimodal representations bridge vision, language, and audio into shared embedding spaces. This file covers fusion strategies, CLIP, ALIGN, SigLIP, contrastive loss functions (InfoNCE, NT-Xent), zero-shot classification, and retrieval evaluation.

- Imagine you are sitting in a cafe. You see a steaming cup on the table, hear the clinking of ceramic, smell roasted coffee beans, and feel warmth radiating from the mug. No single sense tells you everything: your brain fuses these signals into a unified percept of “hot coffee.” **Multimodal learning** does the same thing for machines: it combines information from multiple modalities (vision, language, audio, and others) to build richer, more robust representations than any single modality provides alone.
- A **modality** is a distinct channel of information. In machine learning, the most common modalities are images (pixel grids), text (token sequences), audio (waveforms or spectrograms, as in Chapter 9), video (sequences of frames), and structured data (tables, graphs). Each modality has its own statistical structure: images are spatially coherent, text is sequential and discrete, audio is temporal and continuous. The challenge of multimodal learning is bridging these fundamentally different data types.
- Why bother combining modalities? Because they provide complementary information. A photograph of a dog tells you about its breed and colour but not its name. A caption like “my golden retriever Max” tells you the name and

breed but not the exact pose. Together, the image and text give a fuller picture than either alone. This complementarity is the core motivation: multimodal models can answer questions, generate content, and make decisions that no unimodal model can.



Overview of multimodal learning: separate encoders process image, text, and audio inputs, their representations meet in a shared embedding space

## 46.1 Fusion Strategies

- Think of a group project. You can combine ideas in two ways: everyone works together in the same room from the start (sharing raw notes and drafts), or each person writes their section independently and you merge the final documents. These correspond to **early fusion** and **late fusion** in multimodal learning.
- **Early fusion** (also called feature-level fusion) concatenates or mixes raw or low-level features from different modalities before any serious processing happens. For example, you might concatenate an image's pixel features with a text's token embeddings and feed the combined sequence into a single transformer. The model can learn fine-grained cross-modal interactions from the start, but the input space is large and the model must learn to handle very different data types simultaneously.
- Formally, given feature vectors  $x_{\text{img}} \in \mathbb{R}^{d_1}$  and  $x_{\text{txt}} \in \mathbb{R}^{d_2}$  from two modalities, early fusion simply concatenates them:

$$x_{\text{fused}} = [x_{\text{img}}; x_{\text{txt}}] \in \mathbb{R}^{d_1 + d_2}$$

- This concatenated vector is then processed by a shared network. The advantage is that the model can discover cross-modal correlations at every layer. The

disadvantage is computational cost and the difficulty of aligning very different feature types (dense pixel values vs. sparse token indices).

- **Late fusion** (also called decision-level fusion) processes each modality independently through its own encoder, producing a high-level representation or even a final prediction for each. These outputs are then combined, typically by averaging scores, voting, or a learned combination layer. Late fusion is simpler and lets you reuse pre-trained unimodal models off the shelf, but it cannot capture low-level cross-modal interactions because the modalities never “see” each other’s raw features.

- Given modality-specific predictions

$\hat{y}_1$

$y_1$

and

$\hat{y}_2$

$y_2$

, a simple late fusion rule is:

$\hat{y}$

$y$

$$= \alpha$$

$\hat{y}$

$y$

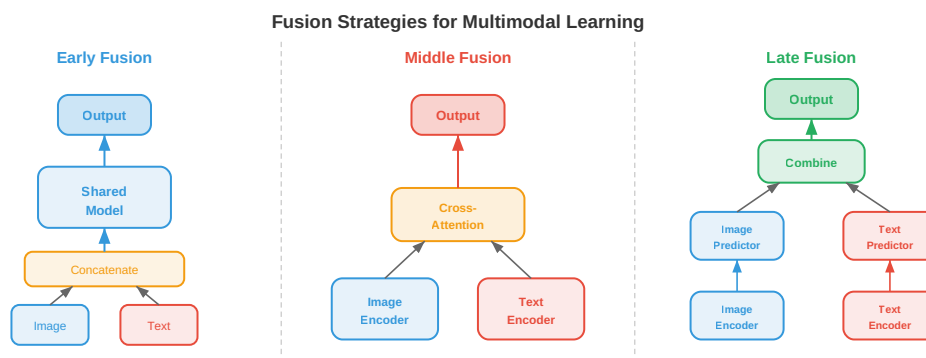
$$= \alpha \hat{y}_1 + (1 - \alpha) \hat{y}_2$$

$\hat{y}$

$y$

$$2$$

- where  $\alpha \in [0, 1]$  is a learned or hand-tuned mixing weight.
- **Middle fusion** (also called intermediate fusion) is the pragmatic middle ground used by most modern systems. Each modality is first processed by its own encoder (extracting modality-specific features), and then the encoded representations are combined partway through the network, often via cross-attention layers. This lets each encoder specialise in its modality while still enabling rich cross-modal interactions. Flamingo, LLaVA, and most vision-language models (file 02) use middle fusion.



Early, middle, and late fusion strategies: early fusion concatenates raw inputs, middle fusion merges intermediate representations via cross-attention, late fusion combines final predictions

- The choice between fusion strategies depends on data availability, computational budget, and the task. Early fusion is powerful but data-hungry. Late fusion is cheap but limited. Middle fusion with cross-attention has become the dominant approach in large-scale multimodal models because it balances expressiveness with modularity.

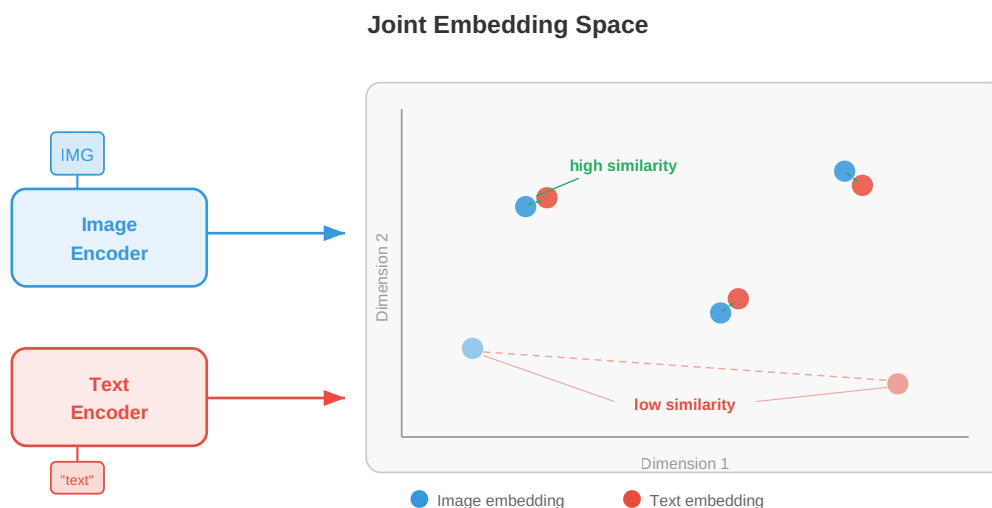
## 46.2 Joint Embedding Spaces

- Imagine a universal translator that can take any sentence in any language and map it to the same point in a shared “meaning space.” The sentence “a dog on a beach” in English, French, or Japanese would all land at the same coordinate. **Joint embedding spaces** do exactly this, but across modalities: an image of a dog on a beach and the text “a dog on a beach” should map to nearby points in the same vector space.
- Formally, we learn two encoder functions:  $f_\theta: \mathcal{X}_1 \rightarrow \mathbb{R}^d$  for modality 1 (e.g., images) and  $g_\phi: \mathcal{X}_2 \rightarrow \mathbb{R}^d$  for modality 2 (e.g., text). Both map their inputs into the same  $d$ -dimensional space. The training objective ensures that semantically matched pairs  $(x_1, x_2)$  have embeddings  $f_\theta(x_1)$  and  $g_\phi(x_2)$  that are close (high cosine similarity), while unmatched pairs are far apart.
- This is a direct generalisation of the word embedding spaces from Chapter 7. Recall that Word2Vec and GloVe placed semantically similar words near each other in a vector space. Joint embedding spaces extend this idea across modalities: instead of word-to-word similarity, we measure image-to-text similarity, audio-to-text similarity, or even image-to-audio similarity.

- The similarity metric is almost always **cosine similarity** (Chapter 1):

$$\text{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}$$

- By  $L_2$ -normalising all embeddings onto the unit hypersphere, cosine similarity reduces to a simple dot product  $\mathbf{u} \cdot \mathbf{v}$ , which is extremely efficient to compute and can be accelerated with approximate nearest-neighbour libraries.



Joint embedding space: an image encoder and a text encoder map their respective inputs into a shared vector space where matched pairs cluster together

- The power of a joint embedding space is that it enables **zero-shot transfer**. Once you have aligned image and text embeddings, you can classify images into categories you have never trained on: just embed the category names as text and find which text embedding is closest to the image embedding. No task-specific fine-tuning is needed. This is the key insight behind CLIP and its successors.

## 46.3 Contrastive Learning for Multimodal Alignment

- Think of a classroom exercise where students are given shuffled pairs of photos and captions, and asked to match each photo with its correct caption. To do this well, you need to understand both the visual content and the language, and know how they relate. **Contrastive learning** trains models in exactly this way: given a batch of (image, text) pairs, the model must figure out which image goes with which text.

- As we saw in Chapter 8 (file 04), contrastive learning in the unimodal setting (SimCLR, MoCo) pulls together augmented views of the same image and pushes apart views of different images. Multimodal contrastive learning replaces “augmented views” with “matched modalities” : an image and its caption are the positive pair; the image paired with any other caption in the batch is a negative pair.

### 46.3.1 CLIP

- **CLIP** (Contrastive Language-Image Pre-training, Radford et al., 2021) is the foundational model for multimodal contrastive learning. It trains an image encoder (a ViT or ResNet, Chapter 8) and a text encoder (a transformer, Chapter 7) jointly on 400 million (image, text) pairs scraped from the internet.
- Given a batch of  $N$  image-text pairs, CLIP computes the  $N \times N$  matrix of cosine similarities between all image embeddings and all text embeddings. The diagonal entries are the matched pairs (positives); all off-diagonal entries are unmatched (negatives). The training loss pushes diagonal entries high and off-diagonal entries low.
- The loss is a symmetric cross-entropy. For image  $i$  paired with text  $j = i$ , the image-to-text loss is:

$$\mathcal{L}_{i \rightarrow t} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(\text{sim}(z_i^{\text{img}}, z_i^{\text{txt}})/\tau)}{\sum_{k=1}^N \exp(\text{sim}(z_i^{\text{img}}, z_k^{\text{txt}})/\tau)}$$

- and the text-to-image loss is the same with the roles swapped:



$$\sum_{i=1}^N$$

$$\mathcal{L}_{t \rightarrow i} = -\frac{1}{N}$$

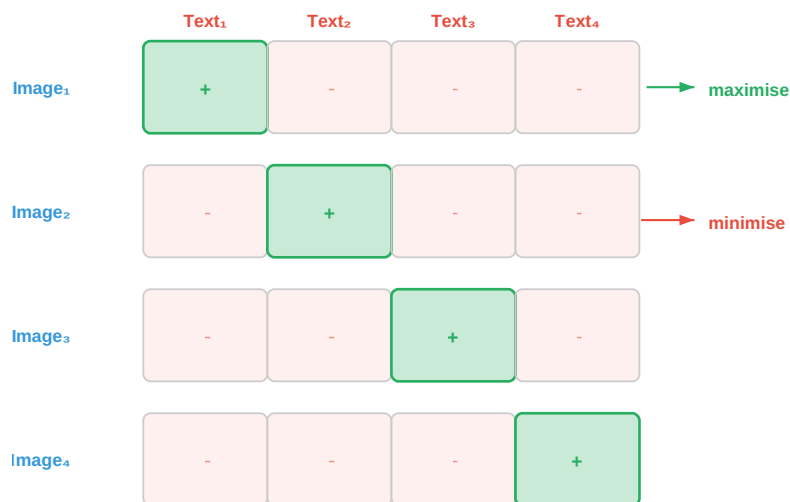
$$\log \frac{\exp(\text{sim}(z_i^{\text{txt}}, z_i^{\text{img}})/\tau)}{\sum_{k=1}^N \exp(\text{sim}(z_i^{\text{txt}}, z_k^{\text{img}})/\tau)}$$

- The total CLIP loss is the average:

$$\mathcal{L}_{\text{CLIP}} = \frac{1}{2}(\mathcal{L}_{i \rightarrow t} + \mathcal{L}_{t \rightarrow i})$$

- Here  $\tau$  is a learned **temperature** parameter (initialised at  $\tau = 0.07$ ). Temperature controls the sharpness of the softmax distribution: low  $\tau$  makes the model focus harder on the closest match, high  $\tau$  spreads probability more evenly. CLIP learns  $\tau$  jointly with the model weights rather than treating it as a fixed hyperparameter.

CLIP Contrastive Learning: N×N Similarity Matrix



Loss =  $\frac{1}{2}$  (CE over rows + CE over columns) — symmetric cross-entropy

CLIP training: a batch of N image-text pairs produces an N×N similarity matrix, training maximises diagonal entries and minimises off-diagonal entries

- CLIP' s image encoder is typically a ViT-L/14 (a large Vision Transformer with 14x14 patches, Chapter 8 file 04). The text encoder is a 12-layer transformer with

causal masking (like GPT, Chapter 7 file 04). Both encoders project their outputs to a shared 512- or 768-dimensional space via a learned linear projection, followed by  $L_2$  normalisation.

- CLIP’ s most remarkable property is **zero-shot image classification**. To classify an image into one of K categories, you create K text prompts like “a photo of a {class name}” , embed each prompt with the text encoder, embed the image with the image encoder, and pick the class whose text embedding has the highest cosine similarity with the image embedding. On ImageNet, CLIP achieves competitive accuracy without ever seeing a single ImageNet training example.

### 46.3.2 ALIGN

- **ALIGN** (Jia et al., 2021) scales CLIP’ s approach to a noisier, larger dataset: 1.8 billion image-text pairs with minimal filtering. Where CLIP carefully curated its data, ALIGN shows that scale can compensate for noise. ALIGN uses an EfficientNet image encoder and a BERT text encoder, and trains with the same contrastive loss. The key finding is that with enough data, you do not need expensive data cleaning: the contrastive objective naturally downweights noisy pairs because they produce inconsistent gradients.

### 46.3.3 SigLIP

- **SigLIP** (Sigmoid Loss for Language-Image Pre-training, Zhai et al., 2023) replaces CLIP’ s softmax-based contrastive loss with a simpler sigmoid loss. Instead of treating the  $N \times N$  similarity matrix as a classification problem (each row is a softmax over columns), SigLIP treats each entry independently as a binary classification: is this (image, text) pair matched or not?

- The SigLIP loss for a single pair (i, j) is:

$$\mathcal{L}_{ij} = -y_{ij} \log \sigma(z_i^{\text{img}} \cdot z_j^{\text{txt}}/\tau) - (1 - y_{ij}) \log (1 - \sigma(z_i^{\text{img}} \cdot z_j^{\text{txt}}/\tau))$$

- where  $y_{ij} = 1$  if  $i = j$  (matched) and  $y_{ij} = 0$  otherwise, and  $\sigma$  is the sigmoid function.
- The crucial advantage of SigLIP is that it eliminates the need for a global softmax normalisation across the entire batch. In CLIP, the softmax denominator requires gathering all embeddings across all devices, which is a communication bottleneck in distributed training. SigLIP’ s per-pair sigmoid

loss can be computed locally, enabling more efficient scaling to very large batches. SigLIP matches CLIP' s quality with lower training cost.

## 46.4 Contrastive Loss Functions in Detail

---

- The loss functions used in contrastive learning share a common structure: they all try to make the similarity score of positive pairs higher than that of negative pairs, with some notion of “margin” or “temperature” controlling how hard the model pushes. Let us formalise the key variants.

### 46.4.1 InfoNCE

- **InfoNCE** (Noise-Contrastive Estimation, van den Oord et al., 2018) is the theoretical foundation behind CLIP' s loss. Given a query  $q$ , one positive key  $k^+$ , and  $K$  negative keys  $\{k_1^-, \dots, k_K^-\}$ , the loss is:

$$\mathcal{L}_{\text{InfoNCE}} = -\log \frac{\exp(q \cdot k^+/\tau)}{\exp(q \cdot k^+/\tau) + \sum_{j=1}^K \exp(q \cdot k_j^-/\tau)}$$

- This is a  $(K + 1)$ -way classification problem: identify the positive among  $K + 1$  candidates. InfoNCE is a lower bound on the mutual information between the query and the positive key, which is why maximising it aligns representations of semantically matched inputs. The bound tightens as the number of negatives  $K$  increases, which explains why contrastive methods benefit from large batch sizes.

### 46.4.2 NT-Xent

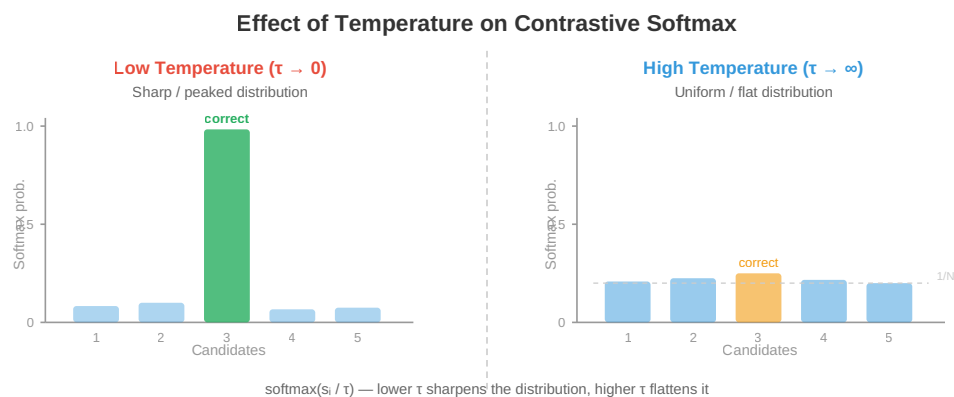
- **NT-Xent** (Normalised Temperature-scaled Cross-Entropy, Chen et al., 2020) is the loss used in SimCLR (Chapter 8 file 04) and is essentially InfoNCE applied symmetrically within a batch. For a batch of  $N$  pairs, the  $2N$  augmented views produce  $2N - 2$  negatives for each anchor (all views except itself and its positive). The loss for a positive pair  $(i, j)$  is:

$$\ell_{i,j} = -\log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k=1}^{2N} \mathbf{1}_{[k \neq i]} \exp(\text{sim}(z_i, z_k)/\tau)}$$

- NT-Xent and InfoNCE are the same mathematical formula; the names differ because they were introduced in different contexts (self-supervised vision vs. representation learning theory).

### 46.4.3 The Role of Temperature

- The **temperature**  $\tau$  is one of the most important hyperparameters in contrastive learning. To build intuition, think of temperature in the physical sense: at high temperature, molecules move randomly (the softmax is flat, all negatives look equally bad); at low temperature, molecules settle into rigid structures (the softmax is peaked, only the hardest negatives matter).
- Formally, as  $\tau \rightarrow 0$ , the softmax approaches a hard argmax that selects only the single hardest negative. As  $\tau \rightarrow \infty$ , all negatives contribute equally. In practice,  $\tau \in [0.01, 0.1]$  works well for normalised embeddings. Too-low temperature causes training instability (gradients become very large for hard negatives); too-high temperature makes the loss insensitive to violations.
- CLIP initialises  $\tau = 0.07$  and learns it as a log-parametrised scalar  $\tau = \exp(t)$ , where  $t$  is updated by gradient descent alongside the model weights. This allows the model to automatically adjust the difficulty of the contrastive task during training.



Effect of temperature on contrastive softmax: low temperature produces a peaked distribution focused on hard negatives, high temperature produces a flat distribution

#### 46.4.4 Triplet Loss and Margin-Based Alternatives

- Before InfoNCE dominated, **triplet loss** was the standard for metric learning. Given an anchor  $a$ , a positive  $p$ , and a negative  $n$ :

$$\mathcal{L}_{\text{triplet}} = \max(0, \|a - p\|^2 - \|a - n\|^2 + m)$$

- where  $m$  is a margin that ensures the positive is at least  $m$  closer than the negative. Triplet loss operates on individual triplets rather than batches, making it less sample-efficient than InfoNCE. It is also sensitive to the mining strategy: random negatives are often too easy (the loss is zero), so **hard negative mining** (selecting the closest incorrect match) or **semi-hard mining** (selecting negatives within the margin) is critical.
- InfoNCE implicitly performs hard negative mining across the entire batch, which is one reason it outperforms triplet loss at scale. The softmax normalisation in InfoNCE automatically upweights hard negatives (those with high similarity to the anchor), providing a natural curriculum without explicit mining.

### 46.5 Image-Text Retrieval and Zero-Shot Classification

---

- Once you have a trained joint embedding space, you can perform **image-text retrieval**: given an image query, find the most relevant texts from a database (image-to-text retrieval), or given a text query, find the most relevant images (text-to-image retrieval). This is simply a nearest-neighbour search in the shared embedding space.
- Imagine a librarian who can instantly compare any photograph with any caption in a million-item catalogue. They do not need to understand every possible category in advance; they just measure how “close” each photo is to each caption. This is how CLIP-style models perform retrieval and zero-shot classification.
- **Zero-shot classification** is a special case of text-to-image retrieval. Given  $K$  class names, you construct text prompts  $\{t_1, \dots, t_K\}$  (e.g., “a photo of a cat” , “a photo of a dog” ) and embed them. For a new image  $x$ , the predicted class is:

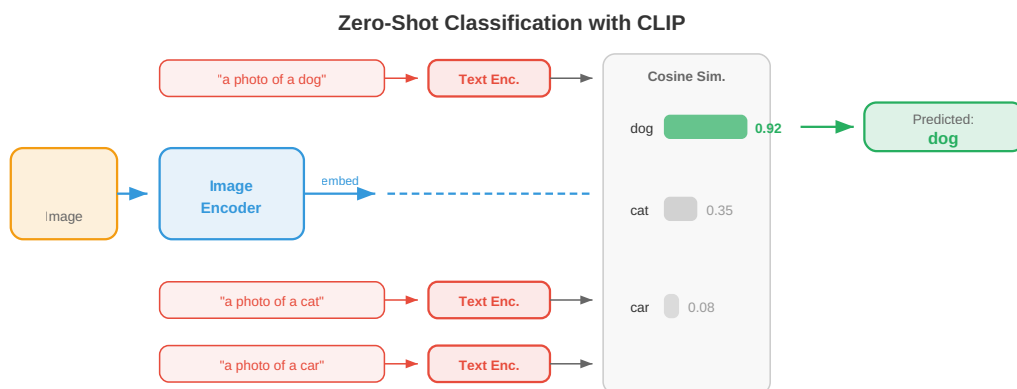
$y$

$= \arg$

$\max_k$

$\text{sim}(f_\theta(x), g_\phi(y))$

- The key insight is that the text encoder acts as a flexible classifier head. Instead of training a new linear layer for each downstream task, you simply describe the task in natural language. This is why CLIP generalises so well: the text encoder has seen millions of diverse descriptions during pre-training.
- **Prompt engineering** matters. CLIP’s zero-shot accuracy on ImageNet improves from 63.2% to 68.4% just by changing the prompt template from “{class name}” to “a photo of a {class name}.” Even better, **prompt ensembling** averages the text embeddings of multiple templates (e.g., “a photo of a {class name}” , “a good photo of a {class name}” , “a drawing of a {class name}” ) to produce a more robust text representation.



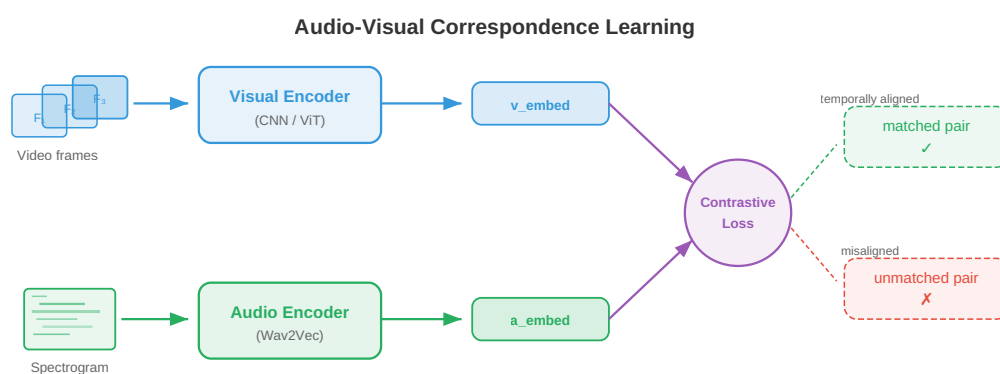
Zero-shot classification: text prompts for each class are embedded alongside the image, the class with highest cosine similarity is selected

## 46.6 Audio-Visual Correspondence

- Close your eyes and listen to someone bouncing a basketball. You can tell when it hits the floor from the rhythmic thuds. Now open your eyes: the visual bounce aligns perfectly with each thud. This tight correspondence between audio and visual events is a free supervisory signal that machines can learn from. **Audio-visual correspondence learning** trains models to associate sounds with their visual sources without any human labels.

- The idea is strikingly similar to CLIP, but replaces text with audio. Given paired video frames and audio segments, the model learns an embedding space where temporally aligned audio-visual pairs are close and misaligned pairs are far apart.
- **Audio-Visual Embedding (AVE)** methods (Arandjelovic and Zisserman, 2017) train a visual encoder  $f$  and an audio encoder  $g$  with a contrastive loss on video data. The positive pair is (video frame, audio clip from the same time), and negatives are audio clips from different videos or different times. The model learns that a barking sound goes with the image of a dog, and a guitar sound goes with the image of a guitar, all without labels.
- The audio encoder typically processes **log-mel spectrograms** (Chapter 9 file 01) using a CNN or audio transformer, producing a fixed-size embedding. The visual encoder processes video frames using a standard image backbone (ResNet, ViT). Both project to a shared  $d$ -dimensional space, and training uses the same InfoNCE loss as CLIP:

$$\mathcal{L}_{AV} = - \log \frac{\exp(\text{sim}(z^{\text{vis}}, z^{\text{aud}})/\tau)}{\sum_{k=1} \exp(\text{sim}(z^{\text{vis}}, z_k^{\text{aud}})/\tau)}$$



Audio-visual correspondence: a visual encoder processes video frames and an audio encoder processes spectrograms, contrastive learning aligns temporally matched pairs

- **Applications** of audio-visual learning include: sound source localisation (where in the image is the sound coming from?), audio-visual speech recognition (combining lip movements with audio, as in Chapter 9 file 02), audio-visual source separation (isolating one speaker's voice by watching their face, the

“cocktail party” problem from Chapter 9 file 05), and video generation conditioned on audio.

- **ImageBind** (Girdhar et al., 2023) extends this to six modalities: images, text, audio, depth, thermal, and IMU data. The key insight is that you do not need paired data for every combination. By aligning each modality to images (using image-text pairs for text, image-audio pairs for audio, etc.), all modalities become implicitly aligned through the shared image embedding space. This “binding” through a common anchor modality produces an emergent alignment: audio and text become similar even though they were never directly trained together.

## 46.7 Evaluation

---

- Evaluating multimodal models requires metrics that capture cross-modal understanding. The two dominant evaluation paradigms are **zero-shot benchmarks** and **retrieval metrics**.

### 46.7.1 Zero-Shot Benchmarks

- Zero-shot evaluation measures whether a model can perform tasks it was never explicitly trained for. The most common benchmark is **ImageNet zero-shot accuracy**: embed all 1,000 ImageNet class names as text, embed each test image, and measure top-1 and top-5 classification accuracy based on cosine similarity. CLIP ViT-L/14 achieves 75.5% top-1 accuracy zero-shot, comparable to a supervised ResNet-50 trained on ImageNet.
- Other zero-shot benchmarks include: CIFAR-10/100, STL-10, Food-101, Oxford Pets, and Flowers-102. Evaluating across many datasets tests whether the model has genuinely general visual understanding or has merely memorised patterns from its pre-training data.
- **Linear probe** evaluation is a complementary test. You freeze the pre-trained image encoder, extract features for a labelled dataset, and train a simple linear classifier on top. This measures the quality of the learned representations independently of the zero-shot retrieval mechanism. CLIP’s features are excellent linear probe features, often matching or exceeding supervised pre-training.



## 46.7.2 Retrieval Metrics

- For retrieval tasks (image-to-text and text-to-image), the standard metric is **Recall@K** ( $R@K$ ): the fraction of queries for which the correct match appears in the top  $K$  retrieved results. Common values are  $R@1$ ,  $R@5$ , and  $R@10$ .

- Formally, for a set of  $Q$  queries:

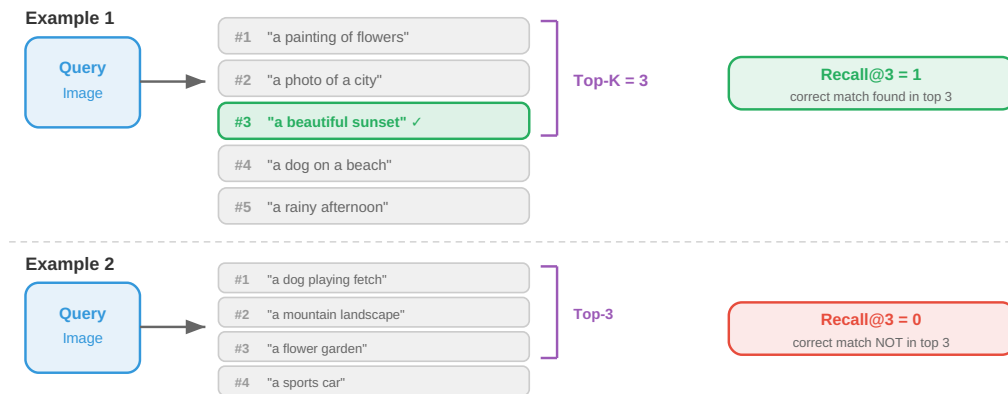
$$R@K = \frac{1}{Q}$$

$\sum_{q=1}^Q$

$$\mathbf{1}[\text{rank}(q) \leq K]$$

- where  $\text{rank}(q)$  is the position of the correct match in the ranked retrieval list for query  $q$ .
- Standard retrieval benchmarks include **Flickr30K** (31,000 images, each with 5 captions) and **MS-COCO** (123,000 images, each with 5 captions). Evaluation is done on the test split: given an image, retrieve the correct caption(s) from the full test set, and vice versa.
- **Median rank** (MedR) is a complementary metric: the median position of the correct match across all queries. A perfect model has  $\text{MedR} = 1$ . Lower is better.
- Beyond retrieval, multimodal models are also evaluated on compositional understanding benchmarks like **Winoground** (which tests whether the model can distinguish “a mug in a dog” from “a dog in a mug” ) and **ARO** (Attribute, Relation, Order), which test whether the model genuinely understands the structure of language or merely matches bags of words. CLIP-style models often struggle on these, revealing a fundamental limitation: contrastive pre-training aligns global semantics but may not capture fine-grained compositional structure.

### Recall@K for Image-Text Retrieval



Retrieval evaluation: given a query image, the model ranks all text candidates by similarity, Recall@K measures whether the correct caption appears in the top K results

## 46.8 Putting It All Together

- The multimodal representations covered in this file form the foundation for everything that follows in this chapter. The joint embedding spaces trained by CLIP and its successors are the “glue” that connects vision and language. File 02 builds on this foundation with vision-language models that go beyond retrieval to generate text about images. File 03 explores how images and video are tokenised for use in sequence models. File 04 covers cross-modal generation (text-to-image, text-to-video). And file 05 examines unified architectures that handle multiple modalities within a single model.
- The core takeaway: contrastive learning on paired data produces embedding spaces where different modalities are interchangeable. An image embedding and a text embedding become “the same kind of thing,” enabling zero-shot classification, retrieval, and seamless integration into larger systems. The simplicity of this idea, just push matched pairs together and unmatched pairs apart, belies its extraordinary effectiveness.

## 46.9 Coding Tasks (use CoLab or notebook)

1. Implement the CLIP contrastive loss from scratch. Create random image and text embeddings, compute the similarity matrix, and calculate the symmetric cross-entropy loss.

```

import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

def clip_loss(image_embeds, text_embeds, temperature=0.07):
 """Compute symmetric CLIP contrastive loss."""
 # L2 normalise embeddings
 image_embeds = image_embeds / jnp.linalg.norm(image_embeds, axis=1,
keepdims=True)
 text_embeds = text_embeds / jnp.linalg.norm(text_embeds, axis=1,
keepdims=True)

 # Compute cosine similarity matrix (N x N)
 logits = image_embeds @ text_embeds.T / temperature # (N, N)

 # Labels: the diagonal (i-th image matches i-th text)
 N = logits.shape[0]
 labels = jnp.arange(N)

 # Symmetric cross-entropy: image-to-text + text-to-image
 loss_i2t = -jnp.mean(jax.nn.log_softmax(logits, axis=1)[jnp.arange(N),
labels])
 loss_t2i = -jnp.mean(jax.nn.log_softmax(logits, axis=0)[labels,
jnp.arange(N)])
 return (loss_i2t + loss_t2i) / 2, logits * temperature

Simulate a batch of 8 image-text pairs in 64-dim space
key = jax.random.PRNGKey(42)
k1, k2 = jax.random.split(key)
N, D = 8, 64
image_embeds = jax.random.normal(k1, (N, D))
text_embeds = jax.random.normal(k2, (N, D))

loss, sim_matrix = clip_loss(image_embeds, text_embeds)
print(f"CLIP loss (random embeddings): {loss:.4f}")

Visualise the similarity matrix
fig, ax = plt.subplots(figsize=(6, 5))
im = ax.imshow(sim_matrix, cmap='coolwarm', vmin=-1, vmax=1)
ax.set_xlabel("Text index"); ax.set_ylabel("Image index")
ax.set_title(f"Cosine Similarity Matrix (loss={loss:.3f})")
plt.colorbar(im); plt.tight_layout(); plt.show()

```

```
Try changing temperature (0.01, 0.1, 1.0) and observe how loss changes
Try making matched pairs similar: set text_embeds = image_embeds + small noise
```

1. Build a toy joint embedding model that learns to align 2D “images” (random vectors) with “captions” (different random vectors) using InfoNCE loss and gradient descent.

```
import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

def info_nce_loss(img_enc, txt_enc, img_data, txt_data, tau=0.1):
 """InfoNCE over a batch of paired (image, text) data."""
 z_img = img_data @ img_enc # (N, D)
 z_txt = txt_data @ txt_enc # (N, D)
 # L2 normalise
 z_img = z_img / jnp.linalg.norm(z_img, axis=1, keepdims=True)
 z_txt = z_txt / jnp.linalg.norm(z_txt, axis=1, keepdims=True)
 logits = z_img @ z_txt.T / tau
 labels = jnp.arange(logits.shape[0])
 return -jnp.mean(jax.nn.log_softmax(logits, axis=1)[jnp.arange(len(labels)),
labels])

Create 32 paired samples: img in R^8, txt in R^6, embed into R^4
key = jax.random.PRNGKey(0)
k1, k2, k3, k4 = jax.random.split(key, 4)
N, d_img, d_txt, d_embed = 32, 8, 6, 4

img_data = jax.random.normal(k1, (N, d_img))
txt_data = jax.random.normal(k2, (N, d_txt))

Learnable projection matrices
img_enc = jax.random.normal(k3, (d_img, d_embed)) * 0.1
txt_enc = jax.random.normal(k4, (d_txt, d_embed)) * 0.1

grad_fn = jax.jit(jax.grad(info_nce_loss, argnums=(0, 1)))
lr = 0.05
losses = []

for step in range(300):
 loss = info_nce_loss(img_enc, txt_enc, img_data, txt_data)
 losses.append(float(loss))
 g_img, g_txt = grad_fn(img_enc, txt_enc, img_data, txt_data)
 img_enc = img_enc - lr * g_img
 txt_enc = txt_enc - lr * g_txt
```

```

print(f"Initial loss: {losses[0]:.3f}, Final loss: {losses[-1]:.3f}")
print(f"Random baseline (log N): {jnp.log(N):.3f}")

plt.figure(figsize=(8, 4))
plt.plot(losses, color='#2c3e50')
plt.axhline(y=0, color='green', linestyle='--', alpha=0.5, label='Perfect alignment')
plt.axhline(y=float(jnp.log(N)), color='red', linestyle='--', alpha=0.5, label='Random (log N)')
plt.xlabel("Step"); plt.ylabel("InfoNCE Loss")
plt.title("Learning a Joint Embedding Space")
plt.legend(); plt.grid(alpha=0.3); plt.tight_layout(); plt.show()
Modify d_embed (try 2, 4, 16) to see how embedding dimension affects alignment

```

1. Implement zero-shot classification with pre-computed embeddings. Simulate class “prototypes” as text embeddings and classify new images by nearest-neighbour lookup.

```

import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

Simulate 5 classes, each with a prototype text embedding in R^32
key = jax.random.PRNGKey(42)
n_classes, d = 5, 32
class_names = ["cat", "dog", "car", "plane", "ship"]

Class prototypes (imagine these came from a text encoder)
k1, k2 = jax.random.split(key)
class_prototypes = jax.random.normal(k1, (n_classes, d))
class_prototypes = class_prototypes / jnp.linalg.norm(class_prototypes, axis=1, keepdims=True)

Generate 200 test "images" (embeddings near their class prototype + noise)
n_per_class = 40
true_labels = jnp.repeat(jnp.arange(n_classes), n_per_class)
keys = jax.random.split(k2, n_classes * n_per_class)

image_embeds = []
for i in range(n_classes):
 noise = jax.random.normal(keys[i], (n_per_class, d)) * 0.5
 cluster = class_prototypes[i] + noise
 image_embeds.append(cluster)
image_embeds = jnp.concatenate(image_embeds, axis=0)
image_embeds = image_embeds / jnp.linalg.norm(image_embeds, axis=1, keepdims=True)

```

```

Zero-shot classification: cosine similarity with each prototype
similarities = image_embeds @ class_prototypes.T # (200, 5)
predicted_labels = jnp.argmax(similarities, axis=1)
accuracy = jnp.mean(predicted_labels == true_labels)
print(f"Zero-shot accuracy: {accuracy:.1%}")

Confusion matrix
conf = jnp.zeros((n_classes, n_classes), dtype=jnp.int32)
for true, pred in zip(true_labels, predicted_labels):
 conf = conf.at[true, pred].add(1)

fig, ax = plt.subplots(figsize=(6, 5))
im = ax.imshow(conf, cmap='Blues')
ax.set_xticks(range(n_classes)); ax.set_xticklabels(class_names, rotation=45)
ax.set_yticks(range(n_classes)); ax.set_yticklabels(class_names)
ax.set_xlabel("Predicted"); ax.set_ylabel("True")
for i in range(n_classes):
 for j in range(n_classes):
 ax.text(j, i, int(conf[i, j]), ha='center', va='center', fontsize=11)
ax.set_title(f"Zero-Shot Confusion Matrix (acc={accuracy:.1%})")
plt.colorbar(im); plt.tight_layout(); plt.show()
Try increasing noise (0.5 -> 1.0 -> 2.0) to see accuracy degrade
Try adding prompt ensembling: average 3 noisy copies of each prototype

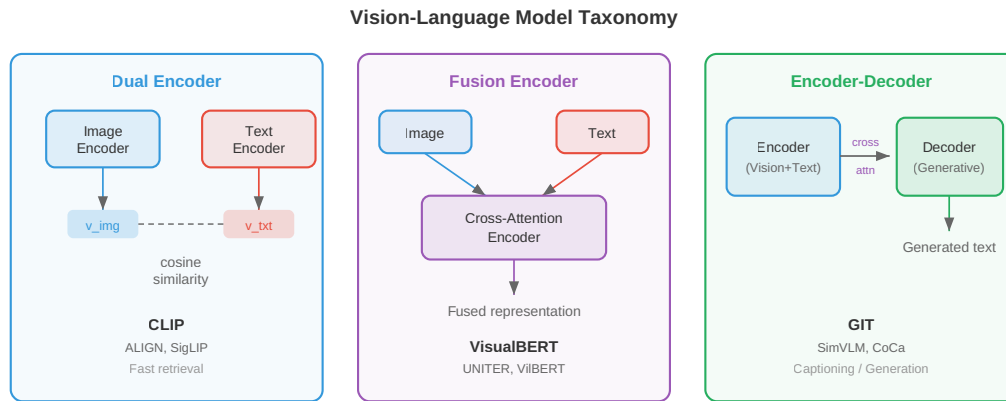
```

## 47 Vision Language Models

Vision language models jointly understand images and text, enabling visual question answering, image captioning, and visual reasoning. This file covers VQA, image captioning, visual grounding, and architectures like VisualBERT, BLIP, LLaVA, Flamingo, PaLI, and Qwen-VL that fuse vision encoders with large language models.

- Think of a museum guide who can look at a painting and articulate everything about it: what objects are present, what story it tells, what emotions it conveys, and answer any question a visitor might pose. A **vision language model (VLM)** is the computational equivalent — a system that jointly understands images and text, enabling it to describe visual scenes, answer questions about them, follow visual instructions, and even locate specific objects within an image given a natural language query.
- VLMs sit at the intersection of the vision encoders you met in Chapter 8 and the language models from Chapter 7. The central engineering challenge is bridging

two very different representational worlds: the spatial, continuous feature maps of a vision backbone and the sequential, discrete token embeddings of a language model. Every architecture in this file is, at its core, a different answer to the question: how do you fuse vision and language?



High-level VLM taxonomy showing dual encoder, fusion encoder, and encoder-decoder families with their inputs and outputs

## 47.1 Visual Question Answering

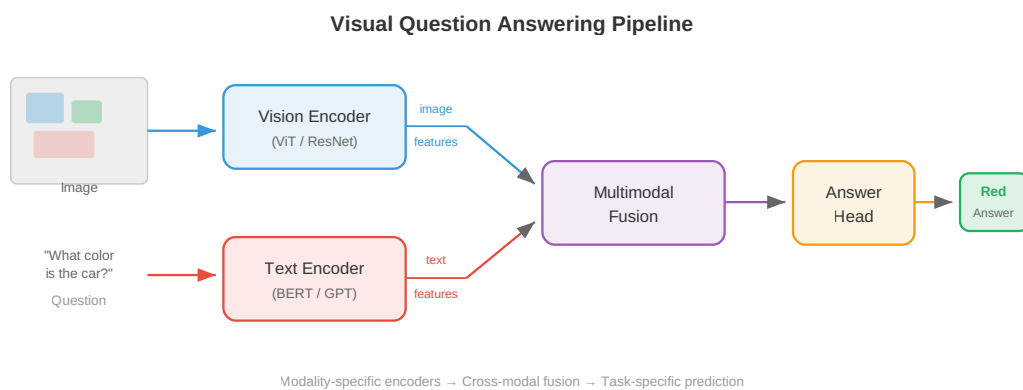
- Imagine someone shows you a photograph and asks “How many dogs are in the park?” You effortlessly parse the image, locate the dogs, count them, and produce an answer. **Visual question answering (VQA)** formalises this: given an image  $I$  and a natural language question  $q$ , predict the answer  $a$ .
- The task can be framed in several ways. The most common treats VQA as **open-ended classification**: the model selects from a fixed vocabulary of the most frequent answers (e.g., the top 3,129 answers in VQA v2). Alternatively, it can be treated as **generative answering**, where the model produces a free-form text string — this is the approach modern VLMs use.
- Formally, you want to learn a function  $f(I, q) \rightarrow a$  that maximises the likelihood of the correct answer. In the classification setup, this becomes:

$$p(a \mid I, q) = \text{softmax}(W \cdot g(v, h))$$

- where  $v$  is a visual feature vector (from a CNN or ViT),  $h$  is a question encoding (from an LSTM or Transformer), and  $g$  is a fusion function that combines them. The design of  $g$  is where the real architectural creativity lies.
- **VQA v1** (Antol et al., 2015) introduced the benchmark with 614,000 questions on 204,000 images from MS COCO. Researchers quickly discovered that models

could achieve surprisingly high accuracy by exploiting **language priors** — answering “2” for “how many” questions or “yes” for “is there” questions without even looking at the image.

- **VQA v2** (Goyal et al., 2017) addressed this by pairing each question with two similar images that yield different answers. This forced models to actually ground their reasoning in visual content. The balanced pair setup roughly doubles the dataset and makes language-only shortcuts much less effective.
- Other important VQA datasets include **GQA** (Hudson & Manning, 2019) with compositional questions requiring multi-step reasoning, **OK-VQA** (Marino et al., 2019) requiring outside knowledge beyond the image, and **TextVQA** (Singh et al., 2019) where the answer depends on reading text within the image.



VQA pipeline: an image passes through a vision encoder and a question passes through a text encoder, their representations are fused, and the fused vector is classified into an answer

- Early VQA models used a simple strategy: extract image features from a pre-trained CNN (typically the penultimate layer of ResNet or VGGNet from Chapter 8), encode the question with an LSTM (Chapter 6), and combine them. The combination function evolved rapidly: from simple element-wise multiplication, to bilinear pooling, to multi-modal Tucker decomposition. **Bilinear attention** computes  $v^T W h$ , where  $W$  is a learnable interaction matrix, but the full bilinear form has  $O(d_v \times d_h)$  parameters, which is prohibitively large. **MLB** (multimodal low-rank bilinear pooling) factorises this into two low-rank projections, making it tractable.
- The breakthrough for VQA was attention. **Stacked Attention Networks** (Yang et al., 2016) used the question encoding to attend over spatial image regions,



iteratively refining which parts of the image to focus on. This idea — letting the question “look at” relevant image regions — became standard.

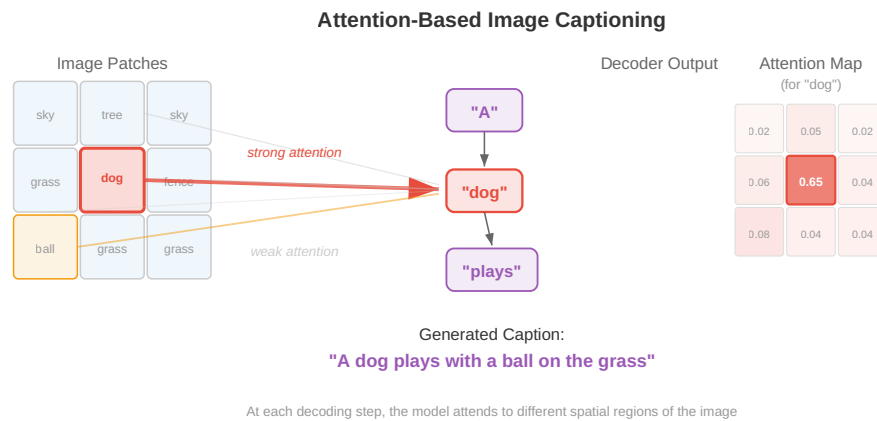
## 47.2 Image Captioning

---

- Picture a friend looking at your holiday photos and narrating what they see: “A golden retriever is catching a frisbee on a sunny beach.” **Image captioning** is the task of generating a natural language description of an image. Unlike VQA, there is no question — the model must decide what is worth describing on its own.
- **Show and Tell** (Vinyals et al., 2015) established the canonical encoder-decoder architecture for captioning. A CNN encoder (e.g., Inception or ResNet) produces a single image feature vector  $v$ . This vector is used as the initial hidden state of an LSTM decoder, which then generates a caption word by word, autoregressively:

$$p(w_t \mid w_{1:t-1}, I) = \text{LSTM}(w_{t-1}, h_{t-1})$$

- The entire model is trained end-to-end by maximising the log-likelihood of ground-truth captions. At inference time, beam search (Chapter 7) is used to find high-probability captions.
- The problem with Show and Tell is that the entire image is compressed into a single vector. For complex scenes, a single vector cannot capture all the relevant details. You lose spatial information — the model cannot “look back” at specific parts of the image while generating different words.
- **Show, Attend and Tell** (Xu et al., 2015) solved this by introducing **attention over image regions**. Instead of encoding the image as one vector, the CNN produces a spatial feature grid (e.g.,  $14 \times 14 \times 512$  from the last convolutional layer of VGGNet). At each decoding step, the model computes attention weights over these spatial locations, producing a context vector that highlights the most relevant region for the current word.
- Recall the attention mechanism from Chapter 6: the decoder hidden state acts as the query, the spatial features act as keys and values, and the attention weights tell the model where to look. The authors proposed two variants: **soft attention** (differentiable, weighted average of all regions) and **hard attention** (stochastic sampling of a single region, trained with REINFORCE).



Attention-based captioning: at each decoding step, the model attends to different spatial regions of the image, focusing on relevant areas like the dog when generating the word dog

- The attention maps produced by these models are remarkably interpretable: when generating “dog,” the attention peaks over the dog region; when generating “beach,” it shifts to the sand and water. This was one of the first compelling demonstrations that attention provides built-in interpretability.
- **CIDEr** (Vedantam et al., 2015), **METEOR**, **BLEU**, and **SPICE** are the standard captioning evaluation metrics. CIDEr computes TF-IDF weighted n-gram similarity between generated and reference captions, specifically designed for captioning evaluation. Modern VLMs are typically evaluated on CIDEr for captioning benchmarks like MS COCO Captions and NoCaps.
- Later captioning models incorporated **bottom-up attention** (Anderson et al., 2018), where an object detector (Faster R-CNN, Chapter 8) first proposes salient image regions, and the captioning model attends over these region features rather than a uniform grid. This was the dominant approach before ViT-based encoders took over.

## 47.3 Architecture Patterns

- Every VLM must answer a fundamental design question: at what point do vision and language interact? The answer defines the model’s architecture family. There are three primary patterns, each with distinct trade-offs.

### 47.3.1 Dual Encoder

- Imagine two translators working independently — one reads a French document, the other reads an English document — and they each produce a

summary in a shared “universal language.” They never communicate during translation, but their summaries are directly comparable. This is the **dual encoder** pattern.

- A vision encoder  $f_v$  and a text encoder  $f_t$  independently map their respective inputs to a shared embedding space of dimension  $d$ . The image embedding is  $v = f_v(I) \in \mathbb{R}^d$  and the text embedding is  $t = f_t(q) \in \mathbb{R}^d$ . Similarity is computed via a dot product or cosine similarity:  $\text{sim}(I, q) = v^T t / (\|v\| \|t\|)$ .
- CLIP (Radford et al., 2021), covered in the previous file on multimodal representations, is the prototypical dual encoder. It is trained with a contrastive objective (InfoNCE) on 400 million image-text pairs scraped from the internet. Because the encoders are independent, you can pre-compute and cache all image embeddings, making retrieval extremely efficient — you only need to encode the query text at search time.
- The dual encoder’s weakness is that vision and language never interact at the feature level. The model cannot perform fine-grained cross-modal reasoning: it cannot, for example, determine whether a specific word in the caption corresponds to a specific region in the image. This limits its usefulness for tasks like VQA or grounded captioning.

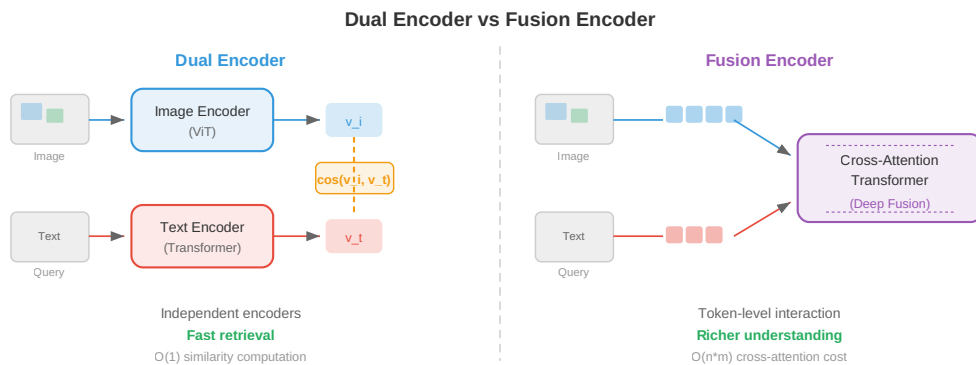
### 47.3.2 Fusion Encoder

- Now imagine the two translators are in the same room, actively discussing both documents. They can point at specific passages, ask each other questions, and build a joint understanding. This is the **fusion encoder** pattern.
- Both modalities are encoded and then fused through **cross-attention layers** where tokens from one modality attend to tokens from the other. The image is first processed by a vision encoder into a sequence of patch or region tokens  $V = [v_1, \dots, v_N]$ . The text is tokenised into  $T = [t_1, \dots, t_M]$ . In the fusion layers, text tokens attend to image tokens via cross-attention:

$$\text{CrossAttn}(T, V) = \text{softmax} \left( \frac{(TW_Q)(VW_K)^T}{\sqrt{d_k}} \right) (VW_V)$$

- This enables fine-grained interaction: each text token can attend to the specific image regions it needs. Models like **VisualBERT**, **VilBERT**, and **UNITER** use this pattern. The cost is that you cannot pre-compute separate embeddings for

retrieval — every image-text pair requires a full forward pass through the fusion layers.



Dual encoder versus fusion encoder: the dual encoder computes separate embeddings and a similarity score, while the fusion encoder merges modalities through cross-attention layers

### 47.3.3 Encoder-Decoder

- The **encoder-decoder** pattern combines the vision encoder with a text decoder that generates output tokens autoregressively, similar to the seq2seq models from Chapter 7. The vision encoder produces contextual image representations, and the text decoder cross-attends to them while generating output text.
- This pattern naturally supports generative tasks: captioning, VQA with free-form answers, and visual dialogue. Models like **GIT** (Generative Image-to-text Transformer, Wang et al., 2022), **CoCa** (Contrastive Captioner, Yu et al., 2022), and **PaLI** use this architecture. CoCa cleverly combines the dual encoder and encoder-decoder patterns: the first half of the text decoder layers operate as a unimodal text encoder (for contrastive learning), while the second half cross-attends to image features (for generative captioning), getting the best of both worlds.
- The choice among these three patterns depends on the target task. Dual encoders are optimal for retrieval at scale. Fusion encoders are best for fine-grained understanding tasks. Encoder-decoders are most versatile for generative tasks. Modern state-of-the-art VLMs increasingly adopt the encoder-decoder or decoder-only paradigm, treating every vision-language task as text generation.

## 47.4 Flamingo: Few-Shot Multimodal Learning

---

- Think of a seasoned expert who, after years of studying both art and literature, can look at a completely new painting style and describe it eloquently after seeing just one or two examples. **Flamingo** (Alonso et al., 2022, DeepMind) is built on the same principle: it leverages a powerful pre-trained language model and a pre-trained vision encoder, connecting them with lightweight architectural components that enable few-shot learning on multimodal tasks.
- Flamingo’ s design philosophy is conservative and effective: keep the pre-trained vision encoder (NFNet) and language model (Chinchilla) frozen, and learn only the “glue” that connects them. This glue consists of two components: a **Perceiver Resampler** and **gated cross-attention layers**.
- The **Perceiver Resampler** takes the variable-length output of the vision encoder (which depends on image resolution) and compresses it into a fixed set of  $N$  visual tokens (typically  $N = 64$ ). It works by initialising a set of  $N$  learnable query vectors and using cross-attention to let these queries attend to the full set of vision encoder outputs. This is essentially the Perceiver architecture (Jaegle et al., 2021) applied as a bottleneck — it produces a compact, fixed-size visual representation regardless of the input image size.

$$z = \text{CrossAttn}(Q_{\text{learned}}, V_{\text{image}}) \in \mathbb{R}^{N \times d}$$

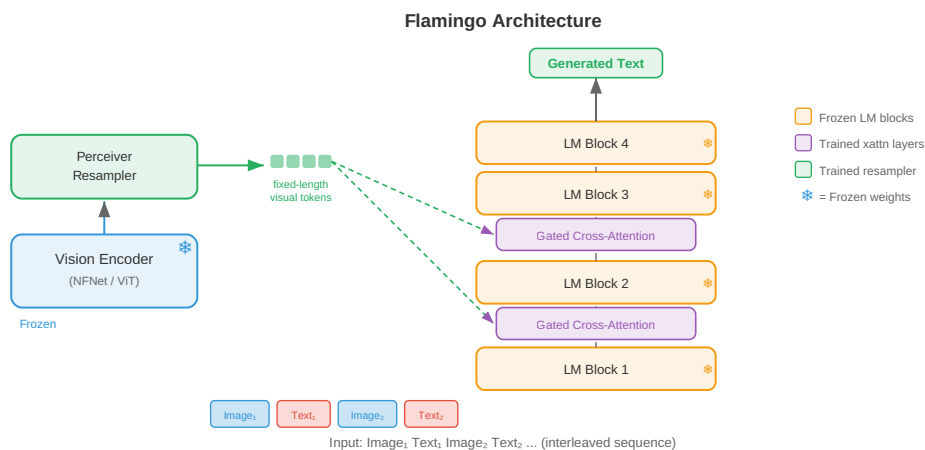
- The **gated cross-attention layers** are interleaved between the frozen language model layers. At each such layer, the language model’ s text tokens cross-attend to the visual tokens produced by the Perceiver Resampler. Critically, each gated cross-attention layer includes a learnable scalar gate  $\alpha$ , initialised to zero, that multiplies the cross-attention output before adding it to the residual stream:

⦿

x

$$= x + \alpha \cdot \text{CrossAttn}(x, z)$$

- Initialising  $\alpha = 0$  means that at the start of training, the cross-attention contributes nothing, and the model behaves exactly like the original frozen language model. The gates gradually open during training, smoothly integrating visual information without disrupting the language model’ s pre-trained representations.



Flamingo architecture: frozen vision encoder feeds into a Perceiver Resampler that produces fixed-length visual tokens, which are injected into a frozen LM via gated cross-attention layers interleaved between LM blocks

- Flamingo natively handles **interleaved image-text sequences**. You can feed it a prompt containing multiple images interspersed with text, such as: “[Image 1] This is a cat. [Image 2] This is a dog. [Image 3] This is a \_\_\_\_.” The model processes each image through the vision encoder and Perceiver Resampler, and the resulting visual tokens are inserted at the corresponding positions in the text sequence. The language model’s causal attention mask ensures that each text token can only attend to visual tokens from the current and preceding images.
- This interleaving enables powerful **few-shot multimodal learning**. By providing a few image-text examples in context, Flamingo can perform new tasks without any gradient updates. On benchmarks like VQAv2, OK-VQA, and captioning, Flamingo with 80B parameters achieved state-of-the-art few-shot performance, often matching or exceeding fine-tuned specialist models with just 4 or 32 examples.

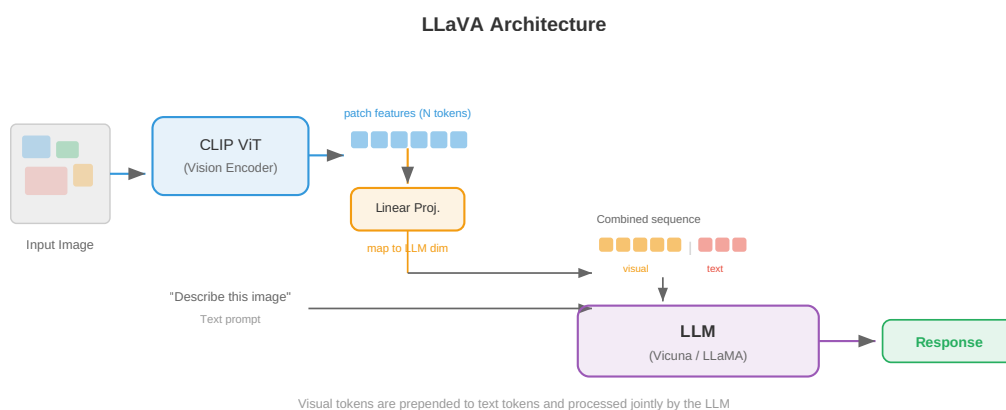
## 47.5 LLaVA and Visual Instruction Tuning

- Imagine you have a brilliant language expert (an LLM) and a brilliant art critic (a vision encoder). If you could teach the art critic to “speak the language expert’s language,” they could collaborate seamlessly. **LLaVA** (Large Language and Vision Assistant, Liu et al., 2023) does exactly this: it projects vision features into the LLM’s token embedding space using a simple linear layer, then fine-tunes the whole system on instruction-following data.

- LLaVA' s architecture is strikingly simple. An image is encoded by a pre-trained CLIP ViT-L/14 vision encoder into a grid of patch features  $V \in \mathbb{R}^N \times d_v$ , where  $N = 256$  patches (for 336px images with 14px patches). A **projection layer**  $W$  maps these vision features into the LLM' s embedding dimension:

$$H_v = VW, \quad W \in \mathbb{R}^{d_v \times d_{LLM}}$$

- The projected visual tokens  $H_v$  are simply concatenated with the text token embeddings and fed into the LLM (Vicuna, a fine-tuned LLaMA) as a single sequence. The LLM processes them with its standard causal self-attention — no special cross-attention layers, no perceiver, just concatenation. The visual tokens are treated as if they were text tokens that happen to encode visual information.



LLaVA architecture: CLIP ViT encodes the image into patch features, a linear projection maps them to the LLM embedding space, projected visual tokens are prepended to text tokens and fed into the LLM

- **Visual instruction tuning** is LLaVA' s key training innovation. The authors used GPT-4 to generate 158,000 multimodal instruction-following examples from COCO images. Each example consists of an image paired with a conversational instruction (e.g., “Describe this image in detail,” “What is unusual about this image?,” “If I were a tourist visiting this place, what should I know?” ). The model is trained to generate the GPT-4-authored response given the image and instruction.
- Training proceeds in two stages. **Stage 1 (pre-training)**: only the projection layer  $W$  is trained on image-caption pairs (595K from CC3M), while both the vision encoder and LLM are frozen. This teaches  $W$  to align visual features with the LLM' s embedding space. **Stage 2 (fine-tuning)**: the projection layer and the LLM are jointly fine-tuned on the instruction-following data, while the

vision encoder stays frozen. This teaches the model to follow complex visual instructions.

- **LLaVA-1.5** improved the original with three key changes: replacing the single linear projection with a two-layer MLP (more expressive mapping), using higher-resolution images (336px instead of 224px, producing more patch tokens), and adding academic VQA datasets to the training mix. These seemingly minor modifications produced a large jump in benchmark performance.
- The LLaVA approach demonstrated that you do not need complex architectural innovations like Flamingo's Perceiver Resampler or gated cross-attention. A simple linear projection, combined with high-quality instruction-tuning data, is enough to connect a vision encoder to an LLM effectively. This simplicity made LLaVA extremely influential — most subsequent open-source VLMs follow a similar recipe.

## 47.6 Scaling Vision-Language Models

---

- The field moved rapidly from proof-of-concept VLMs to industrial-scale systems trained on billions of image-text pairs. Three model families illustrate different approaches to scaling.

### 47.6.1 PaLI

- **PaLI** (Pathways Language and Image model, Chen et al., 2022, Google) scales both the vision encoder and the language model simultaneously. PaLI uses a ViT-e (4B parameters) as the vision encoder and mT5 (13B parameters) as the language model, for a total of 17B parameters. The image is encoded into a sequence of patch tokens, which are prepended to the text tokens and fed into the encoder-decoder mT5.
- PaLI's key insight is that **scaling the vision encoder matters as much as scaling the language model**. Previous work typically used a fixed, moderate-sized vision backbone (e.g., ViT-B or ViT-L) and poured all the parameter budget into the LLM. PaLI showed that a 4B-parameter ViT-e, pre-trained on JFT-4B (4 billion labelled images), dramatically improves performance on fine-grained visual tasks like OCR and spatial reasoning.
- PaLI is trained on WebLI, a dataset of 10 billion image-text pairs in 109 languages, making it inherently multilingual. The model is pre-trained with a



mixture of tasks: image captioning, VQA, and image-text matching, all cast as text-to-text generation (following the T5 paradigm from Chapter 7). **PaLI-X** (55B parameters) and **PaLI-3** (5B, using SigLIP as the vision encoder) are subsequent iterations.

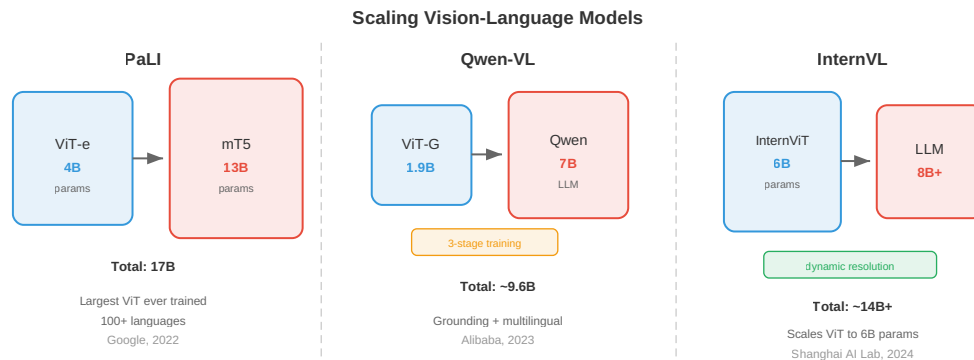
### 47.6.2 Qwen-VL

- **Qwen-VL** (Bai et al., 2023, Alibaba) builds on the Qwen LLM by adding a ViT vision encoder and a single-layer cross-attention module (similar to Flamingo’s Perceiver Resampler) that compresses the vision encoder’s output into a fixed set of 256 visual tokens. The visual tokens are concatenated with text tokens and processed by the Qwen LLM.
- Qwen-VL’s training uses a three-stage recipe. Stage 1: pre-train on 1.4 billion weakly-supervised image-text pairs with only the vision encoder unfrozen. Stage 2: multi-task pre-training on higher-quality data including VQA, captioning, grounding, and OCR datasets, with the full model unfrozen. Stage 3: supervised fine-tuning on instruction-following and dialogue data. This progressive refinement, from noisy web data to curated instruction data, is a pattern shared across most modern VLMs.
- **Qwen2-VL** (2024) introduced **dynamic resolution** support: instead of resizing all images to a fixed size, it processes images at their native resolution by dynamically adjusting the number of visual tokens. Higher-resolution images produce more tokens, and lower-resolution images produce fewer. This improves performance on detail-sensitive tasks like document understanding and fine-grained recognition without wasting computation on low-resolution inputs.

### 47.6.3 InternVL

- **InternVL** (Chen et al., 2024, Shanghai AI Lab) scales the vision encoder aggressively, using InternViT-6B — a 6-billion-parameter vision transformer — paired with a language model. The key architectural contribution is **dynamic high-resolution processing**: images are divided into tiles of 448x448 pixels, each processed independently by the vision encoder, and the resulting tile features are concatenated with a thumbnail feature of the full image. This allows the model to handle images of arbitrary aspect ratios and resolutions.
- InternVL-2 further introduced **progressive alignment training**: first aligning the vision encoder with a contrastive objective (like CLIP), then connecting it to

the LLM through a lightweight MLP connector, and finally fine-tuning end-to-end on instruction data. The progressive strategy prevents catastrophic forgetting of the vision encoder’s pre-trained representations.



Scaling VLMs: comparison of PaLI, Qwen-VL, and InternVL showing different approaches to connecting vision encoders and language models, including their training stages

- A common theme across all three families is the importance of **training data curation**. Raw web-scraped image-text pairs are noisy and often misaligned. Successive training stages progressively filter and refine the data, moving from billions of noisy pairs to millions of high-quality instruction examples. The quality of the final fine-tuning data often matters more than the model’s raw parameter count.

## 47.7 Grounding and Referring

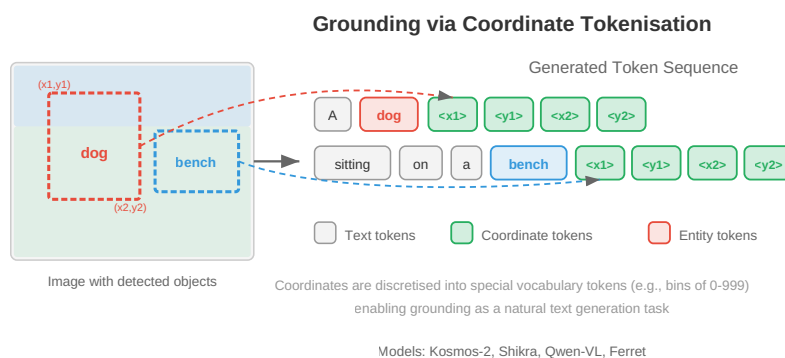
- Imagine pointing at a person in a crowd and saying “the woman in the red hat.” You are using language to refer to a specific spatial region. **Visual grounding** is the reverse: given an image and a natural language expression, the model must identify (localise) the referred object. **Referring expression comprehension** produces a bounding box; **referring expression segmentation** produces a pixel mask.
- Formally, given an image  $I$  and a referring expression  $r$  (e.g., “the large brown dog on the left”), the model predicts a bounding box  $b = (x, y, w, h)$  or a set of coordinates that localise the referent. The datasets include **RefCOCO**, **RefCOCO+**, and **RefCOCOg**, each containing images with multiple objects and unambiguous referring expressions for each.
- Early grounding models used a two-stage approach: first generate region proposals (from Faster R-CNN or similar), then score each proposal against the

language query using a fusion model. The highest-scoring region is the prediction. This is computationally expensive and limited by the quality of the proposals.

- Modern VLMs integrate grounding directly into the generative framework. The key idea is to represent bounding box coordinates as **text tokens**. You discretise the continuous coordinate space into bins (e.g., 1000 bins for each of  $x$ ,  $y$ ,  $w$ ,  $h$ ) and add special location tokens like `<loc_342>` to the vocabulary. The model then generates a bounding box by outputting a sequence of location tokens:

Output: `<loc_102><loc_215><loc_487><loc_398>`

- This tokenisation trick allows any autoregressive language model to perform grounding without any architectural changes — it simply learns to “speak coordinates.” **Pix2Seq** (Chen et al., 2022) pioneered this approach for object detection, and models like Qwen-VL, Ferret, and Kosmos-2 extend it to referring expression comprehension and phrase grounding.
- **Kosmos-2** (Peng et al., 2023, Microsoft) adds grounding capability to a multimodal LLM by representing spatial locations as special tokens embedded within the generated text. For example, it can generate: “A `<phrase>` golden retriever `</phrase>` `<box>` `<loc_102>` `<loc_215>` `<loc_487>` `<loc_398>` `</box>` is catching a frisbee.” This interleaving of text and spatial tokens enables simultaneous captioning and grounding.



Grounding via coordinate tokenisation: the model generates text tokens interspersed with discretised bounding box coordinate tokens, localising objects mentioned in the caption

- **Pointing** takes grounding further: instead of bounding boxes, the model predicts a single point (typically the centre of the referred object). This is useful for interactive applications where a user asks “Where is the nearest exit?” and

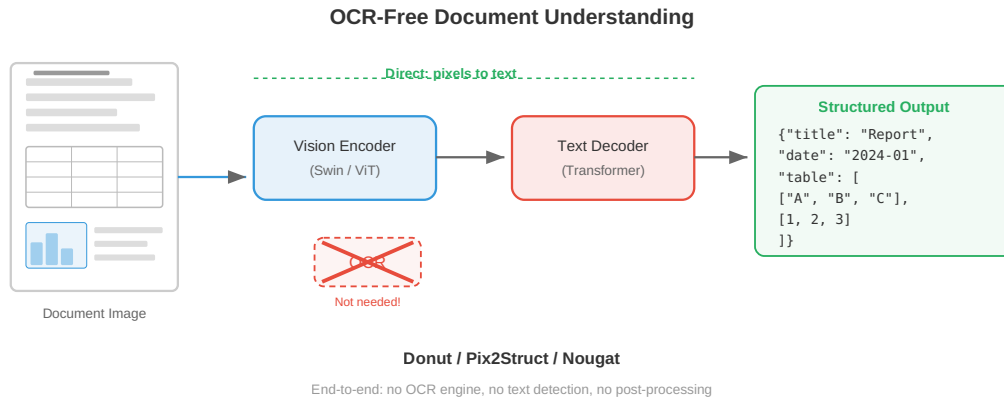
the model responds with a coordinate overlaid on the image. Models like **Shikra** and **Ferret** support point-based referring in addition to box-based grounding.

## 47.8 OCR-Free Document Understanding

---

- Traditional document understanding pipelines are complex: first run an OCR engine to extract text and layout, then feed the extracted text into a language model. This multi-stage approach is fragile — OCR errors propagate downstream, and the spatial layout information is often lost or poorly represented. What if the model could read directly from pixels, the way you do?
- **Donut** (Document Understanding Transformer, Kim et al., 2022) eliminates OCR entirely. It uses a Swin Transformer (Chapter 8) as the vision encoder to process the document image, and a BART-style Transformer decoder to generate structured text output directly from the visual features. The decoder can produce JSON, key-value pairs, or plain text, depending on the task.
- Donut's training is two-stage. **Pre-training**: the model learns to read by performing synthetic OCR — given a document image, it generates the full text content. This is trained on millions of synthetic document images rendered from text corpora, teaching the vision encoder to recognise characters, fonts, and layouts. **Fine-tuning**: the model is adapted to specific downstream tasks like receipt parsing, form understanding, or document classification, by training it to generate task-specific structured output.
- The Donut decoder uses a special prompting scheme: the task is specified by a prompt token (e.g., `<doc_class>` for classification or `<parse_receipt>` for receipt parsing), and the model generates the output conditioned on this prompt. This unified interface allows a single model to handle multiple document understanding tasks.
- **Pix2Struct** (Lee et al., 2023, Google) takes the OCR-free idea and applies it to web page understanding and chart/figure comprehension. The key pre-training objective is **screenshot parsing**: given a masked screenshot of a web page, the model generates the underlying HTML that produced the visible region. This teaches the model to understand the relationship between visual rendering and structured markup.
- Pix2Struct introduces **variable-resolution input processing**: instead of resizing all images to a fixed size (which distorts aspect ratios and destroys fine text), it

packs the image into a fixed number of patches while preserving the original aspect ratio. A tall, narrow document produces a tall, narrow patch grid. This is critical for document understanding, where aspect ratio carries semantic information (a receipt is narrow and tall; a spreadsheet is wide and short).



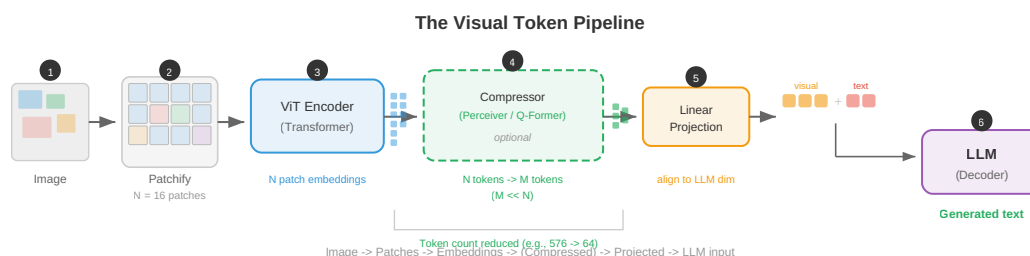
OCR-free document understanding: Donut and Pix2Struct process document images directly through a vision encoder and generate structured text output without any OCR preprocessing

- **Nougat** (Blecher et al., 2023, Meta) applies the Donut architecture specifically to academic papers, generating full LaTeX markup directly from PDF page images. It can handle complex mathematical equations, tables, and figures — tasks where traditional OCR pipelines struggle badly. The model is trained on pairs of PDF page images and their corresponding LaTeX source code.
- The success of OCR-free models demonstrates a broader principle in deep learning: end-to-end models that learn directly from raw inputs (pixels) often outperform complex multi-stage pipelines, because they can jointly optimise all components and learn representations that are specifically tailored to the final task. The intermediate OCR step is a bottleneck that constrains what the model can learn.

## 47.9 The Visual Token Pipeline

- Regardless of architecture family, every VLM must convert an image into a sequence of tokens that a language model can process. Understanding this pipeline is essential. The process varies by model, but the general flow is:
- **Step 1: Patch extraction.** The image (height  $H$ , width  $W$ ) is divided into non-overlapping patches of size  $P \times P$ , producing  $N = HW/P^2$  patches. For a 336x336 image with 14x14 patches,  $N = 576$ .

- **Step 2: Vision encoding.** Each patch is linearly projected and passed through the vision encoder (typically a ViT). The output is a sequence of contextual patch embeddings  $V = [v_1, \dots, v_N] \in \mathbb{R}^N \times d_v$ . These embeddings carry both local appearance information and global context (from self-attention).
- **Step 3: Token compression (optional).** Some models compress the  $N$  visual tokens into a smaller set of  $M \ll N$  tokens to reduce the computational burden on the language model. Flamingo uses a Perceiver Resampler ( $M = 64$ ); Qwen-VL uses cross-attention ( $M = 256$ ); **Q-Former** (used in BLIP-2, Li et al., 2023) uses a set of  $M = 32$  learnable query tokens that cross-attend to the vision encoder's output.
- **Step 4: Projection.** The visual tokens (either the full set or the compressed set) are projected into the language model's embedding space via a linear layer or MLP. After projection, visual tokens have the same dimensionality as text token embeddings and can be concatenated with them.
- **Step 5: Injection into the LLM.** The projected visual tokens are inserted into the token sequence at the position of a special `<image>` placeholder token, and the combined sequence is processed by the language model. The LLM's self-attention allows text tokens to attend to visual tokens and vice versa.



The visual token pipeline: image patches are extracted, encoded by a ViT, optionally compressed by a Perceiver or Q-Former, projected to the LLM dimension, and concatenated with text tokens

- The number of visual tokens directly affects computational cost. Each visual token participates in the LLM's self-attention, which is quadratic in sequence length. A high-resolution image with many patches can produce hundreds or thousands of visual tokens, dominating the LLM's context window. This is why token compression is important: reducing 576 visual tokens to 64 cuts the visual contribution to attention by roughly 9x.

- **BLIP-2** (Li et al., 2023) is notable for its efficient bridging strategy. It introduces a lightweight **Q-Former** (a small Transformer with learnable queries) that sits between the frozen vision encoder and the frozen LLM. The Q-Former is the only trainable component — both the vision encoder and LLM remain frozen. It is pre-trained in two stages: first with image-text contrastive learning, matching, and captioning objectives (connecting it to the vision encoder), then with language generation objectives (connecting it to the LLM). This modular design allows BLIP-2 to plug any vision encoder into any LLM.

## 47.10 Training Objectives

---

- VLMs are trained with a combination of objectives, depending on the architecture pattern:
- **Image-text contrastive loss (ITC)**: aligns image and text representations in a shared embedding space, as in CLIP. This is the primary objective for dual encoders and is often used as a pre-training objective for fusion models. The loss is the InfoNCE loss from the previous file.
- **Image-text matching (ITM)**: a binary classification objective — given an image and a text, predict whether they match. Hard negatives (text that is similar but paired with a different image) make this task challenging and force the model to learn fine-grained alignment.
- **Language modelling (LM)**: the standard autoregressive language modelling objective — predict the next token given all previous tokens. For VLMs, the “previous tokens” include the visual tokens, so the model learns to generate text conditioned on visual input. This is the primary objective for encoder-decoder and decoder-only VLMs.

$$\mathcal{L}_{\text{LM}} = - \sum_{t=1}^T \log p(w_t \mid w_{<t}, V)$$

- **Prefix language modelling**: a variant where the image and a text prefix are provided as context (not trained on), and the model is trained to generate only the continuation. This is used in models like PaLI and SimVLM.



- Most modern VLMs combine multiple objectives during pre-training (e.g., ITC + ITM + LM in BLIP, ITC + LM in CoCa) and then fine-tune with a pure LM objective on instruction data.

## 47.11 Coding Tasks (use CoLab or notebook)

1. Implement a simple attention-based image captioning decoder. Use random “image features” as the encoder output and train the decoder to generate a fixed caption, observing how the attention weights shift across spatial positions at each decoding step.

```
import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

Simulate a 4x4 spatial grid of image features (16 regions, dim=32)
key = jax.random.PRNGKey(42)
k1, k2, k3 = jax.random.split(key, 3)
img_features = jax.random.normal(k1, (16, 32)) # 16 spatial regions, 32-dim

Vocabulary: 0=<start>, 1="a", 2="red", 3="car", 4=<end>
vocab_size, embed_dim, hidden_dim = 5, 16, 32
W_embed = jax.random.normal(k2, (vocab_size, embed_dim)) * 0.1
W_attn_q = jax.random.normal(k3, (hidden_dim, 32)) * 0.1 # query projection

def attend(h, img_feats, W_q):
 """Compute soft attention over image features given decoder state h."""
 query = h @ W_q # (32,)
 scores = img_feats @ query # (16,)
 weights = jax.nn.softmax(scores) # (16,)
 context = weights @ img_feats # (32,)
 return context, weights

Simple GRU-like step (for illustration, just a linear + tanh)
W_h = jax.random.normal(jax.random.PRNGKey(0), (embed_dim + 32, hidden_dim)) *
0.1

def decode_step(h, word_idx, img_feats):
 context, attn_weights = attend(h, img_feats, W_attn_q)
 word_emb = W_embed[word_idx] # (16,)
 inp = jnp.concatenate([word_emb, context]) # (48,)
 h_new = jnp.tanh(inp @ W_h) # (32,)
 return h_new, attn_weights
```



```

Run decoding for the sequence: <start> -> "a" -> "red" -> "car" -> <end>
target_seq = [0, 1, 2, 3, 4]
h = jnp.zeros(hidden_dim)
all_attn = []
for word_idx in target_seq[:-1]:
 h, attn_w = decode_step(h, word_idx, img_features)
 all_attn.append(attn_w)

Visualise attention maps (reshaped to 4x4 grid) at each step
words = ["<start>", "a", "red", "car"]
fig, axes = plt.subplots(1, 4, figsize=(14, 3))
for i, (ax, w) in enumerate(zip(axes, words)):
 ax.imshow(all_attn[i].reshape(4, 4), cmap='viridis')
 ax.set_title(f'Attending when\ngenerating after "{w}"')
 ax.axis('off')
plt.suptitle('Attention Over Image Regions at Each Decoding Step')
plt.tight_layout(); plt.show()
Try changing img_features to see how attention patterns shift!

```

1. Simulate the visual token pipeline: patchify an image, project patches to an embedding space, concatenate with text token embeddings, and run a single self-attention layer over the combined sequence.

```

import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

key = jax.random.PRNGKey(7)

Create a synthetic 8x8 "image" with 3 channels
k1, k2, k3, k4 = jax.random.split(key, 4)
image = jax.random.uniform(k1, (8, 8, 3))

Step 1: Patchify into 4x4 patches -> 4 patches
patch_size = 4
patches = image.reshape(2, patch_size, 2, patch_size, 3)
patches = patches.transpose(0, 2, 1, 3, 4).reshape(4, patch_size * patch_size *
3) # (4, 48)
print(f"Number of patches: {patches.shape[0]}, patch dim: {patches.shape[1]}")

Step 2: Project patches to embedding dim (d=16)
d_model = 16
W_patch = jax.random.normal(k2, (patches.shape[1], d_model)) * 0.1
visual_tokens = patches @ W_patch # (4, 16)

Step 3: Create text token embeddings (simulate 3 text tokens)

```

```

text_tokens = jax.random.normal(k3, (3, d_model)) * 0.1

Step 4: Concatenate visual + text tokens
combined = jnp.concatenate([visual_tokens, text_tokens], axis=0) # (7, 16)
print(f"Combined sequence length: {combined.shape[0]} (4 visual + 3 text)")

Step 5: Single-head self-attention over the combined sequence
W_Q = jax.random.normal(k4, (d_model, d_model)) * 0.1
k5, k6 = jax.random.split(k4)
W_K = jax.random.normal(k5, (d_model, d_model)) * 0.1
W_V = jax.random.normal(k6, (d_model, d_model)) * 0.1

Q = combined @ W_Q
K = combined @ W_K
V = combined @ W_V
attn_scores = (Q @ K.T) / jnp.sqrt(d_model)
attn_weights = jax.nn.softmax(attn_scores, axis=-1) # (7, 7)

output = attn_weights @ V # (7, 16)

Visualise the cross-modal attention pattern
labels = ['V1', 'V2', 'V3', 'V4', 'T1', 'T2', 'T3']
fig, ax = plt.subplots(figsize=(6, 5))
im = ax.imshow(attn_weights, cmap='Blues')
ax.set_xticks(range(7)); ax.set_xticklabels(labels)
ax.set_yticks(range(7)); ax.set_yticklabels(labels)
ax.set_xlabel('Key'); ax.set_ylabel('Query')
ax.set_title('Self-Attention: Visual (V) and Text (T) Tokens')
plt.colorbar(im, ax=ax); plt.tight_layout(); plt.show()
Observe: text tokens attend to visual tokens (cross-modal attention)!

```

1. Implement coordinate tokenisation for visual grounding. Given a bounding box, convert it to discrete tokens; given discrete tokens, reconstruct the bounding box. Visualise the quantisation error at different bin resolutions.

```

import jax.numpy as jnp
import matplotlib.pyplot as plt

def encode_bbox(bbox, num_bins=1000):
 """Convert continuous bbox (x, y, w, h) in [0,1] to discrete tokens."""
 tokens = jnp.round(jnp.array(bbox) * (num_bins - 1)).astype(jnp.int32)
 return tokens

def decode_bbox(tokens, num_bins=1000):
 """Convert discrete tokens back to continuous bbox."""
 return tokens.astype(jnp.float32) / (num_bins - 1)

```

```

Ground-truth bounding box (normalised to [0, 1])
gt_bbox = jnp.array([0.123, 0.456, 0.333, 0.222])

Test quantisation at different bin resolutions
bin_sizes = [10, 50, 100, 500, 1000]
errors = []
for n_bins in bin_sizes:
 tokens = encode_bbox(gt_bbox, n_bins)
 reconstructed = decode_bbox(tokens, n_bins)
 error = jnp.max(jnp.abs(gt_bbox - reconstructed))
 errors.append(float(error))
 print(f"Bins={n_bins:>5d} | Tokens={tokens} | "
 f"Reconstructed={reconstructed} | Max error={error:.6f}")

fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(bin_sizes, errors, 'o-', color='#e74c3c', linewidth=2, markersize=8)
ax.set_xlabel('Number of Bins'); ax.set_ylabel('Max Quantisation Error')
ax.set_title('Bounding Box Quantisation Error vs Bin Resolution')
ax.set_xscale('log'); ax.set_yscale('log')
ax.grid(True, alpha=0.3); plt.tight_layout(); plt.show()
Try: what happens with very few bins (e.g., 5)? When is the error acceptable?

```

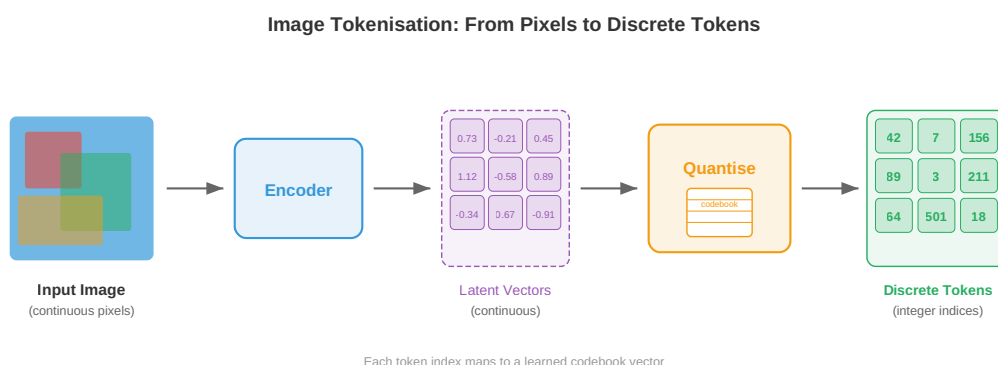
## 48 Image and Video Tokenisation

Image and video tokenisation converts continuous visual data into discrete token sequences that transformers can process like text. This file covers VQ-VAE, VQ-GAN, codebook learning, DALL-E’s dVAE, video tokenisation, and lookup-free quantisation

### 48.1 Why Tokenise Images

- Think of language as a finite alphabet: English has roughly 26 letters, and modern language models carve text into 30,000-100,000 subword tokens. Every sentence becomes a sequence of discrete symbols that a transformer can predict one by one. Images, on the other hand, live in a continuous, high-dimensional space: a single 256x256 RGB image is a point in  $\mathbb{R}^{256 \times 256 \times 3} \approx \mathbb{R}^{196,608}$ . If you want a language model to “speak” images with the same machinery it uses to speak English, you need to convert those continuous pixel arrays into a manageable sequence of discrete tokens drawn from a finite vocabulary. That conversion is **image tokenisation**.

- Imagine you are a mosaic artist. You do not have infinite shades of tile; you have a fixed palette of, say, 8192 distinct tile colours. To reproduce a photograph as a mosaic, you must (1) decide which region of the photo each tile represents, (2) pick the closest tile colour for each region, and (3) accept that some detail is lost but the overall picture is recognisable. Image tokenisation does exactly this: an encoder compresses spatial patches into latent vectors, a codebook maps each vector to its nearest entry, and the result is a grid of integer indices, one per patch, that a discrete model can process.
- The benefits of tokenisation are threefold. First, it compresses the image dramatically: a 256x256 image might become a 16x16 grid of tokens, reducing the sequence length from 65,536 pixels to 256 tokens, which is tractable for attention-based models whose cost scales quadratically with sequence length. Second, it unifies the representation: text tokens and image tokens live in the same discrete vocabulary, enabling a single autoregressive transformer to generate interleaved text and images. Third, it imposes a useful bottleneck that forces the model to learn semantically meaningful codes rather than memorising pixel noise.



Overview of the image tokenisation pipeline: continuous image enters an encoder, latent vectors are quantised against a codebook, producing a grid of discrete token indices

- Recall from Chapter 8 how convolutional networks extract hierarchical feature maps from images, and from Chapter 7 how text tokenisers convert strings into integer sequences. Image tokenisation sits at the intersection: it uses a CNN or vision transformer encoder (Chapter 8) to produce spatial features, then borrows the idea of a discrete vocabulary (Chapter 7) to convert those features into token indices.

## 48.2 VQ-VAE: Vector Quantisation

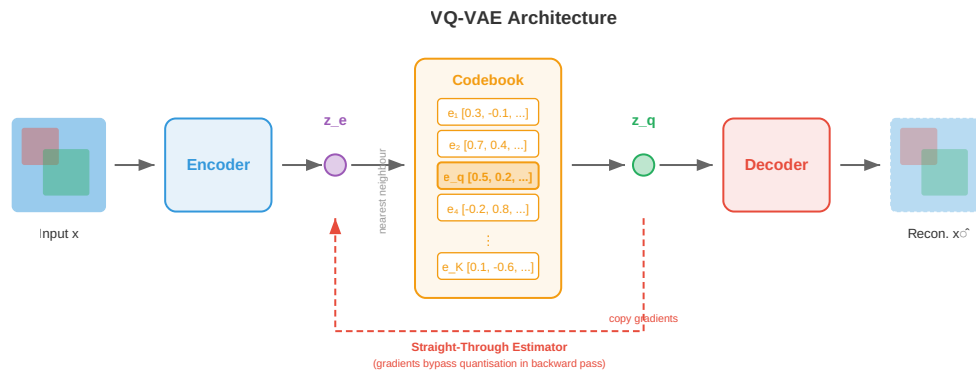
- As we saw in Chapter 6, a standard **variational autoencoder** (VAE) encodes an input into a continuous latent distribution and decodes samples from that distribution back into reconstructions. The latent space is continuous, which makes it awkward to feed into discrete sequence models. The **Vector Quantised Variational Autoencoder** (VQ-VAE), introduced by van den Oord et al. (2017), replaces the continuous latent with a discrete one by introducing a learnable codebook of embedding vectors and snapping each encoder output to its nearest codebook entry.
- Picture a library with exactly  $K$  labelled shelves. When a new book (encoder output) arrives, the librarian places it on the shelf whose existing books (codebook vectors) it most closely resembles, and records the shelf number. Later, to retrieve the book, you only need the shelf number: the codebook entry on that shelf is a good enough stand-in. This is vector quantisation.
- Formally, the VQ-VAE has three components:
- An **encoder**  $E$  that maps an input image  $\mathbf{x} \in \mathbb{R}^{H \times W \times 3}$  to a spatial grid of continuous latent vectors  $\mathbf{z}_e = E(\mathbf{x}) \in \mathbb{R}^{h \times w \times d}$ , where  $h \times w$  is the downsampled spatial resolution and  $d$  is the embedding dimension.
- A **codebook**  $\mathcal{C} = \{\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_K\} \subset \mathbb{R}^d$  containing  $K$  learnable embedding vectors. Typical codebook sizes range from 512 to 16,384 entries.
- A **decoder**  $D$  that reconstructs the image from the quantised latents.
- The **quantisation step** replaces each encoder output  $\mathbf{z}_e(\mathbf{x})$  at spatial position  $(i, j)$  with its nearest codebook entry:

$$\mathbf{z}_q(i, j) = \mathbf{e}_{k^*} \text{ where } k^* = \arg$$

min  
k

$$\| \mathbf{z}_e(i, j) - \mathbf{e}_k \|_2$$

- This is a nearest-neighbour lookup in embedding space, exactly the same operation as k-means assignment (Chapter 6). The index  $k^*$  is the discrete token for spatial position  $(i, j)$ , and the full image is represented as an  $h \times w$  grid of integers from  $\{1, \dots, K\}$ .



VQ-VAE architecture: encoder produces continuous latents, each latent vector is matched to the nearest codebook entry, decoder reconstructs from quantised codes

- The challenge is that argmin is not differentiable: you cannot backpropagate through a discrete selection. VQ-VAE solves this with the **straight-through estimator**: during the forward pass, the decoder receives  $\mathbf{z}_q$  (the quantised vector); during the backward pass, the gradient of the reconstruction loss with respect to  $\mathbf{z}_q$  is copied directly to  $\mathbf{z}_e$ , as if the quantisation step were the identity function. This is written compactly as:

$$\mathbf{z}_q = \mathbf{z}_e + \text{sg}(\mathbf{z}_q - \mathbf{z}_e)$$

- where  $\text{sg}(\cdot)$  is the stop-gradient operator. In the forward pass this evaluates to  $\mathbf{z}_q$ ; in the backward pass, the gradient flows through only the  $\mathbf{z}_e$  term.
- The full VQ-VAE loss has three terms:

$$\mathcal{L} =$$

$$\| \mathbf{x} - D(\mathbf{z}_q) \|_2^2$$

~  
reconstruction

$$+ \| \text{sg}(\mathbf{z}_e) - \mathbf{e} \|_2^2$$

~  
codebook (VQ)

$$+ \beta \| \mathbf{z}_e - \text{sg}(\mathbf{e}) \|_2^2$$

~  
commitment

- The **reconstruction loss** trains the encoder and decoder to faithfully reproduce the input. The **codebook loss** (also called the VQ loss) pulls the codebook

vectors toward the encoder outputs; note that  $\text{sg}(\mathbf{z}_e)$  means the encoder does not receive gradients from this term, so it only updates the codebook. The **commitment loss** does the reverse: it encourages the encoder outputs to stay close to the codebook vectors, preventing the encoder from “running away” from the codebook. The hyperparameter  $\beta$  (typically 0.25) controls the balance between the codebook and commitment terms.

- In practice, the codebook is often updated with an **exponential moving average** (EMA) rather than gradient descent, which is more stable. Let  $\mathbf{n}_k$  be the count of encoder outputs assigned to codebook entry  $k$  and  $\mathbf{s}_k$  be their sum.

The EMA update is:

$$\mathbf{n}_k \leftarrow \gamma \mathbf{n}_k + (1 - \gamma) |\{ (i, j) : k_{ij}^* = k \}|$$

$$\mathbf{s}_k \leftarrow \gamma \mathbf{s}_k + (1 - \gamma) \sum_{(i, j) : k_{ij}^* = k} \mathbf{z}_e(i, j)$$

$$\sum_{(i, j) : k_{ij}^* = k}$$

$$\mathbf{z}_e(i, j)$$

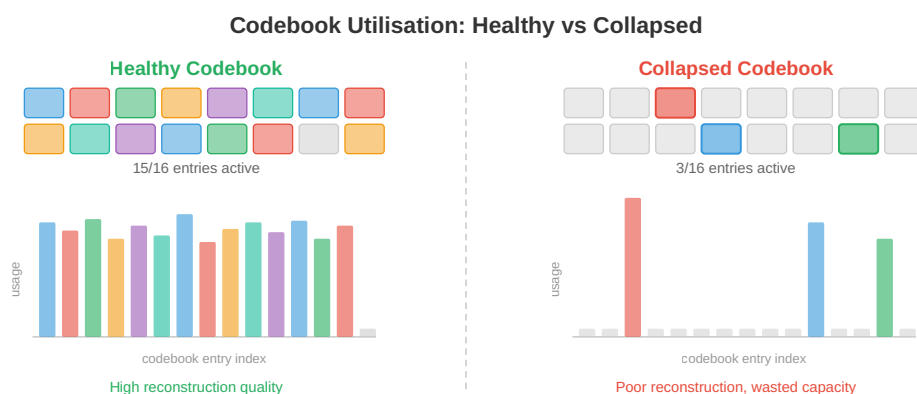
$$\mathbf{e}_k \leftarrow \frac{\mathbf{s}_k}{\mathbf{n}_k}$$

- where  $\gamma$  is the decay rate (typically 0.99). This is equivalent to running an online k-means algorithm on the encoder outputs.

### 48.2.1 Codebook Collapse

- A notorious failure mode of VQ-VAE is **codebook collapse** (also called index collapse): the model learns to use only a small fraction of the  $K$  codebook entries, leaving most entries “dead.” Imagine a library where 90% of the shelves are empty because the librarian always routes books to the same few popular shelves. This wastes representational capacity.
- Codebook collapse occurs because the encoder, codebook, and decoder co-adapt during training. If an entry is not selected for several batches, it drifts away from the encoder manifold, making it even less likely to be selected, creating a positive feedback loop.

- Several techniques mitigate codebook collapse:
  - **Codebook reset:** periodically reinitialise dead entries by copying randomly sampled encoder outputs. This gives dead entries a fresh start near the active region of the latent space.
  - **EMA updates with Laplace smoothing:** add a small constant to  $\mathbf{n}_k$  to prevent any entry from having zero count, ensuring all entries receive gradient signal.
  - **Commitment loss tuning:** increasing  $\beta$  forces encoder outputs to cluster more tightly around codebook entries, distributing assignments more evenly.
  - **Factorised codes:** decompose the codebook lookup into a product of smaller lookups (e.g., two codebooks of size  $\sqrt{K}$  each), which improves utilisation by reducing the effective codebook size for each lookup.
  - **Entropy regularisation:** add a penalty that encourages a uniform distribution over codebook usage, maximising the entropy  $H = -\sum_k p_k \log p_k$  where  $p_k$  is the empirical assignment probability.



Codebook utilisation: healthy codebook with evenly distributed assignments versus collapsed codebook where most entries are unused

## 48.3 VQ-GAN: Adversarial Training for Higher Fidelity

- VQ-VAE produces decent reconstructions, but the pixel-level  $\ell_2$  loss tends to generate blurry outputs because it penalises every pixel deviation equally, averaging over plausible details rather than choosing crisp ones. Imagine asking someone to draw a face that minimises the average difference from all



possible faces — they would draw a blurry average face, not a sharp individual one.

- **VQ-GAN** (Esser et al., 2021) addresses this by combining the VQ-VAE framework with a **discriminator** from generative adversarial networks (Chapter 6). The discriminator is a patch-based convolutional network that judges whether a local image patch is real (from the training data) or fake (from the decoder). This adversarial loss encourages the decoder to produce perceptually sharp, realistic textures instead of pixel-wise averages.
- The VQ-GAN objective adds two terms to the VQ-VAE loss:

$$\mathcal{L}_{\text{VQ-GAN}} = \mathcal{L}_{\text{VQ-VAE}} + \lambda_{\text{adv}} \mathcal{L}_{\text{adv}} + \lambda_{\text{perc}} \mathcal{L}_{\text{perc}}$$

- The **adversarial loss**  $\mathcal{L}_{\text{adv}}$  is the standard GAN objective applied to the decoder output. The discriminator  $\mathcal{D}$  tries to distinguish real patches from decoded patches, and the decoder (generator) tries to fool it. The non-saturating formulation is:

$$\mathcal{L}_{\text{adv}} = -\mathbb{E}[\log \mathcal{D}(\mathbf{D}(\mathbf{z}_q))]$$

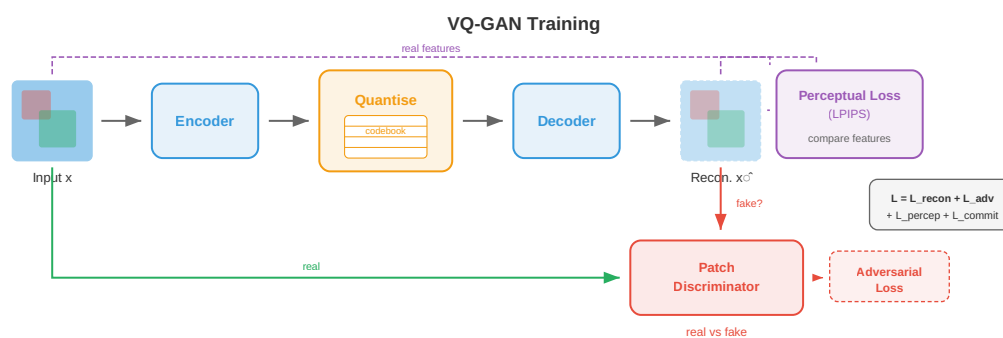
- The **perceptual loss**  $\mathcal{L}_{\text{perc}}$  compares feature activations from a pretrained network (typically VGG or LPIPS) between the original and reconstructed images:

$$\mathcal{L}_{\text{perc}} =$$

$\sum_l$

$$\| \phi_l(\mathbf{x}) - \phi_l(\mathbf{D}(\mathbf{z}_q)) \|_2^2$$

- where  $\phi_l$  denotes the feature map at layer  $l$  of the pretrained network. This loss captures high-level structural similarity rather than pixel-level accuracy.
- The weight  $\lambda_{\text{adv}}$  is adaptively set so that the adversarial gradient and reconstruction gradient are balanced, preventing the adversarial loss from dominating early in training when the reconstructions are poor.



VQ-GAN training: encoder and decoder are connected through a quantisation step, with a patch discriminator providing adversarial feedback on decoded outputs

- The result is a tokeniser that produces dramatically sharper reconstructions than VQ-VAE at the same codebook size. VQ-GAN is the backbone tokeniser behind many major image generation systems, including the original DALL-E, Parti, and numerous text-to-image models. It turns a 256x256 image into a 16x16 or 32x32 grid of discrete tokens from a codebook of size 1024-16384, achieving compression ratios of 16x to 64x in each spatial dimension.

## 48.4 Residual Quantisation and Multi-Scale Codebooks

- A single codebook imposes a hard ceiling on reconstruction quality: each spatial position is represented by exactly one codebook vector, and any detail finer than the codebook can express is lost. Think of describing a colour with a single word from a fixed palette: “teal” is close but not exact. If you could add a refinement — “teal, but slightly more blue and a touch brighter” — you would get much closer.
- **Residual quantisation** (RQ) applies this idea iteratively. After the first quantisation step produces  $\mathbf{z}_q^{(1)}$ , compute the residual  $\mathbf{r}^{(1)} = \mathbf{z}_e - \mathbf{z}_q^{(1)}$ , then quantise the residual against a second codebook to get  $\mathbf{z}_q^{(2)}$ , and so on for T levels:

$$\mathbf{r}^{(0)} = \mathbf{z}_e$$

$$\mathbf{z}_q^{(t)} = \text{Quantise}(\mathbf{r}^{(t-1)}, \mathcal{C}^{(t)})$$

$$\mathbf{r}^{(t)} = \mathbf{r}^{(t-1)} - \mathbf{z}_q^{(t)}$$

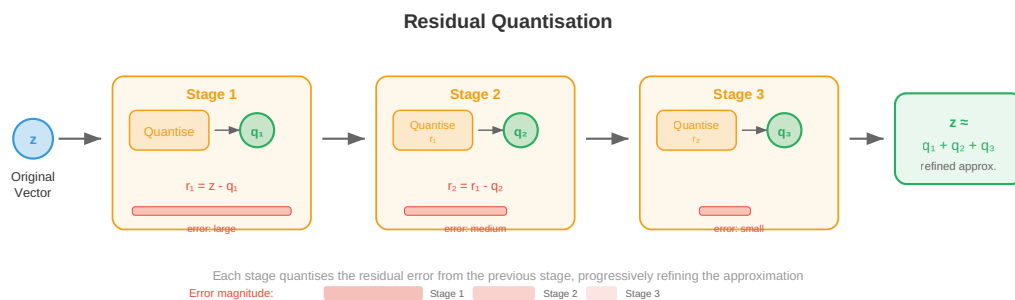
- The final quantised representation is

↺

$\mathbf{z}$

$= \sum_{t=1}^T \mathbf{z}_q^{(t)}$ . With  $T$  levels each using a codebook of size  $K$ , the effective vocabulary size is  $K^T$ , but you only need to store  $T \times K$  vectors rather than  $K^T$ . For example, 8 levels with  $K = 1024$  give an effective  $1024^8 \approx 10^{24}$  entries while storing only 8192 vectors.

- Each successive level captures finer details: the first codebook captures the coarse structure, the second captures medium-frequency corrections, and so on. This is analogous to successive approximation in JPEG or progressive rendering in web images, where a rough version appears first and detail fills in incrementally.



Residual quantisation: the original vector is approximated in successive stages, each stage quantising the residual from the previous stage

- **Multi-scale codebooks** extend this idea by operating at different spatial resolutions. Instead of quantising the same spatial grid repeatedly, you quantise at multiple scales: a coarse grid captures global structure, finer grids capture local detail. This is related to the feature pyramid idea from Chapter 8' s object detection section, where features at different scales capture different levels of detail.
- **Product quantisation** is a related technique where the  $d$ -dimensional latent vector is split into  $M$  sub-vectors of dimension  $d/M$ , and each sub-vector is quantised independently with its own codebook. This gives an effective vocabulary of  $K^M$  while storing only  $M \times K$  vectors. Product quantisation is widely used in approximate nearest-neighbour search (Chapter 13) and has been adapted for image tokenisation.

- **Finite scalar quantisation** (FSQ), introduced by Mentzer et al. (2023), takes a different approach entirely: instead of learning a codebook, it simply rounds each dimension of the latent vector to one of a fixed set of integer levels (e.g.,  $\{-2, -1, 0, 1, 2\}$ ). With  $L$  levels per dimension and  $d$  dimensions, the implicit codebook size is  $L^d$ . FSQ avoids codebook collapse entirely because there are no learned codebook vectors, only learned encoder outputs that are rounded deterministically. The straight-through estimator handles the non-differentiability of rounding.

## 48.5 Image Tokenisers in Practice

---

- The progression from VQ-VAE to VQ-GAN to residual quantisation has spawned a family of practical image tokenisers used in state-of-the-art generative models.

### 48.5.1 DALL-E Tokeniser (dVAE)

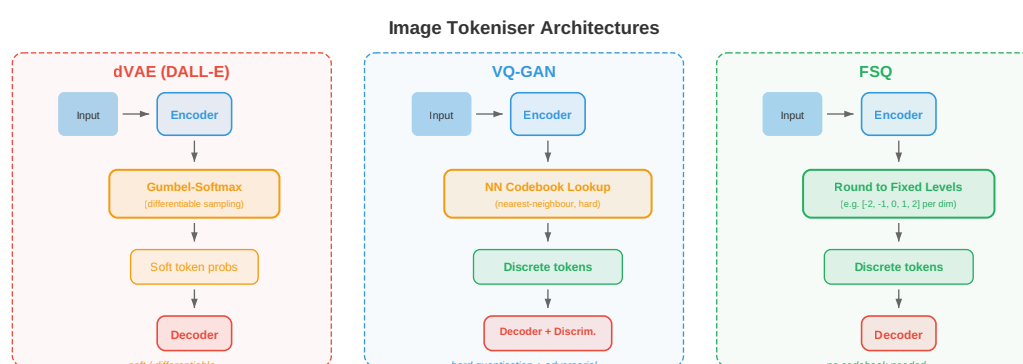
- The original **DALL-E** (Ramesh et al., 2021) used a discrete VAE (dVAE) to tokenise 256x256 images into 32x32 grids of tokens from a codebook of size 8192. The dVAE replaced the hard argmin quantisation with a Gumbel-Softmax relaxation, making the forward pass differentiable during training. At inference time, the argmax is used to produce hard token assignments. The dVAE was trained with a combination of reconstruction loss, KL divergence against a uniform prior, and a learned temperature schedule for the Gumbel-Softmax. DALL-E then trained a 12-billion parameter autoregressive transformer to model the joint distribution of 256 text tokens and 1024 image tokens (32x32).

### 48.5.2 LlamaGen

- **LlamaGen** (Sun et al., 2024) showed that you can repurpose a standard Llama-style language model architecture (Chapter 7) for autoregressive image generation, provided you have a good image tokeniser. LlamaGen uses an improved VQ-GAN tokeniser with a large codebook (16,384 entries) and trains a vanilla autoregressive transformer (with no special image-specific modifications beyond the tokeniser) to predict image tokens left-to-right in raster scan order. The key insight is that once images are tokenised into discrete sequences, the same next-token-prediction paradigm that works for language works for images, validating the idea that tokenisation truly bridges the modality gap.

### 48.5.3 Cosmos Tokeniser

- The **Cosmos tokeniser** (NVIDIA, 2024) is designed for both images and videos in a unified framework. It uses a causal 3D architecture that treats images as single-frame videos, allowing the same tokeniser to handle both modalities. Cosmos supports both continuous and discrete tokenisation modes: the continuous mode outputs real-valued latent vectors (for diffusion model backends), while the discrete mode applies finite scalar quantisation to produce integer tokens (for autoregressive model backends). The encoder uses causal 3D convolutions so that each frame's tokens depend only on the current and previous frames, enabling streaming video tokenisation.



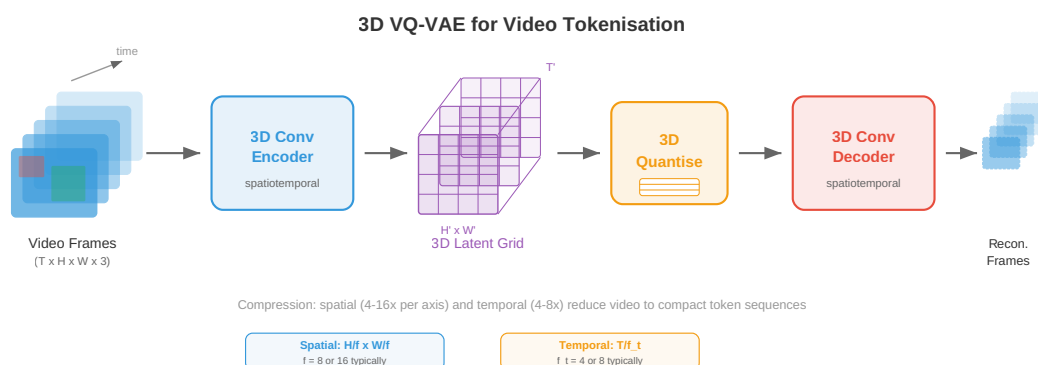
Comparison of image tokeniser architectures: dVAE with Gumbel-Softmax, VQ-GAN with codebook lookup, and FSQ with scalar rounding

## 48.6 Video Tokenisation

- Video adds a third axis — time — to the spatial dimensions of images. A video is a sequence of frames, typically at 24-30 frames per second, and adjacent frames are highly redundant because the visual world does not change drastically in 33 milliseconds. Video tokenisation exploits this temporal redundancy to achieve much higher compression than tokenising each frame independently.
- Think of video compression like a flip-book. If you drew every page from scratch, you would need thousands of detailed drawings. But most pages are nearly identical to their neighbours, so you could draw a full “keyframe” every 10 pages and only note the small changes on the pages in between. Video tokenisers learn this trick automatically.

### 48.6.1 3D VQ-VAE

- The most straightforward extension of VQ-VAE to video is the **3D VQ-VAE**, which replaces 2D convolutions in the encoder and decoder with 3D convolutions that operate over the spatial and temporal dimensions simultaneously. If the encoder downsamples by a factor of  $f_s$  spatially and  $f_t$  temporally, a video clip of  $T \times H \times W$  becomes a token grid of  $(T/f_t) \times (H/f_s) \times (W/f_s)$ .
- For example, with  $f_s = 16$  and  $f_t = 4$ , a 16-frame 256x256 video clip becomes a  $4 \times 16 \times 16 = 1024$  token sequence. This is compact enough for a transformer to model autoregressively, whereas the raw pixel count would be  $16 \times 256 \times 256 \times 3 \approx 3.1$  million values.
- The 3D convolutions jointly learn spatial and temporal features. Early layers capture local motion (edges moving between frames) while deeper layers capture higher-level dynamics (objects appearing, disappearing, or changing shape). This is the same hierarchical feature extraction principle from Chapter 8's convolutional networks, extended along the time axis.



3D VQ-VAE for video: a short video clip is encoded by 3D convolutions into a spatiotemporal grid of latent vectors, quantised, and decoded back to frames

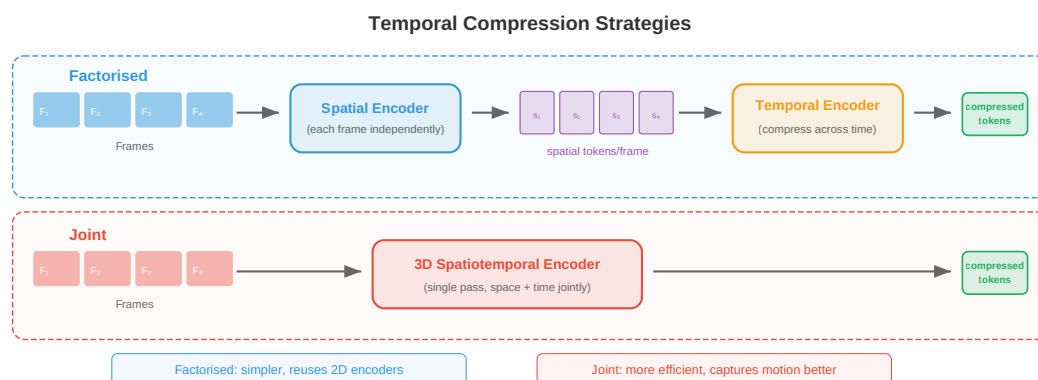
### 48.6.2 Causal Video Tokenisers

- A standard 3D convolution looks at past, current, and future frames, which means you need the entire video clip before you can tokenise any of it. **Causal video tokenisers** constrain the temporal convolutions so that each output depends only on the current and previous frames, never future frames. This is analogous to the causal masking in autoregressive transformers (Chapter 7): information flows forward in time but never backward.

- Causal tokenisation is essential for two use cases. First, **streaming**: you can tokenise video in real time as frames arrive, without buffering future frames. Second, **autoregressive generation**: when a transformer generates video frame-by-frame, the tokens for frame  $t$  must be computable without knowing frame  $t + 1$ , because frame  $t + 1$  has not been generated yet.
- The causal constraint is implemented by padding temporal convolutions asymmetrically: a kernel of temporal size  $k$  is padded with  $k - 1$  zeros on the past side and zero zeros on the future side, ensuring the output at time  $t$  depends only on inputs at times  $t - k + 1, \dots, t$ .
- One elegant property of causal video tokenisers is that they can tokenise a single image (a “video” of one frame) with no special handling. The first frame has no past context, so its tokens are computed from the frame alone. This **image-video unification** means a single tokeniser serves both modalities, simplifying the architecture and enabling models that generate images and videos with the same decoder.

#### 48.6.3 Temporal Compression Strategies

- Different applications demand different temporal compression ratios. For action recognition (where subtle motions matter), gentle compression ( $f_t = 2$ ) preserves temporal detail. For long-form video generation (where storing thousands of frames is prohibitive), aggressive compression ( $f_t = 8$  or higher) is necessary.
- Some tokenisers use **factorised compression**: spatial and temporal compression are performed in separate stages. First, a 2D encoder compresses each frame independently, producing a per-frame latent grid. Then, a 1D temporal encoder compresses across the time dimension. This factorisation is computationally cheaper than full 3D convolution and allows different compression ratios for space and time. The trade-off is that it cannot capture spatiotemporal patterns (like a ball moving diagonally) as efficiently as joint 3D encoding.
- **Temporal interpolation tokens** are a recent innovation where the tokeniser encodes only keyframes fully and represents intermediate frames as lightweight interpolation codes that describe how to morph between keyframes. This mirrors classical video compression (I-frames and P-frames in H.264/HEVC) but in a learned latent space.



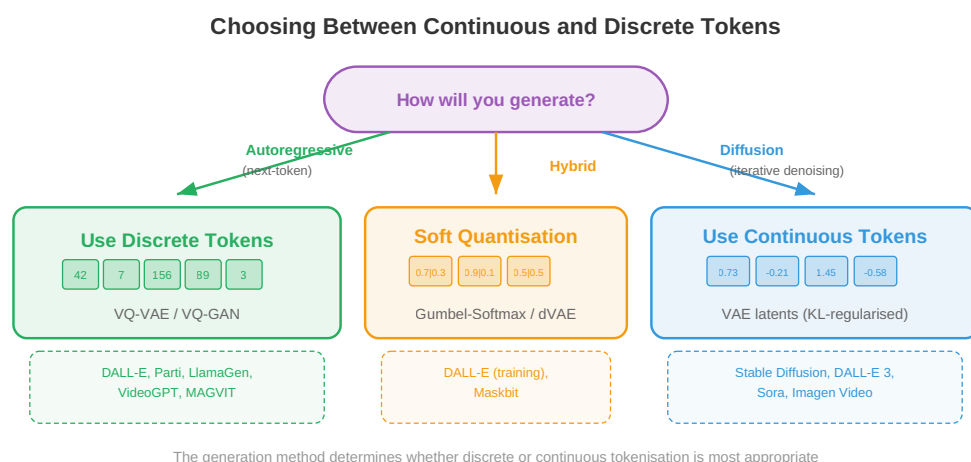
Temporal compression strategies: frame-independent spatial encoding followed by temporal encoding, versus joint spatiotemporal 3D encoding

## 48.7 Continuous vs Discrete Tokens

- Not every downstream model needs discrete tokens. **Diffusion models** (Chapter 10, file 04) work natively with continuous values — they iteratively denoise a Gaussian sample, and their loss functions (denoising score matching) are defined over continuous spaces. For diffusion backends, the tokeniser encoder produces continuous latent vectors that are never quantised. **Latent diffusion models** (Stable Diffusion, DALL-E 3, Flux) use a VQ-GAN-like encoder-decoder but skip the codebook entirely, operating in the continuous latent space.
- Autoregressive models** (GPT-style), on the other hand, predict the next token from a finite vocabulary using a softmax over  $K$  classes. They fundamentally require discrete tokens. Every image generation system that uses an autoregressive transformer (DALL-E, Parti, LlamaGen, Chameleon) depends on a discrete tokeniser.
- The choice between continuous and discrete tokens is therefore driven by the generation backend:
- Use **discrete tokens** when: the model is autoregressive (next-token prediction with cross-entropy loss), you want to share a vocabulary with text tokens for unified multimodal models, or you need exact token-level control (e.g., for retrieval or editing by token replacement).
- Use **continuous tokens** when: the model is a diffusion model or flow-matching model, the task requires very high fidelity reconstruction (continuous latents avoid quantisation error entirely), or you want to use regression losses that operate on real-valued vectors.



- Some recent architectures support both modes. The Cosmos tokeniser, for instance, can output either continuous latents (for its diffusion mode) or FSQ-discretised tokens (for its autoregressive mode) from the same encoder, with a lightweight quantisation head that can be switched on or off.
- Soft quantisation** is a middle ground: instead of hard argmin assignment, compute a weighted average of the top-k nearest codebook entries, with weights given by a softmax over negative distances. This preserves more information than hard quantisation while still being approximately discrete. Some systems use soft quantisation during training and hard quantisation at inference.



Decision tree for choosing between continuous and discrete tokenisation based on the downstream generation model

## 48.8 Applications

### 48.8.1 Autoregressive Image Generation

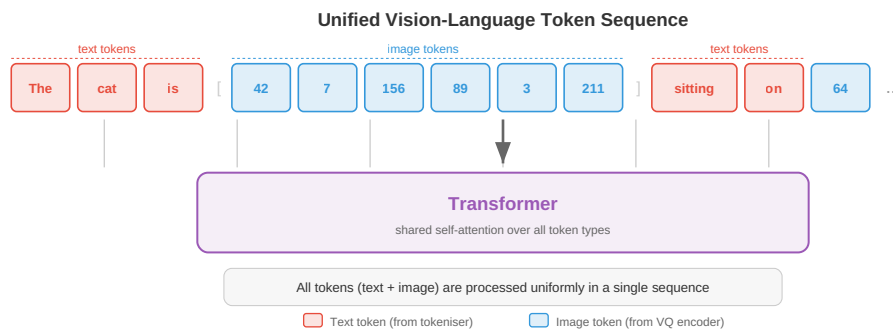
- Once images are discrete token sequences, you can train a standard autoregressive transformer to model them. The image tokens are flattened into a 1D sequence (typically in raster scan order: left-to-right, top-to-bottom) and the transformer learns  $p(\text{token}_i \mid \text{token}_1, \dots, \text{token}_{i-1})$  with the standard cross-entropy loss. At generation time, tokens are sampled one by one and the completed grid is passed through the tokeniser's decoder to produce pixels.
- Conditioning on text is straightforward: prepend text tokens to the image token sequence, so the model learns  $p(\text{image tokens} \mid \text{text tokens})$ . This is exactly how DALL-E, Parti, and LlamaGen perform text-to-image generation. The text and image tokens share the same transformer, the same attention mechanism, and

often the same embedding table (with text and image tokens occupying different index ranges).

- The raster scan order introduces an artificial asymmetry: the top-left of the image is generated first, without any context about the bottom-right. Several works address this. **Masked image modelling** (MaskGIT) trains a bidirectional transformer that generates all tokens simultaneously but with varying confidence, iteratively unmasking the most confident tokens. **Multi-scale generation** generates coarse tokens first (capturing global composition) and then refines with residual tokens. These approaches trade off the simplicity of pure left-to-right generation for better global coherence.

#### 48.8.2 Unified Vision-Language Tokens

- The deepest motivation for image tokenisation is **unification**: putting vision and language into the same representational format so that a single model architecture handles both. As we discussed in Chapter 7, language models are extraordinarily capable sequence-to-sequence machines. By representing images as token sequences, we inherit all the infrastructure of language modelling — pretraining recipes, scaling laws, RLHF, context length extensions — for free.
- **Chameleon** (Meta, 2024) is a prominent example: it uses a VQ-GAN tokeniser with 8192 codebook entries to convert images into tokens that are interleaved with text tokens in a single vocabulary of ~65,000 entries (text + image). A standard transformer is trained on mixed text-image sequences, enabling it to generate text given images, images given text, or interleaved text-and-image content, all with the same forward pass.
- **Gemini** (Google, 2024) takes a similar approach at massive scale, natively understanding and generating images, audio, and text within a single transformer, with modality-specific tokenisers feeding into a shared sequence.
- The key engineering challenge in unified models is **vocabulary balance**: if 8192 out of 65,000 vocabulary entries are image tokens, the model may under-allocate capacity to vision. Solutions include separate embedding layers for each modality (shared only at the attention level), modality-specific loss weighting, and careful data mixing ratios during pretraining.



Unified vision-language model: text and image tokens from separate tokenisers are interleaved into a single sequence processed by one transformer

## 48.9 Coding Tasks (use CoLab or notebook)

1. Implement a minimal VQ layer in JAX: given a batch of encoder output vectors, perform nearest-neighbour codebook lookup and compute the VQ-VAE loss (reconstruction + codebook + commitment). Visualise codebook utilisation as a histogram.

```
import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

--- Minimal VQ layer ---
key = jax.random.PRNGKey(42)
d = 8 # embedding dimension
K = 64 # codebook size
n_vectors = 256 # batch of encoder outputs

Random encoder outputs and codebook
k1, k2 = jax.random.split(key)
z_e = jax.random.normal(k1, (n_vectors, d)) # encoder outputs
codebook = jax.random.normal(k2, (K, d)) * 0.1 # codebook (small init)

Nearest-neighbour lookup: find closest codebook entry for each z_e
distances[i, k] = ||z_e[i] - codebook[k]||^2
distances = (
 jnp.sum(z_e ** 2, axis=1, keepdims=True)
 - 2 * z_e @ codebook.T
 + jnp.sum(codebook ** 2, axis=1, keepdims=True).T
)
indices = jnp.argmin(distances, axis=1) # token indices
z_q = codebook[indices] # quantised vectors
```

```

VQ-VAE loss terms
beta = 0.25
loss_codebook = jnp.mean((jax.lax.stop_gradient(z_e) - z_q) ** 2)
loss_commit = jnp.mean((z_e - jax.lax.stop_gradient(z_q)) ** 2)
loss_total = loss_codebook + beta * loss_commit
print(f"Codebook loss: {loss_codebook:.4f}, Commitment loss: {loss_commit:.4f}")

Codebook utilisation
unique, counts = jnp.unique(indices, return_counts=True, size=K, fill_value=-1)
plt.figure(figsize=(10, 4))
plt.bar(range(K), counts, color='#3498db', alpha=0.8)
plt.xlabel('Codebook Index'); plt.ylabel('Assignment Count')
plt.title(f'Codebook Utilisation ({jnp.sum(counts > 0)}/{K} entries used)')
plt.grid(True, alpha=0.3); plt.tight_layout(); plt.show()
Try: increase K to 512 and observe collapse. Then add codebook reset logic.

```

1. Build a toy 2D vector quantiser that learns to tile a 2D distribution. Generate random 2D points, learn a codebook via EMA updates, and visualise the Voronoi regions.

```

import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

Generate 2D data from a mixture of Gaussians
key = jax.random.PRNGKey(0)
n_points = 2000
K = 16 # codebook entries
gamma = 0.99 # EMA decay

Four clusters
keys = jax.random.split(key, 5)
centres = jnp.array([[2, 2], [-2, 2], [-2, -2], [2, -2]], dtype=jnp.float32)
data = jnp.concatenate([
 jax.random.normal(keys[i], (n_points // 4, 2)) * 0.5 + centres[i]
 for i in range(4)
])

Initialise codebook from random data points
idx = jax.random.choice(keys[4], n_points, (K,), replace=False)
codebook = data[idx]
ema_count = jnp.ones(K)
ema_sum = codebook.copy()

Run EMA-based codebook learning for several epochs
for epoch in range(30):

```

```

Assign each point to nearest codebook entry
dists = jnp.sum((data[:, None, :] - codebook[None, :, :]) ** 2, axis=2)
assignments = jnp.argmin(dists, axis=1)
EMA update
for k in range(K):
 mask = (assignments == k)
 count_k = jnp.sum(mask)
 ema_count = ema_count.at[k].set(gamma * ema_count[k] + (1 - gamma) *
count_k)

 if count_k > 0:
 sum_k = jnp.sum(data[mask], axis=0)
 ema_sum = ema_sum.at[k].set(gamma * ema_sum[k] + (1 - gamma) * sum_k)
 codebook = ema_sum / ema_count[:, None]

Visualise assignments and codebook
fig, ax = plt.subplots(1, 1, figsize=(8, 8))
colors = plt.cm.tab20(jnp.linspace(0, 1, K))
for k in range(K):
 mask = assignments == k
 ax.scatter(data[mask, 0], data[mask, 1], c=[colors[k]], s=5, alpha=0.3)
ax.scatter(codebook[:, 0], codebook[:, 1], c='black', s=120, marker='X',
 edgecolors='white', linewidths=1.5, zorder=10, label='Codebook')
ax.set_title(f'Learned VQ Codebook ({K} entries) on 2D Data')
ax.legend(); ax.set_aspect('equal'); ax.grid(True, alpha=0.3)
plt.tight_layout(); plt.show()
Try: increase K to 64 and observe finer tiling. Reduce gamma and see
instability.

```

1. Demonstrate residual quantisation: encode a batch of vectors with  $T$  successive quantisation stages and measure how the reconstruction error decreases with each level.

```

import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

key = jax.random.PRNGKey(7)
d = 16 # embedding dimension
K = 32 # codebook size per level
T = 8 # number of residual levels
n_vectors = 512

Random data to quantise
k1, *cb_keys = jax.random.split(key, T + 1)
z = jax.random.normal(k1, (n_vectors, d))

```

```

Independent random codebooks for each level
codebooks = [jax.random.normal(cb_keys[t], (K, d)) * (0.5 ** t)
 for t in range(T)]

Residual quantisation loop
residual = z.copy()
z_hat = jnp.zeros_like(z)
errors = []

for t in range(T):
 cb = codebooks[t]
 dists = (jnp.sum(residual ** 2, axis=1, keepdims=True)
 - 2 * residual @ cb.T
 + jnp.sum(cb ** 2, axis=1, keepdims=True).T)
 indices = jnp.argmin(dists, axis=1)
 z_q_t = cb[indices]
 z_hat = z_hat + z_q_t
 residual = residual - z_q_t
 mse = jnp.mean(jnp.sum((z - z_hat) ** 2, axis=1))
 errors.append(float(mse))
 print(f"Level {t+1}: MSE = {mse:.4f}")

plt.figure(figsize=(8, 5))
plt.plot(range(1, T + 1), errors, 'o-', color='#e74c3c', linewidth=2,
markersize=8)
plt.xlabel('Residual Quantisation Level')
plt.ylabel('Reconstruction MSE')
plt.title('Error Reduction with Residual Quantisation')
plt.xticks(range(1, T + 1)); plt.grid(True, alpha=0.3)
plt.tight_layout(); plt.show()
Try: use a single codebook of size K*T and compare with RQ. Which wins?

```

1. Simulate a simple 1D “video tokeniser” : generate a sequence of 1D signals (mimicking video frames), apply causal temporal compression, and compare with non-causal compression in terms of reconstruction quality.

```

import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

key = jax.random.PRNGKey(99)
n_frames = 16
frame_len = 64

Generate a "video": a slowly moving Gaussian bump across frames
x_axis = jnp.linspace(-3, 3, frame_len)

```

```

frames = jnp.stack([
 jnp.exp(-0.5 * (x_axis - (-2 + 4 * t / n_frames)) ** 2)
 for t in range(n_frames)
]) # shape: (n_frames, frame_len)

Causal temporal compression: each frame's code depends only on past frames
Simple approach: average current frame with exponential decay of past
alpha_causal = 0.6
causal_codes = jnp.zeros_like(frames)
causal_codes = causal_codes.at[0].set(frames[0])
for t in range(1, n_frames):
 causal_codes = causal_codes.at[t].set(
 alpha_causal * frames[t] + (1 - alpha_causal) * causal_codes[t - 1]
)

Non-causal: average with both past and future (bilateral smoothing)
kernel = jnp.array([0.2, 0.6, 0.2]) # past, current, future
padded = jnp.concatenate([frames[:1], frames, frames[-1:]], axis=0)
noncausal_codes = jnp.stack([
 kernel[0] * padded[t] + kernel[1] * padded[t+1] + kernel[2] * padded[t+2]
 for t in range(n_frames)
])

Reconstruction error
mse_causal = jnp.mean((frames - causal_codes) ** 2)
mse_noncausal = jnp.mean((frames - noncausal_codes) ** 2)
print(f"Causal MSE: {mse_causal:.6f}, Non-causal MSE: {mse_noncausal:.6f}")

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
for ax, data, title in zip(axes,
 [frames, causal_codes, noncausal_codes],
 ['Original Frames', f'Causal (MSE={mse_causal:.5f})',
 f'Non-causal (MSE={mse_noncausal:.5f})']):
 ax.imshow(data, aspect='auto', cmap='viridis', origin='lower')
 ax.set_xlabel('Spatial Position'); ax.set_ylabel('Frame Index')
 ax.set_title(title)
plt.tight_layout(); plt.show()
Try: vary alpha_causal and the kernel weights. What happens with alpha=1.0?

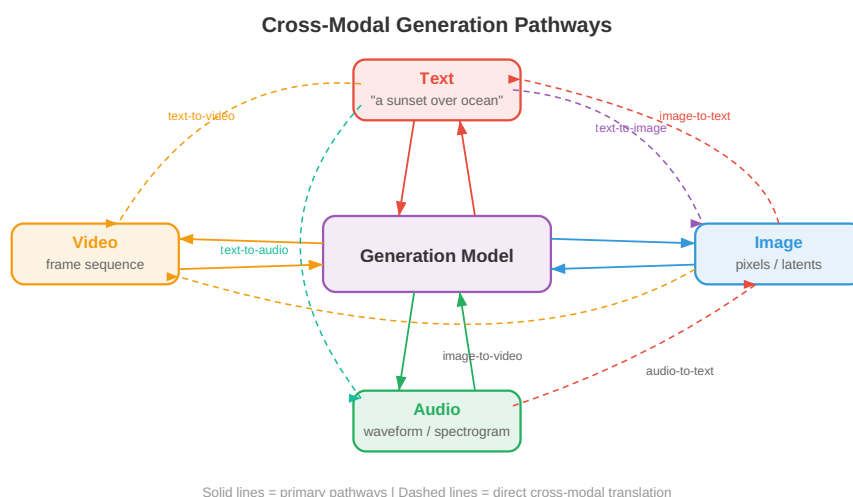
```

## 49 Cross-Modal Generation

Cross-modal generation produces output in one modality conditioned on input from another; text to image, image to text, text to audio, and beyond. This file covers DALL-

E, Stable Diffusion, classifier-free guidance, ControlNet, image captioning, text-to-video (Sora), and text-to-audio generation.

- In files 01-03 of this chapter, you learned how to represent, align, and tokenise different modalities. Now comes the creative act: generating one modality from another. Cross-modal generation is the engine behind text-to-image tools, video synthesis systems, music composition models, and image captioning. Think of it as teaching a machine to be a multimedia artist — you describe what you want in words, and it paints, animates, or composes.
- The core idea is **conditional generation**: given an input from modality A (e.g., text), produce an output in modality B (e.g., image). Formally, you learn a model  $p_{\theta}(y \mid x)$  where  $x$  is the conditioning signal and  $y$  is the generated output. The challenge is that this conditional distribution is enormously complex and high-dimensional — a 512x512 image lives in  $\mathbb{R}^{786432}$ , and there are many valid images for a single text prompt.



Cross-modal generation overview: text, image, audio, and video modalities connected by directional arrows showing generation pathways such as text-to-image, image-to-text, text-to-audio, and text-to-video

## 49.1 Text-to-Image Generation

- Imagine you describe a scene to a courtroom sketch artist. The artist must interpret your words, recall what objects look like, compose them spatially, and render the final picture. Text-to-image models do precisely this, but they must learn all of these skills from data rather than from years of art school.



### 49.1.1 DALL-E: Autoregressive Image Generation

- **DALL-E** (Ramesh et al., 2021) treats image generation as a sequence prediction problem — the same paradigm that powers language models (Chapter 07). The key insight is that if you can represent images as discrete tokens (recall VQ-VAE from file 03), then generating an image is just generating a sequence of tokens, one after another.
- The pipeline has two stages. First, a **discrete VAE (dVAE)** compresses a 256x256 image into a 32x32 grid of discrete tokens from a codebook of 8192 entries, reducing the image to a sequence of 1024 tokens. Second, a **transformer decoder** is trained to model the joint distribution of 256 text tokens (BPE-encoded) concatenated with 1024 image tokens, totalling 1280 tokens:

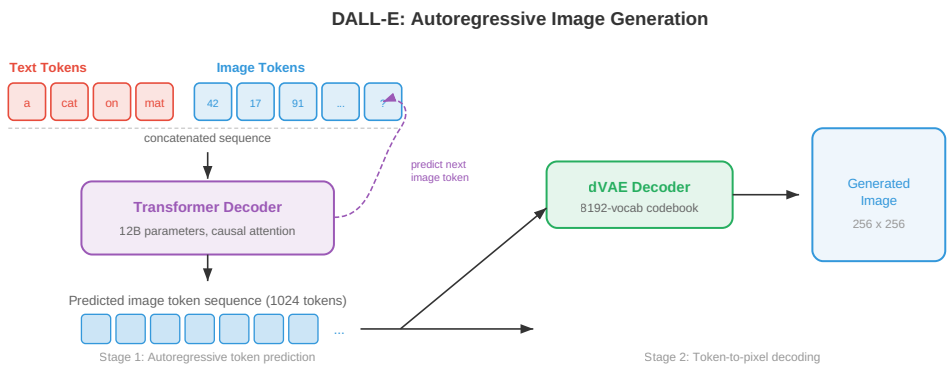
$$p(x_{\text{text}}, x_{\text{img}}) =$$

1280

$\prod_{i=1}$

$$p(x_i \mid x_1, \dots, x_{i-1})$$

- At generation time, you feed in the text tokens and the model autoregressively samples image tokens one by one. This is elegant because it reuses the exact machinery of language modelling — attention, causal masking, top-k sampling — for image synthesis.
- The downside is that autoregressive generation is inherently sequential: generating 1024 tokens one at a time is slow, and any error early in the sequence compounds. DALL-E mitigated this by generating many candidate images and re-ranking them with CLIP (from file 01) to find the best match to the text prompt.



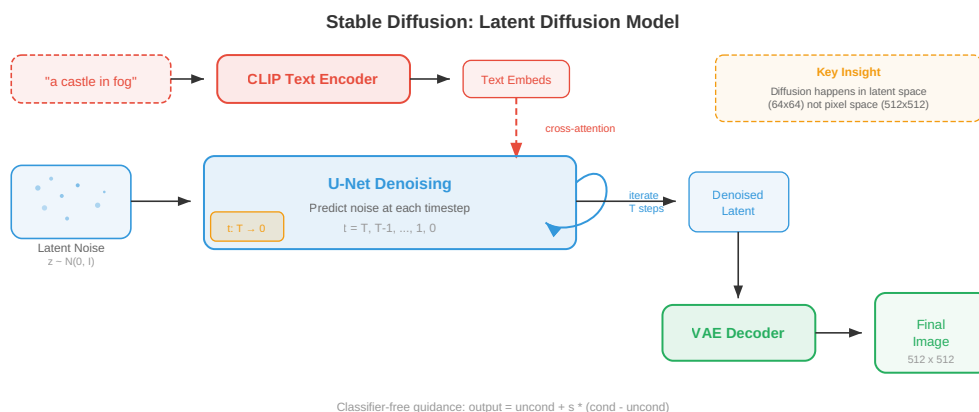
DALL-E pipeline: text tokens and image tokens concatenated into a single sequence, processed by a transformer decoder that autoregressively predicts image tokens conditioned on text tokens

### 49.1.2 Stable Diffusion: Latent Diffusion with Text Conditioning

- **Stable Diffusion** (Rombach et al., 2022) takes a fundamentally different approach. Instead of predicting tokens one by one, it starts with pure noise and gradually denoises it into an image, guided by a text prompt. Recall diffusion models from Chapter 8 — Stable Diffusion operates in a compressed latent space rather than pixel space, making it dramatically more efficient.
- The architecture has three components working in concert. A **VAE encoder** compresses the image from pixel space ( $512 \times 512 \times 3$ ) to a latent representation ( $64 \times 64 \times 4$ ), reducing dimensionality by a factor of 48. A **text encoder** (typically CLIP or OpenCLIP) converts the text prompt into a sequence of embedding vectors. A **U-Net denoiser** takes the noisy latent, the timestep, and the text embeddings, and predicts the noise to subtract at each step. Text conditioning enters the U-Net through **cross-attention** layers:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

- where  $Q$  comes from the noisy image features, and  $K, V$  come from the text embeddings. This lets the model attend to relevant words at each spatial location — when denoising the region where a “red ball” should appear, the model attends to the tokens “red” and “ball”.
- At inference, you sample  $z_T \sim \mathcal{N}(0, I)$  in latent space, iteratively denoise using the U-Net for  $T$  steps (typically 20-50 with DDIM scheduling), and decode the clean latent  $z_0$  back to pixel space with the VAE decoder. The entire forward pass generates a 512x512 image in seconds on a consumer GPU.



Stable Diffusion architecture: text prompt encoded by CLIP, random noise in latent space iteratively denoised by a U-Net with cross-attention to text embeddings, then decoded by VAE to produce the final image

### 49.1.3 Classifier-Free Guidance in Practice

- **Classifier-free guidance (CFG)** is the secret ingredient that makes text-to-image models produce images that actually match their prompts. Recall from Chapter 8 that CFG trains the model both conditionally and unconditionally, then amplifies the conditional signal at sampling time:

$\epsilon$

$$= \epsilon_{\theta}(x_t, \emptyset) + s \cdot (\epsilon_{\theta}(x_t, c) - \epsilon_{\theta}(x_t, \emptyset))$$

- where  $s$  is the guidance scale. Think of the term  $(\epsilon_{\theta}(x_t, c) - \epsilon_{\theta}(x_t, \emptyset))$  as the “direction toward the prompt” — it captures what makes a conditioned prediction different from an unconditioned one. Multiplying by  $s > 1$  exaggerates this direction, pushing the image closer to the text description at the cost of diversity.
- In practice,  $s = 7.5$  is a common default for Stable Diffusion. At  $s = 1.0$  you get the raw model output (diverse but loosely matching the prompt). At  $s = 20+$  the images become oversaturated and repetitive but very closely aligned with the text. The optimal  $s$  depends on the application: creative exploration favours lower guidance, while precise prompt adherence demands higher guidance.

### 49.1.4 Imagen: Cascaded Diffusion with Language Understanding

- **Imagen** (Saharia et al., 2022) demonstrated that a powerful text encoder matters more than a larger image model. Instead of CLIP, Imagen uses a frozen **T5-XXL** language model (from Chapter 07) as the text encoder, which has a

much richer understanding of language semantics, compositionality, and spatial relationships ( “a blue cube on top of a red sphere” ).

- Imagen uses a **cascaded diffusion** approach: a base diffusion model generates a 64x64 image, a first super-resolution model upscales to 256x256, and a second super-resolution model reaches 1024x1024. Each stage is a separate diffusion model conditioned on the text and (for the upscalers) the lower-resolution image. This cascade avoids modelling fine details at the base resolution, allowing the base model to focus on composition and semantics while the upscalers handle texture and sharpness.
- Imagen also introduced **dynamic thresholding**: at each denoising step, predicted pixel values are clipped to a percentile-based range rather than a fixed range  $[-1, 1]$ . This prevents saturation artefacts at high guidance scales, a common problem in diffusion models.

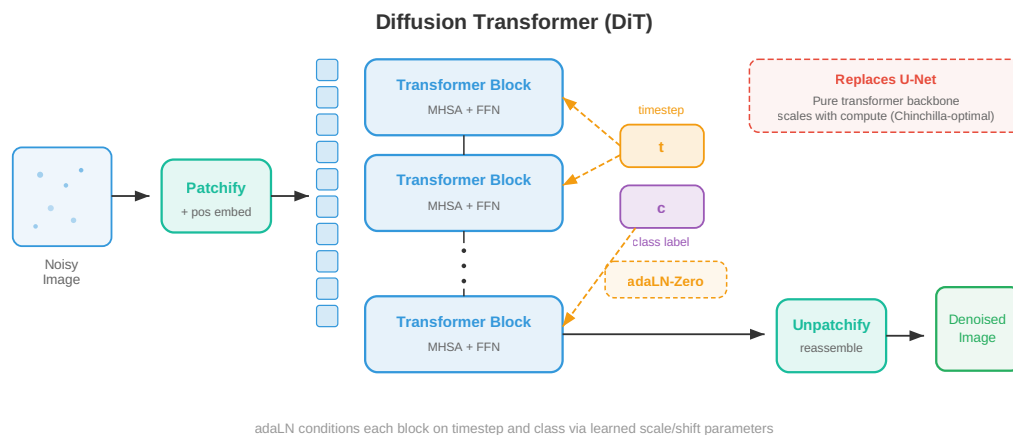
#### 49.1.5 Parti: Autoregressive at Scale

- **Parti** (Pathways Autoregressive Text-to-Image, Yu et al., 2022) revived the autoregressive approach with massive scale. Like DALL-E, it converts images to discrete tokens (using ViT-VQGAN) and generates them sequentially with a transformer. But Parti used a 20-billion-parameter encoder-decoder transformer (based on the Pathways architecture) and showed that autoregressive models can match diffusion quality when scaled sufficiently.
- Parti’ s encoder-decoder architecture is a key difference from DALL-E’ s decoder-only design. The text goes through the encoder; the decoder cross-attends to the encoded text while generating image tokens. This mirrors machine translation (Chapter 07) — you translate from “text language” to “image language” .

#### 49.1.6 DiT and Flow-Based Generation

- **Diffusion Transformers (DiT)** (Peebles and Xie, 2023) replace the U-Net backbone in diffusion models with a plain transformer. Each noisy latent patch is treated as a token (analogous to ViT from Chapter 8), and the transformer processes these tokens with self-attention and cross-attention to the text condition. DiT showed that transformers scale more predictably than U-Nets for diffusion — doubling compute reliably halves the FID score.

- **Flow matching** (recalled from Chapter 8) has emerged as an alternative to the diffusion noise-prediction paradigm. Instead of predicting noise  $\epsilon$  to subtract, the model predicts a velocity  $v_\theta(x_t, t)$  that transports samples along straight paths from noise to data. **Stable Diffusion 3** and **Flux** adopt flow matching with a **multimodal DiT (MM-DiT)** architecture, where text and image tokens are processed jointly by transformer blocks with bidirectional attention — both modalities attend to each other rather than text only conditioning image features via cross-attention.



DiT architecture: noisy latent patches tokenised like ViT, processed by transformer blocks with adaptive layer normalisation for timestep and class conditioning, then decoded back to spatial latent

## 49.2 Text-to-Video Generation

- Text-to-video is text-to-image with a ruthless additional constraint: **temporal coherence**. Every frame must be internally consistent (a valid image), but consecutive frames must also be smoothly connected — objects should move naturally, lighting should change continuously, and the “camera” should follow physically plausible trajectories. Think of the difference between painting a single landscape and directing a film.

### 49.2.1 Temporal Challenges

- Video introduces three challenges beyond image generation. **Temporal consistency** requires that objects maintain their identity across frames — a dog in frame 1 should still be the same dog in frame 100. **Motion modelling** requires learning physical dynamics: how objects move, how gravity works, how fluids flow. **Compute cost** is severe: a 10-second video at 24 fps and

512x512 resolution contains  $10 \times 24 \times 512 \times 512 \times 3 \approx 188$  million values, roughly 240 times more data than a single image.

#### 49.2.2 Make-A-Video and Extend-to-Video Approaches

- **Make-A-Video** (Singer et al., 2022) took a pragmatic approach: start with a pre-trained text-to-image model and add temporal layers. The key insight is that you already have strong text-image models trained on billions of image-text pairs, and you only need to learn motion from (unlabelled) video data.
- Make-A-Video inserts **temporal attention** and **temporal convolution** layers into a pre-trained spatial U-Net. The spatial layers (pre-trained on images) handle appearance, while the new temporal layers (trained on video) handle motion. Spatial self-attention operates within each frame; temporal attention operates across frames at each spatial location. This factorisation is efficient because temporal and spatial patterns are largely separable.
- The generation pipeline mirrors Imagen's cascade: a base model generates 16 frames at 64x64, then spatial and temporal super-resolution models upscale to the final resolution and frame rate. A frame interpolation network increases temporal smoothness.

#### 49.2.3 VideoPoet and Token-Based Video Models

- **VideoPoet** (Kondratyuk et al., 2024) unifies video generation under the language modelling paradigm. All modalities — text, image, video, audio — are tokenised into discrete sequences, and a single large language model (LLM) is trained to predict tokens autoregressively across all modalities. This enables zero-shot capabilities: text-to-video, image-to-video, video-to-audio, video editing, and inpainting all emerge from the same model.
- VideoPoet tokenises video with a MAGVIT-v2 encoder (a 3D VQ-VAE, from file 03) that compresses spatial and temporal dimensions jointly. Audio is tokenised with SoundStream. The LLM backbone is pre-trained on text and fine-tuned on multimodal token sequences, learning the joint distribution across modalities.

#### 49.2.4 Sora-Style Temporal Diffusion

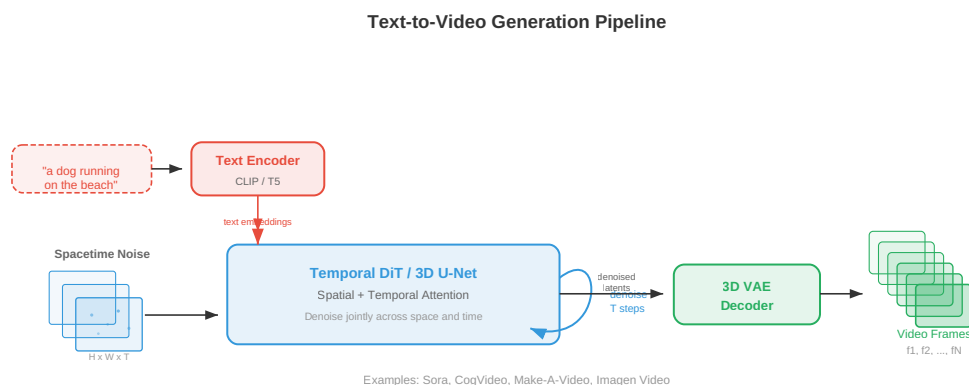
- **Sora** (OpenAI, 2024) brought temporal diffusion to mainstream attention with its ability to generate long, coherent, physically plausible videos. While full architectural details are not published, the key ideas involve scaling DiT to spacetime: video frames are decomposed into **spacetime patches** (3D chunks

across height, width, and time), which are treated as tokens for a large transformer.

- The spacetime patch approach means the model processes video as a native 3D signal rather than a sequence of 2D frames. This allows it to capture long-range temporal dependencies — the model can “plan ahead” across the entire video duration rather than generating frame by frame.
- Sora can handle variable durations, resolutions, and aspect ratios by adjusting the number of spacetime patches. Training on data at its native resolution (rather than cropping everything to squares) improves composition and framing quality.

#### 49.2.5 Wan: Open-Source Video Generation

- **Wan** (Wan et al., 2025) is a family of open-source video generation models (1.3B and 14B parameters) built on a DiT backbone with 3D VAE temporal compression. Wan uses **flow matching** rather than traditional DDPM-style diffusion, learning straight transport paths from noise to video latents. The 3D VAE compresses video spatially and temporally (4x temporal compression), and the DiT processes the resulting spacetime latent tokens with full 3D attention.
- Wan supports text-to-video, image-to-video (animating a still image), and video editing. The 14B model generates coherent videos up to 5 seconds at 720p resolution, demonstrating that open-source models can approach the quality of proprietary systems when architectural and training recipe choices are carefully made.



Text-to-video pipeline: text encoded by language model, spacetime noise denoised by temporal diffusion transformer attending to text embeddings, decoded by 3D VAE into video frames

## 49.3 Text-to-Audio Generation

---

- Picture a film composer reading a screenplay and scoring the soundtrack. Text-to-audio models do something analogous: given a text description ( “a thunderstorm with heavy rain and distant thunder” ), they generate the corresponding audio waveform. The challenge is bridging the gap between the discrete, symbolic nature of text and the continuous, temporal nature of sound.

### 49.3.1 AudioLM: Language Modelling for Audio

- **AudioLM** (Borsos et al., 2023) generates audio by predicting discrete audio tokens autoregressively, drawing on the same language modelling paradigm used by DALL-E for images. It uses a hierarchical token structure: **semantic tokens** (from a self-supervised model like w2v-BERT, recall Chapter 9) capture high-level content (what is being said or played), while **acoustic tokens** (from SoundStream, a neural audio codec) capture fine-grained acoustic details (how it sounds — timbre, recording quality).
- Generation proceeds in two stages. First, a transformer predicts semantic tokens given an optional audio prompt, establishing the high-level content plan. Second, another transformer predicts acoustic tokens conditioned on the semantic tokens, filling in acoustic details. This hierarchy mirrors the text-to-speech pipeline (Chapter 9) — semantic tokens play the role of phonemes, and acoustic tokens play the role of mel spectrogram frames.
- AudioLM can generate speech continuation (given 3 seconds of speech, generate the next 10), music continuation, and sound effects, all from a single model trained on audio-only data (no text labels needed for pre-training).

### 49.3.2 MusicLM: Text-Conditioned Music

- **MusicLM** (Agostinelli et al., 2023) extends AudioLM to text-conditioned music generation. It adds a text-audio joint embedding (from **MuLan**, a CLIP-like model trained on music-text pairs) to condition the generation. The MuLan embedding captures semantic meaning of the text description ( “upbeat jazz with saxophone solo” ) and guides the hierarchical token generation.
- MusicLM generates music at 24 kHz for arbitrary durations, maintaining melodic and rhythmic coherence over minutes-long pieces. It can also condition on a hummed melody (using melody tokens extracted by a pitch



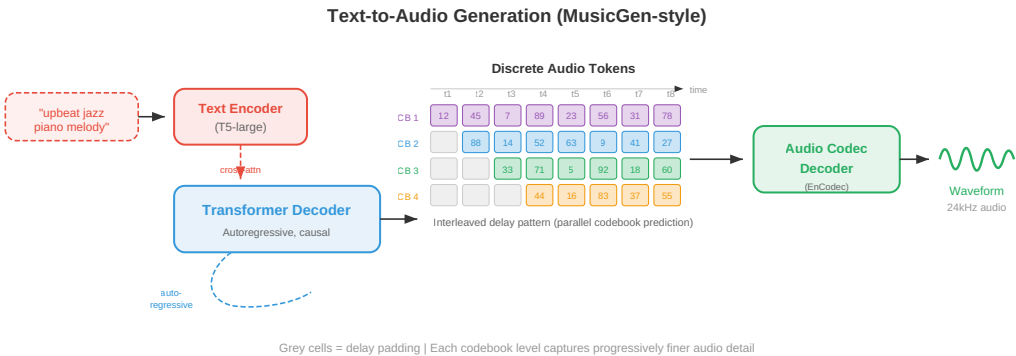
tracker) plus a text description, generating a full arrangement that follows the hummed tune in the style described by the text.

### 49.3.3 MusicGen: Efficient Single-Stage Generation

- **MusicGen** (Copet et al., 2023) simplifies the multi-stage approach. Instead of separate semantic and acoustic models, MusicGen uses a single autoregressive transformer that directly generates multiple codebook levels from the audio codec. The key innovation is an **interleaved codebook pattern**: rather than generating all codebook levels for one timestep before moving to the next, MusicGen interleaves tokens across codebooks and timesteps in a pattern that allows parallel decoding of some codebook levels.
- Conditioning is straightforward: text is encoded by a T5 encoder, and the text embeddings are prepended to the audio token sequence (like a prefix prompt in a language model) or injected via cross-attention. MusicGen also supports melody conditioning: a chromagram (from the spectrogram features discussed in Chapter 9) of a reference melody is encoded and used alongside the text condition.

$$p(a_1, \cdots, a_T) = \prod_{t=1}^T \prod_{k=1}^K p(a_{t,k} \mid a_{<t}, c_{\text{text}})$$

- where  $a_{t,k}$  is the audio token at timestep  $t$  and codebook level  $k$ , and  $c_{\text{text}}$  is the text conditioning. The product over  $k$  factorises depending on the codebook pattern — some levels are predicted in parallel.



Text-to-audio pipeline: text encoded by language model, transformer decoder generates discrete audio tokens across multiple codebook levels in an interleaved pattern, audio codec decoder reconstructs the waveform

## 49.4 Image-to-Text Generation

---

- Now flip the direction: given an image, generate a natural language description. This is **image captioning**, and it is a form of conditional text generation where the image is the condition. Think of a museum guide describing a painting — they must perceive the visual content, understand the relationships between objects, and articulate their observations in fluent language.

### 49.4.1 Captioning as Conditional Generation

- The classic approach uses an **encoder-decoder** architecture (Chapter 07). A pre-trained CNN or ViT (Chapter 8) encodes the image into a set of feature vectors. A language model decoder generates the caption word by word, attending to the image features at each step:

$$p(w_1, \dots, w_L \mid I) = \prod_{l=1}^L p(w_l \mid w_1, \dots, w_{l-1}, I)$$

- where  $w_l$  are the caption words and  $I$  is the image representation. Cross-attention connects the text decoder to the image features, allowing the model to “look at” different regions of the image as it generates different words — attending to the dog region when generating “dog” and the park region when generating “park” .
- **CoCa** (Contrastive Captioners, Yu et al., 2022) unified contrastive learning (file 01’ s CLIP-style objective) with captioning in a single model. The image encoder produces features used both for contrastive alignment with text and for cross-attention in a captioning decoder. This multi-task training gives CoCa strong zero-shot recognition (from contrastive learning) and strong generation (from captioning).

### 49.4.2 Modern Vision-Language Captioning

- Modern approaches often use **large multimodal models** (file 02) for captioning. Models like LLaVA, Qwen-VL, and GPT-4V treat captioning as a special case of visual question answering — the “question” is implicitly “describe this image” . The visual encoder (CLIP ViT or SigLIP) produces patch tokens that are projected into the LLM’ s embedding space, and the LLM generates a free-form description.
- The advantage of LLM-based captioning over dedicated encoder-decoder models is **instruction following**: you can ask for different levels of detail ( “describe in one sentence” vs. “provide a detailed paragraph” ), focus on specific aspects ( “describe the colours” ), or generate structured output ( “list all objects with their positions” ). This flexibility comes from the LLM’ s instruction-tuning (Chapter 07).

## 49.5 Video-Audio Co-Generation

---

- Think of watching a film with the sound off — the experience is hollow. Visual content and audio are deeply coupled: a bouncing ball has a rhythmic thud, rain produces a patter, and a crowd generates cheers. **Video-audio co-generation** aims to produce both modalities together, maintaining temporal alignment between what you see and what you hear.

### 49.5.1 Joint Temporal Modelling

- The core challenge is **temporal synchronisation**: the audio of a drum hit must coincide exactly with the visual frame showing the drumstick striking the drum. This requires a shared temporal representation that both modalities can reference.
- One approach is to generate video and audio from a shared latent timeline. Models like **CoDi** (Composable Diffusion, Tang et al., 2023) use separate diffusion models for each modality but align them through a shared latent space. During training, cross-modal attention layers learn to synchronise visual and audio features at each timestep. During generation, both diffusion processes run simultaneously, conditioning on each other through the shared alignment.

- VideoPoet (discussed above) takes a more unified approach: since all modalities are tokenised into a single sequence, the LLM naturally learns temporal correspondences between video and audio tokens. A video clip of a barking dog followed by the corresponding audio tokens teaches the model to associate visual barking motion with the sound of barking.
- **Temporal alignment loss** functions explicitly enforce synchronisation. One formulation uses contrastive learning at the frame level: the audio segment at time  $t$  should be more similar to the video frame at time  $t$  than to frames at other times:

$$\mathcal{L}_{\text{sync}} = -\mathbb{E}_t \left[ \log \frac{\exp(\text{sim}(v_t, a_t)/\tau)}{\sum_{t'} \exp(\text{sim}(v_t, a_{t'})/\tau)} \right]$$

- where  $v_t$  and  $a_t$  are the video and audio representations at time  $t$ , and  $\tau$  is a temperature parameter. This is structurally identical to the InfoNCE loss from file 01, but applied at the temporal frame level rather than at the clip level.

## 49.6 Instruction-Following Generation

---

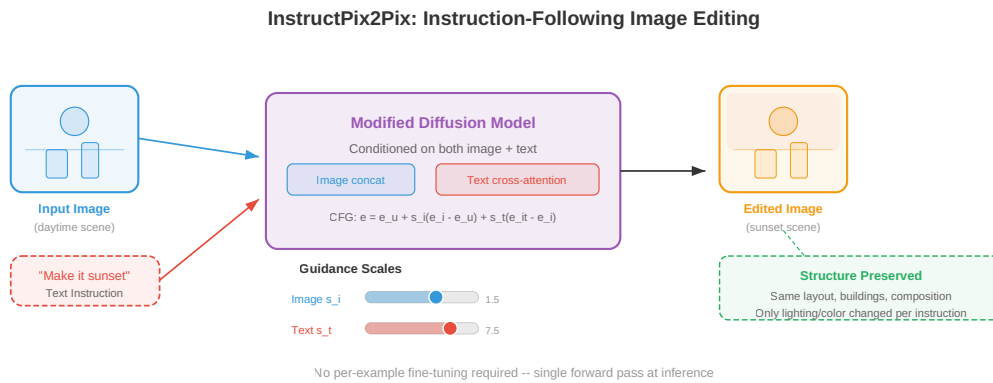
- Imagine telling an artist “make the sky more dramatic” or “replace the hat with a crown” . **Instruction-following generation** lets you edit images using natural language commands rather than precise spatial masks or brush strokes.

### 49.6.1 InstructPix2Pix: Editing by Description

- **InstructPix2Pix** (Brooks et al., 2023) trains a conditional diffusion model that takes an input image and a text instruction, then produces the edited image. The clever part is how the training data is created: GPT-3 generates editing instructions ( “make it winter” , “turn the cat into a dog” ) paired with input-output text captions, and a text-to-image model (Stable Diffusion) generates the corresponding image pairs.
- The model is a modified Stable Diffusion U-Net that receives both the text instruction (via cross-attention) and the input image latent (concatenated channel-wise with the noisy latent). It uses **dual classifier-free guidance** with two guidance scales — one for the text instruction ( $s_T$ ) and one for the input image ( $s_I$ ):

$$= \epsilon_{\theta}(x_t, \emptyset, \emptyset) + s_I \cdot (\epsilon_{\theta}(x_t, c_I, \emptyset) - \epsilon_{\theta}(x_t, \emptyset, \emptyset)) + s_T \cdot (\epsilon_{\theta}(x_t, c_I, c_T) - \epsilon_{\theta}(x_t, c_I, \emptyset))$$

- where  $c_I$  is the input image condition and  $c_T$  is the text instruction. The first guidance term controls how much to preserve the input image; the second controls how strongly to follow the instruction. This gives the user a two-dimensional knob: high  $s_I$  preserves the original closely, while high  $s_T$  makes more dramatic edits.



InstructPix2Pix: input image and text instruction feed into a modified diffusion model, producing an edited image that follows the instruction while preserving unedited regions

## 49.6.2 SDEdit and Noise-Based Editing

- **SDEdit** (Meng et al., 2022) offers a simpler editing approach that requires no special training. You take the input image, add noise to it (running the forward diffusion process to an intermediate timestep  $t_0$ ), then denoise with a text prompt describing the desired output. The amount of noise controls the edit strength: low noise preserves the structure (colour changes, style transfer), while high noise allows major restructuring (object replacement, layout changes).

- The tradeoff is precise: at timestep  $t_0$ , the noisy image retains

—  
 $\alpha$

$t_0$  fraction of the original signal. The denoising process fills in the corrupted details according to the new text prompt. This is mathematically grounded: the

diffusion model samples from the posterior  $p(x_0 \mid x_{t_0}, c)$ , where  $x_{t_0}$  constrains the generation to be “close to” the original.

### 49.6.3 ControlNet: Spatial Conditioning

- **ControlNet** (Zhang et al., 2023) adds fine-grained spatial control to text-to-image diffusion. A copy of the pre-trained U-Net encoder is trained to accept additional input conditions — edge maps (Canny edges), depth maps, pose skeletons, segmentation maps — while the original U-Net weights are frozen. The ControlNet encoder’s outputs are added to the frozen U-Net’s skip connections via **zero convolutions** (1x1 convolutions initialised to zero), ensuring training starts from the pre-trained model’s behaviour and gradually learns the new condition.
- This architecture lets you provide a sketch, a depth map, or a human pose as the structural guide, and the text prompt fills in the appearance. The pre-trained weights handle photorealism and text understanding; the ControlNet layers handle spatial fidelity to the condition.

## 49.7 Consistency and Alignment Metrics

---

- How do you measure whether a generated image is good? “Good” has at least two dimensions: **quality** (does it look like a real image?) and **alignment** (does it match the text prompt?). Several metrics have been developed to quantify these.

### 49.7.1 Frechet Inception Distance (FID)

- **Frechet Inception Distance (FID)** (Heusel et al., 2017) measures the distance between the distribution of generated images and real images in the feature space of a pre-trained Inception network. Think of it as comparing the “fingerprints” of two image collections rather than comparing individual images.
- Both the real and generated image sets are passed through Inception-v3, and the activations from the penultimate layer are collected. These activations are modelled as multivariate Gaussians  $\mathcal{N}(\mu_r, \Sigma_r)$  and  $\mathcal{N}(\mu_g, \Sigma_g)$ . The FID is the Frechet distance (Wasserstein-2 distance) between these Gaussians:

$$\text{FID} = \left\| \mu_r - \mu_g \right\|^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2})$$

- Lower FID is better. FID = 0 means the distributions are identical. FID captures both quality (if generated images are blurry, their features will differ from real images) and diversity (if the model suffers from mode collapse,  $\Sigma_g$  will be smaller than  $\Sigma_r$ ). Typical state-of-the-art values on ImageNet 256x256 are FID < 2.0.
- FID has known limitations: it assumes Gaussian feature distributions (which is approximate), it requires thousands of samples for stable estimates, and it uses Inception features (which may not capture all perceptually relevant differences).

### 49.7.2 Inception Score (IS)

- The **Inception Score (IS)** (Salimans et al., 2016) measures two properties: each generated image should be confidently classifiable (the conditional class distribution  $p(y \mid x)$  should be peaked), and the set of generated images should cover many classes (the marginal  $p(y) = \mathbb{E}_x[p(y \mid x)]$  should be uniform). IS combines these via the KL divergence:

$$\text{IS} = \exp(\mathbb{E}_x[\text{D}_{\text{KL}}(p(y \mid x) \parallel p(y))])$$

- Higher IS is better. The maximum IS equals the number of classes (1000 for ImageNet). IS rewards quality (sharp, recognisable images) and diversity (coverage of classes), but it has significant limitations: it ignores the real data distribution entirely, it cannot detect mode dropping within a class, and it is biased toward ImageNet-like images because it uses Inception's class predictions.

### 49.7.3 CLIPScore: Measuring Text-Image Alignment

- **CLIPScore** (Hessel et al., 2021) directly measures how well a generated image matches its text prompt using a pre-trained CLIP model (file 01). The score is simply the cosine similarity between the CLIP image embedding and the CLIP text embedding:

$$\text{CLIPScore}(I, T) = \max(0, \cos(E_I(I), E_T(T)))$$

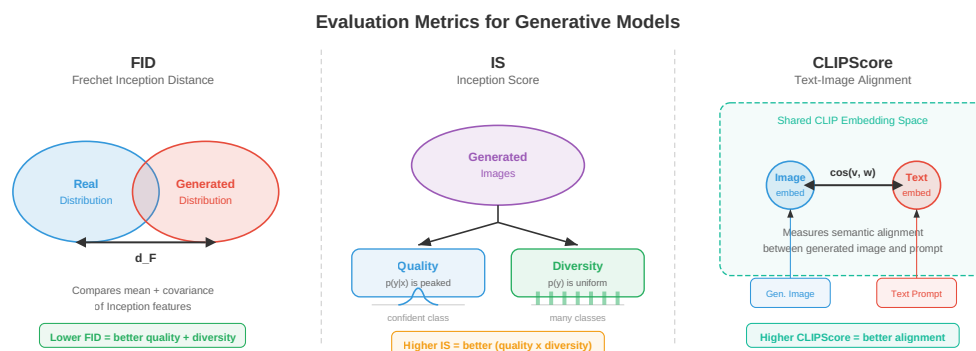
- where  $E_I$  and  $E_T$  are the CLIP image and text encoders. CLIPScore is reference-free — it does not require ground-truth images, only the text prompt. It

correlates well with human judgements of text-image alignment and has become the standard metric for evaluating prompt fidelity in text-to-image models.

- For comparing against a reference caption, **RefCLIPScore** incorporates a reference image:

$$\text{RefCLIPScore} = \text{HarmonicMean}(\text{CLIPScore}(I, T), \max(0, \cos(E_I(I), E_I(I_{\text{ref}}))))$$

- This balances text alignment with visual similarity to a reference.



Evaluation metrics: FID compares feature distributions of real and generated image sets, IS measures quality and diversity of generated images alone, CLIPScore measures cosine similarity between image and text embeddings

#### 49.7.4 Human Evaluation

- Automated metrics are proxies; human judgement remains the gold standard. Common protocols include **pairwise comparisons** (which of two images better matches the prompt?), **Likert scales** (rate quality and alignment from 1-5), and **Elo ratings** (tournament-style ranking across models). The DrawBench and PartiPrompts benchmarks provide standardised prompt sets for systematic human evaluation.

## 49.8 Ethical Considerations

- Cross-modal generation is one of the most ethically consequential areas in AI. The ability to create photorealistic images, videos, and audio from text descriptions raises profound concerns that practitioners must take seriously.



### 49.8.1 Deepfakes and Misinformation

- **Deepfakes** are generated or manipulated media designed to depict events that never occurred. Text-to-image and text-to-video models can create convincing fake photographs of public figures, fabricated evidence, and misleading news imagery. The danger is not just that fakes exist, but that their existence undermines trust in all media — if any image could be fake, no image is fully trusted.
- Detection methods include training classifiers on real vs. generated images, analysing statistical artefacts (GAN-generated images have subtle spectral signatures), and embedding invisible watermarks (Stable Diffusion's invisible watermark, Google's SynthID). However, detection is an arms race: as generators improve, detectors must be constantly updated.

### 49.8.2 Bias in Generation

- Models trained on internet-scale data inherit and amplify societal biases. Text-to-image models disproportionately generate lighter-skinned faces, associate certain professions with specific genders, and default to Western cultural norms for underspecified prompts. These biases are rooted in the training data distribution and the CLIP/T5 text encoders, which encode biases from their own training corpora.
- Mitigation strategies include curating more representative training data, applying debiasing techniques to text encoders, using safety classifiers to filter problematic outputs, and enabling user control over demographic attributes. None of these are complete solutions, and ongoing auditing is essential.

### 49.8.3 Content Filtering and Safety

- Responsible deployment requires multiple layers of protection. **Input filtering** blocks harmful prompts before generation. **Output filtering** classifies generated content and rejects harmful material. **NSFW classifiers** detect sexually explicit, violent, or otherwise harmful content. Stable Diffusion's safety checker, for instance, computes the cosine similarity between the generated image's CLIP embedding and a set of pre-defined harmful concept embeddings, flagging images that exceed a threshold.
- The open-source nature of many generation models (Stable Diffusion, Wan) creates tension between democratising access and preventing misuse. Once

model weights are released, content filtering can be bypassed. This has led to debates about the appropriate level of openness and the responsibilities of model developers.

#### 49.8.4 Intellectual Property and Consent

- Generative models trained on internet data may reproduce copyrighted styles, trademarks, or likeness of real people without consent. The legal and ethical frameworks are still evolving, but responsible practice includes respecting opt-out mechanisms, acknowledging the creative contributions embedded in training data, and developing technical safeguards against memorisation and regurgitation of training examples.

### 49.9 Coding Tasks (use CoLab or notebook)

---

1. Implement classifier-free guidance for a toy 2D diffusion model. Train a conditional diffusion model on a 2D dataset (e.g., labelled clusters), then sample with different guidance scales to observe the quality-diversity tradeoff.

```
import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

Toy 2D conditional diffusion with classifier-free guidance
def noise_schedule(T):
 betas = jnp.linspace(1e-4, 0.02, T)
 alphas = 1.0 - betas
 return jnp.cumprod(alphas)

def forward_diffuse(x0, t, alphaBars, key):
 noise = jax.random.normal(key, x0.shape)
 return jnp.sqrt(alphaBars[t]) * x0 + jnp.sqrt(1 - alphaBars[t]) * noise, noise

Generate labelled 2D data: class 0 = ring, class 1 = cluster
key = jax.random.PRNGKey(42)
k1, k2, k3 = jax.random.split(key, 3)
theta = jax.random.uniform(k1, (200,)) * 2 * jnp.pi
ring = jnp.stack([jnp.cos(theta), jnp.sin(theta)], axis=1) * 2
ring += jax.random.normal(k2, ring.shape) * 0.1
cluster = jax.random.normal(k3, (200, 2)) * 0.3

data = jnp.concatenate([ring, cluster])
```

```

labels = jnp.concatenate([jnp.zeros(200), jnp.ones(200)])

Simulate CFG: show how guidance pushes samples toward class-conditional modes
Try varying guidance_scale from 0.0 to 5.0 and observe results
guidance_scales = [0.0, 1.0, 3.0, 7.0]
fig, axes = plt.subplots(1, 4, figsize=(16, 4))
for ax, s in zip(axes, guidance_scales):
 ax.scatter(ring[:, 0], ring[:, 1], s=8, alpha=0.4, label='Ring (c=0)')
 ax.scatter(cluster[:, 0], cluster[:, 1], s=8, alpha=0.4, label='Cluster
(c=1)')

 ax.set_title(f'Guidance scale s={s}')
 ax.set_xlim(-4, 4); ax.set_ylim(-4, 4)
 ax.set_aspect('equal'); ax.legend(fontsize=7)
plt.suptitle('Experiment: vary guidance scale and observe quality vs diversity')
plt.tight_layout(); plt.show()

Exercise: train a small MLP denoiser with class conditioning,
then implement the CFG formula to sample with different s values.

```

1. Compute FID between two sets of 2D samples using the full Frechet distance formula. Vary the generated distribution and observe how FID changes.

```

import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

def compute_fid(real, generated):
 """Compute Frechet distance between two 2D sample sets."""
 mu_r, mu_g = jnp.mean(real, axis=0), jnp.mean(generated, axis=0)
 sigma_r = jnp.cov(real.T)
 sigma_g = jnp.cov(generated.T)
 diff = mu_r - mu_g
 # Matrix square root via eigendecomposition
 product = sigma_r @ sigma_g
 eigvals, eigvecs = jnp.linalg.eigh(product)
 sqrt_product = eigvecs @ jnp.diag(jnp.sqrt(jnp.maximum(eigvals, 0))) @
eigvecs.T

 fid = jnp.sum(diff ** 2) + jnp.trace(sigma_r + sigma_g - 2 * sqrt_product)
 return fid

key = jax.random.PRNGKey(0)
k1, k2, k3, k4 = jax.random.split(key, 4)

Real distribution: standard 2D Gaussian
real = jax.random.normal(k1, (1000, 2))

Generated distributions with increasing divergence

```

```

shifts = [0.0, 0.5, 1.0, 2.0, 4.0]
fig, axes = plt.subplots(1, len(shifts), figsize=(18, 3.5))
for ax, shift in zip(axes, shifts):
 gen = jax.random.normal(k2, (1000, 2)) * (1 + shift * 0.2) + shift
 fid = compute_fid(real, gen)
 ax.scatter(real[:, 0], real[:, 1], s=3, alpha=0.3, label='Real')
 ax.scatter(gen[:, 0], gen[:, 1], s=3, alpha=0.3, label='Generated')
 ax.set_title(f'Shift={shift}\nFID={fid:.2f}')
 ax.set_xlim(-5, 8); ax.set_ylim(-5, 8)
 ax.set_aspect('equal'); ax.legend(fontsize=7)
plt.suptitle('FID increases as generated distribution diverges from real')
plt.tight_layout(); plt.show()
Try: change the variance of generated samples without shifting the mean.
How does FID respond to a diversity mismatch vs a location mismatch?

```

1. Implement CLIPScore computation between text and image embeddings using random projections as a stand-in for CLIP. Observe how cosine similarity behaves as you vary the “alignment” between modalities.

```

import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

def cosine_similarity(a, b):
 return jnp.dot(a, b) / (jnp.linalg.norm(a) * jnp.linalg.norm(b))

def clip_score(img_emb, txt_emb):
 """CLIPScore: clamped cosine similarity."""
 return jnp.maximum(0.0, cosine_similarity(img_emb, txt_emb))

key = jax.random.PRNGKey(42)
dim = 512 # CLIP embedding dimension

Simulate aligned and misaligned pairs
Aligned: image and text embeddings share a component
k1, k2, k3 = jax.random.split(key, 3)
shared = jax.random.normal(k1, (dim,))
shared = shared / jnp.linalg.norm(shared)

noise_levels = jnp.linspace(0, 5, 20)
scores = []
for noise in noise_levels:
 noise_vec = jax.random.normal(k2, (dim,)) * noise
 img_emb = shared + noise_vec * 0.3
 txt_emb = shared + jax.random.normal(k3, (dim,)) * noise * 0.3
 scores.append(float(clip_score(img_emb, txt_emb)))

```

```
plt.figure(figsize=(8, 4))
plt.plot(noise_levels, scores, 'o-', color='#2c3e50')
plt.xlabel('Noise level (misalignment)')
plt.ylabel('CLIPScore')
plt.title('CLIPScore decreases as text-image alignment degrades')
plt.grid(True, alpha=0.3)
plt.tight_layout(); plt.show()
Experiment: what happens if you normalise embeddings before adding noise?
How does dimensionality affect the score distribution?
```

## 50 Unified Multimodal Architectures

Unified multimodal architectures replace separate specialist models with a single system that reads, reasons, and generates across text, images, audio, and video. This file covers any-to-any models (CoDi, NExT-GPT), natively multimodal LLMs (Gemini, GPT-4o), multimodal tokenisation strategies, and the architectural trade-offs of unification.

### 50.1 The Case for Unification

---

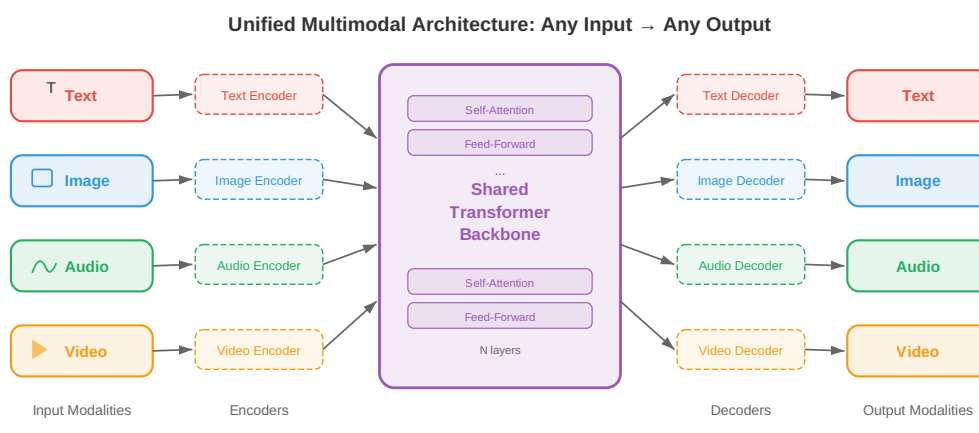
- Imagine a translator who speaks five languages and can switch between them mid-sentence without pausing. Early multimodal systems were more like five separate translators sitting in different rooms, each handling one language and passing notes through a slot in the wall. A **unified multimodal architecture** is the single polyglot: one model with shared weights that reads, writes, and reasons across text, images, audio, video, and even actions, all within a single forward pass.
- The motivation is both practical and theoretical. On the practical side, maintaining separate specialist models for every modality pair (text-to-image, image-to-text, audio-to-text, etc.) leads to a combinatorial explosion:  $k$  modalities require up to  $k(k-1)$  directed pipelines. A unified model collapses all of these into a single system. On the theoretical side, human cognition does not process vision and language in isolated modules; cross-modal binding happens early and deeply, and unification attempts to mirror this.
- Shared weights encourage **transfer across modalities**. A transformer that has learned temporal patterns in text (subject before verb, cause before effect) can

repurpose those same attention circuits for temporal patterns in video (object appears before it moves) or audio (onset before sustain). This is the multimodal analogue of the transfer learning you saw in Chapter 7 with language model fine-tuning and in Chapter 8 with ImageNet pretraining.

- Formally, let  $\mathcal{M} = \{m_1, m_2, \dots, m_k\}$  be a set of modalities. A unified model defines a single parameterised function  $f_\theta$  that maps any subset of input modalities to any subset of output modalities:

$$f_\theta : \mathcal{P}(\mathcal{M}) \rightarrow \mathcal{P}(\mathcal{M})$$

- where  $\mathcal{P}(\mathcal{M})$  is the power set (all subsets) of modalities. The key constraint is that  $\theta$  is largely shared; only thin, modality-specific adapter layers differ.



High-level diagram showing multiple modalities (text, image, audio, video) feeding into a single shared transformer backbone and producing outputs in any modality

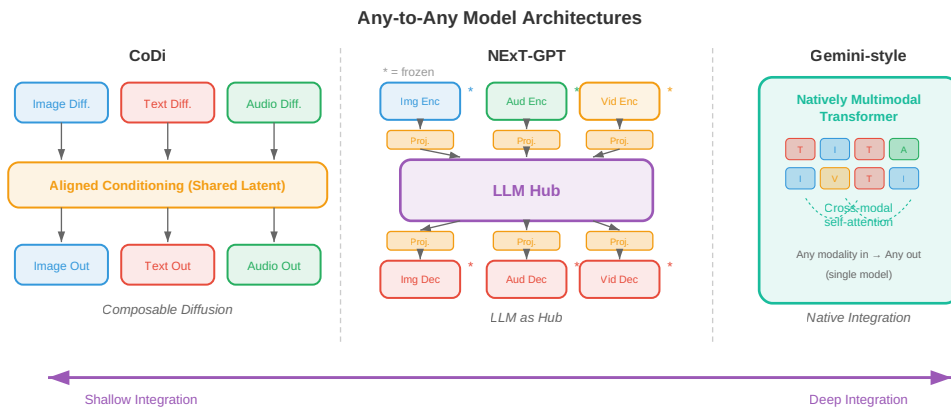
- The promise of unification comes with a fundamental tension: modalities are structurally different. Text is a 1D sequence of discrete tokens. Images are 2D grids of continuous pixel values. Audio is a 1D continuous waveform with a very different temporal scale from text. Video adds a time axis to images. Reconciling these disparate structures into a single sequence that a transformer can digest is the central engineering challenge of this field.

## 50.2 Any-to-Any Models

- Think of a universal remote control that can operate your television, air conditioning, and music system, all through the same interface. **Any-to-any**

**models** are the AI equivalent: they accept any combination of modalities as input and produce any combination as output.

- **CoDi** (Composable Diffusion) achieves any-to-any generation by training modality-specific diffusion models and then aligning their latent spaces through a shared conditioning mechanism. Each modality has its own diffusion process (recall diffusion models from file 04 in this chapter), but the noise prediction networks are conditioned on a joint cross-attention layer that sees embeddings from all input modalities simultaneously. This lets CoDi generate, say, an image and matching audio from a text prompt in a single pass.
- **NExT-GPT** takes a different architectural approach. It connects an LLM backbone (the “brain” ) to modality-specific encoders on the input side and modality-specific decoders on the output side via lightweight **projection layers**. The input encoders (e.g., an image encoder from CLIP, an audio encoder from CLAP) translate each modality into the LLM’s embedding space. The LLM reasons over the combined token sequence and emits special “modality signal tokens” that route information to the appropriate decoder (e.g., Stable Diffusion for images, AudioLDM for audio). Only the projection layers are trained; the LLM and the specialist encoders/decoders are kept frozen.
- **Gemini** (Google DeepMind) is natively multimodal from pretraining. Unlike NExT-GPT’s plug-and-play approach, Gemini’s transformer is trained from scratch on interleaved sequences of text, image, audio, and video tokens. This means cross-modal attention patterns develop organically during pretraining rather than being bolted on afterwards. The model uses the SentencePiece tokeniser for text and learns a visual tokeniser similar to the VQ approaches discussed in file 03 of this chapter.
- **GPT-4o** ( “o” for “omni” ) represents yet another pattern: an end-to-end model where all modalities share the same transformer and the same next-token prediction objective. Audio input is processed as spectral tokens, images as patch tokens, and text as subword tokens, all fed into a single sequence. The model generates output tokens that are decoded by modality-specific heads. The key innovation is the low latency enabled by removing the cascade of separate ASR, LLM, and TTS models that earlier systems like GPT-4V relied on.



Comparison of architectural patterns for CoDi (aligned diffusion), NEXT-GPT (LLM hub with frozen specialists), and Gemini-style (natively interleaved pretraining)

- These models sit on a spectrum of integration depth:
  - **Shallow integration** (NEXT-GPT): frozen specialists connected by trained adapters. Fast to build, limited cross-modal reasoning.
  - **Medium integration** (CoDi): shared conditioning across modality-specific generators. Better alignment, still modular.
  - **Deep integration** (Gemini, GPT-4o): single model trained end-to-end on all modalities. Richest cross-modal reasoning, most expensive to train.

## 50.3 Modality-Specific Encoders and Decoders with a Shared Backbone

- Picture a factory with a single assembly line (the shared backbone) but different loading docks for raw materials (encoders) and different shipping departments for finished goods (decoders). Each dock is specialised for its cargo, but once inside the factory, everything moves along the same conveyor belt.
- The dominant architectural pattern for unified models uses this three-part structure:
  - **Modality encoders**  $E_m$  that convert raw input from modality  $m$  into a sequence of embedding vectors  $\mathbf{h}_1^m, \mathbf{h}_2^m, \dots, \mathbf{h}_{n_m}^m$ , each of dimension  $d$ .
  - A **shared transformer backbone**  $T_\theta$  that processes the concatenated or interleaved embeddings from all input modalities using self-attention.



- **Modality decoders**  $D_m$  that convert the backbone's output embeddings back into the native format of modality  $m$  (text tokens, image pixels, audio waveforms).
- For text, the encoder is typically an embedding lookup table  $E_{\text{text}}(w) = \mathbf{W}_e[w]$  where  $w$  is a token index, identical to what you saw in Chapter 7 with transformers. For images, the encoder is often a **Vision Transformer** (ViT) that splits the image into patches and projects each patch linearly, as covered in Chapter 8. For audio, the encoder computes a mel spectrogram and processes it with either a convolutional frontend or an Audio Spectrogram Transformer (AST), as discussed in Chapter 9.
- The shared backbone is a standard transformer with self-attention across all modality tokens. Given a concatenated input sequence  $\mathbf{H} = [\mathbf{h}_1^{m_1}, \dots, \mathbf{h}_{n_1}^{m_1}, \mathbf{h}_1^{m_2}, \dots, \mathbf{h}_{n_2}^{m_2}]$ , the self-attention allows every token to attend to every other token regardless of modality:

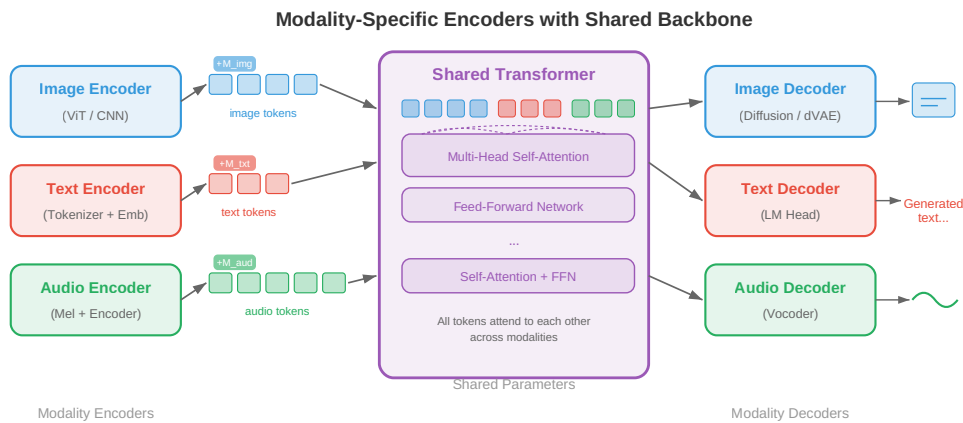
$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

- This is the same attention formula from Chapter 7, but now  $\mathbf{Q}$ ,  $\mathbf{K}$ , and  $\mathbf{V}$  contain tokens from multiple modalities. An image-patch token can attend to a text token, enabling cross-modal reasoning without any separate cross-attention module.
- **Modality embeddings** are added to each token so the backbone knows which modality a token comes from. This is analogous to positional embeddings but encodes modality identity instead of sequence position. A learnable vector  $\mathbf{e}_m \in \mathbb{R}^d$  is added to every token from modality  $m$ :

⦿  
**h**

$$\mathbf{h}_i^m = \mathbf{h}_i^m + \mathbf{e}_m + \mathbf{p}_i$$

- where  $\mathbf{p}_i$  is the positional embedding for position  $i$ .



Encoder-backbone-decoder architecture showing image patches, text tokens, and audio frames all entering a shared transformer, with modality-specific decoders on the output side

## 50.4 Multimodal Tokenisation

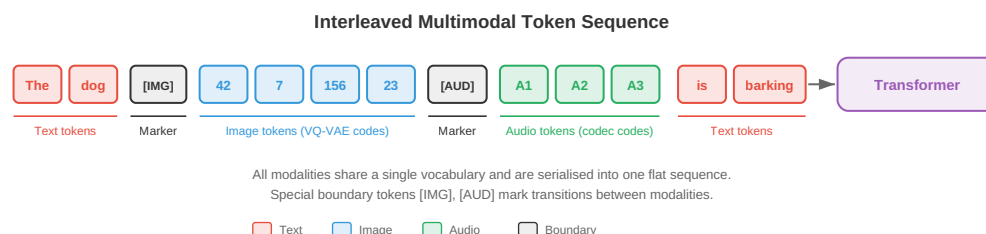
- Imagine you are writing a letter that includes both English text and hand-drawn sketches. You might write a sentence, sketch a diagram, write another sentence referring to the diagram, then paste in a musical score. The letter is a single linear stream that interleaves different “modalities.” Multimodal tokenisation does precisely this: it converts text, images, audio, and video into a single flat sequence of tokens that a transformer processes left-to-right.
- For text, tokenisation is well established: **byte-pair encoding** (BPE) or SentencePiece produce a vocabulary of subword tokens, as covered in Chapter 7. The challenge is extending this idea to continuous modalities.
- For images, there are two broad approaches. The **discrete** approach uses a VQ-VAE or VQ-GAN (detailed in file 03 of this chapter) to map each image to a sequence of codebook indices. If the codebook has  $|\mathcal{C}|$  entries and an image is encoded as  $n$  codes, the image becomes  $n$  discrete tokens drawn from a vocabulary of size  $|\mathcal{C}|$ , directly compatible with a text vocabulary. The **continuous** approach uses a ViT or CNN encoder to produce  $n$  continuous embedding vectors, which are linearly projected into the transformer’s embedding dimension. Gemini and GPT-4o use variants of the continuous approach; autoregressive image generators like Parti and LlamaGen prefer the discrete route.
- For audio, the signal is typically converted to a mel spectrogram and then either discretised with a neural audio codec (e.g., EnCodec, SoundStream, which

produce hierarchical discrete tokens) or projected continuously via a learned encoder. AudioLM, for example, represents audio as a sequence of discrete tokens from multiple codebook levels, then models them autoregressively.

- For video, tokenisation builds on image tokenisation but must also compress the temporal dimension. A common strategy uses a **3D VQ-VAE** (as in VideoGPT or Cosmos Tokeniser from file 03) that quantises spatiotemporal patches into discrete tokens. The temporal compression factor is crucial: raw video at 24 fps produces far too many tokens per second without aggressive temporal downsampling.
- Once all modalities are tokenised, they are **interleaved** into a single sequence with special delimiter tokens marking modality boundaries. A typical format looks like:

```
[TEXT] The cat sits on a mat [/TEXT] [IMAGE] <img_tok_1> <img_tok_2> ...
<img_tok_n> [/IMAGE] [AUDIO] <aud_tok_1> ... <aud_tok_m> [/AUDIO]
```

- The transformer then processes this entire mixed sequence using its standard causal (or bidirectional) attention mechanism. The modality delimiter tokens serve double duty: they inform the model about modality boundaries and act as “pooling points” whose representations summarise each modality segment.



Interleaved token sequence showing text tokens, discrete image tokens, and audio codec tokens flowing through a single transformer with modality boundary markers

- A critical design choice is the **token budget**. A single image tokenised at 256 tokens and a text caption of 50 tokens means the image consumes 5x more of the context window. Models must balance resolution (more tokens = more detail) against context length (more tokens = higher memory and compute cost). Techniques like **token merging** (progressively combining similar tokens) and **adaptive tokenisation** (using fewer tokens for simple regions and more for complex ones) help manage this trade-off.

## 50.5 Training Recipes: Staged Pretraining and Joint Fine-Tuning

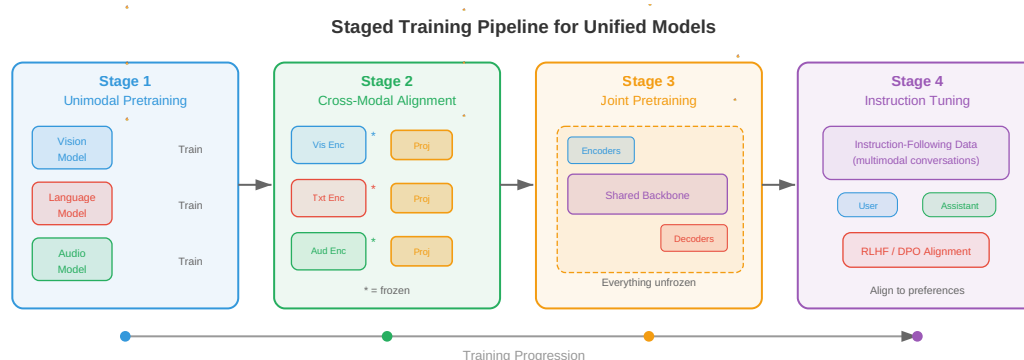
---

- You would not teach a child calculus before arithmetic. Similarly, you cannot train a unified multimodal model on all modalities simultaneously from random initialisation and expect it to converge well. The dominant approach is **staged training**, where the model learns progressively more complex cross-modal capabilities in carefully ordered phases.
- **Stage 1: Unimodal pretraining.** Each modality encoder is trained independently on large unimodal datasets. The text backbone is pretrained with a standard language modelling objective (next-token prediction) on trillions of text tokens, exactly as in Chapter 7. The vision encoder is pretrained on image classification or self-supervised objectives (MAE, DINO) as in Chapter 8. The audio encoder is pretrained on speech recognition or audio classification data as in Chapter 9. This stage produces strong unimodal feature extractors.
- **Stage 2: Cross-modal alignment.** The pretrained encoders are connected to the shared backbone, and the model is trained on paired multimodal data (image-caption pairs, audio-transcript pairs) with a contrastive or generative objective. During this stage, the encoder weights may be frozen (to preserve unimodal knowledge) while only the projection layers and backbone are updated. This is the stage where CLIP-style alignment (from file 01 in this chapter) gets folded into the unified model.
- **Stage 3: Joint multimodal pretraining.** All parameters (or most of them) are unfrozen, and the model is trained on a mixture of unimodal and multimodal data with a single next-token prediction objective across all modality tokens. The loss function is:

$$\mathcal{L} = - \sum_{t=1}^T \log p_{\theta}(x_t \mid x_{<t})$$

- where  $x_t$  can be a text token, an image token, or an audio token. The model must learn to predict the next token regardless of modality, which forces it to develop genuine cross-modal understanding.

- **Stage 4: Instruction tuning and alignment.** The pretrained model is fine-tuned on curated instruction-following datasets that include multimodal instructions (e.g., “Describe this image in detail” , “What sound does this video make?” , “Generate an image of X” ). This stage often uses **reinforcement learning from human feedback** (RLHF) or direct preference optimisation (DPO) to align the model’s outputs with human preferences.
- **Modality-specific warm-up** is a technique used within stages to prevent modality collapse. If one modality (typically text, which has the most training data) dominates the gradient signal, the model may “forget” weaker modalities. Warm-up strategies include:
  - **Gradient balancing:** scaling gradients from each modality so they contribute equally to the parameter update.
  - **Data ratio scheduling:** gradually increasing the proportion of multimodal data relative to unimodal data.
  - **Loss weighting:** assigning modality-specific weights  $\lambda_m$  so the total loss is  $\mathcal{L} = \sum_m \lambda_m \mathcal{L}_m$ , with  $\lambda_m$  tuned to balance learning rates across modalities.



Four-stage training pipeline diagram showing unimodal pretraining, cross-modal alignment, joint multimodal pretraining, and instruction tuning, with arrows indicating which parameters are frozen or trainable at each stage

- **Why not skip stages?** Training everything jointly from scratch is tempting but fails in practice for several reasons. First, the model must simultaneously learn low-level features (edge detection, phoneme recognition) and high-level cross-modal reasoning, which have very different learning dynamics. Second, the data distributions across modalities are wildly imbalanced (trillions of text tokens versus billions of image tokens versus hundreds of millions of audio clips). Third, the optimisation landscape is highly non-convex, and staged

training provides a curriculum that guides the model towards a better basin, similar to the curriculum learning idea from Chapter 6.

## 50.6 Multimodal Chain-of-Thought Reasoning

- When you solve a geometry problem, you might sketch a diagram, label the angles, write out an equation, and then solve it step by step. You do not jump directly from the problem statement to the answer. **Multimodal chain-of-thought** (CoT) reasoning enables models to do the same: generating intermediate reasoning steps that may involve text, visual annotations, or even generated diagrams before arriving at a final answer.
- In text-only CoT (as explored in Chapter 7’s discussion of prompting strategies), the model generates a sequence of reasoning steps in natural language. Multimodal CoT extends this by allowing the intermediate steps to reference or generate visual content. For example, given a chart image and the question “Which year had the highest sales?”, a multimodal CoT model might first describe the chart ( “The chart shows sales from 2018 to 2023...” ), then identify the relevant visual features ( “The tallest bar appears at 2021...” ), and finally output the answer ( “2021” ).
- Formally, let  $\mathbf{x}$  be a multimodal input and  $y$  be the target answer. Standard prediction models  $p(y \mid \mathbf{x})$  directly. Chain-of-thought introduces intermediate reasoning  $\mathbf{r} = (r_1, r_2, \dots, r_L)$  and factorises the prediction as:

$$p(y \mid \mathbf{x}) = \sum_{\mathbf{r}} p(y \mid \mathbf{r}, \mathbf{x}) \cdot p(\mathbf{r} \mid \mathbf{x})$$

- In practice, the sum is approximated by greedy or beam-search decoding over reasoning chains. The reasoning steps  $r_i$  can be text tokens, references to image regions, or even generated visual tokens (e.g., a bounding box annotation overlaid on the input image).
- **Training multimodal CoT** typically involves curating datasets where human annotators provide step-by-step multimodal reasoning traces, then fine-tuning the model on these traces. Some approaches distill CoT capabilities from larger teacher models: the teacher generates reasoning traces for a large dataset, and

the smaller student model is trained on both the inputs and the teacher' s traces.

- Multimodal CoT is especially powerful for tasks that require **spatial reasoning** (e.g., “Is the red ball to the left of the blue cube?” ), **mathematical reasoning over diagrams** (e.g., geometry problems), and **multi-step visual question answering** where the answer depends on combining information from multiple regions of an image.

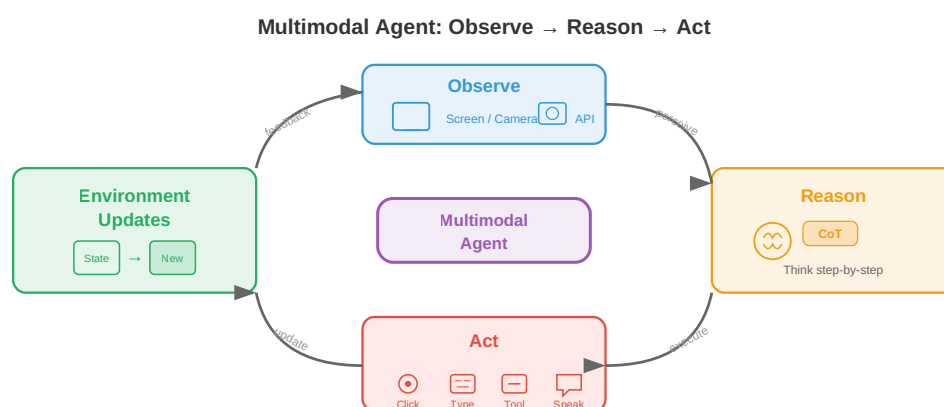
## 50.7 Multimodal Agents

---

- Think of a robot chef in a kitchen. It looks at the ingredients on the counter (vision), reads the recipe on a tablet (text), listens for the timer beeping (audio), and then physically picks up a knife and chops an onion (action). A **multimodal agent** is the digital version of this: a model that perceives the world through multiple modalities, reasons about what to do, and takes actions grounded in its perception.
- The agent loop follows the classic **observe-reason-act** cycle:
  1. **Observe:** The agent receives multimodal input from its environment (a screenshot, a user' s spoken instruction, a video feed).
  2. **Reason:** The unified model processes the multimodal input, possibly using chain-of-thought to plan a sequence of steps.
  3. **Act:** The model outputs an action (a text response, a tool call, a mouse click at coordinates (x , y), a robotic motor command).
- **Tool use** is a key capability of multimodal agents. The model is trained to recognise when it cannot answer a question directly and must instead invoke an external tool: a calculator, a code interpreter, a web browser, or a search engine. The model generates a structured tool call (e.g., `search("current weather in London")`) as part of its output token sequence, the system executes the call, and the result is fed back as additional input tokens for the model to process.
- **Visual grounding** connects language to specific regions in an image or video. When an agent says “click the blue button in the top-right corner,” it must ground the phrase “blue button in the top-right corner” to pixel coordinates. Architecturally, this is achieved by training the model to output bounding box coordinates as special tokens or by having the model produce a heatmap over

the image that indicates the referred region. This extends the grounding and referring work discussed in file 02 of this chapter (Vision Language Models) to the action domain.

- **Web agents** like WebVoyager and SeeAct demonstrate multimodal agents navigating websites. The agent receives a screenshot of a web page, identifies interactive elements (buttons, text fields, links), and outputs actions (click, type, scroll) to accomplish a user-specified goal. The key challenge is the enormous action space: a typical web page has hundreds of possible click targets.



Observe-reason-act loop of a multimodal agent, showing visual input from a screen, the reasoning process inside the unified model, and output actions like clicking, typing, or calling tools

- **Embodied agents** extend this to physical environments. A robot with a camera and microphone receives visual and audio input, processes it through a unified model, and outputs motor commands. Projects like PaLM-E (Google) embed robotic sensor data directly into the token sequence of a language model, allowing the robot to follow instructions like “pick up the green block near the bowl” by grounding the instruction in its visual observation and generating a sequence of motor actions.
- The training recipe for agents adds a **reinforcement learning** (RL) stage on top of the standard staged pretraining. The agent interacts with an environment (a simulated desktop, a web browser, a robotic simulator), receives rewards for task completion, and updates its policy using algorithms like PPO or REINFORCE. The reward signal is typically sparse (1 for task success, 0 otherwise), making this optimisation challenging and heavily reliant on the strong priors from multimodal pretraining.



## 50.8 Benchmarks and Evaluation

---

- Evaluating a model that can see, hear, read, and act requires a diverse suite of benchmarks. No single metric captures multimodal competence, so the field relies on a collection of specialised evaluations.
- **MMLU** (Massive Multitask Language Understanding) tests knowledge across 57 academic subjects. While originally text-only, it serves as a baseline: a unified multimodal model should not lose text-only performance when it gains visual capabilities. A drop in MMLU after multimodal training signals catastrophic forgetting.
- **MMBench** evaluates vision-language understanding across 20 fine-grained ability dimensions, including attribute recognition, spatial relationship understanding, and OCR. Each question presents an image and a multiple-choice question. The benchmark systematically tests whether the model truly understands the image or is relying on text-only shortcuts.
- **SEED-Bench** provides 19,000 multiple-choice questions spanning 12 evaluation dimensions for both image and video understanding. It specifically tests temporal understanding (what happened before/after a given frame) and compositional reasoning (combining multiple visual attributes).
- **MM-Vet** evaluates integrated multimodal capabilities by requiring models to use multiple skills simultaneously: recognition, OCR, spatial awareness, language generation, and knowledge retrieval, all in a single question.
- **MathVista** tests mathematical reasoning over visual inputs: geometry diagrams, statistical charts, function plots, and scientific figures. This benchmark specifically targets multimodal chain-of-thought capabilities.
- **Audio-visual benchmarks** like AVQA (Audio-Visual Question Answering) test whether models can reason about the relationship between what they see and what they hear. For example: “Is the person speaking the one on the left or the right?”
- **Agent benchmarks** like WebArena, OSWorld, and SWE-bench evaluate task completion in interactive environments. The metric is typically the success rate: what fraction of tasks does the agent complete correctly? These benchmarks are particularly challenging because they require long-horizon planning and error recovery.

- **Holistic evaluation** frameworks like LMSYS Chatbot Arena use human preference judgements in a head-to-head format. Two models are shown the same multimodal input, and a human judge selects which response is better. Elo ratings are computed from thousands of such comparisons, providing a single scalar that correlates well with overall model quality.
- A persistent challenge in multimodal evaluation is **data contamination**: because these models are trained on internet-scale data, benchmark images and questions may appear in the training set. Careful deduplication and the creation of held-out test sets are essential but imperfect safeguards.

## 50.9 World Models

---

- Imagine closing your eyes and visualising what will happen if you push a glass off the edge of a table. You “see” it fall, “hear” the shatter, and “feel” that it would be a bad idea. Your brain is running a **world model**: an internal simulation of the physical and causal structure of the environment that can predict future states across multiple modalities.
- In the AI context, a world model is a learned function that predicts the next state of the world given the current state and an action:

↺

s

$$s_{t+1} = g_{\phi}(s_t, a_t)$$

- where  $s_t$  is the current state representation (which may include visual, auditory, and proprioceptive information),  $a_t$  is an action, and

↺

s

$s_{t+1}$  is the predicted next state. The state  $s_t$  lives in a learned latent space rather than raw pixel space, making the prediction problem tractable.

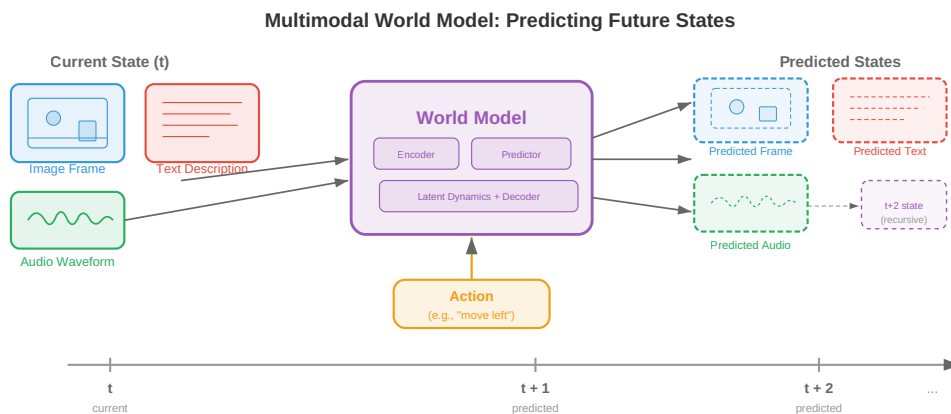
- **Video prediction models** like Sora (OpenAI) and Genie (Google DeepMind) represent a major step towards world models. They learn to generate temporally coherent video frames conditioned on text prompts and/or action sequences. While they are often discussed as video generators, the underlying capability is closer to world simulation: the model has internalised enough

physics (gravity, collision, occlusion, fluid dynamics) to render plausible futures.

- The connection to multimodal architectures is deep. A world model that predicts only pixels is limited; a truly useful world model predicts across modalities. If you push the glass, the world model should predict the visual trajectory (glass falls), the auditory event (glass shatters), and the semantic consequence (you now have broken glass on the floor). Unified multimodal architectures are natural candidates for world models because they already represent all modalities in a shared space.
- Formally, a multimodal world model optimises:

$$\mathcal{L}_{\text{world}} = \mathbb{E} \left[ \sum_{m \in \mathcal{M}} \lambda_m \| s_{t+1}^m - g_{\phi}^m(s_t, a_t) \|^2 \right]$$

- where  $s_{t+1}^m$  is the ground-truth next-state representation in modality  $m$  and  $g_{\phi}^m$  is the modality-specific prediction head of the world model. The shared latent dynamics  $g_{\phi}$  operate in the joint multimodal space, while modality-specific heads decode predictions into each modality's native format.



World model diagram showing a latent state being updated by an action, with decoder heads predicting future visual frames, audio waveforms, and semantic descriptions

- **JEPA** (Joint Embedding Predictive Architecture), proposed by Yann LeCun, offers a framework for world models that avoids the pitfalls of pixel-level prediction. Instead of predicting raw pixels (which wastes capacity on irrelevant details like exact textures), JEPA predicts in embedding space. The

model learns an encoder that maps observations to embeddings and a predictor that forecasts future embeddings:



**z**

$$z_{t+1} = h_{\psi}(z_t, a_t), \quad z_t = \text{Enc}(s_t)$$

- The loss compares embeddings rather than raw observations, which is more robust to perceptual aliasing (many different pixel configurations may represent the same semantic state). This approach is especially promising for multimodal world models because it naturally operates in the shared embedding space that unified architectures already provide.
- World models have practical applications beyond academic interest. In **model-based reinforcement learning**, the agent uses its world model to “imagine” the consequences of actions before taking them, dramatically reducing the number of real-world interactions needed (recall the discussion of model-based RL from Chapter 11). In **autonomous driving**, a world model predicts how the scene will evolve over the next few seconds given different steering decisions. In **robotics**, a world model allows a robot to mentally rehearse a manipulation sequence before executing it.
- The frontier of world model research is moving towards **interactive world models** that run in real-time and respond to arbitrary user actions, essentially becoming general-purpose simulators learned entirely from data. Genie 2 (Google DeepMind) demonstrates this for 3D environments: given a single image, it generates an interactive, controllable 3D world that a user can explore. The convergence of world models and unified multimodal architectures suggests a future where a single model can perceive, predict, simulate, and act across all modalities.

## 50.10 Coding Tasks (use CoLab or notebook)

---

### Task 1: Build a minimal multimodal token interleaver

- Write a function that takes a text string and a dummy “image” (a small 2D array) and interleaves their tokenised representations into a single flat sequence with modality embeddings.

```

import jax
import jax.numpy as jnp

Simulate multimodal tokenisation: text tokens + "image patch" tokens
def interleave_modalities(text_tokens, image_patches, embed_dim=32,
key=jax.random.PRNGKey(0)):
 """Interleave text and image tokens with learned modality embeddings."""
 k1, k2, k3 = jax.random.split(key, 3)
 n_text = text_tokens.shape[0]
 n_img = image_patches.shape[0]
 # Random projection matrices (stand-ins for real encoders)
 W_text = jax.random.normal(k1, (text_tokens.shape[-1], embed_dim)) * 0.02
 W_img = jax.random.normal(k2, (image_patches.shape[-1], embed_dim)) * 0.02
 # Modality embeddings: one for text, one for image
 mod_emb = jax.random.normal(k3, (2, embed_dim)) * 0.02
 text_embs = text_tokens @ W_text + mod_emb[0] # (n_text, embed_dim)
 img_embs = image_patches @ W_img + mod_emb[1] # (n_img, embed_dim)
 # Interleave: [IMG] tokens first, then [TEXT] tokens (like LLaVA)
 combined = jnp.concatenate([img_embs, text_embs], axis=0)
 print(f"Combined sequence: {n_img} image + {n_text} text =
{combined.shape[0]} tokens")
 return combined

Try it: 5 text tokens (dim 16) and 4 image patches (dim 64)
text = jax.random.normal(jax.random.PRNGKey(1), (5, 16))
image = jax.random.normal(jax.random.PRNGKey(2), (4, 64))
seq = interleave_modalities(text, image)
Experiment: change embed_dim, swap the interleaving order, add a third modality

```

## Task 2: Visualise cross-modal attention patterns

- Create a synthetic multimodal sequence and compute self-attention scores to see how image tokens attend to text tokens and vice versa.

```

import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

def cross_modal_attention(n_text=6, n_img=4, d=32, key=jax.random.PRNGKey(42)):
 """Compute and visualise attention between text and image tokens."""
 k1, k2, k3 = jax.random.split(key, 3)
 # Simulate token embeddings for two modalities
 text_embs = jax.random.normal(k1, (n_text, d))
 img_embs = jax.random.normal(k2, (n_img, d))
 seq = jnp.concatenate([img_embs, text_embs], axis=0) # (n_img+n_text, d)
 # Learned Q, K projections

```

```

Wq = jax.random.normal(k3, (d, d)) * 0.1
Wk = jax.random.normal(jax.random.PRNGKey(99), (d, d)) * 0.1
Q, K = seq @ Wq, seq @ Wk
scores = Q @ K.T / jnp.sqrt(d)
attn = jax.nn.softmax(scores, axis=-1)
Plot
labels = [f"img_{i}" for i in range(n_img)] + [f"txt_{i}" for i in
range(n_text)]
fig, ax = plt.subplots(figsize=(7, 6))
ax.imshow(attn, cmap="viridis")
ax.set_xticks(range(len(labels))); ax.set_xticklabels(labels, rotation=45,
fontsize=8)
ax.set_yticks(range(len(labels))); ax.set_yticklabels(labels, fontsize=8)
ax.set_xlabel("Key (attended to)"); ax.set_ylabel("Query (attending from)")
ax.set_title("Cross-modal self-attention map")
plt.colorbar(ax.images[0], ax=ax, shrink=0.8)
plt.tight_layout(); plt.show()

cross_modal_attention()
Experiment: increase d, add a causal mask, observe how attention patterns
change

```

### Task 3: Simulate staged training with modality-specific loss weighting

- Demonstrate how modality-specific loss weights affect a toy multimodal training loop. Observe how balancing losses prevents one modality from dominating.

```

import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

def staged_training_sim(steps=200, key=jax.random.PRNGKey(7)):
 """Simulate multimodal training with adjustable modality loss weights."""
 # Two 'modalities' with different loss scales (text loss ~10x larger than
image loss)
 losses_text, losses_img = [], []
 param = jnp.array([0.0, 0.0]) # Shared param updated by both modality losses
 lr = 0.05
 # Try changing these weights to see the effect on convergence balance
 lambda_text, lambda_img = 1.0, 5.0 # upweight the weaker modality

 for step in range(steps):
 k1, k2, key = jax.random.split(key, 3)
 noise_t = jax.random.normal(k1, ()) * 0.3
 noise_i = jax.random.normal(k2, ()) * 0.1

```

```

 loss_t = (param[0] - 3.0) ** 2 + noise_t # text target = 3.0
 loss_i = 0.1 * (param[1] - 1.0) ** 2 + noise_i # image target = 1.0
(smaller scale)
 # Weighted combined gradient
 grad_t = lambda_text * 2 * (param[0] - 3.0)
 grad_i = lambda_img * 0.2 * (param[1] - 1.0)
 param = param - lr * jnp.array([grad_t, grad_i])
 losses_text.append(float(loss_t)); losses_img.append(float(loss_i))

fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(losses_text, label=f"Text loss (weight={lambda_text})", alpha=0.7)
ax.plot(losses_img, label=f"Image loss (weight={lambda_img})", alpha=0.7)
ax.set_xlabel("Training step"); ax.set_ylabel("Loss"); ax.legend()
ax.set_title("Modality loss balancing during staged training")
plt.tight_layout(); plt.show()

staged_training_sim()
Experiment: set lambda_img=1.0 and watch image loss converge much slower

```

## 51 Perception

Perception is how autonomous systems sense and interpret the physical world. This file covers sensor modalities, calibration, sensor fusion, 3D object detection, depth estimation, occupancy networks, lane detection, and semantic mapping, the sensory foundation that every robot, drone, and self-driving car builds upon.

- For a human, perceiving the world is effortless: you see a car approaching, hear its engine, feel the ground beneath your feet, and instantly build a mental model of your surroundings. An autonomous system must do the same thing, but using electronic sensors and algorithms instead of eyes and ears.
- The fundamental challenge is this: sensors give you raw numbers (pixel intensities, point clouds, signal reflections), and the system must turn those numbers into a structured understanding: “there is a pedestrian 12 metres ahead, moving left at 1.5 m/s.” This is the perception problem.
- Everything downstream (prediction, planning, control) depends on perception. A self-driving car with a perfect planner but poor perception will still crash. Perception is the bottleneck.

## 51.1 Sensor Modalities

- Autonomous systems use multiple sensor types, each with different strengths and failure modes. No single sensor is sufficient on its own.

**Sensor Modalities: Strengths and Weaknesses**

	Resolution	Depth	Weather	Velocity	Cost
<b>Camera</b> RGB images	●●●	●○○	●○○	●●○	●●●
<b>LiDAR</b> laser pulses	●●○	●●●	●●○	●○○	●○○
<b>Radar</b> radio waves	●○○	●●○	●●●	●●●	●●○
<b>IMU</b> accel + gyro	n/a	n/a	●●●	●●●	●●●

●●● = excellent      ●●○ = moderate      ●○○ = poor

Sensor comparison: cameras, LiDAR, radar, and IMU rated on resolution, depth, weather robustness, velocity measurement, and cost

- Cameras** capture dense colour information at high resolution. A single image contains millions of pixels, each recording RGB values (as we saw in chapter 8). Cameras are cheap, lightweight, and provide rich texture and colour information essential for reading signs, detecting traffic lights, and recognising objects.
- Camera types include **monocular** (single lens, no native depth), **stereo** (two lenses separated by a baseline, enabling depth via disparity as covered in chapter 8), and **fisheye** (ultra-wide field of view, 180°+, with heavy radial distortion, used for surround-view parking systems).
- The main weakness of cameras is that they lose depth information during projection. A 3D scene is mapped onto a 2D image plane through the pinhole camera model (recall the intrinsic matrix  $K$  from chapter 8):

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \frac{1}{Z} K \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}$$

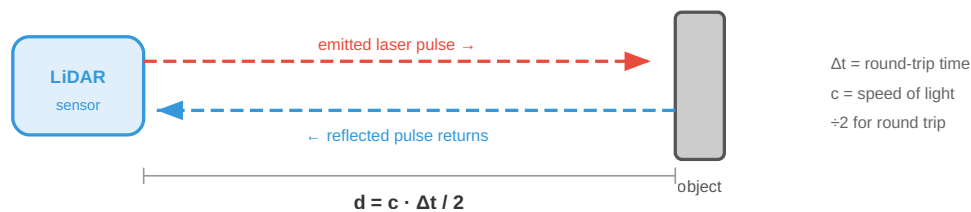
- The division by  $Z$  discards absolute depth. Two objects of different sizes at different distances can produce identical projections. Recovering depth from a single image is ill-posed, which is why stereo cameras or learned monocular depth models are needed.



- Cameras also struggle in adverse conditions: direct sunlight causes glare, darkness reduces signal, and rain or fog scatter light.
- **LiDAR** (Light Detection and Ranging) fires laser pulses and measures the time for each pulse to bounce back. Since light travels at a known speed ( $c \approx 3 \times 10^8$  m/s), the distance to each reflection point is:

$$d = \frac{c \cdot \Delta t}{2}$$

**LiDAR: Time-of-Flight Distance Measurement**



LiDAR time-of-flight: a laser pulse bounces off an object, and distance is computed from the round-trip time

- The factor of 2 accounts for the round trip (out and back). By sweeping the laser across the scene, LiDAR builds a **point cloud**: a set of 3D coordinates ( $x, y, z$ ), often with intensity (reflectance) values.
- **Spinning LiDAR** (e.g., Velodyne) rotates a laser array  $360^\circ$  to produce a full surround view. Typical units generate 300,000+ points per second across 64–128 vertical channels. The result is a sparse but geometrically accurate 3D representation of the scene.
- **Solid-state LiDAR** has no moving parts, using optical phased arrays or MEMS mirrors instead. This makes them cheaper, more compact, and more reliable, but typically with a narrower field of view ( $120^\circ$  vs  $360^\circ$ ).
- LiDAR gives precise depth but produces sparse data (far fewer “pixels” than a camera), has no colour information, and is expensive. It also degrades in heavy rain, snow, or dust, as particles scatter the laser pulses.
- **Radar** (Radio Detection and Ranging) works on the same time-of-flight principle as LiDAR but uses radio waves (millimetre-wave, typically 77 GHz for automotive). Radio waves penetrate rain, fog, dust, and snow far better than light, making radar the most weather-robust sensor.

- Radar also directly measures **velocity** via the Doppler effect. When an object moves towards the sensor, reflected waves are compressed (higher frequency); when moving away, they are stretched (lower frequency). The velocity is:

$$v = \frac{\Delta f \cdot c}{2f_0}$$

- where  $\Delta f$  is the frequency shift and  $f_0$  is the transmitted frequency. This gives instantaneous radial velocity without any tracking or frame-to-frame computation.
- The tradeoff is resolution: radar has much coarser angular resolution than cameras or LiDAR, making it poor at distinguishing nearby objects or detecting fine details. It excels at detecting vehicles at long range (200+ metres) in any weather.
- **Ultrasonic sensors** emit high-frequency sound pulses (40–70 kHz) and measure echo return time. They work at very short range (0.2–5 metres) and are used primarily for parking assistance. Their physics is identical to LiDAR but with sound instead of light, so  $d = \frac{v_{\text{sound}} \cdot \Delta t}{2}$  where  $v_{\text{sound}} \approx 343$  m/s.
- An **IMU** (Inertial Measurement Unit) contains accelerometers and gyroscopes that measure linear acceleration and angular velocity respectively. IMUs provide high-frequency motion data (often 200–1000 Hz) that fills the gaps between slower sensor updates. They do not sense the environment directly but track the robot's own motion, making them essential for dead-reckoning and state estimation.
- IMUs suffer from **drift**: small measurement errors accumulate over time, causing the estimated position to diverge from reality. This is why IMUs are almost always fused with other sensors (cameras, GPS, LiDAR) rather than used alone.
- **GNSS** (Global Navigation Satellite Systems, including GPS) provides absolute position on Earth's surface by triangulating signals from multiple satellites. Standard GPS accuracy is 2–5 metres, which is insufficient for lane-level driving. **RTK-GPS** (Real-Time Kinematic) uses a fixed base station to correct errors, achieving centimetre-level accuracy, but requires clear sky view and base station infrastructure.

## 51.2 Sensor Calibration

---

- Before sensors can work together, they must be **calibrated**: each sensor's measurements must be related to a common coordinate frame.
- **Intrinsic calibration** determines a sensor's internal parameters. For a camera, this means the focal length, principal point, and distortion coefficients (as covered in chapter 8). For LiDAR, it means the precise angular offsets between laser beams. A common method is Zhang's checkerboard calibration, where a known planar pattern is observed from multiple angles to solve for the intrinsic matrix.
- **Extrinsic calibration** determines the rigid transformation (rotation  $R$  and translation  $\mathbf{t}$ ) between two sensors. If a camera and LiDAR are mounted on the same vehicle, extrinsic calibration finds the  $4 \times 4$  transformation matrix that maps points from LiDAR coordinates to camera coordinates:

$$\mathbf{p}_{\text{cam}} = \begin{bmatrix} R & \mathbf{t} \\ \mathbf{0}^T & 1 \end{bmatrix} \mathbf{p}_{\text{lidar}}$$

- This is an affine transformation in homogeneous coordinates, exactly the kind we studied in chapter 2 (linear transformations). Getting this matrix wrong means LiDAR points project to the wrong pixels, and the entire fusion pipeline breaks.
- **Temporal calibration** synchronises sensor clocks. A camera capturing at 30 Hz and a LiDAR at 10 Hz produce data at different timestamps. If a car is moving at 30 m/s (highway speed), a 10 ms timing error corresponds to a 30 cm spatial error. Hardware triggering (a shared clock pulse) or software synchronisation (interpolation between timestamps) is essential.

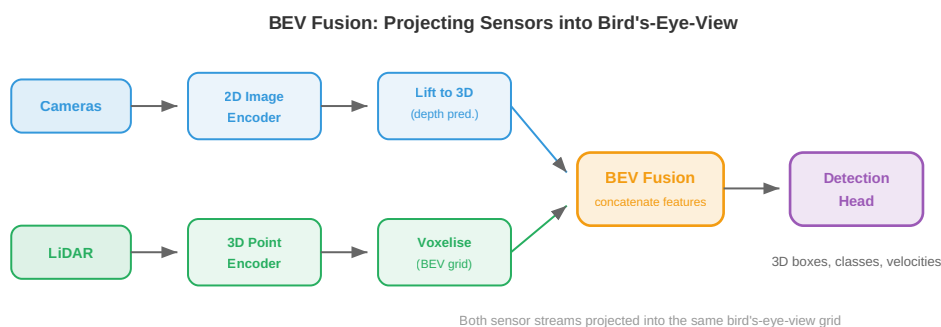
## 51.3 Sensor Fusion

---

- No single sensor covers all conditions. Cameras see colour and texture but lose depth. LiDAR measures depth precisely but is sparse and colourblind. Radar works in any weather but has poor resolution. **Sensor fusion** combines their strengths and compensates for individual weaknesses.
- **Early fusion** (or data-level fusion) combines raw sensor data before any processing. For example, projecting LiDAR points onto the camera image to

create an RGBD representation (colour + depth per pixel), or painting each LiDAR point with the colour of the camera pixel it projects onto. This preserves the most information but requires precise calibration and is sensitive to misalignment.

- **Late fusion** (or decision-level fusion) processes each sensor independently through its own detection pipeline, then merges the final outputs (bounding boxes, class labels, confidence scores). Each sensor votes, and a fusion module reconciles disagreements. This is simpler and more modular, but each pipeline cannot benefit from the other sensor's raw data.
- **Mid-level fusion** operates on intermediate feature representations. Each sensor's raw data is encoded into a learned feature space (using CNNs or transformers), and the features are then combined. This is the dominant approach in modern systems because it lets the network learn what to extract from each modality.



BEV fusion pipeline: cameras and LiDAR are independently encoded then projected into a shared bird's-eye-view grid

- **BEVFusion** is a representative mid-level fusion architecture. It projects both camera features and LiDAR features into a common **bird's-eye-view (BEV)** representation, a top-down grid of the scene. Camera features are “lifted” to 3D using predicted depth distributions, then splattered onto the BEV grid. LiDAR features are already 3D and are directly voxelised onto the same grid. The fused BEV features are then processed by a detection head.
- The BEV representation is powerful because it provides a unified, metric-scale coordinate frame where spatial reasoning (distances, sizes, overlaps) is straightforward. In the camera image, a nearby bicycle and a distant truck might occupy the same number of pixels. In BEV, their true sizes and positions are clear.

## 51.4 3D Object Detection

---

- The core task of perception is detecting objects in 3D: where are they, how big are they, what are they, and which way are they facing? Each detection is a **3D bounding box** with position  $(x, y, z)$ , dimensions  $(l, w, h)$ , heading angle  $\theta$ , class label, and confidence score.
- **LiDAR-based detection** operates directly on point clouds. The challenge is that point clouds are unordered, irregular, and vary in density (nearby objects have thousands of points, distant ones have a handful). Recall from chapter 8 that PointNet handles this with shared MLPs and a permutation-invariant aggregation (max pooling).
- **PointPillars** converts the point cloud into a structured representation by discretising the ground plane into a grid of vertical columns ( “pillars” ). All points within each pillar are encoded by a small PointNet into a fixed-size feature vector. The result is a 2D pseudo-image that can be processed by a standard 2D CNN backbone, followed by a detection head (like the SSD architecture from chapter 8). This is fast and effective.
- **CenterPoint** detects objects as points rather than boxes. It predicts a heatmap of object centres in BEV, then regresses the box attributes (size, height, heading, velocity) at each peak. This is the 3D analogue of CenterNet (chapter 8): anchor-free, no NMS required during training, and naturally extends to tracking by associating centre points across frames.
- **Camera-only 3D detection** must infer depth from 2D images, which is fundamentally harder. Modern approaches like **BEVDet** and **BEVFormer** use transformer architectures to “lift” 2D image features to 3D. BEVFormer uses spatial cross-attention: BEV queries attend to specific 3D reference points projected onto each camera’ s image, pulling features from the relevant locations.
- The accuracy gap between LiDAR-based and camera-based 3D detection has been shrinking rapidly, driven by better depth estimation, larger models, and temporal fusion (using multiple frames to accumulate depth cues, similar to how stereo matching works but across time).

## 51.5 Depth Estimation

---

- Depth estimation is the problem of assigning a distance value to each pixel or point.
- **Stereo matching** uses two cameras separated by a known baseline  $b$ . The same 3D point appears at slightly different horizontal positions in the two images (the **disparity**  $d$ ). Depth is computed as (from chapter 8):

$$Z = \frac{f \cdot b}{d}$$

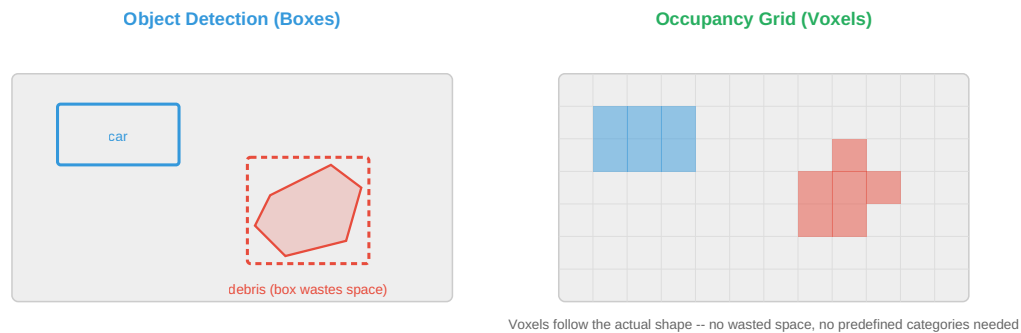
- where  $f$  is the focal length. The challenge is finding correct correspondences between the two images, especially in textureless regions, occlusions, and repetitive patterns. Modern stereo networks (e.g., RAFT-Stereo) use iterative refinement with correlation volumes.
- **Monocular depth estimation** predicts depth from a single image. Since this is ill-posed (infinitely many 3D scenes can produce the same image), the network must learn statistical priors: “floors are flat,” “objects get smaller with distance,” “texture gradient indicates receding surfaces.”
- **Depth Anything** (covered in chapter 8) achieves strong monocular depth by training on massive unlabelled datasets with self-supervision, then fine-tuning on labelled data. The key insight is that scale-invariant losses handle the inherent ambiguity: the model predicts relative depth (ordering) rather than absolute metres.
- **LiDAR-camera depth fusion** projects sparse LiDAR depth measurements onto the camera image as supervision. The network learns to “fill in” the gaps between sparse points, producing dense depth maps that combine LiDAR’s accuracy with the camera’s resolution.

## 51.6 Occupancy Networks

---

- Traditional perception outputs a list of bounding boxes, one per detected object. But the real world contains many things that do not fit neatly into boxes: oddly shaped debris, construction barriers, overhanging branches, a partially collapsed wall.

## Bounding Boxes vs Occupancy Grid



Bounding boxes waste space on irregular shapes; occupancy grids follow the actual geometry

- **Occupancy networks** represent the scene as a dense 3D voxel grid. Each voxel (a small cube of space, e.g.,  $0.2\text{m} \times 0.2\text{m} \times 0.2\text{m}$ ) is classified as either free, occupied, or unknown, and optionally given a semantic label (road, sidewalk, vehicle, vegetation, etc.).
- This is a shift from object-centric perception ( “detect the car” ) to scene-centric perception ( “which parts of 3D space are occupied?” ). The advantage is generality: the system does not need a predefined list of object categories to avoid collisions with arbitrary obstacles.
- Architecturally, occupancy networks take sensor inputs (cameras, LiDAR, or both), encode them into a 3D feature volume, and predict per-voxel labels. The 3D feature volume is typically constructed by lifting 2D features to 3D (similar to BEV construction but extending vertically) and processed with 3D convolutions or sparse convolutions.
- **TPVFormer** (Tri-Perspective View) avoids the cubic cost of full 3D attention by factorising the 3D volume into three orthogonal planes (top-down, front, side). Each plane uses 2D attention, and their features are combined at each voxel. This is reminiscent of how SVD decomposes a matrix into simpler factors (chapter 2): breaking a hard 3D problem into manageable 2D pieces.
- The output voxel grid directly tells the planner which regions of space are safe to occupy and which are not, making it a natural interface between perception and planning.

## 51.7 Lane Detection and Road Topology

---

- For vehicles on structured roads, understanding **lane geometry** is critical. The system must know where lanes are, how they curve, where they merge and split, and which lane the vehicle is in.
- Classical approaches fit parametric curves to detected lane markings. A common model is the third-degree polynomial:

$$x(y) = a_0 + a_1y + a_2y^2 + a_3y^3$$

- where  $y$  is the longitudinal distance ahead and  $x$  is the lateral offset. This is a polynomial approximation (recall Taylor series from chapter 3), chosen because roads are smooth curves well-captured by low-degree polynomials. The coefficients are estimated via least-squares regression on detected lane points.
- Modern approaches use neural networks to detect lanes directly. **LaneNet** treats each lane as an instance and uses an embedding branch to group pixels belonging to the same lane, followed by curve fitting. **GANet** uses a graph-based approach, representing lane topology as a directed graph where nodes are lane points and edges encode connectivity (which lanes merge, split, or connect at intersections).
- **Road topology** goes beyond individual lane curves to capture the full structure: how lanes connect to each other, which lanes allow left turns, where a highway on-ramp merges. This is modelled as a directed graph, where intersections are nodes and lane segments are edges with attributes (speed limit, lane type, turn restrictions).
- The graph structure is essential for route planning: the planner needs to know not just “where are the lanes” but “which sequence of lanes leads to the destination.”

## 51.8 Semantic Mapping

---

- Perception does not end with detecting objects in a single frame. Over time, an autonomous system builds a **semantic map**: a persistent, structured representation of its environment that accumulates information across many observations.



- At its simplest, a semantic map is a 2D grid (an **occupancy grid**) where each cell stores the probability of being occupied. As the robot moves and scans with its sensors, it updates these probabilities using Bayesian updates:

$$P(\text{occupied} \mid z_{1:t}) = \frac{P(z_t \mid \text{occupied}) \cdot P(\text{occupied} \mid z_{1:t-1})}{P(z_t)}$$

- This is Bayes' theorem in action (from chapter 5): each new measurement  $z_t$  updates the prior belief about each cell. The **log-odds** representation is often used to avoid numerical issues with multiplying many small probabilities:

$$l_t = l_{t-1} + \log \frac{P(z_t \mid \text{occupied})}{P(z_t \mid \text{free})}$$

- Adding log-odds is equivalent to multiplying probabilities (recall that  $\log(ab) = \log a + \log b$ ), and the running sum naturally accumulates evidence over time.
- Richer maps assign semantic labels to each cell (road, sidewalk, building, vegetation) and can extend to 3D. These are closely related to occupancy networks but emphasise persistence and temporal aggregation rather than single-frame prediction.
- **SLAM** (Simultaneous Localisation and Mapping), covered in chapter 8, is the algorithm that builds the map while simultaneously tracking the robot's position within it. Visual-inertial SLAM fuses camera and IMU data; LiDAR SLAM uses point cloud registration. The perception pipeline feeds detections and depth estimates into the SLAM system, which maintains the global map.
- Modern approaches increasingly use neural implicit representations (like NeRFs from chapter 8) to build dense, photorealistic maps that can be queried at any 3D point. These neural maps store a compressed representation of the entire scene in network weights, enabling tasks like novel view synthesis and detailed spatial queries.

## 51.9 Coding Tasks (use CoLab or notebook)

---

1. Project 3D LiDAR points onto a 2D camera image using a projection matrix. Visualise which points fall within the image bounds.

```

import jax.numpy as jnp
import matplotlib.pyplot as plt

Simulated LiDAR points in 3D (x=forward, y=left, z=up)
rng = jax.random.PRNGKey(0)
points_3d = jax.random.uniform(rng, (200, 3), minval=jnp.array([5, -10, -2]),
 maxval=jnp.array([50, 10, 3]))

Camera intrinsic matrix (focal length 500, image centre 320x240)
K = jnp.array([[500, 0, 320],
 [0, 500, 240],
 [0, 0, 1.0]])

Extrinsic: LiDAR to camera (identity rotation, small translation)
R = jnp.eye(3)
t = jnp.array([0.0, 0.0, -0.5])

Project: $p_{cam} = K @ (R @ p_{lidar} + t)$
p_cam = (R @ points_3d.T).T + t
p_img = (K @ p_cam.T).T
p_img = p_img[:, :2] / p_img[:, 2:3] # divide by Z

Filter points in front of camera and within image
mask = (p_cam[:, 2] > 0) & (p_img[:, 0] > 0) & (p_img[:, 0] < 640) & \
 (p_img[:, 1] > 0) & (p_img[:, 1] < 480)
depth = p_cam[mask, 2]

plt.figure(figsize=(8, 5))
plt.scatter(p_img[mask, 0], p_img[mask, 1], c=depth, cmap="viridis", s=5)
plt.colorbar(label="Depth (m)")
plt.xlim(0, 640); plt.ylim(480, 0)
plt.title("LiDAR points projected onto camera image")
plt.xlabel("u (pixels)"); plt.ylabel("v (pixels)")
plt.show()

```

1. Build a simple 2D occupancy grid using Bayesian log-odds updates. Simulate a range sensor scanning an environment and watch the map emerge.

```

import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

Grid setup: 50x50 cells, each 0.2m
grid_size = 50
log_odds = jnp.zeros((grid_size, grid_size))

```

```

Sensor model: log-odds update values
l_occ = 0.85 # confidence that a hit means occupied
l_free = -0.4 # confidence that a pass-through means free

Simulated obstacle: a wall from (5,20) to (5,30) in grid coords
wall_y = jnp.arange(20, 30)

Robot at (25, 25), scanning outward
robot = jnp.array([25, 25])

for angle_deg in range(0, 360, 5):
 angle = jnp.radians(angle_deg)
 direction = jnp.array([jnp.cos(angle), jnp.sin(angle)])

 for step in range(1, 25):
 cell = (robot + direction * step).astype(int)
 r, c = int(cell[0]), int(cell[1])
 if r < 0 or r >= grid_size or c < 0 or c >= grid_size:
 break

 # Check if this cell is the wall
 is_wall = (r == 5) and (c >= 20) and (c < 30)
 if is_wall:
 log_odds = log_odds.at[r, c].add(l_occ)
 break
 else:
 log_odds = log_odds.at[r, c].add(l_free)

Convert log-odds to probability
prob = 1.0 / (1.0 + jnp.exp(-log_odds))

plt.figure(figsize=(6, 6))
plt.imshow(prob.T, origin="lower", cmap="RdYlGn_r", vmin=0, vmax=1)
plt.colorbar(label="P(occupied)")
plt.plot(25, 25, "b*", markersize=10, label="Robot")
plt.legend()
plt.title("2D Occupancy Grid from Bayesian Updates")
plt.show()

```

1. Compute the depth from a stereo image pair using disparity. Simulate two camera views of 3D points, compute the disparity, and recover depth.

```

import jax
import jax.numpy as jnp

Camera parameters

```

```

f = 500.0 # focal length in pixels
b = 0.12 # baseline in metres (12 cm)

3D points at known depths
depths_true = jnp.array([5.0, 10.0, 20.0, 50.0, 100.0])

Disparity = f * b / Z
disparities = f * b / depths_true

Recover depth from disparity
depths_recovered = f * b / disparities

for z, d, z_r in zip(depths_true, disparities, depths_recovered):
 print(f"True depth: {z:6.1f}m Disparity: {d:6.2f}px Recovered: {z_r:6.1f}
m")

Notice: disparity is inversely proportional to depth
Close objects have large disparity, far objects have tiny disparity
This is why stereo is most accurate at short range

```

## 52 Robot Learning

Robot learning bridges the gap between algorithms and physical action. This file covers kinematics, dynamics, classical control, imitation learning, sim-to-real transfer, manipulation, locomotion, and safety, the techniques that give robots the ability to move, grasp, walk, and interact with the real world.

- In previous chapters, we studied how to perceive the world (chapter 8, chapter 11 file 1) and how to learn from data (chapter 6). But perception and learning are not enough. A robot must **act**: move its arm to grasp a cup, walk across uneven terrain, or navigate a warehouse. This is where robot learning comes in.
- The central challenge is that the physical world is continuous, high-dimensional, contact-rich, and unforgiving. A classification error in image recognition is an incorrect label. A control error in robotics is a broken robot or a dropped object. The stakes are different.

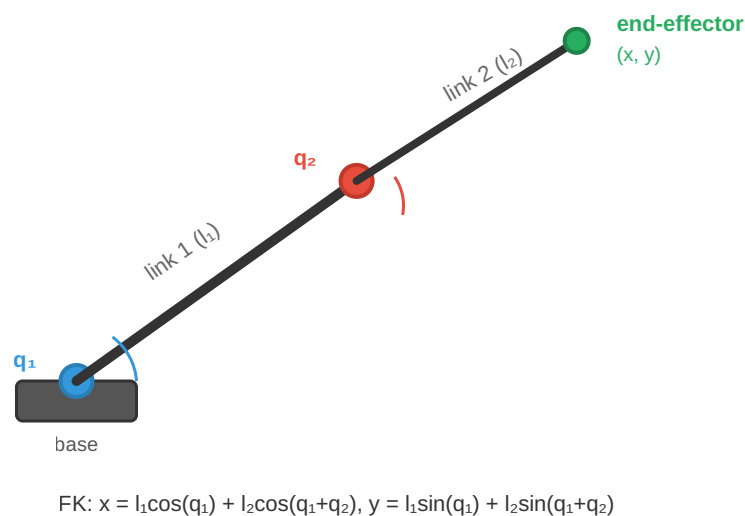
### 52.1 Robot Kinematics

- **Kinematics** describes the geometry of motion without considering forces. A robot arm is a chain of rigid links connected by joints. Each joint has one

degree of freedom (DoF): it either rotates (revolute joint) or slides (prismatic joint).

- The **configuration** of a robot is the set of all joint angles (or displacements)  $\mathbf{q} = [q_1, q_2, \dots, q_n]^T$ . This vector lives in **joint space** (or configuration space), an n-dimensional space where each axis corresponds to one joint. A 6-DoF robot arm has a 6D configuration space.

### 2-Link Robot Arm: Forward Kinematics



2-link robot arm: joint angles  $q_1$  and  $q_2$  determine the end-effector position via forward kinematics

- **Forward kinematics (FK)** computes the position and orientation of the end-effector (the “hand”) given the joint angles. This is a function  $\mathbf{x} = f(\mathbf{q})$  that maps from joint space to **task space** (the 3D position and orientation of the end-effector, also called Cartesian space).
- Each joint is described by a  $4 \times 4$  homogeneous transformation matrix (recall affine transformations from chapter 2). The **Denavit-Hartenberg (DH) convention** parameterises each joint with four numbers: link length  $a$ , link twist  $\alpha$ , link offset  $d$ , and joint angle  $\theta$ . The transformation for joint  $i$  is:

$$T_i = \begin{bmatrix} \cos\theta_i & -\sin\theta_i \cos\alpha_i & \sin\theta_i \sin\alpha_i & a_i \cos\theta_i \\ \sin\theta_i & \cos\theta_i \cos\alpha_i & -\cos\theta_i \sin\alpha_i & a_i \sin\theta_i \\ 0 & \sin\alpha_i & \cos\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- The full forward kinematics is the product of all joint transformations:  
 $T_{0 \rightarrow n} = T_1 T_2 \cdots T_n$ . This is matrix multiplication chaining transformations (chapter 2): each joint's transformation is applied in sequence, rotating and translating the frame from the base to the end-effector.
- **Inverse kinematics (IK)** is the reverse problem: given a desired end-effector pose  $\mathbf{x}^*$ , find the joint angles  $\mathbf{q}$  such that  $f(\mathbf{q}) = \mathbf{x}^*$ . This is much harder because:
  - The mapping is nonlinear (involves sines and cosines).
  - There may be multiple solutions (different arm configurations reach the same point).
  - There may be no solution (the target is out of reach).
- Analytical solutions exist only for specific robot geometries. For general robots, IK is solved iteratively using the **Jacobian**. The Jacobian  $J(\mathbf{q})$  relates small changes in joint angles to small changes in end-effector position (recall the Jacobian from chapter 3):

·

$\mathbf{x}$

$$= J(\mathbf{q})$$

·

$\mathbf{q}$

- To move the end-effector by a small amount  $\Delta\mathbf{x}$ , we need  $\Delta\mathbf{q} = J^{-1}\Delta\mathbf{x}$  (or  $J^+\Delta\mathbf{x}$  using the pseudo-inverse when  $J$  is not square). This is iterated until the end-effector reaches the target, which is essentially Newton's method (chapter 3) applied to the kinematics equation.
- Near **singularities**, the Jacobian loses rank (some columns become linearly dependent, as we studied in chapter 2). Physically, this means the robot loses a degree of freedom: no matter how fast the joints move, the end-effector cannot move in certain directions. The pseudo-inverse blows up near singularities, so damped least-squares (adding a regularisation term  $\lambda^2 I$ ) is used instead:

$$\Delta \mathbf{q} = \mathbf{J}^T (\mathbf{J} \mathbf{J}^T + \lambda^2 \mathbf{I})^{-1} \Delta \mathbf{x}$$

## 52.2 Dynamics and Control

- **Dynamics** adds forces to the picture. The equations of motion for a robot arm follow the **manipulator equation**:

$$\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) + \mathbf{g}(\mathbf{q}) = \boldsymbol{\tau}$$

- where  $\mathbf{M}(\mathbf{q})$  is the mass (inertia) matrix,  $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$  captures Coriolis and centrifugal effects,  $\mathbf{g}(\mathbf{q})$  is the gravity vector, and  $\boldsymbol{\tau}$  is the vector of joint torques (the control input). This is a system of second-order differential equations, one per joint.

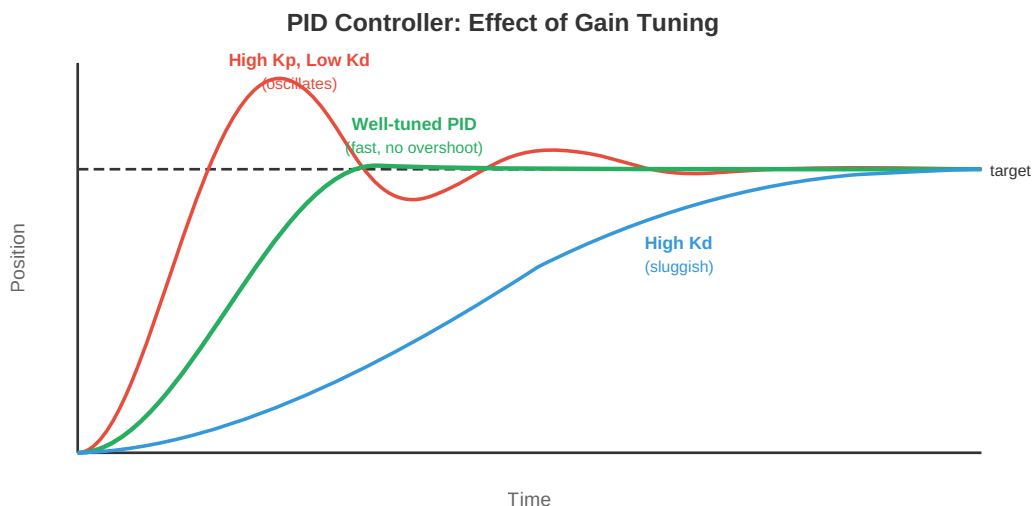
- The mass matrix  $\mathbf{M}$  is always symmetric and positive definite (recall from chapter 2 that positive definite matrices guarantee a unique minimum, here it ensures the system responds predictably to applied torques).
- **PID control** is the most widely used controller in robotics. For each joint, it computes a torque based on the error  $e(t) = q_{\text{desired}}(t) - q_{\text{actual}}(t)$ :

$$\tau(t) = K_p e(t) + K_i \int_0^t e(s) ds + K_d \dot{e}(t)$$

•  
e

(t)

- The three terms have intuitive roles:
  - **Proportional** ( $K_p$ ): corrects proportionally to the current error. Larger error  $\rightarrow$  larger correction. Like a spring pulling the joint towards the target.
  - **Integral** ( $K_i$ ): accumulates past errors to eliminate steady-state offset. If the joint consistently undershoots, the integral term builds up and provides extra push.
  - **Derivative** ( $K_d$ ): reacts to the rate of change of error, providing damping. It slows the response as the error decreases, preventing overshoot and oscillation.



PID controller tuning: high  $K_p$  oscillates, high  $K_d$  is sluggish, well-tuned PID reaches the target quickly

- Tuning  $K_p$ ,  $K_i$ ,  $K_d$  is a balance: too much  $K_p$  causes oscillation, too much  $K_d$  makes the system sluggish, too much  $K_i$  causes wind-up (the integral grows unbounded during sustained error).
- **Model Predictive Control (MPC)** looks ahead. At each timestep, it solves an optimisation problem: find the sequence of future controls that minimises a cost function (e.g., tracking error + control effort) over a finite horizon, subject to the dynamics model and constraints. Only the first control is applied, then the process repeats at the next timestep.



min

$\mathbf{u}_{0:T}$

T

$\sum$

t=0

$$[\|\mathbf{x}_t - \mathbf{x}_t^*\|_Q^2 + \|\mathbf{u}_t\|_R^2] \text{ subject to } \mathbf{x}_{t+1} = f(\mathbf{x}_t, \mathbf{u}_t)$$

- Here  $\|\mathbf{x}\|_Q^2 = \mathbf{x}^T Q \mathbf{x}$  is a weighted norm using positive definite matrix Q (chapter 2), which lets you penalise different state errors differently. MPC naturally handles constraints (joint limits, torque limits, obstacle avoidance) because they are explicitly included in the optimisation.
- **Impedance control** regulates the relationship between force and motion rather than tracking a rigid trajectory. Instead of commanding “go to position x,” it commands “behave like a spring-damper system centred at x” :

$$\mathbf{F} = K_s(\mathbf{x}^* - \mathbf{x})$$

$$+ D(\dot{\mathbf{x}}^* - \dot{\mathbf{x}})$$

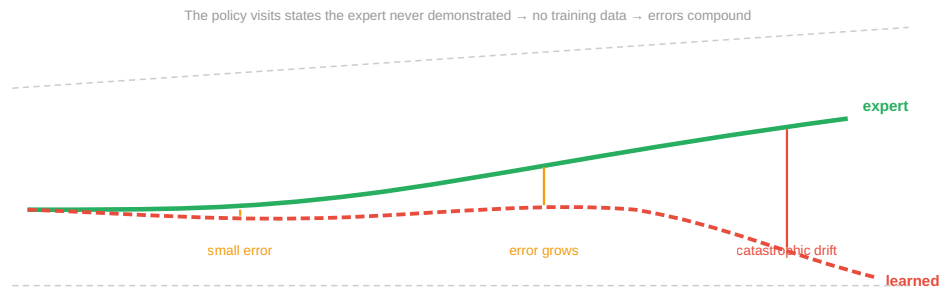
- where  $K_s$  is a stiffness matrix and D is a damping matrix. This makes the robot compliant: if it contacts an obstacle, it yields rather than forcing through. Impedance control is essential for contact-rich tasks like inserting a peg into a hole or handing an object to a human.

## 52.3 Imitation Learning

- Instead of hand-designing controllers, we can learn control policies from demonstrations. A human performs the task, the robot observes, and a learning algorithm extracts a policy. This is **imitation learning** (or learning from demonstration).
- **Behavioural cloning (BC)** is the simplest approach: treat the demonstrations as a supervised learning dataset. Given observation-action pairs  $\{(\mathbf{o}_t, \mathbf{a}_t)\}$  from an expert, train a policy  $\pi_\theta(\mathbf{a} \mid \mathbf{o})$  to predict the expert’s action from the observation. This is standard supervised learning (chapter 6): minimise the loss:

$$\mathcal{L}(\theta) = \mathbb{E}_{(\mathbf{o}, \mathbf{a}) \sim \mathcal{D}} [\| \pi_{\theta}(\mathbf{o}) - \mathbf{a} \|^2]$$

### Behavioural Cloning: Distribution Shift (Compounding Error)



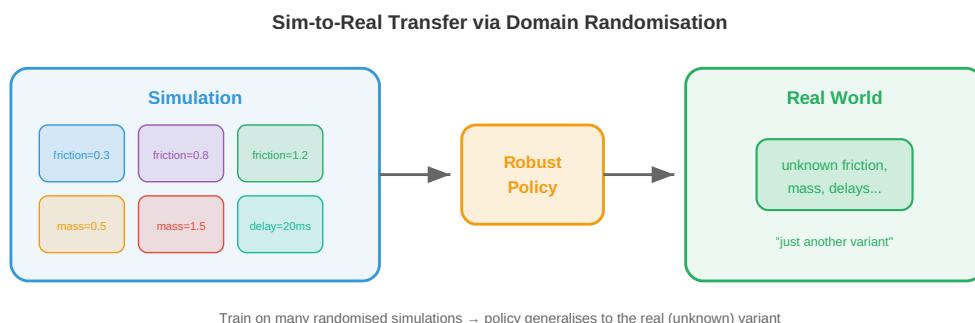
Distribution shift in behavioural cloning: small errors compound, causing the learned policy to drift far from the expert trajectory

- The problem is **distribution shift** (also called the **compounding error problem**). During training, the policy sees the expert's states. During deployment, the policy's own small errors push it into states the expert never visited. These unfamiliar states lead to worse actions, which lead to even more unfamiliar states, and errors compound rapidly.
- Imagine learning to drive by watching a perfect driver. You have never seen what happens after a small swerve because the expert never swerved. The first time you drift slightly, you have no idea how to recover.
- **Dagger** (Dataset Aggregation) addresses this by iterating:
  1. Train a policy on current data.
  2. Run the policy in the environment, collecting new states.
  3. Ask the expert to label these new states with the correct action.
  4. Add the new data to the dataset and retrain.
- Over iterations, the dataset covers states the learned policy actually visits, not just the expert's trajectory. The policy improves because it has seen and learned to recover from its own mistakes.
- **Action Chunking with Transformers (ACT)** is a modern approach where the policy predicts a sequence of future actions (a "chunk") rather than one action at a time. This is implemented using a conditional VAE with a transformer backbone. Predicting action chunks is more robust because it captures temporal correlations: the smoothness of a reaching motion is encoded in the chunk rather than relying on autoregressive single-step predictions that can drift.

- **Diffusion Policy** applies diffusion models (chapter 8) to action generation. Instead of predicting a single action, it models the full distribution of possible actions conditioned on the observation. Starting from noise, it iteratively denoises to produce an action sequence. This handles **multimodality** naturally: when there are multiple valid ways to complete a task (reach from the left or the right), the diffusion model can represent both modes, whereas a regression policy would average them (and reach somewhere in the middle, which might be neither valid).

## 52.4 Sim-to-Real Transfer

- Training robots in the real world is expensive, slow, and dangerous. A robot learning to grasp by trial and error might take thousands of attempts, breaking objects and itself along the way. **Simulation** offers unlimited, safe, fast experience. But simulators are imperfect: physics is approximated, visuals are synthetic, contacts are simplified.
- The **sim-to-real gap** is the difference between simulated and real performance. A policy that works perfectly in simulation may fail completely on the real robot because it has overfit to simulator-specific details.



Sim-to-real via domain randomisation: train on many randomised simulations so the real world is just another variant

- **Domain randomisation** combats this by training across a wide range of simulator settings. Instead of one simulation, use thousands with randomised:
  - Physics: friction coefficients, mass, damping
  - Visuals: lighting, textures, colours, camera position
  - Dynamics: motor delays, noise levels
- The idea is that if the policy works across all these variations, the real world is just “another variation” within the distribution. The policy learns features

that are invariant to the randomised properties, and these invariant features transfer.

- **System identification** takes the opposite approach: instead of randomising everything, carefully measure the real system's physical parameters and tune the simulator to match. This gives a more accurate simulation but is brittle (any unmodelled effect causes a gap).
- In practice, the best results combine both: system identification to get the simulator reasonably close, then domain randomisation to cover the remaining uncertainty.
- **Sim-to-real via fine-tuning** trains primarily in simulation, then does a small amount of real-world fine-tuning. The simulation provides a good initialisation, and the real-world data corrects simulator-specific biases. This requires far less real-world data than training from scratch.

## 52.5 World Models for Robotics

---

- All of the RL and imitation learning approaches above are **model-free**: the policy learns to act through direct interaction (or demonstrations) without explicitly modelling how the world works. An alternative is **model-based** learning: first learn a model of the environment's dynamics, then use that model to plan or to generate synthetic experience.
- A **world model** learns the transition function  $p(s_{t+1} \mid s_t, a_t)$ : given the current state and an action, predict the next state (as introduced in chapter 10). In robotics, this means predicting what will happen if the robot takes a particular action: “if I push this block left, it will slide 3cm and the cup behind it will topple.”
- The appeal is **sample efficiency**. Real-world robot interaction is expensive. If the robot can learn a world model from a modest amount of real data, it can then “imagine” thousands of trajectories by rolling out the model in its head, planning and refining its policy without touching the physical world. This is analogous to how a chess player thinks ahead by simulating moves mentally.

- **DreamerV3** is a general-purpose model-based RL agent. It learns three components jointly:
  - A **representation model** that encodes observations into a compact latent state.
  - A **transition model** (the world model) that predicts the next latent state given the current state and action.
  - A **reward model** that predicts the reward from the latent state.
- The agent then “dreams” by rolling out the transition model for many steps in latent space, trains a policy on these imagined trajectories, and transfers the policy to the real environment. The key innovation is that all imagination happens in latent space (compact learned representations), not in pixel space, making it computationally feasible.



$s$



$$s_{t+1} = f_{\theta}(s_t, a_t), \quad r_t$$

$$r_t = g_{\theta}(s_t)$$

- The transition model  $f_{\theta}$  and reward model  $g_{\theta}$  are trained on real experience, and the policy is trained on imagined rollouts. This decouples data collection from policy optimisation.
- For robot manipulation, world models enable **mental rehearsal**. Before attempting a grasp, the robot can simulate several approaches in its learned model and pick the one most likely to succeed. This is especially valuable for contact-rich tasks where real-world trial and error is slow and risky.
- World models also connect naturally to **sim-to-real**: a world model trained on real data is effectively a learned simulator that automatically captures real-world physics, bypassing the sim-to-real gap entirely. The model may be less accurate than a hand-built simulator for well-understood scenarios, but it captures effects (friction, deformation, contact dynamics) that hand-built simulators often get wrong.
- The **JEPA** (Joint Embedding Predictive Architecture, introduced in chapter 10) offers an alternative to pixel-level prediction. Instead of predicting exact future observations, JEPA predicts in embedding space: “the latent representation of the next state will be close to this vector.” This avoids the difficulty of

predicting pixel-perfect futures (which is both unnecessary and computationally wasteful) and focuses on predicting the aspects of the future that matter for decision-making.

- The limitation of world models is **compounding prediction error**. Small inaccuracies in the transition model accumulate over long rollouts, causing imagined trajectories to diverge from reality. Mitigations include short imagination horizons, ensemble models (using uncertainty to detect when predictions become unreliable), and periodically grounding the model with fresh real-world data.

## 52.6 Manipulation

---

- **Manipulation** is the art of using a robot's end-effector to interact with objects: picking, placing, pushing, inserting, assembling.
- **Grasping** is the foundational manipulation skill. The goal is to find a stable grasp pose: a position and orientation for the gripper that will securely hold the object.
- **Analytical grasp planning** uses physics. A grasp is stable if the contact forces can resist external wrenches (forces and torques). For a parallel-jaw gripper, the simplest criterion is the **force closure** condition: the contact normals must span all directions of force, so the grasp can resist any disturbance. This involves checking the rank of the grasp wrench matrix, a direct application of the rank concept from chapter 2.
- **Data-driven grasping** learns to predict grasp success from sensory input. Given a depth image of objects on a table, a network predicts a grasp quality score for each candidate gripper pose. **GraspNet** and similar architectures use point cloud encoders (PointNet-style, chapter 8) to predict 6-DoF grasp poses (position + orientation) with confidence scores.
- **Dexterous manipulation** goes beyond simple pick-and-place. A multi-fingered hand has 20+ DoF and can perform tasks like in-hand rotation (spinning a pen between fingers), tool use, and delicate assembly. The state space is enormous and contacts are complex, making this one of the hardest problems in robotics.
- Learning dexterous manipulation often uses reinforcement learning (chapter 6) in simulation with heavy domain randomisation. OpenAI's work on Rubik's

cube solving with a Shadow hand trained PPO policies in simulation with randomised physics, achieving transfer to a real robot hand.

- **Contact-rich tasks** like peg-in-hole insertion or wiping a surface require the robot to maintain controlled contact with the environment. These tasks demand force sensing and compliant control (impedance control), and they are difficult to simulate accurately because contact physics is notoriously hard to model.

## 52.7 Locomotion

---

- Locomotion is moving the robot's body through the world: walking, running, climbing, swimming. The key difference from manipulation is that the robot must maintain balance while moving, and the contact points with the ground change over time.
- **Legged locomotion** is challenging because it is inherently unstable. A bipedal robot (humanoid) standing on one leg during a step is like an inverted pendulum. The centre of mass must stay above the support polygon (the convex hull of the feet in contact with the ground), or the robot falls.
- The **Zero Moment Point (ZMP)** is the point on the ground where the net torque from gravity and inertial forces is zero. If the ZMP stays inside the support polygon, the robot will not tip over. Traditional humanoid controllers (like Honda's ASIMO) plan trajectories that keep the ZMP within bounds.
- **Central Pattern Generators (CPGs)** are oscillator-based controllers inspired by biology. Animals generate rhythmic locomotion patterns (walking, trotting, galloping) using neural circuits in the spinal cord, without constant brain involvement. CPG models use coupled differential equations:

$$\ddot{\phi}_i = \omega_i^2 + \sum_j w_{ij} \sin(\phi_j - \phi_i - \psi_{ij})$$

$$\ddot{\phi}_i = \omega_i^2 + \sum_j$$

$$w_{ij} \sin(\phi_j - \phi_i - \psi_{ij})$$

- where  $\phi_i$  is the phase of oscillator  $i$ ,  $\omega_i$  is the natural frequency,  $w_{ij}$  is the coupling strength, and  $\psi_{ij}$  is the desired phase offset. Different phase relationships produce different gaits: all legs in sync (bound), alternating pairs

(trot), sequential (walk). The sine coupling naturally synchronises the oscillators, analogous to how Fourier series (chapter 3) decompose motion into frequency components.

- **Reinforcement learning for locomotion** has become the dominant approach for agile quadruped and humanoid robots. The robot learns a policy  $\pi(\mathbf{a} \mid \mathbf{o})$  through trial and error in simulation (chapter 6), with rewards for forward velocity, stability, and energy efficiency, and penalties for falling, joint limit violations, and jerky motions.
- The key insight from recent work (e.g., by Agility Robotics, Boston Dynamics, and academic labs) is that RL-trained locomotion policies are far more robust than hand-designed controllers. They naturally learn to recover from pushes, adapt to terrain changes, and handle situations that no engineer anticipated. Training typically uses PPO (chapter 6) with domain randomisation.
- **Quadruped robots** (like Boston Dynamics Spot or Unitree Go2) have become the workhorse of legged robotics. Four legs provide inherent stability (a tripod of three legs can always support the body while one leg moves). RL policies for quadrupeds achieve impressive results: running at 3+ m/s, climbing stairs, navigating rocky terrain, and recovering from kicks.
- **Humanoid locomotion** is harder because bipeds have a smaller support polygon and higher centre of mass. Recent advances (Tesla Optimus, Figure, Unitree H1) use RL trained in simulation with careful reward shaping. The humanoid must learn not just to walk but to coordinate arm swings for balance, navigate uneven surfaces, and recover from perturbations.

## 52.8 Safety in Robot Learning

---

- A robot that explores randomly to learn (as in RL) may damage itself, its environment, or nearby humans. **Safe robot learning** constrains exploration to avoid catastrophic outcomes.
- **Constrained RL** adds safety constraints to the MDP (chapter 6). The objective becomes: maximise reward subject to  $J_c(\pi) \leq d$ , where  $J_c$  is the expected cumulative cost (e.g., collision events) and  $d$  is the maximum allowable cost. Algorithms like Constrained Policy Optimisation (CPO) extend PPO to handle these constraints.



- **Safety envelopes** define hard boundaries that the robot must never cross, regardless of what the learned policy says. A safety controller monitors the robot's state and overrides the learned policy when a constraint is about to be violated (e.g., approaching a joint limit, moving too fast near a human, or exceeding a force threshold). This is a layered architecture: the learning algorithm handles performance, and the safety layer handles constraints.
- **Risk-aware planning** explicitly models uncertainty in the environment and the robot's own state estimate. Instead of planning for the most likely outcome, it plans for the worst case within a confidence bound. This connects to the condition number concept (chapter 2): a well-conditioned system is robust to perturbations, and risk-aware planning seeks control strategies that remain safe under perturbation.

## 52.9 Coding Tasks (use CoLab or notebook)

1. Implement forward kinematics for a simple 2-link planar robot arm. Compute and visualise the end-effector position for different joint angles.

```
import jax.numpy as jnp
import matplotlib.pyplot as plt

def forward_kinematics(q1, q2, l1=1.0, l2=0.8):
 """Compute joint and end-effector positions for a 2-link arm."""
 x1 = l1 * jnp.cos(q1)
 y1 = l1 * jnp.sin(q1)
 x2 = x1 + l2 * jnp.cos(q1 + q2)
 y2 = y1 + l2 * jnp.sin(q1 + q2)
 return jnp.array([0, x1, x2]), jnp.array([0, y1, y2])

fig, ax = plt.subplots(figsize=(6, 6))
configs = [(0.5, 0.3), (1.0, -0.5), (1.5, 1.0), (2.0, -1.5)]
colors = ["#e74c3c", "#3498db", "#27ae60", "#9b59b6"]

for (q1, q2), c in zip(configs, colors):
 xs, ys = forward_kinematics(q1, q2)
 ax.plot(xs, ys, "o-", color=c, linewidth=2, markersize=6,
 label=f"q=({q1:.1f}, {q2:.1f})")

ax.set_xlim(-2, 2); ax.set_ylim(-2, 2)
ax.set_aspect("equal"); ax.grid(True); ax.legend()
```

```
ax.set_title("2-Link Robot Arm: Forward Kinematics")
plt.show()
```

1. Implement inverse kinematics using the Jacobian pseudo-inverse. Start from a random configuration and iteratively move the end-effector to a target.

```
import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

l1, l2 = 1.0, 0.8

def end_effector(q):
 x = l1 * jnp.cos(q[0]) + l2 * jnp.cos(q[0] + q[1])
 y = l1 * jnp.sin(q[0]) + l2 * jnp.sin(q[0] + q[1])
 return jnp.array([x, y])

jacobian_fn = jax.jacobian(end_effector)

target = jnp.array([0.5, 1.2])
q = jnp.array([0.1, 0.1])
trajectory = [end_effector(q)]

for _ in range(50):
 pos = end_effector(q)
 error = target - pos
 if jnp.linalg.norm(error) < 1e-4:
 break
 J = jacobian_fn(q)
 # Damped pseudo-inverse to handle near-singularities
 dq = J.T @ jnp.linalg.solve(J @ J.T + 0.01 * jnp.eye(2), error)
 q = q + dq
 trajectory.append(end_effector(q))

traj = jnp.stack(trajectory)
plt.plot(traj[:, 0], traj[:, 1], "b.-", label="end-effector path")
plt.plot(*target, "r*", markersize=15, label="target")
plt.gca().set_aspect("equal"); plt.grid(True); plt.legend()
plt.title(f"IK converged in {len(trajectory)-1} steps")
plt.show()
```

1. Simulate a simple PID controller tracking a desired joint trajectory. Observe the effect of tuning the gains.

```

import jax.numpy as jnp
import matplotlib.pyplot as plt

Desired trajectory: smooth sinusoidal motion
dt = 0.01
t = jnp.arange(0, 5, dt)
q_desired = jnp.sin(2 * t)

Simulate second-order dynamics: $m * q_{ddot} + b * q_{dot} = \tau$
m, b_damp = 1.0, 0.5

for Kp, Kd, Ki, label in [(10, 5, 0, "PD only"), (10, 5, 2, "PID"), (50, 10, 2,
"Aggressive PID")]:
 q, q_dot, integral = 0.0, 0.0, 0.0
 qs = []
 for i in range(len(t)):
 error = q_desired[i] - q
 integral += error * dt
 d_error = -q_dot # derivative of error (desired velocity is known but
simplified here)
 tau = Kp * error + Kd * d_error + Ki * integral
 q_ddot = (tau - b_damp * q_dot) / m
 q_dot += q_ddot * dt
 q += q_dot * dt
 qs.append(float(q))

 plt.plot(t, qs, label=label)

plt.plot(t, q_desired, "k--", label="desired", linewidth=2)
plt.xlabel("Time (s)"); plt.ylabel("Joint angle")
plt.legend(); plt.title("PID Controller Tracking")
plt.show()

```

## 53 Vision-Language-Action Models

Vision-Language-Action models (VLAs) unify seeing, understanding language, and acting into a single neural network. This file covers the VLA architecture, action tokenisation, RT-2, Octo, OpenVLA, pretraining strategies, generalisation, embodiment-agnostic models, and benchmarks

- In the previous files, we covered perception (sensing the world) and robot learning (controlling the body). Traditionally, these were separate pipelines: a perception module detects objects, a language module interprets commands,

and a control module generates actions. Each module was designed, trained, and debugged independently.

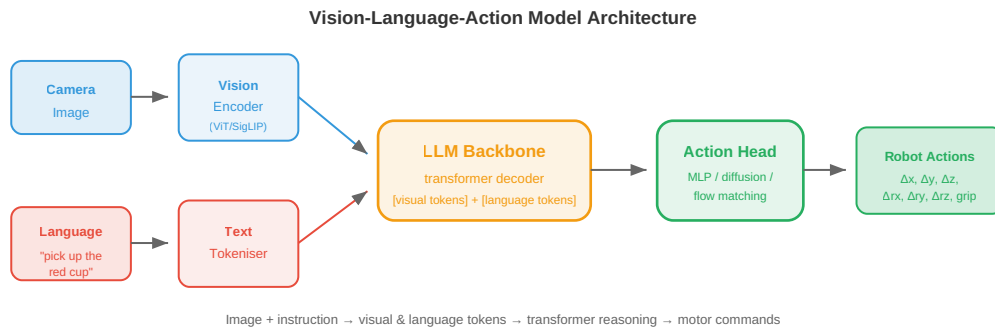
- **Vision-Language-Action models (VLAs)** collapse this pipeline into a single neural network. The model takes in images (vision), a natural language instruction (language), and outputs motor commands (action). One model, end-to-end.
- This follows the same unification trend we saw in chapter 10: just as multimodal models merged vision and language understanding into one architecture, VLAs extend this to physical action. The insight is that language provides a natural, flexible interface for specifying tasks ( “pick up the red cup and place it on the shelf” ), and large pretrained vision-language models already understand both images and instructions.

## 53.1 From Vision-Language to Action

---

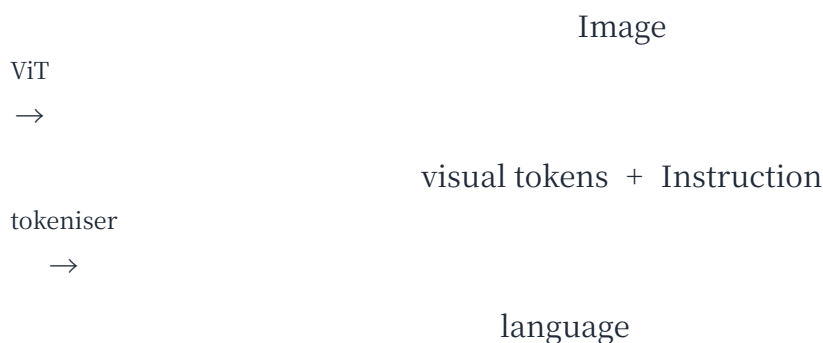
- Recall from chapter 10 that **Vision-Language Models (VLMs)** like LLaVA and Flamingo take an image and text as input and produce text as output. They understand scenes, answer questions, and follow instructions, all in language.
- VLAs ask: what if the output is not text but **robot actions**? Instead of generating “The red cup is on the left side of the table,” the model generates a sequence of motor commands that moves the arm to grasp that cup.
- The key architectural insight is that actions can be represented as tokens, just like words. If a VLM generates language token by token using next-token prediction, a VLA generates action tokens in the same way. The transformer does not fundamentally care whether the output token means “cup” or “move gripper 2cm forward.”
- This reframes robot control as a sequence modelling problem, which transformers excel at (chapter 7). The model learns the mapping: (image observation, language instruction) → (sequence of action tokens).

## 53.2 VLA Architecture



VLA architecture: camera image and language instruction are encoded into tokens, processed by an LLM backbone, and decoded into robot actions

- A typical VLA has three components:
  - **Vision encoder:** processes camera images into visual tokens. Usually a pretrained ViT (chapter 8) or SigLIP encoder (chapter 10). The image is split into patches, each embedded as a token, exactly as in standard vision transformers.
  - **Language model backbone:** a pretrained LLM (e.g., LLaMA, PaLM) that processes the interleaved sequence of visual tokens and language tokens. This is where the reasoning happens: the model understands “pick up the **red** cup” by attending to both the instruction and the visual features.
  - **Action head:** maps the LLM’s output to robot actions. This can be a simple MLP that maps the last hidden state to continuous action values, or a tokenisation scheme that converts actions into discrete tokens predicted by the LLM’s existing vocabulary.
- The architecture looks like:



LLM  
tokens →

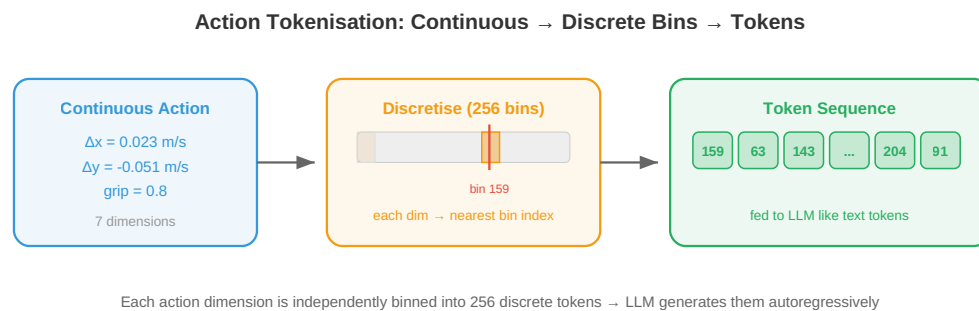
tokens

action

- The visual tokens and language tokens are concatenated (or interleaved) and fed into the transformer backbone, which produces action tokens autoregressively. This is the same architecture as a VLM (chapter 10), but the output modality is action instead of text.

## 53.3 Action Tokenisation

- Robot actions are continuous: joint velocities, end-effector positions, gripper width. These must be converted into discrete tokens for the LLM to generate them.



Action tokenisation: continuous action values are binned into discrete indices that the LLM generates as tokens

- The simplest approach is **uniform discretisation**. Each action dimension is divided into  $N$  bins spanning the range of valid values. For example, if the  $x$ -velocity ranges from  $-0.1$  to  $0.1$  m/s and we use 256 bins, each bin represents  $\frac{0.2}{256} \approx 0.8$  mm/s. An action value is mapped to its nearest bin index, which becomes a token.
- With 7 action dimensions (6 DoF + gripper) and 256 bins each, the action vocabulary has  $7 \times 256 = 1792$  tokens. These are added to the LLM's existing text vocabulary. The model generates one action token per dimension, autoregressively, just like generating words.
- **Action chunking** predicts multiple future timesteps at once rather than a single action. If the chunk size is  $H$ , the model outputs  $H \times d$  tokens (where  $d$  is the action dimensionality). This is critical for smooth, temporally coherent

motions. Predicting one step at a time can produce jerky behaviour because each prediction is independent. Chunking forces the model to plan a short trajectory, capturing temporal structure.

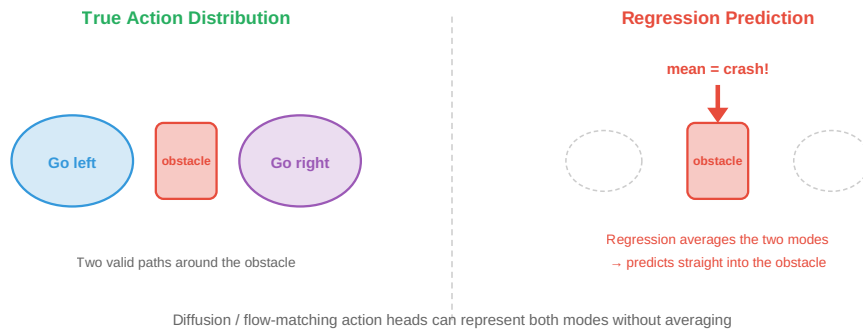
- More sophisticated approaches use **learned tokenisation** via VQ-VAE (chapter 10). A VQ-VAE encoder maps a sequence of continuous actions to a sequence of discrete codebook indices, and the decoder reconstructs the continuous actions from these indices. The LLM then generates codebook indices rather than uniformly binned values. This is analogous to how image tokenisers (chapter 10) compress visual information into a compact discrete code.

## 53.4 Key VLA Models

---

- **RT-2** (Robotic Transformer 2, Google DeepMind) was the first large-scale VLA. It takes a pretrained VLM (PaLM-E or PaLI-X, up to 55B parameters) and fine-tunes it on robot demonstration data. Actions are represented as text strings: the token sequence “1 128 91 241 5 101 127” encodes a 7-dimensional action (each number is a bin index).
- RT-2 demonstrated a remarkable property: **emergent capabilities** from the VLM backbone transfer to robotics. The model can follow instructions involving concepts it never saw in robot data (e.g., “move the banana to the country that starts with A” requires visual object recognition + world knowledge + action). The VLM’s language understanding and visual reasoning “come for free.”
- The limitation of RT-2 is that it was trained on data from a single robot embodiment (a specific arm with a specific gripper). It does not generalise to different robots.
- **Octo** (UC Berkeley) is an open-source, **embodiment-agnostic** VLA designed to work across different robot platforms. The key innovations are:
  - A **diffusion action head** instead of autoregressive token prediction. The action head takes the transformer’s output and produces actions via a denoising diffusion process (chapter 8). This naturally handles multimodal action distributions (see diagram below), where there are multiple valid ways to complete a task.

### Multimodal Actions: Why Regression Averages Fail



Multimodal action distributions: regression averages two valid paths into an invalid one through the obstacle

- **Flexible observation and action spaces**: Octo uses task-specific tokenisers for different robot configurations. It was pretrained on the Open X-Embodiment dataset, which contains demonstrations from 22 different robot embodiments.
- **Efficient fine-tuning**: Octo can be fine-tuned to a new robot with as few as 100 demonstrations, making it practical for labs with limited data.

- **OpenVLA** (Stanford, UC Berkeley) takes the approach of fine-tuning an existing open-source VLM (Llama-based) for robotics. It uses a 7B parameter backbone, uniform action tokenisation (256 bins per dimension), and trains on Open X-Embodiment data. Its strength is simplicity: the architecture is a standard VLM with action tokens appended to the vocabulary, making it easy to train and deploy with existing LLM infrastructure.
- **$\pi_0$**  (Physical Intelligence) represents the state of the art. It uses a pretrained VLM backbone with a **flow matching** action head (chapter 8). Flow matching generates actions by learning a velocity field that transports noise to action distributions, producing smooth, temporally coherent action trajectories.  $\pi_0$  demonstrated remarkable generality, performing tasks across multiple robot embodiments including bimanual manipulation and dexterous hand control.



## 53.5 Pretraining Recipes

---

- VLAs benefit enormously from pretrained VLM backbones, which already understand visual scenes and language. The training pipeline typically follows stages:
  1. **VLM pretraining**: train (or use off-the-shelf) a vision-language model on billions of image-text pairs from the internet (CLIP, SigLIP, LLaVA-style training as covered in chapter 10).
  2. **Robot data co-training**: fine-tune the VLM on a mixture of internet data and robot demonstration data. The internet data prevents catastrophic forgetting of visual and language understanding, while the robot data teaches action generation. The mixing ratio matters: too much robot data degrades language understanding, too little fails to learn actions.
  3. **Task-specific fine-tuning**: optional fine-tuning on demonstrations for a specific task or robot, often with LoRA (chapter 10) to keep the number of trainable parameters small.
- The amount of robot data is orders of magnitude smaller than internet data. A VLM might be pretrained on billions of images, but the largest robot datasets (Open X-Embodiment) contain only millions of frames across all embodiments. This data scarcity is why starting from a pretrained VLM is essential: the visual and language representations transfer, and only the action mapping needs to be learned from limited robot data.

## 53.6 Generalisation

---

- The promise of VLAs is **generalisation**: performing tasks not seen during training, with objects not seen before, in environments not seen before, following instructions not seen before.
- VLAs generalise along several axes:
  - **Novel objects**: the VLM backbone recognises objects from internet pretraining. If the model knows what a “screwdriver” looks like from web images, it can manipulate one even if no robot demonstration ever included a screwdriver.

- **Novel instructions:** compositional language understanding allows the model to follow new combinations of known concepts. “Stack the blue block on the green block” works even if training only showed stacking red blocks, because the model understands colour adjectives from language pretraining.
- **Novel environments:** to a degree, VLAs transfer across visual domains (different tables, lighting, backgrounds) because the vision encoder is pretrained on diverse web images. But this has limits: a robot trained in a lab may struggle in a cluttered kitchen.
- **Novel embodiments:** this is the hardest axis. Different robots have different action spaces (joint angles vs. end-effector velocities), different sensors (wrist cameras vs. overhead cameras), and different physical capabilities. Embodiment-agnostic models like Octo and  $\pi_0$  address this with flexible tokenisers and pretraining across many robot types.
- Generalisation is evaluated on **held-out tasks**: the robot is asked to perform tasks it was never trained on. Success rates of 50–80% on novel tasks are considered strong results, compared to >90% on in-distribution tasks. The gap is shrinking as models scale and robot datasets grow.

## 53.7 Embodiment-Agnostic Models

---

- The field is moving towards **one model, many robots**. Instead of training a separate policy for each robot, a single VLA handles multiple embodiments.
- This requires solving the **action space mismatch** problem. A 7-DoF arm with a parallel-jaw gripper has 7 action dimensions. A bimanual setup has 14. A quadruped has 12. A humanoid has 30+. The action tokenisation must be flexible enough to handle all of these.
- Solutions include:
  - **Padded action vectors:** use the largest action space and pad smaller ones with zeros.
  - **Per-embodiment action heads:** a shared transformer backbone with separate small MLPs for each robot type.
  - **Normalised action representations:** express all actions in a common frame (e.g., end-effector velocity in the world frame) so that different

robots producing similar end-effector motions share the same action tokens.

- The shared backbone learns general visual and language understanding, plus common manipulation strategies (approach from above, align with the object, close the gripper). The embodiment-specific components only need to translate these high-level strategies into specific motor commands.

## 53.8 Benchmarks and Evaluation

---

- Evaluating VLAs is uniquely challenging because it requires physical robot experiments (or high-fidelity simulation).
- **SIMPLER** (Simulated Manipulation Policy Evaluation for Robot learning) provides standardised simulated environments for comparing VLA performance without physical hardware. It correlates well with real-world success rates and enables reproducible benchmarking.
- **Real-world evaluation** remains the gold standard. The typical protocol:
  1. Define a set of tasks with clear success criteria (object reaches target position, correct object selected, task completed within time limit).
  2. Run N trials per task (typically 10–50).
  3. Report success rate with confidence intervals.
  4. Include held-out (never-trained-on) tasks to measure generalisation.
- The **Open X-Embodiment** dataset and benchmark aggregate robot data from 22 institutions across multiple robot platforms. It provides a standardised format for sharing demonstrations and a common evaluation suite for cross-embodiment transfer.

## 53.9 Coding Tasks (use CoLab or notebook)

---

1. Implement action tokenisation: discretise continuous actions into bins and reconstruct them. Observe the quantisation error as a function of bin count.

```
import jax.numpy as jnp

Continuous action: 7 dimensions (6 DoF + gripper)
action_true = jnp.array([0.023, -0.051, 0.012, 0.1, -0.03, 0.005, 0.8])
action_min = jnp.array([-0.1, -0.1, -0.1, -0.5, -0.5, -0.5, 0.0])
```

```

action_max = jnp.array([0.1, 0.1, 0.1, 0.5, 0.5, 0.5, 1.0])

for n_bins in [16, 64, 256, 1024]:
 # Tokenise: map continuous value to bin index
 normalised = (action_true - action_min) / (action_max - action_min)
 tokens = jnp.clip((normalised * n_bins).astype(int), 0, n_bins - 1)

 # Detokenise: map bin index back to continuous value
 reconstructed = (tokens + 0.5) / n_bins * (action_max - action_min) +
action_min

 error = jnp.linalg.norm(action_true - reconstructed)
 print(f"bins={n_bins:4d} tokens={tokens} error={error:.6f}")

```

1. Simulate action chunking vs single-step prediction. Generate a smooth trajectory, add noise to single-step predictions, and compare with chunk-based prediction.

```

import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

Ground truth smooth trajectory (e.g., reaching motion)
t = jnp.linspace(0, 2 * jnp.pi, 100)
gt_x = jnp.sin(t)
gt_y = 1 - jnp.cos(t)

Single-step: each prediction has independent noise
rng = jax.random.PRNGKey(42)
noise_ss = jax.random.normal(rng, (100, 2)) * 0.05
single_step = jnp.stack([gt_x, gt_y], axis=1) + noise_ss
Cumulative drift from single-step errors
single_step_cumulative = jnp.cumsum(noise_ss, axis=0) * 0.3 + jnp.stack([gt_x,
gt_y], axis=1)

Chunked (chunk_size=10): noise is correlated within chunks, smoother
chunk_size = 10
rng2 = jax.random.PRNGKey(7)
chunks = []
for i in range(0, 100, chunk_size):
 chunk_noise = jax.random.normal(jax.random.fold_in(rng2, i), (2,)) * 0.05
 chunk = jnp.stack([gt_x[i:i+chunk_size], gt_y[i:i+chunk_size]], axis=1)
 chunks.append(chunk + chunk_noise)
chunked = jnp.concatenate(chunks, axis=0)

plt.figure(figsize=(8, 4))

```

```
plt.plot(gt_x, gt_y, "k-", linewidth=2, label="Ground truth")
plt.plot(single_step_cumulative[:, 0], single_step_cumulative[:, 1],
 "r-", alpha=0.7, label="Single-step (drifts)")
plt.plot(chunked[:, 0], chunked[:, 1], "b-", alpha=0.7, label="Chunked (stable)")
plt.legend(); plt.axis("equal"); plt.grid(True)
plt.title("Action Chunking vs Single-Step Prediction")
plt.show()
```

1. Visualise how a VLA's action distribution can be multimodal. Use a simple 2D mixture of Gaussians to show why diffusion/flow-matching action heads are preferable to regression.

```
import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

Two valid ways to reach around an obstacle: left or right
rng = jax.random.PRNGKey(0)
k1, k2 = jax.random.split(rng)

model1 = jax.random.normal(k1, (200, 2)) * 0.15 + jnp.array([-1.0, 0.5])
model2 = jax.random.normal(k2, (200, 2)) * 0.15 + jnp.array([1.0, 0.5])
samples = jnp.concatenate([model1, model2])

Regression predicts the mean = average of modes (invalid!)
mean_pred = samples.mean(axis=0)

plt.figure(figsize=(6, 5))
plt.scatter(samples[:, 0], samples[:, 1], s=5, alpha=0.5, label="True action
distribution")
plt.plot(*mean_pred, "rx", markersize=15, markeredgewidth=3, label="Regression
mean (invalid!)")
plt.plot(-1, 0.5, "g^", markersize=12, label="Mode 1 (go left)")
plt.plot(1, 0.5, "b^", markersize=12, label="Mode 2 (go right)")
plt.legend(); plt.grid(True)
plt.title("Multimodal Actions: Why Regression Fails")
plt.xlabel("Action dim 1"); plt.ylabel("Action dim 2")
plt.show()
```

## 54 Self-Driving Cars

Self-driving cars are the most commercially advanced autonomous systems, integrating perception, prediction, planning, and control into a single vehicle. This

file covers the autonomous driving stack, HD maps, motion prediction, planning, end-to-end driving, simulation, safety standards, and levels of autonomy

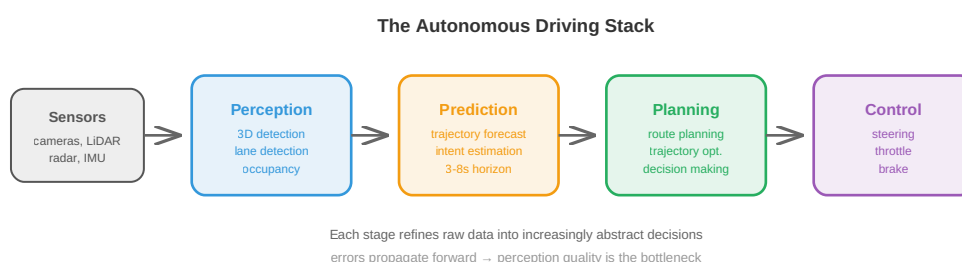
- Self-driving cars are arguably the hardest robotics problem being attempted at scale. Unlike a factory robot that operates in a controlled environment, a self-driving car must handle an open world: unpredictable human drivers, pedestrians who jaywalk, construction zones that appear overnight, and weather that changes by the minute.
- The stakes are also uniquely high. A self-driving car operates at highway speeds among vulnerable road users. The error tolerance is near zero for safety-critical failures.

## 54.1 The Autonomous Driving Stack

---

- The classical self-driving architecture is a **modular pipeline** with four stages, each feeding into the next:

Perception → Prediction → Planning → Control



The autonomous driving stack: sensors feed perception, which feeds prediction, planning, and finally control

- **Perception** (covered in file 1 of this chapter) processes raw sensor data into a structured scene representation: detected objects with 3D positions, velocities, and class labels; lane markings; traffic lights; drivable surface boundaries.
- **Prediction** forecasts how other agents (vehicles, pedestrians, cyclists) will move in the future. Given the current state of the scene, the prediction module outputs trajectories for each agent over a time horizon (typically 3-8 seconds into the future).
- **Planning** decides what the ego vehicle should do: which path to follow, when to change lanes, when to yield, when to accelerate or brake. It takes the predicted scene and produces a trajectory for the ego vehicle that is safe, comfortable, and makes progress towards the destination.

- **Control** converts the planned trajectory into actuator commands: steering angle, throttle, and brake. This is the lowest level, translating the abstract trajectory into physical motion.
- The modular design has clear engineering advantages: each module can be developed, tested, and improved independently. But it also has weaknesses: errors propagate downstream (a missed detection is invisible to the planner), and information is lost at each interface (the planner sees bounding boxes, not the rich sensor data that produced them).

## 54.2 HD Maps

---

- **High-Definition (HD) maps** are detailed, centimetre-accurate digital maps that encode the road structure: lane boundaries, lane connectivity (which lane connects to which at an intersection), traffic sign positions, speed limits, crosswalk locations, and road surface elevation.
- HD maps provide a strong prior for the driving task. The perception module does not need to discover lane boundaries from scratch every frame; it just needs to localise the vehicle within the map and verify that reality matches the stored structure. This dramatically simplifies planning.
- Building HD maps requires specialised survey vehicles equipped with high-end LiDAR, cameras, and RTK-GPS. The maps must be maintained and updated as roads change. This is expensive and does not scale easily to every road on Earth.
- **Mapless driving** (also called “online mapping” ) aims to eliminate the dependency on pre-built HD maps. Instead, the vehicle constructs a local map in real-time from its sensors. Models like **MapTR** and **MapTRv2** use transformer architectures to predict vectorised map elements (lane centrelines, road boundaries, pedestrian crossings) directly from camera images, outputting polylines as ordered point sequences.
- The mapless approach trades map accuracy for scalability: any road the car can drive on, it can map. But it requires the perception system to be robust enough to detect all relevant road structure in real-time, including in complex intersections, highway ramps, and construction zones.
- In practice, many systems use a hybrid approach: a lightweight map with coarse road topology (from existing map providers), enriched in real-time by the vehicle’ s sensors.

## 54.3 Motion Prediction

- Predicting where other road users will go is one of the hardest subproblems in self-driving. Humans are unpredictable, intentions are hidden, and the space of possible futures branches rapidly.
- The input to a prediction model is the **scene context**: the positions and velocities of all detected agents over the recent past (typically 1-2 seconds of history), plus static context (lane geometry, traffic signals, road boundaries).
- The output is a set of **predicted trajectories** for each agent, typically covering 3-8 seconds into the future. Since the future is uncertain, good prediction models output multiple possible trajectories with associated probabilities, not a single point estimate.
- **Trajectory forecasting** as a regression problem: predict the future (x, y) coordinates of each agent at discrete future timesteps. The loss is typically the minimum average displacement error (minADE) over the K predicted trajectories:

$$\min_{k \in \{1, \dots, K\}} \frac{1}{T} \sum_{t=1}^T \left\| \hat{\mathbf{p}}_t^{(k)} - \mathbf{p}_t \right\|_2$$

- This is a “best of K” metric: the model gets credit if any of its K predictions is close to the ground truth. This encourages diverse, multimodal predictions.
- **Social forces** model pedestrian behaviour as a dynamical system where each person experiences attractive forces (towards their goal) and repulsive forces (away from other pedestrians and obstacles). The acceleration of person i is:



$$\mathbf{a}_i = \frac{\mathbf{v}_i^{\text{desired}} - \mathbf{v}_i}{\tau} + \sum_{j \neq i} \mathbf{f}_{ij}^{\text{repulsive}} + \sum_{\text{walls}} \mathbf{f}_{\text{wall}}$$

- This is a system of differential equations similar to the robot dynamics equation from file 2 of this chapter. The model is elegant but relies on hand-tuned force parameters and struggles with complex multi-agent interactions.
- **Graph Neural Networks (GNNs)** for prediction model the scene as a graph: each agent is a node, and edges represent spatial relationships (proximity, lane sharing). Message passing between nodes captures interactions: “this car is yielding to that pedestrian” or “these two vehicles are merging into the same lane.”
- Modern prediction architectures (e.g., **MTR**, **QCNet**) use transformer-based models that jointly reason about agent history, map context, and agent-agent interactions. Agents attend to relevant map features (their current lane, upcoming intersections) and to other agents (the car in front, the pedestrian at the crosswalk) via cross-attention. The output is a set of trajectory hypotheses generated autoregressively or via a mixture model.
- **Goal-conditioned prediction** first predicts where an agent is likely to go (a set of candidate goal points, like lane endpoints or intersection exits), then predicts the trajectory to reach each goal. This decomposes the problem into “where” (discrete, manageable) and “how” (continuous path given the goal), making the multimodal prediction problem more tractable.

## 54.4 Planning

---

- Given the predicted scene, the planner must produce a trajectory for the ego vehicle. This is a constrained optimisation problem: find a trajectory that is safe, comfortable, efficient, and legal.
- **Rule-based planners** encode driving behaviour as a set of if-then rules: “if a pedestrian is in the crosswalk, yield,” “if the gap to the vehicle ahead is less than 2 seconds, do not change lanes,” “if approaching a red light, decelerate

to stop at the stop line.” These rules are interpretable and auditable, but they become unwieldy for complex scenarios (thousands of rules, many edge cases, interactions between rules).

- **Optimisation-based planners** formulate driving as trajectory optimisation. The ego trajectory is parameterised (e.g., as a sequence of  $(x, y, \theta, v)$  states at future timesteps) and an objective function is minimised:

$$\begin{aligned} \min_{\xi} & \\ & w_1 \cdot J_{\text{progress}}(\xi) \\ & \quad \sim \text{get to destination} \\ & + w_2 \cdot J_{\text{comfort}}(\xi) \\ & \quad \sim \text{smooth ride} \\ & + w_3 \cdot J_{\text{safety}}(\xi) \\ & \quad \sim \text{avoid collisions} \end{aligned}$$

subject to: kinematic constraints, speed limits, lane boundaries

- The progress term penalises deviations from the desired route. The comfort term penalises high lateral acceleration, jerk (derivative of acceleration), and abrupt steering, because passengers feel these. The safety term penalises proximity to other agents, using predicted trajectories to evaluate collision risk.
- This is constrained optimisation (chapter 3): minimise a cost function subject to inequality constraints. The weights  $w_1, w_2, w_3$  trade off competing objectives (aggressive driving is faster but less comfortable and less safe).
- **Learning-based planners** use neural networks trained on human driving data to generate trajectories. The model observes the scene and directly outputs a planned trajectory, learning the complex trade-offs implicitly from examples of expert human driving.
- The advantage is that human driving behaviour is captured holistically, including the subtle, hard-to-formalise aspects: how aggressively to merge, when to nudge forward at an intersection, how much space to give a cyclist. The disadvantage is the same distribution shift problem from imitation learning

(file 2): the model may behave unpredictably in situations not well-represented in training data.

## 54.5 End-to-End Driving

---

- **End-to-end driving** removes the modular boundaries entirely. A single neural network takes raw sensor inputs (camera images, LiDAR point clouds) and directly outputs driving commands (steering, throttle, brake) or a planned trajectory. No separate perception, prediction, or planning modules.
- The appeal is that the entire system is jointly optimised for the final task (safe driving), so no information is lost at module boundaries. The perception module learns to extract exactly the features the planner needs, rather than generic object detections that may not capture task-relevant details.
- **UniAD** (Unified Autonomous Driving) is a landmark end-to-end architecture. It processes multi-camera images through a BEV encoder, then applies a cascade of transformer-based modules: tracking, online mapping, motion prediction, occupancy prediction, and planning. While it has internal modules, they are all differentiable and trained jointly end-to-end, with the planning loss backpropagating through the entire network.
- The planning module in UniAD generates future ego-vehicle waypoints by attending to the predicted BEV features, predicted agent trajectories, and predicted occupancy. This is the multivariate chain rule (chapter 3) in action: gradients flow from the planning loss all the way back to the image encoder, telling the perception features how to be more useful for planning.
- More recent end-to-end approaches use VLA-style architectures (file 3 of this chapter). Models like **DriveVLM** take camera images and a navigation instruction (or route), and produce driving actions using a VLM backbone. This brings the benefits of large-scale pretraining (visual understanding, reasoning) directly into the driving stack.
- The tension in end-to-end driving is **interpretability**. A modular system can report “I detected a pedestrian at (x, y) and predicted they will cross” – the failure mode is diagnosable. An end-to-end system is a black box that produces a steering angle. When it fails, diagnosing why is difficult, which is a serious concern for safety certification.

## 54.6 World Models for Driving

---

- A **world model** learns to predict the future state of the driving scene given the current state and the ego vehicle's actions:  $p(s_{t+1} \mid s_t, a_t)$  (as introduced in chapter 10). In driving, this means generating realistic future frames or BEV layouts: “if I accelerate and steer left, the scene will look like this in 3 seconds.”
- World models offer two powerful capabilities for self-driving:
  - **Imagination-based planning**: instead of committing to an action and seeing what happens, the planner can “imagine” multiple candidate trajectories by rolling them out through the world model, evaluate each one for safety and comfort, and pick the best. This is model-based RL (covered in file 2 of this chapter) applied to driving.
  - **Learned simulation**: a world model trained on real driving data is effectively a data-driven simulator. It generates realistic scenarios (including rare edge cases) without the manual effort of building a hand-crafted simulator. Crucially, it captures the statistical patterns of real driving: how other drivers actually behave, how lighting changes, how rain affects visibility.
- **GAIA-1** (Wayve) is a generative world model for driving. Given a sequence of past camera frames and ego-vehicle actions, it autoregressively generates future video frames. It uses a video diffusion architecture conditioned on action inputs. The model learns to generate plausible futures: vehicles that obey traffic rules, pedestrians that walk on sidewalks, and traffic lights that transition correctly, all emergent from training data rather than programmed rules.
- **DriveDreamer** and **GenAD** take a similar approach but operate in BEV space rather than pixel space. Predicting future BEV layouts is more compact than generating full video frames (similar to how DreamerV3 in robotics predicts in latent space rather than pixel space, as discussed in file 2). The BEV world model predicts where all agents will be, what the road structure will look like, and where free space exists, and the planner uses this directly.
- **Neural closed-loop simulation** uses world models to replace hand-built simulators for testing. Given a real driving log as a starting point, the world model generates what would have happened if the ego vehicle had taken a

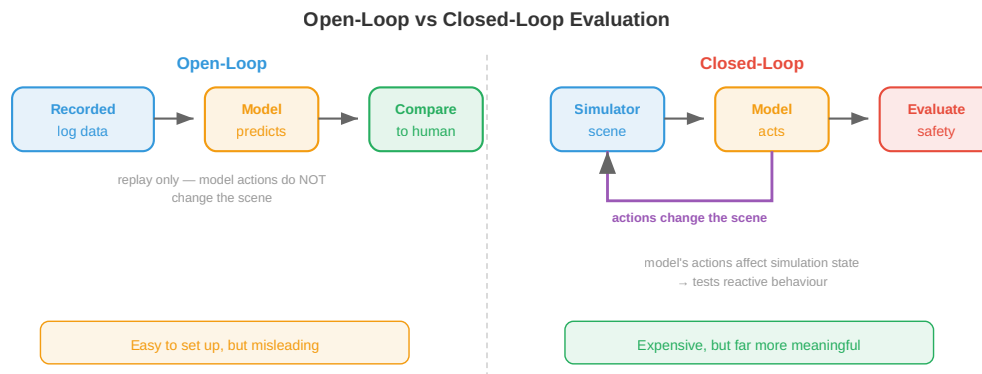
different action. This enables counterfactual evaluation: “what if I had braked 0.5 seconds later?” without ever needing to recreate the scenario physically.

- The connection to the **JEPA** framework (chapter 10) is natural here. Driving world models do not need to predict pixel-perfect futures (exact RGB values of every pixel). They need to predict the aspects that matter for planning: where are the agents, how fast are they moving, where is free space. Embedding-space prediction (JEPA-style) captures these semantically meaningful properties without wasting capacity on irrelevant visual details like exact cloud textures.
- The main challenge is **long-horizon fidelity**. World models accumulate errors over time: a small mistake in frame 2 shifts all subsequent frames. For driving, a 3-second prediction horizon is useful for tactical decisions (should I merge now?), but a 30-second horizon (needed for strategic decisions like route planning) remains unreliable. Current work mitigates this with re-anchoring (periodically resetting the model with real observations) and uncertainty estimation (flagging when predictions become unreliable).

## 54.7 Simulation

---

- Testing a self-driving car by driving on real roads is necessary but insufficient. Dangerous scenarios (near-collisions, edge cases) are rare, so testing by miles driven is inefficient. A car would need to drive hundreds of millions of miles to statistically demonstrate safety, which is infeasible.
- **Simulation** provides unlimited, controllable, and safe testing. Scenarios that are rare in the real world (a child running into the road, a tyre blowout, a sudden obstacle) can be tested millions of times in simulation.
- **CARLA** is an open-source driving simulator built on Unreal Engine. It provides realistic urban environments, dynamic weather, traffic agents, and sensor simulation (cameras, LiDAR, radar). Researchers use CARLA for training RL-based driving agents and evaluating perception algorithms.
- **nuPlan** (Motional) is a closed-loop planning benchmark. Unlike open-loop evaluation (replay logged data and compare the planner’s output to the human driver’s actual trajectory), closed-loop evaluation lets the planner’s decisions affect the simulation: if the planner decides to change lanes, the simulation evolves accordingly. This tests reactive behaviour, not just trajectory similarity.



Open-loop replays logs without interaction; closed-loop lets the model's actions change the simulation state

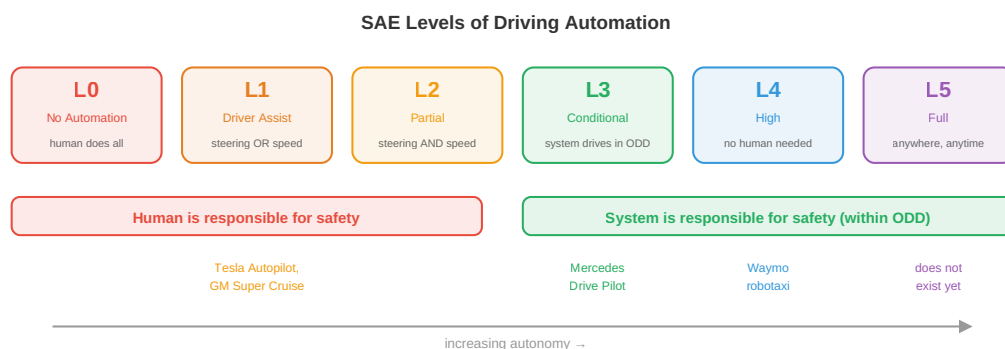
- The distinction between **open-loop** and **closed-loop** evaluation is critical:
  - Open-loop: replay a recorded scenario, compute how similar the model's output is to the human driver's actions. This is easy to set up but misleading: a model that always predicts "go straight" might have low error on highways but would crash at the first turn.
  - Closed-loop: the model's actions change the simulation state, and the simulation evolves in response. This tests the model's ability to recover from its own mistakes and react to dynamic situations. It is much more expensive but far more meaningful.
- **Scenario generation** creates test cases that stress the system. Adversarial scenarios (a vehicle suddenly braking, a pedestrian hidden behind a parked car) are generated by optimising for situations where the self-driving system performs worst. This is related to adversarial training in ML (chapter 6): finding the inputs that maximise the loss.

## 54.8 Safety

- Safety in self-driving is governed by engineering standards, not just ML metrics.
- **ISO 26262** (Functional Safety) is the automotive standard for safety-critical electronic systems. It defines **Automotive Safety Integrity Levels (ASILs)** from A (lowest) to D (highest), based on the severity, exposure, and controllability of potential hazards. A self-driving system's perception and planning components are typically ASIL-D, the highest level, requiring extensive verification, redundancy, and fail-safe design.

- **SOTIF** (Safety of the Intended Functionality, ISO 21448) addresses a different class of hazards: not hardware failures (which ISO 26262 covers), but situations where the system works as designed but still produces an unsafe outcome. A perception model that misclassifies a white truck as sky (a real incident) is a SOTIF issue: the hardware works fine, but the algorithm's limitation causes a hazard.
- The **Operational Design Domain (ODD)** defines the conditions under which the self-driving system is designed to operate: specific geographic areas, road types (highway only, urban, both), weather conditions (no heavy snow), speed ranges, and time of day. Operating outside the ODD is not permitted: if the system cannot handle snow, it must not drive in snow.
- **Fail-safe vs fail-operational** design:
  - Fail-safe: when a fault is detected, the system transitions to a safe state (e.g., pulls over and stops). This is the minimum requirement.
  - Fail-operational: the system continues to operate safely despite the fault, using redundant components. A self-driving car with redundant steering, braking, and compute can survive a single component failure and still drive to a safe location.
- **Redundancy** is fundamental. Critical perception sensors are duplicated: multiple cameras covering overlapping fields of view, both LiDAR and radar providing independent depth measurements, dual compute platforms running the same software. If any single component fails, the others provide sufficient information to drive safely.

## 54.9 Levels of Autonomy



SAE levels of autonomy from L0 (no automation) to L5 (full automation), showing where responsibility shifts from human to system

- The **SAE J3016** standard defines six levels of driving automation, from 0 (no automation) to 5 (full automation):
  - **Level 0 (No Automation)**: the human does everything. The system may provide warnings (lane departure alert) but does not control the vehicle.
  - **Level 1 (Driver Assistance)**: the system controls either steering or speed, but not both. Adaptive cruise control (maintains speed and following distance) or lane keeping assist (keeps the car centred in the lane) are Level 1.
  - **Level 2 (Partial Automation)**: the system controls both steering and speed simultaneously, but the human must monitor at all times and be ready to take over. Tesla Autopilot, GM Super Cruise, and most current “self-driving” features are Level 2. The human is still the responsible driver.
  - **Level 3 (Conditional Automation)**: the system drives and monitors the environment, but only in specific conditions (the ODD). The human can disengage but must be ready to take over when the system requests it (with a time buffer, typically 10+ seconds). Mercedes Drive Pilot (on certain highways, below 60 km/h) is the first certified Level 3 system.
  - **Level 4 (High Automation)**: the system drives and handles all situations within its ODD, with no human intervention needed. If it encounters a situation outside its ODD, it can safely stop itself. Waymo’s robotaxi service operates at Level 4 in specific geographic areas.
  - **Level 5 (Full Automation)**: the system drives everywhere a human can, in all conditions. No steering wheel or pedals needed. This does not exist yet.
- The critical distinction is **who is responsible for safety**. At Levels 0-2, the human is responsible. At Levels 3-5, the system is responsible (within its ODD). This has profound legal, insurance, and ethical implications.
- The current industry state is a mix of Level 2 (widely deployed), Level 3 (beginning deployment), and Level 4 (limited geographic deployment). Level 5 remains a long-term research goal.



## 54.10 Coding Tasks (use CoLab or notebook)

1. Implement a simple trajectory optimisation planner. Given a start position, goal, and an obstacle, find the smoothest collision-free path using gradient descent.

```
import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

Trajectory: N waypoints, each (x, y)
N = 20
start = jnp.array([0.0, 0.0])
goal = jnp.array([10.0, 0.0])
obstacle = jnp.array([5.0, 0.0])
obs_radius = 1.5

Initialise: straight line from start to goal
waypoints_init = jnp.linspace(start, goal, N)

def cost(waypoints):
 wp = jnp.concatenate([start[None], waypoints, goal[None]], axis=0)

 # Smoothness: penalise acceleration (second differences)
 accel = wp[2:] - 2 * wp[1:-1] + wp[:-2]
 smooth_cost = jnp.sum(accel ** 2)

 # Obstacle avoidance: penalise proximity
 dists = jnp.linalg.norm(wp - obstacle, axis=1)
 collision_cost = jnp.sum(jnp.maximum(0, obs_radius + 0.5 - dists) ** 2)

 return 10 * smooth_cost + 100 * collision_cost

grad_cost = jax.grad(cost)

Optimise the interior waypoints
waypoints = waypoints_init[1:-1]
lr = 0.01
for _ in range(500):
 g = grad_cost(waypoints)
 waypoints = waypoints - lr * g

Plot
full_path = jnp.concatenate([start[None], waypoints, goal[None]], axis=0)
theta = jnp.linspace(0, 2 * jnp.pi, 100)
```

```

plt.figure(figsize=(10, 4))
plt.plot(full_path[:, 0], full_path[:, 1], "b.-", label="Optimised path")
plt.plot(waypoints_init[:, 0], waypoints_init[:, 1], "r--", alpha=0.5,
label="Initial (straight)")
plt.fill(obstacle[0] + obs_radius * jnp.cos(theta),
 obstacle[1] + obs_radius * jnp.sin(theta), alpha=0.3, color="red",
label="Obstacle")
plt.plot(*start, "go", markersize=10); plt.plot(*goal, "g*", markersize=15)
plt.legend(); plt.axis("equal"); plt.grid(True)
plt.title("Trajectory Optimisation: Smooth Collision-Free Path")
plt.show()

```

1. Simulate a constant-velocity motion prediction model and compare with ground truth for a turning vehicle.

```

import jax.numpy as jnp
import matplotlib.pyplot as plt

Ground truth: vehicle turning right
dt = 0.1
T = 40 # 4 seconds
v = 10.0 # m/s
omega = 0.3 # rad/s (turning rate)

True trajectory (constant turn rate)
t = jnp.arange(T) * dt
theta = omega * t
gt_x = (v / omega) * jnp.sin(theta)
gt_y = (v / omega) * (1 - jnp.cos(theta))

Constant velocity prediction from t=0
Assumes the car continues straight at its current heading
obs_steps = 10 # observe first 1 second
vx0 = v * jnp.cos(theta[obs_steps - 1])
vy0 = v * jnp.sin(theta[obs_steps - 1])
pred_t = jnp.arange(T - obs_steps) * dt
pred_x = gt_x[obs_steps - 1] + vx0 * pred_t
pred_y = gt_y[obs_steps - 1] + vy0 * pred_t

plt.figure(figsize=(8, 6))
plt.plot(gt_x[:obs_steps], gt_y[:obs_steps], "ko-", label="Observed")
plt.plot(gt_x[obs_steps:], gt_y[obs_steps:], "g-", linewidth=2, label="True
future")
plt.plot(pred_x, pred_y, "r--", linewidth=2, label="Constant velocity
prediction")
plt.legend(); plt.axis("equal"); plt.grid(True)

```

```
plt.xlabel("x (m)"); plt.ylabel("y (m)")
plt.title("Constant Velocity Prediction vs Turning Vehicle")
plt.show()
```

1. Implement a simple rule-based planner that decides between lane-keeping and stopping based on detected obstacles.

```
import jax.numpy as jnp

def rule_based_planner(ego_speed, obstacles, speed_limit=13.9):
 """
 Simple rule-based planner.
 ego_speed: current speed (m/s)
 obstacles: list of (distance, speed) tuples for vehicles ahead
 speed_limit: max allowed speed (m/s), default ~50 km/h

 Returns: (target_speed, action_label)
 """
 min_following_distance = 2.0 * ego_speed # 2-second rule
 emergency_distance = 5.0 # metres

 if not obstacles:
 return speed_limit, "cruise"

 # Find closest obstacle ahead
 closest_dist, closest_speed = min(obstacles, key=lambda o: o[0])

 if closest_dist < emergency_distance:
 return 0.0, "EMERGENCY STOP"
 elif closest_dist < min_following_distance:
 # Match speed of vehicle ahead
 target = min(closest_speed, speed_limit)
 return target, "following"
 else:
 return speed_limit, "cruise"

Test scenarios
scenarios = [
 (13.9, [], "Empty road"),
 (13.9, [(30.0, 10.0)], "Slower car ahead"),
 (13.9, [(3.0, 0.0)], "Stopped car very close"),
 (13.9, [(50.0, 13.9)], "Car ahead at same speed"),
]

for speed, obs, desc in scenarios:
 target, action = rule_based_planner(speed, obs)
```

```
print(f"{desc:30s} → {action:15s} target_speed={target:.1f} m/s
({target*3.6:.0f} km/h)")
```

## 55 Space and Extreme Robotics

Space and extreme environment robotics push autonomy to its limits, where communication delays, radiation, and unstructured terrain demand robots that think for themselves. This file covers planetary rovers, orbital servicing, communication-constrained autonomy, radiation-hardened computing, underwater robotics, search-and-rescue, swarm robotics, and human-robot interaction

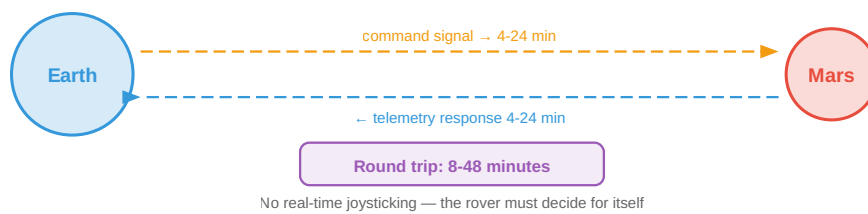
- Throughout this chapter, we have studied autonomous systems that operate in relatively benign environments: roads with lane markings, warehouses with flat floors, kitchens with known object categories. But some of the most impactful applications of robotics are in environments where humans cannot go, or where the cost of human presence is extreme: the surface of Mars, the deep ocean floor, nuclear disaster sites, and burning buildings.
- These **extreme environments** share common challenges: communication is limited or delayed, the terrain is unstructured and unpredictable, hardware must survive harsh conditions, and there is no human nearby to fix things when they go wrong. The robot must be truly autonomous, not just “autonomous with a human watching a screen.”

### 55.1 Space Robotics

---

- Space is the ultimate extreme environment. There is no air, temperatures swing from  $-170^{\circ}\text{C}$  to  $+120^{\circ}\text{C}$ , radiation bombards electronics, and help is millions of kilometres away. Space robots must be extraordinarily reliable, energy-efficient, and autonomous.
- **Planetary rovers** are mobile robots that explore the surfaces of other worlds. NASA’s Mars rovers (Spirit, Opportunity, Curiosity, Perseverance) are the most famous examples. Each generation has been more autonomous than the last.

### Earth-Mars Communication Delay



Earth-Mars communication delay: 4-24 minutes one way, 8-48 minutes round trip, making real-time control impossible

- The fundamental constraint is the **communication delay**. Mars is 4-24 minutes away by radio (depending on orbital positions), so round-trip communication takes 8-48 minutes. A rover cannot be joysticked in real time. If it encounters a rock, it cannot ask Earth for help and wait for a response. It must decide for itself.
- Early rovers (Spirit, Opportunity) relied heavily on ground-in-the-loop planning: humans would study images, plan a path, upload commands, and the rover would execute them. A single drive cycle took an entire Martian day (sol). The rover could traverse maybe 50-100 metres per sol.
- **AutoNav** (Autonomous Navigation) on Curiosity and Perseverance dramatically increased autonomy. The rover uses stereo cameras to build a local 3D map (recall stereo depth from chapter 8), evaluates terrain traversability (slope, roughness, rock size), and plans a safe path using a grid-based planner with a traversability cost map. The rover drives autonomously while the human team sleeps, increasing daily traverse distance to 100+ metres.
- The perception pipeline on Mars rovers is constrained by radiation-hardened processors that are orders of magnitude slower than consumer hardware (discussed below). Algorithms must be computationally frugal: classical stereo matching rather than deep neural networks, simple cost-map planners rather than learned policies.
- **Orbital servicing** involves robots that inspect, repair, refuel, or deorbit satellites in orbit. This is a growing field as space becomes more congested. Missions like **OSAM-1** (NASA) and commercial ventures (Astroscale, Northrop Grumman MEV) use robotic arms and docking mechanisms to service satellites.
- The challenge is **proximity operations**: a servicing spacecraft must approach a target satellite (which may be tumbling, uncooperative, and lacking docking interfaces) and perform precise manipulation in microgravity. Vision-based

pose estimation (determining the target's 3D position and orientation from camera images) is critical. This uses techniques from chapter 8: feature detection, PnP (Perspective-n-Point) solving, and more recently, deep learning-based pose estimators.

- **Satellite inspection** uses small spacecraft to visually examine other satellites for damage or anomalies. The inspector must autonomously navigate around the target, avoid collision, and capture high-resolution imagery from optimal viewpoints. This is a planning problem: find the trajectory that covers all inspection points while respecting fuel constraints, lighting conditions, and collision avoidance.

## 55.2 Communication Constraints

---

- In space, communication is limited by the speed of light, available bandwidth, and orbital geometry (a rover on the far side of Mars cannot communicate with Earth at all without relay satellites).
- These constraints fundamentally change the autonomy architecture. On Earth, a robot can stream HD video to a cloud server, run inference on a GPU cluster, and receive commands in milliseconds. In space, the robot must do everything onboard.
- **High latency** means the robot must plan and act without real-time human guidance. The autonomy software must handle nominal operations, detect anomalies, and respond to hazards without waiting for human input. This requires robust onboard state estimation, fault detection, and contingency planning.
- **Limited bandwidth** means the robot cannot transmit raw sensor data. A single high-resolution image might be several megabytes, but the Mars-to-Earth data rate is only a few kilobits per second through direct-to-Earth links (higher through orbital relays, but still limited). The robot must compress data aggressively, prioritise which data to send, and make most decisions locally.
- **Communication windows** are intermittent. A Mars rover can communicate with Earth only during specific orbital geometries, typically a few hours per sol via relay satellites. Outside these windows, the rover is entirely on its own.
- The implication for AI is that **onboard autonomy** must be highly reliable. The system needs to detect if something is wrong (a wheel is stuck, a sensor has

failed, the terrain ahead is impassable), decide on a safe response, and continue operating until the next communication window when it can report back and receive updated instructions.

## 55.3 Radiation-Hardened Computing

---

- Space is flooded with ionising radiation: cosmic rays, solar particle events, and trapped radiation in planetary magnetic fields. High-energy particles can flip bits in memory (**single-event upsets, SEUs**), permanently damage transistors (**total ionising dose, TID**), or cause destructive latch-up in circuits.
- **Radiation-hardened (rad-hard)** processors are designed to withstand this environment. They use larger transistor geometries, redundant logic (triple modular redundancy: three copies of each circuit vote on the output), and specialised manufacturing processes. The cost is performance: a state-of-the-art rad-hard processor might deliver 200 MIPS, compared to billions of operations per second on a consumer GPU.
- The **RAD750** (BAE Systems) powered Curiosity and many other spacecraft. It runs at 200 MHz with about 400 MIPS of processing power, comparable to a mid-1990s desktop computer. Perseverance uses a similar class of processor. Running a modern neural network (millions of parameters, billions of multiply-accumulate operations) is infeasible on such hardware.
- **Model compression** becomes essential. Techniques from chapter 6 (quantisation, pruning, knowledge distillation) are used to shrink neural networks to fit within the extreme computational budget. A model that runs in milliseconds on a laptop GPU might need minutes on a rad-hard processor, or might not fit in memory at all.
- An alternative approach uses **commercial off-the-shelf (COTS)** processors with radiation mitigation in software: error-correcting codes, watchdog timers, periodic memory scrubbing, and graceful degradation strategies. Some modern missions use this approach to access more powerful compute at the cost of increased software complexity and risk.
- Future planetary missions are exploring **FPGAs** and specialised AI accelerators that can be radiation-tolerant while providing significantly more compute than traditional rad-hard CPUs, potentially enabling onboard deep learning for the first time.

## 55.4 Autonomous Navigation in Unstructured Terrain

---

- On Earth, roads are flat, well-marked, and mapped. On Mars, the Moon, or a disaster site, there are no roads. The terrain is unstructured: rocks, slopes, sand, crevasses, and surfaces that may not support the robot's weight.
- **Terrain classification** evaluates whether each patch of ground is safe to traverse. Features include slope (from 3D reconstruction), roughness (variance of surface normals), rock density, and soil type. Classical approaches compute these features from stereo depth maps; modern approaches use learned classifiers on visual and geometric features.
- **Visual-inertial odometry (VIO)** estimates the robot's motion by tracking visual features across camera frames and fusing with IMU measurements. This is a core SLAM component (chapter 8) adapted for extreme conditions. On Mars, VIO must handle: featureless sandy terrain (few visual features to track), harsh lighting (extreme shadows), and limited compute.
- The estimation fuses visual and inertial data using an **Extended Kalman Filter (EKF)** or factor graph optimisation. The state vector includes position, velocity, orientation, and IMU biases. The prediction step uses IMU integration:

$$\mathbf{x}_{t+1} = f(\mathbf{x}_t, \mathbf{u}_t)$$

- where  $\mathbf{u}_t$  is the IMU measurement (acceleration and angular velocity). The update step corrects the prediction using visual feature observations. This is Bayesian estimation (chapter 5): the IMU provides a prior, and visual observations update the belief.
- **Hazard avoidance** is critical during planetary landing. As a spacecraft descends towards the surface, it must identify safe landing zones in real-time using onboard cameras or LiDAR. NASA's **Terrain Relative Navigation (TRN)** system on Perseverance compared onboard camera images to pre-loaded orbital maps to determine its position during descent, then steered away from hazardous terrain. This enabled landing in Jezero Crater, a scientifically rich but terrain-hazardous site that would have been too risky for previous missions.



## 55.5 Underwater Robotics

---

- The deep ocean is as alien as space: crushing pressure (1000+ atmospheres at full ocean depth), near-zero visibility, no GPS, and limited communication. Underwater robots are essential for ocean science, offshore infrastructure inspection, deep-sea mining, and search operations.
- **AUVs** (Autonomous Underwater Vehicles) operate untethered, carrying their own power and computing. They follow pre-programmed survey patterns or use onboard intelligence to adapt to discoveries. AUVs are used for seafloor mapping, pipeline inspection, and environmental monitoring.
- **ROVs** (Remotely Operated Vehicles) are tethered to a surface ship by a cable that provides power and communication. They are used for tasks requiring real-time human control: deep-sea manipulation, construction, and repair. The tether removes the communication constraint but limits range and adds operational complexity.
- **Acoustic communication** is the primary underwater communication method (radio waves attenuate rapidly in water). Acoustic modems achieve data rates of 1-10 kbps at ranges of a few kilometres, compared to gigabits per second for radio on land. This is even more constrained than Mars communication, forcing AUVs to be highly autonomous.
- **Underwater SLAM** is particularly challenging. Sonar provides range measurements but with poor angular resolution and significant noise (multipath reflections off the seafloor and surface). Cameras work only at very short range (a few metres in clear water, less in turbid conditions). Feature-based visual SLAM (chapter 8) must be adapted for the unique visual characteristics of underwater scenes: colour attenuation (red light is absorbed first), backscatter, and artificial lighting that creates bright spots and deep shadows.
- Navigation without GPS uses **dead reckoning** (integrating velocity from a Doppler Velocity Log, DVL, which measures speed relative to the seafloor using acoustic Doppler shifts), aided by occasional surfacing for GPS fixes or acoustic positioning from surface transponders. This is the same drift problem as IMU-only navigation: small velocity errors accumulate over long missions.

## 55.6 Search and Rescue Robotics

---

- After earthquakes, building collapses, or industrial accidents, robots can enter spaces too dangerous for human rescuers: structurally unstable buildings, toxic environments, fire, or confined spaces.
- The requirements are: rapid deployment (minutes, not hours), operation in GPS-denied environments (inside buildings, underground), robust communication through walls and rubble, and the ability to navigate highly cluttered, partially collapsed spaces with debris, dust, and poor lighting.
- **Multi-robot coordination** is valuable in search and rescue because a team of robots can cover a large area faster than a single robot. The challenge is coordination: the robots must divide the search area, avoid duplicating effort, and share discoveries.
- **Frontier-based exploration** assigns robots to the boundaries between explored and unexplored space (the “frontier” ). Each robot navigates to the nearest unexplored frontier, maps it, and moves on. A central or distributed planner allocates frontiers to robots to minimise total exploration time. This is a coverage optimisation problem.
- Communication through rubble is unreliable. Robots may lose contact with the operator and each other. The system must be robust to intermittent communication: each robot should be able to operate independently, building its own local map and making its own decisions, then merge information when communication is restored.

## 55.7 Swarm Robotics

---

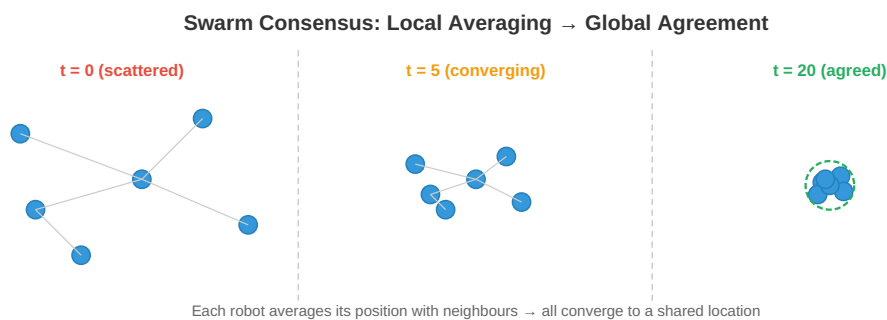
- **Swarm robotics** uses large numbers of simple, low-cost robots that achieve complex collective behaviour through local interactions. No single robot is individually capable, but the swarm as a whole can perform tasks that no individual could.
- Inspiration comes from biological swarms: ants building bridges with their bodies, bees making collective decisions about nest sites, fish schools evading predators through coordinated movement. In each case, simple local rules (follow your neighbours, avoid collisions, move towards food) produce sophisticated global behaviour.

- **Decentralised control** means there is no central commander. Each robot follows the same local rules, reacting only to its neighbours and immediate environment. The global behaviour **emerges** from these local interactions. This makes swarms inherently robust: if one robot fails, the swarm continues. There is no single point of failure.
- **Consensus algorithms** enable a swarm to agree on a collective decision (e.g., which direction to move, which task to prioritise) through local communication only. A simple consensus protocol has each robot average its value with its neighbours:

$$x_i(t+1) = \frac{1}{|N_i| + 1} (x_i(t) + \sum_{j \in N_i} x_j(t))$$

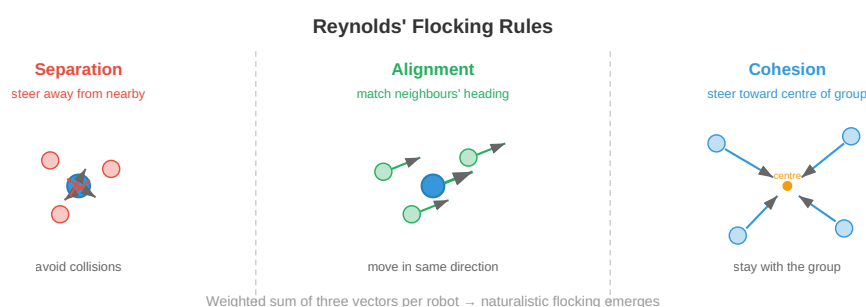
+  $\sum_{j \in N_i}$

$x_j(t)$



Swarm consensus: robots start scattered, iteratively average with neighbours, and converge to a shared location

- where  $N_i$  is the set of robot  $i$ 's neighbours. This is iterated until all robots converge to the same value (the global average). The convergence rate depends on the communication graph's topology, specifically its algebraic connectivity (the second-smallest eigenvalue of the graph Laplacian, connecting to eigenvalues from chapter 2).



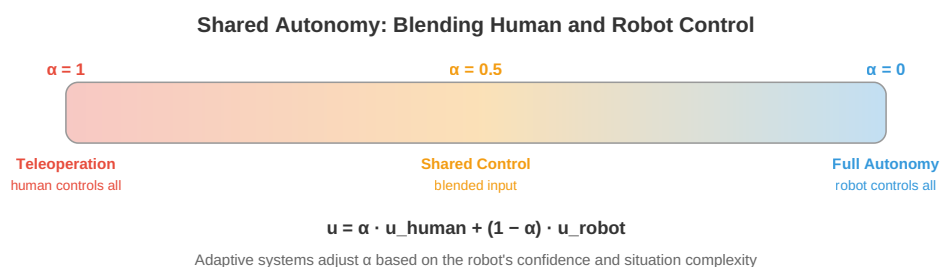
Reynolds' three flocking rules: separation avoids collisions, alignment matches heading, cohesion stays with the group

- **Flocking algorithms** (Reynolds' rules) produce coordinated group motion with three simple rules per robot:
  - **Separation**: steer away from neighbours that are too close (avoid collision).
  - **Alignment**: steer towards the average heading of neighbours (move in the same direction).
  - **Cohesion**: steer towards the average position of neighbours (stay with the group).
- Each rule is a vector contribution to the robot's velocity. The weighted sum of these vectors produces naturalistic flocking behaviour. This is a linear combination of vectors (chapter 1), where the weights control the relative importance of each behaviour.
- Applications of swarm robotics include environmental monitoring (distributing sensors across a large area), precision agriculture (coordinating drones for crop spraying), construction (robots collectively assembling structures), and search operations (covering a large area efficiently).

## 55.8 Human-Robot Interaction

---

- Most real-world autonomous systems operate alongside humans, not in isolation. The interaction between human and robot, how they communicate, share control, and build trust, is as important as the robot's technical capabilities.



Shared autonomy spectrum: from full human teleoperation ( $\alpha=1$ ) through blended control to full robot autonomy ( $\alpha=0$ )

- **Shared autonomy** blends human and robot control. Instead of full teleoperation (human controls everything) or full autonomy (robot controls

everything), shared autonomy lets the human provide high-level intent while the robot handles low-level execution. For example, a human might point to an object and say “pick that up,” and the robot autonomously plans the grasp and arm motion.

- Mathematically, shared autonomy can be modelled as a blending of the human’ s input  $\mathbf{u}_h$  and the robot’ s autonomous action  $\mathbf{u}_r$ :

$$\mathbf{u} = \alpha \mathbf{u}_h + (1 - \alpha) \mathbf{u}_r$$

- where  $\alpha \in [0, 1]$  is the blending parameter. When  $\alpha = 1$ , the human has full control (teleoperation). When  $\alpha = 0$ , the robot is fully autonomous. Adaptive shared autonomy adjusts  $\alpha$  based on the situation: the robot takes more control when it is confident and cedes control when it is uncertain or the situation is novel.
- **Teleoperation** remains important for tasks beyond current autonomous capabilities. A human operator controls the robot remotely, viewing the scene through the robot’ s cameras. The challenge is **latency**: even a 100ms delay makes teleoperation difficult, and the multi-second delays in space make it nearly impossible for fine manipulation. Predictive displays (showing the robot’ s predicted future state) and virtual fixtures (software guides that prevent the operator from commanding dangerous motions) help compensate.
- **Trust calibration** is the problem of ensuring humans have appropriate trust in the robot: not too much (over-trust leads to complacency and failure to intervene when needed), not too little (under-trust leads to unnecessary intervention and underutilisation). Trust should be calibrated to the robot’ s actual capabilities: trust it in situations it handles well, and be sceptical in situations near the edge of its competence.
- Research shows that trust is affected by: the robot’ s transparency (does it explain its decisions?), reliability (does it fail predictably or randomly?), and communication (does it express uncertainty?). A robot that says “I am 40% confident this is a safe path, should I proceed?” enables better human decision-making than one that silently drives forward.
- **Legibility** in robot motion means the robot moves in ways that communicate its intent to nearby humans. If a robot reaches for an object, its path should make it obvious which object it is targeting, even before it arrives. This involves planning trajectories that maximise the observer’ s ability to infer the goal

early, which can be formalised as maximising the posterior probability of the true goal given the observed partial trajectory:

$$\pi^* = \arg \max_{\pi} P(G \mid \xi_{0:t})$$

- where  $G$  is the goal and  $\xi_{0:t}$  is the trajectory observed so far. This uses Bayesian inference (chapter 5): the observer has a prior over possible goals, and the robot's trajectory provides evidence that updates this belief.

## 55.9 Coding Tasks (use CoLab or notebook)

1. Simulate a consensus algorithm for a swarm of robots agreeing on a target position. Start with random initial positions and watch convergence.

```
import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

n_robots = 10
rng = jax.random.PRNGKey(0)
positions = jax.random.uniform(rng, (n_robots, 2), minval=-5, maxval=5)

Communication graph: each robot talks to its 3 nearest neighbours
def get_neighbours(positions, k=3):
 dists = jnp.linalg.norm(positions[:, None] - positions[None, :], axis=-1)
 # For each robot, find k nearest (excluding self)
 neighbours = jnp.argsort(dists, axis=1)[:, 1:k+1]
 return neighbours

history = [positions.copy()]

for step in range(30):
 neighbours = get_neighbours(positions)
 new_positions = jnp.zeros_like(positions)
 for i in range(n_robots):
 nbr_pos = positions[neighbours[i]]
 new_positions = new_positions.at[i].set(
 (positions[i] + nbr_pos.sum(axis=0)) / (len(neighbours[i]) + 1)
)
 positions = new_positions
 history.append(positions.copy())
```

```

Plot convergence
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
for ax, step_idx, title in zip(axes, [0, 10, 29], ["Initial", "Step 10",
"Final"]):
 h = history[step_idx]
 ax.scatter(h[:, 0], h[:, 1], s=50)
 ax.set_xlim(-6, 6); ax.set_ylim(-6, 6)
 ax.set_aspect("equal"); ax.grid(True); ax.set_title(title)
plt.suptitle("Swarm Consensus: Robots Converge to Agreement")
plt.tight_layout()
plt.show()

```

1. Implement Reynolds' flocking rules (separation, alignment, cohesion) and simulate a swarm moving together.

```

import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

n = 30
rng = jax.random.PRNGKey(1)
k1, k2 = jax.random.split(rng)
pos = jax.random.uniform(k1, (n, 2), minval=-5, maxval=5)
vel = jax.random.uniform(k2, (n, 2), minval=-0.5, maxval=0.5)

dt = 0.1
separation_radius = 1.0
neighbour_radius = 3.0

trajectories = [pos.copy()]

for _ in range(200):
 new_vel = jnp.zeros_like(vel)
 for i in range(n):
 diffs = pos - pos[i]
 dists = jnp.linalg.norm(diffs, axis=1)

 # Neighbours within radius (exclude self)
 nbr_mask = (dists < neighbour_radius) & (dists > 0)
 sep_mask = (dists < separation_radius) & (dists > 0)

 # Separation: steer away from very close neighbours
 if sep_mask.any():
 sep = -diffs[sep_mask].sum(axis=0)
 else:

```

```

 sep = jnp.zeros(2)

 # Alignment: match average velocity of neighbours
 if nbr_mask.any():
 align = vel[nbr_mask].mean(axis=0) - vel[i]
 else:
 align = jnp.zeros(2)

 # Cohesion: steer toward average position of neighbours
 if nbr_mask.any():
 cohesion = pos[nbr_mask].mean(axis=0) - pos[i]
 else:
 cohesion = jnp.zeros(2)

 new_vel = new_vel.at[i].set(vel[i] + 1.5 * sep + 0.5 * align + 0.3 *
cohesion)

 # Limit speed
 speeds = jnp.linalg.norm(new_vel, axis=1, keepdims=True)
 vel = jnp.where(speeds > 2.0, new_vel / speeds * 2.0, new_vel)
 pos = pos + vel * dt
 trajectories.append(pos.copy())

 # Plot snapshots
 fig, axes = plt.subplots(1, 3, figsize=(15, 4))
 for ax, idx, title in zip(axes, [0, 50, 199], ["Start", "Step 50", "Step 200"]):
 p = trajectories[idx]
 v = vel if idx == 199 else jnp.zeros_like(vel)
 ax.scatter(p[:, 0], p[:, 1], s=20, c="blue")
 ax.set_aspect("equal"); ax.grid(True); ax.set_title(title)
 lim = max(abs(p).max(), 1, 6)
 ax.set_xlim(-lim, lim); ax.set_ylim(-lim, lim)
 plt.suptitle("Reynolds' Flocking: Separation + Alignment + Cohesion")
 plt.tight_layout()
 plt.show()

```

1. Simulate shared autonomy blending: a human provides noisy directional input, and the robot's autonomous system provides a smooth path to the goal. Blend them with different alpha values.

```

import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

goal = jnp.array([10.0, 5.0])
pos = jnp.array([0.0, 0.0])

```



```

dt = 0.1

rng = jax.random.PRNGKey(3)

fig, axes = plt.subplots(1, 3, figsize=(15, 4))
for ax, alpha in zip(axes, [1.0, 0.5, 0.0]):
 pos = jnp.array([0.0, 0.0])
 path = [pos.copy()]

 for step in range(150):
 # Robot autonomy: smooth path to goal
 direction = goal - pos
 u_robot = direction / (jnp.linalg.norm(direction) + 1e-6) * 1.0

 # Human input: roughly correct direction but noisy
 noise = jax.random.normal(jax.random.fold_in(rng, step), (2,)) * 0.5
 u_human = u_robot + noise

 # Blend
 u = alpha * u_human + (1 - alpha) * u_robot
 pos = pos + u * dt
 path.append(pos.copy())

 if jnp.linalg.norm(pos - goal) < 0.3:
 break

 path = jnp.stack(path)
 ax.plot(path[:, 0], path[:, 1], "b-", alpha=0.7)
 ax.plot(*goal, "r*", markersize=15)
 ax.plot(0, 0, "go", markersize=10)
 ax.set_title(f" $\alpha={alpha:.1f}$ ({'human' if alpha==1 else 'robot' if alpha==0
else 'shared'})")
 ax.set_xlim(-1, 12); ax.set_ylim(-3, 8)
 ax.set_aspect("equal"); ax.grid(True)

plt.suptitle("Shared Autonomy: Blending Human and Robot Control")
plt.tight_layout()
plt.show()

```

## 56 Geometric Deep Learning

Geometric deep learning is the unifying framework that reveals CNNs, transformers, and GNNs as instances of the same principle: exploiting symmetry. This file covers

symmetry groups, group actions, invariance, equivariance, the five geometric domains, and scale separation

- Throughout this book, we have studied many architectures: CNNs for images (chapter 8), transformers for language (chapter 7), and RL policies for sequential decisions (chapter 6). These look like completely different models designed for completely different problems. But there is a deeper pattern.
- **Geometric deep learning** reveals that all of these architectures are instances of the same idea: build networks that respect the **symmetries** of the data. CNNs exploit translation symmetry in images. Transformers exploit permutation symmetry in sequences (attention does not depend on absolute position). GNNs exploit permutation symmetry in graphs. Once you see this, the zoo of architectures becomes a single, coherent framework.

## 56.1 Symmetry and Groups

---

- A **symmetry** of an object is a transformation that leaves it unchanged. A square has 8 symmetries: 4 rotations ( $0^\circ$ ,  $90^\circ$ ,  $180^\circ$ ,  $270^\circ$ ) and 4 reflections. A circle has infinitely many: any rotation about its centre. The key insight is that symmetries tell you what does not matter, and knowing what does not matter is enormously powerful for learning.
- In ML terms: if a task has a symmetry, the model should give the same answer regardless of which “version” of the input it sees. A cat detector should work whether the cat is in the top-left or bottom-right of the image. That is translation symmetry.
- Symmetries are formalised as **groups**. A group  $G$  is a set of transformations with four properties:
  - **Closure**: combining two transformations gives another transformation in the set. Rotating by  $90^\circ$  then  $90^\circ$  gives  $180^\circ$ , which is also in the set.
  - **Associativity**:  $(g_1 \circ g_2) \circ g_3 = g_1 \circ (g_2 \circ g_3)$ . The order of grouping does not matter (recall associativity of matrix multiplication from chapter 2).
  - **Identity**: there is a “do nothing” transformation  $e$  such that  $e \circ g = g \circ e = g$ .
  - **Inverse**: every transformation has an undo:  $g \circ g^{-1} = e$ .

- These are the same axioms as vector spaces (chapter 1) but for transformations instead of vectors. The connection is deep: groups act on vector spaces, and this action is what neural networks must respect.
- Key groups that appear in deep learning:
  - **Translation group**  $(\mathbb{R}^n, +)$ : shifting an image or signal. This is the symmetry that CNNs exploit.
  - **Symmetric group**  $S_n$ : all permutations of  $n$  elements. This is the symmetry that GNNs and transformers exploit (reordering nodes or tokens should not change the result).
  - **Rotation group**  $SO(n)$ : all rotations in  $n$ -dimensional space.  $SO(2)$  is rotations in the plane,  $SO(3)$  is rotations in 3D (crucial for molecular and 3D vision tasks).
  - **Euclidean group**  $E(n)$ : all rotations, reflections, and translations. The symmetry of physical space.
  - **Special Euclidean group**  $SE(n)$ : rotations and translations (no reflections). The symmetry of rigid body motion.
- A **group action** describes how a group transforms the data. If  $G$  is a group and  $X$  is a data space, the action  $\rho : G \times X \rightarrow X$  maps each group element  $g$  and data point  $x$  to a transformed point  $\rho(g, x)$ . For images, the translation group acts by shifting pixel coordinates. For graphs, the symmetric group acts by relabelling nodes.

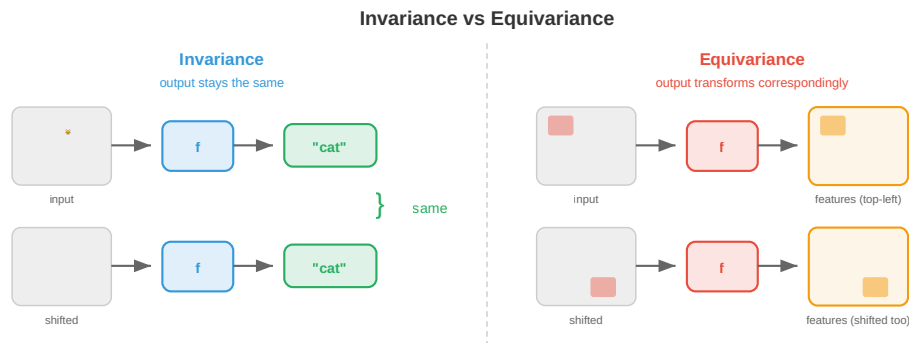
## 56.2 Invariance and Equivariance

---

- Given a symmetry group, a function can relate to it in two important ways:
- A function  $f$  is **invariant** to a group  $G$  if the output does not change when the input is transformed:
 
$$f(\rho(g, x)) = f(x) \text{ for all } g \in G$$
- Example: the total brightness of an image does not change if you shift the image. Image classification should be translation-invariant: the class “cat” is the same regardless of where the cat sits.
- A function  $f$  is **equivariant** to  $G$  if transforming the input transforms the output in a corresponding way:

$$f(\rho_{\text{in}}(g, x)) = \rho_{\text{out}}(g, f(x)) \text{ for all } g \in G$$

- Example: if you shift an image right by 5 pixels, the feature map in a CNN also shifts right by 5 pixels. The convolution operation is translation-equivariant: it preserves the spatial relationship. Object detection should be equivariant: if the cat moves, the bounding box should move with it.



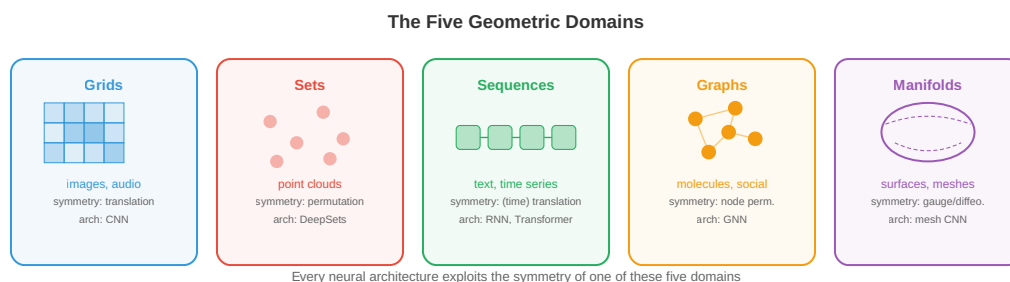
Invariance: output stays the same regardless of transformation.

Equivariance: output transforms correspondingly

- The distinction matters: **intermediate layers** should typically be equivariant (preserving structure for downstream layers), while the **final output** should be invariant (the answer should not depend on the transformation). A CNN achieves this by stacking equivariant convolution layers, then applying global pooling (which is invariant) at the end.
- Building equivariance into the architecture is far more efficient than learning it from data. A translation-equivariant CNN with weight sharing needs far fewer parameters than a fully-connected network that must independently learn “cat at position (10,10)” and “cat at position (200,150).” The symmetry constraint reduces the hypothesis space exponentially.

## 56.3 The Five Geometric Domains

- Geometric deep learning identifies **five fundamental domains** of data, each with its own symmetry group. Every neural network architecture can be understood as exploiting the symmetry of one of these domains.



The five geometric domains: grids, sets, sequences, graphs, and manifolds, each with its own symmetry and architecture

- **1. Grids (Euclidean data):** images, audio spectrograms, volumetric data. The underlying structure is a regular grid with translation symmetry. The group is the translation group (plus possibly rotations and reflections). The architecture that exploits this symmetry is the **CNN**: convolution is exactly the operation that is equivariant to translation. Weight sharing across spatial positions is translation equivariance made concrete.
- **2. Sets (unordered collections):** point clouds, particle systems. The symmetry is permutation invariance: the order of elements does not matter. The architecture is **DeepSets** (and PointNet from chapter 8): apply a shared function to each element, then aggregate with a permutation-invariant operation (sum, mean, or max). Formally,  $f(\{x_1, \dots, x_n\}) = \phi(\sum_i \psi(x_i))$ .
- **3. Sequences (ordered data):** text, time series. Sequences are grids in 1D, but with a twist: the symmetry is more nuanced. Absolute position may or may not matter. RNNs process sequences autoregressively. Transformers with positional encodings can attend to any position, and their self-attention is equivariant to permutation (before positional encoding is added). This is why transformers generalise so well: they start permutation-equivariant and add just enough positional structure.
- **4. Graphs (relational data):** social networks, molecules, knowledge graphs. The symmetry is permutation of nodes: relabelling the nodes should not change the graph's properties. The architecture is the **GNN**: message passing between connected nodes, using shared functions that do not depend on node ordering. This is the focus of the rest of this chapter.
- **5. Manifolds and meshes:** surfaces, 3D shapes. The symmetry includes diffeomorphisms (smooth deformations). The architecture uses intrinsic operators (e.g., Laplace-Beltrami) that are defined by the surface geometry itself, independent of how the surface is embedded in space. This connects to

differential geometry and is relevant for shape analysis, climate modelling on the sphere, and protein surface analysis.

- The power of this framework is unification. A CNN is a GNN on a grid graph. A transformer is a GNN on a fully connected graph. DeepSets is a GNN with no edges. Seeing these as instances of the same principle guides the design of new architectures: identify the symmetry of your data, and build a network that respects it.

## 56.4 Scale Separation and Coarsening

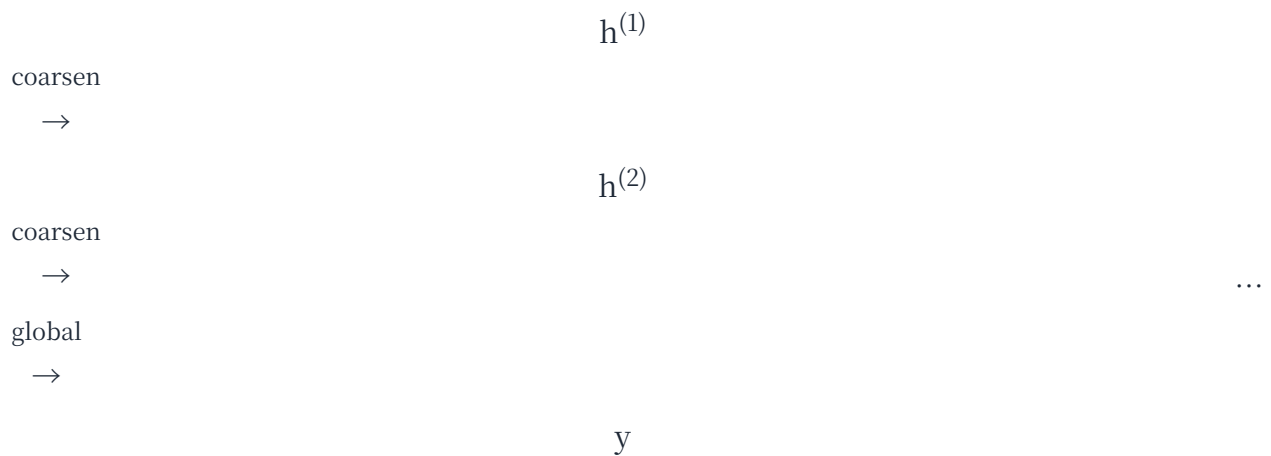
---

- Real-world data has structure at multiple scales. An image has fine-grained texture (pixel level), local patterns (edges, corners), object parts (wheels, windows), and global structure (the entire scene). A molecule has atom-level features, functional groups, and overall molecular shape.
- **Scale separation** is the principle that these levels of detail can be processed hierarchically: first capture local structure, then progressively aggregate into coarser representations. This is **coarsening** or **pooling**.
- In CNNs, pooling layers (max pooling, average pooling) downsample the spatial resolution, forcing higher layers to capture larger-scale patterns. In the receptive field view (chapter 8), deeper layers “see” more of the image. This is scale separation in action.
- In graphs, coarsening means clustering groups of nodes into “supernodes,” producing a smaller graph that preserves the essential structure. This is graph pooling, which we will cover in detail in file 3. The analogy to image pooling is direct: reduce resolution while preserving important features.
- In sequences, hierarchical processing (e.g., sentence → paragraph → document) captures structure at different temporal or semantic scales. The Swin Transformer (chapter 8) applies this idea to images with its shifted window hierarchy.
- Mathematically, coarsening defines a **hierarchy of increasingly abstract representations**:

X

local features

→



- At each level, the representation is equivariant to the symmetry group of that level. The final global representation is invariant, capturing the essence of the input without sensitivity to irrelevant transformations.
- This hierarchy is why deep networks work better than shallow ones for structured data: each layer adds one level of abstraction, and the composition of many equivariant layers builds up complex invariant features from simple local ones.

## 56.5 Coding Tasks (use CoLab or notebook)

1. Verify translation equivariance of convolution. Apply a convolution to an image, then shift the image and convolve again. Check that the outputs are shifted versions of each other.

```
import jax
import jax.numpy as jnp

1D signal and a simple filter
signal = jnp.array([0, 0, 0, 1, 2, 3, 2, 1, 0, 0], dtype=float)
kernel = jnp.array([1, 0, -1], dtype=float)

Convolve then shift
conv_result = jnp.convolve(signal, kernel, mode="same")
shifted_signal = jnp.roll(signal, 3)
conv_shifted = jnp.convolve(shifted_signal, kernel, mode="same")
shifted_conv = jnp.roll(conv_result, 3)

print(f"Conv then shift: {shifted_conv}")
```

```
print(f"Shift then conv: {conv_shifted}")
print(f"Equivariant: {jnp.allclose(shifted_conv, conv_shifted, atol=1e-5)}")
```

1. Verify permutation invariance of DeepSets-style aggregation. Apply a shared function to each element of a set, sum the results, and check that the output is the same regardless of element ordering.

```
import jax
import jax.numpy as jnp

A "set" of 4 vectors (order should not matter)
x = jnp.array([[1.0, 2.0], [3.0, 4.0], [5.0, 6.0], [7.0, 8.0]])

Simple shared function: element-wise square
psi = lambda v: v ** 2

Aggregate by sum
def deepsets(points):
 return jnp.sum(jax.vmap(psi)(points), axis=0)

Original order
result1 = deepsets(x)

Permuted order
perm = jnp.array([2, 0, 3, 1])
result2 = deepsets(x[perm])

print(f"Original order: {result1}")
print(f"Permuted order: {result2}")
print(f"Invariant: {jnp.allclose(result1, result2)}")
```

1. Explore group structure. Verify that 2D rotation matrices form a group by checking closure, associativity, identity, and inverse.

```
import jax.numpy as jnp

def rot2d(theta):
 return jnp.array([[jnp.cos(theta), -jnp.sin(theta)],
 [jnp.sin(theta), jnp.cos(theta)]])

R1 = rot2d(jnp.pi / 6)
R2 = rot2d(jnp.pi / 4)
R3 = rot2d(jnp.pi / 3)

Closure: product of two rotations is a rotation
```



```

R12 = R1 @ R2
print(f"Closure (det=1, orthogonal): det={jnp.linalg.det(R12):.4f}, "
 f"R^T R = I: {jnp.allclose(R12.T @ R12, jnp.eye(2), atol=1e-5)}")

Associativity
print(f"Associative: {jnp.allclose((R1 @ R2) @ R3, R1 @ (R2 @ R3), atol=1e-5)}")

Identity
I = rot2d(0.0)
print(f"Identity: {jnp.allclose(R1 @ I, R1, atol=1e-5)}")

Inverse
R1_inv = rot2d(-jnp.pi / 6)
print(f"Inverse: {jnp.allclose(R1 @ R1_inv, jnp.eye(2), atol=1e-5)}")

```

## 57 Graph Theory

Graph theory provides the mathematical language for describing relationships between entities. This file covers nodes, edges, adjacency matrices, graph types, degree and connectivity, the graph Laplacian, spectral graph theory, and real-world graph applications. We will go deeper into graphs in the pure computer science chapters

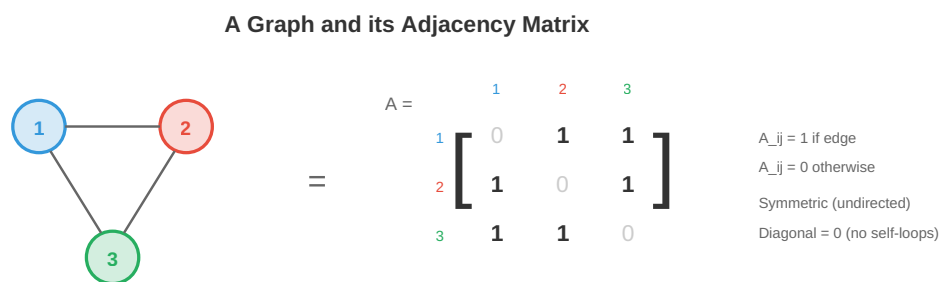
- So far in this book, data has lived on regular structures: vectors in  $\mathbb{R}^n$  (chapter 1), matrices as grids of numbers (chapter 2), images as pixel grids (chapter 8), sequences as ordered lists (chapter 7). But many real-world systems are **irregular**: a social network has no grid structure, a molecule has no left-to-right order, and a road network does not tile neatly into rows and columns.
- **Graphs** are the mathematical tool for representing these irregular, relational structures. A graph captures **entities** (nodes) and **relationships** (edges) between them. Once data is represented as a graph, we can apply the geometric deep learning principles from file 1 to learn from it.

### 57.1 Nodes, Edges, and Adjacency

- A **graph**  $G = (V, E)$  consists of a set of **nodes** (or vertices)  $V = \{v_1, v_2, \dots, v_n\}$  and a set of **edges**  $E \subseteq V \times V$  connecting pairs of nodes.

- Nodes represent entities: people, atoms, cities, web pages, neurons. Edges represent relationships: friendships, chemical bonds, roads, hyperlinks, synapses.
- The **adjacency matrix**  $A$  is the matrix representation of a graph. For a graph with  $n$  nodes,  $A$  is an  $n \times n$  matrix where  $A_{ij} = 1$  if there is an edge from node  $i$  to node  $j$ , and  $A_{ij} = 0$  otherwise.
- For example, a triangle graph (3 nodes, all connected) has:

```
A = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix}
```



A triangle graph and its adjacency matrix: 1 where edges exist, 0 otherwise

- The diagonal is zero because nodes are not connected to themselves (no self-loops by default). The adjacency matrix is a direct application of the Boolean matrices we studied in chapter 2: each entry is a binary relationship.
- The adjacency matrix encodes the graph's structure completely. Matrix operations on  $A$  reveal graph properties:  $A_{ij}^2$  counts the number of paths of length 2 between nodes  $i$  and  $j$  (recall matrix multiplication from chapter 2: each entry is a sum of products over intermediate nodes). More generally,  $A_{ij}^k$  counts length- $k$  paths.
- Each node can carry a **feature vector**  $\mathbf{x}_i \in \mathbb{R}^d$ . For a social network, this might be a user's profile information. For a molecule, it encodes atom type, charge, and other properties. The full set of node features is a matrix  $X \in \mathbb{R}^{n \times d}$ , where each row is one node's features.
- Edges can also carry features: bond type in molecules, distance in spatial graphs, relationship type in knowledge graphs. The **edge feature** for edge  $(i, j)$  is a vector  $\mathbf{e}_{ij} \in \mathbb{R}^{d_e}$ .

## 57.2 Graph Types

- An **undirected graph** has symmetric edges: if  $i$  is connected to  $j$ , then  $j$  is connected to  $i$ . The adjacency matrix is symmetric:  $A = A^T$  (a symmetric matrix, chapter 2). Friendships and chemical bonds are undirected.
- A **directed graph** (digraph) has edges with direction: an edge from  $i$  to  $j$  does not imply an edge from  $j$  to  $i$ . The adjacency matrix is asymmetric. Twitter follows, web hyperlinks, and citation networks are directed.
- A **weighted graph** assigns a numerical weight to each edge. The adjacency matrix has real-valued entries instead of binary:  $A_{ij} = w_{ij}$ . Distances in a road network, correlation strengths in brain connectivity, and interaction frequencies in social networks are weighted.
- A **bipartite graph** has two disjoint sets of nodes, with edges only between the sets (never within). Users and products form a bipartite graph: users rate products, but users do not rate users. The adjacency matrix of a bipartite graph has a block structure:

$$A = \begin{bmatrix} 0 & B \\ B^T & 0 \end{bmatrix}$$

- where  $B$  is the bipartite adjacency matrix between the two node sets.
- A **multigraph** allows multiple edges between the same pair of nodes and/or self-loops. Knowledge graphs are typically multigraphs: two entities can have multiple relationships (e.g., “born in,” “lives in,” “works in” ).
- A **hypergraph** generalises edges to connect more than two nodes at once. A **hyperedge** connects a set of nodes, representing higher-order relationships. A research paper co-authored by five people is a hyperedge connecting five author nodes.
- A **complete graph**  $K_n$  has an edge between every pair of nodes. This is the graph analogue of a fully connected layer, and it is the structure that transformers operate on (every token attends to every other token).

## 57.3 Degree, Paths, and Connectivity

---

- The **degree** of a node is the number of edges connected to it. In an undirected graph, the degree of node  $i$  is  $d_i = \sum_j A_{ij}$ . High-degree nodes are “hubs” with many connections.
- The **degree matrix**  $D$  is a diagonal matrix with degrees on the diagonal:  $D_{ii} = d_i$ . This matrix appears throughout graph theory and GNN formulas.
- A **path** between two nodes is a sequence of edges connecting them. The **shortest path** (or geodesic) between  $i$  and  $j$  is the path with the fewest edges (or lowest total weight in a weighted graph). **Dijkstra’s algorithm** finds shortest paths in  $O((|V| + |E|) \log |V|)$  time.
- A graph is **connected** if there is a path between every pair of nodes. If not, it has multiple **connected components**: isolated subgraphs with no edges between them.
- The **diameter** of a graph is the longest shortest path between any pair of nodes. It measures how “spread out” the graph is. Social networks have famously small diameters ( “six degrees of separation” ).
- A **cycle** is a path that starts and ends at the same node. A graph with no cycles is a **tree**. Trees are the simplest connected graphs:  $n$  nodes and exactly  $n - 1$  edges.
- **Centrality** measures the importance of a node. **Degree centrality** is simply the degree. **Betweenness centrality** counts how many shortest paths pass through a node. **Eigenvector centrality** assigns importance based on the importance of a node’s neighbours, leading to the eigenvector equation  $A\mathbf{x} = \lambda\mathbf{x}$  (chapter 2). Google’s PageRank is a variant of eigenvector centrality for directed graphs.

## 57.4 The Graph Laplacian

---

- The **graph Laplacian** is perhaps the most important matrix in graph theory. It is defined as:

$$L = D - A$$

- where  $D$  is the degree matrix and  $A$  is the adjacency matrix. For our triangle example:

```

L = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 2 \end{bmatrix} -
\begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix} =
\begin{bmatrix} 2 & -1 & -1 \\ -1 & 2 & -1 \\ -1 & -1 & 2 \end{bmatrix}

```

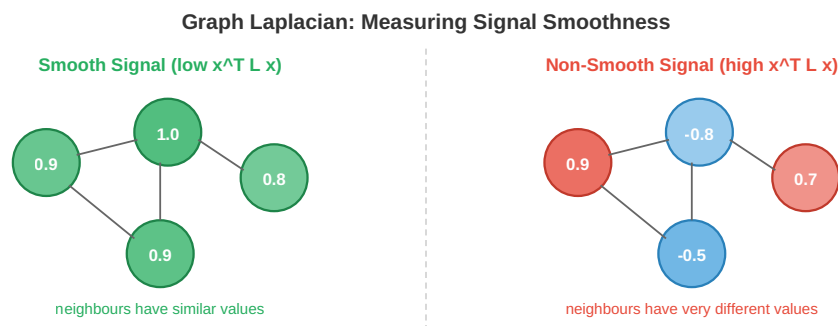
· The Laplacian has remarkable properties:

- It is always **symmetric** and **positive semi-definite** (recall from chapter 2: all eigenvalues are  $\geq 0$ ). For any vector  $\mathbf{x}$ :

$$\mathbf{x}^T \mathbf{L} \mathbf{x} =$$

$$\sum_{(i,j) \in E}$$

$$(x_i - x_j)^2$$



Graph Laplacian measures signal smoothness: smooth signals have similar values on connected nodes, non-smooth signals vary sharply

- This quadratic form measures how much a signal  $\mathbf{x}$  on the graph varies across edges. If neighbouring nodes have similar values,  $\mathbf{x}^T \mathbf{L} \mathbf{x}$  is small. If they differ sharply, it is large. The Laplacian measures **smoothness** of signals on the graph.

- The smallest eigenvalue is always 0, with eigenvector  $\mathbf{1} = [1, 1, \dots, 1]^T$  (a constant signal has zero variation). The number of zero eigenvalues equals the number of connected components.

- The second-smallest eigenvalue  $\lambda_2$  is the **algebraic connectivity** (Fiedler value). It measures how well-connected the graph is:  $\lambda_2 = 0$  means the graph is disconnected, large  $\lambda_2$  means the graph is tightly connected.

· The **normalised Laplacian** scales by the degrees:



L

$$= D^{-1/2} L D^{-1/2} = I - D^{-1/2} A D^{-1/2}$$

- This normalisation ensures that the Laplacian's properties do not depend on the absolute scale of node degrees. The term  $D^{-1/2} A D^{-1/2}$  is the **symmetrically normalised adjacency**, and it appears directly in the GCN formula (file 3).

## 57.5 Spectral Graph Theory

---

- The eigenvalues and eigenvectors of the graph Laplacian define the **spectrum** of the graph, and they serve as the graph analogue of the Fourier transform.
- In classical signal processing, the Fourier transform decomposes a signal into frequency components (sines and cosines). On a graph, the eigenvectors of the Laplacian play the role of these frequency bases. Low-eigenvalue eigenvectors vary slowly across the graph (low frequency, smooth), while high-eigenvalue eigenvectors vary rapidly (high frequency, oscillatory).
- The **Graph Fourier Transform (GFT)** of a signal  $\mathbf{x}$  on the graph is:



$\mathbf{x}$

$$= U^T \mathbf{x}$$

- where  $U$  is the matrix of Laplacian eigenvectors (recall eigendecomposition from chapter 2:  $L = U \Lambda U^T$ ). The inverse transform is  $\mathbf{x} = U$



$\mathbf{x}$

- **Graph convolution** in the spectral domain is pointwise multiplication in the frequency domain, just as convolution in the spatial domain corresponds to multiplication in the Fourier domain (the convolution theorem from chapter 8):

$$g_\theta \star \mathbf{x} = U((U^T g_\theta) \odot (U^T \mathbf{x})) = U \text{diag}(g_\theta) U^T \mathbf{x}$$

- The filter

$$\hat{g}$$

$$g$$

$g_\theta$  is a learnable function of the eigenvalues. This is the foundation of spectral GNNs, which we will simplify into the practical GCN in file 3.

- The computational bottleneck is the eigendecomposition of  $L$ , which costs  $O(n^3)$  for a graph with  $n$  nodes. This is impractical for large graphs (millions of nodes). Polynomial approximations (Chebyshev polynomials) avoid the eigendecomposition entirely, and this approximation leads directly to the GCN.

## 57.6 Community Detection

---

- Many real-world graphs have **community structure**: clusters of densely connected nodes with sparse connections between clusters. Social networks have friend groups, biological networks have functional modules, citation networks have research fields.
- **Spectral clustering** uses the Laplacian eigenvectors to find communities. The idea: embed each node using the  $k$  smallest non-trivial eigenvectors of  $L$ , then apply  $k$ -means (chapter 6) in this embedding space. Nodes in the same community end up close together in the spectral embedding.
- This works because the Fiedler vector (eigenvector of  $\lambda_2$ ) naturally separates the graph into two groups: nodes with positive values and nodes with negative values, cutting through the sparsest connections. Higher eigenvectors refine this into more groups.
- **Modularity**  $Q$  measures the quality of a community partition. It compares the number of within-community edges to the expected number in a random graph:

$$Q = \frac{1}{2|E|}$$

$\sum_{ij}$

$$(A_{ij} - \frac{d_i d_j}{2|E|}) \delta(c_i, c_j)$$

- where  $c_i$  is the community assignment of node  $i$  and  $\delta$  is 1 if nodes are in the same community.  $Q$  ranges from  $-0.5$  to  $1$ , with higher values indicating stronger community structure.

## 57.7 Real-World Graphs

---

- **Social networks:** nodes are people, edges are friendships or interactions. Facebook has billions of nodes and hundreds of billions of edges. These graphs are typically sparse (each person has hundreds of friends, not billions), exhibit small-world properties (short average path length), and have heavy-tailed degree distributions (a few hubs with millions of connections).
- **Molecular graphs:** nodes are atoms, edges are chemical bonds. Each atom has features (element type, charge, hybridisation) and each bond has features (single, double, triple, aromatic). Molecular graphs are small (tens to hundreds of nodes) but highly structured. Predicting molecular properties from graph structure is a major application of GNNs.
- **Knowledge graphs:** nodes are entities (people, places, concepts), edges are typed relationships ( “born in,” “capital of,” “instance of” ). Knowledge graphs power search engines, recommendation systems, and question-answering. They are typically directed multigraphs with millions of entities and billions of relationships.
- **Citation networks:** nodes are papers, edges are citations (directed). Clustering reveals research communities. Node features include title, abstract, and publication year.
- **Protein interaction networks:** nodes are proteins, edges indicate physical interactions or functional associations. Understanding these graphs helps identify drug targets and disease mechanisms.



- **Road networks and transportation:** nodes are intersections, edges are road segments with distance/time weights. Shortest-path algorithms on these graphs power navigation systems. Self-driving motion prediction (chapter 11) represents agent interactions as graphs.

## 57.8 Coding Tasks (use CoLab or notebook)

---

1. Build a small graph as an adjacency matrix and compute basic properties: degree of each node, number of paths of length 2, and whether the graph is connected.

```
import jax.numpy as jnp

A simple graph: 5 nodes
0-1, 0-2, 1-2, 2-3, 3-4
A = jnp.array([[0, 1, 1, 0, 0],
 [1, 0, 1, 0, 0],
 [1, 1, 0, 1, 0],
 [0, 0, 1, 0, 1],
 [0, 0, 0, 1, 0]], dtype=float)

Degree
degrees = A.sum(axis=1)
print(f"Degrees: {degrees}")

Paths of length 2
A2 = A @ A
print(f"Paths of length 2 (node 0 to 3): {int(A2[0, 3])}")

Connected? Check if A^(n-1) has all nonzero entries
An = jnp.linalg.matrix_power(A + jnp.eye(5), 4) # (A+I)^4 for reachability
connected = jnp.all(An > 0)
print(f"Connected: {connected}")
```

1. Compute the graph Laplacian and its eigenvalues. Verify that the smallest eigenvalue is 0 and the corresponding eigenvector is constant.

```
import jax.numpy as jnp

A = jnp.array([[0, 1, 1, 0, 0],
 [1, 0, 1, 0, 0],
 [1, 1, 0, 1, 0],
 [0, 0, 1, 0, 1],
 [0, 0, 0, 1, 0]], dtype=float)
```

```

D = jnp.diag(A.sum(axis=1))
L = D - A

eigenvalues, eigenvectors = jnp.linalg.eigh(L)
print(f"Eigenvalues: {eigenvalues}")
print(f"Smallest eigenvector: {eigenvectors[:, 0]}")
print(f"Fiedler value (algebraic connectivity): {eigenvalues[1]:.4f}")

Verify: $x^T L x$ measures smoothness
x = jnp.array([1.0, 1.0, 1.0, -1.0, -1.0]) # two groups
smoothness = x @ L @ x
print(f"Smoothness of two-group signal: {smoothness:.2f}")

```

1. Perform spectral clustering on a graph with two communities. Embed nodes using the Fiedler vector and separate them by sign.

```

import jax.numpy as jnp
import matplotlib.pyplot as plt

Two communities of 5 nodes each, weakly connected
A = jnp.zeros((10, 10))
Community 1: nodes 0-4 (dense)
for i in range(5):
 for j in range(i+1, 5):
 A = A.at[i, j].set(1).at[j, i].set(1)
Community 2: nodes 5-9 (dense)
for i in range(5, 10):
 for j in range(i+1, 10):
 A = A.at[i, j].set(1).at[j, i].set(1)
One bridge edge
A = A.at[2, 7].set(1).at[7, 2].set(1)

D = jnp.diag(A.sum(axis=1))
L = D - A
eigenvalues, eigenvectors = jnp.linalg.eigh(L)

Fiedler vector (2nd smallest eigenvalue)
fiedler = eigenvectors[:, 1]
communities = (fiedler > 0).astype(int)

print(f"Fiedler vector: {fiedler}")
print(f"Clusters: {communities}")

plt.bar(range(10), fiedler, color=["#3498db" if c == 0 else "#e74c3c" for c in
communities])

```

```
plt.xlabel("Node"); plt.ylabel("Fiedler vector value")
plt.title("Spectral Clustering via Fiedler Vector")
plt.show()
```

## 58 Graph Neural Networks

Graph neural networks learn from graph-structured data by passing messages between connected nodes. This file covers the message-passing framework, GCN, GraphSAGE, GIN, over-smoothing, graph pooling, and node/edge/graph-level tasks; the core architectures that power molecular property prediction, social network analysis, and recommendation systems.

- In the previous files, we established the mathematical foundations: geometric deep learning (file 1) tells us to exploit symmetries, and graph theory (file 2) gives us the language of nodes, edges, and adjacency. Now we build neural networks that operate directly on graphs.
- The core challenge: graph data is **irregular**. Unlike images (fixed grid) or sequences (fixed ordering), graphs have variable numbers of nodes, variable connectivity, and no canonical node ordering. A neural network for graphs must handle all of this while being permutation-equivariant (relabelling nodes should not change the output).

### 58.1 The Message-Passing Framework

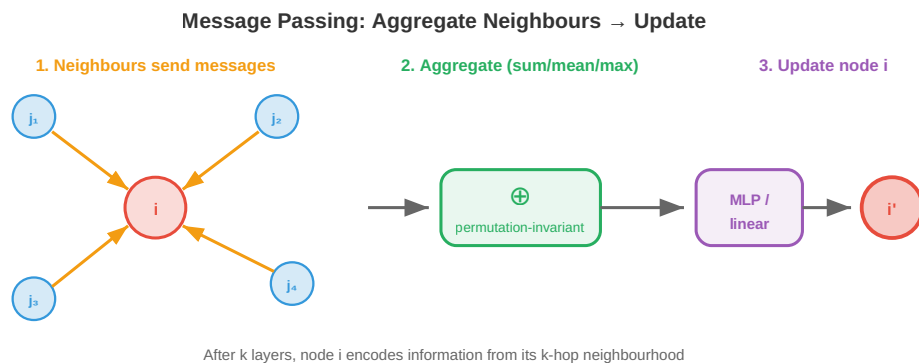
---

- Nearly all GNNs follow the same recipe, called **message passing** (also called neighbourhood aggregation). The idea is simple and elegant: each node updates its representation by collecting information from its neighbours.
- At each layer  $l$ , every node  $i$  does three things:
  1. **Message**: each neighbour  $j$  of node  $i$  computes a message  $\mathbf{m}_{j \rightarrow i}$  based on its current features.
  2. **Aggregate**: node  $i$  collects all incoming messages and combines them with a permutation-invariant function (sum, mean, or max).
  3. **Update**: node  $i$  combines the aggregated message with its own features to produce a new representation.
- Formally:

$$\mathbf{m}_i^{(l)} = \bigoplus_{j \in \mathcal{N}(i)} \phi^{(l)}(\mathbf{h}_i^{(l)}, \mathbf{h}_j^{(l)}, \mathbf{e}_{ij})$$

$$\mathbf{h}_i^{(l+1)} = \psi^{(l)}(\mathbf{h}_i^{(l)}, \mathbf{m}_i^{(l)})$$

- where  $\mathcal{N}(i)$  is the set of neighbours of node  $i$ ,  $\bigoplus$  is a permutation-invariant aggregation (sum, mean, max),  $\phi$  is the message function,  $\psi$  is the update function, and  $\mathbf{e}_{ij}$  is the optional edge feature.



Message passing: neighbours send messages, a permutation-invariant function aggregates them, and the node updates its features

- The aggregation  $\bigoplus$  must be permutation-invariant (it does not matter what order the neighbours are processed in) to ensure the overall function is permutation-equivariant. This directly implements the symmetry principle from file 1.
- After  $k$  layers of message passing, each node's representation encodes information from its  **$k$ -hop neighbourhood**: all nodes reachable within  $k$  edges. Layer 1 sees immediate neighbours, layer 2 sees neighbours of neighbours, and so on. This is how local information propagates to build global understanding.
- The receptive field of a GNN grows with depth, just like the receptive field of a CNN grows with layers (chapter 8). But unlike CNNs on regular grids, the receptive field shape varies per node depending on the graph topology.

## 58.2 Graph Convolutional Network (GCN)

- The **GCN** (Kipf & Welling, 2017) is the foundational GNN architecture. It simplifies spectral graph convolution (from file 2) into an elegant, efficient formula.

- Starting from the spectral convolution

$$\hat{\mathbf{x}} = \mathbf{U} \mathbf{g}_\theta \star \mathbf{x}$$

$$\mathbf{g}_\theta \star \mathbf{x} = \mathbf{U} \text{diag}(\mathbf{g}_\theta)$$

$\mathbf{U}^T \mathbf{x}$ , Kipf and Welling approximate the spectral filter with a first-order Chebyshev polynomial, which avoids computing the eigendecomposition entirely. After simplification, the layer-wise update becomes:

$$\hat{\mathbf{x}} = \mathbf{U} \mathbf{g}_\theta \star \mathbf{x}$$

$$\mathbf{H}^{(l+1)} = \sigma(\mathbf{A}$$

$$\mathbf{H}^{(l)} \mathbf{W}^{(l)})$$

- where:

- $\mathbf{H}^{(l)} \in \mathbb{R}^{n \times d}$  is the matrix of node features at layer  $l$

- $\mathbf{W}^{(l)} \in \mathbb{R}^{d \times d'}$  is a learnable weight matrix

- $\hat{\mathbf{A}} = \mathbf{D}^{-1/2} \mathbf{A} \mathbf{D}^{-1/2}$

$$\mathbf{A}$$

$$=$$

$$\hat{\mathbf{A}}$$

$$\mathbf{D}$$

$$^{-1/2}$$

$$\hat{\mathbf{A}}$$

$$\mathbf{A}$$

$$\hat{\mathbf{A}}$$

$$\mathbf{D}$$

$^{-1/2}$  is the symmetrically normalised adjacency matrix with self-loops

- $\hat{\mathbf{A}} = \mathbf{D}^{-1/2} \mathbf{A} \mathbf{D}^{-1/2}$

$$\mathbf{A}$$

$= \mathbf{A} + \mathbf{I}$  adds self-loops (so each node also receives its own message)

- $\mathbf{D}$

$$\mathbf{D}$$

is the degree matrix of

$$\hat{\mathbf{A}}$$

$$\mathbf{A}$$

- $\sigma$  is a nonlinear activation (ReLU, as in chapter 6)

- The matrix multiplication

$$\hat{\mathbf{x}} = \mathbf{U} \mathbf{g}_\theta \star \mathbf{x}$$

$$\mathbf{A}$$

$H^{(l)}$  is the aggregation step: for each node, it computes a weighted average of its neighbours' features (plus its own, via the self-loop). The weight matrix  $W^{(l)}$  is the learnable transformation, shared across all nodes. The activation adds nonlinearity.

- This is remarkably simple: it is just matrix multiplication followed by a learned linear map and activation. The entire GCN layer can be written in one line of code. The normalisation by

$D$

$D$

$^{-1/2}$  prevents nodes with many neighbours from dominating: high-degree nodes have their messages scaled down.

- In the message-passing framework, GCN uses:
  - Message:  $\phi(\mathbf{h}_j) = \mathbf{h}_j$  (just send your features)
  - Aggregation: normalised sum (weighted by degree)
  - Update: linear transformation + activation

## 58.3 GraphSAGE

---

- GCN is **transductive**: it requires the full graph during training and cannot handle new, unseen nodes. If a new user joins a social network, the GCN must be retrained on the entire graph. **GraphSAGE** (Hamilton et al., 2017) fixes this with an **inductive** approach.
- The key idea is **neighbourhood sampling**: instead of using all neighbours, sample a fixed-size subset. This makes computation independent of the full graph structure and allows generalisation to unseen nodes and graphs.
- The GraphSAGE update for node  $i$ :

$$\mathbf{h}_i^{(l+1)} = \sigma(W^{(l)} \cdot \text{CONCAT}(\mathbf{h}_i^{(l)}, \text{AGG}(\{\mathbf{h}_j^{(l)} : j \in \mathcal{S}(i)\})))$$

- where  $\mathcal{S}(i)$  is a **sampled** subset of neighbours (e.g., randomly sample 10 out of 500 neighbours). The CONCAT operation explicitly separates the node's own features from the aggregated neighbour features, letting the network learn different transformations for “self” and “neighbourhood.”

- GraphSAGE supports multiple aggregation functions:
  - **Mean:**  $AGG = \frac{1}{|S|} \sum_{j \in S} \mathbf{h}_j$  (simple, effective)
  - **LSTM:** feed the sampled neighbours through an LSTM (but this introduces an ordering dependency, somewhat violating permutation invariance)
  - **Pool:**  $AGG = \max(\{\sigma(W_{\text{pool}} \mathbf{h}_j + \mathbf{b})\})$  (nonlinear transform then max)
- The sampling strategy makes GraphSAGE scalable to very large graphs. Training uses mini-batches of nodes: for each target node, sample  $k_1$  neighbours at layer 1, then  $k_2$  neighbours for each of those at layer 2. With  $k_1 = k_2 = 10$  and 2 layers, each node's computation tree has at most  $10 \times 10 = 100$  nodes, regardless of the graph size.

## 58.4 Graph Isomorphism Network (GIN)

---

- Different GNN architectures have different **expressive power**: their ability to distinguish structurally different graphs. GCN and GraphSAGE, despite being effective in practice, are provably limited in what graph structures they can distinguish.
- The theoretical tool for measuring GNN expressiveness is the **Weisfeiler-Lehman (WL) test**, a classical algorithm for testing graph isomorphism (whether two graphs are structurally identical). The WL test iteratively refines node labels by hashing each node's label together with the multiset of its neighbours' labels.
- **GIN** (Xu et al., 2019) is designed to be as expressive as the WL test, making it the most powerful message-passing GNN (within the theoretical limits of message passing). The key insight: the aggregation function must be **injective** on multisets (different multisets of neighbour features must produce different aggregated values).
- Sum aggregation is injective on multisets (summing  $\{1, 1, 2\}$  gives 4, while  $\{1, 3\}$  gives 4 too, but over feature vectors with enough dimensions, sums of different multisets are generically distinct). Mean and max are not injective: mean cannot distinguish  $\{1, 1\}$  from  $\{2, 2\}$ , and max cannot distinguish  $\{1, 2, 3\}$  from  $\{1, 1, 3\}$ .
- The GIN update is:

$$\mathbf{h}_i^{(l+1)} = \text{MLP}^{(l)}((1 + \epsilon^{(l)}) \cdot \mathbf{h}_i^{(l)})$$

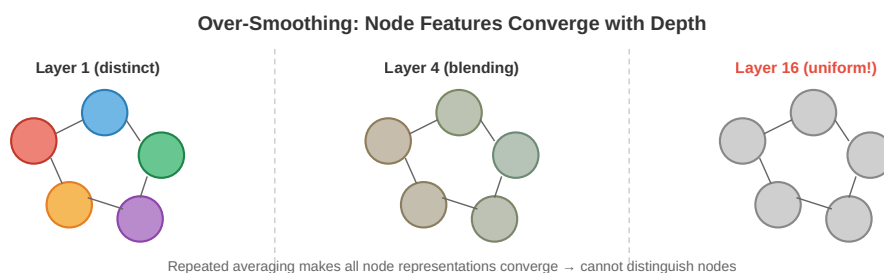
$$+ \sum_{j \in \mathcal{N}(i)}$$

$$\mathbf{h}_j^{(l)}$$

- where  $\epsilon$  is a learnable scalar (or fixed to 0) and the MLP provides the nonlinear, injective mapping. The sum aggregation preserves the multiset structure, and the MLP can learn to distinguish any two different aggregated values.

## 58.5 Over-Smoothing

- A major challenge in GNNs is **over-smoothing**: as the number of layers increases, all node representations converge to the same value, losing the ability to distinguish different nodes.



Over-smoothing: distinct node features at layer 1 gradually blend into uniform features at deeper layers

- The mechanism is intuitive. Each message-passing layer averages a node's features with its neighbours'. After many rounds of averaging, every node has seen (and blended with) every other node in its connected component. The features become a uniform average, the graph equivalent of blurring an image too many times until it becomes a solid colour.

- Formally, repeated application of the normalised adjacency

$$\hat{A}$$

$A$

converges to a rank-1 matrix (every row becomes proportional to the stationary distribution of a random walk on the graph). This is the same convergence as power iteration towards the dominant eigenvector (chapter 2).

- Over-smoothing limits GNNs to shallow depths (typically 2-4 layers), unlike CNNs and transformers that benefit from dozens or hundreds of layers. This



means each node can only see a limited neighbourhood, which is problematic for tasks requiring long-range information.

- Mitigations include:
  - **Residual connections** (from ResNets, chapter 8):  $\mathbf{h}_i^{(l+1)} = \mathbf{h}_i^{(l+1)} + \mathbf{h}_i^{(l)}$ , preserving information from earlier layers.
  - **Jumping knowledge**: concatenate or attention-pool representations from all layers, not just the last.
  - **DropEdge**: randomly remove edges during training, slowing the information spread.
  - **Graph Transformers** (file 4): bypass the local message-passing bottleneck with global attention.

## 58.6 Graph Pooling

---

- For **graph-level tasks** (predicting a property of the entire graph, like a molecule's toxicity), we need to collapse all node representations into a single graph-level vector. This is **graph pooling**, the graph analogue of global average pooling in CNNs (chapter 8).
- The simplest approach is **readout**: apply a permutation-invariant function to the set of all node features:

$$\mathbf{h}_G = \text{READOUT}(\{\mathbf{h}_i^{(L)} : i \in V\}) =$$

$$\sum_i \mathbf{h}_i^{(L)} \quad \text{or} \quad \frac{1}{|V|} \sum_i \mathbf{h}_i^{(L)}$$

$$\mathbf{h}_i^{(L)} \quad \text{or} \quad \max_i \mathbf{h}_i^{(L)}$$

- This is the DeepSets aggregation from file 1, applied after the final GNN layer. Sum preserves size information (a graph with 100 nodes will have a larger sum than one with 10), while mean normalises for size.

- **Hierarchical pooling** progressively coarsens the graph, mirroring how CNNs progressively downsample images. At each level, groups of nodes are merged into “supernodes” :
- **DiffPool** (Differentiable Pooling) learns a soft assignment matrix  $S^{(l)} \in \mathbb{R}^{n_l \times n_{l+1}}$  that assigns each node to a cluster:

$$X^{(l+1)} = S^{(l)T} H^{(l)}, \quad A^{(l+1)} = S^{(l)T} A^{(l)} S^{(l)}$$

- The assignment matrix is predicted by a separate GNN, making the clustering end-to-end differentiable. This creates a hierarchy: the original graph  $\rightarrow$  a coarsened graph with fewer nodes  $\rightarrow$  an even coarser graph  $\rightarrow$  a single node (the graph representation).
- **TopKPool** takes a simpler approach: learn a scalar score for each node, keep the top-k scoring nodes, and drop the rest. This is a hard selection (not soft assignment) and is computationally cheaper than DiffPool.

## 58.7 Heterogeneous Graphs

---

- All GNNs so far assume a **homogeneous graph**: one type of node, one type of edge. But most real-world graphs are **heterogeneous**: multiple node types and multiple edge types. A knowledge graph has person nodes, organisation nodes, and location nodes, connected by “works at,” “born in,” and “located in” edges. A recommender system has user nodes and item nodes connected by “purchased,” “viewed,” and “rated” edges.
- A heterogeneous graph has a **schema** (also called a metagraph) that defines the allowed node types and edge types. Each edge type connects a specific source type to a specific target type. For example, “works at” connects Person  $\rightarrow$  Organisation.
- **Relational GCN (R-GCN)** (Schlichtkrull et al., 2018) handles heterogeneous edges by using a separate weight matrix for each edge type:

$$\mathbf{h}_i^{(l+1)} = \sigma \left( \sum_{r \in \mathcal{R}} \right.$$

$$\left. \sum_{j \in \mathcal{N}_r(i)} \right.$$

$$\frac{1}{|\mathcal{N}_r(i)|} W_r^{(l)} \mathbf{h}_j^{(l)} + W_0^{(l)} \mathbf{h}_i^{(l)})$$

- where  $\mathcal{R}$  is the set of edge types,  $\mathcal{N}_r(i)$  is the set of neighbours connected to node  $i$  via relation  $r$ , and  $W_r$  is the weight matrix specific to relation  $r$ . The self-connection  $W_0$  handles the node's own features separately.
- The problem: with many relation types, the number of parameters explodes (one  $d \times d$  matrix per relation). R-GCN mitigates this with **basis decomposition**:  $W_r = \sum_{b=1}^B a_{rb} V_b$ , where  $V_b$  are shared basis matrices and  $a_{rb}$  are scalar coefficients per relation. This is analogous to low-rank factorisation (chapter 2): the relation-specific matrices live in a low-dimensional subspace.
- **Heterogeneous Graph Transformer (HGT)** (Hu et al., 2020) applies the attention mechanism to heterogeneous graphs. The key insight is that attention should depend on both the node types and the edge type connecting them. HGT uses type-specific projection matrices for queries, keys, and values:

$$\text{Attention}(i, j) = (W_{\tau(i)}^Q \mathbf{h}_i)^T \cdot \frac{W_{\phi(i, j)}^{\text{ATT}}}{\sqrt{d}} \cdot (W_{\tau(j)}^K \mathbf{h}_j)$$

- where  $\tau(i)$  is the type of node  $i$  and  $\phi(i, j)$  is the edge type between them. This ensures that the model attends differently to different relationship types: a paper attending to its authors should use different attention weights than when attending to its references.
- **Metapath-based methods** define meaningful paths through the schema (e.g., Author  $\rightarrow$  Paper  $\rightarrow$  Author for co-authorship) and aggregate information along these paths. **HAN** (Heterogeneous Attention Network) applies attention at two levels: within each metapath (which neighbours along this path matter?) and across metapaths (which relationship patterns matter?).

## 58.8 Link Prediction and Knowledge Graph Completion

---

- **Link prediction** asks: given the existing edges, which missing edges are likely to exist? This is the core task for knowledge graph completion (predict missing facts), recommendation (predict which items a user will like), and social network analysis (predict future friendships).
- **Embedding-based methods** learn a vector for each entity and a transformation for each relation, then score potential edges by how well the entities and relation fit together:
- **TransE** models relations as translations in embedding space: if  $(h, r, t)$  is a valid triple (head entity, relation, tail entity), then  $\mathbf{h} + \mathbf{r} \approx \mathbf{t}$ . The scoring function is  $f(h, r, t) = - \|\mathbf{h} + \mathbf{r} - \mathbf{t}\|$ . Intuitively, the relation vector “moves” the head entity to the tail entity in embedding space.
- **RotatE** models relations as rotations in complex space:  $\mathbf{t} = \mathbf{h} \circ \mathbf{r}$ , where  $\circ$  is element-wise complex multiplication and  $|\mathbf{r}_i| = 1$  (unit complex numbers are rotations). This can model symmetry, antisymmetry, inversion, and composition patterns that TransE cannot.
- **Complex** uses complex-valued embeddings with a Hermitian dot product, enabling it to model asymmetric relations (if A is the boss of B, B is not the boss of A).
- GNN-based link prediction computes node embeddings with message passing, then scores edges using the endpoint embeddings. This combines the structural reasoning of GNNs with the relational modelling of embedding methods. The GNN encoder captures multi-hop neighbourhood structure that single-embedding methods miss.

## 58.9 Task Types

---

- GNNs solve three categories of tasks:
- **Node-level tasks:** predict a property for each node. Examples: classifying users in a social network (bot or human), predicting the function of each protein in an interaction network, semi-supervised node classification (label a few nodes, predict the rest). The output is the node embedding  $\mathbf{h}_i^{(L)}$  passed through a classifier.

- **Edge-level tasks:** predict a property for each edge or predict whether an edge exists. Examples: link prediction (will these two users become friends?), knowledge graph completion (does this relationship hold between these entities?), drug-drug interaction prediction. The output typically uses the embeddings of both endpoint nodes:

$\hat{y}_{ij}$

$y$

$\hat{y}_{ij} = f(\mathbf{h}_i, \mathbf{h}_j)$ , where  $f$  is a dot product, concatenation + MLP, or other combination.

- **Graph-level tasks:** predict a property for the entire graph. Examples: molecular property prediction (is this molecule toxic?), graph classification (is this social network a bot network?), graph generation (design a molecule with desired properties). The output uses graph pooling to produce  $\mathbf{h}_G$ , which is then classified or regressed.

## 58.10 Coding Tasks (use CoLab or notebook)

1. Implement a single GCN layer from scratch using the normalised adjacency matrix. Apply it to a small graph and observe how node features are smoothed.

```
import jax
import jax.numpy as jnp

Graph: 5 nodes, simple chain with a branch
A = jnp.array([[0, 1, 0, 0, 0],
 [1, 0, 1, 0, 0],
 [0, 1, 0, 1, 1],
 [0, 0, 1, 0, 0],
 [0, 0, 1, 0, 0]], dtype=float)

Add self-loops
A_hat = A + jnp.eye(5)
D_hat = jnp.diag(A_hat.sum(axis=1))
D_inv_sqrt = jnp.diag(1.0 / jnp.sqrt(A_hat.sum(axis=1)))
A_norm = D_inv_sqrt @ A_hat @ D_inv_sqrt

Node features: one-hot identity
H = jnp.eye(5)

Weight matrix (random initialisation)
rng = jax.random.PRNGKey(0)
```

```

W = jax.random.normal(rng, (5, 3)) * 0.5

GCN layer: $H' = \text{ReLU}(A_{\text{norm}} @ H @ W)$
H_new = jax.nn.relu(A_norm @ H @ W)

print("Original features (one-hot):")
print(H)
print("\nAfter GCN layer:")
print(jnp.round(H_new, 3))
print("\nNotice: connected nodes now have similar representations")

```

1. Implement message passing with sum aggregation (GIN-style) and compare with mean aggregation (GCN-style). Show that sum can distinguish multisets that mean cannot.

```

import jax.numpy as jnp

Two different neighbourhood multisets that have the same mean
Node A: neighbours have features [1, 1, 1, 1] (four neighbours, all 1)
Node B: neighbours have features [2, 2] (two neighbours, all 2)

neighbours_A = jnp.array([[1.0], [1.0], [1.0], [1.0]])
neighbours_B = jnp.array([[2.0], [2.0]])

Mean aggregation
mean_A = neighbours_A.mean(axis=0)
mean_B = neighbours_B.mean(axis=0)
print(f"Mean A: {mean_A}, Mean B: {mean_B}, Same: {jnp.allclose(mean_A, mean_B)}")

Sum aggregation
sum_A = neighbours_A.sum(axis=0)
sum_B = neighbours_B.sum(axis=0)
print(f"Sum A: {sum_A}, Sum B: {sum_B}, Same: {jnp.allclose(sum_A, sum_B)}")
print("\nSum distinguishes these multisets; mean does not!")

```

1. Demonstrate over-smoothing. Apply the normalised adjacency repeatedly and watch node features converge.

```

import jax.numpy as jnp
import matplotlib.pyplot as plt

Random graph
A = jnp.array([[0, 1, 1, 0, 0, 0],
 [1, 0, 1, 0, 0, 0],

```

```

 [1,1,0,1,0,0],
 [0,0,1,0,1,1],
 [0,0,0,1,0,1],
 [0,0,0,1,1,0]], dtype=float)

A_hat = A + jnp.eye(6)
D_inv_sqrt = jnp.diag(1.0 / jnp.sqrt(A_hat.sum(axis=1)))
A_norm = D_inv_sqrt @ A_hat @ D_inv_sqrt

Initial features: distinct per node
H = jnp.array([[1,0], [0,1], [1,1], [-1,0], [0,-1], [-1,-1]], dtype=float)

distances = []
for k in range(20):
 H = A_norm @ H
 # Measure how distinct the features are (std across nodes)
 spread = jnp.std(H, axis=0).mean()
 distances.append(float(spread))

plt.plot(distances, "o-")
plt.xlabel("Number of message-passing rounds")
plt.ylabel("Feature spread (std across nodes)")
plt.title("Over-Smoothing: Features Converge with Depth")
plt.show()

```

## 59 Graph Attention Networks

Graph attention networks replace uniform neighbour aggregation with learned, data-dependent weighting. This file covers GAT, multi-head graph attention, GATv2, Graph Transformers, positional and structural encodings, and scalability

- In the GCN (file 3), each node aggregates its neighbours' features using fixed weights determined by the graph structure (the normalised adjacency). A node with three neighbours gives each neighbour roughly equal weight ( $\approx 1/3$ ). But not all neighbours are equally important: a message from a close collaborator should matter more than one from a distant acquaintance.
- **Graph Attention Networks** solve this by learning **which neighbours to attend to**, using the same attention mechanism that powers transformers (chapter 7). Instead of fixed, structure-based weights, each node computes dynamic, content-based attention scores over its neighbours.

## 59.1 GAT: Graph Attention Network

- GAT (Veličković et al., 2018) computes attention coefficients between each node and its neighbours. For node  $i$  and neighbour  $j$ :

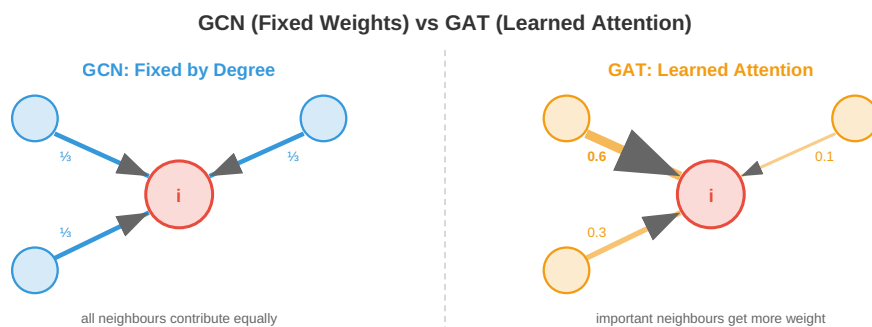
$$e_{ij} = \text{LeakyReLU}(\mathbf{a}^T [\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_j])$$

- where  $\mathbf{W} \in \mathbb{R}^{d' \times d}$  is a shared linear transformation,  $\parallel$  denotes concatenation, and  $\mathbf{a} \in \mathbb{R}^{2d'}$  is a learnable attention vector. The score  $e_{ij}$  measures how important node  $j$ 's features are to node  $i$ .
- The raw scores are normalised across all neighbours using softmax:

$$\alpha_{ij} = \text{softmax}_j(e_{ij}) = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}(i)} \exp(e_{ik})}$$

- This ensures attention weights sum to 1 across each node's neighbourhood, just like transformer attention (chapter 7). The node's updated features are:

$$\mathbf{h}_i' = \sigma \left( \sum_{j \in \mathcal{N}(i)} \alpha_{ij} \mathbf{W}\mathbf{h}_j \right)$$



GCN assigns fixed equal weights to all neighbours; GAT learns data-dependent attention weights

- The crucial difference from GCN: the weights  $\alpha_{ij}$  are **learned from the data**, not fixed by the graph structure. A node can learn to focus on the most informative neighbours while ignoring noisy or irrelevant ones.



- Note that attention is computed only over edges (node  $i$  attends only to its neighbours  $\mathcal{N}(i)$ ), not over all node pairs. This keeps computation proportional to the number of edges, not the square of the number of nodes.

## 59.2 Multi-Head Graph Attention

---

- Just as in transformers (chapter 7), **multi-head attention** runs  $K$  independent attention mechanisms in parallel, each with its own parameters  $W^k$  and  $\mathbf{a}^k$ . The results are concatenated (in intermediate layers) or averaged (in the final layer):

$$\mathbf{h}_i' = \parallel_{k=1}^K \sigma \left( \sum_{j \in \mathcal{N}(i)} \alpha_{ij}^k W^k \mathbf{h}_j \right)$$

- Each head can attend to different aspects of the neighbourhood: one head might focus on structural features, another on semantic similarity. This is the same motivation as multi-head attention in transformers: different heads capture different types of relationships.
- With  $K$  heads and output dimension  $d'$  per head, the concatenated output has dimension  $K \times d'$ . The final layer typically averages instead of concatenating to produce a fixed-size output.

## 59.3 GATv2: Fixing Static Attention

---

- The original GAT has a subtle limitation: its attention function is **static** (also called ranking-based). The attention score depends on the concatenation  $[W\mathbf{h}_i \parallel W\mathbf{h}_j]$ , but because the attention vector  $\mathbf{a}$  is applied after concatenation, it can be decomposed into two independent components:  

$$\mathbf{a}^T [W\mathbf{h}_i \parallel W\mathbf{h}_j] = \mathbf{a}_1^T W\mathbf{h}_i + \mathbf{a}_2^T W\mathbf{h}_j.$$
- This means the ranking of neighbours for a given node  $i$  is determined entirely by the neighbours' features  $\mathbf{h}_j$  (the term  $\mathbf{a}_1^T W\mathbf{h}_i$  is constant across all neighbours of  $i$ ). The attention ranking does not truly depend on the query node's features. Node  $i$  and node  $k$  will rank the same set of neighbours identically, which limits expressiveness.
- **GATv2** (Brody et al., 2022) fixes this by applying the nonlinearity before the attention vector:

$$e_{ij} = \mathbf{a}^T \text{LeakyReLU}(W[\mathbf{h}_i \parallel \mathbf{h}_j])$$

- Moving LeakyReLU inside the computation means the attention score is a nonlinear function of the joint features, not decomposable into independent terms. This makes attention **dynamic**: the ranking of neighbours now depends on the specific query node. GATv2 is strictly more expressive than GAT with no additional computational cost.

## 59.4 Graph Transformers

---

- Standard message-passing GNNs are limited by the graph topology: a node can only attend to its direct neighbours. After  $k$  layers, information from  $k$ -hop neighbours has been mixed through multiple aggregation steps, losing fidelity. This local bottleneck (combined with over-smoothing, file 3) limits the ability to capture long-range dependencies.
- **Graph Transformers** break this bottleneck by applying **global self-attention** to all node pairs, regardless of whether they share an edge. Every node can attend to every other node in a single layer, just like in a standard transformer (chapter 7).
- The basic idea: treat all nodes as tokens and apply transformer self-attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- where  $Q = XW_Q$ ,  $K = XW_K$ ,  $V = XW_V$  are the query, key, and value projections of the node features  $X$  (exactly as in chapter 7). This is a GNN on a fully connected graph (complete graph  $K_n$ , file 2).
- The problem: a fully connected graph ignores the actual graph structure. The edge information (who is actually connected to whom) is lost. Two approaches restore this:
- **Graphormer** (Ying et al., 2021) injects graph structure into the transformer via **bias terms** in the attention scores:

$$A_{ij} = \frac{(\mathbf{h}_i W_Q)(W_K^T \mathbf{h}_j^T)}{\sqrt{d_k}} + b_{\text{spatial}}(i, j) + b_{\text{edge}}(i, j)$$

- The spatial bias  $b_{\text{spatial}}$  encodes the shortest-path distance between nodes  $i$  and  $j$ . The edge bias  $b_{\text{edge}}$  encodes edge features along the shortest path.

Additionally, Graphormer uses a **centrality encoding** that adds the node's degree to its input embedding, giving the model information about each node's structural role.

- **GPS** (General, Powerful, Scalable Graph Transformer, Rampášek et al., 2022) combines local message passing with global attention in each layer:

$$\mathbf{h}_i' = \text{MLP}(\mathbf{h}_i^{\text{MPNN}} + \mathbf{h}_i^{\text{Attention}})$$

- Each layer applies both a standard GNN (for local structure) and a transformer (for global context), then combines the results. This gets the best of both worlds: local structure from message passing and long-range dependencies from attention.

## 59.5 Positional and Structural Encodings

---

- Transformers on sequences use positional encodings (chapter 7) to inject order information. Graphs have no canonical ordering, so graph-specific encodings are needed.
- **Laplacian eigenvector encodings** use the eigenvectors of the graph Laplacian (file 2) as positional features. The  $k$  smallest non-trivial eigenvectors provide a spectral embedding of the graph: nodes that are “nearby” in the graph have similar eigenvector values. These are concatenated to the node features.
- A subtlety: Laplacian eigenvectors have a sign ambiguity (if  $\mathbf{u}$  is an eigenvector, so is  $-\mathbf{u}$ ). The model must be invariant to these sign flips. Solutions include using random sign flips as data augmentation during training, or learning sign-invariant transformations.
- **Random walk encodings** compute the probability of a random walk starting at node  $i$  returning to node  $i$  after  $k$  steps, for  $k = 1, 2, \dots, K$ . These probabilities encode local structural information: nodes in dense clusters have high return probabilities, while nodes in sparse regions have low ones. The landing

probability  $p_{ii}^{(k)} = (A_{\text{rw}}^k)_{ii}$  where  $A_{\text{rw}} = D^{-1}A$  is the random walk transition matrix.

- **Degree encodings** simply add the node degree as a feature. This is surprisingly effective because degree is a strong structural signal: leaf nodes (degree 1), bridge nodes, and hub nodes behave differently.
- These encodings provide the structural information that vanilla transformers lack, enabling Graph Transformers to outperform standard message-passing GNNs on tasks requiring long-range reasoning.

## 59.6 Scalability

---

- The fundamental scalability challenge for GNNs is that graphs can have millions of nodes and billions of edges. Training a GNN on the full graph requires storing all node features and the entire adjacency matrix in memory, which is often infeasible.
- **Mini-batch training** for GNNs is more complex than for images or sequences because nodes are interconnected. Naively sampling a batch of nodes requires their neighbours (layer 1), their neighbours' neighbours (layer 2), and so on. This **neighbourhood explosion** means a batch of 1000 target nodes might require millions of nodes in the computation graph.
- **Neighbourhood sampling** (GraphSAGE-style, file 3) limits the explosion by sampling a fixed number of neighbours per node per layer. With 2 layers and 15 samples per layer, each target node's subgraph has at most  $15^2 = 225$  nodes, regardless of the full graph size.
- **Cluster-GCN** (Chiang et al., 2019) partitions the graph into clusters using a graph clustering algorithm (e.g., METIS), then trains on one cluster at a time. Within-cluster edges are dense (most neighbours are in the same cluster), so the subgraph captures the relevant structure. Cross-cluster edges are handled by occasionally including edges between clusters.
- **Graph Transformer scalability** is harder because global attention is  $O(n^2)$ . For graphs with millions of nodes, full attention is infeasible. Solutions include:
  - Sparse attention patterns (attend only to k-nearest nodes in the graph)
  - Linear attention approximations

- Combining local message passing (cheap,  $O(|E|)$ ) with global attention on a coarsened graph (fewer nodes)

## 59.7 Temporal and Dynamic Graphs

---

- The graphs we have studied so far are **static**: the nodes, edges, and features are fixed. But many real-world graphs **evolve over time**: new users join social networks, financial transactions create edges, traffic patterns shift throughout the day, and molecular interactions fluctuate.
- A **temporal graph** augments each edge with a timestamp:  $(i, j, t)$  means node  $i$  interacted with node  $j$  at time  $t$ . The challenge is to learn representations that capture both the graph structure and the temporal dynamics.
- There are two paradigms:
- **Discrete-time dynamic graphs (DTDG)**: the graph is represented as a sequence of snapshots  $G_1, G_2, \dots, G_T$ , one per timestep. A GNN processes each snapshot, and an RNN or temporal attention mechanism captures the evolution across snapshots. This is simple but loses fine-grained timing information (events between snapshots are lost) and requires choosing a snapshot frequency.
- **Continuous-time dynamic graphs (CTDG)**: events are modelled as a stream of timestamped interactions. Each event  $(i, j, t)$  updates the representations of nodes  $i$  and  $j$  at the exact time it occurs. This preserves all temporal information.
- **Temporal Graph Network (TGN)** (Rossi et al., 2020) is the leading CTDG architecture. Each node maintains a **memory state**  $\mathbf{s}_i(t)$  that is updated whenever the node participates in an interaction:

$$\mathbf{s}_i(t^+) = \text{GRU}(\mathbf{s}_i(t^-), \mathbf{m}_i(t))$$

- where  $\mathbf{m}_i(t)$  is a message computed from the interaction (combining the features of both nodes, the edge features, and the time encoding). The GRU (chapter 6) selectively retains and forgets past information, allowing the memory to capture long-term patterns while adapting to recent events.
- **Time encoding** represents the elapsed time since the last interaction as a feature vector, analogous to positional encoding in transformers (chapter 7). A common approach uses learnable Fourier features:

$$\Phi(t) = [\cos(\omega_1 t), \sin(\omega_1 t), \dots, \cos(\omega_d t), \sin(\omega_d t)]$$

- This gives the model a rich representation of temporal gaps: “this user was last active 5 minutes ago” vs “3 months ago” are embedded differently.
- **Temporal Graph Attention (TGAT)** applies self-attention over a node’s temporal neighbourhood: the set of recent interactions, each weighted by both feature relevance (like GAT) and temporal recency. Interactions from the distant past are naturally down-weighted.
- Applications include fraud detection (anomalous transaction patterns in financial graphs), traffic forecasting (predicting congestion from historical flow patterns), social network dynamics (predicting viral content spread), and drug interaction prediction over time.

## 59.8 Coding Tasks (use CoLab or notebook)

1. Implement a single GAT attention head from scratch. Compute attention weights between a node and its neighbours and verify they sum to 1.

```
import jax
import jax.numpy as jnp

rng = jax.random.PRNGKey(0)
k1, k2, k3 = jax.random.split(rng, 3)

n_nodes, d_in, d_out = 5, 4, 3

Random node features
H = jax.random.normal(k1, (n_nodes, d_in))

Learnable parameters
W = jax.random.normal(k2, (d_in, d_out)) * 0.5
a = jax.random.normal(k3, (2 * d_out,)) * 0.5

Adjacency (node 0 connects to 1, 2, 3)
neighbours_of_0 = [1, 2, 3]

Transform features
Wh = H @ W # (n_nodes, d_out)

Compute attention scores for node 0
h_i = Wh[0]
scores = []
```

```

for j in neighbours_of_0:
 h_j = Wh[j]
 e_ij = jnp.dot(a, jnp.concatenate([h_i, h_j]))
 e_ij = jax.nn.leaky_relu(e_ij, negative_slope=0.2)
 scores.append(float(e_ij))

scores = jnp.array(scores)
alpha = jax.nn.softmax(scores)

print(f"Raw scores: {scores}")
print(f"Attention weights: {alpha}")
print(f"Sum of weights: {alpha.sum():.4f}")

Weighted aggregation
h_new = sum(alpha[k] * Wh[neighbours_of_0[k]] for k in
range(len(neighbours_of_0)))
print(f"Updated node 0 features: {h_new}")

```

1. Compare GCN (fixed weights) vs GAT (learned weights) aggregation. Show that GAT can assign different weights to neighbours while GCN treats them uniformly.

```

import jax
import jax.numpy as jnp

4 nodes: node 0 connects to 1, 2, 3
A = jnp.array([[0,1,1,1],
 [1,0,0,0],
 [1,0,0,0],
 [1,0,0,0]], dtype=float)

Features: node 1 is very relevant, node 2 is noise, node 3 is moderate
H = jnp.array([[0.0, 0.0], # node 0
 [1.0, 0.0], # node 1 (signal)
 [0.0, 0.0], # node 2 (noise)
 [0.5, 0.0]]) # node 3 (moderate)

GCN: normalised adjacency weights
A_hat = A + jnp.eye(4)
D_inv = jnp.diag(1.0 / A_hat.sum(axis=1))
gcn_weights = (D_inv @ A_hat)[0] # weights for node 0
print(f"GCN weights for node 0: {gcn_weights}")
print(" → All neighbours get roughly equal weight")

GAT: learned attention (simulated)
Suppose the attention mechanism learns to focus on node 1

```

```

gat_weights = jnp.array([0.1, 0.7, 0.05, 0.15]) # learned
print(f"\nGAT weights for node 0: {gat_weights}")
print(" → Node 1 (informative) gets most attention")

gcn_output = gcn_weights @ H
gat_output = gat_weights @ H
print(f"\nGCN output: {gcn_output} (diluted by noise)")
print(f"GAT output: {gat_output} (focused on signal)")

```

1. Demonstrate the benefit of positional encodings. Compute Laplacian eigenvector encodings for a graph and show that structurally similar nodes get similar encodings.

```

import jax.numpy as jnp
import matplotlib.pyplot as plt

Barbell graph: two cliques connected by a bridge
n = 10
A = jnp.zeros((n, n))
Clique 1: nodes 0-4
for i in range(5):
 for j in range(i+1, 5):
 A = A.at[i,j].set(1).at[j,i].set(1)
Clique 2: nodes 5-9
for i in range(5, 10):
 for j in range(i+1, 10):
 A = A.at[i,j].set(1).at[j,i].set(1)
Bridge
A = A.at[4,5].set(1).at[5,4].set(1)

D = jnp.diag(A.sum(axis=1))
L = D - A
eigenvalues, eigenvectors = jnp.linalg.eigh(L)

Use first 3 non-trivial eigenvectors as positional encoding
pe = eigenvectors[:, 1:4]

print("Laplacian Positional Encodings:")
for i in range(n):
 group = "Clique 1" if i < 5 else "Clique 2"
 bridge = " (bridge)" if i in [4, 5] else ""
 print(f" Node {i} ({group}{bridge}): {pe[i]}")

plt.scatter(pe[:5, 0], pe[:5, 1], c="#3498db", s=80, label="Clique 1")
plt.scatter(pe[5:, 0], pe[5:, 1], c="#e74c3c", s=80, label="Clique 2")
plt.scatter(pe[[4,5], 0], pe[[4,5], 1], c="black", s=120, marker="*",

```



```
label="Bridge nodes", zorder=5)
plt.legend(); plt.grid(True)
plt.title("Laplacian Eigenvector Positional Encodings")
plt.xlabel("Eigenvector 1"); plt.ylabel("Eigenvector 2")
plt.show()
```

## 60 3D Graph Networks

3D graph networks extend GNNs to data with spatial geometry, where rotations and translations must be handled correctly. This file covers geometric graphs, SE(3)/E(n)-equivariance, SchNet, DimeNet, EGNN, tensor field networks, and applications in molecular property prediction, protein structure, materials science, and drug discovery – the architectures that learn from the 3D physical world.

- The GNNs in files 3 and 4 operate on abstract graphs: nodes have features, edges encode connectivity, but there is no notion of 3D space. A social network graph has no geometry. But many of the most impactful applications of GNNs involve data that lives in **physical 3D space**: molecules, proteins, crystals, point clouds. For these, the spatial positions of nodes carry critical information that abstract GNNs ignore.
- The challenge is that 3D data has **geometric symmetries** (file 1): rotating a molecule does not change its properties, and neither does translating it. A 3D GNN must respect these symmetries. An energy prediction that changes when you rotate the molecule is physically wrong.

### 60.1 Geometric Graphs

---

- A **geometric graph** is a graph embedded in 3D space. Each node  $i$  has a position  $\mathbf{r}_i \in \mathbb{R}^3$  in addition to its feature vector  $\mathbf{h}_i$ . Edges may be defined by spatial proximity (connect nodes within distance  $r_{\text{cut}}$ ) rather than by explicit bonds.
- For molecules, the geometric graph has atoms as nodes (with features: element type, charge, etc.) and chemical bonds as edges. The 3D positions  $\mathbf{r}_i$  are the atom coordinates, determined by quantum mechanics or experimental measurement (X-ray crystallography, cryo-EM).
- For point clouds (from LiDAR or 3D scanners, chapter 8 and chapter 11), each point is a node with position and optional features (colour, intensity). Edges

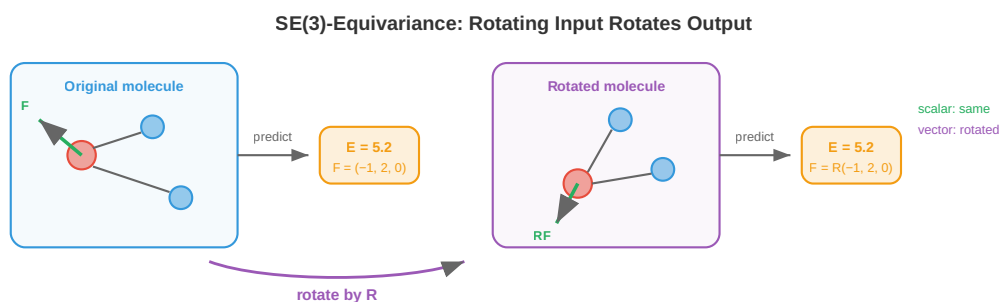
connect nearby points, forming a **k-nearest-neighbour (kNN) graph** or a radius graph.

- The key geometric quantities for message passing are:
  - **Interatomic distances:**  $d_{ij} = \| \mathbf{r}_i - \mathbf{r}_j \|$ . Distances are invariant to rotation and translation. Two molecules with identical interatomic distances are the same shape, regardless of orientation.
  - **Bond angles:** the angle  $\theta_{ijk}$  between vectors  $\mathbf{r}_j - \mathbf{r}_i$  and  $\mathbf{r}_k - \mathbf{r}_i$  at node  $i$ . Angles capture local geometry beyond pairwise distances.
  - **Dihedral (torsion) angles:** the angle  $\phi_{ijkl}$  between the planes defined by  $(i, j, k)$  and  $(j, k, l)$ . Dihedrals capture how the structure twists in 3D, essential for protein backbone geometry.
  - **Relative position vectors:**  $\mathbf{r}_{ij} = \mathbf{r}_j - \mathbf{r}_i$ . These are translation-invariant but NOT rotation-invariant. Using them requires equivariant (not just invariant) architectures.

## 60.2 SE(3) and E(n) Equivariance

---

- The symmetry group for 3D physical data is the **Euclidean group**  $E(3)$ , consisting of all rotations, reflections, and translations. The subgroup **SE(3)** (Special Euclidean) includes rotations and translations but excludes reflections.
- A 3D GNN should be:
  - **Translation-invariant** for scalar outputs (energy, binding affinity): shifting all atoms by the same vector should not change the prediction.
  - **Rotation-invariant** for scalar outputs: rotating the molecule should not change its energy.
  - **Rotation-equivariant** for vector/tensor outputs (forces, dipole moments): rotating the molecule should rotate the predicted force vectors by the same rotation.



SE(3)-equivariance: rotating a molecule leaves scalar predictions (energy) unchanged but rotates vector predictions (forces) correspondingly

- Formally, for a scalar prediction  $f$  and rotation  $R \in \text{SO}(3)$ :

$$f(R\mathbf{r}_1, R\mathbf{r}_2, \dots) = f(\mathbf{r}_1, \mathbf{r}_2, \dots) \text{ (invariance)}$$

- For a vector prediction  $\mathbf{F}$ :

$$\mathbf{F}(R\mathbf{r}_1, R\mathbf{r}_2, \dots) = R \cdot \mathbf{F}(\mathbf{r}_1, \mathbf{r}_2, \dots) \text{ (equivariance)}$$

- These constraints directly mirror the invariance/equivariance framework from file 1, now applied specifically to the 3D rotation and translation groups.
- There are two design approaches:
  - Invariant architectures:** use only invariant geometric features (distances, angles) as inputs to message passing. The internal representations are scalars (invariant). Simple and efficient but cannot produce vector outputs without breaking symmetry.
  - Equivariant architectures:** maintain vector (and higher-order tensor) representations throughout the network, ensuring each layer is equivariant. More expressive, can naturally predict vectors and tensors, but more complex.

## 60.3 SchNet: Distance-Based Message Passing

- SchNet** (Schütt et al., 2017) is the foundational invariant 3D GNN. Its key innovation is **continuous filter convolution**: instead of using a fixed set of edge types (like bond type in molecular GNNs), SchNet generates the message filter directly from the interatomic distance.
- The distance  $d_{ij}$  is first expanded into a feature vector using **radial basis functions (RBFs)**:

$$\text{RBF}(d_{ij}) = [\exp(-\gamma_1(d_{ij} - \mu_1)^2), \dots, \exp(-\gamma_K(d_{ij} - \mu_K)^2)]$$

- Each basis function is a Gaussian centred at  $\mu_k$  with width  $\gamma_k$ . This is analogous to a learnable positional encoding for distances: the continuous distance is mapped to a high-dimensional feature space where the network can learn distance-dependent interactions. The centres  $\mu_k$  are typically evenly spaced from 0 to the cutoff radius.
- The SchNet message from node  $j$  to node  $i$  is:

$$\mathbf{m}_{j \rightarrow i} = \mathbf{h}_j \odot W_{\text{filter}}(\text{RBF}(d_{ij}))$$

- where  $W_{\text{filter}}$  is an MLP that maps the RBF expansion to a filter vector, and  $\odot$  is element-wise multiplication (Hadamard product, chapter 2). The filter depends on the distance, so nearby atoms interact differently from distant atoms. The element-wise multiplication is like a gating mechanism (chapter 6): the distance-dependent filter controls how much of each feature dimension passes through.
- Because SchNet uses only distances (invariant), the entire model is automatically invariant to rotations and translations. No special handling of symmetry is needed beyond this design choice.

## 60.4 DimeNet and SphereNet: Angles and Dihedrals

---

- Distances alone cannot fully specify 3D structure. Two different molecular conformations can have identical pairwise distances but different bond angles (this is the problem of “distance geometry ambiguity”). **DimeNet** (Gasteiger et al., 2020) incorporates **bond angles** into message passing.
- DimeNet uses **directional message passing**: messages flow along directed edges, and the message on edge ( $j \rightarrow i$ ) is influenced by the angle between edges ( $k \rightarrow j$ ) and ( $j \rightarrow i$ ):

$$\mathbf{m}_{kj \rightarrow ji} = f(\mathbf{m}_{kj}, d_{ji}, \theta_{kji})$$

- The angle  $\theta_{kji}$  is expanded using spherical Bessel functions and spherical harmonics (the natural basis for angular information on a sphere, analogous to RBFs for distances). This gives the model access to directional information while maintaining invariance.

- **SphereNet** (Liu et al., 2022) goes further by including **dihedral angles**  $\phi_{lkji}$ , capturing the full 3D torsional structure. The hierarchy is:
  - Distances  $\rightarrow$  captures pairwise proximity
  - Angles  $\rightarrow$  captures local geometry (bent vs. linear)
  - Dihedrals  $\rightarrow$  captures 3D twisting (crucial for protein backbone, drug binding)
- Each level adds geometric resolution at the cost of computational complexity (distances are  $O(|E|)$ , angles are  $O(|E| \cdot k)$ , dihedrals are  $O(|E| \cdot k^2)$  where  $k$  is the average degree).

## 60.5 E(n) Equivariant GNNs (EGNN)

---

- **EGNN** (Satorras et al., 2021) takes the equivariant approach: instead of using only invariant features, it updates both node features **and** node positions at each layer, maintaining equivariance throughout.
- The EGNN update for node  $i$ :

$$\mathbf{m}_{ij} = \phi_e(\mathbf{h}_i, \mathbf{h}_j, d_{ij}^2, a_{ij})$$

$$\mathbf{r}_i' = \mathbf{r}_i + C$$

$$\sum_{j \neq i}$$

$$(\mathbf{r}_i - \mathbf{r}_j) \cdot \phi_r(\mathbf{m}_{ij})$$

$$\mathbf{h}_i' = \phi_h(\mathbf{h}_i,$$

$$\sum_j$$

$$\mathbf{m}_{ij})$$

- The key is the position update: node positions are adjusted by a weighted sum of relative position vectors  $(\mathbf{r}_i - \mathbf{r}_j)$ . The weights come from the message function  $\phi_r$ , which depends only on invariant quantities (features and distances). This construction is **provably equivariant**: if all input positions are rotated by  $R$ , all output positions are rotated by the same  $R$ .
- EGNN is elegant because it achieves equivariance without explicitly using spherical harmonics or irreducible representations. The relative position

vectors carry the directional information, and the invariant message function controls how that directional information is used.

- The simplicity comes with a tradeoff: EGNN only uses vector representations (order 1). It cannot represent higher-order tensors like quadrupole moments or stress tensors without extension.

## 60.6 Tensor Field Networks and Higher-Order Representations

- **Tensor Field Networks** (Thomas et al., 2018) and their successors (**SE(3)-Transformers**, **MACE**, **Equiformer**) use the full machinery of **irreducible representations** of the rotation group to build equivariant layers.
- In representation theory (connecting to the linear algebra of chapter 2), rotations in 3D can be decomposed into irreducible components characterised by an integer order  $\ell$ :
  - $\ell = 0$ : scalars (1 component, invariant). Energy, charge.
  - $\ell = 1$ : vectors (3 components, rotate like position vectors). Forces, dipole moments.
  - $\ell = 2$ : rank-2 symmetric traceless tensors (5 components). Quadrupole moments, stress tensors.
  - Higher  $\ell$ : capture increasingly complex angular structure.
- These are called **spherical tensors**, and they transform under rotation  $R$  via the **Wigner-D matrices**  $D^\ell(R)$ : a scalar is unchanged, a vector rotates by  $R$ , a rank-2 tensor rotates by a more complex matrix.
- **Equivariant message passing** with spherical tensors uses the **Clebsch-Gordan tensor product** to combine features of different orders:

$$(\mathbf{f}_1^\ell \otimes \mathbf{f}_2^\ell)^{\ell_{\text{out}}} = \sum_{m_1, m_2} C_{\ell_1, m_1, \ell_2, m_2}^{\ell_{\text{out}}, m_{\text{out}}} \cdot f_{m_1}^{\ell_1} \cdot f_{m_2}^{\ell_2}$$

- The Clebsch-Gordan coefficients  $C$  are fixed mathematical constants that ensure the tensor product is equivariant. This is the  $SO(3)$ -equivariant analogue of matrix multiplication.

- **MACE** (Batatia et al., 2022) uses higher-order messages (products of multiple neighbour features) to achieve high accuracy with fewer message-passing layers. By constructing body-ordered interactions (2-body from distances, 3-body from angles, many-body from tensor products), MACE captures complex atomic interactions efficiently.
- **Equiformer** (Liao & Smidt, 2023) combines equivariant spherical tensor features with the transformer attention mechanism (file 4), creating an SE(3)-equivariant Graph Transformer. Attention scores are computed from invariant features, while the value aggregation operates on equivariant tensor features.

## 60.7 Applications

---

- **Molecular property prediction:** given a molecule's 3D structure, predict properties like energy, forces, dipole moment, HOMO-LUMO gap, toxicity, solubility. This is the most mature application of 3D GNNs. Models trained on quantum chemistry datasets (QM9, OC20) achieve chemical accuracy on many properties, enabling virtual screening of millions of candidate molecules.
- **Molecular dynamics acceleration:** computing forces between atoms using quantum mechanics (density functional theory, DFT) is extremely expensive ( $O(n^3)$  for  $n$  electrons). A 3D GNN trained to predict forces can replace DFT during molecular dynamics simulations, achieving a speedup of  $10^3$ – $10^6$  while maintaining near-DFT accuracy. This enables simulations of larger systems and longer timescales, revealing phenomena invisible to traditional methods.
- **Protein structure:** proteins are chains of amino acids that fold into complex 3D structures. The protein backbone is a geometric graph where nodes are residues and edges connect spatially proximate residues. 3D GNNs are used for protein function prediction, binding site identification, and protein design (inverse folding: given a desired structure, predict the amino acid sequence). **AlphaFold** uses geometric and graph-based reasoning to predict protein structures from sequences.
- **Materials science and catalysis:** crystalline materials have periodic 3D structures. GNNs model the repeating unit cell and predict material properties: band gap, formation energy, mechanical strength. The Open Catalyst Project (OC20/OC22) benchmarks GNNs for predicting adsorption energies on catalytic surfaces, accelerating the search for new catalysts for renewable energy.

- **Drug discovery:** 3D GNNs predict how a drug molecule will bind to a target protein. The binding affinity depends on the 3D shape complementarity and chemical interactions between the drug and the protein's binding pocket. Models like **DiffDock** use equivariant GNNs with diffusion models (chapter 8) to predict binding poses (the 3D orientation of the drug in the protein pocket).

## 60.8 Graph Generation

---

- All architectures above **analyse** existing graphs. **Graph generation** creates new ones: design a molecule with desired properties, generate a synthetic social network for testing, or propose a novel protein structure. This is the generative counterpart to graph-level prediction.
- The challenge is that graphs are discrete, variable-sized, and combinatorial. Generating a graph means deciding how many nodes to create, what features they have, and which pairs to connect. The space of possible graphs grows super-exponentially with the number of nodes.
- **Autoregressive generation** builds the graph one node (or one edge) at a time. **GraphRNN** (You et al., 2018) generates graphs sequentially: an RNN maintains a state, generates one new node at each step, and decides which existing nodes to connect it to. The generation order imposes an artificial sequence on the inherently unordered graph, but BFS orderings help by keeping recently generated nodes relevant.
- **VAE-based generation** encodes graphs into a continuous latent space (using a GNN encoder), then decodes new graphs from sampled latent vectors. **GraphVAE** generates a probabilistic adjacency matrix  $A \in [0, 1]^{n \times n}$  in one shot, but this scales as  $O(n^2)$  and produces dense outputs that must be thresholded. The latent space allows smooth interpolation: moving between two molecular embeddings generates chemically valid intermediate structures.
- **Diffusion-based generation** applies the diffusion framework (chapter 8) to graphs. The forward process gradually adds noise to node features and edge structures. The reverse process learns to denoise, generating valid graphs from noise. **DiGress** (Vignac et al., 2023) applies discrete diffusion to both node types and edge types, naturally handling the categorical nature of graph data.



- For **molecular generation**, the key constraint is **chemical validity**: generated molecules must obey valence rules (carbon forms 4 bonds, oxygen forms 2, etc.). Methods like **Junction Tree VAE (JT-VAE)** decompose molecules into valid substructures (rings, chains, functional groups) and generate by assembling these building blocks, guaranteeing validity by construction.
- **Goal-directed generation** optimises for specific properties: generate a molecule with high binding affinity to a target protein, low toxicity, and good solubility. This combines graph generation with property prediction (using a 3D GNN as the property evaluator) in a loop: generate → evaluate → refine. Reinforcement learning (chapter 6) or Bayesian optimisation guides the search through chemical space.
- **DiffDock** (Corso et al., 2023) uses SE(3)-equivariant diffusion to predict how a drug molecule docks into a protein binding pocket. The model generates the 3D binding pose (position and orientation of the drug relative to the protein) by denoising from random placements, combining the 3D equivariant networks from this file with the diffusion framework from chapter 8.

## 60.9 Coding Tasks (use CoLab or notebook)

---

1. Build a simple invariant 3D message-passing layer using interatomic distances. Apply it to a small molecule (water: H-O-H) and verify that the output is invariant to rotation.

```
import jax
import jax.numpy as jnp

Water molecule: O at origin, two H atoms
positions = jnp.array([[0.0, 0.0, 0.0], # O
 [0.96, 0.0, 0.0], # H1
 [-0.24, 0.93, 0.0]]) # H2

Node features: [atomic number]
features = jnp.array([[8.0], [1.0], [1.0]])

Compute pairwise distances (invariant)
def pairwise_distances(pos):
 diff = pos[:, None, :] - pos[None, :, :]
 return jnp.sqrt(jnp.sum(diff**2, axis=-1) + 1e-8)

Simple distance-based message passing
```

```

def invariant_message_pass(features, positions):
 dists = pairwise_distances(positions)
 # RBF expansion with 4 centres
 centres = jnp.array([0.5, 1.0, 1.5, 2.0])
 rbf = jnp.exp(-5.0 * (dists[:, :, None] - centres[None, None, :]) ** 2)

 # Message: features weighted by distance-dependent filter
 messages = jnp.einsum("ij,jd->id", rbf.sum(axis=-1), features)
 return messages

output1 = invariant_message_pass(features, positions)

Rotate the molecule by 90 degrees around z-axis
R = jnp.array([[0, -1, 0], [1, 0, 0], [0, 0, 1]], dtype=float)
rotated_positions = (R @ positions.T).T

output2 = invariant_message_pass(features, rotated_positions)

print(f"Original output:\n{output1}")
print(f"\nRotated output:\n{output2}")
print(f"\nInvariant: {jnp.allclose(output1, output2, atol=1e-5)}")

```

1. Compute the bond angle between three atoms and verify it is rotation-invariant.

```

import jax.numpy as jnp

def bond_angle(r_i, r_j, r_k):
 """Angle at node j between edges j->i and j->k."""
 v1 = r_i - r_j
 v2 = r_k - r_j
 cos_angle = jnp.dot(v1, v2) / (jnp.linalg.norm(v1) * jnp.linalg.norm(v2))
 return jnp.arccos(jnp.clip(cos_angle, -1, 1))

Three atoms
r1 = jnp.array([1.0, 0.0, 0.0])
r2 = jnp.array([0.0, 0.0, 0.0])
r3 = jnp.array([0.0, 1.0, 0.0])

angle_original = bond_angle(r1, r2, r3)
print(f"Original angle: {jnp.degrees(angle_original):.1f}°")

Apply random rotation
R = jnp.array([[0.36, 0.48, -0.80],
 [-0.80, 0.60, 0.00],
 [0.48, 0.64, 0.60]])
r1_rot, r2_rot, r3_rot = R @ r1, R @ r2, R @ r3

```

```

angle_rotated = bond_angle(r1_rot, r2_rot, r3_rot)
print(f"Rotated angle: {jnp.degrees(angle_rotated):.1f}°")
print(f"Invariant: {jnp.allclose(angle_original, angle_rotated, atol=1e-4)}")

```

1. Demonstrate equivariant position updates (EGNN-style). Update node positions using distance-weighted relative vectors and verify equivariance.

```

import jax
import jax.numpy as jnp

def egnn_position_update(positions, features):
 """Simple EGNN-style equivariant position update."""
 n = positions.shape[0]
 new_positions = jnp.zeros_like(positions)

 for i in range(n):
 shift = jnp.zeros(3)
 for j in range(n):
 if i != j:
 r_ij = positions[i] - positions[j]
 d_ij = jnp.linalg.norm(r_ij)
 # Weight based on distance (simple: inverse distance)
 weight = 1.0 / (d_ij + 1.0)
 # Scale by feature similarity
 feat_sim = jnp.dot(features[i], features[j])
 shift = shift + weight * feat_sim * r_ij
 new_positions = new_positions.at[i].set(positions[i] + 0.1 * shift)

 return new_positions

3 atoms
pos = jnp.array([[0.0, 0.0, 0.0], [1.0, 0.0, 0.0], [0.0, 1.0, 0.0]])
feat = jnp.array([[1.0, 0.5], [0.5, 1.0], [0.8, 0.3]])

Update positions
pos_new = egnn_position_update(pos, feat)

Now rotate input, update, and check if output is rotated consistently
R = jnp.array([[0.0, -1.0, 0.0], [1.0, 0.0, 0.0], [0.0, 0.0, 1.0]])
pos_rot = (R @ pos.T).T
pos_new_from_rot = egnn_position_update(pos_rot, feat)

Should be the same as rotating the original output
pos_new_then_rot = (R @ pos_new.T).T

print(f"Update then rotate:\n{jnp.round(pos_new_then_rot, 4)}")

```

```
print(f"\nRotate then update:\n{np.round(pos_new_from_rot, 4)}")
print(f"\nEquivariant: {np.allclose(pos_new_then_rot, pos_new_from_rot,
atol=1e-4)}")
```

## 61 Discrete Maths

Discrete maths is the mathematics of countable, separated structures, the foundation that computing is built on. This file covers propositional and predicate logic, proof techniques, sets, relations, functions, graph theory fundamentals, and recurrence relations

- In earlier chapters, we worked with continuous mathematics: calculus (chapter 3), probability distributions (chapter 5), and optimisation over real-valued parameters (chapter 6). But computers are fundamentally **discrete** machines. They store bits (0 or 1), process integers, follow branching logic, and operate on finite data structures. **Discrete maths** provides the formal language for reasoning about these structures.
- Everything in this chapter rests on discrete maths: processor logic gates are Boolean algebra, scheduling algorithms need proof of correctness, memory management uses set operations, and algorithm analysis requires recurrence relations.

### 61.1 Propositional Logic

---

- **Propositional logic** is the algebra of true/false statements. A **proposition** is a statement that is either true (T) or false (F), never both. “It is raining” is a proposition. “What time is it?” is not (it is a question, not a statement with a truth value).
- Propositions can be combined using **logical connectives**:
  - **AND** (conjunction,  $p \wedge q$ ): true only when both  $p$  and  $q$  are true.
  - **OR** (disjunction,  $p \vee q$ ): true when at least one of  $p$  or  $q$  is true.
  - **NOT** (negation,  $\neg p$ ): flips the truth value.
  - **IMPLIES** (implication,  $p \rightarrow q$ ): false only when  $p$  is true and  $q$  is false. “If it rains, the ground is wet” is only violated when it rains and the ground is dry.
  - **IFF** (biconditional,  $p \leftrightarrow q$ ): true when both have the same truth value.

- A **truth table** exhaustively lists all possible input combinations and the resulting output. For  $n$  propositions, the table has  $2^n$  rows. This is how we verify logical equivalences:

<b>p</b>	<b>q</b>	<b><math>p \wedge q</math></b>	<b><math>p \vee q</math></b>	<b><math>p \rightarrow q</math></b>
T	T	T	T	T
T	F	F	T	F
F	T	F	T	T
F	F	F	F	T

- The implication row where  $p$  is false deserves attention:  $F \rightarrow q$  is always true, regardless of  $q$ . This is **vacuous truth**. “If pigs fly, then I am the king of England” is logically true because the premise is false. This seems counterintuitive but is essential for mathematical reasoning.
- **Logical equivalences** are identities that hold for all truth values:
  - **De Morgan’s laws:**  $\neg(p \wedge q) \equiv \neg p \vee \neg q$  and  $\neg(p \vee q) \equiv \neg p \wedge \neg q$ . To negate an AND, negate each part and switch to OR (and vice versa). These appear directly in programming: `!(a && b)` is equivalent to `(!a || !b)`.
  - **Contrapositive:**  $p \rightarrow q \equiv \neg q \rightarrow \neg p$ . “If it rains, the ground is wet” is equivalent to “if the ground is not wet, it is not raining.” This is a powerful proof technique.
  - **Double negation:**  $\neg(\neg p) \equiv p$ .
  - **Distributive:**  $p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$ .
- A formula that is always true (for all truth assignments) is a **tautology**. One that is always false is a **contradiction**. One that is sometimes true and sometimes false is **contingent**. For example,  $p \vee \neg p$  is a tautology and  $p \wedge \neg p$  is a contradiction.

## 61.2 Predicate Logic and Quantifiers

---

- Propositional logic cannot express statements about all or some elements of a set. “Every prime greater than 2 is odd” requires **predicate logic**, which extends propositional logic with variables, predicates, and quantifiers.

- A **predicate** is a statement that depends on a variable:  $P(x) = \text{"x is even."}$  It becomes a proposition when  $x$  is given a specific value:  $P(4)$  is true,  $P(7)$  is false.
- **Quantifiers** express scope:
  - **Universal quantifier** ( $\forall$ ): “for all.”  $\forall x P(x)$  means “ $P(x)$  is true for every  $x$  in the domain.”
  - **Existential quantifier** ( $\exists$ ): “there exists.”  $\exists x P(x)$  means “there is at least one  $x$  for which  $P(x)$  is true.”
- Negating quantifiers flips them:  $\neg(\forall x P(x)) \equiv \exists x \neg P(x)$ . “Not everyone passed” means “someone failed.” And  $\neg(\exists x P(x)) \equiv \forall x \neg P(x)$ . “There is no perfect algorithm” means “every algorithm has a flaw.”
- Nested quantifiers express complex relationships.  $\forall x \exists y (y > x)$  means “for every number, there is a larger one” (true for integers). The order matters:  $\exists y \forall x (y > x)$  means “there is a number larger than all others” (false for integers).
- Predicate logic is the language of formal specification. When we say an algorithm is “correct,” we mean  $\forall \text{inputs } x, \text{ output}(x) = \text{desired}(x)$ . When we say it “terminates,” we mean  $\forall x \exists t \text{halts}(x, t)$ .

## 61.3 Proof Techniques

- A **proof** is a logical argument that establishes a statement’s truth beyond doubt. Unlike empirical evidence (which shows something works for tested cases), a proof guarantees it works for all cases. This is the standard of correctness in CS.
- **Direct proof**: assume the hypothesis, derive the conclusion through logical steps. To prove “if  $n$  is even, then  $n^2$  is even” : assume  $n = 2k$  for some integer  $k$ , then  $n^2 = 4k^2 = 2(2k^2)$ , which is even.

- **Proof by contradiction**: assume the statement is false and derive a contradiction. To prove

---

$\sqrt{2}$

is irrational: assume

---

$\sqrt{2}$

$= a/b$  (fully reduced). Then  $2 = a^2/b^2$ , so  $a^2 = 2b^2$ , meaning  $a^2$  is even, so  $a$  is even,

say  $a = 2c$ . Then  $4c^2 = 2b^2$ , so  $b^2 = 2c^2$ , meaning  $b$  is also even. But we said  $a/b$  was fully reduced – contradiction.

- **Proof by induction:** prove a statement for all natural numbers by showing: (1) the **base case** holds (typically  $n = 0$  or  $n = 1$ ), and (2) the **inductive step**: if the statement holds for  $n = k$  (the inductive hypothesis), then it holds for  $n = k + 1$ .

- For example, prove  $\sum_{i=1}^n i = \frac{n(n+1)}{2}$ :

- Base case:  $n = 1$ :  $1 = \frac{1 \cdot 2}{2} = 1$ . True.

- Inductive step: assume  $\sum_{i=1}^k i = \frac{k(k+1)}{2}$ . Then  $\sum_{i=1}^{k+1} i = \frac{k(k+1)}{2} + (k+1) = \frac{k(k+1) + 2(k+1)}{2} = \frac{(k+1)(k+2)}{2}$ . This is the formula with  $n = k + 1$ . Done.

- Induction is the workhorse for proving properties of recursive algorithms and data structures. Every recursive algorithm has an implicit inductive proof of correctness: the base case is the termination condition, and the inductive step is the recursive call.
- **Strong induction** assumes the statement holds for all values up to  $k$  (not just  $k$ ), then proves it for  $k + 1$ . This is useful when the recursion depends on more than just the previous value.
- The **pigeonhole principle**: if  $n + 1$  objects are placed into  $n$  boxes, at least one box contains two objects. Simple but surprisingly powerful. It proves that in any group of 13 people, at least two share a birthday month. In networking, it proves that hash collisions are inevitable when more items than buckets exist.

## 61.4 Sets

---

- A **set** is an unordered collection of distinct elements. Sets are the most primitive data structure in maths, and they underpin everything from type systems to database queries.
- **Set operations** (connecting to chapter 5 where we used these for probability):
  - **Union**  $A \cup B$ : elements in  $A$  or  $B$  or both.
  - **Intersection**  $A \cap B$ : elements in both  $A$  and  $B$ .

- **Complement**  $A^c$ : elements not in  $A$  (relative to a universal set).
  - **Difference**  $A \setminus B$ : elements in  $A$  but not in  $B$ .
  - **Cartesian product**  $A \times B$ : all ordered pairs  $(a, b)$  with  $a \in A, b \in B$ .
- The **power set**  $\mathcal{P}(A)$  is the set of all subsets of  $A$ . If  $|A| = n$ , then  $|\mathcal{P}(A)| = 2^n$ . For  $A = \{1, 2\}$ :  $\mathcal{P}(A) = \{\emptyset, \{1\}, \{2\}, \{1, 2\}\}$ .
  - **Cardinality** measures set size. Finite sets have integer cardinality. Infinite sets come in different sizes: the natural numbers  $\mathbb{N}$  and rationals  $\mathbb{Q}$  are **countably infinite** (can be listed), while the reals  $\mathbb{R}$  are **uncountably infinite** (cannot be listed, proven by Cantor's diagonal argument). This distinction matters in computability theory: there are uncountably many functions but only countably many programs, so most functions are uncomputable.

## 61.5 Relations

---

- A **relation**  $R$  on a set  $A$  is a subset of  $A \times A$ : a set of ordered pairs specifying which elements are related. For example,  $\leq$  on integers is the set  $\{(a, b) : a \leq b\}$ .
- Important properties of relations:
  - **Reflexive**: every element is related to itself.  $aRa$  for all  $a$ . Example:  $\leq$  (every number is  $\leq$  itself).
  - **Symmetric**: if  $aRb$  then  $bRa$ . Example: “is a sibling of.”
  - **Antisymmetric**: if  $aRb$  and  $bRa$  then  $a = b$ . Example:  $\leq$ .
  - **Transitive**: if  $aRb$  and  $bRc$  then  $aRc$ . Example:  $<$ ,  $\leq$ , “is an ancestor of.”
- An **equivalence relation** is reflexive, symmetric, and transitive. It partitions the set into **equivalence classes** where all elements within a class are related to each other but not to elements in other classes. Modular arithmetic is an equivalence relation:  $a \equiv b \pmod{n}$  partitions integers into  $n$  classes. Type equivalence in programming languages is an equivalence relation.
- A **partial order** is reflexive, antisymmetric, and transitive. It defines a “less than or equal to” structure that may leave some elements incomparable. File system directories form a partial order (parent-child), but sibling directories



are incomparable. A **total order** is a partial order where every pair is comparable (like  $\leq$  on integers).

- Partial orders are essential in concurrency: the “happens-before” relation on events is a partial order. Events that are not ordered by happens-before are concurrent and may execute in any relative order.

## 61.6 Functions

---

- A **function**  $f : A \rightarrow B$  maps each element of  $A$  (the domain) to exactly one element of  $B$  (the codomain). Functions are the mathematical model of deterministic computation: given an input, there is exactly one output.
- **Injective** (one-to-one): different inputs always give different outputs.  $f(a) = f(b) \implies a = b$ . Lossless compression is injective: different inputs must compress to different outputs (otherwise you cannot decompress uniquely).
- **Surjective** (onto): every element of  $B$  is hit by some element of  $A$ . The range equals the codomain. A hash function mapping strings to 256-bit hashes is not surjective if there are fewer strings than possible hashes.
- **Bijective**: both injective and surjective. A perfect one-to-one correspondence between  $A$  and  $B$ . Bijections have inverses. Encryption must be bijective: each plaintext maps to a unique ciphertext, and the decryption function is the inverse.
- **Composition**  $(g \circ f)(x) = g(f(x))$ : apply  $f$  first, then  $g$ . Function composition is associative (chapter 2: same as matrix multiplication being associative). Pipelines in software are function composition: data flows through a chain of transformations.

## 61.7 Graph Theory Fundamentals

---

- We covered graphs extensively in chapter 12 (graph neural networks), including adjacency matrices, graph types, the Laplacian, and spectral theory. Here we focus on the **algorithmic** and **structural** properties relevant to CS.
- A **tree** is a connected graph with no cycles. Equivalently, it has  $n$  nodes and  $n - 1$  edges. Trees are the structure of file systems, XML/HTML documents, decision

processes, and recursive decompositions. A **rooted tree** has a designated root node; every other node has exactly one parent.

- A **spanning tree** of a graph  $G$  is a tree that includes all nodes of  $G$  using a subset of its edges. **Minimum spanning trees (MST)** minimise the total edge weight. Kruskal's algorithm (sort edges, greedily add the lightest that does not create a cycle) and Prim's algorithm (grow the tree from a starting node, always adding the lightest edge to a new node) both find MSTs in  $O(|E| \log |V|)$ .
- **Planarity**: a graph is planar if it can be drawn in the plane with no edge crossings. By **Euler's formula**, for a connected planar graph:  $|V| - |E| + |F| = 2$ , where  $|F|$  is the number of faces (regions, including the outer face). This implies  $|E| \leq 3|V| - 6$  for planar graphs, so planar graphs are sparse. Circuit board routing and map colouring exploit planarity.
- **Graph colouring** assigns colours to nodes such that no two adjacent nodes share a colour. The minimum number of colours needed is the **chromatic number**  $\chi(G)$ . The **four-colour theorem** states  $\chi(G) \leq 4$  for any planar graph. In CS, graph colouring models register allocation (assign variables to CPU registers so that simultaneously live variables get different registers) and scheduling (assign tasks to time slots so conflicting tasks do not overlap).
- **Euler paths** visit every edge exactly once. They exist if and only if the graph has exactly 0 or 2 nodes with odd degree. **Hamiltonian paths** visit every node exactly once. Determining whether a Hamiltonian path exists is NP-complete – one of the classic hard problems in CS. This contrast (Euler: polynomial, Hamilton: NP-complete) illustrates how similar-sounding problems can have vastly different computational complexities.

## 61.8 Recurrence Relations

---

- A **recurrence relation** defines a sequence where each term depends on previous terms. They arise naturally from recursive algorithms.
- The simplest example:  $T(n) = T(n-1) + 1$  with  $T(0) = 0$ . Unrolling:  $T(n) = T(n-1) + 1 = T(n-2) + 2 = \dots = n$ . This is  $O(n)$ , the time complexity of a simple loop.
- **Merge sort** gives  $T(n) = 2T(n/2) + O(n)$ : split the array in half (two subproblems of size  $n/2$ ), recursively sort each half, then merge ( $O(n)$  work). The solution is  $T(n) = O(n \log n)$ .

- The **Master Theorem** solves recurrences of the form  $T(n) = aT(n/b) + O(n^d)$ :
  - If  $d > \log_b a$ :  $T(n) = O(n^d)$  (the work at each level dominates)
  - If  $d = \log_b a$ :  $T(n) = O(n^d \log n)$  (work is balanced across levels)
  - If  $d < \log_b a$ :  $T(n) = O(n^{\log_b a})$  (the number of subproblems dominates)
- For merge sort:  $a = 2$ ,  $b = 2$ ,  $d = 1$ . Since  $d = \log_2 2 = 1$ , we are in the balanced case:  $T(n) = O(n \log n)$ .
- The **Fibonacci recurrence**  $F(n) = F(n-1) + F(n-2)$  with  $F(0) = 0$ ,  $F(1) = 1$  has the closed-form solution  $F(n) = \frac{\phi^n - \psi^n}{\sqrt{5}}$  where  $\phi = \frac{1 + \sqrt{5}}{2}$  (the golden ratio) and  $\psi = \frac{1 - \sqrt{5}}{2}$ . This shows the Fibonacci sequence grows exponentially as  $O(\phi^n)$ , which is why naive recursive Fibonacci is exponentially slow.
- **Combinatorics** (permutations, combinations, the binomial theorem, and inclusion-exclusion) is covered in chapter 5 (probability). These counting techniques are essential for algorithm analysis (how many possible inputs? how many comparisons?), but we will not repeat them here.

## 61.9 Computability

---

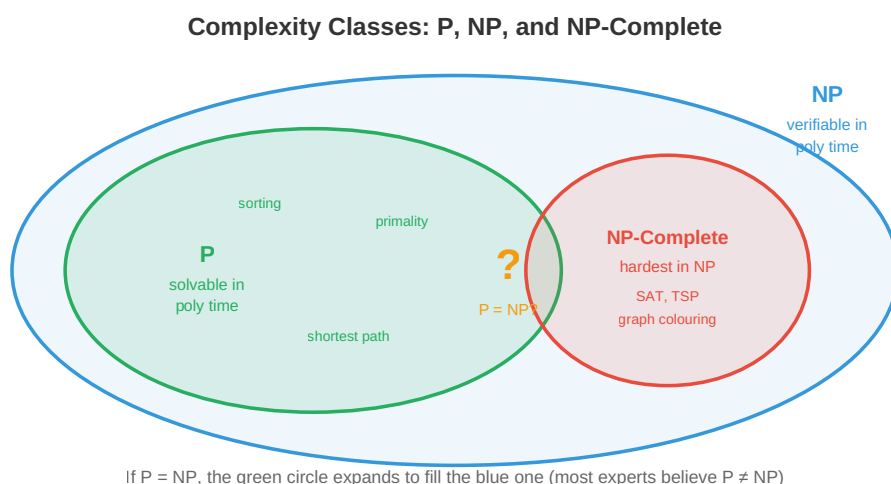
- Not everything can be computed. This is one of the most profound results in all of mathematics, and it sets the fundamental limits of what computers can do.
- A **Turing machine** is an abstract model of computation: an infinite tape of cells (each holding a symbol), a read/write head, and a finite set of states with transition rules. Despite its simplicity, a Turing machine can compute anything that any real computer can. This is the **Church-Turing thesis**: any effectively computable function can be computed by a Turing machine.
- Every programming language (Python, C, Haskell) is **Turing complete**: it can simulate a Turing machine and therefore compute anything that is computable. The differences between languages are in convenience, speed, and safety, not in what they can fundamentally compute.
- The **halting problem** asks: given a program and an input, will the program eventually stop, or will it run forever? Turing proved (1936) that no algorithm can solve this in general. The proof is by contradiction: assume a halting

detector  $H(P, x)$  exists. Construct a program  $D$  that runs  $H(D, D)$  and does the opposite of whatever  $H$  says. If  $H$  says  $D$  halts,  $D$  loops forever. If  $H$  says  $D$  loops,  $D$  halts. Contradiction.

- This is not a limitation of current technology; it is a mathematical impossibility. No amount of compute, cleverness, or AI will ever solve the halting problem in general. It is the computer science analogue of Gödel's incompleteness theorem.
- Practical consequences: you cannot write a perfect deadlock detector, a perfect virus scanner, or a perfect optimising compiler. Each of these would require solving the halting problem (or an equivalent undecidable problem) in general. Real tools use heuristics and approximations that work in common cases but cannot guarantee correctness for all inputs.
- A problem is **decidable** if an algorithm exists that always terminates with the correct yes/no answer. It is **undecidable** if no such algorithm exists. The halting problem is undecidable. Primality testing is decidable. Type checking in most programming languages is decidable (by design).

## 61.10 Complexity Theory

- Even among computable problems, some are vastly harder than others. **Complexity theory** classifies problems by the resources (time, space) required to solve them as the input grows.



P, NP, and NP-Complete: P is contained in NP, NP-Complete sits at the boundary, and whether  $P=NP$  is the central open question

- **P** (Polynomial time): problems solvable in  $O(n^k)$  time for some constant  $k$ . Sorting ( $O(n \log n)$ ), shortest path ( $O(|V|^2)$ ), matrix multiplication ( $O(n^3)$ ). These are considered “efficient” or “tractable.”
- **NP** (Nondeterministic Polynomial time): problems where a proposed solution can be **verified** in polynomial time, even if **finding** the solution might take exponential time. For example, given a claimed Hamiltonian path, you can verify it in  $O(n)$  by checking each edge. But finding one might require trying exponentially many possibilities.
- Every problem in P is also in NP (if you can solve it quickly, you can certainly verify a solution quickly). The central question is whether  $P = NP$ : is every problem whose solution can be quickly verified also quickly solvable? This is the most important open problem in computer science, with a \$1 million Clay Millennium Prize.
- Most experts believe  $P \neq NP$ , meaning some problems are fundamentally harder to solve than to verify. If  $P = NP$ , cryptography would collapse (breaking encryption is in NP), and optimisation, scheduling, and drug design would become trivially easy.
- **NP-complete** problems are the hardest problems in NP. A problem is NP-complete if: (1) it is in NP, and (2) every other NP problem can be **reduced** to it in polynomial time. If you could solve any NP-complete problem efficiently, you could solve all of them (and  $P = NP$ ).
- A **reduction** transforms one problem into another. If problem A reduces to problem B, then B is at least as hard as A. Cook (1971) showed that **SAT** (Boolean satisfiability: given a logical formula, is there an assignment of variables that makes it true?) is NP-complete. Karp (1972) showed 21 other classic problems are NP-complete by reducing SAT to each one.
- Famous NP-complete problems:
  - **Travelling Salesman Problem (TSP)**: find the shortest route visiting all cities exactly once.
  - **Graph colouring**: colour nodes with  $k$  colours such that no adjacent nodes share a colour ( $k \geq 3$ ).

- **Subset sum**: given a set of integers, is there a subset that sums to a target value?
  - **Boolean satisfiability (SAT)**: is there a truth assignment that satisfies a logical formula?
  - **Hamiltonian path** (mentioned above in graph theory).
- When you encounter an NP-complete problem in practice, you do not solve it exactly for large inputs. Instead, you use: **approximation algorithms** (find a solution within a guaranteed factor of optimal), **heuristics** (greedy, local search, simulated annealing), or **special-case solvers** (many NP-complete problems are easy for restricted inputs). Modern SAT solvers, for example, routinely solve instances with millions of variables, despite the worst-case exponential complexity, by exploiting structure in practical instances.
  - **NP-hard** problems are at least as hard as NP-complete but may not be in NP (their solutions might not even be verifiable in polynomial time). Optimisation versions of NP-complete problems are typically NP-hard: “find the shortest TSP tour” is NP-hard, while “is there a TSP tour shorter than k?” is NP-complete.

## 61.11 Coding Tasks (use CoLab or notebook)

---

1. Build a truth table generator. Given a logical expression, enumerate all input combinations and compute the output.

```
import itertools

def truth_table(n_vars, expr_fn):
 """Generate truth table for a boolean function of n_vars variables."""
 headers = [f"p{i}" for i in range(n_vars)]
 print(" | ".join(headers + ["result"]))
 print("-" * (len(headers) * 4 + 10))
 for vals in itertools.product([False, True], repeat=n_vars):
 result = expr_fn(*vals)
 row = [str(v)[0] for v in vals] + [str(result)[0]]
 print(" | ".join(f"{r:>2}" for r in row))

De Morgan's law: NOT(p AND q) == (NOT p) OR (NOT q)
```

```
print("De Morgan's Law verification:")
truth_table(2, lambda p, q: (not (p and q)) == ((not p) or (not q)))
```

1. Prove the sum formula by induction – numerically verify for many values, then implement the closed-form solution.

```
import jax.numpy as jnp

Verify sum formula: sum(1..n) = n(n+1)/2
for n in [1, 5, 10, 100, 1000, 10000]:
 brute = sum(range(1, n + 1))
 formula = n * (n + 1) // 2
 print(f"n={n:5d} sum={brute:>10d} formula={formula:>10d} match={brute == formula}")
```

1. Solve the merge sort recurrence using the Master Theorem and verify empirically by counting operations.

```
import jax.numpy as jnp

def merge_sort_ops(n):
 """Count comparisons in merge sort (recurrence: $T(n) = 2T(n/2) + n$)."""
 if n <= 1:
 return 0
 half = n // 2
 return merge_sort_ops(half) + merge_sort_ops(n - half) + n

for n in [8, 64, 512, 4096, 32768]:
 ops = merge_sort_ops(n)
 predicted = n * jnp.log2(n)
 ratio = ops / predicted
 print(f"n={n:5d} ops={ops:>10d} n log n={int(predicted):>10d} ratio={ratio:.3f}")
```

## 62 Computer Architecture

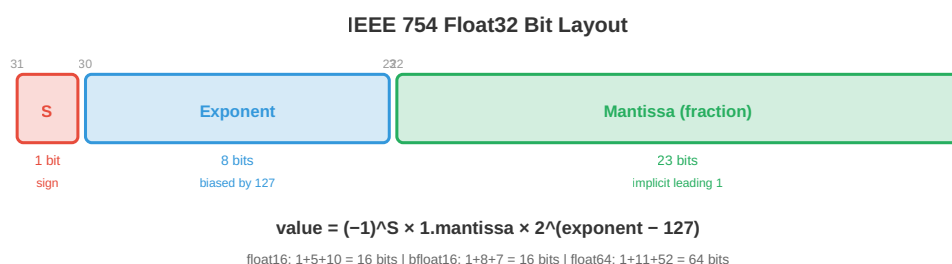
Computer architecture is how we build machines that execute instructions. This file covers number systems, logic gates, CPU design, instruction set architectures,

pipelining, the memory hierarchy, and virtual memory, the hardware foundation that every program, framework, and AI model ultimately runs on.

- Every neural network, every training loop, every inference call eventually becomes a sequence of electrical signals flowing through transistors. Understanding the hardware is not optional for serious ML practitioners: it explains why matrix multiplications are fast, why memory is the bottleneck, why GPUs dominate AI training, and why cache-friendly code can be 100x faster than naive code.

## 62.1 Number Systems

- Computers represent everything as **binary** (base 2): sequences of 0s and 1s. Each digit is a **bit**. A group of 8 bits is a **byte**. The value of a binary number  $b_{n-1}b_{n-2}\dots b_1b_0$  is  $\sum_{i=0}^{n-1} b_i \cdot 2^i$ .
- For example,  $1011_2 = 1 \cdot 8 + 0 \cdot 4 + 1 \cdot 2 + 1 \cdot 1 = 11_{10}$ .
- **Hexadecimal** (base 16) is a compact notation for binary. Each hex digit represents 4 bits: 0-9 map to 0000-1001, and A-F map to 1010-1111. So  $0xFF = 1111\ 1111_2 = 255_{10}$ . Memory addresses and colour codes are typically written in hex.
- **Two's complement** represents signed integers. For an n-bit number, the most significant bit has weight  $-2^{n-1}$  instead of  $+2^{n-1}$ . An 8-bit two's complement ranges from -128 to +127. To negate a number: flip all bits and add 1. This representation makes addition and subtraction use the same hardware circuit, which is why it is universal.
- **IEEE 754 floating point** represents real numbers as  $(-1)^s \times 1.m \times 2^{e - \text{bias}}$ , where s is the sign bit, m is the mantissa (fractional part), and e is the biased exponent.



IEEE 754 float32 layout: 1 sign bit, 8 exponent bits, 23 mantissa bits



- **float32** (single precision): 1 sign + 8 exponent + 23 mantissa = 32 bits. Range:  $\approx \pm 3.4 \times 10^{38}$ , precision:  $\approx 7$  decimal digits.
- **float64** (double precision): 1 sign + 11 exponent + 52 mantissa = 64 bits. Range:  $\approx \pm 1.8 \times 10^{308}$ , precision:  $\approx 15$  decimal digits.
- **float16** (half precision): 1 + 5 + 10 = 16 bits. Limited range and precision, but uses half the memory and bandwidth. Widely used in ML training (mixed precision, chapter 6).
- **bfloat16**: 1 + 8 + 7 = 16 bits. Same exponent range as float32 but lower precision. Designed by Google specifically for ML: the full exponent range prevents overflow during training, and the reduced precision is acceptable for gradient updates.

- Floating-point arithmetic is **not exact**.  $0.1 + 0.2 \neq 0.3$  in float64 (it equals 0.30000000000000004). This is because 0.1 has no exact binary representation, just as  $1/3$  has no exact decimal representation. Accumulating these errors over millions of operations (like gradient descent) can cause numerical instability, which is why techniques like loss scaling (chapter 6) and Kahan summation exist.

## 62.2 Logic Gates

---

- All computation reduces to **logic gates**: physical circuits that implement Boolean operations (the propositional logic from file 1).
- The fundamental gates:
  - **AND**: output is 1 only if both inputs are 1.
  - **OR**: output is 1 if at least one input is 1.
  - **NOT** (inverter): flips the input.
  - **NAND** (NOT-AND): the universal gate. Any other gate can be built from NAND gates alone. This is why NAND is the fundamental building block of digital circuits.
  - **XOR** (exclusive OR): output is 1 if inputs differ. Essential for addition (the sum bit of binary addition is XOR) and cryptography.
- A **half adder** adds two single bits using XOR (sum) and AND (carry). A **full adder** adds two bits plus a carry-in, chaining together to create an n-bit adder. This is how CPUs perform integer addition: a cascade of simple logic gates.
- A **multiplexer** (MUX) selects one of several inputs based on a control signal. With n control bits, it selects from  $2^n$  inputs. Multiplexers are the hardware

equivalent of an if-else chain and are used extensively in CPU datapaths to route data.

- Modern processors contain billions of transistors, each acting as a tiny switch. A transistor is either on (conducting, representing 1) or off (not conducting, representing 0). Gates are built from transistors, adders from gates, ALUs from adders, and CPUs from ALUs. The entire hierarchy of computing rests on this foundation.

## 62.3 CPU Architecture

---

- The **Central Processing Unit (CPU)** executes instructions. Its core components:
  - **ALU (Arithmetic Logic Unit)**: performs integer arithmetic (add, subtract, multiply) and logical operations (AND, OR, XOR, shift). This is where the actual computation happens, built from the logic gates described above.
  - **Registers**: tiny, ultra-fast storage locations inside the CPU. A modern CPU has dozens of general-purpose registers, each holding one word (64 bits on a 64-bit CPU). Registers are the fastest memory in the system: access takes ~0.3 nanoseconds.
  - **Program Counter (PC)**: holds the memory address of the next instruction to execute.
  - **Control Unit**: decodes instructions and orchestrates the datapath, telling the ALU what operation to perform and which registers to use.
- The **instruction cycle** (fetch-decode-execute) repeats billions of times per second:
  1. **Fetch**: read the instruction from memory at the address in the PC.
  2. **Decode**: determine what the instruction does (add? load from memory? branch?) and which operands it uses.
  3. **Execute**: perform the operation (ALU computation, memory access, or branch).
  4. Increment the PC (unless the instruction is a branch/jump).
- A CPU running at 4 GHz performs 4 billion cycles per second. Each cycle takes 0.25 nanoseconds. In that time, light travels about 7.5 centimetres, which is why physical chip size matters: signals cannot cross a large chip in one cycle.

## 62.4 Instruction Set Architectures

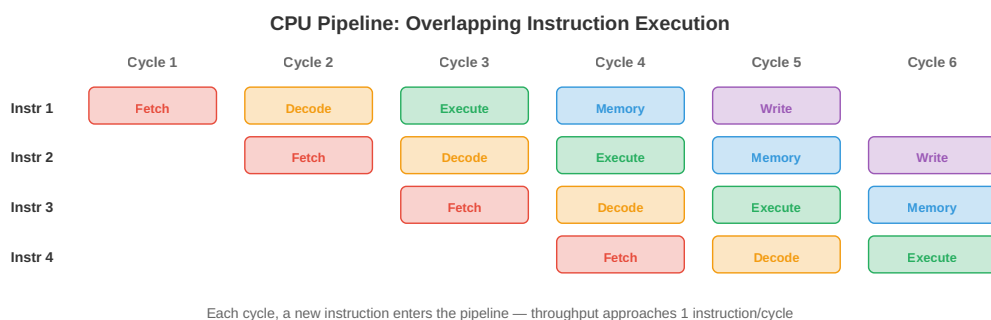
---

- The **Instruction Set Architecture (ISA)** is the contract between hardware and software: it defines the instructions the CPU understands, the register set, the memory model, and the encoding format.
- **CISC** (Complex Instruction Set Computer): instructions can be complex, variable-length, and may access memory directly. A single instruction might multiply two memory values and store the result. **x86** (Intel/AMD) is the dominant CISC ISA, powering most desktops and servers. Its backward compatibility (modern x86 CPUs still run 1980s code) is both its strength and its burden.
- **RISC** (Reduced Instruction Set Computer): instructions are simple, fixed-length, and operate only on registers. Memory access requires separate load/store instructions. Simpler instructions enable faster clock speeds and easier pipelining.
  - **ARM**: the dominant RISC ISA for mobile devices and increasingly for servers and laptops (Apple M-series chips are ARM). ARM's power efficiency makes it ideal for battery-powered and thermally constrained devices.
  - **RISC-V**: an open-source RISC ISA. Anyone can design a RISC-V chip without licensing fees. Growing adoption in embedded systems, research, and AI accelerators.
- The CISC vs RISC distinction has blurred: modern x86 CPUs internally decode complex CISC instructions into simpler micro-operations (essentially RISC internally), getting the benefits of both worlds.

## 62.5 Pipelining

---

- Without pipelining, the CPU completes one instruction fully before starting the next. This wastes hardware: while the ALU executes, the fetch and decode units sit idle.

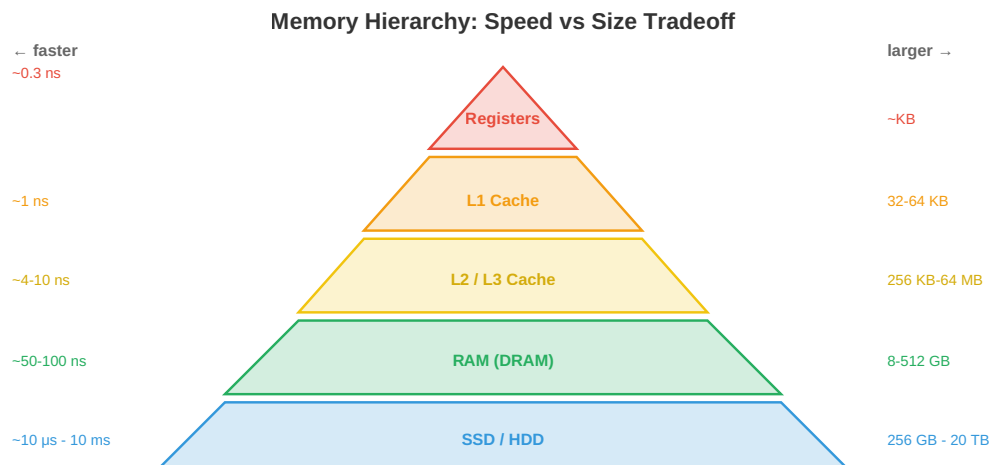


CPU pipeline: instructions overlap across fetch, decode, execute, memory, and writeback stages

- **Pipelining** overlaps instruction execution, like an assembly line. While instruction 1 is executing, instruction 2 is decoding, and instruction 3 is being fetched. A 5-stage pipeline (fetch, decode, execute, memory access, write-back) can have 5 instructions in flight simultaneously.
- The throughput approaches one instruction per cycle (even though each instruction takes 5 cycles to complete). This is the same principle as pipelining in ML: data parallelism overlaps computation and communication (chapter 6).
- **Hazards** are situations where pipelining breaks:
  - **Data hazard:** instruction 2 needs a result that instruction 1 has not yet produced. “Add R1, R2, R3” followed by “Sub R4, R1, R5” – the second instruction needs R1, which the first is still computing. **Forwarding** (bypassing) solves this by routing the result directly from one pipeline stage to another without waiting for the write-back stage.
  - **Control hazard:** a branch instruction (if-else) means the CPU does not know which instruction to fetch next until the branch is resolved. **Branch prediction** guesses which way the branch will go and speculatively fetches instructions along the predicted path. Modern predictors are >95% accurate, using history tables and neural-network-like pattern matching. A misprediction costs ~15 cycles (the pipeline must be flushed and restarted).
  - **Structural hazard:** two instructions need the same hardware resource simultaneously (e.g., both need the memory port). Resolved by duplicating resources or inserting a stall.

## 62.6 Memory Hierarchy

- The fundamental tension in computer memory: fast memory is expensive and small, cheap memory is slow and large. The **memory hierarchy** bridges this gap by exploiting **locality**: programs tend to access the same data repeatedly (temporal locality) and access nearby data (spatial locality).



Memory hierarchy pyramid: registers at the top (fast, small) down to HDD at the bottom (slow, large)

- The hierarchy, from fastest to slowest:
  - **Registers:** ~0.3 ns access, ~KB total. Inside the CPU.
  - **L1 cache:** ~1 ns, 32-64 KB per core. Split into instruction cache and data cache.
  - **L2 cache:** ~4 ns, 256 KB-1 MB per core.
  - **L3 cache:** ~10 ns, 8-64 MB shared across cores.
  - **RAM (DRAM):** ~50-100 ns, 8-512 GB. Main memory.
  - **SSD:** ~10-100 μs, 256 GB-8 TB. Persistent storage.
  - **HDD:** ~5-10 ms, 1-20 TB. Mechanical, very slow for random access.
- The speed gap between registers and RAM is ~300x. Between registers and disk, it is ~30,000,000x. The cache hierarchy hides this gap: if the data the CPU needs is in L1 cache (a **cache hit**), access is fast. If not (a **cache miss**), the CPU stalls while data is fetched from a slower level.
- **Cache associativity** determines where a memory address can be stored in the cache:
  - **Direct-mapped:** each address maps to exactly one cache line. Simple but causes conflicts.

- **Fully associative:** any address can go anywhere. Flexible but expensive to search.
- **Set-associative** (k-way): each address maps to a set of k locations. The practical compromise used in real CPUs (typically 4-way or 8-way).
- **Cache coherence** ensures that all CPU cores see a consistent view of memory. When core 1 writes to an address that core 2 has cached, the coherence protocol (e.g., MESI) invalidates or updates core 2's copy. This is critical for concurrent programming (file 4) and is one reason that shared-memory parallelism is hard.
- For ML practitioners, the memory hierarchy explains why:
  - Matrix operations should access memory sequentially (row-major vs column-major layout matters).
  - Batch size affects performance: larger batches amortise memory latency.
  - Mixed precision (float16/bfloat16) doubles the effective memory bandwidth, which is often the bottleneck.

## 62.7 Virtual Memory

---

- **Virtual memory** gives each process the illusion of having its own large, contiguous memory space, even though physical RAM is limited and shared among processes.
- The address space is divided into fixed-size **pages** (typically 4 KB). The **page table** maps virtual page numbers to physical frame numbers. When a program accesses virtual address 0x1234, the CPU translates it to a physical address by looking up the page table.
- The **Translation Lookaside Buffer (TLB)** is a cache for page table entries. Since the page table lives in RAM (slow), the TLB stores recently used translations in fast hardware. A TLB miss requires walking the page table in memory, costing hundreds of cycles.
- A **page fault** occurs when a program accesses a page that is not in physical RAM. The OS loads the page from disk (swapping), which takes millions of cycles. Excessive page faults (**thrashing**) devastates performance. This is why ML training requires enough RAM to hold the model, optimiser states, and a reasonable batch of data.

- **Page replacement** algorithms decide which page to evict when RAM is full:
  - **LRU** (Least Recently Used): evict the page that has not been accessed for the longest time. Optimal in practice for most workloads. Approximated in hardware with the **clock algorithm** (a circular list with reference bits).
  - **FIFO**: evict the oldest page. Simple but can evict frequently used pages.
  - **Optimal** (Bélády's): evict the page that will not be used for the longest time. Impossible to implement (requires future knowledge) but useful as a theoretical benchmark.
- Virtual memory also provides **isolation**: each process has its own virtual address space. A bug in one process cannot corrupt another process' s memory, because their virtual addresses map to different physical frames. This is the foundation of OS security and stability.

## 62.8 I/O, Interrupts, and DMA

---

- The CPU needs to communicate with the outside world: disks, network cards, keyboards, GPUs. This is the **I/O subsystem**.
- **Programmed I/O** (polling): the CPU repeatedly checks a device' s status register in a loop, waiting for data to be ready. Simple but wastes CPU cycles spinning instead of doing useful work.
- **Interrupt-driven I/O**: the device sends a hardware **interrupt** when data is ready. The CPU continues normal execution until the interrupt arrives, then runs an **interrupt handler** (a kernel function) to process the data. This is far more efficient than polling because the CPU is not idle while waiting.
- The interrupt mechanism:
  1. A device signals an interrupt via a hardware line.
  2. The CPU finishes the current instruction, saves the current state (registers, program counter) onto the stack.
  3. The CPU looks up the interrupt handler address in the **interrupt vector table** (a table of function pointers, one per interrupt type).
  4. The handler runs in kernel mode, processes the I/O, and returns.
  5. The CPU restores the saved state and resumes the interrupted program.
- This is the same save/restore pattern as a context switch (file 3), but triggered by hardware rather than a timer.

- **DMA** (Direct Memory Access): for large data transfers (disk reads, network packets, GPU memory copies), having the CPU copy data byte by byte is wasteful. A **DMA controller** transfers data directly between a device and RAM without involving the CPU. The CPU sets up the transfer (source, destination, size), the DMA controller handles it, and the CPU gets an interrupt when it is done.
- DMA is critical for ML: when you call `model.to('cuda')`, the data is transferred from system RAM to GPU memory via DMA over the PCIe bus. During training, gradient synchronisation across GPUs uses DMA-based RDMA (Remote DMA) for high-bandwidth, low-latency transfers (chapter 6).
- The **bus** connects the CPU to memory and I/O devices. Modern systems use **PCIe** (Peripheral Component Interconnect Express) for high-speed devices (GPUs, NVMe SSDs, network cards). PCIe 4.0 provides ~32 GB/s per x16 slot; PCIe 5.0 doubles this. The bus bandwidth is often the bottleneck for GPU training: the GPU can compute faster than data can be fed to it.
- **MMIO** (Memory-Mapped I/O): device registers are mapped to memory addresses. The CPU reads and writes to these addresses using normal load/store instructions, and the hardware routes the access to the device instead of RAM. This unifies memory and I/O access into a single mechanism, simplifying both hardware and software.

## 62.9 Coding Tasks (use CoLab or notebook)

---

1. Explore IEEE 754 floating-point representation. Convert a float to its binary representation and observe the sign, exponent, and mantissa fields.

```
import struct

def float_to_bits(f):
 """Show the IEEE 754 binary representation of a float32."""
 packed = struct.pack('>f', f)
 bits = ''.join(f'{byte:08b}' for byte in packed)
 sign = bits[0]
 exponent = bits[1:9]
 mantissa = bits[9:]
 return sign, exponent, mantissa

for val in [1.0, -1.0, 0.1, 0.5, 3.14, float('inf'), float('nan')]:
 s, e, m = float_to_bits(val)
```



```
print(f"{val:>10} sign={s} exp={e} ({int(e, 2) - 127:>4d}) mantissa={m[:10]}...")
```

1. Simulate a direct-mapped cache. Track hits and misses for a sequence of memory accesses.

```
def simulate_cache(accesses, cache_size=8, block_size=1):
 """Simulate a direct-mapped cache."""
 cache = [None] * cache_size
 hits, misses = 0, 0

 for addr in accesses:
 cache_line = addr % cache_size
 if cache[cache_line] == addr:
 hits += 1
 status = "HIT "
 else:
 misses += 1
 cache[cache_line] = addr
 status = "MISS"
 print(f" Access {addr:3d} → line {cache_line}: {status}")

 print(f"\nHits: {hits}, Misses: {misses}, Hit rate: {hits/(hits+misses):.1%}")

Sequential access (good locality)
print("Sequential access:")
simulate_cache([0, 1, 2, 3, 4, 5, 6, 7, 0, 1, 2, 3])

Strided access (conflict misses)
print("\nStrided access (stride = cache size):")
simulate_cache([0, 8, 0, 8, 0, 8])
```

1. Demonstrate why floating-point arithmetic is not associative. Show cases where  $(a + b) + c \neq a + (b + c)$ .

```
import jax.numpy as jnp

a = jnp.float32(1e8)
b = jnp.float32(1.0)
c = jnp.float32(-1e8)

left = (a + b) + c # (1e8 + 1) + (-1e8)
right = a + (b + c) # 1e8 + (1 + (-1e8))
```

```
print(f"(a + b) + c = {left}") # should be 1.0
print(f"a + (b + c) = {right}") # might lose the 1.0
print(f"Equal: {left == right}")
print(f"\nThe 1.0 is lost when added to 1e8 because float32 has only ~7 digits of
precision")
```

## 63 Operating Systems

An operating system is the software layer between hardware and applications, managing resources, providing abstractions, and enforcing isolation. This file covers what an OS does, processes, threads, CPU scheduling, memory management, file systems, and system calls

- A computer without an operating system is like a kitchen without a chef: the ingredients (hardware) are there, but nothing coordinates who uses the stove, where to put the dishes, or how to prevent two people from grabbing the same knife. The **OS** is that coordinator.
- For ML practitioners, OS concepts explain: why `nvidia-smi` shows GPU memory usage per process, why training crashes with “out of memory,” why `fork()` duplicates your Python process, and why Docker containers provide isolated environments.

### 63.1 What an Operating System Does

---

- The OS has three core responsibilities:
  - **Abstraction:** hide hardware complexity behind clean interfaces. Programs read and write “files” without knowing whether the underlying storage is an SSD, HDD, or network drive. They allocate “memory” without managing physical RAM chips. They run on a “CPU” without worrying about interrupts and cache coherence.
  - **Resource management:** multiple programs share the CPU, memory, disk, and network. The OS decides who gets what, when, and for how long. A fair, efficient allocation strategy keeps the system responsive.
  - **Isolation and protection:** programs must not interfere with each other. A bug in your web browser should not crash the kernel. A malicious

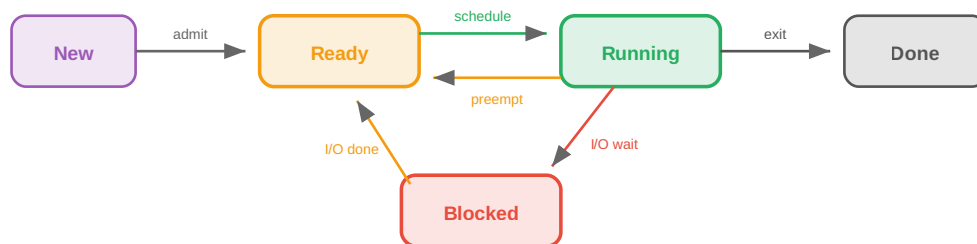
program should not read another program's passwords. The OS enforces boundaries using hardware support (privilege levels, virtual memory).

## 63.2 Processes

---

- A **process** is a running program. It is the OS's fundamental unit of work. Each process has:
  - **Code** (the program instructions, read-only).
  - **Data** (global variables, heap allocations).
  - **Stack** (function call frames, local variables).
  - **State** (register values, program counter, open files, etc.).
- The **Process Control Block (PCB)** is the OS's data structure for tracking a process. It stores the process ID (PID), state, program counter, register contents, memory maps, open file descriptors, and scheduling priority. When the OS switches from one process to another, it saves the current process's state to its PCB and loads the next process's state. This is a **context switch**.
- Context switches are expensive: saving and restoring registers, flushing caches, and invalidating TLB entries takes microseconds. On a system running thousands of processes, the overhead can be significant. This is why process-per-request server architectures (like old Apache) were replaced by thread-based or event-driven architectures.
- **Process creation** in Unix uses `fork()` and `exec()`:
  - `fork()` creates a **copy** of the current process. The child process gets a duplicate of the parent's memory, file descriptors, and state. Both processes continue executing from the same point, but `fork()` returns 0 in the child and the child's PID in the parent.
  - `exec()` replaces the current process's code with a new program. After `fork()`, the child typically calls `exec()` to run a different program.
  - This fork-then-exec model is elegant: creating a new process (`fork`) and loading a new program (`exec`) are separate operations that can be independently customised. Between `fork` and `exec`, the child can redirect I/O, change environment variables, or drop privileges.

### Process State Transitions



Process state transitions: new → ready → running → blocked/terminated, with preemption and I/O waits

- **Process states:** a process is in one of several states:
  - **Running:** currently executing on a CPU core.
  - **Ready:** waiting for a CPU core (runnable but not scheduled yet).
  - **Blocked (waiting):** cannot proceed until some event occurs (I/O completion, lock acquisition, timer expiry).
  - **Terminated:** finished execution, waiting for the parent to collect its exit status.

## 63.3 Threads

- A **thread** is a lightweight execution unit within a process. All threads in a process share the same code, data, and heap, but each has its own stack and register state.
- The advantage over multiple processes: threads share memory, so communication between them is fast (just read/write shared variables). Processes require inter-process communication (pipes, sockets, shared memory mappings), which is slower and more complex.
- The disadvantage: shared memory is dangerous. Two threads writing to the same variable simultaneously cause a **race condition** (the result depends on which thread runs first). This leads us to synchronisation, covered in file 4.
- **Kernel threads** are managed by the OS scheduler. Each thread is independently scheduled onto CPU cores. Creating and switching kernel threads involves system calls, with overhead similar to (but less than) process context switches.

- **User threads** (green threads) are managed by a runtime library in user space, invisible to the OS. They are cheaper to create and switch (no system call needed), but a blocking operation by one user thread blocks all threads in the process (because the OS sees only one kernel thread).
- Modern systems use **hybrid models**: many user threads mapped onto a smaller number of kernel threads (M:N threading). Go's goroutines and Erlang's processes are user-level threads scheduled by the language runtime onto OS threads.
- **Thread pools** pre-create a fixed number of threads that wait for tasks. When a task arrives, it is assigned to an idle thread. This avoids the overhead of creating and destroying threads for each task. Web servers, database engines, and ML inference servers all use thread pools.

## 63.4 CPU Scheduling

---

- The **scheduler** decides which process/thread runs on which CPU core at each moment. The goals are: maximise CPU utilisation, minimise response time (for interactive tasks), maximise throughput (for batch tasks), and ensure fairness.
- **First Come First Served (FCFS)**: processes run in arrival order. Simple but suffers from the **convoy effect**: a long-running process blocks all shorter ones behind it.
- **Shortest Job First (SJF)**: run the shortest process first. Provably minimises average waiting time, but requires knowing job lengths in advance (impossible in general). The preemptive version, **Shortest Remaining Time First (SRTF)**, interrupts a running job if a shorter one arrives.
- **Round Robin (RR)**: each process gets a fixed **time quantum** (e.g., 10 ms), then is preempted and moved to the back of the queue. Fair and responsive, but the time quantum matters: too small means excessive context switching, too large degrades to FCFS.
- **Priority scheduling**: each process has a priority. Higher-priority processes run first. The danger is **starvation**: low-priority processes may never run if high-priority processes keep arriving. **Aging** solves this: a process's priority increases the longer it waits.

- **Multilevel Feedback Queues (MLFQ)**: multiple queues with different priorities and time quanta. New processes start in the highest-priority queue (short quantum). If a process uses its full quantum (CPU-bound), it is demoted to a lower-priority queue (longer quantum). Interactive processes naturally stay in high-priority queues (they block for I/O before using their quantum). This adapts to workload without requiring advance knowledge of job types.
- **Completely Fair Scheduler (CFS)**: the Linux scheduler. It maintains a red-black tree (a balanced binary search tree) of processes sorted by “virtual runtime” – how much CPU time they have consumed. The process with the smallest virtual runtime runs next. This ensures that over time, every process gets its fair share. CFS runs in  $O(\log n)$  per scheduling decision.

## 63.5 Memory Management

---

- The OS manages physical RAM, allocating it to processes and reclaiming it when no longer needed.
- **Paging** (from file 2) divides virtual memory into fixed-size pages and physical memory into frames. The page table maps pages to frames. Paging eliminates external fragmentation (wasted space between allocations) because all pages are the same size.
- **Demand paging** loads pages into RAM only when they are first accessed (not when the process starts). This saves memory: a program with 1 GB of code might only use 50 MB during a typical run. The rest never gets loaded.
- When RAM is full and a new page is needed, the OS must **evict** an existing page. The **page replacement** algorithms (LRU, FIFO, clock, from file 2) decide which page to evict. Good replacement minimises page faults; bad replacement causes thrashing.
- **Segmentation** divides memory into variable-size segments (code, data, stack, heap), each with its own base address and length. Segments provide logical organisation, while paging provides physical management. Modern systems use segmentation minimally (mainly for protection) and rely on paging for memory management.
- The **heap** is where dynamically allocated memory lives (`malloc/free` in C, `new` in Java, implicit in Python). The OS provides large chunks of memory to the process, and a **memory allocator** (e.g., `glibc malloc`, `jmalloc`, `tcmalloc`)

subdivides these chunks into smaller allocations. Allocator design affects performance: fragmentation wastes space, and contention between threads wastes time.

## 63.6 File Systems

---

- A **file system** organises data on persistent storage (SSD, HDD) as a hierarchy of named files and directories.
- An **inode** (index node) stores a file's metadata: size, ownership, permissions, timestamps, and pointers to the data blocks on disk. The file name is stored in the directory, which maps names to inode numbers. This separation means a file can have multiple names (**hard links**) pointing to the same inode.
- **FAT** (File Allocation Table): a simple file system used on USB drives and SD cards. A table maps each cluster (block) to the next cluster in the file, forming a linked list. Simple but does not support permissions, journaling, or large files well.
- **ext4**: the default Linux file system. Uses inodes with direct, indirect, double-indirect, and triple-indirect block pointers to handle files of any size. Supports **extents** (contiguous ranges of blocks) for efficiency with large files. Maximum file size: 16 TB, maximum partition: 1 EB.
- **Journaling** protects against corruption from crashes. Before modifying file system structures, the changes are written to a **journal** (log). If the system crashes mid-operation, the journal is replayed on reboot to complete or undo the operation. Without journaling, a crash during a write could leave the file system in an inconsistent state (a file's data blocks updated but its inode not, or vice versa).
- **B-tree based file systems** (Btrfs, ZFS) use B-trees (balanced search trees) to organise data and metadata, enabling efficient search, copy-on-write snapshots, and built-in checksums for data integrity. These are the same B-trees used in database indexes.

## 63.7 System Calls and Kernel Mode

---

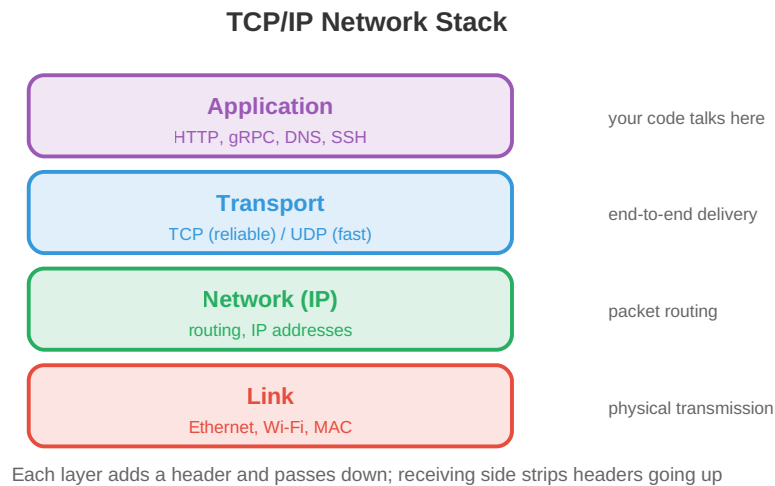
- A **system call** is the interface between user programs and the OS kernel. When a program needs to do something privileged (read a file, allocate memory, create a process, send a network packet), it makes a system call.
- The CPU operates in two modes:
  - **User mode**: restricted. Programs can execute their own code and access their own memory, but cannot directly access hardware, other processes' memory, or OS data structures.
  - **Kernel mode**: unrestricted. The OS kernel can access all hardware and memory. System calls are the controlled gateway from user mode to kernel mode.
- When a program calls `read()`, the following happens:
  1. The program puts arguments in registers and triggers a **trap** (a software interrupt).
  2. The CPU switches to kernel mode and jumps to the system call handler.
  3. The kernel validates the arguments, performs the I/O operation, and copies data to the user's buffer.
  4. The kernel switches back to user mode and returns the result.
- Common system calls: `open`, `read`, `write`, `close` (files), `fork`, `exec`, `wait`, `exit` (processes), `mmap`, `brk` (memory), `socket`, `bind`, `listen`, `accept` (networking).
- **Interrupts** are hardware signals that force the CPU to temporarily stop what it is doing and run an interrupt handler (in the kernel). A keyboard press, a network packet arrival, or a timer tick all generate interrupts. The timer interrupt is particularly important: it is what allows the OS to preempt a running process and switch to another (preemptive multitasking).

## 63.8 Networking Fundamentals

---

- The network stack is an OS subsystem that enables communication between machines. Understanding it explains how distributed training synchronises gradients, how model serving handles requests, and why latency matters.





TCP/IP stack: application, transport, network, and link layers, each adding headers

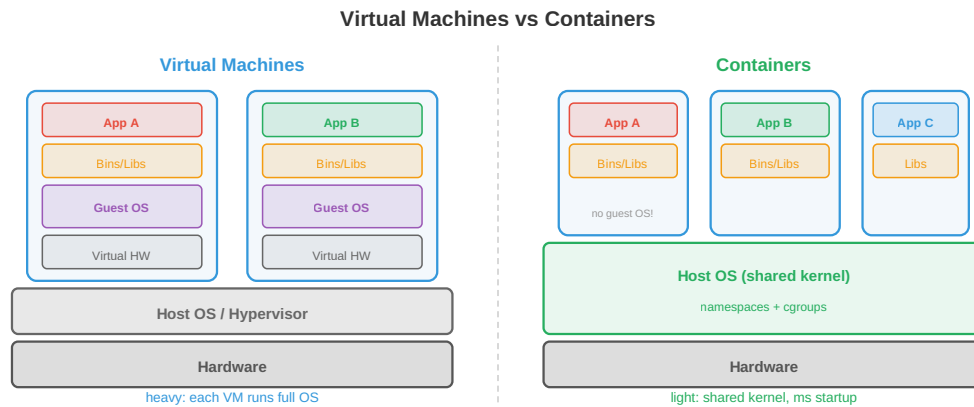
- The **TCP/IP model** organises networking into layers, each providing an abstraction to the layer above:
  - **Link layer:** handles communication over a single physical link (Ethernet, Wi-Fi). Deals with MAC addresses and frames.
  - **Network layer (IP):** routes packets across multiple networks from source to destination. Each machine has an **IP address** (e.g., 192.168.1.1 for IPv4, or a 128-bit IPv6 address). Routers forward packets hop by hop based on destination IP.
  - **Transport layer (TCP/UDP):** provides end-to-end communication between applications.
  - **Application layer:** protocols like HTTP, DNS, gRPC that applications use directly.
- **TCP** (Transmission Control Protocol) provides reliable, ordered delivery. It establishes a connection (three-way handshake: SYN, SYN-ACK, ACK), guarantees that all data arrives in order (using sequence numbers and acknowledgements), retransmits lost packets, and controls the sending rate to avoid overwhelming the network (**congestion control**). The cost is latency: the handshake adds a round trip, and retransmissions add delays.
- **UDP** (User Datagram Protocol) provides unreliable, unordered delivery. No handshake, no retransmission, no ordering guarantee. Much lower latency than TCP. Used where speed matters more than reliability: video streaming, online gaming, DNS lookups. In ML, some gradient synchronisation protocols use UDP-based RDMA for lower latency.

- **Sockets** are the OS API for network communication. A **socket** is an endpoint identified by (IP address, port number). A server creates a socket, binds it to a port (e.g., 80 for HTTP), listens for connections, and accepts them. A client creates a socket and connects to the server's address:port. Data is then read and written through the socket just like a file.
- **DNS** (Domain Name System) translates human-readable names (google.com) to IP addresses (142.250.80.46). It is a distributed, hierarchical database: your machine asks a local resolver, which asks root servers, which delegate to authoritative servers for each domain.
- **HTTP** (HyperText Transfer Protocol) is the request-response protocol of the web. A client sends a request (method + URL + headers + optional body), and the server sends a response (status code + headers + body). ML model serving (e.g., TensorFlow Serving, Triton) exposes models as HTTP or gRPC endpoints.
- **Latency vs bandwidth:** latency is the time for one packet to travel from source to destination (determined by physical distance and network hops). Bandwidth is the data rate (how many bytes per second). A high-bandwidth, high-latency connection (satellite internet) can transfer lots of data but each byte takes a long time to arrive. For distributed training, **latency** matters for synchronisation barriers (all GPUs must wait for the slowest), while **bandwidth** matters for transferring large gradient tensors (chapter 6).

## 63.9 Virtualisation and Containers

---

- **Virtualisation** runs multiple operating systems on a single physical machine. A **hypervisor** (VMware, KVM, Xen) creates **virtual machines (VMs)**, each with its own virtual CPU, memory, disk, and network interface. Each VM runs a complete OS (guest OS) that believes it has dedicated hardware.
- VMs provide strong isolation (one VM crashing does not affect others) and flexibility (run Linux and Windows on the same machine, migrate VMs between physical hosts). The cost is overhead: each VM runs a full OS kernel, consuming memory and CPU for OS operations that are redundant with the host OS.



VMs run separate guest OSes on virtual hardware; containers share the host kernel and are much lighter

- **Containers** (Docker, Podman) provide a lighter alternative. Instead of virtualising the entire hardware, containers share the host OS kernel and use kernel features to isolate processes:
  - **Namespaces** isolate what a process can see: each container gets its own view of the process tree (PID namespace), network interfaces (network namespace), file system mount points (mount namespace), and hostname (UTS namespace). A process inside a container cannot see processes in other containers.
  - **Cgroups** (control groups) limit what a process can use: CPU time, memory, disk I/O, network bandwidth. A container cannot consume more resources than its cgroup allows, preventing one container from starving others.
- Containers start in milliseconds (no OS boot), use minimal overhead (shared kernel), and are defined by a **Dockerfile** that specifies the base image, dependencies, and commands. This makes them reproducible: `docker build` produces the same environment everywhere.
- For ML, containers solve the “it works on my machine” problem. A training environment with specific versions of CUDA, cuDNN, PyTorch, and Python is packaged as a container image. Anyone can reproduce the exact environment on any machine. Cloud training platforms (AWS SageMaker, GCP Vertex AI) run training jobs in containers.
- **Kubernetes** (K8s) orchestrates containers at scale: it schedules containers onto a cluster of machines, restarts failed containers, scales up/down based on load, and manages networking between containers. Large-scale ML serving

(thousands of model replicas handling millions of requests) runs on Kubernetes.

## 63.10 Security Basics

---

- The OS enforces security through multiple mechanisms:
- **Permissions:** every file has an owner, a group, and permission bits (read/write/execute for owner, group, and others). A process runs with the identity (UID) of the user who started it and can only access files that the permission bits allow. The **root** user (UID 0) bypasses all permission checks, which is why running as root is dangerous.
- **Privilege separation:** processes run with the minimum privileges needed. A web server does not need root access; it should run as a restricted user that can only read web files and bind to port 80. If the server is compromised, the attacker's access is limited to what the restricted user can do.
- **Sandboxing:** restrict what a process can do beyond file permissions. **seccomp** (Linux) limits which system calls a process can make. **AppArmor** and **SELinux** define mandatory access control policies. Containers combine namespaces, cgroups, and seccomp for multi-layered isolation.
- **Address Space Layout Randomisation (ASLR):** randomise the memory locations of the stack, heap, and libraries each time a program runs. This makes it harder for attackers to exploit memory corruption bugs (buffer overflows), because they cannot predict where code or data will be in memory.
- Security is a systems-wide concern: a chain is only as strong as its weakest link. A model serving system needs secure network communication (TLS/HTTPS), authenticated API access (API keys, OAuth), input validation (prevent adversarial inputs), and isolated execution (containers with minimal privileges).

## 63.11 Coding Tasks (use CoLab or notebook)

---

1. Explore process creation. Use Python's `os.fork()` (Unix only) to create a child process and observe that both parent and child continue from the same point.

```
import os

pid = os.fork()
```

```

if pid == 0:
 # Child process
 print(f"Child: my PID is {os.getpid()}, parent PID is {os.getppid()}")
else:
 # Parent process
 print(f"Parent: my PID is {os.getpid()}, child PID is {pid}")
 os.wait() # wait for child to finish

```

1. Simulate round-robin scheduling. Given a list of processes with burst times, simulate the scheduling and compute average waiting time.

```

def round_robin(processes, quantum=3):
 """Simulate round-robin scheduling.
 processes: list of (name, burst_time) tuples.
 """
 queue = [(name, burst, 0) for name, burst in processes] # (name, remaining,
wait)
 time = 0
 log = []

 while queue:
 name, remaining, waited = queue.pop(0)
 waited += (time - waited - (processes[[p[0] for p in
processes].index(name)][1] - remaining))
 run_time = min(quantum, remaining)
 log.append(f" t={time:3d}: {name} runs for {run_time} (remaining:
{remaining - run_time})")
 time += run_time
 remaining -= run_time

 if remaining > 0:
 queue.append((name, remaining, time))
 else:
 log.append(f" t={time:3d}: {name} DONE (turnaround: {time})")

 for line in log:
 print(line)

 print("Round Robin (quantum=3):")
 round_robin([("P1", 10), ("P2", 4), ("P3", 6)], quantum=3)

```

1. Simulate page replacement with LRU. Given a sequence of page accesses and a fixed number of frames, count page faults.

```

def lru_page_replacement(pages, n_frames):
 """Simulate LRU page replacement."""
 frames = []
 faults = 0

 for page in pages:
 if page in frames:
 frames.remove(page)
 frames.append(page) # move to most recently used
 status = "HIT "
 else:
 faults += 1
 if len(frames) >= n_frames:
 evicted = frames.pop(0) # remove least recently used
 status = f"MISS (evict {evicted})"
 else:
 status = "MISS (cold)"
 frames.append(page)
 print(f" Page {page}: {status} frames={frames}")

 print(f"\nTotal faults: {faults}/{len(pages)} ({faults/len(pages):.0%}")

print("LRU with 3 frames:")
lru_page_replacement([1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5], n_frames=3)

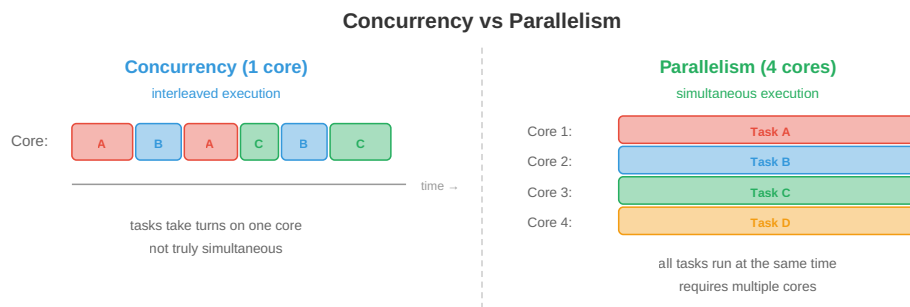
```

## 64 Concurrency and Parallelism

Concurrency and parallelism are how programs do multiple things at once. This file covers the distinction between concurrency and parallelism, synchronisation primitives, classic concurrency problems, deadlock, lock-free data structures, parallel programming models, async programming, and scaling laws, the concepts that underpin multi-threaded servers, distributed training, and every modern application.

- A single CPU core executes one instruction at a time. But modern systems have 8, 64, or even thousands of cores (GPUs). And even on a single core, we want to handle multiple tasks: download a file while rendering a UI while processing user input. **Concurrency** and **parallelism** are the two strategies for managing multiple activities.

## 64.1 Concurrency vs Parallelism



Concurrency interleaves tasks on one core; parallelism runs tasks simultaneously on multiple cores

- **Concurrency** is about managing multiple tasks. Tasks make progress by interleaving: task A runs for a bit, then task B, then back to A. On a single core, concurrency creates the illusion of simultaneous execution. The tasks are not truly simultaneous; they take turns.
- **Parallelism** is about executing multiple tasks simultaneously. With  $n$  cores,  $n$  tasks can genuinely run at the same time. Parallelism requires multiple hardware execution units.
- The analogy: concurrency is one chef alternating between chopping vegetables and stirring the pot. Parallelism is two chefs, each doing one task simultaneously. A system can be concurrent but not parallel (one core, interleaved tasks), parallel but not concurrent (multiple cores running independent programs with no interaction), or both (multiple cores running interleaved tasks that interact).
- In ML, concurrency appears in data loading (overlap data preprocessing with GPU computation), while parallelism appears in distributed training (multiple GPUs computing gradients simultaneously, chapter 6).

## 64.2 Synchronisation Primitives

- When multiple threads share data, **synchronisation** prevents race conditions. A race condition occurs when the outcome depends on the unpredictable order of thread execution.
- Consider two threads both incrementing a shared counter: `counter += 1`. This is actually three operations: (1) read counter, (2) add 1, (3) write counter. If both

threads read the same value (say 5), both add 1, and both write 6, the counter ends up at 6 instead of the correct 7. One increment is lost.

- A **mutex** (mutual exclusion lock) ensures only one thread accesses a critical section at a time. A thread **acquires** the lock before entering the critical section and **releases** it after. Any other thread trying to acquire a held lock blocks until it is released.

```
lock.acquire()
counter += 1 # only one thread at a time here
lock.release()
```

- Mutexes are correct but introduce **contention**: if many threads compete for the same lock, they spend time waiting instead of computing. This limits scalability. The extreme case, where all threads want the same lock, serialises the entire program.
- A **semaphore** generalises a mutex. A counting semaphore maintains a counter: `wait()` decrements the counter (blocking if it would go negative), and `signal()` increments it. A semaphore initialised to 1 behaves like a mutex. A semaphore initialised to `n` allows up to `n` threads into the critical section simultaneously (useful for resource pools like database connections).
- A **condition variable** lets a thread wait until a specific condition is met. The thread releases a lock, waits on the condition variable, and is woken up when another thread signals the condition. This avoids busy-waiting (repeatedly checking a condition in a loop, wasting CPU).
- A **monitor** bundles a mutex with condition variables and shared data into a single abstraction. Java's `synchronized` keyword and Python's `threading.Condition` implement monitor-like semantics.
- **Read-write locks** distinguish between readers (who can share access, since reading does not modify data) and writers (who need exclusive access). Multiple readers can hold the lock simultaneously, but a writer blocks all readers and other writers. This is optimal when reads far outnumber writes (e.g., a cached model serving predictions).



## 64.3 Classic Concurrency Problems

---

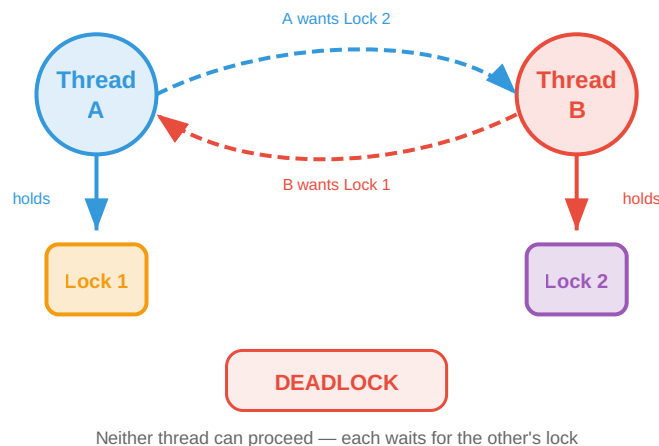
- **Producer-Consumer** (bounded buffer): producers generate items and place them in a fixed-size buffer; consumers remove items. The challenges: producers must wait when the buffer is full, consumers must wait when it is empty, and both must avoid corrupting the buffer.
- Solution uses two semaphores (one counting empty slots, one counting full slots) plus a mutex for the buffer itself. This is the pattern behind most message queues, logging systems, and data pipelines.
- **Readers-Writers**: multiple readers can read simultaneously, but writers need exclusive access. The challenge is fairness: if readers keep arriving, the writer may starve (never get access). Solutions prioritise either readers or writers, or alternate fairly.
- **Dining Philosophers**: five philosophers sit around a table with five forks between them. Each needs two forks to eat. If all five pick up their left fork simultaneously, nobody can pick up their right fork and everyone starves (deadlock). Solutions include: pick up both forks atomically, introduce asymmetry (one philosopher picks up the right fork first), or use a waiter (semaphore limiting diners to 4).

## 64.4 Deadlock

---

- **Deadlock** occurs when a set of threads are each waiting for a resource held by another thread in the set, creating a cycle of dependencies. Nobody can proceed.

### Deadlock: Circular Wait



Deadlock: Thread A holds Lock 1 and wants Lock 2, Thread B holds Lock 2 and wants Lock 1 — circular wait

- Four **necessary conditions** for deadlock (all must hold simultaneously):
  1. **Mutual exclusion**: the resource can only be held by one thread.
  2. **Hold and wait**: a thread holds one resource while waiting for another.
  3. **No preemption**: resources cannot be forcibly taken from a thread.
  4. **Circular wait**: a cycle exists in the wait-for graph.
- **Deadlock prevention** breaks one of the four conditions:
  - Eliminate circular wait: impose a total ordering on resources. All threads acquire resources in the same order. If every thread always acquires lock A before lock B, a cycle is impossible.
  - Eliminate hold and wait: require threads to request all resources at once (atomically).
- **Deadlock avoidance** dynamically decides whether granting a resource request could lead to deadlock. The **Banker's algorithm** maintains the maximum possible demand of each thread and only grants requests that leave the system in a “safe state” (a state where all threads can eventually complete). The algorithm is  $O(n^2m)$  per request ( $n$  threads,  $m$  resource types), which is too expensive for most real systems.
- **Deadlock detection** lets deadlocks happen, then detects them (by finding cycles in the wait-for graph) and recovers (by killing a thread or rolling back a transaction).

- In practice, most systems use prevention (resource ordering) for common cases and detection for rare cases. Database systems are the classic example: they detect deadlocks among transactions and abort one to break the cycle.

## 64.5 Lock-Free and Wait-Free Data Structures

---

- Locks introduce contention, priority inversion, and the risk of deadlock. **Lock-free** data structures avoid locks entirely, using **atomic operations** provided by the hardware.
- The key atomic operation is **Compare-And-Swap (CAS)**: atomically check if a memory location has an expected value, and if so, replace it with a new value. In pseudocode:

```
CAS(address, expected, new_value):
 if *address == expected:
 *address = new_value
 return true
 else:
 return false
```

- CAS is implemented as a single hardware instruction, so it is atomic even without locks. Lock-free algorithms use CAS in a retry loop: read the current value, compute the new value, attempt to CAS. If another thread modified the value in the meantime, the CAS fails and the thread retries.
- **Lock-free**: at least one thread makes progress in a finite number of steps (no deadlock possible, but individual threads might retry indefinitely under contention).
- **Wait-free**: every thread makes progress in a bounded number of steps (strongest guarantee, but hardest to achieve).
- Lock-free stacks, queues, and hash maps are widely used in high-performance systems. Java's `ConcurrentHashMap` and Go's atomic operations are built on CAS.

## 64.6 Parallel Programming Models

---

- **Shared memory** parallelism: all threads access the same memory space. Synchronisation is the programmer's responsibility. **OpenMP** provides compiler directives to parallelise loops:

```
#pragma omp parallel for
for (int i = 0; i < n; i++) {
 result[i] = compute(data[i]);
}
```

- The compiler splits the loop iterations across available cores. OpenMP is effective for data-parallel workloads (same operation on many data points) and is widely used in scientific computing.
- **Message passing** parallelism: each process has its own memory. Communication happens by sending and receiving messages. **MPI** (Message Passing Interface) is the standard for distributed computing across nodes:

```
MPI_Send(data, count, MPI_FLOAT, dest, tag, MPI_COMM_WORLD);
MPI_Recv(data, count, MPI_FLOAT, src, tag, MPI_COMM_WORLD, &status);
```

- MPI scales to thousands of nodes because there is no shared state to synchronise. Distributed deep learning (chapter 6) uses collective operations like `MPI_AllReduce` (ring all-reduce) to synchronise gradients across GPUs.
- **GPU parallelism** follows the **SIMT** (Single Instruction, Multiple Threads) model: thousands of threads execute the same instruction on different data. This is perfect for matrix operations (chapter 2), where the same multiply-add is applied to every element. We will cover GPU programming in detail in a later chapter.

## 64.7 Async and Event-Driven Programming

---

- Not all concurrency needs threads. **Asynchronous** programming handles many I/O-bound tasks with a single thread using an **event loop**.
- The event loop maintains a queue of tasks. When a task needs to wait for I/O (network response, file read), it registers a callback and yields control. The

event loop picks up the next ready task. When the I/O completes, the callback is queued and eventually executed. No thread is blocked during the wait.

- **Coroutines** are functions that can suspend and resume. `async/await` syntax (Python, JavaScript, Rust) makes coroutines look like regular sequential code:

```
async def fetch_data(url):
 response = await http_get(url) # suspends here, event loop runs other tasks
 return process(response) # resumes when response arrives
```

- The `await` keyword suspends the coroutine and returns control to the event loop. When the awaited operation completes, the coroutine resumes where it left off. This is cooperative multitasking: coroutines voluntarily yield, unlike preemptive multitasking where the OS forcibly switches threads.
- Async is ideal for **I/O-bound** workloads with many concurrent connections (web servers handling thousands of clients). It is not suitable for **CPU-bound** work (the single-threaded event loop cannot utilise multiple cores). For CPU-bound work, use threads or processes.
- Python's **Global Interpreter Lock (GIL)** prevents true parallelism with threads: only one thread can execute Python bytecode at a time. This is why Python uses multiprocessing (separate processes, each with its own interpreter) for CPU parallelism, and async for I/O concurrency. The GIL is being removed in Python 3.13+ (free-threaded Python), which will enable true multi-threaded parallelism.

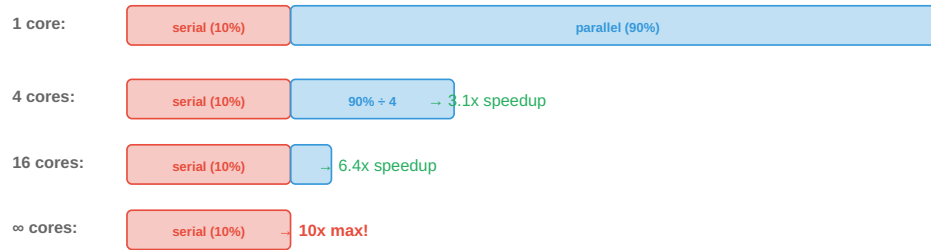
## 64.8 Scaling Laws

---

- **Amdahl's law** describes the theoretical speedup from parallelising a program. If a fraction  $p$  of the program is parallelisable and the rest  $1 - p$  is serial:

$$\text{Speedup}(n) = \frac{1}{(1 - p) + \frac{p}{n}}$$

### Amdahl's Law: The Serial Bottleneck



The serial fraction (red) cannot be parallelised — it limits the maximum speedup to  $1/(1-p)$

Amdahl's law: the serial fraction limits maximum speedup — 10% serial means 10x max, no matter how many cores

- where  $n$  is the number of processors. As  $n \rightarrow \infty$ , the maximum speedup approaches  $\frac{1}{1-p}$ . If 95% of the program is parallel, the maximum speedup is

$\frac{1}{0.05} = 20\times$ , regardless of how many cores you add. The serial portion is the bottleneck.

- This has profound implications for ML: if data loading takes 10% of training time and is serial, adding more GPUs can at most speed up training by 10x. The 10% serial bottleneck limits everything (this is why efficient data pipelines and overlapping compute with I/O matter, chapter 6).
- **Gustafson's law** offers a more optimistic perspective. Instead of fixing the problem size and adding processors, it fixes the total time and asks how much more work can be done. If the parallel portion scales with the problem size:

$$\text{Speedup}(n) = 1 - p + p \cdot n$$

- This is linear in  $n$ . The argument: with more processors, we solve bigger problems, not the same problem faster. In ML, this corresponds to increasing the batch size with more GPUs (weak scaling) rather than keeping the batch size fixed (strong scaling).

## 64.9 Coding Tasks (use CoLab or notebook)

1. Demonstrate a race condition. Two threads increment a shared counter without synchronisation and observe the lost updates.

```
import threading
```

```

counter = 0

def increment(n):
 global counter
 for _ in range(n):
 counter += 1 # NOT atomic: read, add, write

threads = [threading.Thread(target=increment, args=(100000,)) for _ in range(4)]
for t in threads: t.start()
for t in threads: t.join()

print(f"Expected: {4 * 100000}")
print(f"Actual: {counter}")
print(f"Lost updates: {4 * 100000 - counter}")

```

1. Fix the race condition with a lock and measure the overhead.

```

import threading
import time

lock = threading.Lock()
counter = 0

def increment_locked(n):
 global counter
 for _ in range(n):
 with lock:
 counter += 1

start = time.time()
threads = [threading.Thread(target=increment_locked, args=(100000,)) for _ in range(4)]
for t in threads: t.start()
for t in threads: t.join()
elapsed = time.time() - start

print(f"Counter: {counter} (correct: {4 * 100000})")
print(f"Time with lock: {elapsed:.3f}s")

```

1. Visualise Amdahl's law. Plot speedup vs number of processors for different parallel fractions.

```

import jax.numpy as jnp
import matplotlib.pyplot as plt

n_procs = jnp.arange(1, 65)

```

```

for p, color in [(0.5, "#e74c3c"), (0.9, "#f39c12"), (0.95, "#27ae60"), (0.99,
"#3498db")]:
 speedup = 1 / ((1 - p) + p / n_procs)
 plt.plot(n_procs, speedup, color=color, linewidth=2, label=f"p={p}")
 # Max speedup line
 plt.axhline(1 / (1 - p), color=color, linestyle="--", alpha=0.3)

plt.xlabel("Number of processors")
plt.ylabel("Speedup")
plt.title("Amdahl's Law: Serial Fraction Limits Speedup")
plt.legend()
plt.grid(True)
plt.show()

```

## 65 Programming Languages

Programming languages are the interface between human intent and machine execution. This file covers language paradigms, type systems, memory management strategies, the compilation pipeline, interpretation and JIT compilation, key language features, domain-specific languages, and design tradeoffs

- Every piece of software, every ML model, every operating system is written in a programming language. But there are hundreds of languages, each with different strengths. Why? Because language design involves fundamental tradeoffs: performance vs safety, expressiveness vs simplicity, control vs abstraction. Understanding these tradeoffs helps you pick the right tool for the job and understand the constraints you are working within.

### 65.1 Language Paradigms

---

- A **paradigm** is a style of programming: a set of principles that guide how you structure code and think about problems.
- **Imperative** programming describes computation as a sequence of commands that change state. “Set x to 5. Add 3 to x. If x > 7, print it.” C, Python, and Java are imperative at their core. The mental model is a machine with memory that you modify step by step.
- **Object-oriented (OOP)** programming organises code around **objects**: bundles of data (attributes) and behaviour (methods). Objects interact by sending



messages to each other. The key ideas are **encapsulation** (hide internal state behind a public interface), **inheritance** (create new classes by extending existing ones), and **polymorphism** (treat different types uniformly through a shared interface). Java, C++, and Python support OOP.

- **Functional programming (FP)** treats computation as the evaluation of mathematical functions. Core principles: **immutability** (data does not change once created), **pure functions** (output depends only on input, no side effects), and **first-class functions** (functions are values that can be passed as arguments, returned from other functions, and stored in variables). Haskell is purely functional. Python, JavaScript, and Scala support functional style.
- Pure functions are easy to reason about, test, and parallelise (no shared mutable state means no race conditions). This is why functional ideas are increasingly used in distributed systems and data pipelines. JAX (used throughout this book) is functional: `jax.grad` works because JAX functions are pure.
- **Logic programming** describes what should be true, not how to compute it. You state facts and rules, and the runtime finds solutions. Prolog is the classic example: given “Socrates is a man” and “all men are mortal,” the engine derives “Socrates is mortal.” Logic programming is used in AI knowledge bases and type checking.
- Most modern languages are **multi-paradigm**: Python supports imperative, OOP, and functional styles. Rust supports imperative and functional. The paradigm is a tool, not a religion.

## 65.2 Type Systems

---

- A **type** classifies values and determines what operations are valid. The integer 3 and the string “3” are different types: you can add integers but not strings (well, you can concatenate strings, but that is a different operation).
- **Static typing**: types are checked at **compile time**, before the program runs. Type errors are caught early. C, Java, Rust, and Go are statically typed. You must declare types (or the compiler infers them):

```
let x: i32 = 5; // Rust: x is a 32-bit integer
let y: f64 = 3.14; // y is a 64-bit float
// let z = x + y; // compile error: cannot add i32 and f64
```

- **Dynamic typing:** types are checked at **runtime**, when operations are actually executed. More flexible but type errors only surface when the code runs. Python, JavaScript, and Ruby are dynamically typed:

```
x = 5 # x is an int (for now)
x = "hello" # now x is a string -- no error
```

- **Strong typing:** the language prevents implicit type coercion. Python is strongly typed: "3" + 5 raises a TypeError. **Weak typing:** the language silently converts types. JavaScript is weakly typed: "3" + 5 gives "35" (number is coerced to string). C is weakly typed: you can cast a pointer to an integer.
- **Type inference** lets the compiler deduce types without explicit annotations:

```
let x = 5; // compiler infers: i32
let y = x + 3.0; // compile error: mixed types, even with inference
```

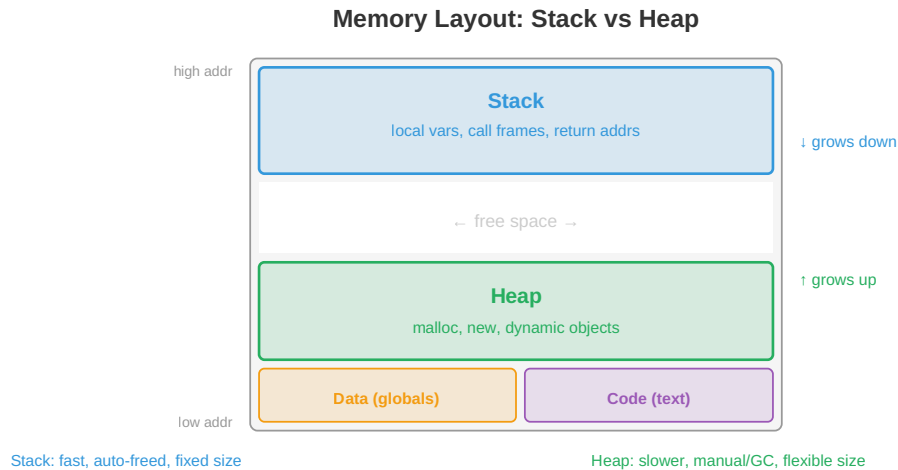
- **Generics** (parametric polymorphism) let you write code that works with any type:

```
fn largest<T: PartialOrd>(list: &[T]) -> &T {
 let mut max = &list[0];
 for item in &list[1..] {
 if item > max { max = item; }
 }
 max
}
// Works for integers, floats, strings -- any type that supports comparison
```

- For ML: Python's dynamic typing makes experimentation fast but hides bugs. Production ML systems increasingly use type hints (def train(model: nn.Module, lr: float) -> float) and static analysis tools (mypy) to catch errors before deployment. PyTorch and JAX use Python for flexibility; TensorRT and ONNX Runtime use C++ for performance.

## 65.3 Memory Management

- Every program allocates and frees memory. How this is managed is one of the most consequential language design decisions.



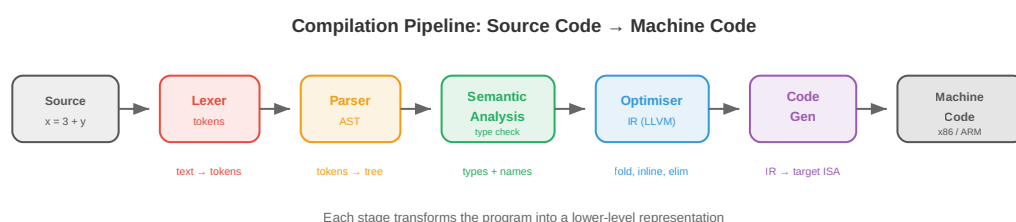
Memory layout: stack grows down from high addresses, heap grows up, with code and data at the bottom

- The **stack** stores local variables and function call frames. Allocation is trivial (move the stack pointer) and deallocation is automatic (pop the frame when the function returns). Stack access is fast because it is always in cache. But the stack has fixed size (typically 1-8 MB) and only supports LIFO (last-in-first-out) allocation.
- The **heap** stores dynamically allocated data (objects, arrays, strings whose size is not known at compile time). Heap allocation is slower (need to find a free block) and requires explicit or automatic deallocation. The heap can grow to fill available memory.
- **Manual memory management** (C, C++): the programmer explicitly allocates (`malloc`) and frees (`free`) heap memory. Maximum control and performance, but extremely error-prone:
  - **Use-after-free**: accessing memory that has been freed. Causes crashes or security vulnerabilities.
  - **Double free**: freeing the same memory twice. Corrupts the allocator's internal data structures.
  - **Memory leak**: allocating memory but never freeing it. The program slowly consumes all available RAM.

- **Garbage collection (GC)**: the runtime automatically detects and frees memory that is no longer reachable. The programmer never calls `free`.
  - **Tracing GC** (Java, Go, Python's cycle collector): periodically traverses all reachable objects from "roots" (stack variables, global variables) and frees unreachable objects. Simple but causes **GC pauses**: the program stops while the collector runs. Modern collectors (Go's concurrent GC, Java's ZGC) minimise pause times to sub-millisecond.
  - **Reference counting** (Python's primary mechanism, Swift, Objective-C): each object tracks how many references point to it. When the count drops to 0, the object is immediately freed. No pauses, but cannot handle **cycles** (A references B, B references A, both have count > 0 but neither is reachable). Python uses a separate cycle detector to handle this.
- **Ownership** (Rust): the compiler enforces memory safety rules at compile time, with zero runtime overhead.
  - Each value has exactly one **owner**. When the owner goes out of scope, the value is dropped (freed).
  - Values can be **borrowed** (referenced) but the compiler enforces: either one mutable reference OR any number of immutable references, never both simultaneously.
  - This prevents use-after-free, double free, data races, and dangling pointers, all at compile time. No GC, no runtime cost.
- The **borrow checker** is Rust's killer feature and its steepest learning curve. It guarantees memory safety and thread safety without garbage collection, which is why Rust is increasingly used for performance-critical systems (OS kernels, game engines, ML inference runtimes like Candle and Burn).

## 65.4 Compilation Pipeline

- A **compiler** translates source code into machine code (or another target language) before the program runs. The pipeline has several stages:



Compilation pipeline: source  $\rightarrow$  lexer  $\rightarrow$  parser  $\rightarrow$  semantic analysis  $\rightarrow$  optimiser  $\rightarrow$  code generation  $\rightarrow$  machine code

1. **Lexing** (tokenisation): convert the source text into a stream of tokens.  $x = 3 + y$  becomes `[IDENT("x"), EQUALS, INT(3), PLUS, IDENT("y")]`. The lexer strips whitespace and comments.
  2. **Parsing**: build an **Abstract Syntax Tree (AST)** from the token stream. The AST represents the hierarchical structure of the program.  $3 + y * 2$  parses as `Add(3, Mul(y, 2))` (multiplication has higher precedence). The parser checks syntax: mismatched parentheses and missing semicolons are caught here.
  3. **Semantic analysis**: check types, resolve variable names, verify that functions are called with the correct arguments. This is where static type checking happens. The output is a typed, annotated AST.
  4. **Optimisation**: transform the program to run faster without changing its behaviour. Common optimisations:
    - **Constant folding**: compute  $3 + 5$  at compile time, replacing it with 8.
    - **Dead code elimination**: remove code that can never execute.
    - **Loop unrolling**: replace a loop with repeated inline code to reduce branch overhead.
    - **Inlining**: replace a function call with the function's body, eliminating call overhead.
  5. **Code generation**: translate the optimised representation into target machine code (x86, ARM) or an intermediate representation.
- **LLVM** is the dominant compiler infrastructure. It provides a common intermediate representation (LLVM IR) that many languages compile to. LLVM's optimiser works on this IR, and its backends generate machine code for many targets. Clang (C/C++), Rust, Swift, Julia, and many other languages use LLVM. This means improvements to LLVM's optimiser benefit all these languages simultaneously.

## 65.5 Interpretation and JIT Compilation

---

- An **interpreter** executes the program line by line (or statement by statement) without producing machine code. This makes startup fast and development interactive, but execution is slower (each line is re-analysed every time it runs).

- Most interpreted languages actually compile to **bytecode**: an intermediate representation that is simpler than source code but not machine-specific. The bytecode runs on a **virtual machine (VM)**.
  - **CPython** (the standard Python implementation) compiles Python source to bytecode (.pyc files), which is executed by the CPython VM. The VM interprets bytecode one instruction at a time. This is why Python is ~100x slower than C for compute-heavy code.
  - **JVM** (Java Virtual Machine): Java compiles to JVM bytecode (.class files). The JVM initially interprets the bytecode, then **JIT-compiles** frequently executed code paths ( “hot spots” ) to native machine code. This is why Java starts slower than C (interpretation overhead) but approaches C’ s speed for long-running programs (JIT-optimised hot paths).
- **JIT (Just-In-Time) compilation** compiles code to machine code at runtime, using information available only during execution. A JIT can optimise based on actual runtime data: if a function is always called with integer arguments, the JIT generates specialised integer-only machine code, skipping type checks.
- **PyPy** is an alternative Python implementation with a JIT compiler. It runs most Python code 5-10x faster than CPython by JIT-compiling hot loops to machine code. However, it has limited compatibility with C extension modules (NumPy, PyTorch), which limits its use in ML.
- The spectrum from interpretation to compilation is not binary:
  - Pure interpretation: Bash shell scripts.
  - Bytecode interpretation: CPython.
  - Bytecode + JIT: JVM, .NET CLR, LuaJIT, PyPy.
  - Ahead-of-time (AOT) compilation: C, C++, Rust, Go.
  - AOT + runtime code generation: JAX’ s `jax.jit` compiles Python functions to optimised XLA code at the first call, then caches the compiled version.

## 65.6 Key Language Features

---

- **Closures**: a function that captures variables from its enclosing scope. The function “closes over” the environment in which it was defined:

```
def make_adder(n):
 def add(x):
```

```

 return x + n # n is captured from the enclosing scope
 return add

add5 = make_adder(5)
print(add5(3)) # 8

```

- Closures are the mechanism behind callbacks, decorators, and partial application. They are fundamental to functional programming.
- **Pattern matching:** a powerful control flow mechanism that destructures data and branches based on its shape:

```

match value {
 Some(x) if x > 0 => println!("Positive: {}", x),
 Some(0) => println!("Zero"),
 Some(x) => println!("Negative: {}", x),
 None => println!("Nothing"),
}

```

- Pattern matching is more expressive than if-else chains: it checks the structure of data (is it Some or None? does it contain a value matching a condition?), not just equality. Python added structural pattern matching in 3.10 (`match/case`).
- **Algebraic data types (ADTs):** types that can be one of several variants, each carrying different data. A `Result` type is either `Ok(value)` or `Err(error)`. A `Tree` is either `Leaf(value)` or `Node(left, right)`. ADTs combined with pattern matching enable exhaustive handling of all cases, eliminating entire classes of bugs (null pointer exceptions, unhandled error codes).
- **Traits and interfaces:** define a set of methods that a type must implement, without specifying how. This enables polymorphism: a function that takes “anything that implements the `Display` trait” works with integers, strings, and custom types. Rust uses traits, Java uses interfaces, Go uses implicit interfaces, Python uses duck typing ( “if it walks like a duck...” ).

## 65.7 Domain-Specific Languages

- A **domain-specific language (DSL)** is designed for a specific problem domain, trading generality for expressiveness within that domain.
- **SQL:** the language of relational databases. `SELECT name FROM users WHERE age > 30` is far more readable and optimisable than the equivalent imperative loop. The

database engine optimises the query execution plan, choosing join strategies and index usage automatically.

- **Regular expressions:** a mini-language for pattern matching in text. `\d{3}-\d{4}` matches phone numbers like “555-1234.” Regex engines compile patterns to finite automata for efficient matching.
- **Shader languages** (GLSL, HLSL, Metal Shading Language): programs that run on GPU cores to compute pixel colours, vertex positions, or compute operations. Shaders are massively parallel: each invocation processes one pixel or one element independently. This is the same execution model used by CUDA for ML computation.
- In ML, frameworks like PyTorch and JAX are essentially DSLs for tensor computation, embedded within Python. They provide domain-specific abstractions (tensors, automatic differentiation, device placement) while leveraging Python’s ecosystem.

## 65.8 Language Design Tradeoffs

---

- No language is best at everything. Design is about choosing which tradeoffs to make:
- **Performance vs safety:** C gives raw speed and hardware control but lets you corrupt memory. Rust gives comparable speed with compile-time memory safety. Java gives memory safety with garbage collection overhead. Python gives maximum safety and expressiveness with 100x slower execution.
- **Expressiveness vs simplicity:** Haskell’s type system can express very precise constraints but has a steep learning curve. Go deliberately omits generics (until recently), inheritance, and exceptions for simplicity. Python’s “there should be one obvious way to do it” philosophy keeps the language learnable.
- **Control vs abstraction:** C/C++ give you control over memory layout, cache behaviour, and hardware interaction. Python hides all of this. For ML training (where GPU compute dominates), Python’s overhead is negligible. For ML inference (where every microsecond counts), C++ or Rust may be necessary.
- **Compilation speed vs runtime speed:** Go compiles in seconds (simple type system, minimal optimisation). Rust compiles in minutes (complex type system,



aggressive optimisation). The tradeoff is developer iteration speed vs deployed performance.

- The ML ecosystem reflects these tradeoffs: Python for experimentation and training (expressiveness wins), C++/CUDA for kernels and inference (performance wins), Rust for infrastructure and safety-critical systems (safety wins).

## 65.9 Coding Tasks (use CoLab or notebook)

---

1. Explore closures and higher-order functions. Implement a simple function factory and verify that closures capture their environment.

```
def make_multiplier(factor):
 """Returns a function that multiplies by factor."""
 def multiply(x):
 return x * factor
 return multiply

double = make_multiplier(2)
triple = make_multiplier(3)

print(f"double(5) = {double(5)}") # 10
print(f"triple(5) = {triple(5)}") # 15

Closures capture by reference, not by value
def make_counter():
 count = [0] # mutable container to allow modification
 def increment():
 count[0] += 1
 return count[0]
 return increment

counter = make_counter()
print(f"counter() = {counter()}") # 1
print(f"counter() = {counter()}") # 2
print(f"counter() = {counter()}") # 3
```

1. Compare dynamic vs static typing behaviour. Show how Python's dynamic typing allows flexibility but can hide bugs.

```
def add(a, b):
 return a + b
```

```

Works with different types -- flexible!
print(add(3, 5)) # 8 (int + int)
print(add("hello ", "world")) # "hello world" (str + str)
print(add([1, 2], [3, 4])) # [1, 2, 3, 4] (list + list)

But type errors only surface at runtime:
try:
 print(add("hello", 5)) # TypeError! str + int
except TypeError as e:
 print(f"Runtime error: {e}")
 print("A static type checker would catch this before running")

```

1. Measure the performance difference between interpreted Python and compiled/JIT approaches for a compute-heavy task.

```

import time
import jax
import jax.numpy as jnp

n = 1_000_000

Pure Python loop (interpreted)
start = time.time()
total = 0.0
for i in range(n):
 total += i * i
python_time = time.time() - start

JAX (compiled via XLA)
@jax.jit
def sum_squares_jax(n):
 return jnp.sum(jnp.arange(n, dtype=jnp.float32) ** 2)

_ = sum_squares_jax(10) # warm up JIT
start = time.time()
result = sum_squares_jax(n)
jax_time = time.time() - start

print(f"Python loop: {python_time:.4f}s")
print(f"JAX (JIT): {jax_time:.6f}s")
print(f"Speedup: {python_time / jax_time:.0f}x")

```

# 66 Foundations: Big O, Recursion, Backtracking, and Dynamic Programming

Before diving into data structures and algorithms, you need four foundational concepts: Big O notation for measuring efficiency, recursion for breaking problems into subproblems, backtracking for exhaustive search with pruning, and dynamic programming for avoiding redundant computation. This file teaches each from first principles.

- The remaining files in this chapter assume you are comfortable with these four ideas. If you skip this file, the  $O(n \log n)$  annotations, recursive tree traversals, backtracking templates, and DP state transitions in later files will feel like magic rather than engineering.

## 66.1 Why Patterns, Not Memorisation

---

- There are thousands of coding problems on LeetCode, NeetCode, and HackerRank. Nobody can memorise them all, and trying to is a losing strategy. Interviewers do not pick problems from a fixed list, they modify, combine, and disguise them. A memorised solution to “Two Sum” will not help you when the interviewer asks a variant you have never seen.
- The good news: there are only about **15-20 core patterns** (two pointers, sliding window, BFS/DFS, DP, backtracking, etc.). Every problem, no matter how novel it looks on the surface, reduces to one or a combination of these patterns. The interview is not testing whether you have seen this exact problem before. It is testing whether you can **peel away the context**, the story, the specific data types, the edge cases, and recognise the underlying pattern.
- Consider these three problems:
  - “Find two numbers in an array that sum to a target.”
  - “Find two molecules whose binding energies sum to a threshold.”
  - “Given a list of account balances, find two accounts whose combined value equals a debt.”
- They look different. They are the same problem: **Two Sum**. The context (numbers, molecules, accounts) is irrelevant. The structure is: search for a complement in a collection → hash map lookup.

- This is why this chapter teaches **patterns through intuition**, not solutions through repetition. For each pattern, we explain:
    - **What structural property** of the problem signals this pattern (sorted input → two pointers; subarray constraint → sliding window; optimal substructure + overlapping subproblems → DP).
    - **Why the pattern works** — the mathematical or logical reasoning, not just “it gives the right answer.”
    - **How to adapt it** — by showing easy, medium, and hard variants where the same core idea applies in different contexts.
  - When you deeply understand why sliding window works (the monotonicity of the constraint means expanding/contracting is sufficient), you can apply it to any problem with that structure, even one you have never seen. When you only memorise the code for “Longest Substring Without Repeating Characters,” you are stuck the moment the problem changes.
  - The practical strategy:
    1. **Learn the pattern** (this chapter).
    2. **Practise recognising it** in disguised problems (the NeetCode take-homes at the end of each file).
    3. **Practise implementing it** under time pressure.
    4. In the interview: read the problem → strip the context → identify the pattern → implement.
- 

## 66.2 Big O Notation

---

- When we say an algorithm is “fast” or “slow,” we need a precise way to measure it. **Big O notation** describes how an algorithm’s runtime (or space usage) grows as the input size  $n$  increases, ignoring constant factors and lower-order terms.
- The formal definition:  $f(n) = O(g(n))$  means there exist constants  $c > 0$  and  $n_0$  such that  $f(n) \leq c \cdot g(n)$  for all  $n \geq n_0$ . In plain English:  $f$  grows no faster than  $g$  for large inputs.
- Why ignore constants? Because a  $2n$  algorithm and a  $5n$  algorithm are both  $O(n)$ : they scale the same way. On a faster computer, the constants change, but

the scaling does not. Big O captures the **intrinsic** difficulty of the problem, independent of hardware.

### 66.2.1 The Growth Rate Hierarchy

- From fastest to slowest:

Big O	Name	Example	$n = 10^6$ operations
$O(1)$	Constant	Array access, hash lookup	1
$O(\log n)$	Logarithmic	Binary search	20
$O(n)$	Linear	Linear scan, single loop	$10^6$
$O(n \log n)$	Linearithmic	Merge sort, efficient sorting	$2 \times 10^7$
$O(n^2)$	Quadratic	Nested loops, brute-force pairs	$10^{12}$ (too slow)
$O(n^3)$	Cubic	Triple nested loops, matrix multiply	$10^{18}$ (way too slow)
$O(2^n)$	Exponential	All subsets, brute-force backtracking	$10^{301030}$ (impossible)
$O(n!)$	Factorial	All permutations	absurd

- **Rule of thumb:** modern computers execute roughly  $10^8$ – $10^9$  simple operations per second. For a 1-second time limit:
  - $O(n)$  works for  $n \leq 10^8$
  - $O(n \log n)$  works for  $n \leq 10^7$
  - $O(n^2)$  works for  $n \leq 10^4$
  - $O(2^n)$  works for  $n \leq 25$

- This table tells you immediately whether your approach is fast enough. If  $n = 10^5$  and your solution is  $O(n^2)$ , it will be  $10^{10}$  operations — too slow. You need a better algorithm.

### 66.2.2 How to Analyse Big O

- **Single loop** over  $n$  elements:  $O(n)$ .

```
total = 0
for x in arr: # n iterations
 total += x # O(1) per iteration
Total: O(n)
```

- **Nested loops**: multiply the iteration counts.

```
for i in range(n): # n iterations
 for j in range(n): # n iterations each
 process(i, j) # O(1)
Total: O(n^2)
```

- **Loop with halving**:  $O(\log n)$ . Each iteration halves the problem size, so it takes  $\log_2 n$  iterations.

```
i = n
while i > 0:
 process(i)
 i //= 2
Total: O(log n)
```

- **Nested loop where inner depends on outer**:

```
for i in range(n):
 for j in range(i): # j goes from 0 to i-1
 process(i, j)
Total: 0 + 1 + 2 + ... + (n-1) = n(n-1)/2 = O(n^2)
```

- **Recursive**: write the recurrence relation and solve it (chapter 13 covered the Master Theorem). For example, merge sort:  $T(n) = 2T(n/2) + O(n) = O(n \log n)$ .

### 66.2.3 Common Traps

- **Hidden loops**:  $x$  in `list` is  $O(n)$  in Python (linear scan), but  $x$  in `set` is  $O(1)$ . Using `in` on a list inside a loop gives  $O(n^2)$ , not  $O(n)$ .

```

BAD: $O(n^2)$ – "in" on a list is $O(n)$
for x in arr:
 if x in another_list:
 process(x)

GOOD: $O(n)$ – convert to set first
another_set = set(another_list)
for x in arr:
 if x in another_set:
 process(x)

```

- **String concatenation:** `s += c` in Python copies the entire string each time. Inside a loop of  $n$  iterations:  $O(1 + 2 + \dots + n) = O(n^2)$ .
- **Sorting dominance:** if your algorithm sorts ( $O(n \log n)$ ) and then does a linear scan ( $O(n)$ ), the total is  $O(n \log n)$  — the sort dominates.
- **Amortised complexity:** some operations are expensive occasionally but cheap on average. Dynamic array append is  $O(1)$  amortised because the rare  $O(n)$  resize is spread across  $n$  cheap appends. Do not confuse amortised  $O(1)$  with worst-case  $O(1)$ .

#### 66.2.4 Space Complexity

- Space complexity follows the same Big O rules, applied to memory usage instead of time.
  - **In-place** algorithms use  $O(1)$  extra space (not counting the input). Quicksort is  $O(\log n)$  space (recursion stack depth). Merge sort is  $O(n)$  (temporary arrays for merging).
  - **Recursion stack:** every recursive call uses stack space. A recursion  $n$  levels deep uses  $O(n)$  space, even if each call does not allocate additional memory. This is why recursive DFS on a graph with  $n$  nodes uses  $O(n)$  space.
  - For interviews, always state both time and space complexity. An  $O(n)$  time,  $O(n)$  space solution is often acceptable, but an  $O(n)$  time,  $O(1)$  space solution is better. The interviewer may ask you to optimise one or the other.
-

## 66.3 Recursion

---

- **Recursion** is when a function calls itself to solve a smaller instance of the same problem. It is the most natural approach for problems with recursive structure: trees, nested structures, divide-and-conquer, and mathematical sequences.
- Every recursive function has two parts:
  1. **Base case:** the smallest instance that can be solved directly (without recursion). This is what stops the recursion.
  2. **Recursive case:** break the problem into smaller subproblems, solve them recursively, and combine the results.

### 66.3.1 Example: Factorial

```
def factorial(n):
 if n <= 1: # base case
 return 1
 return n * factorial(n - 1) # recursive case
```

- How it executes for `factorial(4)`:
  - `factorial(4)` calls `factorial(3)`
  - `factorial(3)` calls `factorial(2)`
  - `factorial(2)` calls `factorial(1)`
  - `factorial(1)` returns 1 (base case)
  - `factorial(2)` returns  $2 * 1 = 2$
  - `factorial(3)` returns  $3 * 2 = 6$
  - `factorial(4)` returns  $4 * 6 = 24$
- Each call is pushed onto the **call stack**. The stack grows until the base case is reached, then unwinds as each call returns. If the recursion is too deep (e.g., `factorial(1000000)` in Python), the stack overflows (`RecursionError`). Python's default recursion limit is 1000.

### 66.3.2 How to Think Recursively

- The key mental shift: **trust the recursion**. When writing a recursive function, assume that the recursive call returns the correct answer for the smaller subproblem. Your job is only to:
  1. Handle the base case.
  2. Break the problem into smaller pieces.
  3. Combine the results.



- You do not need to trace through every recursive call in your head. That is like trying to understand a loop by mentally executing every iteration. Instead, verify: “if the recursive call gives me the right answer for the smaller input, does my combination step give the right answer for the full input?”

### 66.3.3 Example: Recursion on a Linked List

- Reverse a linked list recursively:

```
def reverse(head):
 if not head or not head.next: # base case: 0 or 1 nodes
 return head

 new_head = reverse(head.next) # reverse the rest
 head.next.next = head # point the next node back to me
 head.next = None # I am now the tail
 return new_head
```

- **Trust the recursion:** `reverse(head.next)` correctly reverses the rest of the list and returns the new head. We just need to attach the current node at the end.

### 66.3.4 Example: Recursion on a Tree

- Compute the height of a binary tree:

```
def height(root):
 if not root: # base case: empty tree has height 0
 return 0
 left_h = height(root.left) # height of left subtree
 right_h = height(root.right) # height of right subtree
 return 1 + max(left_h, right_h) # this node adds 1 level
```

- This pattern — “recurse on left, recurse on right, combine” — solves the vast majority of tree problems (see file 03).

### 66.3.5 Recursion vs Iteration

- Every recursive algorithm can be converted to an iterative one (using an explicit stack or loop). Iteration avoids the call stack overhead and the risk of stack overflow.

- **When to prefer recursion:** the problem has natural recursive structure (trees, nested data, divide-and-conquer). The recursive solution is cleaner and easier to reason about.
- **When to prefer iteration:** the recursion depth could be very large (e.g., processing a linked list of  $10^6$  nodes). The iterative solution avoids stack overflow.
- **Tail recursion:** a recursive call is “tail recursive” if it is the last operation in the function (no work is done after the recursive call returns). Some languages (Scheme, Scala) optimise tail calls to use constant stack space. Python does **not** optimise tail calls, so tail recursion in Python still uses  $O(n)$  stack space.

### 66.3.6 Common Pitfalls

Pitfall	Example	Fix
Missing base case	Infinite recursion → stack overflow	Always define when to stop
Wrong base case	Off-by-one in recursive decomposition	Test with the smallest inputs (0, 1, 2)
Not reducing the problem	$f(n)$ calls $f(n)$ instead of $f(n-1)$	Ensure subproblem is strictly smaller
Redundant computation	Fibonacci: $f(n) = f(n-1) + f(n-2)$ recomputes exponentially	Use memoisation (→ DP)
Python recursion limit	<code>factorial(10000)</code> crashes	Use <code>sys.setrecursionlimit</code> or convert to iteration

## 66.4 Backtracking

- **Backtracking** is a systematic way to explore all possible solutions by building them incrementally and abandoning partial solutions that cannot possibly lead to a valid answer.

- Think of it as navigating a maze. At each junction, you pick a path. If you hit a dead end, you go back to the last junction and try a different path. You do not start over from the beginning — you **backtrack** to the most recent decision point.

### 66.4.1 The Three Steps

Every backtracking algorithm follows the same pattern:

1. **Choose**: select a candidate to extend the current partial solution.
2. **Explore**: recursively try to build a complete solution from this candidate.
3. **Unchoose**: undo the choice (backtrack) and try the next candidate.

```
def backtrack(state, choices, result):
 if is_complete(state):
 result.append(state.copy())
 return

 for choice in choices:
 if is_valid(choice, state):
 state.add(choice) # 1. choose
 backtrack(state, choices, result) # 2. explore
 state.remove(choice) # 3. unchoose (backtrack)
```

- The **unchoose** step is what distinguishes backtracking from plain recursion. Without it, the state accumulates all choices and you cannot explore alternative paths.

### 66.4.2 When to Use Backtracking

- The problem asks to **enumerate all valid configurations**: all permutations, all subsets, all valid arrangements (e.g., N-Queens).
- The problem asks to **find any valid configuration**: Sudoku solving, maze path finding.
- The search space is large but can be **pruned**: most partial solutions can be rejected early without exploring them fully.

### 66.4.3 How Pruning Makes It Fast

- Without pruning, backtracking explores every possible combination — exponential time. **Pruning** cuts branches early:

```

for choice in choices:
 if not is_valid(choice, state):
 continue # PRUNE: skip this entire subtree

 state.add(choice)
 backtrack(state, choices, result)
 state.remove(choice)

```

- In N-Queens (file 05), checking column and diagonal conflicts before placing a queen prunes the search tree from  $n^n$  to roughly  $n!$  candidates. For  $n = 8$ , that is 16 million  $\rightarrow$  40,000. Good pruning makes exponential algorithms practical for moderate  $n$ .

#### 66.4.4 Generating All Subsets (The Simplest Backtracking)

```

def subsets(nums):
 result = []

 def backtrack(start, path):
 result.append(path[:]) # every partial solution is a valid subset

 for i in range(start, len(nums)):
 path.append(nums[i]) # choose
 backtrack(i + 1, path) # explore (i+1: no reuse)
 path.pop() # unchoose

 backtrack(0, [])
 return result

```

- For [1, 2, 3], the recursion tree:
  - []  $\rightarrow$  [1]  $\rightarrow$  [1,2]  $\rightarrow$  [1,2,3] (backtrack)  $\rightarrow$  [1,3] (backtrack)  $\rightarrow$  [2]  $\rightarrow$  [2,3] (backtrack)  $\rightarrow$  [3]
- Each node in the tree is one call to `backtrack`. Each leaf (and intermediate node) produces a subset. Total subsets:  $2^n$ .

#### 66.4.5 Generating All Permutations

```

def permutations(nums):
 result = []

 def backtrack(path, remaining):
 if not remaining:
 result.append(path[:])

```

```

 return

 for i in range(len(remaining)):
 path.append(remaining[i]) # choose
 backtrack(path, remaining[:i] + remaining[i+1:]) # explore
 path.pop() # unchoose

 backtrack([], nums)
 return result

```

- Total permutations:  $n!$ . Each requires  $O(n)$  work to construct remaining, so the total is  $O(n \cdot n!)$ .

#### 66.4.6 Common Pitfalls

Pitfall	Example	Fix
Forgetting to copy the path	<code>result.append(path)</code> — all entries share the same list	<code>result.append(path[:])</code> or <code>path.copy()</code>
Not backtracking (unchoosing)	State keeps growing, later candidates see stale state	Always <code>path.pop()</code> or <code>state.remove()</code> after recursive call
Wrong loop start	Subsets with duplicates, or permutations with unwanted reuse	Use start parameter to avoid revisiting earlier indices
Skipping pruning	Exploring obviously invalid branches	Add <code>if not is_valid:</code> <code>continue</code> before the recursive call

### 66.5 Dynamic Programming

- **Dynamic programming (DP)** is an optimisation technique for problems where the same subproblems are solved repeatedly. Instead of recomputing, DP solves each subproblem once and stores the result.
- DP applies when a problem has two properties:
  1. **Optimal substructure:** the optimal solution can be built from optimal solutions to subproblems.

2. **Overlapping subproblems:** the same subproblems appear multiple times in the recursion.

### 66.5.1 The Fibonacci Motivation

- Naive recursive Fibonacci:

```
def fib(n):
 if n <= 1:
 return n
 return fib(n - 1) + fib(n - 2)
```

- For fib(5), the recursion tree:
  - fib(5) calls fib(4) and fib(3)
  - fib(4) calls fib(3) and fib(2)
  - fib(3) is computed **twice**, fib(2) is computed **three times**
- This is  $O(2^n)$  because the tree branches at every level, and most branches recompute the same values. For fib(50), it takes over  $10^{15}$  operations — infeasible.
- With **memoisation** (top-down DP):

```
def fib_memo(n, memo={}):
 if n in memo:
 return memo[n]
 if n <= 1:
 return n
 memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
 return memo[n]
```

- Now fib(3) is computed once, stored, and looked up on subsequent calls. Total:  $O(n)$  time,  $O(n)$  space.
- With **tabulation** (bottom-up DP):

```
def fib_tab(n):
 if n <= 1:
 return n
 dp = [0] * (n + 1)
 dp[1] = 1
 for i in range(2, n + 1):
```

```
dp[i] = dp[i - 1] + dp[i - 2]
return dp[n]
```

- Same  $O(n)$  time, but builds the solution from the bottom up without recursion. Can be further optimised to  $O(1)$  space since each value depends only on the previous two.

### 66.5.2 The DP Recipe

For any DP problem, follow these steps:

1. **Define the state:** what does  $dp[i]$  (or  $dp[i][j]$ ) represent? This is the hardest step. The state must capture enough information to make optimal decisions.
2. **Write the recurrence:** how does  $dp[i]$  relate to smaller subproblems? This is the transition formula.
3. **Identify the base case:** what are the smallest subproblems that can be solved directly?
4. **Determine the iteration order:** which subproblems must be solved before which? Bottom-up: iterate in an order that ensures dependencies are resolved. Top-down: recursion handles this automatically.
5. **Optimise space** (optional): if  $dp[i]$  only depends on the previous row or the previous few entries, you do not need the full table.

### 66.5.3 Example: The Thinking Process

**Problem:** given an array of positive integers, find the maximum sum of non-adjacent elements (House Robber).

**Step 1 — Define the state:**  $dp[i]$  = maximum sum considering elements  $nums[0..i]$ .

**Step 2 — Write the recurrence:** for element  $i$ , we either: - Skip it:  $dp[i] = dp[i-1]$  (best sum without element  $i$ ). - Take it:  $dp[i] = dp[i-2] + nums[i]$  (must skip element  $i-1$ , then add element  $i$ ).

So:  $dp[i] = \max(dp[i-1], dp[i-2] + nums[i])$ .

**Step 3 — Base cases:**  $dp[0] = nums[0]$ ,  $dp[1] = \max(nums[0], nums[1])$ .

**Step 4 — Iteration order:** left to right (each state depends on the two previous states).

**Step 5 — Space optimisation:** only need the last two values.

```
def rob(nums):
 if len(nums) == 1:
 return nums[0]

 prev2, prev1 = nums[0], max(nums[0], nums[1])

 for i in range(2, len(nums)):
 curr = max(prev1, prev2 + nums[i])
 prev2, prev1 = prev1, curr

 return prev1
```

#### 66.5.4 How to Identify DP Problems

- The problem asks for an **optimum** (minimum cost, maximum profit, longest sequence) or a **count** (number of ways).
- The problem has **choices at each step** (take/skip, go left/right, use this coin or not), and the best overall answer depends on the best answers to subproblems.
- Drawing the recursion tree reveals **repeated subproblems**.
- The brute force is exponential, but there are far fewer **distinct states** than recursive calls.

#### 66.5.5 Categories of DP

- **1D DP**: state depends on a single index. Examples: climbing stairs, house robber, maximum subarray.
- **2D DP**: state depends on two indices. Examples: longest common subsequence ( $dp[i][j]$  for first  $i$  chars of string 1 and first  $j$  chars of string 2), edit distance, grid path problems.
- **Interval DP**: state is a range  $dp[i][j]$  representing the subproblem on  $arr[i..j]$ . Examples: matrix chain multiplication, burst balloons.
- **Knapsack DP**: state is an item index and a capacity. Examples: 0/1 knapsack, coin change, subset sum.
- **Bitmask DP**: state includes a bitmask representing which elements have been used. Examples: TSP, assignment problem. The state space is  $O(2^n \cdot n)$ , feasible for  $n \leq 20$ .



### 66.5.6 Top-Down vs Bottom-Up

	Top-Down (Memoisation)	Bottom-Up (Tabulation)
Implementation	Recursive + cache	Iterative + table
Computes	Only subproblems that are actually needed	All subproblems up to the target
Stack overflow risk	Yes (deep recursion)	No
Space optimisation	Harder	Easier (use rolling array)
Ease of coding	Often more natural (write recursion, add cache)	Requires thinking about iteration order

- In interviews, top-down is often faster to code. In production, bottom-up is usually preferred for performance (no recursion overhead, better cache behaviour).

### 66.5.7 Common Pitfalls

Pitfall	Example	Fix
Wrong state definition	<code>dp[i]</code> does not capture enough info to make decisions	Add dimensions (e.g., <code>dp[i][j]</code> instead of <code>dp[i]</code> )
Missing base case	<code>dp[0]</code> is wrong → all subsequent values are wrong	Verify base case by hand
Wrong iteration order	Computing <code>dp[i]</code> before its dependencies	Draw the dependency arrows and iterate accordingly
Not initialising <code>dp</code> correctly	Using 0 when it should be infinity (for min problems)	<code>float('inf')</code> for minimisation,

Pitfall	Example	Fix
		<code>float('-inf')</code> for maximisation
Forgetting to consider “skip” option	Always taking the current element	The recurrence usually has <code>max(take, skip)</code>
Mutable default argument	<code>def f(memo={})</code> shares cache across calls	<code>def</code> <code>f(memo=None):</code> <code>if memo is</code> <code>None: memo = {}</code>
Off-by-one in 2D DP	Indexing <code>text1[i]</code> when <code>dp</code> is 1-indexed	<code>dp</code> has size <code>(m+1) x (n+1)</code> , access <code>text1[i-1]</code>

## 66.6 Putting It All Together

- These four concepts form a progression:
  1. **Big O** tells you whether an approach is fast enough.
  2. **Recursion** breaks problems into subproblems.
  3. **Backtracking** is recursion + choices + undo, for exhaustive search.
  4. **DP** is recursion + caching, for optimisation over overlapping subproblems.
- When you see a new problem:
  - Estimate the input size `n`. What Big O is acceptable?
  - If the brute force is exponential and the problem asks to enumerate/find configurations: **backtracking** (with pruning to make it practical).
  - If the brute force is exponential and the problem asks for an optimum or count, and you see overlapping subproblems: **DP**.
  - If the problem has structure that halves the search space: **binary search** or **divide and conquer**.
  - If the problem is on sequences with a constraint on subarrays: **sliding window** or **two pointers**.
  - If the problem needs fast lookups: **hash map**.

- The remaining files in this chapter apply these ideas to specific data structures and patterns.

## 67 Arrays and Hashing

Arrays and hash tables are the two most fundamental data structures in programming. This file covers how they work under the hood, then builds up the key problem-solving patterns; two pointers, sliding window, prefix sums, and hash-based lookups, through progressively harder problems, with common pitfalls at each step.

- If you understand arrays and hash maps deeply, you can solve ~40% of all coding interview problems. These two structures appear everywhere because they offer the two things algorithms need most: **fast indexed access** (arrays) and **fast lookup by key** (hash maps).
- This file teaches patterns, not solutions. The goal is that when you see a new problem, you recognise which pattern applies and why, rather than trying to recall a memorised solution.

### 67.1 Arrays

---

- An **array** is a contiguous block of memory where elements are stored at fixed offsets. Accessing element  $i$  costs  $O(1)$  because the address is simply  $\text{base} + i * \text{element\_size}$ . This is the fastest possible data access, and it is why arrays are the default choice.
- **Dynamic arrays** (Python's `list`, Java's `ArrayList`, C++'s `vector`) grow automatically when full. The strategy is **amortised doubling**: when the array is full, allocate a new array of double the size and copy everything over. The copy costs  $O(n)$ , but it happens so rarely (every  $n$  insertions) that the amortised cost per insertion is  $O(1)$ .
- **Cache locality** is why arrays are fast in practice, not just in theory. Because elements are stored contiguously, accessing one element loads nearby elements into the CPU cache (chapter 13). Iterating through an array is cache-friendly; following pointers in a linked list is not. This constant-factor difference can be 10-100x in practice.

Operation	Array	Dynamic Array
Access by index	$O(1)$	$O(1)$
Append	n/a	$O(1)$ amortised
Insert at position i	$O(n)$	$O(n)$
Delete at position i	$O(n)$	$O(n)$
Search (unsorted)	$O(n)$	$O(n)$

- **Pitfall:** inserting or deleting in the middle of an array is  $O(n)$  because all subsequent elements must be shifted. If you need frequent middle insertions, consider a linked list or a different approach entirely.

## 67.2 Strings

- A **string** is an array of characters. In Python, strings are immutable: every concatenation creates a new string. Building a string character by character in a loop is  $O(n^2)$  because each concatenation copies the entire string so far.

```
BAD: $O(n^2)$ string concatenation
s = ""
for c in characters:
 s += c # copies entire string each time

GOOD: $O(n)$ using a list then join
parts = []
for c in characters:
 parts.append(c)
s = "".join(parts)
```

- **Pitfall:** in Python, `s += c` inside a loop is one of the most common performance bugs. Always collect into a list and `.join()`.
- **Encoding:** ASCII uses 7 bits (128 characters). **UTF-8** is variable-length: ASCII characters use 1 byte, accented characters use 2, Chinese/Japanese characters use 3, emojis use 4. When a problem says “lowercase English letters,” the alphabet size is 26, which means you can use a fixed-size array instead of a hash map.

## 67.3 Hash Tables

---

- A **hash table** maps keys to values with  $O(1)$  average-case lookup, insertion, and deletion. It works by computing a **hash function**  $h(\text{key})$  that converts the key to an array index.
- The hash function must be: **deterministic** (same key always gives same hash), **uniform** (distributes keys evenly across buckets), and **fast** to compute.
- **Collisions** occur when two different keys hash to the same index. Two main strategies:
  - **Chaining**: each bucket stores a linked list of key-value pairs. On collision, append to the list. Worst case (all keys hash to the same bucket):  $O(n)$ . Average case with a good hash function:  $O(1)$ .
  - **Open addressing**: on collision, probe for the next empty slot. **Linear probing** checks the next slot, then the next, etc. It is cache-friendly but suffers from **clustering** (long runs of occupied slots). **Robin Hood hashing** reduces the variance by displacing entries that are “closer to home.”
- The **load factor**  $\alpha = n/m$  (items / buckets) determines performance. When  $\alpha$  exceeds a threshold (typically 0.75), the table **rehashes**: allocate a larger table and re-insert all elements. This costs  $O(n)$  but happens infrequently.
- **Hash maps** (dict in Python, HashMap in Java) store key-value pairs. **Hash sets** (set in Python, HashSet in Java) store only keys (used for fast membership testing).

Operation	Average	Worst Case
Lookup	$O(1)$	$O(n)$
Insert	$O(1)$	$O(n)$
Delete	$O(1)$	$O(n)$

- **Bloom filters** are space-efficient probabilistic sets. They can tell you “definitely not in the set” or “probably in the set” (with a tunable false positive rate). They use  $k$  hash functions and a bit array. Used in databases (avoid disk reads for absent keys), web caches, and spell checkers.

- **When to reach for a hash map:** whenever you need to answer “have I seen this before?” or “what is the count/index/value associated with this key?” in  $O(1)$ . If you are doing repeated linear scans looking for something, a hash map almost certainly makes it faster.
- 

## 67.4 Pattern: Hash Map Lookups

---

- The most basic pattern: use a hash map to replace  $O(n)$  scans with  $O(1)$  lookups.

### 67.4.1 Easy: Two Sum

- **Problem:** given an array of integers and a target, return indices of two numbers that add up to the target.
- **Brute force**  $O(n^2)$ : check every pair.
- **Pattern insight:** for each number `num`, you need `target - num` to exist somewhere in the array. Instead of scanning the array for it, store previously seen numbers in a hash map.

```
def two_sum(nums, target):
 seen = {} # value -> index
 for i, num in enumerate(nums):
 complement = target - num
 if complement in seen:
 return [seen[complement], i]
 seen[num] = i
```

- **Why this works:** one pass through the array. For each element, the hash map lookup is  $O(1)$ . Total:  $O(n)$  time,  $O(n)$  space.
- **Pitfall:** do not add the current number to the hash map before checking for the complement, or you might match an element with itself. The order in the code above is correct: check first, then insert.

### 67.4.2 Medium: Group Anagrams

- **Problem:** given a list of strings, group the anagrams together. ( “eat” , “tea” , “ate” ) are one group.

- **Pattern insight:** anagrams have the same characters in different orders. If you sort each string, anagrams produce the same sorted key. Use that sorted key as the hash map key.

```
from collections import defaultdict

def group_anagrams(strs):
 groups = defaultdict(list)
 for s in strs:
 key = tuple(sorted(s)) # or use character count tuple
 groups[key].append(s)
 return list(groups.values())
```

- **Optimisation:** sorting each string costs  $O(k \log k)$  where  $k$  is the string length. For a faster key, count character frequencies and use the count tuple as the key:

```
def group_anagrams_fast(strs):
 groups = defaultdict(list)
 for s in strs:
 count = [0] * 26
 for c in s:
 count[ord(c) - ord('a')] += 1
 groups[tuple(count)].append(s)
 return list(groups.values())
```

- This is  $O(k)$  per string instead of  $O(k \log k)$ . The character count tuple is a **canonical form**: a representation that is the same for all members of a group.
- **Pitfall:** in Python, lists are not hashable (cannot be dict keys). You must convert to a tuple. This trips people up when they try `groups[count].append(s)`.

### 67.4.3 Hard: Longest Consecutive Sequence

- **Problem:** given an unsorted array, find the length of the longest consecutive sequence (e.g., `[100, 4, 200, 1, 3, 2]`  $\rightarrow$  4, because `[1, 2, 3, 4]`).
- **Brute force**  $O(n \log n)$ : sort the array, then scan for consecutive runs.
- **Pattern insight:** put all numbers in a hash set for  $O(1)$  lookup. For each number, check if it is the **start** of a sequence (i.e., `num - 1` is not in the set). If so, count how far the sequence extends.

```
def longest_consecutive(nums):
 num_set = set(nums)
```

```

best = 0

for num in num_set:
 # only start counting from the beginning of a sequence
 if num - 1 not in num_set:
 length = 1
 while num + length in num_set:
 length += 1
 best = max(best, length)

return best

```

- **Why  $O(n)$ :** the inner while loop runs at most  $n$  times total across all iterations (each number is visited at most twice: once in the outer loop, once in a while extension). The `if num - 1 not in num_set` guard ensures we only start counting from sequence beginnings.
- **Pitfall:** without the `if num - 1 not in num_set` check, you would start counting from every element, making it  $O(n^2)$  in the worst case (e.g., `[1, 2, 3, ..., n]` would scan the entire sequence from each starting point).

## 67.5 Pattern: Two Pointers

- The **two pointers** pattern uses two indices that move through the array, usually from opposite ends or from the same end at different speeds. It works when the array is sorted or when you need to compare pairs.
- **When to use:** the problem involves pairs, subarrays, or partitioning, and the array is sorted (or can be sorted without losing needed information).

### 67.5.1 Easy: Valid Palindrome

- **Problem:** determine if a string is a palindrome, considering only alphanumeric characters and ignoring case.
- **Pattern:** one pointer at the start, one at the end. Move them inward, comparing characters.

```

def is_palindrome(s):
 left, right = 0, len(s) - 1

 while left < right:

```



```

 # skip non-alphanumeric characters
 while left < right and not s[left].isalnum():
 left += 1
 while left < right and not s[right].isalnum():
 right -= 1

 if s[left].lower() != s[right].lower():
 return False

 left += 1
 right -= 1

 return True

```

- **Pitfall:** forgetting the `left < right` check inside the inner while loops. Without it, the pointer can go out of bounds on strings like “!!!” (all non-alphanumeric).

### 67.5.2 Medium: Three Sum

- **Problem:** find all unique triplets in the array that sum to zero.
- **Pattern:** sort the array. Fix one element, then use two pointers on the remaining portion to find pairs that sum to the negation of the fixed element.

```

def three_sum(nums):
 nums.sort()
 result = []

 for i in range(len(nums) - 2):
 # skip duplicate fixed elements
 if i > 0 and nums[i] == nums[i - 1]:
 continue

 left, right = i + 1, len(nums) - 1
 target = -nums[i]

 while left < right:
 total = nums[left] + nums[right]
 if total < target:
 left += 1
 elif total > target:
 right -= 1
 else:
 result.append([nums[i], nums[left], nums[right]])
 # skip duplicates

```

```

 while left < right and nums[left] == nums[left + 1]:
 left += 1
 while left < right and nums[right] == nums[right - 1]:
 right -= 1
 left += 1
 right -= 1

 return result

```

- **Why this works:** sorting is  $O(n \log n)$ . For each fixed element, the two-pointer scan is  $O(n)$ . Total:  $O(n^2)$ , which is optimal for this problem (you must consider all pairs).
- **Pitfall:** handling duplicates is the hardest part. Without the duplicate-skipping logic (both for the fixed element and for the two-pointer results), you will return duplicate triplets. The `if i > 0 and nums[i] == nums[i-1]: continue` line is critical.

### 67.5.3 Hard: Trapping Rain Water

- **Problem:** given an elevation map (array of non-negative integers), compute how much water it can trap after rain.
- **Pattern insight:** for each position, the water level is determined by the minimum of the maximum height to its left and the maximum height to its right, minus the current height. Two pointers from both ends track these running maxima.

```

def trap(height):
 left, right = 0, len(height) - 1
 left_max, right_max = 0, 0
 water = 0

 while left < right:
 if height[left] < height[right]:
 if height[left] >= left_max:
 left_max = height[left]
 else:
 water += left_max - height[left]
 left += 1
 else:
 if height[right] >= right_max:
 right_max = height[right]
 else:
 water += right_max - height[right]
 right -= 1

 return water

```

```
 water += right_max - height[right]
 right -= 1

 return water
```

- **Why this works:** the key insight is that if `height[left] < height[right]`, the water at position `left` is bounded by `left_max` (we know there is a taller bar on the right side, so the right side cannot be the bottleneck). We process the shorter side, guaranteed that the other side has a taller bar.
  - **Pitfall:** many people try to precompute `left_max[i]` and `right_max[i]` arrays first (which works but uses  $O(n)$  space). The two-pointer approach achieves  $O(1)$  space. Also, confusing `>=` with `>` in the max updates can cause off-by-one water calculations.
- 

## 67.6 Pattern: Sliding Window

---

- The **sliding window** pattern maintains a window (contiguous subarray) that expands and contracts as you iterate. It works for problems asking about subarrays or substrings that satisfy some condition.
- **When to use:** the problem asks for the longest/shortest subarray or substring satisfying a constraint, and expanding/contracting the window is monotonic (adding elements can only make the constraint harder/easier to satisfy, not both).
- **Template:**

```
def sliding_window(arr):
 left = 0
 state = ... # window state (counts, sum, etc.)
 best = ...

 for right in range(len(arr)):
 # expand: add arr[right] to the window state
 update_state(state, arr[right])

 # contract: shrink from the left while constraint is violated
 while constraint_violated(state):
 remove_from_state(state, arr[left])
 left += 1
```

```

 # update answer
 best = max(best, right - left + 1) # or min, depending on problem

 return best

```

### 67.6.1 Easy: Best Time to Buy and Sell Stock

- **Problem:** given daily prices, find the maximum profit from one buy and one sell (buy before sell).
- **Pattern:** track the minimum price seen so far (the left boundary of the window) and compute the profit at each day.

```

def max_profit(prices):
 min_price = float('inf')
 max_profit = 0

 for price in prices:
 min_price = min(min_price, price)
 max_profit = max(max_profit, price - min_price)

 return max_profit

```

- This is a degenerate sliding window: the left pointer (min price) only moves forward when a new minimum is found.  $O(n)$  time,  $O(1)$  space.

### 67.6.2 Medium: Longest Substring Without Repeating Characters

- **Problem:** find the length of the longest substring without any repeating characters.
- **Pattern:** expand the window by moving right. When a duplicate is found, contract from the left until the duplicate is removed.

```

def length_of_longest_substring(s):
 char_index = {} # character -> its most recent index
 left = 0
 best = 0

 for right, char in enumerate(s):
 if char in char_index and char_index[char] >= left:
 left = char_index[char] + 1 # jump past the duplicate

 char_index[char] = right
 best = max(best, right - left + 1)

```

```
return best
```

- **Why `char_index[char] >= left`:** the character might be in the map from before the current window started. Without this check, you would incorrectly shrink the window for a character that is not actually in the current window.
- **Pitfall:** using a set and removing characters one by one from the left is correct but slower. The hash map approach jumps directly to the right position.

### 67.6.3 Hard: Minimum Window Substring

- **Problem:** given strings `s` and `t`, find the minimum window in `s` that contains all characters of `t`.
- **Pattern:** expand the window to include all required characters, then contract from the left to find the minimum valid window.

```
from collections import Counter

def min_window(s, t):
 if not t or not s:
 return ""

 need = Counter(t) # characters we need and their counts
 have = 0 # how many unique characters we have in sufficient
 quantity
 required = len(need) # how many unique characters we need

 left = 0
 best = (float('inf'), 0, 0) # (length, left, right)

 window_counts = {}

 for right in range(len(s)):
 char = s[right]
 window_counts[char] = window_counts.get(char, 0) + 1

 # check if this character's count now meets the requirement
 if char in need and window_counts[char] == need[char]:
 have += 1

 # contract from the left while the window is valid
 while have == required:
 # update best
```

```

 if (right - left + 1) < best[0]:
 best = (right - left + 1, left, right)

 # remove leftmost character
 left_char = s[left]
 window_counts[left_char] -= 1
 if left_char in need and window_counts[left_char] < need[left_char]:
 have -= 1
 left += 1

 length, start, end = best
 return s[start:end + 1] if length != float('inf') else ""

```

- **Pitfall:** the have counter is the crucial optimisation. Without it, you would need to compare the entire `window_counts` dict with `need` at every step, which is  $O(|\text{unique chars}|)$  per step. The have counter makes the validity check  $O(1)$ .
- **Pitfall:** checking `window_counts[char] == need[char]` (not `>=`) ensures we increment have exactly once per character. If we used `>=`, we would over-count.

## 67.7 Pattern: Prefix Sums

- A **prefix sum** array stores cumulative sums: `prefix[i] = sum(arr[0:i])`. Once built in  $O(n)$ , any subarray sum can be computed in  $O(1)$ : `sum(arr[l:r]) = prefix[r] - prefix[l]`.

```

def build_prefix(arr):
 prefix = [0] * (len(arr) + 1)
 for i in range(len(arr)):
 prefix[i + 1] = prefix[i] + arr[i]
 return prefix

sum of arr[l:r] (inclusive l, exclusive r)
def range_sum(prefix, l, r):
 return prefix[r] - prefix[l]

```

- **When to use:** the problem involves multiple subarray sum queries, or finding subarrays with a specific sum.

### 67.7.1 Easy: Range Sum Query

- **Problem:** given an array, answer multiple queries of “what is the sum from index `l` to `r`?”

- Without prefix sums: each query is  $O(n)$ . With prefix sums:  $O(n)$  precomputation, then  $O(1)$  per query.

### 67.7.2 Medium: Subarray Sum Equals K

- **Problem:** count the number of contiguous subarrays that sum to  $k$ .
- **Pattern insight:** a subarray sum from index  $l$  to  $r$  equals  $\text{prefix}[r+1] - \text{prefix}[l]$ . We want this to equal  $k$ , so  $\text{prefix}[l] = \text{prefix}[r+1] - k$ . For each position, count how many earlier prefix sums equal  $\text{current\_prefix} - k$  using a hash map.

```
def subarray_sum(nums, k):
 count = 0
 prefix = 0
 prefix_counts = {0: 1} # empty prefix sum

 for num in nums:
 prefix += num
 # how many earlier prefix sums equal prefix - k?
 count += prefix_counts.get(prefix - k, 0)
 prefix_counts[prefix] = prefix_counts.get(prefix, 0) + 1

 return count
```

- This combines prefix sums with hash map lookups:  $O(n)$  time,  $O(n)$  space.
- **Pitfall:** forgetting to initialise  $\text{prefix\_counts} = \{0: 1\}$ . The empty prefix (before any element) has sum 0. Without this, you miss subarrays starting from index 0.

### 67.7.3 Hard: Product of Array Except Self

- **Problem:** given an array, return an array where each element is the product of all other elements. You cannot use division.
- **Pattern:** build prefix products from the left and suffix products from the right. Each position's answer is  $\text{left\_product} * \text{right\_product}$ .

```
def product_except_self(nums):
 n = len(nums)
 result = [1] * n

 # left pass: result[i] = product of nums[0..i-1]
 prefix = 1
```

```

for i in range(n):
 result[i] = prefix
 prefix *= nums[i]

right pass: multiply by product of nums[i+1..n-1]
suffix = 1
for i in range(n - 1, -1, -1):
 result[i] *= suffix
 suffix *= nums[i]

return result

```

- $O(n)$  time,  $O(1)$  extra space (the output array does not count). This uses the output array itself to store intermediate prefix products, then multiplies in the suffix products in the second pass.
- **Pitfall:** if the array contains a zero, division-based approaches fail. This prefix/suffix approach handles zeros correctly because it never divides.

---

## 67.8 Common Pitfalls Summary

Pitfall	Example	Fix
Off-by-one in window size	<code>right - left</code> vs <code>right - left + 1</code>	Draw out a 2-element example
Mutable default in Python	<code>def f(seen={})</code> shares state across calls	Use <code>def f(seen=None)</code>
String concatenation in loop	<code>s += c</code> is $O(n^2)$ in Python	Use <code>list.append</code> + <code>"".join</code>
Forgetting <code>{0: 1}</code> in prefix sum	Miss subarrays starting at index 0	Always init with empty prefix
Hash map before check	Two Sum: adding <code>num</code> before checking complement	Check first, then insert
Not handling duplicates	Three Sum returns duplicate triplets	



Pitfall	Example	Fix
		Skip consecutive equal values
Integer overflow	Sum of large arrays in C++/Java	Use long or check bounds

## 67.9 Take-Home Problems (NeetCode)

Practice these in order. Each reinforces a pattern from this file.

### 67.9.1 Hash Map Lookups

- [Contains Duplicate](#) — warm-up: hash set for seen-before
- [Two Sum](#) — complement lookup
- [Group Anagrams](#) — canonical form as key
- [Top K Frequent Elements](#) — hash map + bucket sort
- [Longest Consecutive Sequence](#) — hash set with start-of-sequence trick
- [Encode and Decode Strings](#) — design a serialisation scheme

### 67.9.2 Two Pointers

- [Valid Palindrome](#) — inward pointers
- [Two Sum II \(sorted\)](#) — two pointers on sorted array
- [Three Sum](#) — fix + two pointers + dedup
- [Container With Most Water](#) — greedy two pointers
- [Trapping Rain Water](#) — two pointers with running max

### 67.9.3 Sliding Window

- [Best Time to Buy and Sell Stock](#) — degenerate window
- [Longest Substring Without Repeating Characters](#) — expand/contract with hash map
- [Longest Repeating Character Replacement](#) — window + max frequency trick
- [Minimum Window Substring](#) — expand until valid, contract to minimise

### 67.9.4 Prefix Sums

- [Product of Array Except Self](#) — prefix/suffix products

# 68 Linked Lists, Stacks, and Queues

Linked lists, stacks, and queues are the building blocks of more complex data structures. This file covers their mechanics, then builds up the key patterns; fast/slow pointers, monotonic stacks, and heap-based priority queues, through progressively harder problems, with common pitfalls at each step.

- Arrays give you fast random access but expensive insertions. **Linked lists** give you fast insertions but no random access. **Stacks** and **queues** restrict access to one or two ends, and this restriction is what makes them powerful: by limiting what you can do, they simplify what you need to think about.

## 68.1 Linked Lists

- A **singly linked list** is a chain of nodes. Each node stores a value and a pointer to the next node. The last node points to `null`.

```
class ListNode:
 def __init__(self, val=0, next=None):
 self.val = val
 self.next = next
```

- **Advantages over arrays:** inserting or deleting at a known position is  $O(1)$  (just repoint the pointers). No need to shift elements.
- **Disadvantages:** accessing element  $i$  requires  $O(i)$  traversal (no random access). Poor cache locality (nodes are scattered in memory).
- **Doubly linked lists** add a `prev` pointer, enabling backward traversal. Used in LRU caches (constant-time removal of any node) and browser history (back/forward).

Operation	Singly	Doubly
Access by index	$O(n)$	$O(n)$
Insert at head	$O(1)$	$O(1)$
Insert at tail	$O(n)$ or $O(1)^*$	$O(1)$
Delete given node	$O(n)^{**}$	$O(1)$

Operation	Singly	Doubly
Search	O(n)	O(n)

\*with tail pointer. \*\*need predecessor, which requires traversal.

- **Sentinel nodes** (dummy head/tail) simplify edge cases. Without a dummy head, inserting at the head or deleting the head requires special-case code. With a dummy, every real node has a predecessor.

```
Without dummy: special case for head deletion
def delete_head(head):
 if not head:
 return None
 return head.next

With dummy: uniform logic
dummy = ListNode(0)
dummy.next = head
now every deletion is: prev.next = prev.next.next
```

- **Pitfall:** forgetting to handle the empty list (`head is None`) or the single-element list. Always test these edge cases.

---

## 68.2 Pattern: Fast and Slow Pointers (Floyd's)

---

- Use two pointers moving at different speeds to detect properties of linked lists. The **slow** pointer moves one step at a time; the **fast** pointer moves two steps.

### 68.2.1 Easy: Linked List Cycle

- **Problem:** determine if a linked list has a cycle.
- **Pattern:** if there is a cycle, the fast pointer will eventually lap the slow pointer (they will meet). If there is no cycle, the fast pointer reaches `null`.

```
def has_cycle(head):
 slow = fast = head
 while fast and fast.next:
 slow = slow.next
 fast = fast.next.next
 if slow == fast:
```

```
 return True
 return False
```

- **Why it works:** if the cycle has length  $c$ , the fast pointer closes the gap by 1 node per step. They must meet within  $c$  steps of the slow pointer entering the cycle.
- **Pitfall:** checking `fast` and `fast.next` (not just `fast.next`). If `fast` is `None`, calling `fast.next` crashes.

### 68.2.2 Medium: Find the Middle of a Linked List

- **Problem:** return the middle node.
- **Pattern:** when the fast pointer reaches the end, the slow pointer is at the middle.

```
def find_middle(head):
 slow = fast = head
 while fast and fast.next:
 slow = slow.next
 fast = fast.next.next
 return slow # slow is at the middle (or second middle if even length)
```

### 68.2.3 Medium: Linked List Cycle II (Find Cycle Start)

- **Problem:** return the node where the cycle begins.
- **Pattern:** after fast and slow meet, reset one pointer to the head. Move both at speed 1. They meet at the cycle start.

```
def detect_cycle(head):
 slow = fast = head
 while fast and fast.next:
 slow = slow.next
 fast = fast.next.next
 if slow == fast:
 # reset one pointer to head
 slow = head
 while slow != fast:
 slow = slow.next
 fast = fast.next
```

```
 return slow
 return None
```

- **Why this works:** let the distance from head to cycle start be  $a$ , and the distance from cycle start to meeting point be  $b$ . The slow pointer travelled  $a + b$ . The fast pointer travelled  $2(a + b)$ . The difference is one full cycle:  $a + b = c$  (cycle length). So  $a = c - b$ : the distance from head to cycle start equals the distance from meeting point to cycle start (going forward around the cycle).

#### 68.2.4 Hard: Reverse Linked List in Groups of K

- **Problem:** reverse every  $k$  consecutive nodes in a linked list.

```
def reverse_k_group(head, k):
 # check if we have k nodes left
 node = head
 for _ in range(k):
 if not node:
 return head
 node = node.next

 # reverse k nodes
 prev, curr = None, head
 for _ in range(k):
 nxt = curr.next
 curr.next = prev
 prev = curr
 curr = nxt

 # head is now the tail of the reversed group
 # recursively process the rest
 head.next = reverse_k_group(curr, k)
 return prev # prev is the new head of this group
```

- **Pitfall:** the in-place reversal pattern ( $prev$ ,  $curr$ ,  $nxt$ ) is worth memorising. Draw it out: at each step, you point  $curr.next$  backward to  $prev$ , then advance all three pointers. Getting the order wrong corrupts the list.

---

### 68.3 Stacks

- A **stack** is LIFO (Last In, First Out): the most recently added element is removed first. Think of a stack of plates.

- Operations: `push(x)` adds to the top, `pop()` removes from the top, `peek()` looks at the top without removing. All  $O(1)$ .
- Stacks are the implicit structure behind **recursion** (the call stack), **expression evaluation** (converting infix to postfix), and **undo operations** (each action is pushed, undo pops the last one).

### 68.3.1 Easy: Valid Parentheses

- **Problem:** given a string of brackets `()[]{}` , determine if they are balanced.
- **Pattern:** push opening brackets onto the stack. When you see a closing bracket, check that the top of the stack is the matching opener.

```
def is_valid(s):
 stack = []
 matching = {'(': ')', '[': ']', '{': '}'

 for char in s:
 if char in matching:
 if not stack or stack[-1] != matching[char]:
 return False
 stack.pop()
 else:
 stack.append(char)

 return len(stack) == 0
```

- **Pitfall:** forgetting `len(stack) == 0` at the end. The string `"((("` has no mismatches but is not valid because unclosed brackets remain.

---

## 68.4 Pattern: Monotonic Stack

---

- A **monotonic stack** maintains elements in sorted order (either increasing or decreasing). When a new element would violate the order, you pop elements until the order is restored.
- **When to use:** problems asking “for each element, find the next/previous greater/smaller element.” The stack gives  $O(n)$  total because each element is pushed and popped at most once.

### 68.4.1 Medium: Daily Temperatures

- **Problem:** given daily temperatures, for each day find how many days until a warmer temperature.
- **Pattern:** use a stack of indices. When the current temperature is higher than the stack's top, pop and record the distance.

```
def daily_temperatures(temperatures):
 n = len(temperatures)
 result = [0] * n
 stack = [] # stack of indices, temperatures in decreasing order

 for i in range(n):
 while stack and temperatures[i] > temperatures[stack[-1]]:
 prev = stack.pop()
 result[prev] = i - prev
 stack.append(i)

 return result
```

- Each element is pushed once and popped at most once:  $O(n)$  total.
- **Pitfall:** storing indices on the stack (not values). You need the index to compute the distance.

### 68.4.2 Hard: Largest Rectangle in Histogram

- **Problem:** given an array of bar heights, find the area of the largest rectangle.
- **Pattern:** for each bar, find how far left and right it can extend (i.e., the nearest shorter bar on each side). A monotonic increasing stack tracks this efficiently.

```
def largest_rectangle(heights):
 stack = [] # stack of indices, heights in increasing order
 max_area = 0
 heights.append(0) # sentinel to flush the stack at the end

 for i, h in enumerate(heights):
 start = i
 while stack and stack[-1][1] > h:
 idx, height = stack.pop()
 max_area = max(max_area, height * (i - idx))
 start = idx # this bar can extend back to where the popped bar
 started
```

```
stack.append((start, h))

heights.pop() # remove sentinel
return max_area
```

- **Pitfall:** the `start = idx` line is subtle. When we pop a bar that is taller than the current bar, the current bar can extend backward to where the popped bar started (because all bars in between were at least as tall as the popped bar). Missing this line gives incorrect areas.
  - **Pitfall:** the sentinel `heights.append(0)` ensures all remaining bars in the stack are processed. Without it, bars that never encounter a shorter bar to their right are missed.
- 

## 68.5 Queues

---

- A **queue** is FIFO (First In, First Out): elements are added at the back and removed from the front. Think of a line at a shop.
- A **deque** (double-ended queue) supports  $O(1)$  insertion and removal at both ends. Python's `collections.deque` is the standard implementation.
- Queues are the structure behind **BFS** (breadth-first search, chapter 14 file 4), **task scheduling**, and **message passing**.

### 68.5.1 Easy: Implement Queue Using Stacks

- **Problem:** implement a queue using only two stacks.
- **Pattern:** use one stack for pushing and one for popping. When the pop stack is empty, transfer all elements from the push stack (reversing the order).

```
class MyQueue:
 def __init__(self):
 self.push_stack = []
 self.pop_stack = []

 def push(self, x):
 self.push_stack.append(x)

 def pop(self):
 if not self.pop_stack:
 while self.push_stack:
```



```

 self.pop_stack.append(self.push_stack.pop())
 return self.pop_stack.pop()

def peek(self):
 if not self.pop_stack:
 while self.push_stack:
 self.pop_stack.append(self.push_stack.pop())
 return self.pop_stack[-1]

def empty(self):
 return not self.push_stack and not self.pop_stack

```

- Amortised  $O(1)$  per operation: each element is moved between stacks at most once.

---

## 68.6 Priority Queues and Heaps

---

- A **priority queue** returns the smallest (or largest) element first, regardless of insertion order. The standard implementation is a **binary heap**.
- A **min-heap** is a complete binary tree where every parent is smaller than its children. The minimum is always at the root. Stored as an array: the children of node  $i$  are at positions  $2i + 1$  and  $2i + 2$ .

Operation	Time
Insert	$O(\log n)$
Get min	$O(1)$
Extract min	$O(\log n)$
Build heap from array	$O(n)$

- Python's `heapq` module provides a min-heap. For a max-heap, negate the values.

```

import heapq

Min-heap
h = []
heapq.heappush(h, 5)
heapq.heappush(h, 2)
heapq.heappush(h, 8)

```

```
print(heapq.heappop(h)) # 2 (smallest)

Max-heap trick: negate values
heapq.heappush(h, -10)
print(-heapq.heappop(h)) # 10 (largest)
```

### 68.6.1 Medium: Kth Largest Element in an Array

- **Problem:** find the kth largest element.
- **Pattern:** maintain a min-heap of size k. The heap's root is the kth largest. If the heap has k elements and a new element is larger than the root, replace the root.

```
import heapq

def find_kth_largest(nums, k):
 heap = nums[:k]
 heapq.heapify(heap) # O(k)

 for num in nums[k:]:
 if num > heap[0]:
 heapq.heapreplace(heap, num) # pop min, push num: O(log k)

 return heap[0]
```

- $O(n \log k)$  time,  $O(k)$  space. Much better than sorting ( $O(n \log n)$ ) when  $k \ll n$ .
- **Pitfall:** using a max-heap of size n and popping k times also works but is slower:  $O(n + k \log n)$ . The min-heap of size k is the optimal approach.

### 68.6.2 Hard: Merge K Sorted Lists

- **Problem:** merge k sorted linked lists into one sorted list.
- **Pattern:** use a min-heap containing the head of each list. Pop the smallest, add it to the result, and push its next node onto the heap.

```
import heapq

def merge_k_lists(lists):
 heap = []
 for i, lst in enumerate(lists):
 if lst:
 heapq.heappush(heap, (lst.val, i, lst))
```

```

dummy = ListNode(0)
curr = dummy

while heap:
 val, i, node = heapq.heappop(heap)
 curr.next = node
 curr = curr.next
 if node.next:
 heapq.heappush(heap, (node.next.val, i, node.next))

return dummy.next

```

- $O(n \log k)$  where  $n$  is the total number of nodes. The heap always has at most  $k$  elements.
- **Pitfall:** the  $i$  (index) in the heap tuple is a tiebreaker. Without it, Python tries to compare `ListNode` objects when values are equal, which crashes because `ListNode` does not support `<`. The index ensures a valid comparison.

## 68.7 Common Pitfalls Summary

Pitfall	Example	Fix
Null pointer on <code>fast.next</code>	Cycle detection with <code>while fast.next</code>	Check <code>fast</code> and <code>fast.next</code>
Not handling empty list	Reverse of <code>None</code>	Add <code>if not head</code> guard
Stack underflow	Popping from empty stack	Check <code>len(stack) &gt; 0</code> or <code>if stack</code>
Forgetting sentinel	Histogram misses last bars	Append 0 to flush the stack
Missing tiebreaker in heap	Comparing uncomparable objects	Add index to heap tuple
Modifying list during iteration	Removing nodes while traversing	

Pitfall	Example	Fix
		Use prev/curr pattern or dummy head

## 68.8 Take-Home Problems (NeetCode)

### 68.8.1 Linked Lists

- [Reverse Linked List](#) — the fundamental in-place reversal
- [Merge Two Sorted Lists](#) — two-pointer merge
- [Linked List Cycle](#) — fast/slow pointer
- [Reorder List](#) — find middle + reverse + merge
- [Remove Nth Node From End](#) — two pointers with gap n
- [LRU Cache](#) — hash map + doubly linked list

### 68.8.2 Stacks

- [Valid Parentheses](#) — matching brackets
- [Min Stack](#) — track min at each level
- [Evaluate Reverse Polish Notation](#) — stack-based evaluation
- [Daily Temperatures](#) — monotonic decreasing stack
- [Largest Rectangle in Histogram](#) — monotonic increasing stack
- [Car Fleet](#) — stack with time-to-target

### 68.8.3 Heaps / Priority Queues

- [Kth Largest Element in a Stream](#) — min-heap of size k
- [Last Stone Weight](#) — max-heap simulation
- [K Closest Points to Origin](#) — min-heap by distance
- [Task Scheduler](#) — greedy with max-heap + cooldown
- [Find Median from Data Stream](#) — two heaps (max-heap for lower half, min-heap for upper half)

# 69 Trees

Trees are the hierarchical data structure behind file systems, databases, compilers, and countless interview problems. This file covers binary trees, BSTs, balanced trees, tries, segment trees, Fenwick trees, and Union-Find, with traversal patterns, recursive thinking, and progressively harder problems.

- A **tree** is a connected, acyclic graph (chapter 13). The most important variant is the **binary tree**: each node has at most two children (left and right). Trees appear everywhere: parse trees in compilers, DOM trees in browsers, decision trees in ML, and B-trees in databases.
- The key insight for tree problems: **most tree problems are solved recursively**. The structure is recursive (a tree is a root with two subtrees), so the solutions should be too. Master the pattern of “solve for left subtree, solve for right subtree, combine” and you can solve most tree problems.

## 69.1 Binary Tree Traversals

---

- There are four standard ways to visit every node:
  - **Inorder** (left, root, right): for BSTs, this visits nodes in sorted order.
  - **Preorder** (root, left, right): useful for serialisation and copying trees.
  - **Postorder** (left, right, root): useful for deletion and computing sizes.
  - **Level-order** (BFS): visit nodes level by level using a queue.

```
class TreeNode:
 def __init__(self, val=0, left=None, right=None):
 self.val = val
 self.left = left
 self.right = right

def inorder(root):
 if not root:
 return []
 return inorder(root.left) + [root.val] + inorder(root.right)

def preorder(root):
 if not root:
 return []
 return [root.val] + preorder(root.left) + preorder(root.right)
```

```

def postorder(root):
 if not root:
 return []
 return postorder(root.left) + postorder(root.right) + [root.val]

from collections import deque

def level_order(root):
 if not root:
 return []
 result, queue = [], deque([root])
 while queue:
 level = []
 for _ in range(len(queue)):
 node = queue.popleft()
 level.append(node.val)
 if node.left:
 queue.append(node.left)
 if node.right:
 queue.append(node.right)
 result.append(level)
 return result

```

- **Pitfall:** the recursive traversals above create new lists at each step (due to + concatenation), which is  $O(n^2)$ . For efficiency, pass a result list and append in-place:

```

def inorder_efficient(root, result=None):
 if result is None:
 result = []
 if root:
 inorder_efficient(root.left, result)
 result.append(root.val)
 inorder_efficient(root.right, result)
 return result

```

### 69.1.1 Easy: Maximum Depth of Binary Tree

```

def max_depth(root):
 if not root:

```

```

 return 0
 return 1 + max(max_depth(root.left), max_depth(root.right))

```

- **The recursive pattern:** base case (null  $\rightarrow$  0), recurse on children, combine (1 + max). This same pattern applies to dozens of tree problems.

### 69.1.2 Easy: Invert Binary Tree

```

def invert_tree(root):
 if not root:
 return None
 root.left, root.right = invert_tree(root.right), invert_tree(root.left)
 return root

```

### 69.1.3 Medium: Lowest Common Ancestor

- **Problem:** find the lowest node that is an ancestor of both p and q.
- **Pattern:** if both p and q are in the left subtree, the LCA is in the left subtree. If both are in the right, it is in the right. If they split (one left, one right), the current node is the LCA.

```

def lowest_common_ancestor(root, p, q):
 if not root or root == p or root == q:
 return root

 left = lowest_common_ancestor(root.left, p, q)
 right = lowest_common_ancestor(root.right, p, q)

 if left and right:
 return root # p and q are in different subtrees
 return left if left else right

```

- **Pitfall:** this assumes p and q both exist in the tree. If they might not, you need additional checks.

### 69.1.4 Hard: Binary Tree Maximum Path Sum

- **Problem:** find the maximum sum path between any two nodes (the path does not need to go through the root).

```

def max_path_sum(root):
 best = [float('-inf')]

```

```

def dfs(node):
 if not node:
 return 0
 left = max(dfs(node.left), 0) # ignore negative paths
 right = max(dfs(node.right), 0)

 # path through this node (possibly as the "bend")
 best[0] = max(best[0], node.val + left + right)

 # return the max gain this node can contribute to its parent
 return node.val + max(left, right)

dfs(root)
return best[0]

```

- **Key insight:** at each node, there are two questions: (1) what is the best path that goes through this node (left + node + right)? (2) what is the best path this node can contribute to its parent (node + max(left, right), since a path cannot fork at two levels)? Confusing these two is the most common mistake.

## 69.2 Binary Search Trees (BSTs)

---

- A **BST** satisfies: for every node, all values in the left subtree are smaller, all values in the right subtree are larger. This enables  $O(\log n)$  search, insert, and delete (when balanced).

```

def search_bst(root, target):
 if not root:
 return None
 if target < root.val:
 return search_bst(root.left, target)
 elif target > root.val:
 return search_bst(root.right, target)
 else:
 return root

def insert_bst(root, val):
 if not root:
 return TreeNode(val)
 if val < root.val:
 root.left = insert_bst(root.left, val)
 else:

```



```
 root.right = insert_bst(root.right, val)
 return root
```

- **Pitfall:** BST operations are  $O(\log n)$  only when the tree is balanced. A BST built from sorted insertions degenerates to a linked list:  $O(n)$  per operation. This is why balanced BSTs (AVL, red-black) exist.

### 69.2.1 Medium: Validate Binary Search Tree

```
def is_valid_bst(root, lo=float('-inf'), hi=float('inf')):
 if not root:
 return True
 if root.val <= lo or root.val >= hi:
 return False
 return (is_valid_bst(root.left, lo, root.val) and
 is_valid_bst(root.right, root.val, hi))
```

- **Pitfall:** checking only  $\text{left.val} < \text{root.val} < \text{right.val}$  is wrong. The constraint is that all nodes in the left subtree are smaller, not just the immediate child. The  $\text{lo/hi}$  bounds propagate this constraint down.

### 69.2.2 Medium: Kth Smallest Element in a BST

- **Pattern:** inorder traversal of a BST visits nodes in sorted order. The  $k$ th node visited is the answer.

```
def kth_smallest(root, k):
 count = [0]
 result = [None]

 def inorder(node):
 if not node or result[0] is not None:
 return
 inorder(node.left)
 count[0] += 1
 if count[0] == k:
 result[0] = node.val
 return
 inorder(node.right)

 inorder(root)
 return result[0]
```

## 69.3 Tries (Prefix Trees)

---

- A **trie** stores strings character by character in a tree. Each edge represents a character, and paths from root to marked nodes represent stored strings. Tries enable  $O(L)$  lookup where  $L$  is the string length, regardless of how many strings are stored.

```
class TrieNode:
 def __init__(self):
 self.children = {}
 self.is_end = False

class Trie:
 def __init__(self):
 self.root = TrieNode()

 def insert(self, word):
 node = self.root
 for char in word:
 if char not in node.children:
 node.children[char] = TrieNode()
 node = node.children[char]
 node.is_end = True

 def search(self, word):
 node = self.root
 for char in word:
 if char not in node.children:
 return False
 node = node.children[char]
 return node.is_end

 def starts_with(self, prefix):
 node = self.root
 for char in prefix:
 if char not in node.children:
 return False
 node = node.children[char]
 return True
```

- **When to use:** autocomplete, spell check, word games, IP routing tables. Whenever you need prefix-based operations.

### 69.3.1 Hard: Word Search II

- **Problem:** given a board of characters and a list of words, find all words that can be formed by traversing adjacent cells.
- **Pattern:** build a trie from the word list, then DFS from each cell using the trie to prune branches early (if no word starts with the current prefix, stop).
- **Pitfall:** without the trie, you would DFS for each word separately:  
 $O(w \cdot m \cdot n \cdot 4^L)$ . The trie shares prefix computation across words, dramatically reducing work.

## 69.4 Union-Find (Disjoint Set Union)

---

- **Union-Find** tracks a collection of disjoint sets. Two operations: `find(x)` returns the representative of `x`'s set, and `union(x, y)` merges the sets containing `x` and `y`.

```
class UnionFind:
 def __init__(self, n):
 self.parent = list(range(n))
 self.rank = [0] * n
 self.count = n # number of connected components

 def find(self, x):
 if self.parent[x] != x:
 self.parent[x] = self.find(self.parent[x]) # path compression
 return self.parent[x]

 def union(self, x, y):
 rx, ry = self.find(x), self.find(y)
 if rx == ry:
 return False # already connected
 # union by rank
 if self.rank[rx] < self.rank[ry]:
 rx, ry = ry, rx
 self.parent[ry] = rx
 if self.rank[rx] == self.rank[ry]:
 self.rank[rx] += 1
```

```
self.count -= 1
return True
```

- With path compression and union by rank, both operations run in  $O(\alpha(n)) \approx O(1)$  amortised (inverse Ackermann, effectively constant).
- **When to use:** connected components, cycle detection in undirected graphs, Kruskal's MST, grouping equivalent items.

### 69.4.1 Medium: Number of Connected Components

```
def count_components(n, edges):
 uf = UnionFind(n)
 for u, v in edges:
 uf.union(u, v)
 return uf.count
```

### 69.4.2 Medium: Redundant Connection

- **Problem:** find the edge that, when removed, makes the graph a tree (i.e., the edge that creates a cycle).
- **Pattern:** process edges one by one. The first edge where both endpoints are already in the same component creates the cycle.

```
def find_redundant(edges):
 uf = UnionFind(len(edges) + 1)
 for u, v in edges:
 if not uf.union(u, v):
 return [u, v] # already connected → this edge creates a cycle
```

## 69.5 Segment Trees and Fenwick Trees

- **Segment trees** answer range queries (sum, min, max over a subarray) and support point updates, both in  $O(\log n)$ .
- **Fenwick trees** (Binary Indexed Trees) are a simpler, faster alternative for prefix sum queries and point updates. They use a clever bit manipulation trick: each position stores a partial sum covering a range determined by the lowest set bit.

```
class FenwickTree:
 def __init__(self, n):
```

```

self.n = n
self.tree = [0] * (n + 1)

def update(self, i, delta):
 i += 1 # 1-indexed
 while i <= self.n:
 self.tree[i] += delta
 i += i & (-i) # add lowest set bit

def prefix_sum(self, i):
 i += 1
 total = 0
 while i > 0:
 total += self.tree[i]
 i -= i & (-i) # remove lowest set bit
 return total

def range_sum(self, l, r):
 return self.prefix_sum(r) - (self.prefix_sum(l - 1) if l > 0 else 0)

```

- **When to use:** problems requiring repeated range queries with updates. Fenwick trees are preferred when you only need prefix sums; segment trees when you need arbitrary range operations (min, max, GCD).

---

## 69.6 Common Pitfalls Summary

Pitfall	Example	Fix
Checking only direct children for BST	<code>left.val &lt; root.val</code> misses deeper violations	Pass lo/hi bounds
$O(n^2)$ list concatenation in recursion	<code>inorder(left) + [val] + inorder(right)</code>	Append to shared list
Forgetting base case	Infinite recursion on empty tree	<code>if not root: return</code>
Confusing path-through vs path-to-parent	Max path sum: forking at two levels	Return single-branch to parent, track two-branch separately
1-indexed vs 0-indexed Fenwick	Off-by-one in tree array	Always <code>i += 1</code> at entry

Pitfall	Example	Fix
Union-Find without path compression	$O(n)$ per find in worst case	<pre>self.parent[x] = self.find(self.parent[x])</pre>

---

## 69.7 Take-Home Problems (NeetCode)

---

### 69.7.1 Binary Tree Patterns

- [Invert Binary Tree](#) — basic recursion
- [Maximum Depth of Binary Tree](#) — recursive depth
- [Same Tree](#) — simultaneous traversal
- [Subtree of Another Tree](#) — nested recursion
- [Binary Tree Level Order Traversal](#) — BFS with level tracking
- [Binary Tree Maximum Path Sum](#) — DFS with global best
- [Serialize and Deserialize Binary Tree](#) — preorder + null markers

### 69.7.2 BST Patterns

- [Validate Binary Search Tree](#) — bound propagation
- [Kth Smallest Element in a BST](#) — inorder traversal
- [Lowest Common Ancestor of a BST](#) — exploit BST ordering

### 69.7.3 Tries

- [Implement Trie](#) — basic trie operations
- [Design Add and Search Words](#) — trie + DFS with wildcards
- [Word Search II](#) — trie-guided backtracking

### 69.7.4 Union-Find

- [Number of Connected Components](#) — basic union-find
- [Redundant Connection](#) — cycle detection via union-find

## 70 Graphs

Graphs model relationships and connections, from social networks to road maps to dependency chains. This file covers representations, BFS, DFS, shortest paths,

topological sort, and connected components, with the traversal and pathfinding patterns that dominate graph interview problems.

- We covered graph theory in chapter 12 (adjacency matrices, Laplacian, spectral properties) and chapter 13 (trees, planarity, colouring). Here we focus on **algorithmic patterns**: how to traverse, search, and optimise over graphs in code.
- The two fundamental graph algorithms are **BFS** and **DFS**. Nearly every graph problem reduces to one of these, possibly with modifications. Master these two and you can solve the vast majority of graph problems.

## 70.1 Graph Representations

---

- **Adjacency list**: for each node, store a list of its neighbours. Space:  $O(|V| + |E|)$ . Best for sparse graphs (most real-world graphs).

```
Undirected graph
graph = {
 0: [1, 2],
 1: [0, 3],
 2: [0, 3],
 3: [1, 2]
}

From edge list
def build_graph(n, edges):
 graph = {i: [] for i in range(n)}
 for u, v in edges:
 graph[u].append(v)
 graph[v].append(u) # omit for directed
 return graph
```

- **Adjacency matrix**:  $n \times n$  matrix where  $A[i][j] = 1$  if edge  $(i, j)$  exists. Space:  $O(|V|^2)$ . Best for dense graphs or when you need  $O(1)$  edge lookup.
- **When to use which**: adjacency list for almost everything. Matrix only when the graph is dense ( $|E| \approx |V|^2$ ) or you need constant-time edge existence checks.

## 70.2 Pattern: BFS (Breadth-First Search)

- BFS explores nodes **level by level** using a queue. It is the go-to algorithm for:
  - Shortest path in **unweighted** graphs
  - Level-order traversal
  - Finding connected components
  - Any problem asking for the “minimum number of steps”

```
from collections import deque

def bfs(graph, start):
 visited = {start}
 queue = deque([start])

 while queue:
 node = queue.popleft()
 for neighbour in graph[node]:
 if neighbour not in visited:
 visited.add(neighbour)
 queue.append(neighbour)
```

- **Critical:** add to visited when **enqueueing**, not when dequeuing. If you mark visited on dequeue, the same node can be enqueued multiple times from different predecessors, wasting time and potentially causing incorrect results.

### 70.2.1 Easy: Number of Islands

- **Problem:** given a 2D grid of '1' s (land) and '0' s (water), count the number of islands.
- **Pattern:** iterate through the grid. When you find a '1' , BFS/DFS to mark all connected land cells as visited. Each BFS initiation is one island.

```
from collections import deque

def num_islands(grid):
 if not grid:
 return 0

 rows, cols = len(grid), len(grid[0])
 count = 0

 for r in range(rows):
 for c in range(cols):
```



```

 if grid[r][c] == '1':
 count += 1
 # BFS to mark entire island
 queue = deque([(r, c)])
 grid[r][c] = '0' # mark visited
 while queue:
 cr, cc = queue.popleft()
 for dr, dc in [(0,1),(0,-1),(1,0),(-1,0)]:
 nr, nc = cr + dr, cc + dc
 if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] ==
'1':

 grid[nr][nc] = '0'
 queue.append((nr, nc))

 return count

```

- **Pitfall:** the directions = [(0,1),(0,-1),(1,0),(-1,0)] pattern for 4-connected grid neighbours is used in almost every grid problem. Memorise it. For 8-connected, add the diagonals.
- **Pitfall:** modifying the input grid (grid[r][c] = '0') avoids needing a separate visited set. This is acceptable in interviews but be explicit about the tradeoff (mutates input).

### 70.2.2 Medium: Rotting Oranges

- **Problem:** fresh oranges rot if adjacent to a rotten orange. Return the minimum time until all oranges are rotten (or -1 if impossible).
- **Pattern:** multi-source BFS. Start with all initially rotten oranges in the queue simultaneously. Each BFS level is one time step.

```

from collections import deque

def oranges_rotting(grid):
 rows, cols = len(grid), len(grid[0])
 queue = deque()
 fresh = 0

 for r in range(rows):
 for c in range(cols):
 if grid[r][c] == 2:
 queue.append((r, c))
 elif grid[r][c] == 1:
 fresh += 1

```

```

if fresh == 0:
 return 0

time = 0
while queue and fresh > 0:
 time += 1
 for _ in range(len(queue)):
 cr, cc = queue.popleft()
 for dr, dc in [(0,1),(0,-1),(1,0),(-1,0)]:
 nr, nc = cr + dr, cc + dc
 if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == 1:
 grid[nr][nc] = 2
 fresh -= 1
 queue.append((nr, nc))

return time if fresh == 0 else -1

```

- **Key insight:** multi-source BFS processes all sources simultaneously. This gives the shortest distance from any source, which is exactly “how long until the last fresh orange rots.”

## 70.3 Pattern: DFS (Depth-First Search)

- DFS explores as deep as possible before backtracking. It uses a stack (explicit or the call stack via recursion). DFS is the go-to for:
  - Cycle detection
  - Topological sorting
  - Connected components
  - Backtracking / exhaustive search
  - Path-finding with constraints

```

def dfs(graph, node, visited=None):
 if visited is None:
 visited = set()
 visited.add(node)
 for neighbour in graph[node]:
 if neighbour not in visited:
 dfs(graph, neighbour, visited)

```

### 70.3.1 Medium: Course Schedule (Cycle Detection)

- **Problem:** given  $n$  courses and prerequisites, determine if all courses can be completed (i.e., no circular dependencies).
- **Pattern:** detect a cycle in a directed graph. Use DFS with three states: unvisited, in-progress (on the current DFS path), and completed.

```
def can_finish(num_courses, prerequisites):
 graph = {i: [] for i in range(num_courses)}
 for course, prereq in prerequisites:
 graph[course].append(prereq)

 # 0 = unvisited, 1 = in-progress, 2 = completed
 state = [0] * num_courses

 def has_cycle(node):
 if state[node] == 1:
 return True # back edge → cycle
 if state[node] == 2:
 return False # already fully explored

 state[node] = 1 # mark in-progress
 for neighbour in graph[node]:
 if has_cycle(neighbour):
 return True
 state[node] = 2 # mark completed
 return False

 for course in range(num_courses):
 if has_cycle(course):
 return False
 return True
```

- **Why three states:** two states (visited/unvisited) cannot distinguish “I am currently exploring this node” from “I finished exploring this node.” Finding a node that is currently being explored (state = 1) means we have found a cycle. Finding one that is fully explored (state = 2) is just a cross edge, not a cycle.

### 70.3.2 Medium: Course Schedule II (Topological Sort)

- **Problem:** return a valid course order (topological ordering).

- **Pattern (Kahn's algorithm — BFS-based):** start with nodes that have no incoming edges (indegree 0). Process them, reduce indegree of their neighbours. Repeat.

```
from collections import deque

def find_order(num_courses, prerequisites):
 graph = {i: [] for i in range(num_courses)}
 indegree = [0] * num_courses

 for course, prereq in prerequisites:
 graph[prereq].append(course)
 indegree[course] += 1

 queue = deque([i for i in range(num_courses) if indegree[i] == 0])
 order = []

 while queue:
 node = queue.popleft()
 order.append(node)
 for neighbour in graph[node]:
 indegree[neighbour] -= 1
 if indegree[neighbour] == 0:
 queue.append(neighbour)

 return order if len(order) == num_courses else [] # empty = cycle exists
```

- **Pitfall:** if the result has fewer nodes than the graph, a cycle exists (some nodes' indegree never reached 0).

## 70.4 Shortest Paths

### 70.4.1 Dijkstra's Algorithm

- Finds shortest paths from a source to all other nodes in a **non-negative** weighted graph. Uses a priority queue (min-heap).

```
import heapq

def dijkstra(graph, start):
 # graph: {node: [(neighbour, weight), ...]}
 dist = {node: float('inf') for node in graph}
 dist[start] = 0
 heap = [(0, start)]
```

```

while heap:
 d, node = heapq.heappop(heap)
 if d > dist[node]:
 continue # stale entry

 for neighbour, weight in graph[node]:
 new_dist = d + weight
 if new_dist < dist[neighbour]:
 dist[neighbour] = new_dist
 heapq.heappush(heap, (new_dist, neighbour))

return dist

```

- Time:  $O((|V| + |E|) \log |V|)$  with a binary heap.
- **Pitfall:** the `if d > dist[node]: continue` line is essential. Without it, you process stale heap entries, potentially degrading to  $O(|V|^2)$ .
- **Pitfall:** Dijkstra does not work with negative weights. If an edge has negative weight, the greedy assumption (once a node is finalised, its distance is optimal) breaks. Use Bellman-Ford instead.

### 70.4.2 Hard: Network Delay Time

- **Problem:** given  $n$  nodes and weighted directed edges, find the time for a signal to reach all nodes from a source. Return -1 if not all nodes are reachable.

```

def network_delay(times, n, k):
 graph = {i: [] for i in range(1, n + 1)}
 for u, v, w in times:
 graph[u].append((v, w))

 dist = dijkstra(graph, k)
 max_time = max(dist.values())
 return max_time if max_time < float('inf') else -1

```

## 70.5 Strongly Connected Components

- In a directed graph, a **strongly connected component (SCC)** is a maximal set of nodes where every node can reach every other.

- **Kosaraju's algorithm:** (1) DFS on the original graph, recording finish order. (2) Transpose the graph (reverse all edges). (3) DFS on the transposed graph in reverse finish order. Each DFS tree in step 3 is one SCC.
- **When to use:** finding cyclic dependencies, 2-SAT, condensing a directed graph into a DAG of SCCs.

---

## 70.6 Common Pitfalls Summary

---

Pitfall	Example	Fix
Marking visited on dequeue	Same node enqueued multiple times	Mark visited when enqueueing
Two-state visited in directed graphs	Cannot distinguish back edge from cross edge	Use three states: unvisited/in-progress/done
Dijkstra with negative weights	Incorrect shortest paths	Use Bellman-Ford
Forgetting if <code>d &gt; dist[node]: continue</code>	Processing stale heap entries	Always skip if current distance is worse
Grid bounds checking	Index out of range	<code>0 &lt;= nr &lt; rows</code> and <code>0 &lt;= nc &lt; cols</code>
Not counting time=0 edge case	Rotting oranges: no fresh oranges	Check <code>fresh == 0</code> before BFS
Building directed graph as undirected	Prerequisites go one way	Add edge only in one direction

---

## 70.7 Take-Home Problems (NeetCode)

---

### 70.7.1 BFS Patterns

- [Number of Islands](#) — grid BFS/DFS
- [Rotting Oranges](#) — multi-source BFS
- [Clone Graph](#) — BFS + hash map for cloning
- [Pacific Atlantic Water Flow](#) — BFS from both oceans
- [Word Ladder](#) — BFS on implicit graph

### 70.7.2 DFS Patterns

- [Max Area of Island](#) — DFS with area counting
- [Course Schedule](#) — cycle detection in directed graph
- [Course Schedule II](#) — topological sort
- [Number of Connected Components](#) — DFS or Union-Find
- [Graph Valid Tree](#) — connected + no cycle

### 70.7.3 Shortest Paths

- [Network Delay Time](#) — Dijkstra
- [Cheapest Flights Within K Stops](#) — modified BFS/Bellman-Ford with constraint
- [Swim in Rising Water](#) — binary search + BFS or Dijkstra on grid

### 70.7.4 Advanced

- [Alien Dictionary](#) — topological sort from character ordering

## 71 Sorting, Search, and Algorithm Design

Sorting and searching are the most fundamental algorithmic operations. This file covers sorting algorithms, binary search patterns, divide and conquer, greedy algorithms, dynamic programming, and backtracking

- Every data structure enables algorithms, and every algorithm relies on data structures. This file covers the **design paradigms**: the high-level strategies for solving problems. Once you recognise which paradigm applies, the implementation follows naturally.

## 71.1 Sorting Algorithms

- Sorting is the most studied problem in computer science. Understanding the algorithms builds intuition for recursion, divide-and-conquer, and complexity analysis.

Algorithm	Best	Average	Worst	Space	Stable?
Bubble sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Insertion sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No
Counting sort	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(k)$	Yes
Radix sort	$O(d(n + k))$	$O(d(n + k))$	$O(d(n + k))$	$O(n + k)$	Yes

- **Stable** means equal elements preserve their relative order. This matters when sorting by multiple keys.
- The **lower bound** for comparison-based sorting is  $\Omega(n \log n)$ . The proof uses decision trees (chapter 13): any comparison sort must distinguish all  $n!$  permutations, requiring at least  $\log_2(n!) = \Omega(n \log n)$  comparisons. Counting sort and radix sort beat this by not comparing elements.

### 71.1.1 Merge Sort

- Divide the array in half, recursively sort each half, then merge the sorted halves.  $O(n \log n)$  always,  $O(n)$  extra space.

```
def merge_sort(arr):
 if len(arr) <= 1:
 return arr

 mid = len(arr) // 2
 left = merge_sort(arr[:mid])
 right = merge_sort(arr[mid:])

 return merge(left, right)
```



```

def merge(left, right):
 result = []
 i = j = 0
 while i < len(left) and j < len(right):
 if left[i] <= right[j]: # <= for stability
 result.append(left[i])
 i += 1
 else:
 result.append(right[j])
 j += 1
 result.extend(left[i:])
 result.extend(right[j:])
 return result

```

- **Pitfall:** using < instead of <= in the merge breaks stability (equal elements from the right half would come before those from the left).

### 71.1.2 Quick Sort

- Pick a **pivot**, partition elements into “less than pivot” and “greater than pivot,” recursively sort each partition.  $O(n \log n)$  average,  $O(n^2)$  worst case (when the pivot is always the smallest or largest element).

```

def quicksort(arr, lo=0, hi=None):
 if hi is None:
 hi = len(arr) - 1
 if lo >= hi:
 return

 pivot_idx = partition(arr, lo, hi)
 quicksort(arr, lo, pivot_idx - 1)
 quicksort(arr, pivot_idx + 1, hi)

def partition(arr, lo, hi):
 pivot = arr[hi] # Lomuto: pivot is last element
 i = lo
 for j in range(lo, hi):
 if arr[j] < pivot:
 arr[i], arr[j] = arr[j], arr[i]
 i += 1

```

```
arr[i], arr[hi] = arr[hi], arr[i]
return i
```

- **Pivot strategies:** last element (simple, bad for sorted input), random (expected  $O(n \log n)$ ), median-of-three (practical choice). Always prefer random pivot in interviews to avoid worst-case discussion.
- **Pitfall:** quicksort's  $O(n^2)$  worst case occurs on already-sorted arrays with first/last pivot. In practice, random pivot or median-of-three eliminates this.

### 71.1.3 Counting Sort

- When values are integers in a known range  $[0, k)$ , count occurrences and reconstruct:  $O(n + k)$  time. Not comparison-based, so it can beat  $O(n \log n)$ .

```
def counting_sort(arr, k):
 count = [0] * k
 for x in arr:
 count[x] += 1
 result = []
 for val in range(k):
 result.extend([val] * count[val])
 return result
```

- **When to use:** the range  $k$  is not much larger than  $n$ . If  $k = O(n)$ , this is  $O(n)$ . If  $k \gg n$  (e.g., sorting 10 numbers in range  $[0, 10^9]$ ), counting sort wastes memory.
- 

## 71.2 Pattern: Binary Search

- Binary search finds a target in a sorted array in  $O(\log n)$  by repeatedly halving the search space. But binary search is much more than “find a number in a sorted array.” The general pattern is: **search on a monotonic condition**.
- **Template** (the one that avoids off-by-one errors):

```
def binary_search(arr, target):
 lo, hi = 0, len(arr) - 1

 while lo <= hi:
 mid = lo + (hi - lo) // 2 # avoids overflow in other languages
 if arr[mid] == target:
 return mid
```

```

 elif arr[mid] < target:
 lo = mid + 1
 else:
 hi = mid - 1

 return -1 # not found

```

- **Lower bound** (first element  $\geq$  target):

```

def lower_bound(arr, target):
 lo, hi = 0, len(arr)
 while lo < hi:
 mid = (lo + hi) // 2
 if arr[mid] < target:
 lo = mid + 1
 else:
 hi = mid
 return lo

```

- **Pitfall:** the difference between  $lo \leq hi$  and  $lo < hi$ , and between  $hi = mid$  and  $hi = mid - 1$ , determines whether you find the exact match or the boundary. Draw it out with a 2-element array to verify.

### 71.2.1 Easy: Binary Search

- The standard problem. Use the template above.

### 71.2.2 Medium: Search in Rotated Sorted Array

- **Problem:** a sorted array is rotated at some pivot. Find a target.
- **Pattern:** at each step, one half is always sorted. Determine which half is sorted, and check if the target lies in that half.

```

def search_rotated(nums, target):
 lo, hi = 0, len(nums) - 1

 while lo <= hi:
 mid = (lo + hi) // 2
 if nums[mid] == target:
 return mid

 # left half is sorted
 if nums[lo] <= nums[mid]:
 if nums[lo] <= target < nums[mid]:

```

```

 hi = mid - 1
 else:
 lo = mid + 1
 # right half is sorted
 else:
 if nums[mid] < target <= nums[hi]:
 lo = mid + 1
 else:
 hi = mid - 1

 return -1

```

- **Pitfall:** the `<=` in `nums[lo] <= nums[mid]` (not `<`) is critical. When `lo == mid` (2 elements left), we must correctly identify the sorted half.

### 71.2.3 Hard: Median of Two Sorted Arrays

- **Problem:** find the median of two sorted arrays in  $O(\log(m + n))$ .
- **Pattern:** binary search on the partition point of the smaller array. The partition divides both arrays such that all elements on the left are smaller than all elements on the right.

```

def find_median(nums1, nums2):
 if len(nums1) > len(nums2):
 nums1, nums2 = nums2, nums1 # ensure nums1 is shorter

 m, n = len(nums1), len(nums2)
 lo, hi = 0, m
 half = (m + n + 1) // 2

 while lo <= hi:
 i = (lo + hi) // 2 # partition point in nums1
 j = half - i # partition point in nums2

 left1 = nums1[i - 1] if i > 0 else float('-inf')
 right1 = nums1[i] if i < m else float('inf')
 left2 = nums2[j - 1] if j > 0 else float('-inf')
 right2 = nums2[j] if j < n else float('inf')

 if left1 <= right2 and left2 <= right1:
 # correct partition
 if (m + n) % 2 == 1:
 return max(left1, left2)
 return (max(left1, left2) + min(right1, right2)) / 2
 elif left1 > right2:

```

```
hi = i - 1
else:
 lo = i + 1
```

- This is one of the hardest binary search problems. The key insight is that you are not searching for a value but for a **partition point** that satisfies a condition.

#### 71.2.4 Meta-Pattern: Binary Search on Answer

- Many problems that do not look like binary search can be solved by binary searching on the answer. If the answer is a number, and you can write a function `is_feasible(x)` that is monotonic (True for all  $x \geq \text{optimal}$ , or False for all  $x < \text{optimal}$ ), then binary search over  $x$ .
- **Example:** “what is the minimum capacity of a ship to deliver all packages within  $d$  days?” Binary search on the capacity. For each candidate capacity, greedily check if all packages can be delivered in  $d$  days.

```
def ship_within_days(weights, days):
 lo, hi = max(weights), sum(weights)

 while lo < hi:
 mid = (lo + hi) // 2
 # can we ship with capacity mid in <= days?
 current_load, num_days = 0, 1
 for w in weights:
 if current_load + w > mid:
 num_days += 1
 current_load = 0
 current_load += w

 if num_days <= days:
 hi = mid
 else:
 lo = mid + 1

 return lo
```

---

### 71.3 Pattern: Greedy Algorithms

---

- A **greedy** algorithm makes the locally optimal choice at each step, hoping this leads to the globally optimal solution. Greedy works when the problem has

**greedy choice property** (local optimum leads to global optimum) and **optimal substructure** (optimal solution contains optimal solutions to subproblems).

### 71.3.1 Medium: Jump Game

- **Problem:** given an array where `nums[i]` is the max jump length at position `i`, determine if you can reach the last index.

```
def can_jump(nums):
 max_reach = 0
 for i, jump in enumerate(nums):
 if i > max_reach:
 return False # cannot reach this position
 max_reach = max(max_reach, i + jump)
 return True
```

- **Why greedy works:** we only need to know the farthest reachable position. If the current position is beyond the farthest reach, we are stuck. Otherwise, we update the farthest reach.

### 71.3.2 Medium: Merge Intervals

- **Problem:** merge overlapping intervals.

```
def merge_intervals(intervals):
 intervals.sort(key=lambda x: x[0])
 merged = [intervals[0]]

 for start, end in intervals[1:]:
 if start <= merged[-1][1]:
 merged[-1][1] = max(merged[-1][1], end)
 else:
 merged.append([start, end])

 return merged
```

- **Pattern:** sort by start time, then merge greedily. If the current interval overlaps with the last merged one, extend it. Otherwise, start a new merged interval.
  - **Pitfall:** using `merged[-1][1] = end` instead of `merged[-1][1] = max(merged[-1][1], end)`. An interval can be fully contained within another (e.g., `[1, 10]` and `[2, 5]`).
-

## 71.4 Pattern: Dynamic Programming

---

- **Dynamic programming (DP)** solves problems by breaking them into overlapping subproblems, solving each subproblem once, and storing the results. It works when a problem has **optimal substructure** and **overlapping subproblems**.
- **Two approaches:**
  - **Top-down (memoisation):** write the natural recursive solution, then cache results in a dictionary.
  - **Bottom-up (tabulation):** build a table from the smallest subproblems up.
- **How to identify DP:** the problem asks for an optimum (min/max), counting, or existence, and the current decision depends on previous decisions. If you draw the recursion tree and see repeated subproblems, it is DP.

### 71.4.1 Easy: Climbing Stairs

- **Problem:** n steps, you can climb 1 or 2 at a time. How many distinct ways?
- This is Fibonacci:  $f(n) = f(n - 1) + f(n - 2)$ .

```
def climb_stairs(n):
 if n <= 2:
 return n
 a, b = 1, 2
 for _ in range(3, n + 1):
 a, b = b, a + b
 return b
```

- $O(n)$  time,  $O(1)$  space. The full memoisation table is not needed since each state depends only on the previous two.

### 71.4.2 Medium: Coin Change

- **Problem:** given coin denominations and a target amount, find the minimum number of coins needed.
- **State:**  $dp[amount]$  = minimum coins to make amount.
- **Transition:**  $dp[amount] = \min(dp[amount - coin] + 1)$  for each coin.
- **Base case:**  $dp[0] = 0$ .

```
def coin_change(coins, amount):
 dp = [float('inf')] * (amount + 1)
 dp[0] = 0

 for a in range(1, amount + 1):
 for coin in coins:
 if coin <= a and dp[a - coin] + 1 < dp[a]:
 dp[a] = dp[a - coin] + 1

 return dp[amount] if dp[amount] != float('inf') else -1
```

- **Pitfall:** initialising with `float('inf')` (not 0 or -1). The minimum comparison only works if unreachable states are infinity.

### 71.4.3 Medium: Longest Common Subsequence

- **Problem:** given two strings, find the length of their longest common subsequence.
- **State:**  $dp[i][j]$  = LCS of `text1[:i]` and `text2[:j]`.
- **Transition:** if `text1[i-1] == text2[j-1]`, then  $dp[i][j] = dp[i-1][j-1] + 1$ . Otherwise,  $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$ .

```
def longest_common_subsequence(text1, text2):
 m, n = len(text1), len(text2)
 dp = [[0] * (n + 1) for _ in range(m + 1)]

 for i in range(1, m + 1):
 for j in range(1, n + 1):
 if text1[i - 1] == text2[j - 1]:
 dp[i][j] = dp[i - 1][j - 1] + 1
 else:
 dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])

 return dp[m][n]
```

### 71.4.4 Hard: 0/1 Knapsack

- **Problem:** given items with weights and values, and a capacity  $W$ , maximise the total value without exceeding  $W$ .
- **State:**  $dp[i][w]$  = max value using first  $i$  items with capacity  $w$ .



- **Transition:**  $dp[i][w] = \max(dp[i-1][w], dp[i-1][w - \text{weight}[i]] + \text{value}[i])$  (skip or take item i).

```
def knapsack(weights, values, capacity):
 n = len(weights)
 dp = [[0] * (capacity + 1) for _ in range(n + 1)]

 for i in range(1, n + 1):
 for w in range(capacity + 1):
 dp[i][w] = dp[i - 1][w] # skip item i
 if weights[i - 1] <= w:
 dp[i][w] = max(dp[i][w],
 dp[i - 1][w - weights[i - 1]] + values[i - 1])

 return dp[n][capacity]
```

- **Space optimisation:** since each row only depends on the previous row, use a 1D array and iterate w from right to left:

```
def knapsack_optimised(weights, values, capacity):
 dp = [0] * (capacity + 1)
 for i in range(len(weights)):
 for w in range(capacity, weights[i] - 1, -1): # right to left!
 dp[w] = max(dp[w], dp[w - weights[i]] + values[i])
 return dp[capacity]
```

- **Pitfall:** iterating left to right in the 1D version allows using item i multiple times (unbounded knapsack). Right to left ensures each item is used at most once.

## 71.5 Pattern: Backtracking

- **Backtracking** is exhaustive search with pruning. Build a solution incrementally, and abandon (backtrack) as soon as the partial solution cannot possibly lead to a valid complete solution.
- **Template:**

```
def backtrack(candidates, path, result):
 if is_solution(path):
 result.append(path[:]) # copy!
 return

 for candidate in get_candidates(path):
```

```

 if is_valid(candidate, path):
 path.append(candidate) # choose
 backtrack(candidates, path, result) # explore
 path.pop() # unchoose (backtrack)

```

### 71.5.1 Medium: Subsets

```

def subsets(nums):
 result = []
 def backtrack(start, path):
 result.append(path[:])
 for i in range(start, len(nums)):
 path.append(nums[i])
 backtrack(i + 1, path)
 path.pop()
 backtrack(0, [])
 return result

```

### 71.5.2 Medium: Combination Sum

- **Problem:** find all unique combinations that sum to a target (elements can be reused).

```

def combination_sum(candidates, target):
 result = []
 def backtrack(start, path, remaining):
 if remaining == 0:
 result.append(path[:])
 return
 for i in range(start, len(candidates)):
 if candidates[i] > remaining:
 break # prune: sorted, so all further candidates are too large
 path.append(candidates[i])
 backtrack(i, path, remaining - candidates[i]) # i, not i+1: reuse
allowed
 path.pop()

 candidates.sort() # sort for pruning
 backtrack(0, [], target)
 return result

```

- **Pitfall:** `backtrack(i, ...)` allows reuse of the same element. `backtrack(i + 1, ...)` would move to the next element (no reuse). Getting this wrong is the most common backtracking bug.

### 71.5.3 Hard: N-Queens

- **Problem:** place  $n$  queens on an  $n \times n$  board such that no two attack each other.

```
def solve_n_queens(n):
 result = []
 cols = set()
 pos_diag = set() # (row + col) is constant on / diagonals
 neg_diag = set() # (row - col) is constant on \ diagonals

 board = [['.'] * n for _ in range(n)]

 def backtrack(row):
 if row == n:
 result.append([''.join(r) for r in board])
 return

 for col in range(n):
 if col in cols or (row + col) in pos_diag or (row - col) in neg_diag:
 continue

 cols.add(col)
 pos_diag.add(row + col)
 neg_diag.add(row - col)
 board[row][col] = 'Q'

 backtrack(row + 1)

 cols.remove(col)
 pos_diag.remove(row + col)
 neg_diag.remove(row - col)
 board[row][col] = '.'

 backtrack(0)
 return result
```

- **Key insight:** the diagonal encoding. For / diagonals,  $row + col$  is constant. For \ diagonals,  $row - col$  is constant. Using sets for column and diagonal tracking makes the validity check  $O(1)$ .
-

## 71.6 Common Pitfalls Summary

Pitfall	Example	Fix
<code>lo &lt;= hi</code> vs <code>lo &lt; hi</code> in binary search	Off-by-one in bounds	Choose based on whether <code>hi</code> is inclusive or exclusive
Left-to-right 1D knapsack	Items used multiple times	Iterate right-to-left for 0/1 knapsack
Not copying path in backtracking	<code>result.append(path)</code> — all entries point to same list	<code>result.append(path[:])</code> or <code>path.copy()</code>
<code>backtrack(i)</code> vs <code>backtrack(i+1)</code>	Reuse vs no-reuse of elements	Match the problem statement
Missing break in sorted backtracking	Exploring candidates that are too large	Sort + break when candidate exceeds remaining
DP initialisation	<code>dp[0]</code> wrong → all subsequent values wrong	Carefully define the base case
Greedy without proof	Greedy does not always work	Verify greedy choice property
Unstable sort for multi-key	Relative order of equal elements lost	Use stable sort (merge sort, Python's <code>sorted()</code> )

## 71.7 Take-Home Problems (NeetCode)

### 71.7.1 Binary Search

- [Binary Search](#) — standard template
- [Search a 2D Matrix](#) — binary search on flattened matrix
- [Koko Eating Bananas](#) — binary search on answer
- [Search in Rotated Sorted Array](#) — identify sorted half
- [Find Minimum in Rotated Sorted Array](#) — binary search for inflection
- [Median of Two Sorted Arrays](#) — partition-based binary search

### 71.7.2 Greedy

- [Jump Game](#) — track max reach
- [Jump Game II](#) — BFS-style level tracking
- [Merge Intervals](#) — sort + merge
- [Insert Interval](#) — find overlap region
- [Non-overlapping Intervals](#) — sort by end time

### 71.7.3 Dynamic Programming

- [Climbing Stairs](#) — Fibonacci DP
- [House Robber](#) — take-or-skip DP
- [House Robber II](#) — circular: run twice
- [Coin Change](#) — unbounded knapsack
- [Longest Common Subsequence](#) — 2D DP on two strings
- [Word Break](#) — DP with set lookup
- [Longest Increasing Subsequence](#) —  $O(n^2)$  DP or  $O(n \log n)$  with binary search
- [Edit Distance](#) — classic 2D DP
- [Partition Equal Subset Sum](#) — 0/1 knapsack variant

### 71.7.4 Backtracking

- [Subsets](#) — enumerate all subsets
- [Combination Sum](#) — backtrack with reuse
- [Permutations](#) — backtrack with used set
- [Subsets II](#) — skip duplicates
- [Word Search](#) — grid backtracking
- [Palindrome Partitioning](#) — backtrack + palindrome check
- [N-Queens](#) — constraint propagation

## 72 Linux and the Command Line

The command line is the primary interface for ML engineering: training jobs, server management, data pipelines, and cluster administration all happen through the terminal. This file covers the shell, file system, permissions, process management,

package managers, environment variables, SSH, and the essential commands every ML engineer uses daily.

- GUIs are convenient for browsing the web. They are terrible for running a training job on a remote GPU cluster at 2 AM. The **command line** (or terminal, or shell) is the tool that scales: it works on any machine, can be scripted, is composable, and is the same on your laptop, a cloud VM, and an HPC cluster.
- If you are an ML engineer who only uses Jupyter notebooks and VS Code buttons, you are leaving enormous productivity on the table. Every production ML system is deployed, monitored, and debugged through the command line.

## 72.1 The Shell

---

- A **shell** is a program that reads commands from you and executes them. It is the intermediary between you and the operating system (chapter 13). The most common shells are **bash** (the default on most Linux systems) and **zsh** (the default on macOS).
- A command has the form: `command [options] [arguments]`

```
ls -la /home/user # command=ls, options=-la, argument=/home/user
```

- Options modify behaviour (usually prefixed with `-` for short or `--` for long form). `ls -l` lists in long format, `ls --all` shows hidden files. Many options can be combined: `ls -la` means `-l` and `-a` together.

### 72.1.1 Essential Navigation

```
pwd # print working directory (where am I?)
ls # list files in current directory
ls -la # list all files (including hidden) with details
cd /path/to/dir # change directory
cd .. # go up one level
cd ~ # go to home directory
cd - # go back to previous directory
```

### 72.1.2 File Operations

```
cp source dest # copy file
cp -r dir1 dir2 # copy directory recursively
mv old new # move/rename file
```

```
rm file # delete file (no recycle bin – gone forever)
rm -rf dir # delete directory recursively (DANGEROUS – no confirma
mkdir -p a/b/c # create nested directories
touch file.txt # create empty file (or update timestamp)
cat file.txt # print file contents
head -n 20 file # first 20 lines
tail -f logfile # follow a log file in real-time (invaluable for monito
training)
```

- **Pitfall:** `rm -rf` is the most dangerous command in computing. There is no undo. Triple-check the path before pressing enter. Never run `rm -rf /` or `rm -rf ~`.

### 72.1.3 Pipes and Redirection

- The shell's killer feature is **composability**: small commands connected together to do complex things.
- **Pipe** (`|`): sends the output of one command as input to the next.

```
cat training.log | grep "loss" | tail -5 # last 5 lines containing "loss"
ps aux | grep python # find running Python processes
history | grep "docker" # find previous docker commands
```

- **Redirection:** send output to a file instead of the screen.

```
python train.py > output.log 2>&1 # stdout AND stderr to file
python train.py >> output.log # append (don't overwrite)
echo "data" > file.txt # overwrite file
echo "more" >> file.txt # append to file
```

- `2>&1` redirects `stderr` (file descriptor 2) to `stdout` (file descriptor 1). Without it, error messages still appear on screen while only normal output goes to the file.

### 72.1.4 Text Processing

```
grep "error" logfile.txt # find lines containing "error"
grep -r "import torch" src/ # search recursively in directory
grep -i "warning" log.txt # case-insensitive search
grep -c "epoch" train.log # count matching lines

wc -l file.txt # count lines
wc -w file.txt # count words

sort data.txt # sort lines alphabetically
```

```

sort -n numbers.txt # sort numerically
sort -u data.txt # sort and remove duplicates
uniq -c sorted.txt # count consecutive duplicates

cut -d',' -f2,3 data.csv # extract columns 2 and 3 from CSV
awk '{print $1, $3}' data.txt # print 1st and 3rd whitespace-separated
fields
sed 's/old/new/g' file.txt # replace all occurrences of "old" with "new"

```

- These compose beautifully:

```

Find the 10 most common error types in a log file
grep "ERROR" app.log | awk -F': ' '{print $2}' | sort | uniq -c | sort -rn |
head -10

```

### 72.1.5 Finding Files

```

find . -name "*.py" # find all Python files
find . -name "*.pyc" -delete # find and delete compiled Python files
find /data -size +100M # files larger than 100 MB
find . -mtime -1 # files modified in the last 24 hours

which python # where is the python executable?
locate filename # fast file search (uses pre-built index)

```

## 72.2 File System Hierarchy

- Linux organises everything in a single tree rooted at /:

Directory	Purpose
/	Root of the entire file system
/home/user	Your personal files, configs, projects
/etc	System-wide configuration files
/usr	User programs, libraries, documentation
/usr/local	Locally installed software (not from package manager)
/var	Variable data: logs (/var/log), databases, caches
/tmp	Temporary files (cleared on reboot)



Directory	Purpose
/opt	Optional third-party software
/proc	Virtual file system exposing kernel and process info
/dev	Device files (disks, GPUs show up here)

- For ML: your training data is typically in /data or /home/user/data, models in /home/user/models, and CUDA lives in /usr/local/cuda. GPU devices appear as /dev/nvidia0, /dev/nvidia1, etc.

## 72.3 File Permissions

- Every file and directory has three permission types for three user classes:

Permission	File	Directory
<b>r</b> (read)	View contents	List contents
<b>w</b> (write)	Modify contents	Create/delete files inside
<b>x</b> (execute)	Run as program	Enter (cd into) the directory

- Three user classes: **owner** (u), **group** (g), **others** (o).

```
ls -l script.py
-rwxr-xr-- 1 henry ml_team 2048 Mar 28 script.py
^^^ owner permissions: rwx (read, write, execute)
^^^ group permissions: r-x (read, execute, no write)
^^^ others permissions: r-- (read only)
```

```
chmod 755 script.py # owner=rwx, group=rx, others=rx
chmod +x script.py # add execute permission for everyone
chmod u+w,g-w file.txt # add write for owner, remove write for group
chown henry:ml_team file # change owner and group
```

- **Pitfall:** a Python script with `#!/usr/bin/env python3` at the top needs execute permission (`chmod +x`) to be run as `./script.py`. Without it, you must use `python3 script.py`.

## 72.4 Process Management

---

- A **process** is a running program (chapter 13). The shell gives you tools to manage them:

```
ps aux # list all running processes
ps aux | grep python # find Python processes
top # real-time process monitor (CPU, memory)
htop # better version of top (install separately)
nvidia-smi # GPU usage (essential for ML)
watch -n 1 nvidia-smi # refresh nvidia-smi every second

kill PID # gracefully terminate process
kill -9 PID # force kill (use when graceful fails)
killall python # kill all Python processes

Run in background
python train.py & # run in background
nohup python train.py > log.txt & # run in background, survive logout
```

- **nohup** is critical for ML training: without it, closing your SSH connection kills the training job. **nohup** detaches the process from the terminal.
- **screen** and **tmux** are terminal multiplexers that create persistent sessions. You can start a training job in a **tmux** session, disconnect from SSH, reconnect later, and the session (and training) is still running.

```
tmux new -s training # create named session
... start training ...
Ctrl+B, then D # detach from session
tmux attach -t training # reattach later (even after SSH reconnect)
tmux ls # list sessions
```

## 72.5 Package Managers

---

- **System packages** (OS-level software):

```
Debian/Ubuntu
sudo apt update # refresh package list
sudo apt install htop # install a package
sudo apt upgrade # upgrade all packages
```

```
macOS
brew install wget # install via Homebrew
```

- **Python packages:**

```
pip install torch # install from PyPI
pip install -e . # install current project in editable mode
pip install -r requirements.txt # install from requirements file
pip freeze > requirements.txt # export installed packages

Conda (for complex dependencies like CUDA)
conda create -n myenv python=3.11
conda activate myenv
conda install pytorch torchvision cudatoolkit=12.1 -c pytorch
```

- **Pitfall:** never `pip install` into the system Python. Always use a virtual environment (`python -m venv env`, `conda create`, or `uv venv`). System Python is shared by OS tools; breaking it can break your system.

## 72.6 Environment Variables

- **Environment variables** are key-value pairs accessible to all programs. They configure behaviour without changing code.

```
export CUDA_VISIBLE_DEVICES=0,1 # use only GPUs 0 and 1
export PYTHONPATH=/home/user/src # add to Python's import path
export WANDB_API_KEY=abc123 # API key for Weights & Biases

echo $PATH # see current PATH
export PATH=$PATH:/usr/local/cuda/bin # add CUDA to PATH
```

- **.bashrc** (or **.zshrc**): commands run every time you open a shell. Put your `export` statements here so they persist.
- **.env files**: project-specific variables loaded by tools like `python-dotenv`. Keep secrets (API keys, database passwords) in `.env` and add `.env` to `.gitignore`. Never commit secrets to git.

## 72.7 SSH (Secure Shell)

- **SSH** connects you to remote machines over an encrypted channel. This is how you access cloud VMs, GPU servers, and HPC clusters.

```
ssh user@hostname # connect to remote machine
ssh -i ~/.ssh/key.pem user@ip # connect with specific key
ssh -L 8888:localhost:8888 user@server # port forwarding (Jupyter on remote)
```

- **SSH keys** (public/private key pair) replace passwords:

```
ssh-keygen -t ed25519 # generate key pair
ssh-copy-id user@server # copy public key to server
now you can SSH without typing a password
```

- **SSH config** (~/.ssh/config) saves connection details:

```
Host gpu-server
 HostName 10.0.1.42
 User henry
 IdentityFile ~/.ssh/gpu_key
 LocalForward 8888 localhost:8888
```

- Now `ssh gpu-server` connects with all those settings automatically.
- **scp** and **rsync** transfer files between machines:

```
scp model.pt user@server:/data/models/ # copy file to remote
scp -r user@server:/data/results/ ./ # copy directory from remote
rsync -avz --progress data/ user@server:/data/ # sync with progress (smarter
than scp)
```

## 72.8 Essential ML Commands Cheat Sheet

```
GPU monitoring
nvidia-smi # GPU usage snapshot
watch -n 1 nvidia-smi # live monitoring
gpustat # cleaner GPU overview (pip install gpustat)

Training management
nohup python train.py > train.log 2>&1 & # background training that survives
logout

tail -f train.log # monitor training output
kill %1 # kill last background job

Disk usage (datasets are huge)
df -h # disk space on all mounts
```

```

du -sh /data/* # size of each item in /data
du -sh --max-depth=1 . # size of subdirectories

Memory
free -h # RAM usage
cat /proc/meminfo # detailed memory info

Network
curl -O https://example.com/dataset.tar.gz # download file
wget https://example.com/model.bin # alternative downloader
curl -X POST http://localhost:8080/predict \
 -H "Content-Type: application/json" \
 -d '{"text": "hello"}' # test a model serving endpoint

Archives
tar -czf archive.tar.gz directory/ # compress
tar -xzf archive.tar.gz # extract
zip -r archive.zip directory/ # zip
unzip archive.zip # unzip

Quick data inspection
head -5 data.csv # first 5 lines of CSV
wc -l data.csv # count rows
cut -d',' -f1 data.csv | sort -u | wc -l # count unique values in column 1

```

## 73 Git and Version Control

Git is how software teams collaborate without overwriting each other's work. This file covers the Git mental model, branching strategies, merging and rebasing, conflict resolution, pull requests, and managing ML-specific challenges like large files and experiment tracking.

- Every serious software project uses version control. **Git** is the dominant system, used by virtually all open-source projects and companies. Without git, collaboration is emailing zip files and praying nobody overwrites your changes. With git, every change is tracked, reversible, and attributable.
- For ML engineers: git tracks your code, configs, and experiment scripts. Combined with experiment tracking tools, it gives you reproducibility: “what exact code and config produced this model?”

## 73.1 The Mental Model

- Git tracks **snapshots** of your project. Every commit is a full snapshot of all tracked files at that moment, not a diff (internally, git stores diffs for efficiency, but conceptually each commit is a complete state).
- Four “locations” for your files:
  1. **Working directory**: the actual files on disk. You edit these.
  2. **Staging area** (index): files you have marked for the next commit. `git add` moves changes here.
  3. **Local repository**: your commit history, stored in `.git/`. `git commit` saves the staging area as a new snapshot.
  4. **Remote repository** (e.g., GitHub): a shared copy. `git push` uploads your commits, `git pull` downloads others’ .

```
Working Dir → git add → Staging → git commit → Local Repo → git push
→ Remote
← git pull ←
```

- The staging area is what makes git powerful. You can edit 10 files but only commit 3 of them, keeping the other changes for a separate commit. This enables clean, focused commits.

### 73.1.1 Essential Commands

<code>git init</code>	<i># create a new repository</i>
<code>git clone url</code>	<i># download a remote repository</i>
<code>git status</code>	<i># what has changed? (most-used command)</i>
<code>git add file.py</code>	<i># stage a specific file</i>
<code>git add .</code>	<i># stage all changes (use with caution)</i>
<code>git commit -m "descriptive msg"</code>	<i># commit staged changes</i>
<code>git push</code>	<i># upload commits to remote</i>
<code>git pull</code>	<i># download + merge remote changes</i>
<code>git log --oneline</code>	<i># compact commit history</i>
<code>git diff</code>	<i># show unstaged changes</i>
<code>git diff --staged</code>	<i># show staged changes</i>

## 73.2 Branching

---

- A **branch** is a pointer to a commit. The default branch is `main` (or `master`). Creating a branch gives you an independent line of development: you can make changes without affecting `main`.

```
git branch feature-x # create a branch
git checkout feature-x # switch to it
git checkout -b feature-x # create and switch in one step
git branch -d feature-x # delete branch (after merging)
git branch -a # list all branches (local + remote)
```

- **When to branch:** always. Never commit directly to `main`. Every feature, bug fix, or experiment gets its own branch. This keeps `main` stable and deployable.

### 73.2.1 Branching Strategies

- **Feature branches** (most common): each feature/fix gets a branch off `main`. When done, open a pull request (PR) to merge back. Simple, works for most teams.
- **Trunk-based development:** developers commit to `main` frequently (multiple times per day), using feature flags to hide incomplete work. Preferred by teams that deploy continuously (Google, Facebook). Requires excellent CI/CD.
- **Gitflow:** separate branches for features, releases, and hotfixes. More complex, better for software with versioned releases (mobile apps, packaged software). Overkill for most ML projects.
- For ML teams: **feature branches** with short-lived branches (merge within 1-3 days) is the sweet spot. Long-lived branches diverge from `main` and create painful merge conflicts.

## 73.3 Merging and Rebasing

---

- **Merge** creates a new “merge commit” that combines two branches:

```
git checkout main
git merge feature-x
```

- This preserves the full history: you can see that work happened on a branch and when it was merged. The merge commit has two parents.
- **Rebase** replays your branch's commits on top of the target branch:

```
git checkout feature-x
git rebase main
```

- This rewrites history: your branch's commits get new hashes, as if you had started your work from the current tip of `main`. The result is a linear history (no merge commits), which is cleaner to read.
- **When to use which:**
  - **Rebase** for updating your feature branch with the latest `main` changes (keeps your branch clean and up-to-date).
  - **Merge** for integrating your feature branch into `main` (preserves the branch history).
  - **Never rebase commits that have been pushed and shared** with others. Rebasing rewrites history; if someone else has based work on the original commits, rebasing causes chaos.

## 73.4 Resolving Conflicts

---

- A **conflict** occurs when two branches modify the same line of the same file. Git cannot automatically decide which change to keep and asks you to resolve it manually.

```
<<<<<<< HEAD
learning_rate = 0.001
=====
learning_rate = 0.0005
>>>>>> feature-x
```

- Between `<<<<<<< HEAD` and `=====` is the current branch's version. Between `=====` and `>>>>>> feature-x` is the incoming branch's version. You decide which to keep (or combine them), remove the markers, save, and `git add` the resolved file.



- **Pitfall:** do not leave conflict markers in committed files. They are literal text that will break your code. Always search for <<<<<< after resolving.
- **Reducing conflicts:** keep branches short-lived, merge `main` into your branch frequently, and avoid multiple people editing the same file simultaneously.

## 73.5 Writing Good Commit Messages

---

- A commit message is for your future self and your teammates. “fix bug” tells you nothing. “Fix off-by-one in batch size calculation that caused OOM on 8-GPU training” tells you everything.
- **Format:**

Short summary (50 chars or less, imperative mood)

Longer description if needed. Explain WHY, not WHAT  
(the diff shows what changed). Wrap at 72 characters.

Fixes #123

- **Imperative mood:** “Add feature” not “Added feature” or “Adds feature.” Read it as completing the sentence: “If applied, this commit will **add feature.**”
- **Atomic commits:** each commit should do one thing. “Add data loader” is one commit. “Add data loader and fix unrelated bug and update README” should be three commits. This makes `git bisect` (finding which commit introduced a bug) possible.

## 73.6 Pull Requests and Code Review

---

- A **pull request (PR)** proposes merging a branch into `main`. It is the gateway for code review: teammates read your changes, suggest improvements, and approve before merging.
- **Good PR practices:**
  - Keep PRs small (under 400 lines of changes). Large PRs get rubber-stamped because nobody wants to review 2000 lines.
  - Write a clear description: what changed, why, and how to test it.

- Link to the issue or ticket that motivated the change.
  - Respond to review comments promptly.
  - Squash trivial commits before merging (so main has a clean history).
- **Code review is not about finding bugs** (tests do that). It is about: knowledge sharing (the reviewer learns the codebase), design feedback (is this the right approach?), and maintaining standards (naming, style, architecture).

## 73.7 .gitignore

---

- The .gitignore file tells git which files to exclude from tracking. For ML projects:

```
Python
__pycache__/
*.pyc
*.egg-info/
.venv/
env/

Data and models (too large for git)
data/
*.csv
*.parquet
models/
*.pt
*.onnx
*.bin
checkpoints/

Secrets
.env
*.pem
credentials.json

IDE
.vscode/
.idea/
*.swp

OS
.DS_Store
Thumbs.db

Jupyter
```

```
.ipynb_checkpoints/
```

```
Experiment outputs
wandb/
mlruns/
outputs/
logs/
```

- **Pitfall:** adding a file to `.gitignore` after it has been committed does not remove it from the repository. You must also `git rm --cached file` to untrack it. The file stays in the history forever unless you rewrite history (which is messy).

## 73.8 Git for ML

---

- ML introduces challenges that traditional software does not face:
- **Large files:** datasets and model weights are gigabytes or more. Git is designed for text files (source code), not binary blobs. Solutions:
  - **Git LFS** (Large File Storage): tracks pointers in git, stores actual files on a separate server. Simple but has storage/bandwidth limits on GitHub.
  - **DVC** (Data Version Control): manages data and model files separately from git, using remote storage (S3, GCS). Works like git for data: `dvc add data.csv, dvc push, dvc pull`.
- **Experiment tracking:** which commit + which hyperparameters + which data produced which metrics? Git tracks code, but not the full experiment context.
  - **Weights & Biases (W&B):** logs metrics, hyperparameters, system info, and links to the git commit. Provides dashboards for comparing runs.
  - **MLflow:** open-source experiment tracking with model registry. Logs parameters, metrics, and artifacts.
  - **Simple approach:** log the git hash in your training script: `git_hash = subprocess.check_output(['git', 'rev-parse', 'HEAD']).strip()`. Store it alongside your results.
- **Reproducibility checklist** (what to track for each experiment):
  - Git commit hash (exact code version)
  - Config file / hyperparameters
  - Random seeds
  - Python and library versions (`pip freeze`)

- Data version (DVC hash or dataset version tag)
- Hardware (GPU type, number of GPUs)

```
Quick reproducibility snapshot
echo "Commit: $(git rev-parse HEAD)" > experiment_info.txt
echo "Branch: $(git branch --show-current)" >> experiment_info.txt
echo "Dirty: $(git status --porcelain | wc -l) files" >> experiment_info.txt
pip freeze >> experiment_info.txt
nvidia-smi >> experiment_info.txt
```

## 74 Codebase Design and Patterns

Good codebase design is what separates a research prototype from production software. This file covers project structure, clean code principles, design patterns relevant to ML, configuration management, logging, API design, and packaging

- Most ML code starts as a Jupyter notebook. The notebook grows, gets copied, modified, shared, and eventually becomes an unmaintainable tangle of global variables, dead cells, and magic numbers. **Codebase design** is the discipline of organising code so that it remains understandable and modifiable as the project grows.
- This is not about following rules for their own sake. It is about reducing the time between “I want to change X” and “X is changed and working.” In a well-designed codebase, that time is minutes. In a poorly designed one, it is days of archaeology through undocumented spaghetti.

### 74.1 Project Structure

- A consistent project layout lets anyone (including future you) navigate the codebase instantly.

```
my_project/
├─ src/my_project/ # source code (importable package)
│ └─ __init__.py
│ └─ data/ # data loading and preprocessing
│ └─ __init__.py
│ └─ dataset.py
│ └─ transforms.py
└─ models/ # model architectures
```

```

| | | └─ __init__.py
| | | └─ transformer.py
| | | └─ layers.py
| | └─ training/ # training loops, optimisers
| | | └─ __init__.py
| | | └─ trainer.py
| | | └─ losses.py
| | └─ utils/ # shared utilities
| | | └─ __init__.py
| | | └─ logging.py
└─ configs/ # configuration files
 | └─ base.yaml
 | └─ experiment_1.yaml
└─ scripts/ # entry points (train, evaluate, serve)
 | └─ train.py
 | └─ evaluate.py
 | └─ serve.py
└─ tests/ # test files (mirrors src/ structure)
 | └─ test_dataset.py
 | └─ test_model.py
 | └─ test_trainer.py
└─ notebooks/ # exploration only (not production code)
└─ pyproject.toml # project metadata and dependencies
└─ README.md
└─ .gitignore
└─ Dockerfile

```

- **src/ layout:** putting source code under `src/my_project/` prevents accidental imports from the current directory (which masks import errors that would surface in production). Install with `pip install -e .` for development.
- **Monorepo vs multi-repo:** a **monorepo** keeps all related projects in one repository (easier cross-project changes, shared CI). A **multi-repo** gives each project its own repository (cleaner boundaries, independent versioning). Most ML teams start with a monorepo and split later if needed.
- **Scripts vs library:** keep entry points (`train.py`, `evaluate.py`) in `scripts/`. Keep reusable logic in `src/`. A training script should be ~50 lines: parse config, build dataset, build model, build trainer, train. All the complexity lives in the library.

## 74.2 Clean Code Principles

- **Naming:** the single most impactful thing you can do. A variable named `x` requires you to read the surrounding code to understand it. A variable named `learning_rate` is self-documenting.

```
BAD
def proc(d, n, lr):
 for i in range(n):
 for k, v in d.items():
 v -= lr * g[k]

GOOD
def update_parameters(parameters, num_steps, learning_rate):
 for step in range(num_steps):
 for name, param in parameters.items():
 param -= learning_rate * gradients[name]
```

- **Single Responsibility Principle:** each function/class does one thing. A function called `load_data_and_train_model` is doing two things and should be split. This makes each piece independently testable, reusable, and understandable.
- **DRY (Don't Repeat Yourself)** — but not prematurely. If you copy-paste code three times, extract it into a function. But do not create an abstraction for code you have used only once. Premature abstraction is worse than duplication: it adds complexity without proven benefit.

```
Premature abstraction (one use case, over-engineered)
class AbstractDataTransformPipelineFactory:
 ...

Just right (direct, clear, used in three places)
def normalise_image(image, mean, std):
 return (image - mean) / std
```

- **Magic numbers:** never use unexplained literal values.

```
BAD
if len(batch) > 32:
 split_batch(batch, 32)

GOOD
MAX_BATCH_SIZE = 32
```

```
if len(batch) > MAX_BATCH_SIZE:
 split_batch(batch, MAX_BATCH_SIZE)
```

- **Functions should be short:** if a function does not fit on one screen (~30 lines), it is probably doing too much. Extract logical chunks into helper functions with descriptive names. The function body then reads like a high-level summary.

## 74.3 Design Patterns for ML

---

- Design patterns are reusable solutions to common problems. These are the ones most relevant to ML codebases:
- **Factory pattern:** create objects without specifying the exact class. Useful when your config says `model: "transformer"` and you need to instantiate the right class:

```
MODEL_REGISTRY = {
 "transformer": TransformerModel,
 "cnn": CNNModel,
 "mlp": MLPModel,
}

def build_model(config):
 model_cls = MODEL_REGISTRY[config["model"]]
 return model_cls(**config["model_params"])
```

- This decouples the training script from specific model implementations. Adding a new model means adding one line to the registry, not modifying the training loop.
- **Strategy pattern:** swap algorithms at runtime. Useful for losses, optimisers, schedulers:

```
LOSS_FUNCTIONS = {
 "mse": nn.MSELoss,
 "cross_entropy": nn.CrossEntropyLoss,
 "focal": FocalLoss,
}
```

```
loss_fn = LOSS_FUNCTIONS[config["loss"]]()
```

- **Observer pattern** (callbacks/hooks): let modules react to events without tight coupling. Training frameworks (PyTorch Lightning, Keras) use callbacks extensively:

```
class EarlyStopping:
 def __init__(self, patience=5):
 self.patience = patience
 self.best_loss = float('inf')
 self.counter = 0

 def on_epoch_end(self, epoch, val_loss):
 if val_loss < self.best_loss:
 self.best_loss = val_loss
 self.counter = 0
 else:
 self.counter += 1
 if self.counter >= self.patience:
 return "stop"
```

- **Dependency injection:** pass dependencies into a function/class rather than creating them inside. This makes testing easy (inject a mock) and configuration flexible:

```
BAD: hard-coded dependency
class Trainer:
 def __init__(self):
 self.logger = WandbLogger() # cannot test without W&B

GOOD: injected dependency
class Trainer:
 def __init__(self, logger):
 self.logger = logger # can inject any logger, including a mock
```

## 74.4 Configuration Management

- Hard-coding hyperparameters, file paths, and model settings makes experiments unreproducible and modifications painful. **Externalise configuration** into files.
- **YAML** is the most common format for ML configs:



```
configs/experiment_1.yaml
model:
 name: transformer
 d_model: 512
 n_heads: 8
 n_layers: 6

training:
 batch_size: 64
 learning_rate: 3e-4
 max_epochs: 100
 early_stopping_patience: 10

data:
 train_path: /data/train.parquet
 val_path: /data/val.parquet
 max_seq_length: 512
```

- **Hydra** (Facebook) is a configuration framework that supports composition (merge base config with experiment-specific overrides), command-line overrides (python train.py training.lr=1e-3), and multi-run (sweep over hyperparameters).
- **argparse** is simpler for scripts with a few parameters:

```
import argparse

parser = argparse.ArgumentParser()
parser.add_argument("--lr", type=float, default=3e-4)
parser.add_argument("--batch-size", type=int, default=64)
parser.add_argument("--config", type=str, default="configs/base.yaml")
args = parser.parse_args()
```

- **Best practice:** have a base config with all defaults, and per-experiment configs that override only what changes. Track every experiment's config alongside its results.

## 74.5 Logging and Observability

- print statements are for debugging. **Logging** is for production:

```
import logging
```

```

logger = logging.getLogger(__name__)
logger.setLevel(logging.INFO)

logger.debug("Batch loaded: %d samples", len(batch)) # noisy, for debugging
logger.info("Epoch %d: loss=%.4f, lr=%.6f", epoch, loss, lr) # normal operation
logger.warning("GPU memory >90%, consider reducing batch size")
logger.error("Failed to load checkpoint: %s", path) # recoverable error
logger.critical("CUDA out of memory, aborting") # fatal

```

- **Why not print:** logging supports levels (filter out debug messages in production), formatting (timestamps, module names), and handlers (write to file, send to monitoring system) without changing the logging calls.
- **Structured logging** outputs machine-parseable formats (JSON) alongside human-readable messages. This enables searching and alerting on specific fields:

```

logger.info("training_step", extra={
 "epoch": 5, "step": 1200, "loss": 0.0342, "lr": 2.1e-4
})

```

## 74.6 API Design

- If your model will be used by other services (a web app, a mobile app, another ML pipeline), it needs an **API** (Application Programming Interface).
- **REST APIs** use HTTP methods: GET to read, POST to create/predict, PUT to update, DELETE to remove. Endpoints follow resource-based naming:

```

POST /api/v1/predict # send input, get prediction
GET /api/v1/models # list available models
GET /api/v1/models/{id} # get model details
POST /api/v1/models/{id}/predict # predict with a specific model

```

- **FastAPI** is the go-to Python framework for ML serving:

```

from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class PredictRequest(BaseModel):
 text: str

```

```

class PredictResponse(BaseModel):
 label: str
 confidence: float

@app.post("/predict", response_model=PredictResponse)
async def predict(request: PredictRequest):
 result = model.predict(request.text)
 return PredictResponse(label=result.label, confidence=result.score)

```

- FastAPI auto-generates API documentation (Swagger UI at /docs), validates input/output with Pydantic models, and supports async for high throughput.
- gRPC is faster than REST for internal service-to-service communication. It uses Protocol Buffers (binary serialisation, smaller and faster than JSON) and supports streaming. Used by TensorFlow Serving, Triton Inference Server, and many microservice architectures.

## 74.7 Packaging and Distribution

---

- Making your code installable as a package lets others (and your own scripts) import it cleanly:

```

pyproject.toml
[project]
name = "my-ml-project"
version = "0.1.0"
requires-python = ">=3.10"
dependencies = [
 "torch>=2.0",
 "jax>=0.4",
 "pydantic>=2.0",
]

[project.optional-dependencies]
dev = ["pytest", "ruff", "mypy"]

[build-system]
requires = ["setuptools>=64"]
build-backend = "setuptools.backends._legacy:_Backend"

```

```
pip install -e ".[dev]" # install in editable mode with dev dependencies
```

- **Editable install (-e)**: changes to your source code are immediately reflected without reinstalling. Essential during development.
- **Pinning dependencies**: `requirements.txt` with exact versions (`torch==2.2.1`, not `torch>=2.0`) ensures reproducibility. Use `pip freeze > requirements.txt` to capture your current environment. For more sophisticated dependency management, use `uv`, `poetry`, or `pip-tools`.

## 74.8 Working with AI Coding Agents

---

- AI coding agents (Claude Code, GitHub Copilot, Cursor, etc.) are now part of the professional engineering workflow. Used well, they dramatically accelerate development. Used poorly, they introduce subtle bugs, erode your understanding of your own codebase, and create a false sense of productivity.
- The right mental model: **an agent is a fast but inexperienced pair programmer**. It can write code quickly, knows syntax and standard patterns, and has read more documentation than you ever will. But it does not understand your specific system, your business constraints, your edge cases, or the why behind your design decisions. You are the senior engineer; the agent is the junior. You direct, review, and take responsibility.

### 74.8.1 When Agents Excel

- **Boilerplate and scaffolding**: generating Dockerfiles, CI configs, test fixtures, data class definitions, argparse setups. These follow well-known patterns and are tedious to write by hand. Let the agent generate them, then review for correctness.
- **Writing tests**: describe the function's behaviour, and the agent generates test cases. It often catches edge cases you would miss (empty input, negative values, Unicode). Always read the generated tests — they verify your assumptions, not just your code.
- **Refactoring**: “extract this block into a function,” “convert this class to use dataclasses,” “add type hints to this module.” Mechanical transformations where the intent is clear and the risk of subtle errors is low.

- **Exploration and prototyping:** “write a quick script to benchmark inference latency” or “show me how to use the HuggingFace tokeniser API.” The agent gets you a working starting point faster than reading documentation.
- **Documentation and docstrings:** the agent can generate documentation from your code structure. Review for accuracy, but the grunt work is automated.
- **Debugging assistance:** paste an error traceback and ask for diagnosis. The agent can often identify the root cause and suggest a fix, especially for common issues (shape mismatches, import errors, CUDA out of memory).

### 74.8.2 When to NOT Rely on Agents

- **Novel architecture decisions:** if you are designing a new training pipeline, the agent will give you a generic answer. It does not know your data constraints, latency requirements, or team expertise. Use the agent to implement the design you have already thought through.
- **Security-critical code:** authentication, encryption, input sanitisation. The agent may generate code that looks correct but has subtle vulnerabilities (SQL injection, insecure defaults, timing attacks). Security code should be written by someone who understands the threat model, and reviewed by someone else.
- **Performance-critical inner loops:** the agent will write correct but naive code. For GPU kernels, memory-critical data structures, or latency-sensitive serving paths, you need to understand the hardware constraints (chapter 13, chapter 16) and optimise deliberately.
- **Code you don’t understand:** if the agent generates 200 lines and you cannot explain what each line does, do not commit it. You are now maintaining code you do not understand, and when it breaks (it will), you cannot debug it. This is the most common and most dangerous failure mode.

### 74.8.3 The Review Discipline

- **Always read every line** of generated code before committing. This is not optional. The agent’s code is a draft, not a finished product. Treat it exactly like a pull request from a colleague: review it critically.
- **What to check:**
  - **Correctness:** does it actually do what you asked? Agents often solve a subtly different problem than the one you intended.

- **Edge cases:** does it handle empty inputs, None values, negative numbers, very large inputs? Agents frequently omit edge case handling.
- **Hallucinated APIs:** the agent may call functions or use parameters that do not exist, especially for newer or less common libraries. Verify that every API call is real.
- **Over-engineering:** agents tend to produce more code than necessary. A 50-line solution to a 10-line problem adds complexity without benefit. Simplify ruthlessly.
- **Security:** hardcoded secrets, unsanitised user input, insecure defaults. The agent does not think adversarially.
- **Style consistency:** does the generated code match your project's conventions (naming, patterns, error handling)?

#### 74.8.4 How to Write Good Prompts

- The quality of the agent's output is directly proportional to the quality of your instruction. Vague prompts get vague code.
- **Bad:** "write a data loader"
- **Good:** "write a PyTorch DataLoader for a CSV file with columns 'text' and 'label'. Tokenise the text using the HuggingFace tokeniser 'bert-base-uncased' with max\_length=512. Return input\_ids, attention\_mask, and label as tensors. Handle the case where the CSV has missing values in the label column by skipping those rows."
- **Provide context:** tell the agent about your project structure, existing code, constraints, and conventions. The more context, the better the output.
- **Specify constraints:** "use only the standard library," "must work with Python 3.10," "do not use global variables," "follow the existing pattern in src/models/transformer.py."
- **Ask for explanations:** "implement X and explain the key design decisions." This forces the agent to articulate its reasoning, making it easier for you to spot flawed assumptions.

### 74.8.5 Using Quality Gates to Catch Agent Mistakes

- Your existing quality infrastructure (file 04) catches agent errors just as well as human errors:
  - **Type checking (mypy)**: catches hallucinated API signatures and type mismatches.
  - **Linting (ruff)**: catches unused imports, undefined variables, and style violations.
  - **Tests (pytest)**: if the agent's code passes your test suite, it is more likely correct. If you do not have tests, write them before asking the agent to implement the feature (test-driven development works especially well with agents).
  - **CI pipeline**: runs all of the above automatically on every commit.
- The combination of “agent writes code” + “quality gates verify it” is more productive than either alone. The agent is fast but sloppy; the gates are thorough but do not write code. Together, you get speed and correctness.

### 74.8.6 The Productivity Trap

- The biggest risk of coding agents is **the illusion of productivity**. You can generate 500 lines of code in 10 minutes. But if you spend 2 hours debugging those 500 lines because you did not understand them, you were slower than writing 200 lines yourself in 30 minutes.
- True productivity with agents comes from:
  1. **Staying in control**: you decide the architecture, the agent fills in the implementation.
  2. **Understanding what is generated**: if you cannot explain it, rewrite it or ask the agent to simplify.
  3. **Investing in quality gates**: tests, types, and linting amortise their cost across every agent interaction.
  4. **Using the agent for your weaknesses**: if you are great at algorithms but slow at writing tests, let the agent write tests. If you are fast at UI code but unfamiliar with database queries, let the agent draft the SQL. Play to your strengths, delegate your gaps.
- The engineers who get the most out of coding agents are the ones who already know how to code well. The agent amplifies your existing skill; it does not

replace it. Understanding data structures, algorithms, system design, and software engineering (this entire chapter) is what lets you direct the agent effectively and evaluate its output critically.

## 75 Testing and Quality Assurance

Testing is how you know your code works, not just now, but after every change. This file covers the test pyramid, unit tests with pytest, mocking, testing ML-specific code, CI/CD pipelines, linting, formatting, and code review, the practices that catch bugs before they reach production.

- ML code is notoriously undertested. “It trains, therefore it works” is the prevailing attitude. This leads to silent bugs: a data loader that shuffles incorrectly, a loss function with a sign error, a preprocessing step that drops 5% of the data. These bugs do not crash your program. They just make your model quietly worse, and you waste weeks debugging metrics that “should be higher.”
- Testing is not overhead. It is the fastest way to move fast without breaking things.

### 75.1 The Test Pyramid

---

- Tests are organised in layers, from fast and narrow to slow and broad:
  - **Unit tests** (base): test individual functions and classes in isolation. Fast (milliseconds), numerous (hundreds to thousands). “Does `normalise_image` produce values in `[0, 1]`?”
  - **Integration tests** (middle): test that components work together. Slower (seconds). “Does the data loader produce batches in the format the model expects?”
  - **End-to-end tests** (top): test the full pipeline from input to output. Slow (minutes). “Does `python train.py --config test.yaml` complete without errors and produce a valid checkpoint?”
- The pyramid shape means: write many unit tests, fewer integration tests, and a handful of end-to-end tests. Unit tests catch most bugs and run in seconds. End-to-end tests catch integration issues but are slow and fragile.



## 75.2 Unit Tests with pytest

- **pytest** is the standard Python testing framework. A test is a function starting with `test_` in a file starting with `test_`:

```
tests/test_utils.py

def test_normalise_image():
 import numpy as np
 image = np.array([0, 128, 255], dtype=np.uint8)
 result = normalise_image(image, mean=128, std=128)
 assert result.min() >= -1.0
 assert result.max() <= 1.0
 assert abs(result[1]) < 1e-6 # 128 normalised by mean=128 should be ~0

def test_normalise_empty():
 import numpy as np
 image = np.array([], dtype=np.uint8)
 result = normalise_image(image, mean=128, std=128)
 assert len(result) == 0
```

```
pytest tests/ # run all tests
pytest tests/test_utils.py # run one file
pytest -v # verbose output
pytest -x # stop on first failure
pytest -k "normalise" # run tests matching name pattern
pytest --tb=short # shorter tracebacks
```

### 75.2.1 Fixtures

- **Fixtures** provide reusable setup for tests. Instead of repeating setup code in every test, define it once:

```
import pytest

@pytest.fixture
def sample_dataset():
 """Create a small dataset for testing."""
 return {
 "inputs": torch.randn(10, 3, 32, 32),
 "labels": torch.randint(0, 10, (10,))
 }

@pytest.fixture
```

```
def trained_model():
 """Load a small pretrained model."""
 model = SmallModel()
 model.load_state_dict(torch.load("tests/fixtures/small_model.pt"))
 return model

def test_model_output_shape(trained_model, sample_dataset):
 output = trained_model(sample_dataset["inputs"])
 assert output.shape == (10, 10) # batch_size x num_classes
```

- Fixtures can have **scopes**: `scope="function"` (default, fresh per test), `scope="module"` (once per file), `scope="session"` (once per test run). Use `scope="session"` for expensive setup like loading a model.

### 75.2.2 Parametrised Tests

- Test the same function with multiple inputs without duplicating code:

```
@pytest.mark.parametrize("input,expected", [
 ([1, 2, 3], 6),
 ([], 0),
 ([-1, 1], 0),
 ([1000000, 1000000], 2000000),
])
def test_sum(input, expected):
 assert sum(input) == expected
```

## 75.3 Mocking and Patching

- **Mocking** replaces a real dependency with a fake one during testing. This lets you test a function in isolation, without needing a database, API, or GPU.

```
from unittest.mock import patch, MagicMock

def test_training_logs_metrics():
 mock_logger = MagicMock()

 with patch("my_project.training.trainer.wandb") as mock_wandb:
 trainer = Trainer(logger=mock_logger)
 trainer.train_one_epoch()

 # verify that the trainer logged metrics
 mock_logger.log.assert_called()
 # verify it logged a loss value
```

```
call_args = mock_logger.log.call_args
assert "loss" in call_args[1]
```

- **When to mock:** external services (APIs, databases, cloud storage), expensive operations (GPU computation, large file I/O), and non-deterministic behaviour (random number generators, timestamps).
- **When NOT to mock:** your own code. If you mock everything, your tests verify that mocks behave as expected, not that your code works. Mock at the boundaries, test your logic directly.

## 75.4 Testing ML Code

- ML code has unique testing challenges: outputs are probabilistic, training is slow, and “correct” is fuzzy.

### 75.4.1 Deterministic Seeds

- Set random seeds everywhere to make tests reproducible:

```
import random
import numpy as np
import torch

def set_seed(seed=42):
 random.seed(seed)
 np.random.seed(seed)
 torch.manual_seed(seed)
 if torch.cuda.is_available():
 torch.cuda.manual_seed_all(seed)
 torch.backends.cudnn.deterministic = True
 torch.backends.cudnn.benchmark = False
```

### 75.4.2 Numerical Tolerances

- Floating-point comparisons require tolerances (chapter 13, IEEE 754):

```
BAD: exact comparison fails due to floating point
assert model_output == 0.5

GOOD: approximate comparison
import numpy as np
assert np.isclose(model_output, 0.5, atol=1e-5)
```

```
For tensors
assert torch.allclose(output, expected, atol=1e-4)
```

### 75.4.3 What to Test in ML

- **Shape tests:** verify that outputs have the expected dimensions.

```
def test_model_output_shape():
 model = MyModel(d_model=256, n_classes=10)
 x = torch.randn(8, 32, 256) # batch=8, seq=32, dim=256
 output = model(x)
 assert output.shape == (8, 10)
```

- **Gradient flow:** verify that gradients are non-zero for trainable parameters.

```
def test_gradients_flow():
 model = MyModel()
 x = torch.randn(4, 3, 32, 32)
 y = torch.randint(0, 10, (4,))

 output = model(x)
 loss = F.cross_entropy(output, y)
 loss.backward()

 for name, param in model.named_parameters():
 assert param.grad is not None, f"No gradient for {name}"
 assert param.grad.abs().sum() > 0, f"Zero gradient for {name}"
```

- **Overfit on one batch:** a model should be able to memorise a single batch. If it cannot, something is fundamentally wrong.

```
def test_overfit_one_batch():
 model = MyModel()
 optimiser = torch.optim.Adam(model.parameters(), lr=1e-3)
 x, y = get_single_batch()

 for _ in range(100):
 loss = F.cross_entropy(model(x), y)
 loss.backward()
 optimiser.step()
 optimiser.zero_grad()
```

```
assert loss.item() < 0.01, f"Cannot overfit one batch: loss={loss.item()}"
```

- **Data validation:** verify data loading produces valid outputs.

```
def test_dataset_basics():
 dataset = MyDataset("tests/fixtures/small_data.csv")
 assert len(dataset) > 0
 x, y = dataset[0]
 assert x.shape == (3, 224, 224)
 assert 0 <= y < 10
 assert not torch.isnan(x).any()
 assert not torch.isinf(x).any()
```

- **Determinism:** same input + same seed → same output.

```
def test_determinism():
 set_seed(42)
 output1 = model(input_data)
 set_seed(42)
 output2 = model(input_data)
 assert torch.allclose(output1, output2)
```

## 75.5 CI/CD Pipelines

- **Continuous Integration (CI):** automatically run tests on every commit or PR. If tests fail, the PR cannot be merged. This prevents broken code from reaching main.
- **GitHub Actions** example (`.github/workflows/ci.yml`):

```
name: CI
on: [push, pull_request]

jobs:
 test:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4
 - uses: actions/setup-python@v5
 with:
 python-version: "3.11"
 - run: pip install -e ".[dev]"
 - run: ruff check src/
```

```
- run: mypy src/
- run: pytest tests/ -v --tb=short
```

- **Pre-commit hooks:** run checks before each commit (locally), catching issues before they reach CI:

```
.pre-commit-config.yaml
repos:
 - repo: https://github.com/astral-sh/ruff-pre-commit
 rev: v0.3.0
 hooks:
 - id: ruff
 args: [--fix]
 - id: ruff-format
 - repo: https://github.com/pre-commit/pre-commit-hooks
 rev: v4.5.0
 hooks:
 - id: trailing-whitespace
 - id: end-of-file-fixer
 - id: check-yaml
```

```
pip install pre-commit
pre-commit install # now hooks run on every git commit
```

## 75.6 Linting and Formatting

- **Linting** catches bugs and style issues without running the code. **Formatting** enforces consistent style automatically.
- **Ruff:** a fast Python linter and formatter (replaces flake8, isort, and black in one tool):

```
ruff check src/ # lint
ruff check --fix src/ # lint and auto-fix
ruff format src/ # format
```

- **mypy:** static type checker for Python. Catches type errors before runtime:

```
mypy src/
src/model.py:42: error: Argument 1 to "forward" has incompatible type "int";
expected "Tensor"
```

- Type hints make code self-documenting and catch bugs:

```
def train(
 model: nn.Module,
 dataloader: DataLoader,
 optimiser: torch.optim.Optimizer,
 num_epochs: int = 10,
) -> float:
 """Train model and return final loss."""
 ...
```

## 75.7 Code Review Best Practices

---

- **For the author:**
  - Self-review your diff before requesting review. You will catch obvious issues.
  - Keep PRs small and focused. One concern per PR.
  - Write a clear description: what, why, how to test.
  - Respond to every comment (even if just “done” ).
- **For the reviewer:**
  - Be kind. Critique the code, not the person. “This could be clearer” not “this is confusing.”
  - Distinguish blocking issues (bugs, security) from suggestions (style, naming). Use labels: “nit:” , “suggestion:” , “blocking:” .
  - Ask questions instead of making demands. “What happens if this list is empty?” is more helpful than “handle the empty case.”
  - Approve promptly. A PR waiting days for review blocks the author and encourages large, batched PRs (which are harder to review).

## 76 Deployment and DevOps

Deployment is where your model stops being a research artifact and starts being a product. This file covers Docker for ML, model serving, experiment tracking,

reproducibility, monitoring in production, feature stores, and pipeline orchestration, the infrastructure that takes a trained model from a notebook to millions of users.

- A model that only runs on your laptop is a prototype. A model that runs reliably at scale, serves predictions in milliseconds, recovers from failures, and can be updated without downtime is a product. The gap between the two is **deployment and DevOps**.
- Most ML engineers spend more time on deployment, monitoring, and debugging production issues than on training models. Understanding this infrastructure is not optional for anyone building real ML systems.

## 76.1 Docker for ML

---

- We covered containers conceptually in chapter 13 (OS). Here we focus on the practical side: writing Dockerfiles for ML workloads.
- A **Dockerfile** is a recipe for building a container image:

```
Start from an official CUDA base image
FROM nvidia/cuda:12.1.0-cudnn8-runtime-ubuntu22.04

System dependencies
RUN apt-get update && apt-get install -y \
 python3.11 python3-pip git \
 && rm -rf /var/lib/apt/lists/*

Python dependencies (install separately for caching)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

Copy source code (changes frequently, so this layer is last)
COPY src/ /app/src/
COPY configs/ /app/configs/
WORKDIR /app

Entry point
CMD ["python3", "src/scripts/serve.py", "--config", "configs/serve.yaml"]
```

- **Layer caching:** Docker caches each layer. If `requirements.txt` has not changed, `pip install` is skipped on rebuild. Put rarely-changing layers (system packages, `pip install`) before frequently-changing ones (source code). This turns a 10-minute build into a 10-second rebuild.



- **GPU access:** use nvidia/cuda base images and run with `docker run --gpus all`. The nvidia-container-toolkit provides GPU passthrough from host to container.
- **Multi-stage builds** reduce image size by separating the build environment from the runtime:

```
Build stage: install build tools, compile dependencies
FROM python:3.11 AS builder
COPY requirements.txt .
RUN pip install --user -r requirements.txt

Runtime stage: only runtime dependencies
FROM nvidia/cuda:12.1.0-cudnn8-runtime-ubuntu22.04
COPY --from=builder /root/.local /root/.local
COPY src/ /app/src/
ENV PATH=/root/.local/bin:$PATH
```

- The final image contains only the runtime libraries, not compilers, headers, or build tools. A 5 GB build image becomes a 2 GB runtime image.
- **Docker Compose** runs multi-container setups (model server + load balancer + monitoring):

```
docker-compose.yml
services:
 model:
 build: .
 ports:
 - "8080:8080"
 deploy:
 resources:
 reservations:
 devices:
 - capabilities: [gpu]
 prometheus:
 image: prom/prometheus
 ports:
 - "9090:9090"
```

## 76.2 Model Serving

---

- **Model serving** is running inference as a service: receive requests, run the model, return predictions.

- **FastAPI** (covered in file 03) is the simplest approach for low-to-medium throughput. For high throughput and GPU-optimised serving, use dedicated tools:
- **Triton Inference Server** (NVIDIA): serves models in TensorRT, ONNX, PyTorch, and TensorFlow formats. Features:
  - **Dynamic batching**: collects individual requests and batches them for GPU efficiency. A stream of single requests is batched into groups of 32, dramatically improving throughput.
  - **Model ensembles**: chain multiple models (preprocessor → model → postprocessor) in a single request.
  - **Multi-model serving**: serve multiple models on the same GPU, sharing resources.
  - **Concurrent model execution**: run multiple inference requests in parallel on the same GPU.
- **TorchServe** (PyTorch): serves PyTorch models with a REST/gRPC API. Supports model versioning, A/B testing, and custom handlers.
- **vLLM**: specialised for LLM serving. Implements PagedAttention (efficient KV cache management), continuous batching, and tensor parallelism across GPUs. Achieves 10-20x higher throughput than naive serving for large language models.
- **Cactus** ([github.com/cactus-compute/cactus](https://github.com/cactus-compute/cactus)): a low-latency AI engine for on-device serving on mobile and edge. Cactus provides an **OpenAI-compatible API** (chat completion, streaming, tool calling, transcription, embeddings, RAG, vision) that runs entirely on-device, with automatic **cloud fallback** when the local model cannot handle a request. This hybrid architecture means your application code uses the same API regardless of whether inference runs locally or in the cloud — the engine decides based on model confidence and device capability. SDKs available for Python, Swift, Kotlin, Flutter, React Native, and Rust, with pre-converted model weights on HuggingFace. Supports multimodal inference (LLMs, vision, speech) with custom ARM SIMD kernels for fastest inference on ARM CPUs and zero-copy memory mapping for 10x lower RAM usage (chapter 16, chapter 17).

- **Model format optimisation:**
  - **ONNX:** open format for interoperability. Export from PyTorch/ TensorFlow, run anywhere.
  - **TensorRT:** NVIDIA's optimiser. Fuses layers, selects optimal kernels, quantises weights. Typically 2-5x faster than PyTorch on NVIDIA GPUs.
  - **GGUF/GGML:** formats for CPU-efficient inference, popular for running LLMs on consumer hardware.

## 76.3 Experiment Tracking

---

- Without experiment tracking, ML research devolves into: “I think the model from last Tuesday with that config I changed something in was the best one, but I do not remember what I changed.”
- **Weights & Biases (W&B):** the most popular experiment tracker. Log anything from your training script:

```
import wandb

wandb.init(project="my-project", config={
 "model": "transformer",
 "lr": 3e-4,
 "batch_size": 64,
})

for epoch in range(num_epochs):
 train_loss = train_one_epoch()
 val_loss = validate()

 wandb.log({
 "train/loss": train_loss,
 "val/loss": val_loss,
 "epoch": epoch,
 })

 # Log model as artifact
 if val_loss < best_loss:
 wandb.save("best_model.pt")
```

```
wandb.finish()
```

- W&B provides: dashboards for comparing runs, hyperparameter sweep tools, model registry, dataset versioning, and team collaboration.
- **MLflow**: open-source alternative. Runs locally or on a server:

```
import mlflow

mlflow.set_experiment("my-experiment")

with mlflow.start_run():
 mlflow.log_params({"lr": 3e-4, "batch_size": 64})
 mlflow.log_metric("val_loss", 0.042, step=epoch)
 mlflow.pytorch.log_model(model, "model")
```

- **Model registry**: a central store of trained models with versioning, staging (dev → staging → production), and metadata. Both W&B and MLflow provide registries. The registry answers: “what model is currently in production, who trained it, what was its validation accuracy, and what code/data produced it?”

## 76.4 Reproducibility

- Reproducibility means: given the same code, data, and config, produce the same model. This is surprisingly hard in ML due to non-determinism in GPU operations, data shuffling, and floating-point accumulation.
- **The reproducibility checklist:**

What	How
Code version	Git commit hash
Config / hyperparameters	Config file (versioned in git or logged to W&B)
Random seeds	Set and log all seeds (Python, NumPy, PyTorch, CUDA)
Data version	DVC hash, dataset version tag, or S3 object version

What	How
Dependencies	<code>pip freeze</code> , Docker image hash, or lockfile
Hardware	GPU type, number of GPUs, CUDA version
Non-determinism	<code>torch.backends.cudnn.deterministic = True</code> (slower but reproducible)

- **Pinning everything:** `pip install torch==2.2.1` not `torch>=2.0`. A minor version bump can change numerical behaviour, optimiser implementations, or default hyperparameters.
- **Docker for reproducibility:** the Docker image pins the OS, system libraries, Python version, and pip packages. The image hash is a complete environment fingerprint. If you can reproduce the Docker image, you can reproduce the training.

## 76.5 Monitoring in Production

---

- Deploying a model is not the end — it is the beginning of a new set of problems. Models degrade over time as the real world changes (**concept drift**) and as the input data distribution shifts (**data drift**).
- **What to monitor:**
  - **Latency:** how long does inference take? Track p50 (median), p95, and p99. A p99 of 500ms means 1 in 100 users waits half a second, which may be unacceptable.
  - **Throughput:** how many requests per second? Is the system keeping up with demand?
  - **Error rate:** what fraction of requests fail (exceptions, timeouts, invalid inputs)?
  - **Model metrics:** accuracy, precision, recall on a holdout set. If labelled data is available in production (e.g., user corrections), track online metrics.

- **Data drift:** has the distribution of incoming data changed? A model trained on daytime photos may fail on night photos. Statistical tests (KS test, PSI) compare the training distribution to the live distribution.
- **Feature drift:** have individual feature distributions changed? A feature that was normally distributed during training but is now bimodal signals a data pipeline issue.
- **Tools:**
  - **Prometheus + Grafana:** the standard for infrastructure monitoring. Prometheus collects metrics, Grafana visualises them in dashboards with alerts.
  - **Evidently AI:** open-source ML monitoring. Generates reports on data drift, model performance, and data quality.
- **Alerts:** do not just dashboard it — set up automated alerts. “If p99 latency exceeds 200ms for 5 minutes, send a Slack notification.” “If data drift score exceeds threshold, page the on-call engineer.”

## 76.6 Feature Stores

---

- A **feature store** is a centralised repository of precomputed features, shared between training and serving. It solves two problems:
  - **Training-serving skew:** the features used during training must be identical to those used during serving. If training uses `user_age_at_signup` computed one way and serving computes it differently, the model’s predictions are silently wrong.
  - **Feature reuse:** multiple models often use the same features (user demographics, item embeddings, aggregated statistics). Computing them once and sharing avoids duplication and inconsistency.
- **Feast** is the most popular open-source feature store. It manages online features (low-latency, served from Redis or DynamoDB) and offline features (batch, stored in data warehouses for training).
- Feature stores are critical for recommendation systems, fraud detection, and any application where features are computed from raw data pipelines.

## 76.7 Pipeline Orchestration

---

- A production ML system is not just a model. It is a **pipeline**: data ingestion → preprocessing → feature computation → training → evaluation → deployment → monitoring. Each step depends on the previous one, can fail independently, and may need to run on different schedules.
- **Orchestrators** manage these pipelines:
- **Apache Airflow**: the standard for data pipeline orchestration. DAGs (Directed Acyclic Graphs) define task dependencies. Each task runs independently, can be retried on failure, and is monitored via a web UI.

```
airflow DAG example (simplified)
from airflow import DAG
from airflow.operators.python import PythonOperator

dag = DAG("training_pipeline", schedule="@daily")

preprocess = PythonOperator(task_id="preprocess",
python_callable=preprocess_data, dag=dag)
train = PythonOperator(task_id="train", python_callable=train_model, dag=dag)
evaluate = PythonOperator(task_id="evaluate", python_callable=evaluate_model,
dag=dag)
deploy = PythonOperator(task_id="deploy", python_callable=deploy_model, dag=dag)

preprocess >> train >> evaluate >> deploy
```

- **Kubeflow Pipelines**: ML-specific orchestration on Kubernetes. Each step runs in a container, GPU resources are allocated on demand, and experiments are tracked automatically.
- **Prefect** and **Dagster**: modern alternatives to Airflow with better developer experience, native Python APIs, and built-in data lineage.
- **When to orchestrate**: when your pipeline has more than 2-3 steps, runs on a schedule, involves multiple teams or services, or needs automatic recovery from failures. A single-script training job does not need an orchestrator. A daily retraining pipeline that ingests data from 5 sources, trains 3 models, evaluates them, and deploys the best one absolutely does.

# 77 Why C++ and How ML Frameworks Work

Every `jnp.matmul`, every `torch.nn.Linear`, every `np.dot` call in this book has been executing C++ and CUDA code underneath. This file pulls back the curtain: why ML frameworks are built this way, quick C++ fundamentals for Python engineers, when to write custom C++ kernels, and how to bind them into Python, the bridge between the code you write and the hardware it runs on.

- You have spent 15 chapters writing Python. You imported JAX, called `jax.grad`, ran training loops, and built models. It all felt like Python. But here is the truth: **almost none of the actual computation happened in Python.**
- When you write `output = model(input)` in PyTorch or `output = jnp.matmul(W, x)` in JAX, Python does almost nothing. It constructs a description of the computation (a graph of operations), then hands it off to a C++/CUDA backend that does the real work. Python is the steering wheel; C++ is the engine.

## 77.1 Why Python Frontend, C++ Backend

- This two-language architecture exists because Python and C++ are good at opposite things:

	Python	C++
Development speed	Fast (dynamic typing, REPL, no compilation)	Slow (static typing, headers, compile times)
Execution speed	~100x slower than C (interpreted, GIL)	Near-hardware speed (compiled, no overhead)
Memory control	Automatic (GC), no control over layout	Manual, precise control over every byte
Hardware access	None (no SIMD, no GPU, no custom memory)	Full (intrinsics, CUDA, inline assembly)



	Python	C++
Ecosystem	Rich for ML (notebooks, visualisation, data)	Rich for systems (OS, drivers, engines)

- The insight: **use each language for what it is good at**. Python handles the parts where human productivity matters (experiment design, hyperparameter tuning, data exploration). C++ handles the parts where machine performance matters (matrix multiplication, convolution, attention kernels).
- A single matrix multiplication `np.matmul(A, B)` where  $A$  is  $4096 \times 4096$  performs ~137 billion floating-point operations. In pure Python (nested loops), this takes ~30 minutes. In optimised C++ with AVX-512 SIMD and multithreading, it takes ~10 milliseconds. That is a **180,000x** difference. No amount of Python cleverness closes this gap.

## 77.2 How ML Frameworks Are Structured

- Every major ML framework follows the same architecture:

```

User code (Python)
 ↓
Python API layer (torch.nn, jax.numpy, numpy)
 ↓
Dispatch / JIT compiler (torch.compile, XLA, NumPy dispatch)
 ↓
C++ kernel library (ATen/PyTorch, XLA, BLAS/LAPACK)
 ↓
Hardware-specific backends (CUDA, cuDNN, MKL, oneDNN, Metal)
 ↓
Hardware (CPU SIMD units, GPU cores, TPU MXUs)

```

### 77.2.1 NumPy

- NumPy's core is written in C. When you call `np.dot(A, B)`, Python calls a C function that calls BLAS (Basic Linear Algebra Subprograms), typically Intel MKL or OpenBLAS. BLAS is hand-optimised C and Fortran code that uses SIMD instructions, cache-aware memory access patterns, and multithreading. Decades of optimisation went into making matrix multiplication fast.
- NumPy is CPU-only. It does not use GPUs. But on CPU, it is extremely fast because it delegates to the best available BLAS implementation.

## 77.2.2 PyTorch

- PyTorch's computation engine is **ATen** (A Tensor Library), written in C++. ATen implements ~2000 tensor operations (add, matmul, conv2d, softmax, ...), each with CPU and CUDA backends.
- When you call `torch.matmul(A, B)`:
  1. Python dispatches to the ATen C++ function.
  2. ATen checks the device (CPU or CUDA) and dtype.
  3. On CPU: calls MKL/OpenBLAS. On GPU: calls cuBLAS (NVIDIA's GPU-optimised BLAS).
  4. The result is wrapped in a Python tensor object and returned.
- **torch.compile** (PyTorch 2.0+) takes this further: it traces your Python code, builds a computation graph, and compiles it using **Triton** (for GPU) or **C++/OpenMP** (for CPU). The compiled code fuses operations, eliminates Python overhead, and can be 2-5x faster than eager mode.

## 77.2.3 JAX

- JAX compiles Python functions to **XLA** (Accelerated Linear Algebra), Google's compiler for ML workloads. When you `jax.jit` a function:
  1. JAX traces the function, capturing the operations as an XLA computation graph (HLO — High Level Operations).
  2. XLA optimises the graph: fuses operations, eliminates redundant computation, optimises memory layout.
  3. XLA compiles to the target backend: CPU (via LLVM), GPU (via CUDA/PTX), or TPU (via TPU-specific instructions).
  4. The compiled code runs directly on hardware with zero Python involvement.
- This is why `jax.jit` is so important: without it, every operation is a separate Python→C++ round trip. With it, the entire function is a single compiled kernel.

## 77.3 Quick C++ Fundamentals for Python Engineers

---

- You do not need to become a C++ expert. You need to understand enough to read kernel code, write simple extensions, and understand performance discussions. Here are the essentials.

### 77.3.1 Types and Variables

```
// C++ requires explicit types (unlike Python)
int count = 0; // 32-bit integer
float loss = 0.5f; // 32-bit float
double lr = 3e-4; // 64-bit float
bool training = true; // boolean

// Arrays (fixed size, stack-allocated)
float weights[1024]; // 1024 floats, contiguous in memory

// Pointers: a variable that holds a memory address
float* ptr = weights; // ptr points to the first element of weights
float val = ptr[42]; // access element 42 via pointer arithmetic
// ptr[42] is equivalent to *(ptr + 42)
```

- **Pointers** are the biggest conceptual difference from Python. In Python, everything is a reference and you never think about memory addresses. In C++, pointers give you direct access to memory — powerful but dangerous (dangling pointers, buffer overflows).

### 77.3.2 Functions

```
// Function declaration: return_type name(param_type param_name)
float relu(float x) {
 return x > 0.0f ? x : 0.0f;
}

// Passing by reference (avoids copying large objects)
void scale_vector(std::vector<float>& vec, float factor) {
 for (size_t i = 0; i < vec.size(); i++) {
 vec[i] *= factor;
 }
}

// const reference: read-only, no copy
float sum(const std::vector<float>& vec) {
 float total = 0.0f;
 for (float x : vec) { // range-based for loop (like Python's for x in vec)
 total += x;
 }
 return total;
}
```

### 77.3.3 Memory: Stack vs Heap

```
// Stack allocation: fast, automatic lifetime (freed when function returns)
float buffer[256]; // 256 floats on the stack

// Heap allocation: manual, survives beyond the function
float* data = new float[n]; // allocate n floats on the heap
// ... use data ...
delete[] data; // YOU must free it (no garbage collector)

// Modern C++: smart pointers (automatic cleanup, like Python references)
#include <memory>
auto data = std::make_unique<float[]>(n); // freed automatically when out of
scope
```

- **The key rule:** stack is fast but limited (typically 1-8 MB). Large arrays (tensors, feature maps) must go on the heap. In Python, everything is on the heap and the GC handles cleanup. In C++, you manage it yourself (or use smart pointers).

### 77.3.4 Templates (Generics)

```
// A function that works with any numeric type
template <typename T>
T add(T a, T b) {
 return a + b;
}

add<float>(1.5f, 2.5f); // returns 4.0f
add<int>(3, 4); // returns 7
```

- Templates are how C++ libraries (like ATen) write code that works with float16, float32, float64, etc. without duplicating the implementation.

### 77.3.5 The Standard Library Essentials

```
#include <vector> // dynamic array (like Python list)
#include <string> // string type
#include <unordered_map> // hash map (like Python dict)
#include <algorithm> // sort, find, transform, etc.
#include <cmath> // math functions

std::vector<float> vec = {1.0f, 2.0f, 3.0f};
vec.push_back(4.0f); // append
float first = vec[0]; // index
```

```

size_t len = vec.size(); // length

std::unordered_map<std::string, int> counts;
counts["hello"] = 5; // insert
if (counts.count("hello")) { } // check existence

```

## 77.4 When to Write Custom C++ Kernels

- Most ML engineers never need to write C++. The framework's built-in operations cover 99% of use cases. You should consider custom C++ only when:
  1. **Your operation does not exist in the framework:** a novel activation function, a custom attention pattern, a specialised loss function that cannot be expressed as a composition of existing ops.
  2. **Fusing operations for performance:** your model does `relu(layer_norm(matmul(x, W) + b))`. Each operation launches a separate kernel, reads and writes memory, and synchronises. A fused kernel does it all in one pass, avoiding memory round-trips. This can be 2-5x faster.
  3. **Reducing memory usage:** a custom kernel can compute gradients without storing all intermediate activations (gradient checkpointing at the kernel level).
  4. **Targeting novel hardware:** a new accelerator (e.g., Cerebras, Groq) may not have framework support. You write kernels directly.
- For cases 1-2, **Triton** (chapter 16, file 05) is often sufficient and much easier than writing CUDA C directly. Only drop to CUDA C when Triton cannot express what you need.

## 77.5 How to Bind C++ to Python

- Writing C++ is half the job. You also need to call it from Python.

### 77.5.1 pybind11 (General Purpose)

- pybind11 creates Python bindings for C++ functions with minimal boilerplate:

```

// my_ops.cpp
#include <pybind11/pybind11.h>
#include <pybind11/numpy.h>
namespace py = pybind11;

```

```

// A simple custom operation
py::array_t<float> custom_relu(py::array_t<float> input) {
 auto buf = input.request();
 float* ptr = static_cast<float*>(buf.ptr);
 size_t n = buf.size;

 auto result = py::array_t<float>(n);
 float* out = static_cast<float*>(result.request().ptr);

 for (size_t i = 0; i < n; i++) {
 out[i] = ptr[i] > 0 ? ptr[i] : 0;
 }
 return result;
}

PYBIND11_MODULE(my_ops, m) {
 m.def("custom_relu", &custom_relu, "Custom ReLU operation");
}

```

```

Compile
pip install pybind11
c++ -O3 -shared -std=c++17 -fPIC $(python3 -m pybind11 --includes) my_ops.cpp -o
my_ops$(python3-config --extension-suffix)

```

```

Use from Python
import my_ops
import numpy as np

x = np.array([-1.0, 2.0, -3.0, 4.0], dtype=np.float32)
y = my_ops.custom_relu(x)
print(y) # [0. 2. 0. 4.]

```

## 77.5.2 PyTorch C++ Extensions

- PyTorch provides a streamlined way to add custom ops:

```

// custom_op.cpp
#include <torch/extension.h>

torch::Tensor custom_gelu(torch::Tensor x) {
 return x * 0.5 * (1.0 + torch::erf(x / std::sqrt(2.0)));
}

PYBIND11_MODULE(TORCH_EXTENSION_NAME, m) {

```

```
m.def("custom_gelu", &custom_gelu, "Custom GELU activation");
}
```

```
Load and compile on-the-fly
from torch.utils.cpp_extension import load

custom_ops = load(
 name="custom_ops",
 sources=["custom_op.cpp"],
 extra_cflags=["-O3"],
)

x = torch.randn(1000)
y = custom_ops.custom_gelu(x)
```

- `torch.utils.cpp_extension.load` compiles the C++ code, creates a shared library, and loads it as a Python module, all in one call. This is the easiest way to experiment with custom C++ ops in PyTorch.

### 77.5.3 JAX Custom Calls

- JAX uses XLA custom calls. The process is more involved (you register a C function with XLA), but the concept is the same: write C/C++, bind it, call it from Python.
- For most JAX users, **Pallas** (covered in file 05) is the better choice: it lets you write GPU kernels in a Python-like syntax that XLA compiles, without leaving the JAX ecosystem.

## 77.6 The Big Picture

---

- This file explained the layer between Python and hardware. The remaining files in this chapter go deeper:
  - **File 01:** the hardware itself (CPU architecture, GPU architecture, memory systems)
  - **Files 02-03:** SIMD programming on CPU (ARM NEON, x86 AVX) — where you write C++ that uses the CPU's vector units
  - **File 04:** GPU programming with CUDA — where you write C++ that runs on thousands of GPU cores
  - **File 05:** Triton, Pallas, and higher-level GPU programming — where you write Python that compiles to GPU kernels

- The progression mirrors the abstraction ladder: C++ intrinsics (lowest, most control) → CUDA (GPU-specific) → Triton/Pallas (Pythonic, compiled) → JAX/PyTorch (highest, automatic). Each level trades control for convenience. Understanding the lower levels makes you a better user of the higher ones.

## 77.7 Coding Tasks (compile with g++ or clang++)

1. Write your first C++ program. Allocate an array, fill it, compute the sum, and measure the time. This introduces compilation, arrays, pointers, and timing.

```
// task1_basics.cpp
// Compile: g++ -O3 -o task1 task1_basics.cpp
// Run: ./task1

#include <iostream>
#include <chrono>
#include <vector>

int main() {
 const int N = 10'000'000; // C++ allows ' as digit separator
 std::vector<float> data(N);

 // Fill the array
 for (int i = 0; i < N; i++) {
 data[i] = static_cast<float>(i) * 0.001f;
 }

 // Compute sum
 auto start = std::chrono::high_resolution_clock::now();
 float sum = 0.0f;
 for (int i = 0; i < N; i++) {
 sum += data[i];
 }
 auto end = std::chrono::high_resolution_clock::now();
 double elapsed = std::chrono::duration<double, std::milli>(end -
start).count();

 std::cout << "Sum: " << sum << std::endl;
 std::cout << "Time: " << elapsed << " ms" << std::endl;
 std::cout << "Elements: " << N << std::endl;
 std::cout << "Throughput: " << (N * sizeof(float)) / elapsed / 1e6 << " GB/
s" << std::endl;
```



```

 return 0;
}

```

1. Write a C++ function that computes ReLU on an array, then build a Python binding using pybind11. Call it from Python and compare speed against NumPy.

```

// task2_relu.cpp
// Compile: c++ -O3 -shared -std=c++17 -fPIC $(python3 -m pybind11 --includes) \
// task2_relu.cpp -o my_relu$(python3-config --extension-suffix)

#include <pybind11/pybind11.h>
#include <pybind11/numpy.h>
namespace py = pybind11;

py::array_t<float> cpp_relu(py::array_t<float> input) {
 auto buf = input.request();
 float* ptr = static_cast<float*>(buf.ptr);
 int n = buf.size;

 auto result = py::array_t<float>(n);
 float* out = static_cast<float*>(result.request().ptr);

 for (int i = 0; i < n; i++) {
 out[i] = ptr[i] > 0.0f ? ptr[i] : 0.0f;
 }
 return result;
}

PYBIND11_MODULE(my_relu, m) {
 m.def("relu", &cpp_relu, "C++ ReLU");
}

```

```

test_relu.py – run after compiling the C++ module above
import numpy as np
import time
import my_relu # the compiled C++ module

x = np.random.randn(10_000_000).astype(np.float32)

C++ ReLU
start = time.time()
for _ in range(100):
 y_cpp = my_relu.relu(x)
cpp_time = (time.time() - start) / 100

NumPy ReLU

```

```

start = time.time()
for _ in range(100):
 y_np = np.maximum(x, 0)
np_time = (time.time() - start) / 100

print(f"C++ ReLU: {cpp_time*1000:.2f} ms")
print(f"NumPy ReLU: {np_time*1000:.2f} ms")
print(f"Match: {np.allclose(y_cpp, y_np)}")

```

1. Write a C++ program that demonstrates why memory layout matters. Compare row-major vs column-major access patterns and measure the performance difference.

```

// task3_layout.cpp
// Compile: g++ -O3 -o task3 task3_layout.cpp

#include <iostream>
#include <chrono>
#include <vector>

int main() {
 const int N = 4096;
 std::vector<float> matrix(N * N, 1.0f);

 // Row-major access: sequential memory addresses (cache-friendly)
 auto start = std::chrono::high_resolution_clock::now();
 float sum_row = 0.0f;
 for (int i = 0; i < N; i++) {
 for (int j = 0; j < N; j++) {
 sum_row += matrix[i * N + j]; // stride-1 access
 }
 }
 auto end = std::chrono::high_resolution_clock::now();
 double row_ms = std::chrono::duration<double, std::milli>(end -
start).count();

 // Column-major access: stride-N access (cache-unfriendly)
 start = std::chrono::high_resolution_clock::now();
 float sum_col = 0.0f;
 for (int j = 0; j < N; j++) {
 for (int i = 0; i < N; i++) {
 sum_col += matrix[i * N + j]; // stride-N access (cache misses!)
 }
 }
 end = std::chrono::high_resolution_clock::now();
 double col_ms = std::chrono::duration<double, std::milli>(end -

```

```

start).count();

 std::cout << "Row-major (cache-friendly): " << row_ms << " ms" << std::endl;
 std::cout << "Col-major (cache-hostile): " << col_ms << " ms" << std::endl;
 std::cout << "Slowdown: " << col_ms / row_ms << "x" << std::endl;
 std::cout << "(Both sums: " << sum_row << ", " << sum_col << ")" <<
std::endl;

 return 0;
}

```

## 78 Hardware Fundamentals

Before writing SIMD or GPU code, you need to understand the hardware you are programming. This file covers why parallelism replaced clock speed, how modern CPUs execute instructions, what SIMD is, the roofline model for reasoning about performance, and the landscape of chip architectures

- For decades, software got faster for free: buy a new CPU with a higher clock speed, and your program runs faster without changing a line of code. That era ended around 2005. Understanding why it ended, and what replaced it, is essential for anyone who wants to write fast code.

### 78.1 The End of Free Performance

- **Moore's Law** (1965) observed that the number of transistors on a chip doubles roughly every two years. This held for 60 years. More transistors meant smaller transistors, which meant higher clock speeds, which meant faster programs.
- But around 2005, clock speeds hit a wall at ~4 GHz. The problem is **power**. The power consumed by a chip is approximately:

$$P \propto C \cdot V^2 \cdot f$$

- where C is capacitance (proportional to transistor count), V is voltage, and f is clock frequency. To increase frequency, you must increase voltage (to switch transistors faster). But power scales with  $V^2 \cdot f$ , so a small increase in frequency causes a large increase in power (and heat). At 4 GHz, chips were already hitting 100+ watts. Going to 8 GHz would require impractical cooling.

- The solution: instead of making one core faster, put **multiple cores** on the same chip. A 4-core chip at 3 GHz uses similar power to a single core at 4.5 GHz but can do 4x the parallel work. This is why every modern CPU has multiple cores, and why parallelism (SIMD, multithreading, GPU computing) is the only path to more performance.
- **Implication for ML:** a training step that takes 10 minutes on one core cannot be made faster by buying a faster CPU. It can only be made faster by using more cores (data parallelism, chapter 6), wider SIMD units (this chapter), or GPUs (thousands of cores).

## 78.2 How Modern CPUs Execute Instructions

---

- A modern CPU core is far more complex than the simple fetch-decode-execute model from chapter 13. It uses several tricks to execute more instructions per cycle:
- **Superscalar execution:** the CPU has multiple execution units (ALUs, FPUs, load/store units) and can execute multiple independent instructions simultaneously. A modern core might execute 4-6 instructions per cycle if they do not depend on each other.
- **Out-of-order execution (OoO):** the CPU does not execute instructions in program order. It looks ahead in the instruction stream, finds instructions whose inputs are ready, and executes them immediately, regardless of their position. This hides latency: while one instruction waits for data from memory (100+ cycles), the CPU executes other instructions that are ready.
- **Branch prediction:** conditional branches (if statements, loop conditions) create uncertainty: the CPU does not know which path to take until the condition is evaluated. Rather than stall, the CPU **predicts** the outcome and speculatively executes down the predicted path. If the prediction is correct (>95% of the time with modern predictors), no time is lost. If wrong, the speculative work is discarded and the correct path is executed (~15 cycle penalty).
- **Speculative execution:** an extension of branch prediction. The CPU executes instructions that might not be needed, betting that they will be. This fills the pipeline and keeps execution units busy.

- All of these are automatic — the CPU does them without any programmer intervention. But they only help with **instruction-level parallelism** (ILP): independent instructions within a single stream. For **data-level parallelism** (the same operation on many data elements), we need SIMD.

## 78.3 SIMD: Single Instruction, Multiple Data

- **SIMD** is the idea of applying one instruction to multiple data elements simultaneously. Instead of adding two numbers, add two vectors of 4 (or 8, or 16) numbers in a single instruction.
- Without SIMD (scalar):

```
// Add two arrays element by element: 4 add instructions
for (int i = 0; i < 4; i++) {
 c[i] = a[i] + b[i]; // one add per iteration
}
```

- With SIMD (vectorised):

```
// Add two arrays: 1 SIMD instruction does all 4 adds
#include <immintrin.h> // x86 SIMD intrinsics

__m128 va = _mm_load_ps(a); // load 4 floats into a 128-bit register
__m128 vb = _mm_load_ps(b); // load 4 floats into another register
__m128 vc = _mm_add_ps(va, vb); // add all 4 pairs simultaneously
_mm_store_ps(c, vc); // store 4 results
```

- The SIMD version does the same work in 1/4 of the instructions. This is a theoretical 4x speedup, achieved by processing 4 floats per instruction instead of 1.

### 78.3.1 Vector Registers

- SIMD instructions operate on **vector registers**: wide registers that hold multiple data elements.

Register Width	Floats (32-bit)	Doubles (64-bit)	Name
128-bit	4	2	SSE (x86), NEON (ARM)
256-bit	8	4	AVX/AVX2 (x86)

Register Width	Floats (32-bit)	Doubles (64-bit)	Name
512-bit	16	8	AVX-512 (x86)
Variable (128-2048)	varies	varies	SVE/SVE2 (ARM)

- Wider registers = more parallelism. A 512-bit AVX-512 instruction processes 16 floats at once, a theoretical 16x speedup over scalar code. In practice, the speedup is lower due to memory bandwidth limitations (you can compute faster than you can feed data to the CPU).
- For ML: a matrix multiplication of float32 values benefits enormously from SIMD. The inner loop (dot product of two vectors) maps directly to SIMD multiply-accumulate instructions. This is why BLAS libraries (which NumPy and PyTorch call) are so heavily optimised with SIMD.

## 78.4 The Roofline Model

---

- How do you know if your code is fast? The **roofline model** provides a framework by characterising performance in terms of two hardware limits:
  1. **Peak compute** (FLOPS): the maximum floating-point operations per second. For a 4 GHz CPU with 256-bit AVX (8 floats per instruction) and 2 FMA units:  $4 \times 10^9 \times 8 \times 2 = 64 \text{ GFLOPS}$ .
  2. **Peak memory bandwidth** (bytes/second): how fast data can be moved from memory to the CPU. A modern CPU might have 50 GB/s of memory bandwidth.
- The **arithmetic intensity** of your code is the ratio of computation to memory access:

$$\text{Arithmetic Intensity} = \frac{\text{FLOPS}}{\text{Bytes transferred}}$$

- If arithmetic intensity is low (few operations per byte loaded), your code is **memory-bound**: it spends most of its time waiting for data. Making compute faster (wider SIMD, higher clock) will not help.
- If arithmetic intensity is high (many operations per byte), your code is **compute-bound**: it spends most of its time computing. Faster memory will not help.

- The roofline:

$$\text{Achievable FLOPS} = \min(\text{Peak FLOPS}, \text{Bandwidth} \times \text{Arithmetic Intensity})$$

- **Matrix multiplication** has high arithmetic intensity:  $O(n^3)$  operations on  $O(n^2)$  data, so intensity  $\approx O(n)$ . For large matrices, it is compute-bound. This is why GPUs (high compute) dominate matrix-heavy ML workloads.
- **Element-wise operations** (ReLU, add, multiply) have low arithmetic intensity: 1 operation per element loaded. These are memory-bound. Making the GPU faster does not help; you need faster memory (or fuse these operations with compute-heavy ones to avoid separate memory round-trips).
- The roofline model explains why **kernel fusion** matters so much: combining a matmul with a bias add and ReLU into a single kernel avoids writing intermediate results to memory and reading them back, turning three memory-bound operations into one compute-bound operation.

## 78.5 Latency vs Throughput

---

- **Latency** is the time to complete one operation. **Throughput** is the number of operations completed per unit time.
- An analogy: a bus has high latency (waits at every stop) but high throughput (carries 50 people at once). A taxi has low latency (goes directly to your destination) but low throughput (carries 1-4 people).
- GPUs are buses: high latency per operation (each instruction takes many cycles to complete) but enormous throughput (thousands of cores processing simultaneously). CPUs are taxis: low latency (out-of-order execution, branch prediction, deep caches minimise delays) but limited throughput (4-64 cores).
- This is why GPUs are better for ML training (throughput matters: process millions of examples) and CPUs are better for OS tasks (latency matters: respond to a keypress immediately).
- **Pipelining** converts latency into throughput. If an instruction takes 5 cycles but the pipeline starts a new instruction every cycle, the throughput is 1 instruction per cycle (even though each takes 5 cycles to complete). This is the same principle as the CPU pipeline from chapter 13, but it applies at every level: SIMD units, memory controllers, and GPU cores are all pipelined.

## 78.6 Chip Architecture Landscape

---

- The hardware you write code for determines which SIMD instructions are available:

### 78.6.1 x86 (Intel, AMD)

- Dominates desktops, laptops, and data centre CPUs. SIMD: SSE (128-bit), AVX/AVX2 (256-bit), AVX-512 (512-bit). Intel AMX provides dedicated matrix multiply units for AI workloads.
- **Strengths:** highest single-core performance, widest SIMD, mature software ecosystem (MKL, oneDNN).
- **Weaknesses:** high power consumption, complex instruction set, expensive.

### 78.6.2 ARM

- Dominates mobile (every smartphone), growing in servers (AWS Graviton, Ampere Altra) and laptops (Apple M-series). SIMD: NEON (128-bit), SVE/SVE2 (scalable, 128-2048 bit).
- **Strengths:** excellent power efficiency (performance per watt), custom cores (Apple M4 rivals Intel in single-core performance at fraction of the power).
- **Weaknesses:** narrower SIMD (NEON is only 128-bit, though SVE can be wider), smaller software ecosystem for HPC.

### 78.6.3 Apple Silicon (M1/M2/M3/M4)

- ARM-based with custom additions. Includes **AMX** (Apple Matrix eXtensions) — undocumented matrix multiply units that Accelerate framework uses for BLAS operations. Unified memory architecture: CPU and GPU share the same physical memory, eliminating the CPU↔GPU copy bottleneck.
- **For ML:** Apple's Neural Engine (16-core, dedicated ML accelerator) and unified memory make M-series chips surprisingly capable for local ML inference and small-scale training. No CUDA, though: you must use Metal (Apple's GPU API) or MLX (Apple's ML framework).



#### 78.6.4 RISC-V

- Open-source ISA. No licensing fees (unlike ARM). Growing in embedded systems, IoT, and research. SIMD: the “V” (vector) extension provides scalable vector processing similar to ARM SVE.
- **For ML:** not yet competitive with x86/ARM for ML workloads, but watch this space. Several AI accelerator startups use RISC-V cores.

#### 78.6.5 GPUs (NVIDIA, AMD, Intel)

- Covered in depth in files 04-05. Thousands of simple cores optimised for throughput. NVIDIA dominates ML with CUDA; AMD competes with ROCm; Intel enters with Arc GPUs and Gaudi accelerators.

#### 78.6.6 TPUs (Google)

- Custom ASIC designed specifically for ML. Systolic arrays optimised for matrix multiplication. Covered in file 05.

### 78.7 Thermal and Power Constraints

---

- Performance is ultimately limited by power and cooling:
- **TDP** (Thermal Design Power): the maximum sustained power a chip can consume. A laptop CPU might have a 15W TDP; a server CPU 250W; a data centre GPU 700W (NVIDIA B200).
- **Dark silicon:** at any given moment, a significant fraction of transistors must be powered off to stay within the thermal budget. A chip could theoretically use all its transistors simultaneously, but it would melt.
- **Power efficiency** (FLOPS/watt) is increasingly the metric that matters, not raw FLOPS. This is why:
  - ARM is taking over data centres (better FLOPS/watt than x86).
  - TPUs compete with GPUs despite lower peak FLOPS (much better FLOPS/watt for ML workloads).
  - Quantisation (INT8, FP8) is not just about memory: it also reduces power per operation.

- For ML at scale: training a frontier LLM consumes megawatts of power for months. The electricity cost can exceed the hardware cost. Power efficiency directly impacts the economics of AI research.

## 78.8 Practical: Measuring Performance in C++

---

- To reason about performance, you need to measure it. Here is a minimal C++ benchmarking setup:

```
#include <iostream>
#include <chrono>
#include <vector>

// Scalar addition
void add_scalar(const float* a, const float* b, float* c, int n) {
 for (int i = 0; i < n; i++) {
 c[i] = a[i] + b[i];
 }
}

int main() {
 const int N = 1 << 24; // ~16 million elements
 std::vector<float> a(N, 1.0f), b(N, 2.0f), c(N);

 // Warm up (fill caches, trigger frequency scaling)
 add_scalar(a.data(), b.data(), c.data(), N);

 // Benchmark
 auto start = std::chrono::high_resolution_clock::now();

 for (int trial = 0; trial < 100; trial++) {
 add_scalar(a.data(), b.data(), c.data(), N);
 }

 auto end = std::chrono::high_resolution_clock::now();
 double elapsed = std::chrono::duration<double>(end - start).count();

 double total_bytes = 3.0 * N * sizeof(float) *
100; // read a, read b, write c
 double bandwidth = total_bytes / elapsed / 1e9; // GB/s

 std::cout << "Time: " << elapsed << " s\n";
 std::cout << "Bandwidth: " << bandwidth << " GB/s\n";
}
```

```
 return 0;
}
```

```
Compile with optimisations
g++ -O3 -march=native -o bench bench.cpp
./bench
```

- **Key C++ concepts in this code:**

- `#include <vector>`: dynamic array (`std::vector<float>`) — like Python's list but typed and contiguous in memory.
- `a.data()`: returns a raw pointer (`float*`) to the underlying array — needed for SIMD intrinsics.
- `std::chrono`: high-resolution timer for benchmarking.
- `-O3`: maximum compiler optimisation level. The compiler may auto-vectorise your loops (use SIMD automatically). `-march=native` enables all SIMD instructions your CPU supports.

- **Why warm up:** the first run fills caches and may trigger CPU frequency scaling (turbo boost). Subsequent runs are more representative.

- **Why measure bandwidth:** for memory-bound operations (like element-wise add), the meaningful metric is bandwidth (GB/s), not FLOPS. If your measured bandwidth is close to the hardware limit (~50 GB/s for DDR5), you are memory-bound and SIMD will not help much (the bottleneck is memory, not compute).

## 78.9 Coding Tasks (use CoLab or notebook)

1. Compute the arithmetic intensity of common ML operations and classify them as memory-bound or compute-bound.

```
import jax.numpy as jnp

def arithmetic_intensity(flops, bytes_transferred):
 return flops / bytes_transferred

Element-wise ReLU: 1 comparison per element, read + write
n = 1024
relu_flops = n # 1 op per element
relu_bytes = 2 * n * 4 # read input + write output (float32)
print(f"ReLU: {arithmetic_intensity(relu_flops, relu_bytes):.2f} FLOPS/byte →
memory-bound")
```

```

Matrix multiply: 2*n^3 ops, read 2*n^2 + write n^2 floats
matmul_flops = 2 * n**3
matmul_bytes = 3 * n**2 * 4 # read A + read B + write C
print(f"Matmul ({n}x{n}): {arithmetic_intensity(matmul_flops, matmul_bytes):.0f}
FLOPS/byte → compute-bound")

Layer norm: ~5n ops (mean, var, normalise), read + write
ln_flops = 5 * n
ln_bytes = 2 * n * 4
print(f"LayerNorm: {arithmetic_intensity(ln_flops, ln_bytes):.2f} FLOPS/byte →
memory-bound")

Convolution 3x3: 2*9*C_in*C_out*H*W, read kernel + feature map + write output
C_in, C_out, H, W = 64, 128, 32, 32
conv_flops = 2 * 9 * C_in * C_out * H * W
conv_bytes = (9 * C_in * C_out + C_in * H * W + C_out * H * W) * 4
print(f"Conv3x3: {arithmetic_intensity(conv_flops, conv_bytes):.0f} FLOPS/byte →
compute-bound")

```

1. Demonstrate why parallelism matters. Compare sequential vs parallel (NumPy) execution as data size grows.

```

import numpy as np
import time

for n in [1000, 10000, 100000, 1000000, 10000000]:
 a = np.random.randn(n).astype(np.float32)
 b = np.random.randn(n).astype(np.float32)

 # "Sequential" (Python loop)
 start = time.time()
 c = [a[i] * b[i] for i in range(min(n, 100000))] # cap at 100K for sanity
 seq_time = time.time() - start
 if n > 100000:
 seq_time *= n / 100000 # extrapolate

 # "Parallel" (NumPy, uses SIMD + multithreading internally)
 start = time.time()
 c = a * b
 par_time = time.time() - start

 print(f"n={n:>10,} sequential={seq_time:.4f}s parallel={par_time:.6f}s "
 f"speedup={seq_time/par_time:.0f}x")

```

# 79 ARM and NEON

ARM processors power every smartphone, most tablets, Apple's laptops, and an increasing share of data centre servers. This file covers the ARM architecture, NEON SIMD programming with C++ intrinsics, SVE/SVE2 for scalable vector processing, Apple Silicon specifics, and practical vectorised kernel examples

- If you own an iPhone, a MacBook, or use AWS Graviton instances, you are running ARM. ARM's power efficiency makes it dominant in mobile and embedded, and increasingly competitive in servers and ML inference. Understanding ARM SIMD lets you write code that runs fast on the hardware most people actually use.
- For a real-world example of ARM SIMD kernels in production, see **Cactus** — a low-latency AI engine for mobile devices and wearables: [github.com/cactus-compute/cactus](https://github.com/cactus-compute/cactus). Cactus implements custom ARM NEON and NPU-accelerated kernels for attention, KV-cache quantisation, and chunked prefill, achieving the fastest inference on ARM CPUs with 10x lower RAM than other engines. Its three-layer architecture (Engine → Graph → Kernels) is a concrete example of how the SIMD concepts in this file are used to build production ML infrastructure.

## 79.1 ARM Architecture Basics

---

- ARM is a **RISC** (Reduced Instruction Set Computer) architecture (chapter 13). Key characteristics:
  - **Load-store architecture:** arithmetic instructions operate only on registers, never directly on memory. To add two numbers from memory, you must: (1) load them into registers, (2) add the registers, (3) store the result back to memory. This is simpler than x86 (which can add a register and a memory location in one instruction) but enables cleaner pipelining.
  - **Fixed-width instructions:** every ARMv8 (AArch64) instruction is exactly 32 bits. This makes decoding fast and predictable (unlike x86's variable-length instructions that can be 1-15 bytes).
  - **32 general-purpose registers** (x0-x30, each 64-bit) plus the stack pointer (sp) and zero register (xzr). Compare to x86's 16 general-purpose registers. More registers = fewer memory accesses = faster code.

- **32 SIMD/floating-point registers** (v0-v31, each 128-bit) for NEON and floating-point operations.

```
// ARM assembly (just to see the flavour -- you will use intrinsics, not assembly)
// Add two registers
add x0, x1, x2 // x0 = x1 + x2

// Load from memory
ldr x0, [x1] // x0 = *x1 (load 64 bits from address in x1)

// NEON: add four floats
fadd v0.4s, v1.4s, v2.4s // v0 = v1 + v2 (four 32-bit floats)
```

- You will not write assembly. You will use **intrinsics**: C/C++ functions that map 1:1 to specific instructions. The compiler handles register allocation, scheduling, and other low-level details.

## 79.2 NEON: 128-bit SIMD

- **NEON** is ARM's SIMD extension. Each NEON register is 128 bits wide and can hold:

Data type	Elements per register	Notation
float32	4	float32x4_t
float16	8	float16x8_t
int32	4	int32x4_t
int16	8	int16x8_t
int8	16	int8x16_t

- 128 bits is narrower than x86's AVX (256-bit) or AVX-512 (512-bit). But ARM compensates with excellent power efficiency and wide availability.

### 79.2.1 NEON Intrinsics: The Basics

- NEON intrinsics follow a naming convention: `v[operation][qualifier]_[type]`

```
#include <arm_neon.h>
```

```

// Load 4 floats from memory into a NEON register
float32x4_t a = vld1q_f32(ptr); // vld1q = vector load 1, q = 128-bit
(quad)

// Store 4 floats from a NEON register to memory
vst1q_f32(out_ptr, a); // vst1q = vector store 1, q = 128-bit

// Arithmetic
float32x4_t c = vaddq_f32(a, b); // c = a + b (4 floats)
float32x4_t d = vmulq_f32(a, b); // d = a * b (4 floats)
float32x4_t e = vfmaq_f32(c, a, b); // e = c + a * b (fused multiply-add, 4
floats)

// Comparison (returns a mask: all 1s if true, all 0s if false)
uint32x4_t mask = vcgtq_f32(a,
b); // mask[i] = (a[i] > b[i]) ? 0xFFFFFFFF : 0

// Select elements based on mask (like numpy.where)
float32x4_t result = vbslq_f32(mask, a, b); // result[i] = mask[i] ? a[i] : b[i]

// Reduce: sum all 4 elements to a scalar
float total = vaddvq_f32(a); // total = a[0] + a[1] + a[2] + a[3]

```

- **vfmaq\_f32** (fused multiply-add) is the most important SIMD instruction for ML. It computes  $c = c + a \times b$  in one instruction with a single rounding step (more accurate than separate multiply then add). Dot products, matrix multiplications, and convolutions are built from FMA.

## 79.2.2 Practical Example: Vectorised Dot Product

- The dot product is the inner loop of matrix multiplication. Let's write it in scalar C++ and then vectorise it with NEON.

```

#include <arm_neon.h>

// Scalar dot product
float dot_scalar(const float* a, const float* b, int n) {
 float sum = 0.0f;
 for (int i = 0; i < n; i++) {
 sum += a[i] * b[i];
 }
 return sum;
}

// NEON-vectorised dot product
float dot_neon(const float* a, const float* b, int n) {

```

```

float32x4_t sum_vec = vdupq_n_f32(0.0f); // initialise 4 accumulators to 0

int i = 0;
for (; i + 4 <= n; i += 4) {
 float32x4_t va = vld1q_f32(a + i); // load 4 elements from a
 float32x4_t vb = vld1q_f32(b + i); // load 4 elements from b
 sum_vec = vfmaq_f32(sum_vec, va, vb); // sum_vec += va * vb
}

// Reduce the 4 accumulators to a single scalar
float sum = vaddvq_f32(sum_vec);

// Handle remaining elements (if n is not a multiple of 4)
for (; i < n; i++) {
 sum += a[i] * b[i];
}

return sum;
}

```

- **Key C++ concepts:**

- `const float*`: a pointer to read-only float data. `const` promises we won't modify the data through this pointer.
- `a + i`: pointer arithmetic. `a + i` points to the *i*-th element of the array (same as `&a[i]`).
- The “cleanup loop” at the end handles the case where *n* is not a multiple of 4. This is a universal pattern in SIMD code: process the bulk in vectorised chunks, then handle the remainder in scalar code.

- **Why 4 accumulators in `sum_vec`:** instead of a single scalar accumulator, we use 4 independent accumulators (one per SIMD lane). This avoids a data dependency: each iteration's FMA depends on `sum_vec`, but with 4 independent lanes, the CPU can pipeline the FMAs. At the end, we reduce the 4 partial sums to one.

### 79.2.3 Practical Example: Vectorised ReLU

```

#include <arm_neon.h>

void relu_neon(const float* input, float* output, int n) {
 float32x4_t zero = vdupq_n_f32(0.0f);

 int i = 0;
 for (; i + 4 <= n; i += 4) {
 float32x4_t x = vld1q_f32(input + i);

```



```

 float32x4_t result = vmaxq_f32(x, zero); // max(x, 0) = ReLU
 vst1q_f32(output + i, result);
 }

 // Scalar cleanup
 for (; i < n; i++) {
 output[i] = input[i] > 0 ? input[i] : 0;
 }
}

```

- `vmaxq_f32` computes the element-wise maximum of two vectors. Since one vector is all zeros, this is exactly ReLU. No branching, no comparisons — just a single instruction.

## 79.3 I8MM: Integer Matrix Multiply

- **I8MM** (Int8 Matrix Multiply) is an ARMv8.6 extension that adds dedicated instructions for INT8 matrix multiplication with INT32 accumulation — exactly what quantised ML inference needs.
- The key instruction is **SMMLA** (Signed Matrix Multiply-Accumulate): it takes two  $8 \times 2$  blocks of INT8 values and accumulates the result into a  $2 \times 2$  block of INT32:

```

#include <arm_neon.h>

// I8MM: multiply two 8-element INT8 vectors, accumulate into 4 INT32 results
// This computes a 2x2 tile of the output matrix from 2x8 x 8x2 input tiles
void matmul_i8mm_tile(const int8_t* A, const int8_t* B, int32_t* C) {
 // Load 8 bytes from A (2 rows of 4 elements, packed)
 int8x16_t va = vld1q_s8(A); // 16 bytes = 2 rows x 8 elements
 int8x16_t vb = vld1q_s8(B); // 16 bytes = 2 rows x 8 elements

 // Load existing accumulator (2x2 = 4 int32 values)
 int32x4_t acc = vld1q_s32(C);

 // I8MM instruction: acc += A_tile x B_tile^T
 // Computes 2x2 output from 2x8 x 8x2 inputs
 acc = vmmlaq_s32(acc, va, vb); // THE I8MM instruction
}

```

```

 vst1q_s32(C, acc);
 }

```

- **Why I8MM matters:** without I8MM, INT8 matmul on NEON requires widening multiplies (`vmull`) followed by pairwise adds — multiple instructions per output element. With I8MM, the hardware does an 8-element dot product ( $2 \times 8 \times 8 \times 2 = 2 \times 2$ ) in a single instruction. For INT8 inference workloads, this is 4-8x faster than plain NEON.
- **Availability:** Apple M1+ (all Apple Silicon), ARM Cortex-A510/A710/X2+ (ARMv9), AWS Graviton3+. Check with `#ifdef __ARM_FEATURE_MATMUL_INT8`.
- For ML inference: INT8 quantised models (chapter 18) running on ARM servers (Graviton) or Apple Silicon benefit enormously from I8MM. Frameworks like ONNX Runtime and llama.cpp detect I8MM at runtime and use optimised kernels automatically.

## 79.4 SME and SME2: Scalable Matrix Extension

- **SME** (Scalable Matrix Extension) is ARM's answer to Intel AMX and NVIDIA Tensor Cores: dedicated hardware for matrix operations. SME2 (ARMv9.2) extends it further.
- SME introduces **ZA tile registers**: 2D matrices stored in hardware, up to  $SVL \times SVL$  bytes (where SVL is the streaming vector length, typically 128-512 bits per dimension). Unlike NEON (1D vectors) or even SVE (1D scalable vectors), SME operates on **2D tiles** natively.
- The programming model has two modes:
  - **Normal mode:** standard ARM execution (NEON, SVE work as usual).
  - **Streaming SVE mode:** entered via `smstart`, enables SME instructions. SVE instructions also work in this mode but may use different register widths.

```

#include <arm_sme.h>

// SME2: outer product accumulation for matrix multiply
// Accumulates A_col x B_row into the ZA tile register
void sme2_matmul_outer(const float* A_col, const float* B_row, int K) {
 // Enter streaming mode
 // smstart; // (done via compiler intrinsic or inline asm)

```

```

// Zero the ZA tile accumulator
svzero_z();

for (int k = 0; k < K; k++) {
 // Load a column of A and a row of B into SVE registers
 svfloat32_t a = svld1_f32(svptrue_b32(), &A_col[k * SVL]);
 svfloat32_t b = svld1_f32(svptrue_b32(), &B_row[k * SVL]);

 // Outer product: ZA += a × b^T
 // This accumulates an SVL×SVL tile in one instruction
 svmopa_z32_f32_m(0, svptrue_b32(), svptrue_b32(), a, b);
}

// Store the ZA tile to memory
// svst1_z(...);

// Exit streaming mode
// smstop;
}

```

- **Key concepts:**

- **svmopa** (outer product accumulate): the core SME instruction. It computes a full outer product of two vectors and accumulates into the ZA tile. For SVL=512 bits (16 floats), this is a  $16 \times 16$  outer product — 256 FMA operations in one instruction.
- **ZA tile:** persistent across instructions within streaming mode. You accumulate multiple outer products (one per K iteration) into the same tile, building up a full matrix multiply tile.
- **Streaming mode:** SME instructions only work in streaming mode. The overhead of entering/exiting streaming mode means SME is best for sustained matrix computation, not short bursts.

- **SME2 additions:** multi-vector operations (process 2 or 4 SVE vectors simultaneously), additional tile operations, and improved integration with normal mode.
- **Availability:** ARM Neoverse V2 (AWS Graviton4), some upcoming mobile chips. Not yet on Apple Silicon (as of 2026). SME is still early-stage — most ML frameworks do not yet have SME-optimised kernels.
- **The progression:** NEON (128-bit vectors, element-wise) → I8MM (INT8 matrix tiles) → SVE (scalable vectors) → SME (scalable 2D matrix tiles). Each generation moves closer to native matrix operations in hardware.

## 79.5 SVE and SVE2: Scalable Vector Extensions

---

- NEON has a fixed 128-bit width. **SVE** (Scalable Vector Extension) introduces **vector-length agnostic (VLA)** programming: you write code once, and it runs on hardware with any vector width (128 to 2048 bits). The hardware determines the width at runtime.

```
#include <arm_sve.h>

void add_sve(const float* a, const float* b, float* c, int n) {
 int i = 0;
 svbool_t pred = svwhilelt_b32(i, n); // predicate: which lanes are active

 while (svptest_any(svptrue_b32(), pred)) {
 svfloat32_t va = svld1(pred, a + i);
 svfloat32_t vb = svld1(pred, b + i);
 svst1(pred, c + i, svadd_x(pred, va, vb));

 i += svcntw(); // advance by the hardware vector width (in 32-bit elements)
 pred = svwhilelt_b32(i, n);
 }
}
```

- **Predicate registers** (`svbool_t`) replace the scalar cleanup loop. Each lane has a predicate bit: active lanes participate, inactive lanes are masked off. The `svwhilelt_b32(i, n)` instruction creates a predicate where lanes corresponding to `i`, `i+1`, ..., `n-1` are active. This handles the tail automatically.
- `svcntw()` returns the number of 32-bit elements per vector register at runtime. On a CPU with 256-bit SVE, this returns 8. On 512-bit SVE, it returns 16. Your code adapts automatically.
- SVE is available on ARM Neoverse V1/V2 (AWS Graviton3/4, some server chips). It is not yet available on Apple Silicon.

## 79.6 Apple Silicon Specifics

---

- Apple's M-series chips (M1, M2, M3, M4) are ARM-based with custom microarchitecture:

- **Performance and efficiency cores:** P-cores (Firestorm/Avalanche/etc.) for heavy compute, E-cores (Icestorm/Blizzard/etc.) for background tasks. The scheduler assigns threads to the appropriate core type.
- **AMX** (Apple Matrix eXtensions): dedicated matrix multiply units, separate from NEON. AMX is undocumented (Apple does not publish the ISA), but the Accelerate framework uses it internally for BLAS operations. When you call `np.dot` on a Mac, it goes through Accelerate, which uses AMX. You cannot program AMX directly (without reverse engineering).
- **Unified memory:** CPU and GPU share the same physical RAM. On other systems, data must be copied from CPU memory to GPU memory (over PCIe, ~32 GB/s). On Apple Silicon, there is no copy — the GPU reads the same memory the CPU wrote. This eliminates a major bottleneck for ML workloads.
- **Neural Engine:** a 16-core dedicated ML accelerator. Performs ~30 TOPS (trillion operations per second) for INT8 inference. Used by Core ML for on-device inference.
- **For ML on Apple Silicon:** use MLX (Apple's ML framework), which is designed for the unified memory architecture. PyTorch also has MPS (Metal Performance Shaders) backend support, though it is less mature than CUDA.

## 79.7 Auto-Vectorisation

---

- Writing SIMD intrinsics is tedious. Can the **compiler** vectorise your code automatically?
- Yes, with caveats. Modern compilers (GCC, Clang) can auto-vectorise simple loops:

```
// The compiler CAN auto-vectorise this (with -O3 -march=native)
void add_auto(const float* a, const float* b, float* c, int n) {
 for (int i = 0; i < n; i++) {
 c[i] = a[i] + b[i];
 }
}
```

- **Patterns that help auto-vectorisation:**
  - Simple loops with known trip count.

- No data dependencies between iterations ( $c[i]$  does not depend on  $c[i-1]$ ).
- Contiguous memory access (no scatter/gather).
- `const` and `restrict` pointers (tells the compiler arrays do not overlap).

```
// restrict tells the compiler: a, b, c point to non-overlapping memory
void add_restrict(const float* __restrict__ a,
 const float* __restrict__ b,
 float* __restrict__ c, int n) {
 for (int i = 0; i < n; i++) {
 c[i] = a[i] + b[i];
 }
}
```

- Without `restrict`, the compiler must assume `c` might overlap with `a` or `b` (writing to `c[i]` might change `a[i+1]`), preventing vectorisation.

- **Patterns that prevent auto-vectorisation:**

- Data dependencies:  $a[i] = a[i-1] + b[i]$  (each iteration depends on the previous).
- Complex control flow: `if` statements inside the loop (unless the compiler can convert to predication).
- Function calls inside the loop (unless the function is inlined).
- Pointer aliasing (arrays might overlap, without `restrict`).

- **Checking auto-vectorisation:** use compiler flags to see what was vectorised:

```
GCC: show vectorisation decisions
g++ -O3 -march=native -fopt-info-vec-optimized code.cpp

Clang: show vectorisation report
clang++ -O3 -march=native -Rpass=loop-vectorize code.cpp
```

- **When to use intrinsics vs auto-vectorisation:** start with clean C++ and compiler optimisations. If the compiler vectorises your loop, great. If performance is still insufficient, inspect the compiler's vectorisation report to understand why, and only then write intrinsics for the critical inner loop. Premature intrinsics make code unreadable without guaranteed benefit.

## 79.8 Coding Tasks (compile with g++ or clang++ on ARM — Mac M-series or Linux aarch64)

---

1. Write a scalar dot product and a NEON-vectorised dot product. Benchmark both and measure the speedup.

```
// task1_neon_dot.cpp
// Compile (Mac/ARM Linux): clang++ -O3 -o task1 task1_neon_dot.cpp
// Note: NEON is enabled by default on AArch64, no special flags needed

#include <iostream>
#include <chrono>
#include <vector>
#include <arm_neon.h>

float dot_scalar(const float* a, const float* b, int n) {
 float sum = 0.0f;
 for (int i = 0; i < n; i++) {
 sum += a[i] * b[i];
 }
 return sum;
}

float dot_neon(const float* a, const float* b, int n) {
 float32x4_t sum_vec = vdupq_n_f32(0.0f);
 int i = 0;
 for (; i + 4 <= n; i += 4) {
 float32x4_t va = vld1q_f32(a + i);
 float32x4_t vb = vld1q_f32(b + i);
 sum_vec = vfmaq_f32(sum_vec, va, vb);
 }
 float sum = vaddvq_f32(sum_vec);
 for (; i < n; i++) sum += a[i] * b[i];
 return sum;
}

int main() {
 const int N = 10'000'000;
 std::vector<float> a(N, 1.0f), b(N, 2.0f);

 // Warm up
 volatile float s1 = dot_scalar(a.data(), b.data(), N);
 volatile float s2 = dot_neon(a.data(), b.data(), N);

 // Benchmark scalar
```

```

 auto start = std::chrono::high_resolution_clock::now();
 for (int t = 0; t < 100; t++) {
 s1 = dot_scalar(a.data(), b.data(), N);
 }
 auto end = std::chrono::high_resolution_clock::now();
 double scalar_ms = std::chrono::duration<double, std::milli>(end -
start).count() / 100;

 // Benchmark NEON
 start = std::chrono::high_resolution_clock::now();
 for (int t = 0; t < 100; t++) {
 s2 = dot_neon(a.data(), b.data(), N);
 }
 end = std::chrono::high_resolution_clock::now();
 double neon_ms = std::chrono::duration<double, std::milli>(end -
start).count() / 100;

 std::cout << "Scalar: " << scalar_ms << " ms (result: " << s1 << ")\n";
 std::cout << "NEON: " << neon_ms << " ms (result: " << s2 << ")\n";
 std::cout << "Speedup: " << scalar_ms / neon_ms << "x\n";
 return 0;
}

```

1. Implement NEON ReLU and softmax-max-finding. Practice the load→compute→store pattern with different operations.

```

// task2_neon_ops.cpp
// Compile: clang++ -O3 -o task2 task2_neon_ops.cpp

#include <iostream>
#include <vector>
#include <cmath>
#include <arm_neon.h>

void relu_neon(const float* in, float* out, int n) {
 float32x4_t zero = vdupq_n_f32(0.0f);
 int i = 0;
 for (; i + 4 <= n; i += 4) {
 float32x4_t x = vld1q_f32(in + i);
 vst1q_f32(out + i, vmaxq_f32(x, zero));
 }
 for (; i < n; i++) out[i] = in[i] > 0 ? in[i] : 0;
}

float max_neon(const float* data, int n) {
 float32x4_t max_vec = vdupq_n_f32(-INFINITY);

```



```

 int i = 0;
 for (; i + 4 <= n; i += 4) {
 max_vec = vmaxq_f32(max_vec, vld1q_f32(data + i));
 }
 float result = vmaxvq_f32(max_vec);
 for (; i < n; i++) result = result > data[i] ? result : data[i];
 return result;
}

int main() {
 std::vector<float> data = {-3, 1, -1, 4, 2, -5, 0, 7, -2, 3};
 std::vector<float> out(data.size());

 relu_neon(data.data(), out.data(), data.size());
 std::cout << "ReLU: ";
 for (float x : out) std::cout << x << " ";
 std::cout << "\n";

 float mx = max_neon(data.data(), data.size());
 std::cout << "Max: " << mx << " (expected: 7)\n";
 return 0;
}

```

1. Compare auto-vectorised code against hand-written NEON intrinsics. Compile with `-fopt-info-vec` (GCC) or `-Rpass=loop-vectorize` (Clang) to see what the compiler does.

```

// task3_auto_vs_manual.cpp
// Compile: clang++ -O3 -Rpass=loop-vectorize -o task3 task3_auto_vs_manual.cpp
// (or): g++ -O3 -fopt-info-vec-optimized -o task3 task3_auto_vs_manual.cpp

#include <iostream>
#include <chrono>
#include <vector>
#include <arm_neon.h>

// Let the compiler auto-vectorise
void add_auto(const float* __restrict__ a, const float* __restrict__ b,
 float* __restrict__ c, int n) {
 for (int i = 0; i < n; i++) {
 c[i] = a[i] + b[i];
 }
}

// Hand-written NEON
void add_neon(const float* a, const float* b, float* c, int n) {

```

```

 int i = 0;
 for (; i + 4 <= n; i += 4) {
 vst1q_f32(c + i, vaddq_f32(vld1q_f32(a + i), vld1q_f32(b + i)));
 }
 for (; i < n; i++) c[i] = a[i] + b[i];
}

int main() {
 const int N = 10'000'000;
 std::vector<float> a(N, 1.0f), b(N, 2.0f), c(N);

 auto bench = [&](auto fn, const char* name) {
 fn(a.data(), b.data(), c.data(), N); // warm up
 auto start = std::chrono::high_resolution_clock::now();
 for (int t = 0; t < 100; t++) fn(a.data(), b.data(), c.data(), N);
 auto end = std::chrono::high_resolution_clock::now();
 double ms = std::chrono::duration<double, std::milli>(end -
start).count() / 100;
 std::cout << name << ": " << ms << " ms\n";
 };

 bench(add_auto, "Auto-vectorised");
 bench(add_neon, "Hand-written NEON");
 // They should be very close – the compiler auto-vectorises this simple loop
 well

 return 0;
}

```

## 80 x86 and AVX

x86 processors from Intel and AMD dominate data centre servers where most ML training happens. This file covers the evolution of x86 SIMD, AVX/AVX2 intrinsics programming, AVX-512, Intel AMX for matrix operations, memory alignment, performance pitfalls, and profiling – the tools for squeezing maximum performance from the world’s most common server CPUs.

- If your training runs on cloud VMs (AWS, GCP, Azure), it is almost certainly running on x86. Even GPU-heavy training has CPU bottlenecks: data loading, preprocessing, gradient aggregation, and checkpointing all run on the CPU. Optimising these with x86 SIMD can meaningfully reduce end-to-end training time.

## 80.1 The x86 SIMD Evolution

---

- x86 SIMD has evolved through increasingly wider vector registers:

Generation	Year	Register Width	Registers	Key Features
MMX	1997	64-bit	8 (mm0-7)	Integer only, shared with FPU
SSE	1999	128-bit	8 (xmm0-7)	4 floats, dedicated registers
SSE2	2001	128-bit	8/16	2 doubles, integer operations
AVX	2011	256-bit	16 (ymm0-15)	8 floats, 3-operand instructions
AVX2	2013	256-bit	16	Integer 256-bit, FMA, gather
AVX-512	2017	512-bit	32 (zmm0-31)	16 floats, mask registers, scatter
AMX	2023	Tile registers	8 tiles	Matrix multiply (BF16, INT8)

- Each generation doubled the throughput for vectorised code. Code written with SSE intrinsics runs on every x86 CPU made since 2001. AVX2 requires a 2013+ CPU. AVX-512 is Intel Xeon and some consumer chips. AMX is the newest (Sapphire Rapids and later).
- **Backward compatibility:** x86 SSE registers (xmm) are the lower 128 bits of AVX registers (ymm), which are the lower 256 bits of AVX-512 registers (zmm). Old SSE code runs on new CPUs without modification.

## 80.2 AVX2 Programming

---

- AVX2 operates on 256-bit registers (YMM), processing 8 floats or 4 doubles simultaneously. It is the sweet spot for portable high-performance code: available on virtually all modern x86 CPUs (2013+).

## 80.2.1 Intrinsics Naming Convention

- All x86 intrinsics follow the pattern: `_mm[width]_[operation]_[type]`
  - `_mm` = MMX/SSE (128-bit), `_mm256` = AVX (256-bit), `_mm512` = AVX-512 (512-bit)
  - Operation: `add`, `mul`, `fmadd`, `load`, `store`, `set`, etc.
  - Type: `ps` = packed single (float32), `pd` = packed double (float64), `epi32` = packed int32, `si256` = 256-bit integer

```
#include <immintrin.h> // all x86 SIMD intrinsics

// Data types
__m256 a; // 256-bit register holding 8 float32s
__m256d b; // 256-bit register holding 4 float64s
__m256i c; // 256-bit register holding integers (8x32, 16x16, or 32x8)
```

## 80.2.2 Loading and Storing Data

```
// Load 8 floats from memory
__m256 v = _mm256_loadu_ps(ptr); // unaligned load (works with any address)
__m256 v = _mm256_load_ps(ptr); // aligned load (ptr must be 32-byte
aligned, faster)

// Store 8 floats to memory
_mm256_storeu_ps(out_ptr, v); // unaligned store
_mm256_store_ps(out_ptr, v); // aligned store

// Broadcast a single value to all 8 lanes
__m256 ones = _mm256_set1_ps(1.0f); // [1, 1, 1, 1, 1, 1, 1, 1]

// Set individual values (rarely needed)
__m256 v = _mm256_set_ps(7,6,5,4,3,2,1,0); // note: reverse order!

// Zero register
__m256 z = _mm256_setzero_ps();
```

## 80.2.3 Arithmetic

```
__m256 c = _mm256_add_ps(a, b); // c[i] = a[i] + b[i]
__m256 d = _mm256_mul_ps(a, b); // d[i] = a[i] * b[i]
__m256 e = _mm256_sub_ps(a, b); // e[i] = a[i] - b[i]
__m256 f = _mm256_div_ps(a, b); // f[i] = a[i] / b[i] (slower than mul)

// Fused Multiply-Add: r = a * b + c (one instruction, one rounding)
```

```

__m256 r = _mm256_fmadd_ps(a, b, c); // THE most important instruction for ML

// Min and max
__m256 mn = _mm256_min_ps(a, b); // min(a[i], b[i]) – useful for clipping
__m256 mx = _mm256_max_ps(a, b); // max(a[i], b[i]) – useful for ReLU

```

### 80.2.4 Practical Example: AVX2 Dot Product

```

#include <immintrin.h>

float dot_avx2(const float* a, const float* b, int n) {
 __m256 sum = _mm256_setzero_ps(); // 8 accumulators initialised to 0

 int i = 0;
 for (; i + 8 <= n; i += 8) {
 __m256 va = _mm256_loadu_ps(a + i);
 __m256 vb = _mm256_loadu_ps(b + i);
 sum = _mm256_fmadd_ps(va, vb, sum); // sum += va * vb
 }

 // Horizontal reduction: sum the 8 elements of sum
 // Step 1: add upper 128 bits to lower 128 bits
 __m128 hi = _mm256_extractf128_ps(sum, 1);
 __m128 lo = _mm256_castps256_ps128(sum);
 __m128 sum128 = _mm_add_ps(hi, lo); // 4 partial sums

 // Step 2: horizontal add within 128-bit register
 sum128 = _mm_hadd_ps(sum128, sum128); // [a+b, c+d, a+b, c+d]
 sum128 = _mm_hadd_ps(sum128, sum128); // [a+b+c+d, ...]

 float result = _mm_cvtss_f32(sum128); // extract scalar

 // Scalar cleanup
 for (; i < n; i++) {
 result += a[i] * b[i];
 }

 return result;
}

```

- **Why the horizontal reduction is ugly:** SIMD is designed for vertical operations (lane 0 with lane 0, lane 1 with lane 1). Horizontal operations (summing across lanes) fight the hardware. This is why dot products have the awkward reduction code at the end. The vectorised loop is clean; the reduction is boilerplate.

- **Performance:** compared to the NEON version (file 02), AVX2 processes 8 floats per iteration vs NEON's 4. For long vectors, this is a 2x speedup over NEON (ignoring memory bandwidth limits).

### 80.2.5 Practical Example: AVX2 Softmax (Simplified)

- Softmax requires: find the max, subtract it, exponentiate, sum, divide. Here is the max-finding step:

```
float vector_max_avx2(const float* data, int n) {
 __m256 max_vec = _mm256_set1_ps(-INFINITY);

 int i = 0;
 for (; i + 8 <= n; i += 8) {
 __m256 v = _mm256_loadu_ps(data + i);
 max_vec = _mm256_max_ps(max_vec, v);
 }

 // Reduce 8 maxes to 1
 __m128 hi = _mm256_extractf128_ps(max_vec, 1);
 __m128 lo = _mm256_castps256_ps128(max_vec);
 __m128 max128 = _mm_max_ps(hi, lo);

 // Shuffle and max to find the single maximum
 max128 = _mm_max_ps(max128, _mm_shuffle_ps(max128, max128, 0b01001110));
 max128 = _mm_max_ps(max128, _mm_shuffle_ps(max128, max128, 0b10110001));

 float result = _mm_cvtss_f32(max128);

 for (; i < n; i++) {
 result = result > data[i] ? result : data[i];
 }

 return result;
}
```

- The `_mm_shuffle_ps` instruction rearranges elements within a register. The binary constant `0b01001110` controls which elements go where. This is called a **permutation**, and it connects directly to permutation matrices (chapter 2): shuffling SIMD lanes is the hardware equivalent of multiplying by a permutation matrix.

## 80.3 AVX-512

- AVX-512 doubles the width again: 512-bit registers (ZMM), processing 16 floats simultaneously.

```
__m512 a = _mm512_loadu_ps(ptr); // load 16 floats
__m512 c = _mm512_fmadd_ps(a, b, c); // 16 FMAs at once
float sum = _mm512_reduce_add_ps(a); // built-in horizontal sum (no
manual reduction!)

// Mask operations: operate on a subset of lanes
__mmask16 mask = _mm512_cmpgt_ps_mask(a, zero); // which lanes are > 0?
__m512 relu = _mm512_maskz_mov_ps(mask, a); // zero out negative lanes =
ReLU
```

- **Mask registers** (`__mmask16`) are AVX-512's most powerful feature. Each bit controls whether a lane participates in an operation. This replaces the scalar cleanup loop: the last iteration uses a mask where only the valid lanes are active, handling any vector length without a separate scalar loop.
- **AVX-512 frequency throttling**: on many Intel CPUs, using AVX-512 instructions causes the CPU to temporarily reduce its clock frequency (to stay within thermal limits). This means AVX-512 is not always faster than AVX2 for short bursts — the frequency penalty can outweigh the wider vectors. For sustained workloads (like matrix multiplication), AVX-512 wins. For mixed code (some SIMD, some scalar), the frequency transitions can hurt.

## 80.4 Intel AMX: Matrix Multiply Hardware

- **AMX** (Advanced Matrix eXtensions) adds dedicated matrix multiply units. Instead of SIMD vectors, AMX operates on **tiles**: 2D blocks of data (up to 16 rows × 64 bytes each).

```
#include <immintrin.h>

// AMX tile multiply: C += A * B (in BF16)
// A is 16x32 BF16, B is 32x16 BF16, C is 16x16 FP32
_tile_loadadd(0, a_ptr, stride_a); // load tile 0 from A
_tile_loadadd(1, b_ptr, stride_b); // load tile 1 from B
_tile_dpbfbf16ps(2, 0, 1); // tile 2 += tile 0 * tile 1 (BF16 matmul,
```

```
FP32 accumulation)
```

```
_tile_stored(2, c_ptr, stride_c); // store tile 2 to C
```

- AMX performs a full  $16 \times 32 \times 32 \times 16$  matrix multiply in a single instruction. This is hundreds of FMA operations at once, specifically designed for the small matrix multiplies that dominate transformer inference (attention score computation, MLP layers).
- AMX supports BF16 (bfloat16) and INT8, matching the precisions used in ML inference. Combined with AVX-512 for other operations, AMX-equipped CPUs (Intel Sapphire Rapids, Emerald Rapids) can run transformer inference competitively with entry-level GPUs.

## 80.5 Memory Alignment

- **Aligned memory access** is when the data address is a multiple of the vector register width (16 bytes for SSE, 32 bytes for AVX, 64 bytes for AVX-512). Aligned access is faster on some CPUs and required by `_mm256_load_ps` (as opposed to `_mm256_loadu_ps`).

```
// Allocate aligned memory
float* data = (float*)aligned_alloc(32, n * sizeof(float)); // 32-byte aligned
for AVX

// C++ aligned allocation
#include <new>
float* data = new (std::align_val_t(32)) float[n];

// Alternatively, use the compiler attribute
alignas(32) float data[1024];
```

- **In practice:** on modern CPUs (Haswell and later), unaligned loads (`loadu`) are almost as fast as aligned loads when the data does not cross a cache line boundary. The performance penalty of unaligned access has largely disappeared, but cache line splits (data spanning two 64-byte cache lines) can still cause a  $\sim 2\times$  slowdown for that specific load. Aligned allocation avoids this entirely.



## 80.6 Performance Pitfalls

- **AVX-SSE transition penalty:** on older Intel CPUs (pre-Skylake), switching between AVX (256-bit) and SSE (128-bit) instructions caused a penalty (~70 cycles). This is why you should use `_mm256_zeroupper()` (or `vzeroupper` instruction) before returning from a function that uses AVX, to clear the upper 128 bits of YMM registers. Modern CPUs (Skylake+) do not have this penalty.
- **Register pressure:** AVX2 has 16 YMM registers. If your kernel uses too many variables, the compiler spills registers to the stack (memory), destroying performance. Keep inner loops simple with few live variables.
- **Data dependencies:** `sum = _mm256_fmadd_ps(a, b, sum)` has a dependency on `sum`: each iteration must wait for the previous FMA to complete (~4-5 cycles latency). The fix: use multiple independent accumulators and reduce at the end:

```
// Single accumulator: limited by FMA latency (4-5 cycles)
__m256 sum = _mm256_setzero_ps();
for (...) {
 sum = _mm256_fmadd_ps(a, b, sum); // each depends on previous
}

// Four accumulators: 4x throughput (hide latency)
__m256 sum0 = _mm256_setzero_ps();
__m256 sum1 = _mm256_setzero_ps();
__m256 sum2 = _mm256_setzero_ps();
__m256 sum3 = _mm256_setzero_ps();
for (...) {
 sum0 = _mm256_fmadd_ps(a0, b0, sum0); // independent
 sum1 = _mm256_fmadd_ps(a1, b1, sum1); // independent
 sum2 = _mm256_fmadd_ps(a2, b2, sum2); // independent
 sum3 = _mm256_fmadd_ps(a3, b3, sum3); // independent
}
sum0 = _mm256_add_ps(sum0, sum1);
sum2 = _mm256_add_ps(sum2, sum3);
sum0 = _mm256_add_ps(sum0, sum2);
```

- This is **loop unrolling** to hide latency. The CPU can issue the FMAs back-to-back because they write to different registers. This is one of the most impactful micro-optimisations in numerical code.

## 80.7 Profiling

---

- **Performance counters** provide hardware-level measurements:

```
Linux perf (requires kernel support)
perf stat ./my_program

basic counters: cycles, instructions, IPC
perf stat -e cache-misses,cache-references ./my_program # cache behaviour
perf record -g ./my_program && perf report # call graph profiling

Intel VTune (detailed x86 profiling)
vtune -collect hotspots -- ./my_program
vtune -collect memory-access -- ./my_program # memory bandwidth analysis
```

- **What to look for:**
  - **IPC** (Instructions Per Cycle): how efficiently the CPU is used.  $IPC > 2$  is good.  $IPC < 1$  suggests memory stalls or branch mispredictions.
  - **Cache miss rate:** high L1/L2 miss rates indicate poor data locality. Restructure data access patterns.
  - **Branch misprediction rate:**  $> 5\%$  suggests unpredictable branches. Convert to branchless code (SIMD comparisons + blending) if possible.
  - **FLOPS achieved vs roofline:** compare your measured FLOPS to the roofline model (file 01). If you are below the roofline, there is room for improvement.

## 80.8 Coding Tasks (compile with g++ or clang++ on x86 — Intel/AMD)

---

1. Write a scalar dot product and an AVX2 dot product. Benchmark both and measure the speedup from 8-wide SIMD.

```
// task1_avx_dot.cpp
// Compile: g++ -O3 -mavx2 -mfma -o task1 task1_avx_dot.cpp

#include <iostream>
#include <chrono>
#include <vector>
#include <immintrin.h>

float dot_scalar(const float* a, const float* b, int n) {
 float sum = 0.0f;
```

```

 for (int i = 0; i < n; i++) sum += a[i] * b[i];
 return sum;
}

float dot_avx2(const float* a, const float* b, int n) {
 __m256 sum = _mm256_setzero_ps();
 int i = 0;
 for (; i + 8 <= n; i += 8) {
 __m256 va = _mm256_loadu_ps(a + i);
 __m256 vb = _mm256_loadu_ps(b + i);
 sum = _mm256_fmadd_ps(va, vb, sum);
 }
 // Reduce: add upper 128 to lower 128, then horizontal add
 __m128 hi = _mm256_extractf128_ps(sum, 1);
 __m128 lo = _mm256_castps256_ps128(sum);
 __m128 r = _mm_add_ps(hi, lo);
 r = _mm_hadd_ps(r, r);
 r = _mm_hadd_ps(r, r);
 float result = _mm_cvtss_f32(r);
 for (; i < n; i++) result += a[i] * b[i];
 return result;
}

int main() {
 const int N = 10'000'000;
 std::vector<float> a(N, 1.0f), b(N, 2.0f);

 volatile float s1 = dot_scalar(a.data(), b.data(), N);
 volatile float s2 = dot_avx2(a.data(), b.data(), N);

 auto bench = [&](auto fn, const char* name) {
 auto start = std::chrono::high_resolution_clock::now();
 volatile float s;
 for (int t = 0; t < 100; t++) s = fn(a.data(), b.data(), N);
 auto end = std::chrono::high_resolution_clock::now();
 double ms = std::chrono::duration<double, std::milli>(end -
start).count() / 100;
 std::cout << name << ": " << ms << " ms (result: " << s << ")\n";
 return ms;
 };

 double t1 = bench(dot_scalar, "Scalar");
 double t2 = bench(dot_avx2, "AVX2 ");
 std::cout << "Speedup: " << t1 / t2 << "x\n";
}

```

```
 return 0;
}
```

1. Implement AVX2 ReLU using `_mm256_max_ps` and compare against a scalar loop. Then try with multiple accumulators (loop unrolling) to hide FMA latency.

```
// task2_avx_relu.cpp
// Compile: g++ -O3 -mavx2 -o task2 task2_avx_relu.cpp

#include <iostream>
#include <chrono>
#include <vector>
#include <immintrin.h>

void relu_scalar(const float* in, float* out, int n) {
 for (int i = 0; i < n; i++) {
 out[i] = in[i] > 0.0f ? in[i] : 0.0f;
 }
}

void relu_avx2(const float* in, float* out, int n) {
 __m256 zero = _mm256_setzero_ps();
 int i = 0;
 for (; i + 8 <= n; i += 8) {
 __m256 x = _mm256_loadu_ps(in + i);
 _mm256_storeu_ps(out + i, _mm256_max_ps(x, zero));
 }
 for (; i < n; i++) out[i] = in[i] > 0.0f ? in[i] : 0.0f;
}

// Unrolled: process 32 floats per iteration (4 x 8)
void relu_avx2_unrolled(const float* in, float* out, int n) {
 __m256 zero = _mm256_setzero_ps();
 int i = 0;
 for (; i + 32 <= n; i += 32) {
 __m256 x0 = _mm256_loadu_ps(in + i);
 __m256 x1 = _mm256_loadu_ps(in + i + 8);
 __m256 x2 = _mm256_loadu_ps(in + i + 16);
 __m256 x3 = _mm256_loadu_ps(in + i + 24);
 _mm256_storeu_ps(out + i, _mm256_max_ps(x0, zero));
 _mm256_storeu_ps(out + i + 8, _mm256_max_ps(x1, zero));
 _mm256_storeu_ps(out + i + 16, _mm256_max_ps(x2, zero));
 _mm256_storeu_ps(out + i + 24, _mm256_max_ps(x3, zero));
 }
 for (; i + 8 <= n; i += 8) {
 _mm256_storeu_ps(out + i, _mm256_max_ps(_mm256_loadu_ps(in + i), zero));
 }
}
```

```

 }
 for (; i < n; i++) out[i] = in[i] > 0.0f ? in[i] : 0.0f;
}

int main() {
 const int N = 16'000'000;
 std::vector<float> in(N), out(N);
 for (int i = 0; i < N; i++) in[i] = (float)(i % 200) - 100.0f;

 auto bench = [&](auto fn, const char* name) {
 fn(in.data(), out.data(), N); // warm up
 auto start = std::chrono::high_resolution_clock::now();
 for (int t = 0; t < 100; t++) fn(in.data(), out.data(), N);
 auto end = std::chrono::high_resolution_clock::now();
 double ms = std::chrono::duration<double, std::milli>(end -
start).count() / 100;
 double bw = 2.0 * N * sizeof(float) / ms / 1e6; // read + write
 std::cout << name << ": " << ms << " ms (" << bw << " GB/s)\n";
 };

 bench(relu_scalar, "Scalar ");
 bench(relu_avx2, "AVX2 ");
 bench(relu_avx2_unrolled, "AVX2 unrolled ");
 return 0;
}

```

1. Measure the effect of memory alignment. Compare aligned vs unaligned loads on large arrays.

```

// task3_alignment.cpp
// Compile: g++ -O3 -mavx2 -o task3 task3_alignment.cpp

#include <iostream>
#include <chrono>
#include <cstdlib>
#include <immintrin.h>

int main() {
 const int N = 16'000'000;

 // Aligned allocation (32-byte for AVX2)
 float* aligned = (float*)aligned_alloc(32, N * sizeof(float));

 // Unaligned: offset by 4 bytes (1 float) from aligned boundary
 float* raw = (float*)malloc((N + 1) * sizeof(float));
 float* unaligned = raw + 1; // guaranteed misaligned
}

```

```

 for (int i = 0; i < N; i++) {
 aligned[i] = 1.0f;
 unaligned[i] = 1.0f;
 }

 auto bench = [&](float* ptr, bool use_aligned, const char* name) {
 __m256 sum = _mm256_setzero_ps();
 // Warm up
 for (int i = 0; i + 8 <= N; i += 8) {
 __m256 v = use_aligned ? _mm256_load_ps(ptr + i) :
_mm256_loadu_ps(ptr + i);
 sum = _mm256_add_ps(sum, v);
 }

 auto start = std::chrono::high_resolution_clock::now();
 for (int t = 0; t < 100; t++) {
 sum = _mm256_setzero_ps();
 for (int i = 0; i + 8 <= N; i += 8) {
 __m256 v = use_aligned ? _mm256_load_ps(ptr + i) :
_mm256_loadu_ps(ptr + i);
 sum = _mm256_add_ps(sum, v);
 }
 }
 auto end = std::chrono::high_resolution_clock::now();
 double ms = std::chrono::duration<double, std::milli>(end -
start).count() / 100;
 double bw = (double)N * sizeof(float) / ms / 1e6;
 std::cout << name << ": " << ms << " ms (" << bw << " GB/s)\n";
 };

 bench(aligned, true, "Aligned load ");
 bench(unaligned, false, "Unaligned load");

 free(aligned);
 free(raw);
 return 0;
}

```

## 81 GPU Architecture and CUDA

GPUs transformed AI by providing thousands of cores for massive parallelism. This file covers GPU vs CPU design philosophy, the GPU memory hierarchy, CUDA programming in C++, the SIMT execution model, memory access patterns,

synchronisation, streams, profiling, and NVIDIA GPU generations, the knowledge needed to write and understand GPU kernels.

- For hands-on CUDA tutorials with full working examples, see the companion repository: [github.com/HenryNdubaku/cuda-tutorials](https://github.com/HenryNdubaku/cuda-tutorials).
- A modern NVIDIA GPU has over 10,000 CUDA cores. A CPU has 4-128 cores. This 100-1000x core advantage is why GPUs dominate ML: training a transformer requires trillions of multiply-add operations, and GPUs process them in parallel at a scale CPUs cannot match.
- Even if you never write CUDA kernels yourself, understanding GPU architecture explains: why batch size matters (need enough work to saturate the GPU), why memory is usually the bottleneck (not compute), and why certain operations (scatter, conditional branching) are slow on GPUs.

## 81.1 GPU vs CPU: Fundamentally Different Designs

---

- A CPU is designed for **latency**: minimise the time to complete one task. It devotes most of its transistor budget to caches, branch predictors, and out-of-order execution — all tricks to make one thread fast.
- A GPU is designed for **throughput**: maximise the number of tasks completed per second. It devotes most transistors to execution units (ALUs). Individual threads are slow, but there are thousands of them.

	CPU	GPU
Cores	4-128 (complex, fast)	1,000-20,000 (simple, slow)
Clock speed	3-5 GHz	1-2.5 GHz
Cache	Large (32 MB+ L3)	Small (per-SM shared memory)
Branch prediction	Sophisticated	None (all threads follow same path)
Best for	Low-latency, complex control flow	High-throughput, data-parallel work
	1-5 TFLOPS	30-80 TFLOPS

	CPU	GPU
Typical FLOPS (FP32)		
Memory bandwidth	50-100 GB/s	1-3 TB/s

- The GPU's memory bandwidth advantage (10-30x) is often more important than its compute advantage. Many ML operations are memory-bound (element-wise ops, normalization, attention), and the GPU's bandwidth lets it feed data to its cores fast enough.

## 81.2 GPU Memory Hierarchy

- Understanding GPU memory is critical because **memory access is the primary bottleneck**, not computation.

Memory	Size	Latency	Bandwidth	Scope
Registers	~256 KB per SM	0 cycles	Highest	Per thread
Shared memory	48-228 KB per SM	~5 cycles	~20 TB/s	Per thread block
L1 cache	128-256 KB per SM	~30 cycles		Per SM
L2 cache	4-96 MB	~200 cycles	~6 TB/s	Global
Global memory (HBM)	24-192 GB	~400 cycles	1-3.3 TB/s	Global

- **Registers** are the fastest but most limited. Each thread has a private set of registers (typically 255 max). Using too many registers per thread reduces **occupancy** (fewer threads can run simultaneously).



- **Shared memory** is programmer-managed cache shared by all threads in a block. It is the key to writing fast CUDA kernels: load a tile of data from slow global memory to fast shared memory, then compute on it. This is the **tiling** pattern that dominates GPU programming.
- **Global memory (HBM)**: the main GPU memory (VRAM). Large but slow (400 cycle latency). All data starts and ends here. The goal of kernel optimisation is to minimise global memory accesses.

## 81.3 CUDA Programming Model

- CUDA (Compute Unified Device Architecture) is NVIDIA's programming model for GPUs. You write **kernels**: functions that run on the GPU, executed by thousands of threads simultaneously.

### 81.3.1 The Hierarchy: Grids, Blocks, Threads

```

Grid (the entire launch)
├─ Block (0,0)
│ ├── Thread (0,0)
│ ├── Thread (1,0)
│ ├── Thread (2,0)
│ └─ ... (up to 1024 threads per block)
├─ Block (1,0)
│ ├── Thread (0,0)
│ └─ ...
└─ ... (millions of blocks possible)

```

- **Thread**: the smallest unit. Each thread has a unique ID (`threadIdx.x`) within its block.
- **Block**: a group of threads that can share memory and synchronise. Block ID: `blockIdx.x`. Block size: `blockDim.x` (up to 1024 threads).
- **Grid**: all blocks launched by a single kernel. Can be 1D, 2D, or 3D.
- Each thread computes its global index: `int idx = blockIdx.x * blockDim.x + threadIdx.x;`

### 81.3.2 Your First CUDA Kernel

```
// vector_add.cu – CUDA source file (.cu extension)
```

```

#include <stdio.h>

// __global__ marks this as a GPU kernel (called from CPU, runs on GPU)
__global__ void vector_add(const float* a, const float* b, float* c, int n) {
 int idx = blockIdx.x * blockDim.x + threadIdx.x;
 if (idx < n) { // bounds check (grid may be larger than data)
 c[idx] = a[idx] + b[idx];
 }
}

int main() {
 int n = 1 << 20; // ~1 million elements
 size_t bytes = n * sizeof(float);

 // Allocate host (CPU) memory
 float *h_a = new float[n];
 float *h_b = new float[n];
 float *h_c = new float[n];

 // Initialise
 for (int i = 0; i < n; i++) {
 h_a[i] = 1.0f;
 h_b[i] = 2.0f;
 }

 // Allocate device (GPU) memory
 float *d_a, *d_b, *d_c;
 cudaMalloc(&d_a, bytes);
 cudaMalloc(&d_b, bytes);
 cudaMalloc(&d_c, bytes);

 // Copy data from CPU to GPU
 cudaMemcpy(d_a, h_a, bytes, cudaMemcpyHostToDevice);
 cudaMemcpy(d_b, h_b, bytes, cudaMemcpyHostToDevice);

 // Launch kernel: 256 threads per block, enough blocks to cover n elements
 int block_size = 256;
 int grid_size = (n + block_size - 1) / block_size; // ceiling division
 vector_add<<<grid_size, block_size>>>>(d_a, d_b, d_c, n);

 // Copy result from GPU to CPU
 cudaMemcpy(h_c, d_a, bytes, cudaMemcpyDeviceToHost);

 // Verify
 printf("c[0] = %f (expected 3.0)\n", h_c[0]);

 // Free memory

```

```

 cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
 delete[] h_a; delete[] h_b; delete[] h_c;

 return 0;
 }

```

```

Compile with NVIDIA's compiler
nvcc -O3 -o vector_add vector_add.cu
./vector_add

```

#### · Key C++ concepts in CUDA:

- `__global__`: a CUDA keyword marking a kernel function. Called from CPU (host), runs on GPU (device).
- `<<<grid_size, block_size>>>`: kernel launch syntax. Specifies how many blocks and threads to use.
- `cudaMalloc` / `cudaFree`: allocate/free GPU memory (like `new/delete` but for the GPU).
- `cudaMemcpy`: copy data between CPU and GPU. This is often the biggest bottleneck (PCIe bandwidth is ~32 GB/s, while GPU memory bandwidth is ~3 TB/s).

### 81.3.3 Warps and SIMT

- The GPU executes threads in groups of 32 called **warps**. All 32 threads in a warp execute the **same instruction** at the same time (Single Instruction, Multiple Threads — SIMT). This is the GPU's equivalent of SIMD, but at the thread level.
- **Warp divergence** occurs when threads in the same warp take different branches of an `if` statement. The GPU cannot execute two different instructions simultaneously in one warp, so it executes both branches sequentially, masking out the threads that should not participate. This halves performance (or worse).

```

// BAD: warp divergence (threads in same warp take different paths)
if (threadIdx.x % 2 == 0) {
 c[idx] = a[idx] + b[idx]; // even threads do this
} else {
 c[idx] = a[idx] - b[idx]; // odd threads do this (same warp, serialised)
}

// BETTER: branchless (no divergence)
float sign = (threadIdx.x % 2 == 0) ? 1.0f : -1.0f;
c[idx] = a[idx] + sign * b[idx]; // all threads execute the same instruction

```

### 81.3.4 Memory Coalescing

- **Coalesced access:** when consecutive threads access consecutive memory addresses, the GPU combines them into a single memory transaction. This is critical for performance.

```
// GOOD: coalesced – thread 0 reads a[0], thread 1 reads a[1], ...
c[idx] = a[idx] + b[idx];

// BAD: strided – thread 0 reads a[0], thread 1 reads a[stride], ...
c[idx] = a[idx * stride] + b[idx * stride]; // stride > 1 wastes bandwidth
```

- For a warp of 32 threads, coalesced access loads 128 bytes ( $32 \times 4$  bytes for float32) in one transaction. Strided access requires multiple transactions, each loading 128 bytes but using only a fraction. A stride of 32 is the worst case: each transaction loads 128 bytes but only one thread uses 4 bytes (3% utilisation).

### 81.3.5 Shared Memory and Tiling

- The **tiling pattern** is the most important GPU optimisation technique. The idea: load a block of data from slow global memory into fast shared memory, compute on it, then write results back.

```
// Matrix multiply with shared memory tiling (simplified)
__global__ void matmul_tiled(const float* A, const float* B, float* C,
 int M, int N, int K) {
 // Shared memory for one tile of A and one tile of B
 __shared__ float tile_A[TILE_SIZE][TILE_SIZE];
 __shared__ float tile_B[TILE_SIZE][TILE_SIZE];

 int row = blockIdx.y * TILE_SIZE + threadIdx.y;
 int col = blockIdx.x * TILE_SIZE + threadIdx.x;
 float sum = 0.0f;

 // Loop over tiles
 for (int t = 0; t < (K + TILE_SIZE - 1) / TILE_SIZE; t++) {
 // Load one tile of A and B into shared memory
 if (row < M && t * TILE_SIZE + threadIdx.x < K)
 tile_A[threadIdx.y][threadIdx.x] = A[row * K + t * TILE_SIZE +
threadIdx.x];
 else
 tile_A[threadIdx.y][threadIdx.x] = 0.0f;

 if (col < N && t * TILE_SIZE + threadIdx.y < K)
```

```

 tile_B[threadIdx.y][threadIdx.x] = B[(t * TILE_SIZE + threadIdx.y) *
N + col];

 else
 tile_B[threadIdx.y][threadIdx.x] = 0.0f;

 __syncthreads(); // wait for all threads to finish loading

 // Compute partial dot product from this tile
 for (int k = 0; k < TILE_SIZE; k++) {
 sum += tile_A[threadIdx.y][k] * tile_B[k][threadIdx.x];
 }

 __syncthreads(); // wait before loading the next tile
 }

 if (row < M && col < N)
 C[row * N + col] = sum;
}

```

- **\_\_shared\_\_**: declares memory shared by all threads in the block (fast, on-chip).
- **\_\_syncthreads()**: a barrier that waits until all threads in the block have reached this point. Required between writing to shared memory and reading from it (otherwise some threads read stale data).
- **Why tiling works**: without it, each thread loads from global memory for every multiply. With tiling, a  $\text{TILE\_SIZE} \times \text{TILE\_SIZE}$  block of data is loaded once into shared memory and reused by all threads in the block. The reuse factor is  $\text{TILE\_SIZE}$ , reducing global memory traffic by that factor.

## 81.4 Streams and Concurrency

- By default, CUDA operations are sequential: the CPU launches a kernel, waits for it to finish, then launches the next one. **Streams** enable overlapping:

```

cudaStream_t stream1, stream2;
cudaStreamCreate(&stream1);
cudaStreamCreate(&stream2);

// These operations can overlap: different streams execute concurrently
cudaMemcpyAsync(d_a, h_a, bytes, cudaMemcpyHostToDevice, stream1);
cudaMemcpyAsync(d_b, h_b, bytes, cudaMemcpyHostToDevice, stream2);

```

```
kernel1<<<grid, block, 0, stream1>>>(d_a, d_c);
kernel2<<<grid, block, 0, stream2>>>(d_b, d_d);
```

- Streams overlap data transfer with computation: while one stream's kernel runs, another stream copies data. This hides the PCIe transfer latency and keeps the GPU busy.

## 81.5 Profiling CUDA Code

---

```
NVIDIA Nsight Compute: kernel-level profiling
ncu --set full ./my_program

NVIDIA Nsight Systems: system-level timeline
nsys profile ./my_program

Quick metrics
ncu --metrics sm__throughput,dram__throughput ./my_program
```

- **What to look for:**
  - **Occupancy:** fraction of the SM's capacity that is used. Low occupancy (< 50%) means too few threads to hide memory latency. Causes: too many registers per thread, too much shared memory per block.
  - **Memory throughput:** compare to peak bandwidth. If you achieve < 50% of peak, memory access patterns are inefficient (non-coalesced, bank conflicts).
  - **Compute throughput:** compare to peak FLOPS. If both memory and compute throughput are low, the kernel is latency-bound (not enough parallelism).

## 81.6 Advanced Optimisation Techniques

---

- Beyond the basics of coalescing and shared memory tiling, high-performance GPU (and CPU) code uses several advanced techniques:

### 81.6.1 Data Layout: AoS vs SoA

- **Array of Structures (AoS):** each element stores all its fields together.  $\{ \{x, y, z\}, \{x, y, z\}, \{x, y, z\} \}$ .
- **Structure of Arrays (SoA):** each field is stored in its own contiguous array.  $\{ [x, x, x], [y, y, y], [z, z, z] \}$ .

```

// AoS: BAD for SIMD/GPU (accessing all x values touches non-contiguous memory)
struct Particle { float x, y, z, mass; };
Particle particles[N];
// particles[0].x, particles[1].x are 16 bytes apart

// SoA: GOOD for SIMD/GPU (all x values are contiguous)
struct Particles {
 float x[N], y[N], z[N], mass[N];
};
// x[0], x[1] are 4 bytes apart – perfect for coalesced access and SIMD

```

- SoA is almost always faster for data-parallel workloads (SIMD, GPU). AoS is better when you always access all fields of one element together (rare in numerical code). PyTorch tensors are SoA by nature: each feature is a contiguous dimension.

### 81.6.2 Software Prefetching

- The CPU can be told to start loading data before it is needed, hiding memory latency:

```

#include <xmmintrin.h> // for _mm_prefetch

for (int i = 0; i < n; i += 4) {
 _mm_prefetch((char*)(a + i + 64), _MM_HINT_T0); // prefetch 64 elements
 // process a[i:i+4] with SIMD
 __m128 va = _mm_load_ps(a + i);
 // ...
}

```

- The prefetch instruction is a hint: if the data is already in cache, it is a no-op. If not, the CPU starts fetching it in the background while executing other instructions. The prefetch distance (64 elements ahead in this example) should be tuned to match the memory latency and loop iteration time.

### 81.6.3 Kernel Fusion

- **Kernel fusion** combines multiple operations into a single kernel to avoid writing intermediate results to memory. This is the single most impactful GPU optimisation for ML:

```
// UNFUSED: 3 kernel launches, 3 global memory round-trips
y = matmul(x, W) // write y to global memory
z = y + bias // read y, write z
out = relu(z) // read z, write out

// FUSED: 1 kernel launch, 1 global memory write
out = fused_matmul_bias_relu(x, W, bias) // y and z never leave SRAM
```

- For memory-bound operations (bias add, ReLU, layer norm), the memory traffic dominates execution time. Fusing eliminates the traffic entirely. PyTorch's `torch.compile` and Triton enable fusion automatically or with minimal effort.

#### 81.6.4 Mixed-Precision Kernels

- Using lower precision (FP16, BF16, INT8) for computation and higher precision (FP32) for accumulation gives the best of both worlds:

```
// Tensor Core: multiply FP16 matrices, accumulate in FP32
// Each Tensor Core instruction: D (FP32) = A (FP16) × B (FP16) + C (FP32)
nvcuda::wmma::mma_sync(c_frag, a_frag, b_frag, c_frag);
```

- FP16 is 2x smaller than FP32, so it doubles memory bandwidth (the usual bottleneck) and fits 2x more data in cache. Tensor Cores process FP16 at 8-16x the rate of FP32 CUDA cores. This is why mixed-precision training (chapter 6) provides 2-3x speedup with minimal accuracy loss.

#### 81.6.5 Memory Pool Allocators

- `cudaMalloc` is slow (~1 ms per call) because it synchronises with the GPU. In a training loop that allocates temporary buffers every iteration, this adds up.
- **Memory pools** (PyTorch's caching allocator, CUDA memory pools) pre-allocate a large block of GPU memory and sub-allocate from it without system calls:

```
PyTorch does this automatically – but understanding why matters
Each torch.empty() reuses memory from the pool, no cudaMalloc
temp = torch.empty(1024, 1024, device='cuda') # microseconds, not milliseconds
```

- This is why PyTorch's `torch.cuda.memory_allocated()` and `torch.cuda.max_memory_allocated()` differ: allocated is what is currently in use, max is the peak (the pool may hold more than is currently used).



### 81.6.6 Profile-Guided Optimisation

- Do not optimise blindly. **Profile first**, identify the bottleneck, optimise that, and re-profile. The roofline model (file 01) tells you whether the bottleneck is memory or compute:
  - **Memory-bound** (low arithmetic intensity): optimise data layout (SoA), fuse kernels, use lower precision, prefetch.
  - **Compute-bound** (high arithmetic intensity): use Tensor Cores, increase parallelism, use faster instructions (FMA).
  - **Latency-bound** (insufficient parallelism): increase occupancy, reduce register usage, launch more threads.
- Most ML workloads are **memory-bound**. The surprising implication: a faster GPU (more FLOPS) often does not help. Faster memory (HBM3 vs HBM2e) helps more. This is why the A100→H100 upgrade is not just about FLOPS — the H100 also has 2x the memory bandwidth.

## 81.7 NVIDIA GPU Generations

Generation	Year	Key Innovation	AI Relevance
Pascal (P100)	2016	HBM2, NVLink	First serious deep learning GPU
Volta (V100)	2017	<b>Tensor Cores</b> (mixed-precision matmul)	Enabled FP16 training, 125 TFLOPS TF32
Ampere (A100)	2020	TF32, Sparsity, 3rd gen Tensor Cores	312 TFLOPS TF32, structural sparsity 2:4
Hopper (H100)	2022	<b>Transformer Engine</b> (FP8), HBM3	989 TFLOPS FP8, dynamic precision switching
Blackwell (B200)	2024	2nd gen Transformer Engine, NVLink 5	2.5 PFLOPS FP4, multi-die design

- **Tensor Cores** are specialised matrix multiply units. A single Tensor Core instruction computes a  $4 \times 4$  matrix multiply ( $D = A \times B + C$ ) in one cycle.

Regular CUDA cores would need 64 FMA operations. Tensor Cores are why mixed-precision training (float16 compute, float32 accumulation) is fast.

- **The Transformer Engine** (Hopper+) dynamically switches between FP8 and FP16 precision within a single layer, choosing higher precision only where needed. This maximises throughput without sacrificing model quality. It is specifically designed for transformer architectures (attention + MLP), which dominate modern AI.

## 81.8 Coding Tasks (compile with nvcc)

---

1. Write a CUDA kernel that applies ReLU to an array. Measure the time including memory transfers. This teaches kernel writing, cudaMalloc/cudaMemcpy, and the host↔device transfer bottleneck.

```
// task1_relu.cu
// Compile: nvcc -O3 -o task1_relu task1_relu.cu

#include <stdio.h>
#include <stdlib.h>
#include <cuda_runtime.h>

__global__ void relu_kernel(const float* input, float* output, int n) {
 int idx = blockIdx.x * blockDim.x + threadIdx.x;
 if (idx < n) {
 output[idx] = input[idx] > 0.0f ? input[idx] : 0.0f;
 }
}

int main() {
 const int N = 1 << 24; // ~16M elements
 size_t bytes = N * sizeof(float);

 // Allocate host memory
 float* h_input = (float*)malloc(bytes);
 float* h_output = (float*)malloc(bytes);
 for (int i = 0; i < N; i++) {
 h_input[i] = (float)(i % 100) - 50.0f; // mix of positive and negative
 }

 // Allocate device memory
 float *d_input, *d_output;
 cudaMalloc(&d_input, bytes);
 cudaMalloc(&d_output, bytes);
```

```

// Time the full pipeline: copy to GPU, compute, copy back
cudaEvent_t start, stop;
cudaEventCreate(&start);
cudaEventCreate(&stop);

cudaEventRecord(start);
cudaMemcpy(d_input, h_input, bytes, cudaMemcpyHostToDevice);

int block_size = 256;
int grid_size = (N + block_size - 1) / block_size;
relu_kernel<<<grid_size, block_size>>>(d_input, d_output, N);

cudaMemcpy(h_output, d_output, bytes, cudaMemcpyDeviceToHost);
cudaEventRecord(stop);
cudaEventSynchronize(stop);

float ms = 0;
cudaEventElapsedTime(&ms, start, stop);

// Verify
int errors = 0;
for (int i = 0; i < N; i++) {
 float expected = h_input[i] > 0.0f ? h_input[i] : 0.0f;
 if (h_output[i] != expected) errors++;
}

printf("Time (including transfers): %.2f ms\n", ms);
printf("Bandwidth: %.1f GB/s\n", 2.0 * bytes / ms / 1e6); // read + write
printf("Errors: %d / %d\n", errors, N);

cudaFree(d_input); cudaFree(d_output);
free(h_input); free(h_output);
return 0;
}

```

1. Write a tiled matrix multiply in CUDA using shared memory. Compare the performance against a naive (non-tiled) version. This teaches shared memory, \_\_syncthreads, and why tiling matters.

```

// task2_matmul.cu
// Compile: nvcc -O3 -o task2_matmul task2_matmul.cu

#include <stdio.h>
#include <cuda_runtime.h>

```

```

#define TILE 16

// Naive matmul: each thread computes one element of C
__global__ void matmul_naive(const float* A, const float* B, float* C, int N) {
 int row = blockIdx.y * blockDim.y + threadIdx.y;
 int col = blockIdx.x * blockDim.x + threadIdx.x;
 if (row < N && col < N) {
 float sum = 0.0f;
 for (int k = 0; k < N; k++) {
 sum += A[row * N + k] * B[k * N + col];
 }
 C[row * N + col] = sum;
 }
}

// Tiled matmul: use shared memory to reduce global memory accesses
__global__ void matmul_tiled(const float* A, const float* B, float* C, int N) {
 __shared__ float sA[TILE][TILE];
 __shared__ float sB[TILE][TILE];

 int row = blockIdx.y * TILE + threadIdx.y;
 int col = blockIdx.x * TILE + threadIdx.x;
 float sum = 0.0f;

 for (int t = 0; t < (N + TILE - 1) / TILE; t++) {
 sA[threadIdx.y][threadIdx.x] = (row < N && t*TILE+threadIdx.x < N)
 ? A[row * N + t*TILE + threadIdx.x] : 0.0f;
 sB[threadIdx.y][threadIdx.x] = (t*TILE+threadIdx.y < N && col < N)
 ? B[(t*TILE + threadIdx.y) * N + col] : 0.0f;

 __syncthreads();
 for (int k = 0; k < TILE; k++)
 sum += sA[threadIdx.y][k] * sB[k][threadIdx.x];
 __syncthreads();
 }

 if (row < N && col < N)
 C[row * N + col] = sum;
}

int main() {
 const int N = 1024;
 size_t bytes = N * N * sizeof(float);

 float *d_A, *d_B, *d_C;
 cudaMalloc(&d_A, bytes); cudaMalloc(&d_B, bytes); cudaMalloc(&d_C, bytes);

```

```

// Initialise with ones (easy to verify: C should be all N)
float* h_A = new float[N*N];
for (int i = 0; i < N*N; i++) h_A[i] = 1.0f;
cudaMemcpy(d_A, h_A, bytes, cudaMemcpyHostToDevice);
cudaMemcpy(d_B, h_A, bytes, cudaMemcpyHostToDevice);

dim3 block(TILE, TILE);
dim3 grid((N+TILE-1)/TILE, (N+TILE-1)/TILE);

// Benchmark naive
cudaEvent_t start, stop;
cudaEventCreate(&start); cudaEventCreate(&stop);

cudaEventRecord(start);
for (int i = 0; i < 10; i++)
 matmul_naive<<<grid, block>>>(d_A, d_B, d_C, N);
cudaEventRecord(stop);
cudaEventSynchronize(stop);
float naive_ms; cudaEventElapsedTime(&naive_ms, start, stop);

// Benchmark tiled
cudaEventRecord(start);
for (int i = 0; i < 10; i++)
 matmul_tiled<<<grid, block>>>(d_A, d_B, d_C, N);
cudaEventRecord(stop);
cudaEventSynchronize(stop);
float tiled_ms; cudaEventElapsedTime(&tiled_ms, start, stop);

double gflops_naive = 2.0 * N * N * N * 10 / naive_ms / 1e6;
double gflops_tiled = 2.0 * N * N * N * 10 / tiled_ms / 1e6;

printf("Naive: %.2f ms, %.1f GFLOPS\n", naive_ms/10, gflops_naive);
printf("Tiled: %.2f ms, %.1f GFLOPS\n", tiled_ms/10, gflops_tiled);
printf("Speedup: %.1fx\n", naive_ms / tiled_ms);

cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
delete[] h_A;
return 0;
}

```

1. Demonstrate warp divergence. Write a kernel where threads in the same warp take different branches, and compare against a branchless version.

```

// task3_divergence.cu
// Compile: nvcc -O3 -o task3_diverge task3_divergence.cu

```

```

#include <stdio.h>
#include <cuda_runtime.h>

// BAD: warp divergence – even/odd threads take different paths
__global__ void divergent_kernel(float* data, int n) {
 int idx = blockIdx.x * blockDim.x + threadIdx.x;
 if (idx < n) {
 if (idx % 2 == 0) {
 data[idx] = data[idx] * 2.0f + 1.0f;
 } else {
 data[idx] = data[idx] * 0.5f - 1.0f;
 }
 }
}

// GOOD: branchless – all threads execute the same instruction
__global__ void branchless_kernel(float* data, int n) {
 int idx = blockIdx.x * blockDim.x + threadIdx.x;
 if (idx < n) {
 float scale = (idx % 2 == 0) ? 2.0f : 0.5f;
 float offset = (idx % 2 == 0) ? 1.0f : -1.0f;
 data[idx] = data[idx] * scale + offset;
 }
}

int main() {
 const int N = 1 << 24;
 float* d_data;
 cudaMalloc(&d_data, N * sizeof(float));
 cudaMemset(d_data, 0, N * sizeof(float));

 int block = 256, grid = (N + block - 1) / block;

 cudaEvent_t start, stop;
 cudaEventCreate(&start); cudaEventCreate(&stop);

 // Divergent
 cudaEventRecord(start);
 for (int i = 0; i < 100; i++)
 divergent_kernel<<<grid, block>>>(d_data, N);
 cudaEventRecord(stop);
 cudaEventSynchronize(stop);
 float div_ms; cudaEventElapsedTime(&div_ms, start, stop);

 // Branchless
 cudaEventRecord(start);
 for (int i = 0; i < 100; i++)

```

```

 branchless_kernel<<<grid, block>>>(d_data, N);
 cudaEventRecord(stop);
 cudaEventSynchronize(stop);
 float nodiv_ms; cudaEventElapsedTime(&nodiv_ms, start, stop);

 printf("Divergent: %.2f ms\n", div_ms / 100);
 printf("Branchless: %.2f ms\n", nodiv_ms / 100);
 printf("Speedup: %.2fx\n", div_ms / nodiv_ms);

 cudaFree(d_data);
 return 0;
 }

```

## 82 Triton and TPUs

CUDA C is powerful but verbose. Triton lets you write GPU kernels in Python. TPUs offer an alternative to GPUs with different tradeoffs. This file covers Triton kernel programming, Flash Attention as a case study, TPU architecture and JAX/Pallas, and how to choose the right tool. For Vulkan and cross-platform GPU compute, see file 07.

- The previous file taught GPU programming in CUDA C. This file climbs the abstraction ladder: Triton gives you 80% of CUDA's performance with 20% of the effort, in Python. TPUs and Vulkan provide alternative hardware targets for specific use cases.

### 82.1 Triton: GPU Kernels in Python

---

- **Triton** (OpenAI) is a Python-based language for writing GPU kernels. Instead of reasoning about individual threads (CUDA), you reason about **blocks** of data. Triton's compiler handles thread mapping, memory coalescing, shared memory management, and many optimisations automatically.
- **Why Triton matters:** CUDA C requires deep knowledge of warp scheduling, shared memory bank conflicts, register pressure, and coalescing patterns. Triton abstracts most of this away, making GPU kernel development accessible to ML researchers who know Python but not systems programming.

## 82.1.1 Your First Triton Kernel

```
import triton
import triton.language as tl
import torch

@triton.jit
def add_kernel(
 x_ptr, y_ptr, output_ptr,
 n_elements,
 BLOCK_SIZE: tl.constexpr, # compile-time constant
):
 # Each program instance processes one block of BLOCK_SIZE elements
 pid = tl.program_id(axis=0) # which block am I?
 block_start = pid * BLOCK_SIZE

 # Offsets for this block
 offsets = block_start + tl.arange(0, BLOCK_SIZE)

 # Mask to handle the case where n_elements is not a multiple of BLOCK_SIZE
 mask = offsets < n_elements

 # Load data (masked: out-of-bounds reads return 0)
 x = tl.load(x_ptr + offsets, mask=mask)
 y = tl.load(y_ptr + offsets, mask=mask)

 # Compute
 output = x + y

 # Store result
 tl.store(output_ptr + offsets, output, mask=mask)

def add(x: torch.Tensor, y: torch.Tensor) -> torch.Tensor:
 output = torch.empty_like(x)
 n_elements = output.numel()

 # Launch: one program per block
 grid = lambda meta: (triton.cdiv(n_elements, meta['BLOCK_SIZE']),)
 add_kernel[grid](x, y, output, n_elements, BLOCK_SIZE=1024)

 return output

Usage
x = torch.randn(1000000, device='cuda')
```



```
y = torch.randn(1000000, device='cuda')
z = add(x, y)
```

- **Key differences from CUDA:**

- No explicit thread management. You think in **blocks** (programs), not threads.
- `tl.arange(0, BLOCK_SIZE)` creates a vector of offsets for the entire block. All operations on this vector are implicitly vectorised.
- `mask` handles boundary conditions (like AVX-512 mask registers, file 03). No scalar cleanup loop needed.
- `tl.load` and `tl.store` handle coalesced access automatically.
- `@triton.jit` compiles the function to PTX (GPU assembly) at first call, then caches the compiled kernel.

### 82.1.2 Triton Softmax Kernel

- Softmax is a great Triton example because it requires multiple passes over the data (max, subtract, exp, sum, divide) and benefits from keeping data in SRAM (shared memory) between passes:

```
@triton.jit
def softmax_kernel(
 output_ptr, input_ptr, input_row_stride, output_row_stride, n_cols,
 BLOCK_SIZE: tl.constexpr,
):
 # Each program handles one row
 row_idx = tl.program_id(0)
 row_start = input_ptr + row_idx * input_row_stride

 # Load the row
 col_offsets = tl.arange(0, BLOCK_SIZE)
 mask = col_offsets < n_cols
 row = tl.load(row_start + col_offsets, mask=mask, other=-float('inf'))

 # Softmax: max for numerical stability, then exp, then normalise
 row_max = tl.max(row, axis=0)
 numerator = tl.exp(row - row_max)
 denominator = tl.sum(numerator, axis=0)
 softmax_output = numerator / denominator

 # Store result
```

```
output_start = output_ptr + row_idx * output_row_stride
tl.store(output_start + col_offsets, softmax_output, mask=mask)
```

- In PyTorch, `F.softmax(x, dim=-1)` launches 3 separate kernels (max, exp-and-sum, divide), each reading from and writing to global memory. The Triton version does everything in one kernel, keeping data in registers/SRAM. This **kernel fusion** is why custom Triton kernels can be 2-4x faster than PyTorch's built-in operations.

### 82.1.3 Triton Auto-Tuning

- Triton supports **auto-tuning**: try multiple configurations and pick the fastest:

```
@triton.autotune(
 configs=[
 triton.Config({'BLOCK_SIZE_M': 128, 'BLOCK_SIZE_N': 128, 'BLOCK_SIZE_K':
32}),
 triton.Config({'BLOCK_SIZE_M': 64, 'BLOCK_SIZE_N': 256, 'BLOCK_SIZE_K':
32}),
 triton.Config({'BLOCK_SIZE_M': 256, 'BLOCK_SIZE_N': 64, 'BLOCK_SIZE_K':
64}),
],
 key=['M', 'N', 'K'], # re-tune when these change
)
@triton.jit
def matmul_kernel(a_ptr, b_ptr, c_ptr, M, N, K, ...):
 ...
```

- Triton benchmarks each configuration on the actual hardware and selects the fastest. The optimal tile sizes depend on the GPU architecture, matrix dimensions, and memory layout — auto-tuning finds them without manual experimentation.

### 82.1.4 Triton vs CUDA: When to Use Each

	Triton	CUDA C
Language	Python	C/C++
Abstraction	Block-level	Thread-level
Development speed	Fast (10-50 lines per kernel)	Slow (100-500 lines)

	<b>Triton</b>	<b>CUDA C</b>
Performance ceiling	~80-95% of hand-tuned CUDA	100% (full hardware control)
Shared memory	Automatic	Manual
Coalescing	Automatic	Manual
Warp-level primitives	Limited	Full (shuffle, vote, etc.)
Hardware support	NVIDIA only (AMD experimental)	NVIDIA only

- **Use Triton** for: fused kernels, custom attention patterns, activation functions, most ML research kernel needs.
- **Use CUDA C** for: maximum performance (the last 5-20%), warp-level primitives, complex data-dependent parallelism, when Triton cannot express your pattern.

## 82.2 Case Study: Flash Attention

- **Flash Attention** (Dao et al., 2022) is the most impactful custom kernel in recent ML. It computes attention with  $O(n)$  memory instead of  $O(n^2)$ , enabling much longer sequences.
- **The problem:** standard attention computes  $\frac{\text{softmax}(QK^T)}{\sqrt{d}}$   $\cdot V$ .  
The  $QK^T$  matrix is  $n \times n$  where  $n$  is the sequence length. For  $n = 128K$ , this matrix is  $128K \times 128K \times 4 \text{ bytes} = 64 \text{ GB}$ . It does not fit in GPU memory.
- **The insight:** you do not need to materialise the full  $n \times n$  matrix. Compute attention in **tiles**: load a block of  $Q$ , a block of  $K$ , compute their partial attention scores, accumulate, and move to the next block. The  $n \times n$  matrix is never fully materialised — only one tile at a time exists in SRAM.
- **Online softmax:** the tricky part is softmax, which requires knowing the maximum across the entire row (for numerical stability). Flash Attention uses the **online softmax** trick: maintain a running maximum and rescale previously

computed values when a new maximum is found. This allows softmax to be computed incrementally, one tile at a time.

- The algorithm:

```
For each block of Q rows:
 For each block of K columns:
 1. Load Q_block from HBM to SRAM
 2. Load K_block from HBM to SRAM
 3. Compute S_block = Q_block @ K_block.T (in SRAM)
 4. Update running max, rescale previous results
 5. Compute exp(S_block - running_max)
 6. Update running sum and output accumulator
 Load V_block and compute final output
Write output block back to HBM
```

- **Why it is fast:** the inner loop operates entirely in SRAM (shared memory). Global memory (HBM) is only accessed to load blocks of Q, K, V and write the final output. The data reuse factor is proportional to the SRAM size, which is ~100x faster to access than HBM.
- Flash Attention is implemented in both Triton and CUDA C. The CUDA version is faster (~10% more efficient), but the Triton version is far more readable and modifiable, which matters for research on new attention variants.

## 82.3 TPU Architecture

---

- **TPUs** (Tensor Processing Units) are Google's custom ML accelerators. They take a radically different approach from GPUs:
- **Systolic arrays:** the TPU's core compute unit is a **Matrix Multiply Unit (MXU)**, a  $128 \times 128$  or  $256 \times 256$  systolic array that computes matrix multiplications by flowing data through a grid of multiply-accumulate units. Data enters from the edges and propagates through the array, with each unit performing one multiply-add and passing results to the next.
- Unlike GPUs (which schedule thousands of independent threads), the systolic array is a single, deterministic data flow. There is no thread scheduling, no warp divergence, no branch prediction. This simplicity makes the MXU extremely energy-efficient for matrix multiplication.

- **HBM:** TPUs use the same High Bandwidth Memory as GPUs. TPU v5e has 16 GB HBM2e per chip; TPU v5p has 95 GB HBM2e.
- **ICI (Inter-Chip Interconnect):** TPU pods connect hundreds of TPUs with a custom high-speed network. Data parallelism and model parallelism (chapter 6) across TPU pods are supported natively by JAX.
- **BFloat16:** TPUs were the first to use bfloat16 (chapter 13, file 02). BF16 has the same exponent range as float32 (preventing overflow during training) with less mantissa precision. This tradeoff is ideal for ML, where gradient values span a wide range but do not need 23 bits of precision.

### 82.3.1 Programming TPUs: JAX and Pallas

- TPUs are programmed through **JAX** and **XLA**. You write Python/JAX code, `jax.jit` compiles it to XLA HLO, and XLA compiles HLO to TPU-specific instructions. No CUDA, no C++.

```
import jax
import jax.numpy as jnp

@jax.jit
def matmul(a, b):
 return jnp.dot(a, b)

This runs on CPU, GPU, or TPU depending on the device
a = jnp.ones((1024, 1024))
b = jnp.ones((1024, 1024))
c = matmul(a, b)
```

- **Pallas** is JAX' s kernel authoring API — the JAX equivalent of Triton. It lets you write low-level kernels that XLA compiles for GPU or TPU:

```
from jax.experimental import pallas as pl
import jax.numpy as jnp

def add_kernel(x_ref, y_ref, o_ref):
 o_ref[...] = x_ref[...] + y_ref[...]

def add_pallas(x, y):
 return pl.pallas_call(
 add_kernel,
 out_shape=jax.ShapeDtypeStruct(x.shape, x.dtype),
 grid=(x.shape[0] // 128,),
```

```

in_specs=[pl.BlockSpec((128,), lambda i: (i,)),
 pl.BlockSpec((128,), lambda i: (i,))],
out_specs=pl.BlockSpec((128,), lambda i: (i,)),
)(x, y)

```

- Pallas is newer and less mature than Triton, but it is the only way to write custom kernels for TPUs (since TPUs do not support CUDA).

### 82.3.2 GPU vs TPU

	GPU (NVIDIA)	TPU (Google)
Availability	Any cloud, on-premise	Google Cloud only
Programming	CUDA C, Triton, PyTorch	JAX/XLA, Pallas
Flexibility	General-purpose compute	Optimised for matrix-heavy ML
Peak matmul FLOPS	Very high (Tensor Cores)	Very high (MXU)
Non-matmul ops	Good	Slower (routed through vector unit, not MXU)
Multi-chip scaling	NVLink (8 GPUs), InfiniBand	ICI (thousands of TPUs, tighter integration)
Cost efficiency	Competitive	Often cheaper for large-scale training
Ecosystem	Largest (PyTorch, TensorFlow, JAX)	JAX-focused

- **Use GPUs** for: most ML workloads, PyTorch-based research, inference serving, workloads with significant non-matmul computation.
- **Use TPUs** for: large-scale JAX training (thousands of chips), cost-sensitive training on Google Cloud, workloads that are dominated by matrix multiplies.

## 82.4 Choosing the Right Tool

Workload	Best Tool	Why
ML training (PyTorch)	NVIDIA GPU + CUDA/Triton	

Workload	Best Tool	Why
		Largest ecosystem, best tooling
ML training (JAX, large-scale)	TPU or NVIDIA GPU	TPU for cost at Google-scale, GPU for flexibility
Custom fused kernels	Triton (Python) or CUDA C	Triton for speed of development, CUDA for peak performance
JAX custom kernels	Pallas	Only option for TPU, works on GPU too
Cross-platform inference	Vulkan (file 07) or ONNX Runtime	Runs on any GPU vendor
Mobile/edge inference	Metal (Apple), Vulkan (Android), NNAPI	Platform-specific accelerators
Browser inference	WebGPU (file 07)	Only option in the browser
CPU-only inference	ONNX Runtime + AVX/NEON	No GPU needed, uses SIMD (files 02-03)
Novel hardware	Vendor-specific SDK	Each accelerator has its own toolchain

## 82.5 Coding Tasks (use CoLab with GPU runtime)

1. Write and run a Triton kernel for vector addition. Compare its performance against PyTorch's built-in addition.

```
import triton
import triton.language as tl
import torch
import time

@triton.jit
def add_kernel(x_ptr, y_ptr, out_ptr, n, BLOCK: tl.constexpr):
 pid = tl.program_id(0)
 offs = pid * BLOCK + tl.arange(0, BLOCK)
 mask = offs < n
 x = tl.load(x_ptr + offs, mask=mask)
 y = tl.load(y_ptr + offs, mask=mask)
 tl.store(out_ptr + offs, x + y, mask=mask)

n = 10_000_000
x = torch.randn(n, device='cuda')
y = torch.randn(n, device='cuda')

Triton
out_triton = torch.empty_like(x)
grid = lambda meta: (triton.cdiv(n, meta['BLOCK']),)
add_kernel[grid](x, y, out_triton, n, BLOCK=1024)

PyTorch
out_torch = x + y

Verify correctness
assert torch.allclose(out_triton, out_torch, atol=1e-5)

Benchmark
torch.cuda.synchronize()
start = time.time()
for _ in range(1000):
 add_kernel[grid](x, y, out_triton, n, BLOCK=1024)
torch.cuda.synchronize()
triton_time = (time.time() - start) / 1000

start = time.time()
for _ in range(1000):
 out_torch = x + y
torch.cuda.synchronize()
```



```

torch_time = (time.time() - start) / 1000

print(f"Triton: {triton_time*1000:.3f} ms")
print(f"PyTorch: {torch_time*1000:.3f} ms")
print(f"Ratio: {torch_time/triton_time:.2f}x")

```

1. Write a Triton fused kernel that does multiply + add + ReLU in a single pass. Compare against three separate PyTorch operations.

```

import triton
import triton.language as tl
import torch
import time

@triton.jit
def fused_mul_add_relu_kernel(x_ptr, w_ptr, b_ptr, out_ptr, n, BLOCK:
tl.constexpr):
 pid = tl.program_id(0)
 offs = pid * BLOCK + tl.arange(0, BLOCK)
 mask = offs < n
 x = tl.load(x_ptr + offs, mask=mask)
 w = tl.load(w_ptr + offs, mask=mask)
 b = tl.load(b_ptr + offs, mask=mask)
 result = tl.maximum(x * w + b, 0.0) # fused: mul + add + relu
 tl.store(out_ptr + offs, result, mask=mask)

n = 10_000_000
x = torch.randn(n, device='cuda')
w = torch.randn(n, device='cuda')
b = torch.randn(n, device='cuda')

Fused (Triton)
out_fused = torch.empty_like(x)
grid = lambda meta: (triton.cdiv(n, meta['BLOCK']),)
fused_mul_add_relu_kernel[grid](x, w, b, out_fused, n, BLOCK=1024)

Unfused (PyTorch)
out_unfused = torch.relu(x * w + b)

assert torch.allclose(out_fused, out_unfused, atol=1e-5)

Benchmark
torch.cuda.synchronize()
start = time.time()
for _ in range(1000):
 fused_mul_add_relu_kernel[grid](x, w, b, out_fused, n, BLOCK=1024)

```

```

torch.cuda.synchronize()
fused_time = (time.time() - start) / 1000

start = time.time()
for _ in range(1000):
 out_unfused = torch.relu(x * w + b)
torch.cuda.synchronize()
unfused_time = (time.time() - start) / 1000

print(f"Fused (Triton): {fused_time*1000:.3f} ms")
print(f"Unfused (PyTorch): {unfused_time*1000:.3f} ms")
print(f"Speedup: {unfused_time/fused_time:.2f}x")

```

1. Measure how JAX' s XLA compiler automatically fuses operations. Compare a chain of operations with and without jit.

```

import jax
import jax.numpy as jnp
import time

def chain_ops(x):
 x = x * 2.0
 x = x + 1.0
 x = jnp.maximum(x, 0.0) # ReLU
 x = x / jnp.sum(x)
 return x

chain_jit = jax.jit(chain_ops)
x = jax.random.normal(jax.random.PRNGKey(0), (10000, 1000))

Warm up
_ = chain_jit(x)
jax.block_until_ready(_)

Eager (each op is a separate kernel launch)
start = time.time()
for _ in range(100):
 y = chain_ops(x)
jax.block_until_ready(y)
eager_time = (time.time() - start) / 100

JIT (XLA fuses operations)
start = time.time()
for _ in range(100):
 y = chain_jit(x)
jax.block_until_ready(y)

```

```
jit_time = (time.time() - start) / 100

print(f"Eager: {eager_time*1000:.2f} ms")
print(f"JIT: {jit_time*1000:.2f} ms")
print(f"Speedup: {eager_time/jit_time:.1f}x (XLA fuses the 4 operations into 1 kernel)")
```

## 83 RISC-V and Embedded Systems

RISC-V is the open-source instruction set architecture reshaping the chip industry. This file covers the RISC-V philosophy, the V vector extension, embedded ML inference, TinyML on microcontrollers, RISC-V in AI accelerators, and edge deployment constraints

- Every chip architecture we have covered so far (x86, ARM) requires a **licence**. Intel and AMD pay for x86. Apple, Qualcomm, and every smartphone vendor pay ARM billions per year. **RISC-V** is different: it is an open standard. Anyone can design, manufacture, and sell RISC-V chips without paying royalties to anyone. This is changing the economics of chip design, especially for AI.

### 83.1 The RISC-V Philosophy

---

- RISC-V (pronounced “risk five” ) was created at UC Berkeley in 2010 as a clean, modern RISC instruction set. The key principles:
  - **Open standard**: the ISA specification is freely available. You can build a RISC-V CPU without licensing fees, NDAs, or legal agreements. This is like how Linux is to operating systems — anyone can use, modify, and build on it.
  - **Modular design**: the base ISA (RV32I or RV64I) is minimal — just 47 instructions. Everything else is optional **extensions**: M (multiply/divide), A (atomic operations), F/D (floating point), C (compressed instructions), V (vector processing). You pick only what you need, keeping the chip small and efficient.
  - **No legacy baggage**: x86 carries 45 years of backward compatibility. ARM carries 35 years. RISC-V starts clean, incorporating lessons learned from

both. No obscure instructions that exist only for compatibility with 1980s software.

- **Who uses RISC-V:** SiFive (general-purpose cores), Alibaba (Xuantie server cores), Western Digital (storage controllers, billions shipped), Espressif (ESP32-C3, popular IoT chip), and dozens of AI accelerator startups that use RISC-V for the control processor managing their custom compute units.

## 83.2 RISC-V Base Architecture

---

- The base integer ISA (RV64I for 64-bit) has:
  - **32 general-purpose registers** (x0-x31, each 64-bit). x0 is hardwired to zero (useful for implementing common patterns without special instructions).
  - **Fixed 32-bit instruction width** (with the C extension adding 16-bit compressed instructions for code density).
  - **Load-store architecture:** like ARM, arithmetic operates on registers only. Memory access is through explicit load/store instructions.

```
RISC-V assembly (for flavour – you will use C/C++)
add x3, x1, x2 # x3 = x1 + x2
lw x4, 0(x5) # load word from address in x5
sw x4, 8(x5) # store word to address x5 + 8
beq x1, x2, label # branch if x1 == x2
```

- The simplicity of the ISA makes RISC-V cores small and power-efficient. A minimal RV32I core can be implemented in ~10,000 gates (an ARM Cortex-M0 is ~12,000). This matters for embedded systems where every milliwatt and every square millimetre of silicon counts.

## 83.3 The V Extension: RISC-V Vector Processing

---

- The **V extension** (RVV) adds scalable vector processing to RISC-V, similar to ARM SVE. Vector registers have a configurable length (VLEN), specified by the hardware (128 to 65,536 bits). Code is written to be **vector-length agnostic**: it works on any VLEN without recompilation.

```
#include <riscv_vector.h>

// Vector addition using RVV intrinsics
void vadd_rvv(const float* a, const float* b, float* c, int n) {
```

```

while (n > 0) {
 // vsetvl: set vector length – processes min(n, hardware_max) elements
 size_t vl = __riscv_vsetvl_e32m1(n);

 // Load vl elements
 vfloat32m1_t va = __riscv_vle32_v_f32m1(a, vl);
 vfloat32m1_t vb = __riscv_vle32_v_f32m1(b, vl);

 // Add
 vfloat32m1_t vc = __riscv_vfadd_vv_f32m1(va, vb, vl);

 // Store
 __riscv_vse32_v_f32m1(c, vc, vl);

 // Advance pointers
 a += vl; b += vl; c += vl; n -= vl;
}
}

```

- **vsetvl** is the key instruction. It tells the hardware “I want to process this many elements” and the hardware responds “I can process this many” (capped by VLEN). The loop automatically adapts to any vector width, with no scalar cleanup needed (the last iteration simply processes fewer elements).
- **LMUL** (length multiplier): RVV can group multiple vector registers together (m1, m2, m4, m8) to process more elements per instruction at the cost of fewer available registers. m1 uses one register per vector operand; m8 uses eight, processing 8x more elements but leaving only 4 register groups available.
- Compared to x86 AVX (fixed 256/512-bit) and ARM NEON (fixed 128-bit), RVV’s scalability is a major advantage for diverse hardware: the same code runs on a tiny embedded core (VLEN=128) and a high-performance server core (VLEN=1024+).

## 83.4 Embedded ML: TinyML

---

- **TinyML** is machine learning on microcontrollers — devices with kilobytes of RAM, megahertz-class CPUs, and milliwatt power budgets. Think: a sensor that detects keywords ( “Hey Siri” ), an accelerometer that classifies gestures, or a camera that counts people, all running on a chip costing \$0.50 with no internet connection.
- The constraints are extreme:

Resource	Server GPU	Smartphone	Microcontroller
RAM	80 GB	6 GB	256 KB
Storage	TB	128 GB	1 MB
Compute	1000 TFLOPS	10 TFLOPS	0.001 TFLOPS
Power	700 W	5 W	0.001 W
Cost	\$30,000	\$500	\$1

- A model that fits on a server GPU ( $O(10^{10})$  parameters) will not fit on a microcontroller. TinyML models have  $O(10^4)$ – $O(10^6)$  parameters and use INT8 or even INT4 quantisation.

### 83.4.1 TensorFlow Lite Micro (TFLM)

- **TFLM** is Google's inference framework for microcontrollers. It runs quantised TensorFlow Lite models without dynamic memory allocation, without an OS, and with a ~20 KB binary footprint.

```
// TinyML inference on a microcontroller (simplified)
#include "tensorflow/lite/micro/micro_interpreter.h"
#include "tensorflow/lite/micro/micro_mutable_op_resolver.h"

// Model is compiled into a C array (const unsigned char model_data[])
const tflite::Model* model = tflite::GetModel(model_data);

// Allocate a fixed memory arena (no malloc!)
constexpr int kArenaSize = 10 * 1024; // 10 KB
uint8_t tensor_arena[kArenaSize];

// Set up interpreter
tflite::MicroInterpreter interpreter(model, resolver, tensor_arena, kArenaSize);
interpreter.AllocateTensors();

// Set input
float* input = interpreter.input(0)->data.f;
input[0] = sensor_reading;

// Run inference
interpreter.Invoke();

// Read output
float* output = interpreter.output(0)->data.f;
```

```
if (output[0] > 0.8f) {
 trigger_alert();
}
```

- **Key constraints in this code:**

- `tensor_arena` is statically allocated — no `malloc`, no heap. Embedded systems often have no dynamic memory allocator.
- The model is a `const` byte array, stored in flash memory (ROM), not loaded from a file system.
- The entire framework + model + runtime fits in a few tens of KB.

### 83.4.2 Model Optimisation for Edge

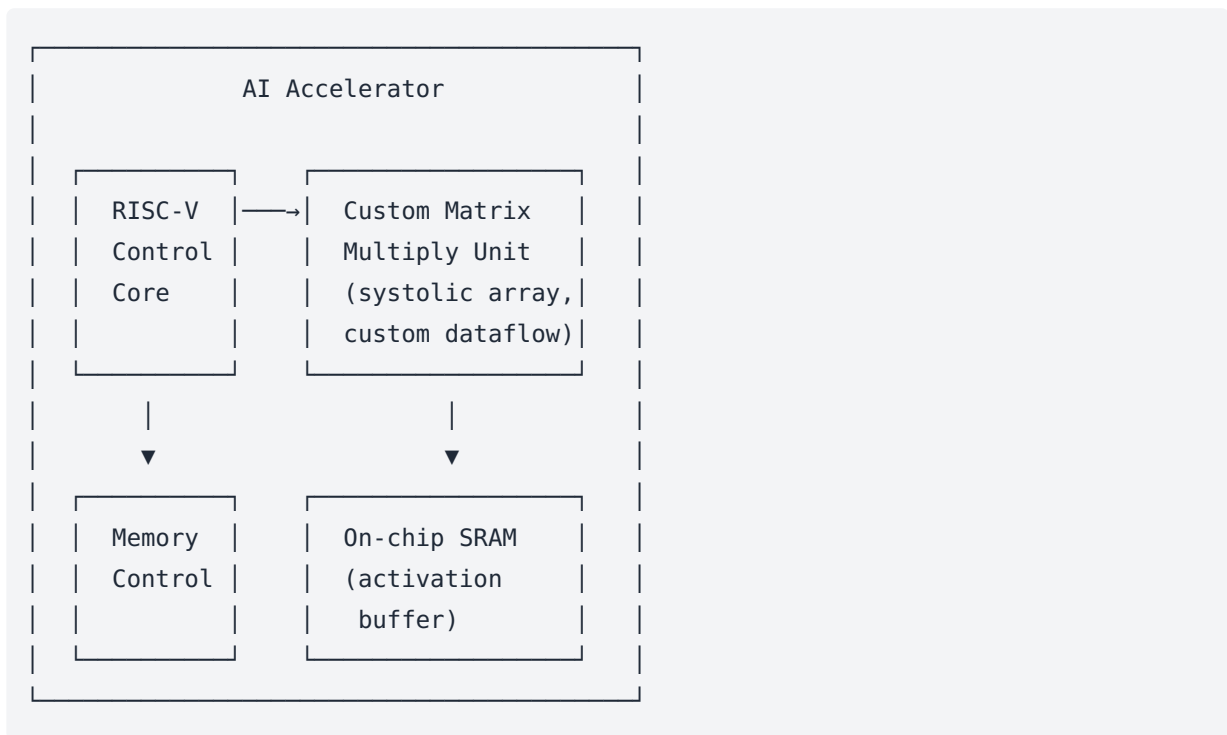
- Getting a model to run on a microcontroller requires aggressive optimisation:

- **Quantisation** (chapter 18): convert float32 weights to INT8 (4x smaller, 2-4x faster on integer-only hardware). Post-training quantisation is simple; quantisation-aware training preserves more accuracy.
- **Pruning**: remove weights close to zero. Structured pruning (remove entire channels/heads) is more hardware-friendly than unstructured pruning (random zeros) because it reduces actual computation, not just storage.
- **Knowledge distillation** (chapter 6): train a small “student” model to mimic a large “teacher” model. The student achieves higher accuracy than training from scratch, because it learns from the teacher’s soft predictions.
- **Neural Architecture Search (NAS)**: automatically search for efficient architectures that fit within a hardware budget (latency, memory, power). **MicroNets** and **MCUNet** find architectures optimised for specific microcontrollers.
- **Operator fusion**: combine conv + batch norm + ReLU into a single fused operation, eliminating intermediate memory writes (the same principle as GPU kernel fusion, but even more critical when you have 256 KB of RAM).

## 83.5 RISC-V in AI Accelerators

---

- Many AI accelerator startups use RISC-V not for running ML models directly, but as the **control processor** that manages the custom compute units:



- The RISC-V core handles: loading model weights from external memory, scheduling layer execution, managing data flow between compute units, and communicating with the host (via PCIe, USB, or SPI). The heavy computation (matrix multiplies, convolutions) is done by the custom hardware, not the RISC-V core.
- **Why RISC-V for control:** no licensing cost (critical for startups), customisable (add domain-specific instructions), small footprint (a control core does not need x86's complexity), and the open ecosystem enables rapid prototyping.
- **Examples:** Esperanto Technologies (1000+ RISC-V cores for ML), Tenstorrent (RISC-V control + custom tensix cores), SiFive (RISC-V cores with vector extensions for edge ML).

## 83.6 Edge Deployment Constraints

- Deploying ML at the edge (on-device, not in the cloud) introduces constraints that cloud deployment does not face:
- **Power:** a battery-powered device might have a total power budget of 100 mW. Running a model that consumes 50 mW leaves only 50 mW for the rest of the system (sensors, radio, display). Power-aware inference schedules computation to avoid thermal throttling and extend battery life.



- **Latency:** edge inference must often be real-time. A wake-word detector ( “Hey Siri” ) must respond within ~200 ms. An autonomous driving perception system (chapter 11) must process frames within ~30 ms. Network round-trip to the cloud (50-200 ms) is too slow for these use cases.
- **Privacy:** processing data on-device means sensitive data (medical images, voice recordings, personal photos) never leaves the device. This is a legal requirement in some jurisdictions (GDPR) and a user trust requirement everywhere.
- **Connectivity:** edge devices may have intermittent or no internet connection. A model running on a Mars rover (chapter 11), a submarine, or a rural farm sensor must work entirely offline.
- **Cost at scale:** deploying ML to a billion smartphones costs \$0 per device (the hardware already exists). Deploying to a billion IoT sensors means each sensor’ s ML hardware budget is pennies. RISC-V’ s zero licensing cost matters enormously at this scale.

## 83.7 Coding Tasks (compile with g++ or riscv64-gcc cross-compiler)

---

1. Write a C program that simulates a TinyML inference pipeline: statically allocate a model buffer, run a mock forward pass, and measure resource usage. This teaches the embedded constraints (no malloc, fixed memory arena).

```
// task1_tinymml_sim.cpp
// Compile: g++ -O2 -o task1 task1_tinymml_sim.cpp

#include <iostream>
#include <chrono>
#include <cmath>
#include <cstring>

// Simulate a microcontroller: fixed memory arena, no dynamic allocation
static constexpr int ARENA_SIZE = 32 * 1024; // 32 KB total RAM budget
static uint8_t arena[ARENA_SIZE];

// Simple 2-layer MLP: 784 -> 64 -> 10 (MNIST-like, INT8 weights)
struct TinyModel {
 int8_t w1[784 * 64]; // layer 1 weights: 50,176 bytes
 int8_t b1[64]; // layer 1 biases
 int8_t w2[64 * 10]; // layer 2 weights: 640 bytes
```

```

 int8_t b2[10]; // layer 2 biases
 // Total: ~51 KB → must go in flash (ROM), not RAM
};

// Check if model fits in flash
void check_model_fit(int flash_kb) {
 int model_bytes = sizeof(TinyModel);
 std::cout << "Model size: " << model_bytes << " bytes ("
 << model_bytes / 1024 << " KB)\n";
 std::cout << "Flash: " << flash_kb << " KB → "
 << (model_bytes <= flash_kb * 1024 ? "FITS" : "TOO LARGE") << "\n";
}

// Mock inference using the fixed arena for activations
void mock_inference(const int8_t* input, int8_t* output) {
 // Activations go in the arena (RAM), not allocated dynamically
 int8_t* act1 = (int8_t*)arena; // 64 bytes for layer 1 output
 int8_t* act2 = (int8_t*)(arena + 64); // 10 bytes for layer 2 output

 // Layer 1: simplified matmul (not real quantised matmul, just structure
demo)
 for (int j = 0; j < 64; j++) {
 int32_t sum = 0; // accumulate in int32 to avoid overflow
 for (int i = 0; i < 784; i++) {
 sum += (int32_t)input[i] * 1; // mock: weight = 1
 }
 act1[j] = (int8_t)std::max(-128, std::min(127, sum / 784)); // quantise
back
 act1[j] = act1[j] > 0 ? act1[j] : 0; // ReLU
 }

 // Layer 2
 for (int j = 0; j < 10; j++) {
 int32_t sum = 0;
 for (int i = 0; i < 64; i++) {
 sum += (int32_t)act1[i] * 1;
 }
 act2[j] = (int8_t)std::max(-128, std::min(127, sum / 64));
 }

 std::memcpy(output, act2, 10);
}

int main() {
 std::cout << "=== TinyML Resource Budget ===\n";
 std::cout << "Arena (RAM): " << ARENA_SIZE << " bytes ("
 << ARENA_SIZE / 1024 << " KB)\n";
}

```

```

 check_model_fit(256); // typical MCU flash

// Activation memory used
int activation_bytes = 64 + 10; // layer 1 + layer 2 outputs
std::cout << "Activation memory: " << activation_bytes
 << " bytes / " << ARENA_SIZE << " available\n\n";

// Benchmark inference
int8_t input[784];
int8_t output[10];
std::memset(input, 1, 784);

auto start = std::chrono::high_resolution_clock::now();
for (int i = 0; i < 10000; i++) {
 mock_inference(input, output);
}
auto end = std::chrono::high_resolution_clock::now();
double us = std::chrono::duration<double, std::micro>(end - start).count() /
10000;

std::cout << "Inference latency: " << us << " us\n";
std::cout << "At 160 MHz MCU (~6.25 ns/cycle): ~"
 << (int)(us * 160) << " cycles\n";

std::cout << "Output logits: ";
for (int i = 0; i < 10; i++) std::cout << (int)output[i] << " ";
std::cout << "\n";

return 0;
}

```

1. Write a C++ program that quantises float32 weights to INT8 and measures the compression ratio and quantisation error.

```

// task2_quantise.cpp
// Compile: g++ -O3 -o task2 task2_quantise.cpp

#include <iostream>
#include <vector>
#include <cmath>
#include <algorithm>
#include <numeric>

// Symmetric quantisation: map float range [-max, +max] to [-127, +127]
void quantise_symmetric(const float* input, int8_t* output, int n, float& scale)
{

```

```

 float max_val = 0.0f;
 for (int i = 0; i < n; i++) {
 max_val = std::max(max_val, std::abs(input[i]));
 }
 scale = max_val / 127.0f;
 for (int i = 0; i < n; i++) {
 float scaled = input[i] / scale;
 output[i] = (int8_t)std::max(-127.0f, std::min(127.0f,
std::round(scaled)));
 }
 }

 // Dequantise: INT8 back to float
 void dequantise(const int8_t* input, float* output, int n, float scale) {
 for (int i = 0; i < n; i++) {
 output[i] = (float)input[i] * scale;
 }
 }

 int main() {
 const int N = 100000;

 // Simulate random weights (roughly normal distribution)
 std::vector<float> weights(N);
 for (int i = 0; i < N; i++) {
 // Simple pseudo-random normal-ish values
 float u1 = (float)(i * 7 % 997 + 1) / 998.0f;
 float u2 = (float)(i * 13 % 991 + 1) / 992.0f;
 weights[i] = std::sqrt(-2.0f * std::log(u1)) * std::cos(6.2832f * u2) *
0.1f;
 }

 // Quantise
 std::vector<int8_t> quantised(N);
 float scale;
 quantise_symmetric(weights.data(), quantised.data(), N, scale);

 // Dequantise and measure error
 std::vector<float> reconstructed(N);
 dequantise(quantised.data(), reconstructed.data(), N, scale);

 float max_error = 0.0f, total_error = 0.0f;
 for (int i = 0; i < N; i++) {
 float err = std::abs(weights[i] - reconstructed[i]);
 max_error = std::max(max_error, err);
 total_error += err;
 }
 }

```

```

std::cout << "=== Quantisation Results ===\n";
std::cout << "Original: " << N * 4 << " bytes (float32)\n";
std::cout << "Quantised: " << N * 1 << " bytes (int8) + 4 bytes (scale)\n";
std::cout << "Compression: " << 4.0f << "x\n";
std::cout << "Scale factor: " << scale << "\n";
std::cout << "Mean abs error: " << total_error / N << "\n";
std::cout << "Max abs error: " << max_error << "\n";
std::cout << "Max abs error / scale: " << max_error / scale
 << " (should be <= 0.5 quantisation levels)\n";

return 0;
}

```

1. Write a C++ program that performs INT8 matrix multiplication with INT32 accumulation — the actual computation that runs on embedded ML accelerators.

```

// task3_int8_matmul.cpp
// Compile: g++ -O3 -o task3 task3_int8_matmul.cpp

#include <iostream>
#include <chrono>
#include <vector>
#include <stdint>

// INT8 matmul with INT32 accumulation (what Tensor Cores and MCU accelerators
do)

void matmul_int8(const int8_t* A, const int8_t* B, int32_t* C,
 int M, int N, int K) {
 for (int i = 0; i < M; i++) {
 for (int j = 0; j < N; j++) {
 int32_t sum = 0;
 for (int k = 0; k < K; k++) {
 sum += (int32_t)A[i * K + k] * (int32_t)B[k * N + j];
 }
 C[i * N + j] = sum;
 }
 }
}

// Float32 matmul for comparison
void matmul_f32(const float* A, const float* B, float* C,
 int M, int N, int K) {
 for (int i = 0; i < M; i++) {
 for (int j = 0; j < N; j++) {

```

```

 float sum = 0.0f;
 for (int k = 0; k < K; k++) {
 sum += A[i * K + k] * B[k * N + j];
 }
 C[i * N + j] = sum;
 }
}

int main() {
 const int M = 128, N = 128, K = 128;

 std::vector<int8_t> A_i8(M * K, 1), B_i8(K * N, 1);
 std::vector<int32_t> C_i32(M * N);

 std::vector<float> A_f32(M * K, 1.0f), B_f32(K * N, 1.0f);
 std::vector<float> C_f32(M * N);

 // Benchmark INT8
 auto start = std::chrono::high_resolution_clock::now();
 for (int t = 0; t < 100; t++) {
 matmul_int8(A_i8.data(), B_i8.data(), C_i32.data(), M, N, K);
 }
 auto end = std::chrono::high_resolution_clock::now();
 double i8_ms = std::chrono::duration<double, std::milli>(end -
start).count() / 100;

 // Benchmark FP32
 start = std::chrono::high_resolution_clock::now();
 for (int t = 0; t < 100; t++) {
 matmul_f32(A_f32.data(), B_f32.data(), C_f32.data(), M, N, K);
 }
 end = std::chrono::high_resolution_clock::now();
 double f32_ms = std::chrono::duration<double, std::milli>(end -
start).count() / 100;

 double gflops_i8 = 2.0 * M * N * K / i8_ms / 1e6;
 double gflops_f32 = 2.0 * M * N * K / f32_ms / 1e6;

 std::cout << "INT8 matmul: " << i8_ms << " ms (" << gflops_i8 << " GOPS)\n";
 std::cout << "FP32 matmul: " << f32_ms << " ms (" << gflops_f32 << " GFLOPS)
\n";

 std::cout << "INT8 speedup: " << f32_ms / i8_ms << "x\n";
 std::cout << "Memory: INT8 = " << M*K + K*N << " bytes vs FP32 = "
 << (M*K + K*N) * 4 << " bytes (4x less)\n";
}

```

```
 return 0;
}
```

## 84 Vulkan Compute and Cross-Platform GPU

Vulkan is the only GPU compute API that runs on every major platform: NVIDIA, AMD, Intel, Apple (via MoltenVK), Android, and even the browser (via WebGPU). This file covers the Vulkan architecture, the compute pipeline, writing compute shaders in GLSL, the full C++ setup for a GPU compute program, shared memory and synchronisation, WebGPU for the browser, and practical ML inference examples

- CUDA dominates ML training on NVIDIA hardware. But not every deployment target has an NVIDIA GPU. A mobile app runs on Qualcomm Adreno or ARM Mali GPUs. A web application runs in the browser. A game engine needs to support AMD, Intel, and NVIDIA simultaneously. For all of these, **Vulkan** is the answer.
- Vulkan is verbose — a “hello world” compute program is ~300 lines of C++. But this verbosity is the price of **explicit control**: you manage every GPU resource (memory, pipelines, command buffers) yourself. This control enables maximum performance and portability, at the cost of development speed.

### 84.1 Vulkan Architecture Overview

---

- Vulkan is a low-level GPU API created by the Khronos Group (the same organisation behind OpenGL). Unlike CUDA (which hides GPU resource management), Vulkan requires you to explicitly manage:
  - **Instance and device**: create a Vulkan instance, enumerate available GPUs, and select one.
  - **Memory**: allocate GPU memory explicitly, specifying the memory type (device-local for speed, host-visible for CPU access).
  - **Buffers**: create buffer objects that reference the allocated memory.
  - **Descriptor sets**: bind buffers to shader inputs (like function arguments for the compute shader).
  - **Compute pipeline**: compile the shader and create a pipeline object.
  - **Command buffer**: record a sequence of GPU commands (bind pipeline, bind descriptors, dispatch compute).

- **Queue submission:** submit the command buffer to the GPU for execution.
- **Synchronisation:** fences and barriers to ensure correct ordering.
- This is radically different from CUDA's `cudaMalloc` + kernel launch model. In CUDA, the driver handles most of this behind the scenes. In Vulkan, you do it yourself.

### 84.1.1 Why So Verbose?

- Vulkan's explicitness exists for two reasons:
  1. **Driver simplicity:** OpenGL drivers were enormously complex (they had to guess what the application intended and optimise accordingly). Vulkan moves that responsibility to the application, making drivers thinner, more predictable, and easier to implement correctly across vendors.
  2. **Performance:** explicit control over memory layout, synchronisation, and command batching lets the application make optimal decisions. In CUDA, the driver might insert unnecessary synchronisation. In Vulkan, you synchronise only when you need to.

## 84.2 Compute Shaders in GLSL

- A **compute shader** is a program that runs on the GPU, similar to a CUDA kernel. It is written in **GLSL** (OpenGL Shading Language) and compiled to **SPIR-V** bytecode (a portable binary format).

### 84.2.1 Vector Addition

```
// add.comp - compile with: glslangValidator -V add.comp -o add.spv
#version 450

// Workgroup size: 256 invocations per workgroup (= threads per block in CUDA)
layout(local_size_x = 256) in;

// Buffer bindings (like kernel arguments)
layout(set = 0, binding = 0) buffer InputA { float a[]; };
layout(set = 0, binding = 1) buffer InputB { float b[]; };
layout(set = 0, binding = 2) buffer Output { float c[]; };

// Push constant: small uniform data (like a kernel parameter)
layout(push_constant) uniform PushConstants {
 uint n; // number of elements
```



```
};

void main() {
 uint idx = gl_GlobalInvocationID.x; // global thread index
 if (idx < n) {
 c[idx] = a[idx] + b[idx];
 }
}
```

· Mapping to CUDA concepts:

Vulkan	CUDA	Meaning
Workgroup	Block	Group of threads that can share memory
Invocation	Thread	Single execution unit
gl_GlobalInvocationID	blockIdx * blockDim + threadIdx	Global thread index
gl_LocalInvocationID	threadIdx	Thread index within workgroup
gl_WorkGroupID	blockIdx	Workgroup index
local_size_x	blockDim.x	Threads per workgroup
Storage buffer	Global memory	Read/write GPU memory
Shared memory (shared)	__shared__	Per-workgroup fast memory
Push constant	Kernel argument	Small uniform data

## 84.2.2 ReLU with Shared Memory

```
// relu_shared.comp
#version 450

layout(local_size_x = 256) in;

layout(set = 0, binding = 0) buffer Input { float input_data[]; };
layout(set = 0, binding = 1) buffer Output { float output_data[]; };

layout(push_constant) uniform PushConstants { uint n; };
```

```

// Shared memory (equivalent to CUDA __shared__)
shared float tile[256];

void main() {
 uint gid = gl_GlobalInvocationID.x;
 uint lid = gl_LocalInvocationID.x;

 // Load into shared memory
 if (gid < n) {
 tile[lid] = input_data[gid];
 }

 // Barrier: wait for all invocations in workgroup to finish loading
 barrier(); // equivalent to CUDA __syncthreads()

 // Compute ReLU
 if (gid < n) {
 output_data[gid] = max(tile[lid], 0.0);
 }
}

```

- For ReLU, shared memory is not strictly necessary (the operation is element-wise). But this demonstrates the pattern: load to shared memory → barrier → compute → store. For operations that need data from neighbouring threads (convolution, reduction, softmax), shared memory is essential.

### 84.2.3 Parallel Reduction (Sum)

```

// reduce_sum.comp
#version 450

layout(local_size_x = 256) in;

layout(set = 0, binding = 0) buffer Input { float input_data[]; };
layout(set = 0, binding = 1) buffer Output { float partial_sums[]; };

layout(push_constant) uniform PushConstants { uint n; };

shared float sdata[256];

void main() {
 uint gid = gl_GlobalInvocationID.x;
 uint lid = gl_LocalInvocationID.x;
 uint wgid = gl_WorkGroupID.x;

 // Load into shared memory

```

```

 sdata[lid] = (gid < n) ? input_data[gid] : 0.0;
 barrier();

 // Tree reduction within the workgroup
 for (uint stride = 128; stride > 0; stride >>= 1) {
 if (lid < stride) {
 sdata[lid] += sdata[lid + stride];
 }
 barrier();
 }

 // Thread 0 writes the workgroup's partial sum
 if (lid == 0) {
 partial_sums[wgid] = sdata[0];
 }
}

```

- This is the classic parallel reduction pattern (same as CUDA). Each workgroup produces one partial sum. A second dispatch reduces the partial sums to a final result. The tree reduction halves the active threads at each step:  $256 \rightarrow 128 \rightarrow 64 \rightarrow \dots \rightarrow 1$ .

## 84.2.4 Matrix Multiply with Tiling

```

// matmul_tiled.comp
#version 450

#define TILE_SIZE 16

layout(local_size_x = TILE_SIZE, local_size_y = TILE_SIZE) in;

layout(set = 0, binding = 0) buffer MatA { float A[]; };
layout(set = 0, binding = 1) buffer MatB { float B[]; };
layout(set = 0, binding = 2) buffer MatC { float C[]; };

layout(push_constant) uniform PushConstants {
 uint M, N, K;
};

shared float tileA[TILE_SIZE][TILE_SIZE];
shared float tileB[TILE_SIZE][TILE_SIZE];

void main() {
 uint row = gl_GlobalInvocationID.y;
 uint col = gl_GlobalInvocationID.x;
 uint lr = gl_LocalInvocationID.y;

```

```

uint lc = gl_LocalInvocationID.x;

float sum = 0.0;

for (uint t = 0; t < (K + TILE_SIZE - 1) / TILE_SIZE; t++) {
 // Load tile of A and B into shared memory
 uint aCol = t * TILE_SIZE + lc;
 uint bRow = t * TILE_SIZE + lr;

 tileA[lr][lc] = (row < M && aCol < K) ? A[row * K + aCol] : 0.0;
 tileB[lr][lc] = (bRow < K && col < N) ? B[bRow * N + col] : 0.0;

 barrier();

 // Compute partial dot product
 for (uint k = 0; k < TILE_SIZE; k++) {
 sum += tileA[lr][k] * tileB[k][lc];
 }

 barrier();
}

if (row < M && col < N) {
 C[row * N + col] = sum;
}
}

```

- This is the same tiling algorithm as the CUDA version (file 04), just in GLSL syntax. The concepts are identical: load tiles into shared memory, barrier, compute, barrier, repeat.

## 84.3 The C++ Vulkan Setup

- The compute shader is the easy part. The hard part is the C++ boilerplate that creates the Vulkan instance, allocates memory, binds buffers, and submits commands. Here is a condensed version of the full pipeline:

```

// vulkan_compute.cpp - a minimal but complete Vulkan compute example
// Compile: g++ -O3 -o vulkan_compute vulkan_compute.cpp -lvulkan
// Requires: Vulkan SDK installed, add.spv compiled from add.comp

#include <vulkan/vulkan.h>
#include <iostream>
#include <vector>
#include <fstream>

```

```

#include <cassert>

// Helper: read SPIR-V file
std::vector<uint32_t> readSPIRV(const std::string& filename) {
 std::ifstream file(filename, std::ios::ate | std::ios::binary);
 size_t fileSize = file.tellg();
 std::vector<uint32_t> buffer(fileSize / sizeof(uint32_t));
 file.seekg(0);
 file.read(reinterpret_cast<char*>(buffer.data()), fileSize);
 return buffer;
}

int main() {
 const uint32_t N = 1024;
 const size_t bufferSize = N * sizeof(float);

 // ===== 1. Create Vulkan Instance =====
 VkApplicationInfo appInfo{};
 appInfo.sType = VK_STRUCTURE_TYPE_APPLICATION_INFO;
 appInfo.apiVersion = VK_API_VERSION_1_2;

 VkInstanceCreateInfo instanceInfo{};
 instanceInfo.sType = VK_STRUCTURE_TYPE_INSTANCE_CREATE_INFO;
 instanceInfo.pApplicationInfo = &appInfo;

 VkInstance instance;
 vkCreateInstance(&instanceInfo, nullptr, &instance);

 // ===== 2. Select Physical Device (GPU) =====
 uint32_t deviceCount = 0;
 vkEnumeratePhysicalDevices(instance, &deviceCount, nullptr);
 std::vector<VkPhysicalDevice> devices(deviceCount);
 vkEnumeratePhysicalDevices(instance, &deviceCount, devices.data());
 VkPhysicalDevice physicalDevice = devices[0]; // use first GPU

 // Print GPU name
 VkPhysicalDeviceProperties props;
 vkGetPhysicalDeviceProperties(physicalDevice, &props);
 std::cout << "Using GPU: " << props.deviceName << "\n";

 // ===== 3. Find Compute Queue Family =====
 uint32_t queueFamilyCount = 0;
 vkGetPhysicalDeviceQueueFamilyProperties(physicalDevice, &queueFamilyCount,
 nullptr);

 std::vector<VkQueueFamilyProperties> queueFamilies(queueFamilyCount);
 vkGetPhysicalDeviceQueueFamilyProperties(physicalDevice, &queueFamilyCount,
 queueFamilies.data());

```

```

uint32_t computeFamily = 0;
for (uint32_t i = 0; i < queueFamilyCount; i++) {
 if (queueFamilies[i].queueFlags & VK_QUEUE_COMPUTE_BIT) {
 computeFamily = i;
 break;
 }
}

// ===== 4. Create Logical Device and Queue =====
float queuePriority = 1.0f;
VkDeviceQueueCreateInfo queueInfo{};
queueInfo.sType = VK_STRUCTURE_TYPE_DEVICE_QUEUE_CREATE_INFO;
queueInfo.queueFamilyIndex = computeFamily;
queueInfo.queueCount = 1;
queueInfo.pQueuePriorities = &queuePriority;

VkDeviceCreateInfo deviceInfo{};
deviceInfo.sType = VK_STRUCTURE_TYPE_DEVICE_CREATE_INFO;
deviceInfo.queueCreateInfoCount = 1;
deviceInfo.pQueueCreateInfos = &queueInfo;

VkDevice device;
vkCreateDevice(physicalDevice, &deviceInfo, nullptr, &device);

VkQueue computeQueue;
vkGetDeviceQueue(device, computeFamily, 0, &computeQueue);

// ===== 5. Allocate Buffers (A, B, C) =====
// For brevity, this uses host-visible memory (slower but simpler)
auto createBuffer = [&](VkBuffer& buffer, VkDeviceMemory& memory) {
 VkBufferCreateInfo bufInfo{};
 bufInfo.sType = VK_STRUCTURE_TYPE_BUFFER_CREATE_INFO;
 bufInfo.size = bufferSize;
 bufInfo.usage = VK_BUFFER_USAGE_STORAGE_BUFFER_BIT;
 vkCreateBuffer(device, &bufInfo, nullptr, &buffer);

 VkMemoryRequirements memReqs;
 vkGetBufferMemoryRequirements(device, buffer, &memReqs);

 // Find host-visible memory type
 VkPhysicalDeviceMemoryProperties memProps;
 vkGetPhysicalDeviceMemoryProperties(physicalDevice, &memProps);
 uint32_t memType = 0;
 for (uint32_t i = 0; i < memProps.memoryTypeCount; i++) {
 if ((memReqs.memoryTypeBits & (1 << i)) &&
 (memProps.memoryTypes[i].propertyFlags &

```

```

 (VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT |
VK_MEMORY_PROPERTY_HOST_COHERENT_BIT))) {
 memType = i;
 break;
 }
 }

 VkMemoryAllocateInfo allocInfo{};
 allocInfo.sType = VK_STRUCTURE_TYPE_MEMORY_ALLOCATE_INFO;
 allocInfo.allocationSize = memReqs.size;
 allocInfo.memoryTypeIndex = memType;
 vkAllocateMemory(device, &allocInfo, nullptr, &memory);
 vkBindBufferMemory(device, buffer, memory, 0);
};

VkBuffer bufA, bufB, bufC;
VkDeviceMemory memA, memB, memC;
createBuffer(bufA, memA);
createBuffer(bufB, memB);
createBuffer(bufC, memC);

// ===== 6. Fill Input Buffers =====
float* ptrA;
vkMapMemory(device, memA, 0, bufferSize, 0, (void**)&ptrA);
for (uint32_t i = 0; i < N; i++) ptrA[i] = 1.0f;
vkUnmapMemory(device, memA);

float* ptrB;
vkMapMemory(device, memB, 0, bufferSize, 0, (void**)&ptrB);
for (uint32_t i = 0; i < N; i++) ptrB[i] = 2.0f;
vkUnmapMemory(device, memB);

// ===== 7. Create Compute Pipeline =====
auto spirvCode = readSPIRV("add.spv");
VkShaderModuleCreateInfo shaderInfo{};
shaderInfo.sType = VK_STRUCTURE_TYPE_SHADER_MODULE_CREATE_INFO;
shaderInfo.codeSize = spirvCode.size() * sizeof(uint32_t);
shaderInfo.pCode = spirvCode.data();
VkShaderModule shaderModule;
vkCreateShaderModule(device, &shaderInfo, nullptr, &shaderModule);

// Descriptor set layout (tells Vulkan about the buffer bindings)
VkDescriptorSetLayoutBinding bindings[3] = {};
for (int i = 0; i < 3; i++) {
 bindings[i].binding = i;
 bindings[i].descriptorType = VK_DESCRIPTOR_TYPE_STORAGE_BUFFER;
 bindings[i].descriptorCount = 1;
}

```

```

 bindings[i].stageFlags = VK_SHADER_STAGE_COMPUTE_BIT;
 }

 VkDescriptorSetLayoutCreateInfo layoutInfo{};
 layoutInfo.sType = VK_STRUCTURE_TYPE_DESCRIPTOR_SET_LAYOUT_CREATE_INFO;
 layoutInfo.bindingCount = 3;
 layoutInfo.pBindings = bindings;
 VkDescriptorSetLayout descLayout;
 vkCreateDescriptorSetLayout(device, &layoutInfo, nullptr, &descLayout);

 // Push constant range
 VkPushConstantRange pushRange{};
 pushRange.stageFlags = VK_SHADER_STAGE_COMPUTE_BIT;
 pushRange.offset = 0;
 pushRange.size = sizeof(uint32_t);

 // Pipeline layout
 VkPipelineLayoutCreateInfo pipeLayoutInfo{};
 pipeLayoutInfo.sType = VK_STRUCTURE_TYPE_PIPELINE_LAYOUT_CREATE_INFO;
 pipeLayoutInfo.setLayoutCount = 1;
 pipeLayoutInfo.pSetLayouts = &descLayout;
 pipeLayoutInfo.pushConstantRangeCount = 1;
 pipeLayoutInfo.pPushConstantRanges = &pushRange;
 VkPipelineLayout pipelineLayout;
 vkCreatePipelineLayout(device, &pipeLayoutInfo, nullptr, &pipelineLayout);

 // Compute pipeline
 VkComputePipelineCreateInfo pipeInfo{};
 pipeInfo.sType = VK_STRUCTURE_TYPE_COMPUTE_PIPELINE_CREATE_INFO;
 pipeInfo.stage.sType = VK_STRUCTURE_TYPE_PIPELINE_SHADER_STAGE_CREATE_INFO;
 pipeInfo.stage.stage = VK_SHADER_STAGE_COMPUTE_BIT;
 pipeInfo.stage.module = shaderModule;
 pipeInfo.stage.pName = "main";
 pipeInfo.layout = pipelineLayout;
 VkPipeline pipeline;
 vkCreateComputePipelines(device, VK_NULL_HANDLE, 1, &pipeInfo, nullptr,
&pipeline);

 // ===== 8. Descriptor Set (bind buffers to shader) =====
 VkDescriptorPoolSize poolSize{};
 poolSize.type = VK_DESCRIPTOR_TYPE_STORAGE_BUFFER;
 poolSize.descriptorCount = 3;

 VkDescriptorPoolCreateInfo poolInfo{};
 poolInfo.sType = VK_STRUCTURE_TYPE_DESCRIPTOR_POOL_CREATE_INFO;
 poolInfo.maxSets = 1;
 poolInfo.poolSizeCount = 1;

```



```

poolInfo.pPoolSizes = &poolSize;
VkDescriptorPool descPool;
vkCreateDescriptorPool(device, &poolInfo, nullptr, &descPool);

VkDescriptorSetAllocateInfo descAllocInfo{};
descAllocInfo.sType = VK_STRUCTURE_TYPE_DESCRIPTOR_SET_ALLOCATE_INFO;
descAllocInfo.descriptorPool = descPool;
descAllocInfo.descriptorSetCount = 1;
descAllocInfo.pSetLayouts = &descLayout;
VkDescriptorSet descSet;
vkAllocateDescriptorSets(device, &descAllocInfo, &descSet);

// Write buffer references into the descriptor set
VkDescriptorBufferInfo bufInfos[3] = {
 {bufA, 0, bufferSize}, {bufB, 0, bufferSize}, {bufC, 0, bufferSize}
};
VkWriteDescriptorSet writes[3] = {};
for (int i = 0; i < 3; i++) {
 writes[i].sType = VK_STRUCTURE_TYPE_WRITE_DESCRIPTOR_SET;
 writes[i].dstSet = descSet;
 writes[i].dstBinding = i;
 writes[i].descriptorCount = 1;
 writes[i].descriptorType = VK_DESCRIPTOR_TYPE_STORAGE_BUFFER;
 writes[i].pBufferInfo = &bufInfos[i];
}
vkUpdateDescriptorSets(device, 3, writes, 0, nullptr);

// ===== 9. Record and Submit Command Buffer =====
VkCommandPoolCreateInfo cmdPoolInfo{};
cmdPoolInfo.sType = VK_STRUCTURE_TYPE_COMMAND_POOL_CREATE_INFO;
cmdPoolInfo.queueFamilyIndex = computeFamily;
VkCommandPool cmdPool;
vkCreateCommandPool(device, &cmdPoolInfo, nullptr, &cmdPool);

VkCommandBufferAllocateInfo cmdAllocInfo{};
cmdAllocInfo.sType = VK_STRUCTURE_TYPE_COMMAND_BUFFER_ALLOCATE_INFO;
cmdAllocInfo.commandPool = cmdPool;
cmdAllocInfo.level = VK_COMMAND_BUFFER_LEVEL_PRIMARY;
cmdAllocInfo.commandBufferCount = 1;
VkCommandBuffer cmdBuf;
vkAllocateCommandBuffers(device, &cmdAllocInfo, &cmdBuf);

VkCommandBufferBeginInfo beginInfo{};
beginInfo.sType = VK_STRUCTURE_TYPE_COMMAND_BUFFER_BEGIN_INFO;
vkBeginCommandBuffer(cmdBuf, &beginInfo);

vkCmdBindPipeline(cmdBuf, VK_PIPELINE_BIND_POINT_COMPUTE, pipeline);

```

```

vkCmdBindDescriptorSets(cmdBuf, VK_PIPELINE_BIND_POINT_COMPUTE,
 pipelineLayout, 0, 1, &descSet, 0, nullptr);
vkCmdPushConstants(cmdBuf, pipelineLayout, VK_SHADER_STAGE_COMPUTE_BIT,
 0, sizeof(uint32_t), &N);
vkCmdDispatch(cmdBuf, (N + 255) / 256, 1, 1); // launch workgroups

vkEndCommandBuffer(cmdBuf);

// Submit
VkFenceCreateInfo fenceInfo{};
fenceInfo.sType = VK_STRUCTURE_TYPE_FENCE_CREATE_INFO;
VkFence fence;
vkCreateFence(device, &fenceInfo, nullptr, &fence);

VkSubmitInfo submitInfo{};
submitInfo.sType = VK_STRUCTURE_TYPE_SUBMIT_INFO;
submitInfo.commandBufferCount = 1;
submitInfo.pCommandBuffers = &cmdBuf;
vkQueueSubmit(computeQueue, 1, &submitInfo, fence);
vkWaitForFences(device, 1, &fence, VK_TRUE, UINT64_MAX);

// ===== 10. Read Results =====
float* ptrC;
vkMapMemory(device, memC, 0, bufferSize, 0, (void**)&ptrC);
std::cout << "Results: c[0]=" << ptrC[0] << " c[1]=" << ptrC[1]
 << " (expected 3.0)\n";
bool correct = true;
for (uint32_t i = 0; i < N; i++) {
 if (ptrC[i] != 3.0f) { correct = false; break; }
}
std::cout << (correct ? "ALL CORRECT" : "ERRORS FOUND") << "\n";
vkUnmapMemory(device, memC);

// ===== Cleanup (abbreviated) =====
vkDestroyFence(device, fence, nullptr);
vkDestroyCommandPool(device, cmdPool, nullptr);
vkDestroyPipeline(device, pipeline, nullptr);
vkDestroyPipelineLayout(device, pipelineLayout, nullptr);
vkDestroyDescriptorPool(device, descPool, nullptr);
vkDestroyDescriptorSetLayout(device, descLayout, nullptr);
vkDestroyShaderModule(device, shaderModule, nullptr);
vkDestroyBuffer(device, bufA, nullptr); vkFreeMemory(device, memA, nullptr);
vkDestroyBuffer(device, bufB, nullptr); vkFreeMemory(device, memB, nullptr);
vkDestroyBuffer(device, bufC, nullptr); vkFreeMemory(device, memC, nullptr);
vkDestroyDevice(device, nullptr);
vkDestroyInstance(instance, nullptr);

```

```
 return 0;
}
```

- **Yes, this is ~200 lines for vector addition.** Compare to CUDA's ~30 lines. This is the cost of explicitness. But notice: every line has a purpose. There are no hidden driver decisions, no implicit synchronisation, no surprise allocations. You control everything.
- In practice, you would wrap this boilerplate in a helper library (or use an existing one like **vk-bootstrap**, **VMA** for memory allocation, or **kompute** for ML-focused Vulkan compute).

## 84.4 Kompute: Simplified Vulkan for ML

- **Kompute** is an open-source C++ library that wraps Vulkan's boilerplate for GPU compute. The same vector addition becomes:

```
#include <kompute/Kompute.hpp>

int main() {
 kp::Manager mgr;

 auto tensorA = mgr.tensor({1, 1, 1, 1, 1});
 auto tensorB = mgr.tensor({2, 2, 2, 2, 2});
 auto tensorC = mgr.tensor({0, 0, 0, 0, 0});

 std::string shader = R"(
 #version 450
 layout(local_size_x = 1) in;
 layout(set=0, binding=0) buffer A { float a[]; };
 layout(set=0, binding=1) buffer B { float b[]; };
 layout(set=0, binding=2) buffer C { float c[]; };
 void main() {
 uint i = gl_GlobalInvocationID.x;
 c[i] = a[i] + b[i];
 }
)";

 auto algorithm = mgr.algorithm({tensorA, tensorB, tensorC},
 kompute::Shader::compile_source(shader));

 mgr.sequence()
 ->record<kp::OpTensorSyncDevice>({tensorA, tensorB, tensorC})
 ->record<kp::OpAlgoDispatch>(algorithm)
```

```

->record<kp::OpTensorSyncLocal>({tensorC})
->eval();

 // tensorC now contains [3, 3, 3, 3, 3]
}

```

- Much more readable. Kompute handles instance creation, device selection, memory allocation, descriptor sets, and command buffer management. You focus on the shader and the data.

## 84.5 WebGPU: GPU Compute in the Browser

- **WebGPU** is the successor to WebGL, providing modern GPU access from JavaScript. It is built on Vulkan (Linux/Android), Metal (macOS/iOS), and DirectX 12 (Windows), abstracting the platform differences.
- WebGPU uses **WGSL** (WebGPU Shading Language) instead of GLSL:

```

// add.wgsl – WebGPU compute shader
@group(0) @binding(0) var<storage, read> a: array<f32>;
@group(0) @binding(1) var<storage, read> b: array<f32>;
@group(0) @binding(2) var<storage, read_write> c: array<f32>;

@compute @workgroup_size(256)
fn main(@builtin(global_invocation_id) id: vec3<u32>) {
 let i = id.x;
 c[i] = a[i] + b[i];
}

```

- **JavaScript setup** (condensed):

```

const adapter = await navigator.gpu.requestAdapter();
const device = await adapter.requestDevice();

// Create buffers
const bufferA = device.createBuffer({ size: N * 4, usage:
GPUBufferUsage.STORAGE, mappedAtCreation: true });
new Float32Array(bufferA.getMappedRange()).fill(1.0);
bufferA.unmap();

// ... (similar for B and C)

// Create pipeline from WGSL shader
const pipeline = device.createComputePipeline({

```

```

 layout: 'auto',
 compute: { module: device.createShaderModule({ code: wgsSource }),
entryPoint: 'main' }
 });

 // Dispatch
 const encoder = device.createCommandEncoder();
 const pass = encoder.beginComputePass();
 pass.setPipeline(pipeline);
 pass.setBindGroup(0, bindGroup);
 pass.dispatchWorkgroups(Math.ceil(N / 256));
 pass.end();
 device.queue.submit([encoder.finish()]);

```

- **Why WebGPU matters for ML:** running inference in the browser means no server cost, no latency, and user data never leaves the device. Libraries like **ONNX Runtime Web** and **Transformers.js** use WebGPU to run models (including small LLMs) entirely client-side.

## 84.6 When to Use Vulkan

Scenario	Use Vulkan?	Why / Alternative
ML training	No	CUDA/Triton is simpler and faster on NVIDIA
Inference on NVIDIA GPUs	No	TensorRT or CUDA is better
Inference on AMD/Intel GPUs	<b>Yes</b>	Only cross-vendor GPU compute option
Mobile inference (Android)	<b>Yes</b>	Vulkan is the standard GPU API on Android
Mobile inference (iOS)	No	Use Metal directly (MoltenVK adds overhead)
Browser inference	<b>WebGPU</b>	Built on Vulkan/Metal/DX12
Game engine + ML	<b>Yes</b>	Engine already uses Vulkan for rendering

Scenario	Use Vulkan?	Why / Alternative
Cross-platform library	Yes	One codebase for all GPU vendors
Learning GPU programming	Maybe	CUDA is easier to start with; Vulkan teaches more

## 84.7 Coding Tasks (compile with g++ -lvulkan, requires Vulkan SDK)

1. Compile and run the vector addition example above. Modify the shader to compute  $c[i] = a[i] * b[i] + a[i]$  (fused multiply-add) and verify the results.
2. Write a compute shader that applies softmax to a row of data using shared memory for the reduction steps (max and sum). Test with known values.

```
// softmax.comp - compile with: glslangValidator -V softmax.comp -o softmax.spv
#version 450

#define WG_SIZE 256

layout(local_size_x = WG_SIZE) in;

layout(set = 0, binding = 0) buffer Input { float input_data[]; };
layout(set = 0, binding = 1) buffer Output { float output_data[]; };

layout(push_constant) uniform PC { uint n; };

shared float sdata[WG_SIZE];

void main() {
 uint gid = gl_GlobalInvocationID.x;
 uint lid = gl_LocalInvocationID.x;

 // Step 1: find max (for numerical stability)
 sdata[lid] = (gid < n) ? input_data[gid] : -1e30;
 barrier();
 for (uint s = WG_SIZE / 2; s > 0; s >>= 1) {
 if (lid < s) sdata[lid] = max(sdata[lid], sdata[lid + s]);
 barrier();
 }
 float maxVal = sdata[0];
 barrier();
```

```

// Step 2: compute exp(x - max)
float expVal = (gid < n) ? exp(input_data[gid] - maxVal) : 0.0;
sdata[lid] = expVal;
barrier();

// Step 3: sum of exp values
for (uint s = WG_SIZE / 2; s > 0; s >= 1) {
 if (lid < s) sdata[lid] += sdata[lid + s];
 barrier();
}
float sumExp = sdata[0];

// Step 4: normalise
if (gid < n) {
 output_data[gid] = expVal / sumExp;
}
}

```

1. Modify the C++ host code to benchmark the compute shader: time the dispatch (excluding setup) using Vulkan timestamp queries or CPU-side fences, and compute the achieved bandwidth in GB/s.

## 85 Quantisation

Quantisation reduces the precision of model weights and activations, making models smaller, faster, and cheaper to run. This file covers number formats, post-training quantisation, quantisation-aware training, weight-only methods (GPTQ, AWQ), activation quantisation, mixed precision, and KV-cache quantisation

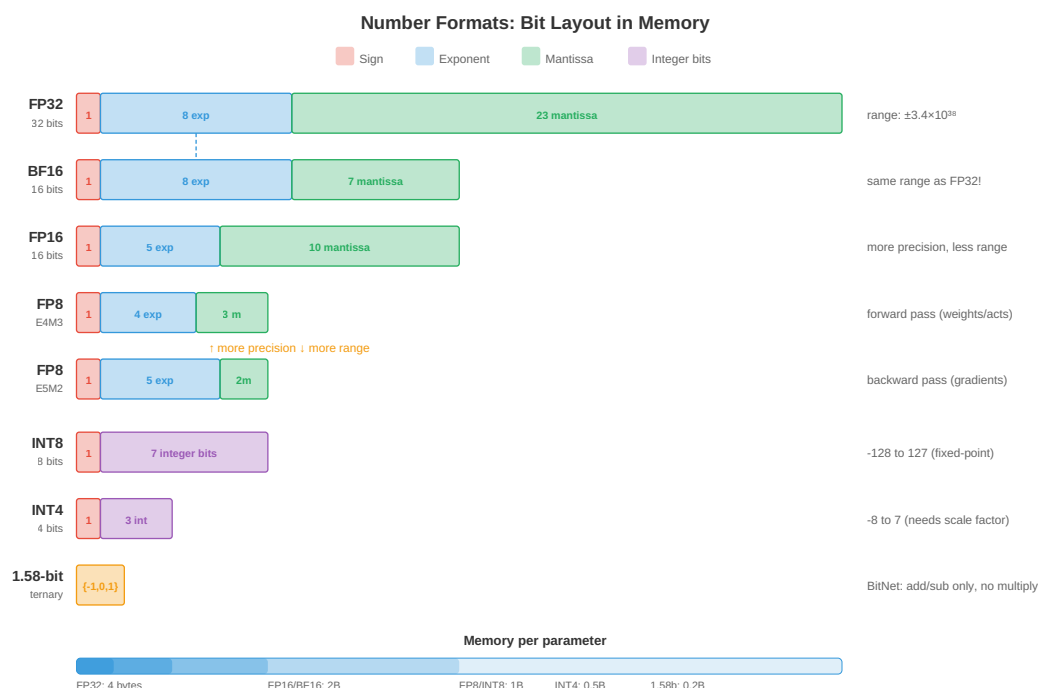
- A 70B parameter model in float16 requires 140 GB of memory, more than any single GPU. Quantise to INT4 and it fits in 35 GB (one A100) or even 20 GB (consumer RTX 4090 with offloading). Quantisation is not an optimisation nicety; it is what makes large model deployment economically viable.
- The fundamental tradeoff: lower precision means less memory, higher throughput, and lower power, but introduces **quantisation error** that can degrade model quality. The art of quantisation is minimising this degradation.

## 85.1 Why Quantise

- **Memory reduction:** INT8 is 2x smaller than FP16, INT4 is 4x smaller. For LLMs, model weights dominate memory. Halving precision halves the memory requirement.
- **Throughput gains:** lower precision means more operations per second. NVIDIA Tensor Cores (chapter 16) achieve 2x throughput for FP16 vs FP32, 2x again for INT8 vs FP16, and 2x again for INT4 vs INT8. An H100 does 989 TFLOPS in FP8 vs 67 TFLOPS in FP32 — a 15x difference.
- **Bandwidth savings:** LLM inference is usually **memory-bandwidth-bound** (chapter 16, roofline model). The bottleneck is loading weights from GPU memory, not computing with them. Smaller weights mean fewer bytes to transfer, directly increasing tokens per second. This is why quantisation often gives nearly linear speedups for LLM inference.
- **Energy savings:** lower precision uses less energy per operation. At data centre scale (thousands of GPUs), this translates to significant electricity cost reduction.

## 85.2 Number Formats

- We covered IEEE 754 floating-point in chapter 13 (computer architecture). Here is the full precision landscape for ML:





Precision formats bit layout: FP32 through ternary, showing how sign, exponent, and mantissa bits are arranged in memory, with memory-per-parameter comparison

Format	Bits	Exponent	Mantissa	Range	Use Case
FP32	32	8	23	$\pm 3.4 \times 10^{38}$	Training (gold standard)
TF32	19	8	10	$\pm 3.4 \times 10^{38}$	Tensor Core training (A100+)
FP16	16	5	10	$\pm 65504$	Mixed-precision training
BF16	16	8	7	$\pm 3.4 \times 10^{38}$	Training (same range as FP32)
FP8 E4M3	8	4	3	$\pm 448$	Forward pass (Hopper+)
FP8 E5M2	8	5	2	$\pm 57344$	Gradients (wider range)
INT8	8	—	—	-128 to 127	PTQ inference
INT4	4	—	—	-8 to 7	Weight-only quantisation
INT2/ Ternary	2	—	—	{-1, 0, 1}	Extreme compression

- **FP8** comes in two variants: **E4M3** (4-bit exponent, 3-bit mantissa, narrower range but more precision) for the forward pass, and **E5M2** (5-bit exponent, 2-bit mantissa, wider range but less precision) for gradients. The Transformer Engine (chapter 16) switches between them automatically per tensor.
- **BF16 vs FP16**: BF16 has the same exponent range as FP32 (no overflow risk) but less mantissa precision. FP16 has more precision but a narrow range (max 65504), requiring loss scaling during training. For inference, both work well; for training, BF16 is safer.
- **Integer formats** have no exponent — they represent fixed-point values. To convert between float and int, you need a **scale factor** and optionally a **zero point**:  $x_{\text{float}} = \text{scale} \times (x_{\text{int}} - \text{zero\_point})$ .

## 85.3 The Quantisation Equation

- All quantisation methods map floating-point values to integers and back:

$$x_q = \text{clamp}(\text{round}(\frac{x}{\text{scale}}) + \text{zero\_point}, q_{\min}, q_{\max})$$

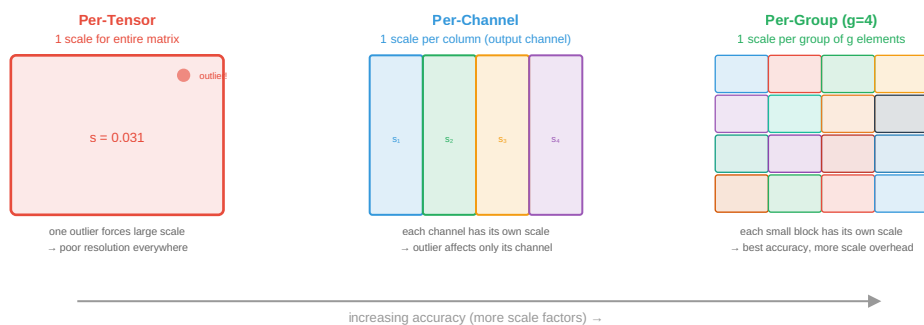
↺

x

$$= \text{scale} \times (x_q - \text{zero\_point})$$

- The **scale** determines the resolution:  $\text{scale} = \frac{x_{\max} - x_{\min}}{q_{\max} - q_{\min}}$ . For INT8:  $q_{\min} = -128$ ,  $q_{\max} = 127$ .
- **Symmetric quantisation** sets  $\text{zero\_point} = 0$ , so  $\text{scale} = \frac{\max(|x|)}{127}$ . Simpler and faster (no zero-point subtraction during inference).
- **Asymmetric quantisation** uses a non-zero  $\text{zero\_point}$  to handle asymmetric distributions (e.g., ReLU outputs are all non-negative). Maps  $[x_{\min}, x_{\max}]$  to  $[0, 255]$  for unsigned INT8.

Quantisation Granularity: Per-Tensor vs Per-Channel vs Per-Group



Quantisation granularity: per-tensor uses one scale for the entire matrix, per-channel one per column, per-group one per small block

- **Quantisation granularity:** how many values share the same scale factor:
  - **Per-tensor:** one scale for the entire tensor. Simplest but lowest accuracy (one outlier distorts the entire tensor's scale).
  - **Per-channel:** one scale per output channel (for convolutions) or per-row (for linear layers). Much better accuracy with minimal overhead.

- **Per-group:** one scale per group of  $g$  elements (e.g.,  $g = 128$ ). Best accuracy, used in modern weight-only quantisation (GPTQ, AWQ).
- **Per-token:** one scale per token for activations. Handles the fact that different tokens have very different activation magnitudes.

## 85.4 Post-Training Quantisation (PTQ)

---

- **PTQ** quantises a pre-trained model without any retraining. You pass a **calibration set** (a small representative dataset, typically 128-512 samples) through the model to collect activation statistics, then compute optimal scale factors.

### 85.4.1 Calibration Methods

- **Min-max:** set scale based on the observed minimum and maximum values. Simple but sensitive to outliers (one extreme value wastes most of the quantisation range on rarely-used values).
- **Percentile:** use the 99.99th percentile instead of the absolute max. Clips extreme outliers, giving better resolution for the majority of values. The clipped values saturate to  $q_{\min}$  or  $q_{\max}$ .
- **MSE-optimal:** find the scale that minimises the mean squared error between the original and quantised tensors. This is a 1D optimisation (search over possible clip values) and usually gives the best PTQ accuracy.
- **Entropy-based** (KL divergence): find the scale that minimises the KL divergence between the original and quantised value distributions. Used in TensorRT's INT8 calibration.

### 85.4.2 PTQ in Practice

```
Simplified PTQ with PyTorch (conceptual)
import torch

def quantise_tensor_symmetric(tensor, bits=8):
 qmax = 2 ** (bits - 1) - 1 # 127 for INT8
 scale = tensor.abs().max() / qmax
 quantised = torch.clamp(torch.round(tensor / scale), -qmax,
 qmax).to(torch.int8)
 return quantised, scale
```

```

def dequantise(quantised, scale):
 return quantised.float() * scale

Quantise a weight matrix
weight = torch.randn(512, 512) # pretrained weight
weight_q, scale = quantise_tensor_symmetric(weight, bits=8)
weight_reconstructed = dequantise(weight_q, scale)

Quantisation error
error = (weight - weight_reconstructed).abs().mean()
print(f"Mean absolute error: {error:.6f}")
print(f"Compression: {weight.numel() * 4 / (weight_q.numel() * 1 + 4):.1f}x") #
+4 bytes for scale

```

- PTQ works well for INT8 on most models with <1% accuracy degradation. For INT4, PTQ quality drops significantly — weight-only methods (below) handle INT4 much better.

## 85.5 Quantisation-Aware Training (QAT)

- QAT inserts fake quantisation operations into the training graph: weights and activations are quantised and dequantised during the forward pass, but gradients flow through as if no quantisation happened (the **straight-through estimator**).

Forward:  W

$$\begin{aligned}
 &= \text{dequant}(\text{quant}(W)) \\
 \text{Backward: } \frac{\partial L}{\partial W} &\approx \frac{\partial L}{\partial} \\
 &\quad \text{W}
 \end{aligned}$$

- The model learns to be robust to quantisation noise during training. QAT typically recovers most or all of the accuracy lost by PTQ, especially at low bit-widths (INT4, INT2).
- **Cost:** QAT requires retraining (or fine-tuning) the model, which is expensive for large models. For a 70B parameter model, QAT might cost \$10,000-\$100,000 in compute. PTQ costs essentially nothing (just calibration).

- **When to use QAT:** when PTQ quality is unacceptable (usually INT4 or lower), when you are deploying to edge devices with strict latency budgets, or when the model will be quantised millions of times (the one-time QAT cost is amortised).

## 85.6 Weight-Only Quantisation

- For LLM inference, the bottleneck is loading weights from memory, not computing with them (memory-bandwidth-bound regime). **Weight-only quantisation** quantises weights to INT4 or INT3 while keeping activations in FP16. The compute happens in FP16 (after dequantising the weights on the fly), but memory consumption and bandwidth are reduced by 4-8x.

### 85.6.1 GPTQ

- **GPTQ** (Frantar et al., 2022) quantises weights one column at a time, compensating for the error of each column by adjusting subsequent columns. It uses the **Hessian** (second-order information from a calibration set) to determine the optimal quantisation order and error compensation:

↺  
W

$$W_{:,j} = \text{quant}(W_{:,j}) - \frac{(W_{:,j} - \text{quant}(W_{:,j})) \cdot H_{j,j+1:}}{H_{j,j}}$$

- The key insight: quantising column  $j$  introduces an error. GPTQ immediately compensates by adjusting all remaining columns so that the overall output of the layer ( $XW$ ) changes as little as possible. This is **optimal brain quantisation** (OBQ) applied to transformers.
- GPTQ with 4-bit group quantisation (group size 128) achieves <1% perplexity degradation on most LLMs. The quantisation takes ~1 hour per 70B model on a single GPU.

### 85.6.2 AWQ

- **AWQ** (Activation-Aware Weight Quantisation, Lin et al., 2023) observes that a small fraction of weight channels (1-3%) are far more important than others — they correspond to activation channels with large magnitudes. Protecting these salient channels dramatically reduces quantisation error.

- AWQ scales these important channels by a factor  $s$  before quantisation (making them larger and thus less affected by rounding) and scales the corresponding activations by  $1/s$  (to preserve the output). The scale  $s$  is optimised per-group to minimise the overall quantisation error.
- AWQ is simpler than GPTQ (no Hessian computation), faster to run, and achieves comparable quality. It has become the default for many open-source LLM quantisation pipelines.

### 85.6.3 GGUF / llama.cpp Quantisation

- **GGUF** (GGML Universal Format) is the format used by llama.cpp for CPU inference. It supports many quantisation schemes:
  - **Q4\_0**: 4-bit, 32-element blocks, symmetric.
  - **Q4\_K\_M**: 4-bit with mixed-precision important channels (k-quants).
  - **Q5\_K\_M**: 5-bit with k-quants (higher quality).
  - **Q8\_0**: 8-bit, simple and fast.
- The “K” variants (k-quants) allocate more bits to important weight blocks, similar to AWQ’s insight but implemented at the format level. Q4\_K\_M is the sweet spot for most models: 4-bit average with minimal quality loss.

### 85.6.4 QuIP and QuIP

- **QuIP** (Chee et al., 2023) introduces **incoherence processing**: rotate the weight matrix using a random orthogonal transformation before quantisation. This spreads the information across all weights, preventing a few outlier weights from dominating the quantisation error.
- The intuition: if one weight is 100 and the rest are  $\sim 1$ , quantising all with the same scale wastes most of the INT4 range on the outlier. After an orthogonal rotation (which preserves the matrix’s mathematical properties), all weights have similar magnitude, and uniform quantisation works much better.
- **QuIP#** extends this with **lattice codebooks**: instead of mapping to a uniform integer grid, map to points in an optimal lattice (the E8 lattice in 8D). Lattice codes pack more quantisation points into the same number of bits, achieving better rate-distortion than uniform quantisation. QuIP# achieves usable quality at **2-bit** precision — half the bits of typical INT4 methods.

### 85.6.5 SpQR

- **SpQR** (Dettmers et al., 2023) observes that a tiny fraction of weights (0.1-1%) are **outliers** that contribute disproportionately to output quality. Instead of quantising everything to the same precision, SpQR:
  1. Identifies outlier weights using sensitivity analysis (how much does quantising this weight change the layer output?).
  2. Stores outliers at **full precision** (FP16) in a sparse format.
  3. Quantises all remaining weights to INT3 or INT4.
- The result: ~99% of weights are aggressively quantised (small), while the critical 1% retain full precision (accurate). The sparse outlier storage adds minimal overhead (<5% of total size).

### 85.6.6 HQQ

- **HQQ** (Half-Quadratic Quantisation, Badri & Shaji, 2023) is a **zero-shot** weight quantisation method that requires no calibration data at all. It formulates quantisation as a half-quadratic optimisation problem, solving for optimal quantised weights and scale factors iteratively.
- The advantage: no calibration set means no data dependency, instant quantisation, and no risk of calibration data mismatch. HQQ is particularly useful for models where representative calibration data is unavailable or sensitive.

### 85.6.7 AQLM

- **AQLM** (Egiazarian et al., 2024) applies **additive quantisation** (multi-codebook vector quantisation) to LLMs. Instead of quantising each weight independently, AQLM groups weights into vectors and represents each vector as the sum of entries from multiple learned codebooks:

$$\mathbf{w} \approx \mathbf{c}_1^{(1)} + \mathbf{c}_2^{(2)} + \dots + \mathbf{c}_M^{(M)}$$

- where  $\mathbf{c}_i^{(m)}$  is an entry from codebook  $m$ . With  $M = 2$  codebooks of 256 entries each, a 8-element vector is encoded as two 8-bit indices = 2 bytes for 8 weights = **2 bits per weight** effective. AQLM achieves state-of-the-art quality at 2-bit precision, outperforming GPTQ and AWQ at this extreme compression level.

85.6.8 BitNet and 1-Bit LLMs

- **BitNet** (Wang et al., 2023) takes quantisation to the extreme: weights are ternary ( $\{-1, 0, +1\}$ ), requiring only  $\sim 1.58$  bits per weight. Matrix multiplication becomes **addition and subtraction only** — no floating-point multiplies needed.
- **BitNet b1.58** (Ma et al., 2024) constrains every weight to  $\{-1, 0, +1\}$ . The “1.58 bits” comes from  $\log_2(3) \approx 1.58$ . At this precision, a 70B model fits in  $\sim 15$  GB and inference requires no multiply operations — just adds, subtracts, and sign flips.
- The matmul becomes:

$$y_j = \sum_i W_{ij} \cdot x_i$$
$$= \sum_{i: W_{ij} = +1} x_i - \sum_{i: W_{ij} = -1} x_i$$

- This is dramatically cheaper than FP16 matmul on any hardware, and could enable LLM inference on devices without floating-point units. The quality tradeoff is significant for current models but improves with scale and training-time quantisation-awareness.

85.6.9 Microscaling (MX) Formats

- **Microscaling** (MX) formats are a new industry standard (supported by AMD, Arm, Intel, Meta, Microsoft, NVIDIA, Qualcomm) that use **block floating point**: a group of elements shares a single exponent, and each element has its own mantissa.

Format	Shared Exponent	Element Bits	Total (per element)	Equivalent
MXFP8	8-bit per block	8 (E4M3/ E5M2)	$\sim 8$	



Format	Shared Exponent	Element Bits	Total (per element)	Equivalent
				Like FP8 with better range
MXFP6	8-bit per block	6	~6.5	Between FP8 and INT4
MXFP4	8-bit per block	4	~4.5	Like INT4 with float-like behaviour
MXINT8	8-bit per block	8 (integer)	~8.5	INT8 with shared scaling

- The shared exponent amortises the exponent cost across a block (typically 16-32 elements). Each element retains more mantissa bits than it would with an individual exponent, giving better precision per bit. MX formats are expected to replace individual FP8 and INT8 formats in future hardware.

### 85.6.10 FP8 Training

- Training in FP8 (not just inference) is now practical on NVIDIA Hopper and Blackwell GPUs. The recipe:
  - **Forward pass:** weights and activations in E4M3 (higher precision, narrower range). The Transformer Engine dynamically computes per-tensor scale factors using delayed scaling (track statistics from the previous iteration, apply them to the current one).
  - **Backward pass:** gradients in E5M2 (wider range, lower precision). Gradients have a broader value range than weights/activations, so the extra exponent bit prevents overflow.
  - **Master weights:** maintained in FP32 for the optimiser state (like standard mixed-precision training with FP16, chapter 6). The FP8 computation is only for the matmuls, not for the weight updates.

- **Loss scaling:** still needed for FP8, just as for FP16. The dynamic loss scaler adjusts the scale factor to keep gradient values within FP8's representable range.
- FP8 training achieves quality comparable to BF16 training for most model sizes, with ~2x throughput improvement. It is the default for new large-scale training runs on H100 clusters.

## 85.7 Activation Quantisation

---

- Activations (the intermediate tensors flowing between layers) can also be quantised, enabling fully INT8 computation (both weights and activations in INT8, with INT32 accumulation).
- **Dynamic quantisation:** compute the scale factor at runtime from the actual activation values. More accurate (adapts to each input) but adds overhead (computing min/max or percentile at each layer).
- **Static quantisation:** compute scale factors once during calibration and fix them. Faster at inference (no runtime statistics) but less accurate if the calibration data is not representative.
- **Per-token quantisation:** compute a separate scale for each token in a sequence. Critical for LLMs because different tokens can have very different activation magnitudes (some tokens produce activations 100x larger than others).
- Activation quantisation is harder than weight quantisation because activations are data-dependent (they change with every input), while weights are fixed. The “outlier” problem is especially severe: a few activation channels have extreme values (100x the mean), and quantising them with the same scale as normal channels wastes precision.
- **SmoothQuant** (Xiao et al., 2022) addresses outliers by mathematically migrating the quantisation difficulty from activations (hard to quantise due to outliers) to weights (easy to quantise): multiply activations by  $1/s$  and weights by  $s$ , where  $s$  balances the difficulty. The output  $XW = (X \cdot \text{diag}(s^{-1})) \cdot (\text{diag}(s) \cdot W)$  is unchanged.

## 85.8 Mixed-Precision Quantisation

---

- Not all layers are equally sensitive to quantisation. Attention layers often tolerate INT4, while embedding layers and the final classifier need higher precision.
- **Sensitivity analysis:** quantise each layer individually and measure the accuracy impact. Layers with high sensitivity get more bits; insensitive layers get fewer bits.
- The Transformer Engine (chapter 16, NVIDIA Hopper) implements dynamic mixed precision at the operation level: each matmul chooses between FP8 and FP16 based on the tensor statistics, maximising throughput while maintaining quality.

## 85.9 KV-Cache Quantisation

---

- During LLM generation, the **KV-cache** stores the key and value tensors for all previous tokens. For long sequences, this dominates memory:

$$\text{KV-cache size} = 2 \times n_{\text{layers}} \times n_{\text{heads}} \times d_{\text{head}} \times \text{seq\_len} \times \text{bytes\_per\_element}$$

- A 70B model with 80 layers, 64 heads, 128-dim heads, at sequence length 128K in FP16:  $2 \times 80 \times 64 \times 128 \times 131072 \times 2 = 330 \text{ GB}$ . This exceeds the GPU memory.
- **KV-cache quantisation** reduces this by storing cached keys and values in INT8 or INT4 instead of FP16. The quantisation error accumulates over the sequence (each new token attends to all cached K/V), but with per-channel or per-head quantisation, the degradation is acceptable.
- **KV-cache quantisation is multiplicatively beneficial:** it enables longer sequences (more context), larger batch sizes (more concurrent users), and faster inference (less memory bandwidth to load the cache). This is one of the highest-impact optimisations for LLM serving.

## 85.10 Coding Tasks (use CoLab or notebook)

1. Implement symmetric INT8 quantisation from scratch. Quantise a weight matrix, dequantise it, and measure the reconstruction error as a function of the value distribution.

```
import jax.numpy as jnp
import jax

def quantise_int8(tensor):
 scale = jnp.max(jnp.abs(tensor)) / 127.0
 quantised = jnp.clip(jnp.round(tensor / scale), -127, 127).astype(jnp.int8)
 return quantised, scale

def dequantise(quantised, scale):
 return quantised.astype(jnp.float32) * scale

Normal weights (typical for trained models)
key = jax.random.PRNGKey(0)
weights = jax.random.normal(key, (1024, 1024)) * 0.02

q, s = quantise_int8(weights)
recon = dequantise(q, s)

print(f"Original: {weights.nbytes / 1024:.0f} KB")
print(f"Quantised: {q.nbytes / 1024:.0f} KB ({weights.nbytes / q.nbytes:.0f}x
smaller)")
print(f"Mean abs err: {jnp.abs(weights - recon).mean():.6f}")
print(f"Max abs err: {jnp.abs(weights - recon).max():.6f}")
print(f"Relative err: {jnp.abs(weights - recon).mean() /
jnp.abs(weights).mean():.4%}")
```

1. Demonstrate the outlier problem. Create activations with a few extreme channels and show how per-tensor quantisation fails while per-channel succeeds.

```
import jax.numpy as jnp
import jax

key = jax.random.PRNGKey(42)

Activations: most channels are normal, 2 channels have 100x outliers
activations = jax.random.normal(key, (32, 512)) * 0.1
activations = activations.at[:, 0].set(activations[:, 0] * 100) # outlier
```

```

channel
 activations = activations.at[:, 1].set(activations[:, 1] * 50) # outlier
channel

Per-tensor quantisation (one scale for entire tensor)
scale_tensor = jnp.max(jnp.abs(activations)) / 127.0
q_tensor = jnp.clip(jnp.round(activations / scale_tensor), -127, 127)
recon_tensor = q_tensor * scale_tensor

Per-channel quantisation (one scale per channel)
scales_channel = jnp.max(jnp.abs(activations), axis=0) / 127.0
q_channel = jnp.clip(jnp.round(activations / scales_channel), -127, 127)
recon_channel = q_channel * scales_channel

err_tensor = jnp.abs(activations - recon_tensor).mean()
err_channel = jnp.abs(activations - recon_channel).mean()

print(f"Per-tensor error: {err_tensor:.6f}")
print(f"Per-channel error: {err_channel:.6f}")
print(f"Per-channel is {err_tensor / err_channel:.1f}x better")
print(f"\nOutlier channels waste {(activations.shape[1] - 2) /
activations.shape[1]:.0%} "
 f"of the quantisation range for {2 / activations.shape[1]:.1%} of
channels")

```

1. Compute the KV-cache memory for different model sizes and sequence lengths. Show why KV-cache quantisation is essential for long-context models.

```

def kv_cache_gb(n_layers, n_heads, d_head, seq_len, bytes_per_elem):
 return 2 * n_layers * n_heads * d_head * seq_len * bytes_per_elem / 1e9

models = [
 ("Llama-7B", 32, 32, 128),
 ("Llama-70B", 80, 64, 128),
 ("GPT-4 (est)", 120, 96, 128),
]

print(f"{'Model':<15} {'SeqLen':>8} {'FP16 (GB)':>10} {'INT8 (GB)':>10} {'INT4 (GB)':>10}")
print("-" * 60)

for name, layers, heads, d_head in models:
 for seq_len in [4096, 32768, 131072]:
 fp16 = kv_cache_gb(layers, heads, d_head, seq_len, 2)
 int8 = kv_cache_gb(layers, heads, d_head, seq_len, 1)
 int4 = kv_cache_gb(layers, heads, d_head, seq_len, 0.5)

```

```
print(f"{name:<15} {seq_len:>8} {fp16:>9.1f} {int8:>9.1f}
{int4:>9.1f}")
print()
```

## 86 Efficient Architectures

Making models faster is not just about lower precision, it is also about smarter architectures that do less work per token. This file covers StreamingLLM, sparse and linear attention, multi-query and grouped-query attention, Mixture of Experts at inference, knowledge distillation, pruning, and neural architecture search

- Quantisation (file 1) makes each operation cheaper. This file makes fewer operations happen in the first place. The two are complementary: a model that is both architecturally efficient and quantised can be 10-100x faster than the original.

### 86.1 StreamingLLM: Infinite-Length Generation

---

- Standard transformers store all previous tokens in the KV-cache, which grows linearly with sequence length. At some point, the cache exceeds GPU memory and generation fails. **StreamingLLM** (Xiao et al., 2023) solves this with a fixed-size **rolling KV-cache**.
- The key observation: the first few tokens in a sequence receive disproportionately high attention scores regardless of their content. These are called **attention sinks**. If you evict them from the cache, the attention distribution collapses and generation quality degrades catastrophically.
- StreamingLLM's solution: keep a small number of **sink tokens** (the first 1-4 tokens) permanently in the cache, plus a **rolling window** of the most recent  $w$  tokens. The total cache size is  $\text{sink} + w$ , which is fixed regardless of how many tokens have been generated.

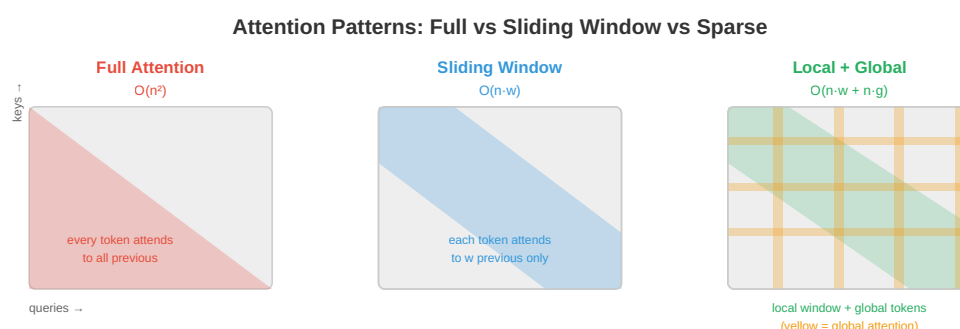
$$\text{Cache} = [\text{token}_0, \text{token}_1, \text{token}_{t-w+1}, \dots, \text{token}_t]$$

- The attention sinks anchor the softmax distribution, and the rolling window provides recent context. This enables **infinite-length generation** with constant memory, at the cost of losing access to middle-of-sequence context.

- StreamingLLM works without any retraining for models that naturally develop attention sinks (most pre-trained LLMs do). For models that do not, adding a single learnable sink token during training fixes it.

## 86.2 Sparse Attention

- Full self-attention is  $O(n^2)$  in sequence length  $n$  because every token attends to every other token. For  $n = 128K$ , the attention matrix has  $128K^2 = 16$  billion entries. **Sparse attention** patterns reduce this by restricting which tokens attend to which.



Attention sparsity patterns: full attention is  $O(n^2)$ , sliding window is  $O(n \cdot w)$ , local+global adds long-range tokens

- **Sliding window attention** (Mistral, Gemma): each token attends only to the previous  $w$  tokens (e.g.,  $w = 4096$ ). Attention is  $O(n \cdot w)$  instead of  $O(n^2)$ . Information propagates beyond the window through multiple layers: after  $L$  layers, the effective context is  $L \times w$ .
- **Local + global attention** (Longformer, BigBird): most tokens use sliding window attention (local), but a few designated tokens (e.g., [CLS], every 512th token) attend to all tokens (global). This captures both local patterns and long-range dependencies.
- **Dilated attention:** attend to every  $k$ -th token within a window, creating a sparse pattern that covers a larger range with the same number of attention scores. Increasing dilation across layers creates a hierarchical pattern similar to dilated convolutions (chapter 8).
- The practical winner for modern LLMs is **sliding window + full attention interleaved**: some layers use sliding window (cheap, handles local context), some layers use full attention (expensive, captures long-range). Mistral/Mixtral use this pattern.

## 86.3 Linear Attention and State-Space Models

- Can we replace the  $O(n^2)$  attention entirely? **Linear attention** and **state-space models (SSMs)** process sequences in  $O(n)$  time by avoiding the explicit attention matrix.
- **Linear attention** replaces the softmax attention with a kernel approximation:

$$\begin{array}{l} \text{Standard: } O = \text{softmax}(QK^T / \sqrt{d})V \\ \text{Linear: } O = \phi(Q)(\phi(K)^TV) \end{array}$$

- By associating the  $K^TV$  product first (which is  $d \times d$ , independent of sequence length), the computation becomes  $O(n \cdot d^2)$  instead of  $O(n^2 \cdot d)$ . For long sequences where  $n \gg d$ , this is a massive saving.
- **RWKV** combines ideas from RNNs and transformers. It uses a recurrent formulation that processes tokens sequentially (like an RNN) but can be parallelised during training (like a transformer). Inference is  $O(1)$  per token (constant memory, no KV-cache growth).
- **Mamba** (Gu & Dao, 2023) is a selective state-space model. It processes sequences through learned state transitions:

$$\begin{aligned} h_t &= A h_{t-1} + B x_t \\ x_t, y_t &= C h_t \end{aligned}$$

- where

$A$

and

$B$

are input-dependent (selective), allowing Mamba to dynamically focus on or ignore parts of the input. Unlike fixed SSMs, the selectivity makes Mamba



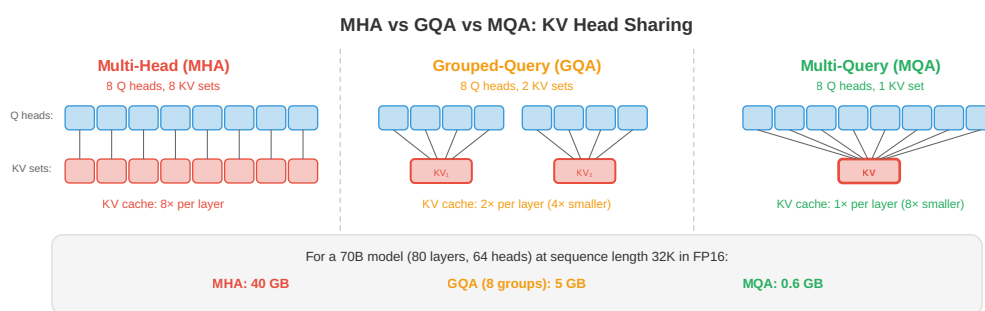
competitive with transformers on language tasks while maintaining  $O(n)$  scaling.

- **The tradeoff:** linear attention and SSMs are faster for long sequences but generally less capable than full attention for tasks requiring precise long-range retrieval. Hybrid architectures (some transformer layers + some Mamba layers) often give the best of both worlds.

## 86.4 Multi-Query and Grouped-Query Attention

- Standard multi-head attention (MHA, chapter 7) uses separate K, V projections for each head. For  $h$  heads, this means  $h$  separate key and value tensors in the KV-cache. **Multi-Query Attention (MQA)** and **Grouped-Query Attention (GQA)** reduce this.
- **MQA** (Shazeer, 2019): all heads share a single set of K, V projections. Each head still has its own Q projection. The KV-cache shrinks by a factor of  $h$  (e.g., 32x for 32 heads).
- **GQA** (Ainslie et al., 2023): a middle ground. Heads are grouped, and each group shares one set of K, V projections. With  $h = 32$  heads and  $g = 8$  groups, each group of 4 heads shares K/V. The KV-cache shrinks by  $h/g = 4x$ .

MHA:  $h$  heads,  $h$  K/V sets  $\rightarrow$  GQA:  $h$  heads,  $g$  K/V sets  $\rightarrow$  MQA:  $h$  heads, 1 K/V set



MHA vs GQA vs MQA: MHA gives each head its own KV, GQA shares KV across groups, MQA uses a single KV for all heads — dramatically reducing KV-cache size

- Most modern LLMs use GQA (Llama 2/3, Gemma, Mistral). It reduces KV-cache memory and inference latency with negligible quality loss compared to MHA.

### 86.4.1 Multi-head Latent Attention (MLA)

- **MLA** (DeepSeek-V2, 2024) goes further than GQA by compressing the KV-cache into a **low-rank latent space**. Instead of caching full key and value vectors, MLA caches a compressed latent vector  $\mathbf{c}_t$  per token and reconstructs K/V on the fly during attention:

$$\mathbf{c}_t = W_{\text{compress}} \cdot [\mathbf{k}_t; \mathbf{v}_t], \quad \mathbf{k}_t = W_K^{\text{up}} \cdot \mathbf{c}_t, \quad \mathbf{v}_t = W_V^{\text{up}} \cdot \mathbf{c}_t$$

- The compressed vector  $\mathbf{c}_t$  is much smaller than the original K and V combined. DeepSeek-V2 achieves a **93.3% reduction** in KV-cache size compared to MHA, outperforming even MQA, while maintaining MHA-level quality.
- The tradeoff: reconstructing K/V from the latent adds a small compute cost per attention operation. But since LLM decode is memory-bandwidth-bound (not compute-bound), this is a net win: less memory to load > slightly more compute per token.

### 86.4.2 Flash Attention

- **Flash Attention** (Dao et al., 2022, covered in detail in chapter 16 file 05) is not an architectural change but an implementation optimisation that belongs in any discussion of efficient attention. It computes exact standard attention with:
  - **O(n) memory** instead of  $O(n^2)$  (the attention matrix is never materialised in HBM).
  - **2-4x faster** than standard attention (by keeping data in SRAM via tiling and online softmax).
  - **No quality loss** — the output is mathematically identical to standard attention.
- Flash Attention is now the default attention implementation in PyTorch (`torch.nn.functional.scaled_dot_product_attention`), JAX, and all major inference frameworks. If you are running attention in 2024+, you are almost certainly using Flash Attention.

### 86.4.3 Ring Attention

- **Ring Attention** (Liu et al., 2023) distributes attention computation across multiple devices for sequences that are too long to fit on a single GPU's memory, even with Flash Attention.

- The idea: partition the sequence across  $N$  devices. Each device holds  $n/N$  tokens'  $Q, K, V$ . The devices are arranged in a ring. In each step:
  1. Each device computes local attention (its  $Q$  against its local  $K/V$ ).
  2. Each device sends its  $K/V$  block to the next device in the ring.
  3. Each device receives  $K/V$  from the previous device and computes attention against those.
  4. After  $N$  steps, every device has attended to every  $K/V$  block.
- Communication is **overlapped** with computation: while computing attention on the current  $K/V$  block, the next block is being transferred. This hides the communication latency almost entirely.
- Ring Attention enables **million-token context windows** by distributing the  $KV$ -cache across a ring of GPUs. The memory per device is  $O(n/N)$ , making arbitrarily long sequences feasible (limited only by the number of devices).

## 86.5 Mixture of Experts at Inference

---

- MoE models (chapter 7) activate only a fraction of their parameters per token (typically 2 out of 8 experts). At inference, the unique challenge is **expert caching**: all experts must be in memory (because any token might route to any expert), but only 2 are active per token.
- For a Mixtral 8x7B model: total parameters = 47B ( $8 \times 7B$  experts, but with shared components). Active parameters per token  $\approx 13B$  (2 experts + shared layers). The model has LLM-70B-class quality with LLM-13B-class inference cost, but requires 47B parameters in memory.
- **Expert offloading**: for GPU-memory-constrained deployment, keep inactive experts on CPU or SSD and load them on demand. This works because token routing is predictable enough to prefetch the likely experts.
- **Expert caching**: maintain an LRU cache of recently used experts in GPU memory. If the same experts are repeatedly activated (common for in-domain data), the cache hit rate is high.

## 86.6 Knowledge Distillation

---

- **Distillation** (chapter 6) trains a small “student” model to mimic a large “teacher.” The student learns from the teacher’s soft predictions

(probability distributions over classes), which contain more information than hard labels alone.

$$\mathcal{L} = \alpha \cdot \text{KL}(p_{\text{teacher}}^T // p_{\text{student}}^T) + (1 - \alpha) \cdot \mathcal{L}_{\text{CE}}(y, p_{\text{student}})$$

- where  $T$  is the temperature (higher  $T$  softens the distributions, revealing the teacher's uncertainty) and  $\alpha$  balances the distillation loss with the standard cross-entropy loss.
- **For LLMs:** distillation is used to create small, fast models from large, capable ones. GPT-4  $\rightarrow$  a 7B student that captures most of GPT-4's behaviour for a specific task. The student can be 10-100x cheaper to serve.
- **Task-specific distillation:** distil only on data relevant to your deployment task. A 7B model distilled on medical Q&A from a 70B teacher can outperform the 70B model on that specific task (because the student's limited capacity is entirely focused on the target domain).

## 86.7 Pruning

---

- **Pruning** removes unnecessary weights (setting them to zero), reducing model size and computation.
- **Unstructured pruning** (magnitude-based): remove individual weights with the smallest absolute values. This creates a sparse weight matrix. Simple and effective for compression, but current hardware (GPUs) cannot accelerate sparse operations efficiently unless the sparsity follows specific patterns.
- **Structured pruning:** remove entire units — attention heads, MLP neurons, or layers. This produces a smaller dense model that is straightforward to accelerate on standard hardware. The tradeoff is coarser granularity (removing a full head might remove useful and useless weights together).
- **2:4 sparsity** (NVIDIA Ampere+): a hardware-supported sparsity pattern where 2 out of every 4 weights are zero. The GPU's sparse Tensor Cores skip the zero multiplications, achieving  $\sim 2\times$  speedup. This is the only sparsity pattern with practical hardware acceleration today.
- **Lottery Ticket Hypothesis** (Frankle & Carlin, 2019): within a randomly initialised network, there exists a subnetwork (the “winning ticket”) that can be trained in isolation to match the full network's performance. Finding these

subnetworks (by training, pruning, and rewinding) is expensive, but the insight motivates pruning research.

## 86.8 Neural Architecture Search (NAS)

---

- **NAS** automates architecture design by searching over a space of possible architectures to find the one that maximises accuracy subject to hardware constraints (latency, memory, power).
- **EfficientNet** (chapter 8) was found by NAS: the compound scaling rule (balance depth, width, resolution) emerged from the search, not from human intuition.
- For inference efficiency, NAS can find architectures optimised for specific hardware targets: “find a model with <5ms latency on an iPhone Neural Engine and >80% accuracy on ImageNet.” The search space includes layer types, widths, activation functions, and attention patterns.
- **Once-for-all networks** train a single overparameterised network and extract subnetworks for different deployment targets. One training run produces models for cloud GPU, mobile GPU, and CPU, each optimised for its target.

## 86.9 Coding Tasks (use CoLab or notebook)

---

1. Implement sliding window attention and compare memory usage against full attention.

```
import jax
import jax.numpy as jnp

def full_attention(Q, K, V):
 """Standard $O(n^2)$ attention."""
 scores = Q @ K.T / jnp.sqrt(Q.shape[-1])
 weights = jax.nn.softmax(scores, axis=-1)
 return weights @ V

def sliding_window_attention(Q, K, V, window_size=128):
 """Sliding window attention: each token attends to window_size previous
tokens."""
 n = Q.shape[0]
 d = Q.shape[-1]
 output = jnp.zeros_like(Q)
```

```

 for i in range(n):
 start = max(0, i - window_size + 1)
 k_window = K[start:i+1]
 v_window = V[start:i+1]
 scores = Q[i] @ k_window.T / jnp.sqrt(d)
 weights = jax.nn.softmax(scores)
 output = output.at[i].set(weights @ v_window)

 return output

n, d = 512, 64
key = jax.random.PRNGKey(0)
Q = jax.random.normal(key, (n, d))
K = jax.random.normal(jax.random.PRNGKey(1), (n, d))
V = jax.random.normal(jax.random.PRNGKey(2), (n, d))

print(f"Full attention memory: O(n^2) = {n*n} entries")
print(f"Window (w=128) memory: O(n*w) = {n*128} entries")
print(f"Reduction: {n*n / (n*128):.1f}x")

```

1. Compare the KV-cache size for MHA, GQA, and MQA. Show why GQA is the practical sweet spot.

```

def kv_cache_size(n_heads, n_kv_heads, d_head, seq_len, bytes=2):
 """KV-cache size in MB."""
 return 2 * n_kv_heads * d_head * seq_len * bytes / 1e6

n_heads = 32
d_head = 128
seq_len = 32768

mha = kv_cache_size(n_heads, n_heads, d_head, seq_len) # 32 KV heads
gqa = kv_cache_size(n_heads, 8, d_head, seq_len) # 8 KV heads
mqa = kv_cache_size(n_heads, 1, d_head, seq_len) # 1 KV head

print(f"MHA (32 KV heads): {mha:.0f} MB per layer")
print(f"GQA (8 KV heads): {gqa:.0f} MB per layer ({mha/gqa:.0f}x smaller)")
print(f"MQA (1 KV head): {mqa:.0f} MB per layer ({mha/mqa:.0f}x smaller)")

```

1. Simulate structured pruning by removing the least important attention heads from a random attention layer and measuring the output change.

```

import jax
import jax.numpy as jnp

```

```

key = jax.random.PRNGKey(0)
n_heads, seq_len, d_head = 8, 64, 32

Random multi-head attention output (one per head)
head_outputs = jax.random.normal(key, (n_heads, seq_len, d_head))

Full output: concatenate all heads
full_output = head_outputs.reshape(seq_len, n_heads * d_head)

Importance: measure each head's contribution by its norm
head_norms = jnp.linalg.norm(head_outputs, axis=(1, 2))
print("Head importance (by norm):", jnp.round(head_norms, 2))

Prune least important heads
for n_keep in [8, 6, 4, 2]:
 top_heads = jnp.argsort(head_norms)[-n_keep:]
 pruned = head_outputs[top_heads].reshape(seq_len, n_keep * d_head)

 # Pad to original size for comparison (zero out pruned heads)
 full_pruned = jnp.zeros_like(head_outputs)
 full_pruned = full_pruned.at[top_heads].set(head_outputs[top_heads])
 full_pruned = full_pruned.reshape(seq_len, n_heads * d_head)

 error = jnp.linalg.norm(full_output - full_pruned) /
jnp.linalg.norm(full_output)
 print(f"Keep {n_keep}/{n_heads} heads: relative error = {error:.4f}, "
 f"memory = {n_keep/n_heads:.0%}")

```

## 87 Serving and Batching

Serving an LLM to thousands of concurrent users requires more than loading a model and running inference. This file covers the prefill-decode split, continuous batching, PagedAttention and vLLM, scheduling strategies, disaggregated serving, multi-model and LoRA serving, and the metrics that matter

- A single LLM inference request is simple: feed in tokens, generate output tokens. But serving an LLM to 10,000 concurrent users at low latency and high throughput is a systems problem. The naive approach (process one request at a time) wastes 90%+ of GPU capacity. Smart batching and scheduling can increase throughput 10-50x without adding hardware.

## 87.1 Prefill vs Decode: Two Very Different Phases

- LLM inference has two distinct phases with fundamentally different computational characteristics:
- **Prefill** (prompt processing): process all input tokens simultaneously. This is a single large matrix multiplication:  $O(\text{prompt\_length} \times d_{\text{model}}^2)$ . The prompt can be processed in parallel (all tokens are known). Prefill is **compute-bound**: the GPU's ALUs are the bottleneck.
- **Decode** (token generation): generate one token at a time, autoregressively. Each new token requires attending to all previous tokens via the KV-cache. Decode is **memory-bandwidth-bound**: the GPU spends most time loading model weights and KV-cache from memory, not computing. Each decode step produces just one token but must load the entire model (~140 GB for a 70B model in FP16).
- The implications:

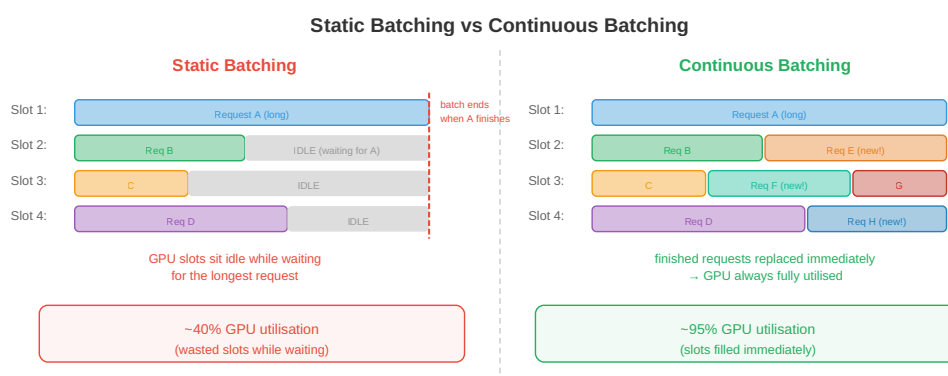
	Prefill	Decode
Tokens processed	All at once (parallel)	One at a time (sequential)
Bottleneck	Compute (FLOPS)	Memory bandwidth
Arithmetic intensity	High	Very low
GPU utilisation	High (50-80%)	Low (1-10%) without batching
Latency metric	<b>Time to First Token (TTFT)</b>	<b>Time Per Output Token (TPOT)</b>

- TTFT matters for user experience (how long until the response starts streaming). TPOT determines the perceived generation speed. Users tolerate higher TTFT (1-5 seconds) but expect fast TPOT (30-100 ms per token for conversational applications).



## 87.2 Static Batching (Naive)

- The simplest batching: collect B requests, pad them to the same length, process them as a single batch.
- **Problem 1:** requests have different prompt lengths and generate different numbers of output tokens. Short requests finish early but must wait for the longest request in the batch before the next batch can start. The GPU sits idle while generating for the one remaining long request.
- **Problem 2:** padding wastes compute. If the longest prompt is 2000 tokens and the shortest is 50, the batch is padded to 2000. The GPU processes 1950 padding tokens for the short request — pure waste.



Static batching wastes GPU slots while waiting for the longest request; continuous batching fills freed slots immediately

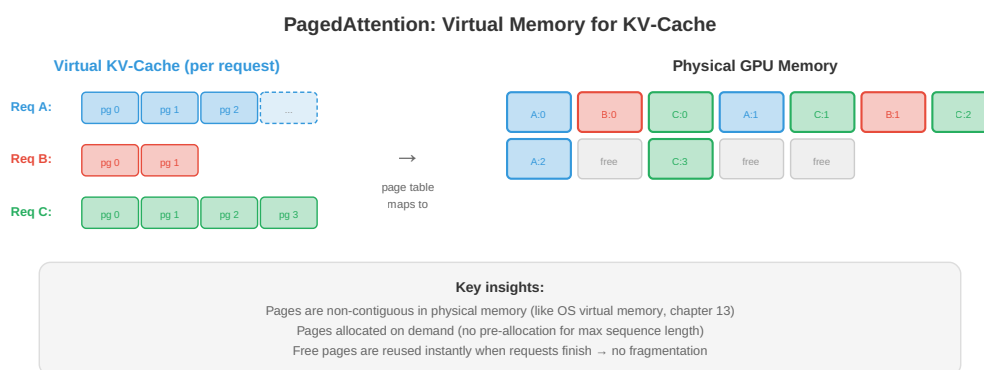
## 87.3 Continuous Batching

- **Continuous batching** (also called iteration-level batching) solves both problems by operating at the granularity of individual decode steps, not entire requests.
- At each decode step:
  1. All in-flight requests generate one token in parallel (as a batch).
  2. Requests that finish (generate EOS token) are **removed** from the batch immediately.
  3. New requests from the queue are **inserted** into the freed slots immediately.

- The batch size changes dynamically every step. The GPU is never idle waiting for stragglers, and there is no wasted padding (each request uses only the slots it needs).
- **Impact:** continuous batching typically increases throughput 2-10x over static batching, with no change to model quality or significant latency increase.

## 87.4 PagedAttention and vLLM

- The KV-cache creates a memory management nightmare. Each request has a KV-cache that grows with each generated token. Different requests are at different stages (different cache sizes). Allocating contiguous memory for each request wastes space (you must allocate for the maximum possible length, even if the request generates only a few tokens).



PagedAttention maps virtual KV-cache pages to non-contiguous physical GPU memory, eliminating fragmentation and enabling on-demand allocation

- **PagedAttention** (Kwon et al., 2023) applies the OS concept of virtual memory (chapter 13) to the KV-cache. The cache is divided into fixed-size **pages** (blocks of token positions). Pages are allocated on demand and can be non-contiguous in physical GPU memory.
- The benefits:
  - **No fragmentation:** pages are uniform size, so there are no “holes” of wasted memory between requests.
  - **Lazy allocation:** memory is allocated only when tokens are actually generated, not pre-allocated for maximum length.

- **Copy-on-write:** requests that share a common prefix (e.g., system prompts) share the same KV-cache pages. Only when the requests diverge are the pages copied.
- **vLLM** is the inference engine built around PagedAttention. It achieves 2-4x higher throughput than static-allocation serving (like HuggingFace's text-generation-inference without paged attention) by virtually eliminating KV-cache memory waste.

## 87.5 Scheduling Strategies

---

- When multiple requests are waiting and the GPU can only process a limited batch, **scheduling** decides which requests to serve:
- **First Come First Served (FCFS):** process requests in arrival order. Simple but unfair: a user submitting a 10K-token generation blocks all users behind them.
- **Shortest Job First (SJF):** process the request that will finish soonest. Minimises average latency but penalises long-running requests (they may starve). In practice, estimated output length is unknown, so SJF uses heuristics (prompt length, user history).
- **Preemption:** if a high-priority request arrives, pause a lower-priority in-progress request (swap its KV-cache to CPU memory or SSD), serve the high-priority request, then resume the paused one. vLLM supports this.
- **Priority-based:** assign priorities to users or request types. Real-time interactive queries get higher priority than batch processing jobs. Combined with preemption, this ensures latency SLOs for high-priority traffic.
- **Token budget:** limit the total number of tokens in the active batch. This prevents a few long requests from monopolising GPU memory and starving new requests.

## 87.6 Disaggregated Serving

---

- Prefill and decode have opposite computational profiles. Running both on the same GPU means the GPU alternates between being compute-bound (prefill) and memory-bandwidth-bound (decode), never fully utilising either resource.

- **Disaggregated serving** separates them:
  - **Prefill nodes:** GPUs optimised for compute (high FLOPS, possibly with less memory). Process all incoming prompts.
  - **Decode nodes:** GPUs optimised for memory bandwidth (large KV-cache capacity, high memory bandwidth). Handle all token generation.
- The prefill node computes the initial KV-cache and sends it to the decode node (over NVLink or network). The decode node generates tokens using the received cache.
- This is the architecture of **Mooncake** (Moonshot AI) and is being explored by several LLM serving teams. The benefit: each GPU type is matched to its workload characteristics, improving overall utilisation.

## 87.7 Multi-Model and LoRA Serving

---

- In production, you often serve multiple models (different sizes for different tiers, different fine-tuned variants for different tasks).
- **Model multiplexing:** load multiple models on the same GPU and route requests to the appropriate model. GPU memory is shared: a 40 GB GPU might hold a 13B model (26 GB) and a 7B model (14 GB) simultaneously.
- **LoRA serving:** instead of deploying separate fine-tuned models, deploy one base model with multiple **LoRA adapters** (chapter 6). Each adapter adds <1% parameters. Requests are routed to the appropriate adapter at inference time.
- **S-LoRA** (Sheng et al., 2023): serves thousands of LoRA adapters from a single base model. Adapters are stored on CPU and paged into GPU memory on demand. The base model's KV-cache and weights are shared; only the small LoRA matrices differ per request.
- **Punica** (Chen et al., 2023): batches requests across different LoRA adapters by using a custom CUDA kernel that applies different LoRA matrices to different requests within the same batch. This avoids the overhead of switching adapters per request.

## 87.8 Constrained and Guided Generation

---

- Many applications need the LLM to produce output in a specific format: valid JSON, SQL queries, code in a particular language, or responses that follow a schema. **Constrained generation** guarantees the output conforms to a grammar or schema.
- **Grammar-constrained decoding**: at each decoding step, mask out tokens that would violate the grammar. If the output so far is `{"name": "Alice", "age":` and the grammar requires an integer next, mask all tokens except digits. The LLM's probability distribution is renormalised over the valid tokens.
- **Outlines** (Willard & Louf, 2023): compiles a JSON schema or regular expression into a finite-state machine (FSM). At each decoding step, the FSM determines which tokens are valid continuations. Invalid tokens get probability 0. This guarantees 100% schema compliance with zero retries.
- **SGLang** integrates constrained generation natively: you specify the output structure in Python, and the engine handles the token masking and caching efficiently. This is combined with RadixAttention (prefix caching) so that structured outputs reuse cached prefixes.
- **Why it matters**: without constrained generation, you generate freely and parse the output, retrying on failure. Retry rates of 10-30% are common for complex JSON schemas, wasting compute. Constrained generation eliminates retries entirely.

## 87.9 Request Routing

---

- Not every query needs the biggest model. **Request routing** directs queries to different models based on estimated difficulty:
- **Cascading**: try a small model first. If the small model's confidence is below a threshold (e.g., the softmax probability of the top token is  $< 0.8$ ), escalate to a larger model. Easy queries (80%+ of traffic) are served cheaply by the small model; only hard queries use the expensive model.
- **Learned routing**: train a lightweight classifier (or use the small model's perplexity) to predict which model tier a query needs. Route “What is 2+2?” to a 3B model and “Explain the mathematical foundations of quantum entanglement” to a 70B model.

- **Impact:** if 80% of queries can be handled by a model that costs 10x less, the average cost per query drops by ~70%. This is one of the highest-impact cost optimisations for multi-model deployments.
- **On-device + cloud hybrid routing: Cactus** ([github.com/cactus-compute/cactus](https://github.com/cactus-compute/cactus)) implements request routing at the device level. It runs a small model on-device (phone, laptop, wearable) via custom ARM SIMD kernels, and automatically routes to a cloud model when the local model's confidence is low or the query exceeds the device's capability. The application uses an OpenAI-compatible API for both paths — the routing is transparent. This is cascading at the infrastructure level: the first tier is free (on-device), the second tier costs money (cloud API). For applications where most queries are simple (assistant Q&A, autocomplete, transcription), on-device handling covers 70-90% of traffic at zero marginal cost.

## 87.10 Inference Metrics

- The right metrics depend on the use case:

Metric	What It Measures	Target (Conversational)	Target (Batch)
<b>TTFT</b>	Time to first token	<1 s	less important
<b>TPOT</b>	Time per output token	<100 ms	less important
<b>Throughput</b>	Tokens/second (total)	less important	maximise
<b>p99 Latency</b>	Worst 1% of requests	<5 s	<30 s
<b>Cost per token</b>	\$/1M tokens	minimise	minimise

Metric	What It Measures	Target (Conversational)	Target (Batch)
SLO compliance	% of requests meeting latency target	>99%	>95%

- **TTFT vs TPOT tradeoff**: aggressive batching increases throughput (more tokens/s total) but increases TPOT (each token takes longer because the GPU processes more requests). The scheduling strategy must balance throughput (revenue) against latency (user experience).
- **Cost per token** is the ultimate metric for production. It combines hardware cost (GPU rental), throughput (tokens/s), and utilisation. A system running at 50% GPU utilisation costs 2x more per token than one at 100%. This is why batching, scheduling, and PagedAttention matter so much — they increase utilisation.

## 87.11 Coding Tasks (use CoLab or notebook)

1. Simulate continuous vs static batching and measure the throughput difference.

```
import random
import time

def simulate_static_batching(requests, batch_size=8):
 """Process requests in fixed batches. Wait for all to finish."""
 total_tokens = 0
 total_time = 0

 for i in range(0, len(requests), batch_size):
 batch = requests[i:i + batch_size]
 max_len = max(r['output_len'] for r in batch)
 # All requests in the batch take as long as the longest
 batch_time = max_len * 0.01 # 10ms per token
 total_time += batch_time
 total_tokens += sum(r['output_len'] for r in batch)

 return total_tokens / total_time # tokens per second

def simulate_continuous_batching(requests, max_batch=8):
 """Process with continuous batching. Remove finished, add new."""
 total_tokens = 0
```

```

total_time = 0
active = []
queue = list(requests)

while active or queue:
 # Fill batch
 while len(active) < max_batch and queue:
 active.append({'remaining': queue.pop(0)['output_len']})

 if not active:
 break

 # One decode step: all active requests generate 1 token
 for req in active:
 req['remaining'] -= 1
 total_tokens += len(active)
 total_time += 0.01 # 10ms per step

 # Remove finished requests
 active = [r for r in active if r['remaining'] > 0]

return total_tokens / total_time

Generate requests with varied output lengths
random.seed(42)
requests = [{'output_len': random.randint(10, 500)} for _ in range(100)]

static_tps = simulate_static_batching(requests)
continuous_tps = simulate_continuous_batching(requests)

print(f"Static batching: {static_tps:.0f} tokens/s")
print(f"Continuous batching: {continuous_tps:.0f} tokens/s")
print(f"Speedup: {continuous_tps / static_tps:.1f}x")

```

1. Calculate the KV-cache memory savings from PagedAttention. Compare pre-allocated (worst case) vs paged (actual usage).

```

def paged_vs_preallocated(n_requests, max_seq_len, avg_seq_len, page_size,
kv_per_token_bytes):
 """Compare memory usage: preallocated vs paged KV-cache."""
 # Preallocated: every request gets max_seq_len slots
 preallocated_gb = n_requests * max_seq_len * kv_per_token_bytes / 1e9

 # Paged: allocate only what is used (with page granularity)
 import math
 avg_pages = math.ceil(avg_seq_len / page_size)

```



```

 paged_gb = n_requests * avg_pages * page_size * kv_per_token_bytes / 1e9

 waste_preallocated = (max_seq_len - avg_seq_len) / max_seq_len
 waste_paged = (avg_pages * page_size - avg_seq_len) / (avg_pages * page_size)

 print(f"Requests: {n_requests}, Max seq: {max_seq_len}, Avg seq:
{avg_seq_len}")
 print(f" Preallocated: {preallocated_gb:.1f} GB (waste:
{waste_preallocated:.0%})")
 print(f" Paged: {paged_gb:.1f} GB (waste: {waste_paged:.0%})")
 print(f" Savings: {preallocated_gb - paged_gb:.1f} GB
({preallocated_gb/paged_gb:.1f}x)")
 print()

 # Llama-70B: ~1.3 KB per token per layer, 80 layers = ~100 KB per token total
 kv_bytes = 100_000

 # Scenario 1: short requests, large max
 paged_vs_preallocated(256, max_seq_len=4096, avg_seq_len=256, page_size=16,
kv_per_token_bytes=kv_bytes)

 # Scenario 2: varied lengths
 paged_vs_preallocated(256, max_seq_len=8192, avg_seq_len=1024, page_size=16,
kv_per_token_bytes=kv_bytes)

 # Scenario 3: long context
 paged_vs_preallocated(64, max_seq_len=131072, avg_seq_len=16000, page_size=16,
kv_per_token_bytes=kv_bytes)

```

## 88 Edge Inference

Edge inference runs models on user devices (phones, laptops, IoT sensors) without sending data to the cloud. This file covers edge constraints, the model compression pipeline, on-device runtimes, the compiler stack, hardware targets (NPUs, Neural Engines), on-device LLMs, federated learning, and latency optimisation

- Cloud inference requires network connectivity, adds latency (50-200 ms round trip), costs money per request, and sends user data to third-party servers. **Edge inference** eliminates all four: the model runs locally, responds instantly, costs nothing per inference, and keeps data private.

- The tradeoff: edge devices have 100-1000x less compute and memory than data centre GPUs. Making models run within these constraints requires aggressive optimisation at every level.
- **Cactus** ([github.com/cactus-compute/cactus](https://github.com/cactus-compute/cactus)) is a low-latency AI engine purpose-built for mobile and wearable devices. It demonstrates many of the techniques covered in this file in production: custom ARM SIMD kernels for attention and matrix operations (chapter 16), KV-cache quantisation (chapter 17 file 01), chunked prefill, NPU-accelerated inference on Apple and Qualcomm chips, zero-copy memory mapping for 10x lower RAM usage, and automatic cloud fallback when on-device compute is insufficient. Cactus supports multimodal inference (LLMs, vision, speech) across iOS, Android, macOS, and embedded Linux, with SDKs for Swift, Kotlin, Python, Flutter, React Native, and Rust. Its benchmarks show 100 tokens/s decode on M4 Pro and 48 tokens/s on iPhone 17 Pro for a 1.2B model at INT4, a concrete example of what optimised edge inference looks like.

## 88.1 Edge Constraints

Resource	Cloud GPU (H100)	Laptop (M4)	Phone (Snapdragon 8 Gen 3)	IoT (ESP32)
RAM	80 GB HBM3	16-36 GB unified	8-12 GB LPDDR5	520 KB
Compute	989 TFLOPS (FP8)	38 TOPS (Neural Engine)	45 TOPS (NPU)	0.001 TOPS
Power	700 W	15-30 W	5-10 W	0.1 W
Storage	TB	256 GB-2 TB	128-512 GB	4 MB

- The compute gap between a cloud GPU and a phone NPU is ~20x. Between a GPU and a microcontroller, it is ~1,000,000x. Different devices require different levels of compression and different model architectures.

## 88.2 The Model Compression Pipeline

---

- For edge deployment, compression is not a single technique — it is a **pipeline** of complementary techniques applied in sequence:

Full model (FP32, 70B params)

- ↓ Knowledge distillation → smaller model (7B params)
- ↓ Structured pruning → remove redundant heads/layers (4B effective)
- ↓ Quantisation (INT4) → 4x smaller (2 GB)
- ↓ Compiler optimisation → fused kernels, optimised memory layout
- ↓ Runtime → on-device execution

- Each step reduces size and latency. The order matters: distil first (reduce architecture), then prune (remove structure), then quantise (reduce precision), then compile (optimise for target hardware). Distilling after quantisation would try to compress an already-lossy model.

## 88.3 On-Device Runtimes

---

- A **runtime** loads a model, allocates memory, and executes inference on the target hardware. Each platform has its preferred runtime:
- **ONNX Runtime**: cross-platform (Windows, Linux, macOS, iOS, Android). Supports CPU, GPU (CUDA, DirectML, CoreML, NNAPI), and many accelerator backends. The most portable option. Models are exported to ONNX format from PyTorch/TensorFlow.
- **TensorFlow Lite (TFLite)**: Google's edge runtime. Optimised for ARM CPUs and Android NPUs. Tiny binary (~1 MB). Supports INT8 and float16. The standard for Android deployment.
- **Core ML**: Apple's runtime for iOS/macOS. Automatically uses the Neural Engine, GPU, or CPU depending on model characteristics. Models are converted from PyTorch/TensorFlow using `coremltools`. Tight integration with Apple hardware (unified memory, Neural Engine).
- **ExecuTorch**: Meta's new runtime for on-device PyTorch. Designed for edge deployment with ahead-of-time compilation and operator-level delegation to hardware accelerators. Successor to PyTorch Mobile.

- **TensorRT**: NVIDIA's runtime for GPU inference optimisation (chapter 15). Fuses layers, selects optimal kernels, and quantises automatically. 2-5x faster than PyTorch eager mode on NVIDIA GPUs.
- **llama.cpp**: single-file C++ inference engine for LLMs. Supports GGUF quantisation (Q4, Q5, Q8), CPU (AVX/NEON), Metal (Apple GPU), CUDA, and Vulkan. The go-to for running LLMs on consumer hardware.

## 88.4 The Compiler Stack

- Between the high-level model (PyTorch graph) and the hardware (NPU instructions) sits the **compiler stack**, which optimises the model for the specific target:

```
PyTorch model
 ↓ Export (torch.export, ONNX, TorchScript)
Graph IR (intermediate representation)
 ↓ Graph optimisations
 - Constant folding (compute constant expressions at compile time)
 - Dead code elimination (remove unused operations)
 - Operator fusion (conv + bn + relu → single fused op)
 - Layout transformation (NCHW → NHWC for ARM, channels-last)
 ↓ Lowering
Hardware-specific IR
 ↓ Backend optimisations
 - Tiling and loop ordering (cache-friendly access patterns)
 - Vectorisation (SIMD, chapter 16)
 - Memory planning (reuse buffers to minimise peak memory)
 - Kernel selection (choose the best implementation per op)
 ↓ Code generation
Machine code / NPU instructions
```

- **Operator fusion** is the most impactful optimisation. A transformer block has ~20 operations (matmul, add, layernorm, softmax, etc.). Without fusion, each writes its output to memory and the next reads it back. With fusion, multiple operations are combined into a single kernel that keeps data in registers/cache. This can be 2-5x faster (chapter 16, roofline model).
- **Memory planning**: the compiler analyses the model graph to determine which tensors overlap in lifetime and can share the same memory buffer. A model with 100 intermediate tensors might only need memory for 10, because most

are consumed and freed before others are created. This is critical on devices with limited RAM.

## 88.5 Hardware Targets

---

### 88.5.1 Mobile GPUs

- **Qualcomm Adreno** (Android): supports OpenCL, Vulkan compute (chapter 16), and Qualcomm's proprietary SNPE (Snapdragon Neural Processing Engine). Adreno GPUs have 256-1024 ALUs with FP16 and INT8 support.
- **ARM Mali** (Android): supports OpenCL and Vulkan. Mali GPUs use a tile-based architecture (different from desktop GPUs), which affects optimal memory access patterns.
- **Apple GPU** (iOS/macOS): accessed via Metal (Apple's GPU API). Unified memory architecture means no CPU↔GPU copy overhead. Metal Performance Shaders (MPS) provide optimised ML primitives.

### 88.5.2 Neural Processing Units (NPUs)

- NPUs are fixed-function accelerators designed specifically for ML inference. They are far more power-efficient than GPUs for standard ML operations (matmul, conv, activation).
- **Apple Neural Engine**: 16 cores, ~38 TOPS (INT8). Accessed via Core ML. Excellent for vision models and on-device diffusion. Cannot run arbitrary code — only operations supported by Core ML.
- **Qualcomm Hexagon NPU**: integrated into Snapdragon SoCs. Supports INT8 and INT4 inference. Accessed via SNPE or ONNX Runtime with QNN backend. Powers on-device features like background blur, speech recognition, and real-time translation.
- **Google Edge TPU**: a small, low-power version of the cloud TPU. 4 TOPS, 2W. Used in Coral devices for on-device inference. Supports only INT8 quantised TFLite models.
- **The delegation pattern**: the runtime splits the model graph between the NPU (for supported operations) and the CPU (for unsupported ones). Maximising the fraction that runs on the NPU is key to performance and power efficiency.

## 88.6 On-Device LLMs

---

- Running LLMs on phones and laptops has become feasible with small models and aggressive quantisation:

Model	Params	Quantised Size	Target Device	Performance
Phi-3 Mini	3.8B	~2 GB (Q4)	Phone/Laptop	~15 tokens/s on iPhone 15
Gemma 2B	2B	~1.5 GB (Q4)	Phone	~20 tokens/s on Pixel 8
Llama 3.2 1B	1B	~700 MB (Q4)	Phone	~30 tokens/s
Llama 3.2 3B	3B	~2 GB (Q4)	Phone/Laptop	~15 tokens/s
Llama 3.1 8B	8B	~4.5 GB (Q4)	Laptop	~20 tokens/s on M2

- **Challenges:**
  - **Memory:** a 3B Q4 model fits in 2 GB, but the KV-cache for long conversations adds significantly. Context length is typically limited to 2-4K tokens on phones.
  - **Thermal throttling:** sustained inference heats the phone. After 30 seconds of continuous generation, the SoC throttles clock speeds to prevent overheating, reducing performance by 30-50%.
  - **Battery:** running a 3B model at 15 tokens/s consumes ~3-5W. A 30-minute conversation drains ~5% of a typical phone battery. Acceptable for occasional use, problematic for always-on applications.
- **llama.cpp** is the standard for on-device LLMs. It runs on CPU (AVX2, NEON, I8MM), Apple GPU (Metal), NVIDIA GPU (CUDA), AMD GPU (ROCm/Vulkan), and even phones (via Termux on Android).

## 88.7 Federated Learning

---

- **Federated learning** trains models across many devices without centralising the data. Each device trains on its local data, computes a gradient update, and

sends only the update (not the data) to a central server that aggregates the updates.

- **The algorithm** (FedAvg):

1. Server sends the current model to K selected devices.
2. Each device fine-tunes the model on its local data for a few steps.
3. Each device sends its updated model (or the difference) back to the server.
4. Server averages the updates:  $W_{\text{new}} = \frac{1}{K} \sum_{k=1}^K W_k$ .
5. Repeat.

- **Privacy**: raw data never leaves the device. The server only sees aggregated model updates. **Differential privacy** adds noise to the updates so that individual data points cannot be reverse-engineered from the gradient.

- **Communication efficiency**: model updates are large (same size as the model). Compression techniques reduce this: **gradient quantisation** (send INT8 gradients instead of FP32), **sparsification** (send only the largest gradients), and **gradient accumulation** (do more local steps, send less often).

- **Applications**: Google's keyboard predictions (Gboard), Apple's voice recognition, health monitoring (train on sensitive health data without centralising it).

## 88.8 Latency Optimisation

---

- Beyond compression, several techniques reduce end-to-end inference latency:

- **Early exit**: add classification heads at intermediate layers. If the model is confident at layer 6 (out of 24), return the prediction without running layers 7-24. Easy inputs exit early, hard inputs use the full model. Average latency drops significantly for tasks with a mix of easy and hard inputs.

- **Model partitioning**: split the model between the NPU (efficient for matmul), GPU (efficient for irregular operations), and CPU (handles everything else). The compiler decides which operations go where based on profiling.

- **Caching**: for applications with repeated queries (autocomplete, code completion), cache recent computations. If the user types "How do I" and the model has recently generated completions for "How do I," the cached KV-cache can be reused, skipping the prefill phase entirely.

- **Speculative prefetching:** predict what the user will do next and start inference before they ask. A chat app might start generating a response to the likely follow-up question while the user is reading the current answer.

## 88.9 Coding Tasks (use CoLab or notebook)

1. Simulate the model compression pipeline. Start with a float32 model, apply distillation (mock), pruning, and quantisation, and track the size at each step.

```
def compression_pipeline(original_params_M, original_bits=32):
 size_mb = original_params_M * 1e6 * original_bits / 8 / 1e6

 print(f"Original: {original_params_M}M params, {original_bits}-bit → {size_mb:.0f} MB")

 # Step 1: Knowledge distillation (reduce params)
 distilled_params = original_params_M * 0.15 # 70B → ~10B equivalent
 size_mb = distilled_params * 1e6 * original_bits / 8 / 1e6
 print(f"After distillation ({distilled_params:.0f}M params): {size_mb:.0f} MB")

 # Step 2: Structured pruning (remove 30% of remaining)
 pruned_params = distilled_params * 0.7
 size_mb = pruned_params * 1e6 * original_bits / 8 / 1e6
 print(f"After pruning ({pruned_params:.0f}M params): {size_mb:.0f} MB")

 # Step 3: INT4 quantisation
 size_mb = pruned_params * 1e6 * 4 / 8 / 1e6
 print(f"After INT4 quantisation: {size_mb:.0f} MB")

 print(f"Total compression: {original_params_M * 1e6 * original_bits / 8 / 1e6 / size_mb:.0f}x")

 print("=== Starting from 70B model ===")
 compression_pipeline(70000)

 print("\n=== Starting from 7B model ===")
 compression_pipeline(7000)
```

1. Estimate on-device inference latency. Given a model's operations count and hardware specs, compute whether it meets a latency target.

```
def estimate_latency(model_name, params_M, bits, compute_tops, mem_bw_gbs, seq_len=256):
```



```

"""Estimate token generation latency for a memory-bandwidth-bound model."""
Model size in bytes
model_bytes = params_M * 1e6 * bits / 8

Decode is memory-bound: must load entire model per token
time_per_token_ms = model_bytes / (mem_bw_gbs * 1e9) * 1000

Tokens per second
tokens_per_sec = 1000 / time_per_token_ms

print(f"{model_name}: {params_M/1000:.1f}B params @ {bits}-bit =
{model_bytes/1e9:.1f} GB")
print(f" Memory bandwidth: {mem_bw_gbs} GB/s")
print(f" Time per token: {time_per_token_ms:.1f} ms")
print(f" Tokens/sec: {tokens_per_sec:.0f}")
print()

Apple M2 Pro: 200 GB/s unified memory bandwidth
print("=== Apple M2 Pro (200 GB/s) ===")
estimate_latency("Llama-7B Q4", 7000, 4, 15.8, 200)
estimate_latency("Llama-7B Q8", 7000, 8, 15.8, 200)
estimate_latency("Llama-70B Q4", 70000, 4, 15.8, 200)

Phone (Snapdragon 8 Gen 3): ~50 GB/s LPDDR5
print("=== Snapdragon 8 Gen 3 (50 GB/s) ===")
estimate_latency("Phi-3 Mini Q4", 3800, 4, 45, 50)
estimate_latency("Llama-3B Q4", 3000, 4, 45, 50)

```

## 89 Scaling and Deployment

Serving large models to millions of users requires distributing inference across multiple GPUs, predicting tokens before they are needed, caching shared context, and choosing the right framework. This file covers parallelism at inference, speculative decoding, prefix caching, inference frameworks, cost optimisation, and monitoring.

- A single H100 GPU serving a 70B model can handle ~100 concurrent users at interactive latency. Serving 10 million users requires 100,000 GPUs — costing ~\$3 billion per year in cloud compute. Every percentage point of efficiency improvement saves tens of millions of dollars. This is why inference optimisation is not academic: it directly determines the economics of AI products.

## 89.1 Model Parallelism at Inference

---

- When a model is too large for a single GPU, it must be split across multiple GPUs. The parallelism strategies from training (chapter 6) apply at inference with different tradeoffs.

### 89.1.1 Tensor Parallelism

- **Tensor parallelism** (Megatron-style, chapter 6) splits individual weight matrices across GPUs. For a linear layer  $Y = XW$ , the weight matrix  $W$  is split column-wise across  $N$  GPUs. Each GPU computes a partial result, then an all-reduce aggregates:

$$W = [W_1 | W_2 | \dots | W_N], \quad Y_i = XW_i, \quad Y = \text{concat}(Y_1, \dots, Y_N)$$

- At inference, tensor parallelism is the default for models that do not fit on one GPU. A 70B model in FP16 requires 140 GB — split across  $2 \times 80$  GB GPUs with tensor parallelism.
- **Latency impact:** tensor parallelism adds an all-reduce communication step per layer. On NVLink (900 GB/s), this adds  $\sim 0.1$  ms per layer. On PCIe (32 GB/s), it adds  $\sim 3$  ms per layer. For a 70B model with 80 layers on 2 GPUs: NVLink adds  $\sim 8$  ms total, PCIe adds  $\sim 240$  ms. This is why NVLink matters enormously for multi-GPU inference.

### 89.1.2 Pipeline Parallelism

- **Pipeline parallelism** assigns different layers to different GPUs. GPU 1 handles layers 0-39, GPU 2 handles layers 40-79. Tokens flow through the pipeline sequentially.
- At inference, pipeline parallelism has higher latency than tensor parallelism (the entire pipeline must be traversed for each token) but lower communication overhead (only activations are passed between GPUs, no all-reduce). It is preferred when GPUs are connected by slow interconnects (different nodes, no NVLink).

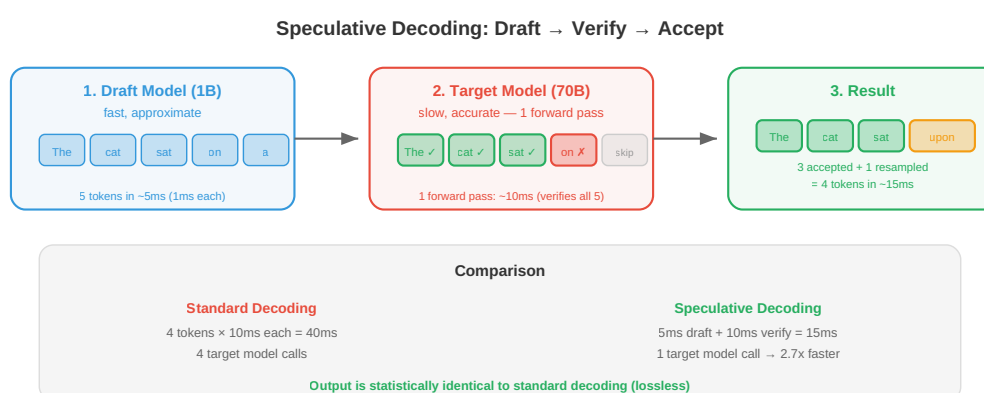
### 89.1.3 Sequence Parallelism

- For very long sequences, the KV-cache itself may not fit on one GPU even if the model does. **Sequence parallelism** shards the KV-cache across GPUs: each GPU stores a portion of the sequence's cached keys and values.

- During attention, each GPU computes partial attention scores over its cached segment, then a reduction combines the results. This is used in long-context inference (128K+ tokens) where the KV-cache exceeds single-GPU memory.

## 89.2 Speculative Decoding

- **Speculative decoding** is one of the most impactful LLM inference optimisations. The insight: decoding is slow because it generates one token at a time, with each token requiring a full forward pass of the large model. But a small model can generate candidate tokens much faster, and the large model can **verify** multiple candidates in parallel.



Speculative decoding: a fast draft model generates 5 candidate tokens, the target model verifies all in one pass, accepted tokens are kept and rejected ones resampled

- **The algorithm:**
  1. A **draft model** (small, fast — e.g., 1B params) generates k candidate tokens autoregressively.
  2. The **target model** (large, accurate — e.g., 70B) runs a single forward pass on the entire draft sequence, computing the probability of each candidate token.
  3. Each candidate is **accepted** if the target model agrees (its probability for that token is high enough). Rejected candidates are resampled from the target model's distribution.
  4. On average, multiple tokens are accepted per verification step, giving a speedup proportional to the acceptance rate.

$$\text{Speedup} \approx \frac{k \times \text{acceptance\_rate}}{\text{cost\_ratio}} \approx 2\text{-}3 \times$$

- **Why it works without quality loss:** the rejection sampling scheme guarantees that the output distribution matches the target model exactly. Speculative decoding is lossless — the output is statistically identical to running the target model alone, just faster.
- **Variants:**
  - **Medusa** (Cai et al., 2024): instead of a separate draft model, add multiple lightweight “heads” to the target model that predict multiple future tokens simultaneously. No separate model needed.
  - **EAGLE** (Li et al., 2024): trains a lightweight draft head that uses the target model’s hidden states to predict future tokens. Higher acceptance rate than independent draft models.
  - **Self-speculative decoding:** the target model itself generates drafts using early exit (run only the first few layers for the draft, then verify with the full model).
  - **Parallel decoding:** generate multiple continuations in parallel (tree of candidates) and verify the entire tree at once. Higher throughput but uses more memory for the branching KV-caches.

## 89.3 Prefix Caching

---

- Many requests share common prefixes: system prompts, few-shot examples, or common query patterns. **Prefix caching** stores the KV-cache for these prefixes and reuses it across requests.
- **System prompt caching:** if every request starts with the same 2000-token system prompt, the KV-cache for those 2000 tokens is computed once and shared across all requests. For a 70B model with 80 layers, this saves ~200 MB per request.
- **Radix tree caching** (SGLang): organise cached prefixes in a radix tree (trie). When a new request arrives, find the longest cached prefix match and start generation from there, skipping the matching prefix’s computation.
- **Impact:** for applications with long shared prefixes (chatbots with system prompts, RAG with common retrieved passages), prefix caching reduces TTFT by 50-90% and saves proportional GPU compute.

## 89.4 KV-Cache Eviction

---

- Beyond quantising the KV-cache (file 01) and using GQA/MLA to reduce its size (file 02), **KV-cache eviction** policies selectively remove cached tokens that are unlikely to be attended to in the future.
- **H2O** (Heavy-Hitter Oracle, Zhang et al., 2023) observes that attention scores follow a power law: a small fraction of tokens ( “heavy hitters” ) receive most of the attention, while the majority receive negligible attention. H2O keeps:
  1. **Recent tokens** (a sliding window of the last  $w$  tokens, like StreamingLLM).
  2. **Heavy-hitter tokens** (the top- $k$  tokens by cumulative attention score across all past decode steps).
- Tokens that are neither recent nor heavy-hitters are evicted. This maintains a fixed-size KV-cache while preserving the tokens that actually influence generation. H2O achieves quality close to full KV-cache with only 20% of the memory.
- **Scissorhands** (Liu et al., 2023) takes a similar approach but uses a more sophisticated importance metric: tokens that receive high attention in the **current** step are kept, while tokens that have not been attended to for  $T$  steps are evicted. This adapts to changing attention patterns during generation.
- **Dynamic eviction + StreamingLLM**: combine attention sinks (keep the first few tokens permanently) with dynamic eviction (keep recent + heavy-hitter tokens). This is the most memory-efficient approach for very long generation, enabling infinite-length generation with bounded quality degradation.
- The key insight across all eviction methods: LLM attention is **sparse in practice** — even though the architecture computes attention over all cached tokens, the actual attention weights concentrate on a small subset. Evicting the rest has minimal impact on output quality.

## 89.5 Inference Frameworks

---

- The LLM serving ecosystem has converged on a few dominant frameworks:

Framework	Strengths	Best For
vLLM		

Framework	Strengths	Best For
	PagedAttention, continuous batching, high throughput	General LLM serving, highest throughput
<b>TensorRT-LLM</b>	NVIDIA-optimised kernels, FP8, in-flight batching	Maximum performance on NVIDIA GPUs
<b>SGLang</b>	Prefix caching (RadixAttention), fast structured generation	Applications with shared prefixes, constrained output
<b>llama.cpp</b>	CPU/Metal/CUDA/Vulkan, GGUF quantisation, portable	Consumer hardware, on-device inference
<b>TGI</b> (HuggingFace)	Simple API, easy to deploy, model hub integration	Quick deployments, HuggingFace ecosystem
<b>Ollama</b>	One-command model download and serving	Personal use, local development
<b>ExLlamaV2</b>	Extreme quantisation optimisation (EXL2 format)	Memory-constrained GPU inference

- **vLLM** is the default choice for production LLM serving. It supports continuous batching, PagedAttention, tensor parallelism, speculative decoding, LoRA serving, and most open-source models.
- **TensorRT-LLM** achieves the highest raw performance on NVIDIA hardware (10-30% faster than vLLM on the same GPU) but is less flexible and harder to customise.
- **SGLang** excels when applications have structured output (JSON, code with specific format) or shared prefixes, thanks to its radix attention cache and constrained decoding engine.

## 89.6 Cost Optimisation

---

- At scale, inference cost dominates ML budgets. Strategies to reduce cost:
- **Right-sizing GPU selection:** not every model needs an H100. A quantised 7B model runs well on an A10G (~\$1/hr) instead of an H100 (~\$8/hr). Match the GPU to the workload.
- **Spot instances:** cloud providers offer unused GPU capacity at 60-90% discount (AWS Spot, GCP Preemptible). Spot instances can be interrupted, so they work for batch inference but not latency-critical serving. Combined with preemption handling (save state, resume on a new instance), spot instances can serve interactive traffic too.
- **Autoscaling:** scale GPU count based on traffic. Scale up during peak hours, scale down at night. Kubernetes HPA (Horizontal Pod Autoscaler) or cloud-native autoscaling (AWS SageMaker, GCP Vertex AI) handle this.
- **Batching + utilisation:** the difference between 30% and 90% GPU utilisation is 3x in cost per token. Continuous batching, smart scheduling, and PagedAttention all increase utilisation.
- **Quantisation:** INT4 vs FP16 is 4x less memory → fits on a smaller GPU → 2-4x lower cost. Plus, more requests fit in the same batch → higher throughput → lower cost per token.
- **Cost per token benchmarks** (approximate, 2026):

Setup	Cost per 1M tokens
GPT-4o API	\$2.50
Claude 3.5 Sonnet API	\$3.00
Llama-70B on H100 (vLLM, FP16)	\$0.50
Llama-70B on H100 (TRT-LLM, INT8)	\$0.25
Llama-8B on A10G (vLLM, INT4)	\$0.05
Llama-3B on-device (llama.cpp)	\$0 (hardware amortised)

## 89.7 Monitoring

---

- Production inference requires continuous monitoring to catch degradation before users are affected:
- **Latency monitoring:** track TTFT and TPOT at p50, p95, and p99. Set alerts for p99 exceeding SLO. A spike in p99 often indicates: KV-cache memory pressure (thrashing), a long-running request monopolising the batch, or GPU thermal throttling.
- **Throughput monitoring:** track tokens per second per GPU. A drop indicates: reduced batch efficiency (many short requests → low batch utilisation), increased sequence lengths (more KV-cache memory per request), or hardware issues (GPU in ECC error correction mode, running slower).
- **GPU utilisation:** track SM occupancy, memory utilisation, and memory bandwidth. Low SM occupancy + high memory utilisation = memory-bound (need more bandwidth or quantisation). High SM occupancy + low memory utilisation = compute-bound (need more FLOPS or smaller model).
- **Model quality monitoring:** track per-request metrics (response length, perplexity on a held-out set, user feedback signals). Model quality can degrade due to: data drift (the distribution of incoming requests changes), KV-cache quantisation error accumulating over long conversations, or bugs in the serving pipeline.
- **Cost monitoring:** track cost per token per model per GPU type. If cost increases without throughput increase, investigate efficiency regressions (new model version with higher memory usage, suboptimal batch configuration, or underutilised GPUs).
- **Tools:** Prometheus + Grafana (chapter 15) for infrastructure metrics, vLLM/ TRT-LLM’ s built-in metrics endpoints, and custom logging for model-level metrics.

## 89.8 Coding Tasks (use CoLab or notebook)

---

1. Simulate speculative decoding. Use a fast “draft” function and a slow “target” function, and measure the speedup from generating and verifying multiple tokens at once.



```

import random
import time

def target_model(tokens):
 """Slow but accurate model. Returns probability of each candidate token."""
 time.sleep(0.01) # simulate 10ms per forward pass
 # For simulation: accept tokens that are even numbers
 return [0.9 if t % 2 == 0 else 0.1 for t in tokens]

def draft_model():
 """Fast but approximate model. Generates one candidate token."""
 time.sleep(0.001) # simulate 1ms per token
 return random.randint(0, 9)

def standard_decoding(n_tokens):
 """Generate one token at a time with the target model."""
 tokens = []
 for _ in range(n_tokens):
 time.sleep(0.01) # target model generates 1 token
 tokens.append(random.randint(0, 9))
 return tokens

def speculative_decoding(n_tokens, k=4):
 """Generate k draft tokens, verify with target, accept/reject."""
 tokens = []
 total_target_calls = 0

 while len(tokens) < n_tokens:
 # Draft: generate k candidates quickly
 candidates = [draft_model() for _ in range(k)]

 # Verify: one target model call for all k candidates
 probs = target_model(candidates)
 total_target_calls += 1

 # Accept tokens until one is rejected
 for i, (tok, prob) in enumerate(zip(candidates, probs)):
 if random.random() < prob:
 tokens.append(tok)
 if len(tokens) >= n_tokens:
 break
 else:
 # Resample from target distribution
 tokens.append(tok + 1) # simplified resampling
 break

 return tokens, total_target_calls

```

```

n = 50

start = time.time()
_ = standard_decoding(n)
standard_time = time.time() - start

start = time.time()
_, target_calls = speculative_decoding(n, k=5)
spec_time = time.time() - start

print(f"Standard: {standard_time:.2f}s ({n} target calls)")
print(f"Speculative: {spec_time:.2f}s ({target_calls} target calls)")
print(f"Speedup: {standard_time / spec_time:.1f}x")

```

1. Estimate the cost savings from different optimisation strategies applied to an LLM serving deployment.

```

def serving_cost_analysis(
 model_name, params_B, precision_bits,
 gpu_name, gpu_mem_gb, gpu_cost_per_hr,
 target_throughput_tps,
):
 """Estimate serving cost for an LLM deployment."""
 model_size_gb = params_B * 1e9 * precision_bits / 8 / 1e9
 gpus_for_model = max(1, int((model_size_gb * 1.2) / gpu_mem_gb + 0.99)) #
1.2x for KV-cache

 # Rough throughput estimate (memory-bandwidth limited)
 tokens_per_gpu = 500 / (params_B * precision_bits / 16) # normalised to 500
tok/s for 7B FP16
 total_throughput = tokens_per_gpu * gpus_for_model

 replicas = max(1, int(target_throughput_tps / total_throughput + 0.99))
 total_gpus = gpus_for_model * replicas
 cost_per_hr = total_gpus * gpu_cost_per_hr
 cost_per_1M_tokens = cost_per_hr / (total_throughput * replicas * 3600 / 1e6)

 print(f"{model_name} @ {precision_bits}-bit on {gpu_name}:")
 print(f" Model size: {model_size_gb:.0f} GB → {gpus_for_model} GPU(s)/
replica")
 print(f" Throughput: {total_throughput:.0f} tok/s/replica")
 print(f" Replicas for {target_throughput_tps} tok/s: {replicas}")
 print(f" Total GPUs: {total_gpus}")
 print(f" Cost: ${cost_per_hr:.0f}/hr, ${cost_per_1M_tokens:.2f}/1M tokens")
 print()

```

```
print("=== Cost Comparison ===\n")

Baseline: FP16 on H100
serving_cost_analysis("Llama-70B", 70, 16, "H100", 80, 8.0, 1000)

Quantised: INT8 on H100
serving_cost_analysis("Llama-70B", 70, 8, "H100", 80, 8.0, 1000)

Quantised: INT4 on A100
serving_cost_analysis("Llama-70B", 70, 4, "A100", 80, 4.0, 1000)

Smaller model: 8B on A10G
serving_cost_analysis("Llama-8B", 8, 4, "A10G", 24, 1.0, 1000)
```

## 90 Systems Design Fundamentals

Systems design is how you build software that works reliably at scale. This file covers client-server architecture, networking protocols, DNS, proxies, load balancing, caching, databases, message queues, consistency models, and resilience patterns

- Every ML system in production is a distributed system. A recommendation engine is not just a model — it is an API server, a feature store, a model registry, a caching layer, a message queue, and a monitoring stack, all communicating over a network. Understanding systems design is what separates “I trained a model” from “I built a product.”
- Systems design interviews at top tech companies (Google, Meta, Amazon, OpenAI) test whether you can design these systems. This chapter gives you the building blocks (this file), the cloud infrastructure (file 02), scaling patterns (file 03), ML-specific design (file 04), and worked examples (file 05).

### 90.1 Client-Server Architecture

---

- The fundamental pattern: a **client** sends a request, a **server** processes it and returns a response. Your browser (client) sends an HTTP request to google.com (server), which returns HTML.
- **Request-response model:** synchronous. The client waits for the response. Simple but creates a bottleneck: the client is idle while waiting, and the server must handle the request before moving on.

- **Stateless servers:** the server does not remember previous requests. Each request contains all the information needed to process it. This makes scaling easy: any server can handle any request, so you can add more servers behind a load balancer.
- **Stateful servers:** the server maintains state between requests (e.g., a user session). Harder to scale because requests from the same user must go to the same server (session affinity). Modern systems avoid server-side state by storing it in a database or cache (Redis).

## 90.2 Networking Protocols

---

- We covered networking in chapter 13 (TCP/IP layers, sockets). Here we focus on the application-level protocols used in systems design:
- **HTTP/HTTPS:** the protocol of the web and most APIs. Request methods: GET (read), POST (create/predict), PUT (update), DELETE (remove). HTTPS adds TLS encryption (chapter 13 security). REST APIs (chapter 15 file 03) are built on HTTP.
- **WebSockets:** persistent bidirectional connection between client and server. Unlike HTTP (request → response → connection closes), WebSocket keeps the connection open for real-time streaming. Used for: LLM token streaming (send tokens as they are generated), live dashboards, chat applications.
- **gRPC:** Google's RPC framework. Uses Protocol Buffers (binary serialisation, ~10x smaller and faster than JSON) over HTTP/2. Supports streaming (server-side, client-side, bidirectional). Used for internal service-to-service communication where performance matters. Triton Inference Server (chapter 15) and TensorFlow Serving use gRPC.
- **Protocol Buffers:** define message schemas in .proto files:

```
message PredictRequest {
 repeated float features = 1;
 string model_version = 2;
}

message PredictResponse {
 float prediction = 1;
 float confidence = 2;
}
```

```
service ModelService {
 rpc Predict(PredictRequest) returns (PredictResponse);
}
```

- The schema is compiled into client and server code in any language (Python, C++, Go, Java). Type safety, backward compatibility, and performance come for free.

## 90.3 DNS

---

- **DNS** (Domain Name System) translates human-readable names to IP addresses (chapter 13). For systems design, DNS also provides:
- **Load balancing via DNS**: return different IP addresses for the same domain name, distributing traffic across multiple servers. Simple but coarse-grained (DNS results are cached for minutes to hours, so traffic does not rebalance quickly).
- **Geographic routing**: return the IP of the nearest data centre based on the client's location. A user in Tokyo gets the Japanese data centre; a user in London gets the European one.
- **Failover**: if a server goes down, DNS stops returning its IP. New clients go to healthy servers. But cached DNS entries mean some clients continue hitting the dead server for minutes (the TTL problem).

## 90.4 Proxies

---

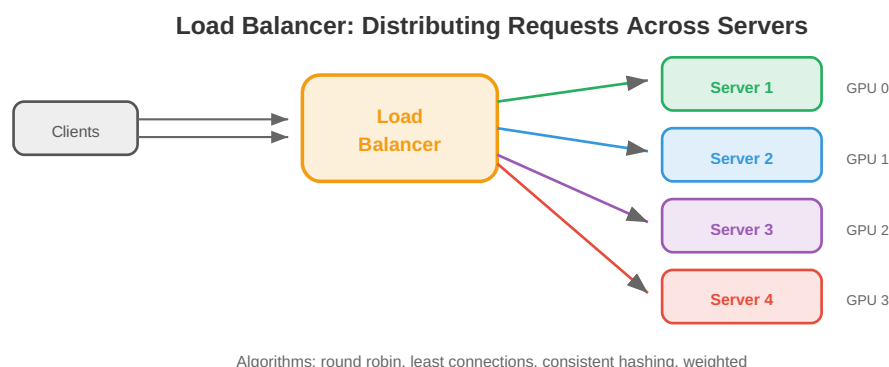
- A **proxy** is an intermediary between client and server:
- **Reverse proxy** (in front of servers): clients connect to the proxy, which forwards requests to backend servers. The client does not know which server handled the request. **Nginx** and **HAProxy** are the standard reverse proxies. They provide: load balancing (distribute requests), SSL termination (decrypt HTTPS at the proxy, send plain HTTP to backends), caching, rate limiting, and compression.
- **API gateway**: a specialised reverse proxy for APIs. Handles authentication, rate limiting, request routing (different paths → different services), and API versioning. **Kong**, **AWS API Gateway**, and **Envoy** are common choices.

- For ML serving: an API gateway sits in front of your model servers. It authenticates API keys, rate-limits free-tier users, routes `/v1/predict` to model server A and `/v2/predict` to model server B, and collects usage metrics.

## 90.5 Load Balancing

---

- When you have multiple servers, a **load balancer** distributes incoming requests across them.

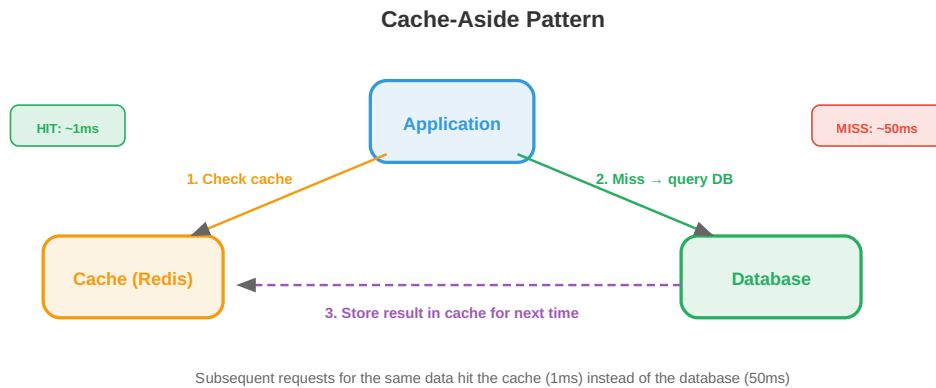


Load balancer distributes incoming requests across multiple backend servers

- **Algorithms:**
  - **Round robin:** send requests to servers in order (1, 2, 3, 1, 2, 3...). Simple, fair, but does not account for server load.
  - **Least connections:** send to the server with the fewest active connections. Better for requests with variable processing time (some LLM requests generate 10 tokens, others generate 1000).
  - **Weighted round robin:** servers with more capacity get more requests. A server with 80 GB GPU memory handles 2x the requests of one with 40 GB.
  - **Consistent hashing:** hash the request key to a specific server. Same key always goes to the same server. Useful for: caching (requests for the same user hit the same cache), session affinity, and prefix caching (chapter 17: requests with the same system prompt go to the server that has the KV-cache for that prompt).
- **L4 vs L7 load balancing:**
  - **L4 (transport layer):** routes based on IP and port. Fast but cannot inspect request content.
  - **L7 (application layer):** routes based on HTTP path, headers, or body content. Can route `/api/chat` to chat servers and `/api/embed` to embedding servers. Slower but more flexible.

## 90.6 Caching

- **Caching** stores frequently accessed data in a fast storage layer (RAM) to avoid recomputing or re-fetching it.



Cache-aside pattern: check cache first, on miss fetch from database and store in cache for next time

- **Cache patterns:**
  - **Cache-aside** (lazy loading): the application checks the cache first. On miss, it fetches from the database, stores in cache, and returns. Most common pattern.
  - **Write-through:** every write goes to both the cache and database simultaneously. Ensures cache is always up to date but slows writes.
  - **Write-back:** writes go to the cache only; the cache asynchronously flushes to the database. Fastest writes but risks data loss if the cache crashes before flushing.
- **Eviction policies** (when the cache is full):
  - **LRU** (Least Recently Used): evict the entry that has not been accessed for the longest time. The most common policy.
  - **LFU** (Least Frequently Used): evict the least-accessed entry. Better when some items are consistently popular.
  - **TTL** (Time To Live): entries expire after a fixed duration. Used for data that becomes stale (model predictions cached for 5 minutes, feature values cached for 1 hour).
- **CDN** (Content Delivery Network): a globally distributed cache for static content (images, JavaScript, CSS). Servers in 100+ locations worldwide serve cached

content from the nearest location to the user. For ML: model weights can be cached on CDNs for fast download.

- **Redis:** the standard in-memory cache/database. Supports strings, lists, sets, sorted sets, hashes, and streams. Sub-millisecond latency. Used for: caching model predictions, storing session data, rate limiting (count requests per user per minute), and real-time feature serving.
- For ML serving: cache predictions for repeated inputs. If many users ask “What is the capital of France?” , compute the answer once and serve the cached result. Cache hit rates of 20-40% are common for chatbot workloads, reducing GPU cost proportionally.

## 90.7 Databases

---

### 90.7.1 SQL (Relational)

- **SQL databases** (PostgreSQL, MySQL) store data in tables with rows and columns. Relations between tables are expressed via foreign keys. Queries use SQL. **ACID** guarantees:
  - **Atomicity:** a transaction either fully completes or fully rolls back. No partial updates.
  - **Consistency:** the database moves from one valid state to another. Constraints (unique keys, foreign keys) are always satisfied.
  - **Isolation:** concurrent transactions do not interfere with each other.
  - **Durability:** committed data survives crashes (written to disk before acknowledging).
- SQL databases excel at: structured data with relationships, complex queries (joins, aggregations), strict consistency requirements, and data integrity.

### 90.7.2 NoSQL

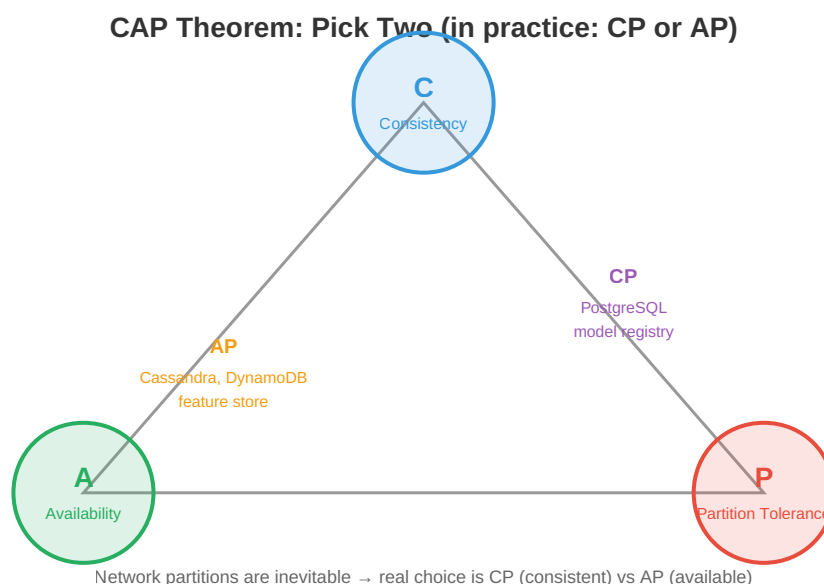
- **NoSQL databases** trade some ACID guarantees for scalability and flexibility:
  - **Key-value stores** (Redis, DynamoDB): simplest model. Fast lookups by key. Used for caching, session storage, and feature stores.
  - **Document stores** (MongoDB, Firestore): store JSON-like documents. Flexible schema (each document can have different fields). Used for user profiles, product catalogues, and configuration.



- **Column-family stores** (Cassandra, HBase): optimised for write-heavy workloads and time-series data. Used for event logging, metrics, and analytics.
- **Graph databases** (Neo4j): store nodes and edges. Optimised for traversal queries. Used for social networks, knowledge graphs, and recommendation systems.
- **Vector databases** (Pinecone, Milvus, Weaviate, FAISS): store high-dimensional embeddings and support approximate nearest neighbour (ANN) search. Essential for semantic search, RAG (retrieval-augmented generation), and recommendation systems.

### 90.7.3 CAP Theorem

- In a distributed database, you can have at most two of three properties:
  - **Consistency**: every read returns the most recent write.
  - **Availability**: every request receives a response (even if some nodes are down).
  - **Partition tolerance**: the system continues operating despite network partitions (nodes cannot communicate).



CAP theorem: choose CP (consistent) or AP (available) since network partitions are inevitable

- Since network partitions are inevitable in distributed systems, the real choice is **CP** (consistent but may be unavailable during partitions — e.g., PostgreSQL) vs **AP** (available but may return stale data during partitions — e.g., Cassandra, DynamoDB).

- For ML: feature stores typically choose AP (a slightly stale feature value is better than no prediction). Model registries choose CP (serving the wrong model version is catastrophic).

#### 90.7.4 Sharding

- **Sharding** splits a database across multiple machines. Each shard holds a subset of the data.
- **Hash sharding**: hash the key to determine the shard. `shard = hash(user_id) % num_shards`. Even distribution but makes range queries impossible.
- **Range sharding**: each shard holds a key range (users A-G on shard 1, H-N on shard 2). Enables range queries but can create hot spots (if many users have names starting with “S” ).
- **The resharding problem**: adding a shard invalidates the hash mapping. **Consistent hashing** minimises the data movement: only  $\sim 1/n$  of keys need to move when adding the  $n$ th shard.

#### 90.7.5 Database Indexing

- An **index** is a data structure that speeds up queries at the cost of extra storage and slower writes. Without an index, a query scans every row ( $O(n)$ ). With an index, it finds the target in  $O(\log n)$ .
- **B-tree index** (the default): a balanced tree (chapter 13, chapter 14) where each node contains multiple keys and pointers. B-trees are cache-friendly (wide nodes fit in cache lines) and support range queries (`WHERE age BETWEEN 20 AND 30`). Most SQL databases use B-trees.
- **Hash index**: maps keys to row locations using a hash function.  $O(1)$  lookup but does not support range queries. Used for exact-match lookups (`WHERE id = 12345`).
- **Composite index**: an index on multiple columns. `CREATE INDEX ON users(country, city)` speeds up queries filtered by country, or by country + city, but NOT by city alone (the leftmost column must be in the query).
- **The tradeoff**: every index speeds up reads but slows down writes (the index must be updated on every insert/update/delete) and uses storage ( $\sim 10\text{-}30\%$  of the table size per index). Do not index everything — index the columns you query frequently.

- **For ML systems:** the feature store's online database needs indexes on entity keys (user\_id, item\_id) for fast feature lookup. The experiment tracking database needs indexes on (experiment\_id, metric\_name) for dashboard queries.

### 90.7.6 API Design

- Systems communicate via APIs. Good API design makes the system usable, evolvable, and debuggable:
- **REST conventions:** use nouns for resources (/users, /models), HTTP methods for actions (GET = read, POST = create, PUT = update, DELETE = remove), and status codes for results (200 = OK, 201 = created, 400 = bad request, 404 = not found, 429 = rate limited, 500 = server error).
- **Pagination:** for endpoints that return lists, never return all results at once. Use cursor-based pagination (GET /items?cursor=abc&limit=50) or offset-based (GET /items?offset=100&limit=50). Cursor-based is more efficient for large datasets (offset-based requires skipping rows).
- **Versioning:** prefix API paths with a version (/v1/predict, /v2/predict). This lets you evolve the API without breaking existing clients. Clients migrate to v2 at their own pace; v1 is deprecated but not removed until traffic drops.
- **Error responses:** return structured errors with enough information to debug:

```
{
 "error": {
 "code": "INVALID_INPUT",
 "message": "Feature 'user_age' must be a positive integer",
 "details": {"field": "user_age", "value": -5}
 }
}
```

## 90.8 Message Queues

---

- **Message queues** decouple producers (services that generate work) from consumers (services that process it). The producer sends a message to the queue; the consumer pulls it when ready.

- **Why queues matter:** without a queue, if the consumer is slow or down, the producer is blocked. With a queue, the producer fires and forgets; the queue buffers messages until the consumer is ready.
- **Apache Kafka:** a distributed, persistent, high-throughput message queue. Messages are stored in **topics**, each partitioned across multiple brokers. Consumers read from partitions, tracking their position (**offset**). Kafka guarantees ordering within a partition and can replay messages (the log is persistent).
- **Pub/sub:** publishers send messages to a topic; all subscribers to that topic receive a copy. Used for event-driven architecture: “a new model was deployed” triggers the monitoring service, the A/B testing service, and the logging service simultaneously.
- For ML: a prediction request arrives via HTTP, is placed in a Kafka queue, processed by a GPU worker, and the result is returned via a callback or WebSocket. The queue buffers bursts of traffic and ensures no requests are lost if a GPU worker crashes.

## 90.9 Consistency Models

---

- In a distributed system, different nodes may have different views of the data. **Consistency models** define what guarantees the system provides:
- **Strong consistency:** after a write, all subsequent reads (from any node) see the new value. Simple to reason about but slow (requires coordination between nodes).
- **Eventual consistency:** after a write, reads may see stale data for some period, but will eventually see the new value. Fast (no coordination) but requires the application to handle stale reads.
- **Causal consistency:** if operation A causally precedes B (e.g., “write X then read X” ), the system guarantees B sees A’ s result. But unrelated operations may be seen in any order.
- **Read-your-writes:** a user always sees their own writes immediately, even if other users see stale data. The minimum consistency most applications need.

## 90.10 Resilience Patterns

---

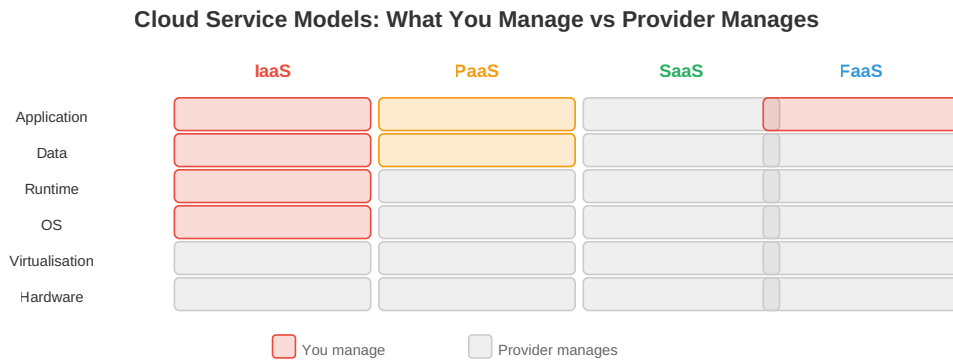
- **Rate limiting:** cap the number of requests per user per time window. Protects against abuse and ensures fair access. Implemented with a token bucket or sliding window counter in Redis.
- **Circuit breaker:** if a downstream service starts failing (error rate exceeds threshold), the circuit breaker “opens” and stops sending requests to it (returning a fallback response immediately). After a timeout, it “half-opens” and sends a test request. If the test succeeds, it closes (normal operation). This prevents cascading failures: if the feature store is down, the model server returns predictions without features rather than timing out on every request.
- **Backpressure:** when a system is overwhelmed, it signals upstream to slow down. Rather than accepting requests and failing, it rejects excess requests early (with a 429 or 503 status code). The client retries with exponential backoff.
- **Retry with exponential backoff:** if a request fails, wait 1 second and retry. If it fails again, wait 2 seconds. Then 4, 8, etc. Add jitter (random delay) to prevent all clients retrying simultaneously (the thundering herd problem).
- **Idempotency:** an operation is idempotent if doing it twice has the same effect as doing it once. `PUT /user/123 {"name": "Alice"}` is idempotent (setting the name twice to “Alice” is fine). `POST /payments` is not (paying twice is bad). Making operations idempotent ensures retries are safe.

## 91 Cloud Computing

Cloud computing provides on-demand infrastructure for ML workloads without owning hardware. This file covers service models, major cloud providers, containers and Kubernetes, storage, cloud networking, serverless computing, cost management, and infrastructure as code

- Training a frontier model requires thousands of GPUs for months. No startup owns that hardware. Cloud computing lets you rent it by the hour, scale up for training, scale down for inference, and pay only for what you use. Understanding cloud infrastructure is essential for anyone building ML systems beyond a laptop.

## 91.1 Cloud Service Models



Cloud service layers: IaaS gives you most control, SaaS gives you least

- Cloud services are layered by how much the provider manages:

Model	You Manage	Provider Manages	Example
<b>IaaS</b> (Infrastructure)	OS, runtime, app	Hardware, virtualisation, networking	AWS EC2, GCP Compute Engine
<b>PaaS</b> (Platform)	App, data	OS, runtime, scaling, patching	AWS SageMaker, GCP Vertex AI
<b>SaaS</b> (Software)	Nothing (just use it)	Everything	OpenAI API, Weights & Biases
<b>FaaS</b> (Function)	Individual functions	Everything else	AWS Lambda, GCP Cloud Functions

- **For ML:** most teams use a mix. IaaS for custom training (full control over GPU instances), PaaS for managed training and serving (SageMaker, Vertex AI handle the orchestration), and SaaS for tools (W&B for experiment tracking, OpenAI API for baseline comparisons).

## 91.2 Major Providers

---

### 91.2.1 AWS (Amazon Web Services)

- The largest cloud provider (~32% market share). Key ML services:
  - **EC2**: virtual machines. GPU instances: p4d (A100), p5 (H100), g5 (A10G for inference).
  - **S3**: object storage. The standard for storing datasets and model weights. Virtually unlimited capacity, ~\$0.023/GB/month.
  - **SageMaker**: managed ML platform. Handles training, hyperparameter tuning, deployment, and monitoring.
  - **EKS**: managed Kubernetes.
  - **Lambda**: serverless functions. Not suitable for GPU workloads but useful for preprocessing and orchestration.

### 91.2.2 GCP (Google Cloud Platform)

- Google's cloud (~11% market share). Key ML services:
  - **Compute Engine**: VMs. GPU instances with A100, H100. **TPU VMs** for TPU access.
  - **GCS**: object storage (like S3).
  - **Vertex AI**: managed ML platform. Native JAX/TPU support.
  - **GKE**: managed Kubernetes (the most mature K8s offering, since Google created Kubernetes).
  - **Cloud TPUs**: exclusive to GCP. v5e and v5p for large-scale training.

### 91.2.3 Azure (Microsoft)

- Microsoft's cloud (~23% market share). Key ML services:
  - **Azure VMs**: GPU instances with A100, H100.
  - **Azure Blob Storage**: object storage.
  - **Azure ML**: managed ML platform.
  - **AKS**: managed Kubernetes.
  - **OpenAI Service**: exclusive access to OpenAI models via Azure API.

## 91.3 Containers and Kubernetes

---

- We covered containers (Docker) and Kubernetes conceptually in chapter 13 (OS) and practically in chapter 15 (deployment). Here we focus on the **cloud-specific** patterns:

### 91.3.1 Kubernetes for ML

- **Kubernetes (K8s)** orchestrates containers at scale. Key concepts:
  - **Pod**: the smallest deployable unit. Contains one or more containers sharing networking and storage. A model serving pod might contain: the model server container + a sidecar container for metrics collection.
  - **Deployment**: manages a set of identical pods. Specifies the desired number of replicas. If a pod crashes, K8s automatically creates a replacement.
  - **Service**: a stable network endpoint for a set of pods. Clients connect to the service; K8s routes to a healthy pod. Types: ClusterIP (internal), NodePort (external via node port), LoadBalancer (external via cloud LB).
  - **StatefulSet**: like a Deployment but for stateful workloads. Each pod gets a persistent identity and stable storage. Used for databases and distributed training (each worker needs a stable identity for communication).
  - **DaemonSet**: runs one pod on every node. Used for: monitoring agents (Prometheus node exporter), log collectors (Fluentd), GPU device plugins (NVIDIA device plugin).
- **GPU scheduling in K8s**: the NVIDIA device plugin exposes GPUs as a K8s resource. Pods request GPUs:

```
resources:
 limits:
 nvidia.com/gpu: 2 # this pod needs 2 GPUs
```

- K8s schedules the pod onto a node with 2 available GPUs. This is how cloud ML platforms allocate GPUs for training and inference.

### 91.3.2 Autoscaling

- **Horizontal Pod Autoscaler (HPA)**: scales the number of pods based on metrics (CPU usage, request rate, custom metrics like GPU utilisation or queue depth).
- **Cluster Autoscaler**: scales the number of nodes. If pods cannot be scheduled because there are not enough nodes, the cluster autoscaler provisions new VMs from the cloud provider. When nodes are underutilised, it drains and terminates them.



- **KEDA** (Kubernetes Event-Driven Autoscaling): scales based on external event sources (Kafka queue depth, HTTP request rate). Perfect for inference: scale up model servers when the request queue grows, scale down when it is empty.

## 91.4 Storage

---

Type	Characteristics	Use Case	Example
<b>Block</b>	Low-latency, attached to one VM	OS disks, databases	AWS EBS, GCP Persistent Disk
<b>Object</b>	Unlimited capacity, HTTP access	Datasets, model weights, logs	AWS S3, GCS, Azure Blob
<b>File</b>	Shared across VMs, POSIX	Shared training data	AWS EFS, GCP Filestore, NFS
<b>Data lake</b>	Schema-on-read, raw data	Analytics, feature engineering	Delta Lake, Iceberg, Hudi

- **For ML training:** datasets are stored in object storage (S3/GCS). Training scripts read data from object storage into RAM. For fast random access (shuffled data loading), either: (1) download the dataset to local SSD before training, (2) use a high-throughput file system (Lustre, FSx), or (3) use a data loading library that streams and caches efficiently (WebDataset, FFCV).
- **Model weights:** stored in object storage with versioning. A 70B model in FP16 is ~140 GB. Loading from S3 at 1 GB/s takes ~2.5 minutes. Caching on local SSD reduces cold start time for inference.

## 91.5 Cloud Networking

---

- **VPC** (Virtual Private Cloud): an isolated network within the cloud. Your VMs, databases, and services communicate within the VPC. External traffic enters through load balancers or gateways.
- **Subnets:** divide the VPC into segments. Public subnets have internet access (for API servers). Private subnets do not (for databases, GPU workers). This is the network equivalent of the security principle of least privilege.

- **Security groups** (AWS) / **Firewall rules** (GCP): control which traffic is allowed. “Allow inbound HTTP on port 80 from anywhere. Allow inbound SSH on port 22 from my IP only. Block everything else.” Misconfigured security groups are the number one cause of cloud security incidents.
- **Service mesh** (Istio, Envoy): manages service-to-service communication within K8s. Provides: mTLS encryption (every service-to-service call is encrypted), traffic routing (A/B testing: route 10% of traffic to the new model), retries, timeouts, circuit breaking, and observability (which service called which, how long did it take).

## 91.6 Serverless

---

- **Serverless** (AWS Lambda, GCP Cloud Functions): you upload a function, and the cloud provider runs it when triggered. No servers to manage, no scaling to configure. You pay per invocation (typically \$0.20 per 1M invocations + compute time).
- **Cold starts**: the first invocation after a period of inactivity takes longer (the provider must allocate a container and load your code). Cold starts are 0.5-5 seconds, making serverless unsuitable for latency-sensitive ML inference.
- **For ML**: serverless is useful for: preprocessing (resize images before sending to the model), postprocessing (format model output, send notifications), orchestration (trigger a training pipeline when new data arrives), and lightweight inference (small models with tolerance for cold starts).
- Serverless is **not** suitable for: GPU inference (no GPU support in most serverless platforms), long-running training jobs (15-minute timeout in Lambda), or stateful services (no persistent state between invocations).

## 91.7 Cost Management

---

- Cloud costs are the number one operational concern for ML teams. A single H100 instance costs ~\$8/hour. A 64-GPU training run costs ~\$500/hour. A month-long training run costs ~\$360,000. Cost optimisation is engineering, not accounting.
- **Spot/preemptible instances**: unused cloud capacity sold at 60-90% discount. The provider can reclaim them with 30-second to 2-minute notice. Use for:

fault-tolerant training (checkpoint frequently, resume on new instances), batch inference, data preprocessing. Not for: latency-sensitive serving (interruption = downtime).

- **Reserved instances:** commit to 1-3 years of usage for 30-60% discount. Use for: steady-state inference serving where you know the baseline load.
- **Autoscaling:** scale up during peak hours, scale down at night/weekends. A model server that needs 10 GPUs at peak and 2 at night saves ~60% by autoscaling vs running 10 GPUs 24/7.
- **Right-sizing:** do not use an H100 for a 7B model that runs fine on an A10G. Match the GPU to the workload. Use profiling (chapter 16) to determine which GPU is the best fit.
- **Storage costs:** object storage is cheap (\$0.023/GB/month for S3 Standard), but accumulates. A team storing every training checkpoint (10 GB each, 100 per experiment, 50 experiments) accumulates 50 TB = \$1,150/month. Set lifecycle policies to delete old checkpoints automatically.

## 91.8 Multi-Region Deployment

---

- For global ML systems (serving users worldwide), deploying in a single region means high latency for distant users (a user in Tokyo hitting a US server adds ~150ms network round trip) and a single point of failure (if the region goes down, the entire service is offline).
- **Multi-region patterns:**
  - **Active-passive:** one primary region handles all traffic. A secondary region has a warm standby (replicated data, ready to receive traffic). On primary failure, DNS switches to the secondary. Downtime during failover: 30 seconds to several minutes.
  - **Active-active:** both regions handle traffic simultaneously. Users are routed to the nearest region. Both regions have up-to-date data (replicated asynchronously or synchronously). No downtime during a single-region failure — traffic is rerouted automatically.
- **Data replication:** the hard part. Model weights can be replicated easily (copy to S3 in each region). Feature store data must be replicated with acceptable

staleness. User data may have **data residency requirements** (GDPR: European user data must stay in Europe).

· **GPU cloud pricing comparison** (approximate, 2026):

GPU	AWS	GCP	Azure	Typical Use
A10G (24 GB)	\$1.00/hr (g5)	\$0.90/hr	\$0.90/hr	Small model inference
A100 (80 GB)	\$4.10/hr (p4d)	\$3.70/hr	\$3.40/hr	Training, large inference
H100 (80 GB)	\$8.00/hr (p5)	\$7.50/hr	\$7.00/hr	Frontier training
TPU v5e	n/a	\$1.20/hr	n/a	JAX training at scale

· Spot/preemptible pricing is typically 60-70% off these rates. Prices vary by region and availability.

## 91.9 Infrastructure as Code

- **IaC** defines infrastructure (VMs, networks, databases, K8s clusters) in version-controlled configuration files. Instead of clicking buttons in the AWS console, you write code that describes what you want, and a tool creates it.
- **Terraform** (HashiCorp): the standard IaC tool. Works with all major cloud providers. Declarative: you describe the desired state, Terraform figures out what to create/modify/delete to reach it.

```
main.tf – create a GPU VM for inference
resource "aws_instance" "model_server" {
 ami = "ami-0abcdef1234567890" # Deep Learning AMI
 instance_type = "g5.xlarge" # A10G GPU

 tags = {
 Name = "model-server-prod"
 }
}

resource "aws_s3_bucket" "model_weights" {
 bucket = "my-model-weights-prod"
```

```
versioning {
 enabled = true
}
}
```

```
terraform init # download provider plugins
terraform plan # show what will change
terraform apply # create the infrastructure
terraform destroy # tear it all down
```

- **Why IaC matters:** reproducibility (recreate the entire infrastructure from code), auditing (git history shows who changed what), disaster recovery (rebuild in a different region from the same config), and environment parity (dev, staging, and prod use the same templates with different parameters).
- **Pulumi:** like Terraform but uses real programming languages (Python, TypeScript, Go) instead of HCL. Useful when your infrastructure logic is complex (conditionals, loops, dynamic configuration).

## 92 Large Scale Infrastructure

Building systems that serve millions of users requires more than a single server. This file covers scalability patterns, distributed systems fundamentals, microservices, data pipelines, database scaling, search and vector systems, observability, reliability engineering, and CI/CD

- A model serving 1 request per second can run on a laptop. Serving 100,000 requests per second with 99.9% availability requires distributed systems, automated failover, and carefully designed data pipelines. This file covers the patterns that bridge that gap.

### 92.1 Scalability

---

- **Vertical scaling** (scale up): get a bigger machine. More CPU, more RAM, bigger GPU. Simple but has hard limits (the biggest available machine) and a single point of failure.

- **Horizontal scaling** (scale out): add more machines. Each handles a fraction of the traffic. No single-machine limit, but requires: load balancing (file 01), data partitioning, and handling distributed state.
- **Stateless services** are horizontally scalable by default. Add more instances behind a load balancer. A model inference server that loads weights at startup and processes requests independently is stateless — any instance can handle any request.
- **Stateful services** (databases, KV-cache, feature stores) are harder to scale. The state must be partitioned across machines (sharding, file 01) and replicated for fault tolerance.
- **The scalability equation:** for a system with  $n$  servers:
  - **Ideal:** throughput scales linearly ( $n$  servers  $\rightarrow n \times$  throughput).
  - **Actual:** overhead from coordination, load balancing, and data transfer means throughput scales sub-linearly. Amdahl's law (chapter 13) applies: the serial fraction (shared state, coordination) limits the speedup.

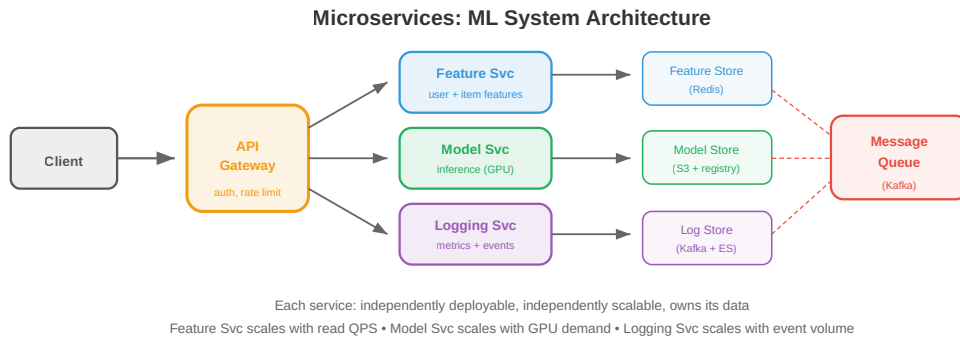
## 92.2 Distributed Systems

---

- A **distributed system** is a group of machines that coordinate to provide a service. The fundamental challenges:
- **Network partitions:** machines cannot always communicate. A network cable is cut, a switch fails, a data centre loses power. The system must handle partial failures.
- **Clock skew:** machines have different clocks. “Event A happened at 10:00:01 on machine 1” and “Event B happened at 10:00:01 on machine 2” does not mean they happened simultaneously. **Logical clocks** (Lamport timestamps, vector clocks) establish ordering without relying on physical clocks.
- **Consensus:** how do multiple machines agree on a value (e.g., who is the leader)? **Raft** is the standard consensus algorithm. A cluster of nodes elects a leader. The leader handles all writes. If the leader fails, the remaining nodes elect a new leader. Requires a majority (3 out of 5 nodes) to operate, so it tolerates  $\lfloor (n - 1)/2 \rfloor$  failures.
- **Distributed locks:** ensure only one machine performs a critical operation. **Redlock** (Redis-based) acquires locks across multiple Redis instances. If a

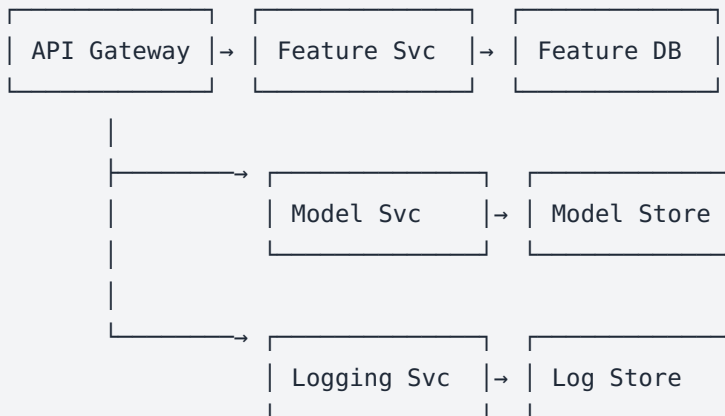
majority of instances grant the lock, it is acquired. Used for: preventing duplicate model deployments, ensuring only one training job writes to the checkpoint.

## 92.3 Microservices



Microservices ML architecture: API gateway routes to feature, model, and logging services, each with its own database, connected via message queue

- **Microservices** decompose a system into small, independently deployable services. Each service owns one domain:



- **Advantages:** independent deployment (update the model service without touching the feature service), independent scaling (scale model servers based on request load, feature servers based on feature store read rate), technology freedom (model service in Python, feature service in Go).
- **Disadvantages:** network overhead (every service call is a network round trip), complexity (debugging spans multiple services), data consistency (no cross-service transactions).

- **Service discovery:** how does the API gateway find the model service? Options: DNS-based (each service registers a DNS name), K8s services (built-in), or a service registry (Consul, Eureka).
- **Saga pattern:** for operations spanning multiple services (create user + allocate resources + send welcome email), use a saga: a sequence of local transactions with compensating actions if any step fails.

## 92.4 Data Pipelines

---

- ML systems consume enormous amounts of data. **Data pipelines** move, transform, and serve this data:

### 92.4.1 Batch Processing

- Process large volumes of data at regular intervals (hourly, daily).
- **MapReduce:** the original batch paradigm. Map (transform each record independently) → Shuffle (group by key) → Reduce (aggregate per group). Conceptually simple but verbose to implement.
- **Apache Spark:** the modern batch engine. In-memory processing (100x faster than MapReduce for iterative algorithms). Supports SQL, DataFrames, and ML pipelines. The standard for feature engineering at scale.
- **Example:** compute user features for a recommendation system. Input: 1B user activity events from the last 30 days. Output: 100M user feature vectors. Run daily as a Spark job, output to the feature store.

### 92.4.2 Stream Processing

- Process data in real-time as it arrives (sub-second latency).
- **Apache Flink:** the leading stream processing engine. Exactly-once processing, event-time handling (process events by when they happened, not when they arrived), windowing (tumbling, sliding, session windows).
- **Kafka Streams:** lightweight stream processing built into Kafka. Good for simpler transformations (filtering, aggregation) without deploying a separate cluster.



- **Example:** real-time fraud detection. Each credit card transaction is a Kafka event. A Flink job computes running statistics (transaction frequency, location changes) and flags anomalies within 100ms.

### 92.4.3 Lambda Architecture

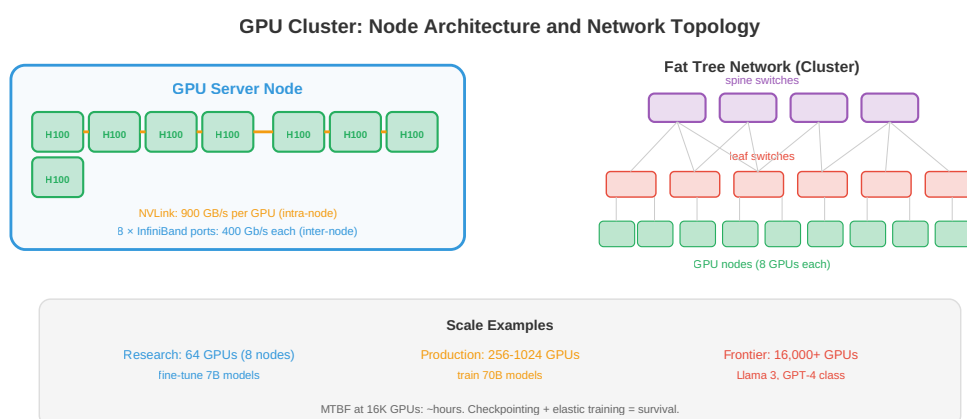
- Combine batch and stream processing. The **batch layer** provides accurate, comprehensive results (but with delay). The **speed layer** provides approximate, real-time results. A **serving layer** merges both.
- In practice, many teams now use the **Kappa architecture**: stream processing only, treating the stream as the source of truth. The stream is replayable (Kafka retains events), so batch can be simulated by replaying the stream.

## 92.5 ML Training Infrastructure

- Training a frontier model (100B+ parameters) is a large-scale infrastructure problem: thousands of GPUs running for months, consuming megawatts of power, generating petabytes of data, and costing tens of millions of dollars. The infrastructure determines whether training succeeds or fails.

### 92.5.1 GPU Clusters

- A training cluster is a collection of GPU servers connected by high-speed networks. Key components:



GPU cluster: each node has 8 GPUs connected by NVLink, nodes connected by InfiniBand in a fat tree topology, scaling from 64 to 16,000+ GPUs

- **GPU servers (nodes):** each server has 4-8 GPUs. A typical configuration:  $8 \times$  H100 GPUs,  $2 \times$  AMD EPYC CPUs, 2 TB RAM, 30 TB NVMe SSD. The GPUs within a node are connected by **NVLink** (900 GB/s per GPU on H100), which is 30x faster than PCIe.
- **Cluster sizes:** a small training cluster has 64-256 GPUs (8-32 nodes). A frontier model training cluster has 4,000-32,000 GPUs (500-4000 nodes). Meta's Llama 3 used 16,384 H100 GPUs. Google trains on TPU pods with 8,000+ chips.
- **Back-of-envelope:** training a 70B model requires ~\$2M in compute. Training a 400B+ frontier model requires ~\$50-100M. The cluster hardware itself costs ~\$500M-\$1B at H100 prices ( $\$30K \text{ per GPU} \times 16,000 \text{ GPUs} = \$480M$ ).

### 92.5.2 Networking Topology

- The network between GPU nodes is the most critical infrastructure component. If GPUs cannot exchange gradients fast enough, they sit idle waiting for communication to complete.
- **InfiniBand** is the standard for GPU cluster networking. NVIDIA's **Quantum-2 InfiniBand** provides 400 Gb/s per port. Each node typically has 8 InfiniBand ports (one per GPU), giving 400 GB/s total bisection bandwidth per node.
- **RDMA** (Remote Direct Memory Access): InfiniBand supports RDMA, which transfers data directly between GPU memory on different nodes without involving the CPU. This reduces latency from  $\sim 100\mu\text{s}$  (TCP) to  $\sim 1\mu\text{s}$  and is essential for efficient gradient all-reduce (chapter 6).
- **Network topology matters:** a **fat tree** (Clos network) provides full bisection bandwidth (any GPU can communicate with any other at full speed). Cheaper topologies (**rail-optimised**, **3D torus**) provide less bandwidth but cost less. The topology must match the parallelism strategy:
  - **Data parallelism:** all-reduce across all GPUs  $\rightarrow$  needs high bisection bandwidth (fat tree).
  - **Tensor parallelism:** communication within a node  $\rightarrow$  NVLink handles this (no network needed).

- **Pipeline parallelism:** communication between adjacent pipeline stages → needs bandwidth only between specific node pairs (rail-optimised is fine).
- **Ethernet alternatives:** **RoCE v2** (RDMA over Converged Ethernet) provides RDMA over standard Ethernet infrastructure. Cheaper than InfiniBand but with higher latency and more congestion. Google uses RoCE for some TPU pod networks. Ultra Ethernet Consortium is developing lossless Ethernet for AI workloads.

### 92.5.3 Storage for Training

- Training requires three storage tiers:
  - **Dataset storage:** the training corpus (1-100 TB of text, or petabytes of multimodal data). Stored in distributed file systems or object storage. Must support high-throughput sequential reads (the data loader reads data in large batches). **Lustre** and **GPFS** are common HPC file systems; cloud alternatives include **FSx for Lustre** (AWS) and **Filestore** (GCP).
  - **Checkpoint storage:** training state (model weights + optimiser state + scheduler state) saved periodically. For a 70B model in mixed precision with Adam optimiser:  $\sim 560$  GB per checkpoint ( $70\text{B} \times 4 \text{ bytes} \times 2$  for optimiser). Saving hourly for a 3-month run =  $\sim 2000$  checkpoints = 1.1 PB. In practice, only the latest N checkpoints are kept, and old ones are deleted. Must be fast enough that checkpointing does not significantly slow training.
  - **Logging and metrics:** experiment tracking data (loss curves, learning rate schedules, gradient norms). Relatively small but must be written in real-time. W&B, MLflow, or TensorBoard handle this.
- **The storage bottleneck:** a 16,000-GPU cluster loading a training batch needs to read  $\sim 100$  GB/s of data continuously. If the file system cannot sustain this throughput, GPUs sit idle waiting for data. Data pipeline optimisation (prefetching, caching, format optimisation using WebDataset or Mosaic Streaming) is critical.

### 92.5.4 Job Scheduling

- A GPU cluster serves multiple teams and projects. A **job scheduler** allocates GPUs to training jobs:

- **SLURM**: the standard HPC job scheduler. Users submit jobs specifying GPU count, memory, and time limit. SLURM allocates resources and manages the queue. Supports priority-based scheduling, preemption, and fair-share allocation between teams.
- **Kubernetes with GPU scheduling** (chapter 18 file 02): cloud-native approach. K8s GPU device plugins expose GPUs as schedulable resources. **Volcano** and **Run:ai** add ML-specific scheduling features: gang scheduling (allocate all GPUs for a job at once, not one at a time), priority queues, and GPU time-sharing.
- **Scheduling challenges**:
  - **Fragmentation**: a cluster with 1000 GPUs might have 200 free, but spread across 50 nodes (4 free per node). A job needing 128 contiguous GPUs cannot run, even though there are enough total GPUs. **Defragmentation** (migrating jobs to consolidate free GPUs) or **topology-aware scheduling** (allocate GPUs that are well-connected) address this.
  - **Priority and preemption**: urgent experiments should preempt lower-priority jobs. But preempting a training job that has been running for 2 days wastes the compute. The scheduler must balance priority with efficiency.
  - **Fair share**: teams should get their allocated fraction of compute over time, even if a team submits more jobs than its share.

### 92.5.5 Fault Tolerance

- At the scale of thousands of GPUs running for months, hardware failures are not exceptional — they are routine. Mean time between failures for a 16,000-GPU cluster is measured in hours, not months.
- **Common failures**: GPU memory errors (ECC-correctable and uncorrectable), NVLink failures (GPU-to-GPU communication within a node), InfiniBand link failures (node-to-node communication), node crashes (kernel panic, PSU failure), and storage failures (disk or controller failure).
- **Checkpointing** is the primary defence. Save the full training state (model, optimiser, data loader position) every N steps. On failure: identify the failed node, replace or remove it, restart training from the latest checkpoint. The cost of a failure is the compute between the last checkpoint and the failure.
- **Checkpoint frequency tradeoff**: frequent checkpoints (every 10 minutes) waste less compute on failure but slow down training (checkpointing 560 GB takes

time). Infrequent checkpoints (every 2 hours) are faster but waste up to 2 hours of compute on failure. Most teams checkpoint every 20-60 minutes.

- **Elastic training:** modern frameworks (PyTorch Elastic, DeepSpeed) support resizing the training run without restarting. If 2 nodes fail out of 500, training continues with 498 nodes. The failed nodes are replaced, and training automatically incorporates them when they come back online.
- **Health monitoring:** continuous monitoring of all GPUs (temperature, memory errors, compute throughput), network links (packet loss, latency), and storage (throughput, error rates). Automatic alerting on anomalies. Some clusters run periodic GPU health checks (a short computation test) to proactively identify degrading hardware before it fails.
- **At scale:** training Meta’ s Llama 3 (16,384 H100s, 54 days) experienced ~466 job interruptions. Effective training time was only ~90% of wall-clock time — 10% was lost to failures and recovery. The infrastructure to achieve 90% (rather than 50% or 70%) is what separates organisations that can train frontier models from those that cannot.

### 92.5.6 Cost and Efficiency

- Training infrastructure cost is dominated by GPU hours:

Component	% of Total Cost
GPU compute	70-80%
Networking (InfiniBand)	10-15%
Storage	5-10%
Cooling and power	5-10%

- **GPU utilisation** (Model FLOPs Utilisation, MFU) measures what fraction of the GPU’ s theoretical peak performance is actually used for useful computation. Peak H100 is 989 TFLOPS (FP8). Achieving 40-50% MFU is good; 50-60% is excellent. The gap is due to: communication overhead (all-reduce, pipeline bubbles), memory bandwidth limitations, and idle time during checkpointing and data loading.
- **Improving MFU:** overlap computation and communication (chapter 6), use efficient attention (Flash Attention, chapter 16), optimise data loading (prevent

GPU starvation), reduce checkpoint overhead (async checkpointing, checkpoint to fast NVMe first, then background-copy to persistent storage).

- **Build vs buy:** at small scale (<256 GPUs), cloud is cheaper (no upfront cost, pay per hour). At large scale (>1000 GPUs, sustained use for 6+ months), owning hardware is cheaper (~2-3x lower TCO over 3 years). Most AI companies use a mix: owned clusters for sustained training, cloud for burst capacity and experimentation.

## 92.6 Database Scaling

---

- **Read replicas:** route read queries to replicas of the primary database. The primary handles writes, replicas handle reads. Since most workloads are read-heavy (95%+ reads), this scales read throughput linearly with the number of replicas.
- **Partitioning** (sharding, from file 01): split data across multiple databases. Each partition is independent, enabling parallel reads and writes. The challenge is cross-partition queries (join data from different shards).
- **Connection pooling:** databases have limited connection capacity. A connection pool (PgBouncer for PostgreSQL) reuses connections across requests, preventing connection exhaustion when hundreds of service instances each try to connect.

## 92.7 Search and Vector Systems

---

### 92.7.1 Text Search

- **Inverted index:** the foundation of text search. For each word, store a list of documents containing that word. A query intersects the lists for each query word. **Elasticsearch** is the standard: distributed, real-time, supports full-text search, aggregations, and geospatial queries.
- **BM25:** the standard text retrieval scoring function. Scores documents by term frequency, inverse document frequency, and document length normalisation. Simple but effective — still competitive with neural methods for keyword-heavy queries.

## 92.7.2 Vector Search

- **Vector databases** store embeddings (high-dimensional vectors) and support fast **approximate nearest neighbour (ANN)** search. Given a query embedding, find the k most similar stored embeddings.
- **FAISS** (Facebook AI Similarity Search): a library (not a database) for ANN search. Supports multiple index types:
  - **Flat**: exact search,  $O(n)$ . Used for small datasets or as ground truth.
  - **IVF** (Inverted File): partition vectors into clusters, search only the nearest clusters.  $O(n/k)$  per query.
  - **HNSW** (Hierarchical Navigable Small World): graph-based. Build a hierarchical graph, navigate from coarse to fine. Extremely fast and accurate, the default choice for most applications.
  - **Product Quantisation (PQ)**: compress vectors into compact codes for memory-efficient search. Trade accuracy for memory.
- **Managed vector databases**: Pinecone, Weaviate, Milvus, Qdrant. They handle scaling, replication, and real-time updates that FAISS does not.
- **For RAG** (Retrieval-Augmented Generation): user query → embed with a text encoder → search the vector database for relevant documents → prepend retrieved documents to the LLM prompt. The quality of the retrieval directly determines the quality of the LLM's response.

## 92.8 Observability

---

- **Observability** is the ability to understand what is happening inside your system from its external outputs. Three pillars:

### 92.8.1 Logging

- **Structured logs** (JSON) are searchable and parseable. Unstructured logs ( “ERROR: something failed” ) are not. Always log: timestamp, service name, request ID (for tracing across services), severity, and the relevant context.
- **ELK stack** (Elasticsearch, Logstash, Kibana): the standard logging pipeline. Logstash collects and transforms logs, Elasticsearch indexes them, Kibana visualises and searches.



## 92.8.2 Metrics

- **Metrics** are numerical measurements over time: request rate, error rate, latency percentiles, GPU utilisation, queue depth. **Prometheus** scrapes metrics from services; **Grafana** visualises them in dashboards with alerting.
- **The RED method** for services: **Rate** (requests/second), **Errors** (error rate), **Duration** (latency). Monitor these for every service.
- **The USE method** for resources: **Utilisation** (% in use), **Saturation** (queue depth), **Errors**. Monitor these for every resource (CPU, GPU, memory, disk, network).

## 92.8.3 Tracing

- **Distributed tracing** follows a single request across multiple services. A user request hits the API gateway → feature service → model service → postprocessing. A **trace** records the timing of each hop, showing where latency is spent.
- **OpenTelemetry**: the open standard for traces, metrics, and logs. Instrument your code once, export to any backend (Jaeger, Zipkin, Datadog).

## 92.9 Reliability

---

- **SLO** (Service Level Objective): the target reliability. “99.9% of requests complete in <200ms.” This gives a concrete error budget: 0.1% of requests (about 43 minutes per month) can be slow or fail.
- **SLI** (Service Level Indicator): the measurement. “The 99th percentile latency over the last 5 minutes.”
- **SLA** (Service Level Agreement): the contractual promise with consequences. “If availability drops below 99.95%, the customer gets a credit.”
- **Error budgets**: if your SLO is 99.9% and you are at 99.99%, you have budget for risky changes (deploy a new model, migrate databases). If you are at 99.85%, freeze all changes and focus on reliability. Error budgets turn reliability from an abstract goal into a measurable resource.
- **Chaos engineering**: deliberately inject failures (kill a server, add network latency, corrupt data) to test whether your system handles them correctly. Netflix’ s Chaos Monkey randomly terminates production instances. If the



system stays up, it is resilient. If it falls over, you found a bug before your users did.

## 92.10 CI/CD

---

- **Continuous Integration:** automatically build and test every code change. Every push triggers: lint, type check, unit tests, integration tests. If any fail, the change is rejected. This catches bugs before they reach production.
- **Continuous Deployment:** automatically deploy changes that pass CI.  
Deployment strategies:
  - **Blue-green:** run two identical environments (blue = current, green = new). Switch traffic from blue to green instantly. If green fails, switch back to blue (instant rollback).
  - **Canary:** route a small fraction of traffic (1-5%) to the new version. Monitor for errors. Gradually increase traffic if metrics look good. This limits the blast radius of a bad deployment.
  - **Feature flags:** deploy new code but hide it behind a flag. Enable the flag for a subset of users (internal testers, then beta users, then everyone). Decouple deployment (code is live) from release (users see the feature).
- For ML: CI/CD includes model-specific steps. A model change triggers: unit tests (shape tests, gradient checks), evaluation on a held-out set (accuracy must not regress), shadow deployment (run the new model alongside the old one, compare outputs), and gradual rollout (canary from 1% → 100%).

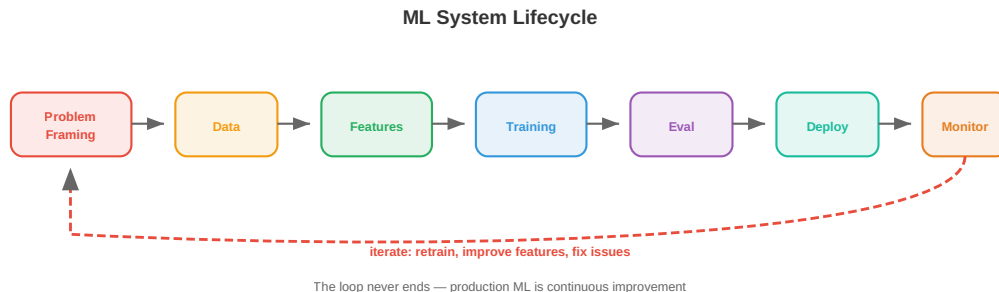
## 93 ML Systems Design

ML systems design applies the infrastructure patterns from files 01-03 to the specific challenges of machine learning. This file covers the ML lifecycle, data management, training infrastructure, model evaluation, serving strategies, feature engineering, ML pipelines, and monitoring.

- A systems design interview question like “Design a recommendation system for YouTube” is not asking you to describe the recommendation algorithm. It is asking you to design the **entire system**: data pipelines, feature engineering,

model training, evaluation, serving, monitoring, and iteration. This file provides the framework.

## 93.1 The ML System Lifecycle



ML system lifecycle: problem framing through deployment and monitoring, with continuous iteration

- Every ML system follows the same lifecycle, whether it is a spam classifier or a foundation model:

Problem Framing → Data → Features → Training → Evaluation → Deployment → Monitoring → Iteration

↑  
|

### 93.1.1 Problem Framing

- Before touching data or models, define:
  - **What** are you predicting? (click probability, next token, object bounding box)
  - **Who** are the users? (end users, internal analysts, other ML models)
  - **What are the constraints?** (latency < 100ms, offline batch is fine, must run on-device)
  - **What is the business metric?** (revenue, engagement, accuracy) and how does the ML metric relate to it?
  - **What is the baseline?** (heuristic, rule-based system, existing model) — you must beat this to justify the ML system.
- **Common mistake:** jumping to model architecture before understanding the problem. “We should use a transformer” is not a system design answer. “We

need to predict click probability for 10M candidates within 200ms, so we need a two-stage system: fast retrieval then a small ranking model” is.

## 93.2 Data Management

---

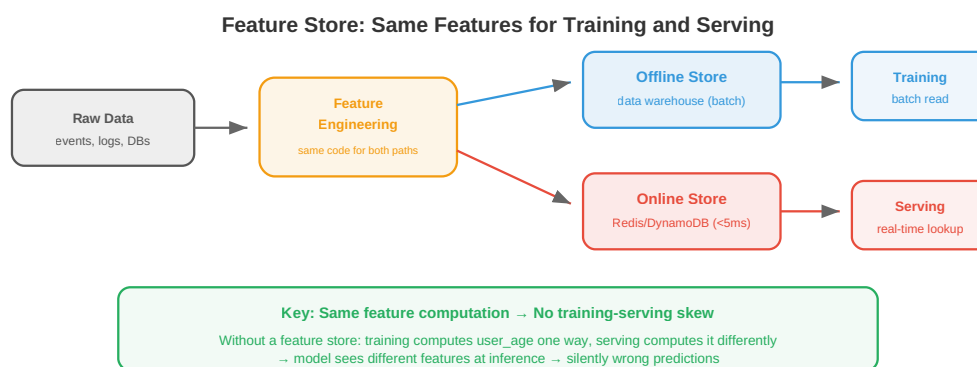
### 93.2.1 Data Collection and Labelling

- **Explicit labels:** humans annotate data (click/no-click, object bounding boxes, conversation quality ratings). Expensive (~\$0.02-\$10 per label depending on complexity), slow, and subjective.
- **Implicit labels:** derive labels from user behaviour. Clicks, dwell time, purchases, skips. Cheap and abundant but noisy (a click does not mean satisfaction; a skip does not mean dislike).
- **Programmatic labelling** (Snorkel): write labelling functions (heuristics, regex, existing models) that vote on each example. Aggregate votes statistically to produce probabilistic labels. Scales to millions of examples with moderate accuracy.
- **Active learning:** the model identifies the examples it is most uncertain about and requests human labels for those. This maximises label efficiency: 1000 actively-selected labels can match 10,000 random labels.

### 93.2.2 Data Quality

- **Data validation:** check every batch of incoming data for schema violations (missing fields, wrong types), distribution shifts (average values changed significantly), and volume anomalies (expected 1M rows, got 500K).
- **Great Expectations** and **TFX Data Validation** are tools that define expectations about data and alert when they are violated.
- **Data versioning:** every training run should be reproducible. **DVC** (chapter 15) tracks data files alongside code. Each dataset version gets a hash; the training config references the hash.

### 93.2.3 Feature Stores



Feature store: same feature computation feeds both the offline store (for training) and online store (for serving), preventing training-serving skew

- **Feature stores** (chapter 15) provide consistent features for training and serving. Key concepts:
  - **Offline features:** computed from batch pipelines (Spark), stored in data warehouses. Used during training and for batch inference. Examples: user' s average session length over 30 days, item' s total purchase count.
  - **Online features:** computed in real-time or precomputed and served from a low-latency store (Redis, DynamoDB). Used during real-time inference. Examples: user' s last 5 actions, current cart contents.
  - **Training-serving skew:** if the feature computation differs between training and serving, the model sees different feature values at inference than it was trained on. Feature stores eliminate this by using the same computation for both.

## 93.3 Training Infrastructure

- For this book' s audience, distributed training was covered in depth in chapter 6 (data parallelism, model parallelism, mixed precision, scaling laws). Here we focus on the **systems** aspects:
- **Experiment tracking** (W&B, MLflow — chapter 15): every training run logs hyperparameters, metrics, git commit, data version, and hardware. This is the ML equivalent of version control for models.
- **Hyperparameter tuning:** automated search over hyperparameters. Methods: grid search (exhaustive, expensive), random search (surprisingly effective),

Bayesian optimisation (model the objective, sample where improvement is likely), and **ASHA** (Asynchronous Successive Halving: start many trials, kill underperforming ones early).

- **Training pipeline orchestration** (Airflow, Kubeflow — chapter 15): automate the sequence of data prep → training → evaluation → registration. Schedule daily retraining. Alert on failures.

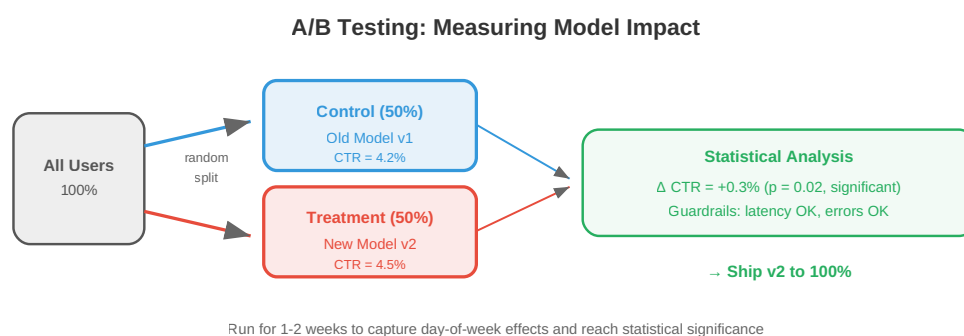
## 93.4 Model Evaluation

---

### 93.4.1 Offline Evaluation

- **Held-out test set**: evaluate on data the model never saw during training. Standard but can be misleading if the test set does not represent production data.
- **Slice-based evaluation**: evaluate on subgroups (by user demographics, content type, language, time period). A model with 95% overall accuracy might have 70% accuracy for a specific minority group — unacceptable.
- **Backtesting**: for time-series or sequential prediction, evaluate on historical data in time order. Train on data up to time  $t$ , evaluate on data from  $t$  to  $t + \Delta t$ . Avoids the leakage of using future data for training.

### 93.4.2 Online Evaluation



A/B testing: randomly split users into control (old model) and treatment (new model), compare metrics with statistical significance

- **A/B testing:** randomly split live traffic into control (old model) and treatment (new model). Compare business metrics (revenue, engagement, retention) with statistical significance. The gold standard for evaluating ML changes.
  - **Sample size:** you need enough data to detect the expected effect size. A 0.1% improvement in click-through rate requires millions of impressions to detect with significance.
  - **Duration:** run for at least one full cycle (1-2 weeks for most products) to capture day-of-week effects.
  - **Guardrail metrics:** monitor metrics that should NOT change (page load time, error rate, crash rate) alongside the target metric. A model that increases revenue but also increases crashes is a net negative.
- **Shadow deployment:** run the new model alongside the old one in production. Both receive the same requests, but only the old model's predictions are served to users. Compare the outputs. This catches bugs and quality issues without risk to users.
- **Interleaving:** for ranking problems, interleave results from the old and new model in a single list. Users interact with the interleaved list, and you measure which model's results get more engagement. Requires fewer users than A/B testing to reach significance.

## 93.5 Model Serving

---

### 93.5.1 Batch vs Real-Time

- **Batch inference:** precompute predictions for all possible inputs. Store in a database/cache. Serve from the cache. Works when: the input space is finite (recommend for all users nightly), freshness is not critical (daily predictions are fine), and latency tolerance is high.
- **Real-time inference:** compute predictions on demand for each request. Works when: the input space is infinite (any user query), freshness matters (predict for this specific query right now), and latency must be low.

- Many systems use **both**: batch precomputes a set of candidates (cheap, covers 80% of traffic), real-time handles the rest (expensive, covers tail queries and new users).

### 93.5.2 Model Versioning and Registry

- A **model registry** (MLflow, W&B, SageMaker) stores trained models with metadata:
  - Version number and training date.
  - Training config and data version.
  - Evaluation metrics (accuracy, latency, memory usage).
  - Stage: development → staging → production → archived.
- **Rollback**: if a new model degrades metrics in production, revert to the previous version immediately. The registry makes this a one-click operation.

## 93.6 Feature Engineering

---

- **Feature engineering** transforms raw data into the inputs the model needs. It is often the highest-leverage activity in ML: better features improve every model, while better models are limited by the features they receive.

### 93.6.1 Online vs Offline Features

- **Offline features** are precomputed and change slowly (user demographics, 30-day aggregates). Computed by batch pipelines (Spark), stored in the feature store.
- **Online features** reflect the current state and change rapidly (items in cart, last action, current location). Computed in real-time from event streams or looked up from a fast store.
- **Feature freshness**: some features need to be seconds-fresh (fraud detection: is this transaction anomalous given the last 5 transactions?). Others can be hours-stale (recommendations: what genres does this user prefer based on their history?). Fresher features are more expensive to compute and serve.

### 93.6.2 Common Feature Patterns

- **Counting features**: count of events in a time window (purchases in last 7 days, logins in last 24 hours).

- **Embedding features:** learned embeddings for categorical variables (user embedding, item embedding, query embedding). These are the inputs to two-tower models and similar architectures.
- **Cross features:** combinations of two or more features (user\_age × item\_category). Capture interactions that individual features miss.
- **Temporal features:** time since last action, day of week, hour of day. Capture temporal patterns.
- **Aggregation features:** mean, median, min, max, std of a numerical feature over a group (average rating of items by this seller).

## 93.7 ML Pipelines

---

- An ML pipeline orchestrates the entire workflow from data to deployed model:

Data ingestion → Validation → Feature engineering → Training → Evaluation →  
Registration → Deployment → Monitoring

- Each step is a task in an orchestrator (Airflow, Kubeflow, Metaflow — chapter 15). The pipeline:
  - Runs on a schedule (daily retraining) or on trigger (new data available).
  - Is idempotent (rerunning produces the same result).
  - Has retry logic (if feature computation fails, retry 3 times with backoff).
  - Produces artifacts (trained model, evaluation report, feature statistics) that are versioned and stored.
- **Metaflow** (Netflix/Outerbounds) is particularly well-suited for ML: it versions code, data, and models together, supports local development and cloud execution with the same code, and integrates with K8s and AWS.

## 93.8 Monitoring

---

- We covered monitoring fundamentals in chapter 15 (Prometheus, Grafana, alerts). Here we focus on **ML-specific monitoring**:

### 93.8.1 Data Drift

- **Data drift** occurs when the distribution of incoming data changes relative to the training data. A model trained on summer data may perform poorly on winter data (different user behaviour, different product availability).



- **Detection:** compare the distribution of incoming features to the training distribution using statistical tests:
  - **KS test** (Kolmogorov-Smirnov): compares two empirical distributions. Tests whether they come from the same underlying distribution.
  - **PSI** (Population Stability Index): measures how much a distribution has shifted.  $PSI < 0.1$  is stable,  $0.1-0.25$  is moderate shift,  $> 0.25$  is significant.
  - **Embedding drift:** compare the embedding distribution of incoming queries to the training set using centroid distance or MMD (Maximum Mean Discrepancy).

### 93.8.2 Concept Drift

- **Concept drift** occurs when the relationship between inputs and outputs changes. The features look the same, but the correct prediction is different. Example: user preferences shift after a cultural event, pandemic, or product change.
- Concept drift is harder to detect than data drift because it requires labelled data. Monitor proxy metrics: click-through rate, conversion rate, user satisfaction scores. A sustained decline suggests concept drift.

### 93.8.3 Model Degradation

- Models degrade over time for multiple reasons: data drift, concept drift, feature pipeline bugs (a feature starts returning null), and upstream data changes (a third-party API changes its response format).
- **Response:** when degradation is detected, the action depends on severity:
  - Mild: retrain on recent data (scheduled retraining handles this).
  - Moderate: investigate the root cause (which feature changed? which user segment is affected?).
  - Severe: roll back to a previous model version immediately, then investigate.

### 93.8.4 Feedback Loops

- ML systems create **feedback loops**: the model's predictions influence user behaviour, which becomes the training data for the next model version. These loops can be virtuous or vicious.

- **Positive feedback loop** (dangerous): a recommendation model shows mostly popular items → users click on popular items (because that is all they see) → the model learns that popular items are even more popular → diversity collapses. The model creates the data that confirms its biases.
- **Negative feedback loop** (also dangerous): a fraud detection model catches all fraud of type A → no type-A fraud reaches the training data → the next model does not learn to detect type A → type-A fraud resurfaces.
- **Mitigations:**
  - **Exploration:** show some items the model is uncertain about (epsilon-greedy, Thompson sampling). This generates diverse training data.
  - **Counterfactual logging:** record what the model would have predicted, not just what the user saw. Train on counterfactual data to debias.
  - **Holdout sets:** randomly serve a fraction of traffic without model filtering. The unfiltered data provides ground truth for evaluating model quality.
  - **Delayed labels:** wait for the true outcome before using the data for training. A recommendation clicked today may be regretted tomorrow. A fraud prediction must wait for the chargeback window (30-90 days).

### 93.8.5 Embedding Table Management

- Large-scale ML systems often have embedding tables with 100M+ entries (one embedding per user, item, ad, or entity). Managing these at scale is a systems challenge:
- **Storage:**  $100\text{M entities} \times 256\text{-dim} \times \text{float16} = 50\text{ GB}$ . Does not fit in GPU memory. Solutions: store in CPU memory with GPU-side caching, shard across multiple machines, or use **hash embeddings** (hash entities to a fixed-size table, accepting collisions).
- **Updates:** embeddings change as the model retrains. Deploying a new embedding table to serving requires: loading 50 GB into memory without disrupting live traffic, verifying correctness, and rolling back if metrics degrade. Use blue-green deployment for embedding tables.
- **Staleness:** a newly created user has no embedding (the cold start problem). Solutions: use a default embedding, derive an embedding from the user's features via a feature-to-embedding model, or fall back to a non-personalised model.

### 93.8.6 Fairness and Bias

- ML systems can systematically treat different groups differently, often reflecting biases in training data. **Fairness monitoring** is a responsibility, not an optional feature.
- **Metrics to monitor:**
  - **Demographic parity:** does the positive prediction rate differ across groups (gender, ethnicity, age)?
  - **Equal opportunity:** does the true positive rate differ across groups? (A hiring model should be equally good at identifying qualified candidates from all groups.)
  - **Calibration:** if the model says  $P(\text{qualified}) = 0.7$  for group A, does 70% of group A actually qualify? And the same for group B?
- **Practical steps:**
  - Evaluate model performance on slices (subgroups) not just overall metrics.
  - Include fairness metrics in the model evaluation pipeline (a model that improves overall accuracy but degrades a specific group should not be deployed without review).
  - Document known limitations and failure modes.
  - Establish a review process for models deployed in sensitive domains (hiring, lending, criminal justice, healthcare).

## 94 ML Design Examples

The best way to learn ML systems design is through worked examples. This file walks through seven complete designs: recommendation systems, search ranking, ads click prediction, fraud detection, content moderation, conversational AI, and large-scale image search.

- Each example follows a consistent framework:
  1. **Problem framing:** what are we building, who are the users, what are the constraints?
  2. **Data:** what data do we have, how is it collected, how is it labelled?
  3. **Features:** what features does the model need?
  4. **Model:** what architecture and training approach?

5. **Serving:** how is the model deployed and served?
  6. **Evaluation:** how do we measure success?
  7. **Iteration:** what improvements would we make over time?
- 

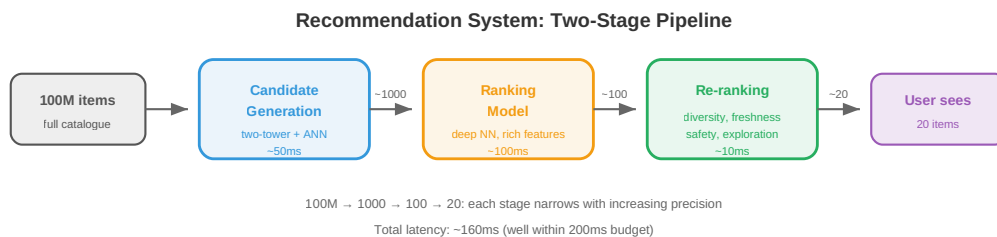
## 94.1 1. Recommendation System (e.g., YouTube, Netflix, Spotify)

---

### 94.1.1 Problem Framing

- **Goal:** show users content they will enjoy, maximising engagement (watch time, listens, clicks).
- **Scale:** 1B+ users, 100M+ items, 10K+ recommendations per second.
- **Latency:** <200ms for the full recommendation pipeline.
- **Key challenge:** the candidate space is enormous (100M items). Cannot score all items for all users in real-time.

### 94.1.2 Architecture: Two-Stage Pipeline



Recommendation pipeline: 100M items narrowed to 1000 by candidate generation, ranked to 100, re-ranked to 20 shown items

```
100M items → Candidate Generation (fast, coarse) → 1000 candidates
 → Ranking (slow, precise) → 100 ranked items
 → Re-ranking (business rules) → 20 shown to user
```

### 94.1.3 Candidate Generation

- **Goal:** reduce 100M items to ~1000 candidates. Must be fast (<50ms).
- **Two-tower model:** encode users and items into the same embedding space. The user embedding captures preferences; the item embedding captures content characteristics. Score = dot product of user and item embeddings.

- **Training:** contrastive learning on (user, positive\_item, negative\_items) triples. Positives = items the user engaged with. Negatives = random items + hard negatives (popular items the user did not engage with).
- **Serving:** precompute all item embeddings. At request time: compute the user embedding, ANN search (HNSW in a vector database) to find the 1000 nearest item embeddings.

#### 94.1.4 Ranking

- **Goal:** precisely score the 1000 candidates. Can afford ~100ms.
- **Model:** a deep neural network (MLP or transformer) that takes rich features: user features (demographics, history, context), item features (content, popularity, freshness), and cross features (user-item interaction history, contextual relevance).
- **Output:** predicted probability of engagement (click, watch 50%+, like, share). Multiple objectives can be combined:  $\text{score} = w_1 \cdot P(\text{click}) + w_2 \cdot P(\text{watch}) + w_3 \cdot P(\text{like})$ .

#### 94.1.5 Re-ranking

- Apply business rules: diversity (do not show 5 videos from the same creator), freshness (boost new content), safety (filter flagged content), and personalised exploration (show some lower-ranked items the user might discover).

#### 94.1.6 Back-of-Envelope Numbers

- **Item embedding index:**  $100\text{M items} \times 256\text{-dim} \times \text{float16} = 50\text{ GB}$ . HNSW index adds ~2x overhead  $\rightarrow \sim 100\text{ GB}$ . Fits on a single machine with 128 GB RAM, or shard across  $4 \times 32\text{ GB machines}$ .
- **User embedding computation:** ~5ms per user (small MLP on user features). At 10K QPS, need ~50 model replicas to handle the load.
- **ANN search:** ~2ms for top-1000 from 100M vectors with HNSW. At 10K QPS, each index replica handles ~500 QPS  $\rightarrow$  need 20 replicas.
- **Ranking model:**  $1000\text{ candidates} \times \sim 0.1\text{ms per candidate} = 100\text{ms per request}$ . At 10K QPS, need 1000 GPU-seconds per second  $\rightarrow \sim 10\text{ A10G GPUs}$  for ranking alone.

- **Total infrastructure:** ~20 embedding index replicas + ~50 user embedding servers + ~10 ranking GPUs + caching + load balancers. Cost: ~\$50K-\$100K/month at cloud prices.

### 94.1.7 Cold Start

- **New users** (no history): use demographic features, device/location context, and popularity-based recommendations. After 5-10 interactions, switch to the personalised model.
- **New items** (no engagement data): use content-based features (title, description, thumbnail embeddings). Allocate an exploration budget: show new items to a fraction of users to gather engagement data quickly. Items with no engagement after a boost period are demoted.
- **Cold start is a systems problem:** the feature store must handle missing features gracefully (return defaults, not errors). The model must be trained with missing features (dropout on user history features during training simulates new users).

### 94.1.8 Evaluation

- **Offline:** NDCG (Normalized Discounted Cumulative Gain), recall@K, precision@K on a held-out set.
  - **Online:** A/B test measuring watch time, DAU, retention. Long-term A/B tests (weeks) to catch effects on user retention that short tests miss.
- 

## 94.2 2. Search Ranking (e.g., Google, Bing)

---

### 94.2.1 Problem Framing

- **Goal:** given a user query, return the most relevant results from a corpus of billions of documents.
- **Latency:** <500ms total (100ms retrieval + 200ms ranking + 100ms rendering + overhead).

### 94.2.2 Architecture: Query Understanding → Retrieve → Rank

### 94.2.3 Query Understanding

- Before retrieval, process the raw query to improve results:

- **Spell correction:** “reccomendation systm” → “recommendation system.”  
Use an edit-distance model or a sequence-to-sequence model trained on (misspelled, corrected) pairs from search logs.
- **Query expansion:** add related terms to improve recall. “Python ML” → “Python machine learning scikit-learn pytorch.” Use synonym dictionaries, word embeddings, or an LLM to generate expansions.
- **Intent classification:** determine what the user wants. “buy Nike shoes” is **transactional** (show product pages). “How does backpropagation work” is **informational** (show articles). “facebook.com” is **navigational** (go directly to the site). Different intents should trigger different retrieval strategies and result layouts.
- **Entity recognition:** extract entities from the query. “best restaurants near Times Square” → location: “Times Square” , entity type: “restaurants.”  
Route to a location-aware search pipeline.

#### 94.2.4 Retrieval

- **BM25** (traditional): term-matching retrieval using an inverted index. Fast, effective for keyword queries. No semantic understanding ( “dog food” does not match “canine nutrition” ).
- **Dense retrieval:** encode queries and documents as embeddings (using a bi-encoder like DPR or ColBERT). Retrieve by ANN search. Captures semantic similarity ( “dog food” matches “canine nutrition” ). Slower than BM25 but better for natural language queries.
- **Hybrid retrieval:** combine BM25 and dense retrieval. BM25 finds exact keyword matches; dense retrieval finds semantic matches. Merge and deduplicate. Best of both worlds.

#### 94.2.5 Ranking

- **Learning to rank:** a model scores each (query, document) pair. Three approaches:
  - **Pointwise:** predict a relevance score for each document independently. Simple but ignores relative ordering.
  - **Pairwise:** predict which of two documents is more relevant. LambdaMART (gradient-boosted trees) is the classic approach.

- **Listwise:** optimise the entire ranked list directly for a list-level metric (NDCG). More complex but best results.
- **Cross-encoder:** a transformer that takes [query, document] as input and outputs a relevance score. More accurate than bi-encoders (which encode query and document independently) because it captures fine-grained interactions. But too slow for the full corpus — used only for re-ranking the top 100-1000 candidates from retrieval.

### 94.2.6 Features

- **Query features:** query length, language, intent classification (navigational, informational, transactional).
  - **Document features:** PageRank, freshness, content quality score, domain authority.
  - **Query-document features:** BM25 score, embedding similarity, exact match count, click-through rate for this (query, document) pair in historical logs.
- 

## 94.3 3. Ads Click Prediction

---

### 94.3.1 Problem Framing

- **Goal:** predict the probability that a user will click on an ad. This determines how much to bid in real-time auctions.
- **Scale:** 100K+ auctions per second, each requiring a prediction within 10ms.
- **Revenue impact:** a 0.1% improvement in click prediction accuracy translates to millions in additional revenue.

### 94.3.2 Architecture

- **Feature engineering** is the core of ads systems. Features include:
  - **User features:** demographics, browsing history, purchase history, device, location, time of day.
  - **Ad features:** creative (image/text), advertiser, category, historical CTR, bid amount.
  - **Context features:** page content, ad position, device type, connection speed.
  - **Cross features:** user\_category  $\times$  ad\_category interaction, user\_region  $\times$  ad\_campaign interaction.



- **Model:** historically logistic regression (simple, fast, interpretable). Modern systems use deep learning: a **DLRM** (Deep Learning Recommendation Model) with embedding tables for categorical features and an MLP for dense features.
- **Calibration:** the predicted probability must be accurate (if the model says  $P(\text{click}) = 0.05$ , then 5% of such impressions should actually be clicked). Calibration is critical because the predicted probability directly determines bid amounts.
- **Exploration-exploitation:** always showing the predicted best ad is suboptimal long-term (you never discover that a new ad might be better). Thompson sampling or  $\epsilon$ -greedy exploration ensures some fraction of impressions go to less-certain ads to gather data.

### 94.3.3 Real-Time Bidding

- When a user loads a page, an ad auction runs in  $<100\text{ms}$ :
    1. Publisher sends a bid request (user info, page context) to multiple ad exchanges.
    2. Each advertiser's bidding server predicts CTR for their ads.
    3.  $\text{Bid} = \text{CTR} \times \text{value\_per\_click}$ . Higher bids win the auction.
    4. The winning ad is shown; if clicked, the advertiser pays.
- 

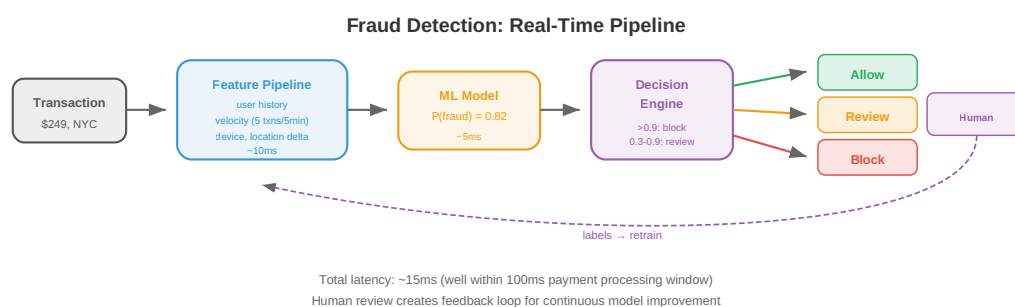
## 94.4 4. Fraud Detection

---

### 94.4.1 Problem Framing

- **Goal:** detect fraudulent transactions in real-time (credit card fraud, account takeover, fake reviews).
- **Latency:**  $<100\text{ms}$  (the transaction must be approved or flagged before the payment processes).
- **Key challenge:** extreme class imbalance (0.1% fraud rate). False positives block legitimate users; false negatives lose money.

## 94.4.2 Architecture



Fraud detection pipeline: transaction → real-time feature pipeline → ML model → decision engine → allow/review/block, with human review feeding labels back for retraining

## 94.4.3 Features

- **Transaction features:** amount, currency, merchant category, time of day, is\_international.
- **User features:** account age, average transaction amount, number of recent transactions, device fingerprint.
- **Velocity features** (real-time, from a streaming pipeline): number of transactions in the last 5 minutes, number of distinct merchants in the last hour, geographic distance from previous transaction.
- **Graph features:** is this merchant connected to known fraud rings? Is this device shared with flagged accounts?

## 94.4.4 Model

- **Gradient-boosted trees** (XGBoost, LightGBM) are the standard for tabular fraud detection. They handle mixed feature types, are interpretable (feature importance), and train fast.
- **Handling imbalance:** undersampling the majority class, oversampling the minority (SMOTE), or using class weights in the loss function. Focal loss (chapter 8) down-weights easy negatives.
- **The cost matrix:** a false positive (blocking a legitimate transaction) has a cost (user frustration, lost sale). A false negative (missing fraud) has a different cost (financial loss). The decision threshold should minimise total expected cost, not maximise accuracy.

### 94.4.5 Human-in-the-Loop

- Uncertain predictions (model confidence between 0.3 and 0.7) are sent to human reviewers. Reviewer decisions become labels for retraining. This creates a feedback loop: the model improves over time as it sees more labelled fraud cases.
- 

## 94.5 5. Content Moderation

---

### 94.5.1 Problem Framing

- **Goal:** automatically detect and remove harmful content (hate speech, violence, misinformation, CSAM) from a platform.
- **Scale:** billions of posts per day (text, images, video).
- **Challenge:** context-dependent (sarcasm, satire, cultural nuance). Must balance free expression with safety.

### 94.5.2 Architecture

- **Multi-modal classification:** separate models for text, images, and video, with a fusion layer that combines their signals.
- **Text moderation:** fine-tuned language model classifies text into categories (harassment, hate speech, misinformation, spam). Multilingual models handle 100+ languages.
- **Image moderation:** vision model detects: explicit content (nudity, violence), text in images (OCR + text classifier), and known harmful content (hash-matching against databases of known CSAM).
- **Video moderation:** sample frames at regular intervals, run image classifier on each frame, combine with audio transcription (ASR → text classifier).
- **Policy-as-code:** moderation policies are defined in structured rules that map model outputs to actions:

```
if text_model.hate_speech_score > 0.9:
 action = "remove"
elif text_model.hate_speech_score > 0.7:
 action = "human_review"
```

```
else:
 action = "allow"
```

- Policies change frequently (new regulations, evolving norms). Separating policy from model ensures changes can be deployed without retraining.

### 94.5.3 Proactive vs Reactive Moderation

- **Proactive** (pre-publication): run classifiers on content before it goes live. High-confidence violations are blocked automatically. This prevents harmful content from ever being visible but adds latency to publishing and risks false positives (blocking legitimate content).
- **Reactive** (post-publication): content goes live immediately. Users can report violations. Reports trigger classifier + human review. Lower latency for publishers but harmful content is visible until detected.
- **Most platforms use both**: proactive for high-severity categories (CSAM: zero tolerance, block before publication) and reactive for nuanced categories (misinformation: needs human judgement, review after reports).

### 94.5.4 Hash Matching

- For known harmful content (CSAM, terrorist propaganda), use **perceptual hashing**: compute a hash of the image/video that is robust to minor modifications (cropping, resizing, compression). Compare against databases of known harmful content (NCMEC's hash database, GIFCT shared hash database). Match → immediate removal without needing a classifier.
- **PhotoDNA** (Microsoft) is the standard perceptual hash for CSAM detection. It is a legal obligation in many jurisdictions, not just a technical choice.

### 94.5.5 Back-of-Envelope Numbers

- **Scale**: 1B posts/day = ~12K posts/second. Each post needs: text classification (~5ms), image classification (~20ms), hash matching (~1ms). At 12K QPS: need ~60 text classifiers, ~240 image classifiers, and ~12 hash matchers (plus redundancy).
- **Human review**: if 2% of posts are flagged for review = 20M/day. At 100 reviews/reviewer/day, need 200K reviewers (this is why automated accuracy matters: every 0.1% reduction in false positives saves 1M reviews/day).

- **Latency budget:** proactive moderation must complete within the publishing pipeline (~500ms). Text (5ms) + image (20ms) + hash (1ms) + overhead = well within budget. Video is the exception: even sampling 1 frame/second from a 10-minute video requires 600 classifier calls → handle asynchronously.

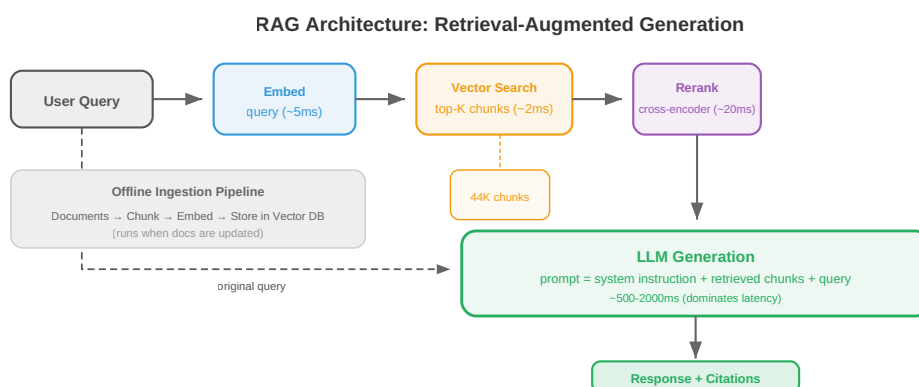
### 94.5.6 Escalation Workflow

- Automatic removal → human review of appeals → specialist review (legal, cultural experts) → policy team for ambiguous cases. Each level handles fewer cases with more nuance.
- **Feedback to model:** human review decisions are the highest-quality labels for retraining. Disagreements between the model and reviewers are prioritised for active learning — they represent the cases the model handles worst.

## 94.6 6. Conversational AI (RAG-Based Chatbot)

### 94.6.1 Problem Framing

- **Goal:** a chatbot that answers questions about a company's products using its documentation.
- **Requirements:** accurate (does not hallucinate), cites sources, handles follow-up questions, and stays within the product domain.



RAG architecture: embed query, search vector DB for relevant chunks, rerank, feed to LLM with original query to generate grounded response

## 94.6.2 Architecture: Retrieval-Augmented Generation (RAG)

```
User query → Query Embedding → Vector Search (documentation) → Top-K chunks
 ↓
User query + Retrieved chunks → LLM → Response (with citations)
```

## 94.6.3 Components

- **Document ingestion:** chunk documents and embed them. **Chunking strategy** matters significantly:
  - **Fixed-size chunking:** split every N tokens (e.g., 500) with M-token overlap (e.g., 50). Simple, predictable chunk sizes, but can split mid-sentence or mid-paragraph, losing context.
  - **Semantic chunking:** split at paragraph or section boundaries. Each chunk is a coherent unit of information. Variable size (some chunks are 100 tokens, others are 800), which requires the retrieval system to handle variable lengths.
  - **Recursive chunking:** try to split at paragraph boundaries. If a paragraph is too long, split at sentence boundaries. If a sentence is too long, split at a fixed size. Best balance of coherence and size consistency.
  - **Embedding:** embed each chunk with a text encoder (e.g., E5, BGE, Cohere embed). Store in a vector database.
- **Retrieval:** embed the user query, search the vector database for the k most similar chunks (typically k = 5-10). Optionally rerank with a cross-encoder for higher precision.
- **Generation:** construct a prompt with the retrieved chunks as context:

```
System: You are a helpful assistant. Answer based ONLY on the provided context.
If the answer is not in the context, say "I don't know."
```

```
Context:
```

```
[chunk 1]
```

```
[chunk 2]
```

```
...
```

User: {question}

- **Guardrails:** prevent the LLM from answering questions outside the product domain, generating harmful content, or contradicting the retrieved context. Implement as: input filtering (reject off-topic queries), output filtering (check response against retrieved context), and constitutional prompting (instruct the model to refuse certain requests).
- **Conversation memory:** maintain the last n turns of conversation. Include them in the prompt so the model understands follow-up questions ( “What about the pricing?” → needs the previous context about which product).

#### 94.6.4 Query Rewriting

- Users often ask ambiguous follow-up questions: “What about the pricing?” (pricing of what?). **Query rewriting** uses the conversation history to produce a standalone query:
  - Input: conversation history + “What about the pricing?”
  - Rewritten: “What is the pricing for the enterprise plan of Product X?”
- This rewritten query is what gets embedded and searched against the vector database. Without rewriting, the retrieval would search for “pricing” without context and return irrelevant chunks.
- Query rewriting can be done with a small LLM call (~50ms) or with a fine-tuned sequence-to-sequence model (~5ms).

#### 94.6.5 Back-of-Envelope Numbers

- **Documentation corpus:** 10K pages, average 2000 tokens each = 20M tokens. At 500 tokens/chunk with 50 overlap = ~44K chunks.
- **Embedding index:** 44K chunks × 768-dim × float16 = ~65 MB. Trivially fits in memory. Even 10M chunks would be ~15 GB.
- **Latency breakdown:** query embedding (5ms) + vector search (2ms) + cross-encoder rerank (20ms for top-50) + LLM generation (500-2000ms) = ~600-2100ms total. The LLM dominates. Use streaming to reduce perceived latency.
- **Cost:** at \$3/1M tokens (Claude/GPT-4 API), 1000 queries/day with ~2K tokens each = ~\$6/day. At scale (1M queries/day), self-host a 7B model on 2 A10G GPUs (~\$50/day) for 100x cost reduction.

## 94.6.6 Evaluation

- **Retrieval quality:** Recall@K (do the top-K chunks contain the answer?), MRR (Mean Reciprocal Rank).
  - **Generation quality:** factual accuracy (does the response match the retrieved context?), groundedness (does the response cite the correct chunks?), answer relevance.
  - **End-to-end:** user satisfaction (thumbs up/down), escalation rate to human agents.
- 

## 94.7 7. Large-Scale Image Search

---

### 94.7.1 Problem Framing

- **Goal:** given an image, find visually similar images from a corpus of 1B+ images.
- **Applications:** reverse image search, product search (photo → matching products), duplicate detection.
- **Latency:** <500ms including network round trip.

### 94.7.2 Architecture

Query image → Embedding Model (ViT/CLIP) → 512-dim vector → ANN Search → Top-K results

### 94.7.3 Embedding Extraction

- **Model:** a pre-trained vision encoder (ViT, CLIP's image encoder, DINOv2). Fine-tuned on the specific domain if needed (fashion, e-commerce, medical imaging).
- **Training:** contrastive learning (chapter 10). Positive pairs = different views of the same image (or image + matching text). Negative pairs = random images. The model learns to produce similar embeddings for similar images and dissimilar embeddings for different images.

### 94.7.4 Indexing

- **Offline:** embed all 1B images and build an ANN index. For HNSW (file 03), building the index takes hours and the index is stored in memory (~128 GB for  $1B \times 512\text{-dim} \times \text{float16} + \text{graph overhead}$ ).



- **Sharding:** split the index across multiple machines. Each machine holds a shard. At query time, search all shards in parallel and merge the top-K results.
- **Incremental updates:** new images (uploads, new products) must be added to the index. HNSW supports incremental insertion without rebuilding. Vector databases (Milvus, Pinecone) handle this natively.

### 94.7.5 Serving

- **Embedding service:** a GPU server running the ViT model. Latency: ~20ms per image. Batch multiple queries for throughput.
- **Search service:** the ANN index server. Latency: ~10ms for top-100 search over 1B vectors (with HNSW).
- **Caching:** cache results for popular queries. For duplicate detection, cache the embedding of recently uploaded images and compare new uploads against the cache before searching the full index.

### 94.7.6 Evaluation

- **Precision@K:** are the top-K results actually similar?
  - **Recall@K:** out of all truly similar images in the corpus, how many are in the top-K?
  - **Mean Average Precision (mAP):** the area under the precision-recall curve.
  - **Human evaluation:** for subjective similarity, human raters judge whether retrieved images are relevant.
- 

## 94.8 The Interview Framework

---

- When you encounter a systems design question, follow this framework:
1. **Clarify requirements** (2-3 minutes): ask about scale, latency, consistency requirements, and edge cases. “How many users? What latency is acceptable? What happens during failures?”
  2. **High-level design** (5-7 minutes): draw the major components and their interactions. Start with the happy path. Use the patterns from files 01-03.
  3. **Deep dive** (15-20 minutes): pick the most interesting/challenging component and design it in detail. This is where you show depth. For an ML system, the

deep dive is often: the model architecture, the feature pipeline, or the serving architecture.

4. **Evaluation and monitoring** (3-5 minutes): how do you measure success? What can go wrong? How do you detect and respond to problems?
  5. **Iteration** (2-3 minutes): what would you improve with more time/resources? This shows you understand tradeoffs and can prioritise.
- **What interviewers look for:** structured thinking (not jumping to solutions), tradeoff awareness (every choice has a cost), practical knowledge (you have actually built systems), and communication (can you explain your design clearly?).

## 95 AI for Finance

- Time series forecasting: ARIMA, exponential smoothing, Prophet, neural approaches (LSTM, Temporal Fusion Transformer, PatchTST)
- Algorithmic trading: signal generation, execution algorithms (TWAP, VWAP), market microstructure
- Portfolio optimisation: mean-variance (Markowitz), Black-Litterman, RL-based portfolio management
- Risk modelling: Value at Risk (VaR), Expected Shortfall, Monte Carlo simulation, credit scoring
- Fraud detection: anomaly detection, graph-based approaches, real-time streaming
- NLP in finance: sentiment analysis of news/earnings calls, financial document understanding
- Alternative data: satellite imagery, social media, web scraping
- Regulatory and ethical: model explainability (SHAP, LIME), fairness in credit, regulatory compliance

## 96 AI for Biology

- Protein structure prediction: AlphaFold 1/2/3, ESMFold, co-evolutionary analysis, MSA transformers

- Protein design: inverse folding (ProteinMPNN), diffusion for protein generation (RFDiffusion), hallucination
- Drug discovery: molecular representations (SMILES, graphs), molecular property prediction, virtual screening, docking
- Generative chemistry: molecular generation (VAE, GAN, diffusion), retrosynthesis prediction
- Genomics: DNA sequence modelling (Enformer, Hyena DNA), variant effect prediction, CRISPR guide design
- Single-cell analysis: scRNA-seq, cell type clustering, trajectory inference
- Medical imaging: radiology (CheXNet), pathology (whole-slide images), segmentation (nnU-Net)
- Clinical NLP: medical entity extraction, clinical trial matching, electronic health records

## 97 Quantum Machine Learning

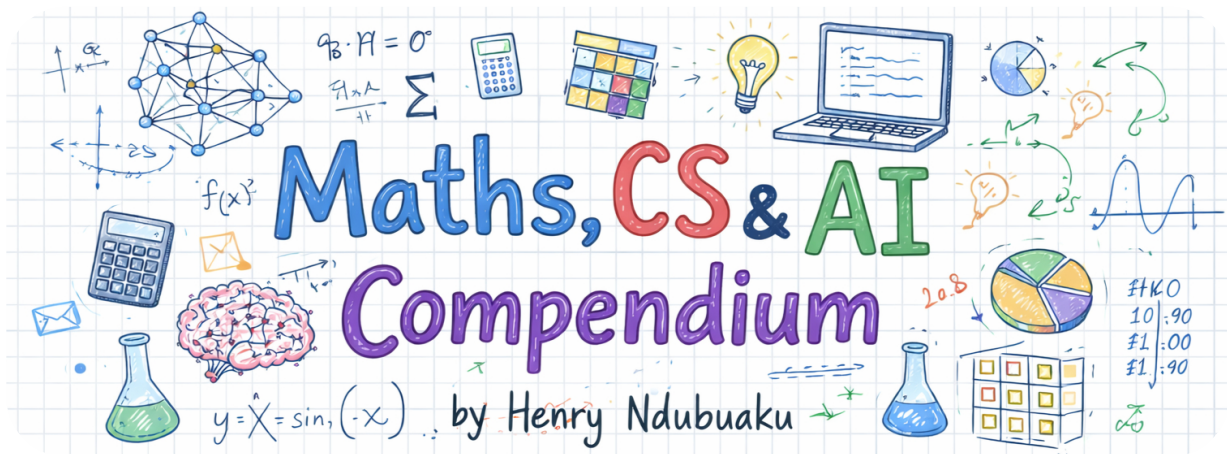
- Quantum computing basics: qubits, superposition, entanglement, measurement
- Quantum gates: Pauli (X, Y, Z), Hadamard, CNOT, Toffoli, rotation gates
- Quantum circuits: circuit model, parameterised circuits, depth and width
- Variational quantum algorithms: VQE, QAOA, variational classifiers
- Quantum kernel methods: quantum feature maps, quantum support vector machines
- Quantum neural networks: parameterised quantum circuits as neural layers
- Barren plateaus: vanishing gradients in quantum circuits, expressibility vs trainability
- Quantum advantage debate: NISQ era limitations, fault-tolerant quantum computing timeline
- Hybrid classical-quantum architectures: quantum layers in classical pipelines

## 98 Neuromorphic Computing

- Biological inspiration: spiking neurons, synaptic plasticity, temporal coding
- Spiking neural networks (SNNs): integrate-and-fire models (LIF, IF), spike timing

- Learning in SNNs: STDP (spike-timing-dependent plasticity), surrogate gradient methods, conversion from ANNs
- Neuromorphic hardware: Intel Loihi 2, IBM TrueNorth, SpiNNaker, BrainScaleS
- Event-driven computation: asynchronous processing, energy efficiency
- Event cameras (DVS): neuromorphic vision sensors, sparse temporal data
- Applications: low-power edge inference, robotics, always-on sensing
- Comparison with conventional deep learning: latency, power, accuracy tradeoffs

## 99 Maths, CS & AI Compendium



Read online: [English](#) | [简体中文](#)



### 99.1 Overview

Most textbooks bury good ideas under dense notation, skip the intuition, assume you already know half the material, and quickly get outdated in fast-moving fields like AI. This is an open, unconventional textbook covering maths, computing, and artificial intelligence from the ground up. Written for curious practitioners looking to deeply understand the stuff, not just survive an exam/interview.

## 99.2 Background

---

Over the past years working in AI/ML, I filled notebooks with intuition first, real-world context, no hand-waving explanations of maths, computing and AI concepts. In 2025, a few friends used these notes to prep for interviews at DeepMind, OpenAI, Nvidia etc. They all got in and currently perform well in their roles. Meanwhile I got in Y Combinator last year. So I’ m sharing to everyone.

## 99.3 MCP Server

---

This repo includes an MCP server that lets any AI assistant (Claude Code, Cursor, VS Code, etc.) use the compendium as a knowledge base. It requires a local clone of the repo. Comes with tools for educational purposes and example implementations.

## 99.4 Outline

---

#	Chapter	Summary	Status
01	<a href="#">Vectors</a>	Spaces, magnitude, direction, norms, metrics, dot/cross/outer products, basis, duality	Available
02	<a href="#">Matrices</a>	Properties, special types, operations, linear transformations, decompositions (LU, QR, SVD)	Available
03	<a href="#">Calculus</a>	Derivatives, integrals, multivariate calculus, Taylor approximation, optimisation and gradient descent	Available
04	<a href="#">Statistics</a>	Descriptive measures, sampling, central limit theorem, hypothesis	Available

#	Chapter	Summary	Status
		testing, confidence intervals	
05	<a href="#">Probability</a>	Counting, conditional probability, distributions, Bayesian methods, information theory	Available
06	<a href="#">Machine Learning</a>	Classical ML, gradient methods, deep learning, reinforcement learning, distributed training	Available
07	<a href="#">Computational Linguistics</a>	syntax, semantics, pragmatics, NLP, language models, RNNs, CNNs, attention, transformers, text diffusion, text OCR, MoE, SSMs, modern LLM architectures, NLP evaluation	Available
08	<a href="#">Computer Vision</a>	image processing, object detection, segmentation, video processing, SLAM, CNNs, vision transformers, diffusion, flow matching, VR/AR	Available
09	<a href="#">Audio &amp; Speech</a>	DSP, ASR, TTS, voice & acoustic activity detection, diarisation, source separation, active noise	Available

#	Chapter	Summary	Status
		cancellation, wavenet, conformer	
10	<a href="#">Multimodal Learning</a>	fusion strategies, contrastive learning, CLIP, VLMs, image/video tokenisation, cross-modal generation, unified architectures, world models	Available
11	<a href="#">Autonomous Systems</a>	perception, robot learning, VLAs, self-driving cars, space robots	Available
12	<a href="#">Graph Neural Networks</a>	geometric deep learning, graph theory, GNNs, graph attention, Graph Transformers, 3D equivariant networks	Available
13	<a href="#">Computing &amp; OS</a>	discrete maths, computer architecture, operating systems, concurrency, parallelism, programming languages	Available
14	<a href="#">Data Structures &amp; Algorithms</a>	Big O, recursion, backtracking, DP, arrays, hashing, linked lists, stacks, trees, graphs, sorting, binary search	Available
15			

#	Chapter	Summary	Status
	<a href="#">Production Software Engineering</a>	Linux, Git, codebase design, testing, CI/CD, Docker, model serving, MLOps, monitoring, best way to use coding agents	Draft (coding tasks pending)
16	<a href="#">SIMD &amp; GPU Programming</a>	C++ for ML, how frameworks work, hardware fundamentals, ARM NEON/I8MM/SME2, x86 AVX, GPU/CUDA, Triton, TPUs, RISC-V, Vulkan, WebGPU	Available
17	<a href="#">AI Inference</a>	quantisation, efficient architectures, serving and batching, edge inference, speculative decoding, cost optimisation	Available
18	<a href="#">ML Systems Design</a>	systems fundamentals, cloud computing, distributed systems, ML lifecycle, feature stores, A/B testing, recommendation/search/ads/fraud design examples	Draft (coding tasks pending)
19	<a href="#">Applied AI</a>	AI for finance, protein design, drug discovery, agentic systems, healthcare	Draft/Stub
20	<a href="#">Bleeding Edge AI</a>	quantum ML, neuromorphic	Draft/Stub



#	Chapter	Summary	Status
		computing, datacentres in space, decentralised AI, brain-machine interfaces	

## 99.5 Foreword

---

A newborn' s brain is a newly initialised neural network, which trains from realworld data and experience into adulthood...until forever. Exceptional understanding of French with the flawless accent implies correct exposure to exceptional French and flawless accent. Similarly, great AI Researchers & engineers with excellent problem-skills imply quality knowledge consumed and exposure rich experience.

Now Kvashchev' s experiment was a long-term Serbian study demonstrating that intensive, three-year training in creative problem-solving can significantly boost intelligence, particularly fluid intelligence, adding 10-15 IQ points. There is such a thing as having a naturally high IQ, similar to how quality weight initialisations yield better training, evidenced by nature-vs-nurture experimental findings.

However, the only advantage a high-IQ individual really has is the ability to learn/ recognise patterns faster. But using a repeated pattern makes any concept absolutely learnable. Charles Darwin was considered a very average, if not below-average, student by his teachers and father. He described himself as not being quick-witted, feeling like a “slow processor” who needed time to soak in data.

Between 3-10yrs, I performed well academically, naturally grasping concepts without ever taking notes or revising. I got a bit cocky between 11-13 and dropped to the bottom half of an 80-student class with this technique. Now between 14-15, I began reading like a normal student, finishing 1st in my final secondary school semester. Early school curriculum works well with natural IQ but real-world talents are powered by quality knowledge consumption and execution intensity.

In fact, most students who perform well academically are just more studious, but the academic system is designed for fast learners. This compendium provides a rounded and well-connected flow of knowledge to facilitate better learning for the Darwins of the world. You only need elementary maths and basic python programming, everything else is picked up, just read and trust the process!

## 99.6 How To Study Better

---

First semester at university, I took 17 modules at once, grades were not great for it, so I used a technique:

**Phase 1:** Cumulative reading after classes Read each material after class, before bed. The next lecture, start all over until the current end, then fill knowledge gaps with additional research. This allows your brain to connect the patterns.

**Phase 2:** Shadow reading before exams Read each slide/note subtitle, close the book, then visualise and write an explanation for that concept. Only re-read what you missed, similar to masked-language modelling in machine learning. After the re-read, ultimately implement the concept in code after. You develop muscle memory for each concept.

This worked really well for my friends who were not very confident. In fact, one of these friends beat me in advanced engineering mathematics module, where we covered Hessians and Optimisation. She works at a big oil & gas firm today. The willingness of the soul matters more than the body we are working with (Rosenthal experiment).

## 99.7 Who is Henry Ndubuaku?

---

Read the GitHub profile!

## 99.8 Citation

---

```
@book{ndubuaku2025compendium,
 title = {Maths, CS & AI Compendium},
 author = {Henry Ndubuaku},
 year = {2026},
 publisher = {GitHub},
 url = {https://github.com/HenryNdubuaku/maths-cs-ai-compendium}
}
```